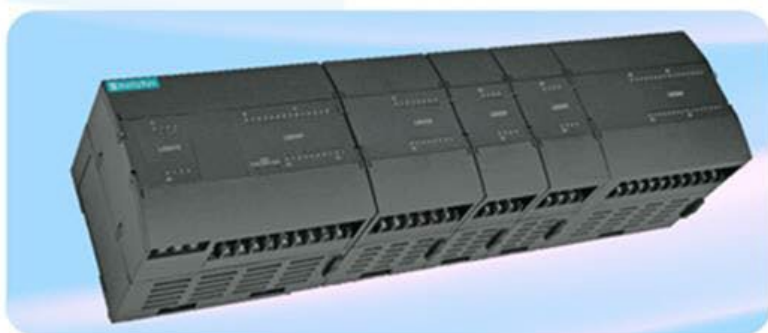




和利时可编程控制器 **PLC 指令手册**

智能化成就卓越

和利时集团
北京和利时智能技术有限公司

版本：01/2025

版权声明

本手册内容，包括文字、图表、标志、标识、商标、产品型号、软件程序、版面设计及其它内容等，均受《中华人民共和国著作权法》、《中华人民共和国商标法》、《中华人民共和国专利法》及与之适用的国际公约中有关著作权、商标权、专利权或其他财产所有权法律的保护，为北京和利时智能技术有限公司专属所有或持有。

由于本手册中所描述的设备有多种使用方法，用户以及设备使用责任人必须保证每种方法的可容许性。对由使用或错误使用这些设备造成的任何直接或间接损失，北京和利时智能技术有限公司将不负法律责任。

由于实际应用时的不确定因素，北京和利时智能技术有限公司不承担直接使用本手册中提供的数据的责任。

本手册仅供商业用户阅读，在未得到北京和利时智能技术有限公司书面授权的情况下，无论出于何种目的和原因，不得以任何形式（包括电子、机械或其它形式）传播或复制本手册的任何内容。违者我公司将依法追究其相关责任。

已核对本手册中的内容、图表与所述硬件设备相符，但误差难以避免，并不能保证完全一致。同时，会定期对手册的内容、图表进行检查、修改和维护，恕不另行通知。

HollySys、和利时、 的字样和徽标均为北京和利时智能技术有限公司的商标或注册商标。

手册中涉及到的其他商标或注册商标属于它们各自的拥有者。

北京和利时智能技术有限公司版权所有。

地址：北京经济技术开发区地盛中路 2 号院

邮编：100176

商务电话：010-5898 1588

产品咨询热线：4008-111-999

技术支持电话：010-58981514

传真：010-5898 1558

网址：<http://www.hollysys.com/>

Email: PLC@hollysys.com

新浪微博：<http://weibo.com/hollysysplc>

目录

第 1 章	关于本文档（指令）	1
1.1	文档更新	1
1.2	文档用途	2
1.3	阅读对象	2
1.4	重要信息	2
1.5	产品文档目录	3
第 2 章	指令系统概述	5
2.1	指令的定义与分类	5
2.2	指令库的分类	11
2.3	指令库的添加	12
2.4	指令系统使用注意事项	13
第 3 章	指令库	15
3.1	标准库	15
3.1.1	数学运算	15
3.1.2	逻辑运算	34
3.1.3	比较运算	37
3.1.4	选择运算	42
3.1.5	移位运算	47
3.1.6	数据类型转换	50
3.1.7	指针	75
3.1.8	字符串	77
3.1.9	沿触发器指令	84
3.1.10	双稳态指令	87
3.1.11	计数器指令	90
3.1.12	定时器指令	96
3.1.13	IEC_SFC	103
3.2	基础应用库	103
3.2.1	BCD 转换指令	103
3.2.2	位_字节操作指令	107
3.2.3	数学辅助功能	113
3.2.4	控制器指令	121
3.2.5	信号发生器指令	125
3.2.6	模拟量处理指令	133
3.2.7	数据块传送指令	147
3.2.8	设置地址内容指令	151
3.3	系统库	158
3.3.1	通讯指令	158
3.3.2	任务指令	215
3.3.3	文件操作	217
3.3.4	时间相关指令	226
3.3.5	模块信息指令	231
3.3.6	系统状态指令	249
3.3.7	冗余相关	262
3.3.8	内存操作指令	275
3.3.9	中断设置指令	281

3.4	行业应用库	291
3.4.1	HMC 运动指令库	291
3.4.2	运动控制指令	617
3.4.3	计数指令	700
3.4.4	语音控制指令	702
3.4.5	MKC3167 保护指令	705
3.4.6	故障录波	716
3.5	现场总线库	719
3.5.1	ETHERCAT 协议相关指令库	719
3.5.2	RTEX 协议相关指令库	733
3.5.3	CANopen 协议相关指令	754
3.6	产品扩展库	757
3.6.1	DI 置反	757
3.6.2	产品内部状态	758
3.6.3	模块扩展信息	761
3.6.4	SOE 相关指令	784

第1章 关于本文档（指令）

和利时公司 PLC 系统分为 MC、LK、LE、MKC 四个平台，通过 PLC 的编程软件 AutoThink 为四个平台提供丰富的指令。选择不同的目标平台，库管理器下包含的库文件目录会不相同，以及加载的可用指令库也不相同。详细内容请参见[指令的定义与分类](#)。

MC 平台的 HMC 运动指令库、IO 操作指令库以及与控制器相关指令库的内容请参考《MC 系列运动控制器指令手册》。

1.1 文档更新

版本	说明	日期
V1.0	创建	2016.12
V1.1	LE 指令更新	2017.04
V1.2	新增 MKC 平台指令	2017.07
V1.3	文件操作指令修改如下： 1、文件目录 “/media/flash/” 修改为 “flash/” 2、文件目录 “/media/sd/” 修改为 “sd/”	2017.12
V1.4	更新章节 3.4.2	2018.04
V1.5	章节 3.1.1 数学运算补充输入变量定义域说明 章节 3.4.5 MKC3167 保护指令更新	2018.07
V1.6	增加 3.2.3.6 SOLVE_POLY6_EQU（一元六次方程求解） 章节 3.4.2.19 PWM（脉冲调制输出,5104/5106/5108/5128）增加 LE5703-QWT 相关参数说明 章节 3.3.1.11 参数说明 Port 参数增加 LE5703-QWT 说明	2018.08
V1.7	AutoThink 软件版本更新 新增 CAN 指令	2019.01
V1.8	新增 GetLE5406ModuleSlaveDiagInfo（获取 LE5406 网关模块从站诊断信息）、 GetLK250ModuleSlaveDiagInfo（获取 LK250 网关模块从站诊断信息）、ControlLK620（LK620 控制监视功能块）	2019.03
V1.9	新增指令 MOVE_BLOCK_EXE、DataMemCopy、DataMemSet	2019.12
V2.0	指令 sysGetEthMac（获取 Mac 地址）新增输入引脚“Slot”、新增指令“sysGetCPUModuleVernExt（获取控制器的固件版本扩展信息）”	2020.04
V2.1	新增指令 GetLK631SOEAlarm(LK631 缓存告警信息查询功能块)、sysGetCPUSOEInfo(获取主控 SOE 诊断信息)、sysClearSOE(清除主控 SOE 数据)	2020.09
V2.2	新增 ControlHSC（LK620 控制监视功能块）	2020.12
V1.0	更新 指令系统使用注意事项	2021.03
V1.1	新增 LK238 相关指令	2021.12
V1.2	新增指令： LK238_ModbusMaster（LK238 主站配置读写功能块） sysReadSOE（读取主控 SOE 信息） 更新以下指令：	2023.12

版本	说明	日期
	ModbusTcpSlaveConfig (Modbus TCP 从站配置) sysGetCPUDiagInfo (获取主控模块诊断)	
V1.3	新增指令： COM_FRAME_BREAK_TIME (通讯口自定义断帧) sysGetHTCP_CycleTime(获取 Hollitcp 主站周期时间) 更新以下指令： sysGetEthMac (获取 Mac 地址) LK238_ModbusMaster (LK238 主站配置读写功能块) sysReadSOE (读取主控 SOE 信息) sysGetComModuleDiagInfo (获取通讯模块诊断) TCP_CONNECT(建立 TCP 通信连接) TCP_SEND (TCP 数据发送) TCP_RECV (TCP 数据接收) ETH_DISCONNECT (关闭以太网通信连接)	2024.06
V1.4	新增指令： MKC3167_IS_ENABLE_CHANNEL(是否使能 IO 板数据上传与下发通道号) MKC3167_NORMAL_READ (正常模式数据读取) MKC3167_GET_ERR_CHNL_ID (获取故障通道 ID) MKC3167_READ_FAULT_DATA (故障模式数据读取)	2025.01

1.2 文档用途

本手册帮助使用 AutoThink 软件的用户，了解在 MC、LK、LE、MKC 平台工程组态的方法，以及软件各功能的使用，并指导用户正确、有效的使用该软件。

1.3 阅读对象

本手册适用于以下人员：

- 负责系统工程实施的工程人员；
- 负责系统维护的技术人员；
- 现场系统运行时的操作人员。

1.4 重要信息

在本手册中使用以下符号：



危险：该符号表示存在造成人身伤亡的潜在威胁。



电击：该符号表示存在造成电击的潜在威胁。



警告：该符号表示存在造成设备或环境损坏的潜在威胁。



提示：该符号表示有用的技巧或建议，以便更高效地使用产品。

SEE ALSO 该符号表示用于参考的信息。

1.5 产品文档目录



AutoThink V3.1 用户手册_工程组态



和利时可编程控制器 PLC 指令手册



LK 系列可编程控制器系统手册



MC 系列运动控制器系统手册



MC 系列运动控制器指令手册

第2章 指令系统概述

和利时公司 PLC 系统为用户提供了丰富的指令，这些指令均可通过 PLC 的编程软件 AutoThink 进行调用，操作简单，使用方便。

本手册将详细讲述 MC、LK、LE、MKC PLC 系统的指令（简称 PLC 指令）。

2.1 指令的定义与分类

在可编程控制器中，使 CPU 完成某种操作或实现某种功能的命令及多个命令的组合称为指令，指令的集合称为指令系统。指令系统是可编程控制器硬件和软件的桥梁，是可编程控制器程序设计的基础。

AutoThink 提供了丰富的指令，按照功能不同可分为转换指令、比较指令、类型转换指令、逻辑运算指令、模拟量应用等类型。为了便于理解和记忆，我们把算法库分为六大类，标准库、基础应用库、系统库、行业应用库、现场总线库（只有 MC1002E 和 MC1002R 控制器支持）、产品扩展库。三个平台均包含标准库、基础应用库、系统库，对于行业应用库、产品扩展库，会根据选择的目标平台不同对相应库文件进行加载。

PLC 指令在编程软件中有函数和功能块两种实现方式。函数和功能块都是 AutoThink 软件的程序组织单元，都是预先编好的、实现某种功能的程序。功能块输出可以是一个或多个结果，每一个功能块实例都有一个相关的标识符（即实例名称）；函数则不需要标识符，而且只有一个输出结果（即函数的返回值）。函数和功能块的具体概念可以参考 POU 类型。下表中注明了指令的实现方式，FUN 表明指令是以函数方式实现，FB 则表明指令是以功能块方式实现的。



- 以函数方式实现的指令，在使用的时候都无需声明。
- 以功能块方式实现的指令，在使用的时候都需声明实例名。

指令列表

指令类型	指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
标准库	ADD(FUN)	加法指令	√	√	√	√	
	SUB(FUN)	减法指令	√	√	√	√	
	MUL(FUN)	乘法指令	√	√	√	√	
	DIV(FUN)	除法指令	√	√	√	√	
	MOD(FUN)	取余指令	√	√	√	√	
	SIZEOF(FUN)	数据类型长度指令	√	√	√	√	

指令类型	指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
	ABS(FUN)	绝对值	√	√	√	√	
	SQRT(FUN)	平方根	√	√	√	√	
	LOG(FUN)	常用对数	√	√	√	√	
	LN(FUN)	自然对数	√	√	√	√	
	EXP(FUN)	指数	√	√	√	√	
	SIN(FUN)	正弦	√	√	√	√	
	COS(FUN)	余弦	√	√	√	√	
	TAN(FUN)	正切	√	√	√	√	
	ASIN(FUN)	反正弦	√	√	√	√	
	ACOS(FUN)	反余弦	√	√	√	√	
	ATAN(FUN)	反正切	√	√	√	√	
	EXPT(FUN)	幂	√	√	√	√	
	MOVE(FUN)	赋值指令	√	√	√	√	
	逻辑运算 指令	AND(FUN)	√	√	√	√	
		OR(FUN)	√	√	√	√	
		XOR(FUN)	√	√	√	√	
		NOT(FUN)	√	√	√	√	
	比较运算 指令	GT(FUN)	√	√	√	√	
		LT(FUN)	√	√	√	√	
		GE(FUN)	√	√	√	√	
		LE(FUN)	√	√	√	√	
		EQ(FUN)	√	√	√	√	
		NE(FUN)	√	√	√	√	
	选择运算 指令	SEL(FUN)	√	√	√	√	
		MAX(FUN)	√	√	√	√	
		MIN(FUN)	√	√	√	√	
		MUX(FUN)	√	√	√	√	
		LIMIT(FUN)	√	√	√	√	
	移位运算 指令	SHL(FUN)	√	√	√	√	
		SHR(FUN)	√	√	√	√	
		ROL(FUN)	√	√	√	√	
		ROR(FUN)	√	√	√	√	
	数据类型 转换指令	BOOL_TO_<TYPE>	√	√	√	√	
		BYTE_TO_<TYPE>	√	√	√	√	
		DATE_TO_<TYPE>	√	√	√	√	
		DINT_TO_<TYPE>	√	√	√	√	
		DT_TO_<TYPE>	√	√	√	√	
		DWORD_TO_<TYPE>	√	√	√	√	
		INT_TO_<TYPE>	√	√	√	√	
		LINT_TO_<TYPE>	√				
		LREAL_TO_<TYPE>	√	√	√		
		LWORD_TO_<TYPE>	√				
		REAL_TO_<TYPE>	√	√	√	√	
		SINT_TO_<TYPE>	√	√	√	√	

指令类型	指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
基础应用库	字符串	STRING_TO_<TYPE>	√	√	√		
		TIME_TO_<TYPE>	√	√	√	√	
		TOD_TO_<TYPE>	√	√	√	√	
		UDINT_TO_<TYPE>	√	√	√	√	
		UINT_TO_<TYPE>	√	√	√	√	
		ULINT_TO_<TYPE>	√				
		USINT_TO_<TYPE>	√	√	√	√	
		WORD_TO_<TYPE>	√	√	√	√	
		TRUNC	√	√	√	√	
	指针	ADR(FUN)	√	√	√	√	
		VAL(FUN)	√	√	√	√	
	字符串	Left(FUN)	√	√			
		Right(FUN)	√	√			
		Concat(FUN)	√	√			
		Delete(FUN)	√	√			
		Insert(FUN)	√	√			
		Mid(FUN)	√	√			
		Replace(FUN)	√	√			
		Find(FUN)	√	√			
		Len(FUN)	√	√			
	沿触发器指令	F_TRIG(FB)	√	√	√	√	
		R_TRIG(FB)	√	√	√	√	
	双稳态指令	RS(FB)	√	√	√	√	
		SR(FB)	√	√	√	√	
	计数器指令	CTD(FB)	√	√	√	√	
		CTU(FB)	√	√	√	√	
		CTUD(FB)	√	√	√	√	
	定时器指令	TP(FB)	√	√	√	√	
		TON(FB)	√	√	√	√	
		TOF(FB)	√	√	√	√	
		TONR(FB)	√	√	√	√	
	IEC_SFC	SFCActionControlBlock(FB)	√	√	√	√	
	BCD 转换指令	BCD_TO_INT(FUN)	√	√	√	√	
		INT_TO_BCD(FUN)	√	√	√	√	
	位_字节操作指令	UNPACK(FB)	√	√	√	√	
		EXTRACT(FUN)	√	√	√	√	
		PACK(FUN)	√	√	√	√	
		PUTBIT(FUN)	√	√	√	√	
	数学辅助功能	DERIVATIVE(FB)	√	√	√	√	
		INTEGRAL(FB)	√	√	√	√	
		STATISTIC_INT(FB)	√	√	√	√	
		STATISTIC_REAL(FB)	√	√	√	√	
		VARIANCE(FB)	√	√	√	√	
	控制器指	PID(FB)	√	√	√	√	

指令类型		指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
系统库	令							
	信号发生器指令	BLINK(FB)	脉冲信号发生器	√	√	√	√	
		GEN(FB)	典型周期信号发生器	√	√	√	√	
		RAND(FB)	随机数发生器	√	√	√	√	
	模拟量处理指令	RAMP_INT(FB)	整型限速	√	√	√	√	
		RAMP_REAL(FB)	实型限速	√	√	√	√	
		CHARCURVE(FB)	特征曲线	√	√	√	√	
		HYSTERESIS(FB)	滞后	√	√	√	√	
		LIMITALARM(FB)	上下限报警	√	√	√	√	
		HEX_ENGIN(FB)	16 进制数转换为工程量数据	√	√	√	√	
		ENGIN_HEX(FB)	工程量数据转换为 16 进制数据 (WORD)	√	√	√	√	
		ENGIN_HEX2(FB)	工程量数据转换为 16 进制数据 (INT)	√	√	√	√	
	数据块传送指令	MOVE_BLOCK(FB)	数据块传送	√	√	√	√	
		MOVE_BLOCK_EXT(FB)	数据块传送扩展指令	√	√	√	√	
	设置地址内容指令	VALSET_BYTE(FUN)	设置地址内容			√		
		VALSET_WORD(FUN)				√		
		VALSET_DWORD(FUN)				√		
		VALSET_REAL(FUN)				√		
	通讯指令	GENERATE_CRC(FB)	CRC 校验计算	√	√	√	√	
		ComConfig(FB)	设置串口通讯参数	√	√			
		ComRecv(FB)	接收串口数据	√	√			
		ComSend(FB)	发送串口数据	√	√			
		CANRecv(FB)	接收 CAN 数据	√	√			
		CANSend(FB)	发送 CAN 数据	√	√			
		CANConfig(FB)	设置 CAN 参数	√	√			
		ComClearRecvFIFO(FB)	清空串口接收数据	√	√			
		COMM_SEND(FB)	自由协议通讯数据发送			√	√	
		COMM_RECEIVE(FB)	自由协议通讯数据接收			√	√	
		MODBUS_MASTER(FB)	Modbus RTU 主站通讯			√	√	
		ModbusTcpMasterConnect (FB)	Modbus TCP 主站连接			√		
		ModbusTcpMasterMsg (FB)	Modbus TCP 主站消息			√		
		ModbusTcpSlaveMsg (FB)	Modbus TCP 从站消息			√		
		ModbusTcpSlaveConfig (FB)	Modbus TCP 从站配置			√		
		IPADDR_CONFIG(FB)	设置 IP 地址				√	
		GATEWAY_CONFIG(FB)	设置网关				√	
		NETMASK_CONFIG(FB)	设置子网掩码				√	
	任务指令	TaskStart(FUN)	运行任务	√	√			
		TaskStop(FUN)	停止任务	√	√			
		TaskSuspend(FUN)	挂起任务	√	√			
		TaskStatus(FUN)	返回任务状态	√	√			
	文件操作指令	FileOpen(FUN)	打开文件	√				
		FileClose(FUN)	关闭文件	√				

指令类型	指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
		FileRead(FUN)	√				
		FileWrite(FUN)	√				
		FileLseek(FUN)	√				
		FileListScan(FUN)	√				
		FindFirstFile(FUN)	√				
		FindNextFile(FUN)	√				
		FindClose(FUN)	√				
		FileRemove(FUN)	√				
		FlashWrite(FUN)	√				
		FlashRead(FUN)	√				
	时间相关 指令	GetTickCnt(FUN)	√	√			
		TimeDelay(FUN)	√	√			
		SET_RTC(FB)	√	√	√	√	
		GET_RTC(FB)	√	√	√	√	
	中断设置 指令	SET_INTERRUPT_DI (FB)			√		
		HD_TIMER0 (FB)			√		
		HD_TIMER1 (FB)			√		
		HD_TIMER2 (FB)			√		
		HD_TIMER3 (FB)			√		
		HD_TIMER4 (FB)			√		
		HD_TIMER5 (FB)			√		
	模块信息 指令	sysGetModel(FB)	√	√			
		sysGetCPUModuleVern(FB)	√	√			
		sysGetComModuleVern(FB)	√	√			
		sysGetModuleSN(FB)	√	√			
		sysGetCheckTime(FB)	√	√			
		sysGetEthMac(FB)	√	√			
		sysGetWorkVoltage	√				
		RW_EXTEN_FLASH(FB)				√	
	系统状态 指令	sysGetRunTime(FB)	√	√			
		sysGetKeyState(FB)	√	√			
		sysGetDPMasterState(FB)	√	√			
		sysGetCPULoad(FB)	√	√			
		GET_PRGTIME(FB)				√	
	冗余相关	sysGetRedState(FB)	√	√			
		sysMasterSwitchToSlave(FB)	√	√			
		sysGetDPSlaveState(FB)	√	√			
		sysGetCPUDiagInfo(FB)	√	√			
		sysGetComModuleDiagInfo(FB)	√	√			
行业 应用 库	HMC 运动 指令库		√				请参见 《MC 系列运 动控制 器指令 手册》

指令类型	指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
运动控制 指令	MF_M(FB)	测频,5128			√		
	MF_H(FB)	测频,5106/5107/5108/5109/5118			√		
	HSC_M(FB)	高速计数,5128			√		
	HSC_H(FB)	高速计 数,5106/5107/5108/5109/5118			√		
	HSC_L(FB)	高速计 数,5104/5105/5107E/5107L/5109L			√		
	PTO_H(FB)	脉冲输出,5106/5108/5118			√		
	PTO_L(FB)	脉冲输出,5104			√		
	MOVERELATIVE_M(FB)	相对定位,5128			√		
	MOVERELATIVE_H(FB)	相对定位,5106/5108/5118			√		
	MOVEABSOLUTE_M(FB)	绝对定位,5128			√		
	MOVEABSOLUTE_H(FB)	绝对定位,5106/5108/5118			√		
	JOG_M(FB)	点动,5128			√		
	JOG_H(FB)	点动,5106/5108/5118			√		
	STOP_M(FB)	停止,5128			√		
	STOP_H(FB)	停止,5106/5108/5118			√		
	HOME_M(FB)	原点回归,5128			√		
	HOME_H(FB)	原点回归,5106/5108/5118			√		
	FORWARD(FB)	连续脉冲输出,5128			√		
	PWM(FB)	脉冲调制输 出,5104/5106/5108/5128/5118			√		
	DEFPOS(FB)	定义位置,5128			√		
	CANCEL(FB)	解除连接,5128			√		
	FASTPID(FB)	快速 PID,5128			√		
	LATCH(FB)	数据锁存,5128			√		
	MOVELINK(FB)	联动功能,5128			√		
	CONNECT(FB)	电子齿轮,5128			√		
计数指令	HSC(FB)	高速计数,5109i				√	
语音控制 指令	PLAY_VOICE_PROMPT(FB)	播放指定语音段,5109i				√	
	PLAY_ENHANCED_VOIVE_PROMPT(FB)	播放“滴”提示音后播放语音 段,5109i				√	
MKC3167 保护指令	MKC3167_KJZY_PROTECT (FB)	一体化移变保护算法,3167				√	
	MKC3167_PJG_PROTECT (FB)	高爆保护算法,3167				√	
	MKC3167_KJG_PROTECT (FB)	移变高压侧保护算法,3167				√	
	MKC3167_BXJ1_PROTECT (FB)	移变低压侧保护算法,3167				√	
	MKC3167_BXJ2_PROTECT (FB)	移变低压侧双回路保护算法,3167				√	
现场总线 库	ETHERCAT 协议相关指令库		√				请参见 《MC 系列运 动控制 器指令 手册》
	RTEX 协议相关指令库		√				请参见 《MC 系列运

指令类型	指令名	指令说明	MC 平台	LK 平台	LE 平台	MKC 平台	备注
							动控制器指令手册》
产品 库	DI 置反	SetInvertDI(FUN)	√				请参见《MC 系列运动控制器指令手册》
		GetInvertDI(FUN)	√				
	产品内部 状态	sysGetBatteryAlarm(FB)	√	√			
		sysGetTEMP(FB)	√	√			
	模块扩展 信息	GetLE5406ModuleSlaveDiagInfo (FB)		√			
		GetLK250ModuleSlaveDiagInfo (FB)		√			
		GetLK250ModuleSlaveDiagInfoExt (FB)		√			
		ControlLK620 (FB)		√			
		GetLK631SOEAlarm (FB)		√			
		COMM_SET_CONFIG (FB)		√			
		COMM_GET_CONFIG (FB)		√			
		COMM_RW_MODBUS_MASTER (FB)		√			
		COMM_RW_MODBUS_SLAVE (FB)		√			
		COMM_RW_FREEPORT (FB)		√			
		COMM_CTL_MODULE (FB)		√			
	SOE 相关 指令	sysGetCPUSOEInfo (FB)		√			
		sysClearSOE (FB)		√			

2.2 指令库的分类

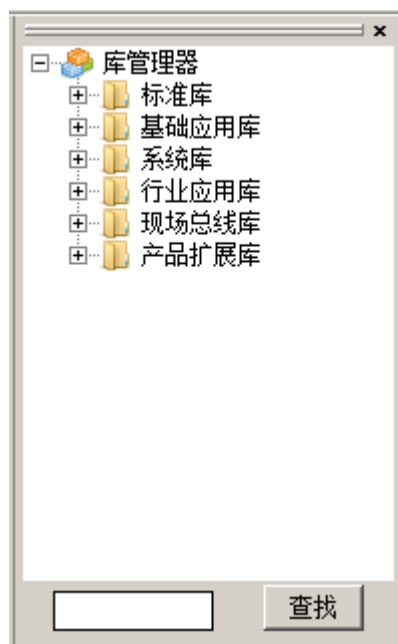
以 MC 平台库管理器为例来说明指令库的分类。

按照库中指令代码功能不同将其分为六大类：

- 标准库：IEC61131-3 标准规定的指令集、编译器通用的指令集。
- 基础应用库：具有通用性的应用库。
- 行业应用库：具体行业相关库，如运动控制行业库、化工行业库。
- 系统库：OS 提供的通用系统接口指令，RTS 提供的通用 PLC 系统接口指令。
- 现场总线库：各种现场总线提供的接口。当添加支持现场总线的控制器时自动显示该指令库。

- 产品扩展库: 特定产品提供的定制接口指令。

“库管理器”窗口如图所示。

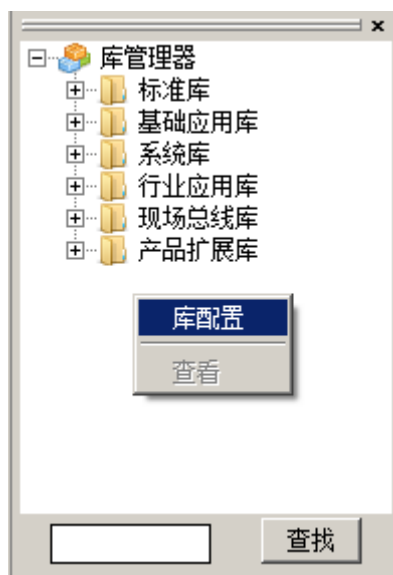


库管理器窗口

2.3 指令库的添加

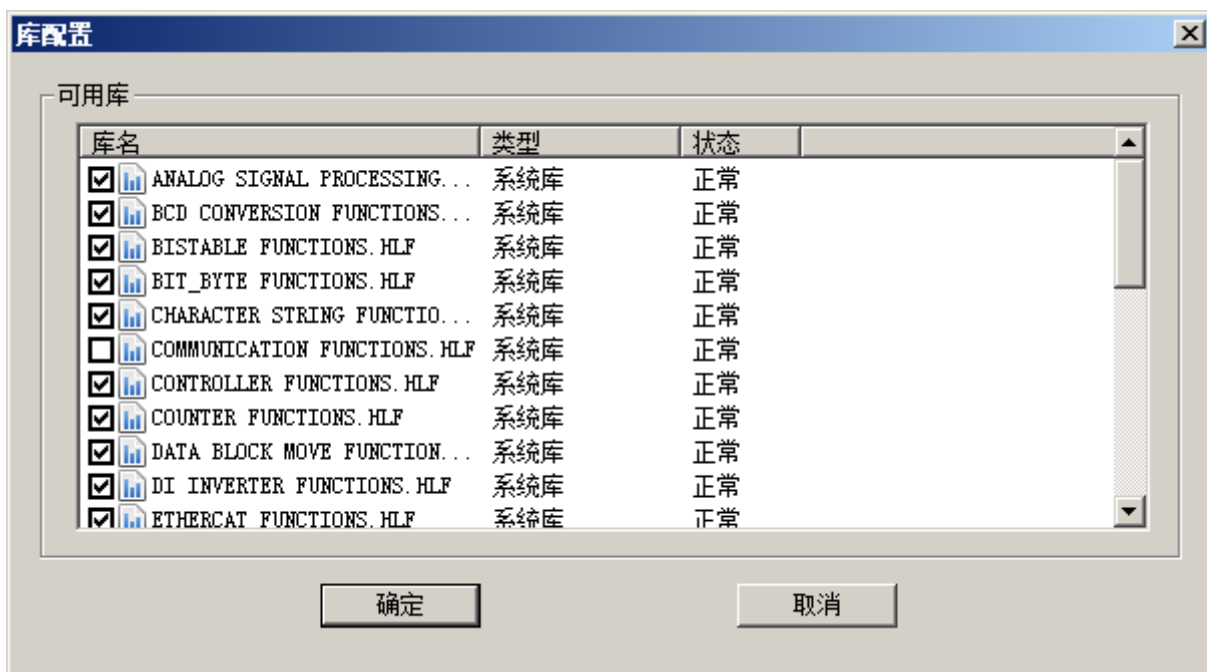
库管理器中的指令默认全部使能，用户也可根据需要进行手动配置。

- 库管理器: 右击空白区域，单击【库配置】。



库配置菜单项

弹出“库配置”对话框，如图所示，勾选需要添加的库文件，单击**确定**按钮，添加到“库管理器”窗口中。



“库配置”对话框

2.4 指令系统使用注意事项

- 上升沿使能，是指每当使能端由低电平变为高电平并一直保持高电平时，执行指令的相关功能。而且，如果指令的输入参数发生变化，只有使能端再次由低电平变为高电平，才能生效。

- 高电平使能，是指当使能端保持高电平时，执行指令的相关功能，并且如果指令的输入参数发生变化，则可以实时生效。
- 在使用数学运算指令时，若输出数据定义的范围小于运算结果，则高位丢失。
- 若指令的实例名声明在 **RETAIN**（掉电保持）区，该指令的所有输入/输出变量都要占用 **RETAIN** 内存区，所以建议不要在 **RETAIN** 内存区声明太多的实例，以免 **RETAIN** 区空间不足。
- **LK** 系列模块 **M** 区的前 4000 个字节，即 **%MB0~%MB3999**，已具有掉电保持功能。**LE** 系列模块 **M** 区的 **%MB1000~%MB1999** 已具有掉电保持功能。
- 在梯形图（**LD**）编程环境下，插入使能运算符与插入功能块是两种不同的指令调用方式。其不同在于，如果采用插入使能运算符调用指令，当使能端低电平时，相应的指令代码不会被扫描，如果采用插入功能块调用指令，不论使能端低电平或高电平，相应指令代码都会将使能端作为一个输入值来扫描。
- 64 位整型数据类型 **LINT**、**ULINT**、**LWORD** 仅 **MC** 控制器支持。
- 对于 64 位整型到 **LREAL** 的隐含转换，编译时报警告。
- 对于数学运算，当输入值超出规定的定义域时，输出将显示无效值：
 - **1.#INF**：表示一个无穷大的数，包括正无穷大 **1.#INF** 和负无穷大 **-1.#INF**。
 - **1.#IND**：表示无穷小。
 - **1.#QNAN**：表示一个非数。
- **LE5128** 电子齿轮不支持反向同步。

第3章 指令库

3.1 标准库

3.1.1 数学运算

3.1.1.1 ADD（加）

3.1.1.1.1 功能

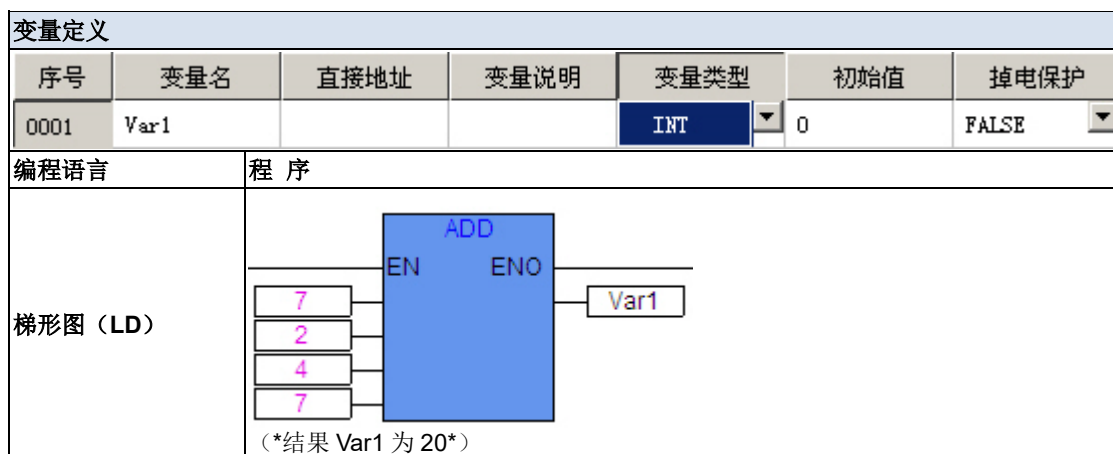
两个（或者多个）变量或常量相加。计算结果超出输出变量的取值范围将显示溢出值，对于 REAL 类型变量，运算超范围溢出，则输出显示无效值“1.#INF”或“-1.#INF”，无效值定义请参见[指令系统使用注意事项](#)。

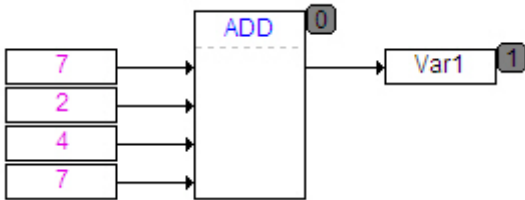
3.1.1.1.2 输入/输出数据类型

BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.1.1.3 指令使用举例



变量定义	
结构化文本 (ST)	Var1:=7+2+4+7; (*结果 Var1 为 20*)
连续功能图 (CFC)	 (*结果 Var1 为 20*)



- TIME 型量也可以使用加法功能，两个 TIME 型量相加得到一个新的时间量。例如：t#45s + t#50s = t#1m35 s。
- 被选择的输出数据类型应可以存储输出结果，否则可能引起数据错误。MUL、SUB、DIV 指令同样。

3.1.1.2 SUB (减)

3.1.1.2.1 功能

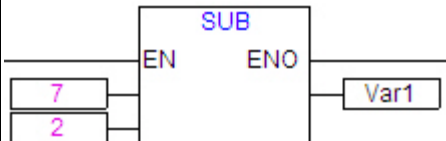
两个变量或常量相减。

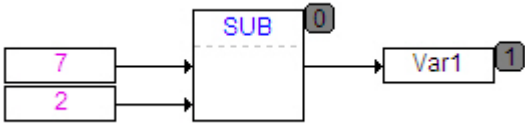
3.1.1.2.2 输入/输出数据类型

BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.1.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言		程序				
梯形图 (LD)						

变量定义	
	(*结果 Var1 为 5*)
结构化文本 (ST)	Var1:=7-2; (*结果 Var1 为 5*)
连续功能图 (CFC)	 (*结果 Var1 为 5*)

i TIME 型量也可以使用减法功能，两个 TIME 型量相减得到一个新的时间量。例如：t#1m35s - t#50s = t#45s，但时间结果不能有负值。

3.1.1.3 MUL (乘)

3.1.1.3.1 功能

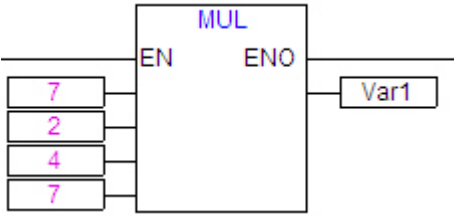
两个（或者多个）变量或常量相乘。

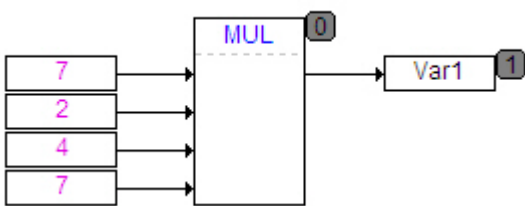
3.1.1.3.2 输入/输出数据类型

BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.1.3.3 指令使用举例

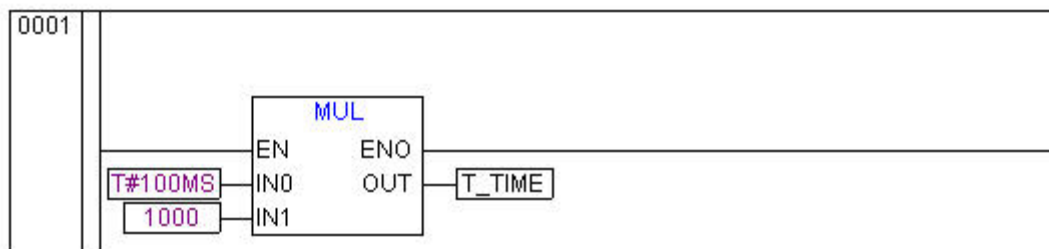
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 392*)				

变量定义	
结构化文本 (ST)	Var1:=7*2*4*7; (*结果 Var1 为 392*)
连续功能图 (CFC)	 (*结果 Var1 为 392*)

对于 TIME 类型的变量或常量可以乘以整型变量或常量，结果为 TIME 类型变量。例如：在 AutoThink 里支持 T#100MS 的时间常量*1000 的常量=时间。如下图所示。

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	T_TIME			TIME	T#0MS	FALSE

Test:



TIME 类型乘法运算

对于 TIME 类型的乘法，只有第一个操作数为 TIME 类型数据，其余均为整型时，编译通过。其余情况报错，例如：T_TIME:=1000*T#100ms;

- LD、CFC 语言报错
 - 类型不匹配：不能将‘TIME’型转为‘WORD’型。
- ST 语言报错
 - 乘除运算：类型不匹配。

3.1.1.4 DIV (除)

3.1.1.4.1 功能

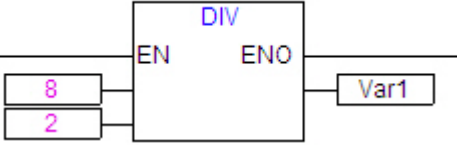
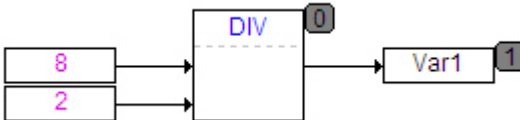
变量或常量相除。当除数为 0 时，结果等于被除数。

3.1.1.4.2 输入/输出数据类型

BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.1.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 4*)				
结构化文本 (ST)		<pre>Var1:=8/2;</pre> (*结果 Var1 为 4*)				
连续功能图 (CFC)		 (*结果 Var1 为 4*)				

i 在工程中使用 DIV 指令时，可使用 CheckDivByte、CheckDivWord、CheckDivDWord 和 CheckDivReal 等指令检查除数是否为零，避免了除数为零的现象。

3.1.1.5 MOD (取余)

3.1.1.5.1 功能

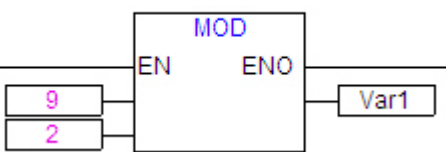
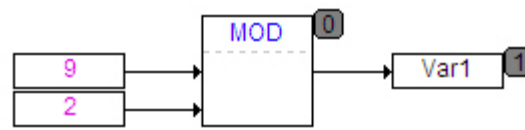
变量或常量相除取余，是一个整数。当除数为 0 时，结果等于被除数，结果符号跟被除数一致。

3.1.1.5.2 输入/输出数据类型

BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、LINT、ULINT、LWORD。

① 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.1.5.3 指令使用举例

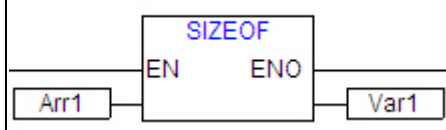
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言		程序				
梯形图 (LD)		 (*结果 Var1 为 1*)				
结构化文本 (ST)		<pre>Var1:=9 MOD 2;</pre> (*结果 Var1 为 1*)				
连续功能图 (CFC)		 (*结果 Var1 为 1*)				

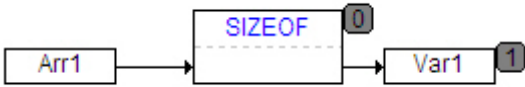
3.1.1.6 SIZEOF (字节长度)

3.1.1.6.1 功能

取得数据类型所需字节数。

3.1.1.6.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Arr1			ARRAY[0..4] OF INT		FALSE
0002	Var1			INT	0	FALSE
编程语言		程序				
梯形图 (LD)		 (*结果 Var1 为 10*)				

变量定义	
结构化文本 (ST)	Var1:=SIZEOF(Arr1); (*结果 Var1 为 10*)
连续功能图 (CFC)	 (*结果 Var1 为 10*)

3.1.1.7 ABS (绝对值)

3.1.1.7.1 功能

把输入数据的绝对值赋予输出变量。

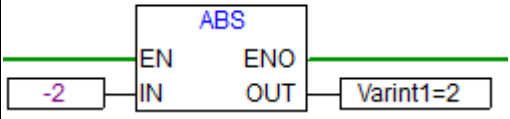
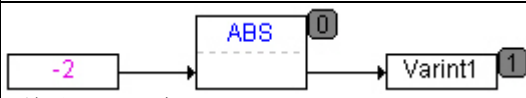
3.1.1.7.2 输入/输出数据类型

输入数据类型	输出数据类型
INT	INT、WORD、DWORD、DINT、UINT、REAL、LREAL
REAL	REAL
LREAL	LREAL
BYTE	INT、BYTE、WORD、DWORD、DINT、UINT、REAL、LREAL
WORD	WORD、DWORD、DINT、REAL、LREAL
DWORD	DWORD、DINT、REAL、LREAL
SINT	INT、BYTE、WORD、DWORD、DINT、UINT、REAL、LREAL
USINT	INT、BYTE、WORD、DWORD、DINT、UINT、REAL、LREAL
UINT	WORD、DWORD、DINT、UINT、REAL、LREAL
DINT	DWORD、DINT、REAL、LREAL
UDINT	DWORD、DINT、UDINT、REAL、LREAL
LINT	LINT、ULINT、LWORD、LREAL
ULINT	LINT、ULINT、LWORD、LREAL
LWORD	LINT、ULINT、LWORD、LREAL



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.7.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varint1			INT	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		Varint1:=ABS(-2); (*结果 Varint1 为 2*)				
连续功能图 (CFC)		 (*结果 Varint1 为 2*)				

3.1.1.8 SQRT (平方根)

3.1.1.8.1 功能

对输入数据求平方根，输入数据必须为非负数。

MC/LK 平台，当输入为负数时，输出为无穷小。

LE/MKC 平台，当输入为负数时，输出无效值。无效值定义请参见[指令系统使用注意事项](#)。

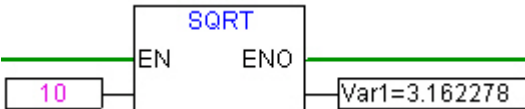
3.1.1.8.2 输入/输出数据类型

BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.8.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		Var1:=SQRT(10); (*结果 Var1 为 3.162278*)				
连续功能图 (CFC)						

3.1.1.9 LOG（常用对数）

3.1.1.9.1 功能

对输入数据求以 10 为底的对数，输入数据必须为正数。

MC/LK 平台，当输入 ≤ 0 时，输出为无穷小。

LE/MKC 平台，当输入 ≤ 0 时，输出为无效值；无效值定义请参见[指令系统使用注意事项](#)。

3.1.1.9.2 输入/输出数据类型

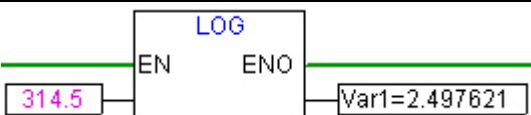
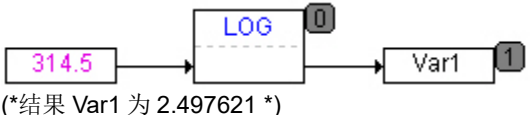
可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型。



- 对于 LE、MKC 工程，输入输出数据不支持 LREAL 类型。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.9.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>Var1:=LOG(314.5);</pre> (*结果 Var1 为 2.497621 *)
连续功能图 (CFC)	 (*结果 Var1 为 2.497621 *)

3.1.1.10 LN（自然对数）

3.1.1.10.1 功能

对输入数据求自然对数，输入数据必须为正数。

MC/LK 平台，当输入 ≤ 0 时，输出为无穷小；

LE/MKC 平台，当输入 ≤ 0 时，输出为无效值，无效值定义请参见[指令系统使用注意事项](#)。

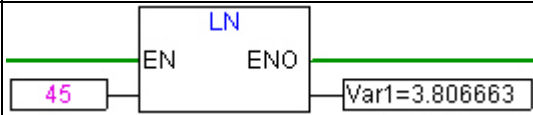
3.1.1.10.2 输入/输出数据类型

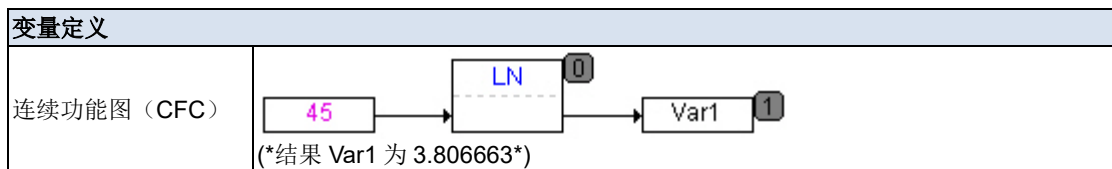
可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.10.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		Var1:=LN(45); （*结果 Var1 为 3.806663*）				



3.1.1.11 EXP（指数）

3.1.1.11.1 功能

以输入数据为指数的幂计算，即 $y=e^x$ ，其中 x 为输入， y 为输出。

MC/LK 平台，当输入 >709.8 时，输出值为无穷大“1.#INF”；当输入 <-745.2 时（MC <-708.3 ），输出值为 0。

LE/MKC 平台，当输入 >88.7228 时，输出为无效值，无效值定义请参见[指令系统使用注意事项](#)。

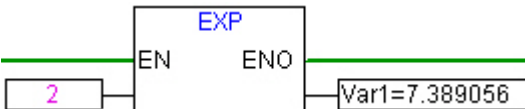
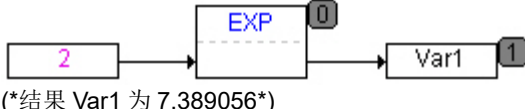
3.1.1.11.2 输入/输出数据类型

可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.11.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		Var1:=EXP(2); (*结果 Var1 为 7.389056*)				
连续功能图（CFC）		 (*结果 Var1 为 7.389056*)				

3.1.1.12 SIN（正弦）

3.1.1.12.1 功能

求输入数据的正弦值，输入数据以弧度表示。

$$\text{弧度(rad)} = \text{角度} * \frac{\pi}{180^{\circ}}$$

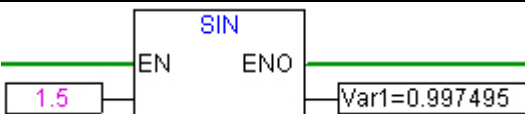
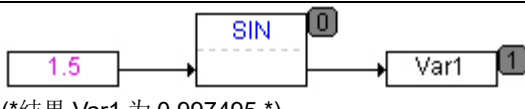
3.1.1.12.2 输入/输出数据类型

输入数据类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.12.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		Var1:=SIN(1.5); (*结果 Var1 为 0.997495 *)				
连续功能图 (CFC)						

3.1.1.13 COS（余弦）

3.1.1.13.1 功能

求输入数据的余弦值，输入数据以弧度表示。

$$\text{弧度(rad)} = \text{角度} * \frac{\pi}{180^{\circ}}$$

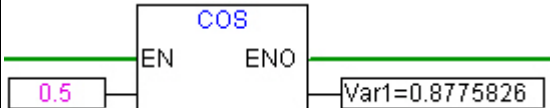
3.1.1.13.2 输入/输出数据类型

输入数据类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.13.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		Var1:=COS(0.5); (*结果 Var1 为 0.8775826 *)				
连续功能图 (CFC)		 (*结果 Var1 为 0.8775826 *)				

3.1.1.14 TAN（正切）

3.1.1.14.1 功能

求输入数据的正切值，输入数据以弧度表示。

$$\text{弧度(rad)} = \text{角度} * \frac{\pi}{180^{\circ}}$$

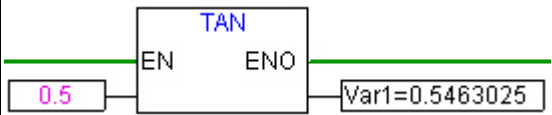
3.1.1.14.2 输入/输出数据类型

输入数据类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.14.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		Var1:=TAN(0.5); (*结果 Var1 为 0.5463025 *)				
连续功能图（CFC）						

3.1.1.15 ASIN（反正弦）

3.1.1.15.1 功能

求输入数据的反正弦值。输入值取值：[-1,+1]。

MC/LK 平台，输入不在[-1,1]范围时，输出为无穷小。

LE/MKC 平台，输入不在[-1,1]范围时，输出为无效值，无效值定义请参见[指令系统使用注意事项](#)。

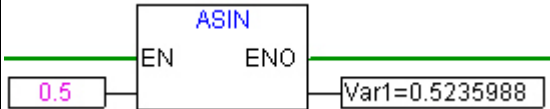
3.1.1.15.2 输入/输出数据类型

输入数据类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型，输出数据以弧度表示。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.15.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		Var1:=ASIN(0.5); (*结果 Var1 为 0.5235988 *)				
连续功能图（CFC）		 (*结果 Var1 为 0.5235988 *)				

3.1.1.16 ACOS（反余弦）

3.1.1.16.1 功能

求输入数据的反余弦值。输入值取值：[-1,+1]。

MC/LK 平台，输入不在[-1,1]范围时，输出为无穷小。

LE/MKC 平台，输入不在[-1,1]范围时，输出为无效值，无效值定义请参见[指令系统使用注意事项](#)。

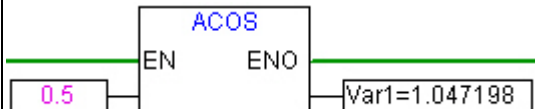
3.1.1.16.2 输入/输出数据类型

输入数据类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型，输出数据以弧度表示。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.16.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		Var1:=ACOS(0.5); (*结果 Var1 为 1.047198 *)				
连续功能图 (CFC)		 (*结果 Var1 为 1.047198 *)				

3.1.1.17 ATAN（反正切）

3.1.1.17.1 功能

求输入数据的反正切值。

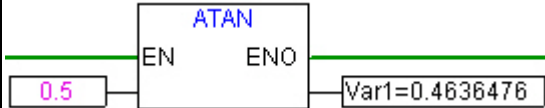
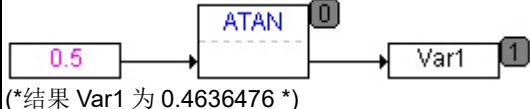
3.1.1.17.2 输入/输出数据类型

输入数据类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 类型，输出数据以弧度表示。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.17.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		<pre>Var1:=ATAN(0.5); (*结果 Var1 为 0.4636476 *)</pre>				
连续功能图（CFC）						

3.1.1.18 EXPT（幂）

3.1.1.18.1 功能

计算以 x 为底，y 为指数的幂。x 为 IN1，y 为 IN2。

MC/LK 平台，如果 x 和 y 都接近 0，则输出为 1。如果 x 接近 0，且 y<0 时，输出为无穷小。当 x<0，且 y 不是整数时，输出为非法浮点值“1.#QNAN”。运算结果溢出时，输出为无效值，无效值定义请参见

指令系统使用注意事项。

LE 平台， $x=0$ ，且 $y \leq 0$ ，输出为无效值（仿真模式， $x=0$ ，且 $y=0$ ，输出为 1）。 $x \neq 0$ ，且 $y=0$ ，输出为 1。运算结果溢出后，输出为无效值。无效值定义请参见[指令系统使用注意事项](#)。

MKC 平台， $x=0$ ，且 $y < 0$ ，输出为无效值。 $y=0$ ，输出为 1。运算结果溢出后，输出为无效值。无效值定义请参见[指令系统使用注意事项](#)。

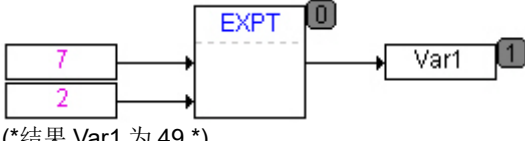
3.1.1.18.2 输入/输出数据类型

输入数据的类型可以是 BYTE、WORD、DWORD、INT、DINT、REAL、LREAL、SINT、USINT、UINT、UDINT、LINT、ULINT、LWORD，输出数据必须是 REAL、LREAL 型。



- 对于 LE、MKC 工程，输入输出数据类型不支持 LREAL。
- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- 当输入是 64 位整型数据类型时，编译会报警告。

3.1.1.18.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			REAL	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		<pre>Var1:=EXPT(7,2); (*结果 Var1 为 49 *)</pre>				
连续功能图 (CFC)						

i EXPT 在不同的平台，运算结果可能存在差异。例如：`expt(0,0)`在仿真（X86 平台）下就是 1，在控制器（XE164 平台）运行结果为 NAN。同样的，LM 系列 PLC 仿真与控制器运行也存在差异。

3.1.1.19 MOVE（赋值）

3.1.1.19.1 功能

将一个常量或者变量的值赋给另外一个变量。

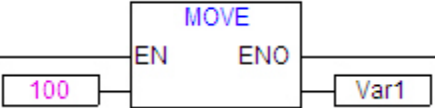
3.1.1.19.2 输入/输出数据类型

BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DT、BOOL、ARRAY、LREAL、STRING、TOD、DATE、LINT、ULINT、LWORD。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.1.19.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言		程 序				
梯形图（LD）		 （*结果 Var1 为 100*）				
结构化文本（ST）		Var1:=100; （*结果 Var1 为 100*）				
连续功能图（CFC）		 （*结果 Var1 为 100*）				

①STRING 类型变量进行赋值运算时，MOVE 块连接的两个变量长度不相等时，以右侧变量的字符长度为准进行取舍。

3.1.2 逻辑运算

3.1.2.1 AND（与）

3.1.2.1.1 功能

变量或常量的相与运算。

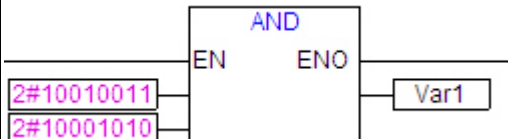
3.1.2.1.2 输入/输出数据类型

BOOL、BYTE、WORD、DWORD、LWORD。

i 64 位整型数据类型 LWORD 仅 MC 控制器支持。

3.1.2.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE	0	FALSE

编程语言	程 序
梯形图（LD）	 (*结果 Var1 为 2#10000010*)
结构化文本（ST）	<pre>Var1:=2#10010011 AND 2#10001010;</pre> (*结果 Var1 为 2#10000010*)
连续功能图（CFC）	 (*结果 Var1 为 2#10000010*)

3.1.2.2 OR（或）

3.1.2.2.1 功能

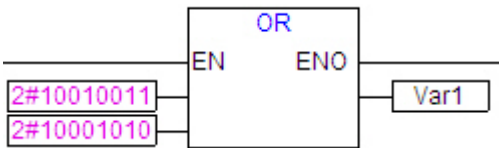
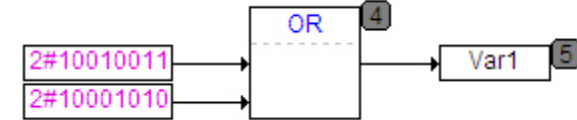
变量或常量的相或运算。

3.1.2.2.2 输入/输出数据类型

BOOL、BYTE、WORD、DWORD、LWORD。

i 64 位整型数据类型 LWORD 仅 MC 控制器支持。

3.1.2.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE	0	FALSE
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 2#10011011*)				
结构化文本 (ST)		<pre>Var1:=2#10010011 OR 2#10001010 ;</pre> (*结果 Var1 为 2#10011011*)				
连续功能图 (CFC)		 (*结果 Var1 为 2#10011011*)				

3.1.2.3 XOR (异或)

3.1.2.3.1 功能

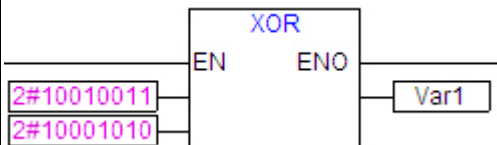
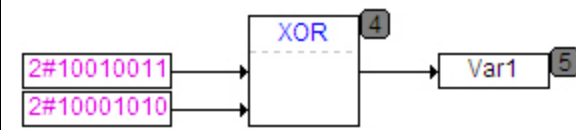
变量或常量的异或运算。

3.1.2.3.2 输入/输出数据类型

BOOL、BYTE、WORD、DWORD、LWORD。

i 64 位整型数据类型 LWORD 仅 MC 控制器支持。

3.1.2.3.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE	0	FALSE
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 2#00011001*)				
结构化文本 (ST)		Var1:=2#10010011 XOR 2#10001010 ; (*结果 Var1 为 2#00011001*)				
连续功能图 (CFC)		 (*结果 Var1 为 2#00011001*)				

3.1.2.4 NOT (非)

3.1.2.4.1 功能

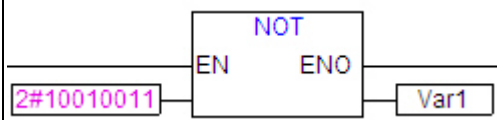
变量或常量的取非运算，逐位取非。

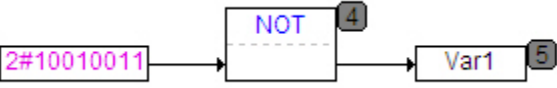
3.1.2.4.2 输入/输出数据类型

BOOL、BYTE、WORD、DWORD、LWORD。

①64 位整型数据类型 LWORD 仅 MC 控制器支持。

3.1.2.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE	0	FALSE
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 2#01101100*)				
结构化文本 (ST)		Var1:= NOT 2#10010011 ; (*结果 Var1 为 2#01101100*)				

变量定义	
连续功能图（CFC）	 (*结果 Var1 为 2#01101100*)

3.1.3 比较运算

3.1.3.1 GT（大于）

3.1.3.1.1 功能

判断两个操作数的大小，当第一个数大于第二个数时输出 TRUE，否则输出为 FALSE。

3.1.3.1.2 输入/输出数据类型

输入数据类型：BOOL、BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DATE、TOD、DT、LREAL、STRING、LINT、ULINT、LWORD；

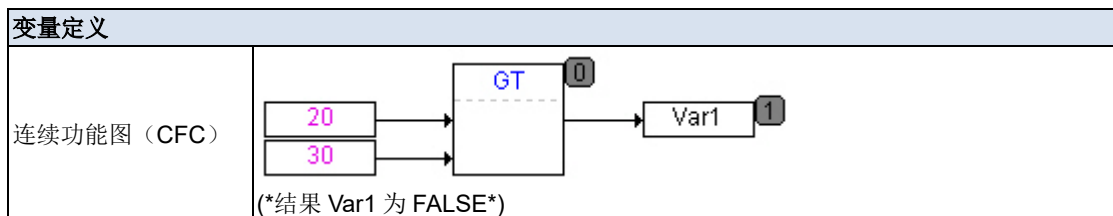
输出数据类型：BOOL。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.3.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
0002	Var2			INT	0	FALSE
0003	Var3			BOOL	FALSE	FALSE
编程语言		程序				
梯形图（LD）		GT EN ENO Var1 Var2 Var3 (*结果 Var3 为 FALSE*)				
结构化文本（ST）		<pre>Var3:=20>30;</pre> (*结果 Var3 为 FALSE*)				



3.1.3.2 LT (小于)

3.1.3.2.1 功能

判断两个操作数的大小，当第一个数小于第二个数时返回 TRUE，否则结果为 FALSE。

3.1.3.2.2 输入/输出数据类型

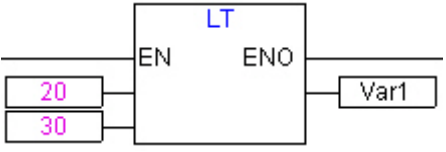
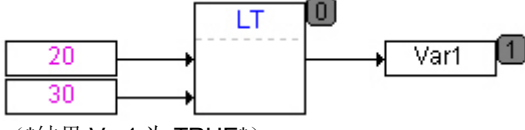
输入数据类型：BOOL、BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DATE、TOD、DT、LREAL、STRING、LINT、ULINT、LWORD；

输出数据类型：BOOL。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.3.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BOOL	FALSE	FALSE
编程语言		程序				
梯形图 (LD)		 (*结果 Var1 为 TRUE*)				
结构化文本 (ST)		Var1:=20<30; (*结果 Var1 为 TRUE*)				
连续功能图 (CFC)		 (*结果 Var1 为 TRUE*)				

3.1.3.3 GE（大于等于）

3.1.3.3.1 功能

判断两个操作数的大小，当第一个数大于等于第二个数时返回 TRUE，否则结果为 FALSE。

3.1.3.3.2 输入/输出数据类型

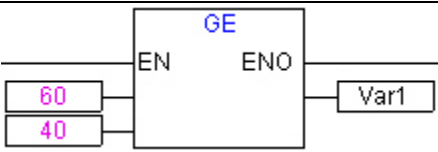
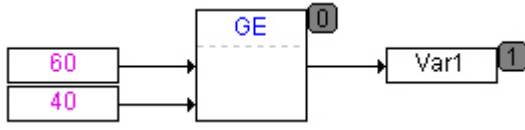
输入数据类型：BOOL、BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DATE、TOD、DT、LREAL、STRING、LINT、ULINT、LWORD；

输出数据类型：BOOL。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.3.3.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BOOL	FALSE	FALSE
编程语言		程序				
梯形图（LD）		 (*结果 Var1 为 TRUE*)				
结构化文本（ST）		<pre>Var1:=60>=40;</pre> (*结果 Var1 为 TRUE*)				
连续功能图（CFC）		 (*结果 Var1 为 TRUE*)				

3.1.3.4 LE（小于等于）

3.1.3.4.1 功能

判断两个操作数的大小，当第一个数小于等于第二个数时返回 TRUE，否则结果为 FALSE。

3.1.3.4.2 输入/输出数据类型

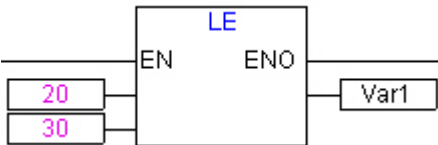
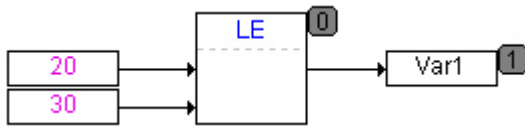
输入数据类型：BOOL、BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DATE、TOD、DT、LREAL、STRING、LINT、ULINT、LWORD；

输出数据类型：BOOL。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.3.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BOOL	FALSE	FALSE
编程语言		程序				
梯形图 (LD)		 (*结果 Var1 为 TRUE*)				
结构化文本 (ST)		<pre>Var1:=20<=30;</pre> (*结果 Var1 为 TRUE*)				
连续功能图 (CFC)		 (*结果 Var1 为 TRUE*)				

3.1.3.5 EQ (等于)

3.1.3.5.1 功能

判断两个操作数是否相等，当第一个数等于第二个数时返回 TRUE，否则结果为 FALSE。

3.1.3.5.2 输入/输出数据类型

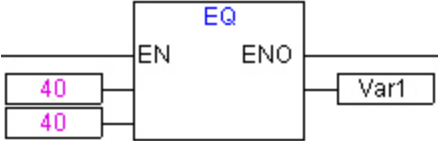
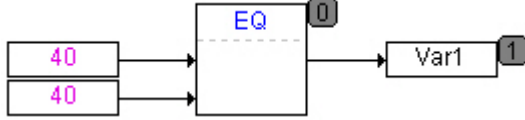
输入数据类型：BOOL、BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DATE、TOD、DT、LREAL、STRING、LINT、ULINT、LWORD；

输出数据类型：BOOL。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.3.5.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BOOL	FALSE	FALSE
编程语言		程序				
梯形图 (LD)		 (*结果 Var1 为 TRUE*)				
结构化文本 (ST)		<pre>Var1:=40=40;</pre> (*结果 Var1 为 TRUE*)				
连续功能图 (CFC)		 (*结果 Var1 为 TRUE*)				

3.1.3.6 NE (不等于)

3.1.3.6.1 功能

判断两个操作数是否不相等，当第一个数不等于第二个数时返回 TRUE，否则结果为 FALSE。

3.1.3.6.2 输入/输出数据类型

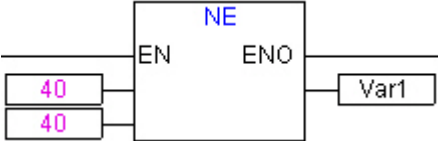
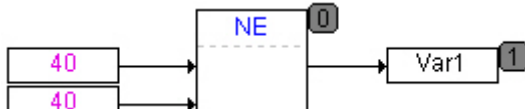
输入数据类型：BOOL、BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、REAL、TIME、DATE、TOD、DT、LREAL、STRING、LINT、ULINT、LWORD；

输出数据类型：BOOL。



- 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。
- LE、MKC 平台不支持 LREAL、STRING 类型数据。

3.1.3.6.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BOOL	FALSE	FALSE
编程语言		程序				
梯形图 (LD)		 (*结果 Var1 为 FALSE*)				
结构化文本 (ST)		<pre>Var1:=40<>40;</pre> (*结果 Var1 为 FALSE*)				
连续功能图 (CFC)		 (*结果 Var1 为 FALSE*)				

3.1.4 选择运算

3.1.4.1 SEL (二选一)

3.1.4.1.1 功能

通过选择开关在两个输入数据中选择一个作为输出，选择开关为 FALSE 时输出为第一个输入数据，选择开关为 TRUE 时输出为第二个输入数据。

3.1.4.1.2 指令格式

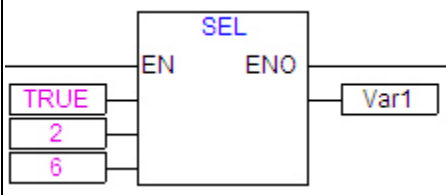
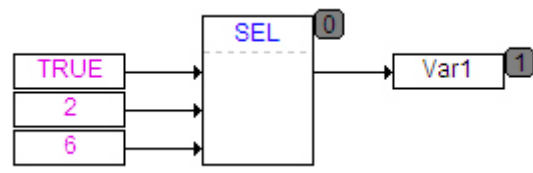
OUT := SEL(G, IN0, IN1)，其中 G 为选择开关，IN0 和 IN1 分别为第一个输入数据和第二个输入数据。

3.1.4.1.3 输入/输出数据类型

G 必须是 BOOL 类型，IN0、IN1 和输出数据可以是任意数据类型（不包括 STRING、数组、指针类型）。

i64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.4.1.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 6 *)				
结构化文本 (ST)		Var1:=SEL(TRUE,3,4); (*结果 Var1 为 4 *)				
连续功能图 (CFC)		 (*结果 Var1 为 6 *)				

3.1.4.2 MAX (取最大值)

3.1.4.2.1 功能

在两个输入数据中选择最大值作为输出。

3.1.4.2.2 指令格式

OUT:=MAX(IN0, IN1), 其中 IN0 和 IN1 分别为第 1 个输入数据和第 2 个输入数据, OUT 是输出数据。

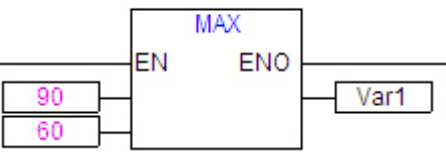
3.1.4.2.3 输入/输出数据类型

IN0, IN1 和 OUT 可以是任意数据类型 (不包括 BOOL、STRING、数组、指针类型)。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.4.2.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
0002	Var2			INT	0	FALSE

编程语言	程 序
梯形图 (LD)	 (*结果 Var1 为 90 *)
结构化文本 (ST)	<pre>Var1:=MAX(90,60); (*结果 Var1 为 90 *) Var2:=MAX(40,MAX(90,60)); (*结果 Var2 为 90 *)</pre>
连续功能图 (CFC)	 (*结果 Var1 为 90 *)

3.1.4.3 MIN（取最小值）

3.1.4.3.1 功能

在两个输入数据中选择最小值作为输出。

3.1.4.3.2 指令格式

OUT:=MIN(IN0, IN1), 其中 IN0 和 IN1 分别为第 1 个输入数据和第 2 个输入数据, OUT 是输出数据。

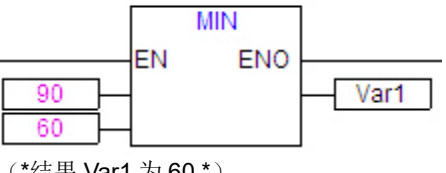
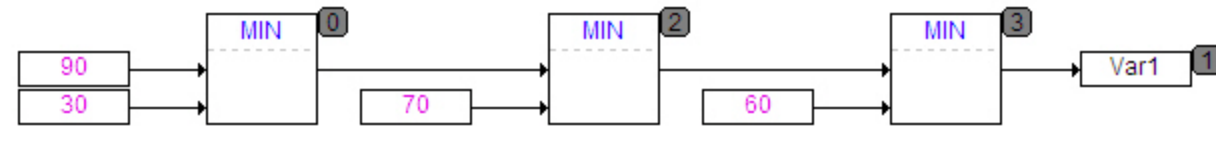
3.1.4.3.3 输入/输出数据类型

IN0, IN1 和 OUT 可以是任意数据类型（不包括 BOOL、STRING、数组、指针类型）。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.4.3.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
0002	Var2			INT	0	FALSE

编程语言程 序	
梯形图 (LD)	 (*结果 Var1 为 60 *)
结构化文本 (ST)	<pre>Var1:=MIN(90,30); (*结果 Var1 为 30 *) Var2:=MIN(MIN(90,30),60); (*结果 Var2 为 30 *)</pre>
连续功能图 (CFC)	 (*结果 Var1 为 30 *)

3.1.4.4 MUX（多选一）

3.1.4.4.1 功能

通过控制数在多个输入数据中选择一个作为输出。

3.1.4.4.2 指令格式

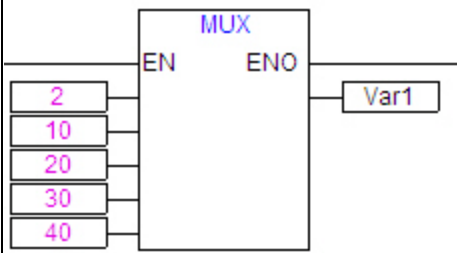
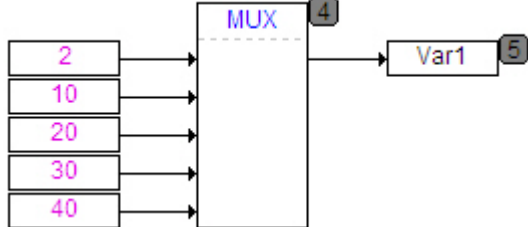
OUT:=MUX(K,IN0,...,INn)，其中 K 为控制数，IN0, ..., INn 为输入数据，OUT 为输出结果。控制数为 K 时选择第 INk 个输入数据作为输出。

3.1.4.4.3 输入/输出数据类型

IN0, ..., INn 和 OUT 可以是任意数据类型（不包括 BOOL、STRING、数组、指针类型），K 必须是 BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.4.4.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言	程 序					
梯形图 (LD)	 (*2 为控制数据, 对应于 30, 所以输出 30 *)					
结构化文本 (ST)	<pre>Var1:=MUX(0,30,40,50,60,70,80);</pre> (*结果 Var1 为 30*)					
连续功能图 (CFC)	 (*2 为控制数据, 对应于 30, 所以结果为 30*)					

3.1.4.5 LIMIT (限幅)

3.1.4.5.1 功能

判断输入数据是否在最小值和最大值之间, 若输入数据在二者之间, 则直接把输入数据作为输出数据进行输出。若输入数据大于最大值, 则把最大值作为输出值。若输入数据小于最小值, 则把最小值作为输出值。

3.1.4.5.2 指令格式

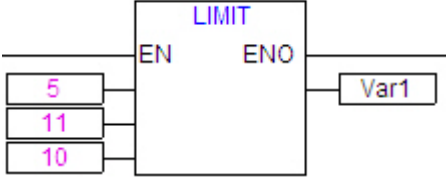
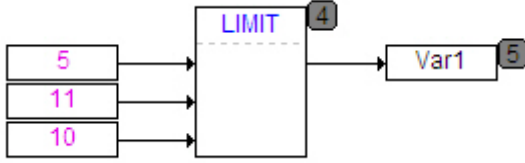
OUT := LIMIT(Min, IN, Max)。

3.1.4.5.3 输入/输出数据类型

IN 和 OUT 可以是任意数据类型 (不包括 BOOL、STRING、数组、指针类型)。

①64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.4.5.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			INT	0	FALSE
编程语言	程 序					
梯形图 (LD)	 (*11 为输入数据, 5 为最小值, 10 为最大值, 结果 Var1 为 10*)					
结构化文本 (ST)	Var1:=LIMIT(30,90,80); (*结果 Var1 为 80 *)					
连续功能图 (CFC)	 (*11 为输入数据, 5 为最小值, 10 为最大值, 结果 Var1 为 10*)					

3.1.5 移位运算

3.1.5.1 SHL (左移)

3.1.5.1.1 功能

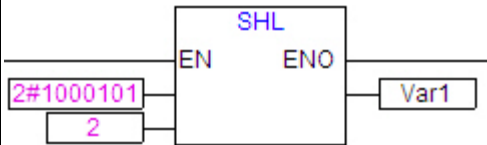
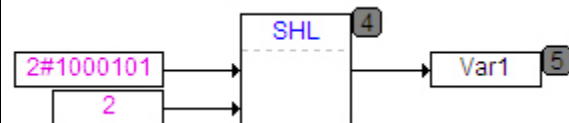
对操作数进行按位左移，左边移出的位不作处理，右边自动补 0。

3.1.5.1.2 输入/输出数据类型

BYTE、INT、WORD、DWORD、SINT、UINT、USINT、DINT、LINT、ULINT、LWORD。

①64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.5.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE ▼	0	FALSE ▼
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 2#10100*)				
结构化文本 (ST)		Var1:=SHL(16#45,2); (*结果 Var1 为 16#14*)				
连续功能图 (CFC)		 (*结果 Var1 为 2#10100*)				

3.1.5.2 SHR (右移)

3.1.5.2.1 功能

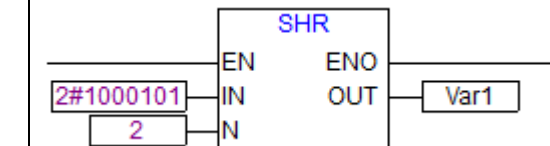
对操作数进行按位右移，右边移出的位不作处理，左边自动补 0。

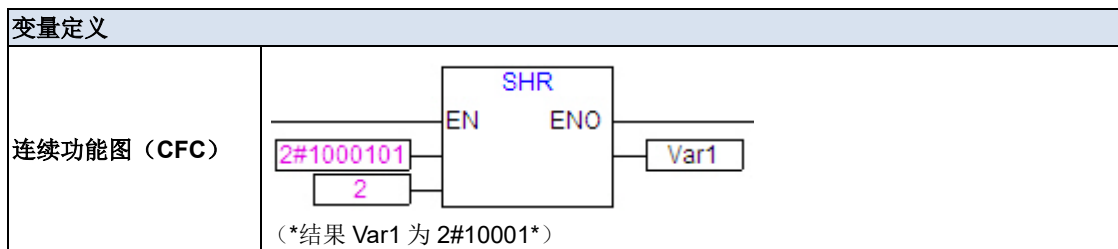
3.1.5.2.2 输入/输出数据类型

BYTE、INT、WORD、DWORD、SINT、UINT、USINT、DINT、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.5.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE ▼	0	FALSE ▼
编程语言		程 序				
梯形图 (LD)		 (*结果 Var1 为 2#10001*)				
结构化文本 (ST)		Var1:=SHR(16#45,2); (*结果 Var1 为 16#11*)				



3.1.5.3 ROL (循环左移)

3.1.5.3.1 功能

对操作数进行按位循环左移，左边移出的位直接补充到右边最低位。

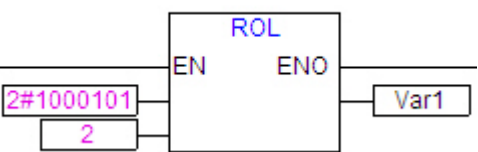
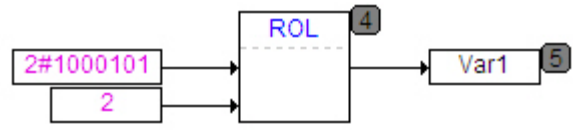
3.1.5.3.2 输入/输出数据类型

BYTE、INT、WORD、DWORD、SINT、UINT、USINT、DINT、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.5.3.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE	0	FALSE

编程语言	程 序
梯形图 (LD)	 (*结果 Var1 为 2#10101*)
结构化文本 (ST)	Var1:=ROL(16#45,2); (*结果 Var1 为 16#15*)
连续功能图 (CFC)	 (*结果 Var1 为 2#10101*)

3.1.5.4 ROR（循环右移）

3.1.5.4.1 功能

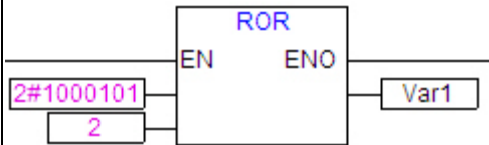
对操作数进行按位循环右移，右边移出的位直接补充到左边最高位。

3.1.5.4.2 输入/输出数据类型

BYTE、INT、WORD、DWORD、SINT、UINT、USINT、DINT、LINT、ULINT、LWORD。

i 64 位整型数据类型 LINT、ULINT、LWORD 仅 MC 控制器支持。

3.1.5.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE	0	FALSE
编程语言		程 序				
梯形图（LD）		 （*结果 Var1 为 2#1010001*）				
结构化文本（ST）		Var1:=ROR(16#45,2); （*结果 Var1 为 16#51*）				
连续功能图（CFC）		 （*结果 Var1 为 2#1010001*）				

3.1.6 数据类型转换

3.1.6.1 数据类型转换

AutoThink 提供了 200 个数据类型转换指令，用于各种数据类型之间相互转换。

语法：<TYPE1>_TO_<TYPE2>

禁止将“较大的”数据类型隐含地转换为“较小的”数据类型使用，当从较大数据类型转为较小数据类型时，有可能丢失信息。

如果被转换的值超出目标数据类型的存储范围，则这个数的高字节将被忽略。例如将 INT 类型转换为 BYTE 类型，或者将 DINT 类型转换为 WORD 类型。

下表列出了所有的数据类型转换指令。

数据类型转换指令列表

BOOL_TO_<TYPE>	BYTE_TO_<TYPE>	DATE_TO_<TYPE>	DINT_TO_<TYPE>
BOOL_TO_BYTE	BYTE_TO_BOOL		DINT_TO_BOOL
BOOL_TO_DATE	BYTE_TO_DATE		DINT_TO_BYTE
BOOL_TO_DINT	BYTE_TO_DINT	DATE_TO_BOOL	DINT_TO_DATE
BOOL_TO_DT	BYTE_TO_DT	DATE_TO_BYTE	DINT_TO_DT
BOOL_TO_DWORD	BYTE_TO_DWORD	DATE_TO_DINT	DINT_TO_DWORD
BOOL_TO_INT	BYTE_TO_INT	DATE_TO_DWORD	DINT_TO_INT
BOOL_TO_LINT	BYTE_TO_LINT	DATE_TO_INT	DINT_TO_LINT
BOOL_TO_LREAL	BYTE_TO_LREAL	DATE_TO_LINT	DINT_TO_LREAL
BOOL_TO_LWORD	BYTE_TO_LWORD	DATE_TO_LREAL	DINT_TO_LWORD
BOOL_TO_REAL	BYTE_TO_REAL	DATE_TO_LWORD	DINT_TO_REAL
BOOL_TO_STRING	BYTE_TO_SINT	DATE_TO_REAL	DINT_TO_SINT
BOOL_TO_SINT	BYTE_TO_STRING	DATE_TO_SINT	DINT_TO_STRING
BOOL_TO_TIME	BYTE_TO_TIME	DATE_TO_STRING	DINT_TO_TIME
BOOL_TO_TOD	BYTE_TO_TOD	DATE_TO_UDINT	DINT_TO_TOD
BOOL_TO_UDINT	BYTE_TO_UDINT	DATE_TO_UINT	DINT_TO_UDINT
BOOL_TO_UINT	BYTE_TO_UINT	DATE_TO_ULINT	DINT_TO_UINT
BOOL_TO_ULINT	BYTE_TO_ULINT	DATE_TO_USINT	DINT_TO_ULINT
BOOL_TO_WORD	BYTE_TO_USINT	DATE_TO_WORD	DINT_TO_USINT
	BYTE_TO_WORD		DINT_TO_WORD
DT_TO_<TYPE>	DWORD_TO_<TYPE>	INT_TO_<TYPE>	LINT_TO_<TYPE>
DT_TO_BOOL	DWORD_TO_BOOL	INT_TO_BOOL	LINT_TO_BOOL
DT_TO_BYTE	DWORD_TO_BYTE	INT_TO_BYTE	LINT_TO_BYTE
DT_TO_DINT	DWORD_TO_DATE	INT_TO_DATE	LINT_TO_DATE
DT_TO_DWORD	DWORD_TO_DINT	INT_TO_DINT	LINT_TO_DINT
DT_TO_INT	DWORD_TO_DT	INT_TO_DT	LINT_TO_DT
DT_TO_LINT	DWORD_TO_INT	INT_TO_DWORD	LINT_TO_DWORD
DT_TO_LREAL	DWORD_TO_LINT	INT_TO_LINT	LINT_TO_INT
DT_TO_LWORD	DWORD_TO_LREAL	INT_TO_LREAL	LINT_TO_LREAL
DT_TO_REAL	DWORD_TO_LWORD	INT_TO_LWORD	LINT_TO_LWORD
DT_TO_SINT	DWORD_TO_REAL	INT_TO_REAL	LINT_TO_REAL
DT_TO_STRING	DWORD_TO_SINT	INT_TO_SINT	LINT_TO_SINT
DT_TO_UDINT	DWORD_TO_STRING	INT_TO_STRING	LINT_TO_STRING
DT_TO_UINT	DWORD_TO_TIME	INT_TO_TIME	LINT_TO_TIME
DT_TO_ULINT	DWORD_TO_TOD	INT_TO_TOD	LINT_TO_TOD
DT_TO_USINT	DWORD_TO_UDINT	INT_TO_UDINT	LINT_TO_UDINT
DT_TO_WORD	DWORD_TO_UINT	INT_TO_UINT	LINT_TO_UINT
	DWORD_TO_ULINT	INT_TO_ULINT	LINT_TO_ULINT
	DWORD_TO_USINT	INT_TO_USINT	LINT_TO_USINT
	DWORD_TO_WORD	INT_TO_WORD	LINT_TO_WORD
LREAL_TO_<TYPE>	LWORD_TO_<TYPE>	REAL_TO_<TYPE>	SINT_TO_<TYPE>
LREAL_TO_BOOL	LWORD_TO_BOOL	REAL_TO_BOOL	SINT_TO_BOOL
LREAL_TO_BYTE	LWORD_TO_BYTE	REAL_TO_BYTE	SINT_TO_BYTE
LREAL_TO_DATE	LWORD_TO_DATE	REAL_TO_DATE	SINT_TO_DATE
LREAL_TO_DINT	LWORD_TO_DINT	REAL_TO_DINT	SINT_TO_DINT
LREAL_TO_DT	LWORD_TO_DT	REAL_TO_DT	SINT_TO_DT
LREAL_TO_DWORD	LWORD_TO_DWORD	REAL_TO_DWORD	SINT_TO_DWORD
LREAL_TO_INT	LWORD_TO_INT	REAL_TO_INT	SINT_TO_INT
LREAL_TO_LINT	LWORD_TO_LINT	REAL_TO_LINT	SINT_TO_LINT
LREAL_TO_LWORD	LWORD_TO_LREAL	REAL_TO_LREAL	SINT_TO_LREAL
LREAL_TO_REAL	LWORD_TO_REAL	REAL_TO_LWORD	SINT_TO_LWORD
LREAL_TO_SINT	LWORD_TO_SINT	REAL_TO_SINT	SINT_TO_REAL
LREAL_TO_STRING	LWORD_TO_STRING	REAL_TO_STRING	SINT_TO_STRING
LREAL_TO_TIME	LWORD_TO_TIME	REAL_TO_TIME	SINT_TO_TIME

BOOL_TO_<TYPE>	BYTE_TO_<TYPE>	DATE_TO_<TYPE>	DINT_TO_<TYPE>
LREAL_TO_TOD	LWORD_TO_TOD	REAL_TO_TOD	SINT_TO_TOD
LREAL_TO_UDINT	LWORD_TO_UDINT	REAL_TO_UDINT	SINT_TO_UDINT
LREAL_TO_UINT	LWORD_TO_UINT	REAL_TO_UINT	SINT_TO_UINT
LREAL_TO_ULINT	LWORD_TO_ULINT	REAL_TO_ULINT	SINT_TO_ULINT
LREAL_TO_USINT	LWORD_TO_USINT	REAL_TO_USINT	SINT_TO_USINT
LREAL_TO_WORD	LWORD_TO_WORD	REAL_TO_WORD	SINT_TO_WORD
STRING_TO_<TYPE>	TIME_TO_<TYPE>	TOD_TO_<TYPE>	UDINT_TO_<TYPE>
STRING_TO_BOOL	TIME_TO_BOOL	TOD_TO_BOOL	UDINT_TO_BOOL
STRING_TO_BYTE	TIME_TO_BYTE	TOD_TO_BYTE	UDINT_TO_BYTE
STRING_TO_DATE	TIME_TO_DATE	TOD_TO_DATE	UDINT_TO_DATE
STRING_TO_DINT	TIME_TO_DINT	TOD_TO_DINT	UDINT_TO_DINT
STRING_TO_DT	TIME_TO_DT	TOD_TO_DT	UDINT_TO_DT
STRING_TO_DWORD	TIME_TO_DWORD	TOD_TO_DWORD	UDINT_TO_DWORD
STRING_TO_INT	TIME_TO_INT	TOD_TO_INT	UDINT_TO_INT
STRING_TO_LINT	TIME_TO_LINT	TOD_TO_LINT	UDINT_TO_LINT
STRING_TO_LREAL	TIME_TO_LREAL	TOD_TO_LREAL	UDINT_TO_LREAL
STRING_TO_LWORD	TIME_TO_LWORD	TOD_TO_LWORD	UDINT_TO_LWORD
STRING_TO_REAL	TIME_TO_REAL	TOD_TO_REAL	UDINT_TO_REAL
STRING_TO_SINT	TIME_TO_SINT	TOD_TO_SINT	UDINT_TO_SINT
STRING_TO_STRING	TIME_TO_STRING	TOD_TO_STRING	UDINT_TO_STRING
STRING_TO_TIME	TIME_TO_TIME	TOD_TO_TIME	UDINT_TO_TIME
STRING_TO_TOD	TIME_TO_TOD	TOD_TO_TOD	UDINT_TO_TOD
STRING_TO_UDINT	TIME_TO_UDINT	TOD_TO_UDINT	UDINT_TO_UDINT
STRING_TO_UINT	TIME_TO_UINT	TOD_TO_UINT	UDINT_TO_UINT
STRING_TO_ULINT	TIME_TO_ULINT	TOD_TO_ULINT	UDINT_TO_ULINT
STRING_TO_USINT	TIME_TO_USINT	TOD_TO_USINT	UDINT_TO_USINT
STRING_TO_WORD	TIME_TO_WORD	TOD_TO_WORD	UDINT_TO_WORD
UINT_TO_<TYPE>	ULINT_TO_<TYPE>	USINT_TO_<TYPE>	WORD_TO_<TYPE>
UINT_TO_BOOL	ULINT_TO_BOOL	USINT_TO_BOOL	WORD_TO_BOOL
UINT_TO_BYTE	ULINT_TO_BYTE	USINT_TO_BYTE	WORD_TO_BYTE
UINT_TO_DATE	ULINT_TO_DATE	USINT_TO_DATE	WORD_TO_DATE
UINT_TO_DINT	ULINT_TO_DINT	USINT_TO_DINT	WORD_TO_DINT
UINT_TO_DT	ULINT_TO_DT	USINT_TO_DT	WORD_TO_DT
UINT_TO_DWORD	ULINT_TO_DWORD	USINT_TO_DWORD	WORD_TO_DWORD
UINT_TO_INT	ULINT_TO_INT	USINT_TO_INT	WORD_TO_INT
UINT_TO_LINT	ULINT_TO_LINT	USINT_TO_LINT	WORD_TO_LINT
UINT_TO_LREAL	ULINT_TO_LREAL	USINT_TO_LREAL	WORD_TO_LREAL
UINT_TO_LWORD	ULINT_TO_LWORD	USINT_TO_LWORD	WORD_TO_LWORD
UINT_TO_REAL	ULINT_TO_REAL	USINT_TO_REAL	WORD_TO_REAL
UINT_TO_SINT	ULINT_TO_SINT	USINT_TO_SINT	WORD_TO_SINT
UINT_TO_STRING	ULINT_TO_STRING	USINT_TO_STRING	WORD_TO_STRING
UINT_TO_TIME	ULINT_TO_TIME	USINT_TO_TIME	WORD_TO_TIME
UINT_TO_TOD	ULINT_TO_TOD	USINT_TO_TOD	WORD_TO_TOD
UINT_TO_UDINT	ULINT_TO_UDINT	USINT_TO_UDINT	WORD_TO_UDINT
UINT_TO_ULINT	ULINT_TO_ULINT	USINT_TO_ULINT	WORD_TO_ULINT
UINT_TO_USINT	ULINT_TO_USINT	USINT_TO_USINT	WORD_TO_USINT
UINT_TO_WORD	ULINT_TO_WORD	USINT_TO_WORD	WORD_TO_USINT

3.1.6.2 BOOL_TO_TYPE (BOOL 类型转换指令)

3.1.6.2.1 功能

把布尔数据类型转换为其它数据类型。

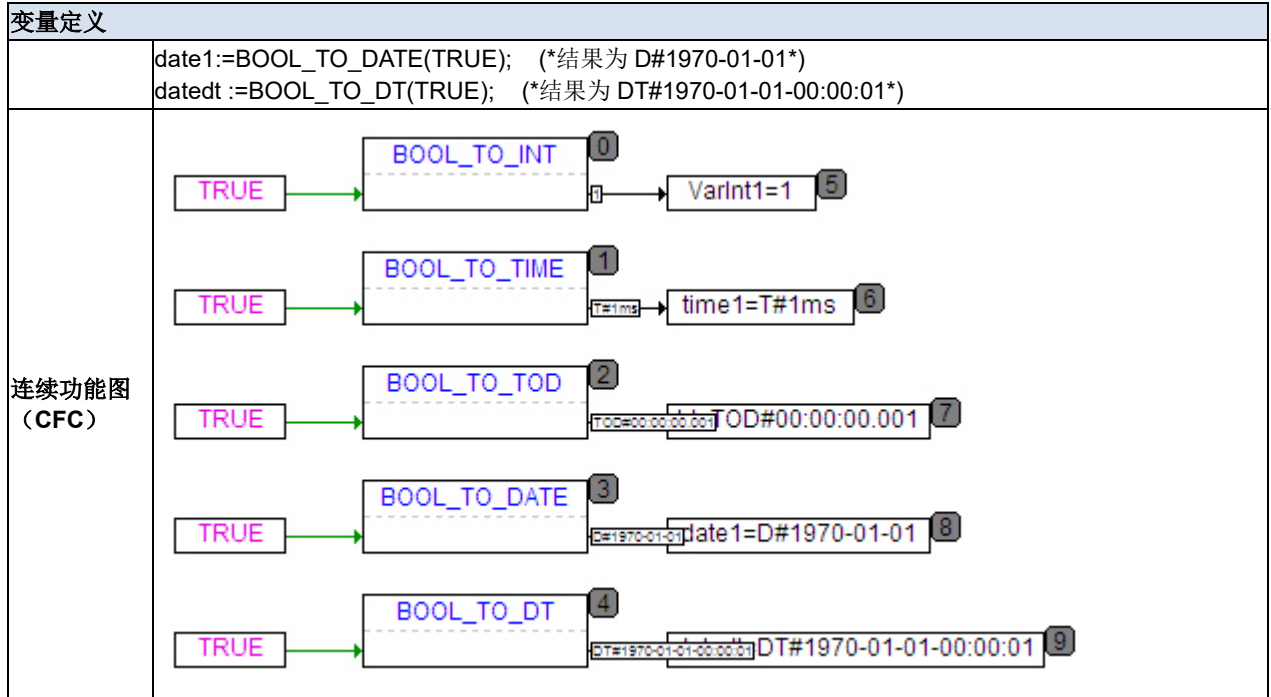
输出为数字类型时，如果输入是 TRUE，则输出 1，如果输入是 FALSE，则输出为 0。

输出为字符串类型时，如果输入是 TRUE，则输出字符串'TRUE'，如果输入是 FALSE，则输出为字符串'FALSE'。

3.1.6.2.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarInt1			INT	0	FALSE
0002	time1			TIME	T#0MS	FALSE
0003	td			TOD	TOD#00:00:00	FALSE
0004	date1			DATE	D#1970-01-01	FALSE
0005	datedt			DT	DT#1970-01-01-00:00:00	FALSE

编程语言	程 序（部分）
梯形图 (LD)	
结构化文本 (ST)	<pre> VarInt1:=BOOL_TO_INT(TRUE); (*结果为 1*) time1:=BOOL_TO_TIME(TRUE); (*结果为 T#1ms*) td:=BOOL_TO_TOD(TRUE); (*结果为 TOD#00:00:00.001*) </pre>



3.1.6.3 BYTE_TO_TYPE (BYTE 类型转换指令)

3.1.6.3.1 功能

把字节类型转换为其他数据类型。

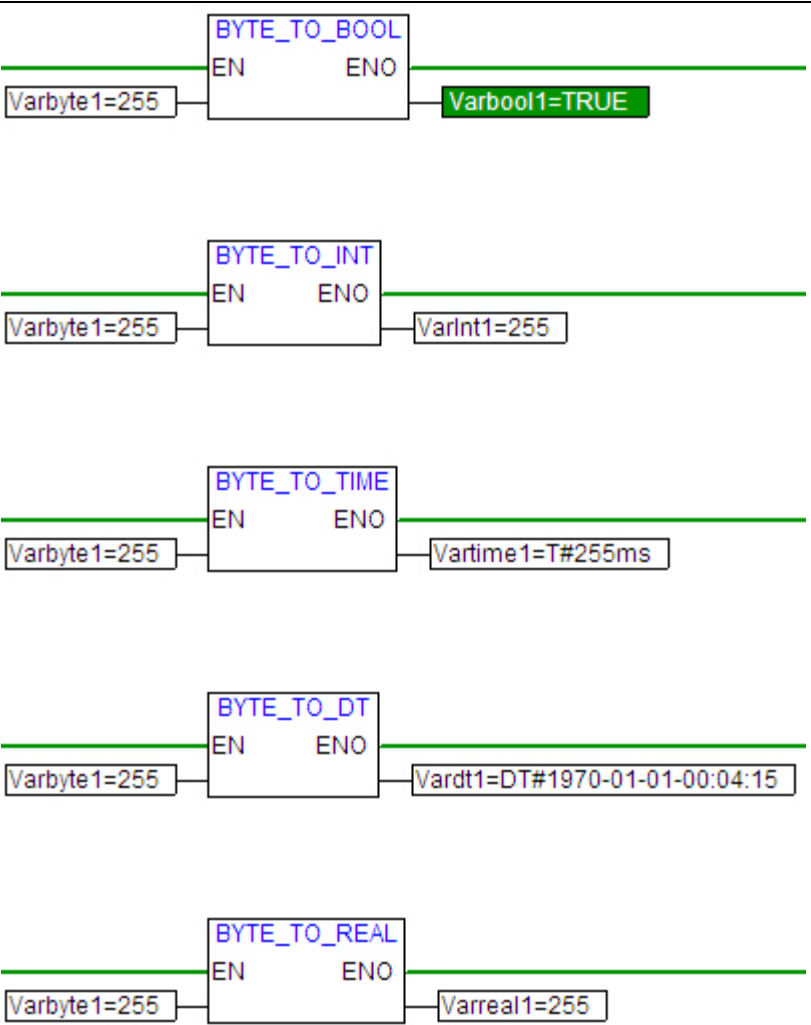
当 BYTE_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE。

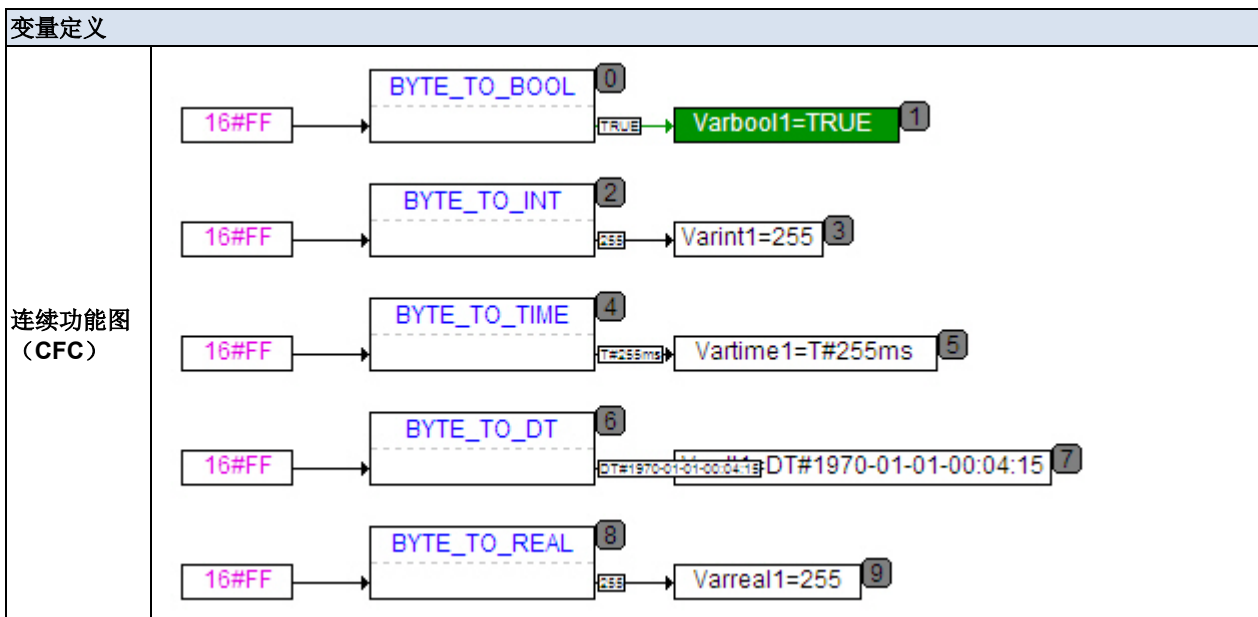
当 BYTE_TO_TIME、BYTE_TO_TOD 时，输入将以毫秒值进行转换。

当 BYTE_TO_DATE、BYTE_TO_DT 时，输入将以秒值进行转换。

3.1.6.3.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varbool1			BOOL	FALSE	FALSE
0002	Varbyte1			BYTE	0	FALSE
0003	Varint1			INT	0	FALSE
0004	Vartime1			TIME	T#0MS	FALSE
0005	Vardt1			DT	DT#1970-01-01-00:00:00	FALSE
0006	Varreal1			REAL	0	FALSE

变量定义	
编程语言	程 序（部分）
梯形图 (LD)	 <pre> graph LR subgraph "BYTE_TO_BOOL" B1[Varbyte1=255] -- EN --> B2[ENO] B2 --> B3[Varbool1=TRUE] end subgraph "BYTE_TO_INT" B4[Varbyte1=255] -- EN --> B5[ENO] B5 --> B6[VarInt1=255] end subgraph "BYTE_TO_TIME" B7[Varbyte1=255] -- EN --> B8[ENO] B8 --> B9[Vartime1=T#255ms] end subgraph "BYTE_TO_DT" B10[Varbyte1=255] -- EN --> B11[ENO] B11 --> B12[Vardt1=DT#1970-01-01-00:04:15] end subgraph "BYTE_TO_REAL" B13[Varbyte1=255] -- EN --> B14[ENO] B14 --> B15[Varreal1=255] end </pre>
结构化文本 (ST)	<pre> Varbyte1:=16#FF; (*Varbyte1 取值*) Varbool1:=BYTE_TO_BOOL(Varbyte1); (*结果为 TRUE *) Varint1:=BYTE_TO_INT(Varbyte1); (*结果为 16# FF *) Vartime1:=BYTE_TO_TIME(Varbyte1); (*结果为 T#255ms *) Vardt1:=BYTE_TO_DT(Varbyte1); (*结果为 DT#1970-01-01-00:04:15 *) Varreal1:=BYTE_TO_REAL(Varbyte1); (*结果为 255 *) </pre>



3.1.6.4 DATE_TO_TYPE (DATE 类型转换指令)

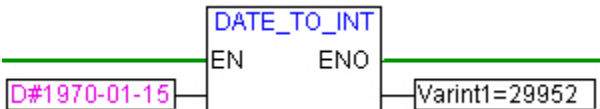
3.1.6.4.1 功能

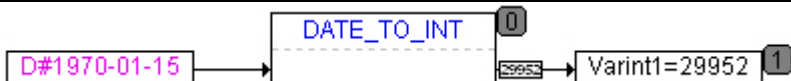
把日期型数据转换为其它类型数据，日期在内部以秒为单位存储，时间从 1970 年 1 月 1 日开始。

3.1.6.4.2 输入/输出数据类型

当 DATE_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

3.1.6.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varint1			INT	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		VarInt1:=DATE_TO_INT(D#1970-01-15); (*结果为 29952*) 说明： 将 D#1970-01-15 (十进制 14×24×3600=1209600=16#127500) 保存为 INT 型变量，则会丢失高位数据，只显示低位数据 16#7500,转换十进制数为 29952				

变量定义	
连续功能图 (CFC)	

3.1.6.5 DINT_TO_TYPE (DINT 类型转换指令)

3.1.6.5.1 功能

双整数类型转换为其它数据类型。

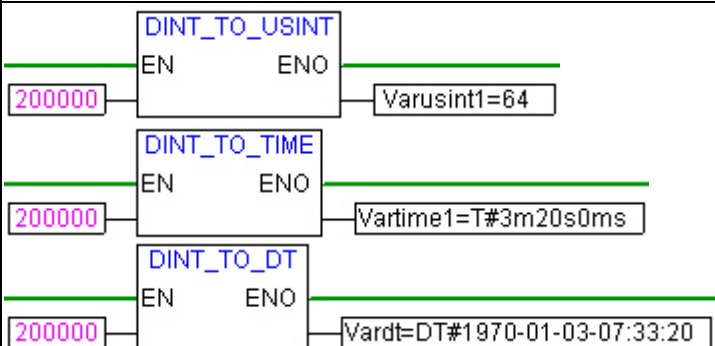
3.1.6.5.2 输入/输出数据类型

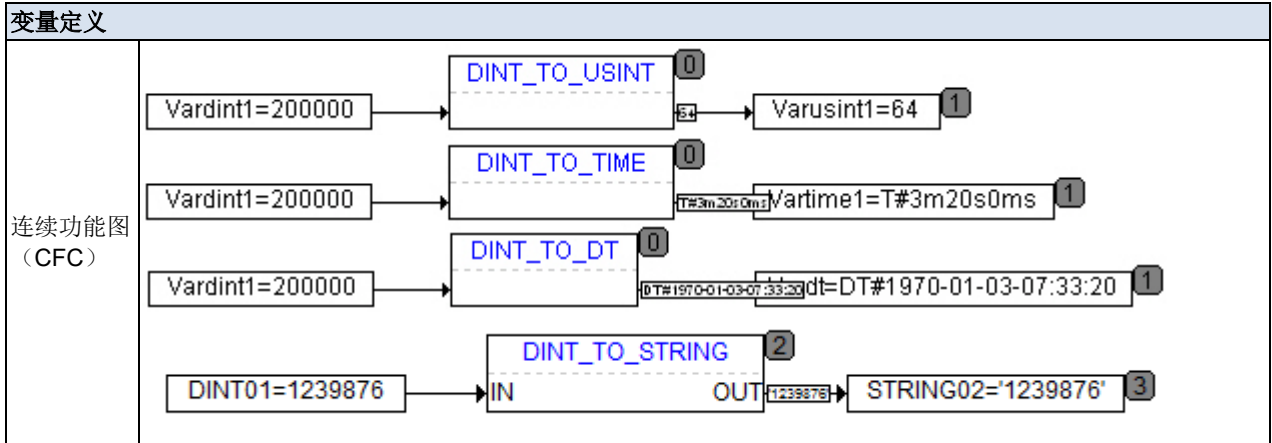
当 DINT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

当 DINT_TO_TIME、DINT_TO_TOD 时，输入将以毫秒值进行转换。

当 DINT_TO_DATE、DINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.5.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Vardint1			DINT	0	FALSE
0002	Varusint1			USINT	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt			DT	DT#1970-01-01-00:00:00	FALSE
编程语言	程序					
梯形图 (LD)						
结构化文本 (ST)	<pre> Vardint1:=200000; Varusint1:=DINT_TO_USINT(Vardint1); (*结果 64*) 说明：如果将整数 200000（十六进制为 16#30D40）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 64（十六进制为 16#40） Vartime1:=DINT_TO_TIME(Vardint1); (*结果 T#3m20s0ms*) Vardt:=DINT_TO_DT(Vardint1); (*结果 DT#1970-01-03-07:33:20 *) </pre>					



3.1.6.6 DT_TO_TYPE (DT 类型转换指令)

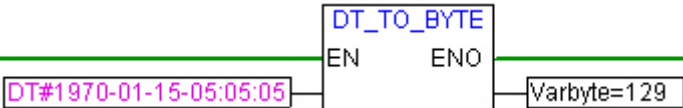
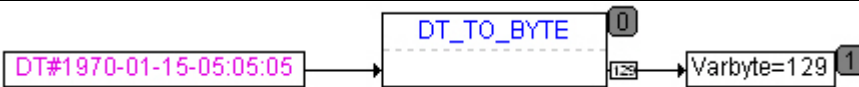
3.1.6.6.1 功能

把日期时间型数据转换为其它类型数据，日期在内部以秒为单位存储，时间从 1970 年 1 月 1 日开始。

3.1.6.6.2 输入/输出数据类型

当 DT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

3.1.6.6.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varbyte			BYTE	0	FALSE
编程语言 程序						
梯形图 (LD)						
结构化文本 (ST)	Varbyte:=DT_TO_BYTE(DT#1970-01-15-05:05:05); (*结果 Varbyte 为 129*) 说明： 将 DT#1970-01-15-05:05:05 转换成秒数，值为(((14*24+5)*60+5)*60+5)=1227905=16#12BC81，保存为 BYTE 型变量，则会丢失高 24 位数据，只显示低 8 位数据，16#81= 129					
连续功能图 (CFC)						

3.1.6.7 DWORD_TO_TYPE（DWORD 类型转换指令）

3.1.6.7.1 功能

把双字类型转换为其它数据类型。

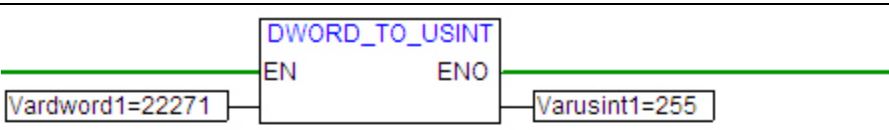
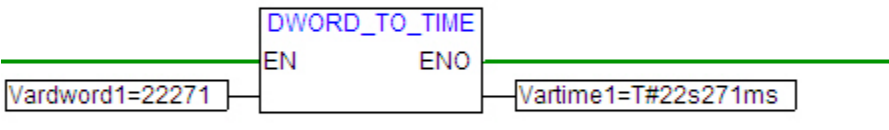
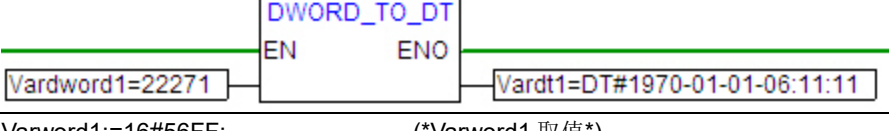
当 DWORD_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

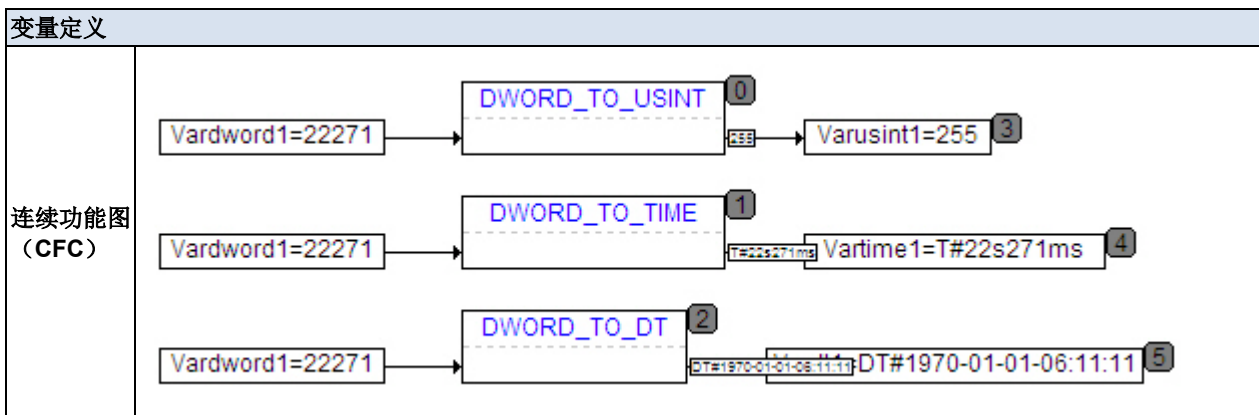
当 DWORD_TO_TIME、DWORD_TO_TOD 时，输入将以毫秒值进行转换。

当 DWORD_TO_DATE、DWORD_TO_DT 时，输入将以秒值进行转换。

3.1.6.7.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varusint1			USINT	0	FALSE
0002	Vardword1			DWORD	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt1			DT	DT#1970-01-01-00:00:00	FALSE

编程语言	程序部分
梯形图 (LD)	
	
	
结构化文本 (ST)	<pre> Varword1:=16#56FF; (*Varword1 取值*) Varusint1:=DWORD_TO_USINT(Varword1); (*结果 255 *) 说明：如果将整数 16#56FF（十进制为 22271）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 255（十六进制为 16#FF） Vartime1:=DWORD_TO_TIME(Varword1); (*结果 T#22s271ms*) Vardt1:=DWORD_TO_DT(Varword1); (*结果 DT#1970-01-01-06:11:11 *) </pre>



3.1.6.8 INT_TO_TYPE (INT 类型转换指令)

3.1.6.8.1 功能

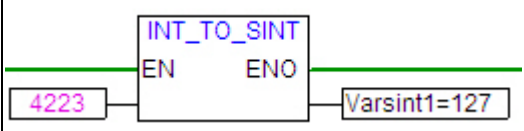
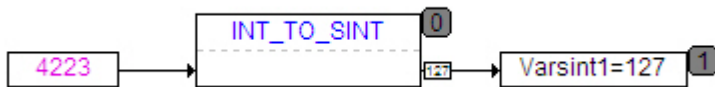
把整型数据类型转换为其它数据类型。

当 INT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

当 INT_TO_TIME、INT_TO_TOD 时，输入将以毫秒值进行转换。

当 INT_TO_DATE、INT_TO_DT 时，输入将以秒值进行转换。

3.1.6.8.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varsint1			SINT	0	FALSE
编程语言	程序部分					
梯形图 (LD)						
结构化文本 (ST)	VarSINT1 := INT_TO_SINT(4223); (*结果 VarSINT1 为 127 *) 说明：如果将整数 4223（十六进制为 16#107F）保存为 SINT 型变量，则会丢失高位数据，只显示低位数据 127（十六进制为 16#7F）。					
连续功能图 (CFC)						

3.1.6.9 LINT_TO_TYPE（LINT 类型转换指令）

3.1.6.9.1 功能

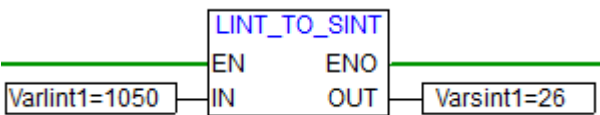
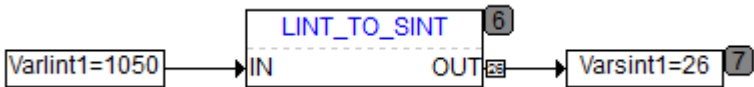
把长整型数据类型转换为其它数据类型。

当 LINT_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE；当 LINT_TO_TIME、LINT_TO_TOD 时，输入将以毫秒值进行转换。

当 LINT_TO_DATE、LINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.9.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varlint1			LINT	0	FALSE
0002	Varsint1			SINT	0	FALSE

编程语言	程序
梯形图（LD）	
结构化文本（ST）	Varsint1:=LINT_TO_SINT(Varlint1); (*结果 Varsint1 为 26*) 说明：如果将整数 1050（十六进制为 16#41A）保存为 SINT 型变量，则会丢失高位数据，只显示低位数据 26（十六进制为 16#1A）。
连续功能图（CFC）	

3.1.6.10 LREAL_TO_TYPE（LREAL 类型转换指令）

3.1.6.10.1 功能

把长实数转换为其它类型数据。

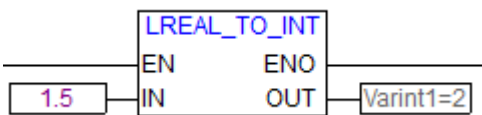
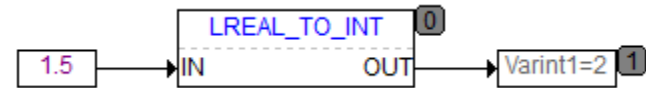
3.1.6.10.2 输入/输出数据类型

当 LREAL_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE；当 LREAL_TO_TIME、LREAL_TO_TOD 时，输入将以毫秒值进行转换；

当 LREAL_TO_DATE、LREAL_TO_DT 时，输入将以秒值进行转换；

当 LREAL_TO_STRING 时，转换后的字符串长度最大为 16。

3.1.6.10.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varint1			INT	0	FALSE
0002	Varint2			INT	0	FALSE
0003	Varint3			INT	0	FALSE
0004	Varint4			INT	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）	VarINT1:= LREAL_TO_INT(1.5); (*结果 VarINT1 为 2*) VarINT2:= LREAL_TO_INT(1.4); (*结果 VarINT2 为 1*) VarINT3:= LREAL_TO_INT(-1.5); (*结果 VarINT3 为 -2*) VarINT4:= LREAL_TO_INT(-1.4); (*结果 VarINT4 为 -1*)					
连续功能图（CFC）						

3.1.6.11 LWORD_TO_TYPE (LWORD 类型转换指令)

3.1.6.11.1 功能

把长字类型转换为其它数据类型。

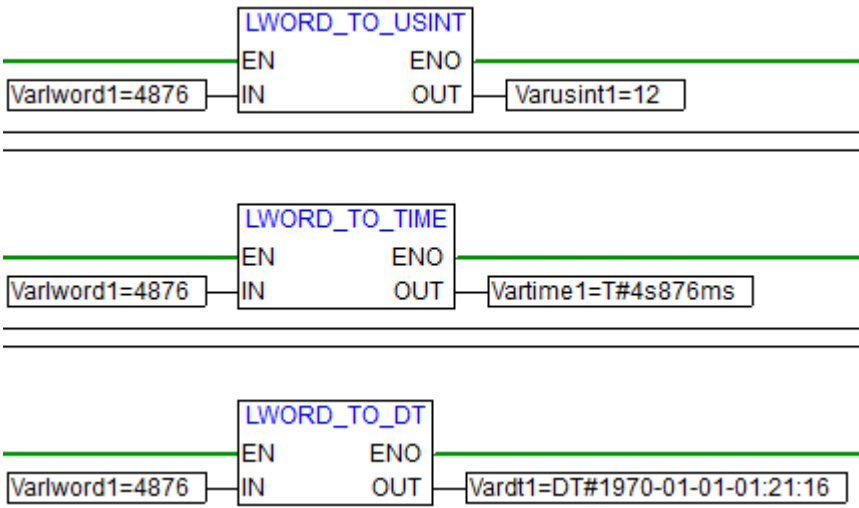
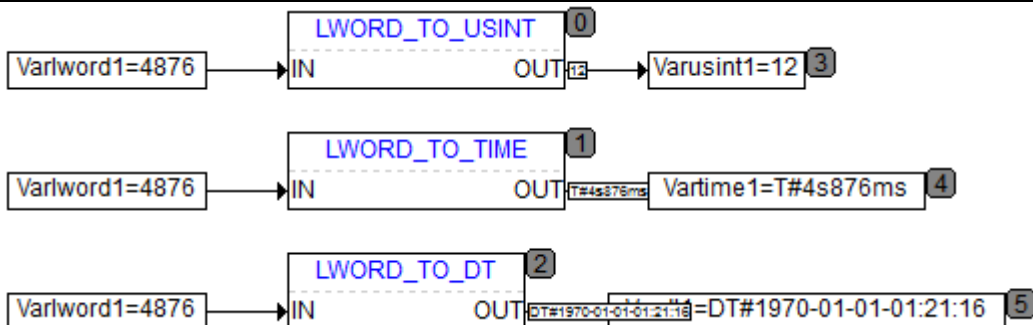
当 LWORD_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE；当 LWORD_TO_TIME、LWORD_TO_TOD 时，输入将以毫秒值进行转换。

当 LWORD_TO_DATE、LWORD_TO_DT 时，输入将以秒值进行转换。

3.1.6.11.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varlword1			LWORD	0	FALSE
0002	Varusint1			USINT	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt1			DT	DT#1970-01-01-00:00:00	FALSE

编程语言	程序
------	----

梯形图 (LD)	
结构化文本 (ST)	<pre> Varlword1:=4876; (*Varlword1 取值*) Varusint1:=LWORD_TO_USINT(Varlword1); (*结果 12*) 说明：如果将整数 4876（十六进制为 16#130C）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 12（十六进制为 16#0C） Vartime1:=LWORD_TO_TIME(Varlword1); (*结果 T#4s876ms*) Vardt1:=LWORD_TO_DT(Varlword1); (*结果 DT#1970-01-01-01:21:16*) </pre>
连续功能图 (CFC)	

3.1.6.12 REAL_TO_TYPE (REAL 类型转换指令)

3.1.6.12.1 功能

把实数转换为其它类型数据。

3.1.6.12.2 输入/输出数据类型

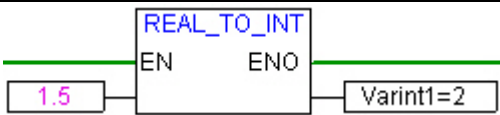
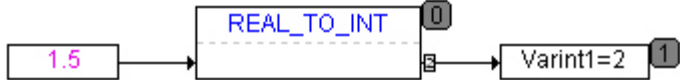
当 REAL_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE；当 REAL_TO_TIME、REAL_TO_TOD 时，输入将以毫秒值进行转换；

当 REAL_TO_DATE、REAL_TO_DT 时，输入将以秒值进行转换；

当 LREAL_TO_STRING 时，转换后的字符串长度最大为 16。

3.1.6.12.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varint1			INT	0	FALSE
0002	Varint2			INT	0	FALSE
0003	Varint3			INT	0	FALSE
0004	Varint4			INT	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> VarINT1:= REAL_TO_INT(1.5); (*结果 VarINT1 为 2*) VarINT2:= REAL_TO_INT(1.4); (*结果 VarINT2 为 1*) VarINT3:= REAL_TO_INT(-1.5); (*结果 VarINT3 为-2*) VarINT4:= REAL_TO_INT(-1.4); (*结果 VarINT4 为-1*) </pre>
连续功能图 (CFC)	

3.1.6.13 SINT_TO_TYPE (SINT 类型转换指令)

3.1.6.13.1 功能

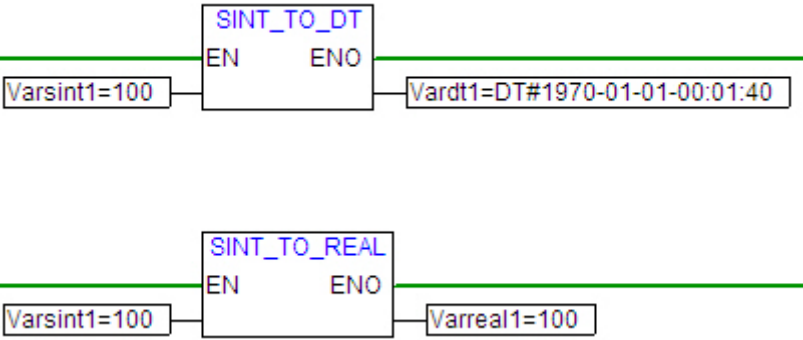
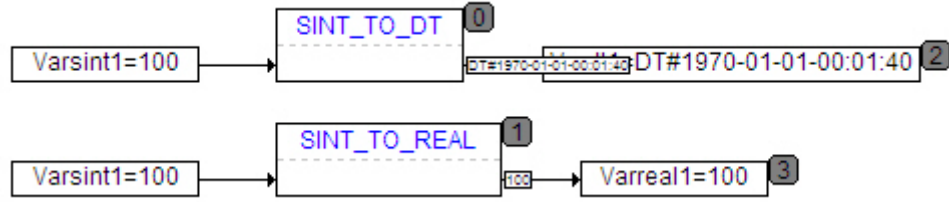
把短整型转换为其它数据类型。

当 SINT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

当 SINT_TO_TIME、SINT_TO_TOD 时，输入将以毫秒值进行转换。

当 SINT_TO_DATE、SINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.13.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varsint1			SINT	100	FALSE
0002	Vardt1			DT	DT#1970-01-01-00:00:00	FALSE
0003	Varreal1			REAL	0	FALSE
编程语言	程 序（部分）					
梯形图 (LD)						
结构化文本 (ST)	<pre>Varsint1:=100; (*Varsint1 取值*) Vardt1:=SINT_TO_DT(Varsint1); (*结果 DT#1970-01-01-00:01:40 *) Varreal1:=SINT_TO_REAL(Varsint1); (*结果为 100.0*)</pre>					
连续功能图 (CFC)						

3.1.6.14 STRING_TO_TYPE（STRING 类型转换指令）

3.1.6.14.1 功能

把 STRING 类型变量转换为其它数据类型。

对于 STRING_TO_BOOL 指令，输入字符串'TRUE'或'true'，则输出 TRUE；除此之外，均输出 FALSE。

3.1.6.14.2 指令使用举例

变量定义							
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护	
0004	STRING02			STRING(80)	'this is a string'	FALSE	

3.1.6.15 TIME_TO_TYPE (TIME 类型转换指令)

3.1.6.15.1 功能

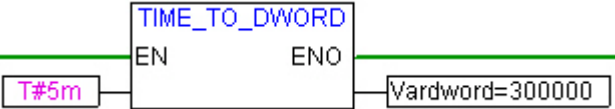
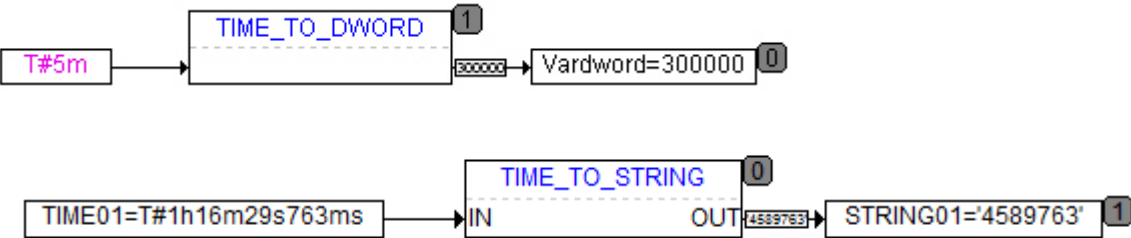
把时间型数据转换为其它类型数据，时间在内部以毫秒为单位存储成 DWORD 类型（对于 TIME_OF_DAY 变量从凌晨 00: 00 开始）。

3.1.6.15.2 输入/输出数据类型

当 TIME_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

3.1.6.15.3 指令使用举例

变量定义							
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护	
0001	Varword			DWORD	0	FALSE	

变量定义	
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	Vardword:=TIME_TO_DWORD(T#5m); (*结果为 300000*)
连续功能图 (CFC)	

3.1.6.16 TOD_TO_TYPE (TOD 类型转换指令)

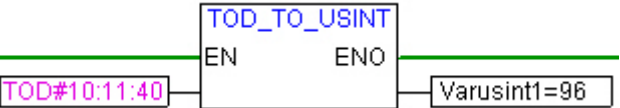
3.1.6.16.1 功能

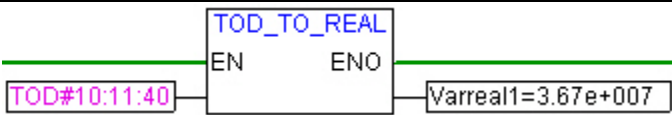
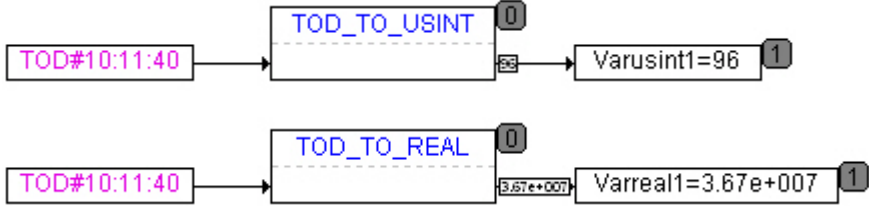
把时间型数据转换为其它类型数据，日期在内部以毫秒为单位进行转化。

3.1.6.16.2 输入/输出数据类型

当 TOD_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

3.1.6.16.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Vartod1			TOD	TOD#00:00:00	FALSE
0002	Varusint1			USINT	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
编程语言		程序				
梯形图 (LD)						

变量定义	
	
结构化文本 (ST)	Vartod1:=TOD#10:11:40; (*Vartod1 取值*) Varusint1:=TOD_TO_USINT(Vartod1); (*结果为 96*) Varreal1:=TOD_TO_REAL(Vartod1); (*结果为 3.67e+007*)
连续功能图 (CFC)	

3.1.6.17 UDINT_TO_TYPE (UDINT 类型转换指令)

3.1.6.17.1 功能

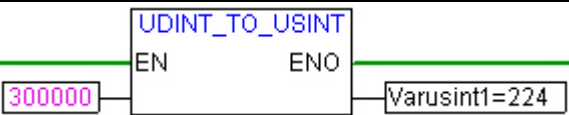
无符号双整数类型转换为其它数据类型。

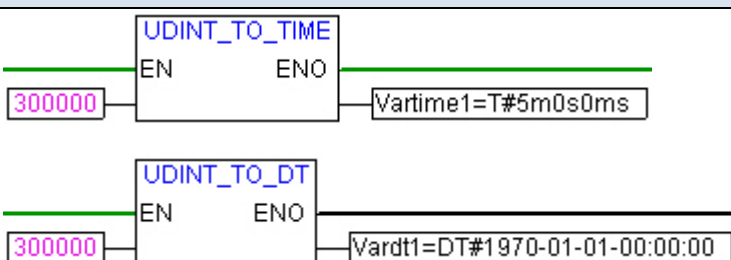
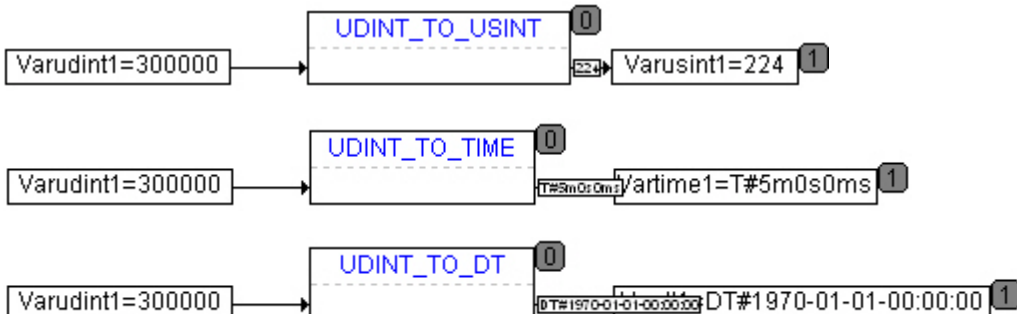
3.1.6.17.2 输入/输出数据类型

当 UDINT_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE；当 UDINT_TO_TIME、UDINT_TO_TOD 时，输入将以毫秒值进行转换；

当 UDINT_TO_DATE、UDINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.17.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varudint1			UDINT	0	FALSE
0002	Varusint1			USINT	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt1			DT	DT#1970-01-01-00:00:00	FALSE
编程语言	程序					
梯形图 (LD)						

变量定义	
	
结构化文本 (ST)	<pre> Varudint1:=300000; Varusint1:= UDINT_TO_USINT(Varudint1); (*结果 224*) 说明：如果将整数 300000（十六进制为 16#493E0）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 224（十六进制为 16#E0）。 Vartime1:=UDINT_TO_TIME(Varudint1); (*结果 T#5m0s0ms*) Vardt1:=UDINT_TO_DT(Varudint1); (*结果 DT#1970-01-01-00:00:00 *) </pre>
连续功能图 (CFC)	

3.1.6.18 UINT_TO_TYPE（UINT 类型转换指令）

3.1.6.18.1 功能

无符号整数类型转换为其它数据类型。

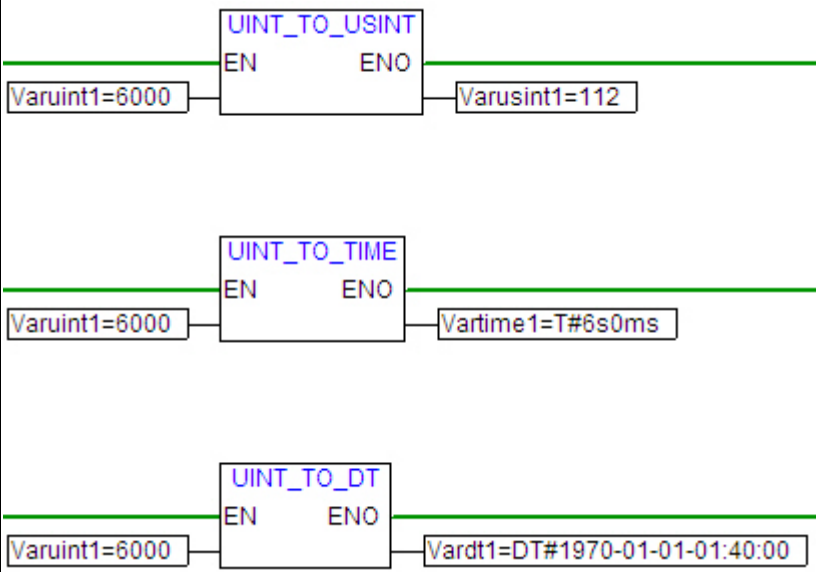
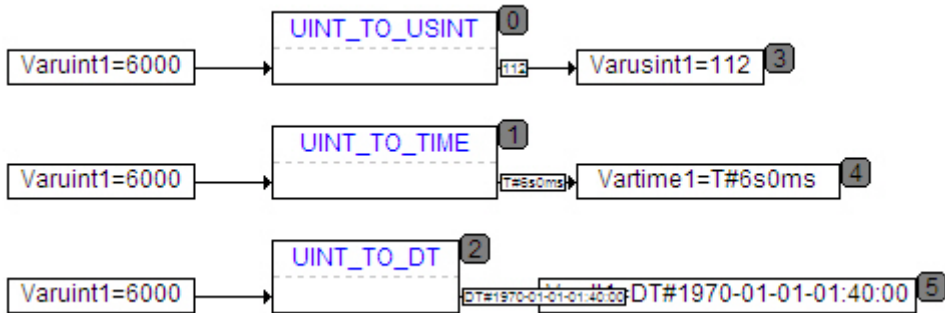
当 UINT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

当 UINT_TO_TIME、UINT_TO_TOD 时，输入将以毫秒值进行转换。

当 UINT_TO_DATE、UINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.18.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varuint1			UINT	6000	FALSE
0002	Varusint1			USINT	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt1			DT	DT#1970-0...	FALSE

编程语言	程序 (部分)
梯形图 (LD)	
结构化文本 (ST)	<pre> Varuint1:=6000; Varusint1:=UINT_TO_USINT(Varuint1); 说明：如果将整数 6000（十六进制为 16#1770）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 112（十六进制为 16#70）。 Vartime1:=UINT_TO_TIME(Varuint1); Vardt1:=UINT_TO_DT(Varuint1); </pre>
连续功能图 (CFC)	

3.1.6.19 ULINT_TO_TYPE（ULINT 类型转换指令）

3.1.6.19.1 功能

把无符号长整型转换为其它数据类型。

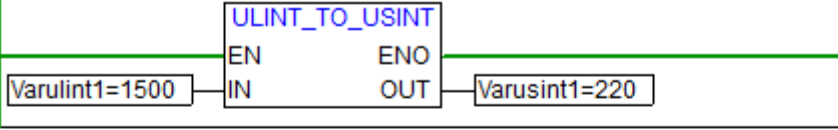
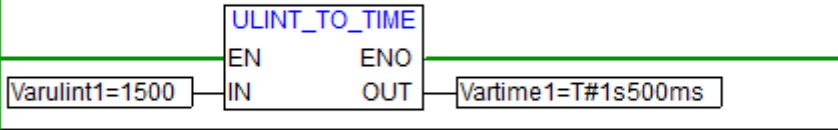
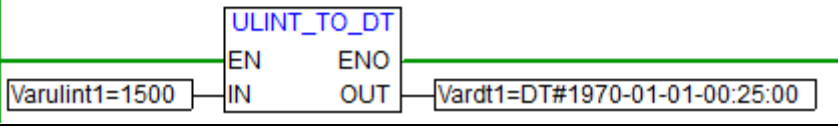
当 ULINT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

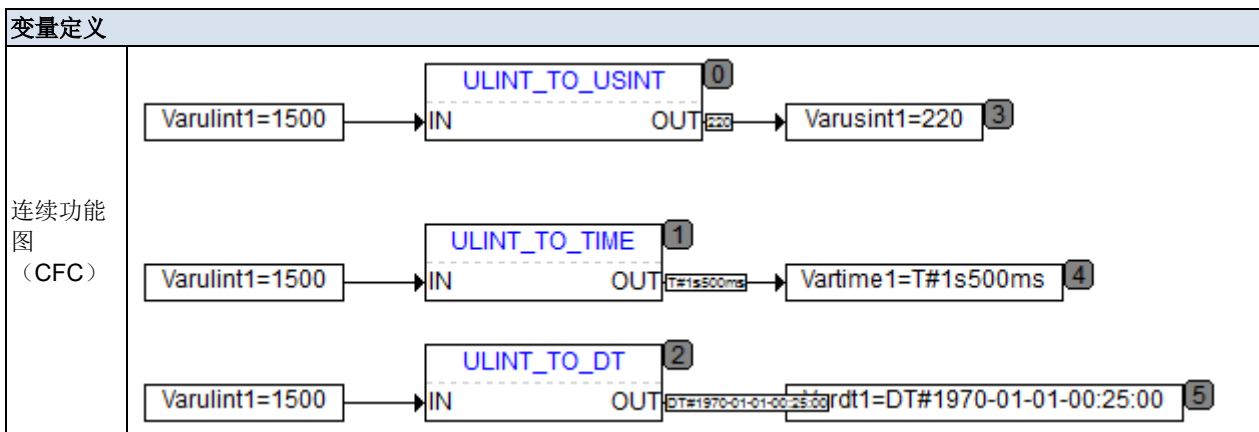
当 ULINT_TO_TIME、ULINT_TO_TOD 时，输入将以毫秒值进行转换。

当 ULINT_TO_DATE、ULINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.19.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varulint1			ULINT	0	FALSE
0002	Varusint1			USINT	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt1			DT	DT#1970-0...	FALSE

编程语言	程序
梯形图 (LD)	
	
	
结构化文 本 (ST)	<pre> Varulint1:=1500; Varusint1:=ULINT_TO_USINT(Varulint1); 说明：如果将整数 1500（十六进制为 16#5DC）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 220（十六进制为 16#DC）。 Vartime1:=ULINT_TO_TIME(Varulint1); Vardt1:=ULINT_TO_DT(Varulint1); </pre>



3.1.6.20 USINT_TO_TYPE (USINT 类型转换指令)

3.1.6.20.1 功能

把无符号短整型转换为其它数据类型。

当 USINT_TO_BOOL 时，输入不等于 0 时，输出为 TRUE；当输入等于 0 时，输出为 FALSE。

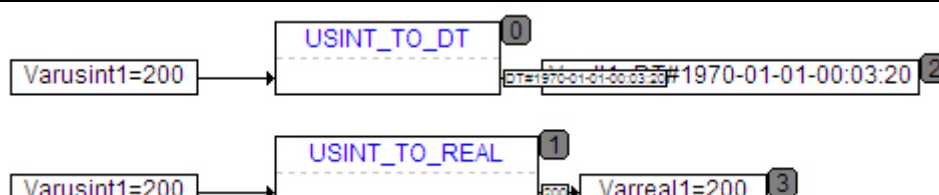
当 USINT_TO_TIME、USINT_TO_TOD 时，输入将以毫秒值进行转换。

当 USINT_TO_DATE、USINT_TO_DT 时，输入将以秒值进行转换。

3.1.6.20.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varusint1			USINT	0	FALSE
0002	Vardt1			DT	DT#1970-0...	FALSE
0003	Varreal1			REAL	0	FALSE

编程语言	程 序 (部分)
梯形图 (LD)	
结构化文本 (ST)	Varusint1:=200; (*Varusint1 取值*)

变量定义	
	Vardt1:=USINT_TO_DT(Varusint1); (*结果 DT#1970-01-01-00:03:20 *) Varreal1:=USINT_TO_REAL(Varusint1); (*结果为 200 .0*)
连续功能图 (CFC)	

3.1.6.21 WORD_TO_TYPE (WORD 类型转换指令)

3.1.6.21.1 功能

把字类型转换为其它数据类型。

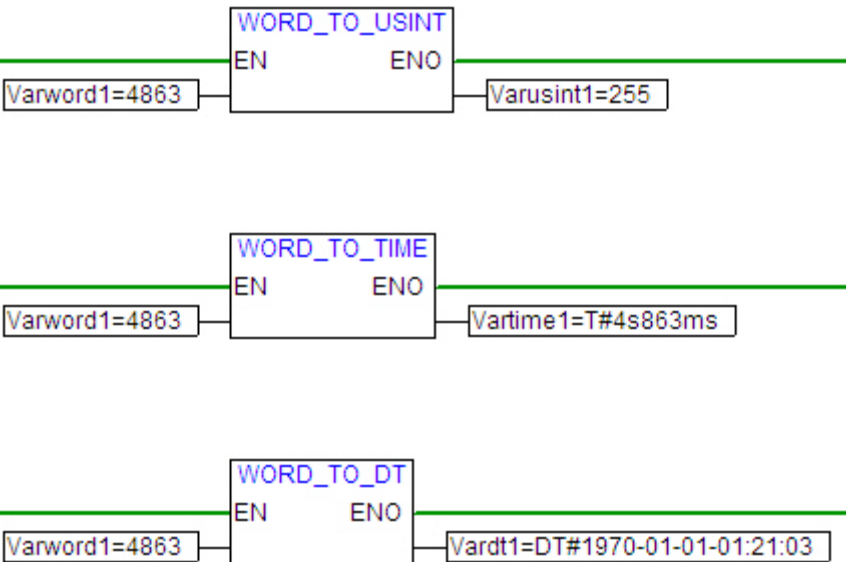
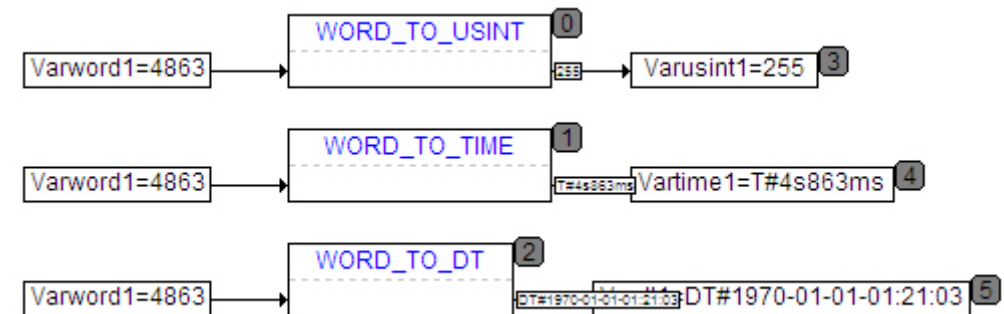
当 WORD_TO_BOOL 时，输入不等于 0 时输出为 TRUE，当输入等于 0 时输出为 FALSE。

当 WORD_TO_TIME、WORD_TO_TOD 时，输入将以毫秒值进行转换。

当 WORD_TO_DATE、WORD_TO_DT 时，输入将以秒值进行转换。

3.1.6.21.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varusint1			USINT	0	FALSE
0002	Varword1			WORD	0	FALSE
0003	Vartime1			TIME	T#0MS	FALSE
0004	Vardt1			DT	DT#1970-01-01-00:00:00	FALSE
编程语言	程 序 (部 分)					

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> Varword1:=4863; (*Varword1 取值*) Varusint1:=WORD_TO_USINT(Varword1); (*结果 255 *) 说明： 如果将整数 4863（十六进制为 16#12FF）保存为 USINT 型变量，则会丢失高位数据，只显示低位数据 255（十六进制为 16#FF）。 Vartime1:=WORD_TO_TIME(Varword1); (*结果 T#4s863ms*) Vardt1:=WORD_TO_DT(Varword1); (*结果 DT#1970-01-01-01:21:03 *) </pre>
连续功能图 (CFC)	

3.1.6.22 TRUNC

3.1.6.22.1 功能

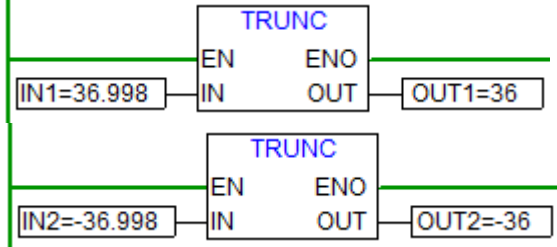
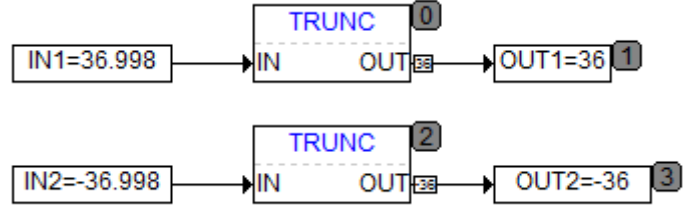
实现对浮点数取整数位功能。

3.1.6.22.2 输入/输出数据类型

输入：REAL；

输出：DINT、LINT、ULINT、LWORD。

3.1.6.22.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	IN1			REAL ▾	0	FALSE ▾
0002	OUT1			DINT ▾	0	FALSE ▾
0003	IN2			REAL ▾	0	FALSE ▾
0004	OUT2			DINT ▾	0	FALSE ▾
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		<pre> 0001 OUT1:= TRUNC (36.998); 0002 OUT2:= TRUNC (-36.998); OUT1 = 36 OUT2 = -36 </pre>				
连续功能图 (CFC)						

3.1.7 指针

3.1.7.1 ADR (取地址)

3.1.7.1.1 功能

取得输入变量的内存地址，并输出。该地址可以在程序内当作指针使用，也可以作为指针传送给函数。

3.1.7.1.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE ▼	0	FALSE ▼
0002	VarAddress			POINTER TO BYTE ▼	0	FALSE ▼
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		VarAddress:= ADR(Var1);				
连续功能图 (CFC)						



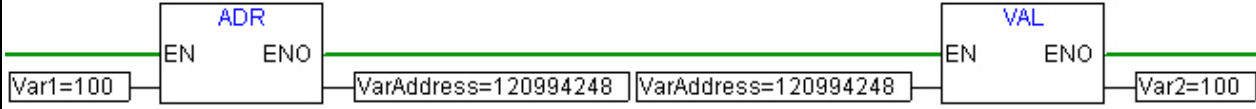
- 慎用取址指令，使用时务必保证指针类型与所指的变量类型匹配。
- 使用该指令存在风险，建议采用其他方式实现。

3.1.7.2 VAL (取值)

3.1.7.2.1 功能

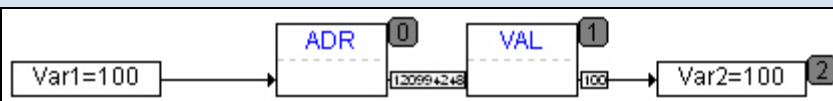
取得指针所指地址的数据。

3.1.7.2.2 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var1			BYTE ▼	0	FALSE ▼
0002	Var2			BYTE ▼	0	FALSE ▼
0003	VarAddress			POINTER TO BYTE ▼	0	FALSE ▼
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		VarAddress:= ADR(Var1); Var2:= VAL(VarAddress); (*结果 Var2 为 100 *)				

变量定义

连续功能图
(CFC)



① 使用该指令存在风险，建议采用其他方式实现。

3.1.8 字符串

3.1.8.1 Left

3.1.8.1.1 函数原型

STRING LEFT (STRING str, INT count)

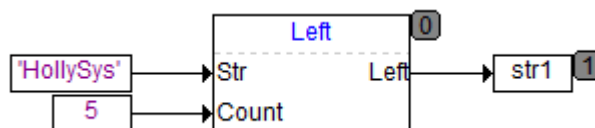
3.1.8.1.2 功能描述

返回指定字符串的最左边的 count 个字符构成的子字符串。

3.1.8.1.3 参数说明

	数据项	数据类型	参数描述
输入端	Str	STRING	输入字符串，最大长度 128
	Count	BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT 和 UDINT	要返回的子字符串长度
输出端	Left	STRING	返回子字符串。 (1) 若 str 为空，则返回空； (2) 若 count 小于或等于 0，则返回空； (3) 若 str 实际长度大于 128，取前 128 个字符； (4) 若 count 大于或等于 str 长度，则返回 str；

3.1.8.1.4 组态示例



Str1 的值为'Holly'。

3.1.8.2 Right

3.1.8.2.1 函数原型

STRING RIGHT (STRING str, INT count)

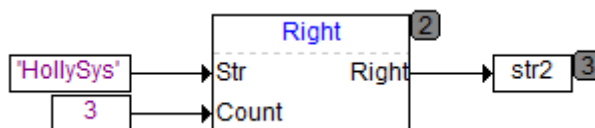
3.1.8.2.2 功能描述

返回指定字符串的最右边的 count 个字符构成的子字符串。

3.1.8.2.3 参数说明

	数据项	数据类型	参数描述
输入端	Str	STRING	输入字符串，最大长度 128
	Count	BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT 和 UDINT	要返回的子字符串长度
输出端	Right	STRING	返回子字符串。 (1) 若 str 为空，则返回空； (2) 若 count 小于或等于 0，则返回空； (3) 若 str 实际长度大于 128，取前 128 个字符； (4) 若 count 大于或等于 str 长度，则返回 str；

3.1.8.2.4 组态示例



str2 的值为'Sys'。

3.1.8.3 Concat

3.1.8.3.1 函数原型

STRING CONCAT (STRING str1, STRING str2)

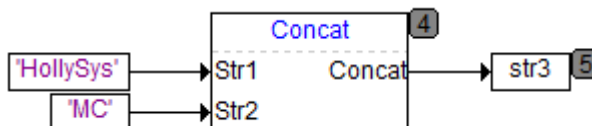
3.1.8.3.2 功能描述

连接两个字符串。

3.1.8.3.3 参数说明

	数据项	数据类型	参 数 描 述
输入端	Str1	STRING	输入字符串，最大长度 128
	Str2	STRING	输入字符串，最大长度 128
输出端	Concat	STRING	返回子字符串。 (1) 输入字符串变量长度大于 128 时，取前 128 个字符； (2) 若 str1 长度+str2 长度大于 128，则返回空。

3.1.8.3.4 组态示例



str3 的值为'HollySysMC'。

3.1.8.4 Delete

3.1.8.4.1 函数原型

STRING DELETE (STRING str, INT len, INT pos)

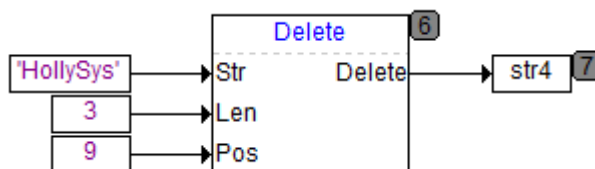
3.1.8.4.2 功能描述

从 str 第 pos 个字符开始删除 len 个字符，并返回结果。

3.1.8.4.3 参数说明

	数据项	数据类型	参数描述
输入端	Str	STRING	输入字符串，最大长度 128
	Len	BYTE、WORD、SINT、USINT、INT、UINT	要删除的字符个数
	Pos	BYTE、WORD、SINT、USINT、INT、UINT	要删除的字符在 str 字符串中的起始位置
输出端	Delete	STRING	(1) 若 str 实际长度大于 128，取前 128 个字符； (2) 若 len 小于 0，则 pos=pos+len ， len=-len ； (3) 若 pos 小于或等于 0，则 len=len+pos-1 ， pos=1 ； (4) 若 pos+len 大于 str 长度，则 len = str 长度-pos+1 ； (5) 若 pos 大于 str 长度或 len 小于等于 0，则返回 str （若其长度大于 128，则返回前 128 个字符）

3.1.8.4.4 组态示例



str4 的值为'HollySys'。

3.1.8.5 Insert

3.1.8.5.1 函数原型

STRING INSERT (STRING str1, STRING str2, INT pos)

3.1.8.5.2 功能描述

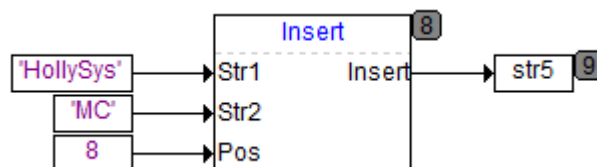
将 **str2** 插入到 **str1** 的第 **pos** 个字符之后，并返回结果。

3.1.8.5.3 参数说明

	数据项	数据类型	参数描述
输入端	Str1	STRING	输入字符串，最大长度 128
	Str2	STRING	输入字符串，最大长度 128

	数据项	数据类型	参数描述
	Pos	BYTE、WORD、SINT、USINT、INT、UINT	str2 字符串要插入到 str1 字符串中的起始位置
输出端	Insert	STRING	(1) 输入字符串变量长度大于 128 时，取前 128 个字符； (2) 若 str1 长度+str2 长度大于 128，则返回空； (3) pos=0，插入到 str1 前面； (4) pos=1，插入到第一个字符之后； (5) 若 pos 大于 str1 长度或小于 0，则返回 str1（若其长度大于 128，则返回前 128 个）

3.1.8.5.4 组态示例



str5 的值为'HollySysMC'。

3.1.8.6 Mid

3.1.8.6.1 函数原型

STRING MID (STRING str, INT len, INT pos)

3.1.8.6.2 功能描述

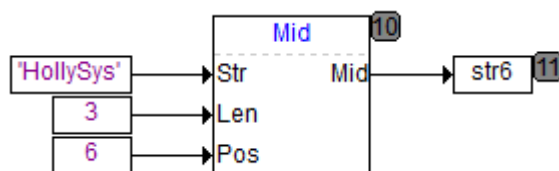
返回字符串中从第 pos 个字符开始连续 len 个字符构成的子串。

3.1.8.6.3 参数说明

	数据项	数据类型	参数描述
输入端	Str	STRING	输入字符串，最大长度 128
	Len	BYTE、WORD、SINT、USINT、INT、UINT	要返回的子字符串长度
	Pos	BYTE、WORD、SINT、USINT、INT、UINT	要返回的子字符串的起始位置
输出端	Mid	STRING	返回子字符串。 (1) 若 str 为空，则返回空； (2) 若 str 实际长度大于 128，取前 128 个字符； (3) 若 pos 小于或等于 0，则返回空；

数据项	数据类型	参数描述
		(4) 若 pos 大于 str 长度，则返回空； (5) 若 len 小于或等于 0，则返回空； (6) 若 len 大于实际可取的字符个数，则返回 pos 开始后面所有字符。

3.1.8.6.4 组态示例



str5 的值为'Sys'。

3.1.8.7 Replace

3.1.8.7.1 函数原型

STRING REPLACE (STRING str1, STRING str2, INT len, INT pos)

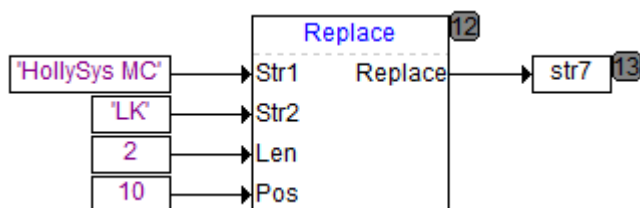
3.1.8.7.2 功能描述

用 str2 替换 str1 第 pos 个字符开始 len 个字符，并返回结果。

3.1.8.7.3 参数说明

数据项	数据类型	参数描述
输入端	Str1	STRING
	Str2	STRING
	Len	BYTE、WORD、SINT、USINT、INT、UINT
	Pos	BYTE、WORD、SINT、USINT、INT、UINT
输出端	Replace	STRING
		(1) 输入字符串变量长度大于 128 时，取前 128 个字符； (2) 若 len 小于 0，pos=pos+len，len=-len； (3) 若 pos 小于等于 0，len=len+pos-1，pos=1； (4) 若 pos+len 大于 str1 长度，len=str1 长度-pos+1； (5) 若 len 大于等于 0，则从 str1 第 pos 个字符 len 个字符开始替换为 str2； (6) 若替换后，str1 长度大于 128，则截取前 128 个返回。

3.1.8.7.4 组态示例



str5 的值为'HollySys LK'。

3.1.8.8 Find

3.1.8.8.1 函数原型

INT FIND (STRING str1, STRING str2)

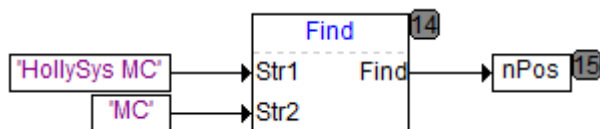
3.1.8.8.2 功能

查找 str2 在 str1 中第一次出现的首字符位置并返回。

3.1.8.8.3 参数说明

	数据项	数据类型	参数描述
输入端	Str1	STRING	输入字符串，最大长度 128
	Str2	STRING	输入字符串，最大长度 128
输出端	Find	WORD、DWORD、INT、UINT、DINT、UDINT、LINT、ULINT、LWORD	(1) 若 str2 在 str1 中没有出现，则返回 0; (2) 若 str1, str2 中有一个为空，则返回 0; (3) 输入字符串变量长度大于 128 时，取前 128 个字符

3.1.8.8.4 组态示例



nPos 的值为 10。

3.1.8.9 Len

3.1.8.9.1 函数原型

INT LEN (STRING str)

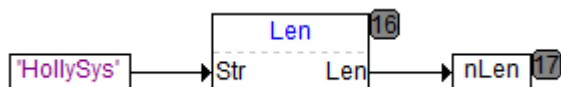
3.1.8.9.2 功能描述

返回输入字符串的长度。

3.1.8.9.3 参数说明

	数据项	数据类型	参数描述
输入端	Str	STRING	输入字符串，最大长度 128
输出端	Len	WORD、DWORD、INT、UINT、DINT、UDINT、LINT、ULINT、LWORD	(1) 若 str 实际长度大于 128，取前 128 个字符； (2) 返回字符串的长度，不包括字符串结束符； (3) 若输入字符串为空，则返回 0；

3.1.8.9.4 组态示例



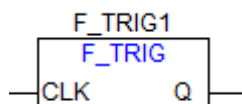
nLen 的值应为 8。

3.1.9 沿触发器指令

3.1.9.1 F_TRIG（下降沿检测触发器）

3.1.9.1.1 功能

用于检测下降沿。



F_TRIG 功能块

3.1.9.1.2 逻辑关系

Q := NOT CLK AND NOT M;

M := NOT CLK;

M 是初始值为 TRUE 的一个中间变量，只要 CLK 是 TRUE，Q 和 M 保持 FALSE。CLK 是 FALSE，Q 首次返回 TRUE，M 设置为 TRUE。每次调用指令时，Q 返回 FALSE，当 CLK 检测到下降沿时，Q 为 TRUE。

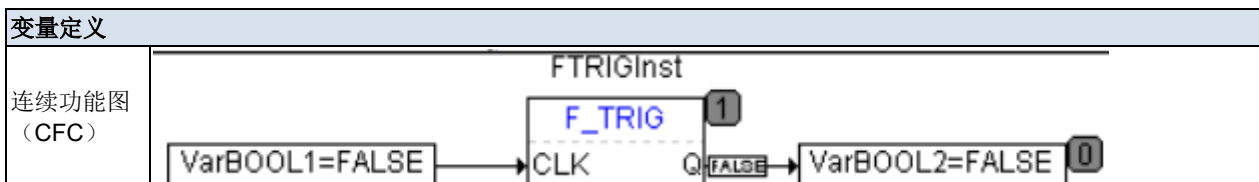
3.1.9.1.3 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
CLK	BOOL	输入	下降沿触发信号	TRUE

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	输出	当检测到下降沿时，Q=TRUE 每次调用指令时，Q=FALSE	0

3.1.9.1.4 指令使用举例

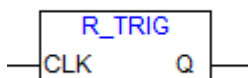
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	F_TRIGInst			F_TRIG		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
编程语言	程序					
梯形图 (LD)	<pre> 0001 ----- comment ----- VarBOOL1=FALSE ----- ----- F_TRIGInst F_TRIG CLK Q ----- ----- VarBOOL2=FALSE ----- ----- </pre>					
结构化文本 (ST)	<pre> FTRIGInst(CLK:=VarBOOL1); VarBOOL2:=FTRIGInst.Q; </pre>					



3.1.9.2 R_TRIG（上升沿检测触发器）

3.1.9.2.1 功能

用于检测上升沿。



R_TRIG 功能块

3.1.9.2.2 逻辑关系

$Q := CLK \text{ AND NOT } M;$

$M := CLK;$

M 是初始值为 FALSE 的一个中间变量，只要 CLK 是 FALSE，Q 和 M 就是 FALSE。每次调用指令时，Q 返回 FALSE。当 CLK 检测到上升沿时，Q 返回 TRUE。

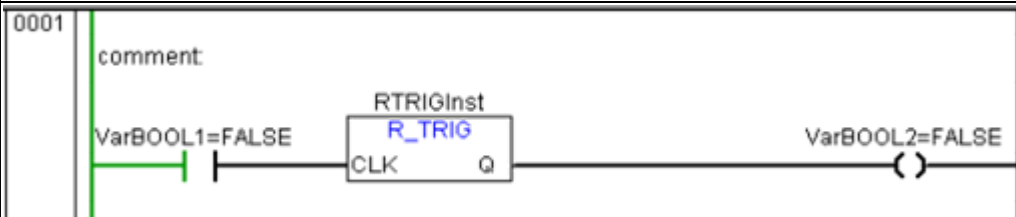
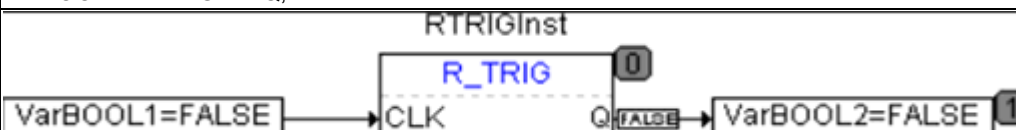
3.1.9.2.3 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
CLK	BOOL	输入	上升沿触发信号	FALSE

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	输出	当检测到上升沿时，Q=TRUE 每次调用指令时，Q=TRUE	0

3.1.9.2.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	R_TRIGInst			R_TRIG		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>RTRIGInst(CLK:=VarBOOL1); VarBOOL2:=RTRIGInst.Q;</pre>
连续功能图 (CFC)	

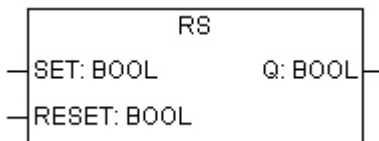
3.1.10 双稳态指令

3.1.10.1 RS（复位优先双稳态器）

3.1.10.1.1 功能

复位双稳态功能算法（RS）实现复位优先的 RS 触发器功能，输入端 SET、RESET，输出端 Q 均为 BOOL 型变量。

详细功能描述如下：Q1 = RS (SET, RESET) 表示：Q1 = NOT RESET AND (Q1 OR SET)。



RS 功能块

3.1.10.1.2 参数说明

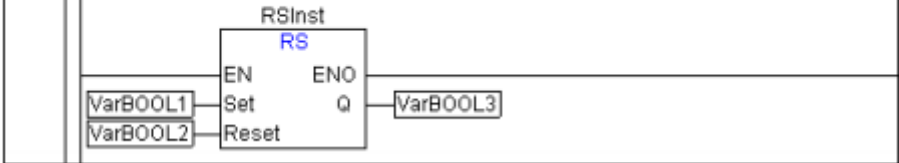
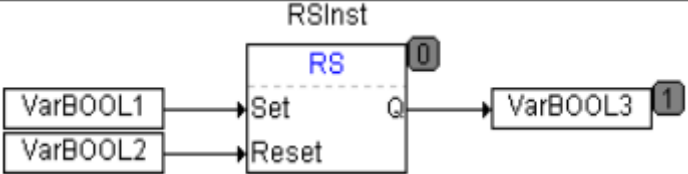
输入参数	数据类型	功能描述	默认值	掉电保护
Set	BOOL	置位	FALSE	FALSE

输入参数	数据类型	功能描述	默认值	掉电保护
Reset	BOOL	复位	FALSE	FALSE

输出参数	数据类型	功能描述	默认值	掉电保护
Q	BOOL	输出端	FALSE	TRUE

3.1.10.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	RSInst			RS		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarBOOL3			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	0001 comment:  说明: $\text{VarBOOL3} = \text{NOT VarBOOL2 AND (VarBOOL3 OR VarBOOL1)}$
结构化文本 (ST)	<pre>RSInst(Set:=VarBOOL1, Reset:=VarBOOL2, Q=>VarBOOL3);</pre>
连续功能图 (CFC)	

指令的真值表如下表所示。

真值表

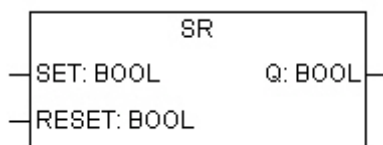
Reset	Set	Q(n+1)
0	0	Q(n)
0	1	1
1	0	0
1	1	0

3.1.10.2 SR（置位优先双稳态器）

3.1.10.2.1 功能

双稳态功能算法（SR）实现置位优先的 RS 触发器功能，输入端 SET、RESET，输出端 Q 均为 BOOL 型变量。

详细功能如下：Q1 = SR (SET, RESET) 表示：Q1 = (NOT RESET AND Q1) OR SET。



SR 功能块

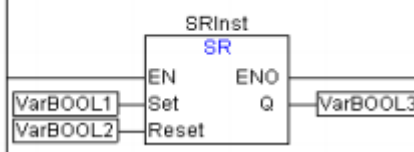
3.1.10.2.2 参数说明

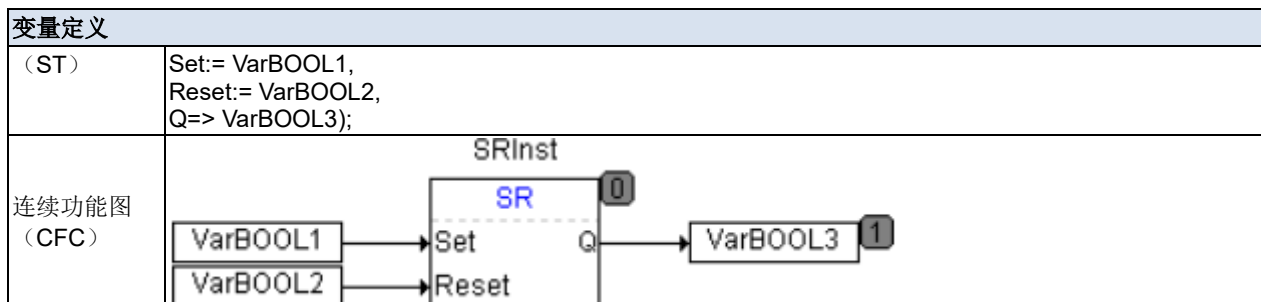
输入参数	说明	初始值（默认）	变量类型	掉电保护
Reset	复位	FALSE	BOOL	FALSE
Set	置位	FALSE	BOOL	FALSE

输出参数	说明	初始值（默认）	变量类型	掉电保护
Q	输出端	FALSE	BOOL	TRUE

3.1.10.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	SRInst			SR		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarBOOL3			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	0001 comment:  The diagram shows a ladder logic network. It starts with a normally open contact labeled 'VarBOOL1' connected to the 'Set' input of a function block labeled 'SRInst' (with 'SR' in blue). Below it, a normally open contact labeled 'VarBOOL2' is connected to the 'Reset' input of the same 'SRInst' block. The 'EN' (enable) input of the 'SRInst' block is connected to a power rail. The 'ENO' (enable output) of the 'SRInst' block is connected to a power rail. The 'Q' (output) of the 'SRInst' block is connected to a coil labeled 'VarBOOL3'. 说明：VarBOOL3=（NOT VarBOOL2 AND VarBOOL3）OR VarBOOL1
结构化文本	SRInst(



指令的真值表如下表所示。

真值表

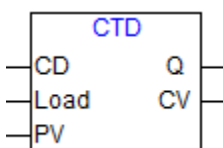
Reset	Set	Q(n+1)
0	0	Q(n)
0	1	1
1	0	0
1	1	1

3.1.11 计数器指令

3.1.11.1 CTD（递减计数器）

3.1.11.1.1 功能

减计数算法（CTD）实现当输入 CD 存在上升沿时计数值递减功能。



CTD 指令示意图

3.1.11.1.2 参数说明

输入参数	数据类型	功能描述	初始值（默认）	掉电保护
CD	BOOL	上升沿检测端	FALSE	FALSE
Load	BOOL	LOAD 为 TRUE 时计数变量 CV 初始化为 PV	FALSE	FALSE
PV	WORD	计数器设定值	0	TRUE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
Q	BOOL	计数标志输出	TRUE	FALSE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
		当 CV 小于等于 0 时，Q 输出 TRUE		
CV	WORD	当前计数值	0	TRUE

中间变量	数据类型	功能描述	初始值（默认）	掉电保护
M	BOOL	CD 检测变量	FALSE	TRUE

3.1.11.1.3 功能详细说明

减计数算法（CTD）实现当输入 CD 存在上升沿时计数值递减功能，输入端 CD、Load、输出端 Q 均为 BOOL 型变量，输入端 PV、输出端 CV 均为 WORD 型变量，详细描述如下：

■ 初始化功能

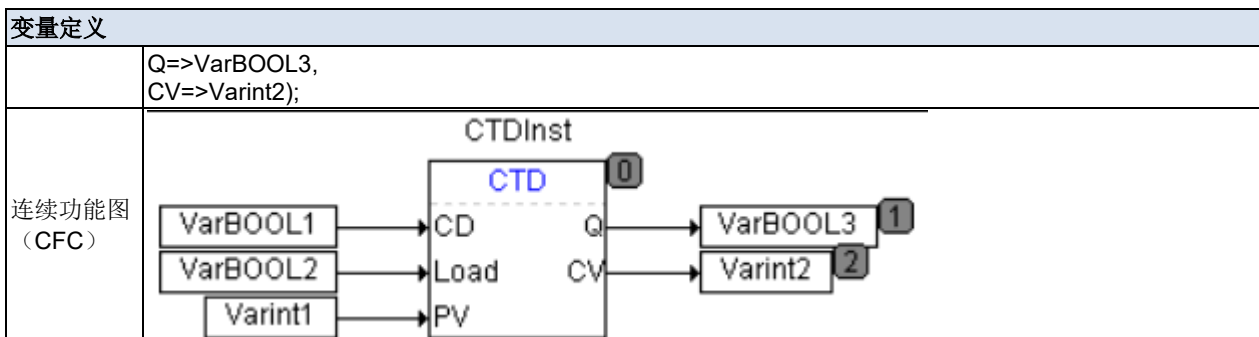
当初始值指令 Load 为真时，计数变量 CV 将被初始化为上限 PV。

■ 递减计数功能

如果 CD 有从 FALSE 到 TRUE 的上升沿时，假定大于 0，计数输出 CV 递减 1（也就是 CV 的值不小于 0）。当 CV 小于等于 0 的时候，输出 Q 置位。

3.1.11.1.4 指令使用举例

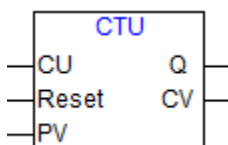
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	CTDInst			CTD		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	Varint1			INT	0	FALSE
0005	VarBOOL3			BOOL	FALSE	FALSE
0006	Varint2			INT	0	FALSE
编程语言		程序				
梯形图 (LD)	0001 comment VarBOOL1 VarBOOL2 Varint1 CTDInst CTD CD Load PV Q CV Varint2 VarBOOL3					
	结构化文本 (ST)					
CTDInst(CD:=VarBOOL1, Load:=VarBOOL2, PV:=Varint1,						



3.1.11.2 CTU（递增计数器）

3.1.11.2.1 功能

加计数算法（CTU）实现当输入 CU 存在上升沿时计数递加功能。



CTU 指令示意图

3.1.11.2.2 参数说明

输入参数	数据类型	功能描述	初始值（默认）	掉电保护
CU	BOOL	上升沿检测端	FALSE	FALSE
Reset	BOOL	Reset 为 TRUE 时计数变量 CV 初始化为 0	FALSE	FALSE
PV	WORD	设定值	0	TRUE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
Q	BOOL	当 CV 值大于等于 PV 时，Q 输出 TRUE	TRUE	FALSE
CV	WORD	计数输出	0	TRUE

中间变量	变量类型	功能描述	初始值（默认）	掉电保护
M	BOOL	CU 检测变量	FALSE	TRUE

3.1.11.2.3 功能详细说明

加计数算法（CTU）实现当输入 CU 存在上升沿时计数递加功能，输入变量 CU 和 Reset、输出变量 Q 都为 BOOL 类型，输入变量 PV 和输出变量 CV 都为 WORD 类型，详细描述如下：

■ 初始化功能

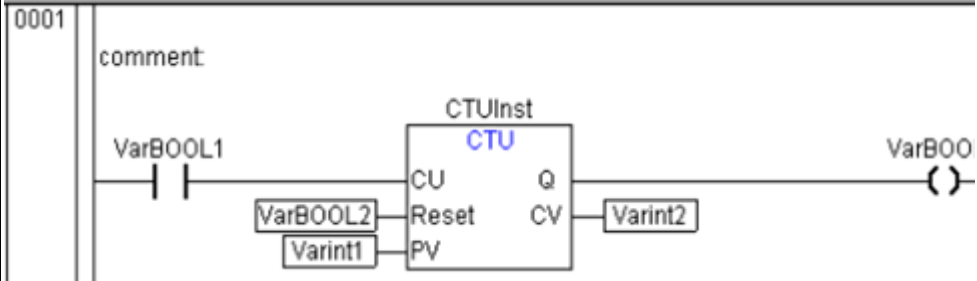
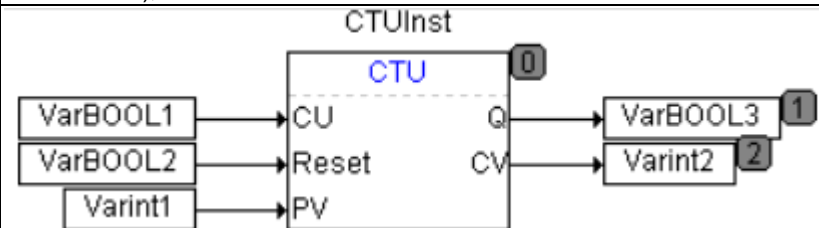
当 Reset 为真时，计数输出 CV 初始化为 0。

■ 计数功能

如果 CU 有从 FALSE 到 TRUE 的上升沿时，计数输出 CV 递增 1，直至 CV 大于等于设定值 PV，输出 Q 将返回 TRUE。

3.1.11.2.4 指令使用举例

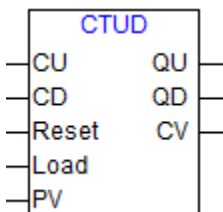
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	CTUInst			CTU		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	Varint1			INT	0	FALSE
0005	VarBOOL3			BOOL	FALSE	FALSE
0006	Varint2			INT	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>CTUInst(CU:=VarBOOL1, Reset:=VarBOOL2, PV:=Varint1, Q=>VarBOOL3, CV=>Varint2);</pre>
连续功能图 (CFC)	

3.1.11.3 CTUD（递增递减计数器）

3.1.11.3.1 功能

递增/递减算法。



CTUD 指令示意图

3.1.11.3.2 参数说明

输入参数	数据类型	功能描述	初始值（默认）	掉电保护
CU	BOOL	上升沿检测端	FALSE	FALSE
CD	BOOL	上升沿检测端	FALSE	FALSE
Reset	BOOL	Reset 为 TRUE 时计数变量 CV 初始化为 0	FALSE	FALSE
Load	BOOL	Load 为 TRUE 时计数变量 CV 初始化为 PV；但当 Reset 也为 TRUE 时，CV=0。	FALSE	FALSE
PV	WORD	设定值	0	TRUE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
QU	BOOL	当 CV 值等于 PV 时，QU 输出 TRUE	TRUE	FALSE
QD	BOOL	当 CV 值等于 0 时，QD 输出 TRUE	TRUE	FALSE
CV	WORD	计数输出	0	TRUE

中间变量	数据类型	功能描述	初始值（默认）	掉电保护
MU	BOOL	CU 检测变量	FALSE	TRUE
MD	BOOL	CD 检测变量	FALSE	TRUE

3.1.11.3.3 功能详细说明

递增/递减算法，输入变量 CU、CD、Reset、Load，输出变量 QU、QD 都为 BOOL 类型，输入变量 PV，输出变量 CV 都为 WORD 类型。详细功能描述如下：

■ 初始化功能

当 RESET 为 1 时，计数输出 CV 初始化为 0；当 LOAD 为 1 时，计数输出 CV 初始化为 PV。

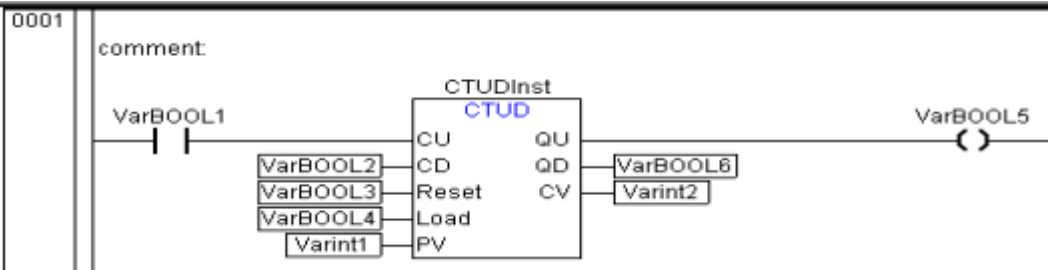
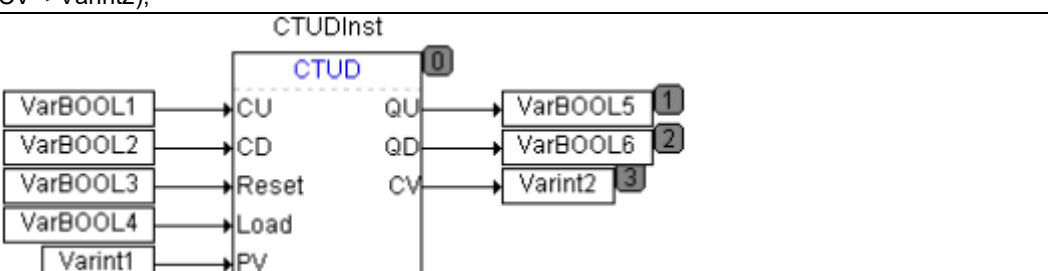
■ 计数功能

如果 CU 有从 FALSE 到 TRUE 的上升沿时，CV 加 1。如果 CD 有从 FALSE 到 TRUE 的上升沿时，CV 减 1（假定不会使 CV 的值小于 0）。

- 当 CV 大于或者等于 PV 时，QU 返回 TRUE。
- 当 CV 等于 0 时，QD 返回 TRUE。

3.1.11.3.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	CTUDInst			CTUD		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarBOOL3			BOOL	FALSE	FALSE
0005	VarBOOL4			BOOL	FALSE	FALSE
0006	VarBOOL5			BOOL	FALSE	FALSE
0007	VarBOOL6			BOOL	FALSE	FALSE
0008	Varint1			INT	0	FALSE
0009	Varint2			INT	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> CTUDInst(CU:=VarBOOL1, CD:=VarBOOL2, Reset:=VarBOOL3, Load:=VarBOOL4, PV:=Varint1, QU=>VarBOOL5, QD=>VarBOOL6, CV=>Varint2); </pre>
连续功能图 (CFC)	

3.1.12 定时器指令

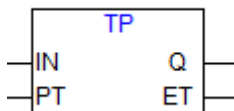
3.1.12.1 定时器指令

定时器包括普通定时器 TP、通电延时定时器 TON、断电延时定时器 TOF 和保持型通电延时定时器 TONR，下面分别描述。

3.1.12.2 TP（普通定时器）

3.1.12.2.1 功能

当 IN 从 FALSE 变成 TRUE 时，Q 为 TRUE，ET 开始以毫秒计时，直到 ET 等于 PT 后 ET 保持常数。在 ET 开始计时后，IN 值无效。当 ET 等于 PT 后，则 Q 为 FALSE；若 IN 变为 FALSE，ET 为 0。



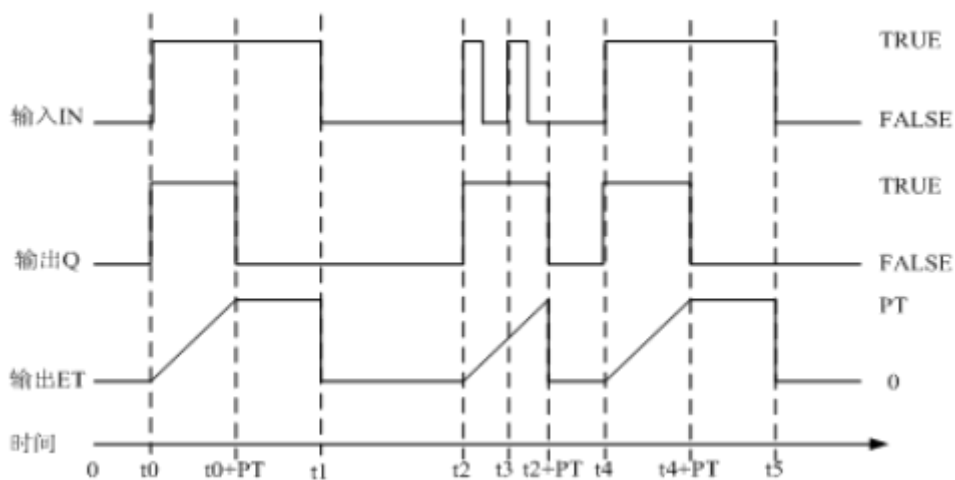
TP 功能块

3.1.12.2.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
IN	BOOL	定时器计时触发信号	当 IN 变成 TRUE 时，ET 以毫秒计时直到 ET 等于 PT，然后 ET 保持常数	FALSE
PT	TIME	定时时间值		0

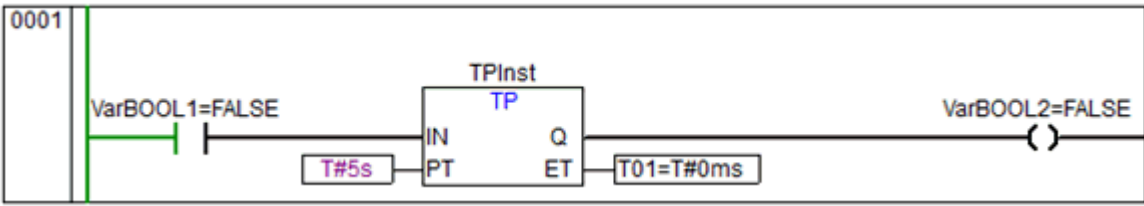
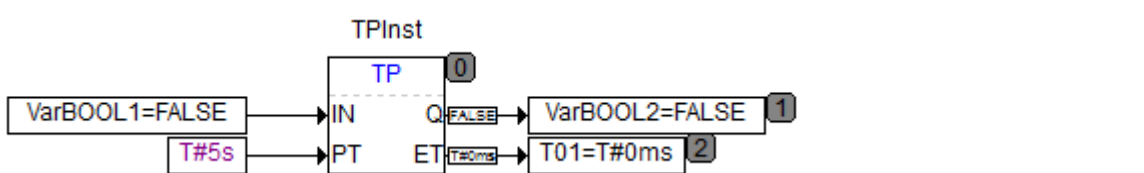
输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	定时器输出	当 IN 为 TRUE 时，定时器开始工作，Q 为 TRUE，在 ET 小于等于 PT 前，IN 无效，ET 等于 PT 时，Q 为 FALSE	FALSE
ET	TIME	当前时间值	当 IN 变成 TRUE 时，ET 以毫秒计时直到 ET 等于 PT，然后 ET 保持常数。计时完毕后，当 IN 为 FALSE 时，ET 等于 0	0

3.1.12.2.3 时序图



3.1.12.2.4 指令使用举例

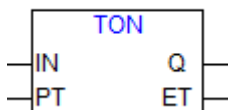
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	TPInst			TP		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	T01			TIME	T#0MS	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>TPInst(IN:=VarBOOL1, PT:=T#5S, Q=>VarBOOL2, ET=>T01);</pre>
连续功能图 (CFC)	

3.1.12.3 TON（通电延时定时器）

3.1.12.3.1 功能

当 IN 从 FALSE 变成 TRUE 时，ET 开始以毫秒计时，直到 ET 等于 PT 后 ET 保持常数，Q 变为 TRUE。ET 开始计时后，若 IN 变为 FALSE，则定时器停止计时，ET 变为 0。



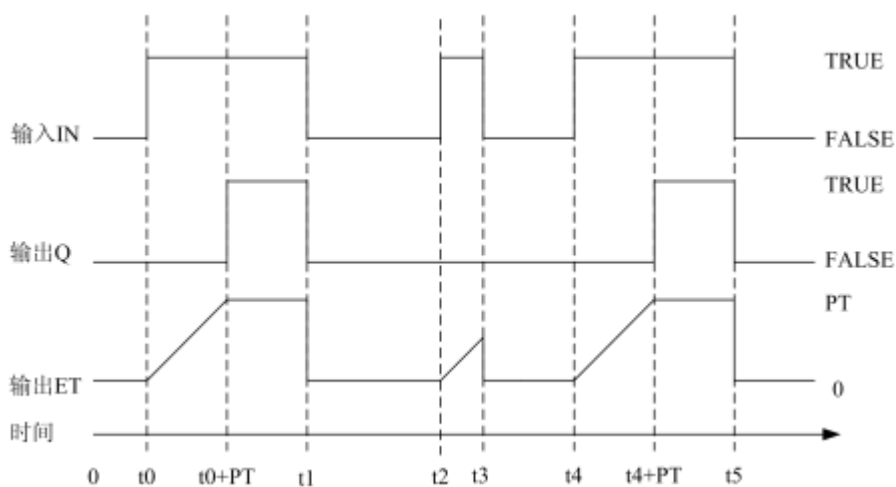
TON 功能块

3.1.12.3.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
IN	BOOL	定时器计时触发信号	当 IN 变成 TRUE 时，触发定时器开始计时。IN 为 FALSE 时停止计时	FALSE
PT	TIME	定时时间值		0

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	定时器输出	当 IN 为 FALSE 时，Q 为 FALSE，ET 为 0。当 IN 为 TRUE 时，定时器开始工作，ET 等于 PT 时，Q 为 TRUE	FALSE
ET	TIME	当前时间值	当 IN 变成 TRUE 时，ET 以毫秒计时直到 ET 等于 PT，然后 ET 保持常数。无论何时，当 IN 为 FALSE 时，ET 等于 0	0

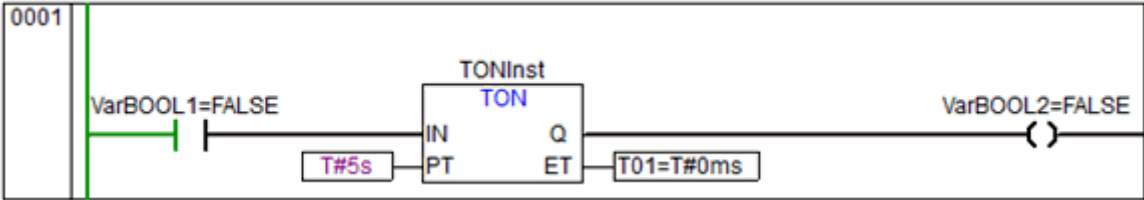
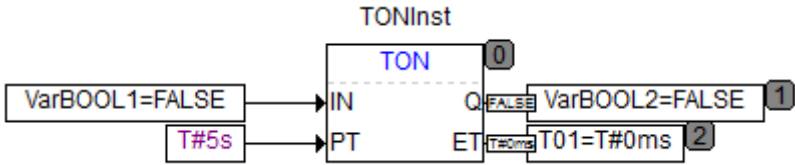
3.1.12.3.3 时序图



TON 时序图

3.1.12.3.4 指令使用举例

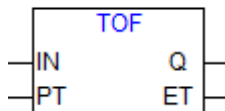
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	TONInst			TON		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	T01			TIME	T#0MS	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>TONInst(IN:=VarBOOL1, PT:=T#5S, Q=>VarBOOL2, ET=>T01);</pre>
连续功能图 (CFC)	

3.1.12.4 TOF（断电延时定时器）

3.1.12.4.1 功能

当 IN 由 TRUE 变成 FALSE 时，ET 以毫秒计时直到 ET 等于 PT 时，Q 变为 FALSE，ET 保持常数。ET 开始计数后，若 IN 重新变为 TRUE 时，定时器停止计时，ET 变为 0，Q 保持值不变。



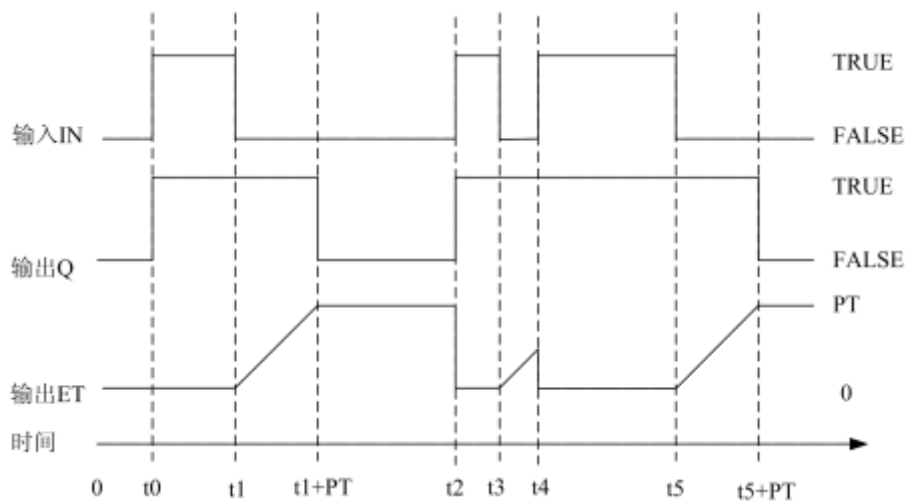
TOF 功能块

3.1.12.4.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
IN	BOOL	定时器计时触发信号	当 IN 由 TRUE 变成 FALSE 时，ET 以毫秒计时直到 ET 等于 PT，然后 ET 保持常数。Q 变为 FALSE	TRUE
PT	TIME	定时时间值		0

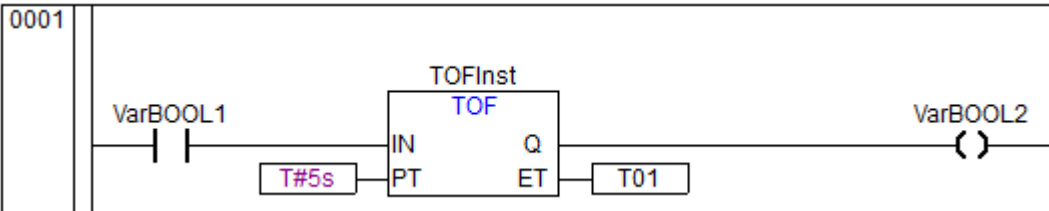
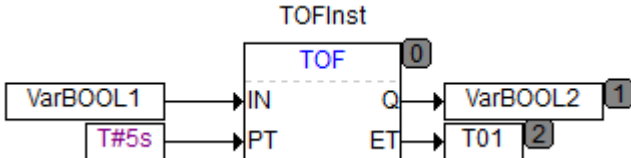
输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	定时器输出	当 IN 为 FALSE 且 ET 等于 PT 时，Q 为 FALSE。否则，Q 为 TRUE	TRUE
ET	TIME	当前时间值	当 IN 为 FALSE 时，ET 以毫秒计数直到 ET 等于 PT，然后 ET 保持常数。无论何时，当 IN 为 TRUE 时，ET 等于 0，Q 为 TRUE	0

3.1.12.4.3 时序图



3.1.12.4.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	TOFInst			TOF		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	T01			TIME	T#0MS	FALSE
编程语言		程序				

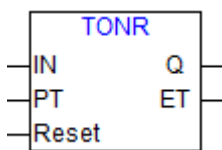
变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>TOFInst(IN:=VarBOOL1, PT:=T#5S, Q=>VarBOOL2, ET=>T01);</pre>
连续功能图 (CFC)	

3.1.12.5 TONR（保持型通电延时定时器）

3.1.12.5.1 功能

保持型通电延时定时器，可以累计输入信号的接通时间。

当 IN 变成 TRUE 时，ET 以毫秒计时直到 ET 等于 PT。如有外部触发导致在延时时间未达设定时间值前 IN 变成 FALSE，此时 ET 保持计时停止前的时间值，等到 IN 再次触发变成 TRUE 时，ET 继续计时，直到 ET 等于 PT，输出 Q 置位。RESET 为 1，则复位所有的输出值和中间变量，结束定时，为 FALSE 时，允许执行延时定时功能。



TONR 功能块

3.1.12.5.2 参数说明

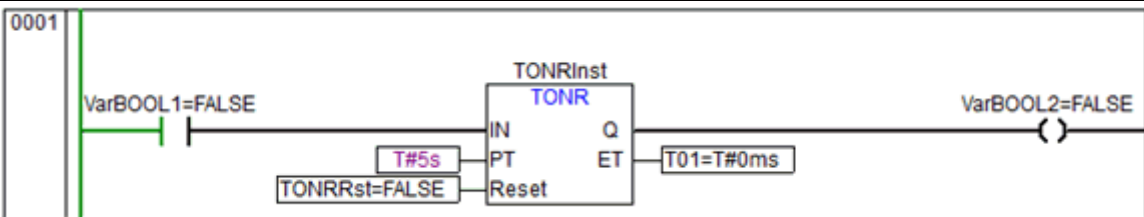
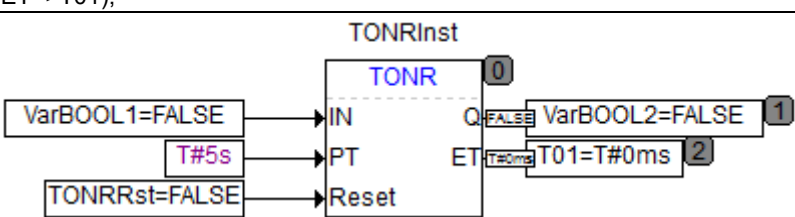
输入参数	数据类型	功能描述	参数值说明	默认值
IN	BOOL	定时器计时触发信号	当 IN 变成 TRUE 时，定时器开始计时，IN 变为 FALSE 时，定时器停止计时	FALSE
PT	TIME	定时时间值	设定的延时时间值，请按照 TIME 数据类型的格式要求来设定，	0

输入参数	数据类型	功能描述	参数值说明	默认值
			单位可以是 d, h, m, s 和 ms, 最小单位为 ms	
Reset	BOOL	复位端	TRUE: 复位所有的输出和中间变量, 结束定时 FALSE: 允许执行延时定时功能	FALSE

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	定时器溢出标志	当 Q 为 FALSE 时, 说明 ET 未达设定值 PT。当 ET 等于 PT 时, Q 为 TRUE 计时开始时, 无论 IN 为 TRUE 还是 FALSE, 都不影响 Q 的输出, Q 的输出只取决于 ET 是否等于 PT 以及 RESET 端状态	TRUE
ET	TIME	当前时间值	当 IN 变成 TRUE 时, ET 以毫秒计时直到 ET 等于 PT 计时开始时, 无论 IN 为 TRUE 还是 FALSE, 只要 Q 为 FALSE, ET 会一直输出当前时间值, 直到 ET 等于 PT	0

3.1.12.5.3 指令使用举例

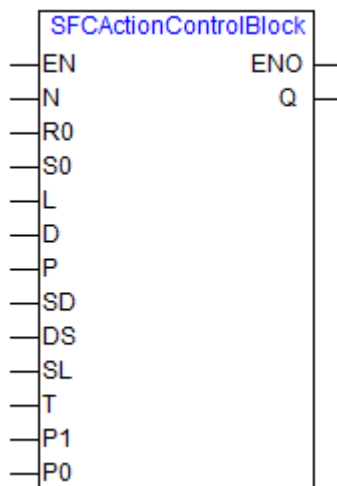
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	TONRInst			TONR		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	TONRRst			BOOL	FALSE	FALSE
0005	T01			TIME	T#0MS	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>TONRInst(IN:=VarBOOL1, PT:=T#5S, Reset:=TONRRst, Q=>VarBOOL2, ET=>T01);</pre>
连续功能图 (CFC)	

3.1.13 IEC_SFC

3.1.13.1 SFCActionControlBlock

该指令实现 SFC 语言中 IEC 步关联动作的控制。



IEC 动作指令

①在 SFC 编程语言中使用 IEC 步，必须加入该库，否则编译提示错误，当 IEC 步有关联动作时，系统自动调用 SFCActionControl 指令。

3.2 基础应用库

3.2.1 BCD 转换指令

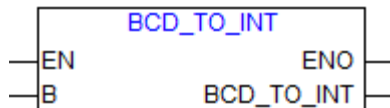
BCD 码的一个字节包含 0 到 99 之间的整数。每个十进制位对应 4 位，十位数存储在 4-7 位，个位数存储在 0-3 位。BCD 码格式和 16 进制表达方式很相似，差别在于 BCD 字节值是 0-99，而 16 进制是 0-FF。

BCD 码用 4 位二进制数表示一个十进制数位，整个十进制数用一串 BCD 码来表示。例如，十进制数 59 表示成 BCD 码为 0101 1001，但表示成二进制为 2#111011。十进制 51 转换成 BCD 码，5 的二进制是 0101，1 的二进制是 0001，那么 51 转换 BCD 码为 0101 0001。

3.2.1.1 BCD_TO_INT (BCD 码转整型)

3.2.1.1.1 功能

BCD 码转换为整型值算法 (BCD_TO_INT) 要求输入值是 BYTE 类型，而输出值则是 INT 类型。完成 BCD 码到整型值的转换。



BCD_TO_INT 功能块

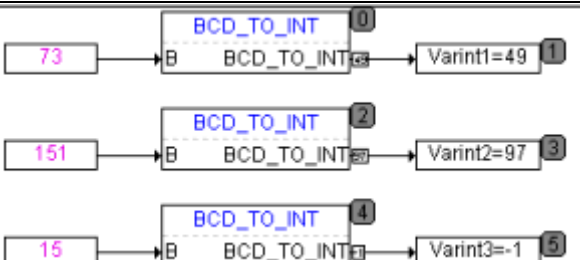
3.2.1.1.2 参数说明

输入参数	数据类型	参数值说明	初始值 (默认)
B	BYTE	需要转换的 BCD 码，输入 BCD 码的二进制形式（或者该二进制对应的十进制和十六进制），比如 BCD 码 49，表示为 2#100 1001（或者对应 10#73、16#49），则此处输入 2#100 1001（或者 10#73，16#49）	0

输出参数	数据类型	参数值说明	初始值 (默认)
BCD_TO_INT	INT	转换后的 INT 值 如果输入的字节不是 BCD 码，输出是-1	0

3.2.1.1.3 指令使用举例

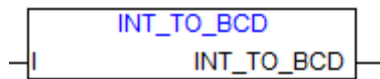
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varint1			INT	0	FALSE
0002	Varint2			INT	0	FALSE
0003	Varint3			INT	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	0001 comment: 
	0002 comment: 
	0003 comment: 
结构化文本 (ST)	Varint1:=BCD_TO_INT(73); (*结果为 49*) Varint2:=BCD_TO_INT(151); (*结果为 97*) Varint3:=BCD_TO_INT(15); (*结果为-1, 因为不是 BCD 码格式*)
连续功能图 (CFC)	

3.2.1.2 INT_TO_BCD（整型转 BCD 码）

3.2.1.2.1 功能

整数值转换为 BCD 码算法（INT_TO_BCD）要求输入值是 INT 类型，而输出值则是 BYTE 类型。完成整型值到 BCD 码的转换。当整数值不能转换成 BCD 码字节时，输出数值 255。



INT_TO_BCD 功能块

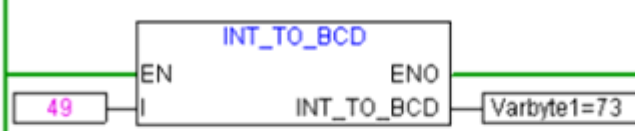
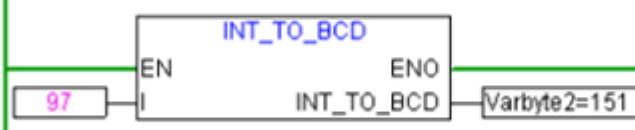
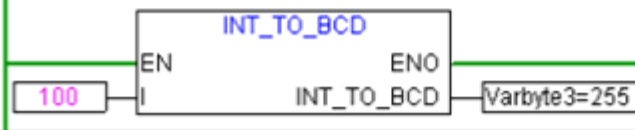
3.2.1.2.2 参数说明

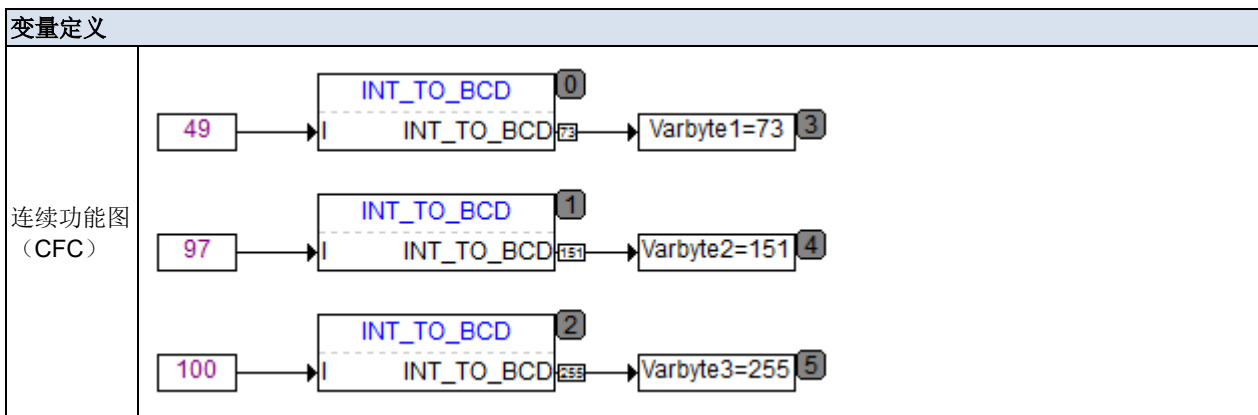
输入参数	数据类型	参数值说明	初始值（默认）
I	INT	需要转换的整型数据值 如果整数值为 49，则此处输入整型数据 49	0

输出参数	数据类型	参数值说明	初始值（默认）
INT_TO_BCD	BYTE	转换后的 BCD 码 比如将 49 转换为 BCD 码为 2#100 1001，则此处输出 2#1001001（或者该二进制对应的 10#73、16#49）	0

3.2.1.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varbyte1			BYTE	0	FALSE
0002	Varbyte2			BYTE	0	FALSE
0003	Varbyte3			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	0001 comment: 
	0002 comment: 
	0003 comment: 
结构化文本 (ST)	<pre> Varbyte1:=INT_TO_BCD(49); (*结果为 73*) Varbyte2:=INT_TO_BCD(97); (*结果为 151*) Varbyte2:=INT_TO_BCD(100); (*错误！输出：255*) </pre>

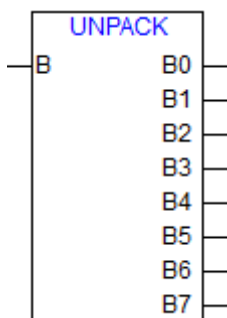


3.2.2 位_字节操作指令

3.2.2.1 UNPACK (位拆分指令)

3.2.2.1.1 功能

将输入 B 的第 7~0 位依次取出赋值给 B7~B0。与 PACK 指令相反。



UNPACK 功能块

3.2.2.1.2 参数说明

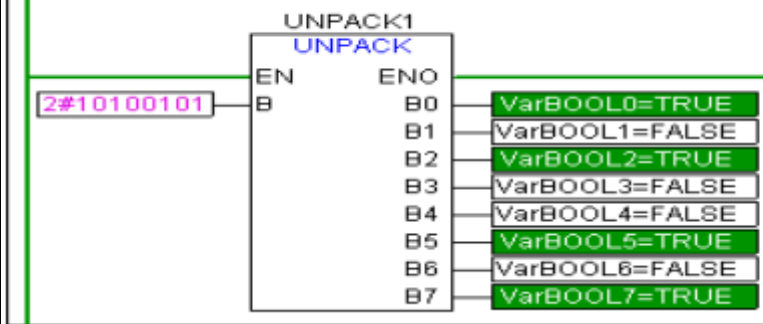
输入参数	说明	变量类型	初始值（默认）	掉电保护
B	BYTE 型输入变量	BYTE	0	FALSE

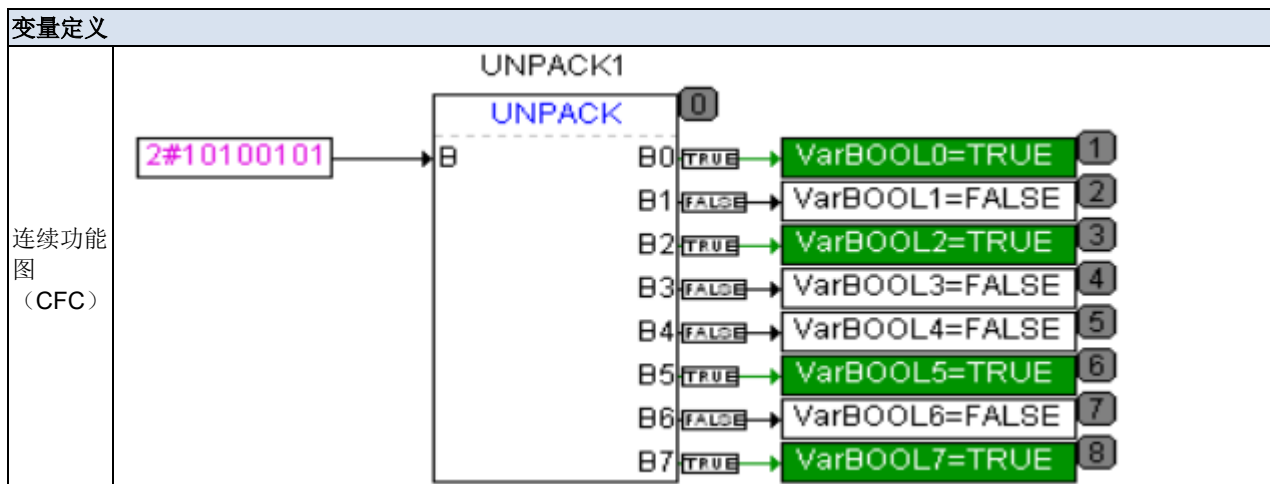
输出参数	说明	变量类型	初始值（默认）	掉电保护
B0	BOOL 型输出变量	BOOL	FALSE	FALSE
B1		BOOL	FALSE	FALSE
B2		BOOL	FALSE	FALSE
B3		BOOL	FALSE	FALSE
B4		BOOL	FALSE	FALSE

输出参数	说明	变量类型	初始值（默认）	掉电保护
B5		BOOL	FALSE	FALSE
B6		BOOL	FALSE	FALSE
B7		BOOL	FALSE	FALSE

3.2.2.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	UNPACK1			UNPACK		FALSE
0002	VarBOOL0			BOOL	FALSE	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	VarBOOL2			BOOL	FALSE	FALSE
0005	VarBOOL3			BOOL	FALSE	FALSE
0006	VarBOOL4			BOOL	FALSE	FALSE
0007	VarBOOL5			BOOL	FALSE	FALSE
0008	VarBOOL6			BOOL	FALSE	FALSE
0009	VarBOOL7			BOOL	FALSE	FALSE

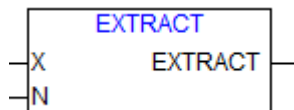
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> UNPACK1(B:=2#10101010, B0=>VarBOOL0, B1=>VarBOOL1, B2=>VarBOOL2, B3=>VarBOOL3, B4=>VarBOOL4, B5=>VarBOOL5, B6=>VarBOOL6, B7=>VarBOOL7); </pre>



3.2.2.2 EXTRACT（位提取指令）

3.2.2.2.1 概述

将输入变量 X 的指定第 N 位输出。



EXTRACT 功能块

3.2.2.2.2 参数说明

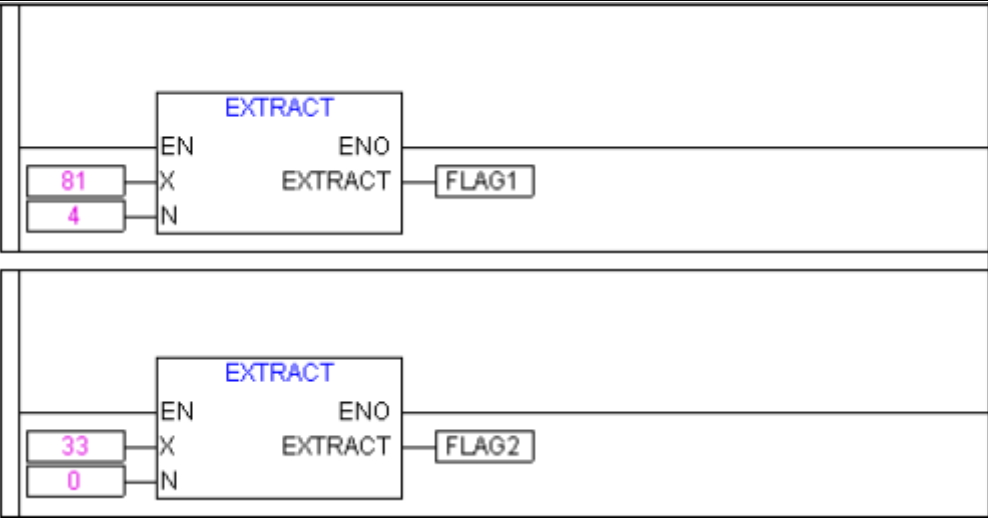
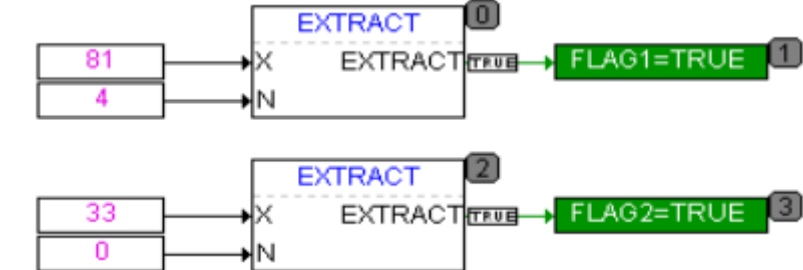
输入参数	说明	变量类型	初始值（默认）
X	被操作变量	DWORD	0
N	位置	BYTE	0

输出参数	说明	变量类型	初始值（默认）
EXTRACT	输出	BOOL	0

3.2.2.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	FLAG1			BOOL	FALSE	FALSE
0002	FLAG2			BOOL	FALSE	FALSE

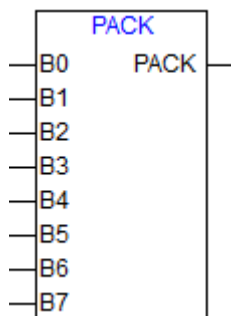
编程语言	程序
------	----

梯形图 (LD)	 (*结果 FLAG1=TRUE, FLAG2=TRUE *)
结构化文本 (ST)	<pre>FLAG1:= EXTRACT (81,4); (*结果: TRUE, 因为 81 的二进制数是 1010001, 所以第四位是 1*) FLAG2:= EXTRACT (33,0); (*结果: TRUE, 因为 33 的二进制数是 100001, 所以第 0 位是 1*)</pre>
连续功能图 (CFC)	 (*结果 FLAG1=TRUE, FLAG2=TRUE *)

3.2.2.3 PACK（位整合指令）

3.2.2.3.1 概述

将 B0~B7 按照从第 0 位到第 7 位的顺序组合成一个 BYTE 变量。与这个指令相对应的指令是 UNPACK。



PACK 功能块

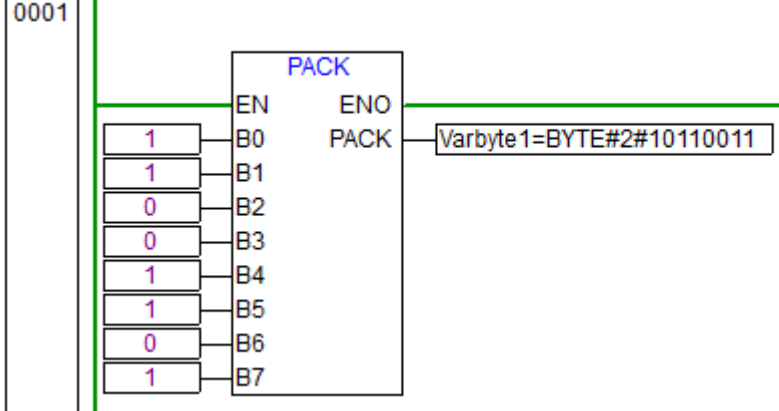
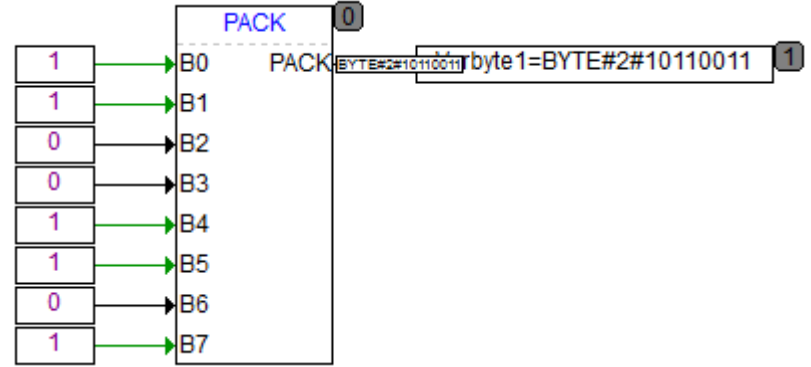
3.2.2.3.2 参数说明

输入参数	变量类型	说明	初始值（默认）
B0	BOOL	第 0 位数据	0
B1	BOOL	第 1 位数据	0
B2	BOOL	第 2 位数据	0
B3	BOOL	第 3 位数据	0
B4	BOOL	第 4 位数据	0
B5	BOOL	第 5 位数据	0
B6	BOOL	第 6 位数据	0
B7	BOOL	第 7 位数据	0

输出参数	变量类型	说明	初始值（默认）
Pack	BYTE	BYTE 字节由高到低：B7~B0	0

3.2.2.3.3 指令使用举例

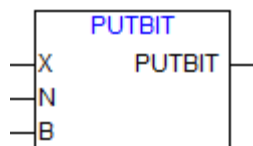
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Varbyte1			BYTE	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>Varbyte1:= PACK(1,1,0,0,1,1,0,1); (*结果为 2#10110011*)</pre>
连续功能图 (CFC)	

3.2.2.4 PUTBIT (位赋值命令)

3.2.2.4.1 功能

置变量指定位为指定值算法 (PUTBIT) 完成置输入值 X 指定的第 N 位为指定值 B。



PUTBIT 功能块

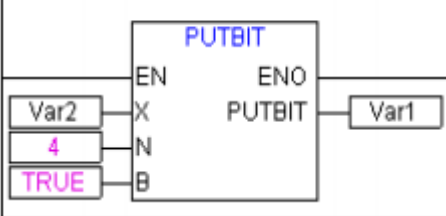
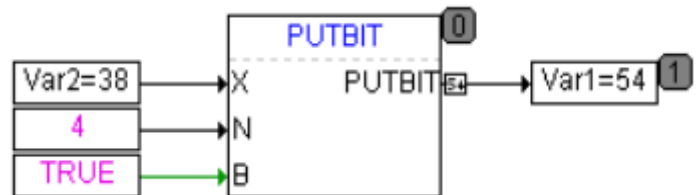
3.2.2.4.2 参数说明

输入参数	数据类型	说明	初始值（默认）
X	DWORD	被操作变量	0
N	BYTE	位置	0
B	BOOL	指定值	FALSE

输出参数	数据类型	说明	初始值（默认）
PutBit	DWORD	输出完成指定值算法的变量值	0

3.2.2.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var2			DWORD ▾	0	FALSE ▾
0002	Var1			DWORD ▾	0	FALSE ▾

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>Var2:=38; (*二进制 100110*) Var1:=PUTBIT(Var2,4,TRUE); (*结果: 54 = 2#110110*)</pre>
连续功能图 (CFC)	

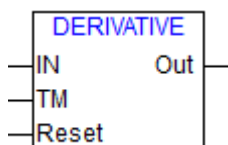
3.2.3 数学辅助功能

3.2.3.1 DERIVATIVE（微分）

3.2.3.1.1 功能

该指令对连续输入的变量进行微分运算。为了获得最好结果，DERIVATIVE 指令只对最新的四个输入

值进行微分，减小因输入参数不精确产生的误差。



DERIVATIVE 功能块

3.2.3.1.2 微分迭代公式

$$OUT = \frac{3 * [IN(k) - IN(k - 3)] + IN(k - 1) - IN(k - 2)}{3 * TM(k - 2) + 4 * TM(k - 1) + 3 * TM(k)} * 1000$$

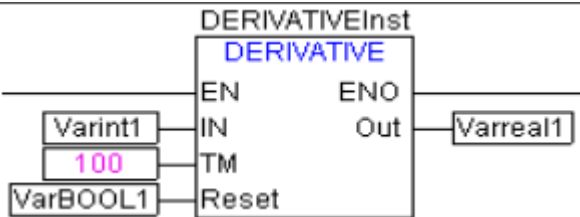
k-3、k-2、k-1、k 为连续四次输入值的标记。

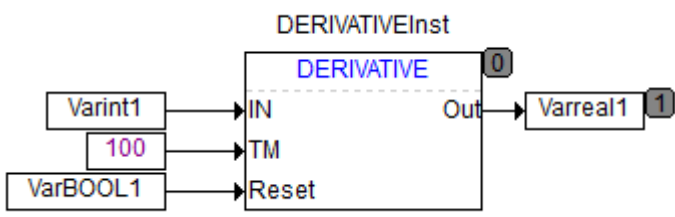
3.2.3.1.3 参数说明

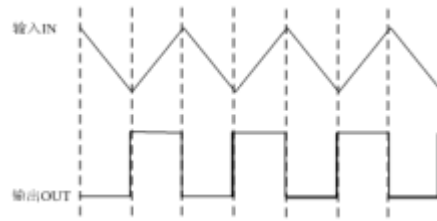
输入参数	数据类型	功能描述	初始值（默认）
IN	REAL	输入变量	0
TM	DWORD	时间常数	0
Reset	BOOL	重启信号	FALSE

输出参数	数据类型	功能描述	初始值（默认）
OUT	REAL	微分算法的输出	0

3.2.3.1.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	DERIVATIVEInst			DERIVATIVE		FALSE
0002	Varint1			INT	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	Varreal1			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		<pre>DERIVATIVEInst(IN:=Varint1,</pre>				

变量定义	
	TM:=100, Reset:=VarBOOL1, Out=>Varreal1);
连续功能图 (CFC)	

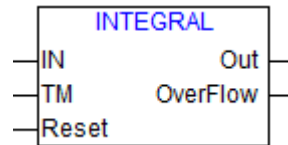


输入输出对应关系

3.2.3.2 INTEGRAL（积分）

3.2.3.2.1 功能

积分算法（INTEGRAL）近似求解输入函数的积分。



INTEGRAL 功能块

3.2.3.2.2 积分迭代公式

$$A(k) = A(k-1) + TM * IN(k-1)$$

$$B(k) = B(k-1) + TM * IN(k)$$

$$OUT(k) = \frac{A(k) + B(k)}{2}$$

k-1、k 为连续两次输入值的标记。

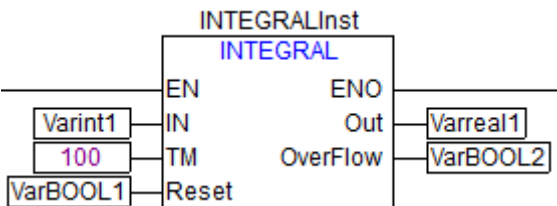
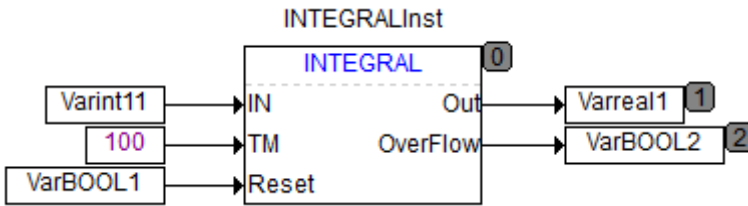
3.2.3.2.3 参数说明

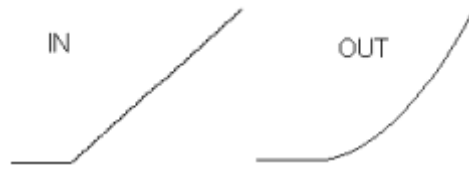
输入参数	数据类型	功能描述	初始值（默认）	掉电保护
IN	REAL	输入变量	0	FALSE
TM	DWORD	时间常数	0	TRUE
Reset	BOOL	重启信号	FALSE	TRUE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
Out	REAL	近似积分值	0	TRUE
OverFlow	BOOL	溢出标志	FALSE	TRUE

3.2.3.2.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	INTEGRALInst			INTE...		FALSE
0002	Varint1			INT	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	Varreal1			REAL	0	FALSE
0005	VarBOOL2			BOOL	FALSE	FALSE

编程语言	程序
梯形图（LD）	
结构化文本（ST）	<pre>INTEGRALInst(IN:=Varint1, TM:=100, Reset:=VarBOOL1, Out=>Varreal1, OverFlow=> VarBOOL2);</pre>
连续功能图（CFC）	

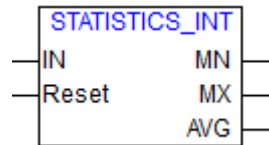


输入输出对应关系

3.2.3.3 STATISTICS_INT（整型统计）

3.2.3.3.1 功能

计算标准统计算法（STATISTICS_INT）统计所有 INT 型输入变量中的最小值、最大值和平均值。



STATISTICS_INT 功能块

3.2.3.3.2 参数说明

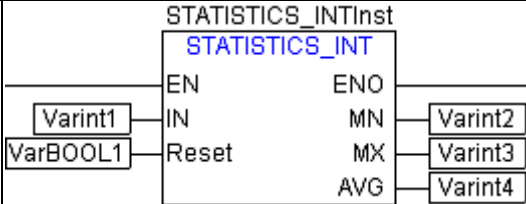
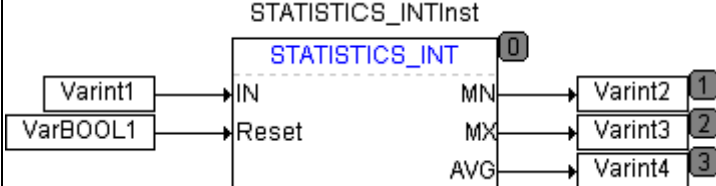
输入参数	数据类型	功能描述	初始值（默认）
IN	INT	被操作变量	0
Reset	BOOL	初始化	FALSE

输出参数	数据类型	功能描述	初始值（默认）
MN	INT	最小值输出	32767
MX	INT	最大值输出	-32768
AVG	INT	平均值输出	0

3.2.3.3.3 指令使用举例

3.2.3.3.4

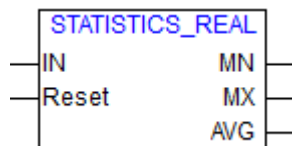
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电值
0001	STATISTICS_INTInst			STATISTICS_INT		FALSE
0002	Varint1			INT	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	Varint2			INT	0	FALSE
0005	Varint3			INT	0	FALSE
0006	Varint4			INT	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> STATISTICS_INTInst(IN:= Varint1, Reset:= VarBOOL1, MN=>Varint2, MX=>Varint3, AVG=>Varint4); </pre>
连续功能图 (CFC)	

3.2.3.4 STATISTICS_REAL (实型统计)

3.2.3.4.1 功能

统计所有 REAL 型输入变量中的最大值、最小值和平均值。



STATISTICS_REAL 功能块

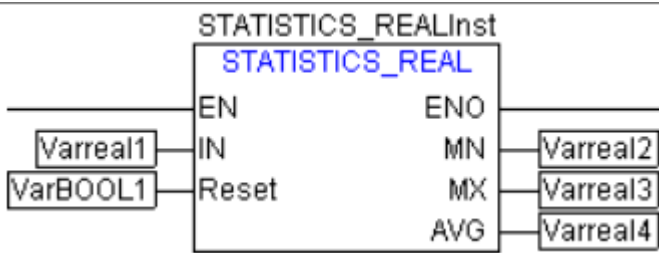
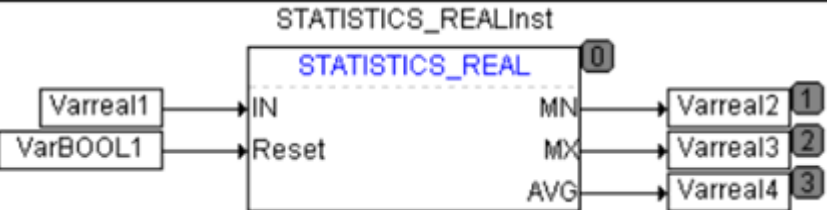
3.2.3.4.2 参数说明

输入参数	数据类型	功能描述	初始值（默认）
IN	REAL	被操作变量	0
Reset	BOOL	初始化	FALSE

输出参数	数据类型	功能描述	初始值（默认）
MN	REAL	最小值输出	3E+38
MX	REAL	最大值输出	1E-37
AVG	REAL	平均值输出	0

3.2.3.4.3 指令使用举例

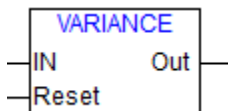
变量定义						
0001	STATISTICS_REAL...			STAT...		FALSE
0002	Varreal1			REAL	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	Varreal2			REAL	0	FALSE
0005	Varreal3			REAL	0	FALSE
0006	Varreal4			REAL	0	FALSE

编程语言	程序
梯形图（LD）	
结构化文本（ST）	<pre> STATISTICS_REALInst(IN:=Varreal1, Reset:=VarBOOL1, MN=>Varreal2, MX=>Varreal3, AVG=>Varreal4); </pre>
连续功能图（CFC）	

3.2.3.5 VARIANCE（平方偏差）

3.2.3.5.1 功能

计算输入值方差算法（VARIANCE）计算所有输入变量的方差。标准偏差可以由平方偏差的平方根得到。



VARIANCE 功能块

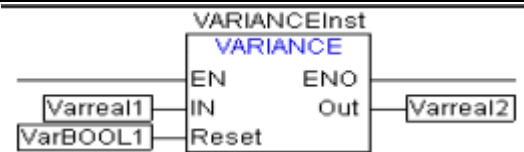
3.2.3.5.2 参数说明

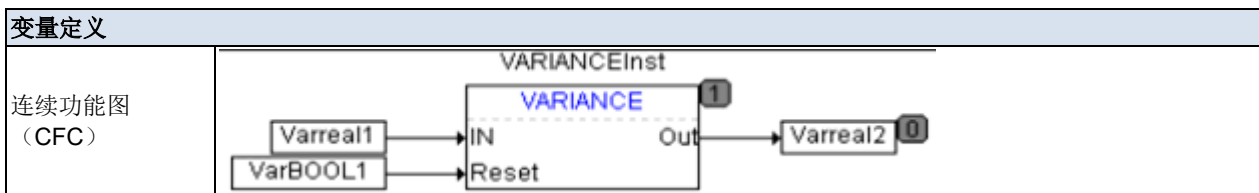
输入参数	数据类型	功能描述	初始值（默认）	掉电保护
IN	REAL	被操作变量	0	FALSE
Reset	BOOL	初始化	FALSE	TRUE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
Out	REAL	方差值输出	0	FALSE

中间变量	数据类型	初始值（默认）	掉电保护
Z	INT	0	TRUE
A	REAL	0	TRUE
B	REAL	0	TRUE

3.2.3.5.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VARIANCEInst			VARI...		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	Varreal1			REAL	0	FALSE
0004	Varreal2			REAL	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		<pre>VARIANCEInst(IN:=Varreal1, Reset:=VarBOOL1, OUT=>Varreal2);</pre>				

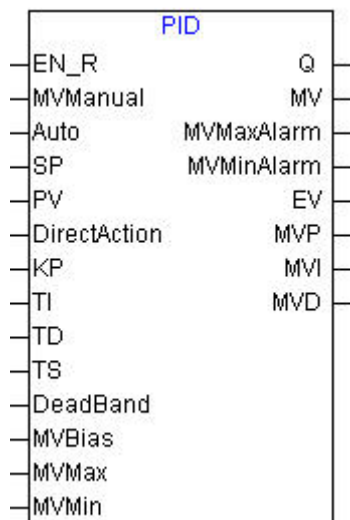


3.2.4 控制器指令

3.2.4.1 PID（比例积分微分控制器）

3.2.4.1.1 概述

比例积分微分控制器，对现场输入的过程值与设定值比较，对其进行PID调节控制。将调节的指令输出给现场设备从而达到及时响应的调节任务，同时对输出值进行限幅操作，以保证输出在许可的范围内操作，并对超限数值及时报警。



PID 功能块

3.2.4.1.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值	掉电保护
EN_R	BOOL	使能	FALSE: 无效 TRUE: 上升沿设定、高电平有效	FALSE	TRUE
MVManual	REAL	手动输入值		0	TRUE
Auto	BOOL	“自动”模式选择	FALSE: 手动 TRUE: 自动	FALSE	TRUE

输入参数	数据类型	功能描述	参数值说明	默认值	掉电保护
SP	REAL	设定值		0	TRUE
PV	REAL	过程值		0	FALSE
DirectAction	BOOL	“正作用”方式选择	FALSE: 反作用 (EV=SP-PV) TRUE: 正作用 (EV=PV-SP)	FALSE	TRUE
KP	REAL	比例增益		1	TRUE
TI	REAL	积分时间 (S)	Ti>=0, 如果积分时间为 0, 表示没有积分环节	1	TRUE
TD	REAL	微分时间 (S)	Td>=0, 如果微分时间为 0, 表示没有微分环节	0	TRUE
TS	REAL	运算周期 (S)	Ts>0	0	TRUE
DeadBand	REAL	偏差死区限	(1) 若 DeadBand=0, 表示没有死区 (2) 偏差绝对值大于该值时有效, 否则偏差为 0	0	TRUE
MVBias	REAL	前馈控制量		0.0	TRUE
MVMax	REAL	输出值上限		+100	TRUE
MVMin	REAL	输出值下限		-100	TRUE

输出参数	数据类型	功能描述	参数值说明	默认值	掉电保护
Q	BOOL	使能状态标志	FALSE: 无效 TRUE: 上升沿设定、高电平有效	FALSE	TRUE
MV	REAL	控制量输出		0	TRUE
MVMaxAlarm	BOOL	输出值超上限报警	FALSE: 未超上限 TRUE: 已超上限	FALSE	TRUE
MVMinAlarm	BOOL	输出值超下限报警	FALSE: 未超下限 TRUE: 已超下限	0	TRUE
EV	REAL	设定值与过程值的偏差	若有死区, EV 则为经死区处理后的偏差值	0	FALSE
MVP	REAL	比例分量		FALSE	TRUE
MVI	REAL	积分分量		1	TRUE
MVD	REAL	微分分量		1	TRUE

3.2.4.1.3 功能详细说明

1. 使能

当使能端 EN_R 为真时, 程序正常运行, 否则程序返回不运算。

2. 判断设置值

■ 输出上下限 MVMax、MVMin 设置判断

输出下限一定要小于输出上限, 否则输出值超上下限标志均置位且程序返回不运算。

■ 积分时间 Ti、微分时间 Td 和死区范围 DeadBand 的设置判断

积分时间 Ti、微分时间 Td 和死区范围 DeadBand 必须为非负数, 否则按照初始值进行运算。

3. 计算

■ 手动模式

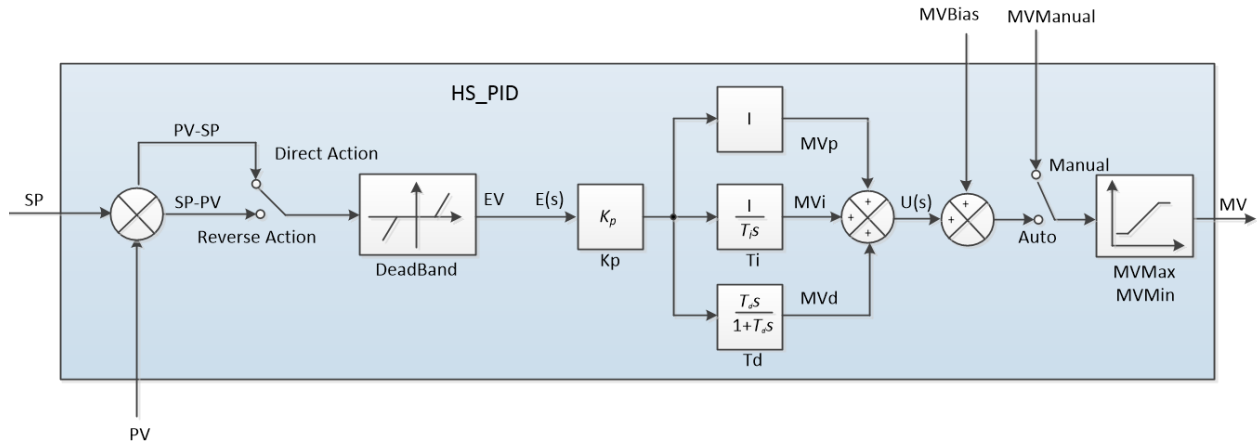
输出 MV 等于手动输入值 MVManual, 并且 SP 跟踪 PV。

■ 自动模式

根据正反作用方式 **DirectAction**、设定值 **SP**、测量值 **PV** 和死区范围 **DeadBand** 计算偏差；再根据输出 **MV**、输出上下限值以及偏差的正负计算积分饱和系数。

根据比例增益 **Kp**、积分时间 **Ti**、微分时间 **Td** 和积分饱和系数计算控制输出的比例分量 **MVp**、积分分量 **MVi**、微分分量 **MVd** 以及控制输出 **MV**。

PID 功能块的逻辑结构如图所示。



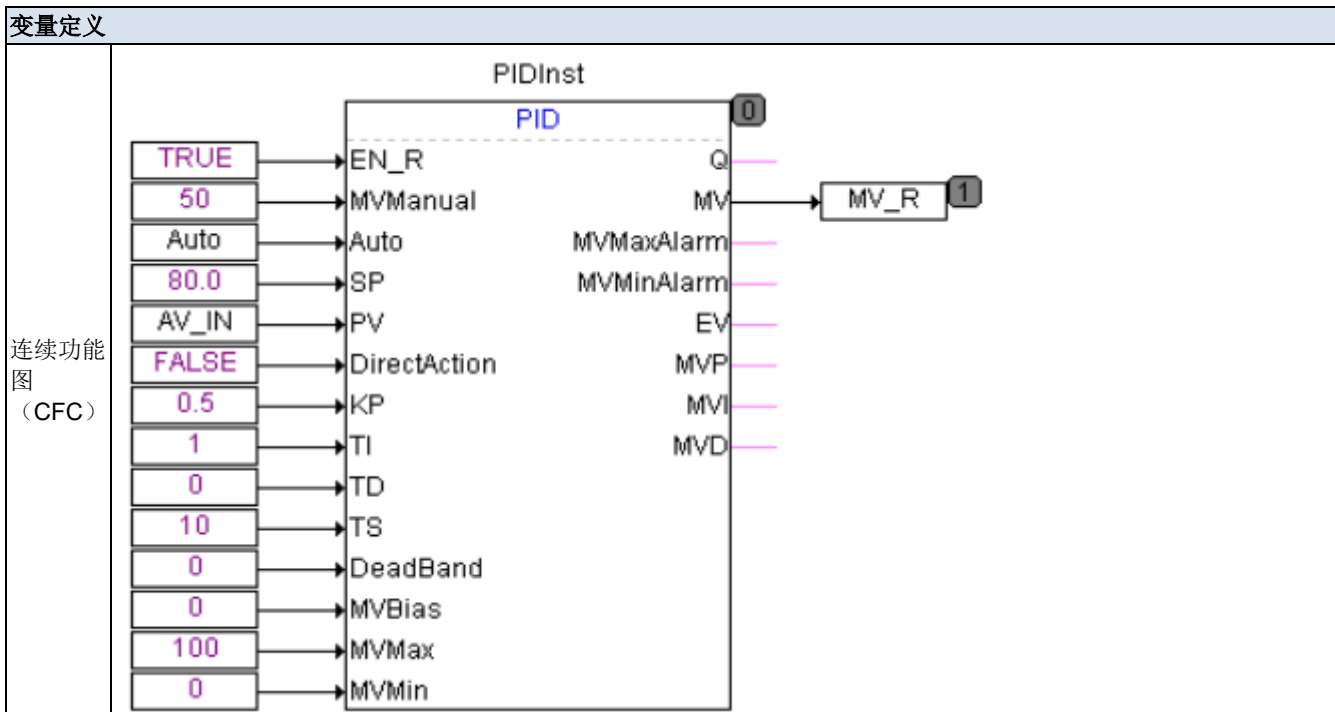
PID 功能块逻辑结构图

3.2.4.1.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	PIDInst			PID		FALSE
0002	Auto			BOOL	FALSE	FALSE
0003	AV_IN			REAL	0	FALSE
0004	MV_R			REAL	0	FALSE
编程语言		程序				

变量定义

梯形图 (LD)	PLC_PRG: 设定值为80, 当Auto为TRUE时自动调节(当Auto为FALSE时, 输出50), 反方向控制(实际值小于设定值输出增大), 最大值、最小值也与实际值和设定值同量纲, 此处设为100和0. 0001 PIDInst PID 50 Auto 80.0 AV_IN FALSE 0.5 1 0 10 0 0 100 0 EN_R MVManual Auto SP PV DirectAction KP TI TD TS DeadBand MVBias MVMax MVMin Q MV MVMaxAlarm MVMinAlarm EV MVP MVI MVD MV_R
	结构化文本 (ST) <pre> PIDInst(EN_R:=TRUE, MVManual:=50, Auto:=Auto, SP:=80.0, PV:=AV_IN, DirectAction:=FALSE, KP:=0.5, TI:=1, TD:=0, TS:=10, DeadBand:=0, MVBias:=0, MVMax:=100.0, MVMin:=0, MV=>MV_R </pre>

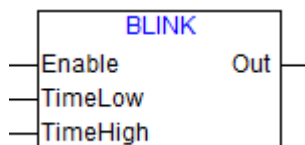


3.2.5 信号发生器指令

3.2.5.1 BLINK（脉冲信号发生器）

3.2.5.1.1 概述

脉冲信号发生器算法（BLINK）完成按指定时间输出高电平和低电平的矩形波。



BLINK 功能块

3.2.5.1.2 输入信号

标识	说明	变量类型	初始值（默认）	掉电保护
Enable	使能端	BOOL	FALSE	TRUE
TimeLow	低电平时间	TIME	T#0S	TRUE

标识	说明	变量类型	初始值（默认）	掉电保护
TimeHigh	高时平时间	TIME	T#0S	TRUE

3.2.5.1.3 输出信号

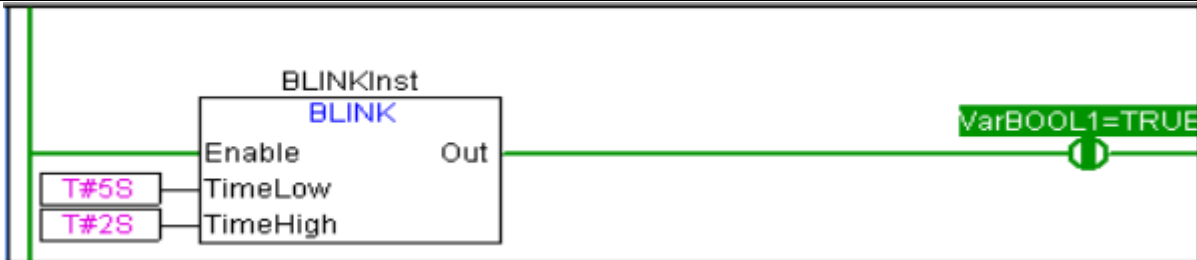
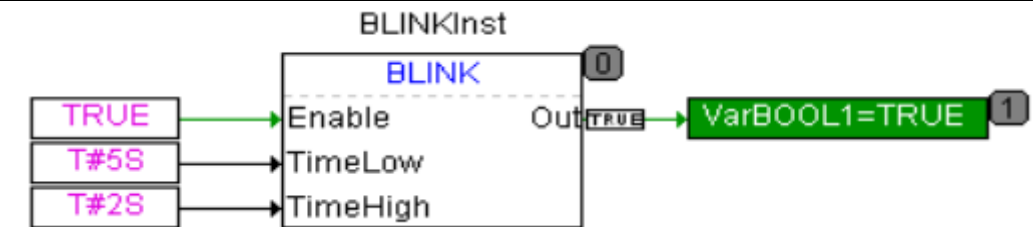
标识	说明	变量类型	初始值（默认）	掉电保护
Out	输出	BOOL	FALSE	TRUE

3.2.5.1.4 中间变量

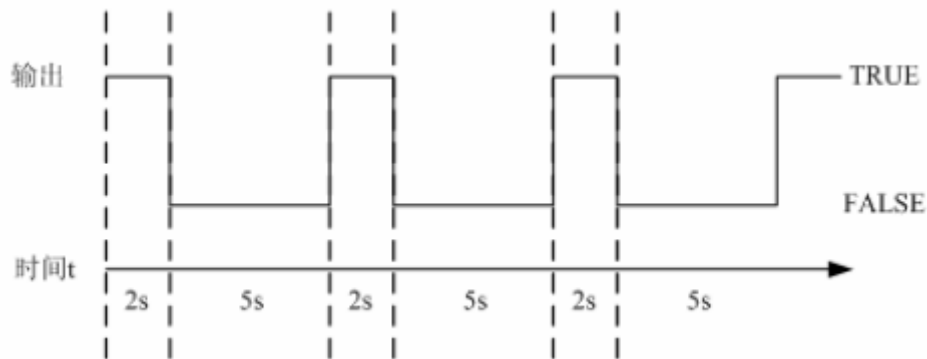
标识	说明	变量类型	初始值（默认）	掉电保护
CLOCK	功能块实例	TP		TRUE

3.2.5.1.5 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	BLINKInst			BLINK		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>BLINKInst(Enable:=TRUE, TimeLow:=T#5S, TimeHigh:=T#2S, Out=>VarBOOL1);</pre>
连续功能图 (CFC)	

当指令执行时，Out 所示波形输出如图所示。

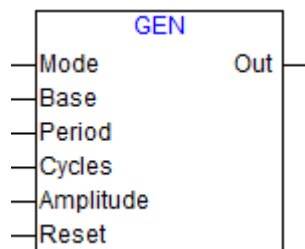


输出波形

3.2.5.2 GEN（典型周期信号发生器）

3.2.5.2.1 概述

产生典型周期函数算法（GEN）生成指定周期函数的类型、周期和振幅的信号。



GEN 功能块

3.2.5.2.2 参数说明

输入参数	变量类型	功能描述	参数值说明	初始值（默认）
Mode	GEN_MODE	指定要产生的信号类型	直接在 MODE 处输入 TRIANGLE、TRIANGLE_POS、SAWTOOTH_RISE、SAWTOOTH_FALL、RECTANGLE、SINUS、COSINUS，则产生对应的波形	TRIANGLE
		TRIANGLE	三角波	
		TRIANGLE_POS	零起点三角波	
		SAWTOOTH_RISE	上升锯齿波	
		SAWTOOTH_FALL	下降锯齿波	
		RECTANGLE	方波	
		SINUS	正弦波	
		COSINU	余弦波	
Base	BOOL	循环方式选择	当 BASE 为 TRUE 时，信号发生器与定义的循环周期有关。当 BASE 为 FALSE 时，信号发生器与特定的发生的个数有关	FALSE

输入参数	变量类型	功能描述	参数值说明	初始值（默认）
Period	TIME	定义循环周期（Base 为真时有效）		T#1S
Cycles	INT	定义循环周期（Base 为假时有效）		1000
Amplitude	INT	信号的振幅		0
Reset	BOOL	初始化	当 RESET=TRUE 时，信号发生器被重新设置为 0	FALSE

输出变量	变量类型	功能描述	初始值（默认）
Out	INT	波形信号输出值	0

中间变量	说明	变量类型	初始值（默认）	掉电保护
CET		DINT	0	FALSE
PER		DINT	0	FALSE
COUNTER	计数变量	INT	0	TRUE
CLOCK	功能块实例	TON		TRUE

i 该指令输出的第一个电平为高是固定的，无法进行高低选择。使能端断开，输出保持。

3.2.5.2.3 功能详细说明

■ 初始化功能

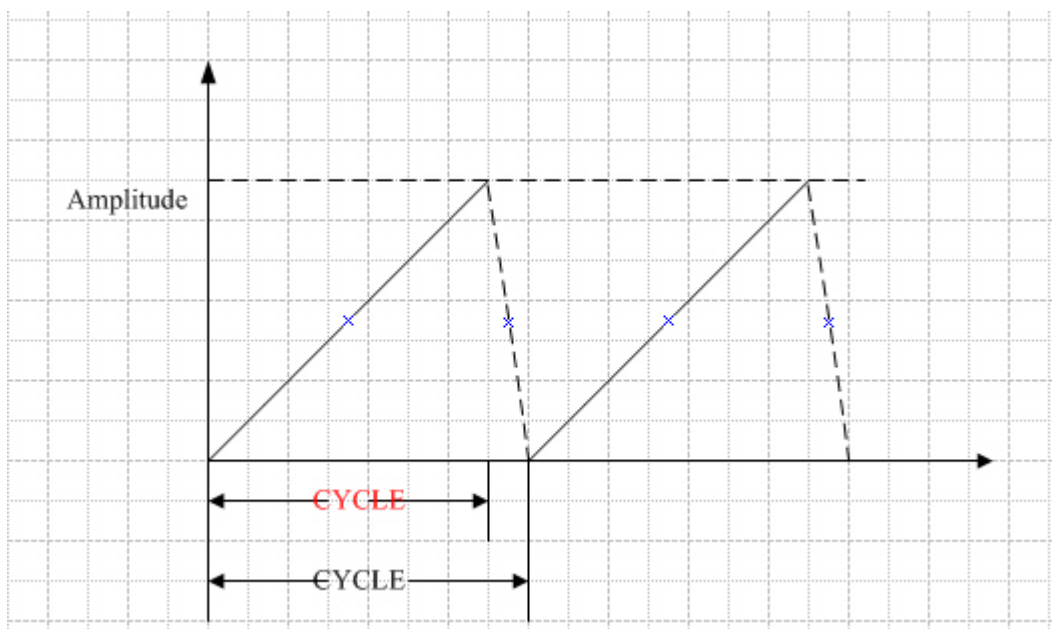
当 RESET=1 时，输出和相关计数变量清零。

■ 计算生成波形

当 BASE=0 时，按照周期 CYCLES 和赋值 AMPLITUDE 生成 MODE 类型的波形；当 BASE=1 时，只是周期不同，周期变为 PERIOD。

3.2.5.2.4 注意事项

- 当 BASE=0 且函数类型为 SAWTOOTH_RISE 或 SAWTOOTH_FALL 时，CYCLE 所示循环周期如下图黑色字体所示，而不是红色字体所示。（因为实际的信号产生中，不会有直上直下的波形，即一个时间点对应两个值的情况。）

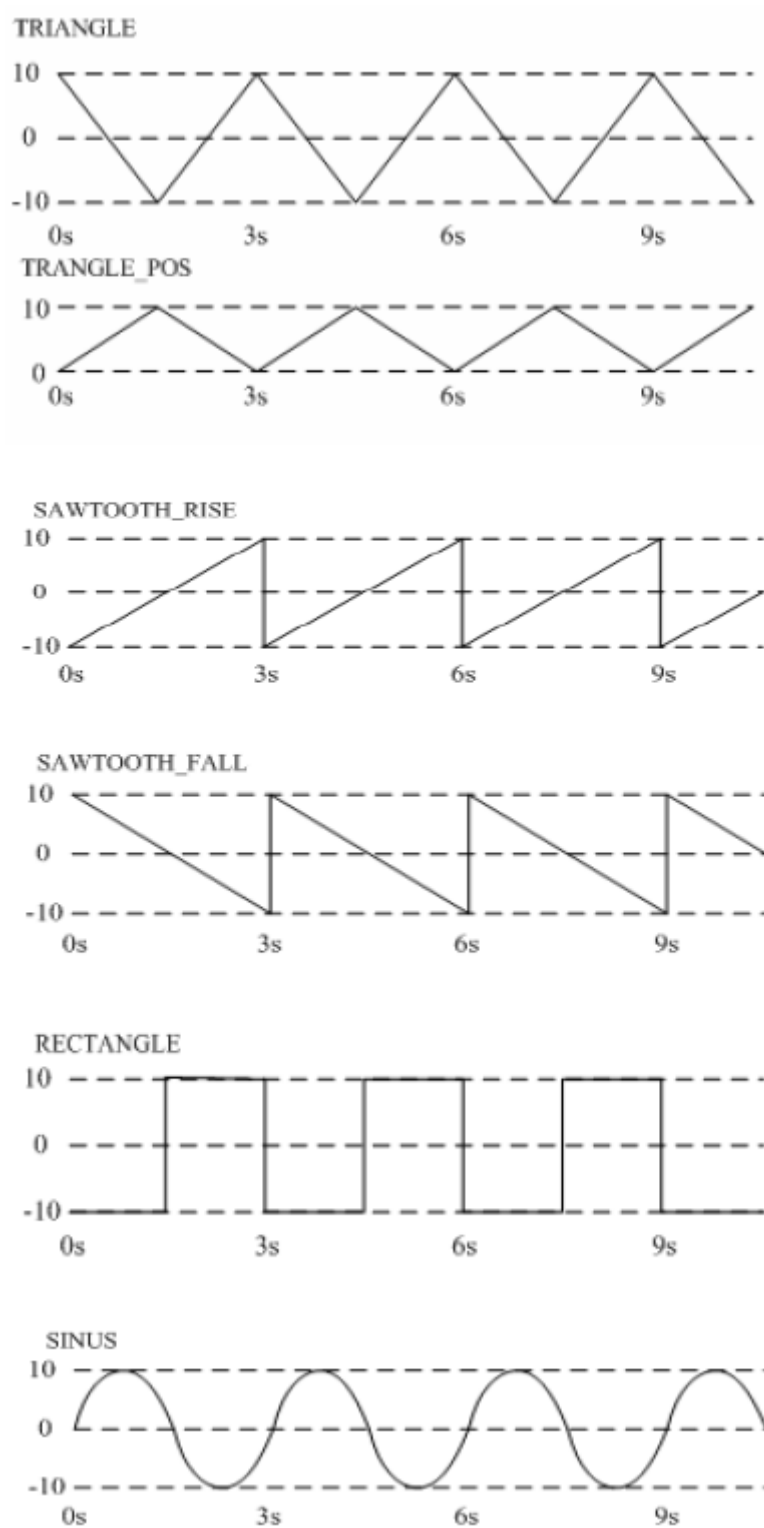


周期图

- 当 $BASE=1$ 且函数类型为 SINUS 或 COSINUS 时，如果 IEC 运算周期与波形循环周期 $PERIOD$ 之比在 0.1 之内，可以实现全波形的采集；

如果比值大于 0.1，有可能采集不到波峰波谷值，必然能采集到的最大值与最小值与比值大小相关。

根据 Mode 处输入不同，产生的波形如下图所示。□

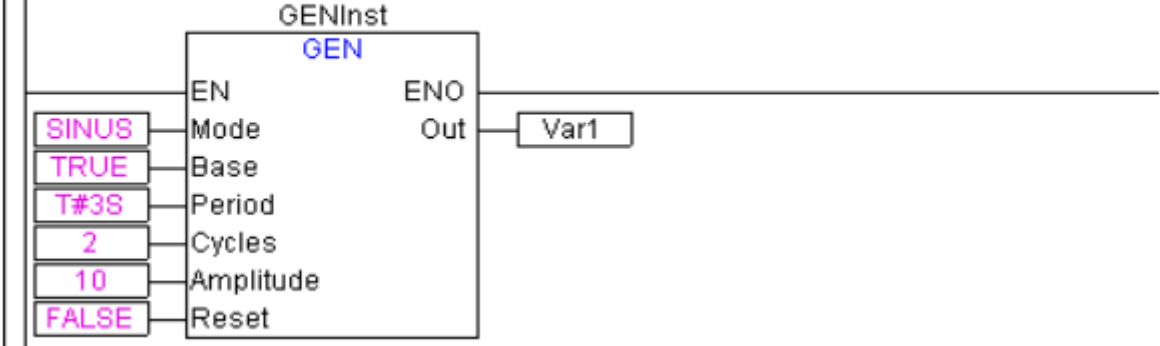
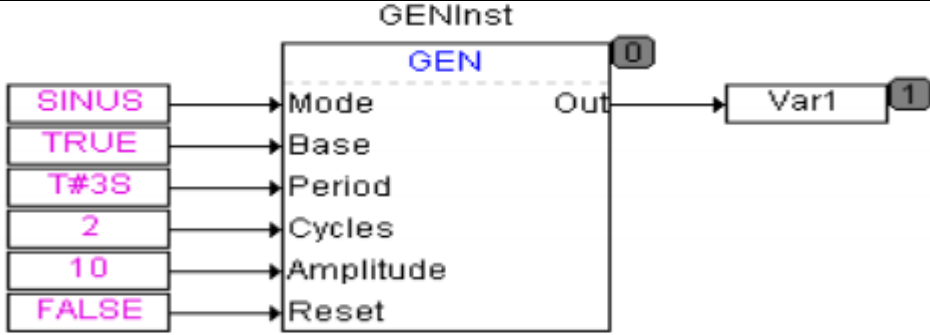


波形图

3.2.5.2.5 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	GENInst			GEN		FALSE
0002	Var1			INT	0	FALSE

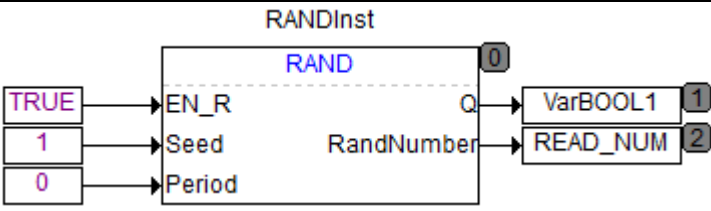
编程语言	程序
------	----

梯形图 (LD)	
结构化文本 (ST)	<pre> GENInst(Mode:=SINUS, Base:=TRUE, Period:=T#3S, Cycles:=2, Amplitude:=10, Reset:=FALSE, Out=>Var1); </pre>
连续功能图 (CFC)	

3.2.5.3 RAND (随机数发生器)

3.2.5.3.1 功能

产生随机数。

变量定义	
结构化文本 (ST)	<pre> RANDInst(EN_R:=TRUE, Seed:=1, Period:=0, Q=>VarBOOL1, RandNumber=>READ_NUM); </pre>
连续功能图 (CFC)	

程序说明：

EN_R 置位时，产生随机数，每个周期产生 1 个随机数。

EN_R 复位时，停止产生随机数，RandNumber 保持最后一次产生的随机数不变。

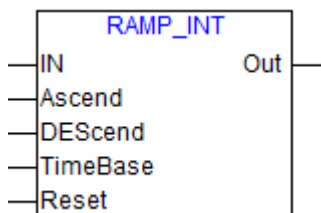
3.2.6 模拟量处理指令

3.2.6.1 RAMP_INT（整型限速）

3.2.6.1.1 功能

限制整型输入函数的升降速度。

输出 OUT 是输入 IN 经上升和下降限制整定后的函数值。当 TIMEBASE 设置为 t#0s，ASCEND 和 DESCEND 与时间间隔无关；当 TIMEBASE 设置为 t#0s 之外的值时，ASCEND 和 DESCEND 是时间间隔 TIMEBASE 时间长短的限制值。



RAMP_INT 功能块

3.2.6.1.2 参数说明

输入参数	数据类型	功能描述	初始值（默认）	掉电保护
IN	INT	被操作变量	0	FALSE

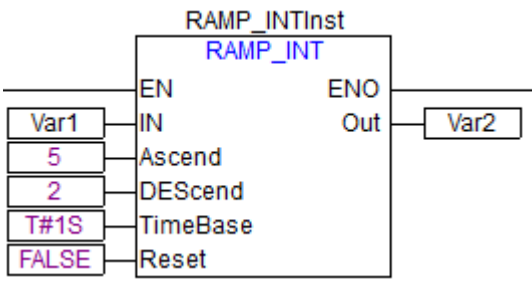
输入参数	数据类型	功能描述	初始值（默认）	掉电保护
Ascend	INT	时间间隔内最大上升值	0	TRUE
DEScend	INT	时间间隔内最大下降值	0	TRUE
TimeBase	TIME	时间间隔	T#0S	TRUE
Reset	BOOL	重启信号	FALSE	TRUE

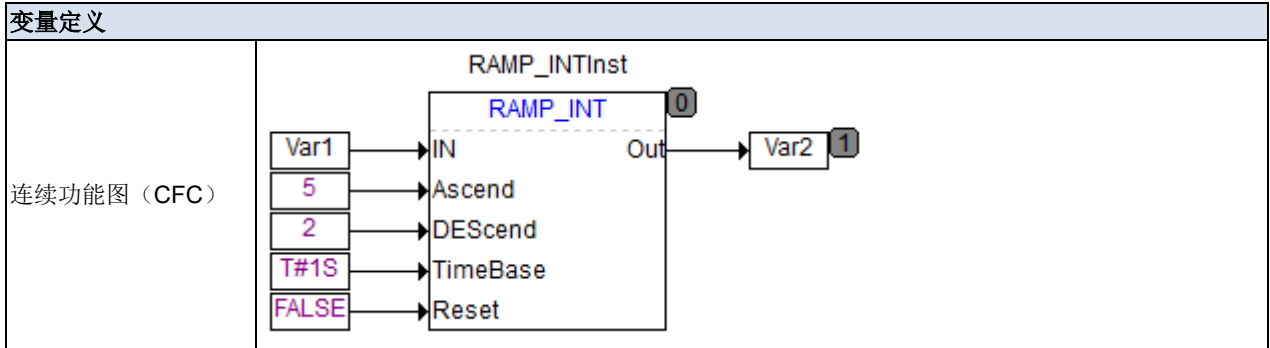
输出参数	数据类型	功能描述	初始值（默认）	掉电保护
Out	INT	经上升和下降限制后的值	0	TRUE

中间变量	数据类型	功能描述	初始值（默认）	掉电保护
DIFF	DINT		0	FALSE
OLD_IN	INT		0	TRUE
TB	DWORD		0	FALSE
CET	DWORD		0	FALSE
CH	INT		0	FALSE
CLOCK	TON	功能块实例		TRUE

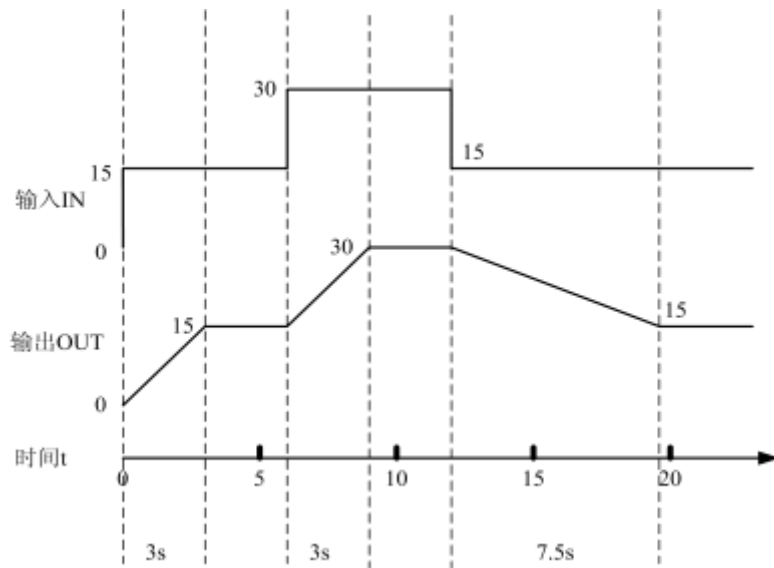
3.2.6.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	RAMP_INTInst			RAMP...		FALSE
0002	Var1			INT	0	FALSE
0003	Var2			INT	0	FALSE

编程语言	程序
梯形图（LD）	
结构化文本（ST）	<pre> RAMP_INTInst(IN:=Var1, Ascend:=5, DEScend:=2, TimeBase:=T#1S, Reset:=FALSE, OUT=>Var2); </pre>



上例中，当指令执行时，根据输入值 IN 的变化，对应的输出值如图所示。



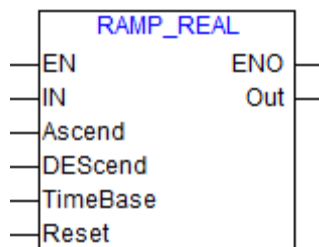
IN 与 OUT 对应关系

3.2.6.2 RAMP_REAL (实型限速)

3.2.6.2.1 功能

RAMP_REAL 指令可以限制实型输入函数的升降速度。

输出 OUT 是输入 IN 经上升和下降限制整定后的函数值。当 TIMEBASE 设置为 t#0s，ASCEND 和 DESCEND 与时间间隔无关；当 TIMEBASE 设置为 t#0s 之外的值时，ASCEND 和 DESCEND 是时间间隔 TIMEBASE 时间长短的限制值。



RAMP_REAL 功能块

3.2.6.2.2 参数说明

输入参数	数据类型	功能描述	初始值（默认）	掉电保护
IN	REAL	被操作变量	0	FALSE
Ascend	REAL	时间间隔内最大上升值	0	TRUE
DEScend	REAL	时间间隔内最大下降值	0	TRUE
TimeBase	TIME	时间间隔	T#0S	TRUE
Reset	BOOL	重启信号	FALSE	TRUE

输出参数	数据类型	功能描述	初始值（默认）	掉电保护
Out	REAL	经上升和下降限制后的值	0	TRUE

中间变量	数据类型	功能描述	初始值（默认）	掉电保护
DIFF	REAL		0	FALSE
OLD_IN	REAL		0	TRUE
TB	DWORD		0	FALSE
CET	DWORD		0	FALSE
CLOCK	TON	功能块实例		TRUE

指令使用举例请参见 [RAMP_INT（整型限速）](#)。

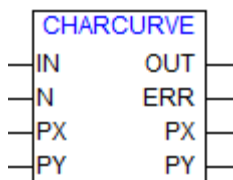
3.2.6.3 CHARCURVE（特征曲线）

3.2.6.3.1 功能

IN 是 DINT 类型，输入要操作的值。N 是字节类型，决定了特征曲线的点数且 $2 \leq N \leq 11$ 。在 DINT 类型数组 PX[0..10]和 PY[0..10]中产生一个特征行。DINT 型输出 OUT 为操作后的值。字节型变量 ERR 输出错误信息。

数组中的点 PX[0]..PX[N-1]必须根据从小到大存储，否则 ERR 的值为 1。如果 IN 不在 PX[0]和 PX[N-1]之间，ERR=2，输出包含相应的限制值 PY[0]或 PY[N-1]。

如果 N 超出了 2 和 11 之间的允许值，ERR=4。



CHARCURVE 指令示意图

3.2.6.3.2 参数说明

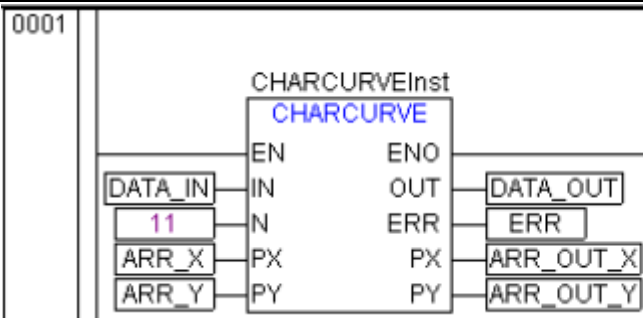
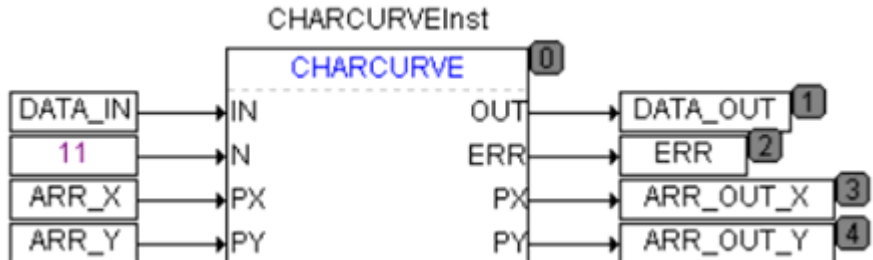
输入参数	数据类型	功能描述	初始值	掉电保护
IN	DINT	被操作变量	0	FALSE
N	BYTE	特征曲线的点数	0	TRUE

输出参数	数据类型	功能描述	初始值	掉电保护
OUT	DINT	操作后的值	0	TRUE
ERR	BYTE	错误信息	0	FALSE

其它参数	数据类型	功能描述	初始值	Retain
PX	ARRAY[0..10] OF DINT	特征曲线的 X 轴点列	0	TRUE
PY	ARRAY[0..10] OF DINT	特征曲线的 Y 轴点列	0	TRUE

3.2.6.3.3 指令使用举例

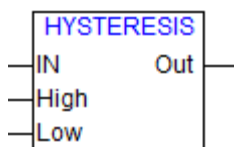
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	CHARCURVEInst			CHARCURVE		FALSE
0002	ERR			BYTE	0	FALSE
0003	DATA_IN			INT	0	FALSE
0004	DATA_OUT			INT	0	FALSE
0005	ARR_X			ARRAY[0..10] OF DINT		FALSE
0006	ARR_Y			ARRAY[0..10] OF DINT		FALSE
0007	ARR_OUT_X			ARRAY[0..10] OF DINT		FALSE
0008	ARR_OUT_Y			ARRAY[0..10] OF DINT		FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> CHARCURVEInst(IN:=DATA_IN, N:=11, PX:=ARR_X, PY:=ARR_Y, OUT=>DATA_OUT, ERR=>ERR); ARR_OUT_X:= CHARCURVEInst.PX; ARR_OUT_Y:= CHARCURVEInst.PY; </pre>
连续功能图 (CFC)	

3.2.6.4 HYSTERESIS (滞后)

3.2.6.4.1 概述

当输入值低于低限值时，输出为 TRUE；当输入值高于高限值时，输出为 FALSE；当输入值在高低限制之间时，输出保持。



HYSTERESIS 功能块

3.2.6.4.2 参数说明

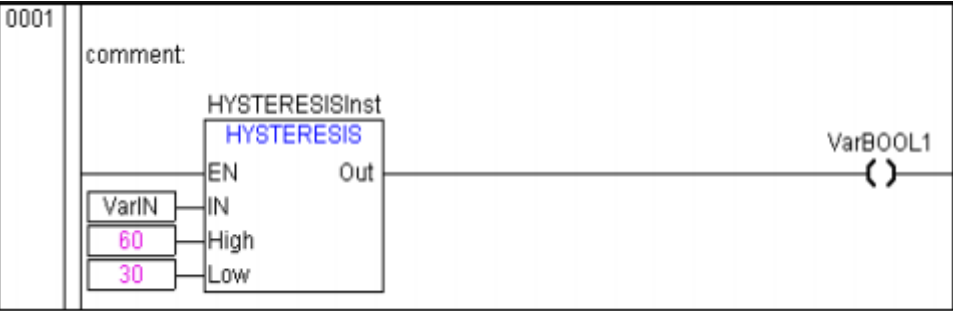
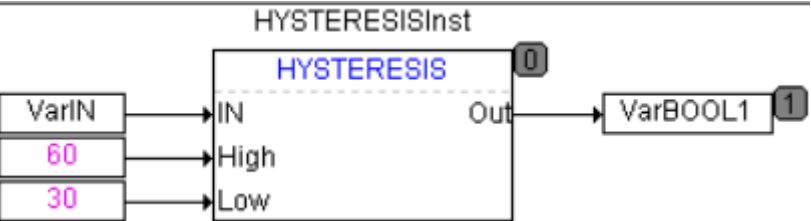
输入参数	说明	变量类型	初始值 (默认)	掉电保护
IN	输入值	INT	0	FALSE

输入参数	说明	变量类型	初始值（默认）	掉电保护
Low	低限值	INT	0	TRUE
High	高限值	INT	0	TRUE

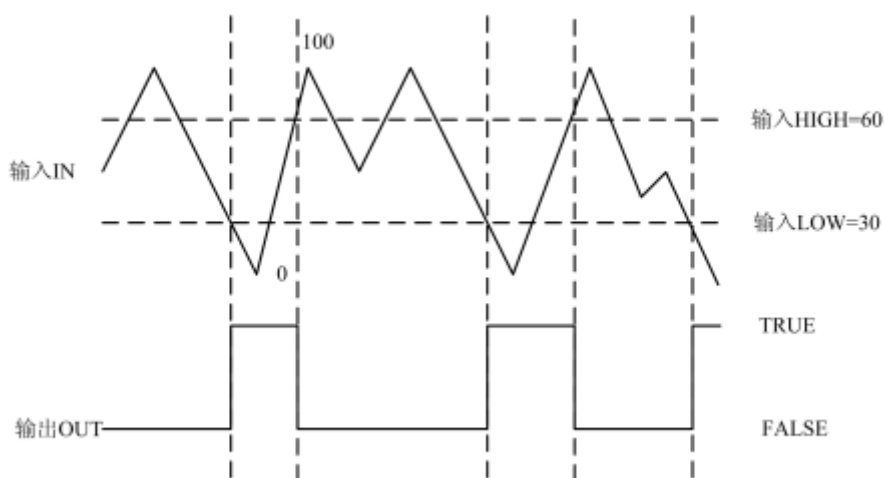
输出参数	说明	变量类型	初始值（默认）	掉电保护
Out	输出，低下限值后为 TRUE，直到上限值后恢复为 FALSE	BOOL	FALSE	TRUE

3.2.6.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HYSTERESISInst			HYST...		FALSE
0002	VarIN			INT	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE

编程语言	程序
梯形图（LD）	
结构化文本（ST）	<pre>HYSTERESISInst(IN:=VarIN, High:=60, Low:=30, Out=>VarBOOL1);</pre>
连续功能图（CFC）	

上例中，当指令执行时，根据输入值的变化，对应的输出值如图所示。

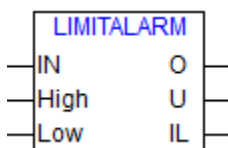


输入与输出变化关系

3.2.6.5 LIMITALARM（上下限报警）

3.2.6.5.1 概述

当输入值低于低限值时，输出 U 为 TRUE；当输入值高于高限值时，输出 O 为 TRUE；当输入值在高低限制之间时，输出 IL 为 TRUE。



LIMITALARM

3.2.6.5.2 参数说明

输入参数	说明	变量类型	初始值（默认）	掉电保护
IN	输入值	INT	0	FALSE
Low	低限值	INT	0	TRUE
High	高限值	INT	0	TRUE

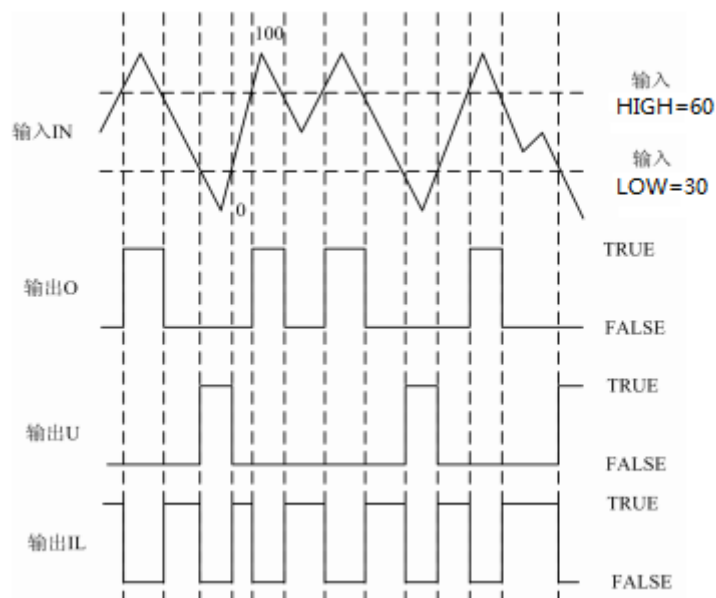
输出参数	说明	变量类型	初始值（默认）	掉电保护
O	输入大于高限值	BOOL	FALSE	FALSE
U	输入小于低限值	BOOL	FALSE	FALSE
IL	输入在高低限之间	BOOL	FALSE	FALSE

3.2.6.5.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0002	VarIN			INT	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	VarBOOL2			BOOL	FALSE	FALSE
0005	VarBOOL3			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	0001 comment:
结构化文本 (ST)	<pre> LIMITALARMInst(IN:=VarIN High:=60, Low:=30, O=>VarBOOL1, U=>VarBOOL2, IL=>VarBOOL3); </pre>
连续功能图 (CFC)	

上例中，当指令执行时，根据输入值的变化，对应的输出值如图所示。

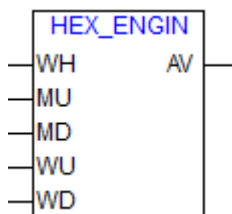


输入与输出变化关系

3.2.6.6 HEX_ENGIN（16 进制数转换为工程量数据）

3.2.6.6.1 概述

将输入的码值 WH 转换成对应的工程量 AV 值。



HEX_ENGIN 功能块

3.2.6.6.2 参数说明

输入参数	说明	变量类型	初始值（默认）	掉电保护
WH	输入码值	DINT	0	FALSE
MU	工程量上限值	REAL	0	TRUE
MD	工程量下限值	REAL	0	TRUE
WU	PLC 的模拟量输入码值上限	DINT	0	TRUE
WD	PLC 的模拟量输入码值下限	DINT	0	TRUE

输出参数	说明	变量类型	初始值（默认）	掉电保护
AV	转换后的工程量	REAL	0	FALSE

3.2.6.6.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HEX_ENGINInst			HEX_...		FALSE
0002	Varreal1			REAL	0	FALSE
编程语言	程序					
梯形图 (LD)	0001 comment: HEX_ENGINInst HEX_ENGIN EN ENO AV %IW2 1.6 0 65535 0 WH MU MD WU WD Varreal1					
结构化文本 (ST)	<pre> HEX_ENGINInst(WH:=%IW2, MU:=1.6, MD:=0, WU:=65535, WD:=0, AV=>Varreal1); </pre>					
连续功能图 (CFC)	HEX_ENGINInst HEX_ENGIN 0 AV %IW2 1.6 0 65535 0 WH MU MD WU WD Varreal1 1					

程序说明：

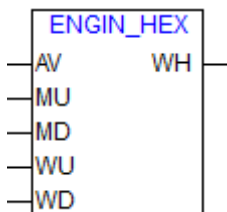
该指令一般用于模拟量输入数据处理；

EN 置位时，进行转换，输入数据为%IW2，工程量上限是 1.6MPa，工程量下限是 0MPa，对应模块码值范围为 0-65535。假设当%IW2 的值为 32767 时，对应输出值为 0.799。

3.2.6.7 ENGIN_HEX（工程量数据转换为 16 进制 WORD 型数据）

3.2.6.7.1 概述

将输入的工程量 AV 转换成相应的码值 WH。



ENGIN_HEX 功能块

3.2.6.7.2 参数说明

3.2.6.7.2.1 输入参数

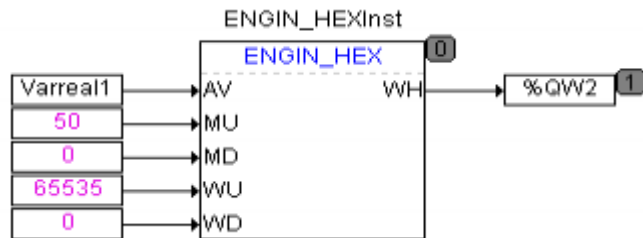
输入参数	说明	变量类型	初始值（默认）	掉电保护
AV	输入工程量	REAL	0	FALSE
MU	工程量上限值	REAL	0	TRUE
MD	工程量下限值	REAL	0	TRUE
WU	PLC 的模拟量输入码值上限	WORD	0	TRUE
WD	PLC 的模拟量输入码值下限	WORD	0	TRUE

3.2.6.7.2.2 输出参数

输出参数	说明	变量类型	初始值（默认）	掉电保护
WH	转换后的码值	WORD	0	FALSE

3.2.6.7.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	ENGIN_HEXInst			ENGI... ▾		FALSE ▾
0002	Varreal1			REAL ▾	0	FALSE ▾
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> ENG_HEXInst(AV:=Varreal1, MU:=50 MD:=0, WU:=65535, WD:=0, WH=>%QW2); </pre>
连续功能图 (CFC)	

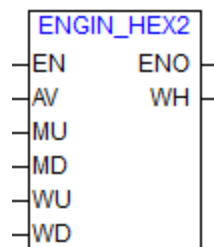
程序说明：

该指令一般用于模拟量输出数据处理；EN 置位时，进行转换，工程量实际数据为 Varreal1，工程量上限是 50Hz，工程量下限是 0Hz，对应模块范围为 0~65535，当 Varreal1 的值为 25Hz 时，对应输出值为 32768。将 32768 赋值给 %QW2，则该通道模拟量输出值为 25Hz。

3.2.6.8 ENG_HEX2（工程量数据转换为 16 进制 INT 型数据）

3.2.6.8.1 概述

将输入的工程量 AV 转换成相应的码值 WH。



ENG_HEX2 功能块

3.2.6.8.2 参数说明

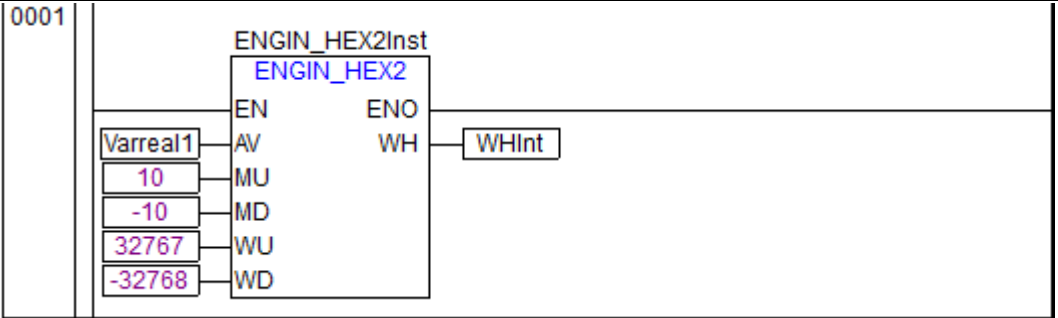
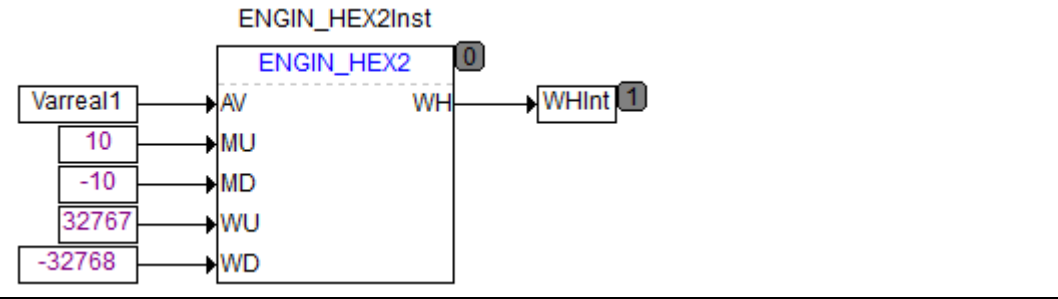
3.2.6.8.2.1 输入参数

输入参数	说明	变量类型	初始值（默认）	掉电保护
AV	输入工程量	REAL	0	FALSE
MU	工程量上限值	REAL	0	TRUE
MD	工程量下限值	REAL	0	TRUE
WU	PLC 的模拟量输入码值上限	INT	0	TRUE
WD	PLC 的模拟量输入码值下限	INT	0	TRUE

3.2.6.8.2.2 输出参数

输出参数	说明	变量类型	初始值（默认）	掉电保护
WH	转换后的码值	INT	0	FALSE

3.2.6.8.3 指令使用举例

变量定义							
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护	SOE使能
0001	ENGIN_HEX2Inst			ENGIN_HEX2		FALSE	☐
0002	Varreal1			REAL	0	FALSE	☐
0003	WHInt	%QW4		INT	0	FALSE	☐
编程语言		程序					
梯形图（LD）							
结构化文本（ST）		<pre>ENGIN_HEX2Inst(AV:=Varreal1, MU:=10, MD:=10, WU:=32767, WD:=-32768, WH=>WHInt);</pre>					
连续功能图（CFC）							

程序说明：

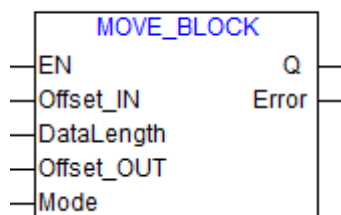
该指令一般用于模拟量输出数据处理；EN 置位时，进行转换，工程量实际数据为 Varreal1，工程量上限是 10V，工程量下限是-10V，对应模块范围为 -32768~32767，当 Varreal1 的值为-5V 时，对应输出值为 -16384。将-16384 赋值给 WHInt(%QW4)，则该通道模拟量输出值为-5V。

3.2.7 数据块传送指令

3.2.7.1 MOVE_BLOCK（数据块传送）

3.2.7.1.1 功能

将 M 区数据从一个位置批量传送到 M 区的另一个位置。支持按字节（BYTE）、字（WORD）或双字（DWORD）批量传送。一次传送数据个数的最大值为 255 个双字，传送数据的长度由传送方式和传送数据的长度共同决定。



MOVE_BLOCK 功能块

3.2.7.1.2 参数说明

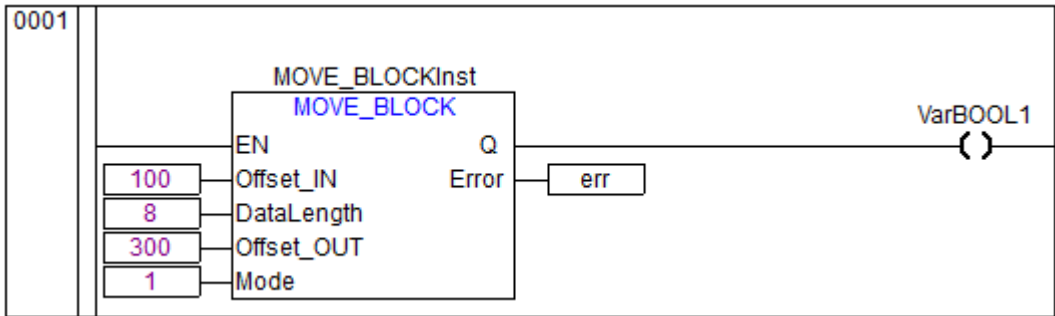
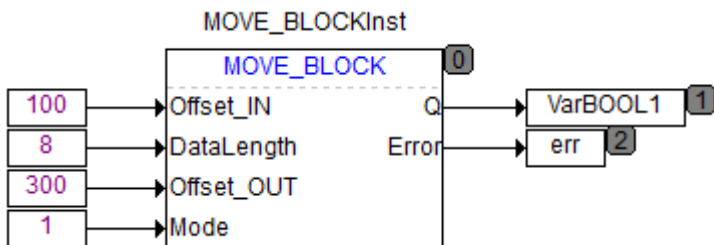
输入参数	数据类型	功能描述	参数值说明	默认值
EN	BOOL	使能	0:无效 1: 使能	
Offset_In	DWORD	数据源区的偏移字地址	数据区的偏移字地址	0
DataLength	BYTE	传送数据的长度	0~255	0
Offset_Out	DWORD	数据目的区的偏移字地址	数据区的偏移字地址	0
Mode	BYTE	数据传送的方式	=0: 表示字节传送； =1: 表示字传送； =2: 表示双字传送；	0

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	使能输出	当数据未能传送/传送未正确时，Q=FALSE。 当 数据传送正确时，Q=TRUE。
Error	BYTE	错误信息	=0: 数据传送正确

输出参数	数据类型	功能描述	参数值说明
			=1: 数据源数据区偏移地址 (Offset_IN) 有误 =2: 数据目的数据区偏移地址 (Offset_OUT) 有误 =3: 数据传送方式 (Mode) 有误

3.2.7.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MOVE_BLOCKInst			MOVE...		FALSE
0002	err			BYTE	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>MOVE_BLOCKInst(EN:=TRUE, Offset_IN:=100, DataLength:=8, Offset_OUT:=300, Mode:=1, Q=>VarBOOL1 Error=> err);</pre>
连续功能图 (CFC)	

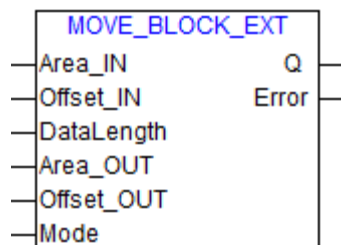
程序说明:

EN 置位时, 将 Offset_IN 所指向的 N 个字节的数据传送给 Offset_OUT 所指的地址。

3.2.7.2 MOVE_BLOCK_EXT（数据块传送扩展指令）

3.2.7.2.1 功能

将 M/I/Q/S 区的数据批量传送到 M/I/Q 区。支持按字节（BYTE）、字（WORD）或双字（DWORD）传送。一次传送数据个数的最大值为 255 个双字。



MOVE_BLOCK_EXT 功能块

3.2.7.2.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
Area_IN	BYTE	源数据区选择	数据区 =0: M 区 =1: I 区 =2: Q 区 =3: S 区	0
Offset_IN	DWORD	源数据区偏移地址	数据区的偏移地址	0
DataLength	BYTE	传送方式对应的传送个数。传送数据长度由 Mode 和 DataLength 共同决定	0~255	0
Area_OUT	BYTE	目的数据区选择	数据区 0: M 区 1: I 区 2: Q 区	
Offset_OUT	DWORD	目的数据区的偏移地址	数据区的偏移地址	0
Mode	BYTE	数据传送的方式	=0: 表示字节传送 =1: 表示字传送 =2: 表示双字传送	0

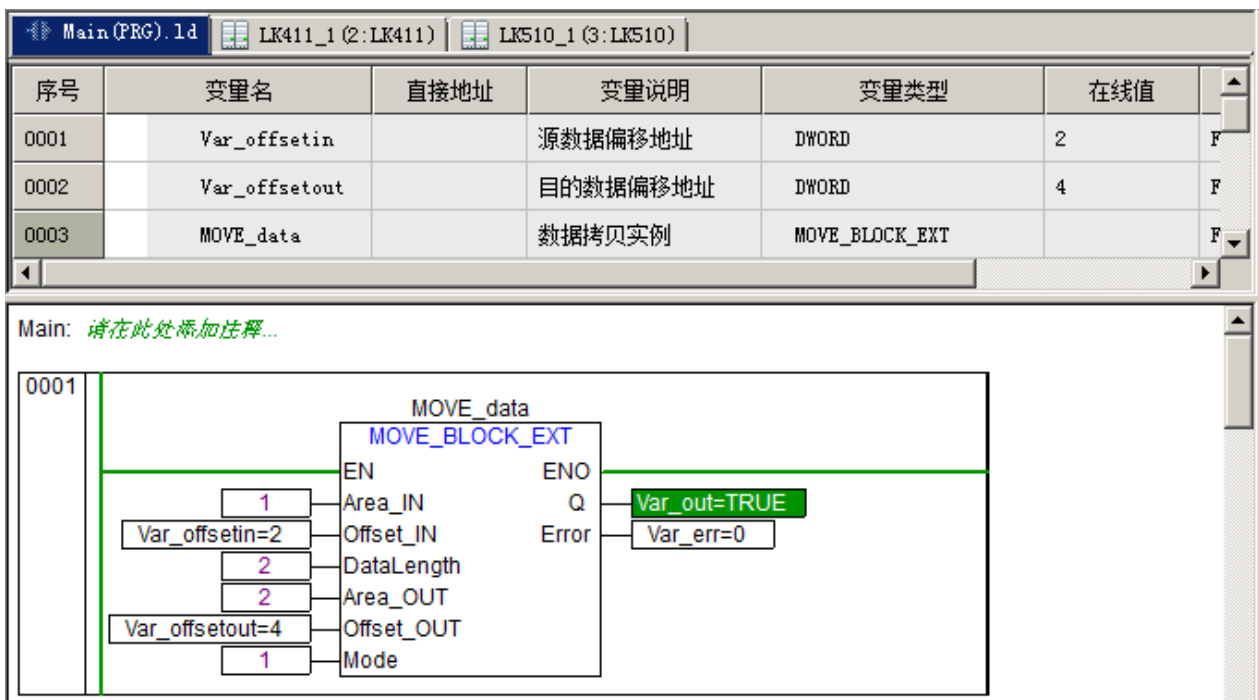
输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	数据传送结果	Q=FALSE, 数据未能传送/传送错误 Q=TRUE, 数据传送正确	FALSE
Error	BYTE	错误信息	0: 数据传送正确	0

输出参数	数据类型	功能描述	参数值说明	默认值
			1: 源数据区选择错误 2: 源数据区偏移地址错误 3: 目的数据区选择错误 4: 目的数据区偏移地址错误 5: 数据传送方式 (Mode) 有误	

3.2.7.2.3 指令使用举例

将%IW2 开始的 4 字节数据拷贝到%QW4 开始的数据区。

设置数据区，选择按字传送，设置源数据偏移地址 Var_offsetin 初始值为 2，目的数据偏移地址 Var_offsetout 初始值为 4，组态示例如图所示。



The screenshot displays the SIMATIC Manager interface for configuring a data transfer instruction. The top section shows a table of variables and their initial values. The bottom section shows the graphical representation of the MOVE_BLOCK_EXT instruction, with its inputs and outputs connected to the appropriate variables and constants.

序号	变量名	直接地址	变量说明	变量类型	在线值
0001	Var_offsetin		源数据偏移地址	DWORD	2
0002	Var_offsetout		目的数据偏移地址	DWORD	4
0003	MOVE_data		数据拷贝实例	MOVE_BLOCK_EXT	

Main: 请在此处添加注释...

0001

MOVE_data
MOVE_BLOCK_EXT

EN 1
Area_IN Var_offsetin=2
Offset_IN 2
DataLength 2
Area_OUT Var_offsetout=4
Offset_OUT 1
Mode 1

ENO Q
Error Var_err=0

Var_out=TRUE

数据拷贝组态示例

运行程序，指令将%IW2~%IW4 数据拷贝到%QW4~%QW6，如下图所示。

LK411_1 (2:LK411)

LK510_1 (3:LK510)

设备配置 | 通道 | 信息

0_8 Channels AI

通道号	通道名称	通道类型	在线值	通道地址	通道说明
1	DPIO_2_1_2_1	WORD	0	%IW0	
2	DPIO_2_1_2_2	WORD	5	%IW2	源数据
3	DPIO_2_1_2_3	WORD	4	%IW4	源数据
4	DPIO_2_1_2_4	WORD	0	%IW6	
5	DPIO_2_1_2_5	WORD	0	%IW8	
6	DPIO_2_1_2_6	WORD	0	%IW10	



LK411_1 (2:LK411)

LK510_1 (3:LK510)

设备配置

通道

信息

0_4 Channels AO

通道号	通道名称	通道类型	在线值	通道地址	通道说明
1	DPIO_2_1_3_1	WORD	0	%QW0	
2	DPIO_2_1_3_2	WORD	0	%QW2	
3	DPIO_2_1_3_3	WORD	5	%QW4	目的数据
4	DPIO_2_1_3_4	WORD	4	%QW6	目的数据

指令运行结果

3.2.8 设置地址内容指令

3.2.8.1 VALSET_BYTE

3.2.8.1.1 功能描述

将 Address 指定地址的内容修改为 Value 所对应的当前值。



VALSET_BYTE 功能块

①使用 VALSET_BYTE 指令具有风险性，在使用过程中，请严格按照指令要求操作。

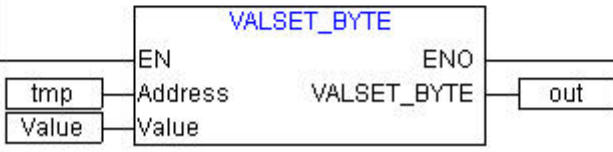
3.2.8.1.2 指令描述

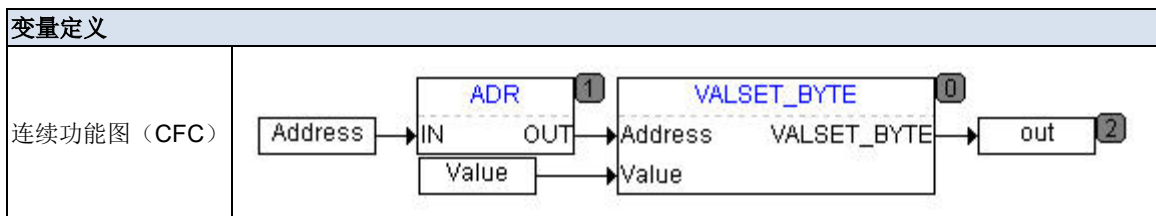
输入参数	数据类型	功能描述	参数值说明	默认值
Address	POINTER TO BYTE	目标地址	指向定义 BYTE 型数据的首地址； 操作数据区范围：M、N、Q、R、特殊寄存器 S 区 %SB270 至 %SB299	\
Value	BYTE	设置值	设置值	\

输出参数	数据类型	功能描述	参数值说明	备注
VALSET_BYTE	BYTE	错误类型	=0: 输出结果正确 =1: 输入参数有误 =2: 目标段地址为 I 区 =3: 目标段地址为特殊寄存器 S 区，目标段地址不在 %SB270 至 %SB299 范围内 =9: 拷贝数据长度错误	\

3.2.8.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Address			BYTE	0	FALSE
0002	Value			BYTE	0	FALSE
0003	out			BYTE	0	FALSE
0004	tmp			POINTER TO BYTE	0	FALSE

编程语言		
梯形图 (LD)	0001	
	0002	
结构化文本 (ST)		out := VALSET_BYTE (ADR(Address),Value);



①使用 VALSET_BYTE 指令时，如果 Address 参数指向的变量占用空间大于 1 个 BYTE 时，该指令仅能修改最低 1 个 BYTE 的内容。

3.2.8.2 VALSET_WORD

3.2.8.2.1 功能描述

将 Address 指定地址起始两字节的内容修改为 Value 所对应的当前值。



VALSET_WORD 功能块



- 使用 VALSET_WORD 指令具有风险性，在使用过程中，请严格按照指令要求操作。
- 使用指令时，如果 Address 参数指向的变量占用空间大于该指令规定数据类型长度时，该指令会意外修改 Address 指向地址之外的内容，可能会造成严重后果。

3.2.8.2.2 指令描述

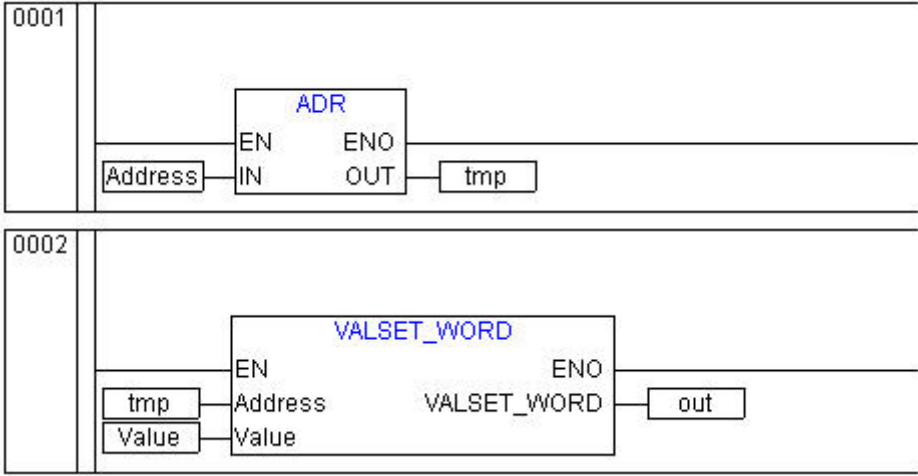
输入参数	数据类型	功能描述	参数值说明	默认值
Address	POINTER TO WORD	目标地址	指向定义 WORD 型数据的首地址； 操作数据区范围：M、Q、R、特殊寄存器 S 区 %SB270 至 %SB299	\
Value	WORD	设置值	设置值	\

输出参数	数据类型	功能描述	参数值说明	备注
VALSET_WORD	WORD	错误类型	=0: 输出结果正确 =1: 输入参数有误 =2: 目标段地址为 I 区 =3: 目标段地址为特殊寄存器 S 区，目标段地址不在 %SB270 至 %SB299 范围内 =4: 目标段地址为 N 区	\

输出参数	数据类型	功能描述	参数值说明	备注
			=5: 目标段地址为 M 区, 目标段地址+拷贝数据长度超出 M 区范围 =6: 目标段地址为 Q 区, 目标段地址+拷贝数据长度超出 Q 区范围 =8: 目标段地址为 R 区, 目标段地址+拷贝数据长度超出 R 区范围 =9: 拷贝数据长度错误	

3.2.8.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Address	%MB100		WORD	0	FALSE
0002	Value			WORD	0	FALSE
0003	tmp			POINTER TO WORD	0	FALSE
0004	out			BYTE	0	FALSE

编程语言	
梯形图 (LD)	
结构化文本 (ST)	out := VALSET_WORD (ADR(Address),Value);
连续功能图 (CFC)	

①使用 VALSET_WORD 指令时, 如果 Address 参数指向的变量占用空间大于 1 个 WORD 时, 该指令仅能修改最低 1 个字的内容。

3.2.8.3 VALSET_DWORD

3.2.8.3.1 功能

将 Address 指定地址起始四字节的内容修改为 Value 所对应的当前值。



VALSET_DWORD 功能块



- 使用 VALSET_DWORD 指令具有风险性，在使用过程中，请严格按照指令要求操作。
- 使用指令时，如果 Address 参数指向的变量占用空间大于该指令规定数据类型长度时，该指令会意外修改 Address 指向地址之外的内容，可能会造成严重后果。

3.2.8.3.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
Address	POINTER TO DWORD	目标地址	指向定义 DWORD 型数据的首地址； 操作数据区范围：M、Q、R、特殊寄存器 S 区 %SB270 至 %SB299	\
Value	DWORD	设置值	设置值	\

输出参数	数据类型	功能描述	参数值说明	备注
VALSET_DWORD	DWORD	错误类型	=0: 输出结果正确 =1: 输入参数有误 =2: 目标段地址为 I 区 =3: 目标段地址为特殊寄存器 S 区，目标段地址不在 %SB270 至 %SB299 范围内 =4: 目标段地址为 N 区 =5: 目标段地址为 M 区，目标段地址+拷贝数据长度超出 M 区范围 =6: 目标段地址为 Q 区，目标段地址+拷贝数据长度超出 Q 区范围 =7: 目标段地址为特殊寄存器 S 区，目标段地址+拷贝数据长度超出 %SB270 至 %SB299 =8: 目标段地址为 R 区，目标段地址+拷贝数据长度超出 R 区范围 =9: 拷贝数据长度错误	\

3.2.8.3.3 指令使用举例

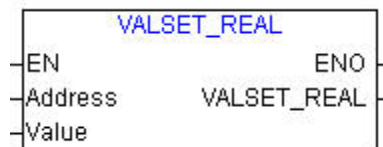
变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Address	%MB100		DWORD	0	FALSE
0002	Value			DWORD	0	FALSE
0003	tmp			POINTER TO DWORD	0	FALSE
0004	out			BYTE	0	FALSE

编程语言	
梯形图 (LD)	0001 ADR EN ENO IN OUT Address tmp 0002 VALSET_DWORD EN ENO Address Value tmp Value

3.2.8.4 VALSET_REAL

3.2.8.4.1 功能描述

将 Address 指定地址起始四字节的内容修改为 Value 所对应的当前值。



VALSET_REAL 功能块



- 使用 VALSET_REAL 指令具有风险性，在使用过程中，请严格按照指令要求操作。

- 使用指令时，如果 **Address** 参数指向的变量占用空间大于该指令规定数据类型长度时，该指令会意外修改 **Address** 指向地址之外的内容，可能会造成严重后果。

3.2.8.4.2 指令描述

输入参数	数据类型	功能描述	参数值说明	默认值
Address	POINTER TO REAL	目标地址	指向定义 REAL 型数据的首地址； 操作数据区范围：M、Q、R、特殊寄存器 S 区 %SB270 至 %SB299	\
Value	REAL	设置值	设置值	\

输出参数	数据类型	功能描述	参数值说明	备注
VALSET_REAL	REAL	错误类型	=0: 输出结果正确 =1: 输入参数有误 =2: 目标段地址为 I 区 =3: 目标段地址为特殊寄存器 S 区，目标段地址不在 %SB270 至 %SB299 范围内 =4: 目标段地址为 N 区 =5: 目标段地址为 M 区，目标段地址+拷贝数据长度超出 M 区范围 =6: 目标段地址为 Q 区，目标段地址+拷贝数据长度超出 Q 区范围 =7: 目标段地址为特殊寄存器 S 区，目标段地址+拷贝数据长度超出 %SB270 至 %SB299 =8: 目标段地址为 R 区，目标段地址+拷贝数据长度超出 R 区范围 =9: 拷贝数据长度错误	\

3.2.8.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Address	%MB100		REAL	0	FALSE
0002	Value			REAL	0	FALSE
0003	tmp			POINTER TO REAL	0	FALSE
0004	out			BYTE	0	FALSE
编程语言						

变量定义	
梯形图 (LD)	0001 
	0002 
结构化文本 (ST)	<pre>out := VALSET_REAL (ADR(Address),Value);</pre>
连续功能图 (CFC)	

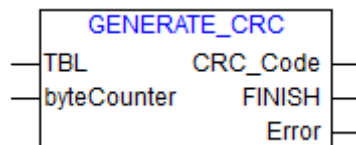
3.3 系统库

3.3.1 通讯指令

3.3.1.1 GENERATE_CRC (CRC 校验计算)

3.3.1.1.1 功能

产生 16 位的 CRC 校验值。



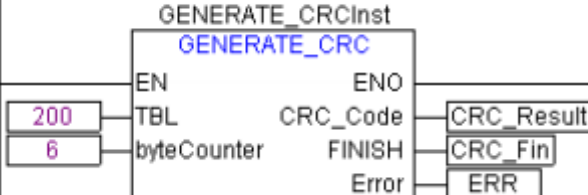
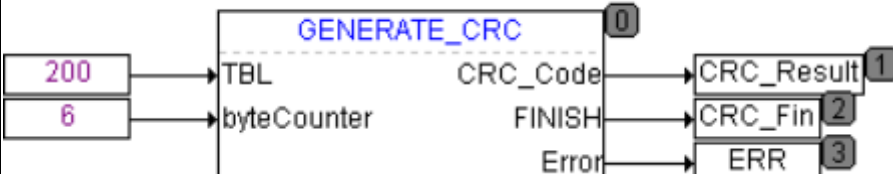
GENERATE_CRC 指令

3.3.1.1.2 参数说明

输入参数	数据类型	功能描述	参数值说明
TBL	WORD	需要校验数据的首字节地址	若设置为 200，则需要校验的数据保存在%MB200 开始的地址中
byteCounter	WORD	需要校验数据的字节数	0 为非法值

输出参数	数据类型	功能描述	参数值说明
CRC_Code	WORD	校验结果	16 位
FINISH	BOOL	是否完成校验	0: 未完成校验 1: 完成校验
Error	BYTE	错误信息	0: 正确 1: 需要校验数据的字节数 byteCounter=0 2: 需要校验的数据超出 M 区范围

3.3.1.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	GENERATE_CRCInst			GENE... ▾		FALSE ▾
0002	CRC_Result	%MB206		WORD ▾	0	FALSE ▾
0003	CRC_Fin			BOOL ▾	FALSE ▾	FALSE ▾
0004	ERR			BYTE ▾	0	FALSE ▾
编程语言		程序				
梯形图（LD）	0001 					
	GENERATE_CRCInst(TBL:=200, byteCounter:=6, CRC_Code=>CRC_Result, FINISH=>CRC_Fin, Error=>ERR);					
连续功能图（CFC）						

程序说明：

通过 CRC 校验计算指令，计算从%MB200 开始的 6 个字节的 CRC 校验码。之后，可根据实际通讯需求，将 CRC 校验码放在所要发送的其它数据后发送出去。

3.3.1.2 ComConfig（设置串口通讯参数）

3.3.1.2.1 功能描述

该功能目前仅支持在线配置各通道通讯参数。

3.3.1.2.2 参数描述

	数据项	数据类型	参数描述		初始值 (默认)	备注
			MC 平台	LE 平台		
输入端	EN_R	BOOL	上升沿使能	上升沿使能	0	
	Port	USINT	0: UART1, 目前支持 232 串口 1: UART2, 目前支持 422/485 串口 2: UART3, 目前支持 485 串口 3: UART4, 目前支持 485 串口	0: CPU 本体的圆形接口 1: CPU 本体端子通讯口 2: 扩展功能板通讯口 串口扩展模块的端口号请参考: 硬件配置->LE5400 串口配置参数 Port	0	
	ComType	USINT	通信类型 0: RS485 1: RS422 2: RS232	通信类型 0: RS485 1: RS232	0	
	Parity	USINT	校验方式 0: 无校验 1: 偶校验 2: 奇校验	校验方式 0: 无校验 1: 偶校验 2: 奇校验	0	
	DataWidth	USINT	数据宽度（支持 8 位） 8: 数据宽度 8 位	数据宽度: 8/7 位	8	
	Stop	USINT	停止位（支持 1、2） 1: 停止位为 1 2: 停止位为 2	停止位（支持 1、2） 1: 停止位为 1 2: 停止位为 2	1	
	BaudRate	UDINT	波特率（bps） 9600: 支持波特率 9600 19,200: 支持波特率 19,200 38,400: 支持波特率 38,400 57,600: 支持波特率 57,600 115,200: 支持波特率 115,200 以下波特率 COM1 口不支持， 但其余 COM 口支持： 500K: 波特率 500,000 1M: 波特率 1,000,000 1.125M: 波特率 1,125,000 1.25M: 波特率 1,250,000	波特率（bps），支持 9600/19200/38400/57600/115200	38,400	

	数据项	数据类型	参数描述		初始值 (默认)	备注
			MC 平台	LE 平台		
			1.5M: 波特率 1,500,000			
输出端	Q	BOOL	输出端 Q 端为 TRUE 只用来说明本次任务周期该条指令执行完毕	输出端 Q 端为 TRUE, 指令执行完毕	0	
	Err	USINT	错误码 0: 无错误 1: ComType 错误 2: Port 错误 4: Parity 错误 5: DataWidth 错误 6: Stop 错误: 7: BaudRate 错误	错误码 0: 无错误 1: Port 通信端口错误 2: ComType 错误 3: BaudRate 错误 4: DataWidth 错误 5: Parity 错误 6: Stop 错误 7: 控制器内部发生错误	0	

i LK220 不支持该指令。

3.3.1.3 ComRecv (接收串口数据)

3.3.1.3.1 功能描述

串口接收数据。

3.3.1.3.2 参数描述

	数据项	数据类型	参数描述	初始值 (默认)
输入端	En_R	BOOL	高电平有效	0
	DataLenth	UDINT	要接收的数据长度 (单位: Byte)	0
	DataOffset	UDINT	数据地址所在 M 区偏移 (单位: Byte)	0
	Port	USINT	0: UART1, 目前支持 232 串口 1: UART2, 目前支持 422/485 串口 2: UART3, 目前支持 485 串口 3: UART4, 目前支持 485 串口	0
输出端	Q	BOOL	输出, Q 端有效只用来说明本次任务周期该条指令执行完毕	0
	Err	USINT	错误码 0: 无错误 1: 输入参数配置错误 2: 本端口没有配置为自由口协议	0
	RecvDataBytes	UDINT	实际接收的字节数	0

i LK220 不支持该指令。

3.3.1.4 ComSend（发送串口数据）

3.3.1.4.1 功能描述

串口发送数据。

3.3.1.4.2 参数描述

	数据项	数据类型	参数描述	初始值（默认）
输入端	En_R	BOOL	上升沿使能	0
	DataLenth	UDINT	要发送的数据长度	0
	DataOffset	UDINT	数据地址所在 M 区的偏移	0
	Port	USINT	0: UART1, 目前支持 232 串口 1: UART2, 目前支持 422/485 串口 2: UART3, 目前支持 485 串口 3: UART4, 目前支持 485 串口	0
输出端	Q	BOOL	输出	0
	Err	USINT	错误码 0: 无错误 1: 输入参数配置错误 2: 本端口没有配置为自由口协议	0
	SendDataBytes	UDINT	实际发送字节数	0

❗ LK220 不支持该指令。

3.3.1.5 ComClearRecvFIFO（清空串口接收数据）

3.3.1.5.1 功能描述

清空串口接收数据。

3.3.1.5.2 参数描述

	数据项	数据类型	参数描述		初始值（默认）
			MC 平台	LE 平台	
输入端	En_R	BOOL	上升沿使能	上升沿使能	0
	Port	BYTE	0: UART1, 目前支持 232 串口 1: UART2, 目前支持 422/485 串口	2: 扩展功能板通讯口（仅 LE5400 支持），请参考：硬件配置->LE5400 串口配置参数 Port	0

	数据项	数据类型	参数描述		初始值 (默认)
			MC 平台	LE 平台	
			2: UART3, 目前支持 485 串口 3: UART4, 目前支持 485 串口		
输出端	Q	BOOL	输出	输出	0
	Err	BYTE(LE 平台) USINT(MC 平台)	错误	错误 0: 无错误 1: Port 端口配置错误 14: 背板通讯失败或清除失败	0

❗ LK220 不支持该指令。

3.3.1.6 CANRecv (接收 CAN 数据)

3.3.1.6.1 功能描述

单次读取 CAN 数据，如有滤波设置，则读到的是滤波后的数据。

3.3.1.6.2 参数描述

输入参数	数据类型	参数描述
IN	BOOL	高电平使能
Port	BYTE	端口号 0: CAN0

输出参数	数据类型	参数描述
Done	BOOL	TRUE: 接收到数据且没有错误 FALSE: 未接收到数据
Busy	BOOL	TRUE: 正在接收 FALSE: 空闲状态
Error	BOOL	TRUE: 处理过程有错误产生, 通过 ErrorID 项查看对应的错误码 FALSE: 无错误
ErrorID	WORD	0: 无错误 1: 端口号错误 2: 功能块未配为 CAN 自由口协议 3: 接收到的数据长度大于 8 字节 21: 硬件接收 FIFO 已满, 此时认为是网络风暴, 控制器会自动复位 CAN 硬件, 并恢复用户正在使用的参数设置 Bit7 为 1 表示 CAN 总线发生错误, 此时 Bit0~Bit4 共计 5 个 bit 位分别表示如下五种错误: Bit0: CRC 错误 (即 CRC ERR) Bit1: 形式错误 (即 Form ERR)

输出参数	数据类型	参数描述
		Bit2: 填充错误 (即 Stuff ERR) Bit3: 位错误 (即 Bit ERR) Bit4: 应答错误 (即 ACK ERR)
EXT	BOOL	TRUE: 扩展帧 FALSE: 标准帧
RTR	BOOL	TRUE: 远程帧 FALSE: 数据帧
MsgID	DWORD	消息 ID 标准帧: 低 11 位有效 扩展帧: 低 29 位有效
DataLen	BYTE	接收到的数据长度
Byte1	BYTE	第 1 字节数据
Byte2	BYTE	第 2 字节数据
Byte3	BYTE	第 3 字节数据
Byte4	BYTE	第 4 字节数据
Byte5	BYTE	第 5 字节数据
Byte6	BYTE	第 6 字节数据
Byte7	BYTE	第 7 字节数据
Byte8	BYTE	第 8 字节数据

 仅 MC 系列控制器 (除 MC1004L) 适用。

3.3.1.7 CANSend (发送 CAN 数据)

3.3.1.7.1 功能描述

单次发送数据。

3.3.1.7.2 参数描述

输入参数	数据类型	参数描述
EN_R	BOOL	上升沿使能
Port	BYTE	端口号 0: CAN0
EXT	BOOL	TRUE: 扩展帧 FALSE: 标准帧
RTR	BOOL	TRUE: 远程帧 FALSE: 数据帧
MsgID	DWORD	消息 ID 标准帧: 低 11 位有效 扩展帧: 低 29 位有效
DataLen	BYTE	发送的数据长度, 0~8

输入参数	数据类型	参数描述
Byte1	BYTE	第 1 字节数据
Byte2	BYTE	第 2 字节数据
Byte3	BYTE	第 3 字节数据
Byte4	BYTE	第 4 字节数据
Byte5	BYTE	第 5 字节数据
Byte6	BYTE	第 6 字节数据
Byte7	BYTE	第 7 字节数据
Byte8	BYTE	第 8 字节数据

输出参数	数据类型	参数描述
Done	BOOL	TRUE: 发送成功
Busy	BOOL	TRUE: 正在发送 FALSE: 空闲状态
Error	BOOL	TRUE: 处理过程有错误产生, 通过 ErrorID 项查看对应的错误码 FALSE: 无错误
ErrorID	WORD	错误码: 0: 发送成功 1: 端口号错误 2: 功能块未配为 CAN 自由口协议 3: 发送的数据长度大于 8 字节

i 仅 MC 系列控制器（除 MC1004L）适用。

3.3.1.8 CANConfig（设置 CAN 参数）

3.3.1.8.1 功能描述

设置波特率。

3.3.1.8.2 参数描述

输入参数	数据类型	参数描述
EN_R	BOOL	上升沿使能
Port	BYTE	端口号 0: CAN0
BaudRate	DWORD	波特率: 1000、800、500、250、125、100、50, 单位: kbps

输出参数	数据类型	参数描述
Done	BOOL	TRUE: 设置成功
Busy	BOOL	TRUE: 正在设置

输出参数	数据类型	参数描述
		FALSE: 空闲状态
Error	BOOL	TRUE: 处理过程有错误产生, 通过 ErrorID 项查看对应的错误码 FALSE: 无错误
ErrorID	WORD	错误码: 0: 设置成功 1: 端口号错误 2: 设置失败 3: 不支持的波特率参数

① 仅 MC 系列控制器（除 MC1004L）适用。

3.3.1.9 CANFilterEnable（使能 CAN 滤波参数）

3.3.1.9.1 功能描述

使能某一组硬件滤波参数，最大支持 4 组。

3.3.1.9.2 参数描述

输入参数	数据类型	参数描述
EN_R	BOOL	上升沿使能
Port	BYTE	端口号 0: CAN0
Group	BYTE	参数组序号，最大 4 组，0~3
MaskEXT	BOOL	是否使能 EXT 项，默认 TRUE
EXT	BOOL	TRUE: 扩展帧；FALSE: 标准帧
MaskRTR	BOOL	是否使能 RTR 项，默认 TRUE
RTR	BOOL	TRUE: 远程帧；FALSE: 数据帧 远程帧/数据帧过滤需要配合其他条件使用，不支持单独使用
MaskMsgID	BOOL	是否使能 MSGID 项，默认 TRUE
MsgID	DWORD	消息 ID 标准帧: 低 11 位有效 扩展帧: 低 29 位有效 如果 MaskEXT 项未使能，则 MSGID 小于 2048 按标准帧格式处理，大于等于 2048 按扩展帧格式处理

输出参数	数据类型	参数描述
Done	BOOL	TRUE: 使能成功

输出参数	数据类型	参数描述
Busy	BOOL	TRUE: 正在设置 FALSE: 空闲状态
Error	BOOL	TRUE: 处理过程有错误产生, 通过 ErrorID 项查看对应的错误码 FALSE: 无错误
ErrorID	WORD	错误码 0: 设置成功 1: 端口号错误 2: 设置失败 3: 不支持的组序号

-  仅 MC 系列控制器（除 MC1004L）适用。

3.3.1.10 CANFilterDisable（不使能 CAN 滤波参数）

3.3.1.10.1 功能描述

不使能某一组滤波参数，最大支持 4 组。

3.3.1.10.2 参数描述

输入参数	类型	参数描述
EN_R	BOOL	上升沿使能
Port	BYTE	端口号 0: CAN0
Group	BYTE	参数组序号，最大 4 组，0~3

输出参数	类型	参数描述
Done	BOOL	TRUE: 设置成功
Busy	BOOL	TRUE: 正在设置 FALSE: 空闲状态
Error	BOOL	TRUE: 处理过程有错误产生, 通过 ErrorID 项查看对应的错误码 FALSE: 无错误
ErrorID	WORD	错误码 0: 成功 1: 端口号错误 2: 设置失败 3: 不支持的组序号

-  仅 MC 系列控制器（除 MC1004L）适用。

3.3.1.11 CANClrRecvFIFO（清空 CAN 接收 FIFO）

3.3.1.11.1 功能描述

清空接收 FIFO。

3.3.1.11.2 参数描述

输入参数	数据类型	参数描述
EN_R	BOOL	上升沿使能
Port	BYTE	端口号 0: CAN0

输出参数	数据类型	参数描述
Done	BOOL	TRUE: 清空成功
Busy	BOOL	TRUE: 正在清空 FALSE: 空闲状态
Error	BOOL	TRUE: 处理过程有错误产生，通过 ErrorID 项查看对应的错误码 FALSE: 无错误
ErrorID	WORD	错误码 0: 成功 1: 端口号错误 2: 清空失败

 仅 MC 系列控制器（除 MC1004L）适用。

3.3.1.12 TCP_CONNECT(建立 TCP 通信连接)

3.3.1.12.1 功能描述

建立 TCP 协议通信连接。

注：该指令在 FA-AT V3.1.10B2 及以上版本支持，兼容的固件版本为 LK220-G02。

-  适用于 LK 系列、MC 系列以及 LE 系列 LE5118-B01、LE5119-B01 及以上版本。LE5118、LE5119 支持 5 路自由口，收发数据最大支持 1K。

3.3.1.12.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	EN_R	BOOL	上升沿：CPU 启动连接操作。EN_R 为 FALSE 时，输出显示连接的当前状态	0
	ConnectID	BYTE	Socket 连接编号 1~16	0
	Active	BOOL	TRUE：创建主动客户端连接 FALSE：创建被动服务器连接	0
	IPAddr1	BYTE	四字节 IP 地址，由高到低依次为 IPAddr1~ IPAddr4	0
	IPAddr2	BYTE		0
	IPAddr3	BYTE		0
	IPAddr4	BYTE		0
	RemotePort	WORD	远程设备上的端口号。远程端口号范围为 1 到 49151，对于被动连接，使用零	0
输出端	LocalPort	WORD	本地设备上的端口号。本地端口号范围为 1 到 49151，但存在一些限制，具体详见 以太网自由协议库指令共用参数 的 LocalPort 参数	0
	Done	BOOL	如果连接处于激活状态并准备发送或接收	0
	Busy	BOOL	如果连接过程仍在进行中	0
	Error	BOOL	如果连接不可用。ErrorID 显示其中一种错误代码，用于指示存在的问题	0
	ErrorID	WORD	如果指令置位 Error 输出，ErrorID 输出会显示错误代码 如果指令置位 Busy 或 Done 输出，ErrorID 为零（无错误） 有关详细信息，请参见 以太网自由协议通信指令错误码	0
	LinkIPAddr1	BYTE	已连接的远程设备 IP 地址的四个八位字节。LinkIPAddr1 是 IP 地址的最高有效字节，LinkIPAddr4 是 IP 地址的最低有效字节	默认隐藏
	LinkIPAddr2	BYTE		
	LinkIPAddr3	BYTE		
	LinkIPAddr4	BYTE		
	LinkPort	WORD	已连接远程设备端口号	

连接操作是异步的，可能需要几次扫描才能完成。连接操作待决时，Busy 输出值为 TRUE。当 CPU 完成操作时，指令置位 Done 或 Error 输出。如果发生错误，则 ErrorID 输出会显示错误代码。

指令处于繁忙状态时不得更改 TCP_CONNECT 的输入参数。CPU 需要凭借这一点了解这是启动连接过程的调用的延续。

您将连接 ID(ConnectID)输入分配给连接，然后当发送、接收或断开连接时使用此 ConnectID 引用该连接。

Active 输入位确定这是主动连接（Active 设置为 TRUE）还是被动连接（Active 设置为 FALSE）。

当 Active 输入设置为 TRUE 时，CPU 会创建主动连接（客户端），则 CPU 尝试联系并创建到指定 IP 地址和远程端口号(RemotePort)的连接。CPU 打开本地端口(LocalPort)以从远程设备接收消息。

当 Active 输入设置为 FALSE 时，CPU 会创建被动（服务器）连接。在这种情况下，CPU 打开请求的本地端口(LocalPort)并接受来自远程设备的连接请求。如果要接受来自任何远程 IP 地址的连接请求，应将 IP 地址设为 0.0.0.0。如果 IP 地址不为零，则 CPU 只接受来自指定 IP 地址的连接请求。对于被动连接，CPU 会忽略远程端口号(RemotePort)，RemotePort 可以设置为零。

您可以随时调用 TCP_CONNECT 指令以确定连接的当前状态。将 EN_R 输入设置为 FALSE 并提供有效的连接 ID(ConnectID)，TCP_CONNECT 返回以下内容：

- Busy, 如果连接过程仍在进行中。
- Done, 如果连接处于激活状态并准备发送或接收。
- Error, 如果连接不可用。ErrorID 显示其中一种错误代码, 用于指示存在的问题。
- LinkIPAddr1, 输出当前连接设备 IP 地址第 1 位
- LinkIPAddr2, 输出当前连接设备 IP 地址第 2 位
- LinkIPAddr3, 输出当前连接设备 IP 地址第 3 位
- LinkIPAddr4, 输出当前连接设备 IP 地址第 4 位
- LinkPort, 输出当前连接设备端口号

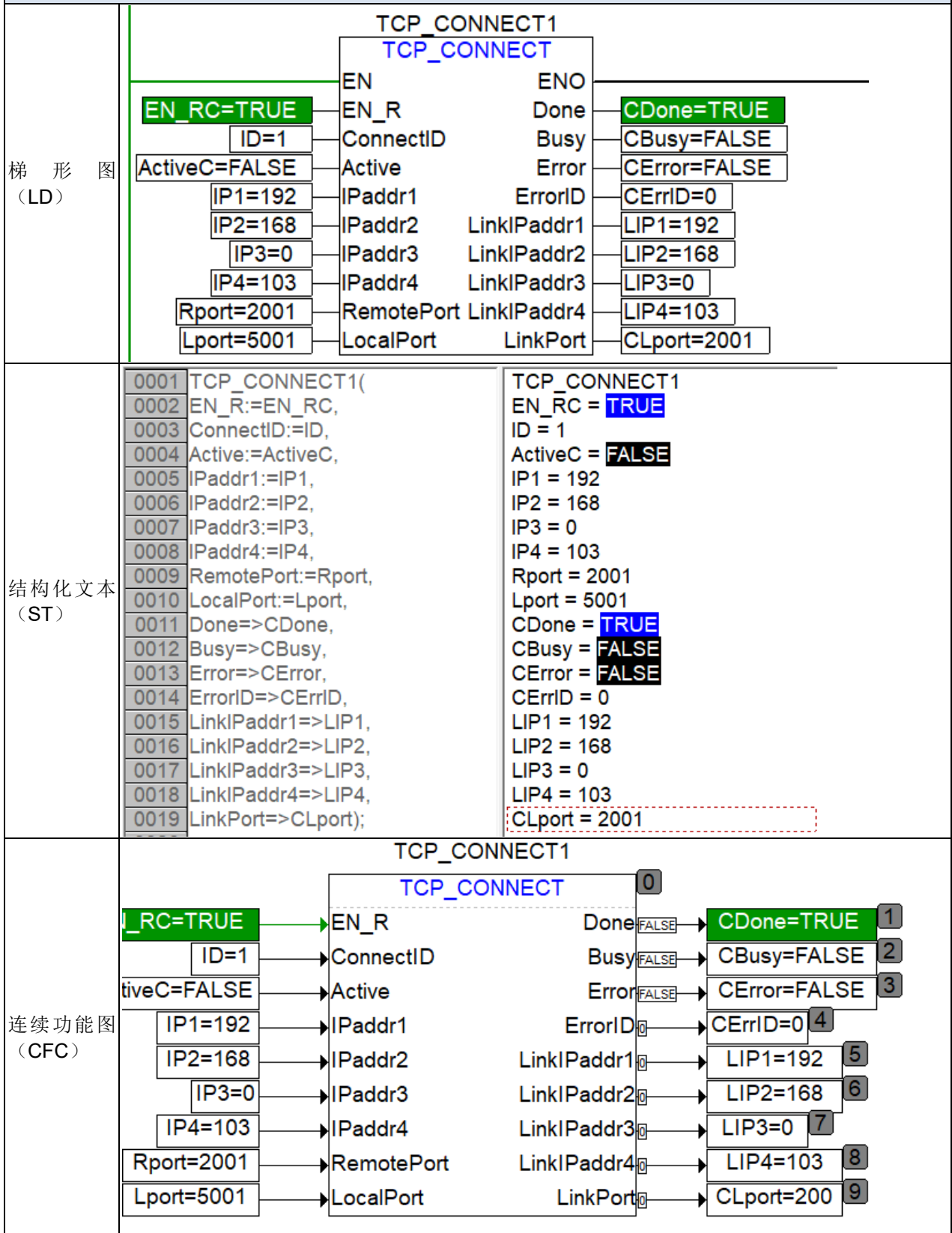
请注意:

1. 主动连接可能最多需要 30 秒的时间来确定远程设备是否允许连接。被动连接显示 **Busy** 状态, 直到远程设备尝试连接到 CPU。
2. 主动连接 EN_R 保持为高电平时, 如果远程设备断开设备连接, 则 CPU 自动尝试重新连接到远程设备
3. 连接关闭后, CPU 不会自动尝试重新连接到设备, 主动连接和被动连接皆如此。

3.3.1.12.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	TCP_CONNECT1			TCP_CONNECT		FALSE
0002	EN_RC			BOOL	TRUE	FALSE
0003	ID			BYTE	1	FALSE
0004	ActiveC			BOOL	FALSE	FALSE
0005	IP1			BYTE	192	FALSE
0006	IP2			BYTE	168	FALSE
0007	IP3			BYTE	0	FALSE
0008	IP4			BYTE	103	FALSE
0009	Rport			WORD	2001	FALSE
0010	Lport			WORD	5001	FALSE
0011	CDone			BOOL	TRUE	FALSE
0012	CBusy			BOOL	FALSE	FALSE
0013	CError			BOOL	FALSE	FALSE
0014	CErrorID			WORD	0	FALSE
0015	LIP1			BYTE	192	FALSE
0016	LIP2			BYTE	168	FALSE
0017	LIP3			BYTE	0	FALSE
0018	LIP4			BYTE	103	FALSE
0019	CLport			WORD	2001	FALSE
编程语言		程序				

变量定义



3.3.1.13 TCP_SEND (TCP 数据发送)

3.3.1.13.1 功能描述

使用 TCP 协议进行以太网数据发送。

注：该指令在 FA-AT V3.1.10B2 及以上版本支持，兼容的固件版本为 LK220-G02。

-  适用于 LK 系列、MC 系列以及 LE 系列 LE5118-B01、LE5119-B01 及以上版本。LE5118、LE5119 支持 5 路自由口，收发数据最大支持 1K。

3.3.1.13.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	EN_R	BOOL	上升沿：CPU 启动发送操作	0
	ConnectID	BYTE	连接 ID(ConnectID)是此发送操作所用连接的编号。使用您为 TCP_CONNECT 操作选择的 ConnectID	0
	DataLen	WORD	请求发送数据长度，1~8K	0
	DataAddr	POINTER TO BYTE	DataAddr 是指向发送数据存储位置的指针	0
输出端	Done	BOOL	当发送操作完成且没有错误时，指令置位 Done 输出	0
	Busy	BOOL	当发送操作正在进行时，指令置位 Busy 输出	0
	Error	BOOL	当发送操作完成但发生错误时，指令置位 Error 输出	0
	ErrorID	WORD	如果指令置位 Error 输出，ErrorID 输出会显示错误代码。如果指令置位 Busy 或 Done 输出，ErrorID 为零（无错误） 有关详细信息，请参见 以太网自由协议通信指令错误码	0
	SentLen	WORD	SentLen 是实际发送的字节数。仅当指令置位 Done 或 Error 输出时 SentLen 才有效。如果指令置位 Done 输出，则整条消息信息发送完成	0

当发生以下情况时，TCP_SEND 指令启动发送指定数量的字节的操作：

- 程序通过将 EN_R 输入设置为 TRUE 来调用指令。
- 连接当前未用于执行其它发送操作。

EN_R 输入使用上升沿触发器，以便指令不启动意外的发送操作。TCP_SEND 处于繁忙状态时，程序会忽略 EN_R 输入。Done、Busy 和 Error 输出及 ErrorID 输出字节显示各调用的 TCP_SEND 状态。

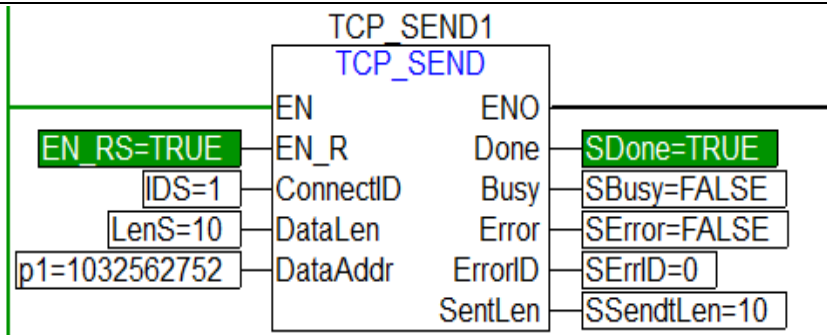
发送操作完成后，指令显示调用一次 TCP_SEND 的 Done 或 Error 状态。此后，TCP_SEND 通过错误代码作出响应(错误描述信息：没有待决操作)，(如果通过将 EN_R 输入设置为 FALSE 进行调用)。如果 EN_R 输入设置为 TRUE，则程序会启动另一个发送操作。下面描述了输入和输出参数之间的关系

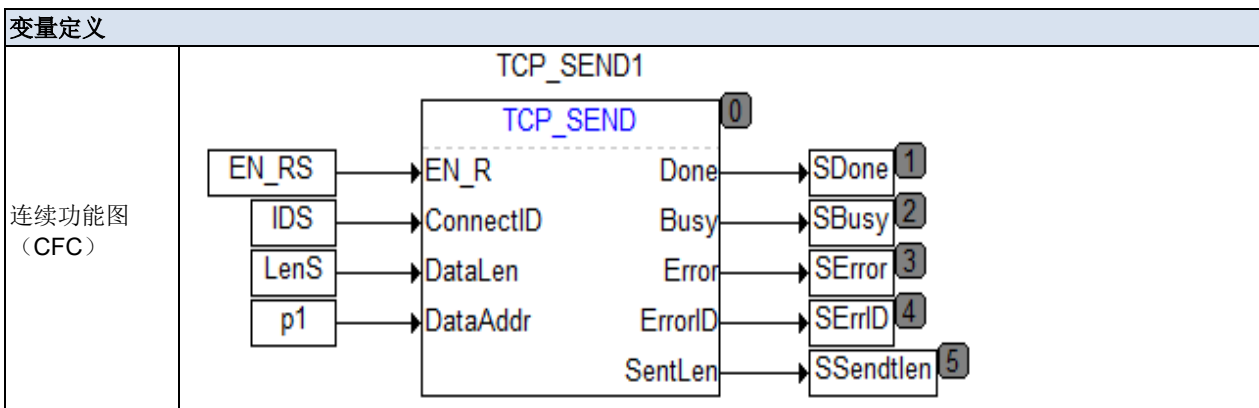
- EN_R 设置为 TRUE 以便开始执行消息发送操作。Busy 设置为 TRUE。

- 消息发送完成。Done 置位，Busy 清零。
- EN_R 为 FALSE，无任何消息发送操作正在执行。
- EN_R 再次设置为 TRUE，因此开始执行另一消息发送操作。Busy 设置为 TRUE。
- 消息发送完成。Done 置位，Busy 在一个扫描周期内清零。

3.3.1.13.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	DataAddrS			ARRAY[0..1024] OF BOOL		FALSE
0002	p1			POINTER TO BYTE	1032562752	FALSE
0003	TCP_SEND1			TCP_SEND		FALSE
0004	EN_RS			BOOL	TRUE	FALSE
0005	IDS			BYTE	1	FALSE
0006	LenS			WORD	10	FALSE
0007	SDone			BOOL	TRUE	FALSE
0008	SBusy			BOOL	FALSE	FALSE
0009	SError			BOOL	FALSE	FALSE
0010	SErrID			WORD	0	FALSE
0011	SSendtLen			WORD	10	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> TCP_SEND1(EN_R:=EN_RS, ConnectID:=IDS, DataLen:=LenS, DataAddr:=p1, Done=>SDone, Busy=>Sbusy, Error=>SError, ErrorID:=SErrID, SentLen=>SSendtlen); </pre>



3.3.1.14 TCP_RECV (TCP 数据接收)

3.3.1.14.1 功能描述

使用 TCP 协议进行以太网数据接收。

注：该指令在 FA-AT V3.1.10B2 及以上版本支持，兼容的固件版本为 LK220-G02。

-  适用于 LK 系列、MC 系列以及 LE 系列 LE5118-B01、LE5119-B01 及以上版本。LE5118、LE5119 支持 5 路自由口，收发数据最大支持 1K。

3.3.1.14.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	IN	BOOL	高电平使能持续接收	0
	ConnectID	BYTE	连接 ID(ConnectID)是此发送操作所用连接的编号。使用您为 TCP_CONNECT 操作选择的 ConnectID	0
	MaxLen	WORD	MaxLen 是要接收的最大字节数（例如，DataPtr 中缓冲区的大小（1 到 8K））	0
	DataAddr	POINTER TO BYTE	DataAddr 是指向接收数据存储位置的指针	0
输出端	Done	BOOL	当接收操作完成且没有错误时，指令置位 Done 输出。当指令置位 Done 输出时，RcvdLen 输出有效	0
	Busy	BOOL	当接收操作正在进行时，指令置位 Busy 输出	0
	Error	BOOL	当接收操作完成但发生错误时，指令置位 Error 输出	0
	ErrorID	WORD	如果指令置位 Error 输出，ErrorID 输出会显示错误代码。如果指令置位 Busy 或 Done 输出，ErrorID 为零（无错误） 有关详细信息，请参见以太网自由协议通信指令错误码	0
	RcvdLen	WORD	RcvdLen 是实际接收的字节数。仅当指令置位 Done 或 Error 输出时 RcvdLen 才有效。如果指令置位 Done 输出，则指令接收整条消息。如果指令置位 Error 输出，则	0

数据项	数据类型	参数描述	初始值
		消息超出缓冲区大小(MaxLen)并被截断	

TCP_RECV 指令具有 IN（使能）输入,由高电平触发。第一次执行 TCP_RECV 指令后，状态位显示指令处于繁忙状态。对 TCP_RECV 的后续调用会显示繁忙状态，直至 CPU 通过指定连接接收数据。

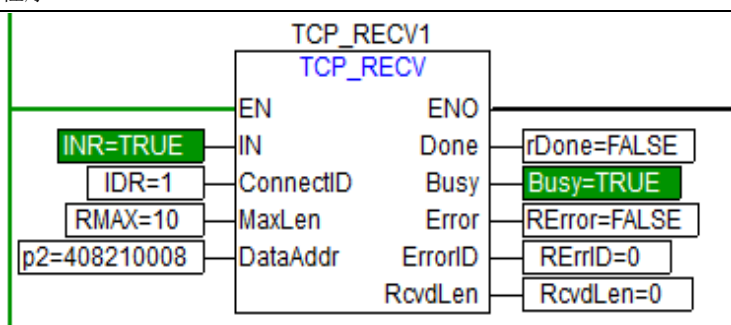
CPU 通过指定连接接收消息后，下一次执行 TCP_RECV 指令时，会执行以下任务：

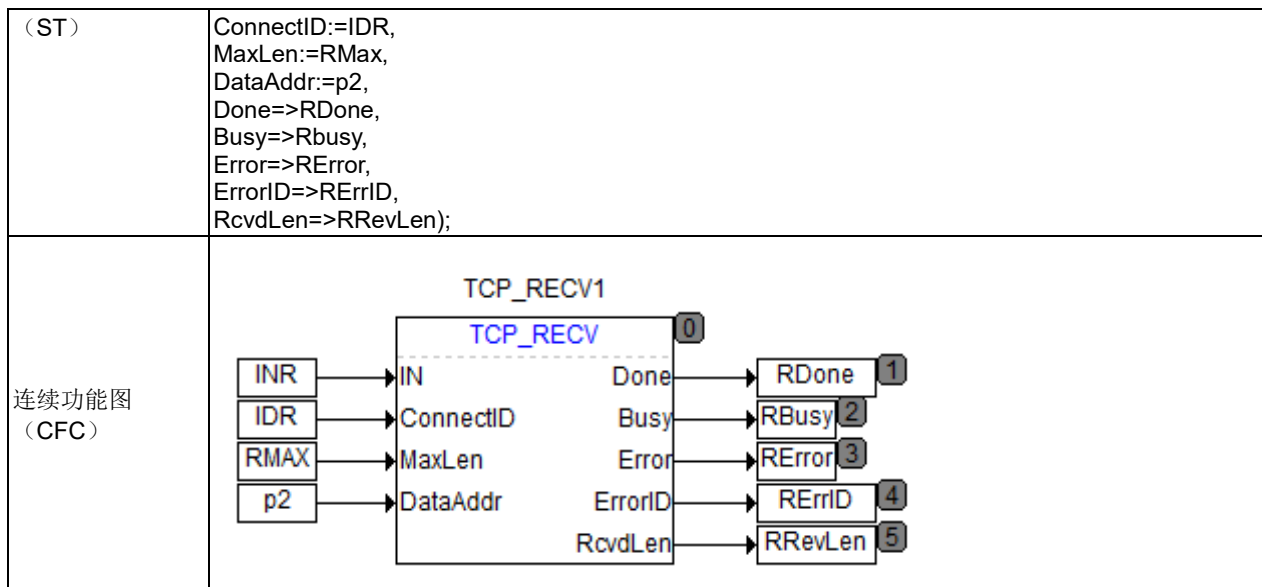
- 将消息数据复制到程序的数据区(DataAddr)。
- 将 RcvdLen 输出设置为接收的字节数。
- 置位 Done 输出，清除 Busy 和 Error 输出，且将 ErrorID 输出字节值设置为零。

使用 TCP 协议时，TCP_RECV 指令返回自程序上次调用 TCP_RECV 指令后 CPU 通过指定连接接收到的所有字节。程序必须足够频繁地调用 TCP_RECV 指令以正确地描绘消息，因为 TCP 充当“流”协议。在 TCP 协议中没有消息描述（没有开始或结束标记）。因此，CPU 并不知晓消息何时开始或结束。

例如，让我们假设存在一个 TCP 客户端接连不断地将 10 条 10 字节的消息发送给 CPU，而且在此期间程序不调用 TCP_RECV 指令。程序在 CPU 接受所有 10 条消息后调用 TCP_RECV 指令时，TCP_RECV 指令以一条 100 个字节的接收消息返回此数据。程序负责足够频繁地调用 TCP_RECV 指令，以便按发送消息的原样接收每条消息。

3.3.1.14.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	TCP_RECV1			TCP_RECV		FALSE
0002	INR			BOOL	TRUE	FALSE
0003	IDR			BYTE	1	FALSE
0004	RMAX			WORD	10	FALSE
0005	p2			POINTER TO BYTE	408210008	FALSE
0006	rDone			BOOL	FALSE	FALSE
0007	Busy			BOOL	TRUE	FALSE
0008	RErrID			WORD	0	FALSE
0009	RcvdLen			WORD	0	FALSE
0010	RError			BOOL	FALSE	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本		<pre>TCP_RECV1(IN:=INR,</pre>				



3.3.1.15 UDP_CONNECT（创建 UDP 通信）

3.3.1.15.1 功能描述

使用 UDP 协议创建被动连接。

3.3.1.15.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	EN_R	BOOL	上升沿触发连接	0
	ConnectID	BYTE	Socket 连接编号 1~5	0
	LocalPort	WORD	本地设备上的端口号。本地端口号范围为 1 到 49151，但存在一些限制，具体详见 以太网自由协议库指令共用参数 的 LocalPort 参数	0
输出端	Done	BOOL	当连接操作完成且没有错误时，指令置位 Done 输出	0
	Busy	BOOL	当连接操作正在进行时，指令置位 Busy 输出	0
	Error	BOOL	如果连接不可用。ErrorID 显示其中一种错误代码，用于指示存在的问题	0
	ErrorID	WORD	如果指令置位 Error 输出，ErrorID 输出会显示错误代码。如果指令置位 Busy 或 Done 输出，ErrorID 为零（无错误） 有关详细信息，请参见 以太网自由协议通信指令错误码	0

UDP_CONNECT 指令只需要连接 ID 和本地端口号即可创建连接。一个 UDP 连接可以将消息发送到任意数量的其它设备，因为 IP 地址和远程端口会随每个 UDP_SEND 指令一起提供。仅当需要多个本地端口时，才需要多个 UDP 连接。不能将同一本地端口号用于多个 UDP 连接。所有本地端口号必须唯一。

连接操作是异步的，可能需要几次扫描才能完成。没有与远程设备建立主动连接，也没有等待另一台设备连接到该 CPU。当连接操作待决时，Busy 输出置位。当连接操作完成时，程序置位 Done 输出。仅当

输入参数发生问题或没有可用的被动连接时，程序才置位 **Error** 输出。如果程序将 **Error** 位置位，**ErrorID** 输出字节将包含错误代码。

指令处于繁忙状态时，不得更改 **UDP_CONNECT** 的参数，CPU 借此可了解这是启动连接过程的调用的延续。

您可以调用 **UDP_CONNECT** 指令以确定连接的当前状态。将 **EN_R** 输入设置为 **FALSE** 并提供有效的连接 ID(**ConnectID**)，**UDP_CONNECT** 指令返回以下内容：

- 如果连接处于激活状态并准备发送或接收，指令置位 **Done** 输出。这仅表示您可以使用该连接，并不表示存在任何远程设备。
- 如果连接仍在进行，指令置位 **Busy** 输出。
- 如果连接不可用，指令置位 **Error** 输出。**ErrorID** 输出字节显示其中一种错误代码，用于指示存在的问题。

3.3.1.16 UDP_SEND (UDP 数据发送)

3.3.1.16.1 功能描述

UDP_SEND 指令将来自请求的缓冲区位置(**DataAddr**)的请求的字节数(**DataLen**)传输到通过 IP 地址 (**IPAddr1**–**IPAddr4**)和端口(**RemPort**)指定的设备。该指令仅用于 **UDP** 协议和通过 **UDP_CONNECT** 创建的连接。

3.3.1.16.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	EN_R	BOOL	上升沿：CPU 启动发送操作	0
	ConnectID	BYTE	连接 ID(ConnectID)是此发送操作所用连接的编号（连接过程中通过 UDP_CONNECT 定义）	0
	DataLen	WORD	发送数据长度，1~64K	0
	DataAddr	POINTER TO BYTE	DataAddr 是指向发送数据存储位置的指针	0
	IPAddr1	BYTE	四字节 IP 地址，由高到低依次为 IPAddr1 ~ IPAddr4	0
	IPAddr2	BYTE		0
	IPAddr3	BYTE		0
	IPAddr4	BYTE		0
	RemotePort	WORD	远程设备上的端口号。远程端口号范围为 1 到 49151。对于被动连接，使用零	0
输出端	Done	BOOL	当发送操作完成且没有错误时，指令置位 Done 输出	0
	Busy	BOOL	当发送操作正在进行时，指令置位 Busy 输出	0
	Error	BOOL	当发送操作完成但发生错误时，指令置位 Error 输出	0
	ErrorID	WORD	如果指令置位 Error 输出， ErrorID 输出会显示错误代码。如果指令置位 Busy 或 Done 输出， ErrorID 为零（无错误） 有关详细信息，请参见 以太网自由协议通信指令错误码	0

当发生以下情况时，UDP_SEND 指令启动发送指定数量的字节的操作：

- 程序通过将 EN_R 输入设置为 TRUE 来调用指令。
- 连接当前未用于执行其它发送操作。

EN_R 输入由电平触发。建议对 EN_R 输入使用上升沿触发器，以便指令不启动意外的发送操作。

UDP_SEND 处于繁忙状态时，程序会忽略 EN_R 输入。Done、Busy 和 Error 输出及 ErrorID 输出字节显示各调用的 UDP_SEND 状态。

发送操作完成后，指令显示调用一次 UDP_SEND 的 Done 或 Error 状态。此后，UDP_SEND 通过错误代码作出响应，这意味着没有待决操作（如果通过将 EN_R 输入设置为 FALSE 进行调用）。如果 EN_R 输入设置为 TRUE，则程序会启动另一个发送操作。下面描述输入和输出参数之间的关系。

- EN_R 设置为 TRUE 以便开始执行消息发送操作。Busy 设置为 TRUE。
- 消息发送完成。Done 置位，Busy 清零。
- EN_R 为 FALSE，但无任何消息发送操作正在执行。
- EN_R 再次设置为 TRUE，因此开始执行另一消息发送操作。Busy 设置为 TRUE。
- 消息发送完成。Done 置位，Busy 在一个扫描周期内清零。

3.3.1.17 UDP_RECV (UDP 数据接收)

3.3.1.17.1 功能描述

UDP_RECV 指令通过现有连接检索数据。该指令仅用于 UDP 协议以及通过 UDP_CONNECT 创建的连接。

3.3.1.17.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	IN	BOOL	高电平使能	0
	ConnectID	BYTE	CPU 将连接 ID(ConnectID)用于此接收操作（UDP_CONNECT 连接过程中定义）	0
	MaxLen	WORD	MaxLen 是要接收的最大字节数（例如，DataAddr 中缓冲区的大小（1 到 64K））	0
	DataAddr	POINTER TO BYTE	DataAddr 是指向接收数据存储位置的指针	0
输出端	Done	BOOL	当接收操作完成且没有错误时，指令置位 Done 输出	0
	Busy	BOOL	当接收操作正在进行时，指令置位 Busy 输出	0
	Error	BOOL	当接收操作完成但发生错误时，指令置位 Error 输出	0
	ErrorID	WORD	如果指令置位 Error 输出，ErrorID 输出会显示错误代码。如果指令置位 Busy 或 Done 输出，ErrorID 为零（无错误） 有关详细信息，请参见 以太网自由协议通信指令错误码	0

	数据项	数据类型	参数描述	初始值
	RcvdLen	WORD	RcvdLen 是实际接收的字节数。仅当指令置位 Done 或 Error 输出时 RcvdLen 才有效。如果指令置位 Done 输出, 则指令接收整条消息。如果指令置位 Error 输出, 则消息超出缓冲区大小(MaxLen)并被截断	0
	SentIPAddr1	BYTE	发送消息的远程设备 IP 地址的四个八位字节。SentIPAddr1 是 IP 地址的最高有效字节, SentIPAddr4 是 IP 地址的最低有效字节	0.0.0.0
	SentIPAddr2	BYTE		
	SentIPAddr3	BYTE		
	SentIPAddr4	BYTE		
	SentPort	WORD	发送消息的远程设备的端口号	0

UDP_RECV 指令仅具有 IN (使能) 输入。UDP_RECV 指令高电平有效。第一次执行 UDP_RECV 指令后, 状态输出显示指令处于繁忙状态。对 UDP_RECV 的后续调用显示繁忙状态, 直至 CPU 通过指定连接接收数据。

CPU 通过指定连接接收消息后, 下一次执行 UDP_RECV 指令时, 会执行以下任务:

- 将消息数据复制到程序的数据区(DataAddr)。
- 将返回的 RcvdLen 设置为接收的字节数。
- 将 SentIPAddr1~SentIPAddr4 地址设置为发送消息的远程设备的地址。
- 将远程端口号(SentPort)设置为远程设备的端口。
- 置位 Done 输出, 清除 Busy 和 Error 输出, 且将 ErrorID 输出字节值设置为零。

3.3.1.18 ETH_DISCONNECT (关闭以太网通信连接)

3.3.1.18.1 功能描述

ETH_DISCONNECT 指令用于终止现有通信连接。

注: 该指令在 FA-AT V3.1.10B2 及以上版本支持, 兼容的固件版本为 LK220-G02。

-  适用于 LK 系列、MC 系列以及 LE 系列 LE5118-B01、LE5119-B01 及以上版本。E5118、LE5119 支持 5 路自由口, 收发数据最大支持 1K。

3.3.1.18.2 参数描述

	数据项	数据类型	参数描述	初始值
输入端	EN_R	BOOL	上升沿触发关闭连接	0
	ConnectID	BYTE	CPU 使用连接 ID(ConnectID)标识要终止的连接 (连接过程中定义)	0
输出端	Done	BOOL	当断开连接操作完成且没有错误时, 指令置位 Done 输出	0
	Busy	BOOL	当断开连接操作正在进行时, 指令置位 Busy 输出	0

	数据项	数据类型	参数描述	初始值
	Error	BOOL	当断开连接操作完成但发生错误时，指令置位 Error 输出	0
	ErrorID	WORD	如果指令置位 Error 输出，ErrorID 输出会显示错误代码。如果指令置位 Busy 或 Done 输出，ErrorID 为零（无错误） 有关详细信息，请参见 以太网自由协议通信指令错误码	0

3.3.1.18.3 以太网自由协议库指令共用参数

共用参数	参数说明
EN_R	输入用于发起操作。由电平触发。应通过上升沿指令将 EN_R 输入连接到库指令，以便操作仅启动一次。指令为 Busy 时程序会忽略 EN_R 输入
Active	Active 输入用于指定连接指令是创建主动客户端连接(Active=TRUE)还是创建被动服务器连接(Active=FALSE)。在主动连接中，本地 CPU 启动到远程设备的通信。在被动连接中，本地 CPU 等待远程设备启动通信
Done	当操作完成且没有错误时，OUC 指令置位 Done 输出。如果指令置位 Done 输出，Busy、Error 和 ErrorID 输出为零。仅当 Done 输出置位时，其它输出（例如，接收到的字节数）才有效
Busy	Busy 输出指示正在进行操作。通过将 EN_R 设为 TRUE 启动操作时，OUC 指令置位 Busy 输出。对于对指令的所有后续调用，Busy 输出保持置位，直到操作完成
Error	Error 输出指示操作完成但有错误。如果 OUC 指令置位 Error 输出，则 Done 和 Busy 输出将设置为 FALSE。如果指令置位 Error 输出，则 ErrorID 输出会指明错误原因。如果 Error 输出置位，所有其它输出均无效
ConnectID	ConnectID 编号是连接的标识符。通过 TCP_CONNECT 创建连接时，会创建 ConnectID 可以选择 1 到 5 范围内的任何值。每个连接必须具有唯一的 ConnectID。程序使用 ConnectID 指定后续发送、接收和断开操作所需的连接
IPAddr1~IPAddr4	这些是远程设备 IP 地址的四个八位字节。IPAddr1 是 IP 地址的最高有效字节，IPAddr4 是 IP 地址的最低有效字节。例如：对于 IP 地址 192.168.2.15，设置以下值： IPAddr1=192 IPAddr2=168 IPAddr3=2 IPAddr4=15 IP 地址不能为以下值： 0.0.0.0（针对主动连接） 任何广播 IP 地址（例如，255.255.255.255） 任何多播地址 本地 CPU 的 IP 地址 可以将 IP 地址 0.0.0.0 用于被动连接。通过选择 IP 地址 0.0.0.0，CPU 接受来自任何远程 IP 地址的连接。如果为被动连接选择一个非零的 IP 地址，CPU 将仅接受来自指定地址的连接
RemotePort	远程设备上的端口号。端口号可用于 TCP 和 UDP 协议，从而路由设备内的消息 远程端口号的规则如下： 有效端口号范围为 1 到 49151 建议采用的端口号范围为 2000 到 49151 对于被动连接，CPU 会忽略远程端口号（可以将其设置为零）
LocalPort	是本地 CPU 上的端口号。端口号可用于 TCP 和 UDP 协议，从而路由设备内的消息。对于所有被动连接，本地端口号必须唯一 本地端口号的规则如下： 有效端口号范围为 1 到 49151 不能使用端口号 20、21、25、80、102、135、161、162、443、1200 以及 34962 至 34964。这些端口具有特定用途 建议采用的端口号范围为 2000 到 49151

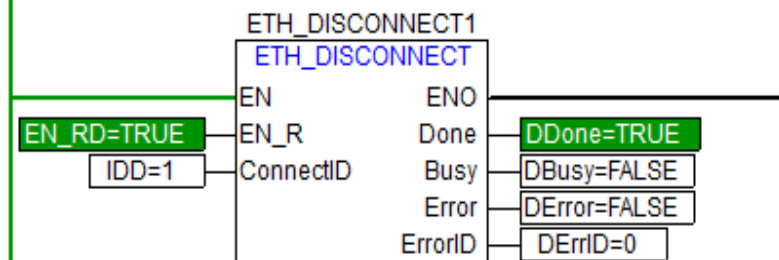
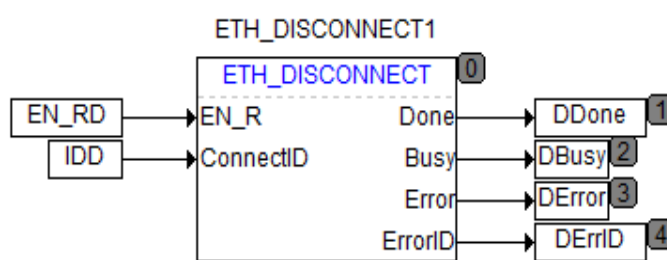
共用参数	参数说明
	对于被动连接，本地端口号必须唯一（不重复）

3.3.1.18.4 以太网自由协议通信指令错误码

错误码	错误信息描述
0	无错误
1	输入 ConnectID 错误
2	输入 IP 地址非法
3	输入的远程端口号非法
4	输入的本地端口号非法
5	正在尝试使用 TCP_CONNECT 指令连接时，输入参数发生变化
6	正在尝试使用 TCP_CONNECT 指令创建主动连接，该路连接输入信息与其它路连接信息重复
7	正在尝试使用 TCP_CONNECT 指令创建被动连接，该路连接输入信息与其它路连接信息重复
8	连接已关闭
9	连接非 TCP 类型
10	建立连接失败
11	连接对象拒绝连接
12	接收到非设定 IP 地址的连接请求
13	以太网协议链路未配置以太网自由协议功能
14	在使用 TCP 或 UDP 自由协议发送或接收数据时，输入数据长度或地址信息错误
15	TCP_SEND 发送数据发生失败
16	TCP_RECV 接收数据发生失败
17	连接非 UDP 类型
18	正在使用 ETH_DISCONNECT 指令关闭已关闭的链路
19	UDP_CONNECT 连接输入信息与其它路连接信息重复
20	正在尝试使用 UDP_CONNECT 连接时，输入参数发生变化
21	TCP_CONNECT 设置套接字选项过程发生错误
22	TCP_CONNECT 绑定端口过程失败
23	TCP_CONNECT 链路监听 listen 过程失败
24	UDP_CONNECT 设置套接字选项过程发生错误
25	UDP_CONNECT 绑定端口过程失败
26	多个任务正在同时进行同一路连接操作

3.3.1.18.5 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	ETH_DISCONNECT1			ETH_DISCONNECT		FALSE
0002	EN_RD			BOOL	TRUE	FALSE
0003	IDD			BYTE	1	FALSE
0004	DDone			BOOL	TRUE	FALSE
0005	DBusy			BOOL	FALSE	FALSE
0006	DError			BOOL	FALSE	FALSE
0007	DErrID			WORD	0	FALSE

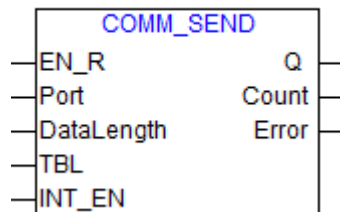
变量定义	
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> ETH_DISCONNECT(EN_R:=EN_RD, ConnectID:=IDD, Done=>DDone, Busy=>DBusy, Error=>DError, ErrorID=>DErrID); </pre>
连续功能图 (CFC)	

3.3.1.19 COMM_SEND (自由协议通讯数据发送)

3.3.1.19.1 功能

提供串口自由协议通讯数据发送端的参数配置和显示发送错误信息的用户接口。

在 MKC 和 LE 平台中，该功能块的引脚会有区别，详见参数说明。



COMM_SEND 功能块

3.3.1.19.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值	备注
EN_R	BOOL	使能	0: 无效 1: 上升沿使能	0	
Port	BYTE	通讯口选择	LE 平台: 0: CPU 本体的圆形接口 1: CPU 本体端子通讯口 (14 点不支持) 2: 扩展功能板通讯口 (14 点不支持) 3: 第一个通讯扩展模块通讯口 4~N: 第二个到第 N 个通讯扩展模块通讯口 MKC 平台: 0: CPU 本体的 PORT0 1: CPU 本体的 PORT1、PORT2 (PORT2 口仅支持 MKC3167)	0	
DataLength	BYTE	发送字节数	发送的字节数, 1~255 个字节	255	
TBL	WORD	数据存放地址	指向 M 区数据存放的首字节地址, 诸如 200, 表示存放地址为 %MB200 开始的一段空间	0	
INT_EN	BOOL	中断使能输入	0: 发送完数据后不产生中断 1: 发送完数据后产生中断 中断程序名称通过中断配置界面配置	0	MKC 平台无此参数

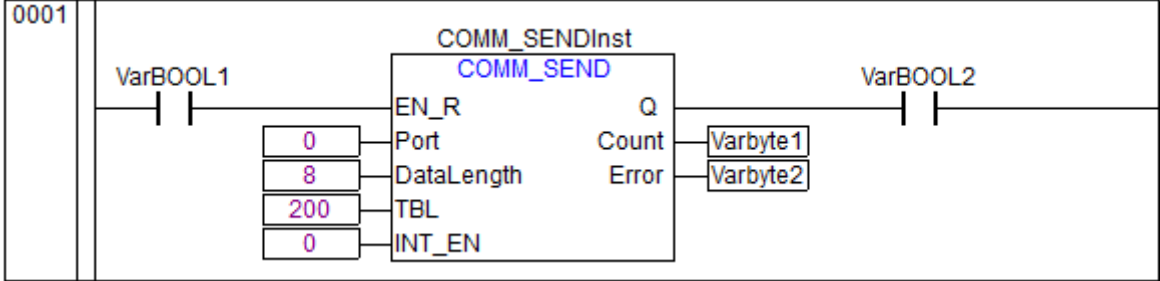
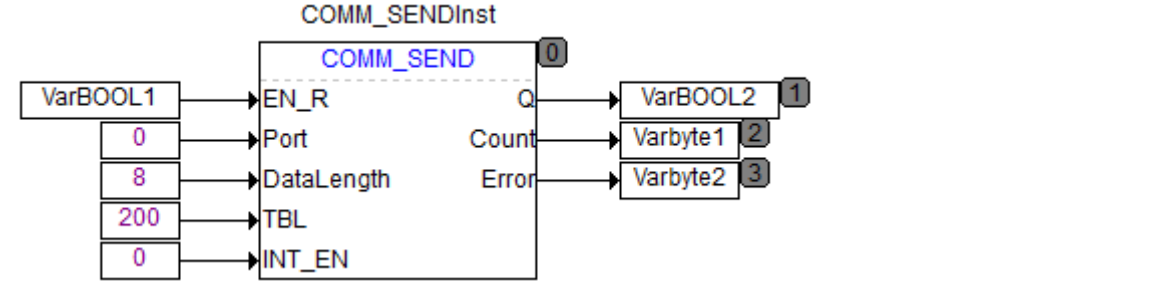
输出参数	数据类型	功能描述	参数值说明
Q	BOOL	发送完成标志	发送完成标志。发送完输出 1, 否则输出 0
Count	BYTE	实际发送的字节数	实际发送的字节数
Error	BYTE	错误信息	LE 平台: =0: 正确 =1: 通讯口 (Port) 设置错误 =2: 数据长度 (DataLength) 错 =3: 数据存放地址 (TBL) 越界 =4: 获取用户空间指针失败 =5: 多个功能块同时使能一个通讯口 =8: 通讯口占用 (请检查该端口是否已用于其它协议方式或与 AT 通讯) =14: 背板通讯失败 (仅 LE5400 模块上报) MKC 平台: =0: 正确 =1: 通讯口 (Port) 设置错误 =2: 数据长度 (DataLength) 错 =3: 通讯口的功能选择非自由口 =4: 数据存放地址 (TBL) 越界 =5: 获取用户空间指针失败 =6: 多个功能块同时使能一个通讯口



- 输出参数在 EN_R 为 FALSE (0) 的状态时, 会恢复成初始值 0。
- 当使用自由口发送指令连续发送多个数据包时, 发送数据包时间间隔必须不小于发送一包数据的时间。

3.3.1.19.3 指令使用举例

以 LE 平台为例进行说明。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	COMM_SENDInst			COMM...		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	Varbyte1			BYTE	0	FALSE
0004	VarBOOL2			BOOL	FALSE	FALSE
0005	Varbyte2			BYTE	0	FALSE
编程语言	程序					
梯形图 (LD)						
结构化文 本 (ST)	<pre> COMM_SENDInst(EN_R:=VarBOOL1, Port:=0, DataLength:=8, TBL:=200, INT_EN:=0, Q=>VarBOOL2, Count=>Varbyte1, Error=>Varbyte2); </pre>					
连续功能 图 (CFC)						

程序说明：

通过 Port 参数将通讯口设置为 CPU 本体的圆形接口。

EN_R 置位并保持时，将%MB200 的连续 8 个字节（即%MB200-%MB207）顺序发送一次，Q 在发送完成的一个周期内等于 TRUE，之后变为 FALSE。

EN_R 复位时，不进行发送。

EN_R 每次由 FALSE 变为 TRUE（置位）时，发送一次数据。

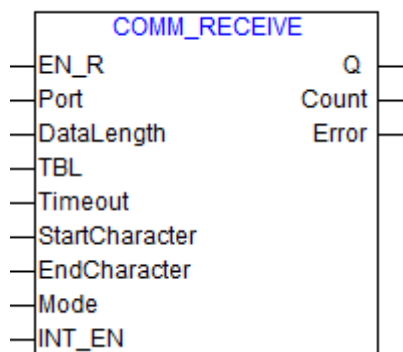
3.3.1.20 COMM_RECEIVE（自由协议通讯数据接收）

在 MKC 和 LE 平台中，该功能块的引脚会有区别，详见各平台参数说明。

3.3.1.20.1 LE 平台

3.3.1.20.1.1 功能

提供串口自由协议通讯数据接收端的参数配置和显示接收错误信息的用户接口。



COMM_RECEIVE 功能块

3.3.1.20.1.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0: 无效 1: 上升沿使能	0
Port	BYTE	通讯口选择	0: CPU 本体的圆形接口 1: CPU 本体端子通讯口（14 点不支持） 2: 扩展功能板通讯口（14 点不支持） 3: 第一个通讯扩展模块通讯口 4~N: 第二个到第 N 个通讯扩展模块通讯口	0
DataLength	BYTE	接收字节数	接收的字节数，0~255 个字节，设为 0 是不定长接收模式	255
TBL	WORD	数据存放地址	指向 M 区数据存放的首字节地址， 诸如 200，表示存放地址为%MB200 开始的一段空间	0
Timeout	WORD	帧超时时间（ms）	从启动接收过程开始计算，在规定的时间内没有再接收到完整的数据包，中止接收过程 最小值为 50ms，最小时间单位为系统调度时间	0
StartCharacter	WORD	开始字符	当启用开始字符时，只有接收到开始字符才表征一帧数据的开始，否则被丢弃	0
EndCharacter	WORD	结束字符	当启用结束字符时，只有接收到结束字符才表征一帧数据的结束，否则继续接收直到缓冲区满（255 字节）	0
Mode	BYTE	控制模式参数设置	详见表 1	0

输入参数	数据类型	功能描述	参数值说明	默认值
INT_EN	BOOL	中断使能输入	0: 接收完数据后不产生中断 1: 接收完数据后产生中断 中断程序名称通过中断配置界面配置	0

输出	数据类型	功能描述	参数值说明
Q	BOOL	接收完成标志	接收完成后第一个扫描周期为 1
Count	BYTE	实际接收的字节数	实际接收的字节数
Error	BYTE	错误信息	Error =0: 正确 Error =1: 通讯口 (Port) 设置错误 Error =2: 数据存放地址 (TBL) 越界 Error =3: 缺少结束字符 Error =4: 超时时间 (Timeout) 设定过小 Error =5: 获取用户空间指针失败 Error =6: 超时 Error =7: 多个功能块同时使能一个通讯口 Error =8: 通讯口占用 (请检查该端口是否已用于其它协议方式或与 AT 通讯) Error =14: 背板通讯失败 (仅 LE5400 模块上报)

表 1 Mode 参数详细说明

Mode (BYTE)								
7	6	5	4	3	2	1	0	
未定义		3.5char 使能控制		开始、结束字符数据长度控制		结束字符控制	开始字符控制	超时控制
超时控制								
0: 禁止超时控制								
1: 启动超时控制								
开始字符控制								
0: 禁止开始字符控制								
1: 启动开始字符控制								
结束字符控制								
0: 禁止结束字符控制								
1: 启动结束字符控制								
开始、结束字符数据长度控制								
0: 开始、结束字符为字								
1: 开始、结束字符为字节								
3.5char 使能控制								
0: 禁止 3.5char 分帧控制								
1: 启动 3.5char 分帧控制								

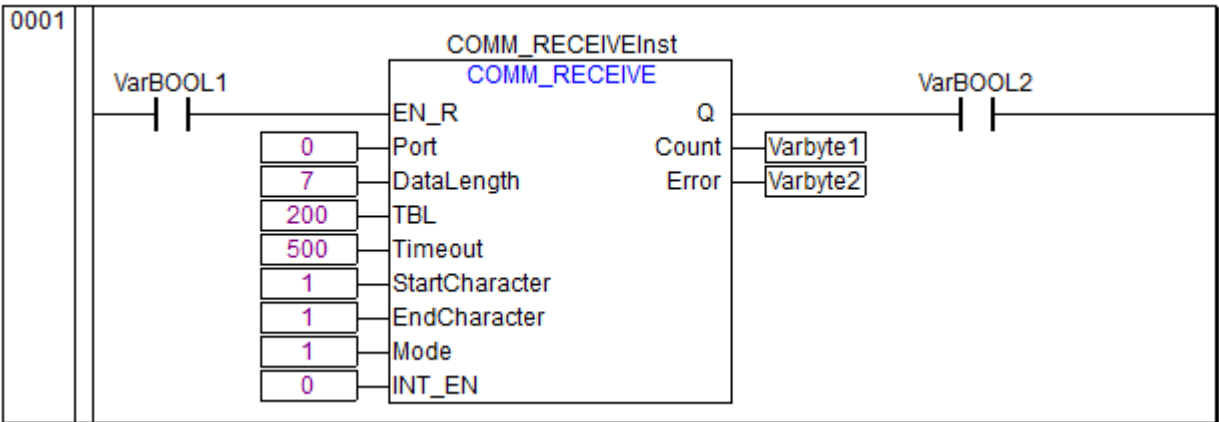
表 2 数据长度与开始字符、结束字符的配合使用

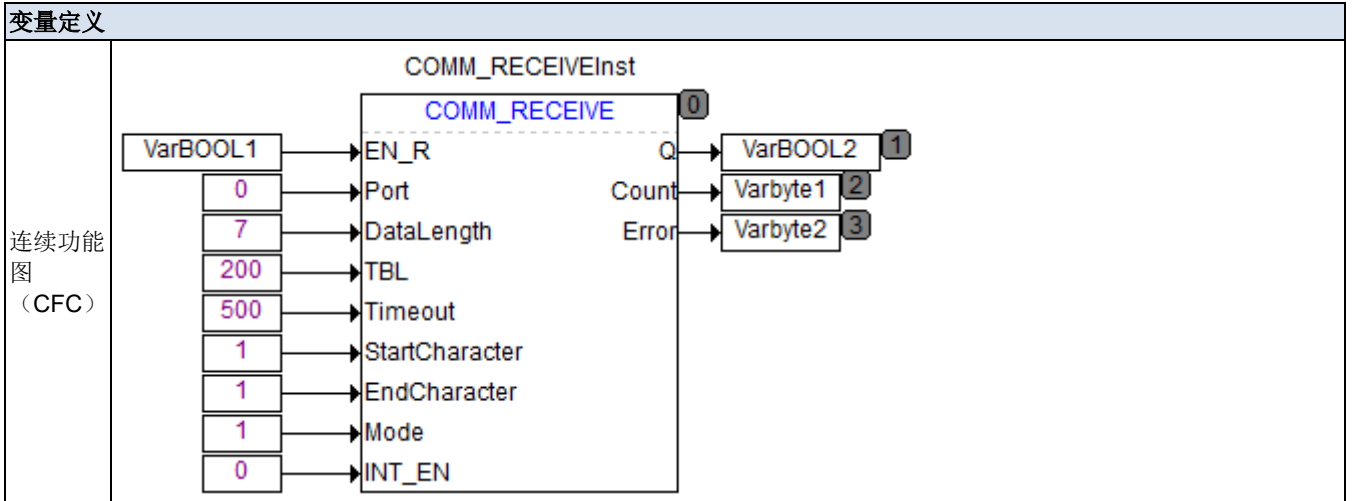
DataLength	StartCharacter	EndCharacter	字符的接收过程
0	使能	使能	接收到开始字符才表征一帧数据的开始, 否则数据被丢弃, 接收到结束字符才表示一帧数据的结束, 否则继续接收直到缓冲区满 (255 字节)
0	禁止	使能	接收到结束字符才表征一帧数据的结束, 否则继续接收直到缓冲区满 (255 字节)
0	使能	禁止	Error 报错, 缺少结束字符
0	禁止	禁止	Error 报错, 缺少结束字符
不为 0	使能	禁止	收到开始字符后继续接收, 并依次排放, 直到接满 DataLength 个字节 (包括开始字符) 后结束本次接收过程

DataLength	StartCharacter	EndCharacter	字符的接收过程
不为 0	使能	使能	收到开始字符后继续接收，直到接满 DataLength 个字节（包括开始字符）或收到结束字符，结束本次接收过程
不为 0	禁止	使能	指令有效即开始接收，直到接满 DataLength 个字节（包括开始字符）或收到结束字符，结束本次接收过程
不为 0	禁止	禁止	指令有效即开始接收，直到接满 DataLength 个字节，结束本次接收过程

① 使用开始\结束字符控制功能时注意大小端问题。

3.3.1.20.1.3 指令使用举例

变量定义	
0001	COMM_RECEIVEInst
0002	VarBOOL1
0003	Varbyte1
0004	VarBOOL2
0005	Varbyte2
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> COMM_RECEIVEInst(EN_R:=VarBOOL1, Port:=0, DataLength:=7, TBL:=200, Timeout:=500, StartCharacter:=1, EndCharacter:=1, Mode:=1, INT_EN:=0, Q=>VarByte1, Count=>VarByte1, Error=>VarByte2); </pre>



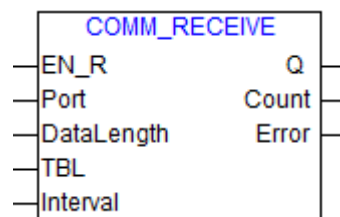
程序说明：

在自由协议通讯状态下，上电 Q 等于 FALSE，当开始运行时第一个扫描周期准备接收 7 个字节数据分别保存到%MB200-%MB206 中，接收成功后 Q 等于 TRUE，从而 EN_R 等于 FALSE。下一个扫描周期时该指令不进行接收，Q 等于 FALSE，从而 EN_R 等于 TRUE。再下一个扫描周期再次准备接收 7 个字节数据分别保存到%MB200-%MB206 中，如此循环接收数据。

3.3.1.20.2 MKC 平台

3.3.1.20.2.1 功能

提供串口自由协议通讯数据接收端的参数配置和显示接收错误信息的用户接口。



COMM_RECEIVE 功能块

3.3.1.20.2.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0: 无效 1: 上升沿使能	0
PORT	BYTE	通讯口选择	=0: CPU 本体的 PORT0 =1: CPU 本体的 PORT1、PORT2 (PORT2 口仅支持 MKC3167)	0
DATALENGTH	BYTE	接收字节数	接收的字节数，0~255 个字节。DATALENGTH 值设置为 0 时是采用 INTERVAL 值指定的时间断帧的方式进行接收数据；DATALENGTH 值设置为非 0 时，采用接收相应个数的数据来断帧	255
TBL	WORD	数据存放地址	指向 M 区数据存放的首字节地址。诸如 200，表示存放地址为%MB200 开始的一段空间	0

输入参数	数据类型	功能描述	参数值说明	默认值
INTERVAL	BYTE	断帧间隔时间设置	断帧间隔时间的字符数目的设定，设定单位是字符，3~255 个字符。如设定字符数的时间之内没有新数据接收到就中止一帧数据的接收	3

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	接收完成标志	接收完成标志，接收完成输出 1，否则为 0
Count	BYTE	实际接收的字节数	实际接收的字节数
Error	BYTE	错误信息	=0: 正确 =1: 通讯口（Port）设置错误 =2: 数据存放地址（TBL）越界 =3: 通讯口的功能选择非自由口 =4: 断帧时间间隔（INTERVAL）设定错误 =5: 获取用户空间指针失败 =6: 多个功能块同时使能一个通讯口



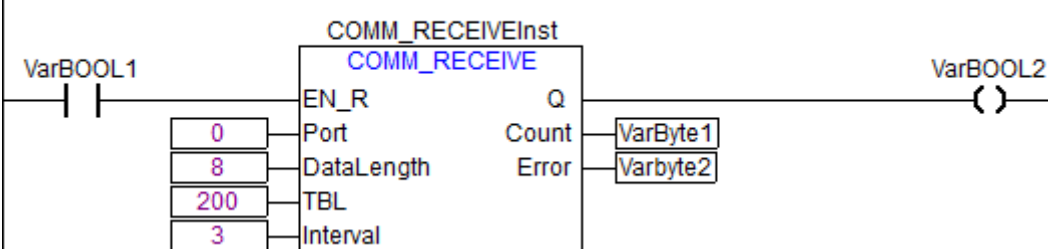
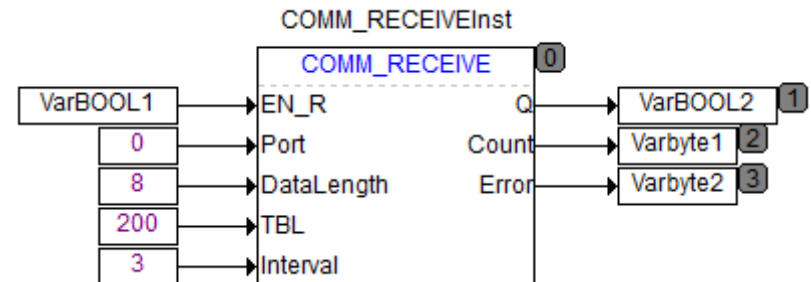
- 输出参数在 EN_R 为 FALSE（0）的状态时，会恢复成初始值 0。
- 当输入参数 DATALENGTH 等于 0，即采用时间断帧接收数据方式时 ERROR=2 的错误不会报出来，但是当断帧接收到数据长度和设定的 TBL 发生地址越界时会将本次接收数据舍弃掉，不会拷贝到指定的 M 区。

数据长度与断帧间隔时间的配合使用

DATALENGTH	INTERVAL	字符的接收过程
0	3~255 个字符任意数值	时间断帧接收数据方式，即以指定 INTERVAL 数值个字符传送时间作为断帧时间，断帧时间之内没有新数据接收到就接收本次接收过程
不为 0	3~255 个字符任意数值	指令有效即开始接收，直到接满 DATALENGTH 个字节，结束本次接收过程

3.3.1.20.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	Varbyte1			BYTE	0	FALSE
0003	COMM_RECEIVEInst			COMM...		FALSE
0004	Varbyte2			BYTE	0	FALSE
0005	VarBOOL2			BOOL	FALSE	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> COMM_RECEIVEInst(EN_R:=VarBOOL1, Port:=0, DataLength:=8, TBL:=200, Interval:=3, Q=>VarBOOL2, Count=>VarByte1, Error=>VarByte2); </pre>
连续功能图 (CFC)	

程序说明：

通过 Port 参数将通讯口设置为 CPU 本体的 PORT0，此时 CPU 的通讯参数设置中 PORT0 功能选择项应设置为<Free Port>，即自由口协议。

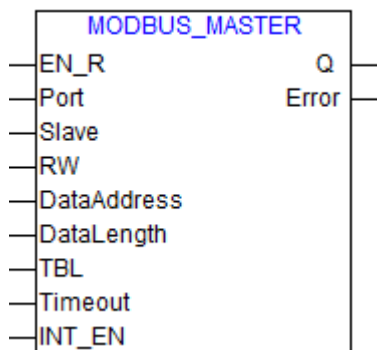
EN_R 置位并保持时，将准备接收 8 个字节数据分别保存到%MB200 的连续 8 个字节（即%MB200-%MB207）中，接收成功后 Q 等于 TRUE。

3.3.1.21 MODBUS_MASTER（Modbus RTU 主站通讯）

3.3.1.21.1 功能

提供实现串口 Modbus 协议的主站数据读写操作的用户接口。

在 MKC 和 LE 平台中，该功能块的引脚会有区别，详见参数说明。



MODBUS_MASTER 功能块

3.3.1.21.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值	备注
EN_R	BOOL	使能	0: 无效 1: 上升沿使能	0	
Port	BYTE	通讯口选择	LE 平台: 0: CPU 本体的圆形接口 1: CPU 本体端子通讯口 (14 点不支持) 2: 扩展功能板通讯口 (14 点不支持) 3: 第一个通讯扩展模块通讯口 4~N: 第二个到第 N 个通讯扩展模块通讯口 MKC 平台: 0: CPU 本体的 PORT0 1: CPU 本体的 PORT1 2: CPU 本体的 PORT2 (PORT2 口仅支持 MKC3167)	0	
Slave	BYTE	从站地址	Modbus 从站地址, 1~247	1	
RW	BOOL	读/写选择	0: 读取数据 1: 写入数据	0	
DataAddress	DWORD	从站存放数据的地址	详见表 DataAddress 参数详细说明		
DataLength	BYTE	数据长度	1~100, 对于开入/开出为所需要传输的总比特数。对于模入/模出为所需要传输的总通道数	1	
TBL	WORD	主站存放数据的首字节地址	指向 M 区数据存放的首字节地址 诸如 200, 表示存放地址为 %MW200 开始的一段空间。如果是读指令, 读回的数据值存放到这个数据区中, 例如 3 号从站 3050 地址的数据为 1000, 则 %MW200 存放 1000。如果是写指令, 要写出的数据值放到这个数据区中, 例如向 3 号从站的 3050 地址写入 500, 则 %MW200 存放 500	0	
Timeout	WORD	超时时间设置 (ms)	从启动接收过程开始计算, 在规定的时间内没有接收到正确的从站应答帧, 中止接收过程。最小时间单位为 50 ms	0	
INT_EN	BOOL	中断使能	0: 禁止中断 1: 中断使能 通讯口发送信息完成和接收信息完成各产生一个中断。中断列表中的	0	MKC 平台无此参数

输入参数	数据类型	功能描述	参数值说明	默认值	备注
			通信口中断 0 和通信口中断 1		

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	完成标志	接收完成后第一个扫描周期为 1
Error	BYTE	错误信息	Error=0: 正确 Error=1: 通讯口 (Port) 设置错误 Error=2: 超时时间 (Timeout) 设定过小 Error=3: 站地址 (Slave) 错 Error=4: 从站存放数据的地址 (DataAddress) 错 Error=5: 数据长度 (DataLength) 错 Error=6: 主站存放数据的地址 (TBL) 越界 Error=7: 功能码错误 Error=8: 获取用户空间指针失败 Error=9: 超时 Error=10: CRC 校验错 Error=11: 应答异常错 Error=12: 通讯口占用 (请检查该端口是否已用于其它协议方式或与 AT 通讯) Error=13: 多个功能块同时使能一个通讯口 Error=14: 强制应答异常错

DataAddress 参数详细说明

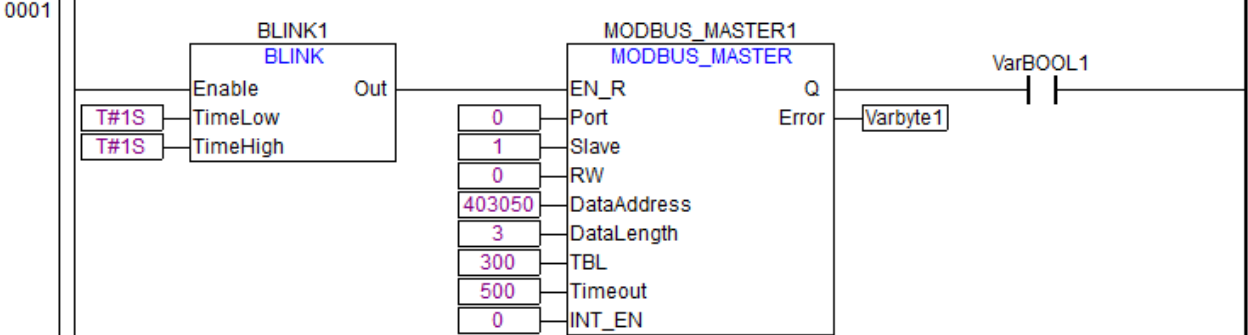
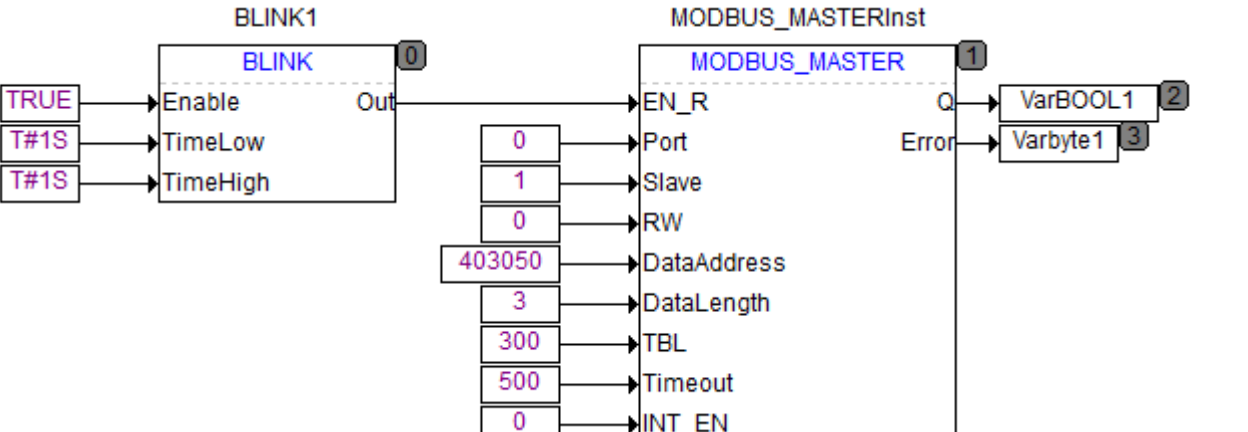
参数说明	
最高位	低五位
0：开出	Modbus 原始地址
1：开入	
3：模入	
4：模出	
从站的地址采用标准 Modbus RTU 驱动的地址填写方式，即采用六位数（十进制），第六位只能选择 0、1、3、4，分别表示 Modbus 不同的数据类型，低五位表示寄存器地址（地址范围 0~65535）。 例如：000001，表示 Modbus 地址为 0001 的开出点；403050，表示 Modbus 地址为 3050 的模出点。	

3.3.1.21.3 指令使用举例

以 LE 平台为例进行说明。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0003	BLINK1			BLINK		FALSE
0004	MODBUS_MASTERInst			MODBUS_MASTER		FALSE
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	Varbyte1			BYTE	0	FALSE
编程语言		程序				

变量定义

梯形图 (LD)	
结构化文本 (ST)	<pre> BLINK1(Enable:=TRUE, TimeLow:=T#1S, TimeHigh:=T#1S, Out=>MODBUS_MASTERInst.EN_R); MODBUS_MASTERInst(EN_R:= BLINK1.Out, Port:=0, Slave:=1, RW:=0, DataAddress:=403050 DataLength:=3, TBL:=300, Timeout:=500, INT_EN:=0 Q=>VarBOOL1, Error=>Varbyte1); </pre>
连续功能图 (CFC)	

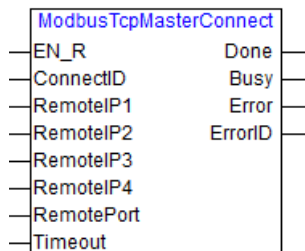
程序说明：

通过 BLINK 脉冲发生器制造方波脉冲，使 MODBUS RTU 主站功能块每 2 秒读取一次 1 号从站地址从 3050 开始的 3 路模拟量输出寄存器，并将得到的 6 个字节（每路模拟量占 2 个字节）存放在 %MW300 开始的地址中，通讯超时时间为 500ms。

3.3.1.22 ModbusTcpMasterConnect (Modbus TCP 主站连接)

3.3.1.22.1 功能

LE5118/LE5119 模块作为 Modbus 主站与从站通讯时，通过该功能块实现 Modbus TCP 通讯连接。



ModbusTcpMasterConnect 功能块

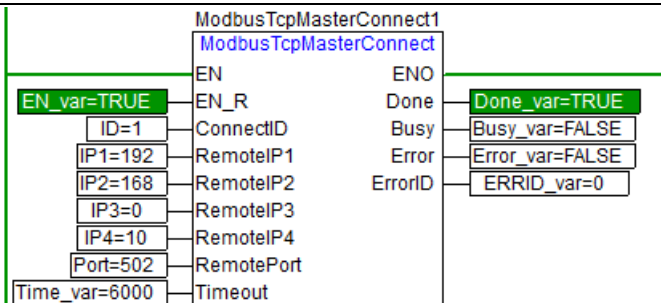
3.3.1.22.2 参数说明

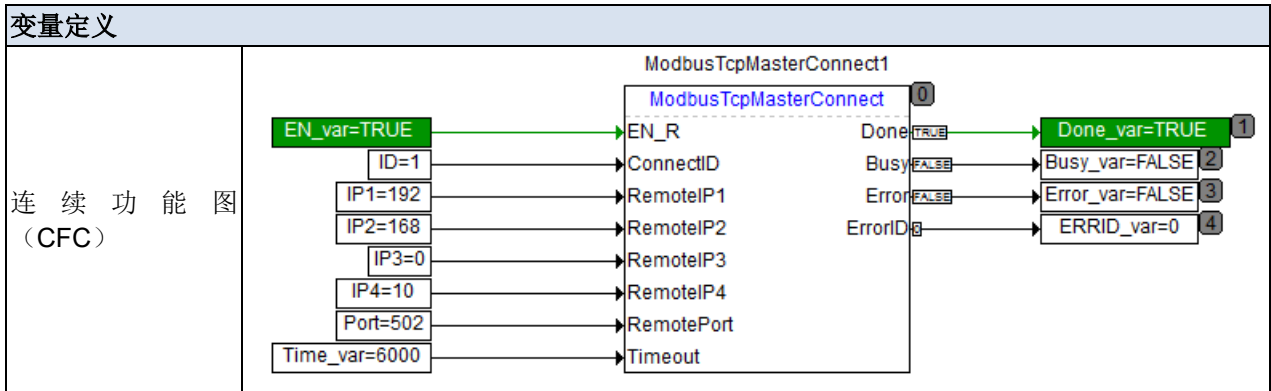
输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	1、上升沿时建立连接，发起连接请求后，修改输入参数不再生效 2、低电平复位，断开连接	FALSE
ConnectID	BYTE	Socket 连接编号	支持 10 路连接，编号取值：1~10，组态时，取值不重复即可	0
RemoteIP1	BYTE	从站 IP 地址	地址最高位	0
RemoteIP2	BYTE	从站 IP 地址	\	0
RemoteIP3	BYTE	从站 IP 地址	\	0
RemoteIP4	BYTE	从站 IP 地址	地址最低位	0
RemotePort	WORD	从站端口号	根据从站设备端口号设置，取值范围：1~49151	0
Timeout	WORD	连接超时时间	连接等待超时后，不再发起连接，设置值≥50ms	0

输出参数	数据类型	功能描述	参数值说明
Done	BOOL	连接完成标记	TRUE：连接成功 FALSE：未建立连接
Busy	BOOL	正在连接	TRUE：正在建立连接 FALSE：连接成功或连接失败
Error	BOOL	连接失败	TRUE：建立连接失败 FALSE：连接成功或正在连接
ErrorID	BYTE	连接失败错误码	=0：无错误 =1：ConnectID 非法，请检查 ConnectID 配置值是否是 1~10 =2：从站 IP 地址非法，请检查 IP 地址是否与从站地址一致 =3：从站端口号非法，请检查端口号是否与从站一致 =4：超时时间非法，请检查取值≥50ms =5：ConnectID 冲突，请检查 ConnectID 配置值是否已被使用 =6：连接超时，请检查从站是否异常或未在线 =7：连接中断，请检查从站是否异常或未在线 =8：连接失败，请检查从站是否异常或未在线 =9：系统错误，模块内部问题，请联系技术支持

3.3.1.22.3 指令使用举例

变量定义						
No.	Variable Name	Address	/variable Descriptor	Variable Type	Online Value	Power Fail Safeguard
0001	EN_var			BOOL	TRUE	FALSE
0002	ID			BYTE	1	FALSE
0003	IP1			BYTE	192	FALSE
0004	IP2			BYTE	168	FALSE
0005	IP3			BYTE	0	FALSE
0006	IP4			BYTE	10	FALSE
0007	Port			WORD	502	FALSE
0008	Time_var			WORD	6000	FALSE
0009	Done_var			BOOL	TRUE	FALSE
0010	Busy_var			BOOL	FALSE	FALSE
0011	Error_var			BOOL	FALSE	FALSE
0012	ERRID_var			BYTE	0	FALSE
0013	ModbusTcpMasterConnect1			MODBUSTCPMASTERCONNECT		TRUE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> ModbusTcpMasterConnect1(EN_R:=EN_var, ConnectID:=ID, RemoteIP1:=IP1, RemoteIP2:=IP2, RemoteIP3:=IP3, RemoteIP4:=IP4, RemotePort:=Port, Timeout:=Time_var, Done=>Done_var, Busy=>Busy_var, Error=>Error_var, ErrorID=>ERRID_var); </pre> <pre> ModbusTcpMasterConnect1 EN_var = TRUE ID = 1 IP1 = 192 IP2 = 168 IP3 = 0 IP4 = 10 Port = 502 Time_var = 6000 Done_var = TRUE Busy_var = FALSE Error_var = FALSE ERRID_var = 0 </pre>



程序说明：

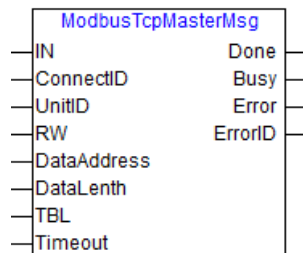
LE5119 作为 Modbus 主站与从站设备建立通讯连接。其中，从站设备 IP 地址：192.168.0.10，从站设备端口号为 502。组态时，我们设置连接 ID 为 1（取值范围 1~10），只要不与其他连接 ID 重复即可。超时时间设置为 6000ms。

在通讯过程中，如果长时间（具体时间各从站不同）没有数据传输，则从站自动断开与主站的连接。

3.3.1.23 ModbusTcpMasterMsg（Modbus TCP 主站消息）

3.3.1.23.1 功能

LE5118/LE5119 模块作为 Modbus 主站与从站通讯时，通过该功能块配置通讯参数和通讯数据。



ModbusTcpMasterMsg 功能块

3.3.1.23.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值	备注
IN	BOOL	使能	上升沿使能，高电平有效，即每次使能必须保持高电平至指令执行结束（成功或失败）；若指令未执行完成之前使能引脚变为 FALSE，则指令终止	0	
ConnectID	BYTE	Socket 连接编号	支持 10 路连接，编号取值：1~10。组态时，与 ModbusTcpMasterConnect 功能块的 ConnectID 值保持一致		
UnitID	BYTE	从站单元 ID	取值：1~247、255		

输入参数	数据类型	功能描述	参数值说明	默认值	备注
RW	BOOL	读/写选择	0: 读取数据 1: 写入数据	0	
DataAddress	DWORD	从站存放数据的地址	详见表 DataAddress 参数详细说明	1	
DataLength	BYTE	数据长度	1~100, 对于开入/开出为所需要传输的总比特数。对于模入/模出为所要传输的总通道数	1	
TBL	WORD	主站存放数据的首字节地址	指向数据存放区的首字节地址 模拟量数据: 诸如 200, 表示存放地址为%MW200 开始的一段空间。如果是读指令, 读回的数据值存放到这个数据区中, 例如 3 号从站 3050 地址的数据为 1000, 则%MW200 存放 1000。如果是写指令, 要写出的数据值放到这个数据区中, 例如向 3 号从站的 3050 地址写入 500, 则%MW200 存放 500 开关量数据: 诸如 100, 表示存放地址为%MX100.0 开始的一段空间。如果是读指令, 读回的数据值存放到这个数据区中, 例如 3 号从站 3050 地址的数据为 1, 则%MX100.0 存放 1。如果是写指令, 要写出的数据值放到这个数据区中, 例如向 3 号从站的 3050 地址写入 0, 则%MX100.0 存放 0	0	
Timeout	WORD	主站请求响应超时 (ms)	从启动接收过程开始计算, 在规定的时间内没有接收到正确的从站应答帧, 中止接收过程。最小时间为 50 ms	0	

输出参数	数据类型	功能描述	参数值说明
Done	BOOL	通信成功标记	TRUE: 本次通信成功 FALSE: 本次通信失败或正在等待应答
Busy	BOOL	正在响应	TRUE: 正在等待从站应答 FALSE: 通信结束
Error	BOOL	通信失败	TRUE: 本次通信失败 FALSE: 本次通信成功或等待从站应答
ErrorID	BYTE	通信失败错误码	=0: 无错误 =1: ConnectID 非法, 检查 ConnectID 配置值是否是 1~10 =2: 单元 ID 非法, 请检查 ID 取值是否是 1~247、255 =3: 从站存放数据的地址 (DataAddress) 非法, 请检查从站数据地址是否满足表 "DataAddress 参数详细说明" 要求 =4: 数据长度 (DataLength) 非法, 请检查数据长度是否是 1~100 =5: 主站存放数据的地址 (TBL) 越界, 请检查主站地址≥16K =6: 超时时间非法, 请检查取值≥50ms =7: 地址+长度越界: 请检查地址+长度≥16K =8: ConnectID 冲突, 请检查 ConnectID 配置值是否已被使用 =9: 发送数据失败 =10: 接收数据失败 =11: 超时 =12: 从站应答帧错误 (MBAP 头校验失败) =13: 从站应答数据长度错误 =14: 从站应答地址错误 (写指令) =15: 从站应答数据错误 (写单个线圈/寄存器指令) =16: 连接中断, 请检查从站是否异常或未在线 =17: 系统错误, 模块内部问题, 请联系技术支持 =129: 从站不支持的功能码, 请与从站配置保持一致

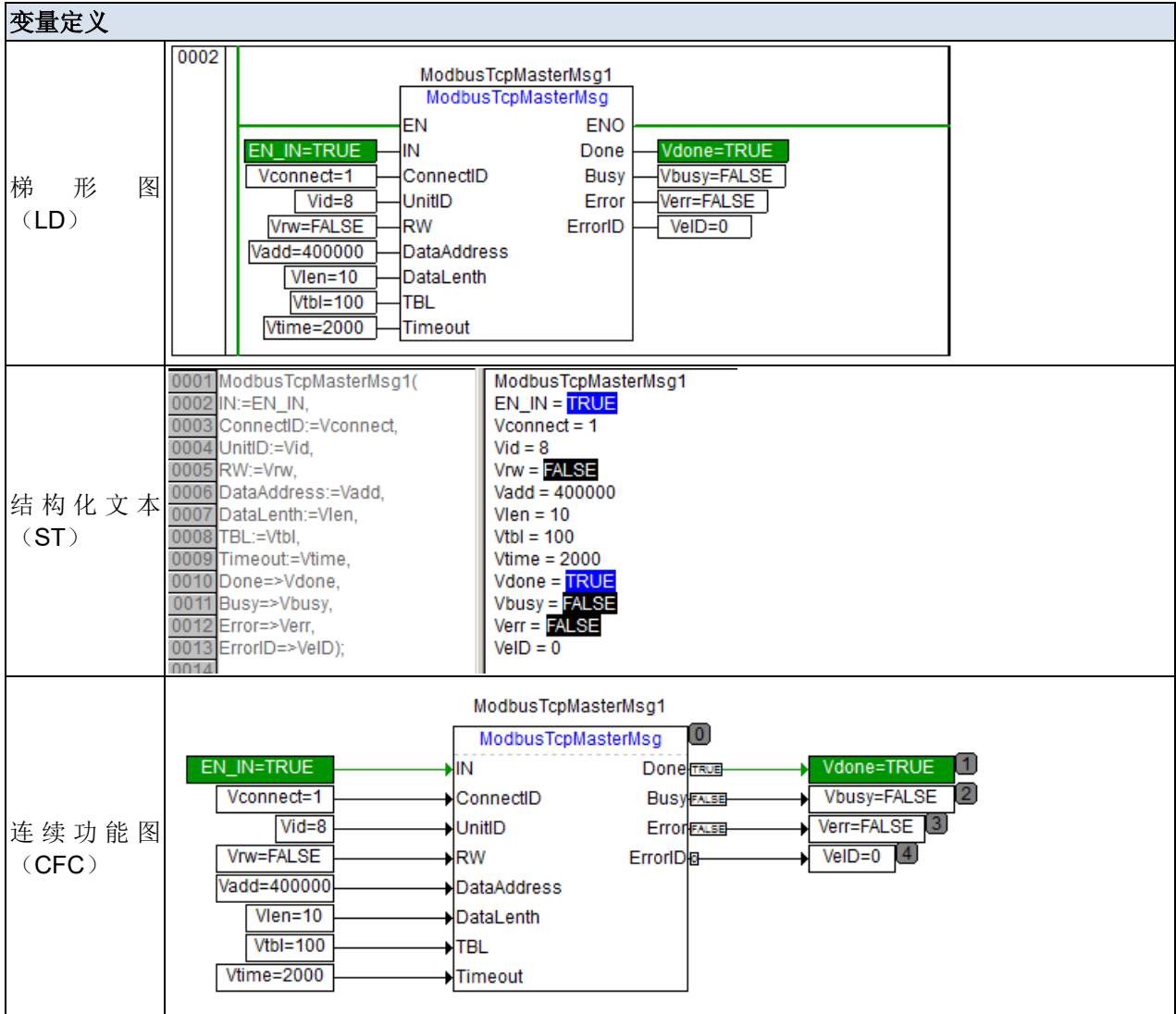
输出参数	数据类型	功能描述	参数值说明
			=130: 从站不支持的地址, 请与从站配置保持一致 =131: 从站不支持的数据, 请与从站配置保持一致 =132: 从站其他类型错误, 请与从站配置保持一致

DataAddress 参数详细说明

参数说明	
最高位	低五位
0: 开出	Modbus 原始地址
1: 开入	
3: 模入	
4: 模出	
从站的地址采用标准 Modbus TCP 驱动的地址填写方式，即采用六位数（十进制），第六位只能选择 0、1、3、4，分别表示 Modbus 不同的数据类型，低五位表示寄存器地址（地址范围 0~65535）。 例如：000001，表示 Modbus 地址为 0001 的开出点；403050，表示 Modbus 地址为 3050 的模出点。	

3.3.1.23.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	ModbusTcpMasterMsg1		功能块实例	MODBUSTCPMASTERMSG		FALSE
0002	EN_IN		使能信号	BOOL	FALSE	FALSE
0003	Vconnect		连接ID	BYTE	1	FALSE
0004	Vid		从站单元ID	BYTE	8	FALSE
0005	Vrw		读写标识	BOOL	FALSE	FALSE
0006	Vadd		从站数据地址	DWORD	400000	FALSE
0007	Vlen		数据长度	BYTE	10	FALSE
0008	Vtbl		主站存放地址	WORD	100	FALSE
0009	Vtime		超时时间	WORD	2000	FALSE
0010	Vdone		通信成功标识	BOOL	FALSE	FALSE
0011	Vbusy		正在响应标识	BOOL	FALSE	FALSE
0012	Verr		通信失败标识	BOOL	FALSE	FALSE
0013	VeID		通信错误码	BYTE	0	FALSE
0014	p1	%MW100		ARRAY[0..20] OF WORD		FALSE
编程语言		程序				



程序说明：

LE5119 作为 Modbus 主站与从站设备建立连接后，当通讯使能信号 IN 为 TRUE 时，即可执行读写操作。

上面的例子中，主站读取从站“读保持寄存器”数据（功能码 0x03），从站数据地址为 0。读取 10 个数据，存放主站 %MW100 开始的存储区。组态时，ConnectID 与 ModbusTcpMasterConnect（Modbus TCP 主站连接）功能块中设置值保持一致。

以下是从站数据存放示意图和主站读取到的数据存放示意图。

Modbus Slave - Mbslave1

File Edit Connection Setup Display View Window Help

Mbslave1

ID = 8: F = 03

	Alias	00000
0		10
1		10
2		10
3		20
4		20
5		20
6		8
7		8
8		8
9		0

从站存放的数据

CFC_DATA.p1

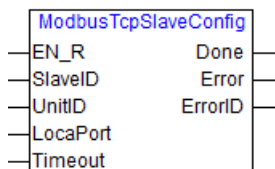
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	p1[0]	%MW100		WORD	10	FALSE
0002	p1[1]	%MW102		WORD	10	FALSE
0003	p1[2]	%MW104		WORD	10	FALSE
0004	p1[3]	%MW106		WORD	20	FALSE
0005	p1[4]	%MW108		WORD	20	FALSE
0006	p1[5]	%MW110		WORD	20	FALSE
0007	p1[6]	%MW112		WORD	8	FALSE
0008	p1[7]	%MW114		WORD	8	FALSE
0009	p1[8]	%MW116		WORD	8	FALSE
0010	p1[9]	%MW118		WORD	0	FALSE
0011	p1[10]	%MW120		WORD	0	FALSE
0012	p1[11]	%MW122		WORD	0	FALSE
0013	p1[12]	%MW124		WORD	0	FALSE
0014	p1[13]	%MW126		WORD	0	FALSE
0015	p1[14]	%MW128		WORD	0	FALSE
0016	p1[15]	%MW130		WORD	0	FALSE

读取到主站中的数据

3.3.1.24 ModbusTcpSlaveConfig (Modbus TCP 从站配置)

3.3.1.24.1 功能

LE5118/LE5119 模块作为 Modbus 从站被主站访问时，通过该功能块实现从站通讯参数配置。每调用一次，可支持 5 路主站同时访问。不调用时，功能块参数将采用默认设置值进行工作。



ModbusTcpSlaveConfig 功能块

3.3.1.24.2 参数说明

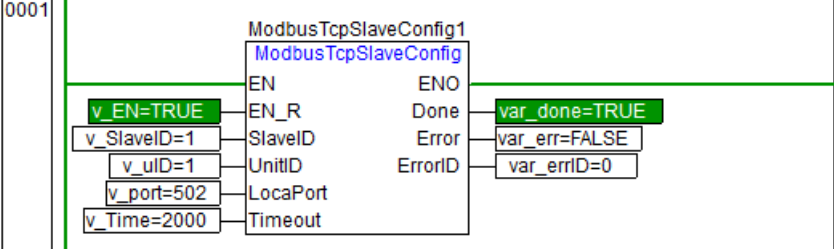
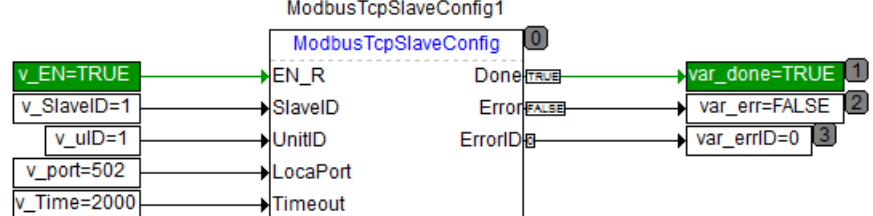
输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	上升沿进行参数配置	FALSE
SlaveID	BYTE	从站 ID	该参数值为固定值 1	0
UnitID	BYTE	单元 ID	可设范围: 1~247、255	0
LocalPort	WORD	本地端口号	可设范围: 1~49151	0
Timeout	DWORD	超时时间 (ms)	第三方主站的最小访问间隔，超过该间隔时间主动断开与第三方主站连接 (为 0 时，从站不主动断连接)	0

注：当不调用 ModbusTcpSlaveConfig (Modbus TCP 从站配置) 功能块时，系统使用默认配置 SlaveID=1, UnitID=1, LocalPort=502, Timeout=2000ms 作为从站通讯参数值。

输出参数	数据类型	功能描述	参数值说明
Done	BOOL	配置完成标记	TRUE: 配置成功 FALSE: 配置失败
Error	BOOL	错误标记	TRUE: 配置失败 FALSE: 未使能或配置成功
ErrorID	BYTE	错误码	=0: 无错误 =1: SlaveID 非法，请检查 SlaveID 是否为 1 =2: 本地单元 ID 设置非法，请检查 ID 是否为: 1~247、255 =3: 本地端口号设置非法，请检查端口配置是否为: 1~49151 =4: IP 地址设置非法，非法地址: 0.0.0.0、255.255.255.255、127.0.0.1 =5: 参数被初始化，请重新配置 =6: 系统错误，模块内部问题，请联系技术支持

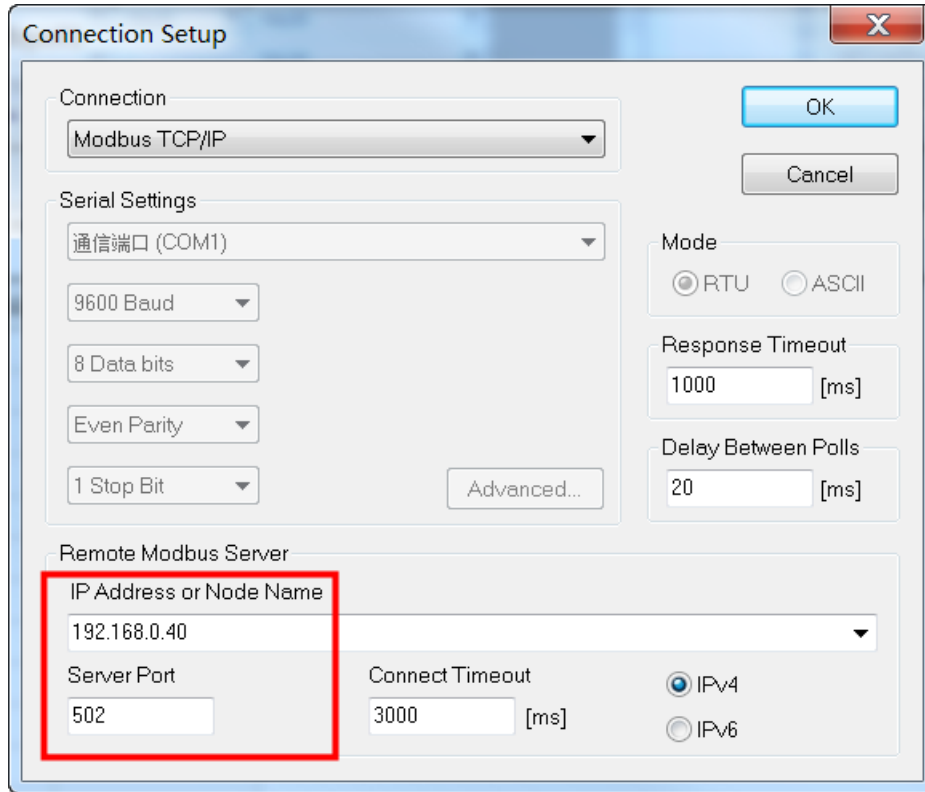
3.3.1.24.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0007	ModbusTcpSlaveConfig1		功能块实例	MODBUSTCPSLAVECONFIG		FALSE
0008	v_EN		使能信号	BOOL	TRUE	FALSE
0009	v_SlaveID		从站ID	BYTE	1	FALSE
0010	v_uID		单元ID	BYTE	1	FALSE
0011	v_port		端口号	WORD	502	FALSE
0012	v_Time		超时时间	DWORD	2000	FALSE
0013	var_done		完成标识	BOOL	TRUE	FALSE
0014	var_err		错误标识	BOOL	FALSE	FALSE
0015	var_errID		错误码	BYTE	0	FALSE
0016	p1	%MW0	从站存储数据1	ARRAY[0..10] OF WORD		FALSE
0017	p3	%MX100.0	从站存储数据2	ARRAY[0..10] OF BOOL		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> 0001 ModbusTcpSlaveConfig1(0002 EN_R:=v_EN, 0003 SlaveID:=v_SlaveID, 0004 UnitID:=v_uID, 0005 LocaPort:=v_port, 0006 Timeout:=v_Time, 0007 Done=>var_done, 0008 Error=>var_err, 0009 ErrorID=>var_errID); </pre>
连续功能图 (CFC)	

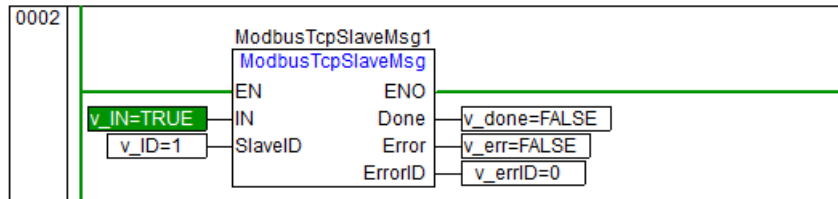
程序说明：

LE5119 作为 Modbus 从站与主站建立通讯连接。SlaveID 设置为固定值 1，UnitID 设置为 1，主站中该参数与其保持一致。从设备 IP 地址 192.168.0.40，端口号为 502，需要在主站中配置该 IP 地址和端口号，如下图所示。当使能信号 EN_R 为 TRUE 时，与主站建立通讯连接。主站可随时读写数据。



主站中配置从站 IP 和端口号

当主从站建立通讯连接后，通过功能块 **ModbusTcpSlaveMsg**（Modbus TCP 从站数据）启动读写操作。如下图所示，当通讯使能信号 **IN** 为 **TRUE** 时，主站开始读写数据。



触发读写操作示例

上面的例子中，主站读取从站“线圈状态”数据（功能码 0x01），从站数据存放在 %MX100.0 开始的数组中，如下图所示。

0017	p1	%MW0	从站存储数据1	ARRAY[0..10] OF WORD		FALSE
0018	p3	%MX100.0	从站存储数据2	ARRAY[0..10] OF BOOL		FALSE

slave_c: 请在此处添加

slave_c.p3

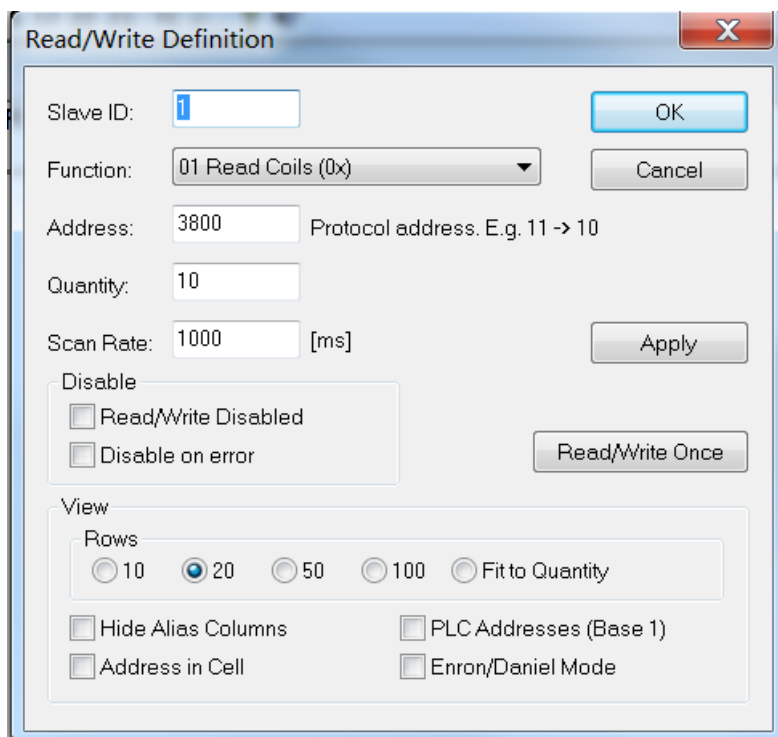
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	p3[0]	%MX100.0		BOOL	TRUE	FALSE
0002	p3[1]	%MX100.1		BOOL	TRUE	FALSE
0003	p3[2]	%MX100.2		BOOL	TRUE	FALSE
0004	p3[3]	%MX100.3		BOOL	TRUE	FALSE
0005	p3[4]	%MX100.4		BOOL	FALSE	FALSE
0006	p3[5]	%MX100.5		BOOL	FALSE	FALSE
0007	p3[6]	%MX100.6		BOOL	FALSE	FALSE
0008	p3[7]	%MX100.7		BOOL	FALSE	FALSE
0009	p3[8]	%MX101.0		BOOL	FALSE	FALSE
0010	p3[9]	%MX101.1		BOOL	FALSE	FALSE
0011	p3[10]	%MX101.2		BOOL	FALSE	FALSE

从站数据

数据区与 MODBUS TCP 从通讯协议的映射关系

数据区	类型	地址范围	MODBUS 地址	映射关系	X (寄存器类型) 选择
I 区	%IX	BOOL %IX0.0、...%IX0.7 %IX1.0、...%IX1.7 ...%IX374.7	X0000、...X0007 X0008、...X0015 ...X2999	IXm.n: m*8+n	只读, X 选 1
	%IW	WORD %IW0、%IW2、...%IW5998	X0000、X0001、...X2999	IWm: m/2	只读, X 选 3
Q 区	%QX	BOOL %QX0.0、...%QX0.7 %QX1.0、...%QX1.7 ...%QX374.7	X0000、...X0007 X0008、...X0015 ...X2999	QXm.n: m*8+n	读写, X 选 0
	%QW	WORD %QW0、%QW2、...%QW5998	X0000、 X0001、...X2999	QWm: m/2	读写, X 选 4
M 区	%MX	BOOL %MX0.0、...%MX0.7 %MX1.0、...%MX1.7 ...%MX7816.7	X3000、...X3007 X3008、...X3015 ...X65535	MXm.n: m*8+n+3000	只读, X 选 1 读写, X 选 0
	%MW	WORD %MW0、%MW2、...%MW125070	X3000、 X3001、...X65535	MWm: m/2+3000	只读, X 选 3 读写, X 选 4

在主站中配置通讯数据，如下图所示。Slave ID 号与 ModbusTcpSlaveConfig 功能块的 UnitID 保持一致。由上表中 M 区对应的映射关系，在主站中配置该读写从站的起始地址为 3800。读取该地址开始的 10 个数据，存放主站。



Read/Write Definition

Slave ID: OK Cancel

Function: Protocol address. E.g. 11 -> 10

Address:

Quantity:

Scan Rate: [ms] Apply

Disable

☐ Read/Write Disabled

☐ Disable on error Read/Write Once

View

Rows

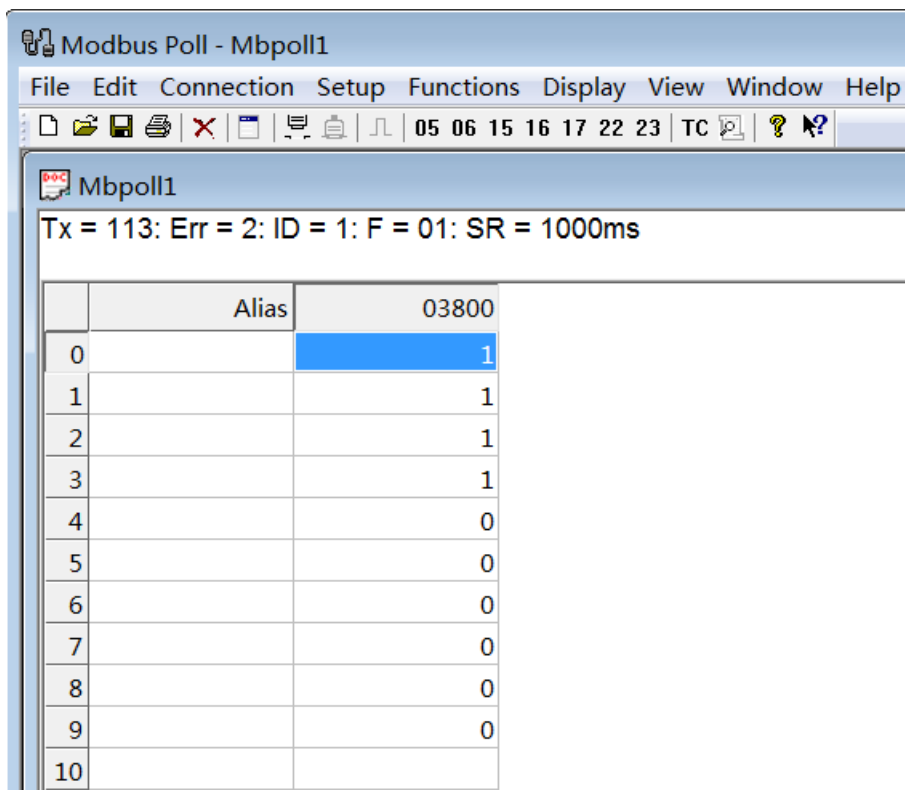
☐ 10 ☒ 20 ☐ 50 ☐ 100 ☐ Fit to Quantity

☐ Hide Alias Columns ☐ PLC Addresses (Base 1)

☐ Address in Cell ☐ Enron/Daniel Mode

配置读写数据

当 ModbusTcpSlaveMsg 功能块的通讯使能信号 IN 为 TRUE 时，主站读取从站%MX100.0 开始的 10 个 BOOL 数据，如下图所示。



Modbus Poll - Mbpoll1

File Edit Connection Setup Functions Display View Window Help

05 06 15 16 17 22 23 TC ? ?

Mbpoll1

Tx = 113: Err = 2: ID = 1: F = 01: SR = 1000ms

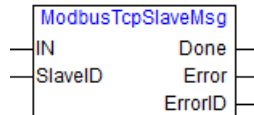
	Alias	03800
0		1
1		1
2		1
3		1
4		0
5		0
6		0
7		0
8		0
9		0
10		

主站读取到的数据

3.3.1.25 ModbusTcpSlaveMsg (Modbus TCP 从站消息)

3.3.1.25.1 功能

LE5118/LE5119 模块作为 Modbus 从站被主站访问时，通过该功能块配置通讯数据。



ModbusTcpSlaveMsg 功能块

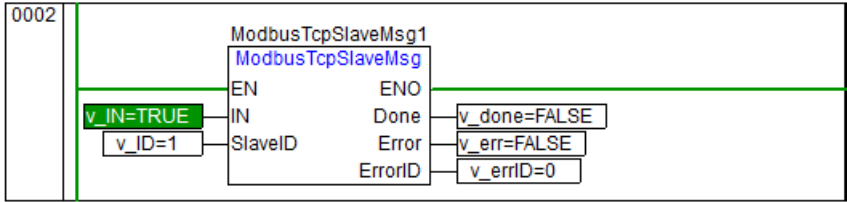
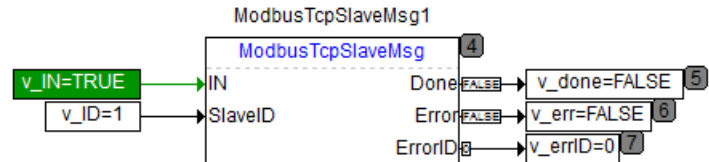
3.3.1.25.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
IN	BOOL	使能	TRUE: 进入监听状态，响应主站连接请求及读写请求 FALSE: 主动断开和第三方主站的连接	FALSE
SlaveID	BYTE	从站 ID	当前固定 1	0

输出参数	数据类型	功能描述	参数值说明
Done	BOOL	完成标记	当前 IEC 周期一条指令处理完成
Error	BOOL	通信错误	\
ErrorID	BYTE	错误码	=0: 无错误 =1: SlaveID 非法，请检查 SlaveID 是否为 1 =2: SlaveID 冲突，请检查 SlaveID 值已被使用 =3: 系统错误，模块内部问题，请联系技术支持 =4: UnitID 非法，请检查主从站 UnitID 是否一致 =5: 长度非法 =6: 数据非法 =7: 接收数据失败 =8: 数据帧非法 =9: 发送数据失败 =10: 连接失败，请检查从站服务是否异常

3.3.1.25.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	v_IN		使能信号	BOOL	FALSE	FALSE
0002	v_ID		从站ID	BYTE	1	FALSE
0003	v_done		完成标记	BOOL	FALSE	FALSE
0004	v_err		错误标记	BOOL	FALSE	FALSE
0005	v_errID		错误码	BYTE	0	FALSE
0006	ModbusTcpSlaveMsg1		功能块实例	MODBUSTCPSLAVMSG		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> 0011 ModbusTcpSlaveMsg1(0012 IN:=v_IN, 0013 SlaveID:=v_ID, 0014 Done=>v_done, 0015 Error=>v_err, 0016 ErrorID:=v_errID); 0017 ModbusTcpSlaveMsg1 v_IN = TRUE v_ID = 1 v_done = FALSE v_err = FALSE v_errID = 0 </pre>
连续功能图 (CFC)	

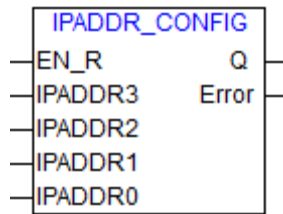
程序说明：

LE5119 作为 Modbus 从站与主站建立连接后，当通讯使能信号 IN 为 TRUE 时，主站开始读写数据。
详细的示例参见[指令使用举例](#)。

3.3.1.26 IPADDR_CONFIG(设置 IP 地址)

3.3.1.26.1 功能

IP 地址配置指令可以实现在线修改 CPU 模块的以太网 IP 地址。



IPADDR_CONFIG 指令模块示意图

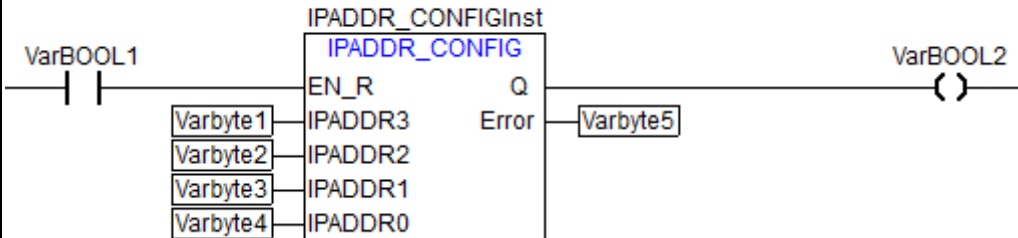
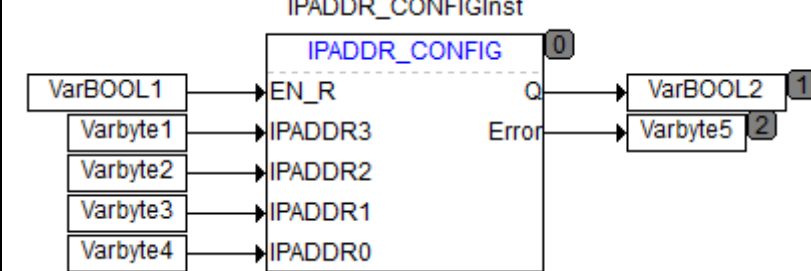
3.3.1.26.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0→1 上升沿使能，电平保持	0
IPADDR3	BYTE	IP 地址第 4 字节	0~255	0
IPADDR2	BYTE	IP 地址第 3 字节	0~255	0
IPADDR1	BYTE	IP 地址第 2 字节	0~255	0
IPADDR0	BYTE	IP 地址第 1 字节	0~255	0

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	配置完成标志	配置完成标志，配置完成输出 1，否则输出 0
ERROR	BYTE	错误信息	=0: IP 地址配置成功 =1: IP 地址全零错误 =2: IP 地址配置失败

3.3.1.26.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	IPADDR_CONFIGInst			IPAD...		TRUE
0002	VarBOOL1			BOOL	FALSE	TRUE
0003	Varbyte1			BYTE	192	TRUE
0004	VarByte2			BYTE	168	TRUE
0005	Varbyte3			BYTE	1	TRUE
0006	Varbyte4			BYTE	90	TRUE
0007	VarBOOL2			BOOL	FALSE	FALSE
0008	Varbyte5			BYTE	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>IPADDR_CONFIGInst(EN_R:=VarBOOL1, IPADDR3:=Varbyte1, IPADDR2:=Varbyte2, IPADDR1:=Varbyte3, IPADDR0:=Varbyte4, Q=>VarBOOL2, Error=> Varbyte5);</pre>
连续功能图 (CFC)	

程序说明：

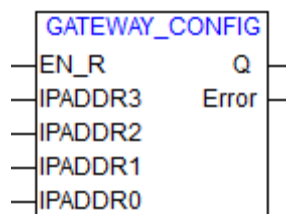
EN_R 置位并保持时，将 IPADDR3(192)、IPADDR2 (168)、IPADDR1 (1)、IPADDR0 (90)组成的 IP 地址 192.168.1.90 设置成功，VarBOOL1 非掉电保持时，在本次在线运行设置成功后生效，控制器掉电上电、冷复位运行后若不使能 EN_R 端，该 IP 不再生效。

VarBOOL1 为掉电保持时，掉电上电运行后，由于 EN_R 端掉电保持使能，程序再次运行又设置为程序中的 IP。

3.3.1.27 GATEWAY_CONFIG(设置网关)

3.3.1.27.1 功能

网关地址配置指令可以实现在线修改 CPU 模块的以太网网关 IP 地址。



GATEWAY_CONFIG 指令模块示意图

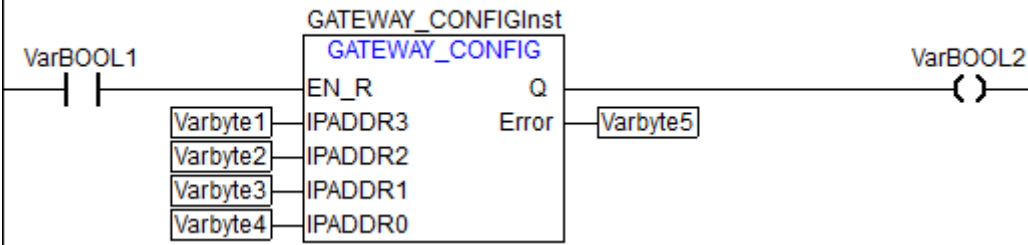
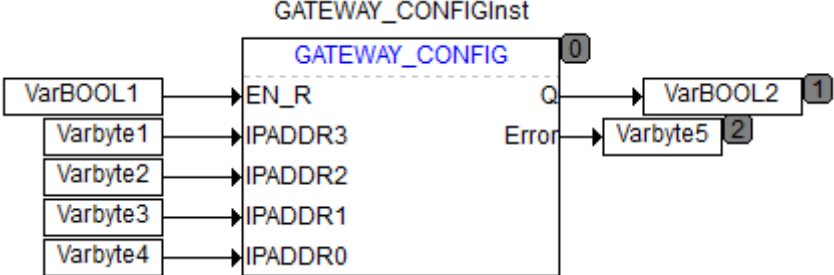
3.3.1.27.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0→1 上升沿使能，电平保持	0
IPADDR3	BYTE	网关 IP 地址第 4 字节	0~255	0
IPADDR2	BYTE	网关 IP 地址第 3 字节	0~255	0
IPADDR1	BYTE	网关 IP 地址第 2 字节	0~255	0
IPADDR0	BYTE	网关 IP 地址第 1 字节	0~255	0

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	配置完成标志	配置完成标志，配置完成输出 1，否则输出 0
ERROR	BYTE	错误信息	=0: 网关 IP 地址配置成功 =1: 网关 IP 地址全零错误 =2: 网关 IP 地址配置失败

3.3.1.27.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	GATEWAY_CONFIGInst			GATEWAY_CONFIG ▾		TRUE ▾
0002	VarBOOL1			BOOL ▾	FALSE ▾	TRUE ▾
0003	Varbyte1			BYTE ▾	192	TRUE ▾
0004	VarByte2			BYTE ▾	168	TRUE ▾
0005	Varbyte3			BYTE ▾	1	TRUE ▾
0006	Varbyte4			BYTE ▾	1	TRUE ▾
0007	VarBOOL2			BOOL ▾	FALSE ▾	FALSE ▾
0008	Varbyte5			BYTE ▾	0	FALSE ▾
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>GATEWAY_CONFIGInst(EN_R:=VarBOOL1, IPADDR3:=Varbyte1, IPADDR2:=Varbyte2, IPADDR1:=Varbyte3, IPADDR0:=Varbyte4, Q=>VarBOOL2, Error=>Varbyte5);</pre>
连续功能图 (CFC)	

程序说明：

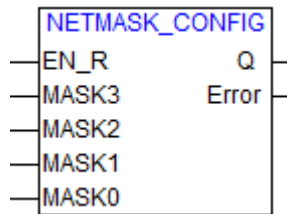
EN_R 置位并保持时，将 IPADDR3(192)、IPADDR2 (168)、IPADDR1 (1)、IPADDR0 (1)组成的网关地址 192.168.1.1 设置成功，VarBOOL1 非掉电保持时，在本次在线运行设置成功后生效，控制器掉电上电、冷复位运行后若不使能 EN_R 端，该网关 IP 地址不再生效。

VarBOOL1 为掉电保持时，掉电上电运行后，由于 EN_R 端掉电保持使能，程序再次运行又设置为程序中的网关 IP 地址。

3.3.1.28 NETMASK_CONFIG(设置子网掩码)

3.3.1.28.1 功能

子网掩码配置指令可以实现在线修改 CPU 模块的以太网子网掩码长度。



NETMASK_CONFIG 指令模块示意图

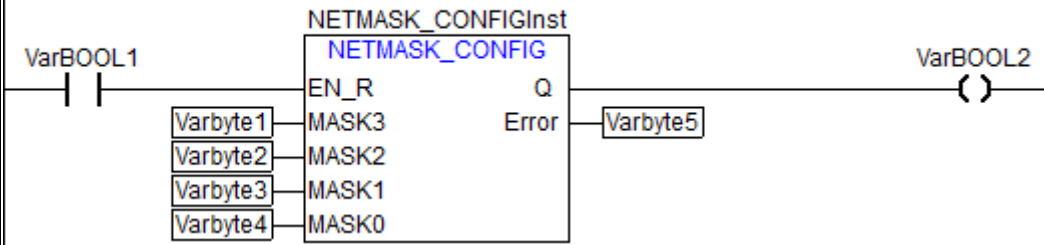
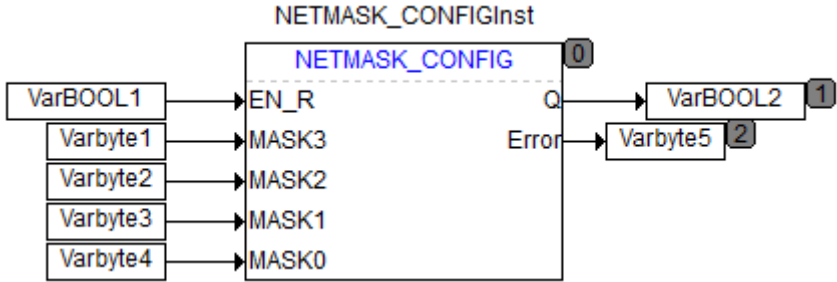
3.3.1.28.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0→1 上升沿使能，电平保持	0
MASK3	BYTE	掩码第 4 字节	0~255	0
MASK2	BYTE	掩码第 3 字节	0~255	0
MASK1	BYTE	掩码第 2 字节	0~255	0
MASK0	BYTE	掩码第 1 字节	0~255	0

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	配置完成标志	配置完成标志，配置完成输出 1，否则输出 0
ERROR	BYTE	错误信息	=0: 掩码配置成功 =1: 掩码全零错误 =2: 掩码配置失败

3.3.1.28.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	NETMASK_CONFIGInst			NETMASK_CONFIG		TRUE
0002	VarBOOL1			BOOL	FALSE	TRUE
0003	Varbyte1			BYTE	255	TRUE
0004	VarByte2			BYTE	255	TRUE
0005	Varbyte3			BYTE	255	TRUE
0006	Varbyte4			BYTE	0	TRUE
0007	VarBOOL2			BOOL	FALSE	FALSE
0008	Varbyte5			BYTE	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>NETMASK_CONFIGInst(EN_R:=VarBOOL1, MASK3:=Varbyte1, MASK2:=Varbyte2, MASK1:=Varbyte3, MASK0:=Varbyte4, Q=>VarBOOL2, Error=>Varbyte5);</pre>
连续功能图 (CFC)	

程序说明：

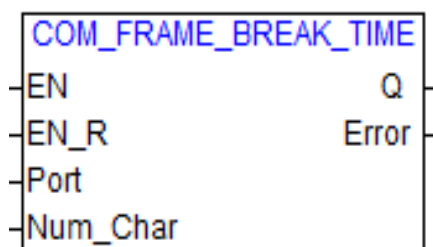
EN_R 置位并保持时，将 MASK3 (255)、MASK2 (255)、MASK1 (255)、MASK0 (0)组成的掩码地址 255.255.255.0 设置成功，VarBOOL1 非掉电保持时，在本次在线运行设置成功后生效，控制器掉电上电、冷复位运行后若不使能 EN_R 端，该子网掩码不再生效。

VarBOOL1 为掉电保持时，掉电上电运行后，由于 EN_R 端掉电保持使能，程序再次运行又设置为程序中的子网掩码。

3.3.1.29 COM_FRAME_BREAK_TIME（通讯口自定义断帧）

3.3.1.29.1 功能

实现 Modbus RTU 和自由口接收数据的断帧间隔，支持自定义 modbus RTU 通讯数据为设置。



通讯口自定义断帧

-  适用于 LE 系统中 LE5107E-F01、LE5118-B01 和 LE5119-B01 及以上版本。

3.3.1.29.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0: 无效 1: 上升沿使能	0
Port	BYTE	通讯口选择	LE 平台: 0: CPU 本体的圆形接口 1: CPU 本体端子通讯口 (14 点不支持) 2: 扩展功能板通讯口 (14 点不支持)	0
Num_Char	BYTE	自定义断帧字节数	自定义断帧数选择: 0~255。 0~3: 选择端口默认的 3.5char 断帧时间。 4~255: 设置为相应字节的断帧时间。	0

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	完成标志	接收完成后第一个扫描周期为 1	0
Error	BYTE	错误信息	Error=0: 正确 Error =1: 通讯口 (Port) 设置错误 Error =2: 选择端口 2 配置 485 断帧数时, 没有配置 LE5601 Error =3: 波特率 (BaudRate) 错误	0

协议栈公式: 断帧时间 (ms) = (断帧字节数*1000*单字节位数) /波特率

其中, 单字节位数 = 校验位+停止位+数据位+起始位。

校验方式:

- 校验位: 无校验: 0 位; 奇/偶校验: 1 位;
- 停止位: 0, 1, 2 位;
- 数据位: 7, 8 位;
- 起始位: 1 位。

3.3.1.29.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	COM_FRAME_BREAK_TIME1			COM_FRAME_BREAK_TIME		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	Varbyte			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	CHAR: 请在此处添加注释...
结构化文 本 (ST)	<pre>COM_FRAME_BREAK_TIME1(EN_R:=VarBOOL1, Port1:=1, Num_Char:=10, Q=>VarBOOL2, Error=>Varbyte1);</pre>
连续功能 图 (CFC)	

3.3.2 任务指令

3.3.2.1 TaskStart（运行任务）

3.3.2.1.1 函数原型

DINT TaskStart (UDINT uiTaskID);

3.3.2.1.2 功能描述

运行指定的任务。

3.3.2.1.3 参数描述

	数据项	数据类型	参数描述
输入端	uiTaskID	UDINT	任务 ID 号
输出端	TaskStart	DINT	成功返回 0；失败返回-1

❗ LK220 不支持该指令。

3.3.2.2 TaskStop（停止任务）

3.3.2.2.1 函数原型

DINT TaskStop (UDINT uiTaskID);

3.3.2.2.2 功能描述

停止指定的任务。

3.3.2.2.3 参数描述

	数据项	数据类型	参数描述
输入端	uiTaskID	UDINT	任务 ID 号
输出端	TaskStop	DINT	成功返回 0；失败返回-1

❗ LK220 不支持该指令。

3.3.2.3 TaskSuspend（挂起任务）

3.3.2.3.1 函数原型

DINT TaskSuspend (UDINT uiTaskID);

3.3.2.3.2 功能描述

挂起指定的任务。

3.3.2.3.3 功能描述

	数据项	数据类型	参数描述
输入端	uiTaskID	UDINT	任务 ID 号
输出端	TaskSuspend	DINT	成功返回 0；失败返回-1

i LK220 不支持该指令。

3.3.2.4 TaskStatus（返回任务状态）

3.3.2.4.1 函数原型

DINT TaskStatus (UDINT uiTaskID);

3.3.2.4.2 功能描述

返回任务的当前状态。

3.3.2.4.3 参数描述

	数据项	数据类型	参数描述
输入端	uiTaskID	UDINT	任务 ID 号
输出端	TaskStatus	DINT	任务状态： 运行 1 挂起 2 停止 3 失败 -1

i LK220 不支持该指令。

3.3.3 文件操作

3.3.3.1 文件操作

i

- 文件操作指令仅 MC 平台支持。

- “/media/sd/”、“/media/flash/”目录方式仅在 V1.1.0_R 之前的固件版本中适用，固件版本 V1.1.0_R 及以后版本上，目录方式为：“sd/”、“flash/”。关于控制器固件版本信息详见《MC 系列运动控制器指令手册》的附录 4 控制器固件版本信息。
- 对“sd/”目录下的文件进行打开（FileOpen）、查找（FindFirstFile、FindNextFile）、扫描（FileListScan）、删除（FileRemove）等操作前，请确保 SD 卡已正确插入卡槽。

3.3.3.2 FileOpen（打开文件）

3.3.3.2.1 函数原型

DINT FileOpen (STRING Name, UDINT uiAccess);

3.3.3.2.2 功能描述

根据用户设定的方式，打开文件。

3.3.3.2.3 参数说明

	数据项	数据类型	参数描述
输入端	Name	STRING(63)	文件名，最大支持 63 个字符，且只支持全路径方式 “sd/”为 SD 卡目录 “flash/”为 Flash 目录
	uiAccess	UDINT	文件操作方式 0: 读写（默认以读写方式打开文件） 1: 只读（以只读方式打开文件） 2: 只写（以只写方式打开文件） 8: 创建（创建并打开文件） 16: 清空（清除文件中的内容并打开文件） 32: 追加（将光标定位到文件末尾并打开文件） 注意： 1) 除了追加方式，其他方式都会将光标定位到文件头 2) 写入操作会对光标后的内容进行覆盖 3) 通过 FileLseek 指令可以移动光标
输出端	FileOpen	DINT	成功:返回文件访问 ID 失败:返回-1



- FileOpen 打开文件执行读写操作完成后，需要进行 [FileClose](#) 关闭操作，否则会造成资源无法释放。
- 文件命名规则受文件系统限制，SD 卡采用 FAT32 文件系统，那么文件命名规则与 FAT32 文件系统命名规则一致，Flash 上的文件命名受 JFFS2 文件系统限制，那么文件命名规则与 JFFS2 文件系统命名规则一致。

- LK220 不支持该指令。

3.3.3.3 FileClose（关闭文件）

3.3.3.3.1 函数原型

DINT FileClose (DINT iFd);

3.3.3.3.2 功能描述

关闭文件。

3.3.3.3.3 参数说明

	数据项	数据类型	参数描述
输入端	iFd	DINT	FileOpen 返回文件 ID
输出端	FileClose	DINT	成功:返回 0 失败:返回-1

❗ LK220 不支持该指令。

3.3.3.4 FileRead（读文件）

3.3.3.4.1 函数原型

DINT FileRead (DINT iFd, POINTER TO USINT pcBuf, UDINT uiSize);

3.3.3.4.2 功能描述

读文件。

3.3.3.4.3 参数描述

	数据项	数据类型	参数描述
输入端	iFd	DINT	FileOpen 返回文件 ID
	pcBuf	POINTER TO USINT	读数据的内存地址

	数据项	数据类型	参数描述
	uiSize	UDINT	读文件的大小 (byte)
输出端	FileRead	DINT	成功: 实际读取文件的字节数 失败: 返回-1



- 在使用 FileRead/ FileWrite 读写文件过程中，如操作返回-1，在确认输入参数无误的情况下，请关闭并重新打开文件，则可继续进行读写操作。
- LK220 不支持该指令。

3.3.3.5 FileWrite（写文件）

3.3.3.5.1 函数原型

DINT FileWrite (DINT iFd, POINTER TO USINT pcBuf, UDINT uiSize);

3.3.3.5.2 功能描述

写文件。

3.3.3.5.3 参数描述

	数据项	数据类型	参数描述
输入端	iFd	DINT	FileOpen 返回文件 ID
	pcBuf	POINTER TO USINT	写数据所在内存地址
	uiSize	UDINT	写文件的大小 (byte)
输出端	FileWrite	DINT	成功: 返回实际写入数据的大小 失败: 返回-1



- 在使用 FileRead/ FileWrite 读写文件过程中，如操作返回-1，在确认输入参数无误的情况下，请关闭并重新打开文件，则可继续进行读写操作。
- LK220 不支持该指令。

3.3.3.6 FileLseek（移动文件读写指针）

3.3.3.6.1 函数原型

DINT FileLseek (DINT iFd, DINT iOffset, UDINT uiFrom);

3.3.3.6.2 功能描述

移动文件读写指针。

3.3.3.6.3 参数描述

	数据项	数据类型	参数描述
输入端	iFd	DINT	FileOpen 返回文件 ID
	iOffset	DINT	移动文件指针的偏移量
	uiFrom	UDINT	0: 定位到文件头 1: 定位到文件当前位置 2: 定位到文件尾
输出端	FileLseek	DINT	成功: 返回文件当前的读写位置 失败: 返回-1

❗LK220 不支持该指令。

3.3.3.7 FileListScan（扫描目录文件）

3.3.3.7.1 函数原型

SINT FileListScan (STRING PathName, POINTER TO USINT pFileNameList, SINT cFileNum, SINT cFileNameSize)

3.3.3.7.2 功能描述

扫描指定目录下包含的所有文件。

3.3.3.7.3 参数描述

	数据项	数据类型	参数描述
输入	PathName	STRING(63)	待扫描的目录名，最大支持 63 个字符，且只支持全路径方式

	数据项	数据类型	参数描述
端			“sd/”为 SD 卡目录 “flash/”为 Flash 目录
	pFileNameList	POINTER TO USINT	字符串指针数组的地址。返回扫描到的文件名列表
	cFileNum	SINT	需要扫描文件的个数，目前最大支持 32 个文件
	cFileNameLen	SINT	为每一个文件名开辟的存储空间大小，字符串以'\0'结束，（包括结束符'\0'，最大支持 64 个字符），建议设置为 64
输出端	FileListScan	SINT	成功：返回扫描到文件的个数 失败：返回-1

❗ LK220 不支持该指令。

3.3.3.8 FindFirstFile（查找第一个文件）

3.3.3.8.1 函数原型

DINT FindFirstFile (STRING PathName, STRING FileName);

3.3.3.8.2 功能描述

打开遍历目录文件句柄，并获得该目录下的第一个文件名。

3.3.3.8.3 参数描述

	数据项	数据类型	参数描述
输入端	PathName	STRING(63)	待遍历的目录名，最大支持 63 个字符，且只支持全路径方式 “sd/”为 SD 卡目录 “flash/”为 Flash 目录
	FileName	STRING(63)	返回遍历到的文件名（不含路径，最大支持 63 个字符）
输出端	FindFirstFile	DINT	成功：返回扫描文件句柄 失败：返回（-1）

❗ LK220 不支持该指令。

3.3.3.9 FindNextFile（查找下一个文件）

3.3.3.9.1 函数原型

SINT FindNextFile (DINT iFileFindHandle, STRING FileName);

3.3.3.9.2 功能描述

遍历目录，获取指定目录的下一个文件名。

3.3.3.9.3 参数描述

	数据项	数据类型	参数描述
输入端	iFileFindHandle	DINT	文件遍历句柄(FindFirstFile 的返回值)
	FileName	STRING(63)	返回遍历到的文件名（不含路径，最大支持 63 个字符）
输出端	FindNextFile	SINT	成功：返回 0 失败：返回-1

❗LK220 不支持该指令。

3.3.3.10 FindClose（查找结束）

3.3.3.10.1 函数原型

SINT FindClose (DINT iFileFindHandle);

3.3.3.10.2 功能描述

关闭遍历句柄。

3.3.3.10.3 参数描述

	数据项	数据类型	参数描述
输入端	iFileFindHandle	DINT	文件遍历句柄（FindFirstFile 的返回值）
输出端	FindClose	SINT	成功：返回 0 失败：返回-1

❗ LK220 不支持该指令。

3.3.3.11 FileRemove（删除文件）

3.3.3.11.1 函数原型

DINT FileRemove (STRING pStrFileName);

3.3.3.11.2 功能描述

删除文件。

3.3.3.11.3 参数描述

	数据项	数据类型	参数描述
输入端	Name	STRING(63)	文件名，最大支持 63 个字符，且只支持全路径方式 “sd/”为 SD 卡目录 “flash/”为 Flash 目录
输出端	FileRemove	DINT	成功：返回 0 失败：返回-1

❗ LK220 不支持该指令。

3.3.3.12 FlashWrite（写入掉电保持数据）

3.3.3.12.1 函数原型

DINT FlashWrite (DINT iFileID, POINTER TO USINT pDataAddr);

3.3.3.12.2 功能描述

文件 ID 取值 0~15，分别对应 Flash 上的 16 个文件。用户通过调用此函数可以完成将指定地址 1k bytes 大小的数据保存在 Flash 对应的文件中。

3.3.3.12.3 参数描述

	数据项	数据类型	参数描述
输入端	iFileID	DINT	iFileID
	pDataAddr	POINTER TO USINT	掉电保护区地址
输出端	FlashWrite	DINT	成功：0 失败：-1

❗ LK220 不支持该指令。

3.3.3.13 FlashRead（读取掉电保持数据）

3.3.3.13.1 函数原型

DINT FlashRead (DINT iFileID, POINTER TO USINT pDataAddr);

3.3.3.13.2 功能描述

文件 ID 取值 0~15，分别对应 Flash 上的 16 个文件。用户通过调用此函数可以完成将 Flash 上指定文件的数据恢复至数据区。

3.3.3.13.3 参数描述

	数据项	数据类型	参数描述
输入端	iFileID	DINT	iFileID
	pDataAddr	POINTER TO USINT	掉电保护区地址
输出端	FlashRead	DINT	成功：0 失败：-1

❗ LK220 不支持该指令。

3.3.4 时间相关指令

3.3.4.1 GetTickCnt(获取开机计数)

3.3.4.1.1 函数原型

UDINT GetTickCnt (void);

3.3.4.1.2 功能

获取控制器自开机以来的时间。MC 平台能够获取的时间数值范围为 $0 \sim 2^{32}/10$ 微秒（≈7 分 9 秒 496 毫秒 729.6 微秒），LK 平台时间范围为 $0 \sim 2^{32}$ 微秒，若超过上限，则从 0 开始计数。



GetTickCnt 指令

3.3.4.1.3 参数说明

返回值	类型	含义
GetTickCnt	UDINT	控制器开机以来所经过的时间，溢出后从零重新计时（ μs ）

3.3.4.2 TimeDelay（时间延时）

3.3.4.2.1 函数原型

DINT TimeDelay (UDINT uiTimeDelay);

3.3.4.2.2 功能描述

调用此函数后，该函数执行 uiTimeDelay 个毫秒后返回。最终达到时间延时的效果。

3.3.4.2.3 参数描述

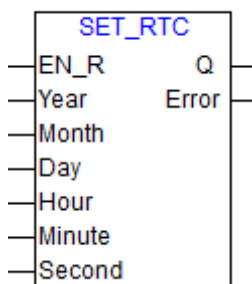
	数据项	数据类型	参数描述
输入端	uiTimeDelay	UDINT	延时时间（单位为毫秒）

	数据项	数据类型	参数描述
输出端	TimeDelay	DINT	0 执行延时成功；-1 执行延时失败

3.3.4.3 SET_RTC（设置实时时钟）

3.3.4.3.1 功能

将用户规定的时间设置到 PLC 实时时钟中。



SET_RTC 功能块

3.3.4.3.2 参数说明

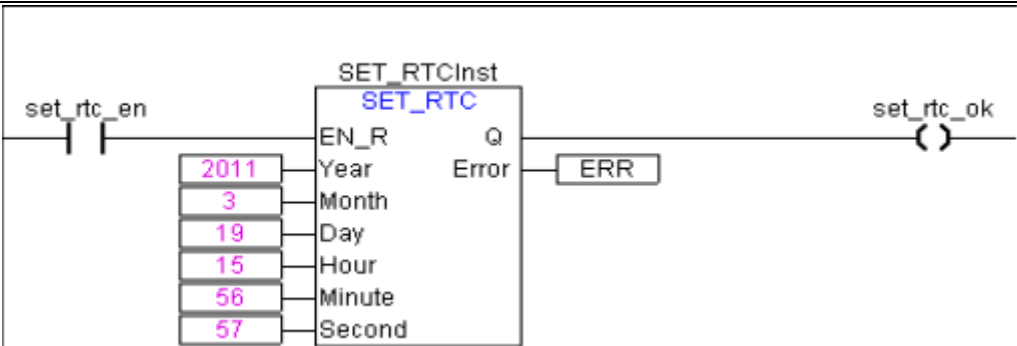
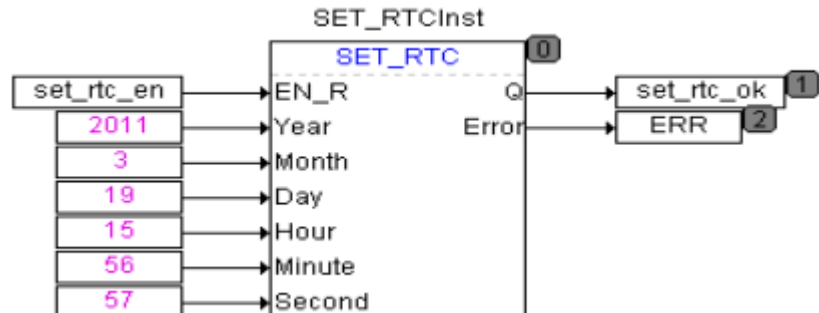
输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0: 无效 1: 上升沿有效	0
Year	WORD	年	LE/MKC 平台: 2000~2099 MC 平台: MC1000 系列为 1971~2036, MC2000 系列、MC1002R、MC1002E 为 2000~2036 LK 平台: LK 系统 V2.1.0 版本前为 1971~2036, V2.1.0 版本后为 2000~2036	2011
Month	BYTE	月	1-12	1
Day	BYTE	日	1-31	1
Hour	BYTE	时	0-23	0
Minute	BYTE	分	0-59	0
Second	BYTE	秒	0-59	0

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	设置完成	0: 未完成 1: 已完成	0
Error	BYTE	参数设置错误	=0: 设置 RTC 时间没有错误 非 0: 表示设置 RTC 时间时出现错误 =1: 设置年超出规定范围 =2: 设置月不合理 =3: 设置日不合理 =4: 设置时不合理 =5: 设置分不合理	0

输出参数	数据类型	功能描述	参数值说明	默认值
			=6: 设置秒不合理 =200: 模块重复定义并调用	

3.3.4.3.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	set_rtc_en			BOOL	FALSE	FALSE
0002	set_rtc_ok			BOOL	FALSE	FALSE
0003	ERR			BYTE	0	FALSE
0004	SET_RTCInst			SET_RTC		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> SET_RTCInst(EN_R:=set_rtc_en , Year:=2011 , Month:=3 , Day:= 19, Hour:= 15, Minute:= 56, Second:=57 , Q=>set_rtc_ok Error=>ERR); </pre>
连续功能图 (CFC)	

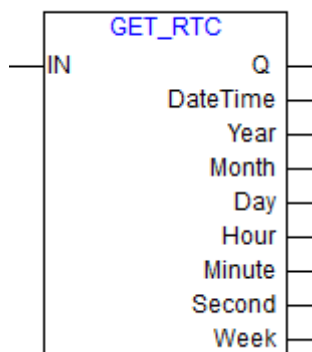
程序说明:

EN_R 置位时，如图所示的日期、时间将被设置到 PLC 硬件实时时钟之中，当前的实时时钟是 2011 年 3 月 19 日 15 点 56 分 57 秒。

3.3.4.4 GET_RTC（读取实时时钟）

3.3.4.4.1 功能

该功能块读取 PLC 硬件实时时钟中的时间，该时间为东 8 区时间。



GET_RTC 功能块

3.3.4.4.2 参数说明

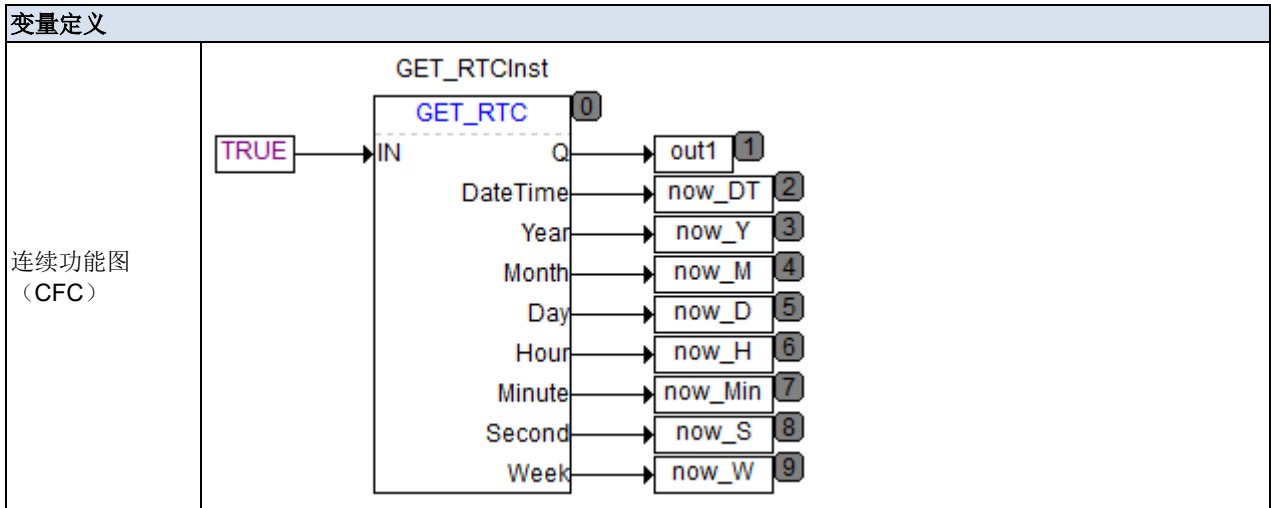
输入参数	数据类型	功能描述	参数值说明	默认值
IN	BOOL	使能	0: 无效 1: 有效	0

输出参数	数据类型	功能描述	参数值说明	默认值
Q	BOOL	是否读出数据	0: 未读出数据 1: 正确取出数据	0
DateTime	DT	日期/时间	LE/MKC 平台: DT#2000-01-01-00:00:00~ DT#2099-12-31-23:59:59 MC/LK 平台: DT#1971-01-01-00:00:00~ DT#2036-12-31-23:59:59	DT#1970-01-01-00:00:00
Year	WORD	年	LE/MKC 平台: 2000~2099 MC 平台: MC1000 系列为 1971~2036, MC2000 系列、MC1002R、MC1002E 为 2000~2036 LK 平台: LK 系统 V2.1.0 版本前为 1971~2036, V2.1.0 版本后为 2000~2036	0
Month	BYTE	月	1-12	0
Day	BYTE	日	1-31	0
Hour	BYTE	时	0-23	0
Minute	BYTE	分	0-59	0
Second	BYTE	秒	0-59	0
Week	BYTE	星期	1-7	0

3.3.4.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	now_DT			DT	DT#1970-0...	FALSE
0002	now_Y			WORD	0	FALSE
0003	now_M			BYTE	0	FALSE
0004	now_D			BYTE	0	FALSE
0005	now_H			BYTE	0	FALSE
0006	now_Min			BYTE	0	FALSE
0007	now_S			BYTE	0	FALSE
0008	now_W			BYTE	0	FALSE
0009	GET_RTCInst			GET_RTC		FALSE
0010	out1			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	<pre> 0001 GET_RTCInst GET_RTC IN Q out1 () DateTime=now_DT Year=now_Y Month=now_M Day=now_D Hour=now_H Minute=now_Min Second=now_S Week=now_W </pre>
结构化文本 (ST)	<pre> GET_RTC(IN:=TRUE, Q=>out1, DateTime=>now_DT, Year=>now_Y, Month=>now_M, Day=>now_D, Hour=>now_H, Minute=>now_Min, Second=>now_S, Week=>now_W); </pre>



程序说明：

IN 置位时，DateTime 以 DT 型数据格式输出 PLC 实时时钟的日期、时间，Year 以 WORD 型数据格式输出年，Month、Date、Hour、Minute、Second、Week 分别以 BYTE 型数据格式输出月、日、时、分、秒、星期，Q 等于 TRUE，当前的实时时钟是：2012-03-19-15:56:57，星期一。

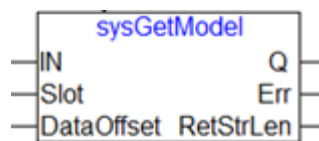
EN 复位时，Q 等于 FALSE，DateTime、Year、Month、Date、Hour、Minute、Second、Week 保持最后一次输出值不变。

3.3.5 模块信息指令

3.3.5.1 sysGetModel（获取模块型号）

3.3.5.1.1 功能描述

获取模块型号。



sysGetModel 指令

3.3.5.1.2 参数描述

输入参数	类型	参数描述（MC）	参数描述（LK）
IN	BOOL	高电平使能（默认值 FALSE）	与 MC 一致
Slot	BYTE	槽位号（有效值为 1）	槽位号（默认值 1）

输入参数	类型	参数描述（MC）	参数描述（LK）
			主控模块槽位号固定为 1 其它模块依据与主控位置向右递增 使用时输入想得到版本信息模块所在的槽位号
DataOffset	DWORD	M 区的偏移（模块型号以字符串格式保存在指定偏移位置） （默认值 0）	M 区的偏移（模块型号以字符串格式保存在指定偏移位置） （默认值 0，有效值范围 0~1048575）

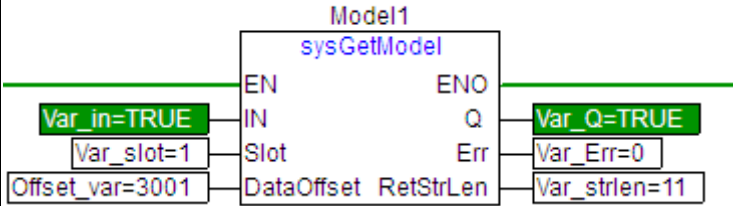
输出参数	类型	参数描述（MC）	参数描述（LK）
Q	BOOL	输出（默认值 FALSE）	与 MC 一致
Err	BYTE	错误（默认值 0） 1: 槽号 slot 错误 2: DataOffset 偏移配置错误 129: 返回的字符串超出 M 区范围 130: 返回信息错误	错误（默认值 0） 1: 槽号 slot 错误 2: DataOffset 偏移配置错误 128: 对应槽上不存在模块 129: 返回的字符串超出 M 区范围 130: 返回信息错误 131: 该槽位模块类型未知
RetStrLen	BYTE	模块型号字符串长度（默认值 0）	与 MC 一致

3.3.5.1.3 指令使用举例

变量定义

Main (PRG).ld

序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	Model1			SYSGETMODEL		FALSE
0002	Var_in			BOOL	TRUE	FALSE
0003	Var_slot			BYTE	1	FALSE
0004	Offset_var			DWORD	3001	FALSE
0005	Var_Q			BOOL	TRUE	FALSE
0006	Var_Err			BYTE	0	FALSE
0007	Var_strlen			BYTE	11	FALSE
0008	address_var	%MB3001		STRING (80)	'Model:LK220'	FALSE

编程语言	程序
梯形图（LD）	 <pre> graph LR subgraph Model1_sysGetModel [Model1 sysGetModel] EN[EN] --> IN[IN] IN --> Slot[Slot] Slot --> DataOffset[DataOffset] DataOffset --> RetStrLen[RetStrLen] ENO[ENO] --> Q[Q] Q --> Err[Err] Err --> RetStrLen end Var_in_TRUE[Var_in=TRUE] --> EN Var_slot_1[Var_slot=1] --> Slot Offset_var_3001[Offset_var=3001] --> DataOffset Q --> Var_Q_TRUE[Var_Q=TRUE] Err --> Var_Err_0[Var_Err=0] RetStrLen --> Var_strlen_11[Var_strlen=11] </pre>

变量定义	
结构化文本 (ST)	<pre> 0001 Model1(0002 IN:=Var_in, 0003 Slot:=Var_slot, 0004 DataOffset:=Offset_var, 0005 Q=>Var_Q, 0006 Err=>Var_Err, 0007 RetStrLen=>Var_strlen); 0008 0009 0010 0011 </pre>
连续功能图 (CFC)	

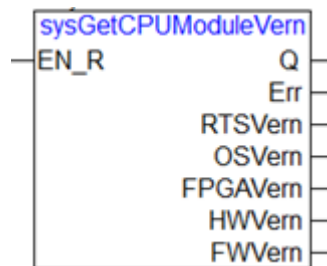
程序说明：

用户需要定义字符串类型的变量存放读取的模块型号，如示例中的变量 `address_var`，变量需关联 M 区地址。参数 `DataOffset` 的值引用该 M 区偏移地址即可，如示例中设置 `Offset_var` 的值为 3001。

3.3.5.2 sysGetCPUModuleVern（获取控制器的固件版本信息）

3.3.5.2.1 功能

获取控制器的固件版本信息，版本信息以十进制方式显示。



sysGetCPUModuleVern 指令

3.3.5.2.2 参数说明

输入参数	类型	含义
EN_R	BOOL	使能 FALSE: 无效（默认值）

输入参数	类型	含义
		TRUE: 上升沿有效

输出参数	类型	参数描述
Q	BOOL	输出（默认值 FALSE）
Err	BYTE	错误（默认值 0） 128: 获取版本失败
RTSVern	DWORD	RTS 固件版本号 高 16 位为快核 HEROS 侧 RTS 版本号，对应小工具 RTOS_RTS 版本 Vx.y.z 低 16 位为慢核 LINUX 侧 RTS 版本号，对应小工具 LINUX_RTS 版本 Vx.y.z 其中，高 5bit 代表 x，中间 6bit 代表 y，最后 5bit 代表 z
OSVern	DWORD	OS 版本号 高 16 位为快核 HEROS 侧 OS 版本号，对应小工具 HEROS 版本 Vx.y.z 低 16 位为慢核 LINUX 侧 OS 版本号，对应小工具 LINUX 版本 Vx.y.z 其中，高 5bit 代表 x，中间 6bit 代表 y，最后 5bit 代表 z
FPGAVern	WORD	FPGA 版本号，对应小工具 FPGA 版本 Vx.y.z 共 16 位，其中高 5bit 代表 x，中间 6bit 代表 y，最后 5bit 代表 z
HWVern	WORD	硬件版本号 返回值为 1~26，对应 A~Z
FWVern	WORD	模块固件版本号，对应小工具 SW 版本 Vx.y.z 共 16 位，其中高 5bit 代表 x，中间 6bit 代表 y，最后 5bit 代表 z

3.3.5.2.3 指令使用举例

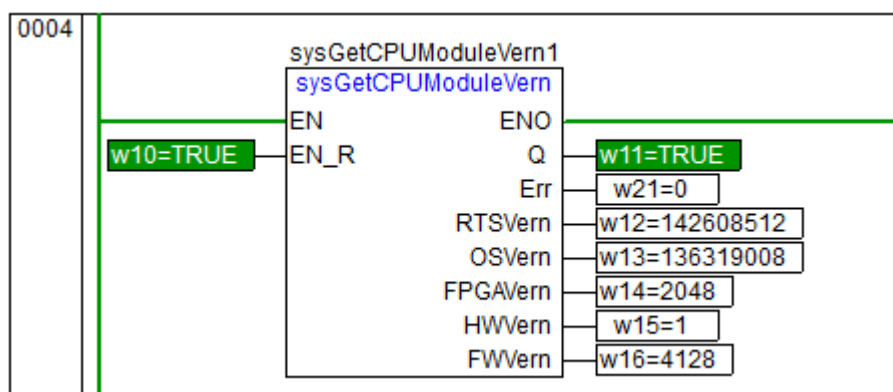
通过 sysGetCPUModuleVern 指令和小工具分别获取控制器版本信息，如下图所示。

以 RTS 版本信息为例进行说明。

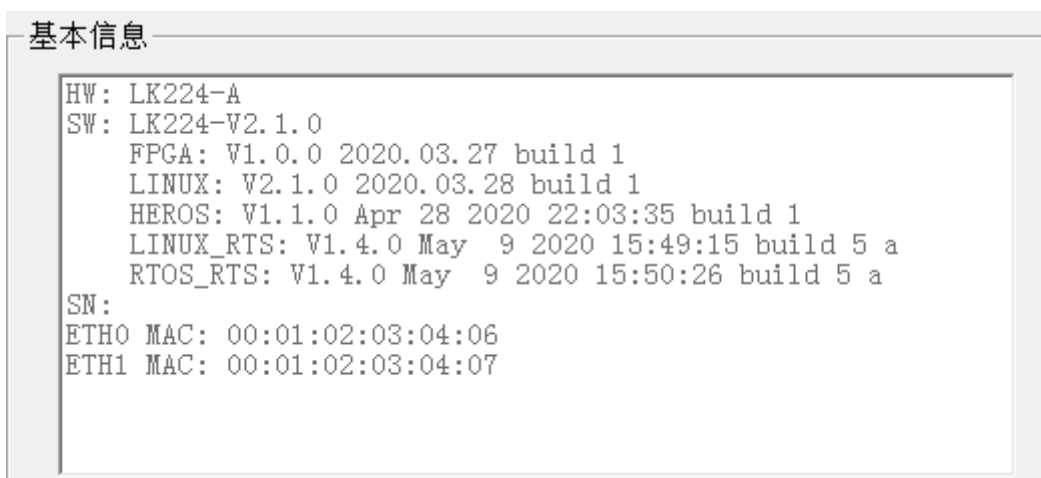
RTSVern 输出为 142608512（0x08800880），其中高 16 位表示快核侧 RTS 版本号，低 16 位为慢核侧 RTS 版本号。

高 16 位 0x0880 对应二进制为 0000 1000 1000 0000b，高 5bit 值为 1，中间 6bit 值为 4，最后 5bit 值为 0，则版本号为 V1.4.0，对应小工具第 7 行 RTOS_RTS:V1.4.0。

低 16 位 0x0880 对应二进制为 0000 1000 1000 0000b，高 5bit 值为 1，中间 6bit 值为 4，最后 5bit 值为 0，则版本号为 V1.4.0，对应小工具第 6 行 LINUX_RTS:V1.4.0。



指令获取控制器版本信息

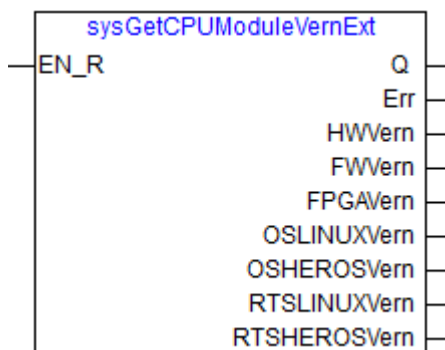


小工具读取控制器版本信息

3.3.5.3 sysGetCPUModuleVernExt（获取控制器的固件版本扩展信息）

3.3.5.3.1 功能

获取控制器的固件版本信息，版本信息以字符串格式显示。与“sysGetCPUModuleVern”指令功能一致，仅版本信息显示方式不同。



sysGetCPUModuleVernExt 指令

3.3.5.3.2 参数说明

输入参数	类型	含义
EN_R	BOOL	使能 FALSE: 无效（默认值） TRUE: 上升沿有效

输出参数	类型	参数描述
Q	BOOL	输出（默认值 FALSE）

输出参数	类型	参数描述
Err	BYTE	错误（默认值 0） 128: 获取版本失败
HWVern	STRING	硬件版本号，对应小工具 HW 版本信息 返回值为 1~26，对应 A~Z，显示格式'A'~'Z'
FWVern	STRING	模块固件版本号，对应小工具 SW 版本 Vx.y.z
FPGAVern	STRING	FPGA 版本号，对应小工具 FPGA 版本 Vx.y.z
OSLINUXVern	STRING	LINUX 侧 OS 版本号，对应小工具 LINUX 版本 Vx.y.z
OSHEROSVern	STRING	HEROS 侧 OS 版本号，对应小工具 HEROS 版本 Vx.y.z
RTSLINUXVern	STRING	LINUX 侧 RTS 版本号，对应小工具 LINUX_RTS 版本 Vx.y.z
RTSHEROSVern	STRING	HEROS 侧 RTS 版本号，对应小工具 RTOS_RTS 版本 Vx.y.z

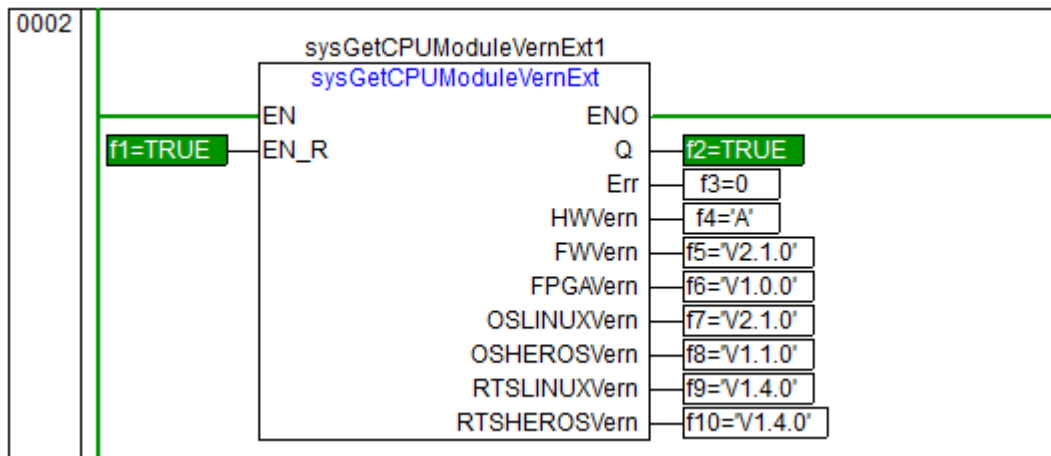
3.3.5.3.3 指令使用举例

通过 sysGetCPUModuleVernExt 指令和小工具分别获取控制器版本信息，如下图所示。

以 HW 和 FPGA 版本信息为例进行说明。

HWVern 显示字符串'A'，对应小工具第一行 HW:LK224-A 中的'A'，表示硬件版本号为 A。

FPGAVern 显示字符串 V1.0.0，对应小工具第三行 FPGA:V1.0.0，表示 FPGA 版本为 V1.0.0。



指令获取控制器版本信息

基本信息

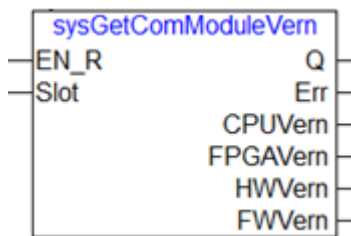
```
HW: LK224-A
SW: LK224-V2.1.0
  FPGA: V1.0.0 2020.03.27 build 1
  LINUX: V2.1.0 2020.03.28 build 1
  HEROS: V1.1.0 Apr 28 2020 22:03:35 build 1
  LINUX_RTS: V1.4.0 May 9 2020 15:49:15 build 5 a
  RTOS_RTS: V1.4.0 May 9 2020 15:50:26 build 5 a
SN:
ETH0 MAC: 00:01:02:03:04:06
ETH1 MAC: 00:01:02:03:04:07
```

小工具读取控制器版本信息

3.3.5.4 sysGetComModuleVern（获取通信模块固件版本信息）

3.3.5.4.1 功能

获取通信模块固件版本信息。



sysGetComModuleVern 指令

3.3.5.4.2 参数说明

输入参数	类型	含义
EN_R	BOOL	使能 FALSE: 无效（默认值） TRUE: 上升沿有效
Slot	BYTE	槽位号（默认值 2，有效值范围 2-5） 主控模块槽位号固定为 1 其它模块依据与主控位置向右递增 使用时输入想得到版本信息模块所在的槽位号

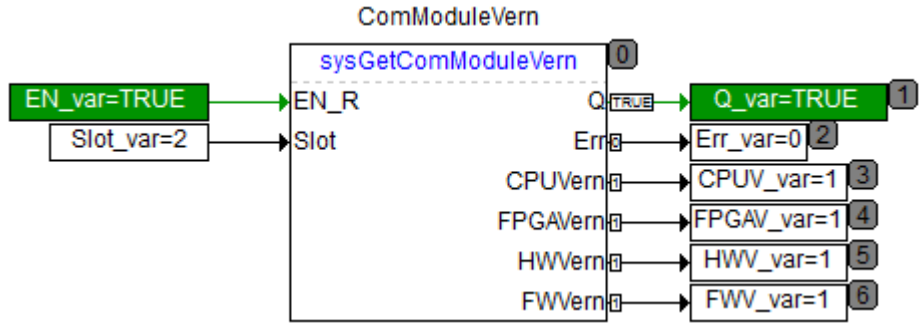
输出参数	类型	参数描述
Q	BOOL	输出（默认值 FALSE）
Err	BYTE	错误（默认值 0） 0: 正确

输出参数	类型	参数描述
		1: Slot 配置错误 128: 对应槽位上不存在模块 129: 获取失败 200: 模块重复定义并调用
CPUVer	WORD	CPU 固件版本号
FPGAVer	WORD	FPGA 版本号
HWVer	WORD	硬件版本号 返回值为 1~26, 对应 A~Z
FWVer	WORD	模块固件版本号 所示的为“LK240-A01”中的“01”

3.3.5.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	EN_var			BOOL	TRUE	FALSE
0002	Slot_var			BYTE	2	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	Err_var			BYTE	0	FALSE
0005	CPUV_var			WORD	1	FALSE
0006	FPGAV_var			WORD	1	FALSE
0007	HWV_var			WORD	1	FALSE
0008	FWV_var			WORD	1	FALSE
0009	ComModuleVer			SYSGETCOMMODULEVERN		FALSE

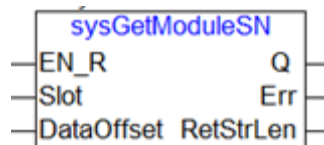
编程语言	程序
梯形图 (LD)	

变量定义	
结构化文本 (ST)	<pre> 0001 ComModuleVern(0002 EN_R:=EN_var, 0003 Slot:=Slot_var, 0004 Q=>Q_var, 0005 Err=>Err_var, 0006 CPUVern=>CPUV_var, 0007 FPGAVern=>FPGAV_var, 0008 HWVern=>HWV_var, 0009 FWVern=>FWV_var); 0010 0011 0012 </pre>
连续功能图 (CFC)	

3.3.5.5 sysGetModuleSN（获取模块序列号）

3.3.5.5.1 功能

获取模块序列号。



sysGetModuleSN 指令

3.3.5.5.2 参数说明

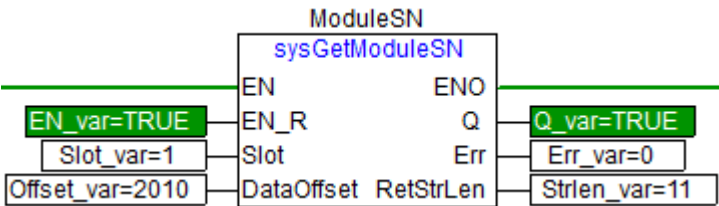
输入参数	类型	参数描述（MC）	参数描述（LK）
EN_R	BOOL	使能 FALSE: 无效（默认值） TRUE: 上升沿有效	与 MC 一致
Slot	BYTE	槽位号（有效值为 1）	槽位号（默认值 1,有效值 1~5） 主控模块槽位号固定为 1 其它模块依据与主控位置向右递增 使用时输入想得到版本信息模块所在的槽位号

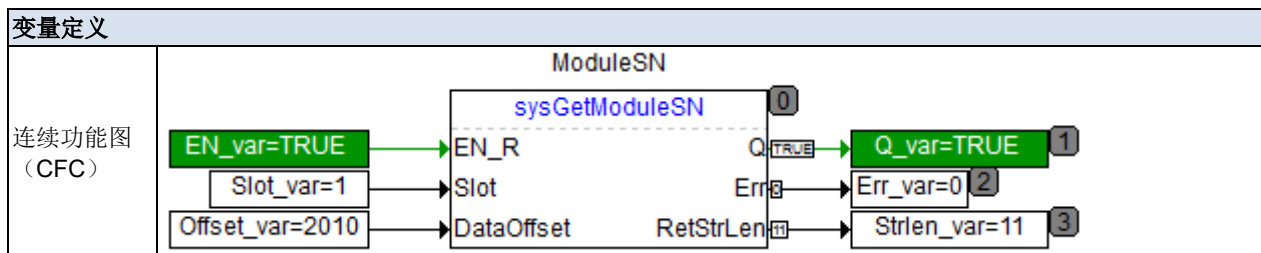
输入参数	类型	参数描述（MC）	参数描述（LK）
DataOffset	DWORD	M 区的偏移（序列号以字符串格式将保存在指定偏移位置） （默认值 0）	M 区的偏移（序列号以字符串格式将保存在指定偏移位置） （默认值 0，有效值范围 0~1, 048, 575）

输出参数	类型	参数描述（MC）	参数描述（LK）
Q	BOOL	输出（默认值 FALSE）	与 MC 一致
Err	BYTE	错误（默认值 0） 0: 正确 1: 槽号 slot 错误 2: DataOffset 偏移配置错误 129: 返回的字符串超出 M 区范围 130: 返回信息错误或读取过程中暂时未得到正确数据（最终读取成功后会变为 0）	错误（默认值 0） 0: 正确 1: 槽号 slot 错误 2: DataOffset 偏移配置错误 128: 对应槽上不存在模块 129: 返回的字符串超出 M 区范围 130: 返回信息错误或读取过程中暂时未得到正确数据（最终读取成功后会变为 0） 200: 模块重复定义并调用
RetStrLen	BYTE	模块序列号字符串长度（默认值 0）	模块序列号字符串长度（默认值 0），最长支持 18 位 ASCII 码的字符串，超出部分自动截断

3.3.5.5.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	EN_var			BOOL	TRUE	FALSE
0002	Slot_var			BYTE	1	FALSE
0003	Offset_var			DWORD	2010	FALSE
0004	Q_var			BOOL	TRUE	FALSE
0005	Err_var			BYTE	0	FALSE
0006	Strlen_var			BYTE	11	FALSE
0007	ModuleSN			SYSGETMODULESN		FALSE
0008	address_var	%MB2010		STRING(80)	'01234567891'	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> 0001 ModuleSN(0002 EN_R:=EN_var, 0003 Slot:=Slot_var, 0004 DataOffset:=Offset_var, 0005 Q:=Q_var, 0006 Err:=Err_var, 0007 RetStrLen:=Strlen_var); 0008 </pre>



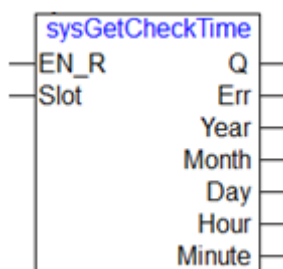
程序说明：

用户需要定义字符串类型的变量存放读取的模块序列号，如示例中的变量 `address_var`，变量需关联 M 区地址。参数 `DataOffset` 的值引用该 M 区偏移地址即可，如示例中设置 `Offset_var` 的值为 2010。

3.3.5.6 sysGetCheckTime（获取检验时间）

3.3.5.6.1 功能描述

获取检测时间。



sysGetCheckTime 指令

3.3.5.6.2 参数说明

输入参数	类型	参数描述
EN_R	BOOL	使能 FALSE：无效（默认值） TRUE：上升沿有效
Slot	BYTE	槽位号（默认值 1，有效值 1-5） 主控模块槽位号固定为 1 其它模块依据与主控位置向右递增 使用时输入想得到版本信息模块所在的槽位号

输出参数	类型	参数描述（MC）	参数描述（LK）
Q	BOOL	输出(默认值 FALSE)	与 MC 一致
Err	BYTE	错误（默认值 0） 0：正确	错误（默认值 0） 0：正确

输出参数	类型	参数描述（MC）	参数描述（LK）
		1: Slot 配置错误 129: 获取失败	1: Slot 配置错误 128: 对应槽位上不存在模块 129: 获取失败 130: 年份错误（2015~2099） 131: 月份错误 132: 日期错误 133: 小时错误 134: 分钟错误 200: 模块重复定义并调用
Year	WORD	年（默认值 0）	与 MC 一致
Month	BYTE	月（默认值 0）	与 MC 一致
Day	BYTE	日（默认值 0）	与 MC 一致
Hour	BYTE	时（默认值 0）	与 MC 一致
Minute	BYTE	分（默认值 0）	与 MC 一致

3.3.5.6.3 指令使用举例

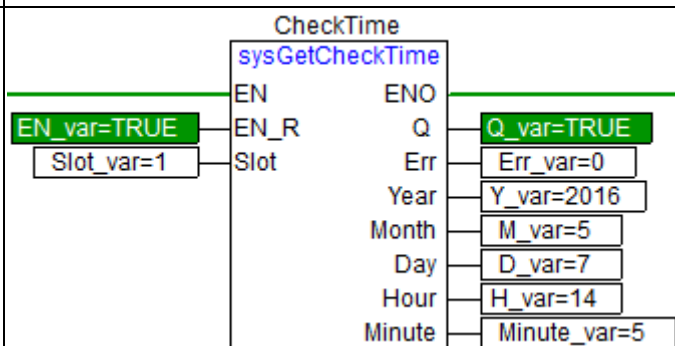
变量定义

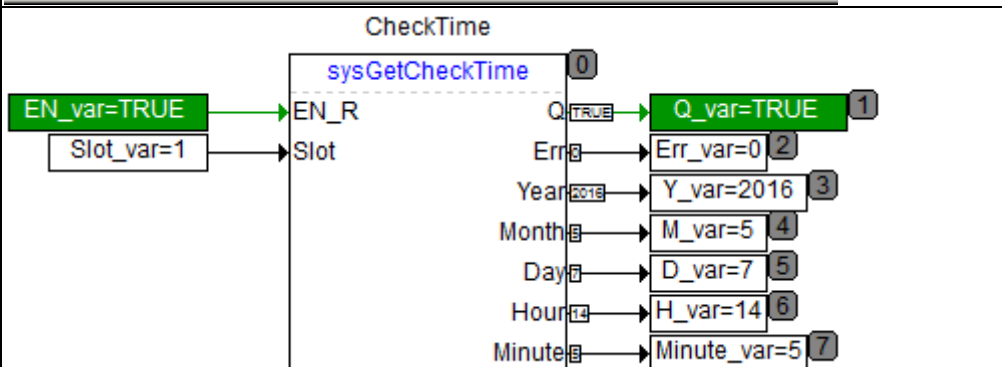
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	EN_var			BOOL	TRUE	FALSE
0002	Slot_var			BYTE	1	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	Err_var			BYTE	0	FALSE
0005	Y_var			WORD	2016	FALSE
0006	M_var			BYTE	5	FALSE
0007	D_var			BYTE	7	FALSE
0008	H_var			BYTE	14	FALSE
0009	Minute_var			BYTE	5	FALSE
0010	CheckTime			SYSGETCHECKTIME		FALSE

编程语言

程序

梯形图（LD）

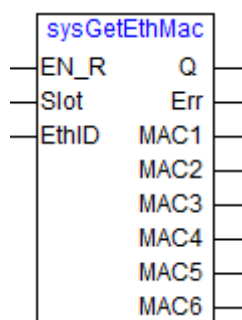


变量定义		
结构化文本 (ST)	<pre> 0001 CheckTime(0002 EN_R:=EN_var, 0003 Slot:=Slot_var, 0004 Q=>Q_var, 0005 Err=>Err_var, 0006 Year=>Y_var, 0007 Month=>M_var, 0008 Day=>D_var, 0009 Hour=>H_var, 0010 Minute=>Minute_var); 0011 </pre>	<pre> CheckTime EN_var = TRUE Slot_var = 1 Q_var = TRUE Err_var = 0 Y_var = 2016 M_var = 5 D_var = 7 H_var = 14 Minute_var = 5 </pre>
连续功能图 (CFC)		

3.3.5.7 sysGetEthMac (获取 Mac 地址)

3.3.5.7.1 功能

获取 Mac 地址。



sysGetEthMac 指令

3.3.5.7.2 参数说明

输入参数	类型	参数描述 (MC)	参数描述 (LK)
EN_R	BOOL	使能	与 MC 一致

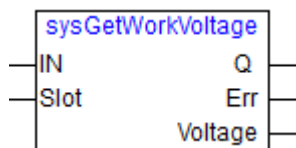
输入参数	类型	参数描述 (MC)	参数描述 (LK)
		FALSE: 无效 (默认值) TRUE: 上升沿有效	
Slot	BYTE	槽位号 (有效值为 0)	1: 本机 2~5: 槽位 2~5 上的 CP 卡
EthID	BYTE	网卡 ID 号 (有效值为 0) 0: 第 1 块网卡	网卡 ID 号 (默认值 0, 有效值为 0-1) 0: 第 1 块网卡 1: 第 2 块网卡

输出参数	类型	参数描述 (MC)	参数描述 (LK)
Q	BOOL	输出 (默认值 FALSE)	与 MC 一致
Err	BYTE	错误 (默认值 0) 1: EthID 配置错误 128: 获取 MAC 失败	错误 (默认值 0) 1: EthID 配置错误 128: 获取 mac 失败 200: 模块重复定义并调用
MAC1	BYTE	MAC 地址第 1 部分	与 MC 一致
MAC2	BYTE	MAC 地址第 2 部分	与 MC 一致
MAC3	BYTE	MAC 地址第 3 部分	与 MC 一致
MAC4	BYTE	MAC 地址第 4 部分	与 MC 一致
MAC5	BYTE	MAC 地址第 5 部分	与 MC 一致
MAC6	BYTE	MAC 地址第 6 部分	与 MC 一致

3.3.5.8 sysGetWorkVoltage(获取当前系统工作电压)

3.3.5.8.1 功能

获取当前系统工作电压。



sysGetWorkVoltage 指令

3.3.5.8.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能 (默认值 FALSE)
Slot	BYTE	槽位号, 设置为 1

输出参数	类型	含义
Q	BOOL	输出 (默认值 FALSE) TRUE: 输出成功

输出参数	类型	含义
		FALSE: 无输出
Err	BYTE	错误（默认值 0） 0: 正常 1: 槽位号错误 2: 模块型号错误，不支持 MC1004L 3: 获取当前电压错误
Voltage	REAL	当前电压（默认值 0）

3.3.5.8.3 指令使用举例

变量定义

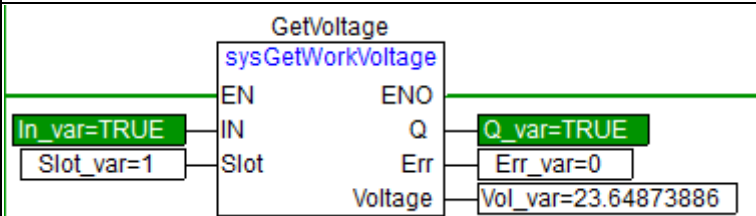
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	GetVoltage			SYSGETWORKVOLTAGE		FALSE
0002	In_var			BOOL	TRUE	FALSE
0003	Slot_var			BYTE	1	FALSE
0004	Q_var			BOOL	TRUE	FALSE
0005	Err_var			BYTE	0	FALSE
0006	Vol_var			REAL	23.59865761	FALSE

Main(PRG).ld

编程语言

程序

梯形图
(LD)



结构化文本
(ST)

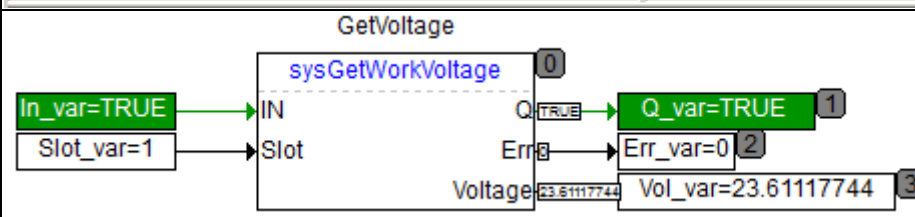
```

0001 GetVoltage(
0002 IN:=In_var,
0003 Slot:=Slot_var,
0004 Q:=Q_var,
0005 Err:=Err_var,
0006 Voltage:=Vol_var;
0007

```

GetVoltage
In_var = TRUE
Slot_var = 1
Q_var = TRUE
Err_var = 0
Vol_var = 23.61117744

连续功能图
(CFC)

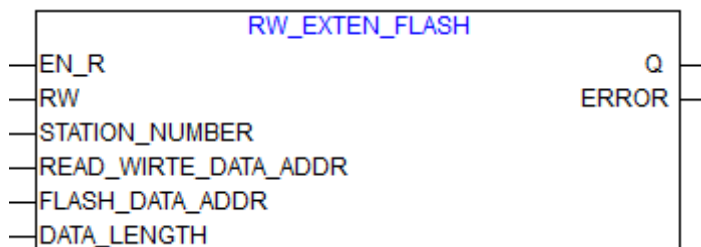


 该指令仅 MC 平台支持。

3.3.5.9 RW_EXTEN_FLASH(读写扩展模块 FLASH)

3.3.5.9.1 功能

实现 MKC3167 CPU 模块 RTS 和保护 IO 固件版本信息的读取。



RW_EXTEN_FLASH 指令

3.3.5.9.2 参数说明

输入参数	类型	含义
EN_R	BOOL	使能功能块，上升沿使能
RW	BOOL	读写控制 0: 读操作 1: 写操作（写操作有风险，请谨慎使用）
STATION_NUMBER	BYTE	模块地址 0: CPU 模块自身（含保护 IO）
READ_WIRTE_DATA_ADDR	DWORD	写入扩展模块 FLASH 的数据的 M 区起始地址;或者是读取扩展模块 FLASH 数据的保存到 M 区的起始地址
FLASH_DATA_ADDR	WORD	读取或写入的偏移地址 保护 RTS 模块: 0x200 保护 IO 信息: 0x00
DATA_LENGTH	WORD	读取或写入的长度 保护 RTS 模块: 46 保护 IO 信息: 46

输出参数	类型	含义
Q	BOOL	操作完成标志 0: 未完成 1: 完成
Error	BYTE	功能块故障状态 0: 运行 OK 1: 读取错误 2: 写入错误 4: 数据区不存在错误（此处应该不用） 5: M 区起始地址有误 6: 读取或写入 M 区数据长度超限

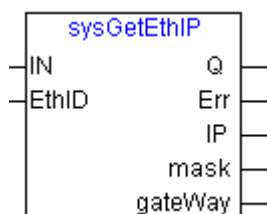
输出参数	类型	含义
		7: 读取或写入 M 区数据越界 8: 不能写模块内部型号和固件信息 9: 读取或写入 Flash 数据长度超限

❗ 该功能块仅 MKC3167 模块可用。

3.3.5.10 sysGetEthIP（获取 IP 信息）

3.3.5.10.1 功能

获取控制器的 IP 地址信息。



sysGetEthIP

- ❗ 适用于 LE5118-B01、LE5119-B01 及以上版本。

3.3.5.10.2 参数说明

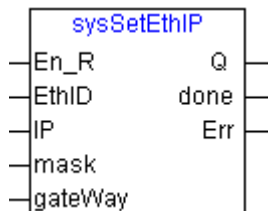
输入参数	类型	含义
IN	BOOL	高电平有效
EthID	BYTE	网卡编号，LE5118/LE5119 只有一张网卡，默认 0

输出参数	类型	含义
Q	BOOL	读完成标志 FALSE: 设置未完成 TRUE: 设置完成
Err	BYTE	错误码 0: 读成功 1: 网卡编号错误 2: 获取 IP 信息错误
IP	DWORD	IP 地址，32 位二进制，每一个字节表示 IP 的一位，高字节存放 IP 的第一位，如 192.168.0.250，高字节存放 192，低字节存放 250。
mask	DWORD	子网掩码，存储格式同 IP
gateWay	DWORD	网关，存储格式同 IP

3.3.5.11 sysSetEthIP（设置 IP 信息）

3.3.5.11.1 功能

为控制器配置 IP 地址。



sysSetEthIP 指令

-  适用于 LE5118-B01、LE5119-B01 及以上版本。

3.3.5.11.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿有效，上升沿使能配置
EthID	BYTE	网卡编号，LE5118/LE5119 只有一张网卡，默认 0
IP	DWORD	IP 地址，32 位二进制，每一个字节表示 IP 的一位，高字节存放 IP 的第一位，如 192.168.0.250，高字节存放 192，低字节存放 250
mask	DWORD	子网掩码，存储格式同 IP
gateWay	DWORD	网关，存储格式同 IP

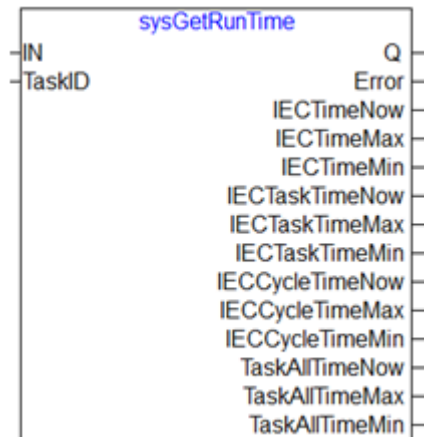
输出参数	类型	含义
Q	BOOL	设置完成标志 TRUE：设置完成 FALSE：设置未完成
done	BOOL	设置成功标志 TRUE：设置成功 FALSE：设置失败或功能块未触发
Err	BOOL	错误码 0：设置成功， 1：网卡编号错误， 2：获取 IP 信息错误 3：入参 IP 检查错误 4：IP 写失败

3.3.6 系统状态指令

3.3.6.1 sysGetRunTime（获取任务运行时间）

3.3.6.1.1 功能

获取控制器任务运行时间。



sysGetRunTime 指令

3.3.6.1.2 参数说明

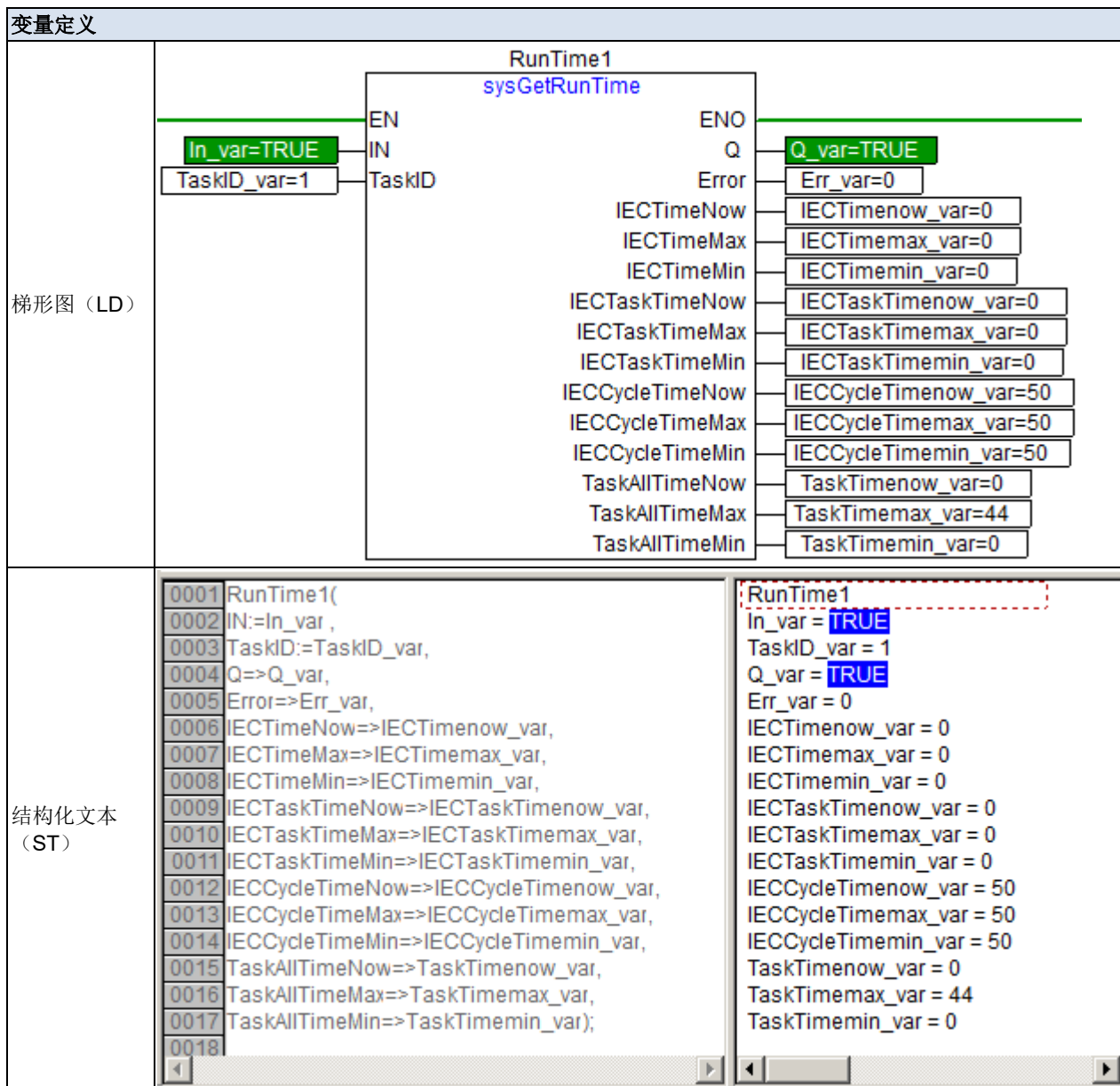
输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
TaskID	BYTE	IEC 任务 ID，1~8 LK220 只支持一个 IEC 任务，ID 默认为 1；如果为 2~8 则前三组 9 个时间数据返回 0 值

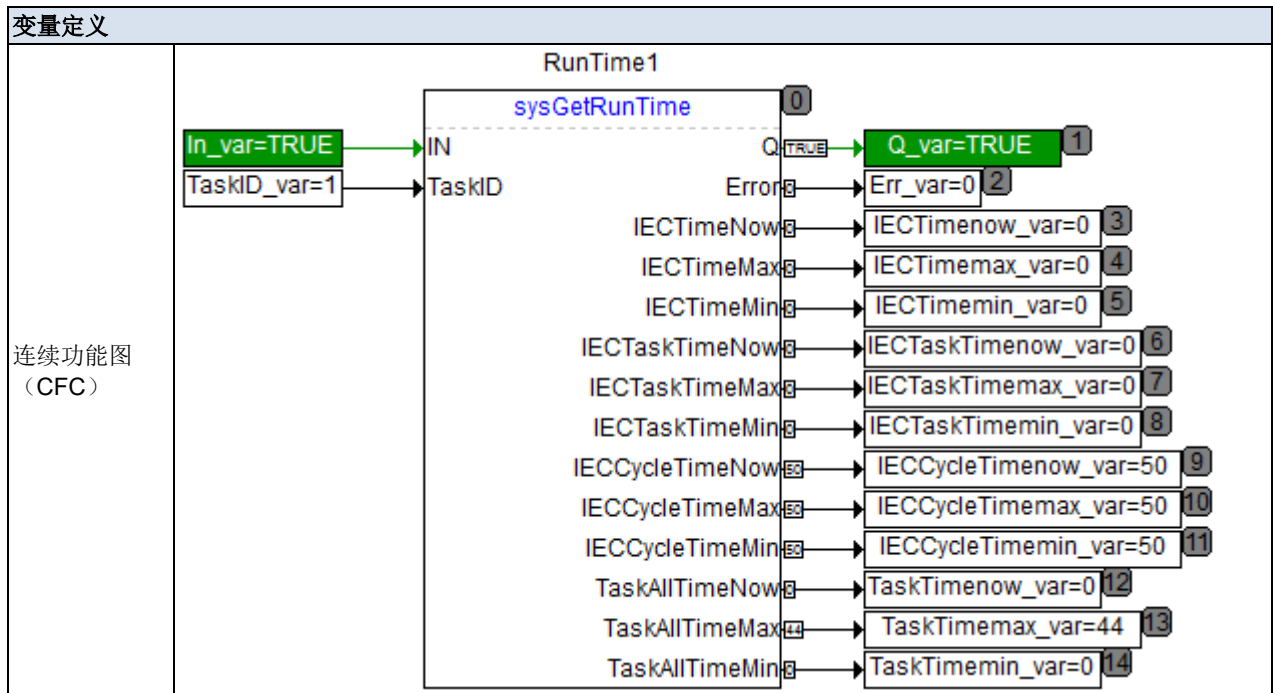
输出参数	类型	含义
Q	BOOL	输出，默认值 FALSE
Error	BYTE	错误，默认值 0 1: TaskID 配置错误
IECTimeNow	DWORD	当前组态逻辑本次运行时间，默认值 0
IECTimeMax	DWORD	当前组态逻辑最大运行时间，默认值 0
IECTimeMin	DWORD	当前组态逻辑最小运行时间，默认值 0
IECTaskTimeNow	DWORD	当前 IEC 任务本次运行时间，默认值 0
IECTaskTimeMax	DWORD	当前 IEC 任务最大运行时间，默认值 0
IECTaskTimeMin	DWORD	当前 IEC 任务最小运行时间，默认值 0
IECCycleTimeNow	DWORD	当前 IEC 任务周期本次运行时间，默认值 0
IECCycleTimeMax	DWORD	当前 IEC 任务周期最大运行时间，默认值 0
IECCycleTimeMin	DWORD	当前 IEC 任务周期最小运行时间，默认值 0

输出参数	类型	含义
TaskAllTimeNow	DWORD	当前任务本次运行时间，默认值 0
TaskAllTimeMax	DWORD	当前任务最大运行时间，默认值 0
TaskAllTimeMin	DWORD	当前任务最小运行时间，默认值 0

3.3.6.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	TaskID_var			BYTE	1	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	Err_var			BYTE	0	FALSE
0005	IECTimenow_var			DWORD	0	FALSE
0006	IECTimemax_var			DWORD	0	FALSE
0007	IECTimemin_var			DWORD	0	FALSE
0008	IECTaskTimenow_var			DWORD	0	FALSE
0009	IECTaskTimemax_var			DWORD	0	FALSE
0010	IECTaskTimemin_var			DWORD	0	FALSE
0011	IECCycleTimenow_var			DWORD	50	FALSE
0012	IECCycleTimemax_var			DWORD	50	FALSE
0013	IECCycleTimemin_var			DWORD	50	FALSE
0014	TaskTimenow_var			DWORD	0	FALSE
0015	TaskTimemax_var			DWORD	44	FALSE
0016	TaskTimemin_var			DWORD	0	FALSE
0017	RunTime1			SYSGETRUNTIME		FALSE
编程语言		程序				





3.3.6.2 sysGetRunTimeInfo（获取任务运行时间信息）

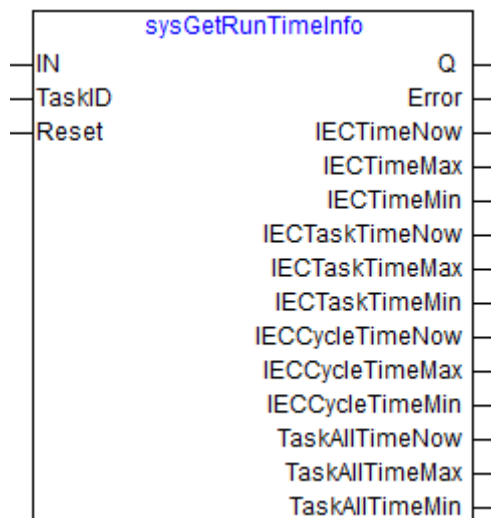
V3.1.5B5 及以上版本支持。

3.3.6.2.1 功能

获取控制器任务运行时间信息。

建议用户使用前，先通过 **Reset** 引脚对指令进行复位。

通过组态软件执行下装、冷复位等操作，可能导致任务运行时间变长。



sysGetRunTimeInfo 指令

3.3.6.2.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
TaskID	BYTE	IEC 任务 ID, 1~8 LK220 只支持一个 IEC 任务, ID 默认为 1; 如果为 2~8 则前三组 9 个时间数据返回 0 值
Reset	BOOL	将输出变量的各个时间变量清零

输出参数	类型	含义
Q	BOOL	输出, 默认值 FALSE
Error	BYTE	错误, 默认值 0 1: TaskID 配置错误
IECTimeNow	DWORD	当前组态逻辑本次运行时间, 默认值 0
IECTimeMax	DWORD	当前组态逻辑最大运行时间, 默认值 0
IECTimeMin	DWORD	当前组态逻辑最小运行时间, 默认值 0
IECTaskTimeNow	DWORD	当前 IO 刷新本次运行时间, 默认值 0
IECTaskTimeMax	DWORD	当前 IO 刷新最大运行时间, 默认值 0
IECTaskTimeMin	DWORD	当前 IO 刷新最小运行时间, 默认值 0
IECCycleTimeNow	DWORD	当前设置的任务周期, 默认值 0
IECCycleTimeMax	DWORD	当前设置的任务周期, 默认值 0
IECCycleTimeMin	DWORD	当前设置的任务周期, 默认值 0
TaskAllTimeNow	DWORD	当前任务本次运行时间, 默认值 0
TaskAllTimeMax	DWORD	当前任务最大运行时间, 默认值 0
TaskAllTimeMin	DWORD	当前任务最小运行时间, 默认值 0

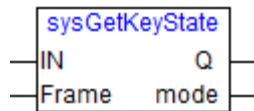
3.3.6.2.3 指令使用举例

请参见 [sysGetRunTime（获取任务运行时间）](#) 指令的使用举例。

3.3.6.3 sysGetKeyState（获取钥匙开关状态）

3.3.6.3.1 功能

获取主机架 CPU 模块钥匙开关状态。



sysGetKeyState 指令

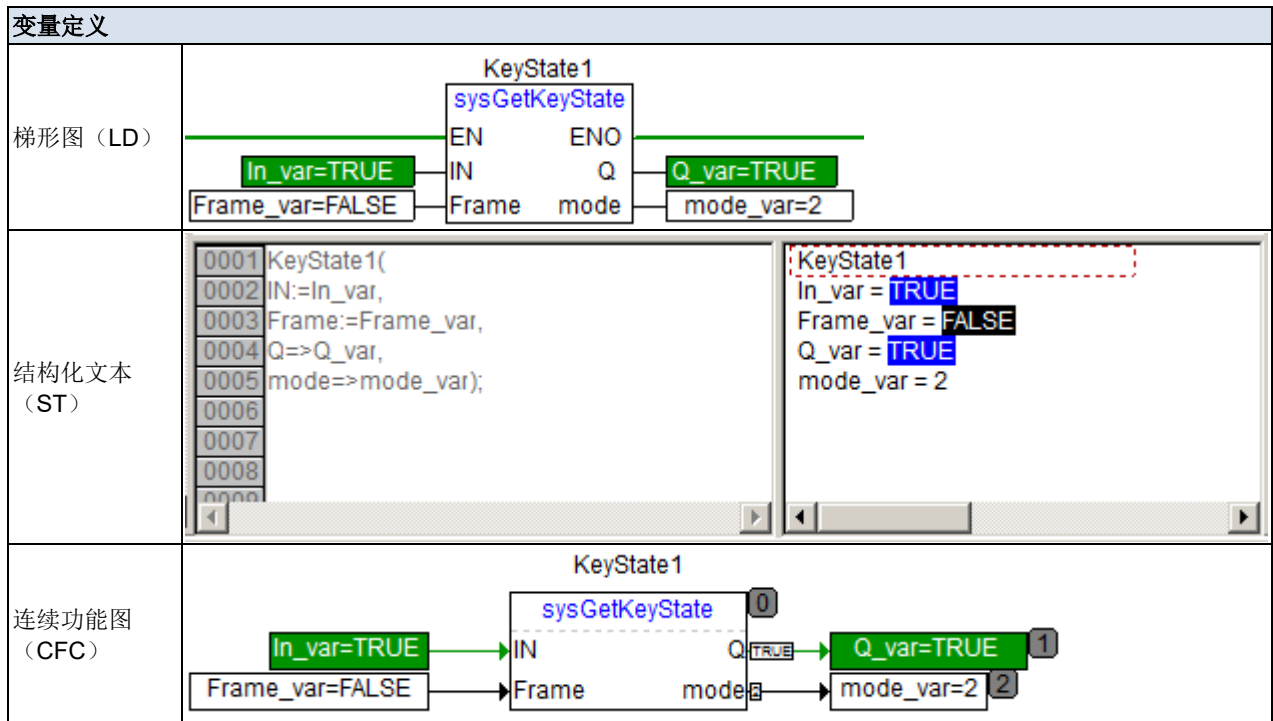
3.3.6.3.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能 (默认值 FALSE)
Frame	BOOL	机架选择（默认值 FALSE） TRUE：非主机架 FALSE：主机架

输出参数	类型	含义
Q	BOOL	输出 (默认值 FALSE)
Mode	BYTE	钥匙开关状态（默认值 0） 1: RUN 2: REM 3: PRG

3.3.6.3.3 指令使用举例

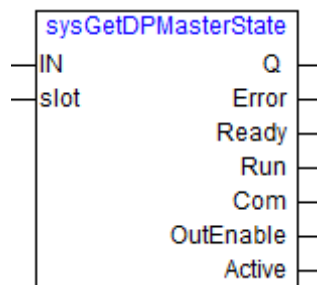
变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Frame_var			BOOL	FALSE	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	mode_var			BYTE	2	FALSE
0005	KeyState1			SYSGETKEYSTATE		FALSE
编程语言		程序				



3.3.6.4 sysGetDPMasterState (获取 DP 主卡状态)

3.3.6.4.1 功能

获取 DP 主站诊断信息。



sysGetDPMasterState 指令

3.3.6.4.2 参数说明

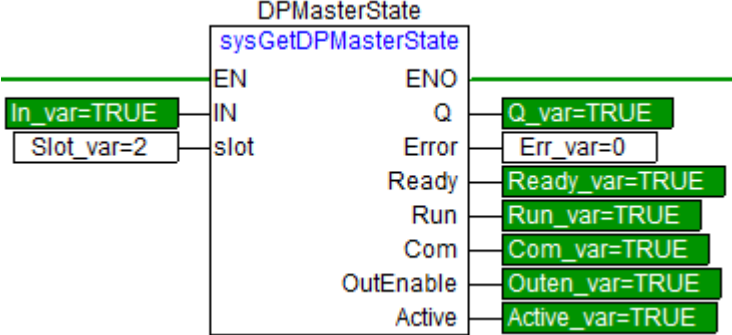
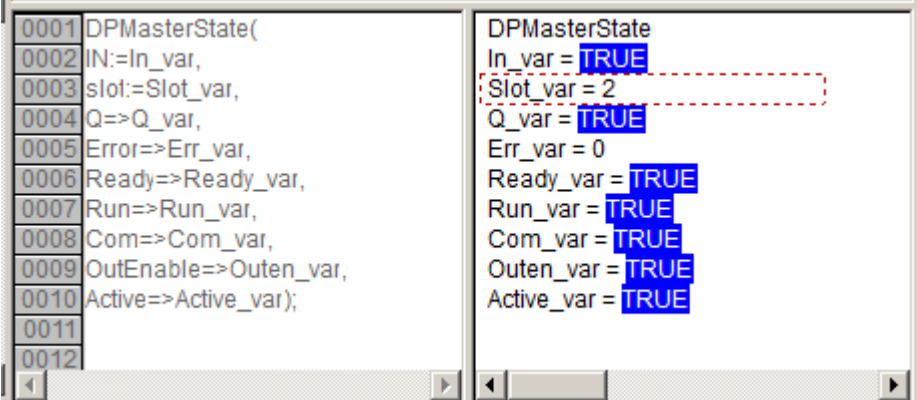
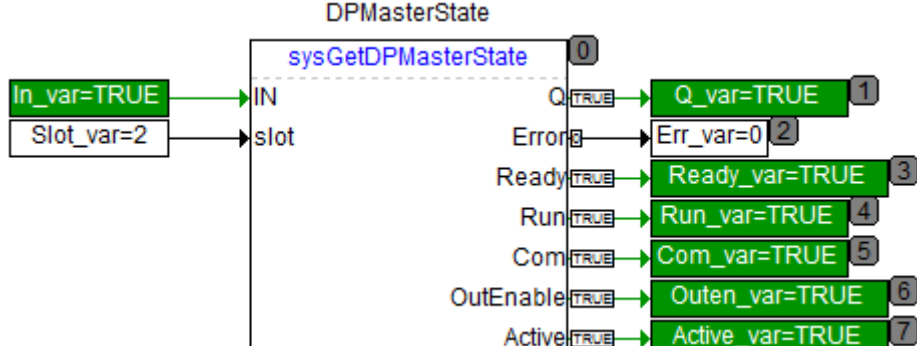
输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
Slot	BYTE	槽位号 （默认值 2，有效值为 2~5） 主控模块槽位号固定为 1

输入参数	类型	含义
		其它模块依据与主控位置向右递增 使用时输入想得到版本信息模块所在的槽位号

输出参数	类型	含义
Q	BOOL	输出，默认值 FALSE
Error	BYTE	错误，默认值 0 0: 正确 1: Slot 配置错误 128: 在对应槽位上没有检测到 DP 主站
Ready	BOOL	主站是否已准备好标识 FALSE: 主站初始化未完成 TRUE: 主站初始化完成
Run	BOOL	主站是否已运行标识 FALSE: 主站未接收完下装数据 TRUE: 主站接收下装数据完成，开始与从站通讯
Com	BOOL	主站是否与从站建立数据交换标识 FALSE: 还未与从站建立数据交换 TRUE: 至少与一个从站建立了数据交换
OutEnalbe	BOOL	主站输出使能标识 FALSE: 输出不使能 TRUE: 输出使能
Active	BOOL	主站活动状态标识 FALSE: 热备状态 TRUE: 活动状态

3.3.6.4.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Slot_var			BYTE	2	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	Err_var			BYTE	0	FALSE
0005	Ready_var			BOOL	TRUE	FALSE
0006	Run_var			BOOL	TRUE	FALSE
0007	Com_var			BOOL	TRUE	FALSE
0008	Outen_var			BOOL	TRUE	FALSE
0009	Active_var			BOOL	TRUE	FALSE
0010	DPMasterState			SYSGETDPMMASTERSTATE		FALSE
编程语言		程序				

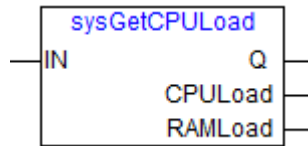
变量定义	
梯形图 (LD)	
结构化文本 (ST)	 <pre> 0001 DPMasterState(0002 IN:=In_var, 0003 slot:=Slot_var, 0004 Q:=Q_var, 0005 Error=>Err_var, 0006 Ready=>Ready_var, 0007 Run=>Run_var, 0008 Com=>Com_var, 0009 OutEnable=>Outen_var, 0010 Active=>Active_var); 0011 0012 </pre> <pre> DPMasterState In_var = TRUE Slot_var = 2 Q_var = TRUE Err_var = 0 Ready_var = TRUE Run_var = TRUE Com_var = TRUE Outen_var = TRUE Active_var = TRUE </pre>
连续功能图 (CFC)	

3.3.6.5 sysGetCPULoad（获取控制器负荷）

3.3.6.5.1 功能

获取控制器 CPU 使用率。

通过组态软件执行下装、冷复位等操作，可能导致 CPU 使用率变高。



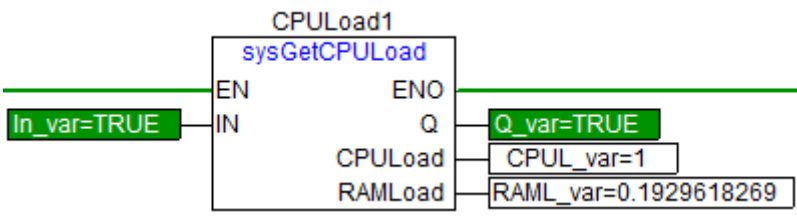
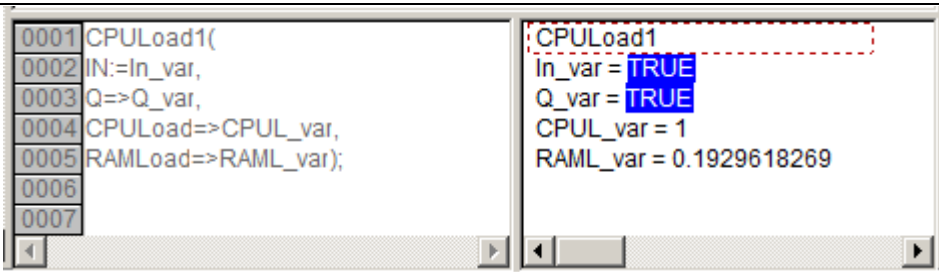
sysGetCPULoad 指令

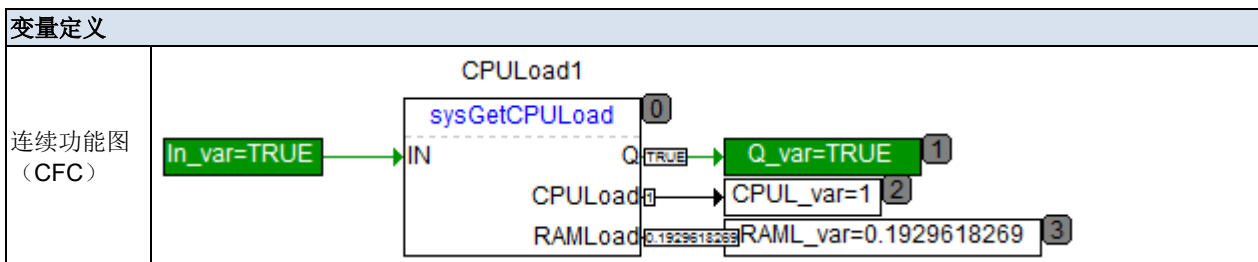
3.3.6.5.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）

输出参数	类型	含义
Q	BOOL	输出（默认值 FALSE）
CPULoad	REAL	CPU 使用率（默认值 0）
RAMLoad	REAL	内存使用率（默认值 0）

3.3.6.5.3 指令使用举例

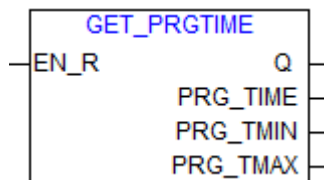
变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Q_var			BOOL	TRUE	FALSE
0003	CPUL_var			REAL	1	FALSE
0004	RAML_var			REAL	0.1929618269	FALSE
0005	CPULoad1			SYSGETCPULOAD		FALSE
编程语言	程序					
梯形图 (LD)						
结构化文本 (ST)						



3.3.6.6 GET_PRGTIME(获取工程时间信息)

3.3.6.6.1 功能

用户工程信息获取指令用于获取当前用户组态的用户工程实际运行时长信息。



GET_PRGTIME 指令模块示意图

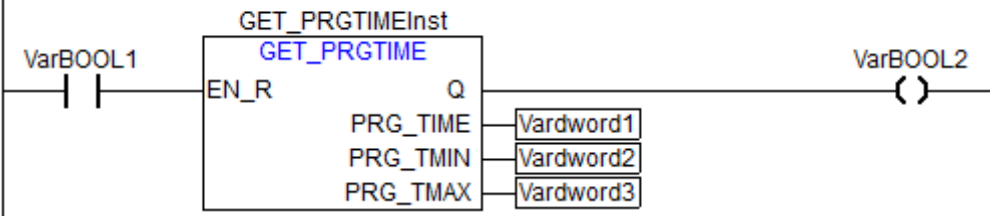
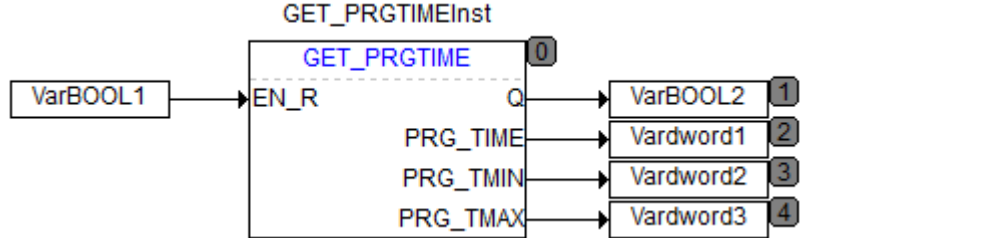
3.3.6.6.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	高电平使能保持	0

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	获取完成标志	获取完成标志，获取完成输出 1，否则输出 0
PRG_TIME	DWORD	工程运行时长	使能指令模块后，当前周期用户组态工程运算时长。单位：毫秒
PRG_TMIN	DWORD	工程运行最小时长	自工程运行开始，用户工程运算最小时长。单位：毫秒
PRG_TMAX	DWORD	工程运行最大时长	自工程运行开始，用户工程运算最大时长。单位：毫秒

3.3.6.6.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	GET_PRGTIMEInst			GET_PRGTIME		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	Vardword1			DWORD	0	FALSE
0004	Vardword3			DWORD	0	FALSE
0005	Vardword2			DWORD	0	FALSE
0006	VarBOOL2			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>GET_PRGTIMEInst(EN_R:=VarBOOL1, Q=>VarBOOL2, PRG_TIME=>Vardword1, PRG_TMIN=>Vardword2, PRG_TMAX=>Vardword3);</pre>
连续功能图 (CFC)	

3.3.6.7 sysGetHTCP_CycleTime(获取 Hollitcp 主站周期时间)

3.3.6.7.1 功能

获取 Hollitcp 主站下 IO 的周期时间

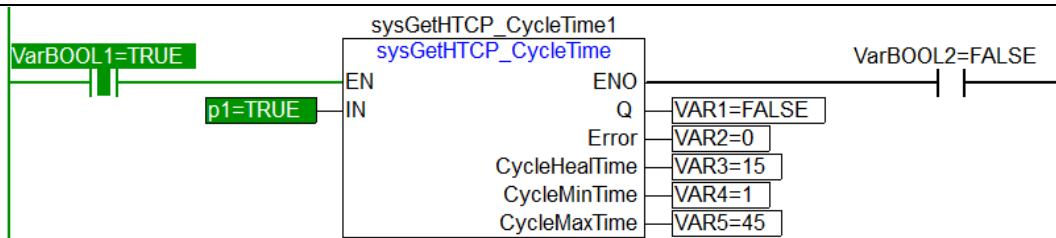
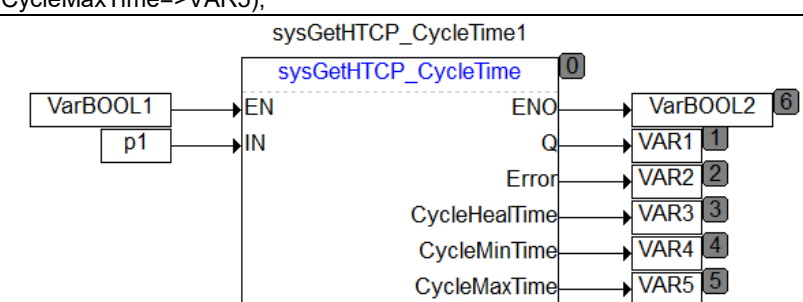
3.3.6.7.2 参数

输入参数	类型	含义
IN	BOOL	高电平使能

输出参数	类型	含义
Q	BOOL	输出 FALSE: 输出未完成 TRUE: 输出完成
Error	BYTE	错误码 0: 成功 128:Hollitcp 主站周期未输出
CycleHealTime	DWORD	Hollitcp 主站周期实时时间
CycleMinTime	DWORD	Hollitcp 主站周期最小时间
CycleMaxTime	DWORD	Hollitcp 主站周期最大时间

3.3.6.7.3 指令使用举例

变量定义							
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护	SOE使能
0001	sysGetHTCP_CycleTime1			SYSGETHTCP_CYCLETIME		FALSE	☐
0002	VarBOOL1			BOOL	TRUE	FALSE	☐
0003	VarBOOL2			BOOL	FALSE	FALSE	☐
0004	p1			BOOL	TRUE	FALSE	☐
0005	VAR1			BOOL	FALSE	FALSE	☐
0006	VAR2			BYTE	0	FALSE	☐
0007	VAR3			DWORD	15	FALSE	☐
0008	VAR4			DWORD	1	FALSE	☐
0009	VAR5			DWORD	45	FALSE	☐

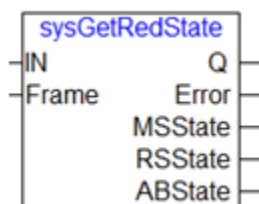
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> sysGetHTCP_CycleTime1(IN:=p1, Q=>VAR1, Error=>VAR2, CycleHealTime=>VAR3, CycleMinTime=>VAR4, CycleMaxTime=>VAR5); </pre>
连续功能图 (CFC)	

3.3.7 冗余相关

3.3.7.1 sysGetRedState（获取系统冗余状态）

3.3.7.1.1 功能

获取主从机架冗余状态信息。



sysGetRedState 指令

3.3.7.1.2 参数说明

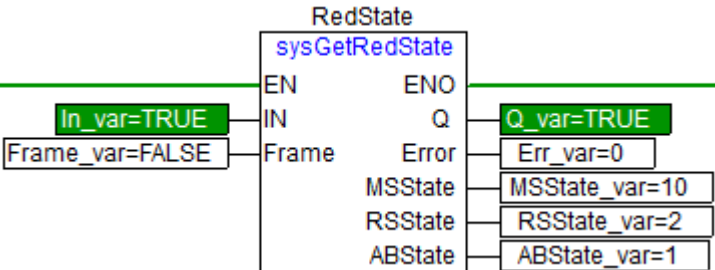
输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
Frame	BOOL	主从机架标识 FALSE：主机架 TRUE：从机架

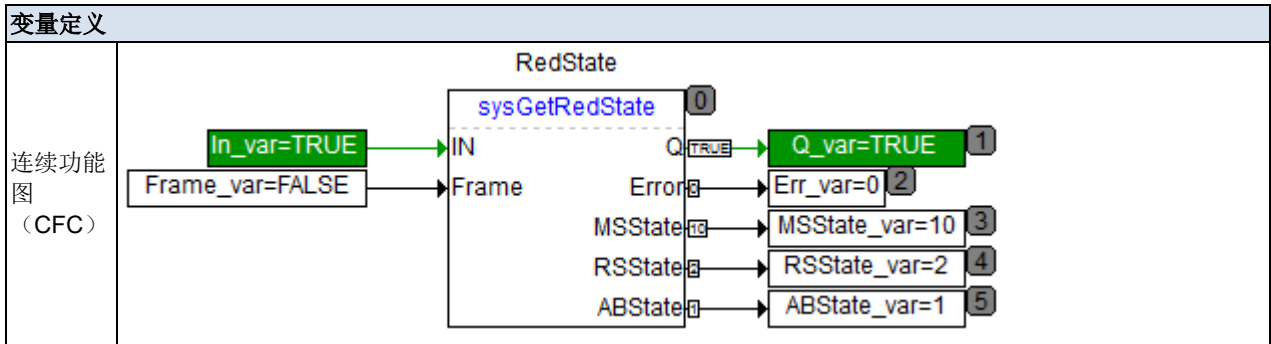
输出参数	类型	含义
Q	BOOL	输出（默认值 FALSE）
Error	BYTE	错误（默认值 0） 128：本机目前为冗余异常状态（对应主从机状态中的 0~6 及 11 项） 129：主从机钥匙开关状态不一致 130：双机系统下任务运行时间超出设置的任务周期 131：主从机之间冗余链路异常
MSState	BYTE	主从机状态（默认值 0） 0：未知态 1：初始态 2：硬件 Ready 态 3：双机 Ready 态 4：单机 Ready 态 5：故障态 6：错误态 7：工程冗余 8：工程验证 9：从机 10：主机 11：不合格主机
RSState	BYTE	单双机状态（默认值 0）

输出参数	类型	含义
		0: 未知态 1: 单机 2: 双机
ABState	BYTE	AB 机状态（默认值 0） 0: 未知态 1: A 机 2: B 机

3.3.7.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Frame_var			BOOL	FALSE	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	Err_var			BYTE	0	FALSE
0005	MSState_var			BYTE	10	FALSE
0006	RSState_var			BYTE	2	FALSE
0007	ABState_var			BYTE	1	FALSE
0008	RedState			SYSGETREDSTATE		FALSE

编程语言		程序
梯形图 (LD)		
结构化文本 (ST)		<pre> 0001 RedState(0002 IN:=In_var, 0003 Frame:=Frame_var, 0004 Q=>Q_var, 0005 Error=>Err_var, 0006 MSState=>MSState_var, 0007 RSState=>RSState_var, 0008 ABState=>ABState_var); 0009 0010 0011 </pre>



3.3.7.2 sysMasterSwitchToSlave (主从切换)

3.3.7.2.1 功能

将主机切换为从机。



sysMasterSwitchToSlave 指令

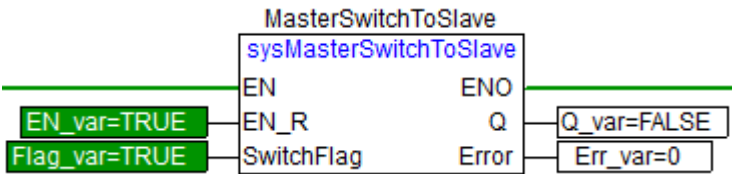
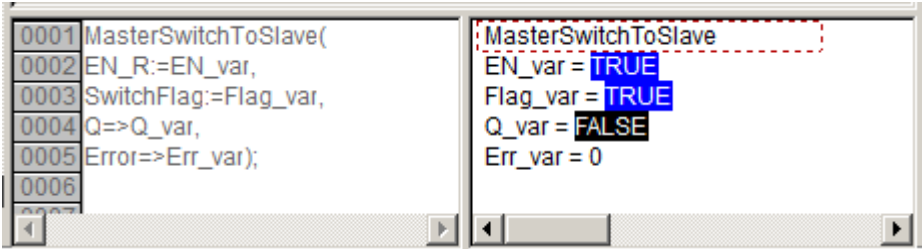
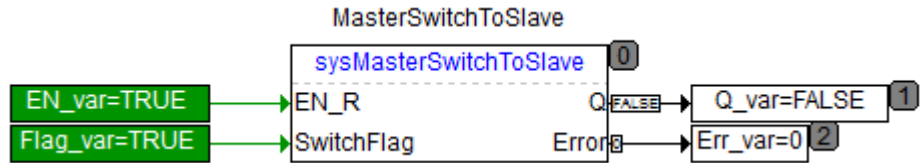
3.3.7.2.2 参数说明

输入参数	类型	含义
EN_R	BOOL	使能 FALSE: 无效 (默认值) TRUE: 上升沿有效
SwitchFlag	BOOL	FALSE: 不允许切换 TRUE: 允许切换

输出参数	类型	含义
Q	BOOL	输出, 默认值 FALSE
Error	BYTE	错误, 默认值 0 0: 正确 128: 单机系统 129: 主从同步尚未完成 130: 从机的钥匙开关在 PRG 状态

3.3.7.2.3 指令使用举例

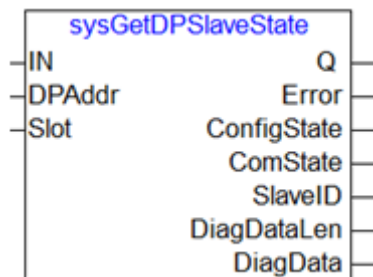
变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	EN_var			BOOL	TRUE	FALSE
0002	Flag_var			BOOL	TRUE	FALSE
0003	Q_var			BOOL	FALSE	FALSE
0004	Err_var			BYTE	0	FALSE
0005	MasterSwitchToSlave			SYSMASTERSWITCHTOSLAVE		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	
连续功能图 (CFC)	

3.3.7.3 sysGetDPSlaveState (获取 DP 从站诊断)

3.3.7.3.1 功能

获取 DP 从站诊断信息。



sysGetDPSlaveState 指令

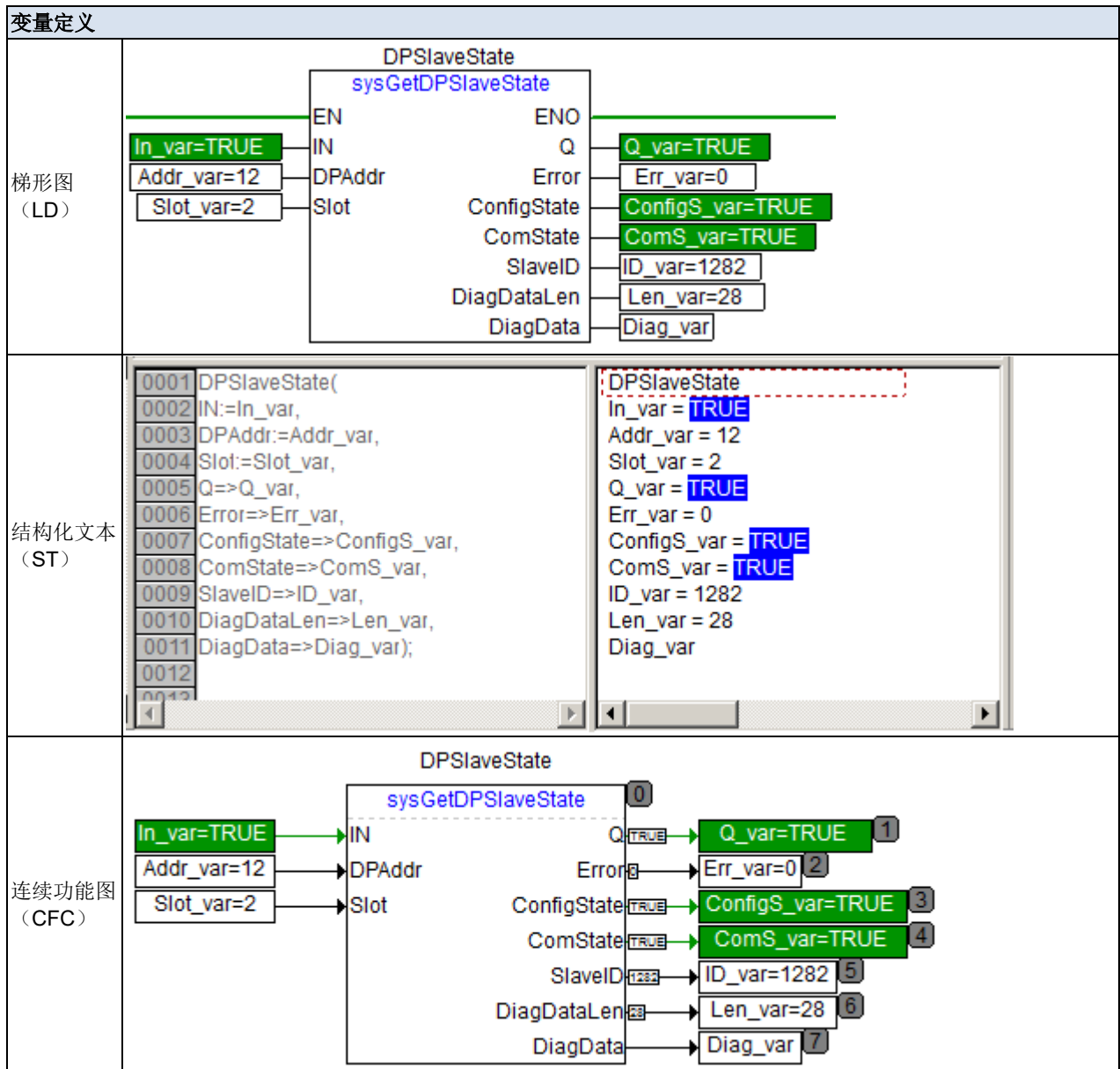
3.3.7.3.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能，默认值 FALSE
DPAddr	BYTE	从站地址 (默认值为 0，有效值为 2~125)
Slot	BYTE	DP 主卡所在槽号

输出参数	类型	参数描述
Q	BOOL	输出，默认值 FALSE
Error	BYTE	错误（默认值 0） 0：正确 1：槽号错误 2：从站地址配置错误 128：没有 DP 主卡或槽号不匹配
ConfigState	BOOL	从站是否被配置标识： TRUE：已被配置 FALSE：未被配置
ComState	BOOL	从站通讯状态标识： TRUE：通讯正常 FALSE：通讯异常
SlaveID	WORD	从站 ID（默认值 0） 注：LK 从站模块的 ID 范围为 0x0500 ~ 0x05FF
DiagDataLen	BYTE	从站诊断信息长度
DiagData	ARRAY[0...51] OF BYTE	诊断信息（具体含义根据从站类型而变） DiagData[50]：从站 DP 网故障状态 bit0：DP1 网故障状态，1 为故障，0 为正常 bit1：DP2 网故障状态，1 为故障，0 为正常

3.3.7.3.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Addr_var			BYTE	12	FALSE
0003	Slot_var			BYTE	2	FALSE
0004	Q_var			BOOL	TRUE	FALSE
0005	Err_var			BYTE	0	FALSE
0006	ConfigS_var			BOOL	TRUE	FALSE
0007	ComS_var			BOOL	TRUE	FALSE
0008	ID_var			WORD	1282	FALSE
0009	Len_var			BYTE	28	FALSE
0010	Diag_var			ARRAY[0..51] OF BYTE		FALSE
0011	DPSlaveState			SYSGETDPSLAVESTATE		FALSE
编程语言		程序				



程序说明：

示例为获取从站地址为 12 的模块 LK411 的诊断信息。

双击数组变量 Diag_var，查看各字节诊断信息，如图所示。

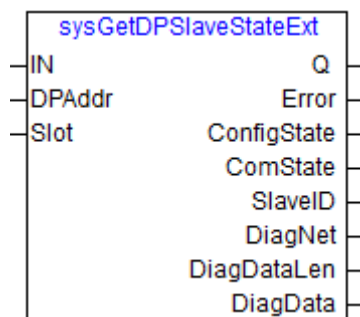
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	Diag_var[0]			BYTE	2	FALSE
0002	Diag_var[1]			BYTE	0	FALSE
0003	Diag_var[2]			BYTE	66	FALSE
0004	Diag_var[3]			BYTE	1	FALSE
0005	Diag_var[4]			BYTE	128	FALSE
0006	Diag_var[5]			BYTE	64	FALSE
0007	Diag_var[6]			BYTE	160	FALSE
0008	Diag_var[7]			BYTE	128	FALSE
0009	Diag_var[8]			BYTE	65	FALSE
0010	Diag_var[9]			BYTE	160	FALSE
0011	Diag_var[10]			BYTE	128	FALSE
0012	Diag_var[11]			BYTE	66	FALSE
0013	Diag_var[12]			BYTE	160	FALSE
0014	Diag_var[13]			BYTE	128	FALSE
0015	Diag_var[14]			BYTE	67	FALSE
0016	Diag_var[15]			BYTE	160	FALSE

查看诊断信息

3.3.7.4 sysGetDP SlaveStateExt（获取 DP 从站诊断）

3.3.7.4.1 功能

获取 DP 从站诊断信息，该功能块适用于诊断数据字节较大情况，一般超过 52 Bytes 时，建议使用该功能块进行 DP 诊断。



sysGetDP SlaveStateExt 指令

3.3.7.4.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能，默认值 FALSE
DPAddr	BYTE	从站地址 (默认值为 0，有效值为 2~125)
Slot	BYTE	DP 主卡所在槽号

输出参数	类型	参数描述
Q	BOOL	输出，默认值 FALSE
Error	BYTE	错误（默认值 0） 0：正确 1：槽号错误 2：从站地址配置错误 128：没有 DP 主卡或槽号不匹配
ConfigState	BOOL	从站是否被配置标识： TRUE：已被配置 FALSE：未被配置
ComState	BOOL	从站通讯状态标识： TRUE：通讯正常 FALSE：通讯异常
SlaveID	WORD	从站 ID（默认值 0） 注：LK 从站模块的 ID 范围为 0x0500 ~ 0x05FF
DiagNet	BYTE	从站 DP 网故障状态 bit0：DP1 网故障状态 bit1：DP2 网故障状态
DiagDataLen	BYTE	从站诊断信息长度
DiagData	ARRAY[0...237] OF BYTE	从站诊断信息，具体含义根据从站类型而变

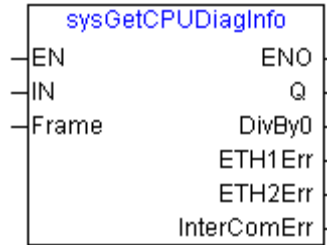
3.3.7.4.3 指令使用举例

请参见 [sysGetDPSlaveState（获取 DP 从站诊断）](#)。

3.3.7.5 sysGetCPUDiagInfo（获取主控模块诊断）

3.3.7.5.1 功能

获取主控模块诊断信息。



sysGetCPUDiagInfo 指令

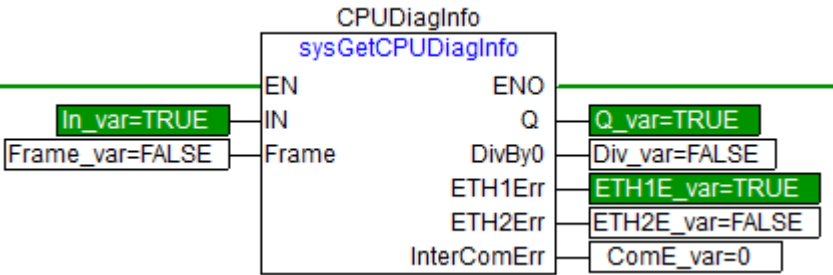
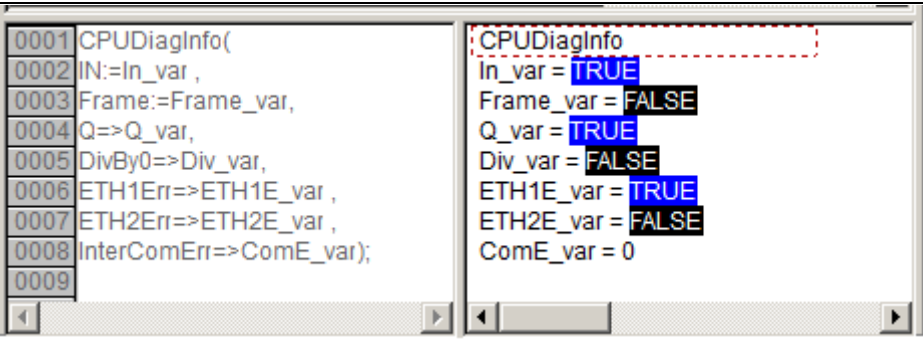
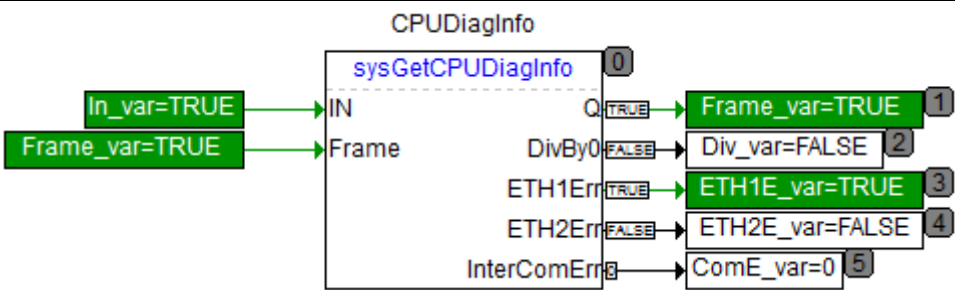
3.3.7.5.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能，默认值 FALSE
Frame	BOOL	机架选择，FALSE 为本机架(主)，TRUE 为对方机架（从）

输出参数	类型	含义
Q	BOOL	输出（默认值 FALSE）
DivBy0	BOOL	默认值为 FALSE，当尝试用零除时，DivBy0 被置位为 TRUE，此时，需要检查工程逻辑，确保除零运算消除后，重新下装工程才可恢复为 FALSE
ETH1Err	BOOL	第 1 路以太网故障信息，当断线时此值为 TRUE（默认值 FALSE）
ETH2Err	BOOL	第 2 路以太网故障信息，当断线时此值为 TRUE（默认值 FALSE）
InterComErr	BYTE	内部通信链路故障信息，当不同通道出现故障时，此值对应 bit 为 1(默认值 0): bit0 = 1：槽 2 内部通信链路故障 bit1 = 1：槽 3 内部通信链路故障 bit2 = 1：槽 4 内部通信链路故障(6 槽背板预留) bit3 = 1：槽 5 内部通信链路故障(6 槽背板预留) bit4 = 1：槽位不匹配故障 bit5~bit7：保留

3.3.7.5.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Frame_var			BOOL	FALSE	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	Div_var			BOOL	FALSE	FALSE
0005	ETH1E_var			BOOL	TRUE	FALSE
0006	ETH2E_var			BOOL	FALSE	FALSE
0007	ComE_var			BYTE	0	FALSE
0008	CPUDiagInfo			SYSGETCPUDIAGINFO		FALSE

变量定义	
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> 0001 CPUDiagInfo(0002 IN:=In_var, 0003 Frame:=Frame_var, 0004 Q=>Q_var, 0005 DivBy0=>Div_var, 0006 ETH1Err=>ETH1E_var, 0007 ETH2Err=>ETH2E_var, 0008 InterComErr=>ComE_var); 0009 </pre> 
连续功能图 (CFC)	

程序说明：

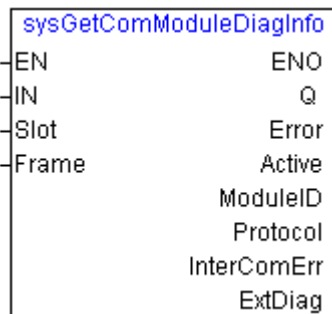
示例为获取主机架中 CPU 模块的诊断信息，此时，第一路以太网故障。

3.3.7.6 sysGetComModuleDiagInfo（获取通讯模块诊断）

3.3.7.6.1 功能

获取通讯模块诊断信息。

注：本功能块可获取主控模块、LK249、LK240 模块的诊断信息。LK246 和 LK241 模块诊断信息通过双击模块树节点，在模块信息配置页面的【诊断信息】标签页中可查看。【诊断信息】标签在工程编译通过后显示。



sysGetComModuleDiagInfo 指令

3.3.7.6.2 参数说明

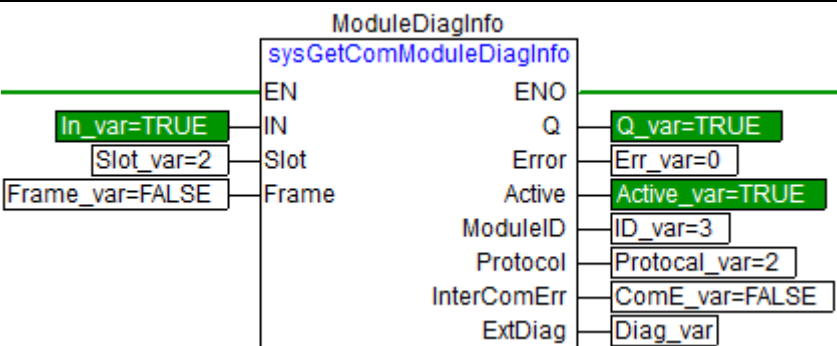
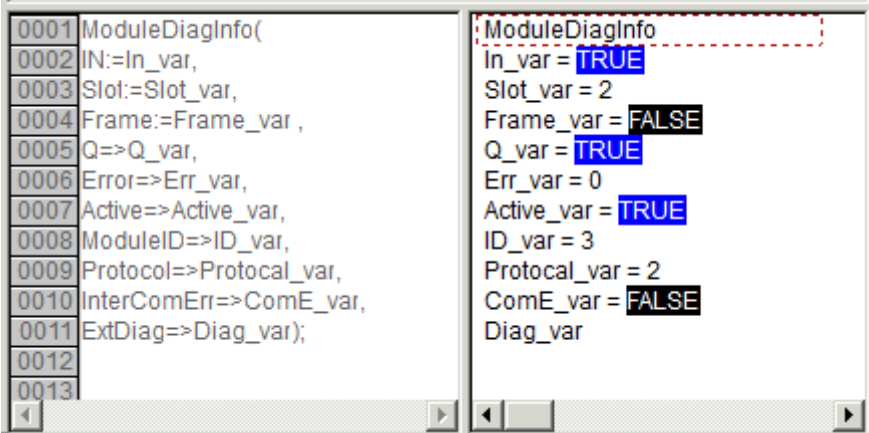
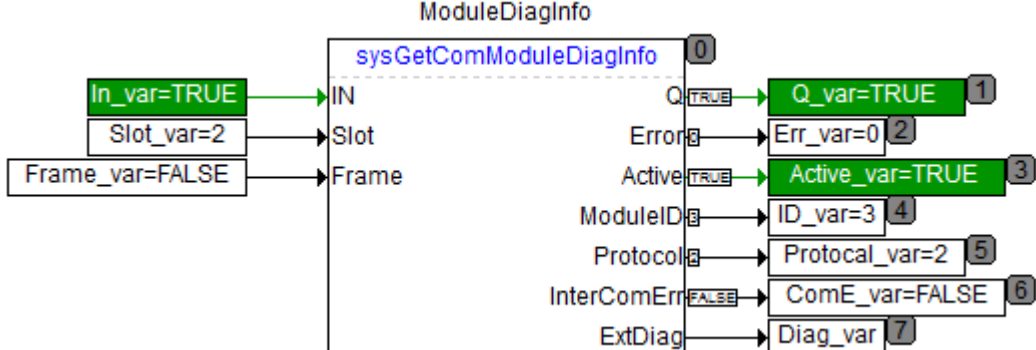
输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
Slot	BYTE	槽位号（默认值 2，有效值为 2~5） 主控模块槽位号固定为 1 其它模块依据与主控位置向右递增 使用时输入想得到版本信息模块所在的槽位号
Frame	BOOL	机架选择，FALSE 为本机架（主），TRUE 为对方机架（从）

输出参数	类型	含义
Q	BOOL	输出（默认值 FALSE）
Error	BYTE	错误（默认值 0） 0: 正确 1: Slot 配置错误，实际配置地址与安装槽位不匹配 128: 本机架槽位上不存在模块 129: 对方机架槽位上不存在模块 130: 该槽位模块不支持此功能
Active	BOOL	通信模块工作状态（默认值 0） FALSE: 离线 TRUE: 在线
ModuleID	BYTE	模块 ID 号 1: LK220、LK222、LK224（保留） 2: LK240 3: LK249 4: LK247 5: LK248 6: LK245
Protocol	BYTE	通信协议标识号 1: 内部冗余协议 2: DP 协议 3: CANOpen 协议 4: EtherCAT 协议 5: PowerLink 协议 6: ModbusRTU 协议 7: 自由口协议 8: ModbusTCP 协议
InterComErr	BOOL	内部通信链路故障信息，当出现故障时此值为 TRUE

输出参数	类型	含义
		(默认值 0)
ExtDiag	ARRAY[0...31] OF BYTE	扩展诊断信息 LK240: 第 1 个字节代表第 1 路光纤断线故障 (对应值为 1) 第 2 个字节代表第 2 路光纤断线故障 (对应值为 1) 第 3 个字节代表 A/B 系开关冲突故障 (对应值为 1) 第 4 个字节代表清看门狗复位结果标志 (对应值为 1) 第 5 个字节保留 第 6 个字节代表第 1 路光纤的误码率数值 (0~100 之间) 第 7 个字节代表第 2 路光纤的误码率数值 (0~100 之间) 其它字节目前为预留
		LK249: 第 1 个字节代表第 1 路 485 断线故障 (对应值为 1) 第 2 个字节代表第 2 路 485 断线故障 (对应值为 1) 第 3 个字节保留 第 4 个字节代表清看门狗复位结果标志 (对应值为 1) 其它字节目前为预留
		LK245: 第 1 个字节代表第 1 路以太网故障 (对应值为 1) 第 2 个字节代表第 2 路以太网故障 (对应值为 1) 其它字节目前为预留

3.3.7.6.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	In_var			BOOL	TRUE	FALSE
0002	Slot_var			BYTE	2	FALSE
0003	Frame_var			BOOL	FALSE	FALSE
0004	Q_var			BOOL	TRUE	FALSE
0005	Err_var			BYTE	0	FALSE
0006	Active_var			BOOL	TRUE	FALSE
0007	ID_var			BYTE	3	FALSE
0008	Protocal_var			BYTE	2	FALSE
0009	ConE_var			BOOL	FALSE	FALSE
0010	Diag_var			ARRAY[0..31] OF BYTE		FALSE
0011	ModuleDiagInfo			SYSGETCOMMModuleDIAGINFO		FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	
连续功能图 (CFC)	

程序说明:

示例为获取主机架中 LK249 模块的诊断信息。

双击数组变量 **Diag_var**，打开诊断信息如图所示。第一个字节在线值为 1，表示第一路 DP 链路故障。

Comdiag1.Diag_var						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0001	Diag_var[0]			BYTE	1	FALSE
0002	Diag_var[1]			BYTE	0	FALSE
0003	Diag_var[2]			BYTE	0	FALSE
0004	Diag_var[3]			BYTE	0	FALSE
0005	Diag_var[4]			BYTE	0	FALSE
0006	Diag_var[5]			BYTE	0	FALSE
0007	Diag_var[6]			BYTE	0	FALSE
0008	Diag_var[7]			BYTE	0	FALSE
0009	Diag_var[8]			BYTE	0	FALSE
0010	Diag_var[9]			BYTE	0	FALSE
0011	Diag_var[10]			BYTE	0	FALSE
0012	Diag_var[11]			BYTE	0	FALSE
0013	Diag_var[12]			BYTE	0	FALSE
0014	Diag_var[13]			BYTE	0	FALSE
0015	Diag_var[14]			BYTE	0	FALSE
0016	Diag_var[15]			BYTE	0	FALSE

[查看诊断信息](#)

3.3.8 内存操作指令

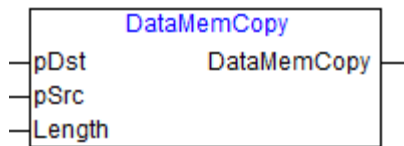
3.3.8.1 DataMemCopy（从源地址拷贝数据到目的地址）

3.3.8.1.1 功能

以字节为单位，将数据从源地址拷贝到目的地址。源数据可选择所有数据区数据，目的数据不能为 S 区数据。

使用指令时，首先需要获取源数据和目的数据内存地址，条件满足后，将源数据内存地址开始的一定长度的数据传送到目的内存地址段。

需要注意的是，数据长度必须小于等于指针所指源数据和目的数据的大小。



DataMemCopy 块

3.3.8.1.2 参数说明

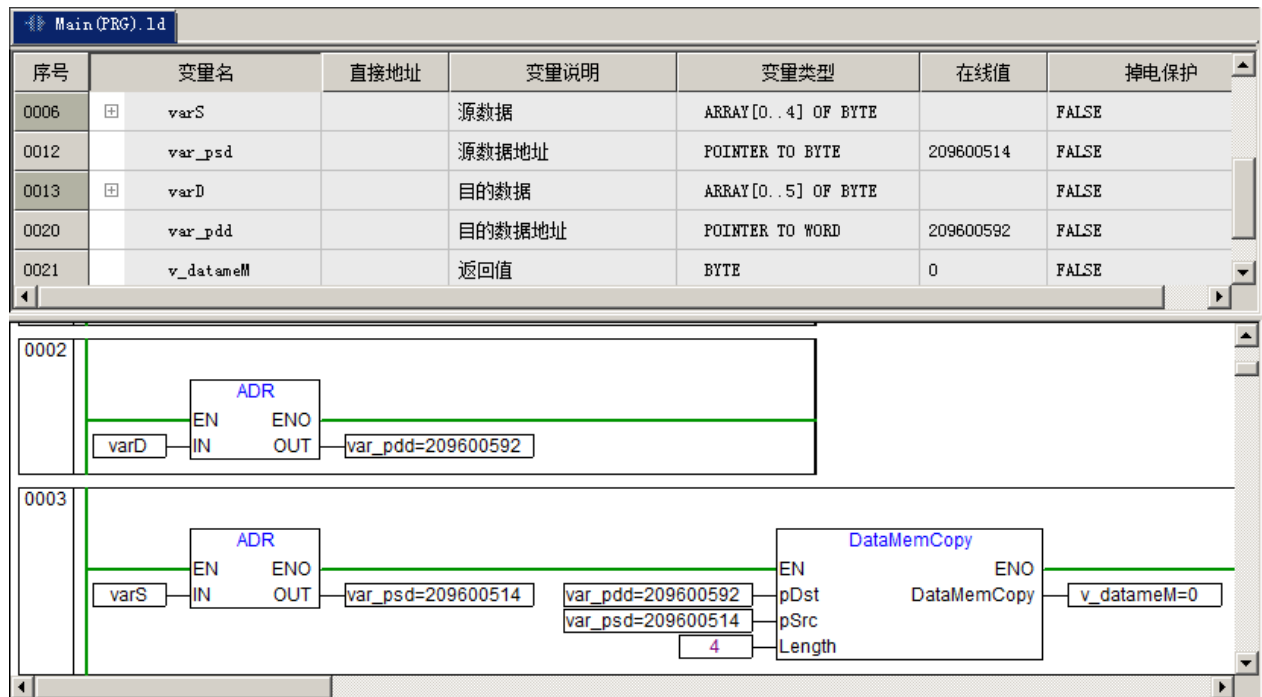
输入参数	数据类型	功能描述	参数值说明	默认值
pDst	POINTER TO BYTE	目的数据地址	数据区有效地址	0
pSrc	POINTER TO BYTE	源数据地址	数据区有效地址	0
Length	DWORD	数据长度	单位是 1 字节	0

输出参数	数据类型	功能描述	参数值说明
DataMemCopy	BYTE	错误信息	0: 执行成功 1: 源数据地址错误 2: 目的数据地址错误

3.3.8.1.3 指令使用举例

将数组 varS 中的四字节数据拷贝到数组 varD 中。

设置指针变量 var_psd 和 var_pdd，分别获取源数据和目的数据内存地址。设置拷贝数据长度为 4 字节，组态示例如下所示。



DataMemCopy 组态示例

运行程序，数组元素 varS[0]~varS[3]被拷贝到数组元素 varD[0]~varD[3]中，如下图所示。

Main(PRG).ld						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0006	varS		源数据	ARRAY[0..4] OF BYTE		FALSE
0007	varS[0]			BYTE	1	FALSE
0008	varS[1]			BYTE	2	FALSE
0009	varS[2]			BYTE	3	FALSE
0010	varS[3]			BYTE	4	FALSE
0011	varS[4]			BYTE	1	FALSE
0012	var_psd		源数据地址	POINTER TO BYTE	209600514	FALSE
0013	varD		目的数据	ARRAY[0..5] OF BYTE		FALSE
0014	varD[0]			BYTE	1	FALSE
0015	varD[1]			BYTE	2	FALSE
0016	varD[2]			BYTE	3	FALSE
0017	varD[3]			BYTE	4	FALSE
0018	varD[4]			BYTE	0	FALSE

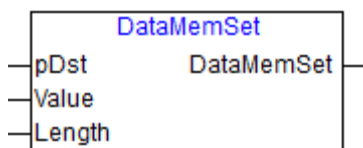
运行结果

3.3.8.2 DataMemSet(设置目的地址为指定数据值)

3.3.8.2.1 功能

向指定地址中写入数据值。目的数据不能为 S 区数据。

需要注意的是，数据长度必须小于等于指针所指目的数据的大小。



DataMemSet 块

3.3.8.2.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
pDst	POINTER TO BYTE	目的数据地址	数据区有效地址	0
Value	BYTE	数据值	设置的数据值	0

输入参数	数据类型	功能描述	参数值说明	默认值
Length	DWORD	数据长度	单位是 1 字节	0

输出参数	数据类型	功能描述	参数值说明
DataMemSet	BYTE	错误信息	0: 执行成功 1: 目的数据地址错误

3.3.8.2.3 指令使用举例

拷贝内存数据前，请先将目的地址数据复位为设定值 6。

设置指针变量 `var_psd` 和 `var_pdd`，分别获取源数据和目的数据内存地址。

内存拷贝指令的输出使能信号取反，作为 `DataMemSet` 指令的使能信号。在内存拷贝前，先将目的数据复位为设定值。

当内存拷贝使能信号 `var_en` 为 TRUE，执行 `DataMemCopy` 指令，源数据被拷贝到目的地址。

Main (PRG).ld

序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0006	varS		源数据	ARRAY[0..4] OF BYTE		FALSE
0012	var_psd		源数据地址	POINTER TO BYTE	209600514	FALSE
0013	varD		目的数据	ARRAY[0..5] OF BYTE		FALSE
0020	var_pdd		目的数据地址	POINTER TO WORD	209600519	FALSE
0021	v_datameM		返回值	BYTE	0	FALSE
0022	V_datameS			BYTE	0	FALSE
0023	V_out		内存复位使能信号	BOOL	TRUE	FALSE
0024	var_en		数据拷贝使能信号	BOOL	TRUE	FALSE

0002

varD

ADR

ENENO

INOUT

var_pdd=209600519

0003

varS

ADR

ENENO

INOUT

var_psd=209600514

0004

V_out=TRUE

var_pdd=209600519

64

Value

Length

DataMemSet

ENENO

DataMemSet

V_datameS=0

0005

var_en=TRUE

var_pdd=209600519

var_psd=209600514

4

Length

DataMemCopy

ENENO

DataMemCopy

v_datameM=0

V_out=TRUE

0006

复位内存值组态示例

Main (PRG).ld

序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0006	varS		源数据	ARRAY[0..4] OF BYTE		FALSE
0007	varS[0]			BYTE	1	FALSE
0008	varS[1]			BYTE	2	FALSE
0009	varS[2]			BYTE	3	FALSE
0010	varS[3]			BYTE	4	FALSE
0011	varS[4]			BYTE	1	FALSE
0012	var_psd		源数据地址	POINTER TO BYTE	209600514	FALSE
0013	varD		目的数据	ARRAY[0..5] OF BYTE		FALSE
0014	varD[0]			BYTE	6	FALSE
0015	varD[1]			BYTE	6	FALSE
0016	varD[2]			BYTE	6	FALSE
0017	varD[3]			BYTE	6	FALSE
0018	varD[4]			BYTE	0	FALSE

0004

V_out=FALSE

EN

var_pdd=209600519

pDst

Value

Length

4

6

DataMemSet

ENO

DataMemSet

V_datameS=0

0005

var_en=FALSE

EN

var_pdd=209600519

pDst

var_psd=209600514

pSrc

Length

4

DataMemCopy

ENO

DataMemCopy

v_datameM=0

V_out=FALSE

内存被写入设定值

Main(PRG).ld

序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护
0006	varS		源数据	ARRAY[0..4] OF BYTE		FALSE
0007	varS[0]			BYTE	1	FALSE
0008	varS[1]			BYTE	2	FALSE
0009	varS[2]			BYTE	3	FALSE
0010	varS[3]			BYTE	4	FALSE
0011	varS[4]			BYTE	1	FALSE
0012	var_psd		源数据地址	POINTER TO BYTE	209600514	FALSE
0013	varD		目的数据	ARRAY[0..5] OF BYTE		FALSE
0014	varD[0]			BYTE	1	FALSE
0015	varD[1]			BYTE	2	FALSE
0016	varD[2]			BYTE	3	FALSE
0017	varD[3]			BYTE	4	FALSE
0018	varD[4]			BYTE	0	FALSE

0004

V_out=TRUE

0005

var_en=TRUE

执行内存数据拷贝

3.3.9 中断设置指令

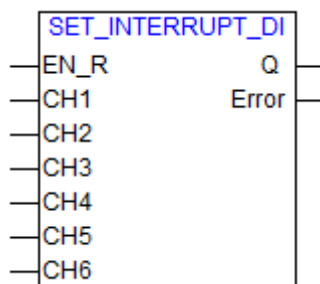
3.3.9.1 SET_INTERRUPT_DI（设置快速外部中断）

3.3.9.1.1 模块支持

14 点（LE5104、LE5105）支持 CH1、CH2。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118）支持全部 6 路通道。

3.3.9.1.2 功能描述

设置快速外部中断参数指令可以对一个或者多个输入通道进行快速外部中断设置，但只针对 6 个通道，默认为 0，即外部中断禁止。



SET_INTERRUPT_DI 功能块

3.3.9.1.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能
CH1	BYTE	第 1 个通道外部中断参数设置	通道 1: 0 中断禁止; 1 上升沿触发中断; 2 下降沿触发中断; 3 上升沿和下降沿触发中断
CH2	BYTE	第 2 个通道外部中断参数设置	通道 2: 0 中断禁止; 1 上升沿触发中断; 2 下降沿触发中断; 3 上升沿和下降沿触发中断
CH3	BYTE	第 3 个通道外部中断参数设置	通道 3: 0 中断禁止; 1 上升沿触发中断; 2 下降沿触发中断; 3 上升沿和下降沿触发中断
CH4	BYTE	第 4 个通道外部中断参数设置	通道 4: 0 中断禁止; 1 上升沿触发中断; 2 下降沿触发中断; 3 上升沿和下降沿触发中断
CH5	BYTE	第 5 个通道外部中断参数设置	通道 5: 0 中断禁止; 1 上升沿触发中断; 2 下降沿触发中断; 3 上升沿和下降沿触发中断
CH6	BYTE	第 6 个通道外部中断参数设置	通道 6: 0 中断禁止; 1 上升沿触发中断; 2 下降沿触发中断; 3 上升沿和下降沿触发中断

输出参数	数据类型	功能描述	值
Q	BOOL	设置完成标志	0: 设置参数失败 1: 设置参数成功
Error	BYTE	错误信息	0: 无错误信息 1: 配置参数错误 2: 调用多个功能块 8: 资源被占用 (例如: DI 通道已被配置为脉冲捕获或高速计数。)



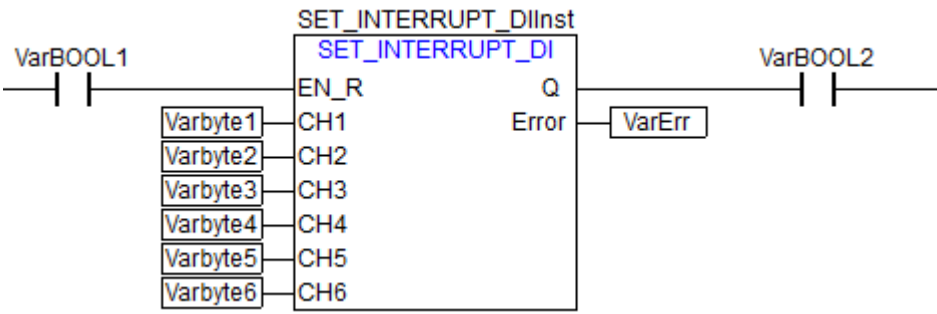
- 第 1 个通道设置外部中断功能端 CH1: 参数取值范围为 0~3; 当设置 CH1=0 时, 功能块禁止外部中断; 当设置 CH1=1 时, 功能块使能后, 输入脉冲有上升沿变化时, 就可以触发外部中断; 当设置 CH1=2 时, 功能块使能后, 输入脉冲有下降沿变化时, 就可以触发外部中断; 当设置 CH1=3 时, 功能块使能后,

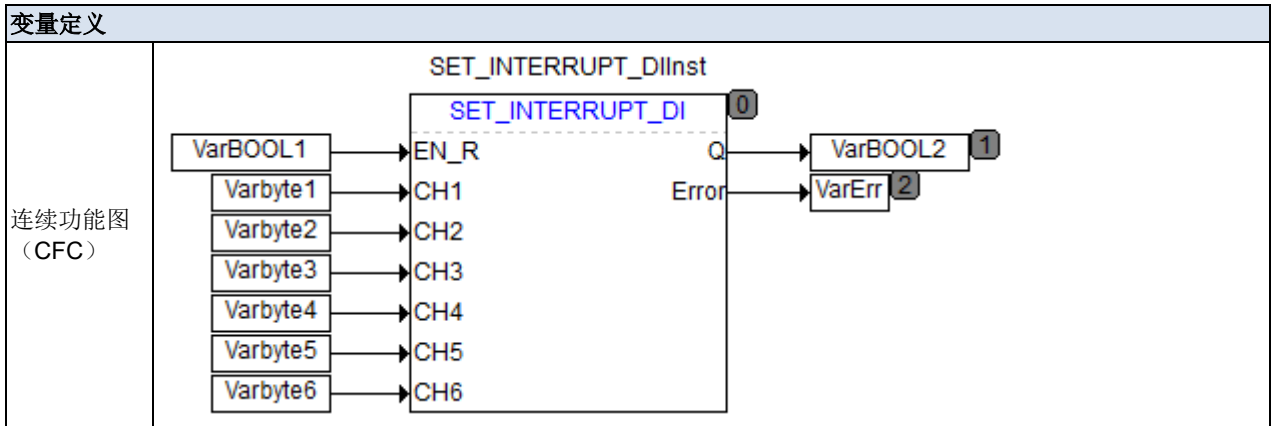
输入脉冲有上升沿或者下降沿变化时，就可以触发外部中断。其它通道依此类推。

- 14 点，快速外部中断通道安排在 IO.0~IO.1；24 点，快速外部中断安排在 IO.0~IO.5；40 点，快速外部中断前四路安排在 IO.0~IO.3，后两路安排在 I1.4~1.5。

3.3.9.1.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	SET_INTERRUPT_DIInst			SET_INTERRUPT_DI		FALSE
0002	Varbyte1			BYTE	0	FALSE
0003	Varbyte2			BYTE	0	FALSE
0004	Varbyte3			BYTE	0	FALSE
0005	Varbyte4			BYTE	0	FALSE
0006	Varbyte5			BYTE	0	FALSE
0007	Varbyte6			BYTE	0	FALSE
0008	VarErr			BYTE	0	FALSE
0009	VarBOOL1			BOOL	FALSE	FALSE
0010	VarBOOL2			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> SET_INTERRUPT_DIInst(EN_R:=VarBOOL1, CH1:=Varbyte1, CH2:=Varbyte2, CH3:=Varbyte3, CH4:=Varbyte4, CH5:=Varbyte5, CH6:=Varbyte6, Q=>VarBOOL2, Error=> VarErr); </pre>



程序说明：

EN_R 置位并保持时，Q 等于 TRUE，相应的输入通道每到达一个上升沿，就触发相应的快速外部中断。

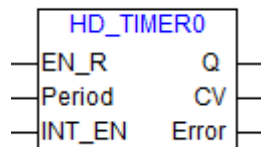
EN_R 复位时，相应的输入通道停止接收中断脉冲，Q 等于 FALSE。

3.3.9.2 HD_TIMER0（硬件中断定时器 0）

3.3.9.2.1 模块支持

14 点（LE5104、LE5105）支持前 4 个中断定时器。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118、LE5119）支持全部 6 路通道。

3.3.9.2.2 功能



HD_TIMER0 功能块

3.3.9.2.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0: 无效 1: 上升沿使能
Period	DWORD	设定时间 (us)	500 us ~800000 us 可设置
INT_EN	BOOL	中断使能	0: 设定时间到不产生中断 1: 设定时间到产生中断

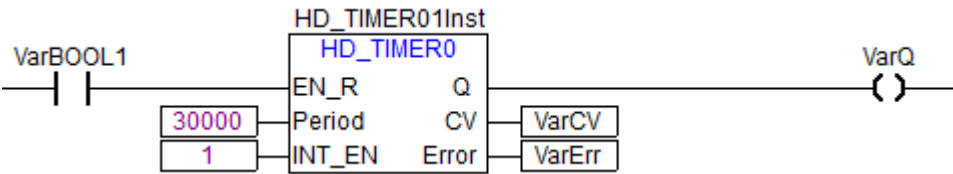
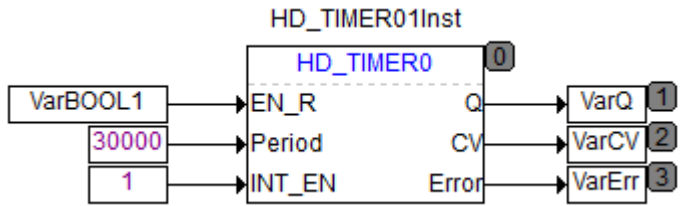
输出参数	数据类型	功能描述	参数值说明
Q	BOOL	是否设置完毕	0: 未设置 1: 设置完毕, 保持
CV	DWORD	已过时间 (us)	0~800000 us
Error	BYTE	错误信息	=0: 无错误信息 =1: 参数设置错误 =2: 硬件定时器未运算 =3: 调用多个相同功能块

① 共有 6 个硬件定时器, 名称为 HD_TIMER0~5, 每个硬件定时器对应 1 路定时器中断。

3.3.9.2.4 指令使用举例

HD_TIMER1~HD_TIMER5 与此类似。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HD_TIMER01Inst			HD_TIMER0		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarCV			DWORD	0	FALSE
0004	VarErr			BYTE	0	FALSE
0005	VarQ			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> HD_TIMER01Inst(EN_R:=VarBOOL1, Period:=30000, INT_EN:=1, Q=>VarQ, CV=>VarCV, Error=> VarErr); </pre>
连续功能图 (CFC)	

程序说明:

EN_R 置位并保持时（上升沿），Q 等于 1，因为 INT_EN 等于 1，所以此后每隔 30000us 就触发一次 HD_TIMER0 中断事件，通过 AutoThink 中的系统事件来调用 HD_TIMER0 中断事件执行程序，CV 显示当前已过时间值，单位为 us。

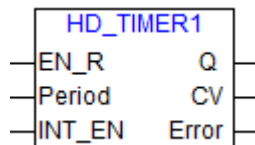
EN 复位时，停止产生 HD_TIMER0 中断事件，Q 等于 0。

3.3.9.3 HD_TIMER1（硬件中断定时器 1）

3.3.9.3.1 模块支持

14 点（LE5104、LE5105）支持前 4 个中断定时器。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118、LE5119）支持全部 6 路通道。

3.3.9.3.2 功能



HD_TIMER1 功能块

3.3.9.3.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0: 无效 1: 上升沿使能
Period	DWORD	设定时间（us）	500 us~800000 us 可设置
INT_EN	BOOL	中断使能	0: 设定时间到不产生中断 1: 设定时间到产生中断

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	是否设置完毕	0: 未设置 1: 设置完毕，保持
CV	DWORD	已过时间（us）	0~800000 us
Error	BYTE	错误信息	=0: 无错误信息 =1: 参数设置错误 =2: 硬件定时器未运算 =3: 调用多个相同功能块

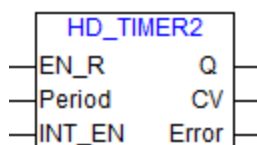
i 共有 6 个硬件定时器，名称为 HD_TIMER0~5，每个硬件定时器对应 1 路定时器中断。

3.3.9.4 HD_TIMER2（硬件中断定时器 2）

3.3.9.4.1 模块支持

14 点（LE5104、LE5105）支持前 4 个中断定时器。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118、LE5119）支持全部 6 路通道。

3.3.9.4.2 功能



HD_TIMER2 功能块

3.3.9.4.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0: 无效 1: 上升沿使能
Period	DWORD	设定时间 (us)	500 us ~800000 us 可设置
INT_EN	BOOL	中断使能	0: 设定时间到不产生中断 1: 设定时间到产生中断

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	是否设置完毕	0: 未设置 1: 设置完毕, 保持
CV	DWORD	已过时间 (us)	0~800000 us
Error	BYTE	错误信息	=0: 无错误信息 =1: 参数设置错误 =2: 硬件定时器未运算 =3: 调用多个相同功能块

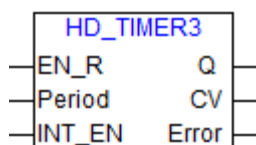
i 共有 6 个硬件定时器，名称为 HD_TIMER0~5，每个硬件定时器对应 1 路定时器中断。

3.3.9.5 HD_TIMER3（硬件中断定时器 3）

3.3.9.5.1 模块支持

14 点（LE5104、LE5105）支持前 4 个中断定时器。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118、LE5119）支持全部 6 路通道。

3.3.9.5.2 功能



HD_TIMER3 功能块

3.3.9.5.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0: 无效 1: 上升沿使能
Period	DWORD	设定时间 (us)	500 us ~800000 us 可设置
INT_EN	BOOL	中断使能	0: 设定时间到不产生中断 1: 设定时间到产生中断

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	是否设置完毕	0: 未设置 1: 设置完毕, 保持
CV	DWORD	已过时间 (us)	0~800000 us
Error	BYTE	错误信息	=0: 无错误信息 =1: 参数设置错误 =2: 硬件定时器未运算 =3: 调用多个相同功能块

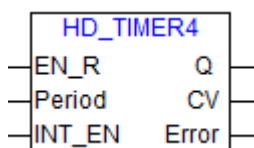
① 共有 6 个硬件定时器，名称为 HD_TIMER0~5，每个硬件定时器对应 1 路定时器中断。

3.3.9.6 HD_TIMER4（硬件中断定时器 4）

3.3.9.6.1 模块支持

14 点（LE5104、LE5105）支持前 4 个中断定时器。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118、LE5119）支持全部 6 路通道。

3.3.9.6.2 功能



HD_TIMER4 功能块

3.3.9.6.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0: 无效 1: 上升沿使能
Period	DWORD	设定时间（us）	500 us ~800000 us 可设置
INT_EN	BOOL	中断使能	0: 设定时间到不产生中断 1: 设定时间到产生中断

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	是否设置完毕	0: 未设置 1: 设置完毕，保持
CV	DWORD	已过时间（us）	0~800000 us
Error	BYTE	错误信息	=0: 无错误信息 =1: 参数设置错误 =2: 硬件定时器未运算 =3: 调用多个相同功能块

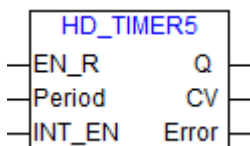
① 共有 6 个硬件定时器，名称为 HD_TIMER0~5，每个硬件定时器对应 1 路定时器中断。

3.3.9.7 HD_TIMER5（硬件中断定时器 5）

3.3.9.7.1 模块支持

14 点（LE5104、LE5105）支持前 4 个中断定时器。24 点（LE5106、LE5106M、LE5107）和 40 点（LE5108、LE5109、LE5118、LE5119）支持全部 6 路通道。

3.3.9.7.2 功能



HD_TIMER5 功能块

3.3.9.7.3 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0: 无效 1: 上升沿使能
Period	DWORD	设定时间（us）	500 us ~800000 us 可设置
INT_EN	BOOL	中断使能	0: 设定时间到不产生中断 1: 设定时间到产生中断

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	是否设置完毕	0: 未设置 1: 设置完毕，保持
CV	DWORD	已过时间（us）	0~800000 us
Error	BYTE	错误信息	=0: 无错误信息 =1: 参数设置错误 =2: 硬件定时器未运算 =3: 调用多个相同功能块

i 共有 6 个硬件定时器，名称为 HD_TIMER0~5，每个硬件定时器对应 1 路定时器中断。

3.4 行业应用库

3.4.1 HMC 运动指令库

3.4.1.1 AXIS（轴参数）

每个轴对应一组同样的轴参数，用户通过查看轴参数实时值可了解轴的当前运行状态，也可通过直接修改或运控控制指令间接修改轴参数，改变轴的运行状态。

轴参数按照读写属性可分为“只读”和“读写”两种。“只读”参数在组态时不能被赋值，也不能通过 AutoThink 来“写变量”；“读写”参数在组态时可以被赋值，也可以通过 AutoThink 写变量。“只读”参数又分为两种：第一种只反映轴的一些状态（例如 LimitState）或当前计算或测量出来的值（如 VpSpeed、MpSpeed），此类参数是由控制算法解算刷新；第二种是用户可通过设置函数设置的只读参数（如 AxisType）。

在 AutoThink 中，轴参数以功能块的方式组织，轴参数是其成员，每一个轴的所有轴参数构成一个功能块变量，该变量支持用户改名。举例来说，用户将以类似 Axis0.Speed 的方式访问轴 0 的 Speed，如果用户将轴 0 的功能块变量名改为 AxisLaser，则可以用 AxisLaser.Speed 的方式来访问轴 0 的 Speed。

AutoThink 中将所有轴的轴参数定义在数据区的某个固定区域中，为防止数据冲突，AutoThink 不允许用户在该区中再定义其他变量。

3.4.1.1.1 基本参数（Basic）

3.4.1.1.1.1 AxisID（轴号）

基本属性

类型	USINT
读写属性	只读
设置函数	-
初始值	0
取值范围	0~63
相关章节	无

含义描述

轴号，表示各组轴参数的轴号。

3.4.1.1.1.2 AxisType（轴类型）

基本属性

类型	EmAxisType
读写属性	只读
设置函数	HMC_SetAxisType
初始值	Virtual（0）：虚轴
取值范围	Unused（-1）：该轴未使用 Virtual（0）：虚轴 Stepper（10）：直连步进轴（脉冲+方向） Stepper_CW_CCW（11）：直连步进轴（正反脉冲） Stepper_AB（12）：直连步进轴（AB相脉冲） Servo_AO（20）：直连伺服轴（增量式编码器+AO输出） Encoder（30）：编码器轴（增量式编码器，AB相计数） Encoder_PLS_DIR（31）：编码器轴（脉冲方向计数） ENCODER_CW_CCW（32）：编码器轴（正反脉冲计数） ENCODER_SSI（33）：SSI绝对值编码器轴 （该类轴（30~39）不接受运动控制指令，仅仅跟踪编码器值） RTEX_Position（40）：RTEX总线连接轴，位置控制（仅MC1002R支持） RTEX_Speed（41）：RTEX总线连接轴，速度控制（仅MC1002R支持） RTEX_Torque（42）：RTEX总线连接轴，转矩控制（仅MC1002R支持） RTEX_PP（43）：RTEX总线连接轴，Profile Position控制（该模式仅用于RTEX伺服原点回归） EtherCAT_Position（50）：EtherCAT总线连接轴，位置控制（仅MC1002E支持） EtherCAT_Speed（51）：EtherCAT总线连接轴，速度控制（仅MC1002E支持） EtherCAT_Torque（52）：EtherCAT总线连接轴，转矩控制（暂不支持）
相关章节	HMC_SetAxisType（设置轴类型）

含义描述

轴类型，每10个为一大类，例如类型10~19代表的大类为直连步进轴。通过函数HMC_SetAxisType设置轴类型，不同型号的控制器的同一型号控制器的不同轴号，所支持的轴类型不同，参见[HMC_SetAxisType（设置轴类型）](#)。

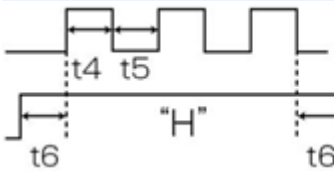
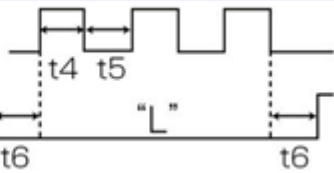
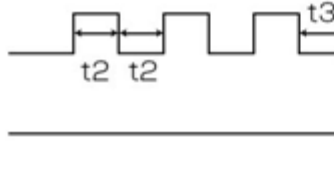
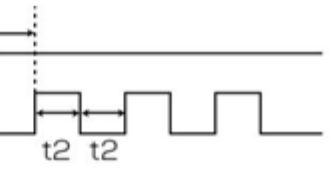
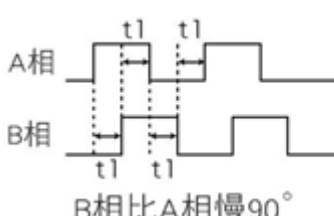
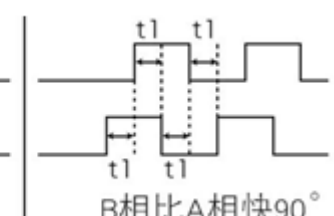
轴类型的不同，会影响该轴支持的指令类型。例如，编码器轴类型，不支持运动控制指令。

轴类型的不同，会影响该轴的输入、输出方式，各轴类型的输入、输出信号见下表。更改轴类型后，对应端子的输入、输出信号会发生变化，请确认无误后，再进行对应操作。

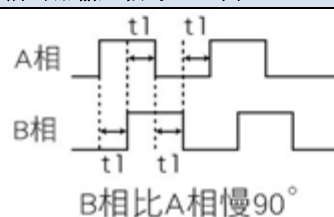
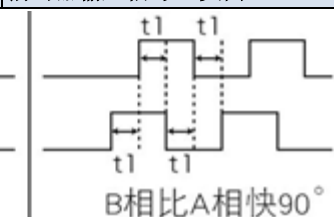
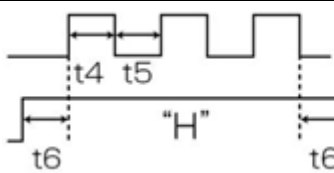
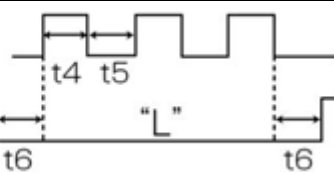
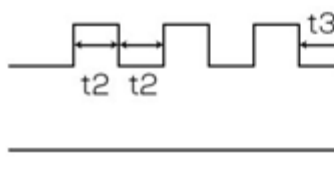
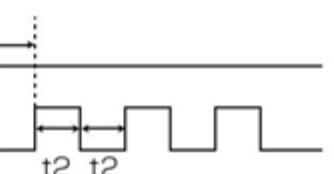
轴类型标示	轴类型代码	与控制器连接方式	输入	输出	备注
Unused	-1	未启用轴	无	无	运动控制指令不允许投放到该轴； 算法不对此类型轴进行解算，可节约算法解算时间
Virtual	0	控制器内置	无	无	一般用作插补指令的合轴或中间解算轴
Stepper	10	直连	无	脉冲+方向差分信号输出，详见 AxisType 10示意图	Z+、Z-为编码器输入
Stepper_CW_CCW	11	直连	无	正负脉冲差分信号输出，详见 AxisType 11示意图	Z+、Z-为编码器输入
Stepper_AB	12	直连	无	AB相正交脉冲差分信号输出，详见	输出脉冲数量方面： Stepper_AB是Stepper

轴类型标示	轴类型代码	与控制器连接方式	输入	输出	备注
				AxisType 12 示意图	的 4 倍频，Z+、Z-为编码器输入
Servo_AO	20	直连	AB 相差分信号输入	使用 AO：使用对应轴号的 AO 口输出，例如 AO5 对应的是轴 5 的输出	-
Encoder	30	直连	AB 相差分信号输入，详见图 3-4	无	运动控制指令不允许投放到该轴
Encoder_PLS_DIR	31	直连	脉冲+方向差分信号输入，详见图 3-5	无	运动控制指令不允许投放到该轴
ENCODER_CW_CCW	32	直连	正反向脉冲差分信号输入，详见图 3-6	无	运动控制指令不允许投放到该轴
ENCODER_SSI	33	直连	SSI 绝对值编码器轴	无	运动控制指令不允许投放到该轴
RTEX_Position	40	RTEX 总线连接	通过 RTEX 总线接收端获得编码器反馈位置（轴的位置信息），不作位置闭环控制	通过 RTEX 总线发送端输出指令位置	-
RTEX_Speed	41	RTEX 总线连接	通过 RTEX 总线接收端获得编码器反馈位置（轴的位置信息），servoLoop = 1 时作位置闭环控制运算	通过 RTEX 总线发送端输出指令速度	-
RTEX_Torque	42	RTEX 总线连接	通过 RTEX 总线接收端获得编码器反馈位置（轴的位置信息），servoLoop = 1 时作位置闭环控制运算	通过 RTEX 总线发送端输出指令转矩	-
RTEX_PP	43	RTEX 总线连接	通过 RTEX 总线接收端获得编码器反馈位置（轴的位置信息），不作位置闭环控制	无	该模式仅用于 RTEX 伺服的原点回归功能（RTEX_Home 指令）
EtherCAT_Position	50	EtherCAT 总线连接	通过 EtherCAT 总线获得编码器反馈位置（轴的位置信息），不作位置闭环控制	通过 EtherCAT 总线输出指令位置	-
EtherCAT_Speed	51	EtherCAT 总线连接	通过 EtherCAT 总线获得编码器反馈位置（轴的位置信息），servoLoop = 1 时作位置闭环控制运算	通过 EtherCAT 总线输出指令速度	-
EtherCAT_Torque	52	EtherCAT 总线连接	通过 EtherCAT 总线获得编码器反馈位置（轴的位置信息），servoLoop = 1 时作位置闭环控制运算	通过 EtherCAT 总线输出指令转矩	-

步进模式

轴类型	脉冲输出形态	信号名称	正向脉冲输出	负向脉冲输出
AxisType=10 (脉冲+方向输出)	脉冲序列+符号	Step Direction	 AxisType=10 示意图	 AxisType=10 示意图
AxisType=11 (正负脉冲输出)	正脉冲序列+负脉冲序列	Step Direction	 AxisType=11 示意图	 AxisType=11 示意图
AxisType=12 (AB 相脉冲输出)	90° 相位差 2 相脉冲 (A 相+B 相)	Step Direction	 B相比A相慢90°	 B相比A相快90°
说明	(1) 信号输出类型为差分信号； (2) 脉冲序列输出最高频率 2 MHz，占空比 50%，AB 相输出时相位差为 90°； (3) 最小时间宽度：t1 为 0.125 μs，t2、t4、t5 为 0.25 μs，t3、t6 为 0.5 μs。			

编码器模式

增量式编码器	脉冲输入形态	信号名称	编码器输入信号 (正向)	编码器输入信号 (负向)
AxisType=30 (AB 相编码器轴)	90° 相位差 2 相脉冲 (A 相+B 相)	Enc.A Enc.B Enc.Z	 B相比A相慢90°	 B相比A相快90°
AxisType=31 (脉冲+方向编码器轴)	脉冲序列+符号	Enc.A Enc.B Enc.Z	 AxisType=31 示意图	 AxisType=31 示意图
AxisType=32 (正负脉冲编码器轴)	正脉冲序列+负脉冲序列	Enc.A Enc.B Enc.Z	 AxisType=32 示意图	 AxisType=32 示意图
说明	(1) 信号输入类型为差分信号； (2) 脉冲输入最高频率 6 MHz (单相)，占空比 50%，AB 相输入时相位差为 90°； (3) 最小时间宽度：t1 为 0.125 μs，t2、t4、t5 为 0.25 μs，t3、t6 为 0.5 μs；			

增量式编码器	脉冲输入形态	信号名称	编码器输入信号（正向）	编码器输入信号（负向）
		(4) Enc.Z	相用来接收编码器每转一圈时产生的 Z 相脉冲。	

SSI 总线绝对值编码器	说明
AxisType=33 (SSI 总线绝对值编码器轴)	(1) 在设定轴类型为 SSI 编码器轴之前，可使用 HMC_SetSSIEncoderBit、HMC_SetSSIEncoderDataType、HMC_SetSSIEncoderFreq、HMC_SetSSIEncoderTp 配置相关参数； (2) 初始化 Mpos 为 SSI 编码器绝对位置的时机： 使用 HMC_SetAxisType 成功设定该轴类型为 SSI 轴类型时； 轴类型已经为 SSI 轴类型，使用 HMC_SetSSIEncoderBit、HMC_SetSSIEncoderDataType 更改编码器位数/编码器码制成功时； (3) 初始化绝对位置后，后续采用初始位置+每个伺服周期位置增量的方式计算 Mpos； 如果需要 MPos 的值始终与编码器绝对位置一致，则可设定该轴为循环行程（RepMode=1），且循环行程的行程长度与编码器的位置数据范围一致（如 12 位编码器，在 Units、Ke 均为 1 的情况下，RepMode=1 时应设置 RepDist = 2¹²； 在设置 RepMode=2、RepDist=编码器位置数据范围/2 的情况下，可实现该轴 Mpos 区间长度=编码器位置数据区间长度，并且 Mpos 的范围区间是关于零点对称的（如 12 位编码器，在 Units、Ke 均为 1 的情况下，应设置 RepMode=2、RepDist=2¹²/2）； 如果设定该轴为有限行程，则如果编码器未发生数据溢出时，Mpos 与编码器绝对位置一致；当编码器发生数据溢出时，则控制器仍按实际增量位移计算 Mpos，而此时编码器内部的绝对位置数据则因数据溢出而发生了跳变； (4) 支持 SSI 轴的模块：MC1008-B02 支持 4 个 SSI 轴（轴号 0、1、2、3），MC1004-B01 支持 2 个 SSI 轴（轴号 0、1），MC1008-A01/MC1008-A02/MC1008-B01/MC1004-A01/ MC1002R-A01/MC1002E-A01 不支持。

示例

因为 AxisType 是枚举类型，所以可按照如下方式使用。

Result := HMC_SetAxisType(1, 20); (*Servo_AO 的轴类型枚举值为 20*)

IF 20 = Axis1.AxisType THEN

Axis1.Speed := 4000;

END_IF

3.4.1.1.1.3 Units（单位转换因子）

基本属性

类型	LREAL
读写属性	只读
设置函数	HMC_SetUnits
初始值	1
取值范围	正浮点数
相关章节	HMC_SetUnits（设置用户单位）

含义描述

单位转换因子（只可为正值），单位：线/用户单位。

设置指定轴的单位转换因子轴参 Units，该参数只可为正值，单位：线/用户单位，“线”是指轴走一个脉冲对应的刻度单位。

通过设置轴参 Units，可将编码器反馈的计数值或脉冲输出值转换为用户方便使用的单位数值，例如 mm、角度等。

Units 在系统中的默认值是 1，表示用户单位是“线”，因此 Speed、Accel、Decel、FastDecel 等运动参数的单位均是“线/秒”，或者“线/（秒*秒）”；当 Units 被修改后，这些运动参数的单位自然变为了“用户单位/秒”，或者“用户单位/（秒*秒）”，但是这些运动参数的数值需要用户根据工程需要手动修改。

该参数设置方式，参见 [HMC SetUnits（设置用户单位）](#)。

3.4.1.1.2 状态参数（Status）

3.4.1.1.2.1 MCmdType（当前指令类型）

基本属性

类型	EmCmdType
读写属性	只读
设置函数	-
初始值	HMC_McIdle（空）
取值范围	系统的所有运动控制指令类型

含义描述

轴的当前指令类型。每个轴有 64 级运动指令缓存(指令队列 / FIFO)，位于指令队列中队首位置的指令为当前正在被执行的指令。

系统支持的所有运动控制指令类型

值	名称	说明	备注
0	HMC_McIdle	空闲	无运动指令
1	HMC_ERR1	预留 1	系统预留
2	HMC_ERR2	预留 2	系统预留
3	HMC_ERR3	预留 3	系统预留
4	HMC_ERR4	预留 4	系统预留
5	HMC_ERR5	预留 5	系统预留
6	HMC_Home	原点回归	-
7	HMC_HomeB	原点回归（阻塞）	暂不支持
8	HMC_HomeEx	高级原点回归	-
9	HMC_HomeExB	高级原点回归（阻塞）	暂不支持
10	HMC_Move	相对运动	-
11	HMC_MoveB	相对运动（阻塞）	暂不支持
12	HMC_MoveEx	高级相对运动	-
13	HMC_MoveExB	高级相对运动（阻塞）	暂不支持
14	HMC_MoveAbs	绝对运动	-
15	HMC_MoveAbsB	绝对运动（阻塞）	暂不支持
16	HMC_MoveAbsEx	高级绝对运动	-
17	HMC_MoveAbsExB	高级绝对运动（阻塞）	暂不支持
18	HMC_Forward	正向运动	-

值	名称	说明	备注
19	HMC__ForwardEx	高级正向运动	-
20	HMC__Reverse	反向运动	-
21	HMC__ReverseEx	高级反向运动	-
22	HMC__Move2D	2 轴直线插补	-
23	HMC__Move2DB	2 轴直线插补（阻塞）	暂不支持
24	HMC__Move2DEx	高级 2 轴直线插补	-
25	HMC__Move2DExB	高级 2 轴直线插补（阻塞）	暂不支持
26	HMC__Move3D	3 轴直线插补	-
27	HMC__Move3DB	3 轴直线插补（阻塞）	暂不支持
28	HMC__Move3DEx	高级 3 轴直线插补	-
29	HMC__Move3DExB	高级 3 轴直线插补（阻塞）	暂不支持
30	HMC__MoveND	N 轴直线插补	-
31	HMC__MoveNDB	N 轴直线插补（阻塞）	暂不支持
32	HMC__MoveNDEx	高级 N 轴直线插补	-
33	HMC__MoveNDExB	高级 N 轴直线插补（阻塞）	暂不支持
34	HMC__MoveCircle	圆弧插补	-
35	HMC__MoveCircleB	圆弧插补（阻塞）	暂不支持
36	HMC__MoveCircleEx	高级圆弧插补	-
37	HMC__MoveCircleExB	高级圆弧插补（阻塞）	暂不支持
38	HMC__MoveHelical	螺旋线插补	-
39	HMC__MoveHelicalB	螺旋线插补（阻塞）	暂不支持
40	HMC__MoveHelicalEx	高级螺旋线插补	-
41	HMC__MoveHelicalExB	高级螺旋线插补（阻塞）	暂不支持
42	HMC__MoveSpherical	球弧插补	-
43	HMC__MoveSphericalB	球弧插补（阻塞）	暂不支持
44	HMC__MoveSphericalEx	高级球弧插补	-
45	HMC__MoveSphericalExB	高级球弧插补（阻塞）	暂不支持
46	HMC__Gear	电子齿轮	-
47	HMC__Superimpose	叠加	-
48	HMC__SuperimposeEx	高级叠加	-
49	HMC__Cam	电子凸轮	-
50	HMC__MoveAbs2D	2 轴绝对值直线插补	-
51	HMC__MoveAbs2DEx	高级 2 轴绝对值直线插补	-
52	HMC__MoveAbs3D	3 轴绝对值直线插补	-
53	HMC__MoveAbs3DEx	高级 3 轴绝对值直线插补	-
54	HMC__MoveAbsND	N 轴绝对值直线插补	-
55	HMC__MoveAbsNDEx	高级 N 轴绝对值直线插补	-
56	HMC__MoveTang	切向追踪	-
57	HMC__MoveSplineND	样条插补	-
59	HMC__MoveSplineNDEx	高级样条插补	-
62	HMC__MoveLink	追剪运动	-

3.4.1.1.2.2 NCmdType（下一条指令类型）

基本属性

类型	EmCmdType
读写属性	只读
设置函数	-
初始值	HMC_McIdle (空)
取值范围	系统的所有运动控制指令类型

含义描述

轴的下一条指令类型。每个轴有 64 级运动指令缓存(指令队列 / FIFO)，位于指令队列中队首位置指令的下一条指令。

3.4.1.1.2.3 AxisStatus (轴状态)

基本属性

类型	UDINT
读写属性	只读
设置函数	-
初始值	0
取值范围	0 表示正常； 其他数值，转换为 2 进制数后按位解析，各 Bit 位含义见 轴状态解析表 。
相关章节	AxisErrMask (轴状态掩码)

含义描述轴

当前运行状态，为 0 时表示该轴无异常。不为 0 时，按位解析如下：

轴状态解析表

Bit	含义	消除方式	可能原因
0	随动误差 FE 连续 10 次超限 FELimit	状态锁定，需要用户清除	1、轴编码器线未连接； 2、电机驱动器接线错误导致伺服使能未开； 3、电机驱动器报错导致伺服使能关闭； 4、轴 FELimit 参数设置过小； 5、轴 PID 参数不合理。
1	随动误差 FE 超限 1 次警告	连续 1000 次不超限后该警告，自动解除	1、轴 FELimit 参数设置过于临界； 2、轴 PID 参数不合理。
2	轴参数设定错误	状态锁定，需要用户清除	轴参数设置非法值（例如 Speed、CreepSpeed 为负，Accel、Decel 为 0 或负）。
3	到达正向或反向限位，包括软限位和硬限位	限位条件不满足后，自动清除	有限形成轴发生满足以下某条件： 1、轴 $Dpos \geq FSLimit$ （正向软限位）； 2、轴 $Dpos \leq RSLimit$ （反向软限位）； 3、轴 $FHLimitIn$ 关联 DI 为 1（正向硬限位）； 4、轴 $RHLimitIn$ 关联 DI 为 1（反向硬限位）；
4	与总线轴通讯错误	MC1002R、MC1002E 等总线型控制器支持	1、总线断线或输入输出连接错误； 2、配置总线从站数与实际连接总线从站数不一致； 3、总线主站配置参数与从站参数不同； 4、配置从站型号和实际从站型号不同； 5、总线从站报错（错误信息查看从站使用手册）； （AT 中的全局变量 RtexDiagGroup 或 EtherCATDiagGroup 包含总线部分详细诊断信息）
5	龙门轴偏差超限报警	偏差消除后自动清除	龙门轴存在轴间偏差超报警阈值
6	龙门轴偏差超限停车	手动消除	龙门轴存在轴间偏差超紧急停车阈值

Bit	含义	消除方式	可能原因
7~31	未使用	预留	预留

备注：当发生轴状态错误时，排除导致错误的原因后，可使用 HMC_ClearAxisErr 清除该轴状态错误，也可使用 HMC_ClearAllErr 清除所有轴状态错误。如果是与总线轴通讯错误，清除错误前需使用 Rtex_BusReset()或 EtherCAT_BusReset()复位相应总线协议栈。

示例

轴 0 到达正向或反向限位，关闭伺服使能开关。程序如下：

```
IF (UDINT_TO_DWORD (Axis0.AxisStatus) AND 8) >0 THEN  (*轴 0 AxisStatus 与 8 位与，判断是否到达正向或反向限位*)
```

```
    HMC_DriveEnable(0); (*关闭伺服使能开关*)
```

```
END_IF
```

3.4.1.1.2.4 AxisErrMask（轴状态掩码）

基本属性

类型	UDINT
读写属性	读写
设置函数	-
初始值	1
取值范围	0 表示各种 AxisStatus 值均不进行异常处理 其他值设置方式参考 AxisStatus 的各 bit 位含义，可设置一个或多个 bit 位为 1
相关章节	AxisStatus（轴状态）

含义描述

轴状态异常掩码，32 个 bit 位与 AxisStatus 的 32 个 bit 位对应。当 AxisErrMask 的某个 bit 位为 1 时，轴状态 AxisStatus 的对应 bit 位如果也为 1 时，认为出现系统异常，此时系统处理方式：将 DriveEnable 信号禁用，将 ServoLoop 置为 0。

示例

轴 0 的 AxisStatus Bit0 和 Bit4 为 1 时，断开伺服使能开关，将闭环输出开关 ServoLoop 置为 0（开环）。程序如下：

```
Axis0.AxisErrMask=17;  (*设置轴 0 的 AxisErrMask 参数 Bit0 和 Bit4 为 1，则在轴 0 的 AxisStatus Bit0 和 Bit4 为 1 时，断开伺服使能开关，将闭环输出开关 ServoLoop 置为 0（开环）*)
```

3.4.1.1.2.5 LimitState（超限状态）

基本属性

类型	USINT
读写属性	只读
设置函数	-
初始值	0
取值范围	LimitState 为 0 表示正常，即未发生任何超限。 不为 0 时，先转换为 2 进制数，各 bit 位为 1 时含义如下：

类型	USINT
	Bit 0: 超过前向硬限位 Bit 1: 超过后向硬限位 Bit 2: 超过前向软限位 Bit 3: 超过后向软限位 Bit 4: 软限位相关参数错误 (前向软限位 $\leq$ 后向软限位) Bit 5: 各个状态冲突错误 (超越前向软/硬限位, 同时也超越后向软/硬限位)
相关章节	RepMode (位置行程模式) FsLimitMode/RsLimitMode (前/后限位时指令撤销方式) FHLimitIn/RHLimitIn (前/后向硬限位通道号) FSLimit/RSLimit (前/后向软限位边界值)

含义描述

轴为有限行程时 (RepMode=0), 该轴参数用来描述当前超限状态。

取值

Bit	说明	值	备注
0	到达正向硬限位	1	正向硬限位关联 DI 通道为 1
1	到达反向硬限位	2	反向硬限位关联 DI 通道为 1
2	到达正向软限位	4	$Dpos \geq FSLimit$
3	到达反向软限位	8	$Dpos \leq RSLimit$
4	软限位相关参数错误	16	正向软限位 $\leq$ 反向软限位
5	同时到达正向和反向限位	32	包括软限位和硬限位
6~7	未使用	-	预留

示例

轴 0 到达正向软限位时, 反向位移 100。

程序如下:

```
IF (USINT_TO_WORD (Axis0.LimitState) AND 4) >0 THEN  (*轴 0 LimitState 与 4 位与, 判断是否到达正向软限位*)
```

```
  HMC_Move(0,-100); (*轴 0 反向位移 100*)
```

```
END_IF
```

■ 备注

(1) 轴到达正向限位 (含软限位和硬限位) 时, 投送加重正向限位的指令将被立即撤销, 轴不正向运动。此时投送减缓正向限位的指令可投送成功, 轴可向减缓正向限位的方向运动;

(2) 轴到达反向限位 (含软限位和硬限位) 时, 投送加重反向限位的指令将被立即撤销, 轴不反向运动。此时投送减缓反向限位的指令可投送成功, 轴可向减缓反向限位的方向运动;

(3) 对于正在执行单轴指令的轴超限, 将撤销该指令, 并使用 $\text{Max}(\text{FastDecel}, \text{Decel})$ 快速停止当前轴的运动;

(4) 对于正在执行插补指令的合轴超限, 将撤销整个插补指令, 并使用合轴的 $\text{Max}(\text{FastDecel}, \text{Decel})$ 快速停止当前轴的运动;

(5) 对于正在执行插补指令的分轴超限, 将撤销整个插补指令, 超限分轴使用 $\text{Max}(\text{FastDecel}$ 分轴, Decel 分轴, 合轴分解的减速度) 快速停止当前分轴的运动。合轴和未超限的分轴不受影响, 继续运动;

(6) 对于执行联动指令的从轴超限，将撤销该指令，并使用 **Max(FastDecel,Decel)**快速停止当前从轴的运动；

(7) 轴同时到达正向和反向限位（含软限位和硬限位）时，轴指令停止解算但不撤销。当恢复为一种限位时将按照上面几种情况处理。

3.4.1.1.3 位置相关参数（Positions）

3.4.1.1.3.1 MPos（反馈位置）

基本属性

类型	LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	浮点数范围
相关章节	DPos（本周目标位置） RepMode（位置行程模式） RepDist（循环行程长度） HMC_DefPos（设置当前位置）

含义描述

当前测量反馈位置，对于有实际输入信号的轴类型，MPos 反映了当前实际位置。用户单位。

(1) Mpos 取值原则

轴类型有实际输入信号时，如伺服轴、RTEX 总线轴、EtherCAT 总线轴、编码器轴，其 Mpos 来自于编码器的实时测量值；但若当 **FeedBackSrcMode=1** 时，Mpos 来自用户自定义输入源。

轴类型为虚轴时，其 MPos 等于 DPos。

轴类型为直连步进轴时，其 MPos 根据当前轴实际已发送的脉冲数计算。

(2) Mpos 计算方式

Mpos 均采用增量计算方法。将前后两周期的编码器反馈值的增量累积到上周期 MPos 值上，得到本周 MPos 值。

对于 SSI 编码器轴类型，在设置轴类型后，或通过 **HMC_SetSSIEncoderBit** 设置 SSI 编码器位数，或通过 **HMC_SetSSIEncoderDataType** 设置编码器轴码制时，系统会自动使用绝对值编码器当前位置初始化轴的 Mpos，但后续仍采用增量计算。

3.4.1.1.3.2 DPos（本周目标位置）

基本属性

类型	LREAL
读写属性	只读
设置函数	-

类型	LREAL
初始值	0
取值范围	浮点数范围
相关章节	RepMode（位置行程模式） RepDist（循环行程长度）

含义描述

当前目标位置，是算法实时计算出来的本周周期目标位置，据此得到最终的输出信号，用户单位。

系统按照以下几个原则处理该轴参数：

（1）系统运行过程中实时更新该轴参数，该参数的值由运动指令实时解算；

（2）在循环行程模式下，MPos 始终在[0,RepDist)或[-RepDist,RepDist)之间，Dpos 和 Mpos 之间保持实际的距离，这样 Dpos 大部分情况下也在[0,RepDist)或[-RepDist,RepDist)之间，一些个别情况，例如 RepDist 值设置的比较小的时候，伺服轴的 Dpos 会出现在循环行程区间之外；

（3）对于伺服轴（轴类型为 20~29）、RTEX_Speed 轴（轴类型为 41）、EtherCAT_Speed 轴（轴类型为 51），当 ServoLoop 为 0 时，Dpos 实时等于 Mpos。

3.4.1.1.3.3 FE（偏差）

基本属性

类型	LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	浮点数范围
相关章节	StopFETol（反馈位置是否到达目标位置的死区） FELimits（随动误差限制）

含义描述

实时的随动误差， $FE = DPOS - MPOS$ 。

伺服模式时，会根据 FE 进行 PID 运算，得到最终输出值；对于总线位置模式时，该 FE 不参与 PID 运算，但用来进行偏差报警。当轴类型为非伺服轴、非总线轴时，该值始终为 0。

3.4.1.1.3.4 StopFETol（反馈位置是否到达目标位置的死区）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	10
取值范围	正的浮点数
相关章节	FE（偏差）

含义描述

FE 误差容忍范围，用于判断速度模式时 MPos 是否到达目标位置。

当轴类型为速度模式时，即包括伺服轴、总线速度轴类型，如果 $\text{CmdStopMode}=1$ ，则判断指令是否结束时，除了需满足 DPos 解算完成到达 EndPos ，还需满足 $\text{Abs}(\text{FE}) < \text{StopFETol}$ ，即认为 Mpos 到达指令目标位置；如果 $\text{CmdStopMode}=0$ 则只需满足 DPos 解算完成到达 EndPos ，就认为指令结束。

3.4.1.1.3.5 RepMode（位置行程模式）

基本属性

类型	SINT
读写属性	读写
设置函数	-
初始值	2
取值范围	0: 有限行程，此时 RepDist 无意义， FsLimit 和 RsLimit 为软限位，硬限位通过函数 HMC_HwLimitConfig 来配置 1: 循环行程， $\text{FsLimit}/\text{RsLimit}$ 、 $\text{FHLimitIn}/\text{RHLimitIn}$ 无意义， $[0, \text{RepDist}]$ 为循环行程上下限值 2: 循环行程， $\text{FsLimit}/\text{RsLimit}$ 、 $\text{FHLimitIn}/\text{RHLimitIn}$ 无意义， $[-\text{RepDist}, \text{RepDist}]$ 为循环行程上下限值
相关章节	RepDist（循环行程长度） MPos（反馈位置）

含义描述

设置该轴的位置行程模式：循环行程/有限行程。

(1) 有限行程定义

表示轴只在一个限定的范围内运动，不能超过该限定的范围，例如由滚珠丝杆带动的运动物体，只能在有限的行程范围内运动。

当用户设置 $\text{RepMode}=0$ 时，代表该轴为有限行程，此时 RepDist 无意义，该轴存在行程限位。

FsLimit 和 RsLimit 表示软限位，而硬限位通道（ $\text{FHLimitIn}/\text{RHLimitIn}$ ）通过函数 HMC_HwLimitConfig 来配置。当有限行程的轴超过硬限位或者软限位时，系统的处理方式详见“有限行程轴超限处理”章节。

(2) 循环行程定义

轴在不断转动，其位置在不断的重复，没有限位，例如钟表的运动。

当用户设置 $\text{RepMode}=1$ 或 $\text{RepMode}=2$ 时，代表该轴为循环行程。其中 $\text{RepMode}=1$ 时， $[0, \text{RepDist}]$ 为循环行程上下限值， $\text{RepMode}=2$ 时， $[-\text{RepDist}, \text{RepDist}]$ 为循环行程上下限值。

循环行程时，软限位 $\text{FsLimit}/\text{RsLimit}$ 和硬限位通道（ $\text{FHLimitIn}/\text{RHLimitIn}$ ）无意义。

(3) 轴类型设置后自动修改行程方式

当用户通过函数 HMC_SetAxisType 设置某个轴类型后，系统会自动修改该轴的行程模式。当设置为虚轴或编码器轴时，系统将设置 $\text{RepMode}=2$ ；当用户设置为其他轴类型时，系统将设置 $\text{RepMode}=0$ 。

3.4.1.1.3.6 RepDist（循环行程长度）

基本属性

类型	LREAL
读写属性	读写

类型	LREAL
设置函数	-
初始值	10,000,000,000
取值范围	正浮点数
相关章节	RepMode（位置行程模式）

含义描述

循环行程模式（RepMode 为 1 或 2 时）时有效：EndPos 和 MPOS 显示的数值范围限值。伺服轴时 DPos 有可能会有短时间超出循环行程的情况。

当用户设置 RepMode=1 时，代表该轴为循环行程，[0,RepDist]为循环行程上下限值；当用户设置 RepMode=2 时，代表该轴为循环行程，[-RepDist,RepDist]为循环行程上下限值。

3.4.1.1.3.7 Remain（剩余位置）

基本属性

类型	LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	浮点数范围
相关章节	-

含义描述

当前运动指令执行过程中，离指令目标位置的剩余位移，用户单位。系统运行过程中实时更新该轴参数，Remain=EndPos-DPos。

3.4.1.1.4 速度相关参数（Velocity Profile）

3.4.1.1.4.1 Speed（目标速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	2000
取值范围	正浮点数
相关章节	-

含义描述

设定指令解算的目标速度，用户单位/秒。该参数可以实时修改，该值只可为正或 0，如果设置为负值，系统自动取其绝对值，同时在 AxisStatus 的 Bit 2 位报一个警告。

轴正在执行运动指令时，Speed 可以被实时修改，系统处理方法如下：

- （1）执行单轴指令时，Speed 可以实时修改，且实时生效，指令将按照新的 Speed 进行运动解算；
- （2）执行插补指令且是插补合轴时，Speed 可以实时修改，实时生效，合轴将按照新的 Speed 进行运

动解算；因合轴分轴插补关系不变，这些实时解算出来的运动量自然会被分解到分轴上去；

（3）执行插补指令且是插补分轴时，**Speed** 可以被实时修改，但不会影响该插补分轴的运动；

（4）执行联动指令且该轴此时是联动从轴时，**Speed** 可以被实时修改，但不会影响该轴的运动。

3.4.1.1.4.2 Accel（加速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	200,000
取值范围	正浮点数
相关章节	-

含义描述

加速度，用户单位/（秒*秒）。

该值只可为正，如果设置为负值，系统自动取其绝对值；如果为 0，该参数恢复系统默认值；对于非法值，同时在 **AxisStatus** 的 Bit 2 位报一个警告。

某个轴正在执行运动指令时，**Accel** 可以被实时修改，系统处理方法如下：

（1）执行单轴指令时，**Accel** 可以实时修改，实时生效，指令将按照新的 **Accel** 进行运动解算；

（2）执行插补指令且是插补合轴时，**Accel** 可以实时修改，实时生效，合轴将按照新的 **Accel** 进行运动解算；因合轴分轴插补关系不变，这些实时解算出来的运动量自然会被分解到分轴上去；

（3）执行插补指令且是插补分轴时，**Accel** 可以被实时修改，但不会影响该插补分轴的运动；

（4）执行联动指令且该轴此时是联动从轴时，**Accel** 可以被实时修改，但不会影响该轴的运动。

3.4.1.1.4.3 Decel（减速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	200,000
取值范围	正浮点数
相关章节	-

含义描述

减速度，用户单位/（秒*秒）。

该值只可为正，如果设置为负值，系统自动取其绝对值；如果为 0，该参数恢复系统默认值；对于非法值，同时在 **AxisStatus** 的 Bit 2 位报一个警告。

某个轴正在执行运动指令时，**Decel** 可以被实时修改，系统处理方法如下：

(1) 执行单轴指令时，Decel 可以实时修改，实时生效，指令将按照新的 Decel 进行运动解算；

(2) 执行插补指令且该轴是插补合轴时，Decel 可以实时修改，实时生效，合轴将按照新的 Accel 进行运动解算；因合轴分轴插补关系不变，这些实时解算出来的运动量自然会被分解到分轴上去；

(3) 执行插补指令且该轴是插补分轴时，Decel 可以被实时修改，但不会影响该插补分轴的运动；

(4) 执行联动指令（该轴此时是联动从轴）时，Decel 可以被实时修改，但不会影响该轴的运动。

3.4.1.1.4.4 MpSpeed（反馈速度）

基本属性

类型	LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	浮点数
相关章节	-

含义描述

实时测量到的反馈速度，测速周期为系统伺服周期，单位同 Speed。

MpSpeed 的计算原则为：

(1) 伺服轴（轴类型为 20~29）、编码器轴（轴类型为 30~39）、RTEX_Position 轴（轴类型为 40）、RTEX_Speed 轴（轴类型为 41）、RTEX_Torque 轴（轴类型为 42）、RTEX_PP 轴（轴类型为 43）、EtherCAT_Position 轴（轴类型为 50）、EtherCAT_Speed 轴（轴类型为 51）、EtherCAT_Torque（轴类型为 52），MpSpeed 是根据来自编码器的实时测量位置值计算得到，即此时反映了 MPos 的变化速度；

(2) 对于其他轴，该值在数值上等于 VpSpeed；

(3) MpSpeed 的正负值表示当前轴的实际运动方向。

3.4.1.1.4.5 VpSpeed（解算速度）

基本属性

类型	LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	正浮点数
相关章节	Direction（解算速度的方向）

含义描述

运动指令实时解算出来的速度数值，解算周期为系统伺服周期，单位同 Speed。其反映了 Dpos 的变化速度值，该参数仅表示速度值，不包含方向，即为正值。

3.4.1.1.4.6 Direction（解算速度的方向）

基本属性

类型	SINT
读写属性	只读
设置函数	-
初始值	1
取值范围	1: 表示正在向正向运动 -1: 表示正在向负向运动
相关章节	VpSpeed（解算速度）

含义描述

运动指令实时解算出来的速度方向，反映了 DPos 的速度变化方向。

3.4.1.1.4.7 Merge（融合功能模式）

基本属性

类型	USINT
读写属性	只读
设置函数	HMC_SetMerge
初始值	0
取值范围	该值只可为 0、1、2、3、4 0: Merge 关闭，每个指令块单独完成各自的运动 1: Merge 打开，当前指令块结束时目标速度为该指令块的 Speed 2: Merge 打开，当前指令块结束时目标速度为下一指令块的 Speed 3: Merge 打开，当前指令块结束时目标速度为该指令块 Speed 和下条指令块 Speed 最小值 4: Merge 打开，当前指令块结束时目标速度为该指令块 Speed 和下条指令块 Speed 最大值
相关章节	运动前瞻

含义描述

设置速度融合功能，决定是否开启前瞻预处理的速度规划策略。Merge（融合功能）是将轴运动控制指令缓存中一连串运动指令连贯起来，使负载运动速度连续，告别走走停停，可以显著提升效率。

3.4.1.1.4.8 CreepSpeed（原点回归爬行速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	200
取值范围	正浮点数
相关章节	HMC_Home（原点回归）

含义描述

爬行速度，用户单位/秒。

用于原点回归往回找原点阶段，在此阶段系统会将 CreepSpeed 作为目标速度进行运动解算，用户在此阶段修改 CreepSpeed 将实时生效。

该值只可为正或 0，如果给负值，取其绝对值，同时在 AxisStatus 的 Bit 2 位报一个警告。

3.4.1.1.4.9 FastDecel（快速减速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	5,000,000
取值范围	正浮点数

含义描述

快速减速度，用户单位/（秒*秒），用于有限行程轴发生软超限或硬超限后的自动紧急停止。详见有限行程轴超限处理章节。

该值只可为正（如果为负值，取其绝对值；如果为 0，该参数恢复系统默认值；对于非法值，同时在 AxisStatus 的 Bit 2 位报一个警告）。

3.4.1.1.4.10 Jerk（加加速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	5,000,000,000
取值范围	正浮点数
相关章节	HMC_ClcJerk（冲击运算）

含义描述

加加速度（加速度或减速度的导数），用户单位/（秒*秒*秒）。

该值只可为正。如果为负值，取其绝对值；如果为 0，该参数恢复系统默认值；对于非法值，在 AxisStatus 的 Bit 2 位报一个警告。

- 不支持的模块版本

MC1008-A01

3.4.1.1.4.11 JerkMode（加速模式）

基本属性

类型	USINT
读写属性	只读
设置函数	-
初始值	0
取值范围	0: S 形运动禁用 1: S 形运动使能
相关章节	-

含义描述

S 形运动使能/禁用（是否使用 Jerk）。

当不使用 Jerk 时，将按照加速度 Accel 和减速度 Decel 进行加速或减速运动，此时速度曲线表现为梯形；当使用 Jerk 时，将按照 Jerk 作为加加速度 Jerk 来改变加（减）速度和速度来进行运动，一般情况下，速度曲线表现为“S”形。

■ S 形加减速不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-B02、MC1004-A01、MC1002R-A01、MC1002E-A01

3.4.1.1.4.12 CmdStopMode（指令结束判断方式）

基本属性

类型	USINT
读写属性	读写
设置函数	-
初始值	0
取值范围	0 1
相关章节	-

含义描述

判断伺服轴的运动指令是否执行结束时的模式。

（1）CmdStopMode=0

系统判断当前指令运算结束的标准是：DPOS 已经到达目标位置 EndPos。即达到该条件后，认为当前指令运算结束，立即开始执行下一条指令。

（2）CmdStopMode=1

系统判断当前指令运算结束必须同时满足以下两个条件，才会执行下一条指令。

（a）DPOS 已经到达目标位置

（b）MPOS 到达目标位置或者 DPOS 到达目标位置已经足够长时间（默认 500ms，也可通过 HMC_SetCmdStopMaxTime 指令修改此时间）。MPos 到达目标位置是指 $Abs(FE) < StopFETol$ 。例如系统控制周期为 1 ms，则 DPOS 到达目标位置 500 ms 后，即使 MPOS 没有到达指定位置，系统也认为该指令结束了。

3.4.1.1.4.13 MoveAbsMode（循环行程轴绝对运动时方向选取方式）

基本属性

类型	USINT
读写属性	读写
设置函数	-
初始值	0
取值范围	0、1、2、3、4

类型	USINT
相关章节	-

含义描述

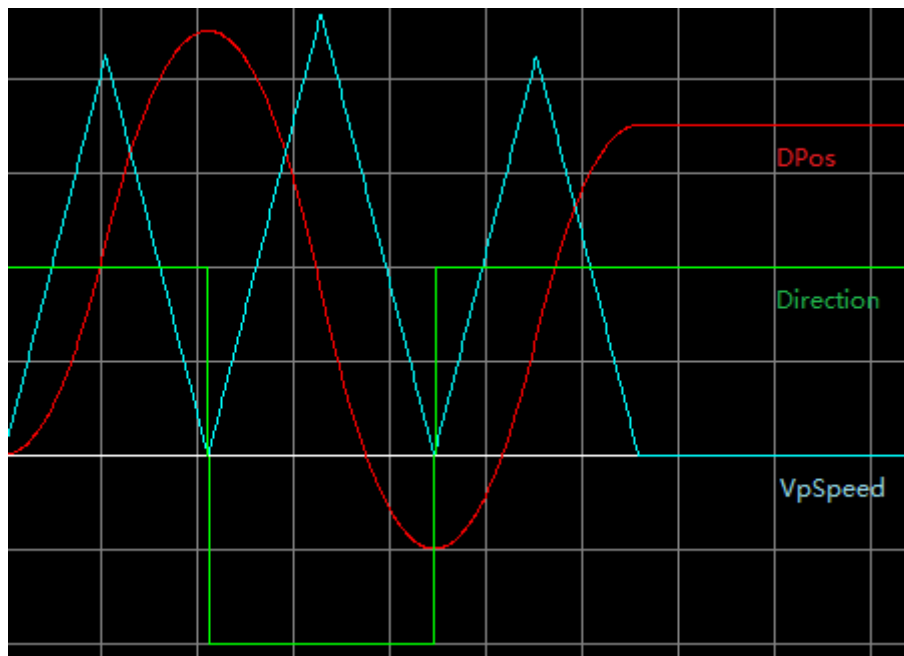
仅对循环行程轴（RepMode=1 或 RepMode=2）的绝对运动指令有效，例如 HMC_MoveAbs、HMC_MoveAbs2DEx、HMC_MoveAbsND。

- 1 MoveAbsMode=0: 在本次循环行程内确认方向
- 2 MoveAbsMode=1: 算法选取距离最短的方向，正负向运动等距离时，选取正方向
- 3 MoveAbsMode=2: 按照正方向运动
- 4 MoveAbsMode=3: 按照负方向运动
- 5 MoveAbsMode=4: 按照当前运动方向运动。即 MoveAbs 之前的 Direction 方向即为本次 MoveAbs 的方向

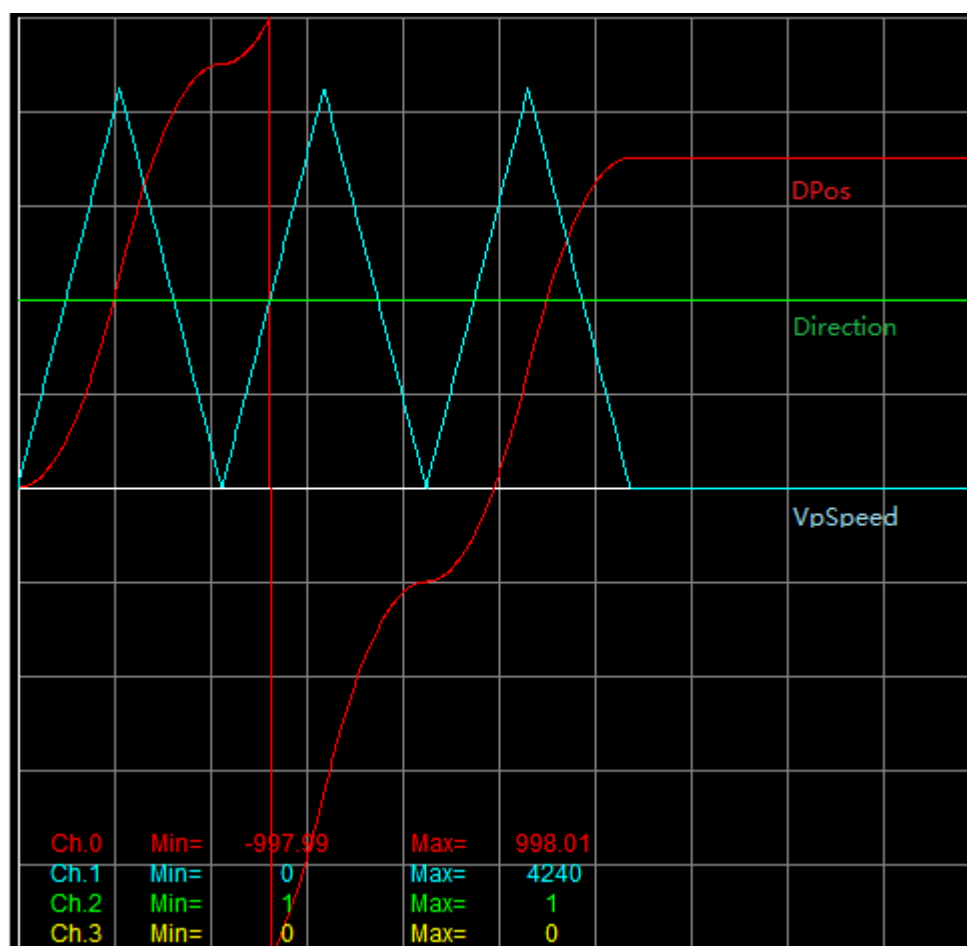
以下例详细描述以上几个模式时，方向确定方法。

编写如下程序，然后分别修改 MoveAbsMode，查看轴 0 的运行情况。

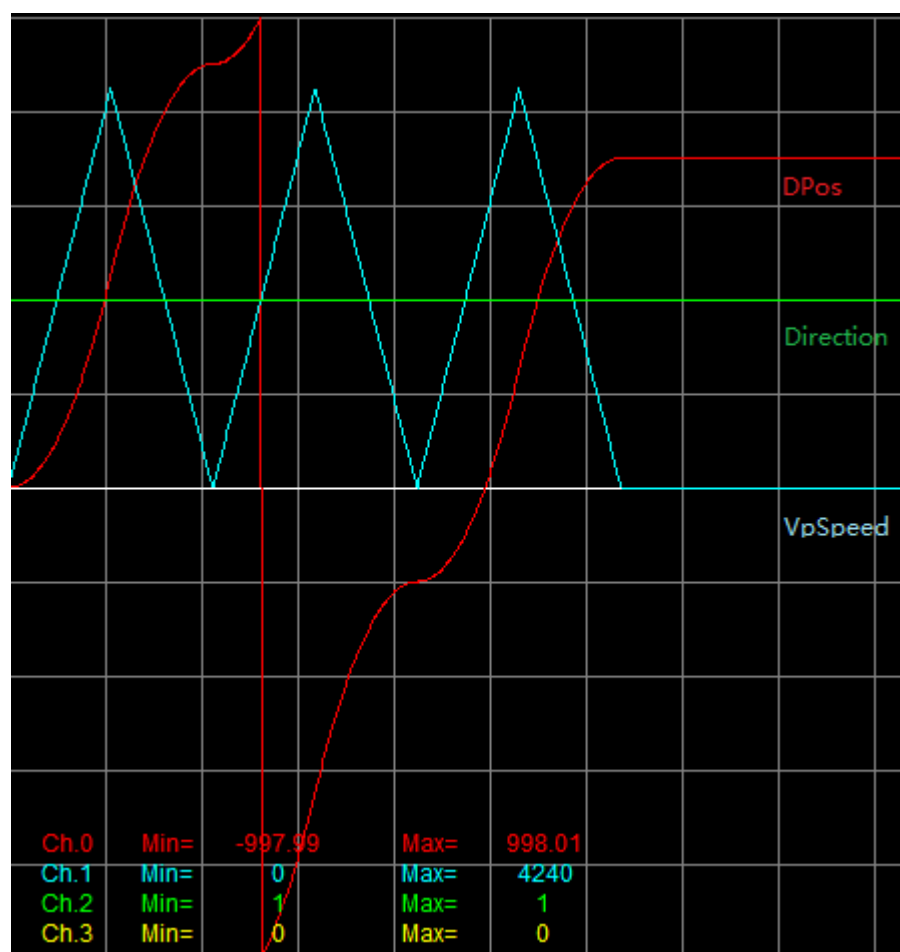
```
axis0.RepMode := 2;
axis0.RepDist := 1000;
hmc_defpos(0,0);
hmc_moveabs(0, 900);
hmc_moveabs(0, -200);
hmc_moveabs(0, 700);
```



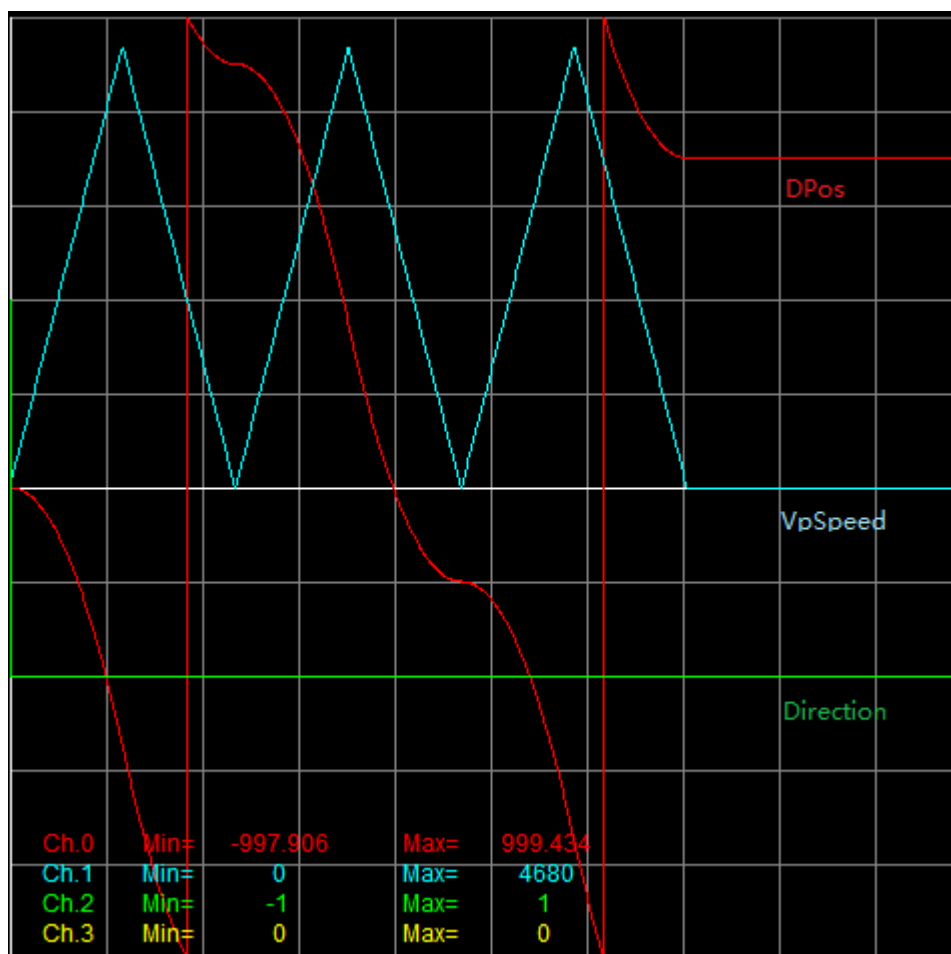
MoveAbsMode=0 在循环行程内确认方向



MoveAbsMode=1 取最短距离方向



MoveAbsMode=0 始终正方向



MoveAbsMode=0 始终负方向

3.4.1.1.4.14 CancelMode（Cancel 时指令停止模式）

基本属性

类型	USINT
读写属性	读写
设置函数	-
初始值	0
取值范围	0、1、2
相关章节	CancelPos（Cancel 时停止位置） HMC Cancel（撤销指令）

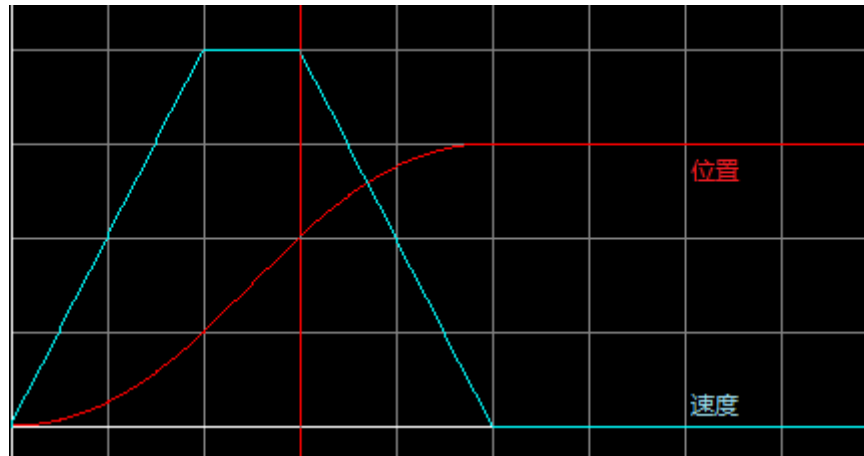
含义描述

Cancel 时指令停止模式。

指令撤销过程中，在接受到 Cancel 指令后，轴会以当前速度开始进行减速直到停止。该减速停止过程中具体停止方式，由轴参数 CancelMode 指定。具体如下：

- (1) CancelMode=0 自然停止

默认值，按照 Decel 自然停止。此模式支持所有指令类型。

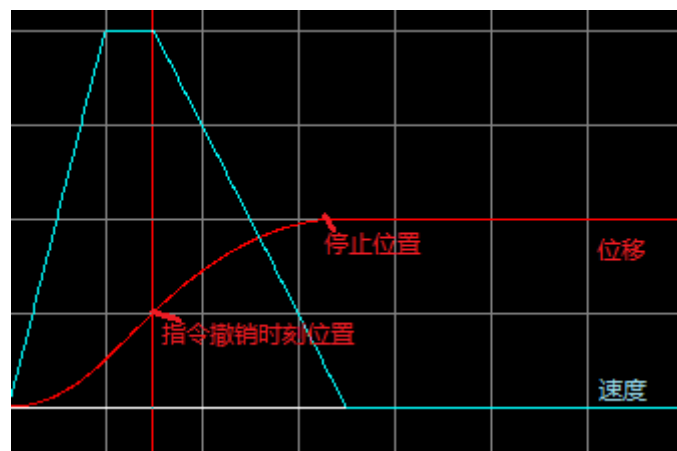


指令停止示意图

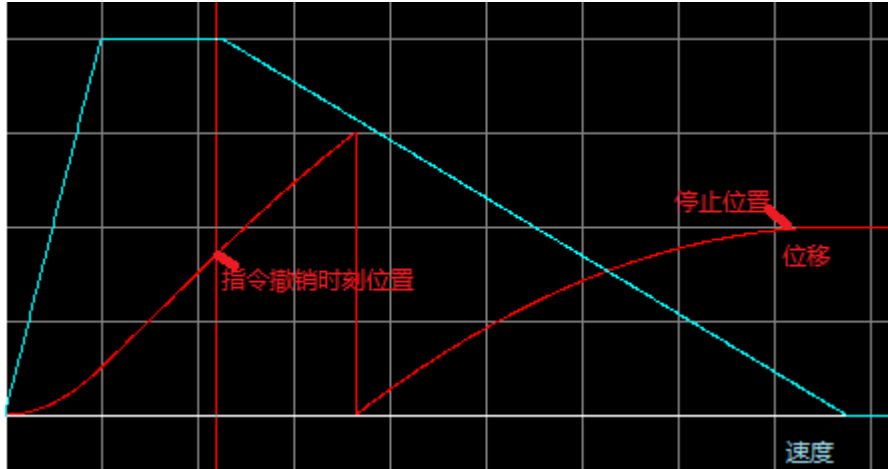
(2) CancelMode=1 指定位置停止

按照不超过减速度 Decel 停到指定位置，该位置由参数 CancelPos 指定。此模式仅对循环行程模式 (RepMode=1 或 2) 且当前指令为单向运动 (HMC_Forward、HMC_Reverse) 的情况生效。若单向运动指令，但不满足循环行程模式，则按照 CancelMode=0 处理。

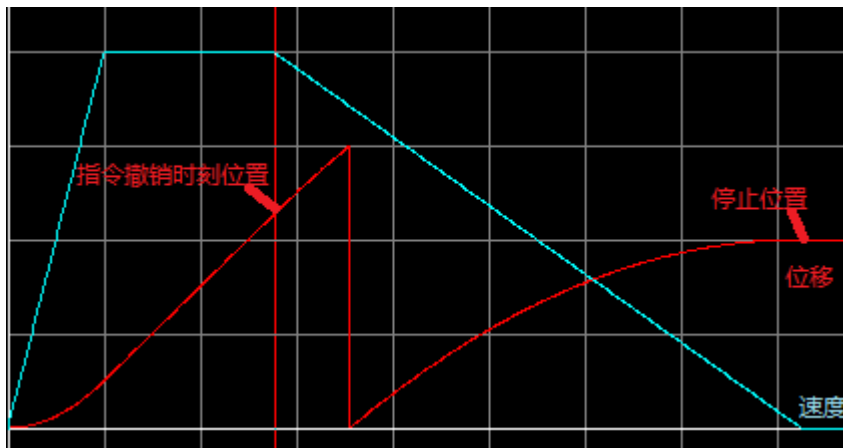
根据当前位置和 CancelPos 的相对关系，会有三种情况：1、当前位置 < 停止位置，且两者距离位移差足够当前速度以减速度停止为 0；2、当前位置 < 停止位置，但两者距离位移差不够当前速度以减速度停止为 0；3、当前位置 > 停止位置。以上三种情况分别对应如下三图：



指令停止示意图



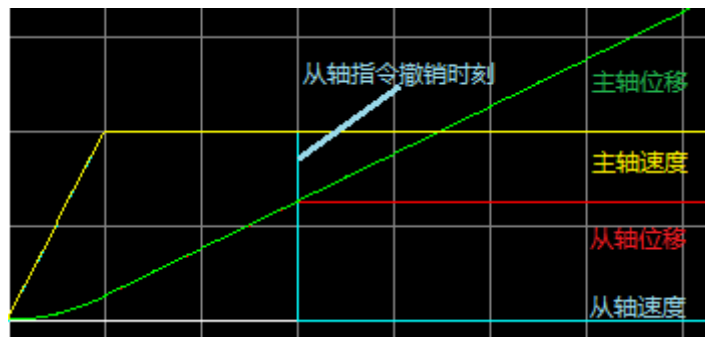
指令停止示意图



指令停止示意图

(3) CancelMode=2 立即停止

立即停止，速度从当前速度直接减速为 0。此模式仅对正在执行联动指令的轴生效，若状态不满足，则默认按照 CancelMode=0 处理。



指令停止示意图

- 不支持的模块版本

MC1008-A01

3.4.1.1.4.15 CancelPos (Cancel 时停止位置)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	该值应在循环行程的上下限之间
相关章节	CancelMode (Cancel 时指令停止模式)

含义描述

循环行程轴且指令为 HMC_Forward 或 HMC_Reverse，同时 CancelMode=1 时，该参数有效，该参数表示在指令 Cancel 完成时轴将到达的位置。但 CancelPos 超行程限时，按照 CancelMode=0 处理。

- 不支持的模块版本

MC1008-A01

3.4.1.1.5 系数因子参数 (Gains)

3.4.1.1.5.1 Kp (PID 的比例系数)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	1
取值范围	浮点数

含义描述

当控制方式为位置闭环模式，需要经过 PID 模型运算，Kp 即 PID 模型的比例系数，其作用与比例项，比例项作用= $Kp \cdot Fe$ 。

3.4.1.1.5.2 Ki (PID 的积分增益)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	浮点数

含义描述

当控制方式为位置闭环模式，需要经过 PID 模型运算，Ki 即 PID 模型的积分增益，其作用与积分项，积分项作用= $Ki \cdot (\sum (Fe \cdot dt))$ 。

3.4.1.1.5.3 Kd (PID 的微分增益)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	浮点数

含义描述

当控制方式为位置闭环模式，需要经过 PID 模型运算，Kd 即 PID 模型的微分增益，其作用与微分项，微分项作用 = $Kd \cdot (d(Fe)/dt)$ 。

3.4.1.1.5.4 Kov (测量速度增益)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	浮点数

含义描述

当控制方式为位置闭环模式，会在 PID 计算输出基础上叠加测量速度增益项。测量速度增益作用 = $Kov \cdot MpSpeed$ 。

3.4.1.1.5.5 Kvff (速度前馈增益)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	浮点数

含义描述

当控制方式为位置闭环模式，会在 PID 计算输出基础上叠加速度前馈增益项。速度前馈增益作用 = $Kvff \cdot ((+1/-1) \cdot VpSpeed)$ 。

3.4.1.1.5.6 Kaff (加速度前馈增益)

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	浮点数

含义描述

加速度前馈增益，加速度前馈= K_{aff} *解算加速度。

3.4.1.1.5.7 Ko（PID 输出放大因子）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	1
取值范围	浮点数

含义描述

伺服输出比例，PID 计算后伺服输出值的放大因子。

3.4.1.1.6 限幅限位参数（Limits）

3.4.1.1.6.1 FELimits（随动误差限制）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	1000
取值范围	正的双精度浮点数（64 位）
相关章节	-

含义描述

FE（随动误差）的正常范围。当 $Abs(FE) > FELimit$ 时，则认为 FE 超限。当 FE 超限时，系统将会将轴参数 AxisStatus 中 bit0 置 1，并关闭伺服使能。如果要屏蔽 FE 超限关闭伺服使能动作，可以把 AxisErrMask 的 bit0 置 0。

3.4.1.1.6.2 FsLimitMode/RsLimitMode（前/后限位时指令撤销方式）

基本属性

类型	USINT
读写属性	只读
设置函数	-
初始值	0
取值范围	0: 遇到前/后向限位后自动撤销当前指令 1: 遇到前/后向限位后手动撤销当前指令(暂不支持)
相关章节	-

含义描述

表示前/后向限位控制模式，对软限位及硬限位均有效。该参数仅对有限行程的轴是有效的，对于循环行程的轴无效。

当一个有限行程轴超限时，会开始紧急停止动作。当前指令如何处理由该轴相关参数决定，详见“异常处理”章节中“有限行程轴超限处理”。

0：遇到前/后向限位后自动撤销当前指令。需要特别说明的是，对插补指令而言，无论合轴还是分轴遇到限位时，整个插补指令都将被撤销。

1：遇到前/后向限位后手动撤销当前指令。需要特别说明的是，当前指令在遇到函数 HMC_Cancel 之前是不会被撤销的，在轴完成紧急停止后，当原运动指令解算出来的 DPos 值加重超限时，轴继续停止不动；当原运动指令解算出来的 DPos 值减缓超限时，轴将按照 DPos 值运动。

这里的“加重超限”指：当超越前向软/硬软限位时，当前指令解算的 DPos 值越来越大；后者当超越后向软/硬软限位时，当前指令解算的 DPos 值越来越小。直观来讲，就是超限的越来越严重。“减缓超限”的含义刚好与之相反。

3.4.1.1.6.3 FHLimitIn/RHLimitIn（前/后向硬限位通道号）

基本属性

类型	INT
读写属性	只读
设置函数	HMC_HwLimitConfig
初始值	0
取值范围	-1：未启用； DI 通道号：快速、慢速或虚拟 DI 通道号
相关章节	-

含义描述

设置前/后向硬限位行程开关所对应的 DI 通道号。

循环行程时（RepMode =1 或 2）无意义。

3.4.1.1.6.4 FSLimit/RSLimit（前/后向软限位边界值）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	RSLimit: -1e100 FSLimit: 1e100
取值范围	双精度浮点数（64 位）。需确保 RSLimit< FSLimit。
相关章节	-

含义描述

有限行程的前/后向软限位界限设定。循环行程时（RepMode =1 或 2）无意义。

3.4.1.1.6.5 DACLimitHigh/DACLimitLow（DAC 输出上限/下限）

基本属性

类型	DINT
读写属性	读写
设置函数	-
初始值	DACLimitLow: -32,768 DACLimitHigh: 32,767
取值范围	轴类型为 20 时取值范围: -32,768~32,767; 其他轴类型时取值范围: -2147483648~2147483647 需满足 DACLimitHigh>DACLimitLow
相关章节	-

含义描述

DACOut 的上限/下限，确保 DACOut 的输出不会超过设定范围。

3.4.1.1.7 原点和保持参数（Home&Hold）

3.4.1.1.7.1 HomeState（原点回归状态）

基本属性

类型	USINT
读写属性	只读
设置函数	-
初始值	0
取值范围	以下描述与 HMC_Home 或 HMC_HomeEx 函数的 Mode 参数相关。 0: 尚未开始寻找原点或者已经完成了一个找原点过程 1: 该功能块处于第一阶段运行找原点状态（找 DI（MODE=0、1、4 或 5）或 Z 相（MODE=2 或 3）） 2: 该功能块处于第二阶段运行找原点状态（找到 DI（MODE=0、1、4 或 5）或 Z 相（MODE=2 或 3）后减速到 0） 3: 该功能块处于第三阶段运行找原点状态（找到 DI（MODE=0、1、4 或 5）并减速到 0 后，反向运动找 DI 下降沿或 Z 相） 4: 该功能块处于第四阶段运行找原点状态（找到 DI（MODE=0、1、4 或 5）并减速到 0 后，反向运动找到 DI 下降沿或 Z 相，减速到 0）
相关章节	HMC_Home（原点回归） HMC_HomeEx（高级原点回归）

含义描述

原点回归状态，用于指示 HMC_Home、HMC_HomeEx 在原点回归过程所处状态。原点回归过程需经历 0、1、2、3、4 等 5 个状态，各个状态含义见上标取值范围。用户可通过查看该参数，清楚了解当前原点回归情况。

3.4.1.1.7.2 HomeIn（原点回归所用的 DI 通道）

基本属性

类型	INT
读写属性	只读
设置函数	HMC_Home、HMC_HomeEx
初始值	-1

类型	INT
取值范围	HMC_Home 时取值范围 0~63（DI 通道号） HMC_HomeEx 时取值范围 0~11（快速 DI 通道） -1 时表示未使用原点回归功能
相关章节	HMC_Home（原点回归） HMC_HomeEx（高级原点回归）

含义描述

原点回归所用的 DI 通道号，该参数值是在投放 HMC_Home、HMC_HomeEx 指令时设置。

3.4.1.1.7.3 HomeCapIn（原点回归使用的高速捕获通道）

基本属性

类型	INT
读写属性	只读
设置函数	HMC_HomeEx
初始值	-1
取值范围	当前控制器型号所支持的高速捕获通道数目，MC1008、MC1004 为 8 路，MC1002R、MC1002E 为 2 路
相关章节	HMC_HomeEx（高级原点回归）

含义描述

HMC_HomeEx 所使用的高速捕获通道号，在投放 HMC_HomeEx 指令时设置。

3.4.1.1.7.4 HoldSpeed（强制速度）

基本属性

类型	LREAL
读写属性	读写
设置函数	-
初始值	0
取值范围	正浮点数
相关章节	HMC_HoldConfig(设置强制速度开关)

含义描述

当速度强制开关被触发时，轴会从当前速度加减速至 HoldSpeed。一种常用的使用方法是设置 HoldSpeed=0，当 Hold 触发源（HoldIn）为 1 时，Speed 被强制成 0，这样可以实现“暂停”功能。

速度强制功能详细描述可参考章节 [HMC_HoldConfig\(设置强制速度开关\)](#)。

3.4.1.1.7.5 HoldIn（速度强制开关 DI 通道号）

基本属性

类型	INT
读写属性	只读
设置函数	HMC_HoldConfig
初始值	-1
取值范围	0~63（DI 通道号） -1 表示关闭该功能

类型	INT
相关章节	HMC HoldConfig(设置强制速度开关)

含义描述

速度强制功能的触发源 DI 通道号。

3.4.1.1.8 输入相关参数 (Input)

3.4.1.1.8.1 KeNumerator/KeDenominator (编码器输入比例 Ke 的分子/分母)

基本属性

类型	DINT
读写属性	只读
设置函数	HMC_SetKe
初始值	1
取值范围	DINT 整数: 分子、分母均不为 0, 但可异号
相关章节	HMC_SetKe (设置 Ke)

含义描述

Ke 的分子和分母。参数 Ke 仅对有实际位置反馈的轴类型起作用 (伺服轴、编码器轴、总线轴), 可为负。该参数和 Units 有类似的作用, (Ke /Units)共同构成了伺服轴或编码器轴的“单位转换因子”, 其单位为“线/用户单位”, 可查看“系统控制模式”章节中的位置闭环框图了解 Ke 的作用方式。

3.4.1.1.8.2 EncoderValue (内部编码器值)

基本属性

类型	DINT
读写属性	只读
设置函数	-
初始值	0
取值范围	-2147483648~+2147483647
相关章节	-

含义描述

当该轴通过编码器反馈位置信息时, 该轴参数表示内部编码器值, 每个伺服周期进行刷新。根据该值增量计算得到实际位置值 MPos。

3.4.1.1.8.3 FeedBackSrcMode (轴位置反馈源模式)

基本属性

类型	USINT
读写属性	读写
设置函数	-
初始值	0
取值范围	0: 反馈输入源为轴实际物理连接输入, 如 20、30 模式时为编码器输入, 40、41、50、51 时为总线输入 1: 反馈输入源为用户自定义输入

类型	USINT
相关章节	HMC_SetFeedBackSrcAddr（设置用户自定义反馈输入地址） FeedBackSrcValue（轴反馈自定义输入实时值）

含义描述

设定轴反馈值 Mpos 输入源方式，仅对有实际输入的轴类型生效，例如编码器轴、伺服轴、总线轴类型等。

通过设置 FeedBackSrcMode=0 使得 Mpos 输入源为轴实际物理连接输入；通过设置 FeedBackSrcMode=1，使得 MPos 输入源为用户自定义输入，该用户自定义地址通过 HMC_SetFeedBackSrcAddr 函数设置。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.1.8.4 FeedBackSrcValue（轴反馈自定义输入实时值）

基本属性

类型	LREAL
读写属性	只读
设置函数	HMC_FeedBackSrcAddr
初始值	0
取值范围	浮点数
相关章节	HMC_SetFeedBackSrcAddr（设置用户自定义反馈输入地址） FeedBackSrcMode（轴位置反馈源模式）

含义描述

对于有实际反馈输入的轴，可通过设置 FeedBackSrcMode=1，调用 HMC_SetFeedBackSrcAddr 配置某参数地址作为此时轴的反馈输入源，即此时该轴的 Mpos 根据该地址对应参数来计算。FeedBackSrcValue 实时显示关联的自定义参数值。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.1.8.5 FeedBackDataType（轴反馈自定义输入变量数据类型）

基本属性

类型	USINT
读写属性	只读
设置函数	HMC_SetFeedBackSrcAddr
初始值	0
取值范围	USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、REAL=7、LREAL=8
相关章节	HMC_SetFeedBackSrcAddr（设置用户自定义反馈输入地址） FeedBackSrcMode（轴位置反馈源模式）

含义描述

FeedBackDataType 表示用户自定义输入变量的数据类型。FeedBackSrcValue 实时显示关联的自定义

参数值。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-B02、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A01

3.4.1.1.9 输出相关参数（Output）

3.4.1.1.9.1 KsNumerator/KsDenominator（步进输出比例 Ks 的分子/分母）

基本属性

类型	DINT
读写属性	只读
设置函数	HMC_SetKs
初始值	1
取值范围	DINT 整数：分子、分母均不为 0，且同号
相关章节	HMC_SetKs（设置 Ks）

含义描述

步进输出比例 Ks 的分子/分母，通过 HMC_SetKs 设置。该参数仅对步进轴（10~19）、RTEX_Position（40）、EtherCAT_Position（50）有效。参见 [HMC_SetKs（设置 Ks）](#)。

3.4.1.1.9.2 PulseNum（步进轴周期输出脉冲数）

基本属性

类型	UDINT
读写属性	只读
设置函数	-
初始值	0
取值范围	0xffffffff
相关章节	-

含义描述

每个伺服周期，步进轴输出的脉冲数。

3.4.1.1.9.3 ServoLoop（闭环输出开关）

基本属性

类型	USINT
读写属性	读写
设置函数	-
初始值	0
取值范围	0：表示开环，模拟量输出（DACOut）采用 DAC 参数所给定的数值 1：表示闭环，模拟量输出（DACOut）为 PID 运算的结果
相关章节	-

含义描述

闭环输出开关，该参数仅对位置闭环模式的轴类型生效，如伺服轴 20、RTEX 总线速度模式 41，EtherCAT 总线速度模式 51 等。通过设置该开关决定位置闭环是否生效：

(1) ServoLoop=0 表示开环

该轴处于位置开环模式。此时 Mpos、MpSpeed 来自于编码器的测量值，而 DPos 跟踪 MPos。输出值 DACOut 采用 DAC 参数所给定的数值，即开环模式。

(2) ServoLoop=1 表示闭环

该轴处于位置闭环控制模式。Dpos、VpSpeed 为运动控制指令实时计算值，Mpos、MpSpeed 是来自于编码器的测量值，通过对 DPos 和 MPos 进行 PID 计算，PID 将计算结果更新至输出值 DACOut，达到位置闭环的控制效果。

需要说明的是：当系统出现异常时，会通过轴参数 AxisStatus 来显示，当 AxisStatus 和 AxisErrMask 的逻辑与运算非零时，系统将进行异常处理，此时，ServoLoop 将被系统强制修改为 0。

3.4.1.1.9.4 DAC（速度模式开环输出值）

基本属性

类型	DINT
读写属性	读写
设置函数	-
初始值	0
取值范围	-2,147,483,648~2,147,483,647
相关章节	-

含义描述

ServoLoop=0 时的 DACOut 的输出值。

3.4.1.1.9.5 DACOut（速度模式输出值）

基本属性

类型	DINT
读写属性	只读
设置函数	-
初始值	0
取值范围	-2,147,483,648~2,147,483,647
相关章节	-

含义描述

当前轴的速度量输出结果（直连速度模式（20），该速度量转换为 AO 模拟量来控制伺服驱动器速度；RTEX_Speed（41）/EtherCAT_Speed（51），该速度量直接通过总线给伺服驱动器）。

3.4.1.1.9.6 PosOffset (DACOut 正向死区值)

基本属性

类型	DINT
读写属性	读写
设置函数	-
初始值	0
取值范围	直连轴: -32,768~32,767 总线轴: -2,147,483,648~2,147,483,647
相关章节	-

含义描述

伺服轴输出正向死区值。

当伺服轴计算的输出值 DACOut 为正时，则在此 DACOut 基础上叠加正向死区值 PosOffset。DACOut 为 0 时，固定输出 $(\text{PosOffset} + \text{NegOffset}) / 2$ 。即通过设置死区后，最终输出的 DACOut 永远在伺服执行机构的电压死区范围外，避免执行机构长期处于死区影响控制效果的现象。

对于直连轴，DACOut 取值范围为-32,768~32,767。（-32768，32767）线性对应输出电压（-10，10）V。

对于伺服总线轴，DACOut 取值范围为-2,147,483,648~2,147,483,647。需根据执行机构情形进行计算并设置。

应用举例

若伺服轴执行机构存在死区，其死区对应电压为（-1，2）V，则需设置 NegOffset=-3278，PosOffset=6553。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.1.9.7 NegOffset (DACOut 负向死区值)

基本属性

类型	DINT
读写属性	读写
设置函数	-
初始值	0
取值范围	直连轴: -32768~32767 总线轴: -2147483648~2147483647
相关章节	-

含义描述

伺服轴输出负向死区值。

当伺服轴计算的输出值 DACOut 为负时，则在此 DACOut 基础上叠加负向死区值 NegOffset。DACOut 为 0 时，固定输出 $(\text{PosOffset} + \text{NegOffset}) / 2$ 。即通过设置死区后，最终输出的 DACOut 永远在伺服执行机构的电压死区范围外，避免执行机构长期处于死区影响控制效果的现象。

3.4.1.1.9.8 ZeroWindow (DACOut 零点间隙死区)

基本属性

类型	DINT
读写属性	读写
设置函数	-
初始值	0
取值范围	直连轴: -32768~32767 总线轴: -2147483648~2147483647
相关章节	-

含义描述

DACOut 零点间隙死区。DACOut 位于死区窗口[-ZeroWindow, ZeroWindow]时，输出 NegOffset 和 PosOffset 的中值（即 $(\text{PosOffset} + \text{NegOffset}) / 2$ ）。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.1.9.9 OutDstMode (DACOut 输出目标模式)

基本属性

类型	USINT
读写属性	读写
设置函数	-
初始值	0
取值范围	0: 轴计算的 DACOut 输出值的目标为实际物理连接。 1: 轴计算的 DACOut 输出值的目标为用户自定义变量。
相关章节	HMC_SetOutDstAddr (设置用户自定义输出地址)

含义描述

对于速度模式的轴类型，如伺服轴 20、RTEX 总线速度模式 41，EtherCAT 总线速度模式 51 等，可以通过设置 OutDstMode 来指定输出值 DACOut 的最终输出目标。

(1) OutDstMode=0

轴计算的 DACOut 输出目标为实际物理连接，如 20 模式目标输出为对应 AO 通道，41、51 时目标输出为总线输出通道。

(2) OutDstMode=1

轴计算的 DACOut 输出目标为用户自定义变量，自定义变量值通过 OutDstValue 显示。通过 HMC_SetOutDstAddr 设置用户自定义变量。

此时物理输出通道的状态如下：20 轴类型时，AO 通道不与轴关联，AO 通道变为普通模拟量输出通道；对于 41、51 模式下，为了安全起见，此时会置总线驱动器速度为 0，使总线驱动器停止转动。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.1.9.10 OutDstValue (DACOut 输出自定义地址实时值)

基本属性

类型	LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	-
相关章节	HMC_SetOutDstAddr (设置用户自定义输出地址)

含义描述

OutDstMode=1 时, OutDstValue 实时显示用户自定义变量数值。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.1.9.11 OutDstDataType (DACOut 输出自定义变量数据类型)

基本属性

类型	USINT
读写属性	只读
设置函数	HMC_SetOutDstAddr
初始值	0
取值范围	USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、REAL=7、LREAL=8
相关章节	HMC_SetOutDstAddr (设置用户自定义输出地址)

含义描述

设置 OutDstMode=1 时, 调用 HMC_SetOutDstAddr 配置轴的 DACOut 输出至用户自定义量, OutDstDataType 表示用户自定义变量的数据类型。OutDstValue 实时显示用户自定义变量数值。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-B02、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A01

3.4.1.1.10 预留参数 (Reserved)

3.4.1.1.10.1 SystemValue*

基本属性

类型	SystemValue0、SystemValue1: UDINT SystemValue2、SystemValue3: LREAL
读写属性	只读
设置函数	-
初始值	0
取值范围	-
相关章节	-

含义描述

用于系统 Debug，或者记录系统的故障信息。目前有部分固定轴号的预留轴参数已经使用，其具体含义如下：

轴号	SystemValue0	SystemValue1	SystemValue2	SystemValue3
0	算法解算时间	算法的小版本号	-	-
1	算法调度间隔时间	最后一次系统错误码	-	-
2	算法最大解算时间		-	-
3	最后一次用户错误码	最后一次用户错误信息	-	-
4	运动控制算法、协议模块的总执行时间	运动控制算法、协议模块总执行时间超出伺服周期的累计次数	-	-

3.4.1.2 系统启动指令

3.4.1.2.1 HMC_DriveEnable（伺服使能）

3.4.1.2.1.1 函数原型

UDINT HMC_DriveEnable(USINT bMode)

3.4.1.2.1.2 功能说明

控制与控制器相连的所有伺服驱动器使能信号的开启与关闭。

（1）对于直连脉冲型的伺服驱动器，DriveEnable 信号对应的引脚直接与驱动器上的伺服使能信号相连接；对于总线类型伺服驱动器，DriveEnable 信号通过总线通信发出。

（2）DriveEnable 禁止时，运动控制器各轴停止输出（例如直连伺服轴 DACOut 为 0，直连步进轴也停止发脉冲）。

3.4.1.2.1.3 输入参数

输入参数	类型	参数描述
bMode	USINT	0: 使能禁止 1: 使能开启

3.4.1.2.1.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
0xFFFFFFFF	UDINT	函数参数错误

3.4.1.2.1.5 指令类型

非轴运动缓存指令。

■ 补充说明

(1) 该指令控制与控制器相连的所有驱动器的伺服使能状态，为伺服使能总开关。对于总线轴类型，还可实现伺服分使能控制，即单独控制某个轴的使能状态，详见总线相关指令 RTEX_DriveEnable、EtherCAT_DriveEnable。

(2) 在 AutoThink 菜单栏点击使能按钮，也可控制伺服使能状态切换；

(3) 当某个轴出现某些异常时，其 AxisStatus 对应的 bit 位会被置 1，如果 AxisErrMask 对应的 bit 位也为 1，此时异常处理程序可能会自动禁用 DriveEnable 信号。

3.4.1.2.1.6 应用举例

HMC_DriveEnable (1); (*开启伺服使能*)

HMC_DriveEnable (0); (*关闭伺服使能*)

3.4.1.2.2 HMC_GetDriveEnableState（获取伺服使能状态）

3.4.1.2.2.1 函数原型

USINT HMC_GetDriveEnableState ()

3.4.1.2.2.2 功能说明

获取伺服总使能（DriveEnable）状态。

3.4.1.2.2.3 返回值

返回值	类型	参数描述
0	USINT	禁用
1	USINT	使能

3.4.1.2.2.4 指令类型

非轴运动缓存指令。

3.4.1.2.2.5 补充说明

对于总线轴类型，还可获取某个轴的伺服分使能状态，详见 RTEX_GetDriveEnableState、EtherCAT_GetDriveEnableState。

3.4.1.2.2.6 应用举例

State:=HMC_GetDriveEnableState (); (*获取伺服总使能状态*)

3.4.1.3 单轴运动指令

3.4.1.3.1 原点搜寻及位置定义

3.4.1.3.1.1 HMC_Home（原点回归）

函数原型

UDINT HMC_Home (USINT ucAxisID, USINT ucHomeMode, USINT ucDiChI)

功能

设备在上电后正常工作前一般首先需要寻找机械零点，本函数通过检测原点开关信号或伺服电机编码器 Z 相信号为该轴确定原点，原点回归方式由参数 **ucHomeMode** 确定。原点开关信号的获取使用的是控制器的某个快速/慢速 DI 通道，也可用虚拟 DI 通道模拟产生；Z 相信号从原点回归轴 DB9 接口的 Z 相接口获取。

慢速 DI 由于响应频率较低，原点回归精度不高，可使用快速 DI 通道提高回归精度。受安装精度及振动因素影响，编码器 Z 相信号比常用的开关传感器（如接近开关、机械式行程开关等）信号具有更高的精度与可靠性，因此在精度要求较高的场合，可使用 原点开关+Z 相信号相结合的方式进行原点回归。

参数说明

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucHomeMode	USINT	0: 仅使用 DI 信号正向定位原点 1: 仅使用 DI 信号负向定位原点 2: 仅使用本轴 Z 相信号正向定位原点 3: 仅使用本轴 Z 相信号负向定位原点 4: 使用 DI 及本轴 Z 相信号正向定位原点 5: 使用 DI 及本轴 Z 相信号负向定位原点 注：详见应用示例
ucDiChI	USINT	原点回归时使用的 DI 通道号：0~63（快速/慢速/虚拟 DI）

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

注意事项

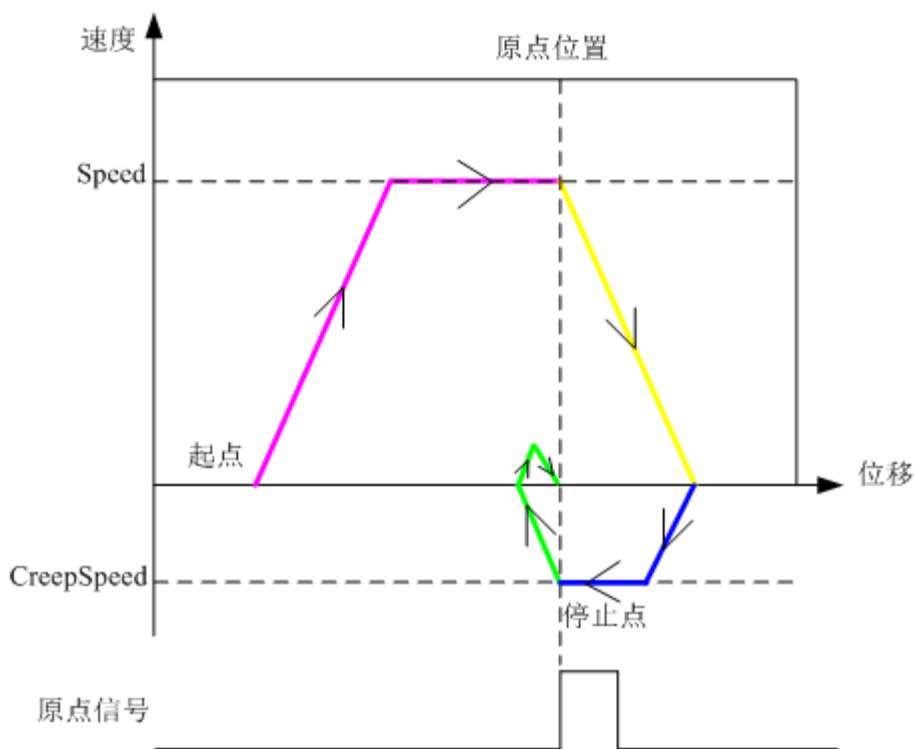
- 对于 RTEXT 和 EtherCAT 总线轴类型，由于总线类型的驱动器再生脉冲口未提供 Z 相信号输出，所以涉及到 Z 相信号的原点回归功能不可用。
- HMC_Home 设计采用查询机制获取触发信号，查询周期为一个伺服周期，故触发信号高电平持续时间 $\geq$ 一个伺服周期时，可在每个查询周期下捕获到触发信号。如果触发信号高电平持续时间 $<$ 一个伺服周期时，不能保证每个查询周期下都能捕获到原点信号，影响原点回归正常工作。

应用示例

- 模式 0 原点回归（ucHomeMode = 0）

仅使用 DI 信号正向定位原点。轴以 **Speed** 设定的速度正向运动（阶段 1），当遇到 DI 上升沿时减速停止（阶段 2），然后以 **CreepSpeed** 设定的速度反向运动（阶段 3），遇到 DI 下降沿时，设定此处为原点位置并开始减速停止，然后再返回到原点位置（阶段 4）。

特殊情形：如果一开始 DI 为 1，则直接进入阶段 3。



模式 0 原点回归

（紫色、黄色、蓝色、绿色 对应回归阶段 1、2、3、4）

示例程序如下（ST 语言）：

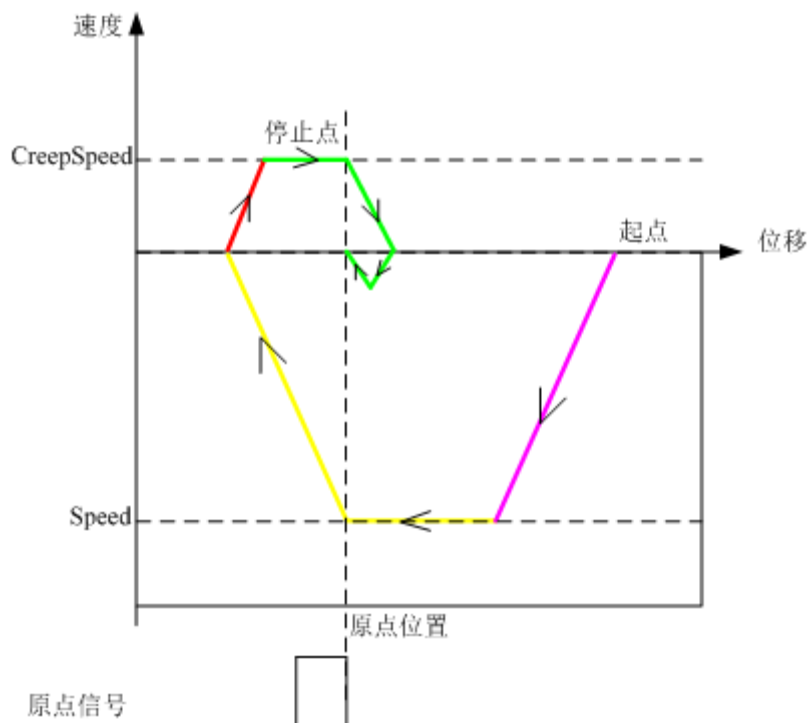
```
Axis0.Speed := 1000;    (*设定原点回归速度(阶段 1 速度)*)
Axis0.CreepSpeed := 100;  (*设定爬行速度(阶段 3 速度)*)
Axis0.Accel := 2000;     (*设定加速度*)
Axis0.Decel := 2000;    (*设定减速度*)

HMC_Home(Axis0.AxisID, 0, 0); (*向 Axis0 轴投送原点回归指令(模式 0, 使用 0 号快速 DI 通道)*)
```

(2) 模式 1 原点回归 (ucHomeMode = 1)

仅使用 DI 信号负向定位原点。轴以 **Speed** 设定的速度负向运动（阶段 1），当遇到 DI 上升沿时减速停止（阶段 2），然后以 **CreepSpeed** 设定的速度反向运动（阶段 3），遇到 DI 下降沿时，设定此处为原点位置并开始减速停止，然后再返回到原点位置（阶段 4）。

特殊情形：同模式 0。



模式 1 原点回归

（紫色、黄色、红色、绿色 对应回归阶段 1、2、3、4）

示例程序如下（ST 语言）：

Axis0.Speed := 1000; (*设定原点回归速度(阶段 1 速度)*)

Axis0.CreepSpeed := 100; (*设定爬行速度(阶段 3 速度)*)

Axis0.Accel := 2000; (*设定加速度*)

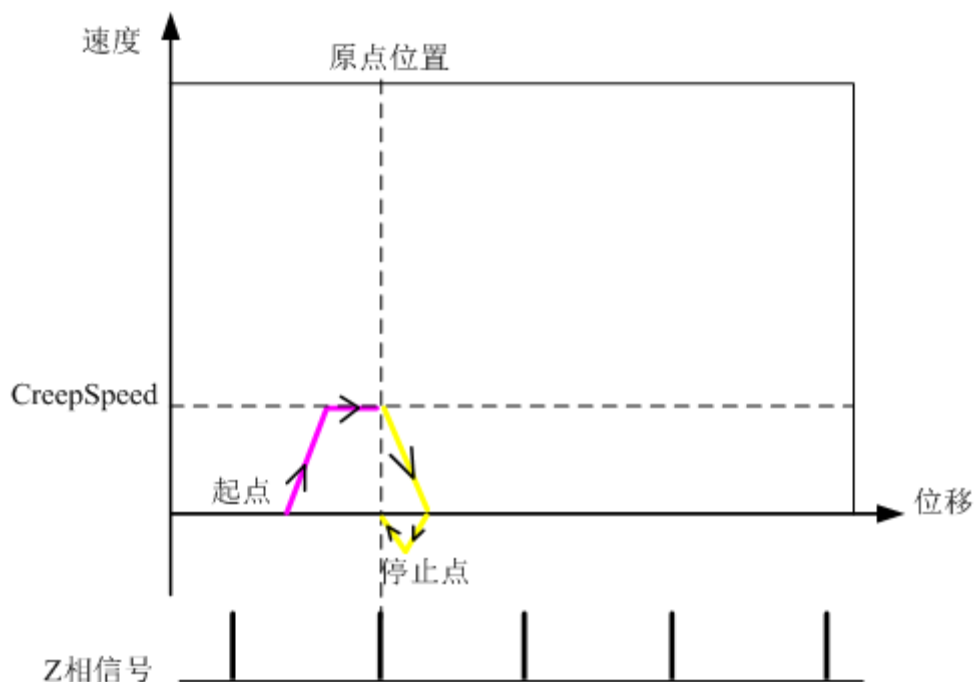
Axis0.Decel := 2000; (*设定减速度*)

HMC_Home(Axis0.AxisID, 1, 0); (*向 Axis0 轴投送原点回归指令(模式 1, 使用 0 号快速 DI 通道)*)

(3) 模式 2 原点回归 (ucHomeMode = 2)

仅使用本轴 Z 相信号正向定位原点。轴以 CreepSpeed 设定的速度正向运动（阶段 1），当 Z 相为 1 时，设定此处为原点位置并开始减速停止，然后再返回到原点位置。（阶段 2）。

特殊情形：如果一开始 Z 相为 1，则直接进入阶段 2。



模式 2 原点回归（紫色、黄色 对应回归阶段 1、2）

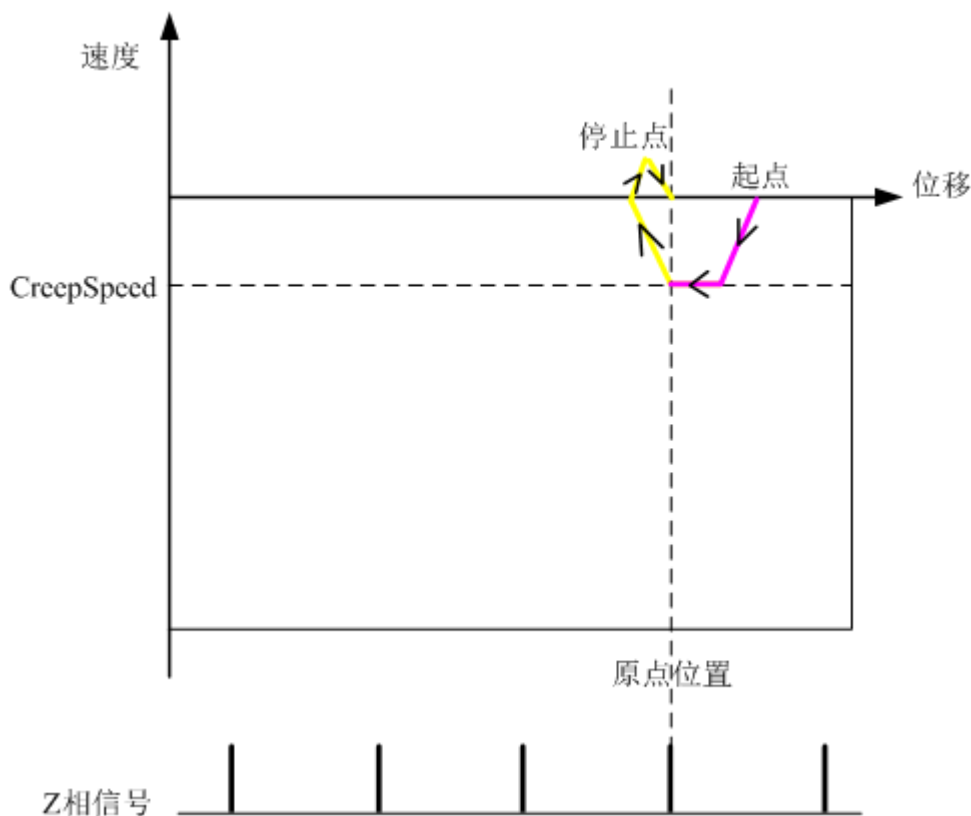
示例程序如下（ST 语言）：

```
Axis0.Speed := 1000;      (*设定原点回归速度(阶段 1 速度)*)
Axis0.CreepSpeed := 100;  (*设定爬行速度(阶段 3 速度)*)
Axis0.Accel := 2000;      (*设定加速度*)
Axis0.Decel := 2000;      (*设定减速度*)
HMC_Home(Axis0.AxisID, 2, 0); (*向 Axis0 轴投送原点回归指令(模式 2, 使用 Axis0 轴 Z 相通道)*)
```

(4) 模式 3 原点回归（ucHomeMode = 3）

仅使用本轴 Z 相信号负向定位原点。轴以 CreepSpeed 设定的速度负向运动（阶段 1），当 Z 相为 1 时，设定此处为原点位置并开始减速停止，然后再返回到原点位置（阶段 2）。

特殊情形：同模式 2。



模式 3 原点回归

(紫色、黄色 对应回归阶段 1、2)

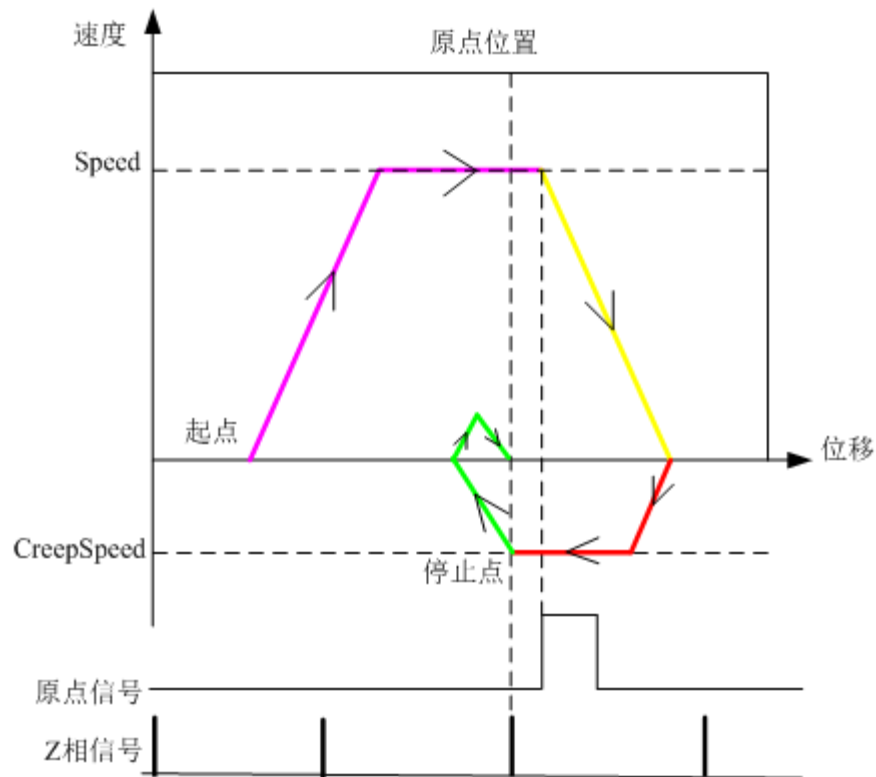
示例程序如下 (ST 语言) :

```
Axis0.Speed := 1000;      (*设定原点回归速度(阶段 1 速度)*)
Axis0.CreepSpeed := 100;  (*设定爬行速度(阶段 3 速度)*)
Axis0.Accel := 2000;      (*设定加速度*)
Axis0.Decel := 2000;      (*设定减速度*)
HMC_Home(Axis0.AxisID, 3, 0); (*向 Axis0 轴投送原点回归指令(模式 3, 使用 Axis0 轴 Z 相通道)*)
```

(5) 模式 4 原点回归 (ucHomeMode=4)

使用 DI 及本轴 Z 相信号正向定位原点。轴以 Speed 设定的速度正向运动 (阶段 1)，当遇到 DI 上升沿时减速停止 (阶段 2)，然后以 CreepSpeed 设定的速度反向运动 (阶段 3)，遇到 DI 下降沿后且遇到 Z 相为 1 时，设定此处为原点位置并开始减速停止，然后再返回到原点位置 (阶段 4)。

特殊情形：如果一开始 DI 为 1，则直接进入阶段 3。



模式 4 原点回归

（紫色、黄色、红色、绿色 对应回归阶段 1、2、3、4）

示例程序如下（ST 语言）：

Axis0.Speed := 1000; (*设定原点回归速度(阶段 1 速度)*)

Axis0.CreepSpeed := 100; (*设定爬行速度(阶段 3 速度)*)

Axis0.Accel := 2000; (*设定加速度*)

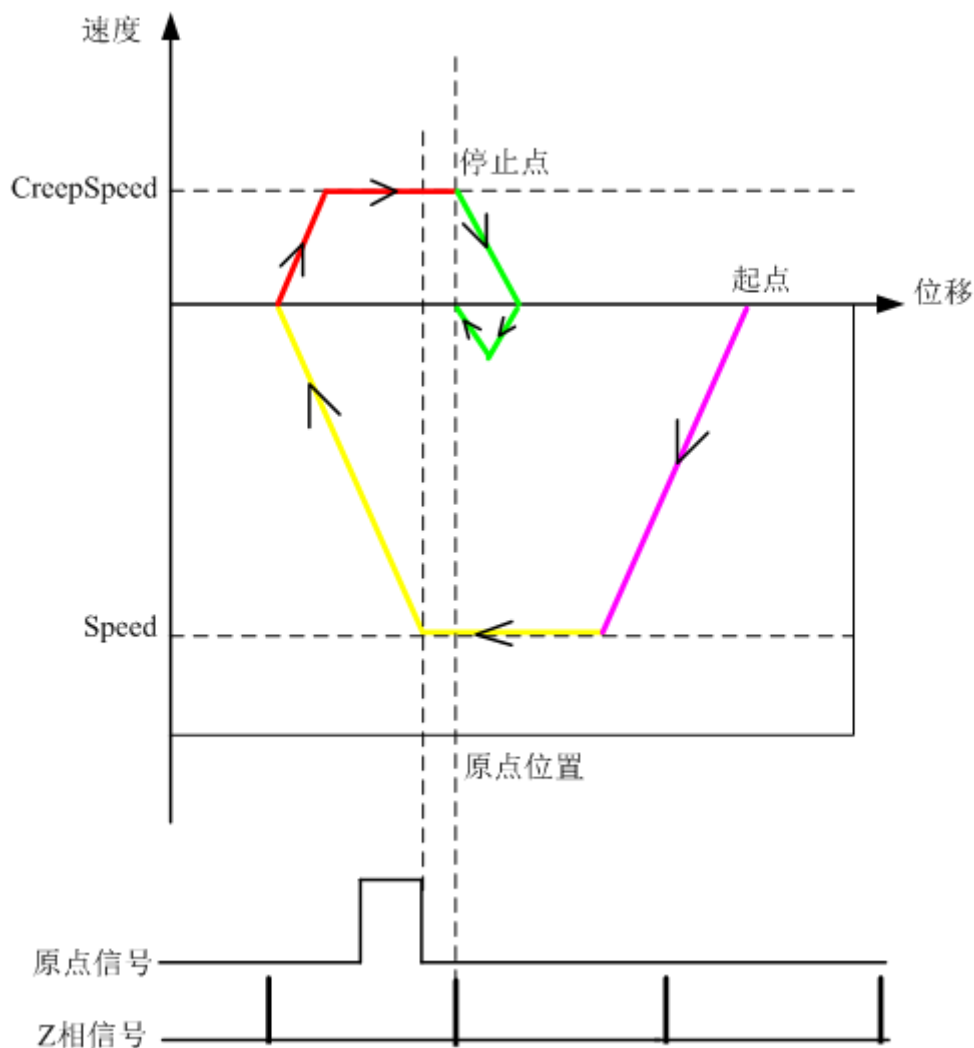
Axis0.Decel := 2000; (*设定减速度*)

HMC_Home(Axis0.AxisID, 4, 0); (*向 Axis0 轴投送原点回归指令(模式 4，使用 0 号快速 DI 及本轴 Z 相通道)*)

(6) 模式 5 原点回归 (ucHomeMode=5)

使用 DI 及本轴 Z 相信号负向定位原点。轴以 Speed 设定的速度负向运动（阶段 1），当遇到 DI 上升沿时减速停止（阶段 2），然后以 CreepSpeed 设定的速度反向运动（阶段 3），遇到 DI 下降沿后且遇到 Z 相为 1 时，设定此处为原点位置并开始减速停止，然后再返回到原点位置（阶段 4）。

特殊情形：同模式 4。



模式 5 原点回归

(紫色、黄色、红色、绿色 对应回归阶段 1、2、3、4)

示例程序如下 (ST 语言)：

Axis0.Speed := 1000; (*设定原点回归速度(阶段 1 速度)*)

Axis0.CreepSpeed := 100; (*设定爬行速度(阶段 3 速度)*)

Axis0.Accel := 2000; (*设定加速度*)

Axis0.Decel := 2000; (*设定减速度*)

HMC_Home(Axis0.AxisID, 5, 0); (*向 Axis0 轴投送原点回归指令(模式 5, 使用 0 号快速 DI 及本轴 Z 相通道)*)

3.4.1.3.1.2 HMC_HomeEx (高级原点回归)

函数原型

UDINT HMC_HomeEx(USINT ucAxisID,

USINT ucHomeMode,

USINT ucDiChI,

USINT ucCaptureChI)

功能

通过检测原点开关信号或伺服电机编码器 Z 相信号为该轴确定原点，原点回归方式由参数 ucHomeMode 确定。原点开关信号的获取使用的是控制器的某个快速 DI 通道，Z 相信号从原点回归轴 DB9 接口的 Z 相接口获取。

与 HMC_Home 的不同在于：该指令使用高速捕获通道来捕获原点位置，原点精确位置的获得通过硬件完成，消除了原点信号触发时刻与原点回归程序处理时刻的延时所引入的偏差，因此更加精准。

参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucHomeMode	USINT	0: 仅使用 DI 信号正向定位原点 1: 仅使用 DI 信号负向定位原点 2: 仅使用本轴 Z 相信号正向定位原点 3: 仅使用本轴 Z 相信号负向定位原点 4: 使用 DI 及本轴 Z 相信号正向定位原点 5: 使用 DI 及本轴 Z 相信号负向定位原点 注：详见应用示例。
ucDiChI	USINT	原点回归时使用的快速 DI 通道号 MC1008&MC1004&MC1002R&MC1002E(0~11) MC1004L (0~3)
ucCaptureChI	USINT	原点回归时使用的高速捕获通道号

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

注意事项

(1) 对于 RTEX 和 EtherCAT 总线轴类型，由于总线类型的驱动器再生脉冲口未提供 Z 相信号输出，所以涉及到 Z 相信号的原点回归功能不可用该指令。

应用举例

参见 [HMC_Home（原点回归）](#) 示例。

3.4.1.3.1.3 HMC_DefOrigin（设置原点）

函数原型

UDINT HMC_DefOrigin(USINT ucAxisID, LREAL dMpos)

功能

移动坐标系，改变某轴坐标系的原点，将参数 dMpos 所标定的原坐标系下的位置设置为新的坐标系原点。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dMpos	LREAL	将会被定义为原点的位置（当前坐标系下）

返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令。

应用举例

HMC_DefOrigin(Axis0.AxisID,1000); (*将轴 Axis0 坐标为 1000 的位置设定为新的坐标原点*)

3.4.1.3.1.4 HMC_DefPos（设置当前位置）

函数原型

UDINT HMC_DefPos (USINT ucAxisID, LREAL dMpos)

功能

移动坐标系，将当前位置定义为 dMpos。

在轴高速运动的时候调用该函数，由于控制器处理时序的问题可能会产生偏差，此时要确保原点位置定义精确，可采用 HMC_DefOrigin 指令实现。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dMpos	LREAL	当前位置指定值

返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令。

应用举例

(1) 示例 1

HMC_DefPos(Axis0.AxisID,0); (*将轴 Axis0 当前坐标位置设定为坐标原点*)

(2) 示例 2

HMC_DefPos(Axis0.AxisID,1000); (*将轴 Axis0 当前坐标位置设定为 1000*)

3.4.1.3.2 单向运动

3.4.1.3.2.1 HMC_Forward（正向运动）

函数原型

UDINT HMC_Forward(USINT ucAxisID)

功能说明

持续正向运动，运动速度由轴参数 **Speed** 决定，加减速由轴参数 **Accel**、**Decel** 确定。在运动过程中，可以实时修正 **Speed**、**Accel**、**Decel** 参数，实时生效。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

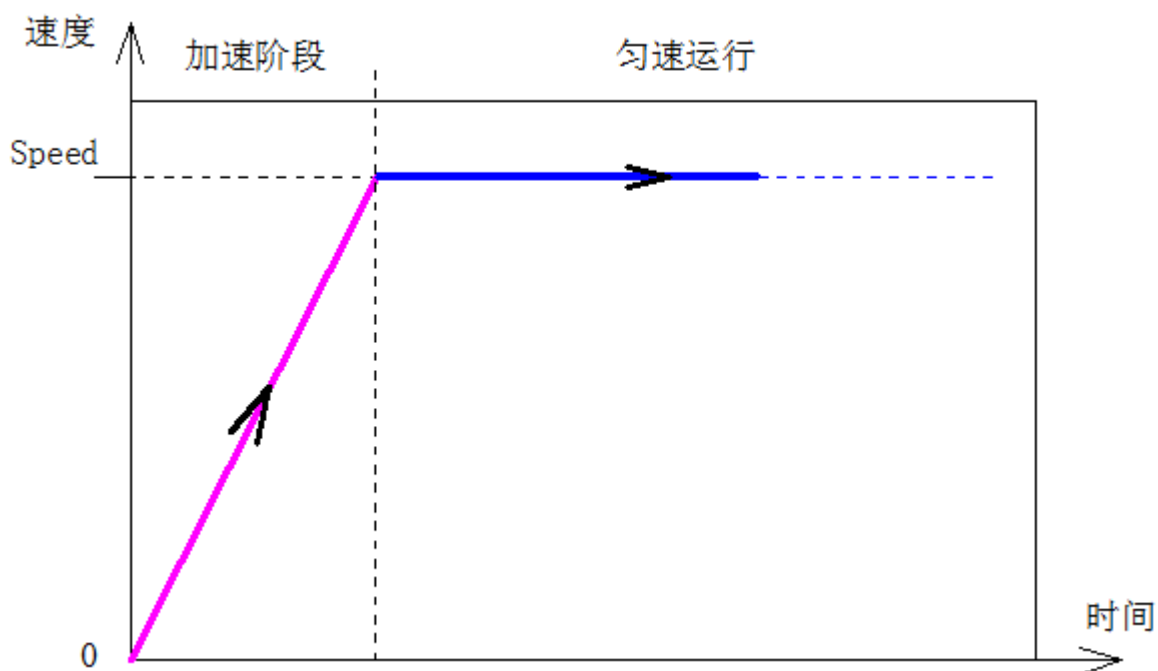
1. 补充说明

运动过程中如果需要撤消停止，可使用 **HMC_Cancel** 指令；

运动过程中若遇到正限位，将减速停止，参见限位时的处理。

应用举例

示例 1：



HMC_Forward

示例程序如下（ST 语言），运行过程的速度-时间曲线如上图所示：

```
Axis0.Speed := 1000;    (*设定运动速度*)
Axis0.Accel := 2000;    (*设定加速度*)
Axis0.Decel := 2000;    (*设定减速度*)
HMC_Forward(Axis0.AxisID); (*向 Axis0 轴投送 HMC_Forward 指令*)
```

示例 2：实时调整速度、加速度，撤消停止

示例程序如下（ST 语言）：

```
Axis0.Speed := 2000;    (*设定运动速度*)
Axis0.Accel := 1000;    (*设定加速度*)
Axis0.Decel := 2000;    (*设定减速度*)

HMC_Forward(Axis0.AxisID); (*向 Axis0 轴投送 HMC_Forward 指令*)
TimeDelay(500);          (*延时 500 毫秒*)

Axis0.Accel := 2000;    (*改变加速度，实时生效*)
TimeDelay(1500);        (*延时 1500 毫秒*)
```

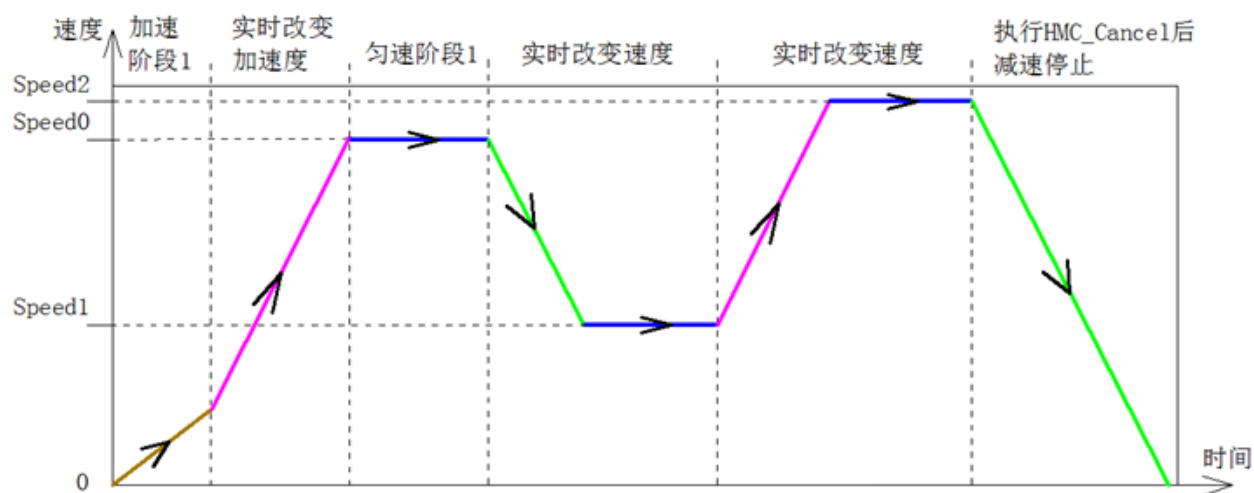
Axis0.Speed := 1000; (*降低速度, 实时生效*)

TimeDelay(1500); (*延时 1500 毫秒*)

Axis0.Speed := 2500; (*增大速度, 实时生效*)

TimeDelay(1500); (*延时 1500 毫秒*)

HMC_Cancel(Axis0.AxisID,1); (*撤消 Axis0 轴当前 HMC_Forward 指令*)



HMC_Forward 运动中实时调整速度、加速度

3.4.1.3.2.2 HMC_Reverse (反向运动)

函数原型

UDINT HMC_Reverse(USINT ucAxisID)

功能说明

持续负向运动，运动的速度由轴参数 Speed 决定。在运动过程中，可以实时修正 Speed、Accel、Decel 等参数，实时有效。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

运动过程中如果需要撤消停止，可使用 HMC_Cancel 指令；

运动过程中若遇到负限位，将减速停止，参见限位时的处理。

应用举例

参见 [HMC_Forward（正向运动）](#)。

3.4.1.3.2.3 HMC_ForwardEx（高级正向运动）

函数原型

UDINT HMC_ForwardEx(USINT ucAxisID, LREAL dSpeed)

功能说明

该指令执行时首先修改轴参数 Speed=函数参数 dSpeed，然后以速度 Speed 持续向前运动。运动过程中可以实时修正 Speed、Accel、Decel 等轴参数，改变运动过程，实时有效。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dSpeed	LREAL	执行该运动时的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

运动过程中如果需要撤消停止，可使用 HMC_Cancel 指令；

运动过程中若遇到正限位，将减速停止，参见限位时的处理。

应用举例

示例 1

示例程序如下（ST 语言）：

Axis0.Speed := 1000; (*设定运动速度*)

Axis0.Accel := 2000; (*设定加速度*)

Axis0.Decel := 2000; (*设定减速度*)

HMC_ForwardEx(Axis0.AxisID,2000); (*向 Axis0 轴投送 HMC_Forward 指令,将以速度 2000 运行（而非 1000）*)

示例 2：实时修改速度、加减速

可参见 [HMC_Forward（正向运动）](#) 中示例。

■ 不支持的模块版本

3.4.1.3.2.4 HMC_ReverseEx（高级反向运动）

函数原型

UDINT HMC_ReverseEx(USINT ucAxisID, LREAL dSpeed)

功能说明

该指令执行时首先修改轴参数 Speed=函数参数 dSpeed，然后以速度 Speed 持续负向运动。运动过程中可以实时修正 Speed、Accel、Decel 等轴参数，改变运动过程，实时有效。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dSpeed	LREAL	执行该运动时的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

运动过程中如果需要撤消停止，可使用 HMC_Cancel 指令；

运动过程中若遇到负限位，将减速停止，参见限位时的处理。

应用举例

参见 [HMC_ForwardEx（高级正向运动）](#)。

■ 不支持的模块版本

MC1008-A01

3.4.1.3.3 点位运动

3.4.1.3.3.1 HMC_Move（相对运动）

函数原型

UDINT HMC_Move(USINT ucAxisID, LREAL dDistance)

功能说明

让某个轴按照预先设定的速度及加减速参数运动到相对于本指令开始执行时刻所在位置的相对位移处（正负均可）。运动速度由对应轴参数 Speed 设定，加减速分别由轴参 Accel、Decel 设定。在运动过程中，可以实时修改 Speed、Accel、Decel 等轴参数，立即生效。

HMC_Move 指令是 HMC 运动控制系统中最典型的运动控制函数，可控制单轴独立运动。没有插补或同步关系的轴可通过简单的执行 HMC_Move 进行独立的运动控制。

HMC_Move 的第一个参数是轴号，第二个参数是从当前位置开始该轴要移动的位移，正负代表运动方向。位移单位可通过轴参 Units、Ke、Ks 进行调整，详见相关轴参数。

(1) 在匀加速、匀减速运动情况下，如果设定的目标速度 Speed 能够达到且存在匀速段，则运动过程中的速度与时间曲线分为三段（加速段、匀速段、减速段），呈现为梯形；否则分为两段（加速段、减速段），呈现为三角形。

(2) 指令执行过程中，对于有限行程，对应的轴参有如下关系：

$$\text{Endpos} = \text{Dpos} + \text{Remain}$$

对于循环模式，Move 指令完成过程中，Endpos、Remain、Dpos、Mpos 的关系详见 RepMode、RepDist。

(3) 在位置开环模式下（如步进轴），当满足 Dpos=Endpos、Remain=0 时，指令执行完成。在位置闭环模式下（如伺服轴），判断执行是否完成还可能要看 Mpos 是否到位，详见轴参数 CmdStopMode、StopFETol。

(4) 当 Jerk 模式使能时，指令实际执行的最大加减速会受 Jerk 功能限制，详见 [Jerk（加加速度）](#)。

(5) Merge 功能关闭时，HMC_Move 指令执行完成时，速度为 0。当 Merge 功能打开且满足 Merge 生效条件时，多条指令可以被连接在一起连续不间断执行，以提高效率，详见 [Merge（融合功能模式）](#)。

参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dDistance	LREAL	增量位移（正负均可）

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

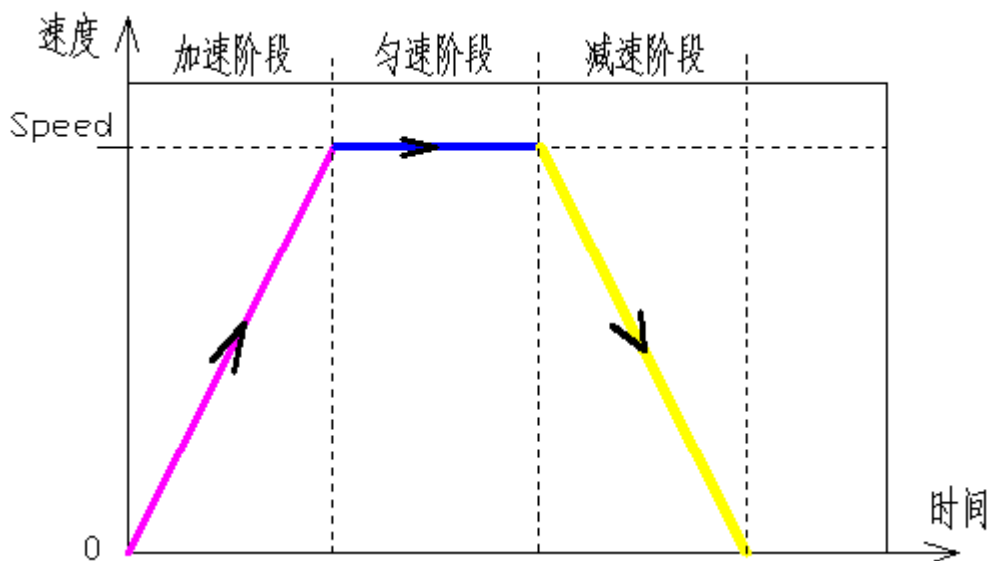
轴运动缓存指令。

应用举例

示例 1：梯形速度曲线

当设定的运动学参数满足下述关系式时，指令执行过程的速度曲线为梯形。

$$\text{dDistance} > \frac{\text{Speed}^2}{2} \left(\frac{1}{\text{Accel}} + \frac{1}{\text{Decel}} \right)$$



梯形加减速曲线

示例程序如下（ST 语言）：

```
Axis0.Speed := 1000;      (*设定运动速度*)
Axis0.Accel := 2000;      (*设定加速度*)
Axis0.Decel := 2000;      (*设定减速度*)
HMC_Move(Axis0.AxisID, 4000); (*向 Axis0 轴投送 HMC_Move 指令,运动增量距离 4000*)
```

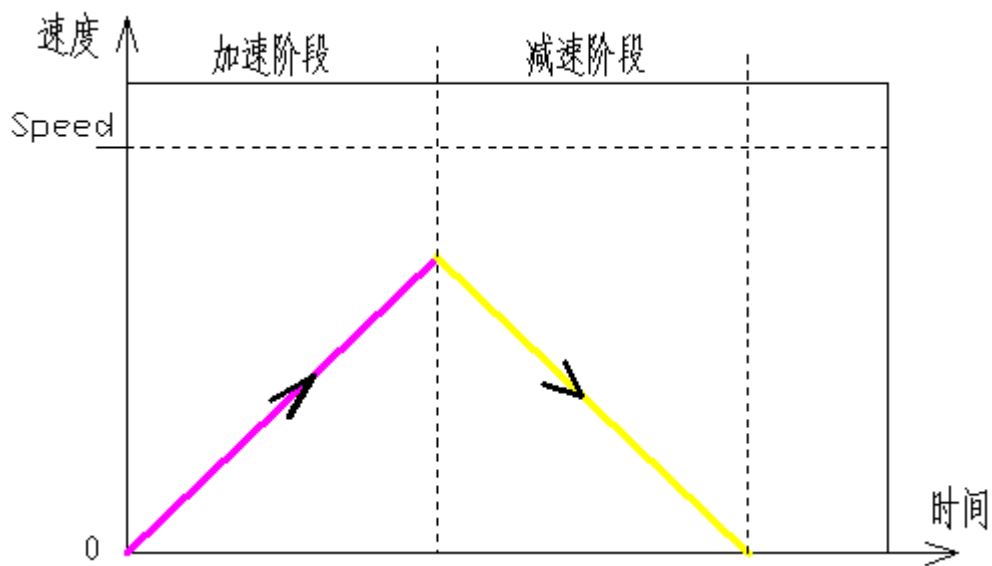
示例 2：三角形速度曲线

当设定的运动学参数满足下述关系式时，指令执行过程的速度曲线为三角形。

$$dDistance \leq \frac{Speed^2}{2} \left(\frac{1}{Accel} + \frac{1}{Decel} \right)$$

示例程序如下（ST 语言）：

```
Axis0.Speed := 2000;      (*设定运动速度*)
Axis0.Accel := 1000;      (*设定加速度*)
Axis0.Decel := 1000;      (*设定减速度*)
HMC_Move(Axis0.AxisID, 1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 1000*)
```

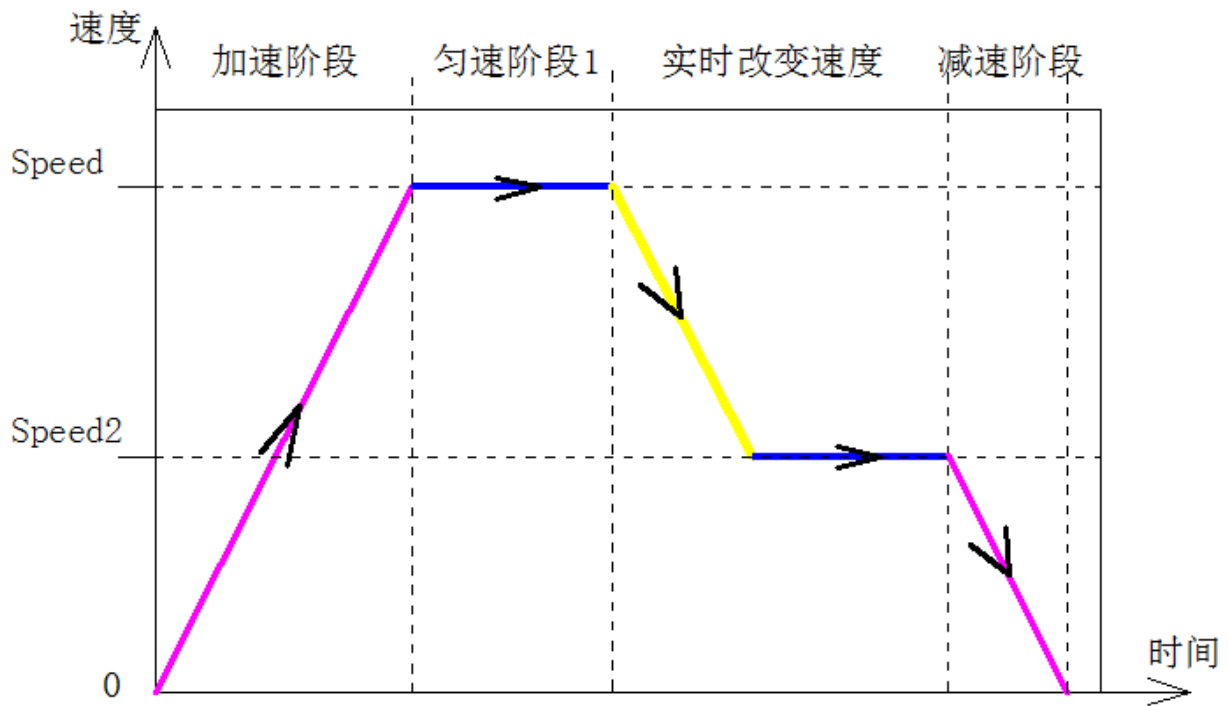


三角形加减速曲线

示例 3：运动中实时修改参数

在运动过程中，可以实时修改 Speed、Accel、Decel 等轴参数，立即生效。程序如下（ST 语言）：

```
Axis0.Speed := 1000; (*设定运动速度*)
Axis0.Accel := 1000; (*设定加速度*)
Axis0.Decel := 1000; (*设定减速度*)
HMC_Move(Axis0.AxisID, 3000); (*向 0 号轴投送 HMC_Move 指令*)
TimeDelay(2000); (*延时等待 2000 毫秒*)
Axis0.Speed := 500; (*速度降为 500*)
```



运动中实时修改速度参数

3.4.1.3.3.2 HMC_MoveAbs (绝对运动)

函数原型

UDINT HMC_MoveAbs(USINT ucAxisID, LREAL dDistance)

功能说明

让某个轴按照预先设定的加速度、速度及减速度参数运动到坐标位置 dDistance 处。运动速度由对应轴参数 Speed 设定，加减速分别由轴参 Accel、Decel 设定。在运动过程中，可以实时修正 Speed、Accel、Decel 等轴参数，实时生效。

相比而言，HMC_MoveAbs 在绝对坐标下使用较为方便，HMC_Move 适合在增量坐标下使用。

详细内容可参考 [HMC_Move \(相对运动\)](#) 中功能说明部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dDistance	LREAL	目标位置

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

应用举例

这里重点举例说明 HMC_MoveAbs 与 HMC_Move 的区别，其它方面的示例可参见 HMC_Move。

示例 1：使轴 0 依次运动 1000、3000、-1000、2000 的增量位移，采用 HMC_Move 与 HMC_MoveAbs 两种方式实现的程序如下（ST 语言）：

HMC_Move 实现程序：

```
Axis0.Speed := 1000;      (*设定运动速度*)
```

```
Axis0.Accel := 2000;      (*设定加速度*)
```

```
Axis0.Decel := 2000;      (*设定减速度*)
```

```
HMC_Move(Axis0.AxisID, 1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 1000*)
```

```
HMC_Move(Axis0.AxisID, 3000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 3000*)
```

```
HMC_Move(Axis0.AxisID, -1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离-1000*)
```

```
HMC_Move(Axis0.AxisID, 2000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 2000*)
```

HMC_MoveAbs 实现程序（假设起点为 0 点）：

```
Axis0.Speed := 1000;      (*设定运动速度*)
```

```
Axis0.Accel := 2000;      (*设定加速度*)
```

```
Axis0.Decel := 2000;      (*设定减速度*)
```

```
HMC_MoveAbs (Axis0.AxisID, 0 + 1000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动至绝对位置 1000 处*)
```

```
HMC_MoveAbs (Axis0.AxisID, 0 + 1000 + 3000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动绝对位置 4000 处*)
```

```
HMC_MoveAbs (Axis0.AxisID, 0 + 1000 + 3000 + (-1000)); (*向 0 号轴投送 HMC_MoveAbs 指令,运动绝对位置 3000 处*)
```

```
HMC_MoveAbs (Axis0.AxisID, 0 + 1000 + 3000 + (-1000) + 2000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动绝对位置 5000 处*)
```

示例 2：使轴 0 依次运行至绝对位置坐标 1000、3000、-1000、2000，分别采用 HMC_Move 与 HMC_MoveAbs 两种方式实现的程序如下（ST 语言）：

HMC_Move 实现程序(假设起点为 0 点)：

```
Axis0.Speed := 1000;      (*设定运动速度*)
```

```
Axis0.Accel := 2000;      (*设定加速度*)
```

```
Axis0.Decel := 2000;      (*设定减速度*)
```

HMC_Move(Axis0.AxisID, 1000 - 0); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 1000*)
HMC_Move(Axis0.AxisID, 3000 - 1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 2000*)
HMC_Move(Axis0.AxisID, - 1000 - 3000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离-1000*)
HMC_Move(Axis0.AxisID, 2000 - (-1000)); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 2000*)

HMC_MoveAbs 实现程序:

Axis0.Speed := 1000; (*设定运动速度*)
Axis0.Accel := 2000; (*设定加速度*)
Axis0.Decel := 2000; (*设定减速度*)

HMC_MoveAbs (Axis0.AxisID, 1000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动至绝对位置 1000 处*)
HMC_MoveAbs (Axis0.AxisID, 3000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动绝对位置 3000 处*)
HMC_MoveAbs (Axis0.AxisID, -1000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动绝对位置-1000 处*)
HMC_MoveAbs (Axis0.AxisID, 2000); (*向 0 号轴投送 HMC_MoveAbs 指令,运动绝对位置 2000 处*)

通过上述例子可见，在增量坐标下使用 HMC_Move 较为适合，在绝对坐标下使用 HMC_MoveAbs 则较为方便。

3.4.1.3.3.3 HMC_MoveEx（高级相对运动）

函数原型

UDINT HMC_MoveEx(USINT ucAxisID, LREAL dDistance, LREAL dSpeed)

功能说明

该指令执行时首先修改轴参数 Speed=函数参数 dSpeed，然后以速度 Speed 运动到相对于本指令开始执行时刻所在位置的相对位移处（正负均可）。在运动过程中，可以实时修正 Speed、Accel、Decel 等轴参数，改变运动过程，实时有效。

详细内容可参考 [HMC_Move（相对运动）](#) 中功能说明部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dDistance	LREAL	位移（正负均可）

输入参数	类型	参数描述
dSpeed	LREAL	执行该运动时的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

应用举例

示例程序如下（ST 语言），其余示例可参见 HMC_Move。

Axis0.Speed := 1000; (*设定运动速度*)

Axis0.Accel := 2000; (*设定加速度*)

Axis0.Decel := 2000; (*设定减速度*)

HMC_MoveEx(Axis0.AxisID, 10000,2000); (*向 0 号轴投送 HMC_Move 指令,将以最大速度 2000 运行（而非 1000）*)

■ 不支持的模块版本

MC1008-A01

3.4.1.3.3.4 HMC_MoveAbsEx（高级绝对运动）

函数原型

UDINT HMC_MoveAbsEx(USINT ucAxisID, LREAL dDistance, LREAL dSpeed)

功能说明

轴参数 Speed=函数参数 dSpeed，让某个轴按照本指令设定的目标速度运动到绝对位置。在运动过程中，可以实时修正 Speed、Accel、Decel 等轴参数，改变运动过程，实时有效。

详细内容可参考 [HMC_MoveAbs（绝对运动）](#) 中功能说明部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dDistance	LREAL	目标位置
dSpeed	LREAL	执行该运动时的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

应用举例

参见 [HMC_MoveAbs（绝对运动）](#)。

- 不支持的模块版本

MC1008-A01

3.4.1.3.4 指令修正

3.4.1.3.4.1 HMC_Cancel（撤销指令）

函数原型

UDINT HMC_Cancel(USINT ucAxisID, USINT ucCancelMode)

功能说明

撤销指定轴运动缓存中的指令，支持撤销当前指令、撤销下一条指令、撤销该轴所有指令三种模式。

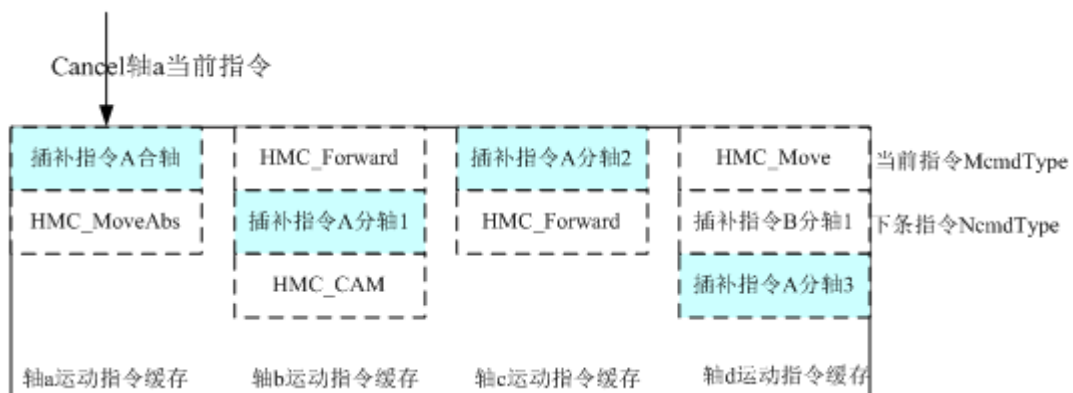
(1) 不同指令类型 **Cancel** 响应方式

(a) 单轴指令

撤销该轴当前或所有指令。

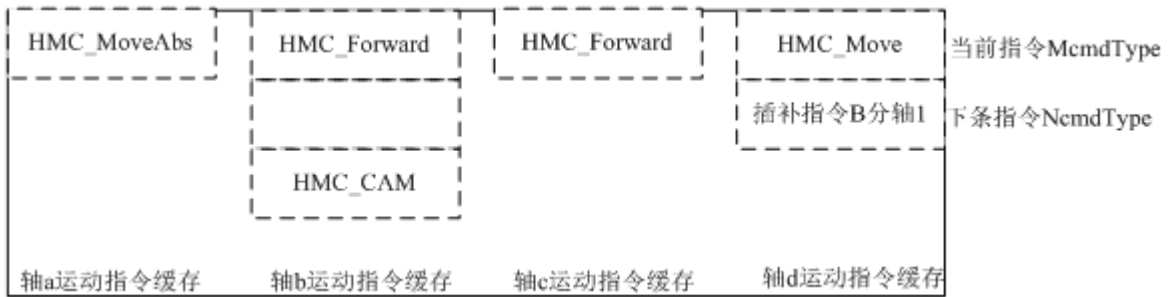
(b) 插补合轴

撤销插补合轴指令时，会撤销与该插补指令相关的所有轴的该插补指令，即包括合轴、分轴，且无论该指令在各插补分轴是否为当前指令。如当前各轴的运动指令缓存状态如下，现欲对轴 **a** 撤销当前指令。



运动指令缓存状态示意图

则调用 **Cancel** 指令后，轴 **a** 立即响应，无论其从轴的该指令是否处于当前指令。待 **Cancel** 执行完成后，各轴的运动指令缓存状态如下：



运动指令缓存状态示意图

注意，**Cancel** 插补指令后，会直接删除合轴、分轴的该指令缓存为空，但此时不会调整后续指令上移，而是等待执行到此指令时，判断为空则直接执行后续指令，对实际执行效率无影响。

（c）插补分轴

Cancel 指令对插补分轴无效。

（d）联动主轴

对于联动指令主轴执行 **Cancel**，仅对主轴生效，对从轴无影响。

（e）联动从轴

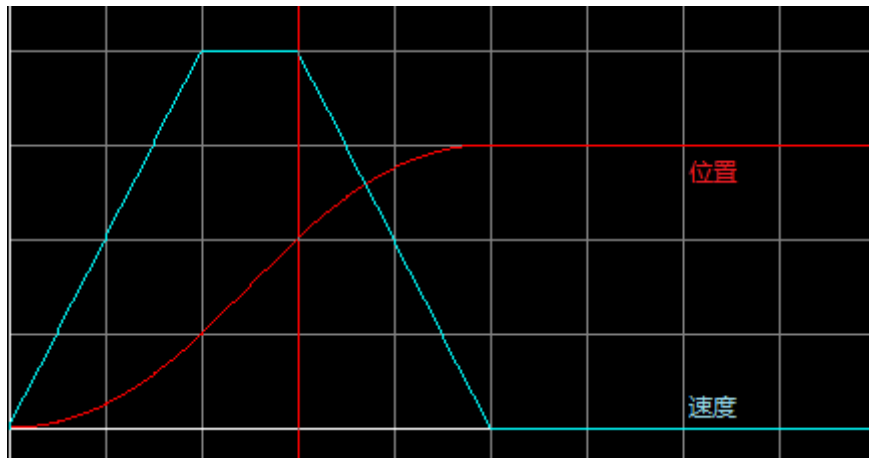
对于联动指令从轴执行 **Cancel**，仅对从轴生效，对主轴无影响。

（2）指令撤销时轴的停止方式

指令撤销过程中，在接受到 **Cancel** 指令后，轴会以当前速度开始进行减速，直到停止。但该减速停止过程中具体停止方式，如实际减速度、最终停止时刻位置等可由轴参数 **CancelMode**、**CancelPos** 指定。具体如下：

（a）**CancelMode=0** 自然停止

默认值，按照 **Decel** 自然停止。此模式支持所有指令类型。



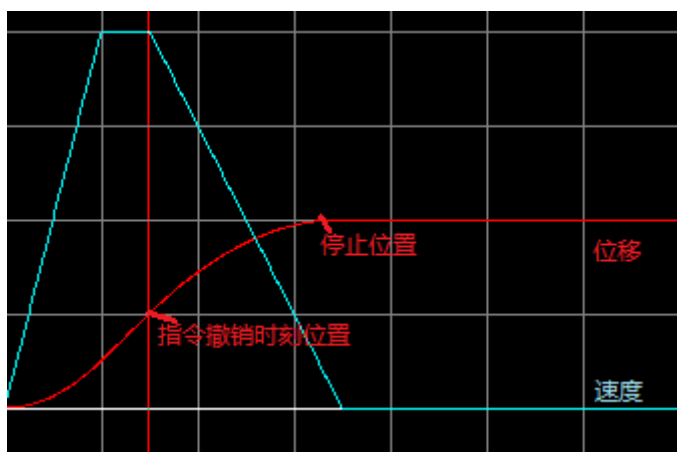
运动轨迹示意图

（b）**CancelMode=1** 指定位置停止

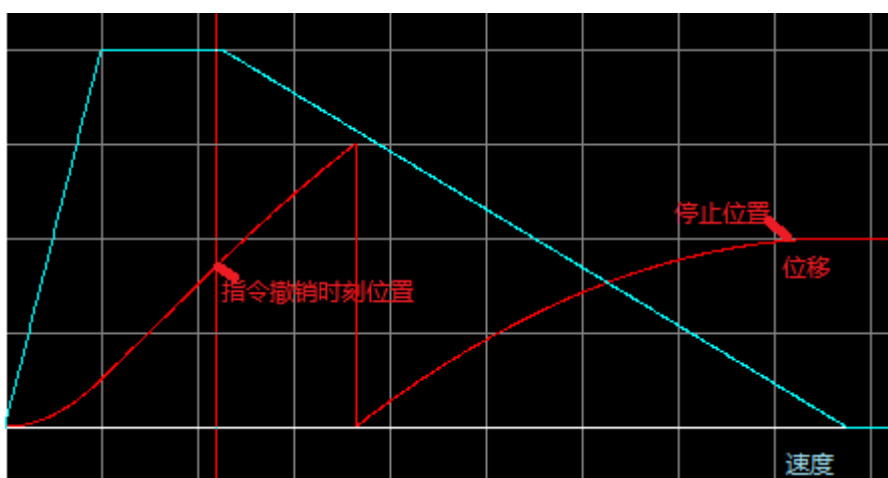
按照不超过减速度 **Decel** 停到指定位置，该位置由参数 **CancelPos** 指定。此模式仅对循环行程模式

(RepMode=1 或 2)且当前指令为单向运动（HMC_Forward、HMC_Reverse）的情况生效。若单向运动指令，但不满足循环行程模式，则按照 CancelMode=0 处理。

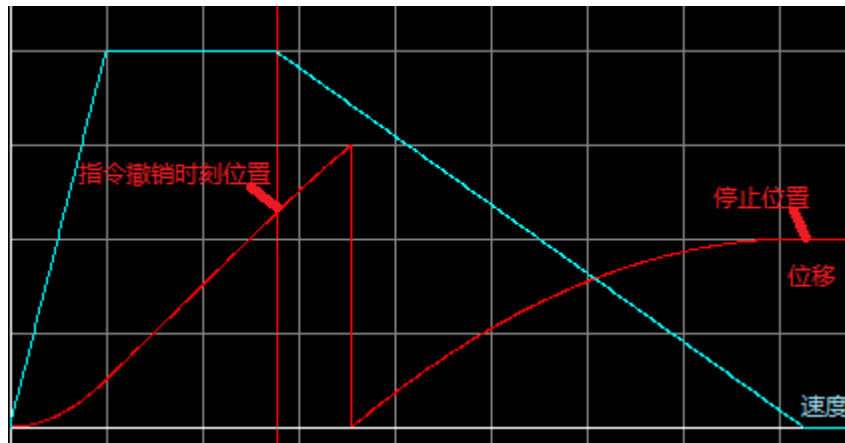
根据当前位置和 CancelPos 的相对关系，会有三种情况：1、当前位置<停止位置，且两者距离位移差足够当前速度以减速度停止为 0；2、当前位置<停止位置，但两者距离位移差不够当前速度以减速度停止为 0；3、当前位置>停止位置。以上三种情况分别对应如下三图：



运动轨迹示意图



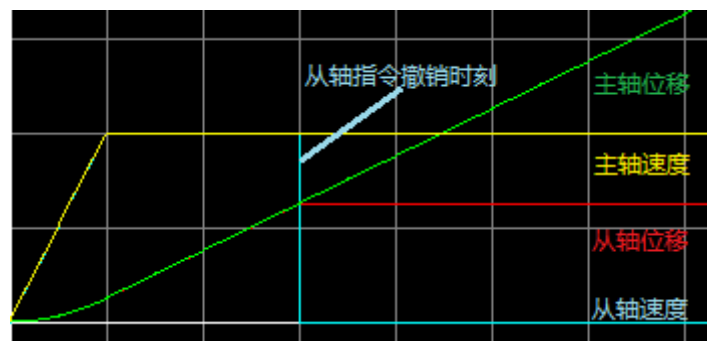
运动轨迹示意图



运动轨迹示意图

(c) CancelMode=2 立即停止

立即停止，速度从当前速度直接减速为 0。此模式仅对正在执行联动指令的轴生效，若状态不满足，则默认按照 CancelMode=0 处理。



运动轨迹示意图

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucCancelMode	USINT	0: 撤销下一条指令 1: 撤销当前指令 2: 撤销该轴所有指令

返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令

应用举例

轴 0 单向正向运动 10s 之后，以 Decel 减速停止。

```
Axis0.Speed := 100000;
```

```
Axis0.Accel :=100000;
```

```
Axis0.Decel :=100000;
Axis0.CancelMode:=0;
HMC_Forward(0);
TimeDelay (10000);
HMC_Cancel(0, 1);
```

3.4.1.3.4.2 HMC_MoveModify (目标位置变更)

函数原型

UDINT HMC_MoveModify (USINT ucAxisID, LREAL dDpos)

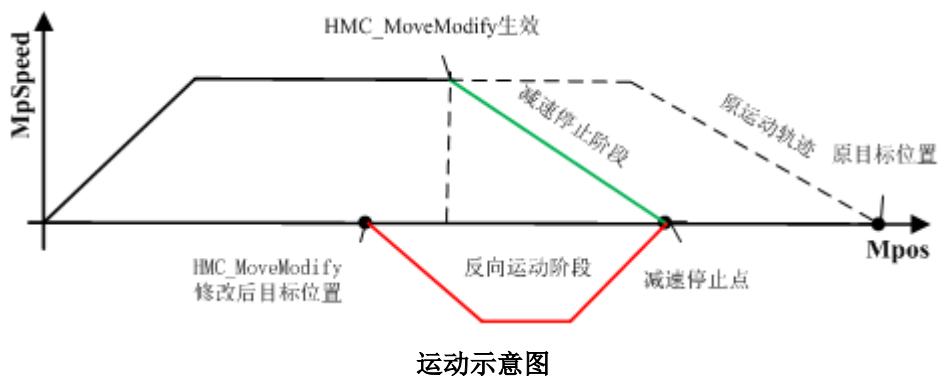
功能说明

实时修改 HMC_Move/HMC_MoveAbs 指令的目标位置，修改后的目标位置为 dDpos。

执行 HMC_MoveModify 时，若当前轴指令队列中没有指令，则直接投送 MoveAbs 指令；若有指令但当前正在执行的指令不是 Move 或 MoveAbs，则忽略本次修改请求。

在执行 HMC_MoveModify 时，存在以下几种处理方式：

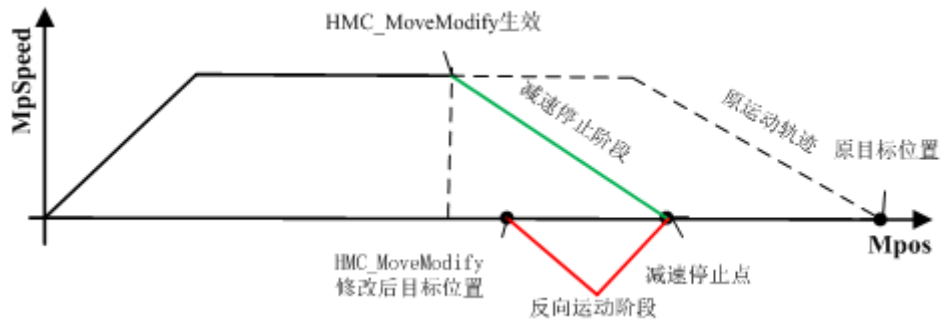
(1) 若轴的运动方向是远离 HMC_MoveModify 所指定的终点位置的方向，则该轴将减速直至停止，然后反向运动到 HMC_MoveModify 所设定的位置。



示例程序：

```
p1 := HMC_Move(0, 10000);
p2:= TimeDelay(15); (* 此时的位置值 axis0.dpos 已经超过 5000 并继续远离 5000 *)
p3:= HMC_MoveModify(0, 5000)
```

(2) 若轴的运动方向是朝向 HMC_MoveModify 所指定的终点位置的方向，但没有足够的减速距离，则当前轴将减速停止，然后反向运动到 HMC_MoveModify 所设定的位置。



运动示意图

示例程序:

```
p1 := HMC_Move(0, 10000);
```

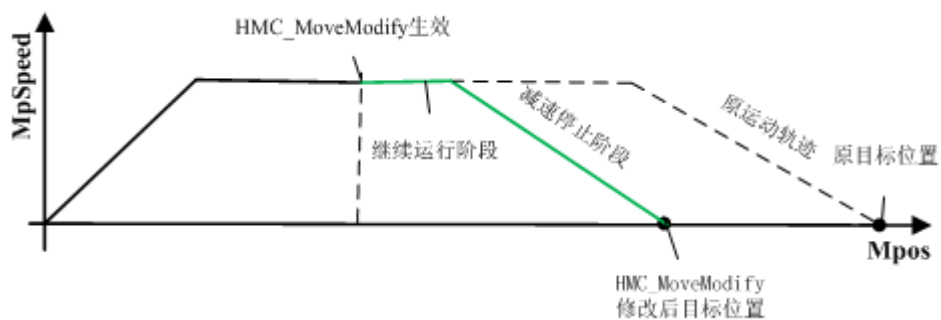
```
p2 := TimeDelay(10); (* 此时的位置值 axis0.dpos 已接近 6000，没有足够的减速距离使其减速停止 *)
```

```
p3 := HMC_MoveModify(0, 6100)
```

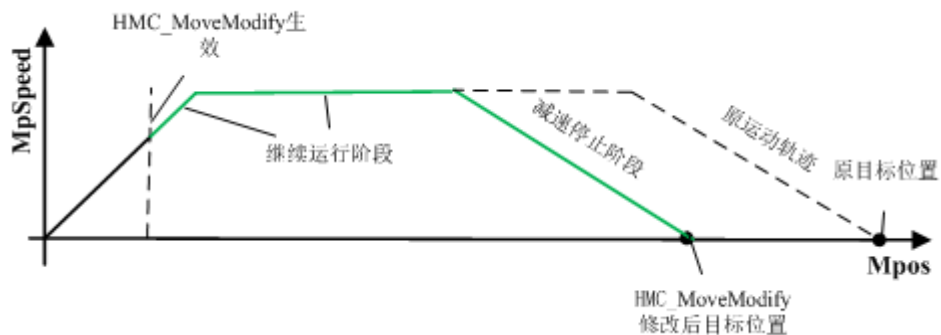
(3) 若轴的运动方向是朝向 HMC_MoveModify 所指定的终点位置的方向，且有足够的减速距离，则当前轴将继续运动至减速点，然后减速停止到 HMC_MoveModify 所设定的位置。

由于 HMC_MoveModifyEx 执行时机的不同以及修改后的终点位置的不同，“继续运行阶段”的运动过程不尽相同，并且匀速阶段不一定存在。以下列出几种典型的运动过程示意图：

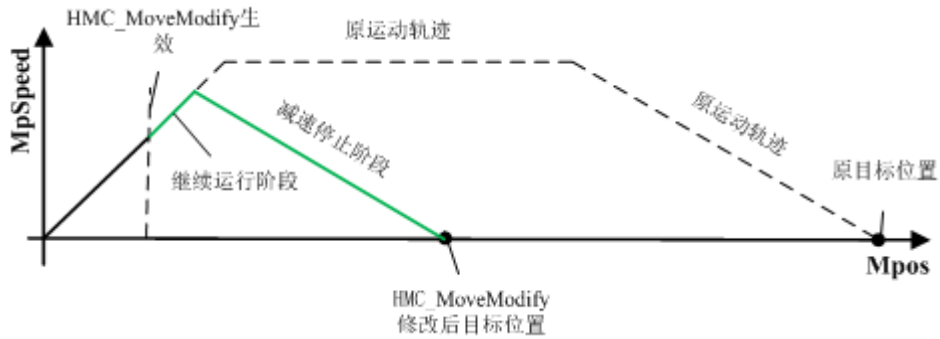
(a) HMC_MoveModify 所指定的终点位置在原目标位置之前。



设定位置在原目标位置之前

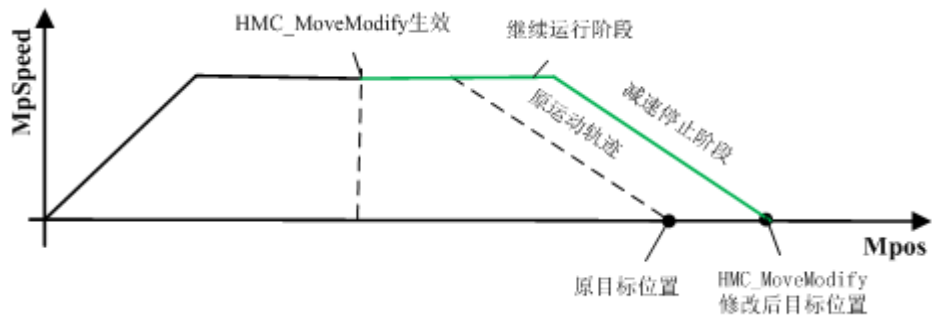


设定位置在原目标位置之前（加速阶段执行 HMC_MoveModify，存在匀速段）

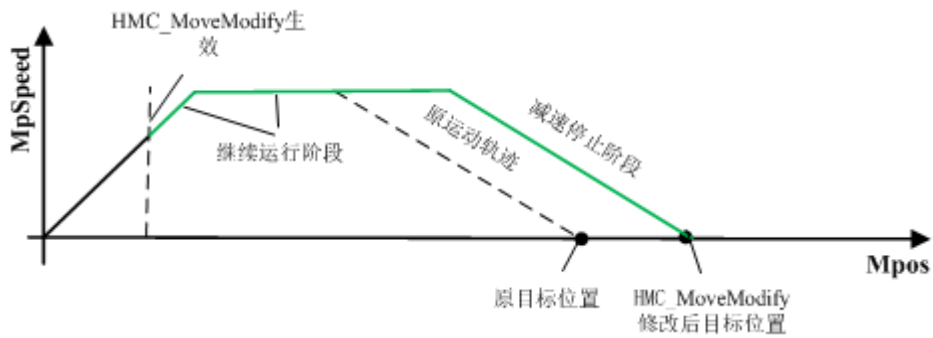


设定位置在原目标位置之前（加速阶段执行 **HMC_MoveModify**，无匀速段）

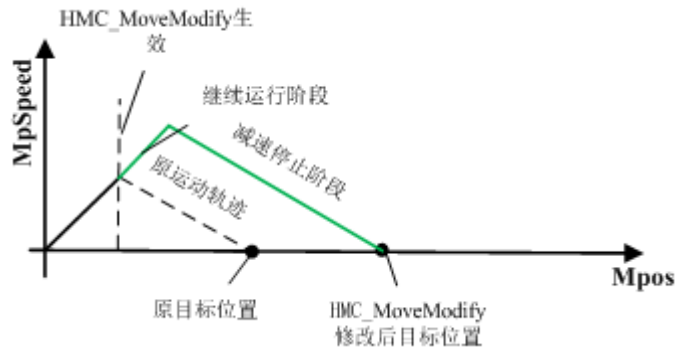
(b) **HMC_MoveModify** 所指定的终点位置在原目标位置之后。



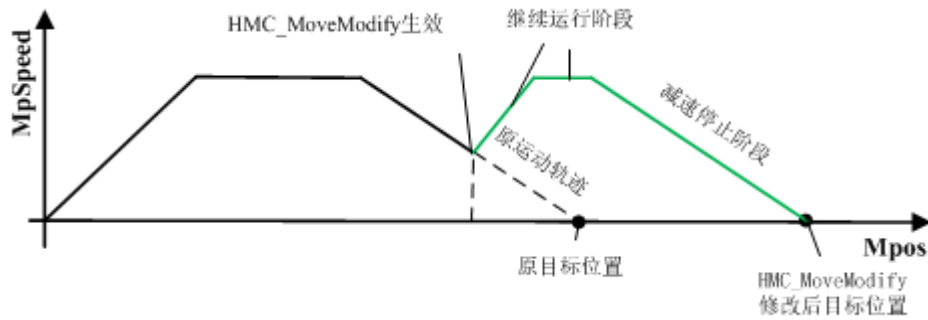
设定位置在原目标位置之后（匀速阶段执行 **HMC_MoveModify**）



设定位置在原目标位置之后（加速阶段执行 **HMC_MoveModify**，存在匀速段）



设定位置在原目标位置之前（加速阶段执行 **HMC_MoveModify**，无匀速段）



设定位置在原目标位置之后（减速阶段执行 HMC_MoveModify）

示例程序：

```
p1 := HMC_Move(0, 10000);
```

```
p2 := TimeDelay(2); (* 此时的位置值 axis0.dpos 远小于 5000，有足够的剩余减速距离 *)
```

```
p3 := HMC_MoveModify(0, 5000)
```

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dDpos	LREAL	修改当前指令的位移

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	当前指令不是 Move 或 MoveAbs，忽略本次修改申请
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

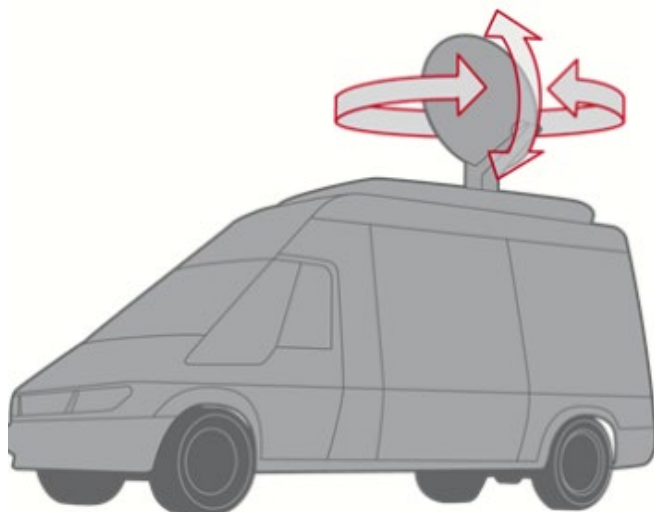
■ 补充说明

若当前 Merge 打开，则在投送 HMC_MoveModify 指令时将会自动关闭 Merge。

应用举例

卫星信号接收车。

在进行野外测控火箭发射时的运行状态和接收卫星信号时，车载卫星接收器要根据上位机电脑系统发过来的数据，对接收天线的水平方向和上下俯仰方向进行实时的调整，以准确跟踪测控目标的状态。那么我们控制器中的 HMC_MoveModify 函数能很好的满足这个需求。可在此指令执行过程中不断加载修正目标位置，完成随动追踪任务。



卫星信号接收车示意图

设上位机电脑系统发送过来的水平矫正位置为 $g_rHorizontalAdjPos$ ，垂直俯仰矫正位置为 $g_rVerticalAdjPos$ ，假设水平方向转 360° 指令脉冲为 X ，垂直方向转 360° 指令脉冲数为 Y 。具体程序如下：

(**** 参数设定 ****)

Axis0UnitReval := HMC_SetUnits(0, X/3600); (* 设定 AXIS0 单位为 0.1 度 *)

Axis1UnitReval := HMC_SetUnits(1, Y/3600); (* 设定 AXIS1 单位为 0.1 度 *)

AxisEnableReval := HMC_DriveEnable(1);

axis0.Speed := 100;

axis0.Accel := 1000;

axis0.Decel := 1000;

axis1.Speed := 100;

axis1.Accel := 1000;

axis1.Decel := 1000;

(****后续程序计算，循环扫描****)

WHILE true DO

IF bStart=1 THEN (*设备运行*)

(*矫正水平方向*)

Axis0ModifyReval := HMC_MoveModify(0, $g_rHorizontalAdjPos$);

(*矫正垂直方向*)

```
Axis1ModifyReval := HMC_MoveModify(1, g_rVerticalAdjPos);
```

```
END_IF
```

```
IF bStop=1 THEN (*设备停止*)
```

```
Axis0StopReVal := HMC_Cancel(0, 2);
```

```
Axis1StopReVal := HMC_Cancel(1, 2);
```

```
RESULT := HMC_MoveAbs(0, 0);
```

```
RESULT := HMC_MoveAbs(1, 0);
```

```
RESULT := HMC_WaitAxisIdle(0);
```

```
RESULT := HMC_WaitAxisIdle(1)
```

```
END_IF
```

```
END_WHILE
```

■ 不支持的模块版本

MC1008-A01

3.4.1.3.4.3 HMC_MoveModifyEx (高级目标位置变更)

函数原型

```
UDINT HMC_MoveModifyEx(USINT ucAxisID, LREAL dDpos, USINT ucModifyMode)
```

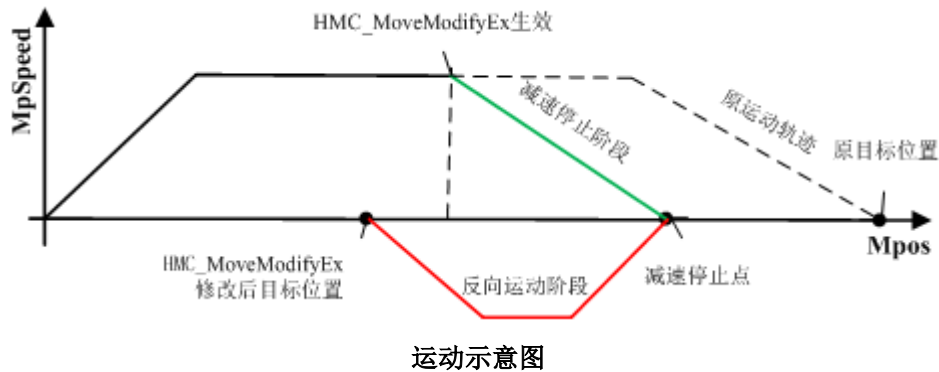
功能说明

实时修改 HMC_Move/HMC_MoveAbs 指令的目标位置，修改方式由 ucModifyMode 决定：当 ucModifyMode=0 时，本指令与 HMC_MoveModify 相同，为绝对位置修改模式，修改后的目标位置为 dDpos；当 ucModifyMode=1 时，为增量位置修改模式，修改后的目标位置为当前 dPos + dDpos。

执行 HMC_MoveModifyEx 时，若当前轴指令队列中没有指令，则 ucModifyMode=0 时投送 HMC_MoveAbs 指令，ucModifyMode=1 时投送 HMC_Move 指令；若当前轴指令队列中有指令，且正在执行的指令是 Move 或 MoveAbs，则修改当前指令的位移；若当前轴有指令，且正在执行的指令不是 Move 或 MoveAbs，则忽略本次修改请求。

执行 HMC_MoveModifyEx 时，存在以下几种处理方式：

- (1) 若轴的运动方向是远离 HMC_MoveModifyEx 所指定的终点位置的方向，则该轴将减速直至停止，然后反向运动到 HMC_MoveModifyEx 所设定的位置。匀速阶段执行 HMC_MoveModify 的运动过程示意图如下。



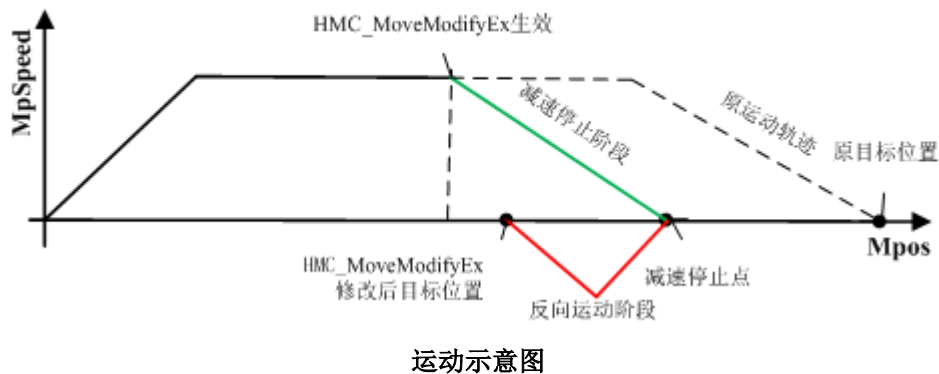
示例程序：

```
p1 := HMC_Move(0, -10000);
```

```
p2 := TimeDelay(10); (* 此时的位置值已经超过 5000 并继续远离 5000 *)
```

```
p3 := HMC_MoveModifyEx(0, 5000, 1);
```

- (2) 若轴的运动方向是朝向 HMC_MoveModifyEx 所指定的终点位置的方向，但没有足够的减速距离，则当前轴将减速停止，然后反向运动到 HMC_MoveModifyEx 所设定的位置。匀速阶段执行 HMC_MoveModify 的运动过程示意图如下。



示例程序：

```
p1 := HMC_Move(0, 10000);
```

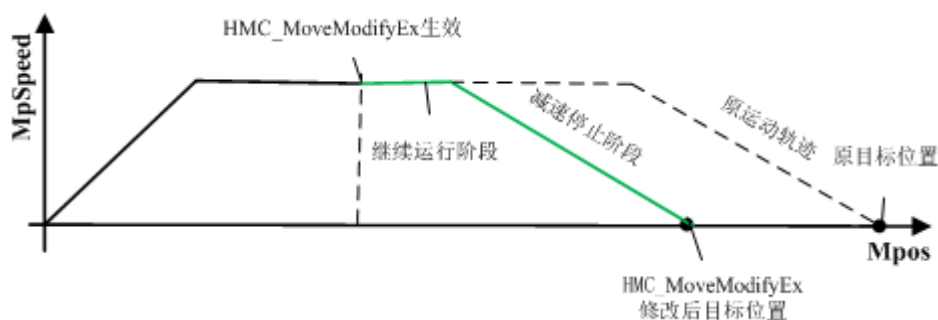
```
p2 := TimeDelay(10);
```

```
p3 := HMC_MoveModifyEx(0,50,1); (* 增量位置值太小，不足以减速 *)
```

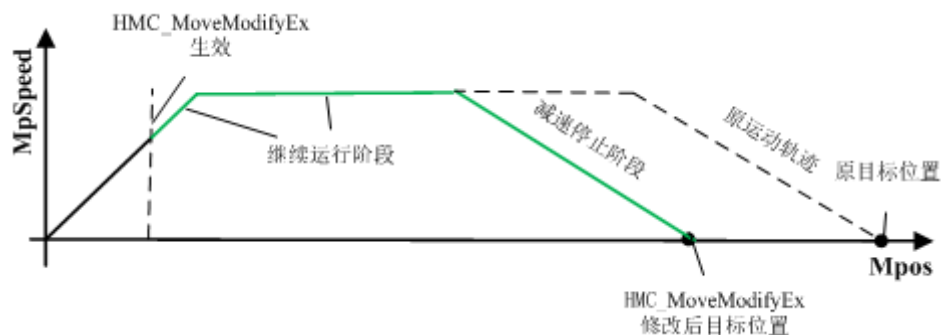
- (3) 若轴的运动方向是朝向 HMC_MoveModifyEx 所指定的终点位置的方向，且有足够的减速距离，则当前轴将继续运动至减速点，然后减速停止到 HMC_MoveModifyEx 所设定的位置。

由于 HMC_MoveModifyEx 执行时机的不同以及修改后的终点位置的不同，“继续运行阶段”的运动过程不尽相同，并且匀速阶段不一定存在。以下列出几种典型的运动过程示意图：

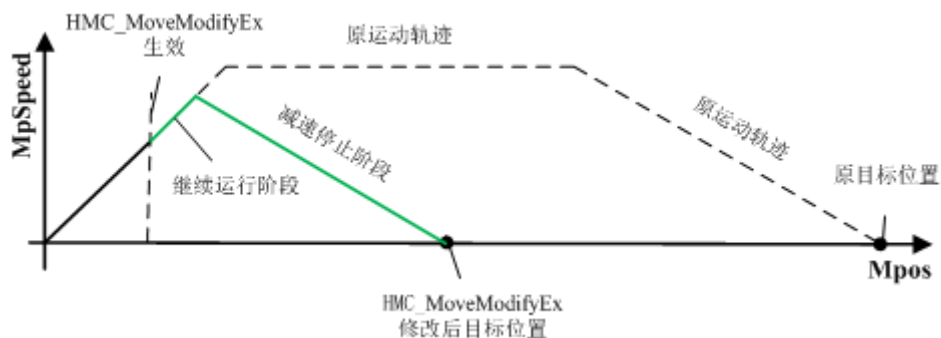
- (a) HMC_MoveModifyEx 所指定的终点位置在原目标位置之前。



设定位置在原目标位置之前（匀速阶段执行 HMC_MoveModifyEx）

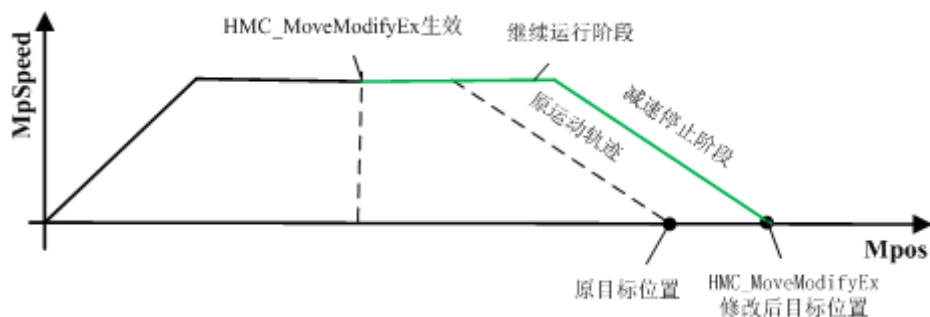


设定位置在原目标位置之前（加速阶段执行 HMC_MoveModifyEx，存在匀速段）

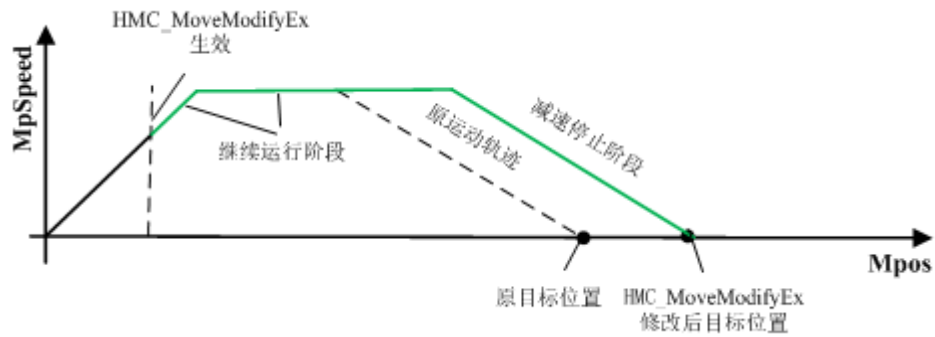


设定位置在原目标位置之前（加速阶段执行 HMC_MoveModifyEx，无匀速段）

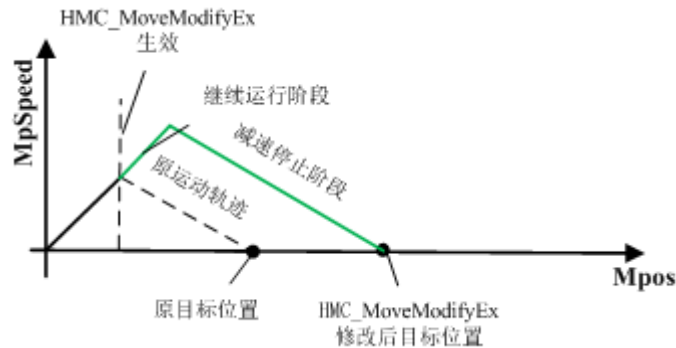
(b) HMC_MoveModifyEx 所指定的终点位置在原目标位置之后。



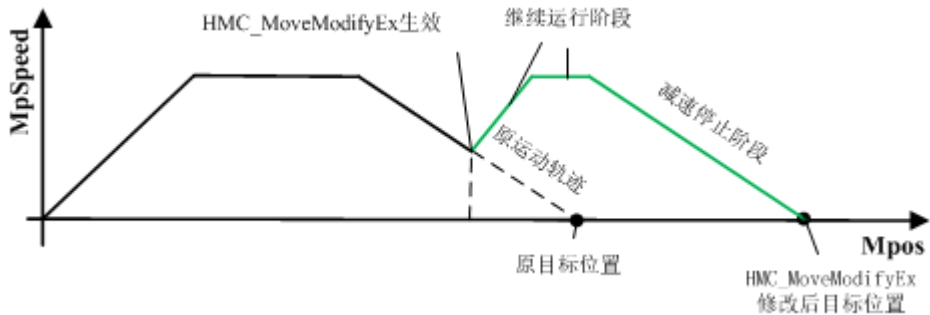
设定位置在原目标位置之后（匀速阶段执行 HMC_MoveModifyEx）



设定位置在原目标位置之后（加速阶段执行 **HMC_MoveModifyEx**，存在匀速段）



设定位置在原目标位置之前（加速阶段执行 **HMC_MoveModifyEx**，无匀速段）



设定位置在原目标位置之后（减速阶段执行 **HMC_MoveModifyEx**）

示例程序：

```
p1 := HMC_Move(0, 10000);
p2 := TimeDelay(2);
p3 := HMC_MoveModifyEx(0, 5000, 1); (* 此时增量位置为 5000，有足够时间减速 *)
```

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	待修改轴轴号
dDpos	LREAL	修改当前指令的位移
ucModifyMode	USINT	0: 绝对位置修改模式 1: 增量位置修改模式

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFFE	UDINT	当前指令不是 Move 或 MoveAbs，忽略本次修改申请
0xFFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

■ 补充说明

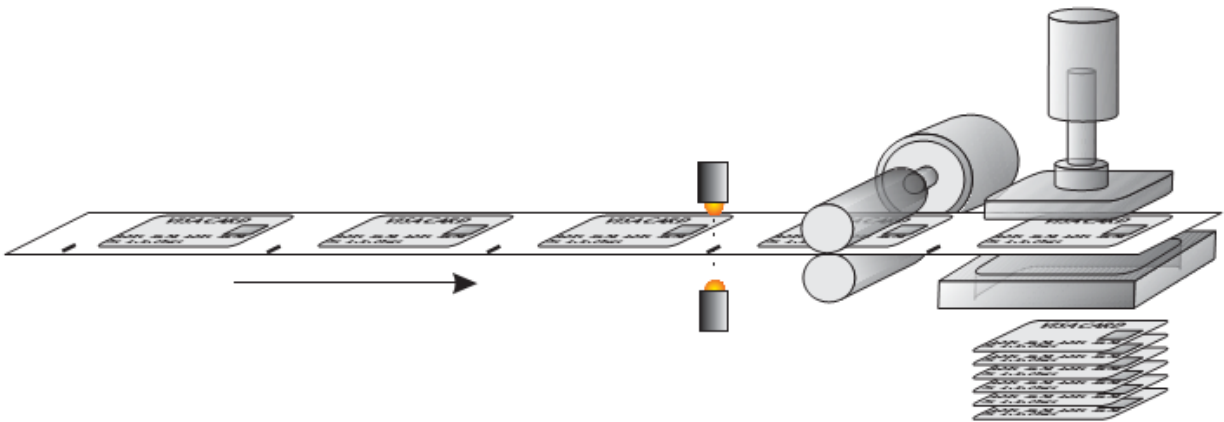
若当前 Merge 打开，则在投送 HMC_MoveModifyEx 指令时自动关闭 Merge。

应用举例

包装机械在运行过程中，由于输送带机械振动将导致材料偏移、传送速度发生变化、包装材料拉伸，加之包装材料上的色标定位之间存在累计误差等因素的影响，将极大影响封切质量。

通过使用高速捕捉功能，配以目标位置变更的修正功能，在每一个色标定位周期进行补偿，可有效抑制累计误差，将误差控制到允许范围内。举例如下。

夹送辊输送一印刷的卡片，卡片在印刷的时候印上了一个色标块（下图中两图案中黑色的方块）。要求把卡片模切下来，模切的位置刚好在卡片中心，不能有累计误差。实现原理：我们在夹送辊的左方加装一个色标传感器，用以检测色标块的到来，当色标块到来时使用高速捕获功能捕获色标块的精确位置，每次行走的距离以色标块的位置为参考偏移行走合适的距离。由于是同一个印刷版印出来的，所以色标和卡的图案位置是固定的。这样每次走过的距离都是以色标的位置作为参考，消除了累计的误差。色标传感器接到快速 DI 通道 0 上，使用高速捕获通道 0 捕获色标位置。



传输示意图

我们假设两卡片图案之间的间距是 300 mm，要模切的位置距离色标的位置是 offset（位置可以调整）。具体程序如下：

```
Start_MPOS := 0.0;
End_MPOS := 300.00;
Offset := 30.0;
```

```
Axis0TypeReval := HMC_SetAxisType(Axis0.AxisID, 10); (*设置轴类型*)
```

```
Axis0UnitReval := HMC_SetUnits(Axis0.AxisID, 1); (*设置用户单位*)
```

```
AxisEnableReval := HMC_DriveEnable(1); (*开启伺服使能*)
```

```
axis0.Speed := 100;
```

```
axis0.Accel := 1000;
```

```
axis0.Decel := 1000;
```

```
WHILE true DO (* 后续程序计算，循环扫描 *)
```

```
IF bStart = 1 THEN (*运行按钮按下*)
```

```
Axis0DefReval := HMC_DefOrigin(Axis0.AxisID, Axis0.MPos); (*当前位置清零*)
```

```
Config_result := HMC_CaptureConfig(0, 0, 1, 0, 0, 1, Start_MPOS,
```

```
End_MPOS); (*配置高速捕获*)
```

```
Axis0moveReval := HMC_Move(0, 300); (*走一个卡片图案间距*)
```

(* 读取高速捕捉通道 0 的状态，等待高速捕获完成。当色标传感器有信号时，高速捕获将会完成，此时可获得色标传感器的精确位置 *)

```
WHILE (HMC_CaptureGetState(0) <> 3) DO
```

```
Reg_pos := HMC_CaptureGetMpos(0);
```

```
END_WHILE
```

```
Axis0ModifyReval := HMC_MoveModifyEx(0,offset,1);
```

(* 通过高速捕获到的精确位置，以色标传感器位置为参考修正位移，保证能准确定位到卡片中心位置。例如捕获到位置为 280，Offset 为 30，则补偿后的绝对目标位置为 310，补偿了 10 的误差 *)

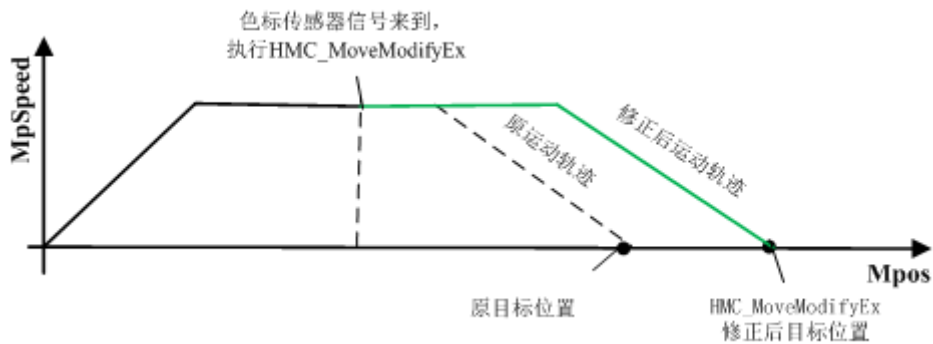
```
Idlereval := HMC_WaitAxisIdle(0); (*等待 MoveModify 补偿完毕，再进行下一循环*)
```

```
(****模切卡片（略）****)
```

```
END_IF
```

```
END_WHILE
```


传送带轨迹:



传送带轨迹示意图

■ 不支持的模块版本

MC1008-A01

3.4.1.4 多轴运动指令

3.4.1.4.1 线性插补

3.4.1.4.1.1 两轴直线插补

HMC_Move2D(2 轴直线插补)

函数原型

```
UDINT HMC_Move2D (USINT ucAxisID,  
USINT ucAxisX,  
USINT ucAxisY,  
LREAL dDistanceX,  
LREAL dDistanceY)
```

功能说明

二维平面上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的两分轴相对位置（以指令起点时刻位置为坐标原点的坐标位置）为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是根据上一次运动的终点位置做增量运动，即每一步都是增量模式运动。

插补合轴的目标位置在指令开始执行时刻位置基础上，增加以两分轴增量位置为直角边的直角三角形的斜边长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由合轴轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。

插补分轴的目标位置在指令开始执行时刻位置基础上，增量该轴指定的相对运动距离。插补运动分轴的运行速度、加减速与该分轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

插补分轴遇到限位时，选取合轴分解计算的分轴减速度和自身快减速度两者的最大值进行最优保护停止，详见 [HMC_HwLimitConfig（设置硬限位开关）](#)。

支持 Merge 功能。多条指令均为支持 Merge 功能的 2 轴插补指令时，如 HMC_Move2D、HMC_MoveCircl，且合轴、X 轴、Y 轴都一致时，满足 Merge 生效条件，开启 Merge 功能可以提高指令执行效率。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
dDistanceX	LREAL	轴 X 运动距离
dDistanceY	LREAL	轴 Y 运动距离

返回值

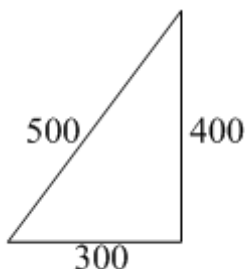
返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

以一个平面切割轮廓为三角形的物料为例，三角形如下：



三角形轮廓

编写程序如下：

(*分别设置使用的三个轴的轴类型*)

```
HMC_SetAxisType (10 ,0);
```

```
HMC_SetAxisType (0 ,10);
```

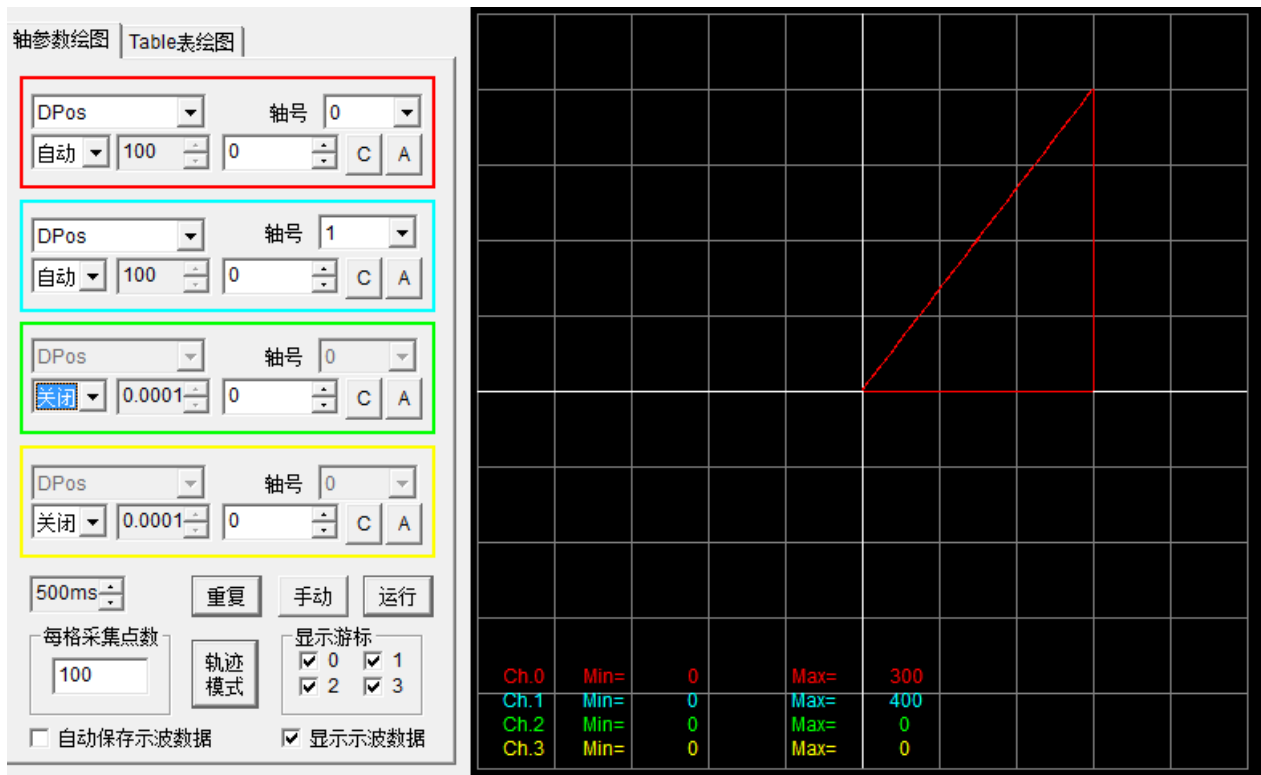
```
HMC_SetAxisType (1,10);
```

```
HMC_Move2D(10,0,1,300,0);    (*X 轴正向移动 300，Y 轴静止。第 1 条直角边*)
```

```
HMC_Move2D(10,0,1,0,400);    (*X 轴保持静止，Y 轴正向移动 400。第 2 条直角边*)
```

```
HMC_Move2D(10,0,1,-300,-400); (*X 轴负向移动 300，Y 轴负向移动 400。斜边*)
```

通过 AT 示波器观察实际运行轨迹如下：



2 轴直线插补实现三角形轮廓轨迹

HMC_Move2DEx(高级 2 轴直线插补)
函数原型

```
UDINT HMC_Move2DEx (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
LREAL dDistanceX,
LREAL dDistanceY,
LREAL dSpeed)
```

功能说明

二维平面上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的两分轴相对位置（以指令起点时刻位置为坐标原点的坐标位置）为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是根据上一次运动的终点位置做增量运动，即每一步都是增量模式运动。

此指令执行前自动设定合轴的轴参数 **Speed** = 函数参数 **dSpeed**，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_Move2D\(2 轴直线插补\)](#) 中功能说明部分。

参数

输入参数	类型	参数描述
------	----	------

ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
dDistanceX	LREAL	轴 X 运动距离
dDistanceY	LREAL	轴 Y 运动距离
dSpeed	LREAL	合轴目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

Axis10.Speed := 1000; (*设定运动速度*)

HMC_Move2DEx(10, 0, 1, 30000, 40000, 2000); (*投送 HMC_Move2DEx 指令, 合轴轴 10 将以最大速度 2000 运行 (而非 1000) *)

■ 不支持的模块版本

MC1008-A01

HMC_MoveAbs2D(2 轴绝对值直线插补)

函数原型

```
UDINT HMC_MoveAbs2D (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
LREAL dDposX,
LREAL dDposY)
```

功能说明

二维平面上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的两分轴绝对位置为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是绝对位置，绝对位置指的是相对于该轴原点的位置。

详细内容可参考 [HMC_Move2D\(2 轴直线插补\)](#) 中功能说明部分。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
dDposX	LREAL	轴 X 目标位置
dDposY	LREAL	轴 Y 目标位置

返回值

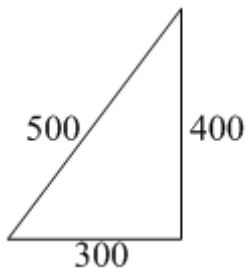
返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

以一个平面切割轮廓为三角形的物料为例，三角形如下：



三角形轮廓

编写程序如下：

(*分别设置使用的三个轴的轴类型*)

```
HMC_SetAxisType(10,0);
```

```
HMC_SetAxisType(0,10);
```

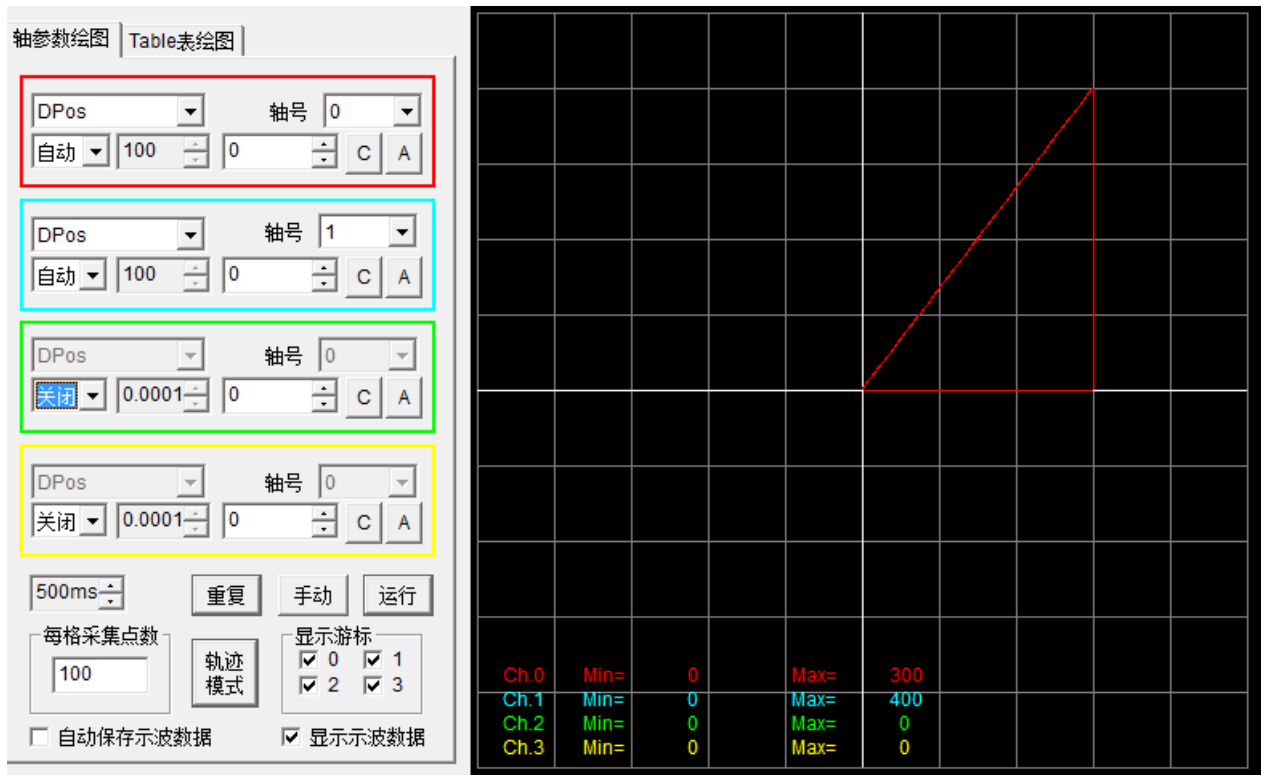
```
HMC_SetAxisType(1,10);
```

```
HMC_MoveAbs2D(10,0,1,300,0); (*X Y 轴运行至 (300,0) 点。第 1 条直角边*)
```

```
HMC_MoveAbs2D(10,0,1,300,400); (*X Y 轴运行至 (300,400) 点。第 2 条直角边*)
```

```
HMC_MoveAbs2D(10,0,1,0,0); (*X Y 轴运行至 (0,0) 点。斜边*)
```

通过 AT 示波器观察实际运行轨迹如下：



2 轴直线插补实现三角形轮廓轨迹

- 不支持的模块版本

MC1008-A01

HMC_MoveAbs2DEx(高级 2 轴绝对值直线插补)

函数原型

UDINT HMC_MoveAbs2DEx (USINT ucAxisID

USINT ucAxisX,

USINT ucAxisY,

LREAL dDposX,

LREAL dDposY,

LREAL dSpeed)

功能说明

二维平面上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的两分轴绝对位置为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是绝对位置，绝对位置指的是相对于该轴原点的位置。

此指令可设定合轴的轴参数 Speed=函数参数 dSpeed，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_Move2D\(2 轴直线插补\)](#) 中功能说明部分。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
dDposX	LREAL	轴 X 目标位置
dDposY	LREAL	轴 Y 目标位置
dSpeed	LREAL	合轴目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

参见 [HMC_MoveAbs2D\(2 轴绝对值直线插补\)](#)。

■ 不支持的模块版本

MC1008-A01

3.4.1.4.1.2 三轴直线插补

HMC_Move3D(3 轴直线插补)

函数原型

```
UDINT HMC_Move3D (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
USINT ucAxisZ,
LREAL dDistanceX,
LREAL dDistanceY,
LREAL dDistanceZ)
```

功能说明

三维空间上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的三分轴相对位置（以指令起点时刻位置为坐标原点的坐标位置）为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是根据上一次运动的终点位置做增量运动，即每一步都是增量模式运动。

插补合轴的目标位置在指令开始执行时刻位置基础上，增加起始点至终点的连线长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由合轴轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。

插补分轴的目标位置在指令开始执行时刻位置基础上，增量该轴指定的相对运动距离。插补运动分轴

的运行速度、加减速度与该轴设定的速度参数 Speed、Accel、Decel 等无关。分轴运动速度由合轴速度参数 VpSpeed 实时关联计算。

插补分轴遇到限位时，选取合轴分解计算的分轴减速度和自身快减速度两者的最大值进行最优保护停止，详见 [HMC HwLimitConfig（设置硬限位开关）](#)。

不支持 Merge 功能。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
dDistanceX	LREAL	轴 X 运动距离
dDistanceY	LREAL	轴 Y 运动距离
dDistanceZ	LREAL	轴 Z 运动距离

返回值

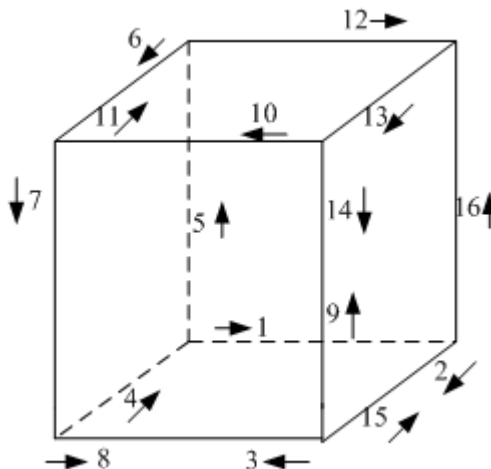
返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

画一个空间的边长为 1000 的正立方体，允许同一条边重复绘制，预期按如下顺序依次画各条边，数字代表轨迹顺序，箭头代表轨迹方向，最终完成正方体的绘制。



正方体示意图

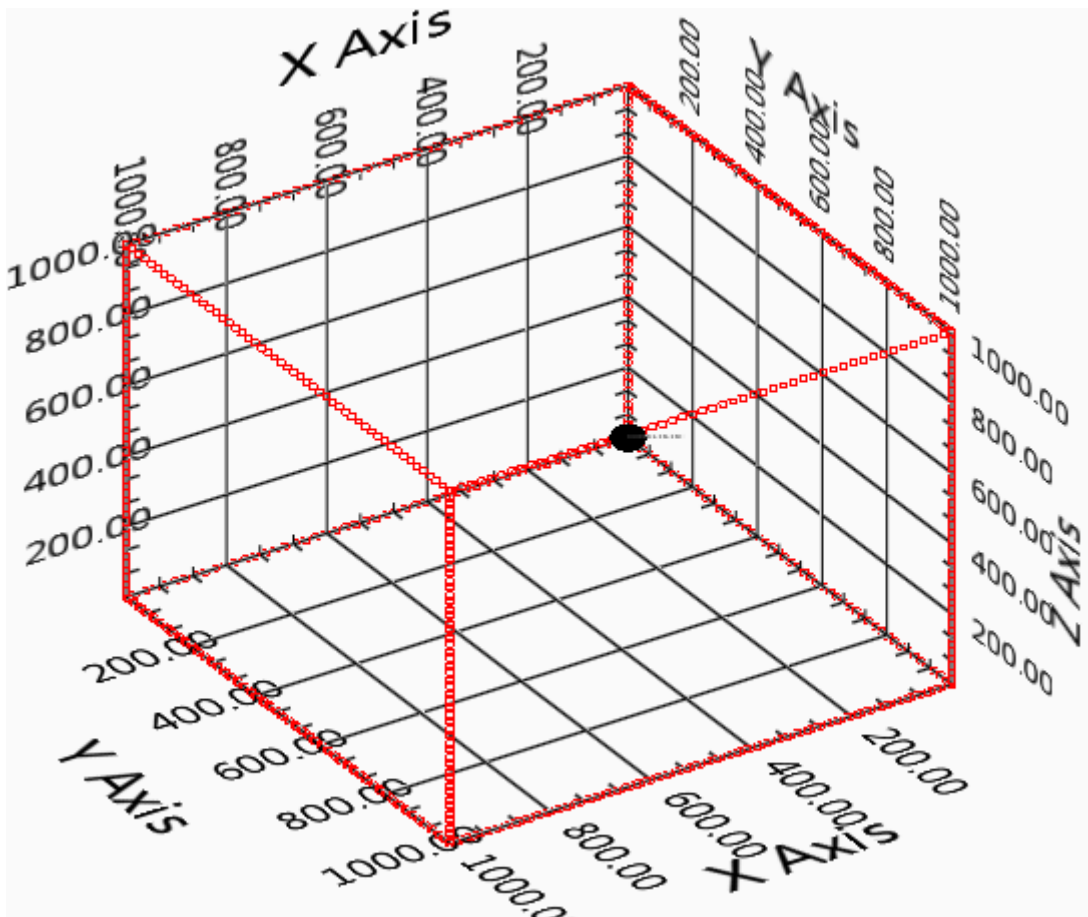
编写程序如下：

(*起始位置为原点*)

HMC_Move3D (5,0,1,2, 1000, 0, 0); (*轨迹 1*)

HMC_Move3D (5 ,0,1,2, 0, 1000, 0); (*轨迹 2*)
HMC_Move3D (5 ,0,1,2, -1000, 0, 0); (*轨迹 3*)
HMC_Move3D (5 ,0,1,2, 0, -1000, 0); (*轨迹 4*)
HMC_Move3D (5 ,0,1,2, 0,0, 1000); (*轨迹 5*)
HMC_Move3D (5 ,0,1,2, 0, 1000, 0); (*轨迹 6*)
HMC_Move3D (5 ,0,1,2, 0, 0, -1000); (*轨迹 7*)
HMC_Move3D (5 ,0,1,2, 1000, 0, 0); (*轨迹 8*)
HMC_Move3D (5 ,0,1,2, 0, 0, 1000); (*轨迹 9*)
HMC_Move3D (5 ,0,1,2, -1000, 0, 0); (*轨迹 10*)
HMC_Move3D (5 ,0,1,2, 0, -1000, 0); (*轨迹 11*)
HMC_Move3D (5 ,0,1,2, 1000, 0, 0); (*轨迹 12*)
HMC_Move3D (5 ,0,1,2, 0, 1000, 0); (*轨迹 13*)
HMC_Move3D (5 ,0,1,2, 0, 0, -1000); (*轨迹 14*)
HMC_Move3D (5 ,0,1,2, 0, -1000, 0); (*轨迹 15*)
HMC_Move3D (5 ,0,1,2, 0, 0, 1000); (*轨迹 16*)

运行以上程序，由于目前 AT 示波器不支持绘制三维轨迹，因此通过在示波器记录三个分轴的 Dpos 值，并将记录数据导出，在三维图形工具中导入记录数据：



三维示意图

HMC_Move3DEx(高级 3 轴直线插补)

函数原型

```
UDINT HMC_Move3DEx (USINT ucAxisID,  
USINT ucAxisX,  
USINT ucAxisY,  
USINT ucAxisZ,  
LREAL dDistanceX,  
LREAL dDistanceY,  
LREAL dDistanceZ,  
LREAL dSpeed)
```

功能说明

三维空间上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的三分轴相对位置（以指令起点时刻位置为坐标原点的坐标位置）为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是根据上一次运动的终点位置做增量运动，即每一步都是增量模式运动。

此指令可设定合轴的轴参数 **Speed**=函数参数 **dSpeed**，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_Move3D\(3 轴直线插补\)](#) 中功能说明部分。

不支持 Merge 功能。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
dDistanceX	LREAL	轴 X 运动距离
dDistanceY	LREAL	轴 Y 运动距离
dDistanceZ	LREAL	轴 Z 运动距离
dSpeed	LREAL	合轴目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

Axis10.Speed := 1000; (*设定运动速度*)

HMC_Move3DEx(10, 0, 1, 2, 30000, 40000, 5000, 2000); (*投送 HMC_Move3DEx 指令, 合轴轴 10 将以最大速度 2000 运行 (而非 1000) *)

■ 不支持的模块版本

MC1008-A01

HMC_MoveAbs3D(3 轴绝对值直线插补)

函数原型

```
UDINT HMC_MoveAbs3D (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
USINT ucAxisZ,
LREAL dDposX,
LREAL dDposY,
LREAL dDposZ)
```

功能说明

三维平面上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的三分轴绝对位置为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是绝对位置，绝对位置指的是相对于该轴原点的位置。

详细内容可参考 [HMC_Move3D\(3 轴直线插补\)](#) 中功能说明部分。

本指令目前暂不支持 merge 功能。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
dDposX	LREAL	轴 X 目标位置
dDposY	LREAL	轴 Y 目标位置
dDposZ	LREAL	轴 Z 目标位置

返回值

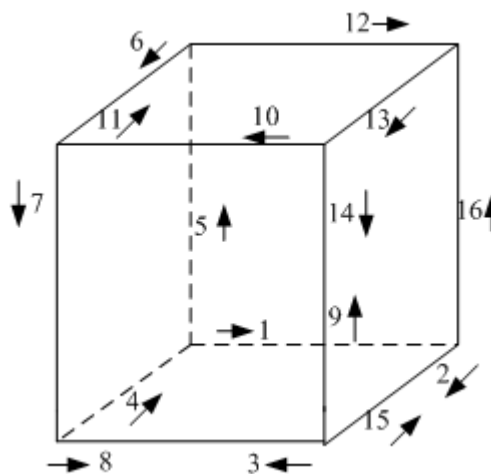
返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

画一个空间的边长为 1000 的正立方体，允许同一条边重复绘制，预期按如下顺序依次画各条边，数字代表轨迹顺序，箭头代表轨迹方向，最终完成正方体的绘制。



正方体示意图

编写程序如下：

(*起始位置为原点*)

HMC_MoveAbs3D (5,0,1,2,1000,0,0); (*轨迹 1*)

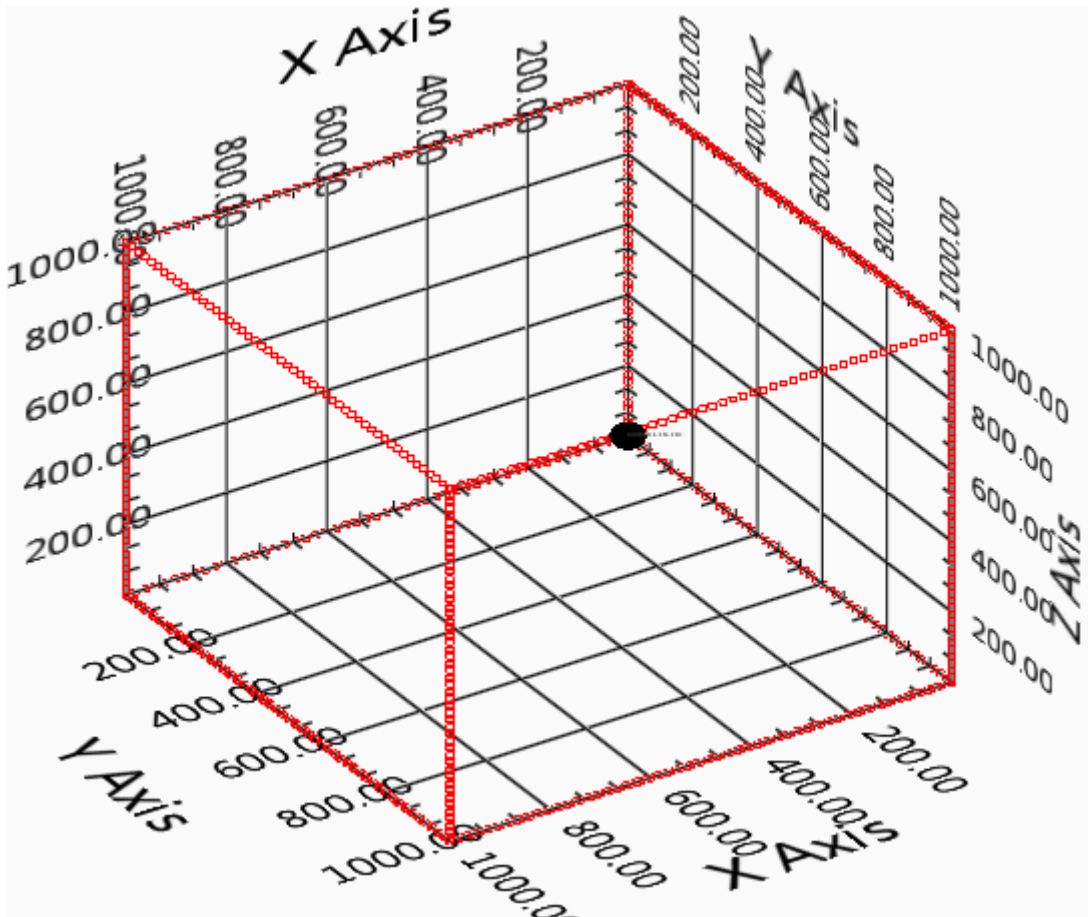
HMC_MoveAbs3D (5,0,1,2,1000,1000,0); (*轨迹 2*)

HMC_MoveAbs3D (5,0,1,2,0,1000,0); (*轨迹 3*)

HMC_MoveAbs3D (5,0,1,2,0,0,0); (*轨迹 4*)

HMC_MoveAbs3D (5,0,1,2,0,0,1000); (*轨迹 5*)
HMC_MoveAbs3D (5,0,1,2,0,1000,1000); (*轨迹 6*)
HMC_MoveAbs3D (5,0,1,2,0,1000,0); (*轨迹 7*)
HMC_MoveAbs3D (5,0,1,2,1000,1000,0); (*轨迹 8*)
HMC_MoveAbs3D (5,0,1,2,1000,1000,1000); (*轨迹 9*)
HMC_MoveAbs3D (5,0,1,2,0,1000,1000); (*轨迹 10*)
HMC_MoveAbs3D (5,0,1,2,0,0,1000); (*轨迹 11*)
HMC_MoveAbs3D (5,0,1,2,1000,0,1000); (*轨迹 12*)
HMC_MoveAbs3D (5,0,1,2,1000,1000,1000); (*轨迹 13*)
HMC_MoveAbs3D (5,0,1,2,1000,1000,0); (*轨迹 14*)
HMC_MoveAbs3D (5,0,1,2,1000,0,0); (*轨迹 15*)
HMC_MoveAbs3D (5,0,1,2,1000,0,1000); (*轨迹 16*)

运行以上程序，由于目前 AT 示波器不支持绘制三维轨迹，因此通过在示波器记录三个分轴的 Dpos 值，并将记录数据导出，在三维图形工具中导入记录数据：



三维示意图

■ 不支持的模块版本

MC1008-A01

HMC_MoveAbs3DEx(高级 3 轴绝对值直线插补)

函数原型

```

UDINT HMC_MoveAbs3DEx (USINT ucAxisID,
    USINT ucAxisX,
    USINT ucAxisY,
    USINT ucAxisZ,
    LREAL dDposX,
    LREAL dDposY,
    LREAL dDposZ,
    LREAL dSpeed)
    
```

功能说明

三维平面上运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的三分

轴绝对位置为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是绝对位置，绝对位置指的是相对于该轴原点的位置。

此指令可设定合轴的轴参数 $Speed = \text{函数参数 } dSpeed$ ，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_Move3D\(3 轴直线插补\)](#) 中功能说明部分。

不支持 Merge 功能。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
dDposX	LREAL	轴 X 目标位置
dDposY	LREAL	轴 Y 目标位置
dDposZ	LREAL	轴 Z 目标位置
dSpeed	LREAL	合轴目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

参考 [HMC_MoveAbs3D\(3 轴绝对值直线插补\)](#)

■ 不支持的模块版本

MC1008-A01

3.4.1.4.1.3 多轴直线插补

HMC_MoveND(N 轴直线插补)

函数原型

```
UDINT HMC_MoveND (USINT ucAxisID,
    POINTER TO USINT pAxis,
    POINTER TO LREAL pDistance,
    USINT ucLineNum,
    USINT ucSelfNum)
```

功能说明

可以实现任意 N 维坐标系统的运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的线性插补分轴相对位置（以指令起点时刻位置为坐标原点的坐标位置）为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是根据上一次运动的终点位置做增量运动，即每一步都是

增量模式运动。

插补合轴的目标位置在指令开始执行时刻位置基础上，增加起始点至终点的连线长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。

插补分轴的目标位置在指令开始执行时刻位置基础上，增量该轴指定的相对运动距离。插补运动分轴的运行速度、加减速与该轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

N 轴插补指令的分轴分为两种类型：线性插补分轴和独立插补分轴。线性插补分轴决定插补合运动的实际轨迹，独立插补分轴与合运动轨迹无关，独立插补分轴可实现与合轴或分轴的同步运动，即指令结束时刻，合轴和线性插补分轴完成指令运动距离时，独立插补分轴也同时完成运动距离。

各个线性插补分轴的运动距离决定了合轴的运动距离，合轴运动距离等于各个线性插补分轴运动距离的平方之和再开方。线性插补分轴速度由合轴速度、线性插补分轴运动距离与合轴运动距离的比值决定。

独立插补分轴的运动距离不影响合轴的运动距离，因此称为独立轴，可实现与合轴或插补线性轴的同步运动。独立轴的速度由合轴速度、独立插补分轴运动距离与合轴运动距离的比值决定。

N 轴线性插补中合运动与分运动关系详细描述如下：

数组 **pAxis** 和 **pDistance** 分别代表分轴轴号和各分轴运动距离。**LN** 为线性插补分轴个数（**ucLineNum** 记为 **LN**），**SN** 为独立插补分轴个数（**ucSelfNum** 记为 **SN**），即 **LN+SN** 为分轴总个数，因此 **pAxis** 和 **pDistance** 分别指向数组的元素个数 **N** 应满足：**N ≥ LN+SN**。

合运动轴 **ucAxisID** 的运动距离为 **D**，速度为 **Speed**；

LN 个线性插补轴的运动距离为 **L(0)**, ..., **L(LN-1)**，速度为：**SpeedL(0)**, ..., **SpeedL(LN-1)**；

SN 个独立插补轴的运动距离为 **S(0)**, ..., **S(SN-1)**，速度为：**SpeedS(0)**, ..., **SpeedS(SN-1)**；

合轴运动距离由线性插补分轴运动距离决定，具体如下：

$$D^2 = L(0)^2 + L(1)^2 + \dots + L(LN - 1)^2$$

线性插补分轴速度与合轴速度关系如下：

$$SpeedL(i) = \frac{L(i)}{D} \times Speed \quad (0 \leq i \leq LN - 1)$$

独立插补分轴速度与合轴速度关系如下：

$$SpeedS(j) = \frac{S(j)}{D} \times Speed \quad (0 \leq j \leq SN - 1)$$

本指令不支持 **Merge** 功能。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
pAxis	POINTER TO USINT	插补分运动轴号数组首地址
pDistance	POINTER TO LREAL	插补分运动轴运动距离首地址
ucLineNum	USINT	线性插补轴个数。pAxis 的前 ucLineNum 个元素即为线性插补分轴的轴号，pDistance 的前 ucLineNum 个元素即为线性插补分轴的运动距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucLineNum>=1
ucSelfNum	USINT	独立插补轴个数。pAxis 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的轴号，pDistance 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的运动距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucSelfNum>=0

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

```
FOR i:=0 TO 4 DO
```

```
    AxisArray[i] := i;
```

```
END_FOR      (*数组 AxisArray 前 5 个元素为插补分轴的轴号，依次为 0~4*)
```

```
DistanceArray[0] := 10000;
```

```
DistanceArray[1] := 20000;
```

```
DistanceArray[2] := 30000;
```

```
DistanceArray[3] := 180;
```

```
DistanceArray[4] := 120;
```

HMC_MoveND (15 ,ADR(AxisArray), ADR(DistanceArray), 3, 2); (*前 3 个分轴为线性插补分轴，轴序号为 0、1、2，接下来 2 个分轴为独立插补分轴，轴序号为 3、4*)

HMC_MoveNDEx(高级 N 轴直线插补)

函数原型

```
UDINT HMC_MoveNDEx (USINT ucAxisID,
```

```
    POINTER TO USINT pAxis,
```

```
    POINTER TO LREAL pDistance,
```

```
    USINT ucLineNum,
```

USINT ucSelfNum,

LREAL dSpeed)

功能说明

可以实现任意 N 维坐标系统的运动轮廓为直线的插补功能。以本指令开始执行时刻为起点，以指令输入参数设置的线性插补分轴相对位置（以指令起点时刻位置为坐标原点的坐标位置）为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是根据上一次运动的终点位置做增量运动，即每一步都是增量模式运动。

此指令可设定合轴的轴参数 Speed=函数参数 dSpeed，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_MoveND\(N 轴直线插补\)](#) 中功能说明部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
pAxis	POINTER TO USINT	插补分运动轴号数组首地址
pDistance	POINTER TO LREAL	插补分运动轴运动距离首地址
ucLineNum	USINT	线性插补轴个数。pAixs 的前 ucLineNum 个元素即为线性插补分轴的轴号，pDistance 的前 ucLineNum 个元素即为线性插补分轴的运动距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucLineNum>=1
ucSelfNum	USINT	独立插补轴个数。pAixs 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的轴号，pDistance 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的运动距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucSelfNum>=0
dSpeed	LREAL	合轴 ucAxisID 的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

参考 [HMC_MoveND\(N 轴直线插补\)](#)。

- 不支持的模块版本

MC1008-A01

HMC_MoveAbsND (N 轴绝对值直线插补)

函数原型

UDINT HMC_MoveAbsND (USINT ucAxisID,

POINTER TO USINT pAxis,

POINTER TO LREAL pDpos,

USINT ucLineNum,

USINT ucSelfNum)

功能说明

可以实现任意 **N** 维坐标系统的运动轮廓为直线的插补功能。以指令输入参数设置的线性插补分轴的绝对位置为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是绝对位置，绝对位置指的是相对于该轴原点的位置。

详细内容可参考 [HMC_MoveND\(N 轴直线插补\)](#) 中功能说明部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
pAxis	POINTER TO USINT	插补分运动轴号数组首地址
pDpos	POINTER TO LREAL	插补分运动轴目标位置首地址
ucLineNum	USINT	线性插补轴个数。pAixs 的前 ucLineNum 个元素即为线性插补分轴的轴号，pDpos 的前 ucLineNum 个元素即为线性插补分轴的目标距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucLineNum>=1
ucSelfNum	USINT	独立插补轴个数。pAixs 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的轴号，pDpos 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的目标距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucSelfNum>=0

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

参考 [HMC_MoveND\(N 轴直线插补\)](#)

■ 不支持的模块版本

MC1008-A01

HMC_MoveAbsNDEx (高级 N 轴绝对值直线插补)

函数原型

```
UDINT HMC_MoveAbsNDEx (USINT ucAxisID,
    POINTER TO USINT pAxis,
    POINTER TO LREAL pDpos,
    USINT ucLineNum,
    USINT ucSelfNum,
    LREAL dSpeed)
```

功能说明

可以实现任意 **N** 维坐标系统的运动轮廓为直线的插补功能。以指令输入参数设置的线性插补分轴的绝对位置为终点，起点和终点的连线即为该指令的运动轨迹。此指令的目标位置都是绝对位置，绝对位置指的是相对于该轴原点的位置。

此指令可设定合轴的轴参数 **Speed=**函数参数 **dSpeed**，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_MoveND\(N 轴直线插补\)](#) 中功能说明部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
pAxis	POINTER TO USINT	插补分运动轴号数组首地址
pDpos	POINTER TO LREAL	插补分运动轴目标位置首地址
ucLineNum	USINT	线性插补轴个数。pAixs 的前 ucLineNum 个元素即为线性插补分轴的轴号，pDpos 的前 ucLineNum 个元素即为线性插补分轴的目标距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucLineNum>=1
ucSelfNum	USINT	独立插补轴个数。pAixs 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的轴号，pDpos 从 ucLineNum 个元素开始往后的 ucSelfNum 个元素即为独立插补分轴的目标距离，从前到后依次与 pAxis 指定的轴号对应 取值范围：ucSelfNum>=0
dSpeed	LREAL	合轴 ucAxisID 的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

参考 [HMC_MoveND\(N 轴直线插补\)](#)

■ 不支持的模块版本

MC1008-A01

3.4.1.4.2 圆弧插补

3.4.1.4.2.1 平面圆弧插补

HMC_MoveCircle（圆弧插补）

函数原型

```
UDINT HMC_MoveCircle (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
USINT ucMode,
LREAL dAx,
LREAL dAy,
LREAL dBx,
```

LREAL dBy)

功能说明

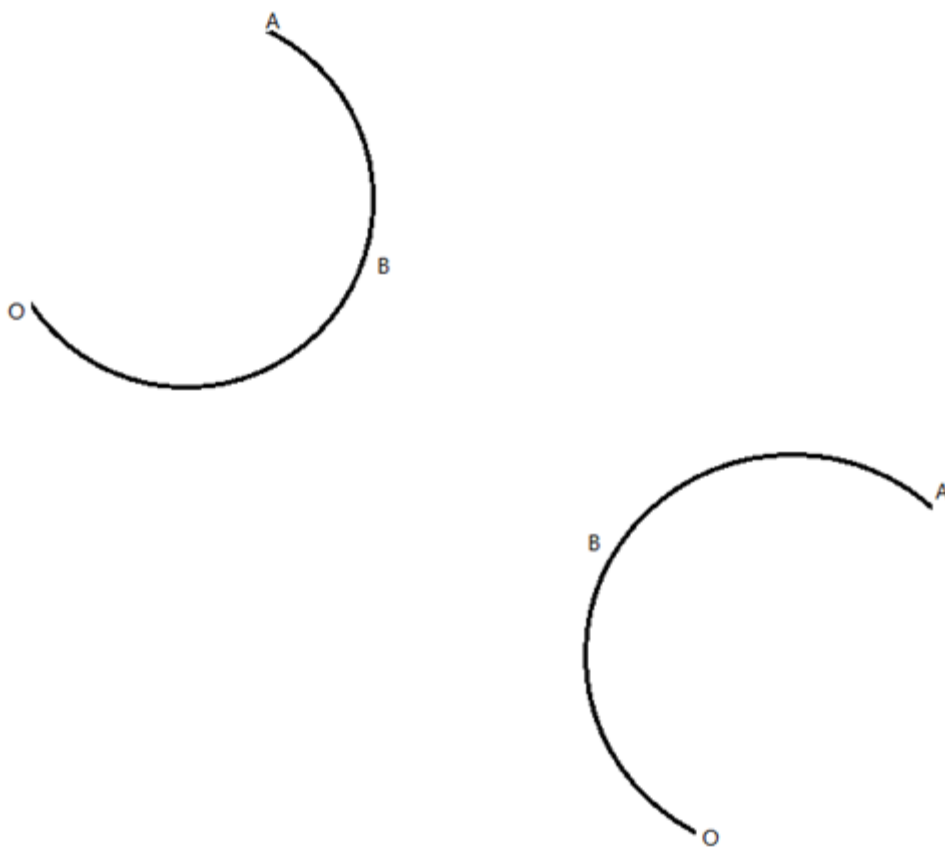
二维平面上运动轮廓为圆弧的插补功能。

插补合轴的目标位置在指令开始执行时刻位置基础上，增加指令轨迹圆弧长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由合轴轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。插补运动分轴的运行速度、加减速与该轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

本指令支持三种模式：

(1) ucMode=0

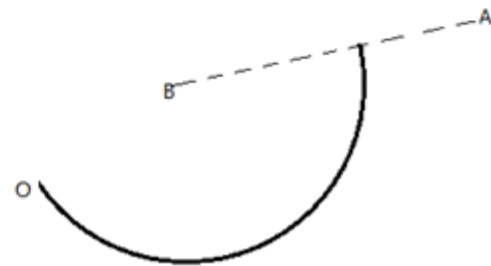
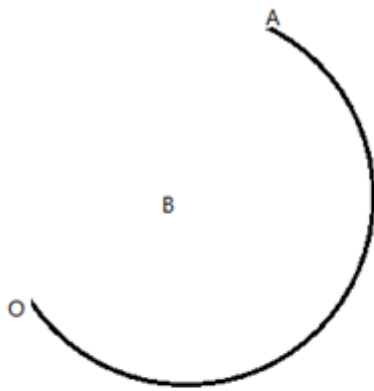
以圆弧指令开始时刻位置为起点，A 为终点，B 为中间点，一段圆弧。A、B 点的坐标都是以 O 点为坐标原点。如果 O、B、A 三点一线且 B 在中间时，按照直线处理。



运动轨迹

(2) ucMode=1

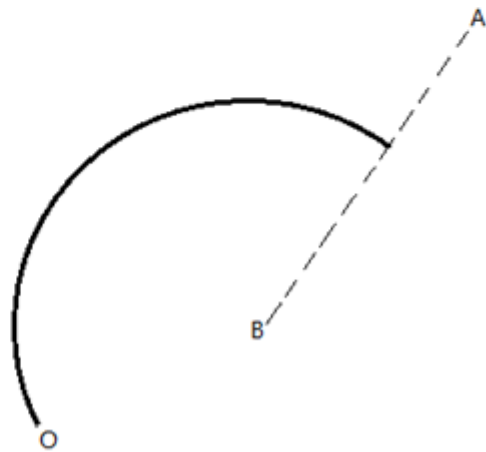
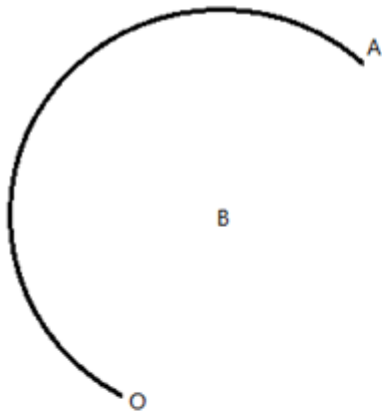
以圆弧指令开始时刻为起点，B 为圆心，A 为终点，逆时针方向一段圆弧。A、B 点的坐标都是以 O 点为坐标原点。起点和圆心即可确定圆弧，如果 A 是圆上一点，则 A 就为指令终点，如下图左图；否则即为圆心到 A 点射线与圆弧的交点为终点，如下图右图。如果 A 点为 (0,0) 则逆时针画一整圆。



运动轨迹

(3) ucMode=3

以圆弧指令开始时刻为起点，B 为圆心，A 为终点，顺时针方向一段圆弧。A、B 点的坐标都是以 O 点为坐标原点。起点和圆心即可确定圆弧，如果 A 是圆上一点，则 A 就为指令终点，如下图左图；否则即为圆心到 A 点射线与圆弧的交点为终点，如下图右图。如果 A 点为 (0,0) 则顺时针画一整圆。



运动轨迹

支持 Merge 功能。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucMode (O 为当前点)	USINT	0: A 为终点, B 为中间某点的一段圆弧; O、B、A 三点一线且 B 在中间时, 按照直线处理 1: A 为终点, B 为圆弧的圆心, 且按照逆时针方向走 3: A 为终点, B 为圆弧的圆心, 且按照顺时针方向走
dAx	LREAL	A 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, A 的 X 坐标)
dAy	LREAL	A 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, A 的 Y 坐标)
dBx	LREAL	B 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, B 的 X 坐标)
dBy	LREAL	B 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, B 的 Y 坐标)

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功

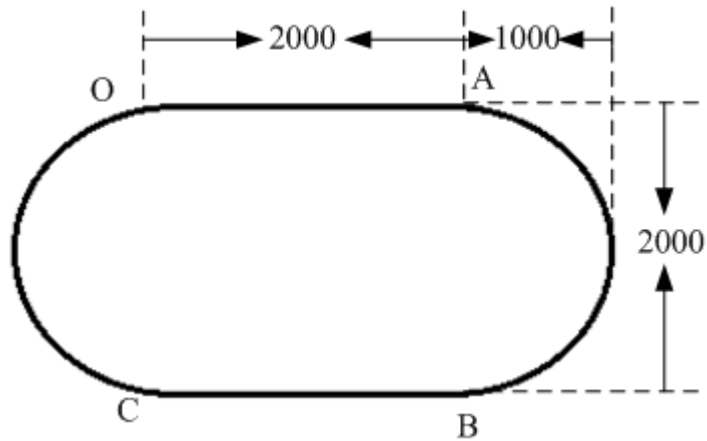
返回值	类型	参数描述
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

切割如下工件。



工件尺寸图

以上图 O 点为起始点，编写程序如下：

```
HMC_Move2D(6,0,1,2000,0);
```

```
HMC_MoveCircle(6,0,1,3,0,-2000,0,-1000);
```

(*以当前点 A 为起始点，终点和圆心都以 A 点为原点

*)

```
HMC_Move2D(6,0,1,-2000,0);
```

```
HMC_MoveCircle(6,0,1,3,0,2000,0,1000);
```

(*以当前点 C 为起始点，终点和圆心都以 C 点为原点

*)

运行程序，在 AT 示波器中得到轨迹如下：



轨迹示意图

HMC_MoveCircleEx（高级圆弧插补）

函数原型

```
UDINT HMC_MoveCircleEx(USINT ucAxisID,
    USINT ucAxisX,
    USINT ucAxisY,
    USINT ucMode,
    LREAL dAx,
    LREAL dAy,
    LREAL dBx,
    LREAL dBy,
    LREAL dSpeed)
```

功能说明

二维平面上运动轮廓为圆弧的插补功能。

插补合轴的目标位置在指令开始执行时刻位置基础上，增加指令轨迹圆弧长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。插补运动分轴的运行速度、加减速与该轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

此指令可设定合轴的轴参数 $Speed = \text{函数参数 } dSpeed$ ，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC_MoveCircle（圆弧插补）](#) 中功能说明部分。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucMode (O 为当前点)	USINT	0: A 为终点, B 为中间某点的一段圆弧; O、B、A 三点一线且 B 在中间时, 按照直线处理 1: A 为终点, B 为圆弧的圆心, 且按照逆时针方向走 3: A 为终点, B 为圆弧的圆心, 且按照顺时针方向走
dAx	LREAL	A 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, A 的 X 坐标)
dAy	LREAL	A 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, A 的 Y 坐标)
dBx	LREAL	B 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, B 的 X 坐标)
dBy	LREAL	B 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, B 的 Y 坐标)
dSpeed	LREAL	合轴目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令

应用举例

参考 [HMC_MoveCircle（圆弧插补）](#)。

■ 不支持的模块版本

MC1008-A01

3.4.1.4.2.2 球面插补

HMC_MoveSpherical（球弧插补）

函数原型

```
UDINT HMC_MoveSpherical (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
USINT ucAxisZ,
USINT ucMode,
LREAL dAx,
LREAL dAy,
```

LREAL dAz,

LREAL dBx,

LREAL dBy,

LREAL dBz)

功能说明

三维空间上运动轮廓为球弧的插补功能。球弧即空间某个平面的圆弧，或者说球面与某个平面相交的空间圆上的一段圆弧。

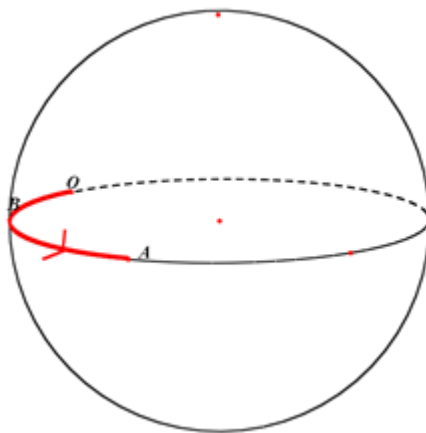
插补合轴的目标位置在指令开始执行时刻位置基础上，增加指令轨迹球弧长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。插补运动分轴的运行速度、加减速与该轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

本指令支持四种模式：

(1) ucMode=0

以指令开始时刻为起点 **O**，**B** 为中间点，**A** 为终点，三点确定一段球弧。**A**、**B** 点的坐标都是以 **O** 点为坐标原点。如果 **O**、**B**、**A** 三点一线且 **B** 点为中间点时，按照直线处理。

下图是 **O**、**B**、**A** 三点确定的空间一段球弧，运动方向为 **O**->**B**->**A**，红色曲线为球弧运动轨迹曲线，箭头表示方向。

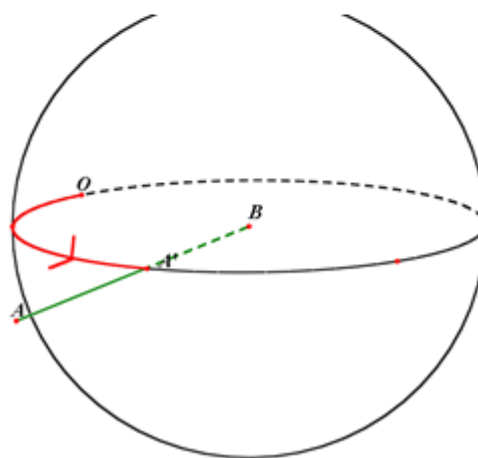
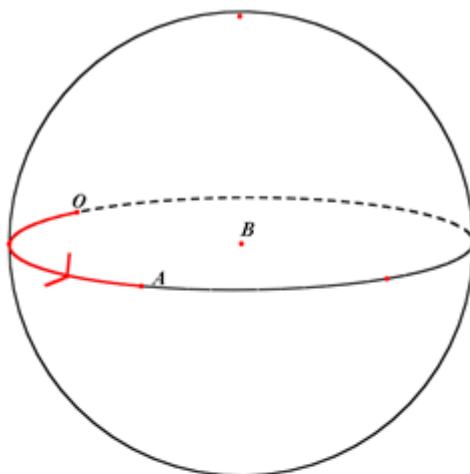


运动轨迹

(2) ucMode=1

以指令开始时刻为起点 **O**，**B** 为空间圆弧的圆心，**A** 为终点，按照 **O** 到 **A** 的最短距离为运动方向，确定一段空间圆弧。

下图是以 **O** 为起点、**B** 为球弧圆心、**A** 为终点的空间一段球弧，当 **A** 点不在球弧上时，**AB** 连线与球弧交点则为终点，红色曲线为球弧运动轨迹曲线，箭头表示方向。

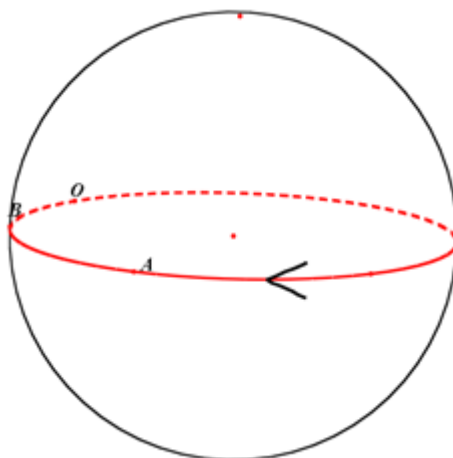


运动轨迹

(3) ucMode=2

以指令开始时刻为起点 O ， A 、 B 为中间点，方向为 $O \rightarrow A \rightarrow B$ ，运动完整圆弧。

下图是 O 、 B 、 A 三点确定的空间球弧，运动方向为 $O \rightarrow A \rightarrow B$ ，红色曲线即三点确定的完整球弧为运动轨迹曲线，箭头表示方向。

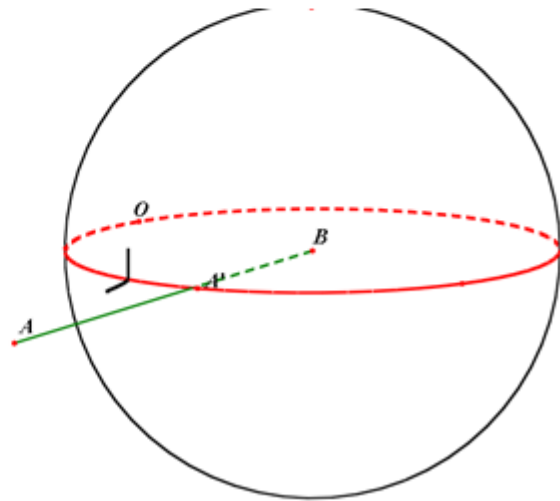
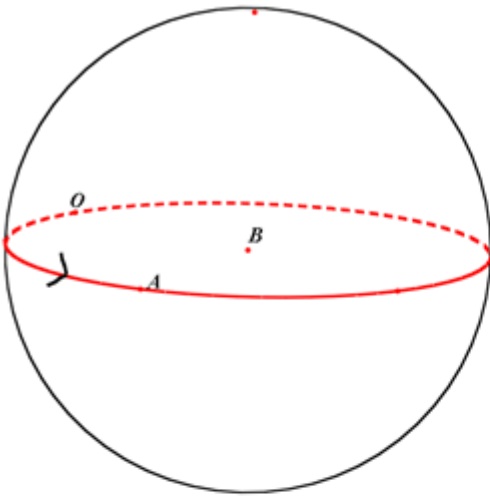


运动轨迹

(4) ucMode=3

以指令开始时刻为起点 O ， B 为空间圆弧的圆心， A 为中间点，按照 O 到 A 的最短距离为运动方向，运动完整圆弧。

下图是以 O 为起点、 B 为球弧圆心、 A 为中间点的空间一段球弧，按照 O 到 A 的最短距离为运动方向，即为逆时针方向。红色曲线为球弧运动轨迹曲线，箭头表示方向。



运动轨迹

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
ucMode (O 为当前点)	USINT	0: A 为终点, B 为中间某点的一段圆弧, O、B、A 三点一线且 B 点为中间点时, 按照直线处理 1: A 为终点, B 为空间圆弧的圆心的一段圆弧, 系统确定方向方法: 使 O 到 A 的最短距离 (等距特殊情况系统自动处理)。注意: 用户设置时 OBA 三点不可共线 2: A、B 为中间某点, 运动完整圆弧, 方向为 O->A->B 3: A 为中间某点, B 为空间圆弧的圆心, 运动完整圆弧, 系统确定方向方法: 使 O 到 A 的最短距离 (等距特殊情况系统自动处理)。注意: 用户设置时 OBA 三点不可共线
dAx	LREAL	A 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, A 的 X 坐标)
dAy	LREAL	A 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, A 的 Y 坐标)
dAZ	LREAL	A 点在 Z 轴位置-O 点在 Z 轴的位置

输入参数	类型	参数描述
		(将 O 视为坐标系原点时, A 的 Z 坐标)
dBx	LREAL	B 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, B 的 X 坐标)
dBy	LREAL	B 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, B 的 Y 坐标)
dBZ	LREAL	B 点在 Z 轴位置-O 点在 Z 轴的位置 (将 O 视为坐标系原点时, B 的 Z 坐标)

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令属性

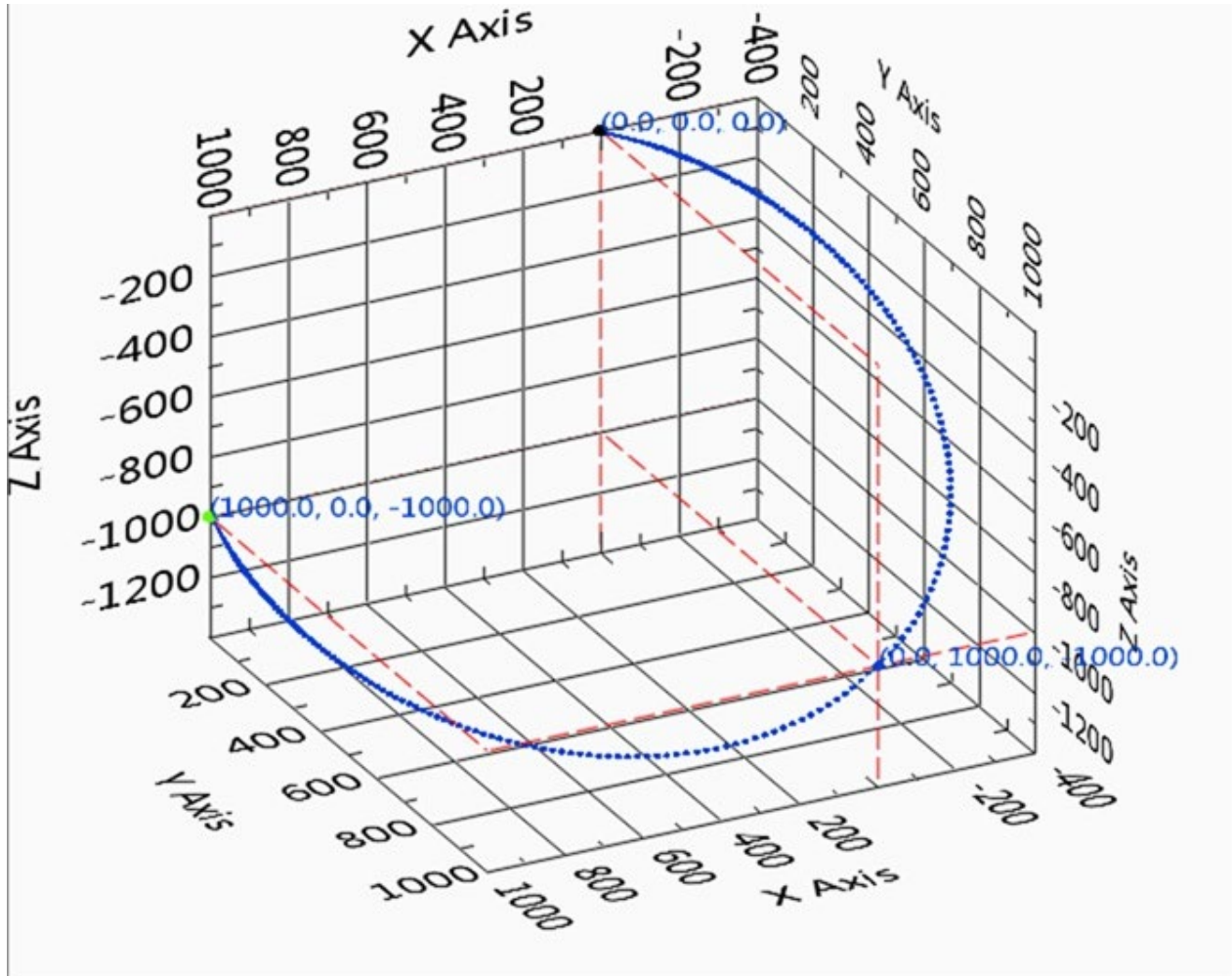
非阻塞指令, 轴运动缓存指令

应用举例

以(0, 0, 0)为起始点, 经过(0, 1000, -1000)点, 终点为(1000, 0, -1000)点, 使用模式 0, 编写程序如下:

```
HMC_MoveSpherical (0,1,2,3,0,1000,0,-1000,0,1000,-1000 );
```

AT 示波器记录各分轴 Dpos, 将数据导出, 通过第三方软件读取数据绘制三维图形如下:



三维示意图

HMC_MoveSphericalEx（高级球弧插补）
函数原型

```

UDINT HMC_MoveSphericalEx (USINT ucAxisID,
USINT ucAxisX,
USINT ucAxisY,
USINT ucAxisZ,
USINT ucMode,
LREAL dAx,
LREAL dAy,
LREAL dAz,
LREAL dBx,
LREAL dBy,

```


LREAL dBz,

LREAL dSpeed)

功能说明

三维空间上运动轮廓为球弧的插补功能。球弧即空间某个平面的圆弧，或者说球面与某个平面相交的空间圆上的一段圆弧。

插补合轴的目标位置在指令开始执行时刻位置基础上，增加指令轨迹球弧长度。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。插补运动分轴的运行速度、加减速与该轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

此指令可设定合轴的轴参数 **Speed**=函数参数 **dSpeed**，合轴以该速度为目标速度进行解算。

本指令支持四种模式，详细可见 [HMC MoveSpherical（球弧插补）](#)。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
ucMode (O 为当前点)	USINT	0: A 为终点, B 为中间某点的一段圆弧, O、B、A 三点一线且 B 点为中间点时, 按照直线处理 1: A 为终点, B 为空间圆弧的圆心的一段圆弧, 系统确定方向方法: 使 O 到 A 的最短距离 (等距特殊情况系统自动处理)。注意: 用户设置时 OBA 三点不可共线 2: A、B 为中间某点, 运动完整圆弧, 方向为 O->A->B 3: A 为中间某点, B 为空间圆弧的圆心, 运动完整圆弧, 系统确定方向方法: 使 O 到 A 的最短距离 (等距特殊情况系统自动处理)。注意: 用户设置时 OBA 三点不可共线
dAx	LREAL	A 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, A 的 X 坐标)
dAy	LREAL	A 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, A 的 Y 坐标)
dAZ	LREAL	A 点在 Z 轴位置-O 点在 Z 轴的位置 (将 O 视为坐标系原点时, A 的 Z 坐标)
dBx	LREAL	B 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, B 的 X 坐标)
dBy	LREAL	B 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, B 的 Y 坐标)
dBZ	LREAL	B 点在 Z 轴位置-O 点在 Z 轴的位置 (将 O 视为坐标系原点时, B 的 Z 坐标)
dSpeed	LREAL	合轴目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令属性

非阻塞指令，轴运动缓存指令

应用举例

参考 [HMC MoveSpherical（球弧插补）](#)。

- 不支持的模块版本

MC1008-A01

3.4.1.4.3 螺旋线插补

3.4.1.4.3.1 HMC_MoveHelical（螺旋线插补）

函数原型

```
UDINT HMC_MoveHelical (USINT ucAxisID,  
USINT ucAxisX,  
USINT ucAxisY,  
USINT ucAxisZ,  
USINT ucMode,  
LREAL dAx,  
LREAL dAy,  
LREAL dBx,  
LREAL dBy,  
LREAL dDistanceZ)
```

功能说明

螺旋线插补运动。**ucAxisID** 为合运动轴，在坐标架 O-XYZ 下，**ucAxisX**、**ucAxisY** 两轴执行平面圆弧插补运动，**ucAxisZ** 轴执行 Z 相线性运动，三轴共同完成圆柱螺旋线轨迹。

插补合轴的目标位置根据不同模式计算方法不同。插补运动合轴运动速度由对应轴参数 **Speed** 设定，加减速分别由轴参 **Accel**、**Decel** 设定。在插补运动过程中，可以实时修改合轴 **Speed**、**Accel**、**Decel** 等轴参数，立即生效。插补运动分轴的运行速度、加减速与该轴设定的速度参数 **Speed**、**Accel**、**Decel** 等无关。分轴运动速度由合轴速度参数 **VpSpeed** 实时关联计算。

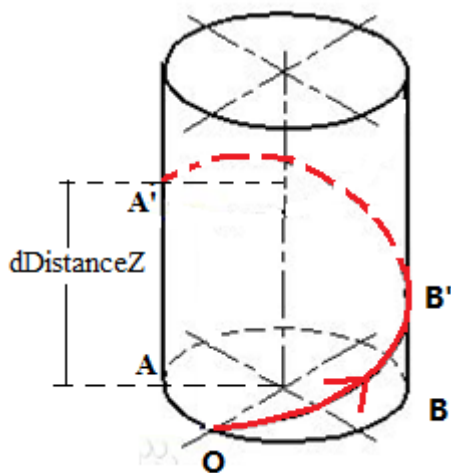
该指令支持 6 种模式：

(1) ucMode=0

平面圆弧是以 A 为终点，B 为中间点。合轴是 XYZ 轴合运动，合轴的目标位置在指令开始执行时刻位置基础上，增量以平面圆弧长度、Z 轴运动位置长度为直角边的三角形斜边长度。合轴与分轴位置、速度关系如下：

$$dDistance_AxisID^2 = dCircleLength^2 + dDistance_AxisZ^2$$
$$\frac{dSpeed_AxisZ}{dSpeed_AxisID} = \frac{dDistance_AxisZ}{dDistance_AxisID}$$

下图为该模式运动示意图，红色曲线为三轴联动的空间轨迹曲线。

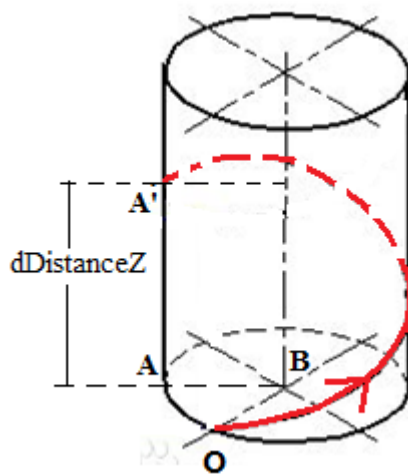


运动轨迹

(2) ucMode=1

平面圆弧以 B 为圆心，A 为终点，按照逆时针方向。合轴是 XYZ 轴合运动，合轴与分轴位置、速度关系与模式 0 一致。

下图为该模式运动示意图，红色曲线为三轴联动的空间轨迹曲线。

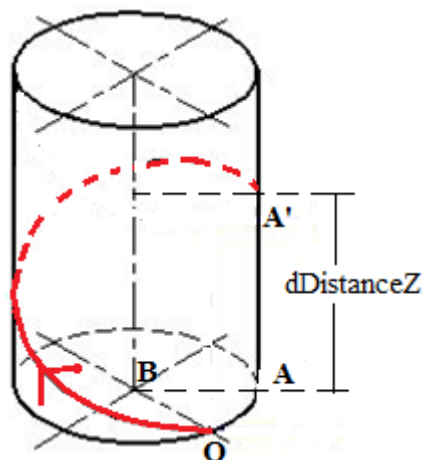


运动轨迹

(3) ucMode=3

平面圆弧以 B 为圆心，A 为终点，按照顺时针方向。合轴是 XYZ 轴合运动，合轴与分轴位置、速度关系与模式 0 一致。

下图为该模式运动示意图，红色曲线为三轴联动的空间轨迹曲线。



运动轨迹

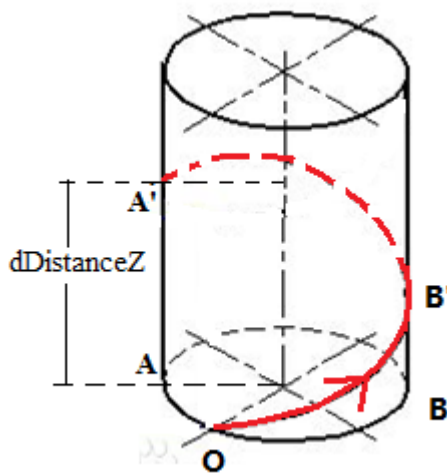
(4) ucMode=4

平面圆弧是以 A 为终点，B 为中间点。合轴是 XY 轴合运动，合轴的目标位置在指令开始执行时刻位置基础上，增量以平面圆弧长度。合轴与分轴位置、速度关系如下：

$$dDistance_AxisID = dCircleLength$$

$$\frac{dSpeed_AxisZ}{dSpeed_AxisID} = \frac{dDistance_AxisZ}{dDistance_AxisID}$$

下图为该模式运动示意图，红色曲线为三轴联动的空间轨迹曲线。

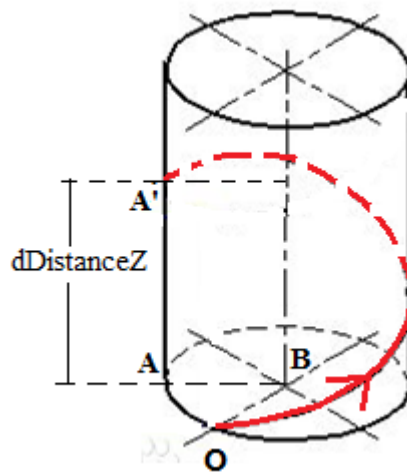


运动轨迹

(5) ucMode=5

平面圆弧以 B 为圆心，A 为重点，按照逆时针方向。合轴是 XY 轴合运动，合轴与分轴位置、速度关系与模式 4 一致。

下图为该模式运动示意图，红色曲线为三轴联动的空间轨迹曲线。

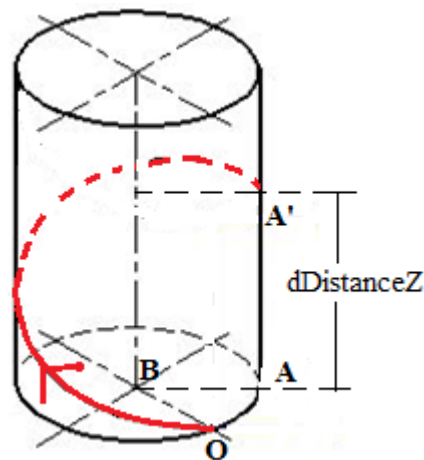


运动轨迹

(6) ucMode=7

平面圆弧以 B 为圆心，A 为重点，按照顺时针方向。合轴是 XY 轴合运动，合轴与分轴位置、速度关系与模式 4 一致。

下图为该模式运动示意图，红色曲线为三轴联动的空间轨迹曲线。



运动轨迹

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
ucMode (O 为当前点)	USINT	0: A 为终点，B 为中间某点的一段圆弧；ucAxisID 的速度 dSpeed 是三轴的合运动的速度。注意：用户设置时 OBA 三点不可共线 1: A 为终点，B 为圆弧的圆心，且按照逆时针方向走；ucAxisID 的速度 dSpeed 是三轴的合运动的速度 3: A 为终点，B 为圆弧的圆心，且按照顺时针方向走；ucAxisID 的速度 dSpeed 是三轴的合运动的速度

输入参数	类型	参数描述
		4: A 为终点, B 为中间某点的一段圆弧; ucAxisID 的速度 dSpeed 是 X、Y 轴合运动的速度。 注意: 用户设置时 OBA 三点不可共线 5: A 为终点, B 为圆弧的圆心, 且按照逆时针方向走; ucAxisID 的速度 dSpeed 是 X、Y 轴合运动的速度 7: A 为终点, B 为圆弧的圆心, 且按照顺时针方向走; ucAxisID 的速度 dSpeed 是 X、Y 轴合运动的速度
dAx	LREAL	A 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, A 的 X 坐标)
dAy	LREAL	A 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, A 的 Y 坐标)
dBx	LREAL	B 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, B 的 X 坐标)
dBy	LREAL	B 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, B 的 Y 坐标)
dDistanceZ	LREAL	Z 轴位移

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令属性

非阻塞指令, 轴运动缓存指令

应用举例

螺旋铣削运动轨迹为一螺旋线, 可通过数控机床的三轴联动来实现, 工件水平面进行 X、Y 轴的圆弧插补, 工件的垂直方向进行 Z 轴线性运动。在 XOY 平面完成整圆运动, 即沿螺纹孔旋转 360 度后, 在 Z 轴方向上升一个螺距。可利用螺旋线插补铣削螺纹。

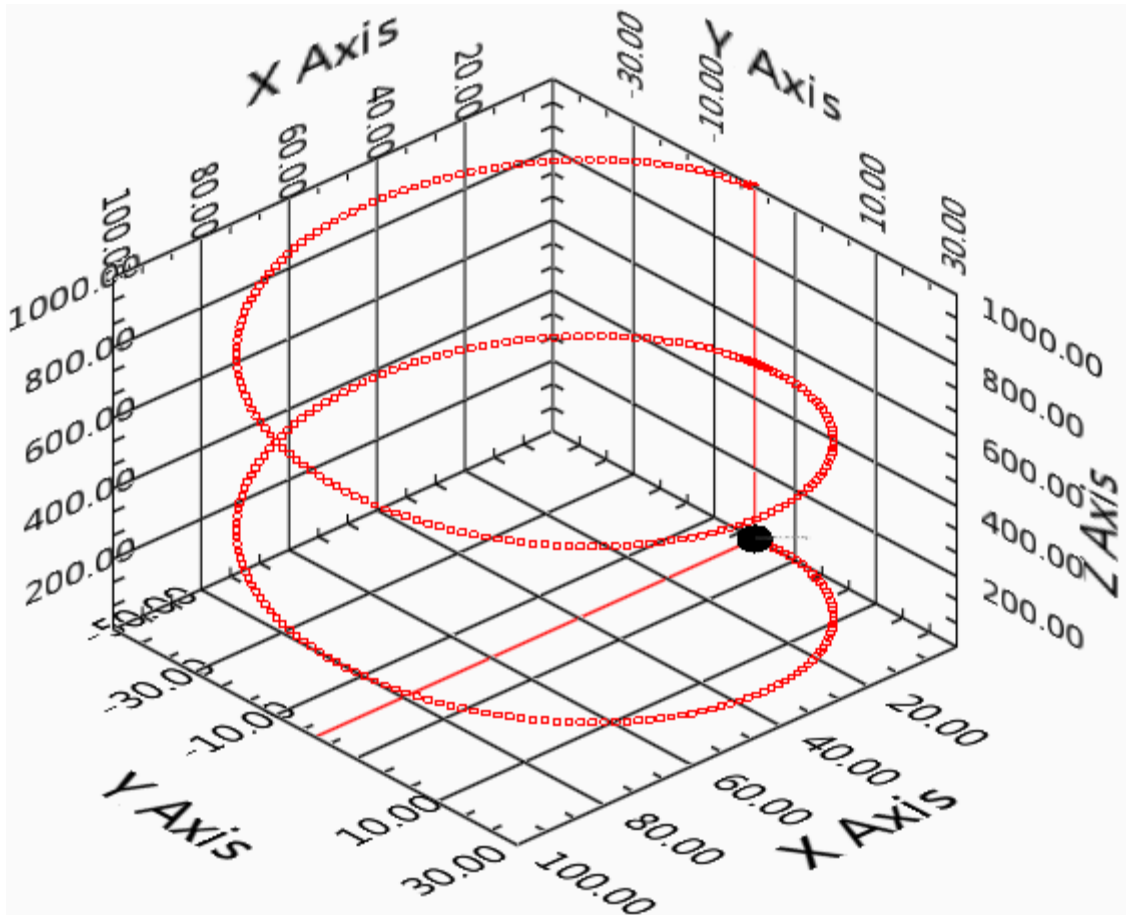
例如, 要加工一个直径为 100 mm, 螺距是 500 mm, 高度为 1000 mm 的螺纹。

FOR i:=1 TO 2 BY 1 DO

HMC_MoveHelical (0, 1, 2, 3, 7, 0, 0, 50, 0, 500);

END_FOR

AT 示波器记录各分轴 Dpos, 将数据导出, 通过第三方软件读取数据绘制三维图形:



三维示意图

3.4.1.4.3.2 HMC_MoveHelicalEx (高级螺旋线插补)

函数原型

```

UDINT HMC_MoveHelicalEx (USINT ucAxisID,
  USINT ucAxisX,
  USINT ucAxisY,
  USINT ucAxisZ,
  USINT ucMode,
  LREAL dAx,
  LREAL dAy,
  LREAL dBx,
  LREAL dBy,
  LREAL dDistanceZ,
  LREAL dSpeed)
  
```

功能说明

螺旋线插补运动。**ucAxisID** 为合运动轴，在坐标架 O-XYZ 下，**ucAxisX**、**ucAxisY** 两轴执行平面圆弧插补运动，**ucAxisZ** 轴执行 Z 相线性运动，三轴共同完成圆柱螺旋线轨迹。

此指令可设定合轴的轴参数 **Speed** = 函数参数 **dSpeed**，合轴以该速度为目标速度进行解算。

详细内容可参考 [HMC MoveHelical（螺旋线插补）](#) 中功能说明部分。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
ucAxisX	USINT	插补轴 X 的轴号
ucAxisY	USINT	插补轴 Y 的轴号
ucAxisZ	USINT	插补轴 Z 的轴号
ucMode (O 为当前点)	USINT	0: A 为终点, B 为中间某点的一段圆弧; ucAxisID 的速度 dSpeed 是三轴的合运动的速度。注意: 用户设置时 OBA 三点不可共线 1: A 为终点, B 为圆弧的圆心, 且按照逆时针方向走; ucAxisID 的速度 dSpeed 是三轴的合运动的速度 3: A 为终点, B 为圆弧的圆心, 且按照顺时针方向走; ucAxisID 的速度 dSpeed 是三轴的合运动的速度 4: A 为终点, B 为中间某点的一段圆弧; ucAxisID 的速度 dSpeed 是 X、Y 轴合运动的速度。注意: 用户设置时 OBA 三点不可共线 5: A 为终点, B 为圆弧的圆心, 且按照逆时针方向走; ucAxisID 的速度 dSpeed 是 X、Y 轴合运动的速度 7: A 为终点, B 为圆弧的圆心, 且按照顺时针方向走; ucAxisID 的速度 dSpeed 是 X、Y 轴合运动的速度
dAx	LREAL	A 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, A 的 X 坐标)
dAy	LREAL	A 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, A 的 Y 坐标)
dBx	LREAL	B 点在 X 轴位置-O 点在 X 轴的位置 (将 O 视为坐标系原点时, B 的 X 坐标)
dBy	LREAL	B 点在 Y 轴位置-O 点在 Y 轴的位置 (将 O 视为坐标系原点时, B 的 Y 坐标)
dDistanceZ	LREAL	Z 轴位移
dSpeed	LREAL	合轴 ucAxisID 的目标速度

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令属性

非阻塞指令，轴运动缓存指令

应用举例

参考 [HMC MoveHelical（螺旋线插补）](#) 示例。

- 不支持的模块版本

MC1008-A01

3.4.1.4.4 样条插补

3.4.1.4.4.1 HMC_MoveSplineND（样条插补）

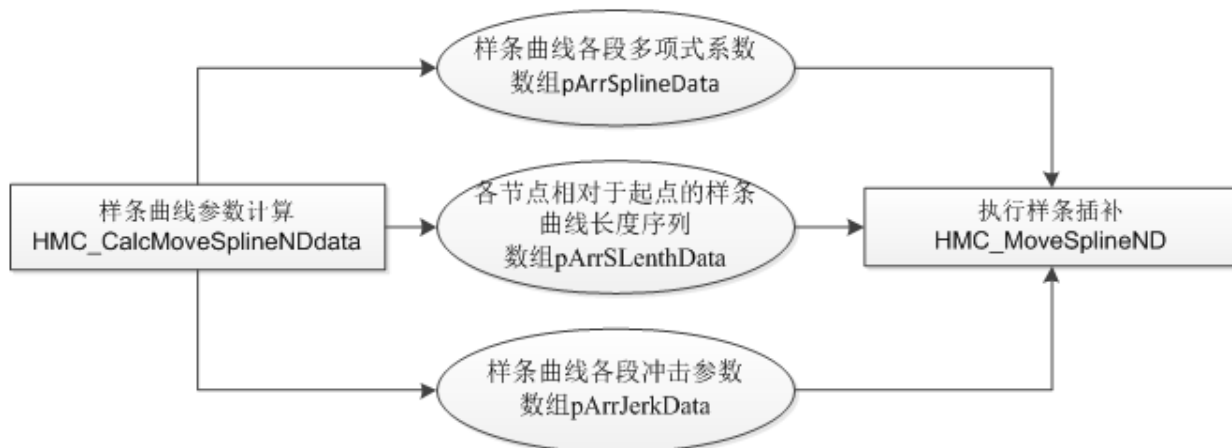
函数原型

```
UDINT HMC_MoveSplineND (USINT ucAxisID,
    POINTER TO USINT pAxis,
    USINT ucEndPointAxisNum,
    USINT ucFollowAxisNum,
    POINTER TO LREAL pArrPos,
    UDINT uiSplineNum,
    POINTER TO LREAL pArrSplineData,
    POINTER TO LREAL pArrSlenthData,
    POINTER TO LREAL pArrJerkData)
```

功能说明

以当前点为起点，执行 N 轴样条插补运动（相对坐标/增量坐标）。

该指令使用 HMC_CalcMoveSplineNDdata 函数计算所得到的样条曲线的相关参数(pArrSplineData、pArrSplineData、pArrSlenthData)，通过指定合轴轴号 ucAxisID 及各个插补分轴轴号数组 pAxis、样条曲线节点坐标数组 pArrPos，实现 N 轴样条插补功能。



样条插补功能

使用 N 轴样条插补指令最常用的是实现 2 维或 3 维样条插补，可如下设置参数：

- （1）2 轴样条插补：跟随轴 ucFollowAxisNum = 0，端点轴 ucEndPointAxisNum=2；
- （2）3 轴样条插补：跟随轴 ucFollowAxisNum = 0，端点轴 ucEndPointAxisNum=3。

执行上述 2 轴或 3 轴样条插补时，插补合轴按照端点轴合运动的轨迹运动，端点轴速度的合速度近似等于插补合轴速度 VpSpeed。通过设定合轴 Speed 参数，可控制插补轮廓速度。

pAxis 数组中放置端点轴（EndPointAxis）与跟随轴（FollowAxis）的轴号，排列顺序为先端点轴后跟随轴。

pArrPos 数组中的坐标由各节点在端点轴与跟随轴上的坐标分量组成，端点轴与跟随轴的数目由 ucEndPointAxisNum 与 ucFollowAxisNum 确定。数组中坐标排列顺序：首先是各端点轴坐标，然后是各跟随轴坐标。可定义该数组为二维数组，这样数组元素 pArrPos[i,j]便表示第 i 个轴在第 j 个节点上的坐标分量。

端点轴与跟随轴的区别（如果仅使用常规的 2 轴/3 轴样条插补，则跟随轴数目为 0，此段可不作了解）：类似于 N 轴直线插补（HMC_MoveND）的线性轴与独立轴，作样条插补运动时，插补合轴按照各端点轴合运动的轨迹进行，而不受跟随轴的影响。各端点轴速度的合速度约等于插补合轴速度 VpSpeed，各跟随轴的平均速度约为（跟随轴总位移/合轴总位移）*合轴 VpSpeed，瞬时速度与样条曲线在该点的瞬时斜率有关。端点轴的个数只能为 2 或 3，而跟随轴个数可为 0，总轴数不能超过系统最大轴数（64）。

插补合轴运动速度由对应轴参数 Speed 设定，加减速分别由轴参 Accel、Decel 设定。在插补运动过程中，可以实时修改合轴 Speed、Accel、Decel 等轴参数，立即生效。插补运动分轴的运行速度、加减速与该轴自身的 Speed、Accel、Decel 等无关，由合轴实时关联计算得到。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
pAxis	POINTER TO USINT	插补分运动轴号数组首地址。排列顺序为先端点轴后跟随轴。
ucEndPointAxisNum	USINT	端点轴的个数。端点轴合运动的速度近似为插补合轴 Speed。取值范围：2 或 3
ucFollowAxisNum	USINT	跟随轴的个数。
pArrPos	POINTER TO LREAL	样条曲线节点坐标，2 维数组 pArrPos[length1][length2]首地址。数组第一维长度 length1 为坐标轴个数（维度），第二维长度 length2 为样条曲线节点数目。有如下关系： length1 = n length2 = uiSplineNum 其中 n = ucEndPointAxisNum +ucFollowAxisNum
uiSplineNum	UDINT	pArrPos 中的样点个数，1~1000 个。
pArrSplineData	POINTER TO LREAL	数组首地址，HMC_CalcMoveSplineNDdata 中计算得到的样条曲线各段的多项式系数参数。
pArrSlenthData	POINTER TO LREAL	数组首地址，HMC_CalcMoveSplineNDdata 中计算得到的 各节点相对于起点的样条曲线长度序列。
pArrJerkData	POINTER TO LREAL	数组首地址，HMC_CalcMoveSplineNDdata 中计算得到的 样条曲线各段冲击参数。

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

（1）pArrJerkData 为 HMC_CalcMoveSplineNDdata 中计算得出的每段样条函数所对应的最大冲击计

算参数，插补运动过程中会根据 **pArrJerkData** 与合轴速度实时速度 **VpSpeed** 计算实时冲击数据，如果大于合轴轴参中的 **Jerk** 限定值，则将降低合轴速度，以保证运动过程中冲击不超限；

（2）在转折较为急剧的节点附近，实际插补速度可能会比设定的合轴插补速度 **Speed** 略大一些，但上述中的处理措施可保证此处的冲击不会超过用户设定的 **Jerk** 值。

应用示例

示例 1：2 轴样条插补

参见 **HMC_CalcMoveSplineNDdata** 中示例 1

示例 2：3 轴样条插补

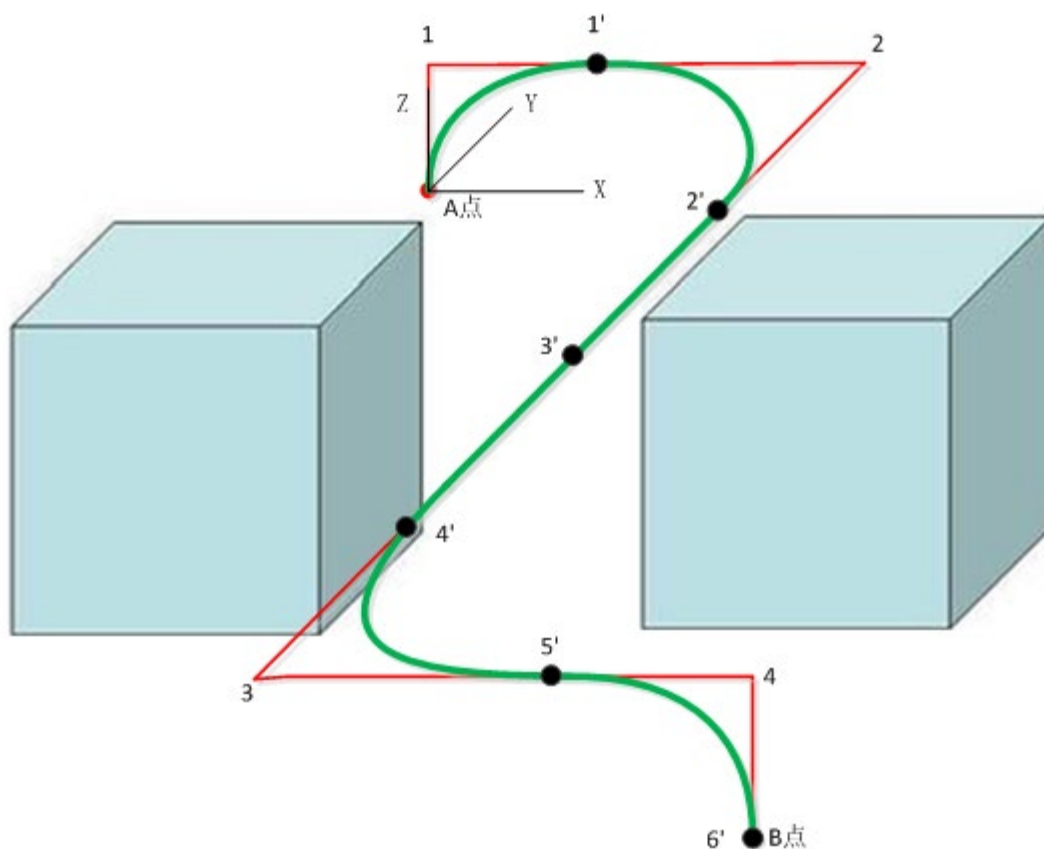
参见 **HMC_CalcMoveSplineNDdata** 中示例 2

示例 3：避障取放物料

三轴直角坐标机械手需把物料从位置 **A** 搬运至位置 **B**，然后从 **B** 返回 **A**，周而复始。由于 **AB** 之间有一些障碍物，因此需规划合适的路径避开障碍。

如果采用常规的三轴直线插补，则路径规划如下图中红色曲线所示。由图可知在每个转折点 1、2、3、4 均需减速至 0，然后开始下一段运动，大大降低了搬运效率。

若使用样条插补，以 **A** 点为当前起点运动至 **B** 点，在路径上取节点 1'、2'、3'、4'、5'、6'，则样条插补轨迹示意图如下。由于样条插补轨迹的方向变化是平滑的，不会像直线插补那样在转折点处减速至 0，因此可以大大提高搬运效率及运动的平稳性。



样条插补示意图

设 A 点为坐标原点，B 点坐标为 (3000,-4000,0)。

若采用三轴直线插补，则从 A 点运行至 B 点经过的停止点 1、2、3、4 到 B 点，坐标依次为 (0,0,500)、(1500,0,500)、(1500,-4000,500)、(3000,-4000,500)、(3000,-4000,0)。

从 A 点运行至 B 点的三轴直线插补程序如下：

(****首先定义大小为 pArrPos[3][5] 的节点数组，然后对目标位置数组赋值****)

pArrPos[0][0] := 0; (*第 1 个目标点在轴 X 轴上的坐标分量*)

pArrPos[0][1] := 1500; (*第 2 个目标点在轴 4(X 轴)上的坐标分量*)

pArrPos[0][2] := 1500; (*第 3 个目标点在轴 4(X 轴)上的坐标分量*)

pArrPos[0][3] := 3000; (*第 4 个目标点在轴 4(X 轴)上的坐标分量*)

pArrPos[0][4] := 3000; (*第 5 个目标点在轴 4(X 轴)上的坐标分量*)

pArrPos[1][0] := 0; (*第 1 个目标点在轴 5(Y 轴)上的坐标分量*)

pArrPos[1][1] := 0; (*第 2 个目标点在轴 5(Y 轴)上的坐标分量*)

pArrPos[1][2] := -4000; (*第 3 个目标点在轴 5(Y 轴)上的坐标分量*)

pArrPos[1][3] := -4000; (*第 4 个目标点在轴 5(Y 轴)上的坐标分量*)

pArrPos[1][4] := -4000; (*第 5 个目标点在轴 5(Y 轴)上的坐标分量*)

pArrPos[2][0] := 500; (*第 1 个目标点在轴 6(Z 轴)上的坐标分量*)

pArrPos[2][1] := 500; (*第 2 个目标点在轴 6(Z 轴)上的坐标分量*)

pArrPos[2][2] := 500; (*第 3 个目标点在轴 6(Z 轴)上的坐标分量*)

pArrPos[2][3] := 500; (*第 4 个目标点在轴 6(Z 轴)上的坐标分量*)

pArrPos[2][4] := 0; (*第 5 个目标点在轴 6(Z 轴)上的坐标分量*)

(*定义分轴轴号数组 pArrAxis[3]*)

pArrAxis[0] := 4; (*X 轴轴号*)

pArrAxis[1] := 5; (*Y 轴轴号*)

pArrAxis[2] := 6; (*Z 轴轴号*)

(*定义合轴轴号 AxisNo_3d *)

AxisNo_3d := 7;

(*执行三轴直线插补指令*)

HMC_MoveABS3d(AxisNo_3d,pArrAxis[0],pArrAxis[1],pArrAxis[2], pArrPos[0][0], pArrPos[1][0],
pArrPos[2][0]);

HMC_MoveABS3d(AxisNo_3d,pArrAxis[0],pArrAxis[1],pArrAxis[2], pArrPos[0][1], pArrPos[1][1],
pArrPos[2][1]);

HMC_MoveABS3d(AxisNo_3d,pArrAxis[0],pArrAxis[1],pArrAxis[2], pArrPos[0][2], pArrPos[1][2],
pArrPos[2][2]);

HMC_MoveABS3d(AxisNo_3d,pArrAxis[0],pArrAxis[1],pArrAxis[2], pArrPos[0][3], pArrPos[1][3],
pArrPos[2][3]);

HMC_MoveABS3d(AxisNo_3d,pArrAxis[0],pArrAxis[1],pArrAxis[2], pArrPos[0][4], pArrPos[1][4],
pArrPos[2][4]);

若采用样条插补，从 A 点运行至 B 点，在路径上取节点 1'、2'、3'、4'、5'、6'，存入节点位置数组 pArrPos 中。1'、2'、3'、4'、5'、6'的坐标依次为 (750,0,500)、(1500,-1000,500)、(1500,-2000,500)、(1500,-3000,500)、(2250,-4000,500)、(3000,-4000,0)。

从 A 点运行至 B 点的样条插补程序如下：

(****首先定义大小为 pArrPos[3][6]的节点数组，然后对节点数组赋值****)

pArrPos[0][0] := 750; (*第 1 个目标点在轴 0(X 轴)上的坐标分量*)

```
pArrPos[0][1] := 1500; (*第 2 个节点在轴 0(X 轴)上的坐标分量*)  
pArrPos[0][2] := 1500; (*第 3 个节点点在轴 0(X 轴)上的坐标分量*)  
pArrPos[0][3] := 1500; (*第 4 个节点点在轴 0(X 轴)上的坐标分量*)  
pArrPos[0][4] := 2250; (*第 5 个节点点在轴 0(X 轴)上的坐标分量*)  
pArrPos[0][5] := 3000; (*第 6 个节点点在轴 0(X 轴)上的坐标分量*)
```

```
pArrPos[1][0] := 0; (*第 1 个节点点在轴 1(Y 轴)上的坐标分量*)  
pArrPos[1][1] := -1000; (*第 2 个节点点在轴 1(Y 轴)上的坐标分量*)  
pArrPos[1][2] := -2000; (*第 3 个节点点在轴 1(Y 轴)上的坐标分量*)  
pArrPos[1][3] := -3000; (*第 4 个节点点在轴 1(Y 轴)上的坐标分量*)  
pArrPos[1][4] := -4000; (*第 5 个节点点在轴 1(Y 轴)上的坐标分量*)  
pArrPos[1][5] := -4000; (*第 6 个节点点在轴 1(Y 轴)上的坐标分量*)
```

```
pArrPos[2][0] := 500; (*第 1 个节点点在轴 2(Z 轴)上的坐标分量*)  
pArrPos[2][1] := 500; (*第 2 个节点点在轴 2(Z 轴)上的坐标分量*)  
pArrPos[2][2] := 500; (*第 3 个节点点在轴 2(Z 轴)上的坐标分量*)  
pArrPos[2][3] := 500; (*第 4 个节点点在轴 2(Z 轴)上的坐标分量*)  
pArrPos[2][4] := 500; (*第 5 个节点点在轴 2(Z 轴)上的坐标分量*)  
pArrPos[2][5] := 0; (*第 6 个节点点在轴 2(Z 轴)上的坐标分量*)
```

(*定义数组 pArrSplineData[72], pArrSlenthData[6], pArrJerkData [6],

保存计算得到的样条参数*)

(*调用函数生成样条曲线参数*)

```
HMC_CalcMoveSplineNDdata(pArrPos,6,3,0, pArrSplineData,72, pArrSlenthData, pArrJerkData);
```

(*定义分轴轴号数组 pArrAxis[3]*)

```
pArrAxis[0] := 0; (*第 1 个端点轴轴号(X 轴)*)  
pArrAxis[1] := 1; (*第 2 个端点轴轴号(Y 轴)*)  
pArrAxis[2] := 2; (*第 3 个端点轴轴号(Z 轴)*)
```

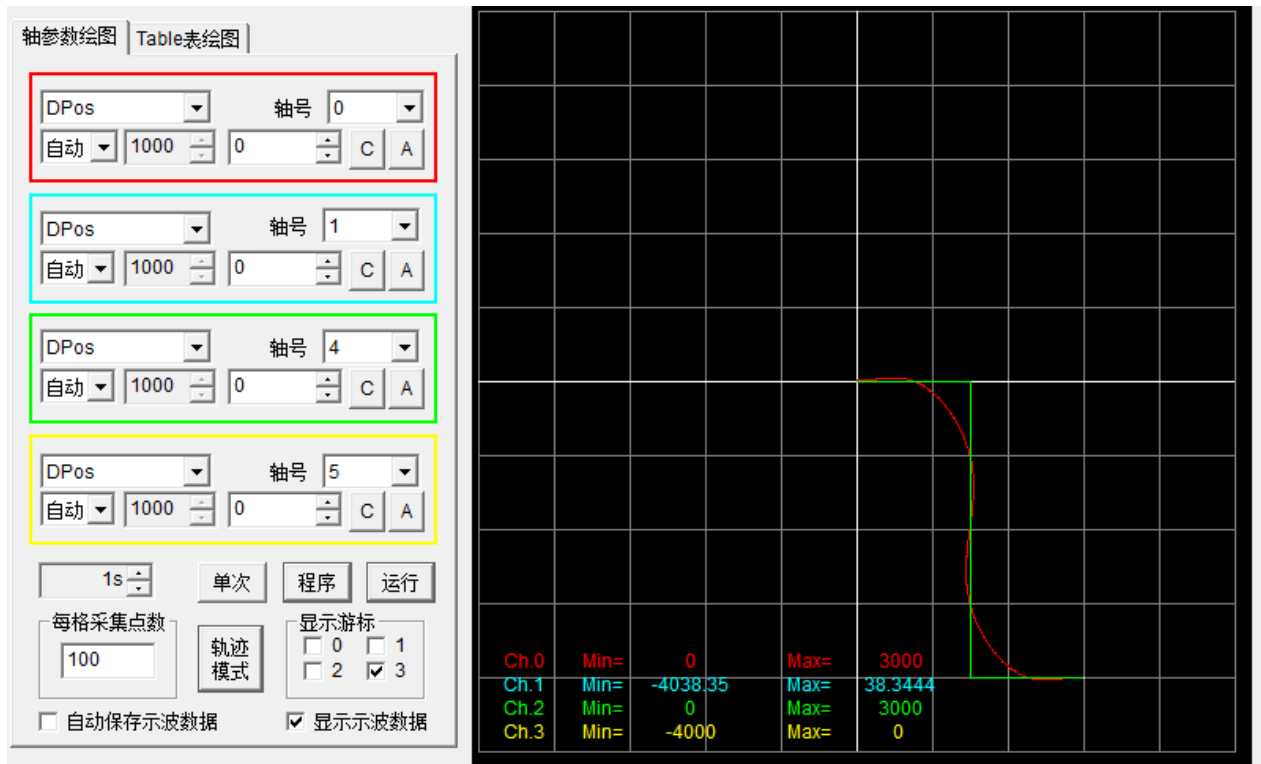
(*定义合轴轴号 AxisNo_Spline *)

AxisNo_Spline := 3;

(*执行样条插补指令*)

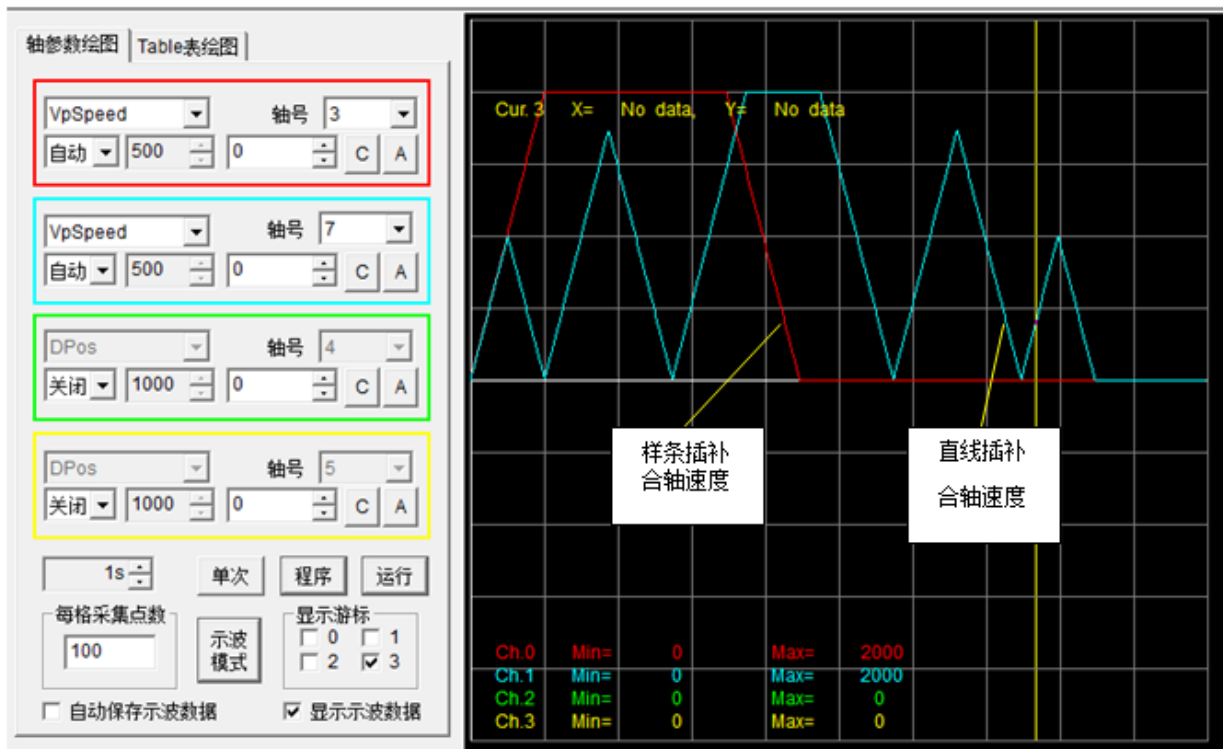
HMC_MoveSplineND(AxisNo_Spline,pArrAxis,3,0, pArrPos,6, pArrSplineData, pArrSlenthData, pArrJerkData);

在 AT 中运行，直线插补轨迹（绿色）与样条插补轨迹（红色）的 XY 平面俯视图如下：



运动轨迹

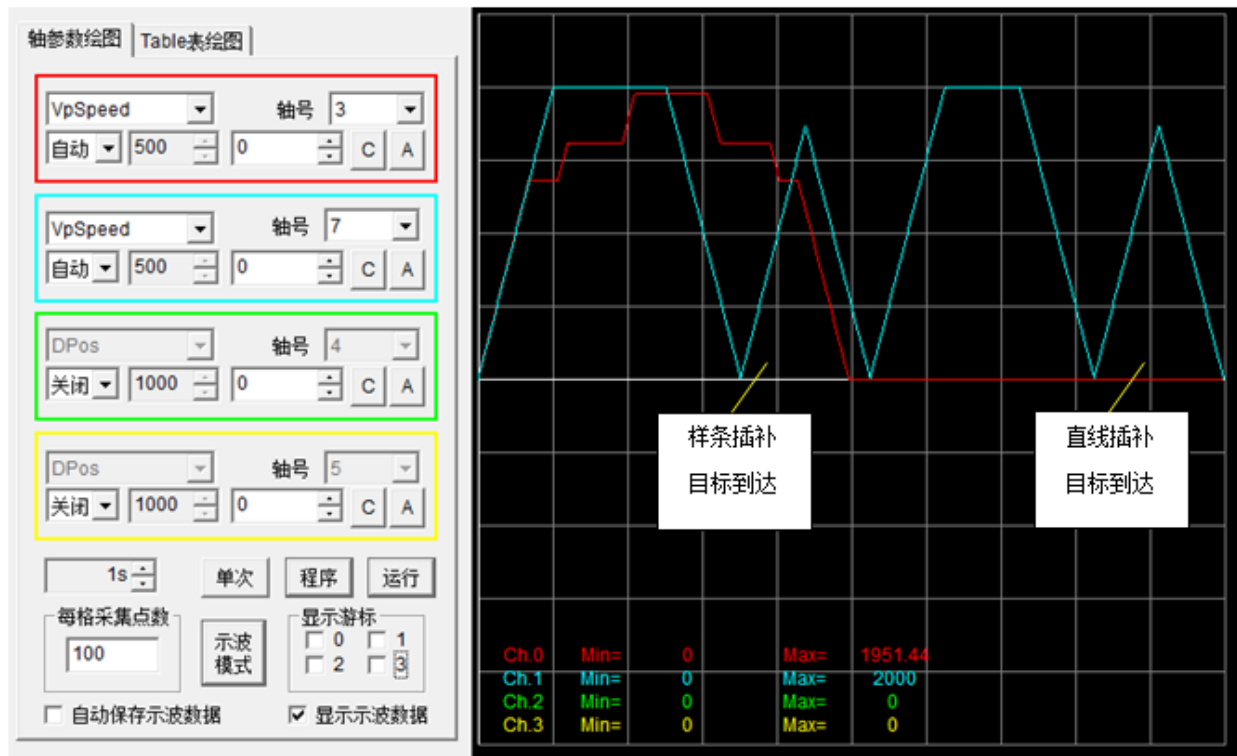
直线插补与样条插补采用相同的加减速及最大速度参数，在 AT 中运行，运行过程中直线插补速度曲线（青色）与样条插补速度曲线（红色）如下图所示。可以看出，直线在每个转折点 1、2、3、4 均需减速至 0，然后开始下一段运动，大大降低了搬运效率。样条插补轨迹整个运动过程比较平滑，不会像直线插补那样在转折点处减速至 0，因此可以大大提高效率以及运动的平稳性。



运动轨迹

（红色：样条插补合轴速度，青色：直线插补合轴速度）

为限制样条插补在转折较为急剧之处的冲击，设置样条插补中合轴轴参 **Jerk** 限制值为 5000，AT 中运行速度曲线如图所示（直线插补速度曲线（青色）与样条插补速度曲线（红色））。由图可知，在转折较为急剧之处，通过降低该段的最大速度，保证运行中的冲击不超过设定值 5000。



运动轨迹

(红色：样条插补合轴速度，青色：直线插补合轴速度)

若要从 B 点运行至 A 点，则只需把样条节点 1'、2'、3'、4'、5'、6'顺序倒置，存入 pArrayPos 数组中，即可生成回程的样条参数模型。这里不再详述。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.4.4.2 HMC_MoveSplineNDEx (高级样条插补)

函数原型

```
UDINT HMC_MoveSplineNDEx(USINT ucAxisID,
    POINTER TO USINT pAxis,
    USINT ucEndPointAxisNum,
    USINT ucFollowAxisNum,
    POINTER TO LREAL pArrPos,
    UDINT uiSplineNum,
    POINTER TO LREAL pArrSplineData,
    POINTER TO LREAL pArrSlenthData,
```

POINTER TO LREAL pArrJerkData,

LREAL dSpeed)

功能说明

以当前点为起点，执行 N 轴样条插补运动（相对坐标/增量坐标）。

可参见 [HMC MoveSplineND（样条插补）](#) 指令，该指令与 HMC_MoveSplineND 的区别在于：执行该指令时会把插补合轴轴参 Speed 修改为函数参数中的 dSpeed 值，以该值为插补合速度进行样条插补运动。

参数

输入参数	类型	参数描述
ucAxisID	USINT	插补合运动轴号
pAxis	POINTER TO USINT	插补分运动轴号数组首地址。排列顺序为先端点轴后跟随轴。
ucEndPointAxisNum	USINT	端点轴的个数。端点轴合运动的速度近似为插补合轴 Speed。取值范围：2 或 3
ucFollowAxisNum	USINT	跟随轴的个数。
pArrPos	POINTER TO LREAL	样条曲线节点坐标，2 维数组。第一维长度 length1 为坐标轴个数（维度），第二维长度 length2 为样条曲线节点数目。有如下关系： length1 = n length2 = uiSplineNum 其中 n = ucEndPointAxisNum + ucFollowAxisNum
uiSplineNum	UDINT	pArrPos 中的样点个数，1~1000 个。
pArrSplineData	POINTER TO LREAL	数组首地址，HMC_CalcMoveSplineNDdata 中计算得到的样条曲线各段的多项式系数参数。
pArrSlenthData	POINTER TO LREAL	数组首地址，HMC_CalcMoveSplineNDdata 中计算得到的 各节点相对于起点的样条曲线长度序列。
pArrJerkData	POINTER TO LREAL	数组首地址，HMC_CalcMoveSplineNDdata 中计算得到的 样条曲线各段冲击参数。
dSpeed	LREAL	合轴目标速度（插补轮廓速度）

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	生成数据成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

应用示例

参见 [HMC MoveSplineND（样条插补）](#)。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.4.4.3 HMC_CalcMoveSplineNDdata（生成样条数据）

函数原型

UDINT HMC_CalcMoveSplineNDdata(POINTER TO LREAL pArrPos,
UDINT uiSplineNum,

```

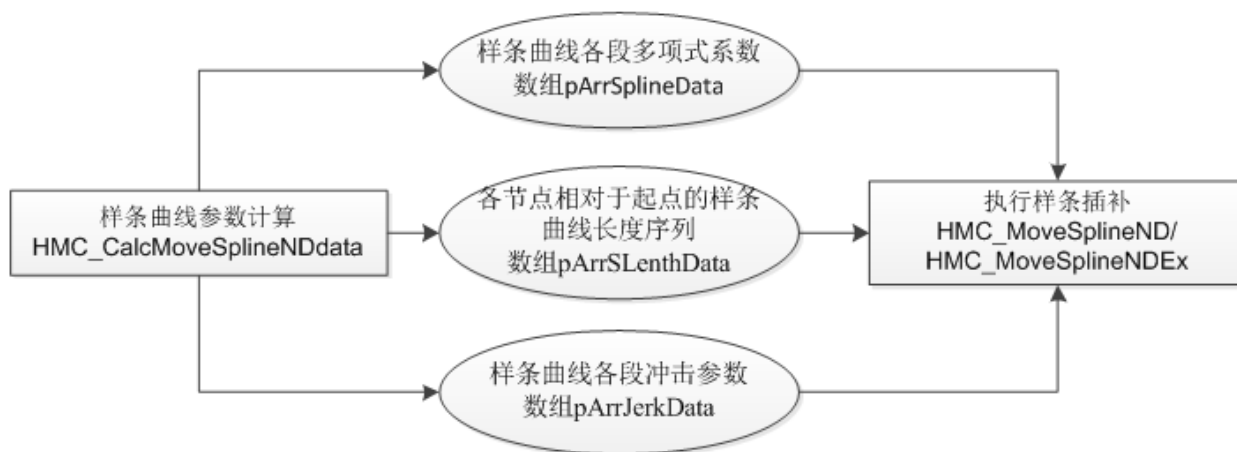
USINT ucEndPointAxisNum,
USINT ucFollowAxisNum,
POINTER TO LREAL pArrSplineData,
UDINT uiSplineDataSize,
POINTER TO LREAL pArrSLenthData,
POINTER TO LREAL pArrJerkData)

```

功能说明

样条曲线由于具有二阶连续可导性，可有效的减小插补运动过程中因方向变化而引入的速度/加速度跃变，从而降低机械冲击，实现平滑的运动效果。

该函数生成 **N** 轴插补的三次样条曲线，计算得到的样条曲线参数被保存在用户指定的数组中，用户不必关心该数组中的具体内容，后续使用样条插补指令 **HMC_MoveSplineND** / **HMC_MoveSplineNDEx** 调用该数组中的数据，即可实现 **N** 轴样条插补功能。



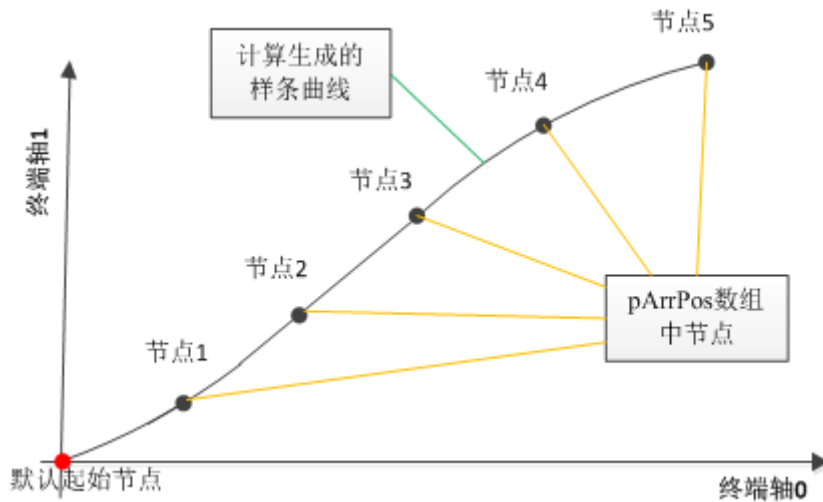
样条插补功能

函数根据用户指定的样条曲线节点坐标 **pArrPos**（起点默认为原点）、节点个数 **uiSplineNum**，计算生成样条曲线的参数。参数包括各段的多项式系数、各节点相对于起点的样条曲线长度序列、样条曲线各段冲击参数，分别保存在用户指定的数组 **pArrSplineData**、**pArrSLenthData**、**pArrJerkData** 中。

pArrPos 数组中的坐标由各节点坐标在端点轴（**EndPointAxis**）与跟随轴（**FollowAxis**）上的分量组成，端点轴与跟随轴的数目由 **ucEndPointAxisNum** 与 **ucFollowAxisNum** 确定。数组中坐标排列顺序：首先是各端点轴坐标，然后是跟随轴坐标。建议定义该数组为二维数组，这样数组元素 **pArrPos[i,j]** 便表示第 **j** 个节点在 **i** 号轴上的坐标分量。详见示例。

使用 **N** 轴样条插补功能最常用的是实现 2 维或 3 维样条插补，可如下设置参数：

- （1）2 轴样条插补：跟随轴 **ucFollowAxisNum** = 0，端点轴 **ucEndPointAxisNum**=2。



2 轴样条插补

(2) 3 轴样条插补：跟随轴 $ucFollowAxisNum = 0$ ，端点轴 $ucEndPointAxisNum=3$ 。

使用样条插补指令 HMC_MoveSplineND / HMC_MoveSplineNDEx 作插补运动时，插补合轴按照各端点轴合运动的轨迹进行，而不受跟随轴的影响。各端点轴速度的合速度约等于插补合轴速度 Speed。

端点轴与跟随轴的概念可类比 N 轴直线插补 (HMC_MoveND) 的线性轴与独立轴（如果仅使用常规的 2 轴/3 轴样条插补，可不作了解），详见 [HMC_MoveSplineND（样条插补）](#)。

函数首先计算出样条曲线长度序列 pArrSlenthData，然后使用该长度序列计算各个分轴（端点轴、跟随轴）的样条曲线模型参数。样条长度序列 pArrSlenthData 由节点坐标在端点轴上的分量计算得到，表示样条插补指令 HMC_MoveSplineND / HMC_MoveSplineNDEx 中合轴在各节点处的位移，合轴速度 speed 近似等于各端点轴合运动的速度。

参数

输入参数	类型	参数描述
pArrPos	POINTER TO LREAL	样条曲线节点坐标，2 维数组 pArrPos[length1][length2] 首地址。数组第一维长度 length1 为坐标轴个数（维度），第二维长度 length2 为样条曲线节点数目。有如下关系： length1 = n length2 = uiSplineNum 其中 n = ucEndPointAxisNum + ucFollowAxisNum
uiSplineNum	UDINT	pArrPos 中的样点个数，1~1000 个。
ucEndPointAxisNum	USINT	端点轴的个数，样条曲线长度序列 pArrSlenthData 由节点坐标在端点轴上的分量计算得到。 端点轴合运动的速度近似为插补合轴 Speed。 取值范围：2 或 3
ucFollowAxisNum	USINT	跟随轴的个数。跟随轴的样条曲线模型是根据样条曲线长度序列 pArrSlenthData 和样条节点在跟随轴上的坐标分量计算得到的。
pArrSplineData	POINTER TO LREAL	用户指定的一维数组 pArrSplineData[length] 首地址，用于保存计算得到的样条曲线各段的多项式系数参数数据。应满足： 数组长度 length $\geq n \cdot uiSplineNum \cdot 4$ 其中 n = ucEndPointAxisNum + ucFollowAxisNum
uiSplineDataSize	UDINT	pArrSplineData 数组长度。
pArrSlenthData	POINTER TO	用户指定的一维数组 pArrSlenthData[length] 首地址，用于保存计算得到的各节点

输入参数	类型	参数描述
	LREAL	相对于起点的样条曲线长度序列。应满足： 数组总长度 $\geq$ uiSplineNum
pArrJerkData	POINTER TO LREAL	用户指定的数组首地址，用于保存计算得到的 样条曲线各段冲击参数。应满足： 数组长度 length $\geq$ uiSplineNum

返回值

返回值	类型	参数描述
0	UDINT	成功
非零	UDINT	错误码

指令类型

非轴运动缓存指令。

■ 补充说明

(1) 各节点之间的空间距离尽量不要相差过大，否则在相差较大的节点附近的数值计算偏差可能会影响精度。

(2) 当样条曲线上各个节点在经过原点的直线上时，生成的样条曲线为直线。

(3) 如果只有一个节点，则生成的样条曲线为连接原点与该节点的线段。因此 N 轴样条插补可完成 2 轴、3 轴、N 轴直线插补的功能。

应用示例

示例 1：2 轴样条插补

端点轴数量为 2 个，跟随轴数量为 0 个。2 个端点轴轴号为 0、1，分别对应 X、Y 轴。节点数目为 3 个，节点坐标分别为 (500,500)、(500,1500)、(1500,1000)、(2000,0)。使用 HMC_CalcMoveSplineNDdata 生成样条曲线参数，然后使用该参数调用 HMC_MoveSplineND 执行样条插补指令(当前点为原点)，生成样条曲线轨迹。程序如下：

(***首先定义大小为 pArrPos[2][3]的节点数组，然后对节点数组赋值***)

pArrPos[0][0] := 500; (*第 1 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[0][1] := 500; (*第 2 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[0][2] := 1500; (*第 3 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[0][3] := 2000; (*第 4 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[1][0] := 500; (*第 1 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[1][1] := 1500; (*第 2 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[1][2] := 1000; (*第 3 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[1][3] := 0; (*第 4 个节点在轴 1(Y 轴)上的坐标分量*)

(*定义数组 pArrSplineData[32], pArrSlenthData[4], pArrJerkData [4]，保存计算得到的样条参数*)

(*调用函数生成样条曲线参数*)

```
HMC_CalcMoveSplineNDdata(pArrPos, 4, 2, 0, pArrSplineData, 32, pArrSlenthData, pArrJerkData);
```

(*定义轴号数组 pArrAxis[2]*)

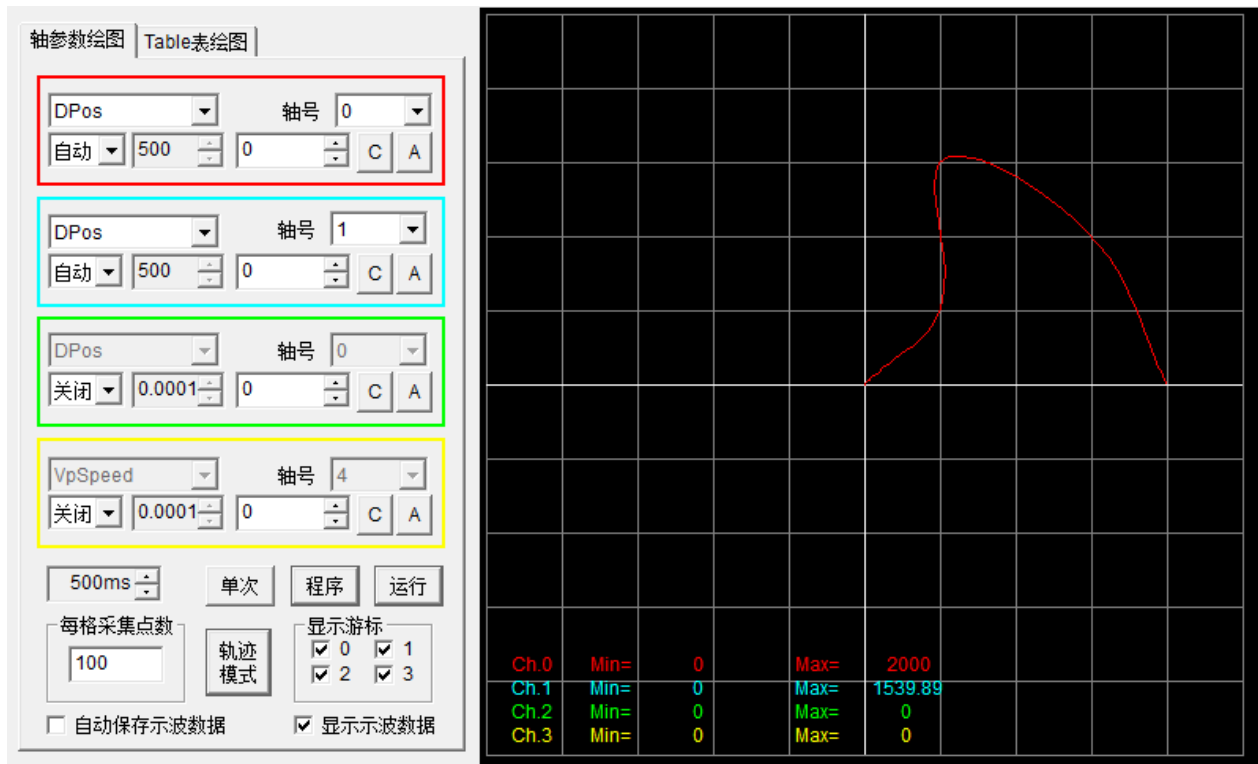
```
pArrAxis[0] := 0; (*第 1 个端点轴轴号*)
```

```
pArrAxis[1] := 1; (*第 2 个端点轴轴号*)
```

(*执行样条插补指令*)

```
HMC_MoveSplineND(2,pArrAxis,2,0, pArrPos,4,  
pArrSplineData, pArrSlenthData, pArrJerkData);
```

AT 示波器运行结果如下:



运动轨迹

示例 2: 3 轴样条插补曲线

端点轴数量为 3 个, 跟随轴数量为 0 个。3 个端点轴轴号为 0、1、2, 分别对应 X、Y 轴。节点数目为 4 个, 节点坐标分别为 (500,500,500)、(500,1500,500)、(1500,1000,1000)、(2000,0,0)。使用 HMC_CalcMoveSplineNDdata 生成样条曲线参数, 然后使用该参数调用 HMC_MoveSplineND 执行样条插补指令(当前点为原点), 生成样条曲线轨迹。程序如下:

(****首先定义大小为 pArrPos[3][3] 的节点数组, 然后对节点数组赋值****)

pArrPos[0][0] := 500; (*第 1 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[0][1] := 500; (*第 2 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[0][2] := 1500; (*第 3 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[0][3] := 2000; (*第 4 个节点在轴 0(X 轴)上的坐标分量*)

pArrPos[1][0] := 500; (*第 1 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[1][1] := 1500; (*第 2 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[1][2] := 1000; (*第 3 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[1][3] := 0; (*第 4 个节点在轴 1(Y 轴)上的坐标分量*)

pArrPos[2][0] := 500; (*第 1 个节点在轴 2(Z 轴)上的坐标分量*)

pArrPos[2][1] := 500; (*第 2 个节点在轴 2(Z 轴)上的坐标分量*)

pArrPos[2][2] := 1000; (*第 3 个节点在轴 2(Z 轴)上的坐标分量*)

pArrPos[2][3] := 0; (*第 4 个节点在轴 2(Z 轴)上的坐标分量*)

(*定义数组 pArrSplineData[48], pArrSlenthData[4], pArrJerkData [4], 保存计算得到的样条参数*)

(*调用函数生成样条曲线参数*)

HMC_CalcMoveSplineNDdata(pArrPos,4,3,0, pArrSplineData,48, pArrSlenthData, pArrJerkData);

(*定义轴号数组 pArrAxis[3]*)

pArrAxis[0] := 0; (*第 1 个端点轴轴号*)

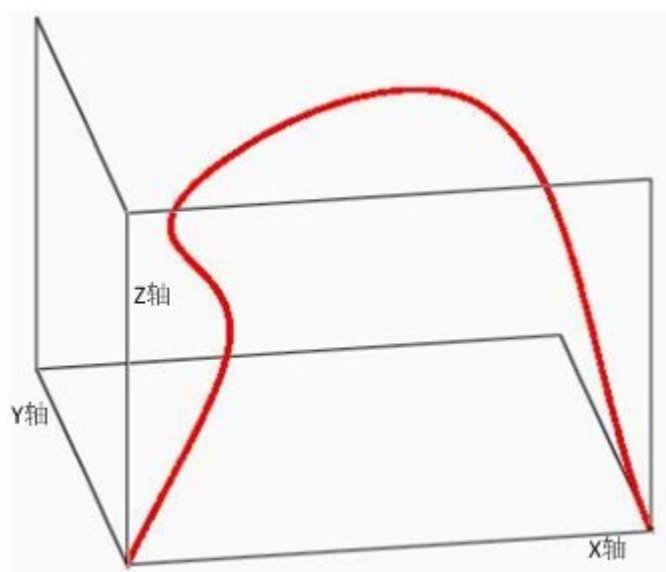
pArrAxis[1] := 1; (*第 2 个端点轴轴号*)

pArrAxis[2] := 2; (*第 3 个端点轴轴号*)

(*执行样条插补指令*)

HMC_MoveSplineND(3,pArrAxis,3,0, pArrPos,4, pArrSplineData, pArrSlenthData, pArrJerkData);

运行结果如下:



三维轨迹

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.4.5 电子齿轮

3.4.1.4.5.1 HMC_Gear（电子齿轮）

函数原型

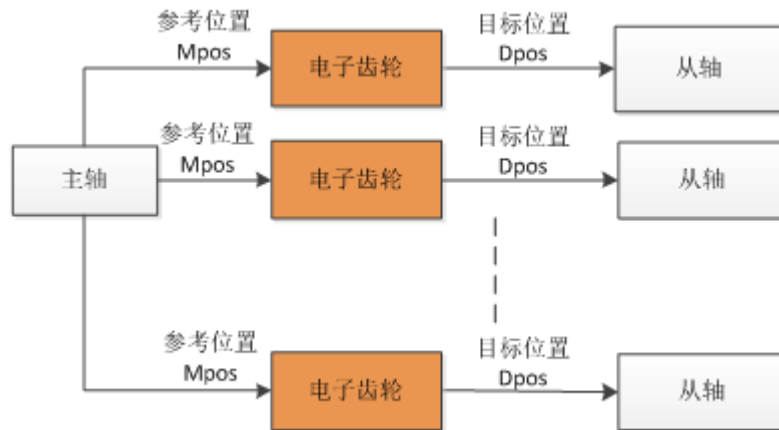
```
UDINT HMC_Gear(USINT ucSlaveAxis,  
USINT ucMasterAxis,  
DINT iRatioNumerator,  
DINT iRatioDenominator)
```

功能说明

电子齿轮主要用于实现无轴传动。无轴传动技术具有传动精度高、结构简单、传动比范围宽、调整方便等优点，因此可以代替传统的机械传动实现准确的传动关系。在印染、纺织、造纸、齿轮加工机床内联传动等要求实现多轴同步传动的领域，无轴传动技术有着广阔的应用前景。

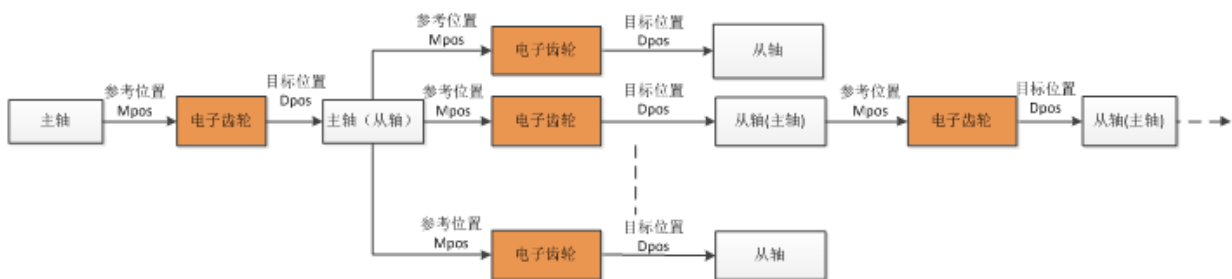


串联式电子齿轮结构图



并联式电子齿轮结构图

电子齿轮使用中有串联式与并联式两种基本结构，如上图所示。串联式结构中从轴作为下一级电子齿轮的主轴，并联式结构中各从轴拥有共同的主轴。实际使用中可根据需要选用，或使用该两种基本结构组合构建复合式电子齿轮结构。



复合式电子齿轮结构图

使用电子齿轮后，从轴 Dpos 增量与主轴 Mpos 增量满足如下关系：

$$\Delta Dpos_S = \Delta Mpos_M * \left(\frac{iRatioNumerator}{iRatioDenominator} \right)$$

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
ucMasterAxis	USINT	主轴轴号
iRatioNumerator	DINT	齿轮比分子
iRatioDenominator	DINT	齿轮比分母（不可为 0）

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

(1) 在该指令执行过程中，可以通过调用 HMC_SetGearRatio 函数动态修改齿轮比例；

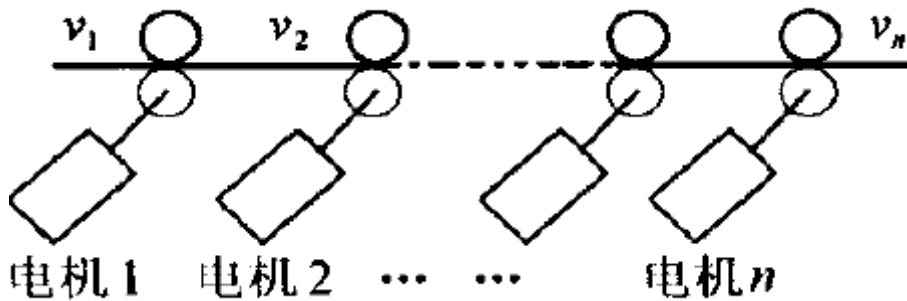
(2) 从数学关系上看, HMC_Gear 和 HMC_Superimpose 均是 HMC_SuperimposeEx 的特例;

(3) HMC_Gear 和 HMC_Superimpose 配合使用可实现 HMC_SuperimposeEx 的所有功能。

应用举例

示例 1: 连续工艺过程装备无轴控制技术

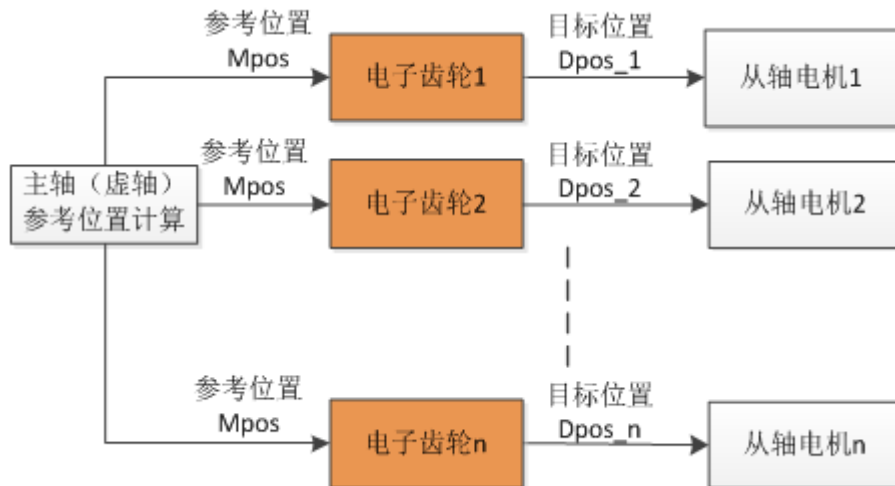
许多连续工艺过程的工业设备, 如热连轧机和冷连轧机、造纸机、聚合物加工生产线、纺织染整机械等等, 对传动系统的要求是相似的, 即采用多单元分布传动。单元数量往往多至几十个, 各单元由单独电动机驱动; 从工艺过程和保持 2 个单元间给定张力的条件出发, 各单元机之间的速度关系必须保持严格不变, 必须在某一精度下长时间地稳定所加工带材(金属、纸张、聚合胶片、纺织物等)的线速度; 各单元机通过同步传动, 保持加工时的适度张力, 避免加工过程中带材伸长或产生折皱。



连续工艺过程装备传动示意图

这类设备的传动结构如上图所示, 通常采用并联式电子齿轮箱结构, 通过共同加工的带材负载以及通过公用的速度给定和各个分部速度比值控制, 使分部系统相互耦合起来。

主轴为虚轴, 用于产生公共参考位置, 通过对该公共参考位置进行电子齿轮变换产生各从轴目标位移, 驱动各从轴按预定位置及速度关系同步运转。



连续工艺过程装备传动电子齿轮实现方式

如果某个分部内部还存在传动关系, 则可在分部中构建子电子齿轮控制系统, 与整个系统组成复合式电子齿轮控制系统。

示例 2：无轴传动实现螺纹车削

数控车床主轴电机为 0 号轴，轴向进给电机为 1 号轴，进给丝杠导程为 l_f ，现欲车削导程为 l_x 的螺纹，可用电子齿轮功能实现。

计算可得，加工该导程的螺纹时进给电机与主轴的传动比为：

$$\frac{l_x}{l_f}$$

把该数值化简为最简分数形式，设：

$$\frac{l_x}{l_f} = \frac{iRatioNumerator}{iRatioDenominator}$$

程序如下（ST 语言）（设主轴电机、轴向进给电机、径向进给电机分别为轴 0、1、2）：

(**设置轴类型为步进轴**)

```
HMC_SetAxisType(Axis0.AxisID, 10);
```

```
HMC_SetAxisType(Axis1.AxisID, 10);
```

```
HMC_SetAxisType(Axis2.AxisID, 10);
```

(**设置各轴速度、加减速参数（略）**)

(**设置各轴速度、加减速参数（略）**)

(**运动至进刀点（略）**)

(**径向进给电机给定螺纹深度（略）**)

(*投送 HMC_Gear 指令，1 号轴（轴向进给电机）为从轴，0 号轴（主轴电机）为主轴，齿轮比为 $iRatioNumerator/iRatioDenominator$ *)

```
HMC_Gear(Axis1.AxisID, Axis0.AxisID, iRatioNumerator, iRatioDenominator);
```

(*驱动主轴运转，轴向进给电机将在电子齿轮的驱动下同步运动，完成螺纹车削*)

示例 3：H 形同步齿形带并联机构运动控制（参见 HMC_Superimpose 中例 1）。

3.4.1.4.5.2 HMC_SetGearRatio（设置电子齿轮比例）

函数原型

```
UDINT HMC_SetGearRatio(USINT ucSlaveAxis,  
DINT iRatioNumerator,  
DINT iRatioDenominator)
```

功能说明

设置齿轮比例，在 HMC_Gear 指令启动后，可实时调用该指令动态改变运转中电子齿轮比例，从而达到动态改变电子齿轮的效果。

该指令只对当前正在运行的 HMC_Gear 指令起作用。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
iRatioNumerator	DINT	齿轮比分子
iRatioDenominator	DINT	齿轮比分母

返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令。

3.4.1.4.5.3 HMC_GetGearMaster（获取电子齿轮主轴）

函数原型

```
INT HMC_GetGearMaster(USINT ucSlaveAxis)
```

功能说明

获取电子齿轮的主轴号。该指令只可获取当前正在运行的 HMC_Gear 指令主轴号。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号

返回值

返回值	类型	参数描述
电子齿轮主轴号	INT	成功
-1	INT	当前未配置电子齿轮或函数参数错误

指令类型

非轴运动缓存指令。

3.4.1.4.5.4 HMC_GetGearRatio（获取电子齿轮比例）

函数原型

LREAL HMC_GetGearRatio(USINT ucSlaveAxis)

功能说明

获取电子齿轮的齿轮比。该指令只可获取当前正在运行的 HMC_Gear 指令齿轮比。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号

返回值

返回值	类型	参数描述
电子齿轮比例	LREAL	-

指令类型

非轴运动缓存指令。

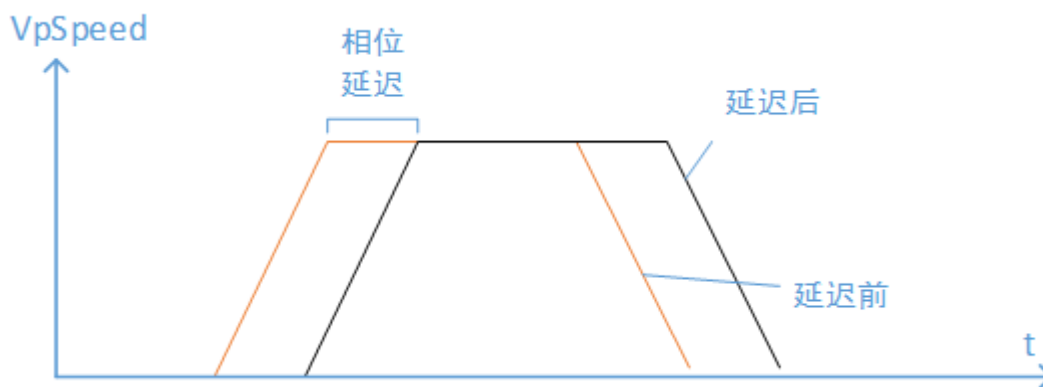
3.4.1.4.5.5 HMC_SetGearPhaseOffset（设置电子齿轮相位延迟）

函数原型

UDINT HMC_SetGearPhaseOffset(USINT ucAxisID, DINT iPhaseOffset)

功能说明

该指令可调整电子齿轮从轴的跟随相位，即把原来的从轴轨迹在时间轴上作一个相位延迟。调整后的从轴轨迹比原轨迹滞后 iPhaseOffset 个伺服周期（最大为 128）。如图所示：



从轴相位延迟轨迹

可通过电子齿轮间接实现其它指令的相位延迟。如凸轮指令，可把凸轮主轴作为 1:1 电子齿轮的从轴，通过调整电子齿轮的相位延迟量间接实现凸轮的相位延迟。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	从轴轴号
iPhaseOffset	DINT	相位延迟量，0~128，单位为伺服周期

返回值

返回值	类型	参数描述
成功: 0 错误: 错误码	UDINT	--

指令类型

非轴运动缓存指令。

■ 补充说明

必须在电子齿轮指令执行前调用 `HMC_SetGearPhaseOffset` 设定好所需的相位延迟，无法在电子齿轮指令执行过程中动态调整相位延迟。

3.4.1.4.5.6 HMC_GetGearPhaseOffset（获取电子齿轮相位延迟）

函数原型

DINT HMC_GetGearPhaseOffset (USINT ucAxisID)

功能说明

获取电子齿轮相位延迟设定量。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	从轴轴号，0~63

返回值

返回值	类型	参数描述
相位延迟量	DINT	单位为伺服周期

指令类型

非轴运动缓存指令。

3.4.1.4.5.7 HMC_SetGearTransationSwitch（电子齿轮过渡段开关）

函数原型

UDINT HMC_SetGearTransationSwitch(USINT ucAxisID, BOOL ucValue)

功能说明

开启/关闭电子齿轮平滑功能。开关开启时，电子齿轮在启动阶段或电子齿轮比发生切换时，会自动插入一段平滑过渡过程，过渡过程采用轴自身的加减速，防止机械冲击。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucValue	BOOL	0: 关闭 1: 开启

返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

指令类型

非轴运动缓存指令。

3.4.1.4.6 电子凸轮

3.4.1.4.6.1 HMC_Cam（电子凸轮）

函数原型

```
UDINT HMC_Cam(USINT ucSlaveAxis,  
USINT ucMasterAxis,  
POINTER TO LREAL pArrPos,  
UDINT uiPosNum,  
LREAL dScale,  
USINT ucMode,  
LREAL dTrigMes)
```

功能说明

通过选取目标运动轨迹（二维）上的离散坐标点，使用电子凸轮功可以通过主从轴联动的方式逼近目标运动轨迹。逼近方式为“以折代曲”，即用连接上述离散坐标点的折线段去逼近目标曲线。

电子凸轮可完成与机械式凸轮相似的功能，而没有机械式凸轮设计难度大、加工成本高、运动高副易磨损等缺点，在许多场合是机械式凸轮的理想替代品。电子凸轮功能也可用来进行不规则曲线的逼近。与 HMC_ClcMoveLinkData 或 HMC_ClcFlexLinkData 指令配合使用，还可实现追剪、飞剪、旋切等应用场景所需的动作流程。

电子凸轮有主轴、从轴之分，凸轮主从轴位置预先存储在主从轴位置数组 pArrPos 中。pArrPos 为二维数组，第一维 pArrPos[0]为主轴位置，第二维 pArrPos[1]为从轴位置，二者为一一对应关系。凸轮启动时，根据不同的启动方式设定凸轮主轴起点位置；凸轮运行过程中，主轴运行正常的运动控制指令，控制器根据主轴当前 Mpos 值和 pArrPos 中数据计算出从轴的理论位置，驱动从轴进行相应的联动动作，同时根据当前主轴 Mpos 值计算判断是否满足凸轮撤销条件。

按主轴位置的边界范围划分，凸轮可分为：

（1）单次凸轮——凸轮启动后，若凸轮主轴位置在有效行程范围内，则凸轮可保持正常联动运行（主轴单向、往复运动均可。当主轴在有效行程范围内往复运动时，从轴也会进行往复性的联动动作），主轴超过有效行程范围时凸轮指令将自动撤销。

（2）循环凸轮——凸轮主轴位置没有边界范围限制，启动后如果不执行相应的撤销指令，凸轮功能将一直有效。

按凸轮启动条件的触发方式划分，凸轮可分为如下几种：

（1）立即启动——凸轮指令投放后立即启动凸轮功能，启动位置为主轴当前 Mpos 位置；

（2）固定位置启动（到位启动）——凸轮指令投放后，当凸轮主轴 Mpos 从负向跨过凸轮启动位置时，凸轮功能启动，启动位置为该固定位置；

（3）事件启动——凸轮指令投放后，当设定的某高速捕获通道被触发时启动凸轮，启动位置为高速捕

获通道所捕获到的主轴 Mpos 位置；

（4）DI 启动—— 凸轮指令投放后，当设定的某 DI 信号为 1 时启动凸轮，启动位置为程序检测到 DI 为 1 时主轴 Mpos 位置。

事件启动凸轮与 DI 启动凸轮启动方式类似，均为外部信号触发。不同的是事件启动凸轮可利用高速捕获通道记录下 DI 信号被触发瞬间的精确位置，因而可获得更为精准的启动位置，相比 DI 启动凸轮具有更高的精度。

凸轮主轴的一个行程长度 Delta 为凸轮主轴位置数组的最后一个元素值减去第一个元素值，即：Delta = pArrPos[0,uiPosNum-1] - pArrPos[0,0]，其中 uiPosNum 为位置个数。设凸轮起点为 Mpos0，则单次凸轮的有效行程范围为：[Mpos0, Mpos0+ Delta)，单次凸轮主轴位置超出该行程范围时，凸轮将撤消；凸轮主轴在有效行程范围内时，主轴可进行单向、往复运动，当主轴在有效行程范围内往复运动时，从轴也会进行往复性的联动动作。

对于循环凸轮，当凸轮主轴位置越过一个凸轮主轴行程长度 Delta 时，将会进入下一个凸轮行程，此时从轴将会以上一个凸轮行程结束时的从轴位置为起点，在此基础上作增量位移计算，重复上一个凸轮行程的联动动作，周而复始。凸轮主轴也可做单向、往复运动。如果需要撤消循环凸轮，可执行 Hmc_Cancel 立即撤消，或执行 HMC_CancelREPCam 指令在凸轮主轴行程的整数倍处撤消，详细使用方法请参考相关指令。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
ucMasterAxis	USINT	主轴轴号
pArrPos	POINTER TO LREAL	位置数组首地址（二维）。在二维数组 pArrPos[2,uiPosNum]中，pArrPos[0]为主轴位置数组，pArrPos[1]为从轴位置数组。
uiPosNum	UDINT	位置个数，应大于等于 3
dScale	LREAL	凸轮从轴位置数据比例因子。凸轮从轴实际位置输出 = 从轴位置数组设定输出 * dScale
ucMode	USINT	0: 单次立即启动； 1: 单次固定位置启动； 2: 单次事件启动； 3: 单次 DI 启动； 4: 循环立即启动； 5: 循环固定位置启动； 6: 循环事件启动； 7: 循环 DI 启动。
dTrigMes	LREAL	启动信息： 1、凸轮为位置启动时，该值表示主轴的启动位置； 2、凸轮为事件触发启动时，表示某高速捕获通道号； 3、凸轮为 DI 为 1 触发启动时，表示 DI 通道（0~63）。

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

(1) 主轴位置数组 **pArrPos[0]** 中的元素必须是单调递增的（不可有相同数），数据正负均可。凸轮运行过程中不能修改主从轴位置数据，否则可能会造成系统错误！

(2) 凸轮为事件触发启动时，**dTrigMes** 表示某高速捕获通道，所以应事先配置一个高速捕获通道来捕获启动（**HMC_CaptureConfig** 功能块中的 **CaptureAxis** 必须为凸轮主轴号）。

(3) 凸轮启动时的参考启动点、运行时用于计算的动态坐标点、撤消时的参考坐标点均以凸轮主轴的 **Mpos** 值为参考基准。当主轴为位置闭环控制时，由于主轴 **Mpos** 来自于外部检测装置，为避免在凸轮有效行程范围的边界处由于 **Mpos** 的小扰动所造成的凸轮意外撤消，撤消条件也会兼顾主轴 **Dpos** 值。

(4) 对于单次凸轮，当 **Mpos** 来自外部检测装置时，为避免小扰动所引起的凸轮异常撤消，应确保凸轮正常工作区距离凸轮边界位置有一定余量（保证 **Mpos** 的扰动不会超出凸轮边界位置）。

应用举例

(1) 单次立即启动凸轮(**ucMode = 0**)

定义 **ArrPos** 为主从轴位置数组，示例程序如下（**ST** 语言）。通过选取直线上三个坐标点，该程序实现凸轮主轴与从轴联动运行的轨迹为倾角为 **45** 度的直线。

凸轮指令投放运行后，以当前主轴 **Mpos** 值为启动位置立即启动凸轮功能，向凸轮主轴投放 **Hmc_Move** 指令驱动凸轮主轴运动，此时凸轮从轴作相应的联动动作，当主轴在凸轮有效行程范围内往复运行时，从轴也作往复性的联动动作（主从轴起始位置均为 **0**）。

(*主从轴位置数组赋值*)

ArrPos[0,0] := 0;

ArrPos[1,0] := 0; (*点 1 坐标*)

ArrPos[0,1] := 500;

ArrPos[1,1] := 500; (*点 2 坐标*)

ArrPos[0,2] := 1000;

ArrPos[1,2] := 1000; (*点 3 坐标*)

pArrPos := ADR(ArrPos); (*取主从轴位置数组 ArrPos 首地址*)

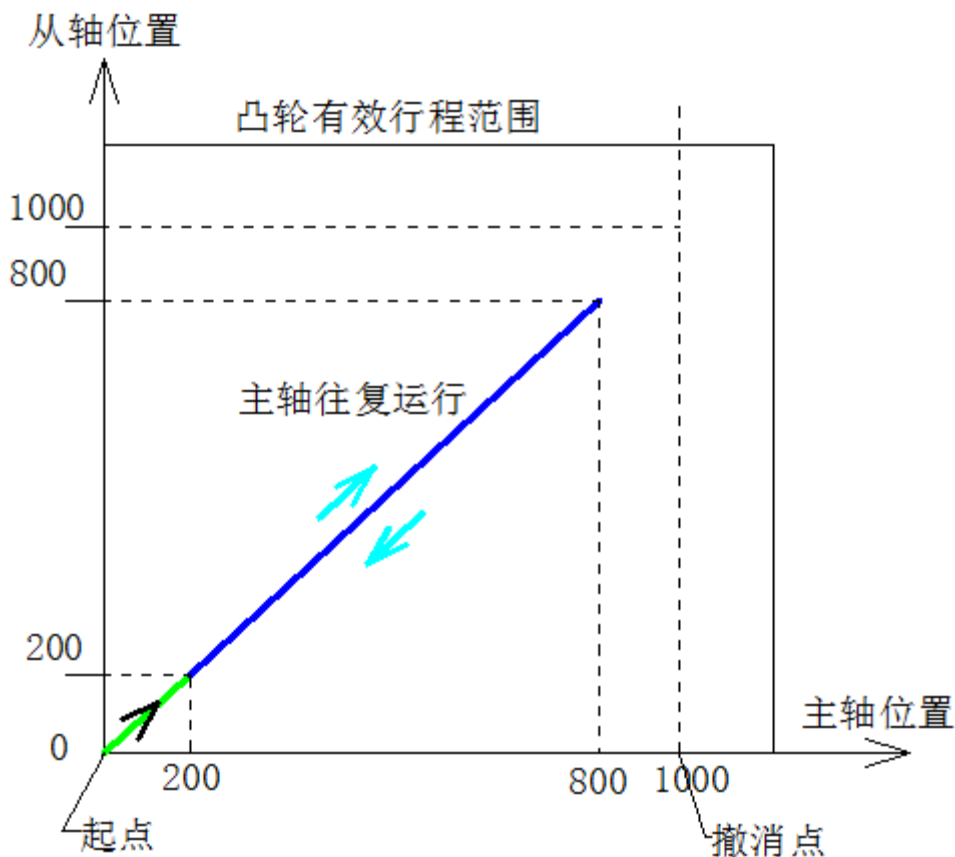
HMC_Cam(Axis1.AxisID,Axis0.AxisID,pArrPos,3,1,0,0); (*投放单次立即启动凸轮指令，0 轴为主轴，1 轴为从轴，凸轮主轴行程范围为*)

(*向主轴投送 **Move** 指令，驱动凸轮主轴往复运动。此时凸轮从轴将进行相应联动动作*)

HMC_Move(Axis0.AxisID,800);

HMC_Move(Axis0.AxisID,-600);

```
HMC_Move(Axis0.AxisID,600);
HMC_Move(Axis0.AxisID,-600);
HMC_Move(Axis0.AxisID,600);
```



单次立即启动凸轮联动轨迹

(2) 单次固定位置启动凸轮(ucMode = 1)

示例程序如下（ST 语言），该程序凸轮主轴与从轴联动运行的轨迹为倾角为 45 度的直线。向凸轮主轴投放 Hmc_Move 指令驱动凸轮主轴运动，当凸轮主轴达到启动位置时，凸轮指令启动运行，凸轮从轴开始作相应的联动动作。当主轴越过凸轮有效行程边界时，凸轮功能自动撤销（主从轴起始位置均为 0）。

(**主从轴位置数组赋值**)

```
ArrPos[0,0] := 0;
```

```
ArrPos[1,0] := 0;    (*点 1 坐标*)
```

```
ArrPos[0,1] := 500;
```

```
ArrPos[1,1] := 500;  (*点 2 坐标*)
```

```
ArrPos[0,2] := 1000;
```

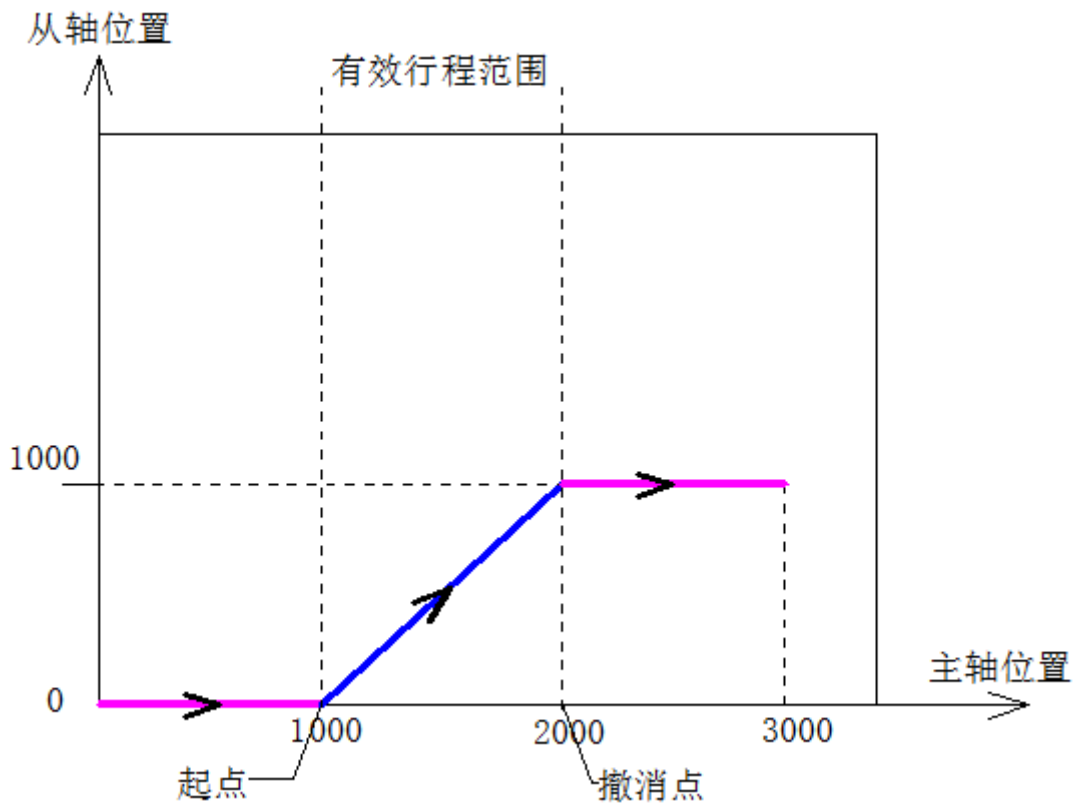
```
ArrPos[1,2] := 1000; (*点 3 坐标*)
```

(*投放单次到位启动凸轮指令，0 轴为主轴，1 轴为从轴，启动位置为 1000*)

```
HMC_Cam (Axis1.AxisID,Axis0.AxisID,ADR(ArrPos2),101,1,1,1000);
```

(*向主轴投送 Move 指令，驱动凸轮主轴单向运动。单次凸轮将启动——运行——撤销*)

```
HMC_Move(Axis0.AxisID,3000);
```



单次固定位置启动凸轮联动轨迹

(3) 单次事件启动凸轮(ucMode = 2)

示例程序如下（ST 语言），该程序实现凸轮主轴与从轴联动运行的轨迹为正弦曲线。向凸轮主轴投放 Hmc_Move 指令驱动凸轮主轴运动，当 0 号通道 DI 上升沿到来时，0 号高速捕获通道被触发，凸轮指令以捕获到的主轴位置为启动位置启动运行，凸轮从轴开始作相应的联动动作。当主轴越过凸轮有效行程边界时，凸轮功能自动撤销。

(*生成主从轴位置数组*)

```
delta := 3.1415926*2/100;
```

```
FOR n := 0 TO 100 DO
```

```
ArrPos2[0,n] := (delta*n)*150;
```

```
ArrPos2[1,n] := SIN(delta*n)*1000;
```

END_FOR

(*配置高速捕获通道: 0 号通道 DI 触发, 上升沿触发, 待捕获轴为 0 号轴*)

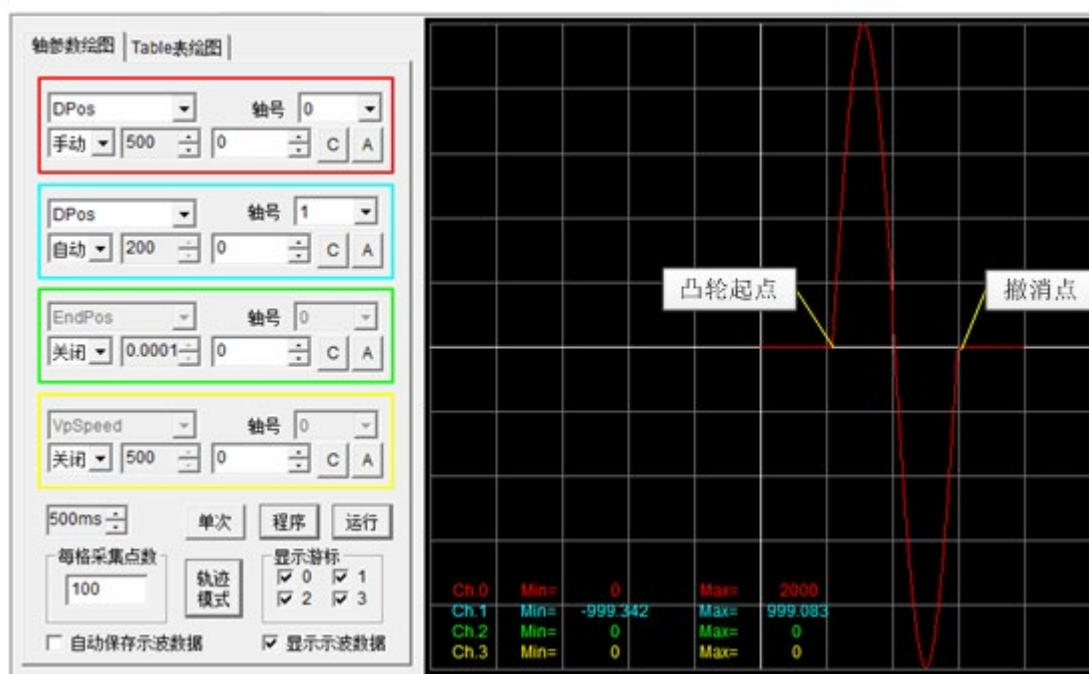
HMC_CaptureConfig (0, 0, 1, 0, Axis0.AxisID, 0, 0, 0);

(*投放单次事件启动凸轮指令, 0 轴为主轴, 1 轴为从轴, 高速捕获通道号为 0*)

HMC_Cam (Axis1.AxisID,Axis0.AxisID,ADR(ArrPos2),101,1,2,0);

(*向主轴投放 Move 指令, 驱动凸轮主轴单向运动。*)

HMC_Move(Axis0.AxisID,2000);



单次事件启动凸轮联动轨迹

(4) 单次 DI 启动凸轮(ucMode = 3)

示例程序如下 (ST 语言), 该程序实现凸轮主轴与从轴联动运行的轨迹为正弦曲线。当 0 号通道 DI 上升沿到来时, 凸轮指令以当前主轴 Mpos 位置为启动位置启动运行。此时若向凸轮主轴投放 Hmc_Move 指令驱动凸轮主轴运动, 凸轮从轴便会开始作相应的联动动作。

(*生成主从轴位置数组*)

delta := 3.1415926*2/100;

FOR n := 0 TO 100 DO

ArrPos2[0,n] := (delta*n)*150;

```
ArrPos2[1,n] := SIN(delta*n)*1000;  
END_FOR
```

(*投放单次 DI 启动凸轮指令，0 轴为主轴，1 轴为从轴，DI 道号为 0*)

```
HMC_Cam (Axis1.AxisID,Axis0.AxisID,ADR(ArrPos2),101,1,3,0);
```

(5) 循环立即启动凸轮(ucMode = 4)

示例程序如下 (ST 语言)，该程序凸轮主轴与从轴联动运行的轨迹为倾角为 45 度的直线。

凸轮指令投放运行后，以当前主轴 Mpos 值为启动位置立即启动凸轮功能，向凸轮主轴投放 Hmc_Move 指令驱动凸轮主轴往复运动，此时凸轮从轴作相应的联动动作，当主轴一个凸轮行程结束时，进入下一个凸轮行程，从轴以上一个凸轮行程的终点为起点作增量计算，执行周期性的联动动作（主从轴起始位置均为 0）。

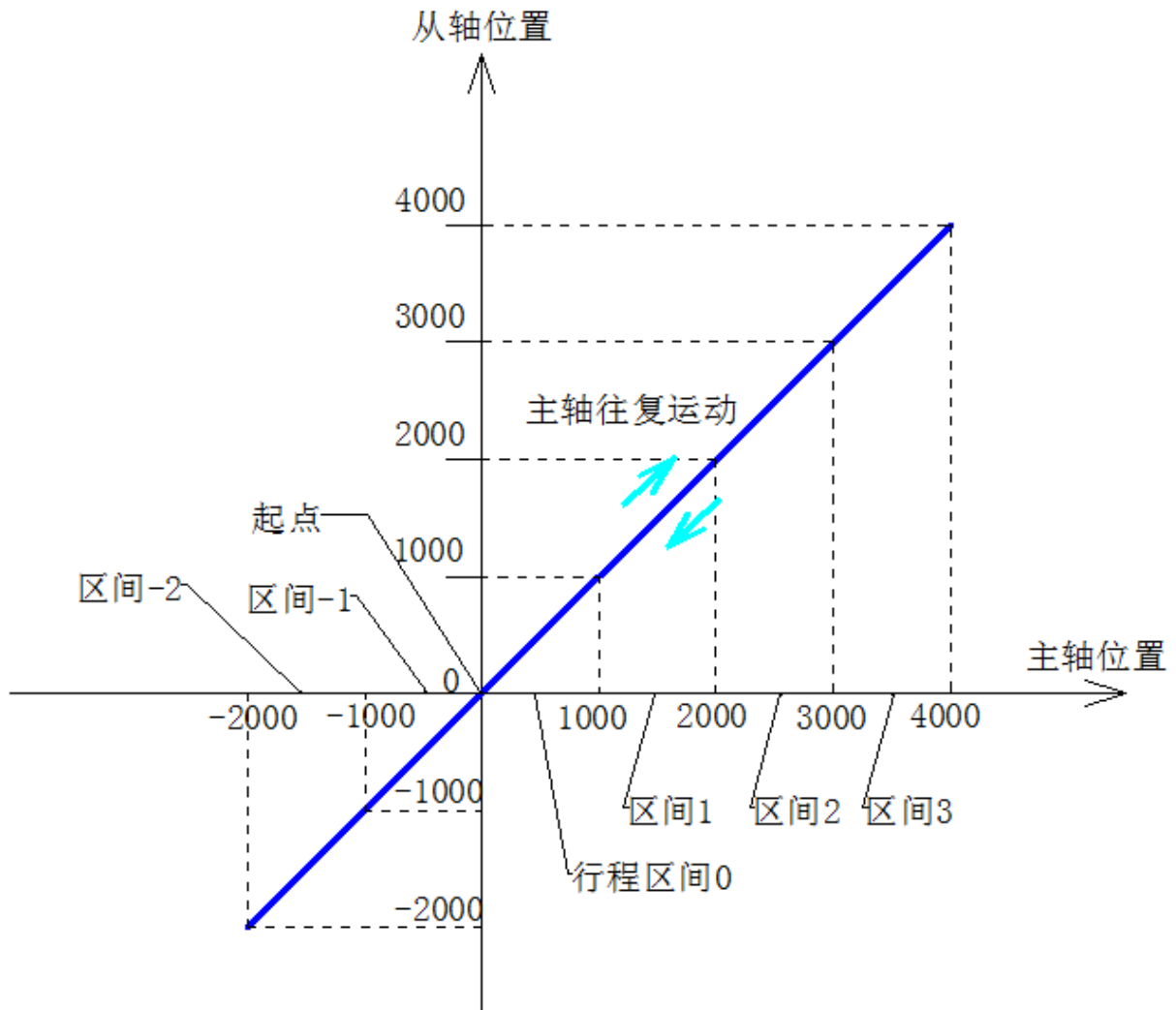
(*主从轴位置数组赋值*)

```
ArrPos[0,0] := 0;  
ArrPos[1,0] := 0;    (*点 1 坐标*)  
ArrPos[0,1] := 500;  
ArrPos[1,1] := 500;  (*点 2 坐标*)  
ArrPos[0,2] := 1000;  
ArrPos[1,2] := 1000; (*点 3 坐标*)
```

HMC_Cam(Axis1.AxisID,Axis0.AxisID, ADR(ArrPos),3,1,0,0); (*投放单次立即启动凸轮指令，0 轴为主轴，1 轴为从轴，凸轮主轴行程范围为*)

(*向主轴投送 Move 指令，驱动凸轮主轴往复运动。此时凸轮从轴将进行相应联动动作*)

```
HMC_Move(Axis0.AxisID,4000);  
HMC_Move(Axis0.AxisID,-6000);  
HMC_Move(Axis0.AxisID,6000);  
HMC_Move(Axis0.AxisID,-4000);
```



循环立即启动凸轮联动轨迹

(6) 循环固定位置启动凸轮(ucMode = 5)

示例程序如下 (ST 语言)，该程序凸轮主轴与从轴联动运行的轨迹为倾角为正弦曲线。向凸轮主轴投放 Hmc_Move 指令驱动凸轮主轴运动，当凸轮主轴达到启动位置时，凸轮指令启动运行，凸轮从轴开始作相应的联动动作（主从轴起始位置均为 0）。

(**生成主从轴位置数组**)

```
delta := 3.1415926*2/100;
```

```
FOR n := 0 TO 100 DO
```

```
ArrPos2[0,n] := (delta*n)*150;
```

```
ArrPos2[1,n] := SIN(delta*n)*1000;
```

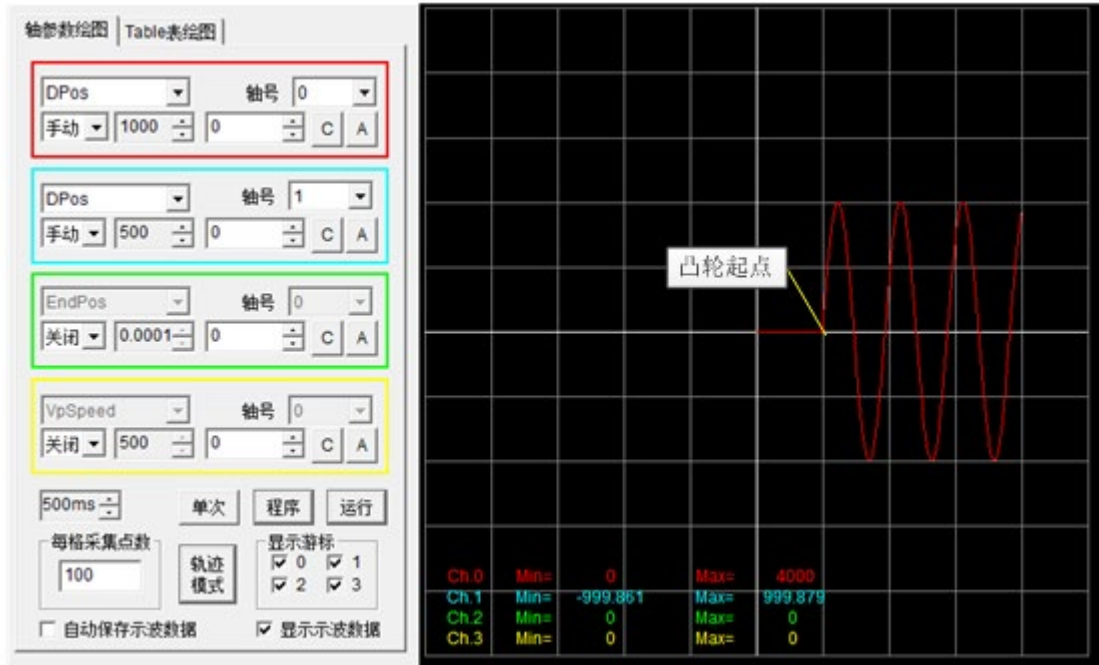
```
END_FOR
```

(*投放单次事件启动凸轮指令，0 轴为主轴，1 轴为从轴，固定启动位置为 1000*)

HMC_Cam (Axis1.AxisID,Axis0.AxisID,ADR(ArrPos2),101,1,5,1000);

(*向主轴投送 Move 指令，驱动凸轮主轴运动。*)

HMC_Move(Axis0.AxisID,4000);



循环固定位置启动凸轮联动轨迹

(7) 循环事件启动凸轮(ucMode = 6)

除启动后撤消条件不同之外，其余与单次事件启动凸轮类似，可参考（3）中示例。

(8) 循环 DI 启动凸轮(ucMode = 7)

除启动后撤消条件不同之外，其余与单次 DI 启动凸轮类似，可参考（4）中示例。

3.4.1.4.6.2 HMC_SplineCam（样条凸轮）

函数原型

```
UDINT HMC_SplineCam(USINT ucSlaveAxis,
USINT ucMasterAxis,
POINTER TO LREAL pArrPos,
UDINT uiPosNum,
LREAL dScale,
USINT ucMode,
```

LREAL dTrigMes)

功能说明

1 该指令功能与 HMC_Cam 功能类似，区别在于 HMC_SplineCam 使用样条曲线拟合的方式把主轴位置点连接起来，而 HMC_Cam 采用的是折线连接的方式。因此，样条凸轮生成的轨迹更为顺滑，在运动过程中没有速度的跃变。

2 其它相关的说明及指令示例可参见 [HMC Cam（电子凸轮）](#)。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
ucMasterAxis	USINT	主轴轴号
pArrPos	POINTER TO LREAL	位置数组首地址（二维） 在二维数组 pArrPos[2,uiPosNum]中，pArrPos[0]为主轴位置数组，pArrPos[1]为从轴位置数组
uiPosNum	UDINT	位置个数，应大于等于 3
dScale	LREAL	凸轮从轴位置数据比例因子。凸轮从轴实际位置输出=从轴位置数组设定输出*dScale
ucMode	USINT	0: 单次立即启动 1: 单次固定位置启动 2: 单次事件启动 3: 单次 DI 启动 4: 循环立即启动 5: 循环固定位置启动 6: 循环事件启动 7: 循环 DI 启动
dTrigMes	LREAL	启动信息： 1、凸轮为位置启动时，该值表示主轴的启动位置 2、凸轮为事件触发启动时，表示某高速捕获通道号 3、凸轮为 DI 为 1 触发启动时，表示 DI 通道（0~63）

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

3.4.1.4.6.3 HMC_GetCamMasterPosIndex（获取电子凸轮主轴位置索引）

函数原型

UDINT HMC_GetCamMasterPosIndex(USINT ucAxisID)

功能说明

获取当前正在运行的凸轮主轴位置索引（当前凸轮主轴位置在凸轮主轴位置数组中的区间索引，应为 0 到 uiPosNum-1）。如果当前指令不为凸轮，函数返回 0。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
位置索引	UDINT	当前正在运行的凸轮的主轴位置索引（在凸轮主轴位置数组中的区间索引）

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01

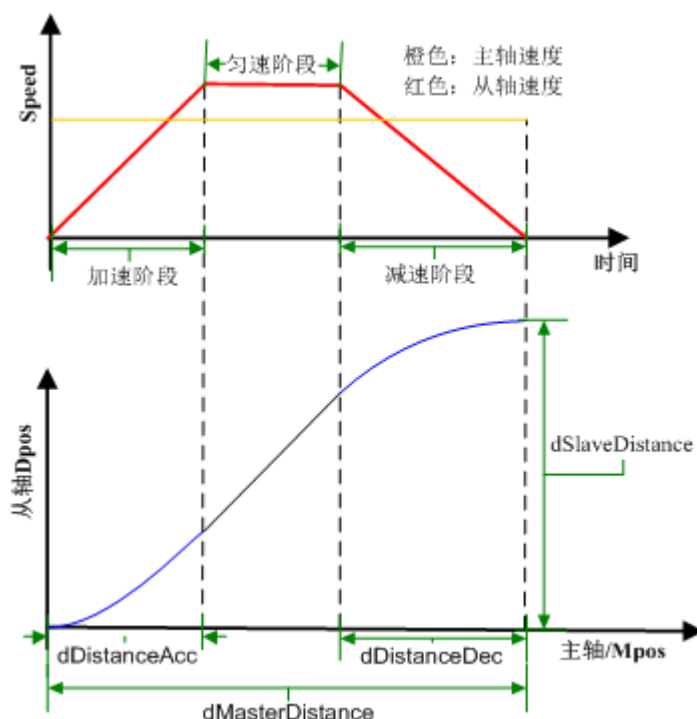
3.4.1.4.6.4 HMC_ClcMoveLinkData（追剪运算）

函数原型

```
UDINT HMC_ClcMoveLinkData(LREAL dSlaveDistance,
LREAL dMasterDistance,
LREAL dDistanceAcc,
LREAL dDistanceDec,
POINTER TO LREAL pArrPos,
UDINT uiPosNum)
```

功能说明

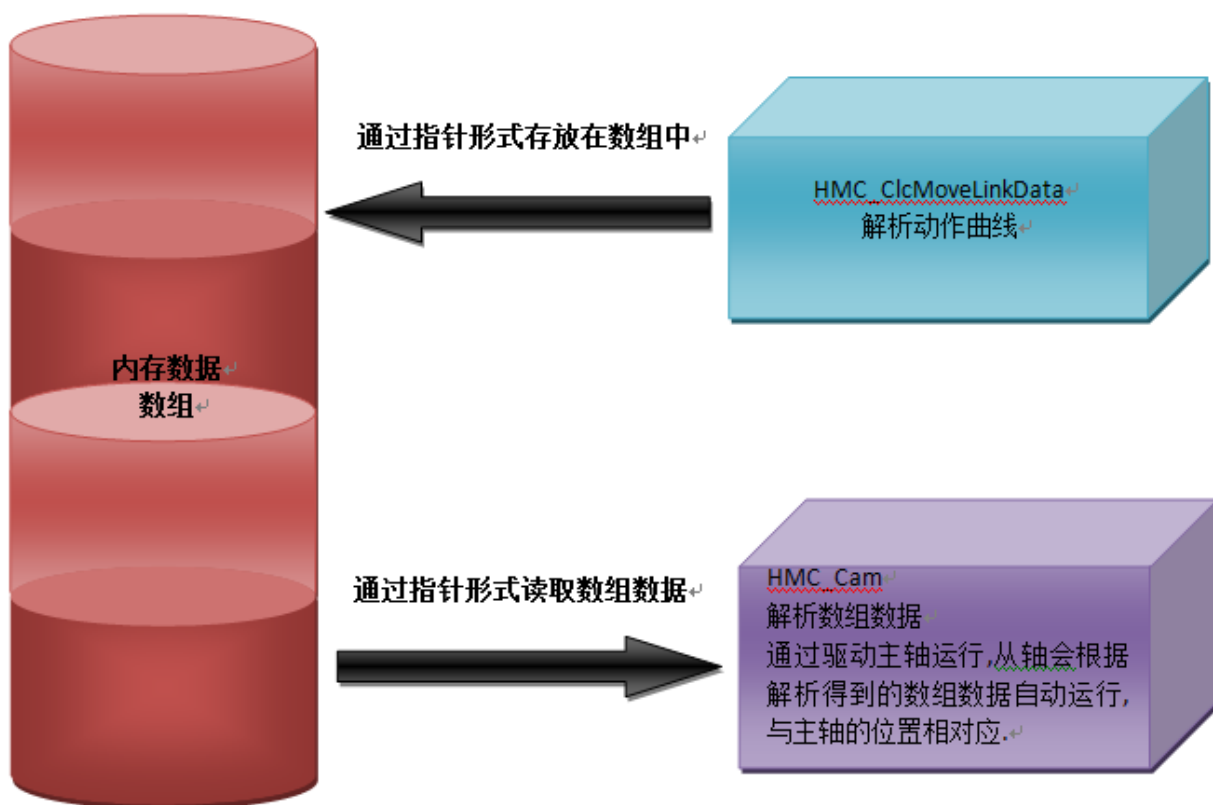
该指令可按照设定的关系生成主从轴位置数据，位置数据保存在数组中，可被凸轮指令使用，控制从轴跟随主轴按一定规律运动。使用该指令所生成的主从轴位置数据，当凸轮主轴匀速运动时，主从轴曲线如下图所示。该运动带有分离可变的加减速阶段。



主/从轴示意图

通过设定从轴位移 dSlaveDistance、主轴位移 dMasterDistance、从轴加速阶段主轴位移 dDistanceAcc、从轴减速阶段主轴位移 dDistanceDec，HMC_ClcMoveLinkData 可生成满足上图所示的主从轴位置数据，数据保存在数组 pArrPos 中，主/从轴位置个数由 uiPosNum 参数指定。

在 HMC_ClcMoveLinkData 与 HMC_Cam 指令配合使用时，使用数组传递主从轴位置数据。首先将曲线通过 HMC_ClcMoveLinkData 指令运算的主从轴位置数据放在数组中，然后通过 HMC_Cam 指令调用该数组数据，最后驱动主轴运行，从轴通过 HMC_Cam 指令解析数组中位置数据实现对主轴位置的跟随。



HMC_ClcMoveLinkData 与 HMC_Cam 数据流

该指令可用在追剪场景中，配合凸轮指令 HMC_Cam，快速生成追剪所需的动作流程，无需进行复杂的中间计算与编程。

输入参数

输入参数	类型	参数描述
dSlaveDistance	LREAL	从轴运动距离
dMasterDistance	LREAL	主轴运动距离（必须为正）
dDistanceAcc	LREAL	在从轴加速阶段主轴移动的正向距离，非负
dDistanceDec	LREAL	在从轴减速阶段主轴移动的正向距离，非负
pArrPos	POINTER TO LREAL	位置数组（二维） 在二维数组 pArrPos[2][uiPosNum]中，pArrPos[0]为主轴位置数组，pArrPos[1]为从轴位置数组
uiPosNum	UDINT	位置个数，应大于等于 6

返回值

返回值	类型	参数描述
0	UDINT	生成数据成功
0xFFFFFFFF	UDINT	函数参数错误

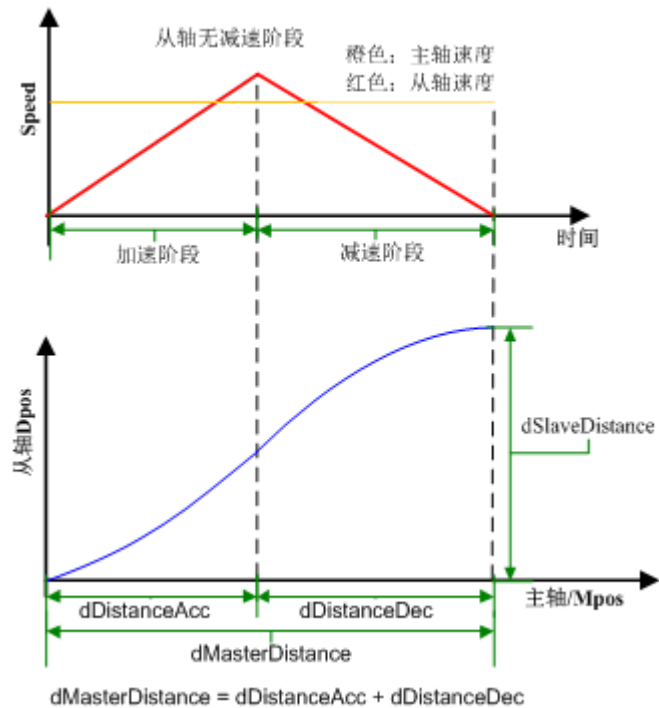
指令类型

非轴运动缓存指令。

■ 补充说明

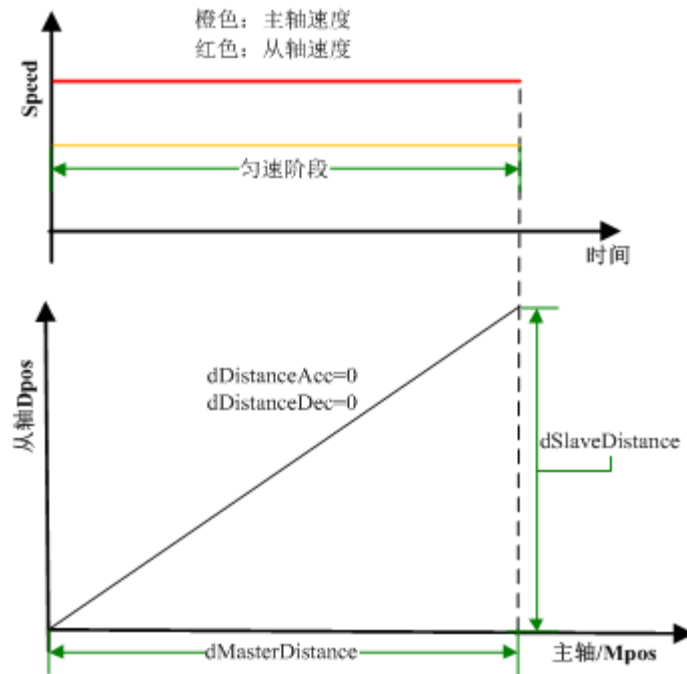
(1) 该函数中的距离均采用用户单位。需满足 $dDistanceAcc + dDistanceDec \leq dMasterDistance$ ，否则返回参数错误。

(2) 当 $dDistanceAcc + dDistanceDec = dMasterDistance$ 时，[主/从轴示意图](#)中凸轮从轴将无匀速阶段，此时凸轮主从轴曲线如下：



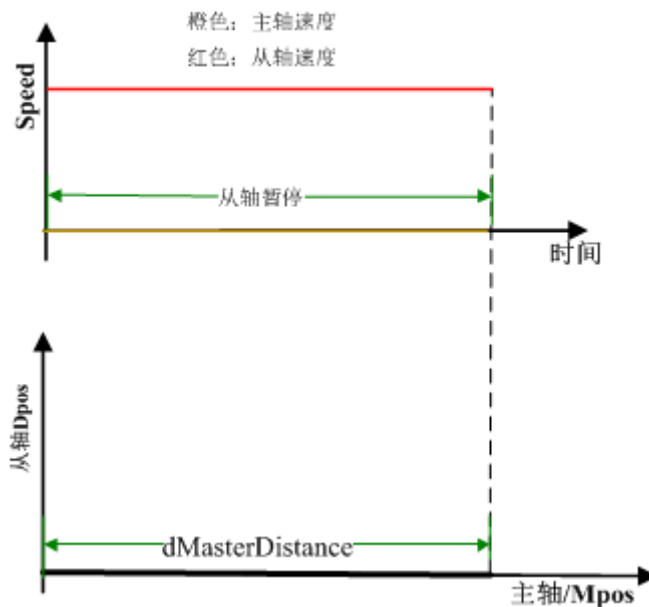
主/从轴示意图

(3) 当 $dDistanceAcc$ 、 $dDistanceDec$ 均为零时，即无分离的加、减速时，此时的联动等效于电子齿轮，[主/从轴示意图](#)中的凸轮主从轴曲线如下：



主/从轴示意图

(4) 若 $dSlaveDistance$ 为 0，当凸轮主轴在设定区域运动时，从轴将暂停等待。[主/从轴示意图](#)中的凸轮主从轴曲线如下：



主/从轴示意图

应用举例

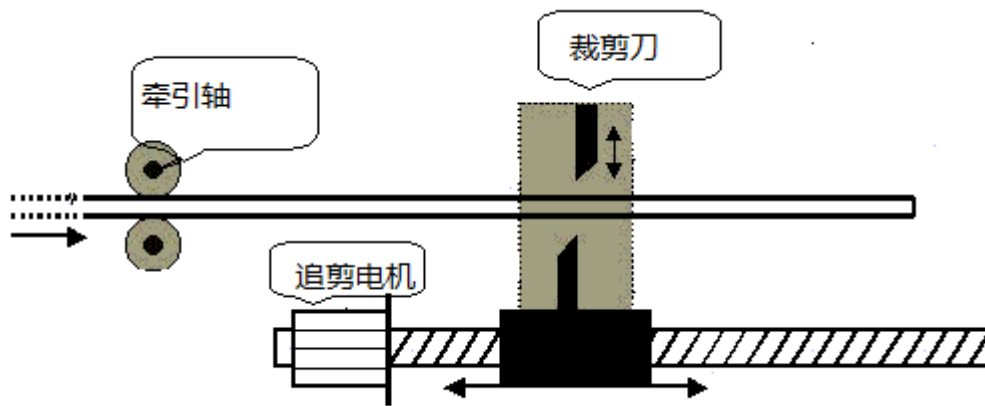
(1) 追剪

在传统的定长度物料加工系统中，一般是由送料伺服电机将物料输出设定的长度后停止送料电机，然后驱动加工机构（如裁刀等）进行加工，加工完毕后才能继续送料。传统的此类设备有钢板校平轧断机、各

类管材的定长裁断、纺织机械的落砂装置等。由于加工中送料动作需要间歇性停止，因此此类设备生产效率低下，且只能用在定长输出的物料可以间歇停止的场合。在无缝钢管生产线、送料装置为挤出机等场合，或对生产效率有较高要求的应用场景，设备不允许间歇停止，上述传统设备便无法满足客户需求，由此需要引入追剪方案。

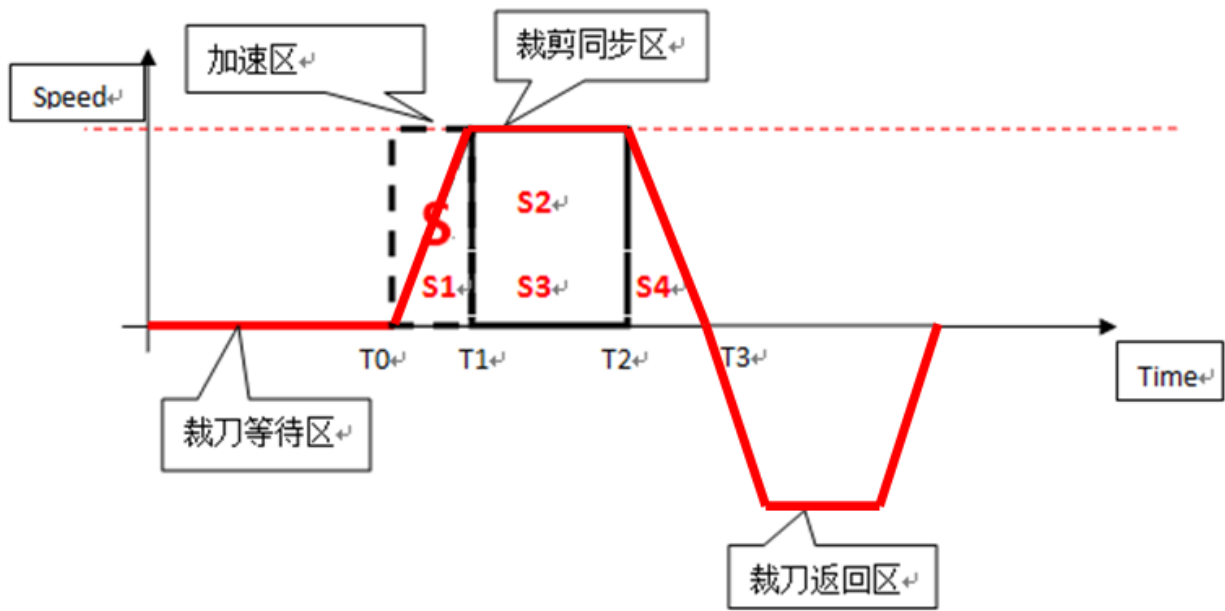
追剪就是在物料剪裁过程中，物料保持不间断的传送，而裁断装置做往复运动，通过对裁断装置进行速度及位置规划，使得剪裁装置与物料速度达到同步时，所要剪裁的物料长度刚好满足预定要求，此时进行物料裁剪，剪裁动作完成后返回等待位置等待一定时间，之后再次追踪物料同步裁剪，周而复始。以下举例说明。

物料通过牵引轴从左至右匀速运行，裁刀从初始位置开始追踪物料到达同步区后进行裁剪动作，裁剪完毕后，裁刀快速返回至初始位置，等待一段时间后继续下一个动作循环。

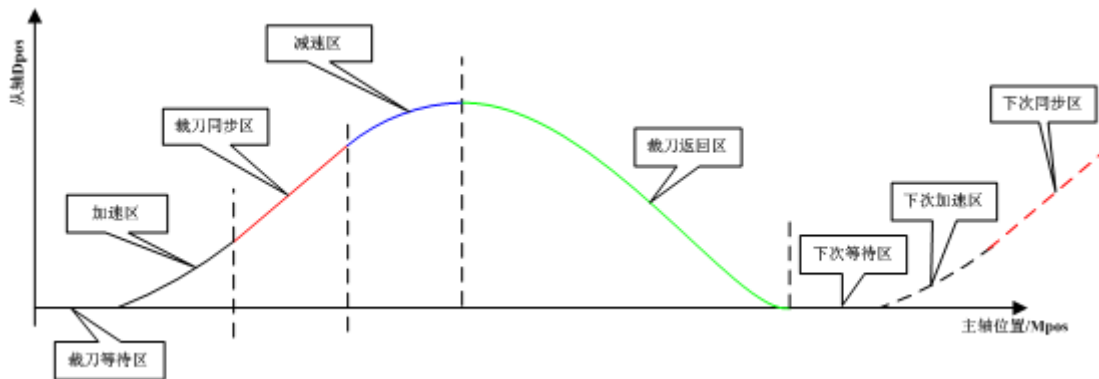


追剪方案示意图

满足追剪要求的速度曲线如下图所示，途中红色虚线为牵引轴速度，红色实线为裁剪轴速度。通过 HMC_ClcMoveLinkData 及 HMC_Cam 指令即可实现该追剪动作流程。设牵引轴为系统的主轴，追剪电机为系统从轴。



追剪速度规划图



追剪主从轴位移曲线

裁剪长度设定 100 m，裁刀移动行程 3 m，丝杠导程 30 mm，追剪电机转速 2000 r/min，速比 1:1。将运动轨迹分成以下几部分：等待区、加速区、同步区、减速区、返回区。在常速区域可以细化曲线来配合裁剪刀动作。设在加速区（ $T_0 \sim T_1$ 时刻）裁剪轴的行走距离是 S_1 ，牵引轴行走距离是 S ，计算可知： $S=2S_1$ ；在同步区（ $T_1 \sim T_2$ 时刻），裁刀速度 裁剪轴行走距离 $S_2 =$ 牵引轴行走距离 S_3 ；若加减速时间相等，则在减速区（ $T_2 \sim T_3$ 时刻）裁剪轴行走距离 $S_4=S_1$ ，牵引轴行走距离为 S 。

由已知参数得知裁剪轴速度 $=2000 \times 30 = 60,000 \text{ mm/min} = 1000 \text{ mm/s}$ 。设加减速时间为 1 s，则计算可得：

$$S=1000 \text{ mm}$$

$$S_1=S_4=500 \text{ mm}$$

$$S_2=S_3=2000 \text{ mm}$$

裁刀等待区域的主轴位移 = 总裁剪长度 - 裁刀正向动作区主轴位移 - 裁刀反向动作区主轴位移。

本例设定裁刀正向动作与反向动作所用的加减速、匀速速度大小相同，则有裁刀正反向动作时间相等，因此：

裁刀反向动作区主轴位移 = 裁刀正向动作区主轴位移 = $S+S_2+S = 4000$ (mm)

裁刀正向动作区从轴位移 = $S_1+S_2+S_4 = 3000$ (mm)

裁刀反向动作区从轴位移 = - (裁刀正向动作区从轴位移 = -3000 (mm)

裁刀等待区域的主轴位移 = $100000 - 4000 - 4000 = 92000$ (mm)

裁刀等待区域的从轴位移 = 0

程序如下：

TASK1(任务 1):初始化轴参数及追剪数据

(**** 使用 HMC_SetUnits 把用户单位转换为 mm (略) ****)

(****使用 ADR 取位置数组首地址****)

add1 := ADR(Pos);

add2 := ADR(Pos1);

add3 := ADR(Pos2);

add4 := ADR(Pos3);

add5 := ADR(Pos4);

(**** 生成主从轴位置数组 ****)

HMC_ClcMoveLinkData(0, 92000, 0, 0, add1, PosNum); (* 裁剪等待区 *)

HMC_ClcMoveLinkData(500, 1000, 1000, 0, add2, PosNum); (* 裁剪加速区 *)

HMC_ClcMoveLinkData(2000, 2000, 0, 0, add3, PosNum); (* 裁剪同步区 *)

HMC_ClcMoveLinkData(500, 1000, 0, 1000, add4, PosNum); (* 裁剪减速区 *)

HMC_ClcMoveLinkData(-3000, 4000, 1000, 1000, add2, PosNum); (* 裁剪返回区 *)

(**** 使用各阶段主从轴位置数组投放凸轮指令****)

While 1 do (* 循环投送追剪凸轮*)

HMC_Cam(1, 0, add1, PosNum, 1, 0, 0); (* 裁剪等待区 *)

HMC_Cam(1, 0, add2, PosNum, 1, 0, 0); (* 裁剪加速区 *)

Cut_Enable := HMC_Cam(1, 0, add3, PosNum, 1, 0, 0); (* 裁剪同步区 *)

HMC_Cam(1, 0, add4, PosNum, 1, 0, 0); (* 裁剪减速区 *)

HMC_Cam(1, 0, add5, PosNum, 1, 0, 0); (* 裁剪返回区 *)

(**** 在同步区进行裁剪动作 ****)

Camload_Ok := HMC_WaitCmdLoaded(Cut_Enable); (* 在同步区驱动裁剪 *)

%QX0.0:= 1; (* 裁刀动作。假设裁刀动作为开关量控制。 *)

(****同步区结束后收回裁刀，裁剪结束 ****)

CamFinish_Ok := HMC_WaitCmdFinish(Cut_Enable); (* 同步区结束后立即停止裁剪 *)

%QX0.0 := 0; (* 裁刀收回 *)

End_While;

(**** 注意:在实际应用过程中,为了保护裁刀可以将同步区再细化。 ****)

TASK2(任务 2):

HMC_Forward(0); (*主轴位移启动 *)

■ 不支持的模块版本

MC1008-A01

3.4.1.4.6.5 HMC_ClcFlexLinkData (飞剪/旋切运算)

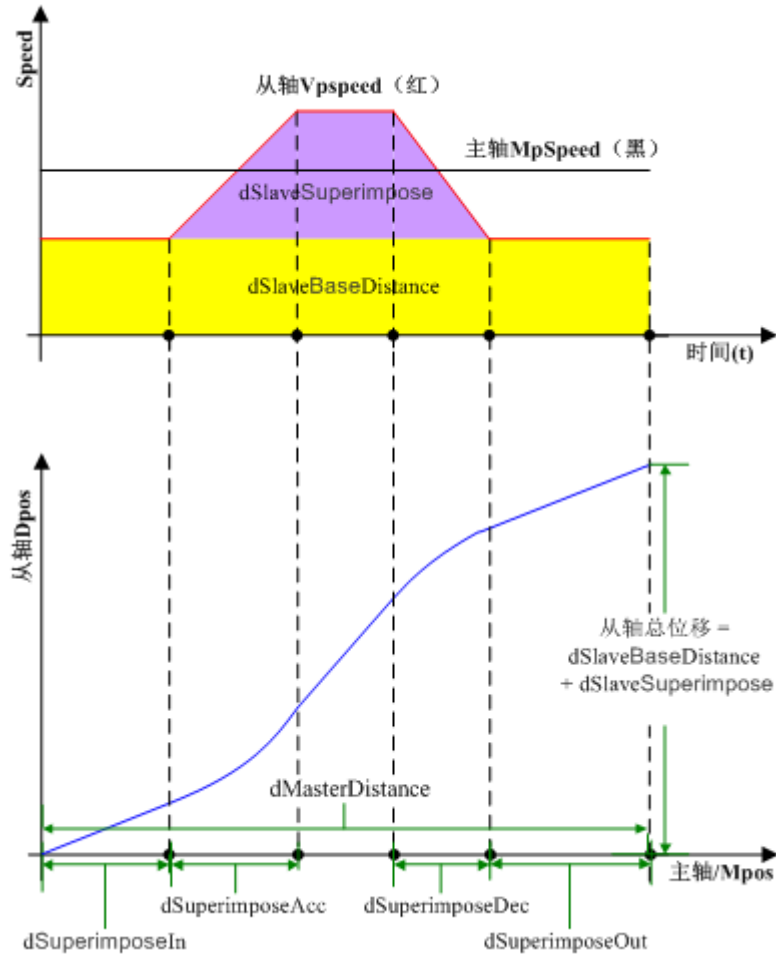
函数原型

```
UDINT HMC_ClcFlexLinkData(LREAL dSlaveBaseDistance,  
LREAL dSlaveSuperimpose,  
LREAL dSlaveBaseDistance,  
LREAL dSlaveSuperimpose,  
LREAL dMasterDistance,  
LREAL dSuperimposeIn,  
LREAL dSuperimposeOut,  
LREAL dSuperimposeAcc,  
LREAL dSuperimposeDec,  
POINTER TO LREAL pArrPos,  
UDINT uiPosNum)
```

功能说明

该指令可按照设定的关系生成主从轴位置数据，位置数据保存在数组中，可被凸轮指令使用，控制从

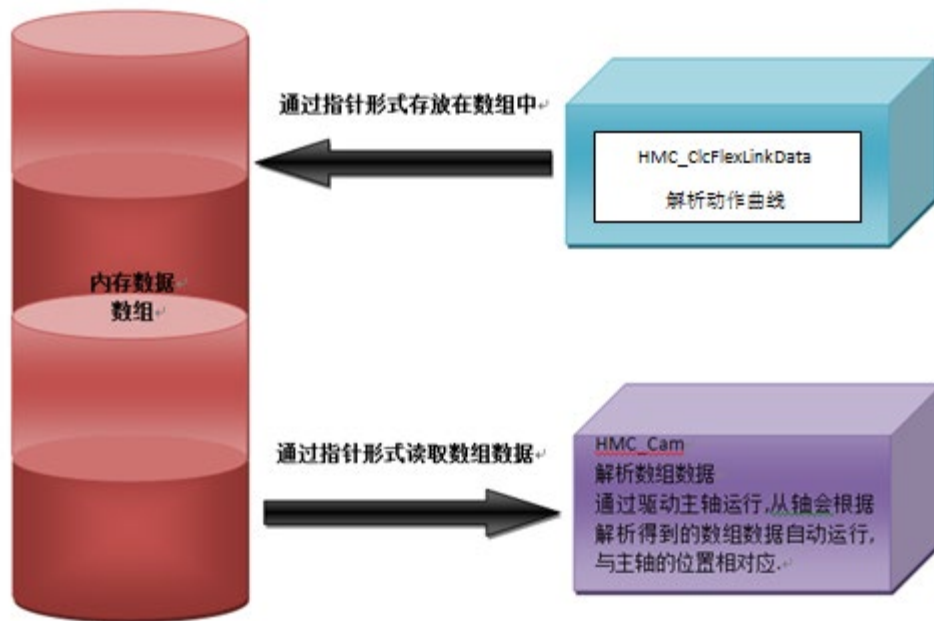
轴跟随主轴按一定规律运动。使用该指令所生成的主从轴位置数据，当凸轮主轴匀速运动时，主从轴曲线如下图所示。从轴的跟随运动由恒速距离（ $dSlaveBaseDistance$ ）和带分离加减速的变速距离（ $dSlaveSuperimpose$ ）叠加而成。



主/从轴示意图

通过设定所需的参数（见输入参数列表），`HMC_ClcMoveLinkData` 即可生成满足上图所示的主从轴位置数据，数据保存在数组 `pArrPos` 中，主/从轴位置个数由 `uiPosNum` 参数指定。

在 `HMC_ClcFlexLinkData` 与 `HMC_Cam` 指令配合使用时，使用数组传递主从轴位置数据。首先将曲线通过 `HMC_ClcFlexLinkData` 指令运算的主从轴位置数据放在数组中，然后通过 `HMC_Cam` 指令调用该数组数据，最后驱动主轴运行，从轴通过 `HMC_Cam` 指令解析数组中位置数据实现对主轴位置的跟随。



HMC_ClcFlexLinkData 与 HMC_Cam 数据流

该指令可用在飞剪/旋切场景中，配合凸轮指令 HMC_Cam，快速生成追剪所需的动作流程，无需进行复杂的中间计算与编程。

输入参数

输入参数	类型	参数描述
dSlaveBaseDistance	LREAL	从轴恒速跟随距离
dSlaveSuperimpose	LREAL	从轴叠加运动距离
dMasterDistance	LREAL	主轴运动距离（必须为正）
dSuperimposeIn	LREAL	叠加运动（dSlaveSuperimpose）开始时主轴已完成正向距离，非负
dSuperimposeOut	LREAL	叠加运动（dSlaveSuperimpose）结束时主轴剩余正向距离，非负
dSuperimposeAcc	LREAL	叠加运动加速阶段主轴移动的正向距离，非负
dSuperimposeDec	LREAL	叠加运动减速阶段主轴移动的正向距离，非负
pArrPos	POINTER TO LREAL	位置数组（二维） 在二维数组 pArrPos[2][uiPosNum]中，pArrPos[0]为主轴位置数组，pArrPos[1]为从轴位置数组
uiPosNum	UDINT	位置个数，应大于等于 6

返回值

返回值	类型	参数描述
0	UDINT	生成数据成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

■ 补充说明

（1）主轴运动距离（dMasterDistance）严格为正，否则返回参数错误。

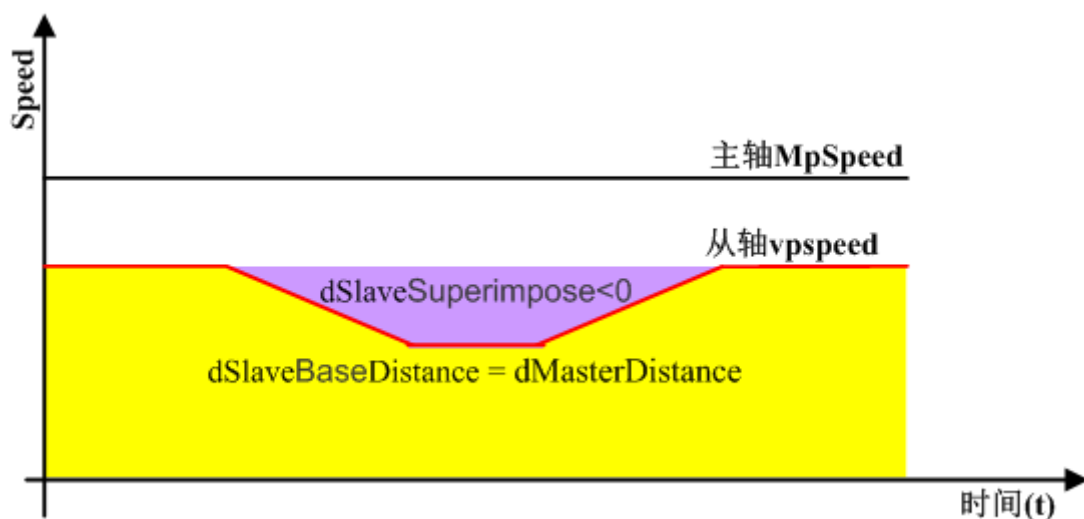
从轴位移等于 dSlaveBaseDistance 与 dSlaveSuperimpose 之和。

（2）应满足：

$$dSuperimposeIn + dSuperimposeOut + dDistanceAcc + dDistanceDec \leq dMasterDistance$$

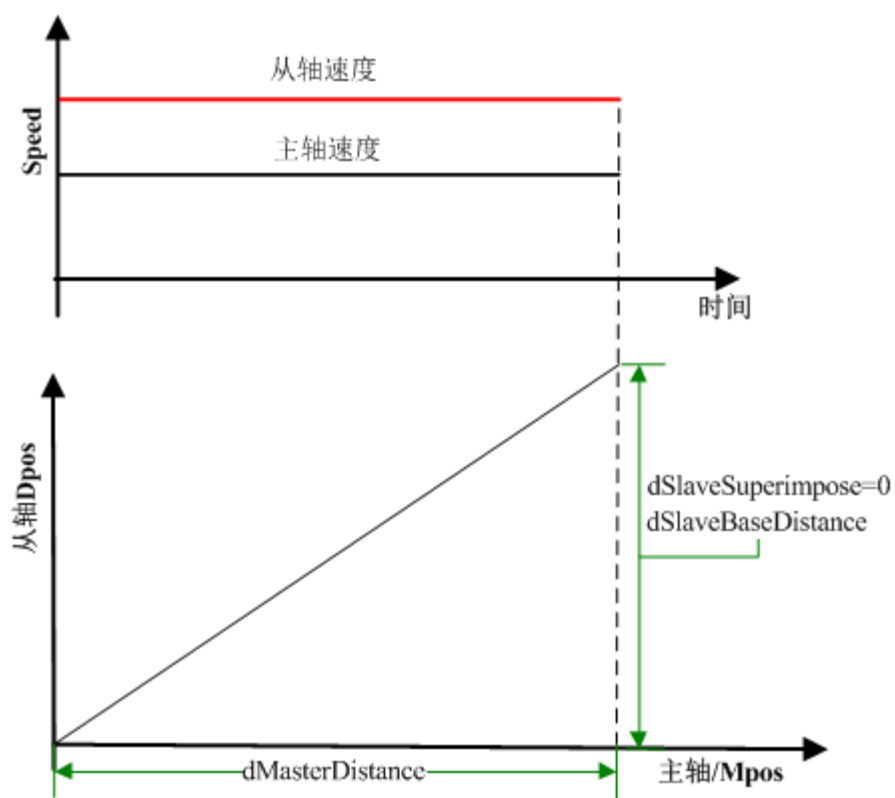
否则返回参数错误（0xFFFFFFFF）。

（3）dSlaveBaseDistance、dSlaveSuperimpose 正负均可，若两个参数值均为 0 时，在主轴未执行完 dMasterDistance 之前，从轴一直处于等待状态，与 HMC_MoveLink 的暂停等待类似。



从轴叠加距离为负时的情形

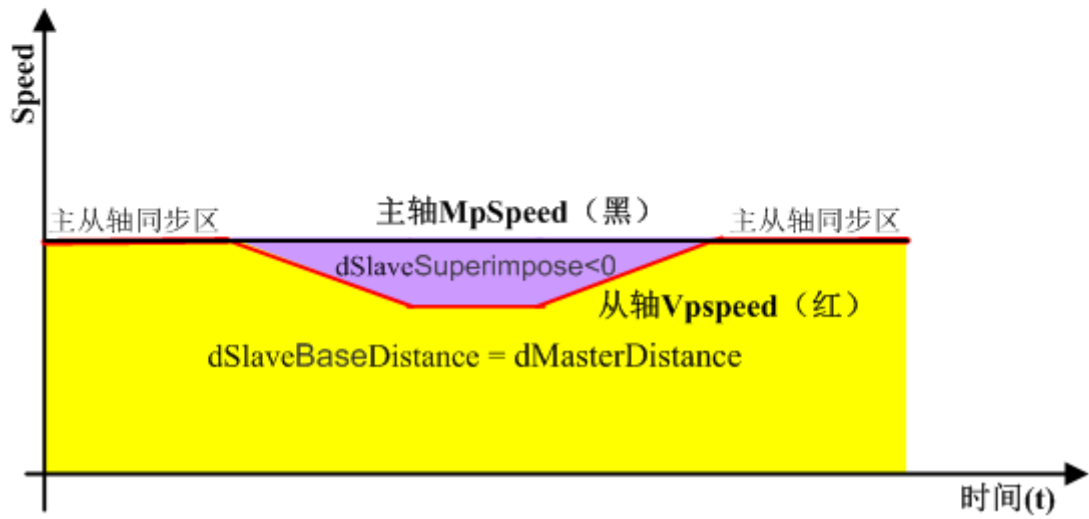
（4）dSlaveSuperimpose 为 0 时，主从轴将按照一定的速度比例关系运动，参数 dSuperimposeIn、dSuperimposeOut、dSuperimposeAcc、dSuperimposeDec 的设置将失效。此时的联动等效于电子齿轮。



运动示意图

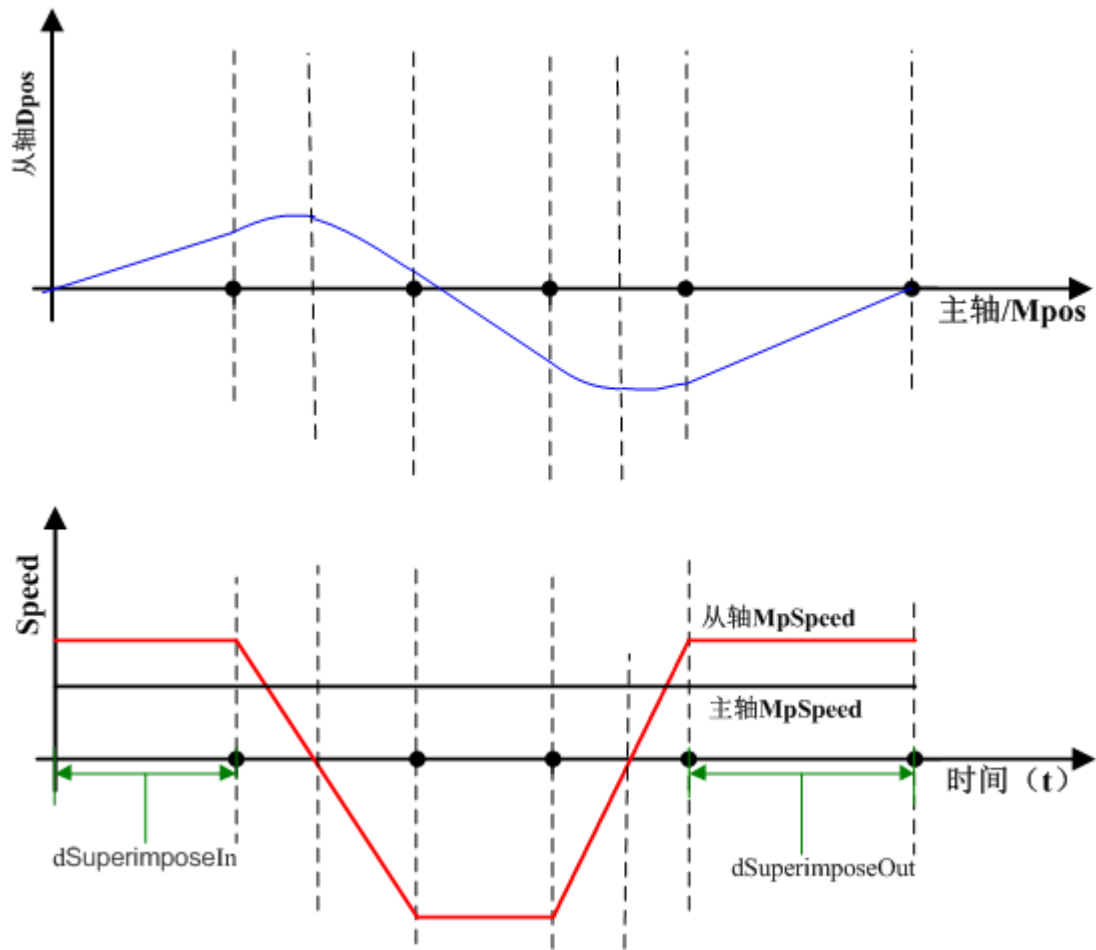
(5) 当从轴恒速跟随位移与主轴位移相等时，即：

$dSlaveBaseDistance = dMasterDistance$ ，在从轴叠加区域之外，从轴与主轴的运动将保持同步。



运动示意图

(6) 当 $dSlaveBaseDistance + dSlaveSuperimpose = 0$ ，且 $dSlaveBaseDistance$ 不为 0 时，通过叠加补偿后从轴位置回到了起始点，此情况也可用在需通过反转返回到触发位置的情况。

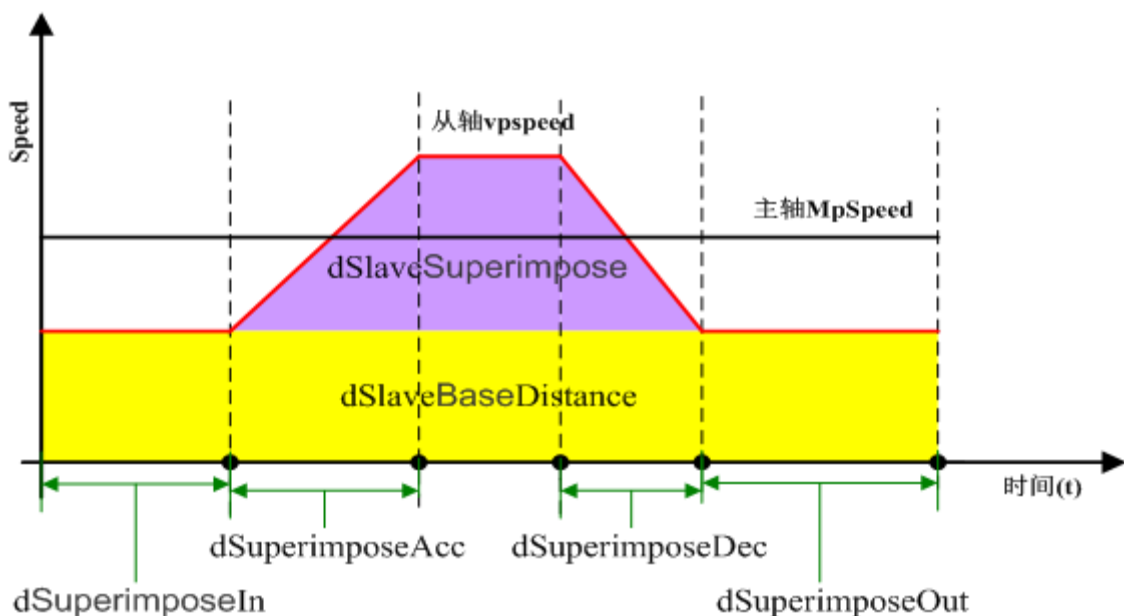
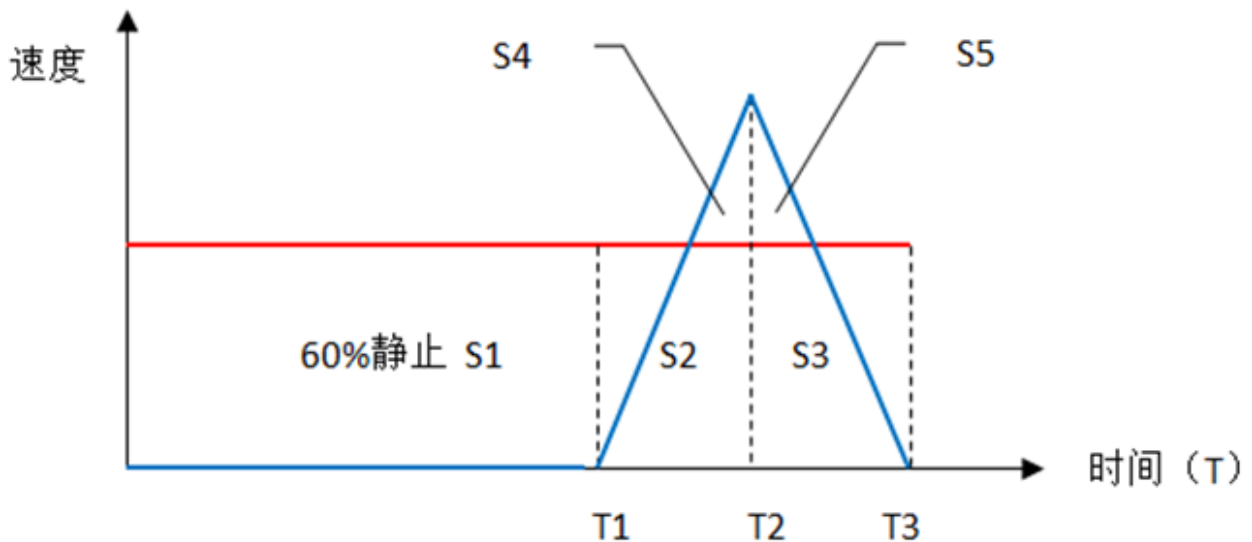


主/从轴示意图

应用举例

(1) 生成指定的联动动作

在一个循环动作中，有 40% 的距离是平滑的加减速，余下的部分保持静止不动，现使用 HMC_ClcFlexLinkData 生成该动作流程。



```
dSlaveBaseDistance=0;  
dSlaveSuperimpose=1000;  
dSuperimposeIn=S1=1200;  
dSuperimposeOut=0;  
dSuperimposeAcc=dSuperimposeDec=S2=400。
```

具体程序如下：

(****初始化程序如下 ****)

(*使用 HMC_SetUnits 把用户单位转换为 mm (略) *)

```
DR_RESULT := HMC_DriveEnable(1);    (* 信号使能操作 *)
```

```
X_TYPE := HMC_SetAxisType(0, 0);    (* 设置 axis0 属性 *)
```

```
X_UNIT := HMC_SetUnits(0, 100);    (* 假设 100 个脉冲对应 1 mm *)
```

```
AXIS0.SPEED := 100;                (* 单位 mm/s *)
```

```
AXIS0.ACCEL := 1000;
```

```
AXIS0.DECEL := 1000;
```

```
Y_TYPE := HMC_SetAxisType(1, 0);    (* 设置 axis1 属性 *)
```

```
Y_UNIT := HMC_SetUnits(1, 100);
```

```
AXIS1.SPEED := 100;                (* 单位 mm/s *)
```

```
AXIS1.ACCEL := 1000;
```

```
AXIS1.DECEL := 1000;
```

```
Pt_array := ADR(arraympos);
```

```
NumOfPos := 100;
```

(* 主要程序如下 *)

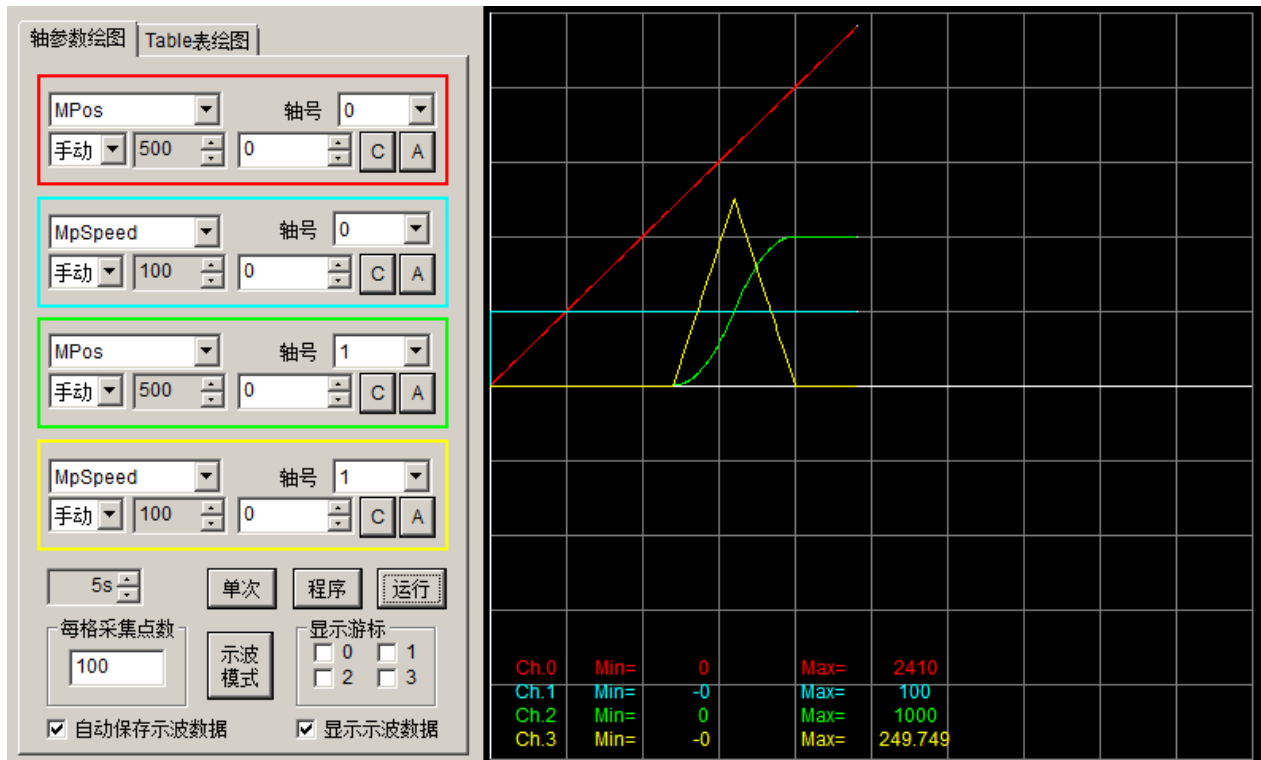
```
HMC_ClcFlexLinkData(0, 1000, 2000, 1200, 0, 400, 400, Pt_array, NumOfPos);
```

```
fff := HMC_TriggerScope();    (* 示波器使能 *)
```

```
camid := HMC_Cam(1, 0, Pt_array, NumOfPos, 1, 4, 0); (* 投送凸轮，循环立即启动 *)
```

```
X_RESULT := HMC_Forward(0);    (* 主轴启动 *)
```

在控制器中运行结果如下（青色与黄色曲线分别为主轴与从轴速度）：



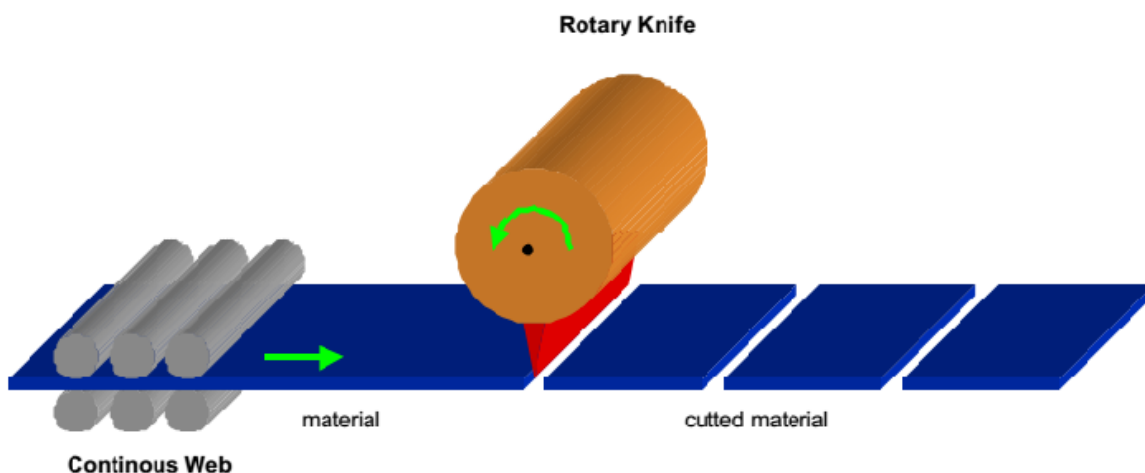
示波器显示结果

(2) 飞剪/旋切运算

飞剪常用于包装、造纸、轧钢等生产线上。在连轧钢坯车间或小型型钢车间里，它安放在轧制线的后部，将轧件切成定尺寸或仅切头切尾。在冷、热带钢车间的横剪机组、重剪机组、镀锌机组和镀锡机组里都配置有各种不同类型的飞剪，将带钢剪成定尺或裁成规定重量的钢卷。广泛采用飞剪有利于使造纸、包装、轧钢生产线迅速向高速化、连续化方向发展。因此，它是高速生产线发展的重要环节之一。

旋切工艺流程中，切刀的动作有同步区和非同步区。切刀剪切材料的区域为同步区，在剪切过程中，刀头与被剪切的材料同步运动，以保证剪切质量，同时保护刀具；在剪切完成后，根据剪切的长度不同，刀辊做加减速运动来调整刀头做下一次剪切，即为非同步区。

使用 `HMC_ClcFlexLinkData` 指令，我们无需使用复杂的数学表达式去求解位置点，底层的算法由控制器自动完成，只需要规划好同步、加速、减速的距离，便可自动生成飞剪/旋切运动所需的主从轴位置数据，然后在 `HMC_Cam` 中使用该位置数据执行即可。



飞剪/旋切示意图

假定：刀辊的周长为 600 mm，剪切同步区长度为 150 mm（两边各 75 mm）。

易知：dSuperimposeIn = dSuperimposeOut = 75mm；

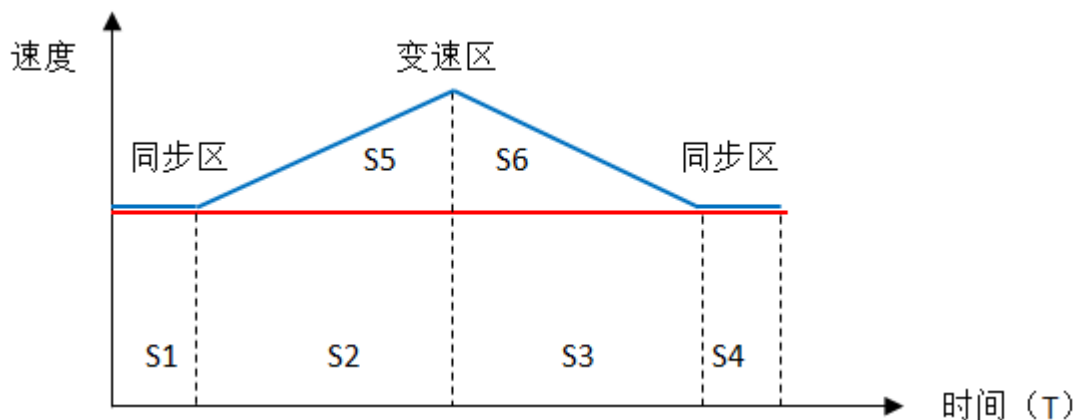
由于存在同步区，故有：dSlaveBaseDistance = dMasterDistance = 待剪切料长度。

根据所需要的剪切长度的不同，可分为短料剪切、中料剪切、长料剪切三种情况，可通过调整从轴叠加距离 dSlaveSuperimpose 来实现（刀辊周长= dSlaveBaseDistance+dSlaveSuperimpose）。

初始化程序如下：

```
DR_RESULT := HMC_DriveEnable(1);    (* 信号使能操作 *)
X_TYPE := HMC_SetAxisType(0, 0);    (* 设置 axis0 属性 *)
X_UNIT := HMC_SetUnits(0, 100);     (* 假设 100 个脉冲对应 1 mm *)
AXIS0.SPEED := 100;                 (* 单位 mm/s *)
AXIS0.ACCEL := 1000;
AXIS0.DECEL := 1000;
Y_TYPE := HMC_SetAxisType(1, 0);    (* 设置 axis1 属性 *)
Y_UNIT := HMC_SetUnits(1, 100);
AXIS1.SPEED := 100;                 (* 单位 mm/s *)
AXIS1.ACCEL := 1000;
AXIS1.DECEL := 1000;
Pt_array := ADR(arraympos)          (* 取得数组指针，数组维数应为[2,NumOfpos] *)
NumOfPos := 100;
```

1) 情形一：短料剪切（L<剪刀周长）L=400 mm



短料剪切示意图（红色代表主轴，蓝色代表从轴）

由于剪切材料的时候主轴和从的线速度要同步，在这里我们取同步的长度为 150 mm，这样以切断点为中心，两边同步距离就是 75 mm， $S1=S4=75\text{ mm}$ 。我们假设从轴加速和减速区的距离相等，那么相对应的主轴 $S2=S3=(400-150)/2=125\text{ mm}$ ，而从轴整个位移，即蓝线所包含的面积就是剪刀的周长即 600 mm。 $S5+S6=(600-400)=200\text{ mm}$ 。 $S5+S6$ 区域为蓝线区域减红线区域。

故有：

$$dSuperimposeIn = dSuperimposeOut = S1 = S4 = 75\text{ mm}$$

$$dSlaveBaseDistance = dMasterDistance = L = 400\text{ mm}$$

$$dSlaveSuperimpose = S5 + S6 = 200\text{ mm}$$

$$dSuperimposeAcc = dSuperimposeDec = S2 = S3 = 125\text{ mm}。$$

具体凸轮程序如下：

```
FlexLinkReVal := HMC_ClcFlexLinkData(400, 200, 400, 75, 75, 125, 125, Pt_array, NumOfPos);
```

```
fff := HMC_TriggerScope(); (* 示波器使能 *)
```

```
camid := HMC_Cam(1, 0, Pt_array, NumOfPos, 1, 4, 0); (* 投凸轮，循环立即启动 *)
```

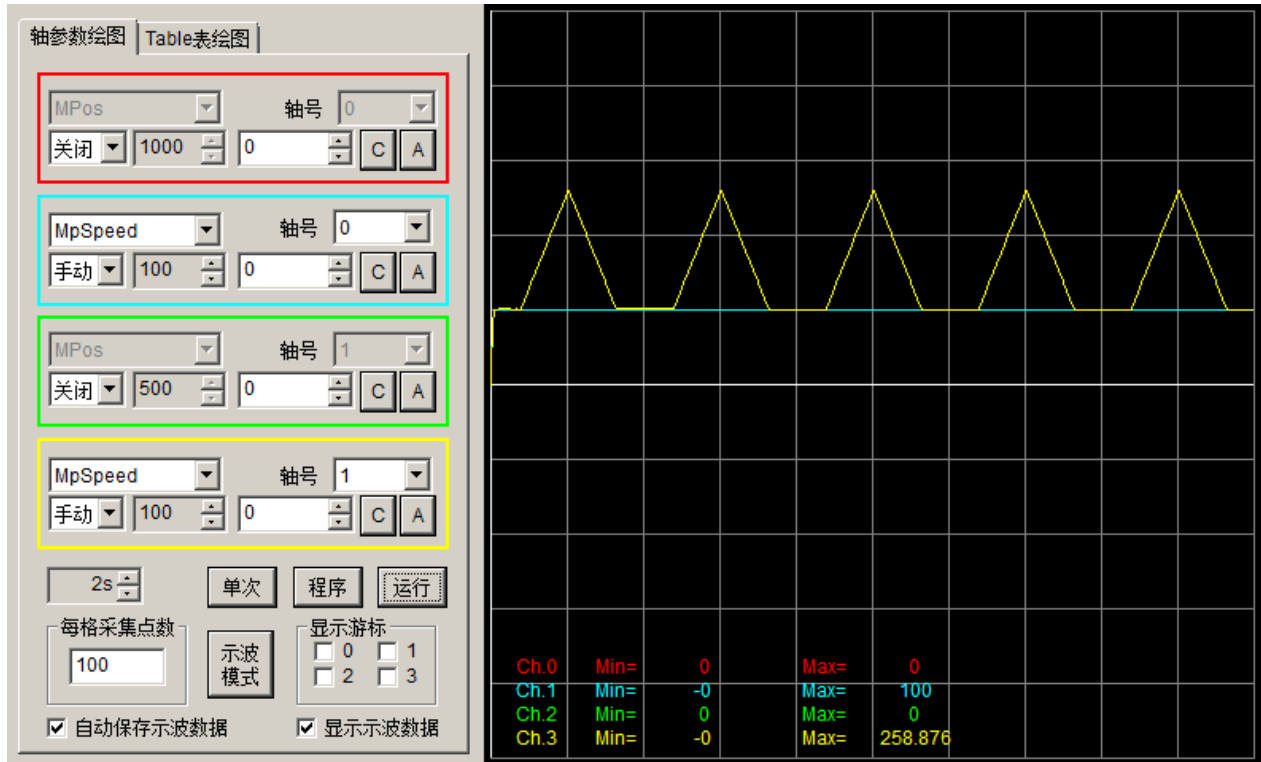
```
X_RESULT := HMC_Forward(0); (* 主轴启动 *)
```

AT 工程中 arraympos 数组维数应为[2,NumOfpos]。

变量名	直接地址	变量说明	变量类型	在线值	掉电保护
arraympos	%MDO		ARRAY[0..1, 0..1000] OF LREAL		FALSE

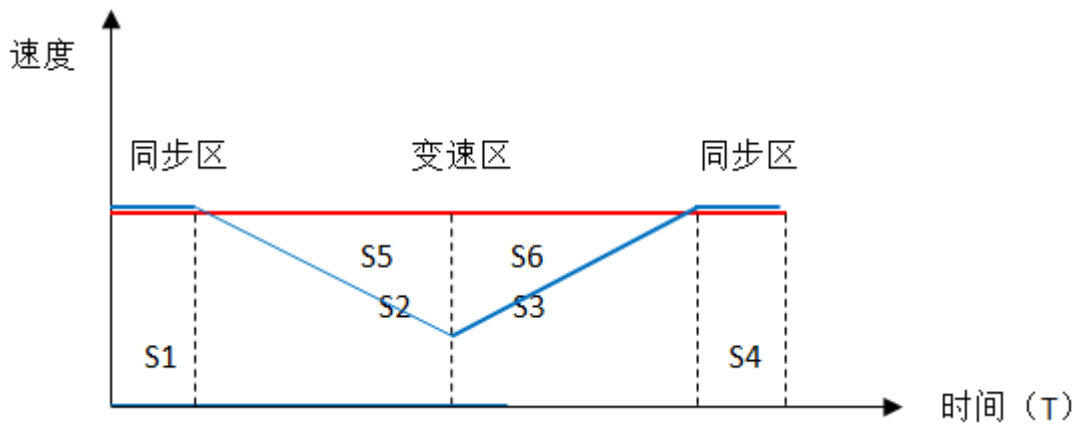
Arraympos 变量定义

运行任务。示波器显示：



示波器显示结果

2) 情形二：中料剪切 ($L \geq$ 剪刀周长) $L=800$ mm



中料剪切示意图

红色区域面积(剪切料长/主轴位移): $S1+S2+S3+S4=800$ mm;

蓝色区域面积(刀辊周长/从轴位移): $S1+(S2-S5)+(S3-S6)+S4=600$;

$S2 = S3 = (800-S1-S4)/2 = 325$ mm;

$S5+S6 = 800-600 = -200$ mm;

$S5+S6$ 区域为红线区域减蓝线区域。

故有：

$d_{SuperimposeIn} = d_{SuperimposeOut} = S1 = S4 = 75\text{mm}$

$d_{SlaveBaseDistance} = d_{MasterDistance} = L = 800\text{ mm}$

$d_{SlaveSuperimpose} = S5 + S6 = -200\text{mm}$

$d_{SuperimposeAcc} = d_{SuperimposeDec} = S2 = S3 = 325\text{mm}。$

具体的凸轮运动程序：

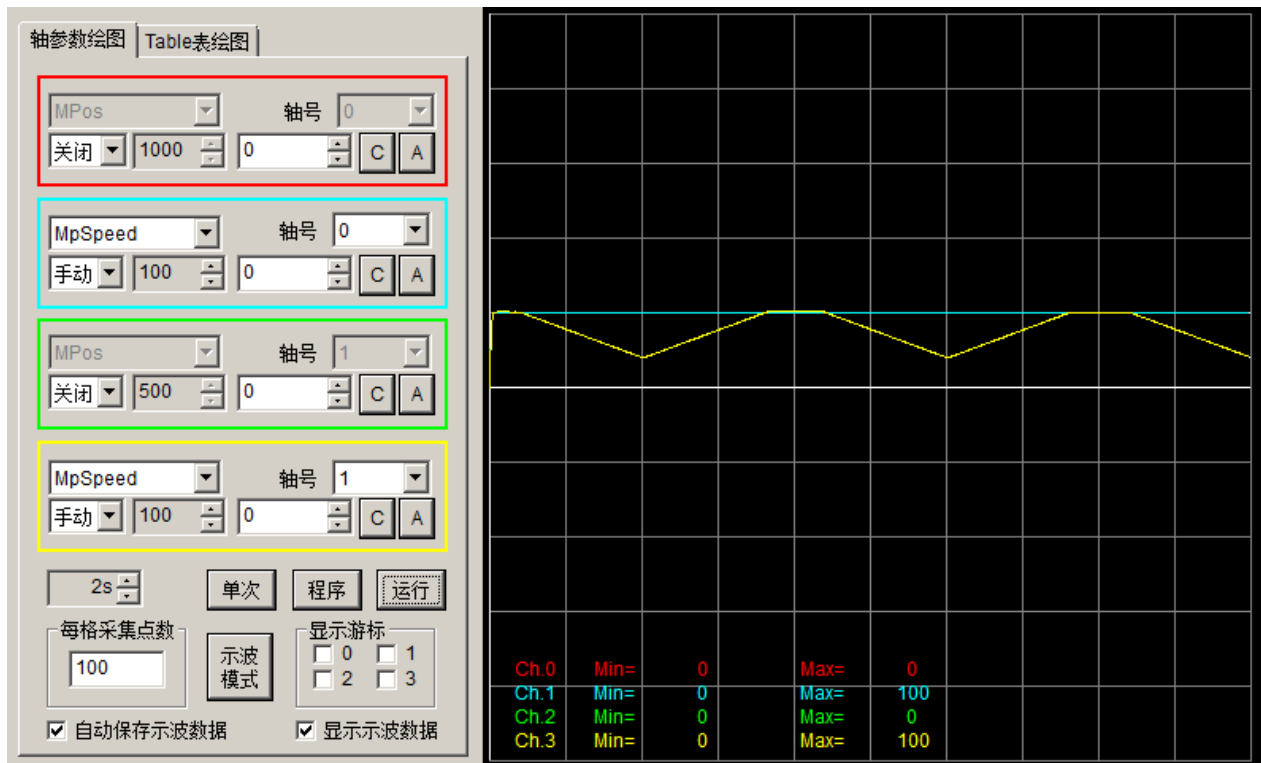
$FlexLinkReVal := HMC_ClcFlexLinkData(800, -200, 800, 75, 75, 325, 325, Pt_array, NumOfPos);$

$fff := HMC_TriggerScope();$ (* 示波器使能 *)

$camid := HMC_Cam(1, 0, Pt_array, NumOfPos, 1, 4, 0);$ (* 投凸轮，循环立即启动 *)

$X_RESULT := HMC_Forward(0);$ (* 主轴启动 *)

运行任务。示波器显示如下：



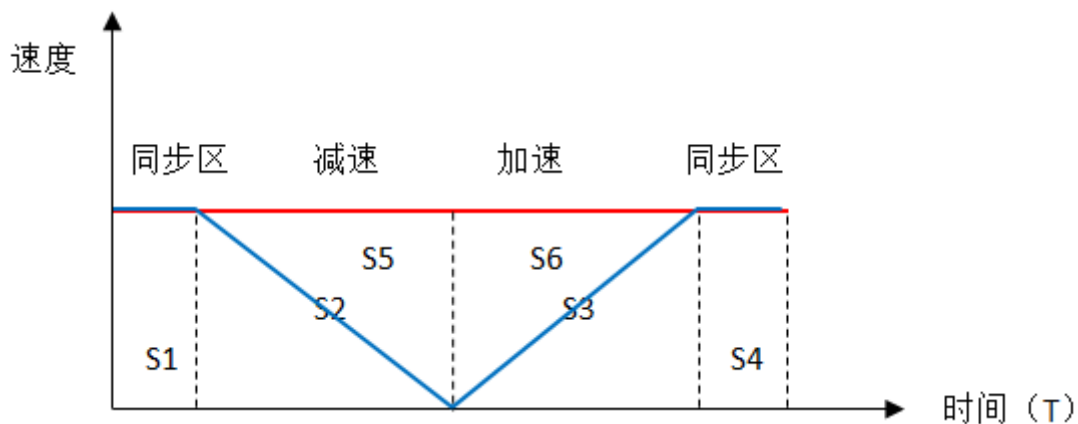
示波器显示结果

3) 情形三：长料剪切

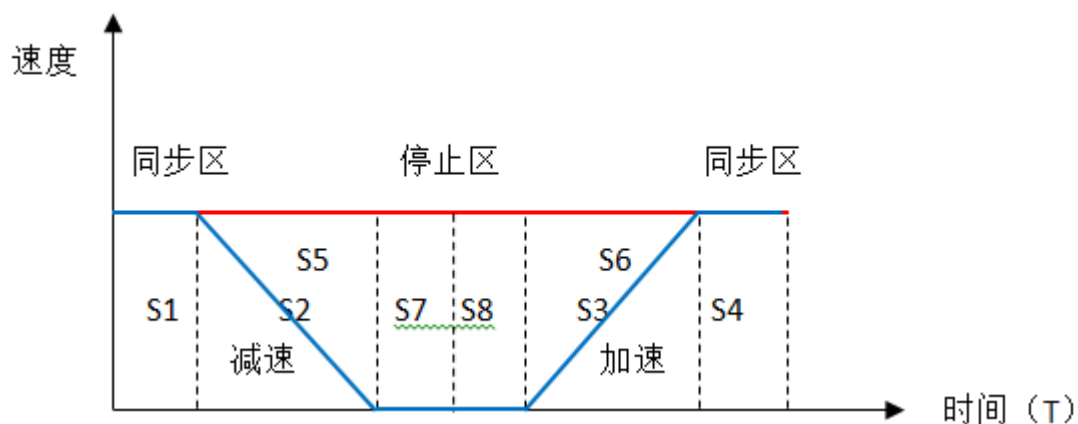
先计算刀辊刚好停止时（长料剪切与中料剪切分界点），剪切的长度(红色包围的面积)：

$L0 = S1 + S2 + S3 + S4 = (S1 + S4) + 2 * (S5 + S6) = (600 - 150) * 2 + 150 = 1050\text{ mm}$

当 $L > 1050\text{ mm}$ 时，刀辊出现停止等待区域，属长料剪切。



长料剪切与中料剪切分界点计算



长料剪切示意图

假设 $L=1500\text{ mm}$ ，那么刀辊停止等待时，主轴走过的距离：

$$S7+S8 = L - L0 = 1500 - 1050 = 450\text{ mm}$$

故有：

$$dSuperimposeIn = dSuperimposeOut = S1 = S4 = 75\text{mm}$$

$$dSlaveBaseDistance = dMasterDistance = L = 1500\text{ mm}$$

$$dSlaveSuperimpose = \text{刀辊周长} - L = 600 - 1500 = -900\text{mm}$$

$$dSuperimposeAcc = dSuperimposeDec = (L - (S1+S4) - (S7+S8)) / 2 = (1500 - 150 - 450) / 2 = 450$$

具体的运动程序如下：

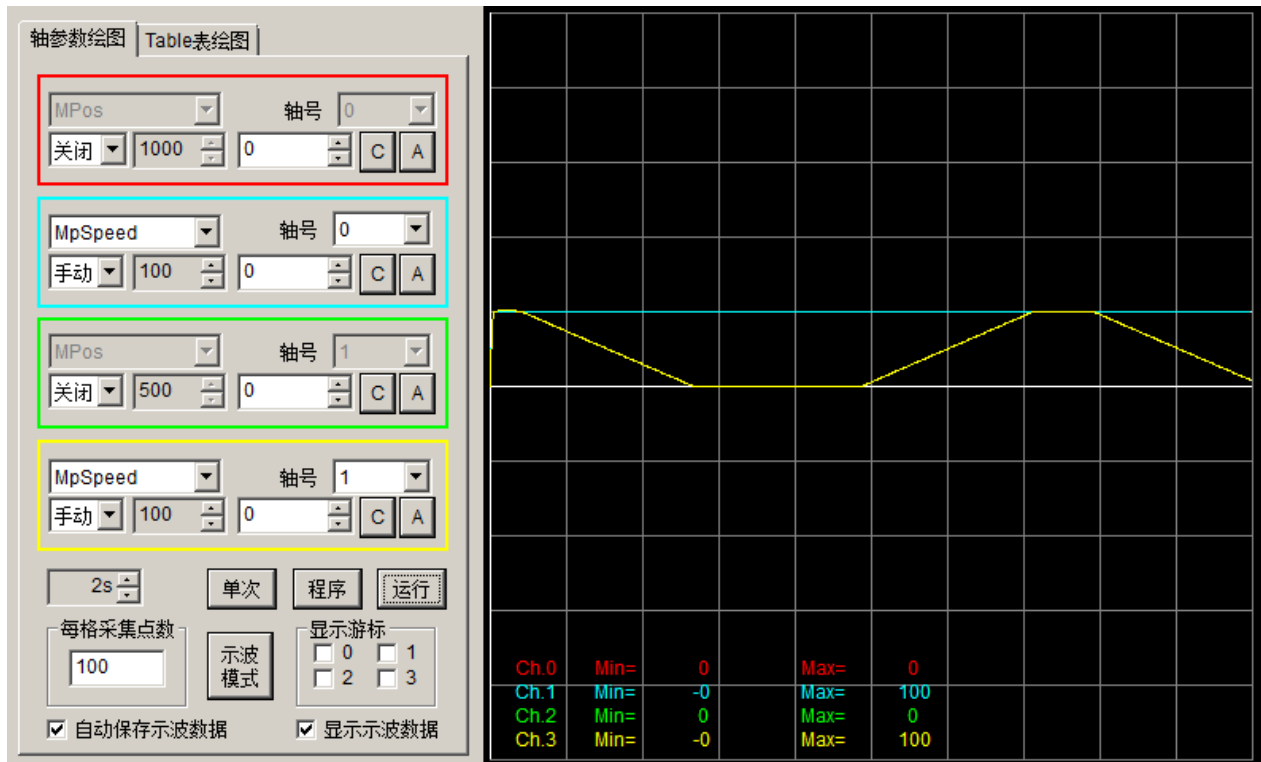
```
FlexLinkReVal := HMC_ClcFlexLinkData(1500, -900, 1500, 75, 75, 450, 450, Pt_array, NumOfPos);
```

```
fff := HMC_TriggerScope(); (* 示波器使能 *)
```

```
X_RESULT := HMC_Forward(0); (* 主轴启动 *)
```

```
camid := HMC_Cam(1, 0, Pt_array, NumOfPos, 1, 4, 0);
```

运行任务。示波器显示：



示波器显示结果

- 不支持的模块版本

MC1008-A01

3.4.1.4.6.6 HMC_CancelREPCam（循环凸轮条件撤销控制）

函数原型

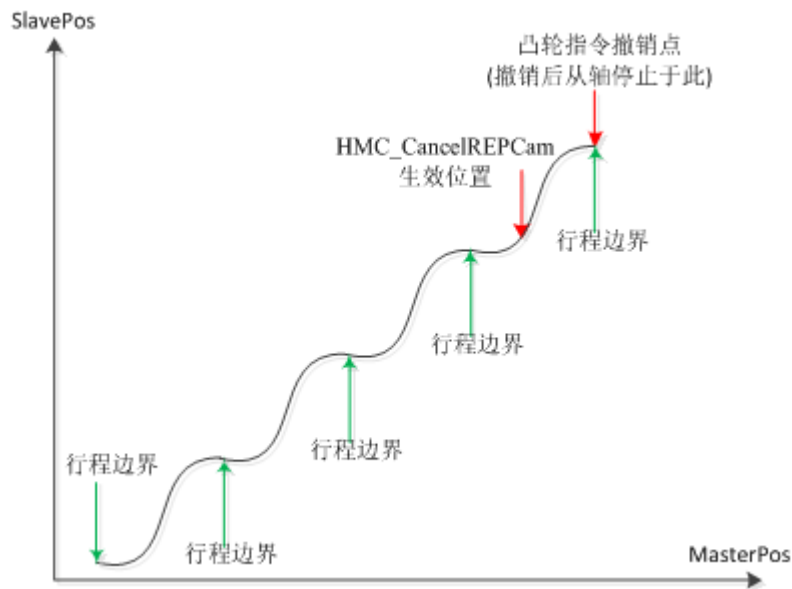
UDINT HMC_CancelREPCam(USINT ucSlaveAxis, USINT ucCancelCycMode)

功能说明

对正在运行的循环凸轮联动指令使用条件撤销功能，功能生效时，循环凸轮指令将在完成当前凸轮完整行程后撤销。

与 HMC_Cancel 指令直接撤销当前指令不同，HMC_CancelREPCam 会等待循环凸轮指令完成当前凸轮完整行程后才撤销当前凸轮指令，凸轮从轴会跟随主轴运行至凸轮主轴行程边界处，撤销后凸轮从轴准确停止在凸轮主轴行程边界处所对应的从轴位置处。

HMC_CancelREPCam 有两种模式：使能撤销与禁止撤销。HMC_CancelREPCam 生效后，会等待循环凸轮指令完成当前凸轮完整行程后才撤销当前凸轮指令，在凸轮主轴运行至行程边界之前，尚可撤回已生效的 HMC_CancelREPCam 指令，只需调用 HMC_CancelREPCam(ucSlaveAxis,0)即可。



循环凸轮执行 HMC_CancelREPCam 指令示意图

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
ucCancelCycMode	USINT	撤销模式。 1: 使能撤销 0: 禁止撤销

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令

■ 补充说明

HMC_CancelREPCam 生效后，凸轮从轴会跟随主轴运行至凸轮主轴行程边界处，然后直接停止，若此时凸轮从轴速度不为 0，则会造成较大冲击。请设计合适的主从轴位置曲线，确保在行程边界处从轴速度为 0 或较小。

注：可采用 HMC_ClcMoveLinkData、HMC_ClcFlexLinkData 指令自动生成主从轴位置数组来满足行程边界处从轴速度为 0 的要求。

应用举例

使用 HMC_ClcMoveLinkData 生成主从轴位置数组，保存在用户自定义数组 arrPos 中，然后使用 arrPos 中的主从轴位置数组投放 HMC_Cam 凸轮指令，模式为循环凸轮，启动方式为立即启动。使用 Hmc_Move 驱动主轴运动，延时 7 秒后使用 HMC_CancelREPCam 撤销循环凸轮。

程序如下：

MasterAxisID := 1; (*主轴轴号*)

SlaveAxisID := 0; (*从轴轴号*)

HMC_TriggerScope(); (*程序触发示波器开始示波采样*)

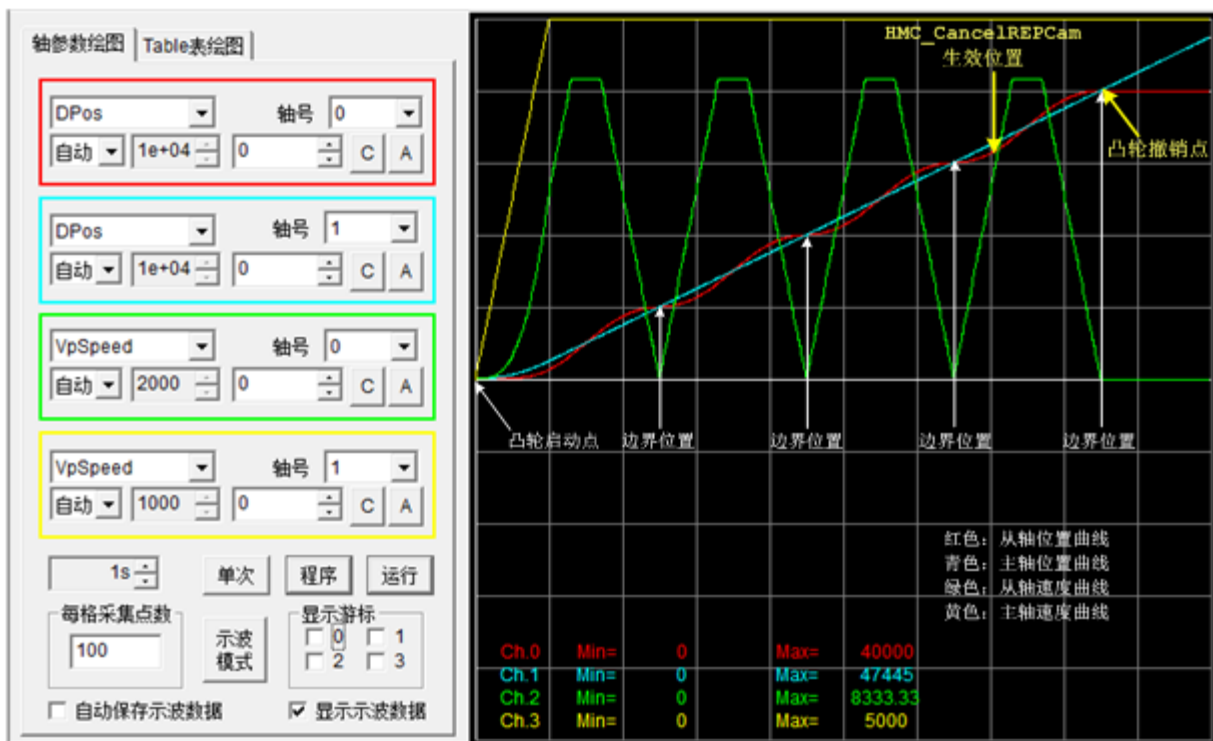
HMC_ClcMoveLinkData(10000,10000,4000,4000,ADR(arrPos),100); (*生成主从轴位置数组，保存在用户自定义数组 arrPos 中*)

HMC_Cam(SlaveAxisID,MasterAxisID,ADR(arrPos),100,1,4,0); (*使用 arrPos 中的主从轴位置数组投放凸轮指令，模式为循环凸轮，启动方式为立即启动*)

Hmc_Move(MasterAxisID,200000); (*驱动主轴运动*)

TimeDelay(7000); (*延时 7 秒*)

HMC_CancelREPCam(SlaveAxisID,1); (*撤销循环凸轮*)



循环凸轮执行 HMC_CancelREPCam 指令运行结果图

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.4.6.7 HMC_MoveLink (追剪运动)

函数原型

```
UDINT HMC_MoveLink (LREAL dSlaveDistance,  
LREAL dMasterDistance,  
LREAL dMasterDistAcc,  
LREAL dMasterDistDec,  
LREAL dSRatio,  
USINT ucSlaveAxis,  
USINT ucMasterAxis,  
USINT ucMode,  
LREAL dTrigMes)
```

功能说明

该指令集成了凸轮和追剪运算功能，控制从轴跟随主轴按一定规律运动，可用在追剪场景中，快速生成追剪所需的动作流程，无需进行复杂的中间计算与编程。

该运动带有分离可变的加减速阶段。各个阶段参数设定方式可参考[HMC_ClcMoveLinkData \(追剪运算\)](#)，此处不再重复。

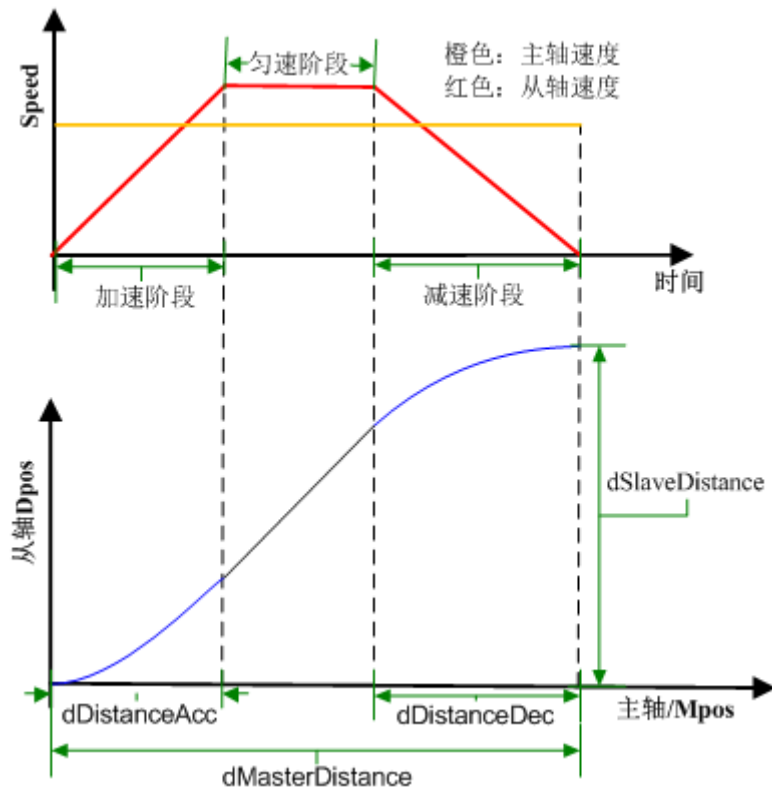
该功能块支持加减速阶段按梯形或 S 形曲线加减速两种方式。且不采用计算生成主从轴位置数组功能块，然后再线性插的方式控制从轴跟随主轴；而是根据设定好的加减速阶段距离、加减速方式等，选择梯形、S 形加减速模型，之后实时根据主轴位置计算从轴位置。这种方法可避免因主从轴位置数组选取点数少时，带来的速度阶梯。

加减速阶段规划方式：

(1) 梯形加减速 dSRatio=0

当 dSRatio=0 时，算法将对从轴的速度按照梯形曲线加减速规划，根据梯形模型、主轴位置实时计算本周期从轴的目标位置。

梯形加减速时，以主轴匀速运动为例，主、从轴的速度、位移曲线示意图如下



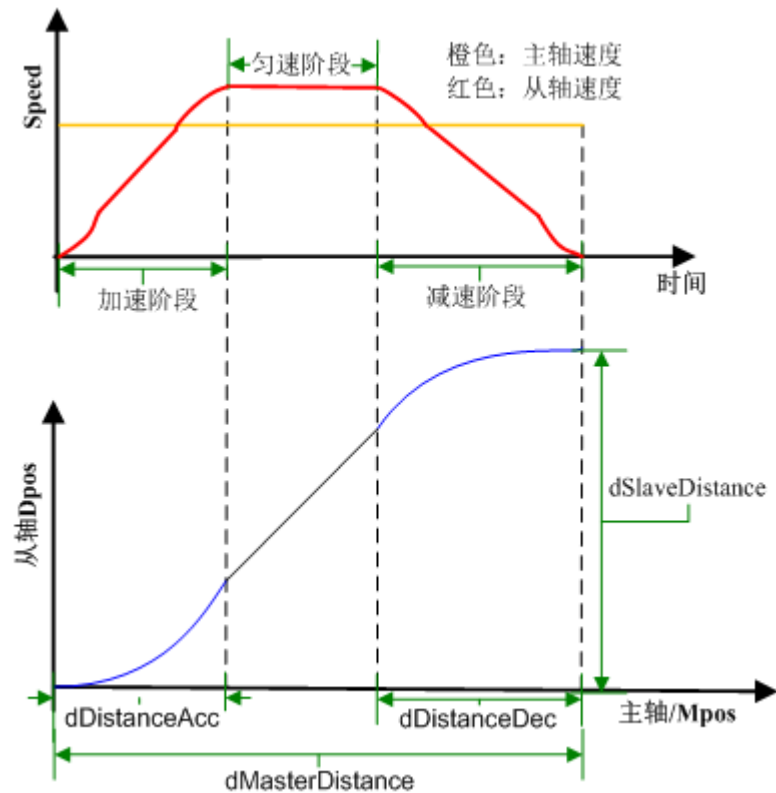
运动示意图

(2) S 型曲线加减速 dSRatio 在(0,1]

当 dSRatio 在(0,1]范围时，算法将对从轴的速度按照 S 形曲线加减速规划，根据 S 型模型、主轴位置实时计算本周期从轴的目标位置。

dSRatio 表示 S 形加减速在整个加速/减速过程所占比例，以加速阶段为例，加加速阶段所用时间占整个加速阶段时间的比例为 $dSRatio/2$ ，匀加速阶段所用时间占整个加速阶段时间的比例为 $1-dSRatio$ ，减加速阶段所用时间占整个加速阶段时间的比例为 $dSRatio/2$ 。

以 dSRatio=0.5，主轴匀速运动为例，主、从轴的速度、位移曲线示意图如下：



运动示意图

当设置 $dSRatio=1$ 时，加减速阶段就没有匀加速/匀减速阶段，加速阶段的加加速和减加速比例分别为 0.5，减速阶段类似。

输入参数

输入参数	类型	参数描述
dSlaveDistance	LREAL	从轴运动距离
dMasterDistance	LREAL	主轴运动距离（必须为正）
dMasterDistAcc	LREAL	在从轴加速阶段主轴移动的正向距离，非负
dMasterDistDec	LREAL	在从轴减速阶段主轴移动的正向距离，非负
dSRatio	LREAL	从轴加速/减速阶段中 S 形加减速所占的比例 取值范围：[0,1]
ucSlaveAxis	USINT	从轴轴号
ucMasterAxis	USINT	主轴轴号
ucMode	USINT	0: 单次立即启动 1: 单次固定位置启动 2: 单次事件启动 3: 单次 DI 启动 4: 循环立即启动 5: 循环固定位置启动 6: 循环事件启动 7: 循环 DI 启动 13: 循环固定位置启动，主轴负向运动时启动 29: 单次固定位置启动，主轴负向运动时启动
dTrigMes	LREAL	启动信息： 1: 凸轮为位置启动时，该值表示主轴的启动位置 2: 凸轮为事件触发启动时，表示某高速捕捉通道号

输入参数	类型	参数描述
		3: 凸轮为 DI 为 1 触发启动时, 表示 DI 通道 (0~63)

指令类型

轴运动缓存指令

应用举例

功能块各个参数及算法方式同 HMC_ClcMoveLinkData, 区别仅在于加速方式可选, 因此应用举例可参数 HMC_ClcMoveLinkData。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

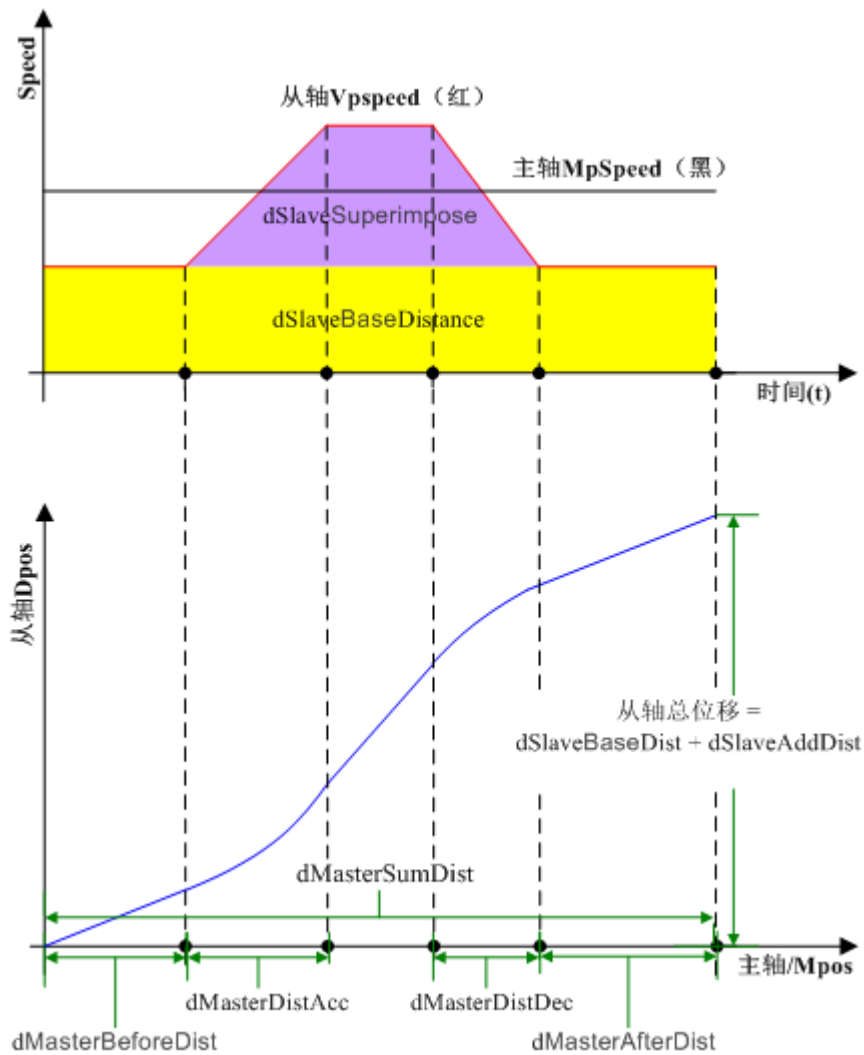
3.4.1.4.6.8 HMC_MoveFlexLink (飞剪/旋切运动)

函数原型

UDINT HMC_MoveFlexLink(LREAL dSlaveBaseDist, LREAL dSlaveAddDist, LREAL dMasterSumDist, LREAL dSlaveBeforeDist, LREAL dSlaveAfterDist, LREAL dMasterAccDist, LREAL dMasterDecDist, LREAL dSRatio, USINT ucSlaveAxis, USINT ucMasterAxis, USINT ucMode, LREAL dTrigMes)

功能说明

完成具有主从轴位置及加减速规划的凸轮功能 (dSRatio 设置 0 时为梯形加减速; S 形加减速所占的比例可通过 dSRatio 参数调整), 可用在追剪、飞剪、旋切场景。



主/从轴示意图

输入参数

输入参数	类型	含义
dSlaveBaseDist	LREAL	从轴恒速跟随距离，正负均可
dSlaveAddDist	LREAL	从轴叠加运动总距离，正负均可
dMasterSumDist	LREAL	主轴运动总距离（必须为正）
dMasterBeforeDist	LREAL	叠加运动（dSlaveAddDist）开始时主轴已完成正向距离，非负
dMasterAfterDist	LREAL	叠加运动（dSlaveAddDist）结束时主轴剩余正向距离，非负
dMasterDistAcc	LREAL	从轴叠加运动加速阶段主轴移动的正向距离，非负
dMasterDistDec	LREAL	从轴叠加运动减速阶段主轴移动的正向距离，非负
dSRatio	LREAL	dSRatio=0 时，从轴为梯形加减速； $dSRatio \in (0,1]$ 时，从轴执行 S 形加减速，S 形加减速所占的比例由 dSRatio 确定 取值范围：[0,1]
ucSlaveAxis	USINT	从轴轴号
ucMasterAxis	USINT	主轴轴号
ucMode	USINT	0：单次立即启动 1：单次固定位置启动

输入参数	类型	含义
		2: 单次事件启动 3: 单次 DI 启动 4: 循环立即启动 5: 循环固定位置启动 6: 循环事件启动 7: 循环 DI 启动 13: 循环固定位置启动, 主轴负向运动时启动 29: 单次固定位置启动, 主轴负向运动时启动
dTrigMes	LREAL	启动信息: (1) 凸轮为位置启动时, 该值表示主轴的启动位置 (2) 凸轮为事件触发启动时, 表示某高速捕捉通道号 (3) 凸轮为 DI 为 1 触发启动时, 表示 DI 通道 (0~63)

返回值

返回值	类型	含义
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

■ 补充说明

当 dSlaveBaseDist、dMasterBeforeDist、dMasterAfterDist 均为 0 时, 即与 HMC_MoveLink 功能等价。

3.4.1.5 高级应用指令

3.4.1.5.1 运动叠加

3.4.1.5.1.1 HMC_Superimpose (叠加)

函数原型

UDINT HMC_Superimpose(USINT ucSlaveAxis, USINT ucMasterAxis0, USINT ucMasterAxis1)

功能说明

该函数可实现两个主轴增量位移的简单叠加, 叠加结果通过从轴输出。叠加功能生效后, 设两个主轴增量位移分别为 $\Delta Mpos_M0$ 、 $\Delta Mpos_M1$, 则从轴 Dpos 增量位移为:

$$\Delta Mpos_S = \Delta Mpos_M0 + \Delta Mpos_M1$$

使用叠加功能时, 两个主轴执行正常运动控制指令, 从轴跟随两个主轴位置的叠加结果作相应的联动动作。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号

输入参数	类型	参数描述
ucMasterAxis0	USINT	第一个主轴轴号
ucMasterAxis1	USINT	第二个主轴轴号

返回值

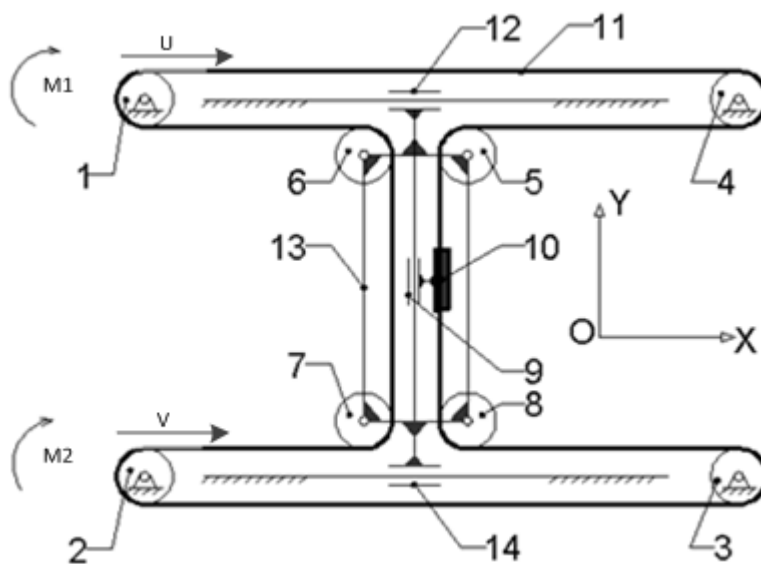
返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

应用举例

示例 1：H 形同步齿形带并联机构运动控制



- H 形并联机构简图
- 1、2、3、4——静轮（固定于基座，选择 1、2 做驱动轮 M1、M2）
- 5、6、7、8——动轮
- 9、12、14——移动副
- 10——滑块
- 11——同步齿形带
- 13——动平台

设用户坐标空间为 X-Y，电机坐标空间为 U-V。通过机构学分析，可得出该机构的正反解运动学方程如下：

正解：

$$\begin{cases} \Delta x = (-\Delta u + \Delta v) / 2 \\ \Delta y = -(\Delta u + \Delta v) / 2 \end{cases}$$

逆解:

$$\begin{cases} \Delta u = -(\Delta x + \Delta y) \\ \Delta v = \Delta x - \Delta y \end{cases}$$

现欲实现在用户坐标空间 X-Y 中编写正常的用户程序, 驱动电机在电机坐标空间 U-V 运行至相应的位置。根据该机构运动学反解模型, 用 HMC_Superimpose 与 HMC_Gear (电子齿轮) 功能实现该并联机构控制的程序如下 (设 X-Y 对应主轴 0、1; U-V 对应从轴 2、3 (电机轴); 10、11 号轴用来完成电子齿轮变换功能 (实现 X、Y 轴换向)):

(**主轴设置为虚轴。**)

```
HMC_SetAxisType(Axis0.AxisID, 0);
```

```
HMC_SetAxisType(Axis1.AxisID, 0);
```

(**完成电子齿轮变换功能的 10、11 号轴设置为虚轴。**)

```
HMC_SetAxisType(Axis10.AxisID, 0);
```

```
HMC_SetAxisType(Axis11.AxisID, 0);
```

(**从轴 (电机轴) 设置为步进脉冲输出轴。**)

```
HMC_SetAxisType(Axis2.AxisID, 10);
```

```
HMC_SetAxisType(Axis3.AxisID, 10);
```

(**设定主轴运动参数**)

```
Axis0.Speed := 1000; (*设定 0 轴运动速度*)
```

```
Axis0.Accel := 2000; (*设定 0 轴加速度*)
```

```
Axis0.Decel := 2000; (*设定 0 轴减速度*)
```

```
Axis1.Speed := 1000; (*设定 1 轴运动速度*)
```

```
Axis1.Accel := 2000; (*设定 1 轴加速度*)
```

```
Axis1.Decel := 2000; (*设定 1 轴减速度*)
```

(*投送 HMC_Gear 指令, 10 号轴为从轴, 0 号轴为主轴, 齿轮比为-1 (X 轴换向)*)

```
HMC_Gear(Axis10.AxisID, Axis0.AxisID,-1,1);
```

(*投送 HMC_Gear 指令, 11 号轴为从轴, 1 号轴为主轴, 齿轮比为-1 (Y 轴换向)*)

```
HMC_Gear(Axis11.AxisID, Axis1.AxisID,-1,1);
```

(*投送 HMC_SuperImpose 指令, 2 号轴为从轴, 10、11 号轴为主轴。*)

```
HMC_Superimpose(Axis2.AxisID,Axis10.AxisID, Axis11.AxisID);
```

(*投送 HMC_SuperImpose 指令, 3 号轴为从轴, 0、11 号轴为主轴。*)

```
HMC_Superimpose(Axis3.AxisID,Axis0.AxisID, Axis11.AxisID);
```

(*之后可向主轴 0、1 投放所需的运动控制指令, 驱动主轴运动, 从轴 2、3 将驱动电机运动至相应位置。*)

有关该机构更为简单的控制实现方法, 可参考 HMC_SuperImposeEx 中十字形并联机构运动控制示例。

示例 2: 同步跟踪点胶

待点胶工件通过传送带传送, 直角坐标机构负责完成工件表面的点胶, 点胶轨迹为不规则几何曲线。为提高生产效率, 要求在传送带保持正常运转的情况下, 点胶头同步完成传送带上工件的点胶。

直角坐标机构 X、Y、Z 对应的轴号分别为 0、1、2, 安装时保证直角坐标机构的 X 轴与传送带平行。轴 4 用于 X 方向叠加运动。同步动作开始时, 通过使用 HMC_Superimpose 指令, 把轴 4 的运动叠加在传送带轴 (轴 3) 的运动之上, 叠加结果通过 X 轴 (轴 0) 电机输出。

通过运动叠加, 在同步运动开始之后, 用户只需对轴 4、1、2 (对应 X、Y、Z) 进行编程即可, 点胶时只需按照点胶轨迹的原始几何曲线控制轴 4、1、2 进行插补运动, 而无需考虑传送带的附加运动, X 轴方向的同步跟踪动作由叠加功能自动完成。

(*设定轴类型。Stepper 轴——直角坐标 X 轴:轴 0, Y 轴:轴 1, Z 轴:轴 2, 传送带轴:轴 3。虚轴——用于 X 方向叠加运动的轴: 轴 4*)

```
HMC_SetAxisType(Axis0.AxisID, 10);
```

```
HMC_SetAxisType(Axis1.AxisID, 10);
```

```
HMC_SetAxisType(Axis2.AxisID, 10);
```

```
HMC_SetAxisType(Axis3.AxisID, 10);
```

```
HMC_SetAxisType(Axis4.AxisID, 0); (*用于 X 方向叠加运动的轴设定为虚轴*)
```

(*设定各轴速度、加减速参数。(略)*)

(*检测工件是否到来, 没有到来时等待(略)*)

(*工件到来, 机械手首先加速到传送带相同的速度(略)*)

(*投送 HMC_Superimpose 指令, 0 号轴(X 轴)为从轴, 3 号轴(传送带轴)、4 号轴(圆弧插补轨迹计算轴)为主轴。*)

HMC_Superimpose(Axis0.AxisID, Axis3.AxisID, Axis4.AxisID);

(*之后控制机械手运动时对应的 X、Y、Z 轴轴号分别为: 4、1、2, 无需考虑传送带的附加运动, X 轴方向的同步跟踪动作由叠加功能自动完成*)

(*首先移动机械手到工件涂胶起始点 (略)*)

(*按照点胶轨迹的原始几何曲线控制轴 4、1、2 进行插补运动(略)*)

3.4.1.5.1.2 HMC_SuperImposeEx (高级叠加)

函数原型

```
UDINT HMC_SuperImpose(USINT ucSlaveAxis,
    USINT ucMasterAxis0,
    USINT ucMasterAxis1,
    DINT iRatioNumerator0,
    DINT iRatioDenominator0,
    DINT iRatioNumerator1,
    DINT iRatioDenominator1)
```

功能说明

该函数可实现两个主轴增量位移的线性叠加, 叠加结果通过从轴输出。叠加功能生效后, 设两个主轴增量位移分别为 $\Delta Mpos_M0$ 、 $\Delta Mpos_M1$, 则从轴 Dpos 增量位移为:

$$\Delta Dpos_s = \left(\frac{iRatioNumerator0}{iRatioDenominator0} \right) \times \Delta Mpos_{M0} + \left(\frac{iRatioNumerator1}{iRatioDenominator1} \right) \times \Delta Mpos_{M1}$$

支持用户动态修改叠加比例 (详见示例 3: 电子齿轮箱实现斜齿轮滚齿加工)。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
ucMasterAxis0	USINT	第一个主轴轴号
ucMasterAxis1	USINT	第二个主轴轴号
iRatioNumerator0	DINT	第一个主轴的叠加比例的分子

输入参数	类型	参数描述
iRatioDenominator0	DINT	第一个主轴的叠加比例的分母
iRatioNumerator1	DINT	第二个主轴的叠加比例的分子
iRatioDenominator1	DINT	第二个主轴的叠加比例的分母

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

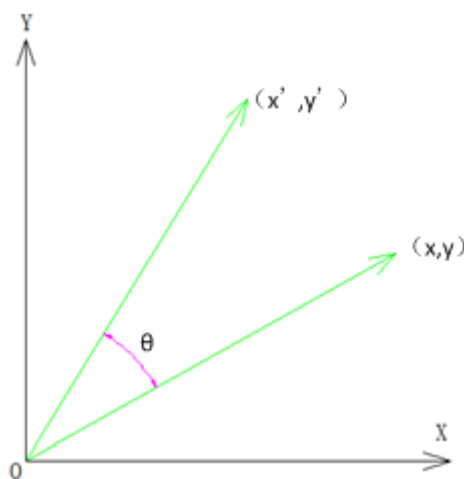
■ 补充说明

从数学关系上看，HMC_SuperImpose 和 HMC_Gear 均是 HMC_SuperImposeEx 的特例；

在该指令执行过程中，可以调用 HMC_SetSuperImposeRatio 函数动态修改叠加比例。

应用举例

示例 1：坐标变换



坐标变换示意图

轴 0、1 的合运动轨迹坐标点(x,y)与轴 2、3 的合运动轨迹坐标点(x',y')存在如下关系：(x,y)逆时针旋转 θ 得到(x',y')。由坐标变理论可知：

$$\begin{cases} x' = x \cos \theta - y \sin \theta \\ y' = x \sin \theta + y \cos \theta \end{cases}$$

使用 HMC_SuperImposeEx 可实现该两组坐标的变换。程序如下(设 $\theta=45^\circ$)：

(**设定主轴运动参数**)

Axis0.Speed := 1000; (*设定 0 轴运动速度*)

Axis0.Accel := 2000; (*设定 0 轴加速度*)

Axis0.Decel := 2000; (*设定 0 轴减速度*)

Axis1.Speed := 1000; (*设定 1 轴运动速度*)

Axis1.Accel := 2000; (*设定 1 轴加速度*)

Axis1.Decel := 2000; (*设定 1 轴减速度*)

(*设置当前点为零点*)

HMC_DefPos(Axis0.AxisID,0);

HMC_DefPos(Axis1.AxisID,0);

HMC_DefPos(Axis2.AxisID,0);

HMC_DefPos(Axis3.AxisID,0);

(*投送 HMC_SuperImpose 指令，2 号轴为从轴，0、1 号轴为主轴。 $\sin 45^\circ = \cos 45^\circ \approx 707/1000$ *)

HMC_SuperImposeEx(Axis2.AxisID, Axis0.AxisID, Axis1.AxisID, 707, 1000, -707, 1000);

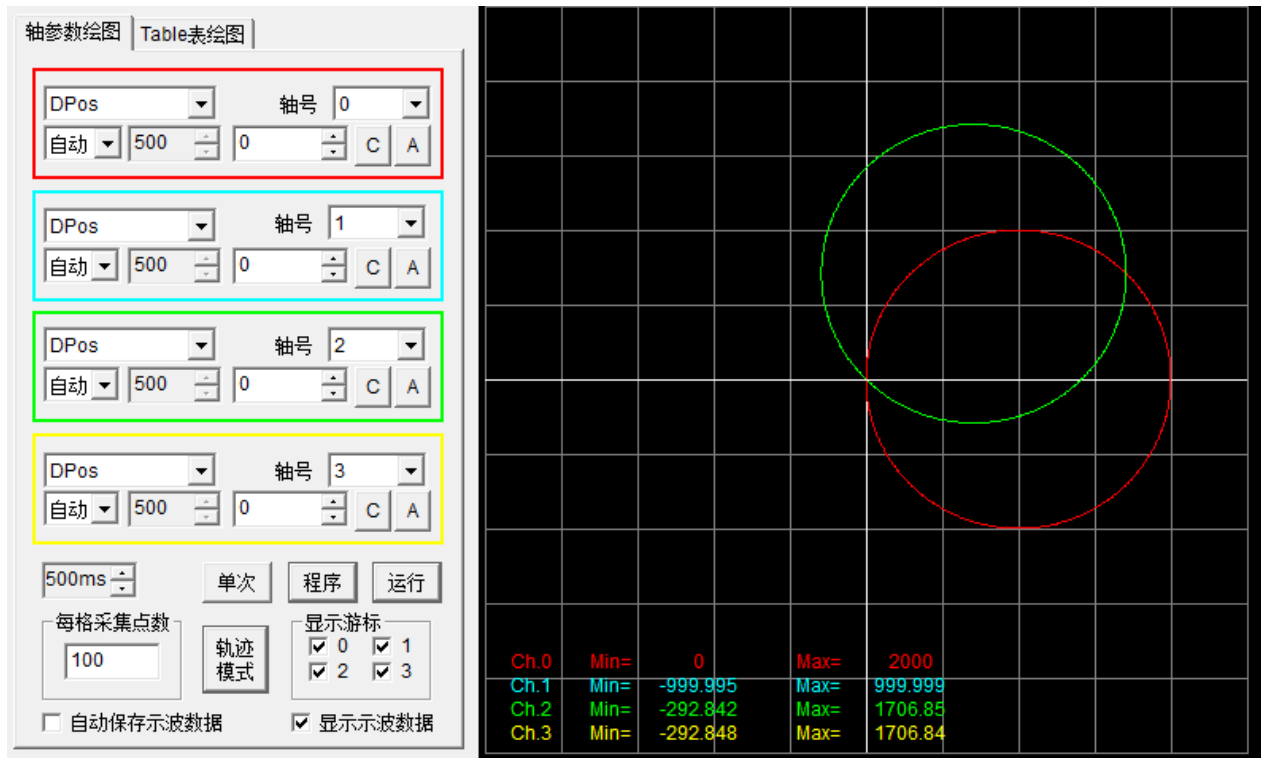
(*投送 HMC_SuperImpose 指令，3 号轴为从轴，0、1 号轴为主轴。*)

HMC_SuperImposeEx(Axis3.AxisID, Axis0.AxisID, Axis1.AxisID, 707, 1000, 707, 1000);

(*之后向主轴 0、1 投放圆弧插补指令指令，驱动主轴运动（红色），从轴 2、3 将输出变换后的结果（绿色）。*)

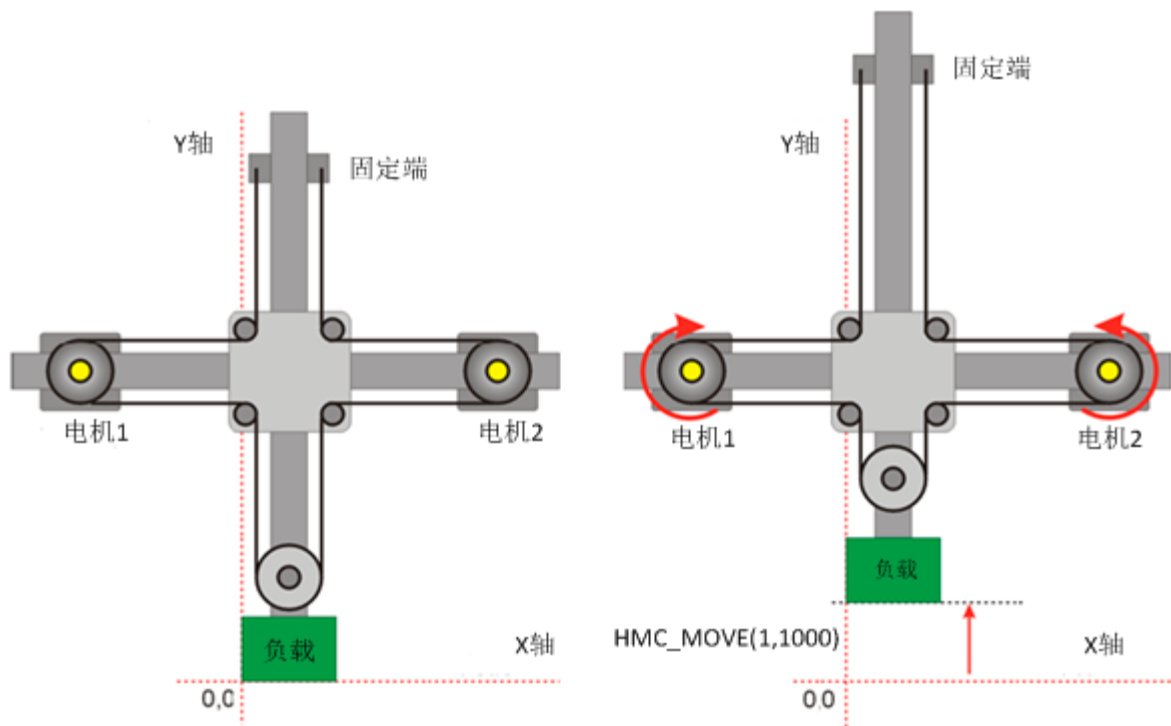
HMC_MoveCircle(10, Axis0.AxisID, Axis1.AxisID, 1, 0, 0, 1000, 0);

AT 中执行结果如下：



HMC_SuperImposeEx 实现坐标变换

示例 2: 十字形同步齿形带并联机构运动控制



十字形同步齿形带并联机构

设用户坐标空间为 X-Y，电机坐标空间为 U-V。通过机构学分析，可得出该机构的正反解运动学方程如

下:

正解:

$$\begin{cases} \Delta x = 0.5 * \Delta u + 0.5 * \Delta v \\ \Delta y = 0.5 * \Delta u - 0.5 * \Delta v \end{cases}$$

反解:

$$\begin{cases} \Delta u = \Delta x + \Delta y \\ \Delta v = \Delta x - \Delta y \end{cases}$$

现欲实现在用户坐标空间 X-Y 中编写正常的用户程序, 驱动电机在电机坐标空间 U-V 运行至相应的位置。用 HMC_SuperimposeEx 实现的程序如下(设 X-Y 对应主轴 0、1, U-V 对应从轴 2、3):

(**主轴设置为虚轴。**)

HMC_SetAxisType(Axis0.AxisID, 0);

HMC_SetAxisType(Axis1.AxisID, 0);

(**从轴设置为步进脉冲输出轴。**)

HMC_SetAxisType(Axis2.AxisID, 10);

HMC_SetAxisType(Axis3.AxisID, 10);

(**设定主轴运动参数**)

Axis0.Speed := 1000; (*设定 0 轴运动速度*)

Axis0.Accel := 2000; (*设定 0 轴加速度*)

Axis0.Decel := 2000; (*设定 0 轴减速度*)

Axis1.Speed := 1000; (*设定 1 轴运动速度*)

Axis1.Accel := 2000; (*设定 1 轴加速度*)

Axis1.Decel := 2000; (*设定 1 轴减速度*)

(*投送 HMC_SuperImpose 指令, 2 号轴为从轴, 0、1 号轴为主轴。*)

HMC_SuperimposeEx(Axis2.AxisID, Axis0.AxisID, Axis1.AxisID, 1, 1, 1, 1);

(*投送 HMC_SuperImpose 指令, 3 号轴为从轴, 0、1 号轴为主轴。*)

HMC_SuperimposeEx(Axis3.AxisID, Axis0.AxisID, Axis1.AxisID, 1, 1, -1, 1);

(*之后可向主轴 0、1 投放所需的运动控制指令, 驱动主轴运动, 从轴 2、3 将驱动电机运动至相应位置。*)

示例 3：电子齿轮箱实现斜齿轮滚齿加工

数控滚齿机和插齿机以电子齿轮箱取代了原始的机械式内联传动链，与常规的机械内联传动链相比，采用电子齿轮箱的数控齿轮加工机床传动链缩短，提高了传动刚度和传动精度；各向运动轴既可单独动作也可多轴联动，加工过程由程序控制，运动灵活、定位准确、精度高、效率高，能加工普通齿轮加工机床无法加工的零件。

HMC_SuperImposeEx 指令可实现数控滚齿机和插齿机所需的电子齿轮箱功能，以下以滚齿机为例分析。

首先针对滚齿机床各轴运动特征，建立无轴传动电子齿轮箱控制的数学模型。滚齿机床各运动轴为：主运动轴 B 轴（滚刀主轴）、工件轴（C 轴）、X 轴（径向进给轴）、Y 轴（窜刀轴）和 Z 轴（轴向进给轴）。

设滚刀（B 轴）头数为 z_B ，转速为 n_B ；工件（C 轴）齿数为 z_C 。在用“差动法”加工斜齿轮或采用“对角滚切法”加工齿轮时，机床工件主轴与机床刀具主轴之间不仅要有准确的速比关系，在滚刀轴有 Z 向进给或 Y 向连续窜刀运动时，工件主轴还要完成对 Z 轴或 Y 轴的准确跟随，使滚刀与工件之间保持严格的展成运动。选取工件轴作为电子齿轮箱的从动轴，其运动速度可描述为：

$$n_C = K_B \frac{z_B}{z_C} n_B + K_Z \frac{\sin \beta}{\pi m_n z_C} v_Z + K_Y \frac{\cos \lambda}{\pi m_n z_C} v_Y$$

式中， v_Y 、 v_Z 分别为 Y 轴、Z 轴的移动速度，mm/min； β 为斜齿轮的螺旋角； λ 为刀具的安装角； m_n 为齿轮的法面模数； K_B 、 K_Z 、 K_Y 为系数。

由上式可知，滚齿加工时，C 轴不仅与 B 轴速度有关，Y 轴和 Z 轴的运动也会产生 C 轴的附加运动。假设 Y 轴与 Z 轴均采用伺服电机+滚珠丝杠的传动方式实现，则上述公式可简化为(最简分数形式)：

$$n_C = \frac{i_{Bn}}{i_{Bd}} n_B + \frac{i_{Zn}}{i_{Zd}} n_Z + \frac{i_{Yn}}{i_{Yd}} n_Y$$

n_B 、 n_C 、 n_Y 、 n_Z 分别为 B、C、Y、Z 轴的驱动电机转速。

设 B、C、X、Y、Z 轴的驱动电机对应的轴号分别为 0、1、2、3、4，用 HMC_SuperImposeEx 指令实现该运动关系的示例程序如下（ST 语言）：

(**设置 B、C、X、Y、Z 轴为闭环伺服轴。**)

HMC_SetAxisType(Axis0.AxisID, 20);

HMC_SetAxisType(Axis1.AxisID, 20);

HMC_SetAxisType(Axis2.AxisID, 20);

HMC_SetAxisType(Axis3.AxisID, 20);

HMC_SetAxisType(Axis4.AxisID, 20);

(**用于中间运算的轴 5 设置为虚轴。**)


```
HMC_SetAxisType(Axis5.AxisID, 0);
```

(**设定 B、C、X、Y、Z 轴运动参数（略）**)

(**调整设备至加工起始位置（略）**)

(*投送 HMC_SuperImpose 指令，5 号轴为从轴，B 轴（0）、Z 轴（4）为主轴。*)

```
HMC_SuperimposeEx(Axis5.AxisID, Axis0.AxisID, Axis4.AxisID, iBn, iBd, iZn, iZd);
```

(*投送 HMC_SuperImpose 指令，C 轴（1）为从轴，Y 轴（3）、5 号轴为主轴。*)

```
HMC_SuperimposeEx(Axis1.AxisID, Axis3.AxisID, Axis5.AxisID, iYn, iYd, 1, 1);
```

(*通过 X 轴调整合适的径向进给量，然后驱动 B/Y/Z 轴运动，从轴 C 将按上述关系运动，实现斜齿轮范成加工*)

3.4.1.5.1.3 HMC_SetSuperimposeRatio（设置叠加比例）

函数原型

```
UDINT HMC_SetSuperimposeRatio(USINT ucSlaveAxis,  
DINT iRatioNumerator0,  
DINT iRatioDenominator0,  
DINT iRatioNumerator1,  
DINT iRatioDenominator1)
```

功能说明

设置叠加比例，关于叠加详见 [HMC_Superimpose（叠加）](#)、[HMC_SuperImposeEx（高级叠加）](#)。在 HMC_Superimpose 或 HMC_SuperimposeEx 指令启动后，可以实时调用该指令，可以动态改变运转中的叠加比例，从而达到动态改变叠加比例的效果。

该指令只允许当前正在运行 HMC_Superimpose 或 HMC_SuperimposeEx 指令时设置。

输入参数

输入参数	类型	参数描述
ucSlaveAxis	USINT	从轴轴号
iRatioNumerator0	DINT	第一个主轴的叠加比例的分子
iRatioDenominator0	DINT	第一个主轴的叠加比例的分母
iRatioNumerator1	DINT	第二个主轴的叠加比例的分子
iRatioDenominator1	DINT	第二个主轴的叠加比例的分母

返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令。

应用举例

示例程序如下：

Axis0.Speed := 1000; (*设定运动速度*)

Axis0.Accel := 2000; (*设定加速度*)

Axis0.Decel := 2000; (*设定减速度*)

Axis1.Speed := 1000; (*设定运动速度*)

Axis1.Accel := 2000; (*设定加速度*)

Axis1.Decel := 2000; (*设定减速度*)

(*投送 HMC_SuperImpose 指令, 3 号轴为从轴, 0、1 号轴为主轴。*)

HMC_SuperimposeEx(Axis3.AxisID, Axis0.AxisID, Axis1.AxisID, 1, 1, 1, 1);

Cmd_move0 := HMC_Move(Axis0.AxisID, 1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 1000*)

Cmd_move1 := HMC_Move(Axis1.AxisID, 2000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 2000*)

HMC_WaitCmdFinish(Cmd_move0); (* 等待指令完成 *)

HMC_WaitCmdFinish(Cmd_move1); (* 等待指令完成 *)

HMC_SetSuperimposeRatio(Axis3.AxisID, 1, 1, -1, 1); (*改变叠加比例*)

HMC_Move(Axis0.AxisID, 1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离 1000*)

HMC_Move(Axis1.AxisID, -1000); (*向 0 号轴投送 HMC_Move 指令,运动增量距离-1000*)

3.4.1.5.1.4 HMC_GetSuperimposeMaster1 (获取叠加主轴 1)

函数原型

INT HMC_GetSuperimposeMaster1(USINT ucSlaveAxis)

功能描述

获取叠加的第一个主轴。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	从轴轴号

返回值

返回值	类型	参数描述
叠加的第一个主轴	INT	成功
-1	INT	当前未进行叠加运动或函数参数错误

指令类型

非轴运动缓存指令。

3.4.1.5.1.5 HMC_GetSuperimposeMaster2（获取叠加主轴 2）

函数原型

INT HMC_GetSuperimposeMaster2(USINT ucSlaveAxis)

功能说明

获取叠加的第二个主轴。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	从轴轴号

返回值

返回值	类型	参数描述
叠加的第二个主轴	INT	成功
-1	INT	当前未进行叠加运动或函数参数错误

指令类型

非轴运动缓存指令。

3.4.1.5.1.6 HMC_GetSuperimposeRatio1（获取叠加主轴 1 比例）

函数原型

LREAL HMC_GetSuperimposeRatio1(USINT ucSlaveAxis)

功能说明

获取叠加的第一个主轴比例。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	从轴轴号

返回值

返回值	类型	参数描述
叠加的第一个主轴比例	LREAL	成功

指令说明

非轴运动缓存指令。

3.4.1.5.1.7 HMC_GetSuperimposeRatio2（获取叠加主轴 2 比例）

函数原型

LREAL HMC_GetSuperimposeRatio2(USINT ucSlaveAxis)

功能说明

获取叠加的第二个主轴比例。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	从轴轴号

返回值

返回值	类型	参数描述
叠加的第二个主轴比例	LREAL	成功

指令类型

非轴运动缓存指令。

3.4.1.5.1.8 HMC_SetAddAxis（设定和运动轴）

函数原型

UDINT HMC_SetAddAxis(USINT ucBaseAxisID, USINT ucAddAxisID)

功能说明

该指令用于完成二轴运动求和。指令调用成功后，在后续的运动过程中，有：

基础轴位移=基础轴自身指令位移+叠加轴位移

上式中位移均为 Dpos 增量。

当 ucAddAxisID=255 时，表示取消 ucBaseAxisID 轴与其他轴的和运动。

可以通过 HMC_GetAddAxis(ucBaseAxisID)指令查询叠加轴轴号。

输入参数

输入参数	类型	参数描述
ucBaseAxisID	USINT	基础轴轴号，0~63
ucAddAxisID	USINT	叠加轴轴号（有效关联轴号：0~63；255 表示取消关联。）

返回值

返回值	类型	参数描述
成功：0 错误：错误码	UDINT	--

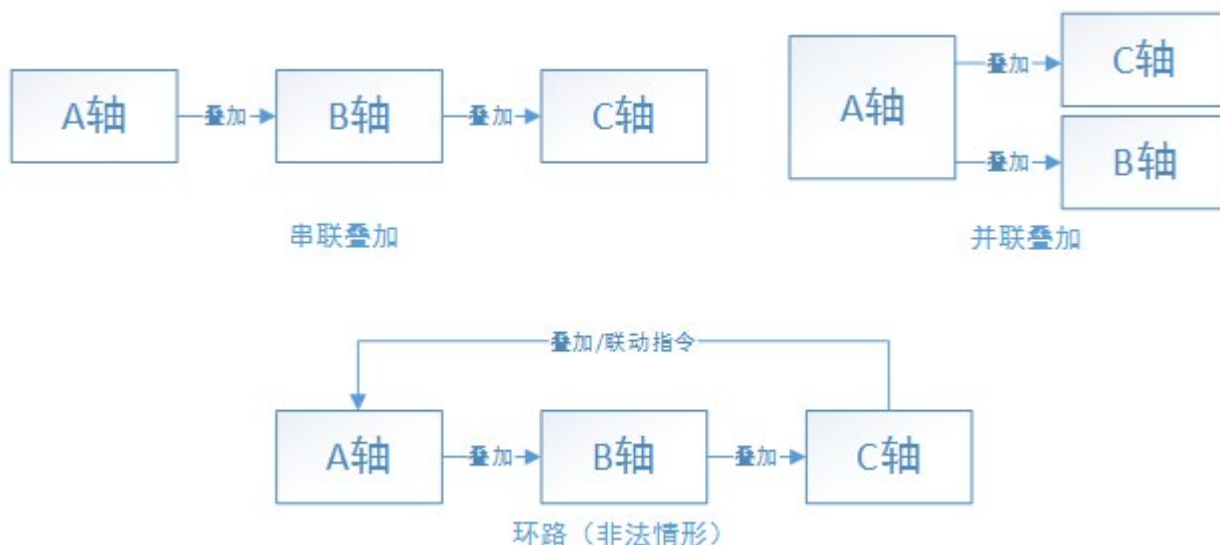
指令类型

非轴运动缓存指令。

■ 补充说明

（1）叠加指令使用时不可形成环路，否则会导致轴速度发散失控。下图依此为正常的串/并联开路情

形、异常的环路情形。因异常环路的多样性，检测比较困难，用户使用时需自行保证。



(2) 指令调用成功后，基础轴的 Dpos/Mpos 反映的是轴实际的位置，MpSpeed 反映的是实际叠加后的速度，但 VpSpeed 为基础轴自身指令速度，而非叠加后的值。

3.4.1.5.1.9 HMC_GetAddAxis (获取和运动叠加轴)

函数原型

UDINT HMC_GetAddAxis(USINT ucBaseAxisID)

功能说明

用于获取和运动叠加轴轴号，255 表示未关联任何叠加轴。

输入参数

输入参数	类型	参数描述
ucBaseAxisID	USINT	基础轴轴号，0~63

返回值

返回值	类型	参数描述
叠加轴轴号	UDINT	255 表示未关联任何叠加轴

指令类型

非轴运动缓存指令。

3.4.1.5.2 高速脉冲捕获

3.4.1.5.2.1 HMC_CaptureConfig (设置高速捕捉)

函数原型

UDINT HMC_CaptureConfig (USINT ucChlID,
USINT bTrigEdge,

```
USINT ucTrigType,
USINT ucTrigID,
USINT ucCaptureAxis,
USINT bWinModeEnable,
LREAL dWinStart,
LREAL dWinEnd)
```

功能说明

该函数主要用来配置高速捕获通道。高速捕获功能主要用来实现通过外部触发信号（高速 DI 或 Z 相）快速获取待捕获轴当前位置的功能，该过程由底层硬件（FPGA）完成，不存在软件处理时的滞后误差，所以具有很高的捕获精度。

用户通过指定待捕获轴、捕获信号的类型等参数，配置高速捕获通道。配置成功后，高速捕获功能立即生效。当捕获信号产生时，高速捕获功能被触发，此时待捕获轴的 Mpos 值被记录下来，之后可通过 HMC_CaptureGetMpos 指令获取此位置。

在高速捕获功能使用过程中，可通过 HMC_CaptureGetState 指令实时查询高速捕获状态，以判断高速捕获通道是否被触发。

当高速捕获功能的窗口模式（bWinModeEnable）禁用时，在任意时刻捕获信号到来之时都将触发高速捕获功能进行位置捕获；当窗口模式（bWinModeEnable）使能时，仅当待捕获轴的 Mpos 值在 dWinStart 与 dWinEnd 之间（窗口范围之内），捕获信号到来时高速捕获功能才会被触发。在该窗口区间之外时，即使捕获信号到来，高速捕获功能也不会被触发。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	高速捕获通道号 MC1008&MC1004&MC1004L (0~7) MC1002R&MC1002E (0~1)
bTrigEdge	USINT	捕获触发信号的有效沿，0：上升沿，1：下降沿
ucTrigType	USINT	捕获触发信号类型，0：轴接口 Z 相信号，1：高速 DI 信号
ucTrigID	USINT	捕获触发信号的通道号，即高速 DI 通道号或 Z 相所在轴号
ucCaptureAxis	USINT	待捕获轴号，当捕获信号有效时获取该轴 MPos 值
bWinModeEnable	USINT	高速捕获窗口使能，0：禁止，1：窗口内使能，2：窗口外使能
dWinStart	LREAL	捕获窗口开启位置，待捕获轴的 Mpos
dWinEnd	LREAL	捕获窗口关闭位置，待捕获轴的 Mpos

返回值

返回值	类型	参数描述
0	UDINT	配置成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

■ 补充说明

- MC1008/MC1004 支持 8 路独立的高速捕获功能，MC1002R/MC1002E 支持 2 路高速捕获。

(2) 选择 DI 作为触发捕获信号时, 只可选择 DI0~DI11 (高速 DI); 选择 Z 相作为触发捕获信号时, 可选择处于空闲状态的 Z 相接口。

(3) 高速捕获待捕获轴可为任意轴类型。当待捕获轴为伺服轴 (20)、增量编码器 (30/31/32)、SSI 编码器 (33) 或安川绝对值编码器时, 捕获过程由硬件 FPGA 完成, 不存在软件滞后偏差, 精度较高; 虚轴、总线轴类型捕获过程由软件完成, 存在时间滞后 (最大一个伺服周期), 捕获精度稍差。

(4) 由于 DI 输入较 Z 相输入偏慢, 因此当用作高速设备时建议选择 Z 相作为捕获触发信号, 当使用低速设备时使用 DI 输入。

(5) 两路独立的高速捕获可同时捕获同一个轴。

(6) 对于高速捕获的窗口, 需满足 $dWinStart \leq dWinEnd$, 可正可负。

应用举例

假如某设备外接一数字量传感器, 算法要获取滑块到达该传感器时的精确位置, 可配置一高速捕获通道, 当检测到传感器有效时, 捕获此时 Mpos 值。

```
axis0.AxisType := 30;    (*设定轴 0 为编码器轴*)
```

```
cap_chID := 0;          (*选择高速捕获通道 0*)
```

```
cap_type := 0;          (*捕获信号选择 Z 相输入*)
```

```
cap_trigID := 0;        (*选择轴 0、Z 相作为捕获信号*)
```

```
cap_edge := 0;          (*选择轴 0 的 Z 相上升沿触发高速捕获*)
```

```
cap_axis := 0;          (*捕获轴 0 的位置*)
```

```
cap_win_mode := 1;      (*使能窗口内有效*)
```

```
cap_winstart := -100;   (*左窗口位置*)
```

```
cap_winend := 1000;     (*右窗口位置*)
```

```
cap_status := HMC_CaptureConfig (cap_chID, cap_edge, cap_type, cap_trigID, cap_axis,  
cap_win_mode, cap_winstart, cap_winend);
```

```
while capture_state <> 3 do    (*等待捕获完成*)
```

```
    capture_state := HMC_CaptureGetState(cap_chID);    (*查询捕获状态*)
```

```
end_while
```

```
capture_pos := HMC_CaptureGetMpos(cap_chID);    (*读取捕获值*)
```

3.4.1.5.2.2 HMC_CaptureGetMpos（获取高速捕捉 Mpos）

函数原型

LREAL HMC_CaptureGetMpos(USINT ucChIID)

功能说明

返回 ucChIID 指定高速捕捉通道获取到的 Mpos。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	高速捕捉通道号

返回值

返回值	类型	参数描述
Mpos 值	LREAL	捕捉到的 Mpos 值

指令类型

非轴运动缓存指令。

3.4.1.5.2.3 HMC_CaptureGetState（获取高速捕捉状态）

函数原型

USINT HMC_CaptureGetState(USINT ucChIID)

功能说明

返回某个高速捕捉通道的状态，参见返回值参数描述。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	高速捕捉通道号

返回值

返回值	类型	参数描述
0	USINT	正在配置
1	USINT	不在窗口期（等待窗口打开，或者窗口已经关闭，此状态时窗口模式应该被使能）
2	USINT	正在等待捕捉信号
3	USINT	空闲，捕捉完成或未被使用

指令类型

非轴运动缓存指令。

3.4.1.5.2.4 HMC_CaptureGetError（获取高速捕捉错误）

函数原型

UDINT HMC_CaptureGetError(USINT ucChIID)

功能说明

返回某个高速捕捉通道的错误码。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	高速捕捉通道号

返回值

返回值	类型	参数描述
0	UDINT	正确
其他值	UDINT	错误码

指令类型

非轴运动缓存指令。

3.4.1.5.3 高速到位输出

3.4.1.5.3.1 HMC_SwitchConfig(设置高速输出)

函数原型

```
UDINT HMC_SwitchConfig(USINT ucChIID,
    USINT ucOutputMode,
    USINT ucOutputID,
    USINT ucTrigAxis,
    USINT bSwitchValue,
    POINTER TO LREAL pMpos,
    UDINT uiMposNum)
```

功能说明

该指令主要完成对高速到位输出的配置。高速到位输出功能（简称 **Switch**）可以设置一系列目标位置，当被选择的伺服轴或编码器轴到达当前目标值时，被选择的输出通道电平发生翻转。其中，输出通道的首个电平值由 **bSwitchValue** 指定。该系列目标值被顺序执行。

该指令可实现 DO 输出快速响应轴接口位置的功能，无需组态其它逻辑，操作简单、响应快速。



功能示意图

输入参数

输入参数	类型	参数描述
ucChIID	USINT	高速输出通道号 MC1008&MC1004&MC1004L (0~3) MC1002R&MC1002E (0)
ucOutputMode	USINT	输出类型选择, 0: DO 输出; 1: Axis Z 相输出
ucOutputID	USINT	输出位置选择, 使用该轴 Z 相或该 DO 输出。
ucTrigAxis	USINT	触发轴, switch 比较该轴与目标 Mpos 的值决定输出电平

输入参数	类型	参数描述
bSwitchValue	USINT	首次输出电平：0 或 1
pMpos	POINTER TO LREAL	目标位置存放地址，一般为一个数组的地址，该数组存放目标 Mpos 设定值。
uiMposNum	UDINT	目标 Mpos 序列的个数

返回值

返回值	类型	参数描述
0	UDINT	配置成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

■ 补充说明

(1) 针对不同的模块，通道数有所不同，即 ucChIID 数不同。对于 MC1008、MC1004、MC1004L 共有 4 路高速到位输出通道，MC1002R、MC1002E 只有 1 路高速到位输出通道。

(2) 针对不同的模块，可选输出位置不同。对于 MC1008,共有 8 路轴接口或 8 路 DO 输出可供选择；对于 MC1004,共有 4 路轴接口和 8 路 DO 可供选择；对于 MC1002,共有 2 路轴接口和 8 路 DO 通道可供选择。

(3) 该指令 ucTrigAxis 触发轴只支持伺服轴、编码器轴、步进轴类型，不支持虚轴、总线轴类型。

(4) 该指令可设置一系列目标位置，该位置存于数组中，并将数组的首地址写入 pMpos 参数，Switch 将依次执行。该命令阻塞执行，即未到达当前目标位置时，后续目标位置将不响应。当目标位置被执行，该命令才被释放。

(5) 在位置数组的设置中，应遵循数据递增方向同电机运行方向保持一致，否则会产生异常。例如当前位置为 0，电机将向负向运行，则目标位置设定为-10、-100、-500 可工作正常。但设定为-10、10000、-500，则功能块会出现异常。

(6) 在配置高速到位输出前，触发轴当前位置 Mpos 必须在触发位置 Mpos 序列的起始位置之前。不允许在配置高速到位输出后，使用 Defpos 定义轴当前位置位于触发位置 Mpos 序列的起始位置之前。

(7) 当采集轴为绝对值编码器轴时，命令输出与实际位置会存在一些偏差。例如设定目标位置为 1000，相临两次从编码器获得的数据分别为 990、1100 时，在 1100 位置会产生输出跳变。

(8) 在 Switch 命令执行过程中，不得人为改变 Mpos 的值。例如，在 Switch 命令未完成之前对 DefPos 的执行会导致该功能的异常。如执行了修改 Mpos 的相关指令，应执行一次 HMC_CloseSwitch 命令，关闭高速到位输出并重新配置。

应用举例

在某激光雕刻应用中，要求在钢板上每隔 100 单位的距离打长度为 100 单位的矩形孔。从 0 位开始，当到达 100 时激光打开，到达 200 时激光关闭，到达 300 时激光打开....直到到达 1000。

在上述应用中，如使用 HMC_SwitchConfig 指令，用户只需关心电机的运行，无需关注激光的开闭。该指令会快速及时的响应被采集轴的位置。

```
FOR i:=0 TO 9 BY 1 DO      (*填充目标位置设定值*)
    target_pos[i] := (i+1)*100;
```

END_FOR

axis0.AxisType := 20; (*设定轴 0 为伺服轴*)

(*轴 0 定位到起始位置...略...*)

ChIID := 0; (*选择 switch 通道 0*)

OutputMode := 1; (*使用 Z 相输出*)

OutputID := 0; (*使用轴 0 的 Z 相作为 switch 的输出*)

TrigAxis := 0; (*采集轴 0 的当前位置*)

SwitchValue := 0; (*首个输出为低电平*)

ppMpos := ADR(target_pos); (*获取目标位置数组的地址*)

MposNum := 10; (*设定设置的有效目标地址个数共 10 个*)

(*配置 SwitchConfig*)

switch_ack :=HMC_SwitchConfig(ChIID, OutputMode, OutputID, TrigAxis, SwitchValue, ppMpos, MposNum);

(*轴 0 开始位移...略...*)

3.4.1.5.3.2 HMC_GetSwitchState(获取高速输出状态)

函数原型

USINT HMC_GetSwitchState(USINT ucChIID)

功能说明

获取指定 ID 的 SwitchConfig 的当前状态。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	switchconfig 通道号

返回值

返回值	类型	参数描述
0	USINT	当前通道正在工作
3	USINT	当前通道空闲，表示已完成或未配置

指令类型

非轴运动缓存指令

3.4.1.5.3.3 HMC_GetSwitchNum(获取已输出数目)

函数原型

UDINT HMC_GetSwitchNum(USINT ucChIID)

功能说明

获取指定 ID 的 SwitchConfig 已执行的段数。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	switchconfig 通道号

返回值

返回值	类型	参数描述
已完成执行的段数	UDINT	已完成执行的段数

指令类型

非轴运动缓存指令

3.4.1.5.3.4 HMC_SwitchTZConfig(设置高速 TZ 输出)

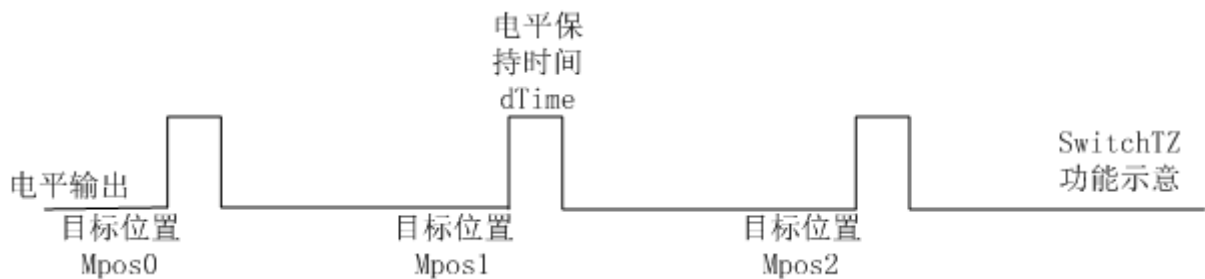
函数原型

```
UDINT HMC_SwitchTZConfig(USINT ucChIID,
    USINT ucOutputAxis,
    USINT ucTrigAxis,
    USINT ucTimeMode,
    LREAL dTime,
    USINT bSwitchValue,
    POINTER TO LREAL pMpos,
    UDINT uiMposNum)
```

功能说明

SwitchTZ 功能是 Switch 功能的增强，即到达指定位置使指定轴的 Z 相输出产生响应。当时间使能被禁止时，该功能与相当于 Switch 使用 Z 相输出。开启时间使能 (ucTimeMode:= 1) 后，被采集轴（由 ucTrigAxis 设定）到达指定位置（pMpos 所指的数组设定）后，输出与 bSwitchValue 相同的电平值。电平持续 dTime 时间后恢复至相反电平，继续等待下一个目标位置，直到完成 uiMposNum 段的输出。

该指令与 Switch 功能类似，主要完成在指定位置完成固定时间的电平输出。AT 工程无需关心具体的操作逻辑，由底层硬件完成电平输出与当前位置的完美结合。



功能示意图

输入参数

输入参数	类型	参数描述
ucChIID	USINT	高速输出通道号 MC1008 (0~1) 其它型号不支持
ucOutputAxis	USINT	输出轴号 (使用该轴 Z 相进行输出)
ucTrigAxis	USINT	触发轴, switch 比较该轴与目标 Mpos 的值决定输出电平
ucTimeMode	USINT	时间模式设置。0: 关闭; 1: 开启
dTime	LREAL	输出电平保持时间, 单位: us , 有效范围: 1~100,000, 分辨率: 0.1 μs 。
bSwitchValue	USINT	时间模式关闭时: 首个输出电平值 (0 或 1) 时间模式开启时: 输出脉冲电平值 (0 或 1)
pMpos	POINTER TO LREAL	目标位置存放地址, 一般为一个数组的地址, 该数组存放目标 Mpos 设定值。
uiMposNum	UDINT	目标 Mpos 序列的个数

返回值

返回值	类型	参数描述
0	UDINT	配置成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

■ 补充说明

- (1) MC 系列产品中, 只有 MC1008 支持该功能, 其它模块不支持。
- (2) 该指令只支持伺服轴、编码器轴、步进轴的采集。不支持虚轴、总线轴使用。
- (3) 该指令可设置一系列目标位置, 该位置存于数组中, 并将数组的首地址写入 **pMpos** 参数, **SwitchTZ** 将依次执行。该命令阻塞执行, 即未到达当前目标位置时, 后续目标位置将不响应。只有最后一个目标位置被执行, 该命令才可释放。
- (4) 在位置数组的设置中, 应遵循数据递增方向同电机运行方向保持一致, 否则会产生异常。例如当前位置为 0, 电机将向负向运行, 则目标位置设定为-10、-100、-500 可工作正常。但设定为-10、10000、-500, 则功能块会出现异常。
- (5) 在位置设定中应注意有效电平维持时间同两次有效位置的间隔。例如电机速度为 **V**, 电平有效时间设为 **T**, 当前位置为 **S0**, 下一有效位置为 **S1**, 则应保证 $S1-S0 > V \cdot T$ 。否则, 如果下一刻的目标位置在 **T** 时间内到达, **SwitchTZ** 模块将不响应。严重时发生阻塞。
- (6) 在配置高速到位输出前, 触发轴当前位置 **Mpos** 必须在触发位置 **Mpos** 序列的起始位置之前。不允许在配置高速到位输出后, 使用 **Defpos** 定义轴当前位置位于触发位置 **Mpos** 序列的起始位置之前。
- (7) 当采集轴为绝对值编码器轴时, 命令输出与实际位置会存在一些偏差。例如设定目标位置为 1000, 相临两次从编码器获得的数据分别为 990、1100 时, 在 1100 位置会产生输出跳变。
- (8) 在 **SwitchTZ** 命令执行过程中, 不得人为改变 **Mpos** 的值。例如, 在 **SwitchTZ** 命令未完成之前对 **DefPos** 的执行会导致该功能的异常。如执行了修改 **Mpos** 的相关指令, 应执行一次 **HMC_CloseSwitchTZ** 命令, 关闭高速到位输出并重新配置。

3.4.1.5.3.5 HMC_GetSwitchTZState(获取高速 TZ 输出状态)

函数原型

USINT HMC_GetSwitchTZState (USINT ucChIID)

功能说明

获取指定 ID 的 SwitchTZConfig 的当前状态。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	switchTZConfig 通道号

返回值

返回值	类型	参数描述
0	USINT	当前通道正在工作
3	USINT	当前通道空闲，表示已完成或未配置

指令类型

非轴运动缓存指令。

仅 MC1008 支持。

3.4.1.5.3.6 HMC_GetSwitchTZNum(获取 TZ 已输出数目)

函数原型

UDINT HMC_GetSwitchTZNum(USINT ucChIID)

功能说明

获取指定 ID 的 SwitchConfig 已执行的段数。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	switchTZconfig 通道号

返回值

返回值	类型	参数描述
已完成执行的段数	UDINT	已完成执行的段数

指令类型

非轴运动缓存指令。

仅 MC1008 支持。

3.4.1.5.3.7 HMC_CloseSwitch(关闭高速输出)

函数原型

UDINT HMC_CloseSwitch(USINT ucChIID)

功能说明

关闭对 CloseSwitch 的操作，常用于 CloseSwitch 阻塞或需要重新配置时。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	switchconfig 通道号

返回值

返回值	类型	参数描述
0	UDINT	关闭成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

3.4.1.5.3.8 HMC_CloseSwitchTZ(关闭高速 TZ 输出)

函数原型

UDINT HMC_CloseSwitchTZ(USINT ucChIID)

功能说明

关闭对 SwitchTZ 的操作，常用于 SwitchTZ 阻塞或需要重新配置时。

输入参数

输入参数	类型	参数描述
ucChIID	USINT	switchTZconfig 通道号

返回值

返回值	类型	参数描述
0	UDINT	关闭成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

仅 MC1008 支持。

3.4.1.5.4 低速到位输出

3.4.1.5.4.1 低速到位输出

低速到位输出与高速到位输出功能类似，但是位置比较的精准度低于高速到位输出功能，位置比较精度与伺服周期和轴的速度都有关系，适用于对控制精度要求不高的应用场景。

低速到位输出支持更丰富的子功能：

■ 输入

支持所有轴类型，包括直连、总线、虚轴等；

信号源比较支持目标位置 Dpos 和实际反馈位置 Mpos。

■ 输出

可通过任意类型 DO 输出，包括高速、低速、虚拟 DO；

信号输出方式支持电平翻转和指定宽度的脉冲输出。

■ 位置比较

比较位置序列随意设置，但必须符合轴的位置变换，否则会出现一直等待某点判断中；

支持一维位置比较、二维位置比较。

3.4.1.5.4.2 HMC_NormalSwitch1Dconfig（设置一维低速输出）

功能说明

配置一维低速输出。按照设置的位置比较序列，与轴的位置进行比较，当轴的位置在本周期经过目标比较位置时，则激活输出。

支持目标位置序列不严格单调。按目标位置序列数组从前到后比较，若当前待比较位置一直不能触发，则一直处于判断该目标点状态，不会对后续位置进行比较。

输入参数

输入参数	类型	含义
ucChlID	USINT	普通到位输出通道号，0~7 通道号与 HMC_NormalSwitch2DConfig 共用
ucOutputID	USINT	输出位置选择，0~63
ucTrigAxis	USINT	采集轴，switch 比较该轴与目标位置值决定输出电平
ucTrigPosType	USINT	位置比较源方式： 0：比较采集轴的 Mpos 1：比较采集轴的 Dpos
ucOutputType	USINT	输出方式： 0：电平翻转 1：可设脉宽的脉冲
bSwitchValue	USINT	当 ucOutputType=0 时，表示第一个位置输出电平值：0 或 1 当 ucOutputType=1 时，表示输出脉冲电平值：0 或 1
uiPluseWidth	UDINT	当 ucOutputType=1 时，该参数有效，表示脉冲宽度，单位 ms（设置值需≥1 个伺服周期，分辨率正负 1 个伺服周期。）
pPos	POINTER TO LREAL	目标位置存放地址，该数组存放目标位置设定值
uiPosNum	UDINT	目标 Mpos 序列的个数

返回值

返回值	类型	参数描述
0	UDINT	配置成功
其它	UDINT	错误码

3.4.1.5.4.3 HMC_NormalSwitch2DConfig（设置二维低速输出）

功能说明

配置二维低速输出，按照设置的位置比较序列进行比较，当两轴位置对应的二维坐标进入目标位置相等区域并选取较优位置，激活输出。具体的位置判定方法如下：

- 当前点与目标点的距离小于等于位置比较阈值 dThreshold，认为进入目标位置相等区域，是触发

输出的必要条件：

- 进入相等区域后，尽量选择在距离目标点距离最近处输出；通过连续比较与目标点距离，确认逼近还是远离，从而寻找最优时机；
- 当比较信号源为轴的实际反馈位置时，需做防抖处理，使用防抖动阈值 `dDisturbValue` 参数；
- 只要进入过相等区域就一定会触发输出。

综合以上多项处理，输出的时机不一定在距离目标点最近处，但在其附近，这取决于用户设定的判断阈值和轴的速度等多项因素。

输入参数

输入参数	类型	含义
<code>ucChlID</code>	USINT	普通到位输出通道号，0~7 通道号与 <code>HMC_NormalSwitch1DConfig</code> 共用
<code>ucOutputID</code>	USINT	输出位置选择，0~63
<code>ucTrigAxisX</code>	USINT	采集轴 X， <code>switch</code> 比较该轴与 X 目标位置值决定输出电平
<code>ucTrigAxisY</code>	USINT	采集轴 Y， <code>switch</code> 比较该轴与 Y 目标位置值决定输出电平
<code>ucTrigPosType</code>	USINT	位置比较源方式： 0：比较采集轴的 <code>Mpos</code> 1：比较采集轴的 <code>Dpos</code>
<code>ucOutputType</code>	USINT	输出方式： 0：电平翻转 1：可设脉宽的脉冲
<code>bSwitchValue</code>	USINT	当 <code>ucOutputType=0</code> 时，表示第一个位置输出电平值：0 或 1 当 <code>ucOutputType=1</code> 时，表示输出脉冲电平值：0 或 1
<code>uiPluseWidth</code>	UDINT	当 <code>ucOutputType=1</code> 时，该参数有效，表示脉冲宽度，单位 <code>ms</code> （设置值需 ≥ 1 个伺服周期，分辨率正负 1 个伺服周期。）
<code>pPosX</code>	POINTER TO LREAL	X 目标位置存放地址，该数组存放目标位置设定值。当为二维位置比较时，该数组描述 X 轴位置值
<code>pPosY</code>	POINTER TO LREAL	Y 目标位置存放地址，当为二维位置比较时该参数有效，该数组描述 Y 轴位置值，与 <code>pMposX</code> 共同描述二维比较目标位置
<code>uiPosNum</code>	UDINT	目标 <code>Mpos</code> 序列的个数
<code>dThreshold</code>	LREAL	位置比较偏差阈值，判断是否到达目标位置相等区域(≥ 0)
<code>dDisturbValue</code>	LREAL	防抖动阈值。位置比较源为 <code>Mpos</code> ，且反馈位置来自物理输入时才有效。反馈位置与目标位置进行比较时，用来进行防抖动处理。（ ≥ 0 ）

返回值

返回值	类型	参数描述
0	UDINT	配置成功
其它	UDINT	错误码

3.4.1.5.4.4 HMC_GetNormalSwitchNum（获取已输出数目）

功能说明

获取指定 ID 的低速到位输出已执行的段数。

输入参数

输入参数	类型	含义
<code>ucChlID</code>	USINT	低速输出通道号，0~7

返回值

返回值	类型	参数描述
已完成执行的段数	UDINT	已完成执行的段数

3.4.1.5.4.5 HMC_GetNormalSwitchState（获取低速输出状态）

功能说明

获取指定 ID 的低速输出的当前状态。

输入参数

输入参数	类型	含义
ucChIID	USINT	低速输出通道号，0~7

返回值

返回值	类型	含义
0	USINT	当前通道正在工作
3	USINT	当前通道空闲，表示已完成或未配置

3.4.1.5.4.6 HMC_CloseNormalSwitch（关闭低速输出）

功能说明

关闭该 ID 的低速到位输出。

输入参数

输入参数	类型	含义
ucChIID	USINT	低速输出通道号，0~7

返回值

返回值	类型	参数描述
0	UDINT	关闭成功
其它	UDINT	错误码

3.4.1.5.5 示波器功能指令

3.4.1.5.5.1 HMC_SetSampleVar（设置采样变量）

函数原型

```
UDINT HMC_SetSampleVar (USINT ucSampleChl,
    POINTER TO BYTE pVar0,
    POINTER TO BYTE pVar1,
    POINTER TO BYTE pVar2,
    POINTER TO BYTE pVar3,
    UDINT uiVarType0,
    UDINT uiVarType1,
```

UDINT uiVarType2,

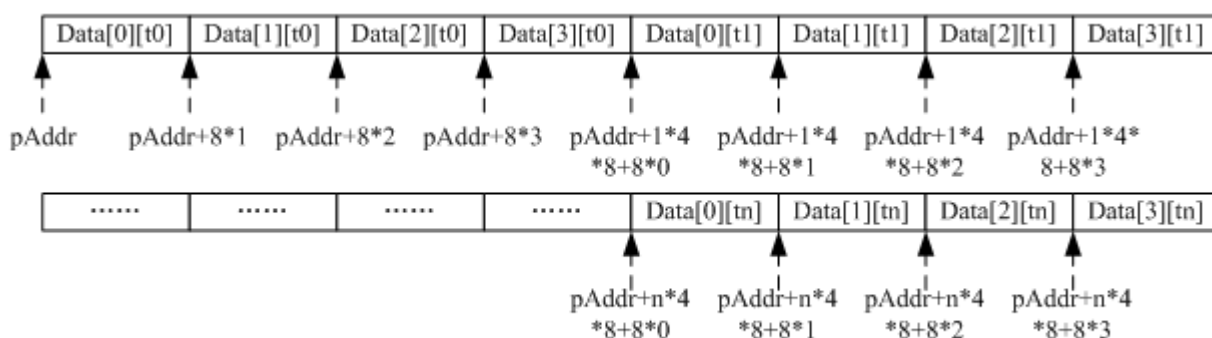
UDINT uiVarType3)

功能说明

MC1004 和 MC1008 型号控制器支持 4 路高速采样通道，MC1002R 和 MC1002E 型号控制器支持 1 路高速采样通道。每个采样通道有 4 个子通道，最多可以采集 4 个数据。

数据采样功能可以对系统的轴参数、I/O 数据、中间/系统变量等进行数据采集，按照配置的数据采样间隔时间进行采样，将采样获得的实时数据记录存放至用户指定区域，该指定区域必须位于 M 区内。

无论待采样数据其自身数据类型是整形还是浮点，采样数据最终都按照双精度浮点 LREAL 类型存放至用户指定区域，一个通道的四个子通道数据按照时间先后顺序在用户指定区域存放顺序如下：



Data[i][ti]:第i个子通道在第i次采样时刻的采样数据

采样存放示意图

可以根据以上存储格式解析查看记录数据。当仅使用部分子通道时，如只使用 0、1 通道，则 1 号子通道数据后紧促排列存储 0 号子通道下一采样时刻的值，中间无空闲。

本功能块作用是配置待采样数据来源，最多支持采集 4 个变量。

采样功能须结合系统其它几个采样相关功能块一起使用。先调 SetSampleVar 配置待采样数据，再调用 SetSamplePara 设置存储区域、采样间隔时间、采样总点数，最后还需通过 HMC_TriggerSample 触发采样执行。采样执行过程中，可通过 HMC_GetSampleNum 查看当前已经完成的采样数目，采样执行完成后，可通过 HMC_GetSampleData 查看某一子通道指定采样时刻序号的采样值。其它采样相关功能块请查看对应章节。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道号 MC1008&MC1004 (0~3) MC1002R&MC1002E (0)
pVar0	POINTER TO BYTE	待采集的第 0 子通道数据的地址
pVar1	POINTER TO BYTE	待采集的第 1 子通道数据的地址
pVar2	POINTER TO BYTE	待采集的第 2 子通道数据的地址
pVar3	POINTER TO BYTE	待采集的第 3 子通道数据的地址
uiVarType0	UDINT	待采集的第 0 子通道数据类型，数据类型为：USINT=1、SINT=2、UINT=3、INT=4、

输入参数	类型	参数描述
		UDINT=5、DINT=6、F32=7、LREAL=8
uiVarType1	UDINT	待采集的第 1 子通道数据类型，数据类型为：USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、F32=7、LREAL=8
uiVarType2	UDINT	待采集的第 2 子通道数据类型，数据类型为：USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、F32=7、LREAL=8
uiVarType3	UDINT	待采集的第 3 子通道数据类型，数据类型为：USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、F32=7、LREAL=8

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令

应用举例

对轴 0 的 Mpos（LREAL）和 VpSpeed（LREAL）进行采样，采样时间间隔分别为 1 倍伺服周期和 2 倍伺服周期，伺服周期为 1 ms，采样数都为 3000。

```
Axis0.Speed:=2000;
```

```
HMC_Forward(0);
```

```
HMC_SetSampleVar (0, ADR(axis0.MPos), ADR(axis0.VpSpeed), ADR(Temp), ADR(Temp), 8, 8, 7, 7);
```

（*使用 0 号通道的子通 0、1 进行采样，2、3 通道采样 Temp 变量，类型为 F32*）

```
HMC_SetSamplePara(0,1,3000,ADR(RecordAry0));
```

（*设置 0 号通道的采样间隔时间为 1 个伺服周期,采样点数 3000 个/子通道，采集数据存放到数组 RecordAry0*）

```
HMC_SetSampleVar (1, ADR(axis0.MPos), ADR(axis0.VpSpeed), ADR(Temp), ADR(Temp), 8, 8, 7, 7);
```

```
HMC_SetSamplePara(1, 10, 3000, ADR(RecordAry1));
```

（*设置 1 号通道的采样间隔时间为 10 个伺服周期,采样点数 3000 个/子通道，采集数据存放到数组 RecordAry1*）

```
HMC_TriggerSample(0);
```

（*触发采样通道 0*）

```
HMC_TriggerSample(1);
```

（*触发采样通道 1*）

查看数组 RecordAry0 和 RecordAry1 数组存储的采样结果如下：

变量名	直接地址	在线值	变量名	直接地址	在线值
RecordAry0[0,0]	%MD0	5238	RecordAry1[0,0]	%MD5000	8320
RecordAry0[0,1]	%MD8	2000	RecordAry1[0,1]	%MD5008	2000
RecordAry0[1,0]	%MD16	5240	RecordAry1[1,0]	%MD5016	8340
RecordAry0[1,1]	%MD24	2000	RecordAry1[1,1]	%MD5024	2000
RecordAry0[2,0]	%MD32	5242	RecordAry1[2,0]	%MD5032	8360
RecordAry0[2,1]	%MD40	2000	RecordAry1[2,1]	%MD5040	2000
RecordAry0[3,0]	%MD48	5244	RecordAry1[3,0]	%MD5048	8380
RecordAry0[3,1]	%MD56	2000	RecordAry1[3,1]	%MD5056	2000
RecordAry0[4,0]	%MD64	5246	RecordAry1[4,0]	%MD5064	8400
RecordAry0[4,1]	%MD72	2000	RecordAry1[4,1]	%MD5072	2000
RecordAry0[5,0]	%MD80	5248	RecordAry1[5,0]	%MD5080	8420
RecordAry0[5,1]	%MD88	2000	RecordAry1[5,1]	%MD5088	2000
RecordAry0[6,0]	%MD96	5250	RecordAry1[6,0]	%MD5096	8440
RecordAry0[6,1]	%MD104	2000	RecordAry1[6,1]	%MD5104	2000
RecordAry0[7,0]	%MD112	5252	RecordAry1[7,0]	%MD5112	8460
RecordAry0[7,1]	%MD120	2000	RecordAry1[7,1]	%MD5120	2000

存储示意图

可以看出 RecordAry0 中存储的 Mpos 采样间隔的差值为 2，符合速度为 2000、伺服周期为 1 ms、采样间隔时间为 1 个伺服周期；RecordAry1 中存储的 Mpos 采样间隔的差值为 20，符合速度为 2000、伺服周期为 1 ms、采样间隔时间为 10 个伺服周期。

3.4.1.5.5.2 HMC_SetSamplePara（设置采样参数）

函数原型

```
UDINT HMC_SetSamplePara (USINT ucSampleChl,
UDINT uiTime,
UDINT uiNum,
POINTER TO LREAL pRecordArea)
```

功能说明

配置指定采样通道的采样间隔时间、采样点数、采样数据存储区地址。需注意，采样间隔时间 uiTime 的单位为伺服周期，uiTime=2，表示采样间隔为 2 个伺服周期，MC 控制器支持 0.25、0.5、1、2、4 ms 伺服周期。

采样功能详细描述请查看 [HMC_SetSampleVar（设置采样变量）](#)。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道
uiTime	UDINT	记录间隔时间。不可为 0，单位：伺服周期
uiNum	UDINT	记录点数
pRecordArea	POINTER TO LREAL	采样记录数组的起始地址（采样记录数组只允许定义在 M 区，系统不允许记录区域大小超过 M 区；用户在定义记录数组时应计算好大小，避免越界情况发生）

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令

应用举例

参考 [HMC_SetSampleVar（设置采样变量）](#)

3.4.1.5.5.3 HMC_GetSampleData（获取采样数据）

函数原型

LREAL HMC_GetSampleData (USINT ucSampleChl, USINT ucChannel, UDINT uiDataNum)

功能说明

获取指定通道的某子通道的序号为 uiDataNum 的采样数据。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道
ucChannel	USINT	子通道号（0~3）
uiDataNum	UDINT	数据号（该子通道的第 uiDataNum 个数据），数据号从 0 开始

返回值

返回值	类型	参数描述
采样数据	LREAL	-
其它	LREAL	错误码

指令类型

非轴运动缓存指令

3.4.1.5.5.4 HMC_TriggerSample（触发采样）

函数原型

UDINT HMC_TriggerSample (USINT ucSampleChl)

功能说明

触发指定采样通道开始执行采样。待采样通道的相关参数配置完毕后，HMC_TriggerSample 用来触发采样动作的执行。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令

应用举例

参考 [HMC_SetSampleVar（设置采样变量）](#)

3.4.1.5.5.5 HMC_DisableSample（停止采样）

函数原型

UDINT HMC_DisableSample (USINT ucSampleChl)

功能说明

禁止指定采样通道执行采样。对于正在执行采样的通道，通过本函数可以停止采样，若需要再次触发采样，必须重新配置采样和触发采样。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.5.5.6 HMC_TriggerScope（触发示波器）

函数原型

UDINT HMC_TriggerScope ()

功能说明

触发示波器开始执行。

示波器功能需配合 AT 的示波器软件使用，仅在触发方式为“程序”时该功能才起作用。示波器窗口中，选择触发方式为“程序”，切换“停止/运行”按钮进入运行模式后，通过调用此功能块，可触发算法进行采样，同时示波器软件根据采样数据绘制过程曲线。

编写 IEC 程序，当触发条件满足时调用 HMC_TriggerScope，可实现满足某条件后立刻采样的功能。

输入参数

无

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令

应用举例

轴 0 投放持续正转指令，当轴 0 位置达到 5000 时，利用示波器获取轴 0 的速度和位置信息。

设置 AT 触发方式为“程序”，切换“停止/运行”按钮进入运行模式。调用如下程序所在 IEC 任务：

```
HMC_Forward (0);
```

```
WHILE axis0.Mpos < 5000 DO
```

```
;
```

```
END_WHILE
```

```
HMC_TriggerScope();    (*触发示波器采样*)
```

3.4.1.5.5.7 HMC_GetSampleNum（获取已采样数目）

函数原型

```
UDINT HMC_GetSampleNum (USINT ucSampleChl)
```

功能说明

获取指定采样通道当前已完成的采样数目。完成该采样通道下的 1 个子通道的采样，则已采样数目累加 1。

即如果 HMC_SetSamplePara 中设置采样点数为 N，且 HMC_SetSampleVar 设置四个子通道采样均有效，则当该通道完成采样后，通过 HMC_GetSampleNum 获取的已采样数目应为 4*N。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道

返回值

返回值	类型	参数描述
已采样数目	UDINT	-
其它	UDINT	错误码

指令类型

非轴运动缓存指令

3.4.1.5.5.8 HMC_SetSampleSingleVar（设置单个采样变量）

函数原型

```
UDINT HMC_SetSampleSingleVar (USINT ucSampleChl,
```


POINTER TO BYTE pVar,

UDINT uiVarType)

功能说明

配置指定采样通道的 0 号子通道的待采样数据来源，即仅采集单个变量。该函数仅使用 0 号子通道，且同时将 1~3 号子通道的待采样数据均设置为空，即后续 3 个子通道均不进行采样。

输入参数

输入参数	类型	参数描述
ucSampleChl	USINT	采样通道
pVar	POINTER TO BYTE	待采集的数据的地址
uiVarType	UDINT	待采集的数据类型，数据类型为： USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、F32=7、LREAL=8

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令

3.4.1.5.6 运动保持功能指令

3.4.1.5.6.1 HMC_HoldConfig(设置强制速度开关)

函数原型

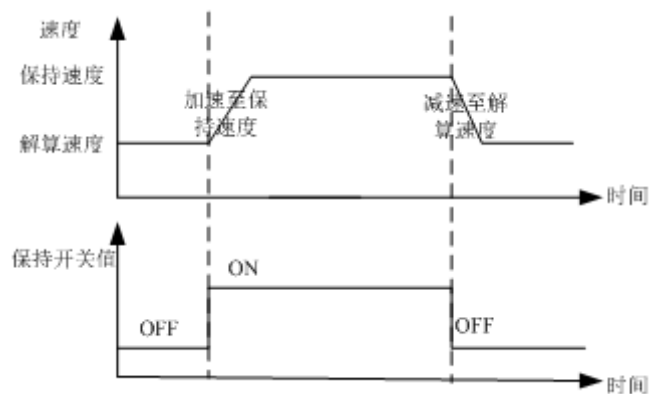
UDINT HMC_HoldConfig (USINT ucAxisID, INT sDiChan)

功能说明

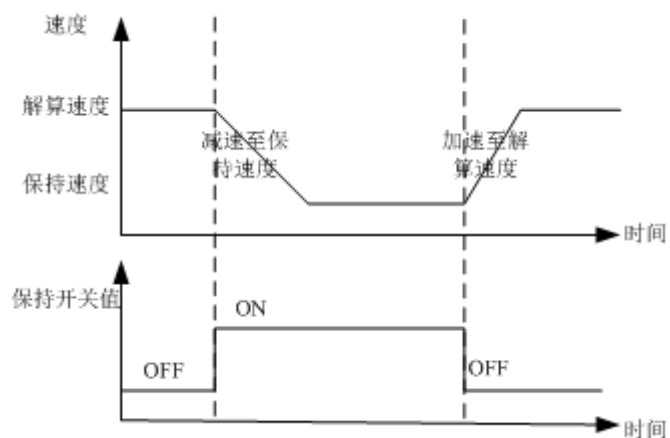
速度强制功能是指某个轴关联的速度保持开关 DI 通道被触发即值为 1 时，该轴会从当前速度按照 Accel 或 Decel 加减速到 HoldSpeed 设定速度；当 DI 通道解除触发状态即值为 0 时，该轴又按照 Accel 或 Decel 加减速到指令解算速度。

该算法块用于配置轴的速度强制开关为指定 DI，sDiChan 参数指定 DI 通道号。当参数 sDiChan 等于-1 时，表示关闭此功能。

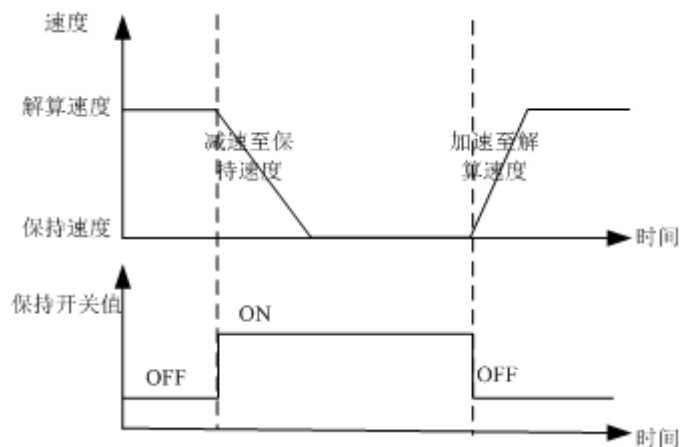
以保持速度为 0，保持速度大于运行速度，保持速度小于运行速度，三种常见情况下轴的速度变化如下：



保持速度大于解算速度



保持速度小于解算速度



保持速度等于 0

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
sDiChan	INT	关联的 DI 通道 (0~63) 参数为-1 时表示取消 HoldSpeed (强制 Speed) 开关。当取消 HoldSpeed 开关后, 原来关联的 DI 通道的值将不再影响该轴的 HoldSpeed 状态, 也就是即便当前处于

输入参数	类型	参数描述
		HoldSpeed 状态也会立即解除

返回值

返回值	类型	参数描述
0	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

应用举例

轴在正常运动过程中，当某条件满足时，如按下暂停按钮，需使得该轴暂停运动，保持静止；当暂停按钮弹起，保持条件解除时，该轴再恢复之前运动。

该按钮状态可通过 DI 检测，使用第 10 路 DI，编写程序如下：

```
axis0.HoldSpeed:=0;           (*设置保持速度为 0*)
HMC_HoldConfig (0, 10);      (*配置第 10 路 DI 为轴 0 速度保持开关*)
HMC_ForwardEx (0, 10000);    (*投入运动指令，轴 0 以 10000 速度运动*)
```

(*当现场的暂停按钮按下时，第 10 路 DI 检测为 1，算法检测到该 DI 为 1 时，从当前速度减速至保持速度 0，实现该轴暂停；当现场的暂停按钮弹起时，第 10 路 DI 为 0，算法检测到该 DI 为 0，从速度 0 加速至设定速度 10000*)

3.4.1.5.7 运动前瞻

3.4.1.5.7.1 运动前瞻

本章节是运动前瞻功能，该功能是对轴运动持续不断地追踪、统计、分析，在计划时间内得出解算结果，是对轴运动存在的多影响因素的综合考虑。

控制器提供段内梯形或 S 形加减速控制功能，即对每一段都进行加减速处理，指令间速度独立。因此各个指令段连接处，一定会经历速度先减速为零，然后再下条指令开始后再重新进行加速的过程，宏观上，速度是持续的加减速、加减速。实际应用时，不仅影响加工效率，而且在长度极短连续微线段加工时，会对机床产生巨大的加速度冲击，造成机床振动，甚至导致工件过切。因此需要更加高级的速度控制方法。

系统的速度前瞻功能，一方面可以对指令进行整体规划，即对各段速度进行整体规划，再配合指令段内的加减速控制，可以使机床保持高速运行提高效率，使负载运动更加流畅，告别停停走走，系统通过 Merge 速度融合功能实现；另一方面，再保证高速运行基础上为了限制机械冲击和过切等，还需进行减速识别，通过提前识别轨迹变化，从而按照安全的减速度提前减速，系统通过减速/停止融合功能、抑制冲击功能实现。整体来看，速度前瞻功能既可提升整机效率，也可减少冲击增加柔性，降低零部件磨损，增加设备使用寿命。

系统的运动前瞻功能包括运动两大类，1-融合功能，其包括速度融合、减速/停止融合功能，2-抑制冲击功能。

Merge（融合功能）

Merge 功能是将控制器运动缓存中一连串运动指令连贯起来，使负载运动更加流畅，告别停停走走。其针对指令间的处理：开启 Merge 功能后实现了速度衔接，指令间速度不再减速为 0；而另一方面，为了避免指令间夹角过大时如果速度较大会造成冲击，引入了减速/停止角融合功能。

■ 生效范围

- Merge 功能是否能够成功开启，除了依赖 Merge 开关外，还依赖于指令类型、指令涉及轴的统一等条件，如下：
- 目前仅支持一维指令（单轴指令）、二维指令（两轴插补指令）、N 轴插补指令的 Merge 处理。且支持 S 形加减速 Merge；
- 单轴指令中，只支持 HMC_Move、HMC_MoveEx、HMC_MoveAbs、HMC_MoveAbsEx 指令；
- 两轴插补指令中，只支持 HMC_Move2D、HMC_Move2DEx、HMC_MoveAbs2D、HMC_MoveAbs2DEx、HMC_MoveCircle、HMC_MoveCircleEx 指令；
- N 轴插补指令中，只支持 HMC_MoveND、HMC_MoveAbsND、HMC_MoveNDEx、HMC_MoveAbsNDEx 指令；
- 两轴/N 轴插补指令中，若插补的合轴、分轴所对应的轴号序列发生变化时 Merge 无效；
- 维度（轴数）不同的指令之间相互连接时 Merge 无效；
- 若当前指令正在执行 HMC_MoveModify，HMC_SetMerge 参数设置失败；
- 二维、N 维指令中，不允许第 N 段、N+1 段的指令转角 θ 过大， θ 为 Merge 所连接的两相邻指令之间的连接角。指令转角 θ 小于减速角（减速角由 HMC_SetMergeAngle 函数设置）时可以正常 Merge；减速角 θ 大于等于停止角（停止角由 HMC_SetMergeAngle 函数设置）时 Merge 无效，指令转角 θ 大于等于减速角且小于停止角时，当前指令以小于 Speed 的速度结束（按比例）；
- 如果 Merge 功能处于开启状态，在投放指令 HMC_MoveModify 后 Merge 自动关闭；用户需手动重新开启 Merge 功能；
- 如果 Merge 功能处于开启状态，在使用 Cancel 撤销指令后 Merge 自动关闭；用户需手动重新开启 Merge 功能。

■ Merge 功能开启

通过 HMC_SetMerge 函数设置 Merge 功能是否开启及开启方式。

- Merge=0
Merge 关闭，当前指令块结束时的速度为 0。
- Merge=1
Merge 打开，当前指令块结束时目标速度为该指令块的 Speed

- Merge=2

Merge 打开，当前指令块结束时目标速度为下一指令块的 Speed

- Merge=3

Merge 打开，当前指令块结束时目标速度为该指令块 Speed 和下条指令块 Speed 最小值

- Merge=4

Merge 打开，当前指令块结束时目标速度为该指令块 Speed 和下条指令块 Speed 最大值

■ Merge 的主要设计准则如下：

- 1 D 指令衔接处，如果本条指令剩余位移不足，则提前输出下一条指令的部分位移，防止每条指令的最后一个周期速度跃变；

- 2 D 插补指令衔接处，不强制插补轨迹必须经过两条指令的衔接点，如果本条指令剩余位移不足，则提前输出下一条指令的部分位移；

- 对于改变速度 speed 的 EX 指令：

1) 尽量保证所有 EX 指令衔接点预设的速度够达到，如果只能保证部分衔接点的预设速度，则优先保证速度较低的衔接点，不能达到预设速度的衔接点需保证实际运行速度小于其预设速度。

2) 必须保证 VpSpeed 的变化过程始终满足预设的梯形/S 形规律；

3) 在满足上述准则所需约束条件的前提下，各指令在各自的生命周期内保证尽可能多的时间以自身的 speed 为目标速度运动。

- 对于存在拐角限速/Jerk 限速的指令：

1) 在逐级前瞻计算的过程中判断当前前瞻的衔接点是否需要拐角限速，如果需要则计算限制速度。

尽量保证所有指令衔接处的限制速度速度够达到，如果只能保证部分衔接点的限制速度，则优先保证限制速度较低的衔接点，不能达到限制速度的衔接点需保证实际运行速度小于其限制速度。

2) 运动过程中必须保证 VpSpeed 的变化过程始终满足预设的梯形/S 形规律；

3) 在满足上述准则所需约束条件的前提下，各指令在各自的生命周期内保证尽可能多的时间以自身的 speed 为目标速度运动。

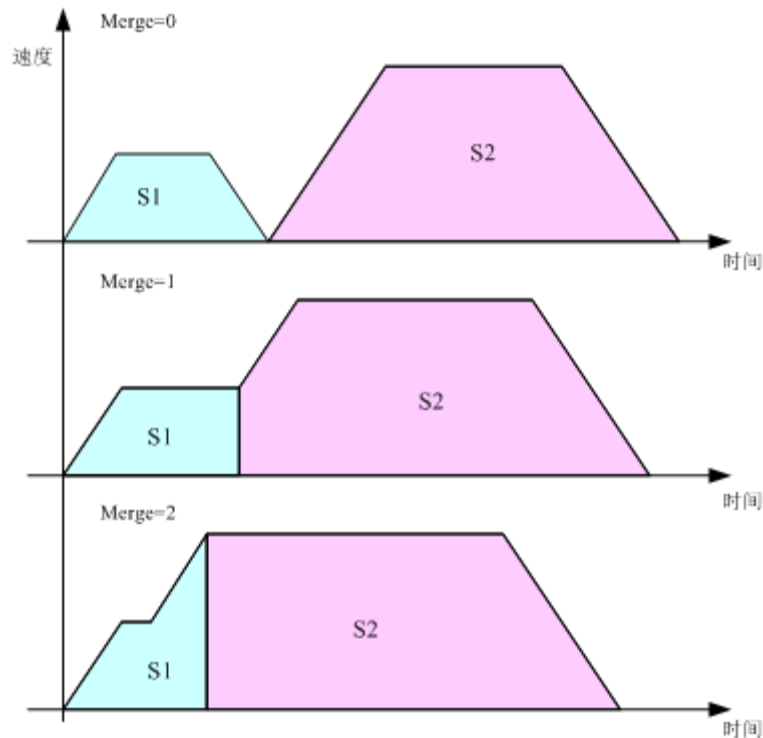
- 如下条件可作为前瞻计算的结束判据

1) 前瞻已达到指令缓存队列尾部，则前瞻结束；

2) 前瞻计算尚未达到指令缓存队列尾部，但当前条件下可以保证：在前瞻范围之外的指令即使出现任意低速的限制速度，都可以在位移条件满足的前提下，确保能够按照预设的梯形/S 形加速减速规律到达该速度。则此时可以结束当前前瞻计算。

这样可以有效减小前瞻计算深度，节省计算时间。

例如当前指令速度为 1000，下条指令目标速度为 2000，五种模式下计算结果如下：



Merge 功能示意图

减速/停止融合功能

减速/停止融合功能解决的问题是：当指令间夹角过大时，如果仍以较大速度运行，会在夹角处产生较大的机械冲击，轨迹偏离。

控制器会对指令间轨迹变化的夹角进行提前识别，比较其与减速/停止角的大小关系，提前决定是否进行减速，保证在指令连接处平稳过渡。

■ 指令间夹角 < 减速角

减速/停止角对速度不做限制，Merge 融合功能决定指令结束时的速度。

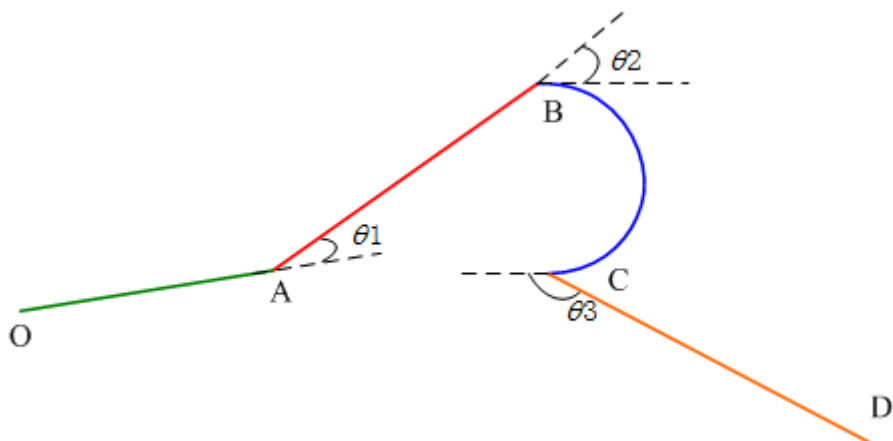
■ 减速角 < 指令间夹角 < 停止角

对速度做限制，限制后速度为 Merge 融合决定指令结束速度的 k 倍，该比例系数 k 属于 $[0,1]$ 范围内， $k = \frac{\text{停止角} - \text{指令间夹角}}{\text{停止角} - \text{减速角}}$ 。

■ 停止角 < 指令间夹角

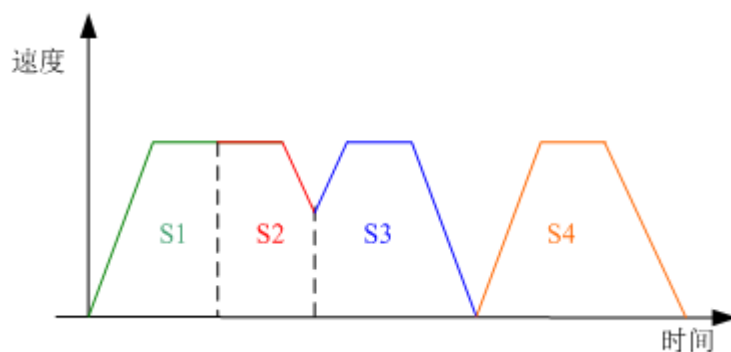
指令结束速度为 0。

如下，在二维平面走如下轨迹，依次经过 O→A→B→C→D 点执行 4 条指令后完成整个轨迹。假设，4 条指令间不通过其它方式修改合轴的 Speed 参数。



运动轨迹示意图

θ_1 为指令 1 和指令 2 间的夹角， θ_2 、 θ_3 依次为后续相邻指令间的夹角。假设角度满足关系： $\theta_1 < \text{减速角} < \theta_2 < \text{停止角} < \theta_3$ ，则合轴的速度变化如图所示：



合轴速度示意图

抑制冲击功能

抑制冲击功能主要解决的问题是：运动轨迹的曲率较大时，如果速度过大会带来较大的惯性离心力，造成机械冲击损坏工件，甚至会导致轨迹偏离失准。

■ 生效范围

区别与 **Merge** 只作用于多条指令之间，抑制冲击功能不仅支持多条 2 维指令之间，还对单条指令生效，具体如下：

单条指令，无需开启 **Merge**

- ☐ 平面圆弧插补指令、球弧插补指令；
- ☐ 螺旋线插补指令；
- ☐ 样条插补指令；

多条指令之间，需开启 **Merge**

- ☐ 2 轴直线插补（HMC_Move2D、HMC_Move2DEx）和平面圆弧插补（HMC_MoveCircle、HMC_MoveCircleEx）任意组合，插补的合轴、分轴所对应的轴号序列必须相同。

■ 抑制冲击

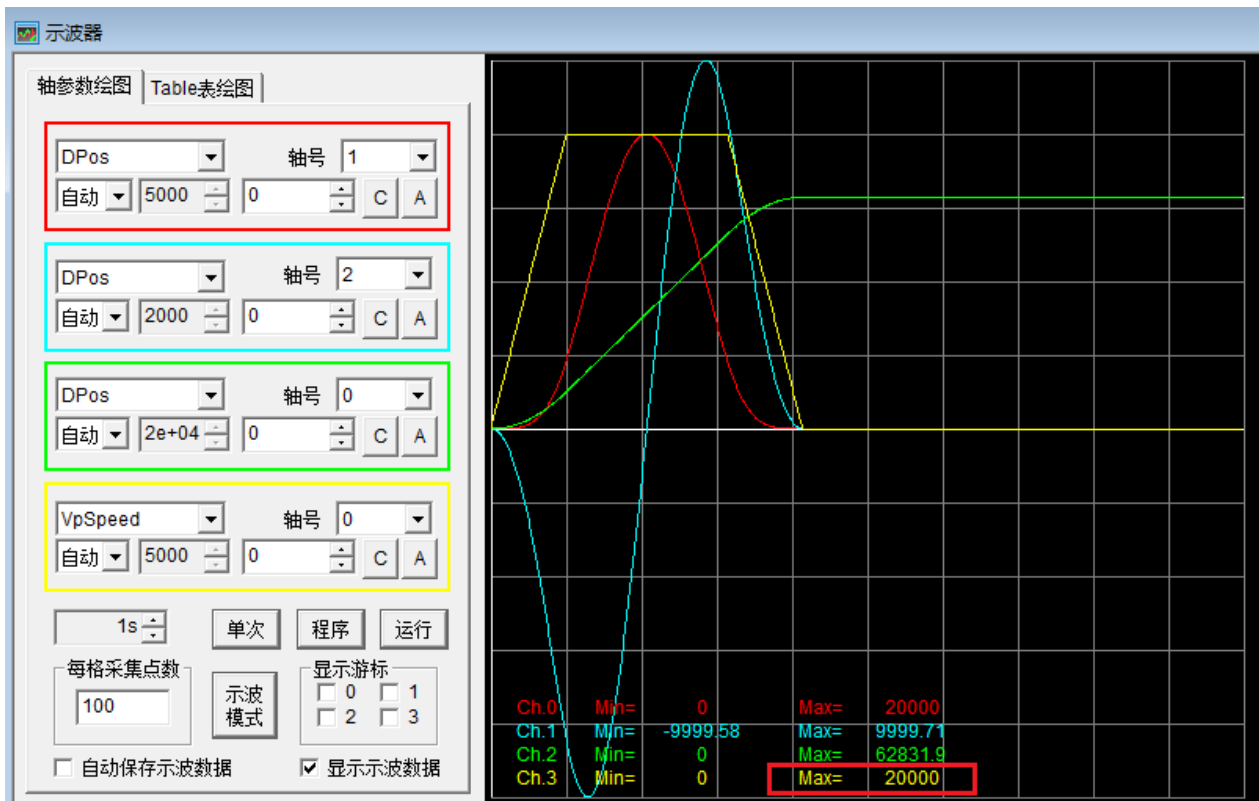
计算指令轨迹的曲率 ρ ，然后根据曲率、Jerk 计算出最大速度。

$$V_{max} = f(\rho, Jerk)$$

与指令当前的目标速度比较，是否需要对其进行限制减速，以保证全部轨迹过程内的机械冲击限制在合理范围内，同时保证轨迹的精准。

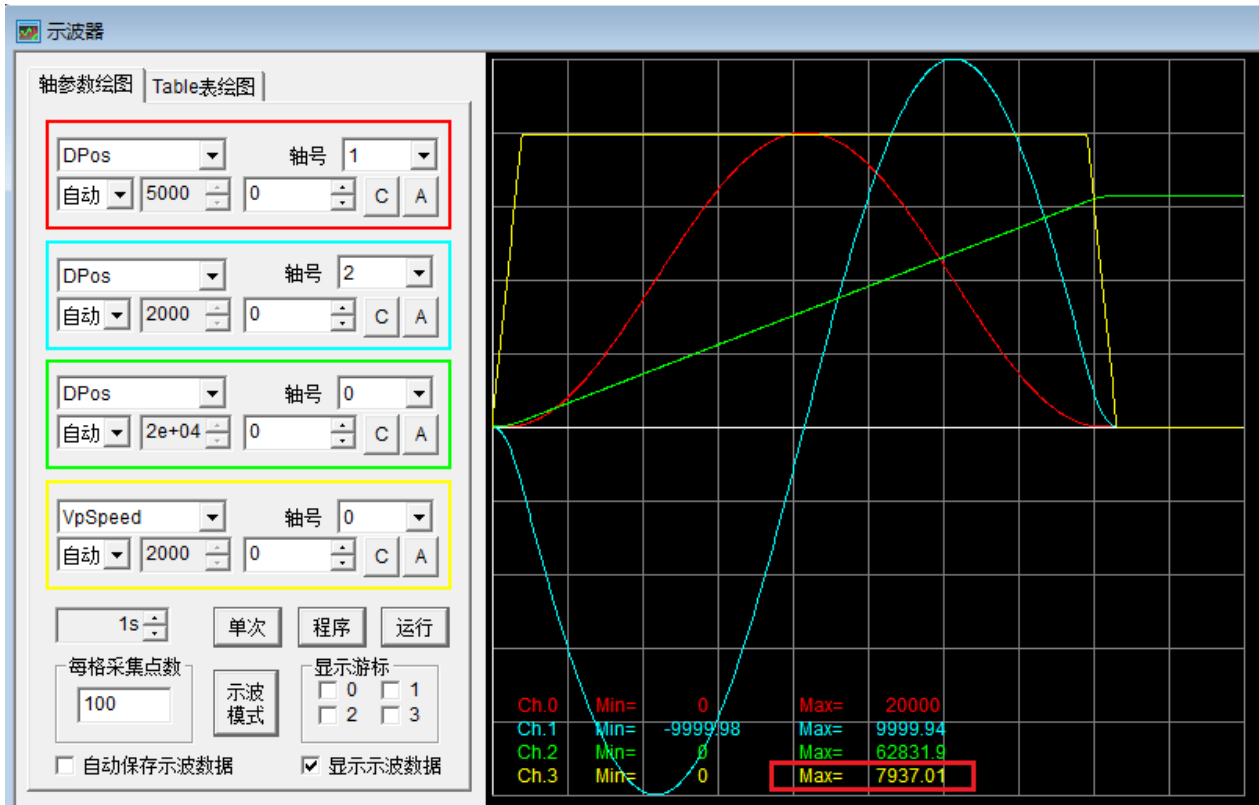
如执行一个圆弧指令，设定相同的 Speed=20000，但设定不同的 Jerk 值，实际运行时合轴的速度发生变化。

Jerk=5000000000 默认值时，合轴速度达到 20000，指令花费较短时间就解算完成。



运动轨迹示意图

修改 Jerk=5000 时，合轴速度最高仅达到 7937.01，指令花费较长时间解算完成。



运动轨迹示意图

3.4.1.5.7.2 HMC_SetMerge（设置融合功能）

函数原型

UDINT HMC_SetMerge (USINT ucAxisID, USINT ucMerge)

功能说明

设置轴参数 Merge 模式。Merge 相关详细描述见轴参数 Merge 和本章节的运动前瞻功能描述部分。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucMerge	USINT	Merge 模式，该值只可为 0、1、2、3、4。 0: Merge 关闭，每个指令块单独完成各自的运动 1: Merge 打开，当前指令块结束时目标速度为该指令块的 Speed 2: Merge 打开，当前指令块结束时目标速度为下一指令块的 Speed 3: Merge 打开，当前指令块结束时目标速度为该指令块 Speed 和下条指令块 Speed 最小值 4: Merge 打开，当前指令块结束时目标速度为该指令块 Speed 和下条指令块 Speed 最大值

返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令

应用举例

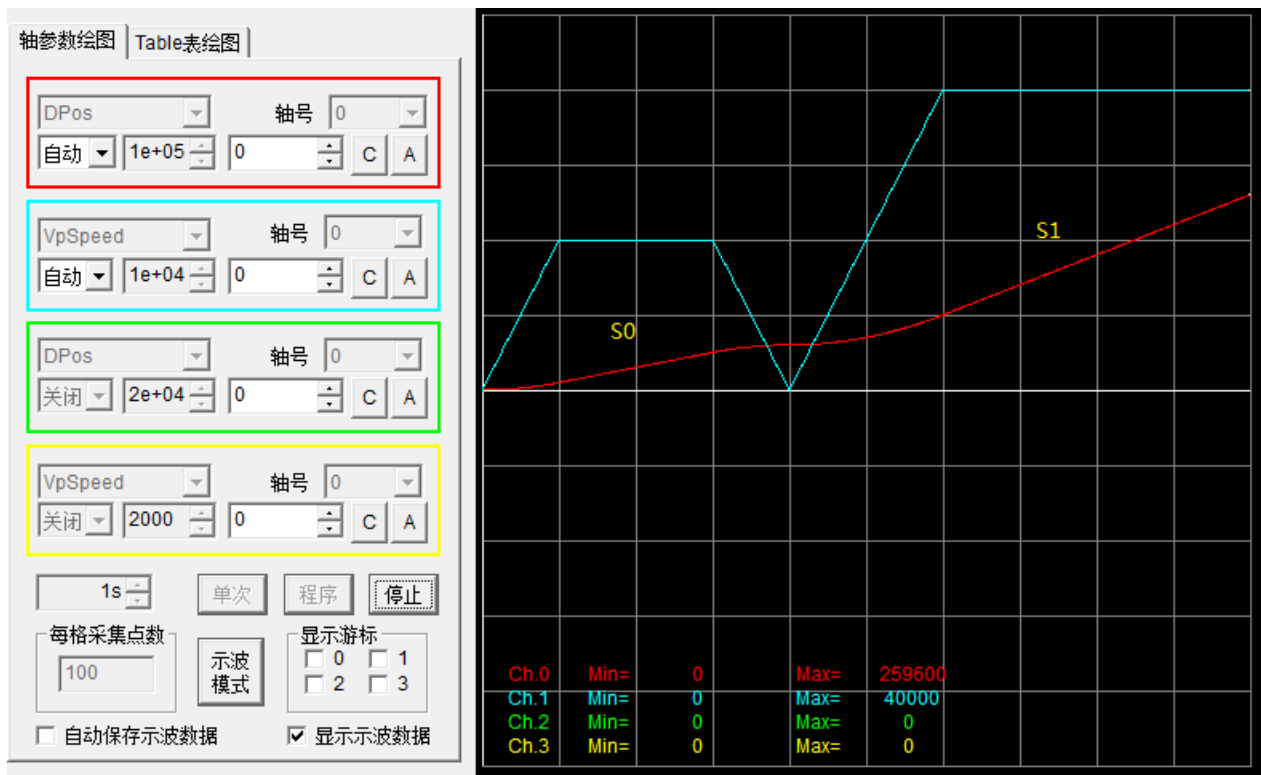
单轴指令 Merge（两轴指令的举例见设置融合角章节）程序如下：

axis0.Speed := 20000; (* 轴 0 速度为 20,000（用户单位/秒） *)

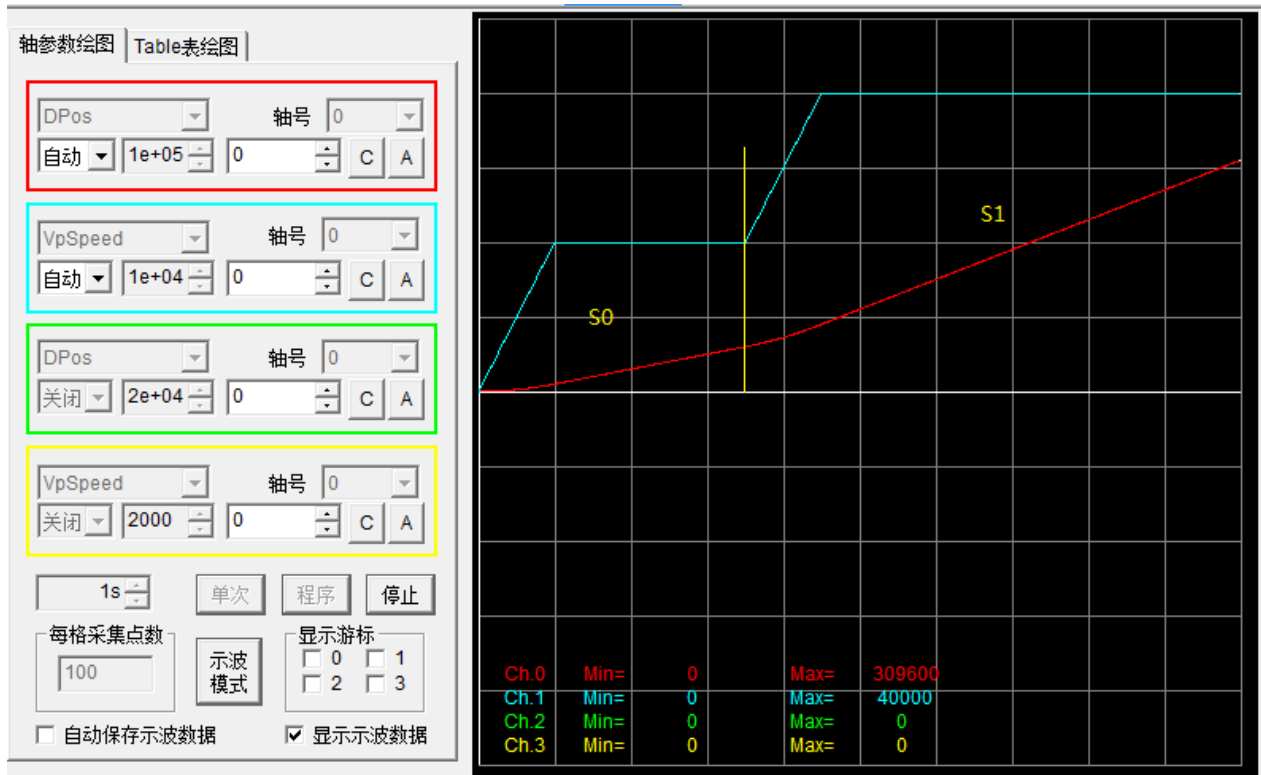
HMC_Move(0, 60000); (* 指令 0 向前运动 60000（用户单位） *)

HMC_ForwardEx(0, 40000); (* 指令 1：将 axis0.Speed 置为 40,000（用户单位/秒）然后持续向前运动 *)

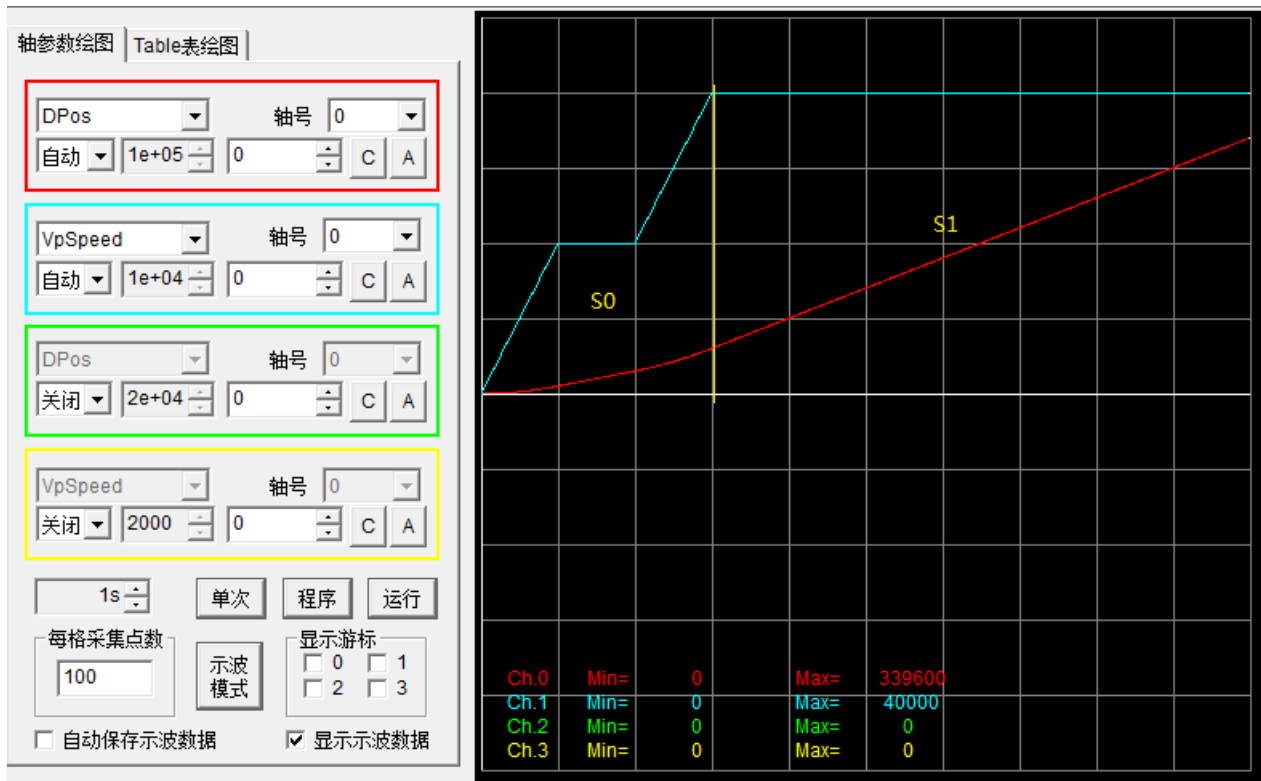
当 axis0.Merge 为 0、1、2 时，曲线形状分别如下所示（其中 S0 均为 60000 用户单位）：



Merge 为 0 示意图



Merge 为 1 示意图



Merge 为 2 示意图

■ 不支持的模块版本

3.4.1.5.7.3 HMC_ClcJerk（冲击运算）

函数原型

LREAL HMC_ClcJerk (LREAL dCurvature, LREAL dSpeed)

功能说明

根据 Speed 和曲率计算 Jerk。

$Jerk = dCurvature^2 * dSpeed^3$

输入参数

输入参数	类型	参数描述
dCurvature	LREAL	曲率（非负）
dSpeed	LREAL	速度（非负）

返回值

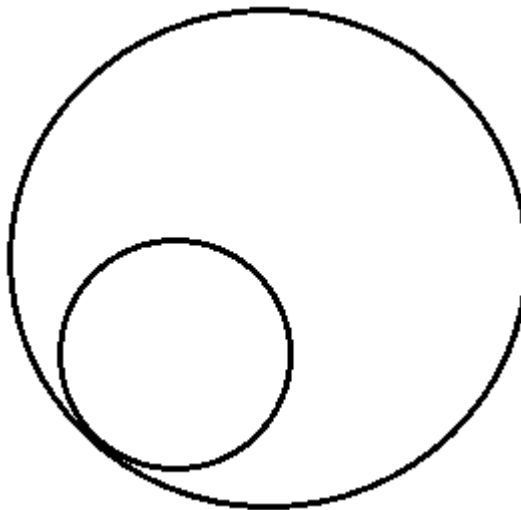
返回值	类型	参数描述
其他值	LREAL	返回 Jerk 值
-1	LREAL	参数错误

指令类型

非轴运动缓存指令

应用举例

要走如下轨迹，从两圆的交点处为起始点，先经过大圆达到起始点后，在经过小圆最终到达起始点。假设设备能承受的最大 Jerk=100000，希望速度为 1000。



不同轨迹圆

考虑 Jerk 与速度和曲率有关系，而以上两个不同轨迹圆的曲率不同，如果设置 Jerk 过小，会导致实际运行速度受限影响效率，如果设置过大，超过设备能承受的最大 Jerk，则会影响设备安全。故对两个圆分别计算出 Jerk 后，与最大 Jerk 比较，若大于系统承受最大 Jerk 则设置为最大 Jerk 否则按照计算得到的 Jerk。故编写程序如下：

```
Axis7.Speed := 1000;
```

```
Axis7.Accel := 5000;
```

```
Axis7.Decel:= 5000;
```

```
HMC_MoveCircle(7, 0, 1, 1, 0, 0, 100, 100); (*第一个大圆*)
```

```
Jerk1 := HMC_ClcJerk(1/(100*SQRT(2)), 1000); (*冲击运算函数*)
```

```
MaxJerk := 100000;
```

```
IF Jerk1 > MaxJerk THEN (*与设备所能承受的最大冲击比较，原则为：安全第一效率第二*)
```

```
    axis7.Jerk := MaxJerk;
```

```
ELSE
```

```
    axis7.Jerk := Jerk1;
```

```
END_IF
```

```
HMC_MoveCircle(7, 0, 1, 1, 0, 0, 50, 50);
```

```
Jerk2 := HMC_ClcJerk(1/(50*SQRT(2)), 1000); (*冲击运算函数*)
```

```
MaxJerk := 100000;
```

```
IF Jerk2 > MaxJerk THEN (*与设备所能承受的最大冲击比较，原则为：安全第一效率第二*)
```

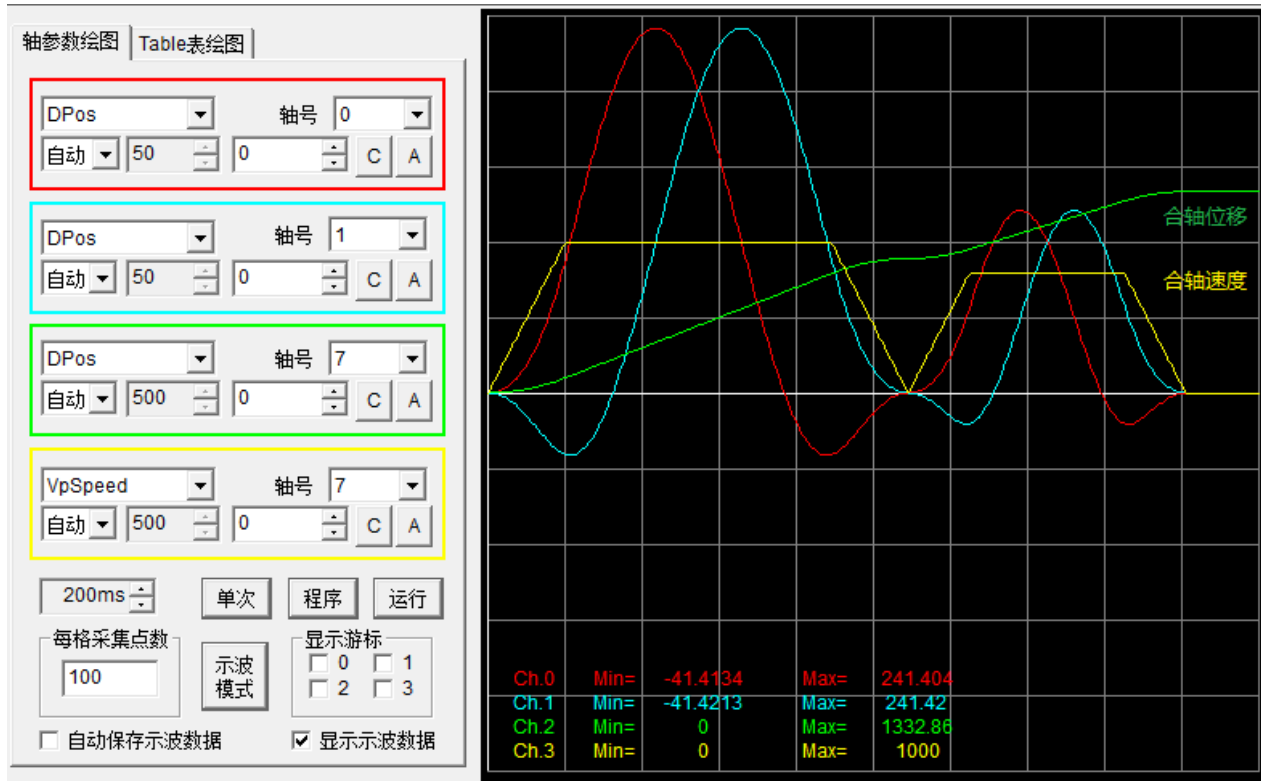
```
    axis7.Jerk := MaxJerk;
```

```
ELSE
```

```
    axis7.Jerk := Jerk2;
```

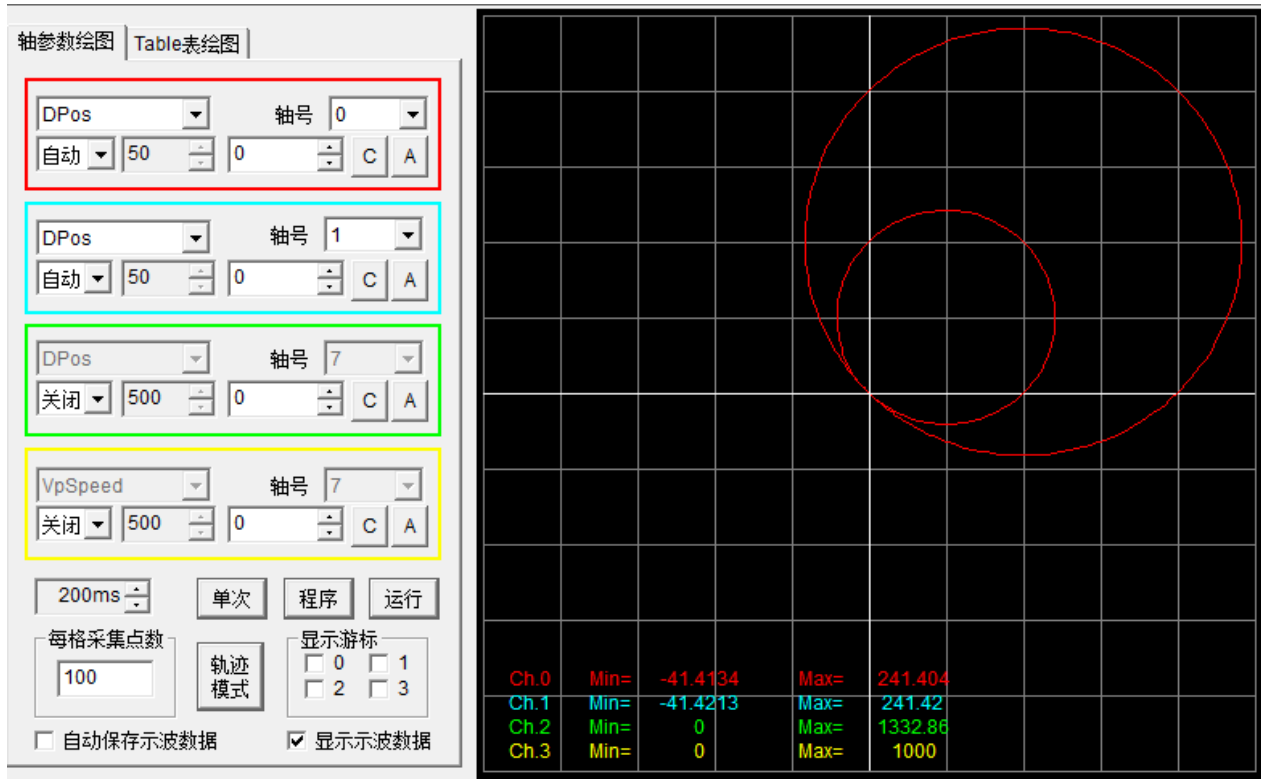
```
END_IF
```

执行后结果如下：



运动轨迹示意图

从上图可以看出，虽然设置了同样的指令速度 $Speed=1000$ ，但小圆轨迹执行时的实际运行速度并未达到 1000，这是因为因为小圆的曲率较大，以 1000 速度运行时需要设备承受的冲击超过了设备能承受的最大冲击 100000,此时设置 $Jerk=100000$ ，则限制了实时结算速度。



运动轨迹示意图

- 不支持的模块版本

MC1008-A01

3.4.1.5.7.4 HMC_SetMergeAngle（设置融合角度）

函数原型

UDINT HMC_SetMergeAngle (USINT ucAxisID, LREAL dDecelAngle, LREAL dStopAngle)

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dDecelAngle	LREAL	设置二维指令的减速角，对合轴设置有效，不可为负
dStopAngle	LREAL	设置二维指令的停止角，对合轴设置有效，不可为负

返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

指令类型

非轴运动缓存指令

$dDecelAngle \leq \pi/6$, $dStopAngle \leq \pi/2$, 且 $dDecelAngle \leq dStopAngle$ 。

$dDecelAngle$ 默认值为 $\pi/6$, $dStopAngle$ 默认值为 $\pi/2$ 。

应用举例

两轴指令 Merge，程序如下：

```
axis0.Speed := 20000;    (*轴 0 速度为 20,000 (用户单位/秒) *)
```

```
axis0.Accel := 200000;
```

```
axis0.Decel := 200000;
```

```
HMC_Move2D(7, 0, 1, 10000, 0);
```

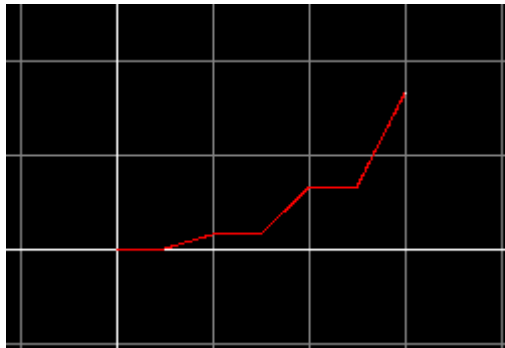
```
HMC_Move2D(7, 0, 1, 10000, 3000);
```

```
HMC_Move2D(7, 0, 1, 10000, 0);
```

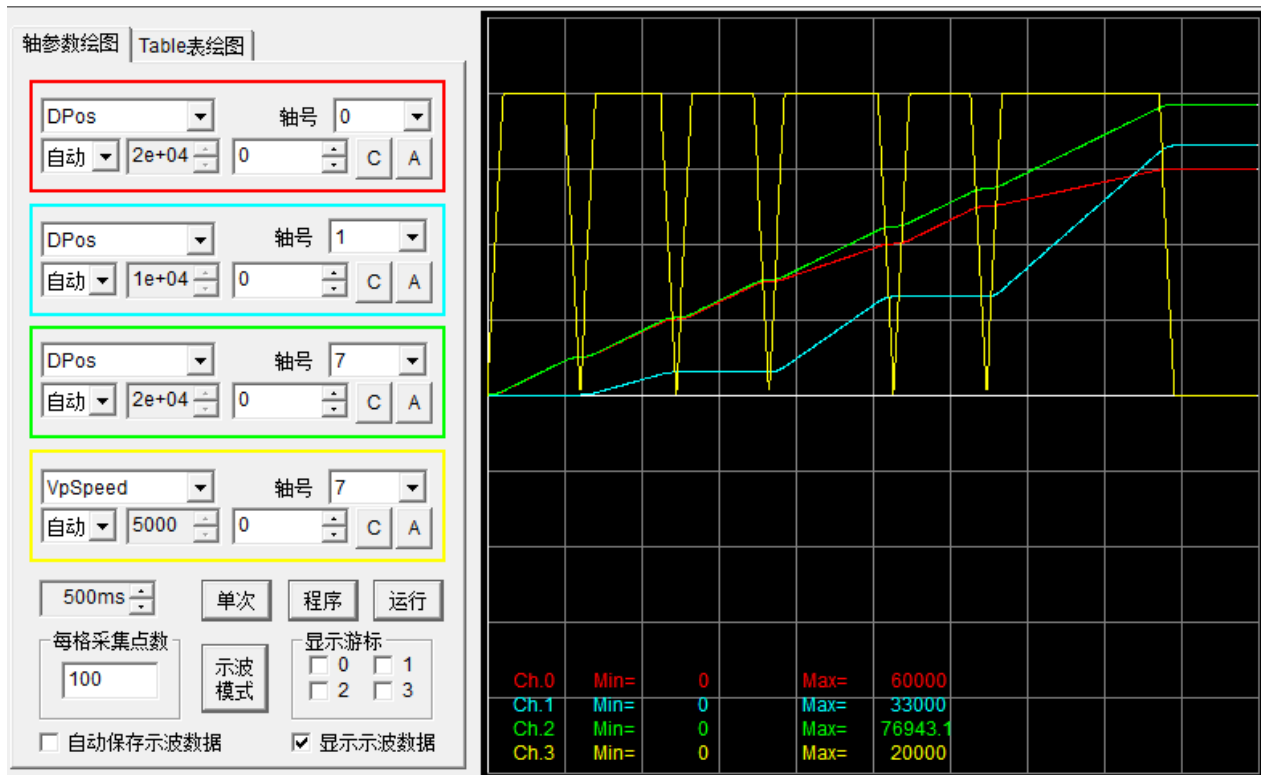
```
HMC_Move2D(7, 0, 1, 10000, 10000);
```

```
HMC_Move2D(7, 0, 1, 10000, 0);
```

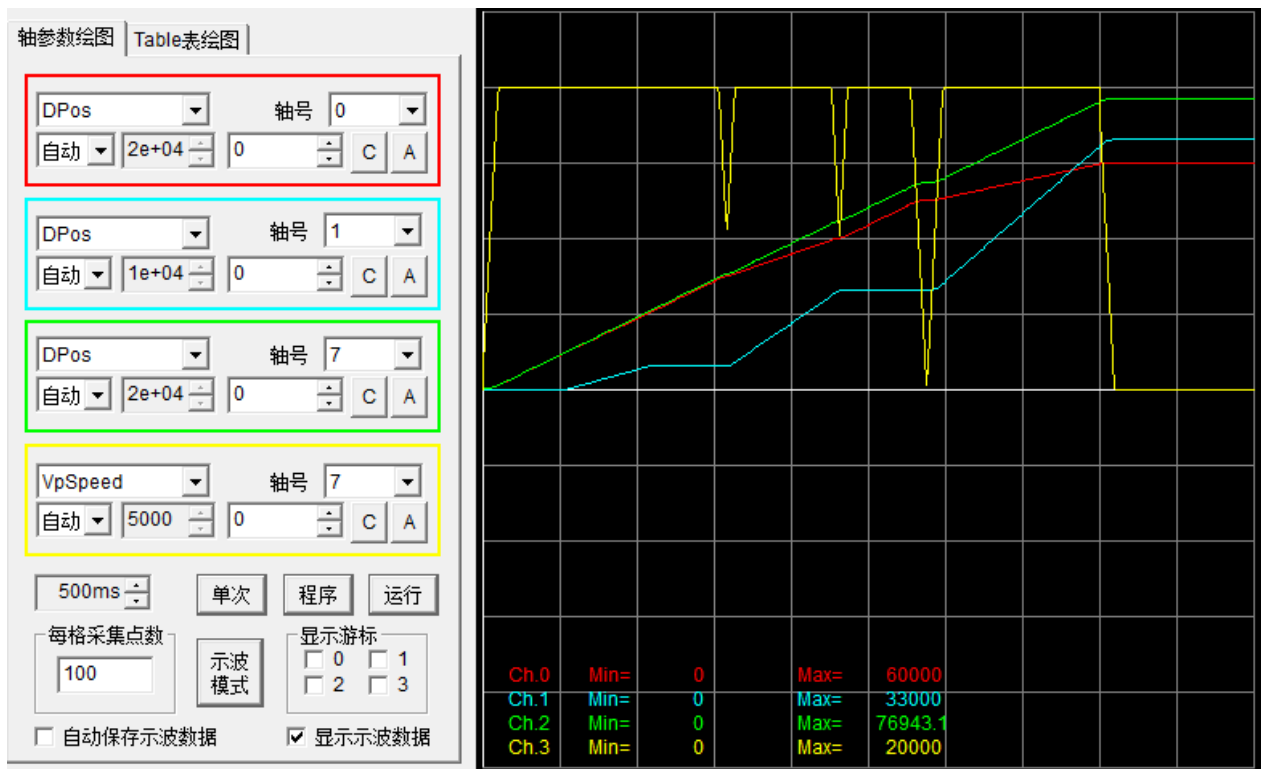
```
HMC_Move2D(7, 0, 1, 10000, 20000);
```



Merge 在二维指令下的情况



Merge 为 0 时的情况



Merge 为 1 时的情况

- 不支持的模块版本

MC1008-A01

3.4.1.5.7.5 HMC_GetMergeStopAngle（获取融合停止角）

函数原型

LREAL HMC_GetMergeStopAngle (USINT ucAxisID)

功能说明

获取插补运动停止角。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
运动停止角	LREAL	弧度

指令类型

非轴运动缓存指令

■ 不支持的模块版本

MC1008-A01

3.4.1.5.7.6 HMC_GetMergeDecelAngle（获取融合减速角）

函数原型

LREAL HMC_GetMergeDecelAngle (USINT ucAxisID)

功能说明

获取插补运动减速角。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
运动减速角	LREAL	弧度

指令类型

非轴运动缓存指令

■ 不支持的模块版本

MC1008-A01

3.4.1.5.7.7 HMC_SetMergeSpeedTolerRatio（设定速度波动抑制比）

函数原型

UDINT HMC_SetMergeSpeedTolerRatio (USINT ucAxisID, LREAL dTolerRatio)

功能说明

该指令设定运动前瞻过程中的速度波动抑制比参数，速度波动抑制功能允许在用户设定的误差比率范围内抑制速度波动。

在运动前瞻过程中，为降低冲击，运行速度受曲率限速以及拐角限速机制的约束，因此速度常常会有一定波动。当使用小折线逼近曲线时，如果用户提供的原始点位数据精度过低，可能会造成曲率限速及拐角限速机制计算出的约束速度频繁上下跳动，从而造成速度高频波动。

速度波动抑制功能可解决上述问题，减小乃至消除这种波动。速度波动抑制比 dTolerRatio 越大，抑制作用越强，但过大时可能会增加系统冲击。因此，在可以接受的速度波动范围内，尽量取 dTolerRatio 为较小值。如果开启 S 形加减速并设置合适的加加速度，则可在同样的 dTolerRatio 下达到更好的波动抑制效果。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dTolerRatio	LREAL	速度波动抑制比，[0,1] 值越大抑制作用越强，但过大可能会增加系统冲击

返回值

返回值	类型	参数描述
0	UDINT	成功
错误码	UDINT	参数错误

指令类型

非轴运动缓存指令。

3.4.1.5.7.8 HMC_GetMergeSpeedTolerRatio (读取速度波动抑制比)

函数原型

LREAL HMC_GetMergeSpeedTolerRatio (USINT ucAxisID)

功能说明

读取当前的速度波动抑制比。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
速度波动抑制比	LREAL	当前速度波动抑制比

指令类型

非轴运动缓存指令。

3.4.1.5.8 曲线限速

3.4.1.5.8.1 HMC_SetCurveJerkLimit（设定曲线冲击上限）

函数原型

UDINT HMC_SetCurveJerkLimit (USINT ucAxisID, LREAL dLimitVal)

功能说明

该指令设定曲线插补过程中物理冲击的上限值，由此可限定曲线的插补速度。

通过合理设定该参数，可使插补过程中曲线的大曲率处（如小圆圆弧）有较大的降速，而小曲率处（如大圆圆弧）降速较少或不降速。

可通过设备在线调试找到满足具体设备的冲击参数值，也可通过 HMC_ClcJerk 指令计算出在特定曲率（半径的倒数）、特定速度下的冲击值，作为冲击上限值。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dLimitVal	LREAL	设定的上限值，非负

返回值

返回值	类型	参数描述
0	UDINT	成功
错误码	UDINT	参数错误

指令类型

非轴运动缓存指令

3.4.1.5.8.2 HMC_GetCurveJerkLimit（读取曲线冲击上限）

函数原型

LREAL HMC_GetCurveJerkLimit (USINT ucAxisID)

功能说明

读取设定的曲线插补物理冲击的上限值。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
冲击上限值	LREAL	曲线插补物理冲击上限值 指令参数错误时返回 0

指令类型

非轴运动缓存指令

3.4.1.5.9 切向追踪

3.4.1.5.9.1 HMC_MoveTang（切向追踪）

函数原型

UDINT HMC_MoveTang (USINT ucAxisC, USINT ucAxisX, USINT ucAxisY, LREAL dOffset)

功能说明

该功能块根据 X、Y 运动轨迹，实时计算出二者合运动轨迹与 X 轴正向夹角，然后驱动切刀旋转轴 C 轴跟踪至目标角度，可用在物料切割场景，自动实时调整刀具方向与切割轨迹的切向方向（切割速度矢量方向）保持一致，达到最好的切割效果。可设置刀具倾角的偏移量 dOffset，以调节刀具切割时的前角、后角角度。

该指令运行过程中，C 轴（刀具旋转轴）工作在循环行程模式下，模式 RepMode 为 2(对称区间)。使用该指令前，用户需首先把循环行程长度（[-Repdist, Repdist]）设为该轴旋转一圈所对应的用户单位下的位移量(假设为 RoundLength)。即：

$$\text{Repdist} = \text{RoundLength}/2$$

这样，C 轴的绝对位置[-Repdist, Repdist]就对应于 $[-\pi, \pi]$ 的切向角度。

指令执行过程中实时计算出 X 轴（轴号 ucAxisX）、Y 轴（轴号 ucAxisY）合运动轨迹的切向方向与 X 轴正向所成的角度 TangAngle，即 C 轴的目标切向角；同时结合目标切向角的变化速度（角速度）。实时调整 C 轴目标位置，驱动 C 轴（轴号 ucAxisC）定位至目标切向角，实现对两轴合运动轨迹角度的最优跟随。

使用切向追踪功能时，用户最好先估算出刀具旋转轴 C 轴的最大加减速能力，以及最大运行速度，这些参数可根据电机的额定转矩、额定转速等参数，结合负载情况以及设备机械结构对振动、扭矩的要求估算得到。根据上述计算把 C 轴的加减速、speed 参数设定为设备所允许的最大值（或乘以一定的安全余量系数）。在切向追踪执行过程中，算法会按照不超过 C 轴设定的加减速及 speed 的运动参数驱动 C 轴至目标切向角度。

若 C 轴的最大加减速能力小于实际切向角变化的加/减速度，或最大运行速度小于实际切向角变化的最大速度，则 MoveTang 运行过程中 C 轴不能实现无偏差跟随切向角度。

在指令执行过程中，当目标切向角发生跃变或目标切向角的变化速度（角速度）发生跃变时，刀具旋转轴 C 将以轴参中设定的加减速及速度朝目标位置运动，而不会发生因运行速度的跃变而造成的冲击。但此处 C 轴的实际角度位置与目标切削角之间将存在一定的跟随偏差。

设备调试时可用 HMC_GetMoveTangDeviation 实时获取切向角跟随偏差，查看切向追踪实际运行效果。如果跟随偏差超出应用所要求的范围，则可用 HMC_GetMoveTangDestDirection 和 HMC_GetMoveTangDestVelocity 指令分别获取目标切向角位置、切向角变化速度，通过对比 C 轴 DPOS 变化曲线与 VpSpeed 变化曲线，可分析出偏差产生的具体原因，一般可分为以下 3 类：

(1) C 轴加减速/Speed 过小，无法跟随切向角变化。解决措施：

- 加大 C 轴加减速/Speed;

- 减小 XY 轴插补速度以降低切向角变化速率；
- 若 XY 轴插补速度不可降低，且 C 轴加减速/Speed 已达到 C 轴电机扭矩极限或机械结构极限，则考虑更换更大扭矩的电机或变更机械结构。

(2) 切向角位置 发生跃变。（XY 插补轨迹存在一阶不可导点，如两条折线）。解决措施：

- 优化 XY 插补路径。比如对于折线路径，可在折线之间插入圆弧；
- 如果需要的就是折线轨迹，则可加入辅助进退刀路径，使刀具首先从一段折线的延长线切出，再从另一段折线的延长线切入。
- 加大 C 轴加减速/Speed 或 减小跃变点处 XY 轴插补速度 可降低该偏差的影响范围，但无法根本消除。

(3) 切向角变化速度 发生跃变。解决措施：

- 优化 XY 插补路径。比如使用样条插补轨迹。
- 如果插补路径不可更改，则可加大 C 轴加减速/Speed 或 减小跃变点处 XY 轴插补速度。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号
ucAxisX	USINT	X 方向运动轴轴号
ucAxisY	USINT	Y 方向运动轴轴号
dOffset	LREAL	刀具旋转轴偏移量（用户单位）

返回值

返回值	类型	参数描述
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

补充说明

- (1) 该指令一旦被投送成功，则一直生效，直到执行 **cancel** 指令撤销为止。
- (2) 切向追踪支持 X、Y 轴为任意指令类型。
- (3) 刀具旋转轴 C 轴的最大速度、加减速由其自身轴参设定值决定，应根据运动情况及该轴自身的特性设定合适的值（在轴自身能承受的范围内保证该值足够大），以确保 C 轴能够快速完成跟随切向角的动作。
- (4) 当两段运动轨迹的衔接处发生急剧的角度变化时，此时程序会自动选择最短的路径追踪定位切向角度，不会出现打转现象。
- (5) 在指令执行过程中，当目标切向角或目标切向角的变化速度（角速度）发生跃变时，刀具旋转轴 C 将以轴参中设定的加减速及速度朝目标位置运动，而不会发生因运行速度的跃变而造成冲击。但此处 C 轴的实际角度位置与目标切削角之间将存在一定的跟随偏差。
- (6) X/Y 轴遇到限位时，在减速停止过程中切向角会按照实际减速轨迹计算。在限位停止动作完成

后，若行程超限状态尚未解除，并且后续所投送到主轴 XY 中的指令不是朝 X/Y 限位的反方向运行的（减轻位置超限方向），则按照指令使用异常处理，出于安全考虑，此时刀具旋转轴 C 轴将保持原位不动。

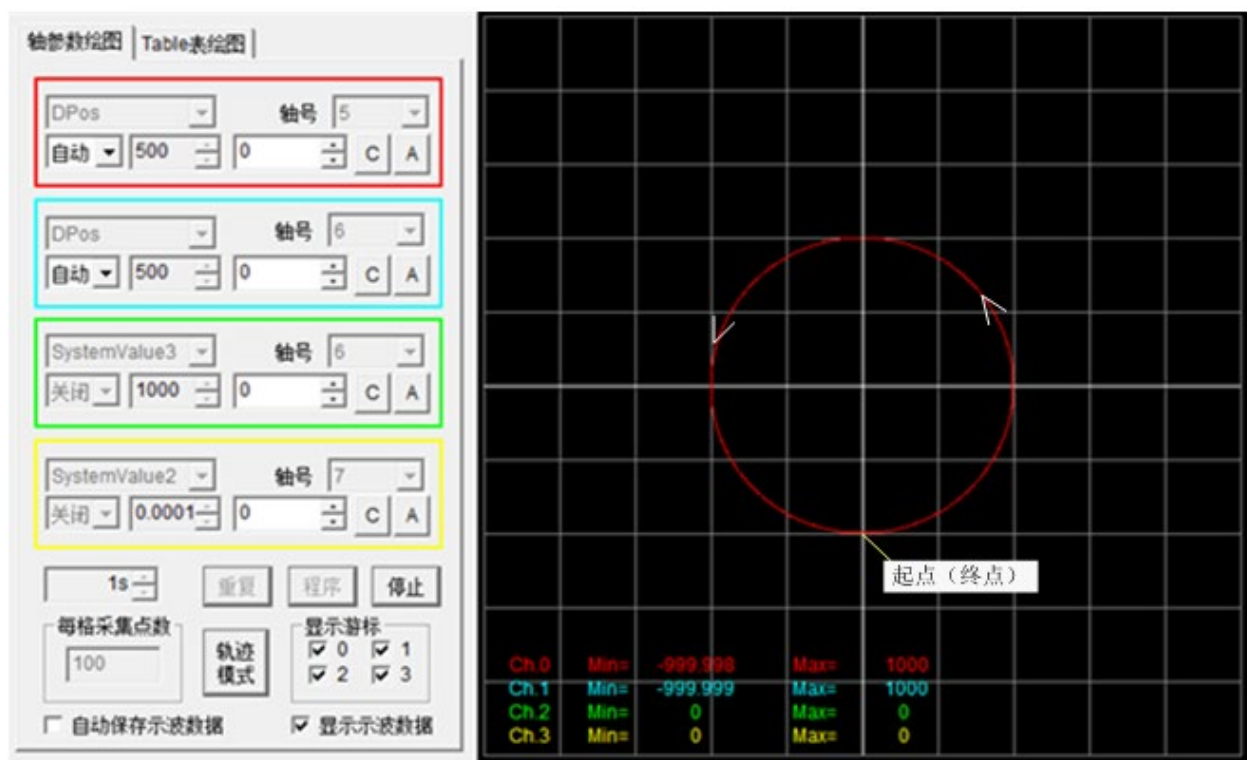
应用举例

为说明 C 轴的参数设定对跟随偏差的影响，现对一些典型情况的处理举例如下。设切向追踪刀具旋转轴 C 轴为 7 号轴，跟随 5、6 号轴的插补轨迹的切向角。设定 7 号轴为循环模式 2，循环行程区间为[-10000,10000)。

首先投送指令 hmc_movetang(7,5,6,0)，主轴 5、6 执行如下不同的指令。

1. 主轴 XY 执行圆弧插补

投送 hmc_movecircle(4,5,6,1,0,0,0,1000)执行圆弧插补，半径为 1000。插补 Speed = 1000，accel = decel = 2000。圆弧轨迹如下：



XY 轴插补轨迹

可算得执行圆弧插补过程中一些关键的运动参数（C 轴单位下）：

$$\text{切向角变化的最大速度} = \frac{\text{速度}}{\text{半径}} * \text{行程长度} = \frac{1000}{1000} * 20000 = 3,183.09886$$

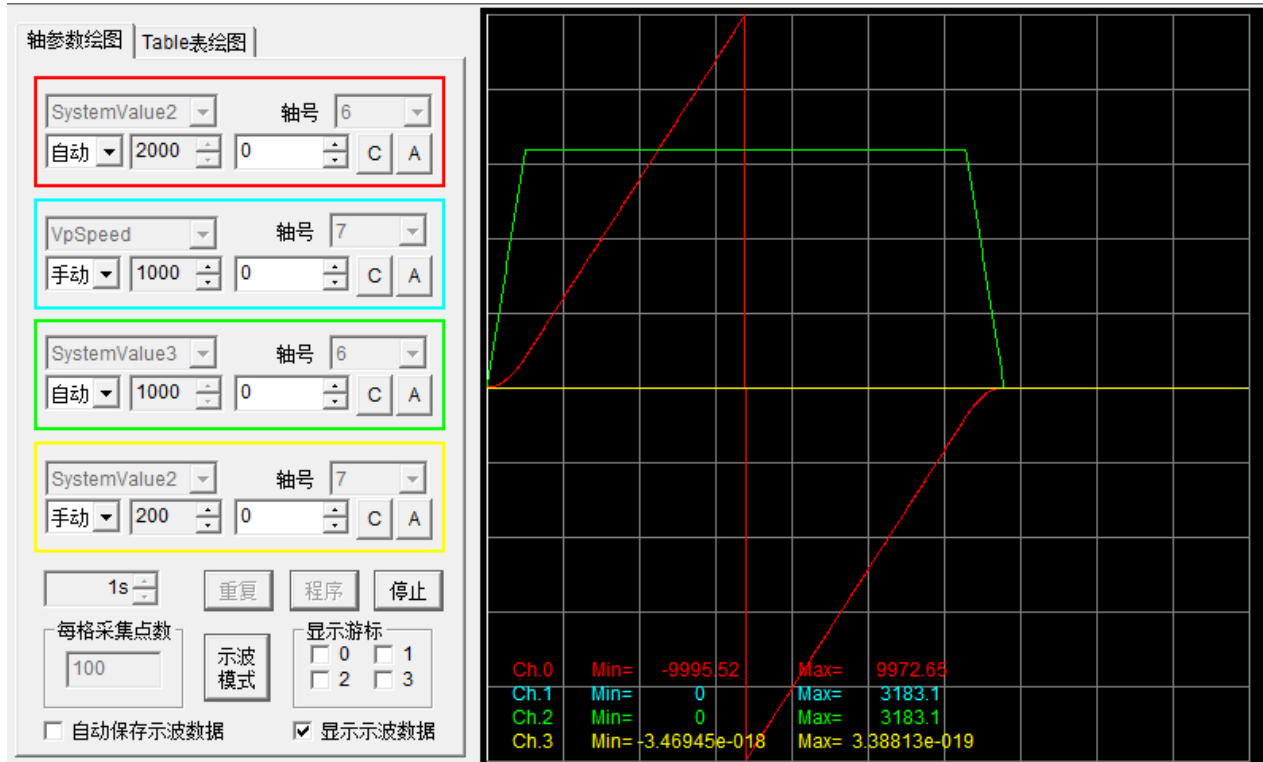
$$\text{切向角变化的加速度} = \frac{\text{加速度}}{\text{半径}} * \text{行程长度} = \frac{2000}{1000} * 20000 = 6,366.19772$$

$$\text{切向角变化的减速度} = \frac{\text{减速度}}{\text{半径}} * \text{行程长度} = \frac{2000}{1000} * 20000 = 6,366.19772$$

C 轴加减速及速度参数若大于以上计算参数，便可实现无差跟随。不同的 C 轴参数的运动效果如下图所示，红色曲线为 moveTang 执行过程中的实时目标角度位置，青色为刀具旋转轴 C 轴 VpSpeed，绿色为目标切向角的变化速度，黄色为 moveTang 执行过程中指令解算位置与理论目标角度位置的差值（随动偏差）。

(1) accel = decel = 10000, speed = 10000

由下图可知此时 C 轴可实现无差跟随，C 轴 VpSpeed 曲线与切向角的变化速度曲线完全重合。红色曲线的跃变是由于循环行程边界处的折算。



圆弧插补切向追踪实时数据（C 轴加减速、最大速度 Speed 足够）

红色：目标切向角

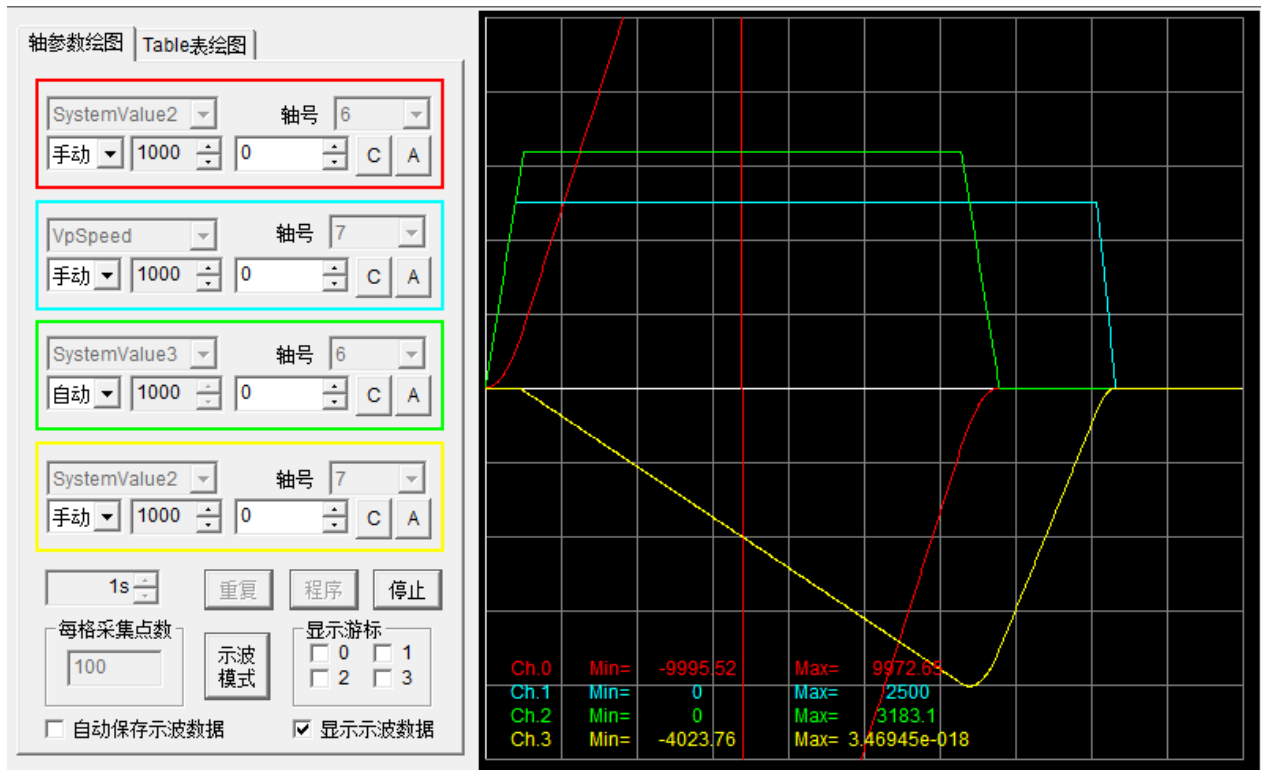
青色：刀具旋转轴 VpSpeed

绿色：目标切向角变化速度

黄色：解算位置与目标切向角的差值

(2) accel = decel = 10000, speed = 2500

此时 C 轴最大速度小于切向角最大变化速度，无法实现无差跟随。



圆弧插补切向追踪实时数据（C 轴最大速度 Speed 过小）

红色：目标切向角

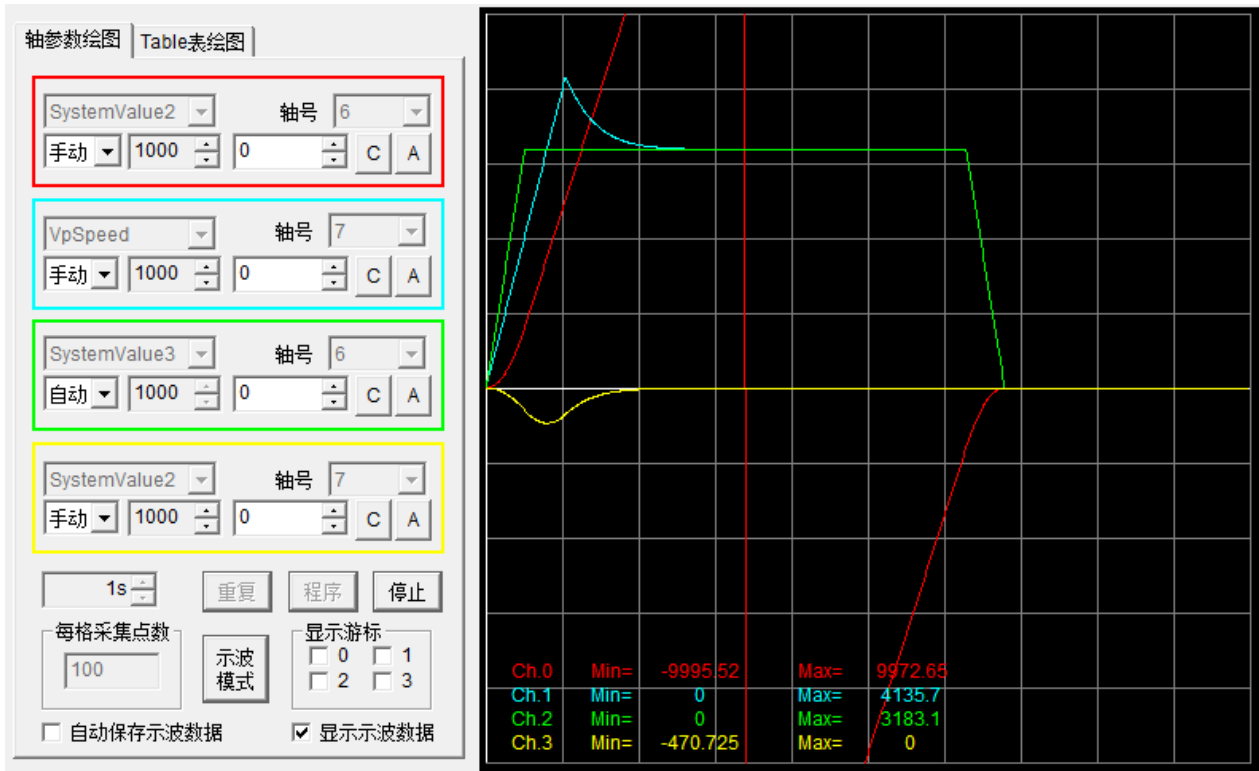
青色：刀具旋转轴 VpSpeed

绿色：目标切向角变化速度

黄色：解算位置与目标切向角的差值

(3) accel = 4000, decel = 10000, speed = 10000

此时 C 轴最大加速度小于切向角变化的最大加速度，加速阶段无法实现无差跟随。



圆弧插补切向追踪实时数据（C 轴加速能力不足）

红色：目标切向角

青色：刀具旋转轴 VpSpeed

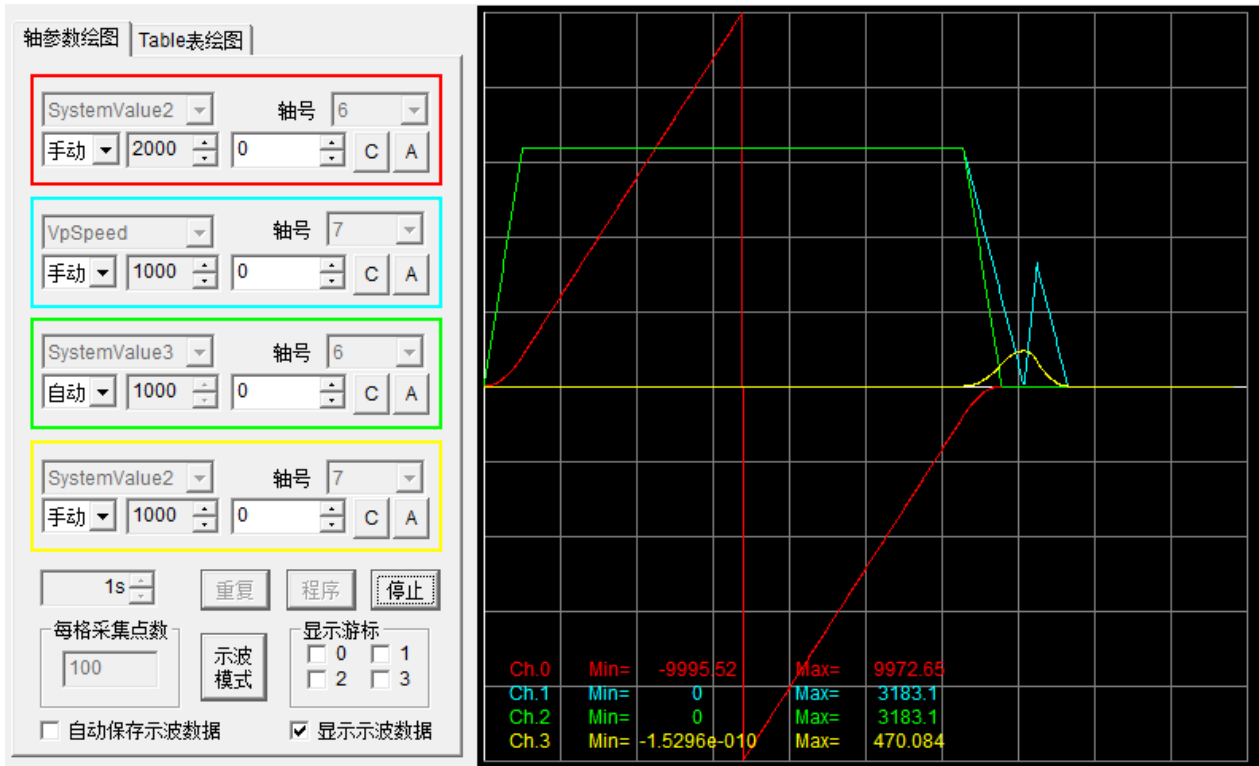
绿色：目标切向角变化速度

黄色：解算位置与目标切向角的差值

(4) accel = 10000, decel = 4000, speed = 10000

此时 C 轴最大减速度小于切向角变化的最大减速度，减速阶段无法实现无差跟随。

因此最后减速阶段 C 轴首先减速至 0（已经超过目标位置），再反向定位到目标位置。



圆弧插补切向追踪实时数据（C 轴减速能力不足）

红色：目标切向角

青色：刀具旋转轴 VpSpeed

绿色：目标切向角变化速度

黄色：解算位置与目标切向角的差值

2. 主轴 XY 执行样条插补

样条插补过程中的切向角度变化的速度及加减速均为变值，最大加减速及最大速度无法精确计算得出。只要 C 轴加减速及速度参数大于样条插补过程中的切向角度变化的速度及加减速，便可实现无差跟随。如图所示：

轴参数绘图 | Table表绘图

DPos 轴号 5
 手动 500 0 C A

DPos 轴号 6
 自动 500 0 C A

SystemValue3 轴号 6
 关闭 2000 0 C A

SystemValue2 轴号 7
 手动 1 0 C A

500ms 重复 程序 停止

每格采集点数 100 轨迹模式 显示游标

☐ 自动保存示波数据 ☒ 显示示波数据



XY 轴 2D 样条插补轨迹

轴参数绘图 | Table表绘图

SystemValue2 轴号 6
 手动 2000 0 C A

VpSpeed 轴号 7
 自动 1e+04 0 C A

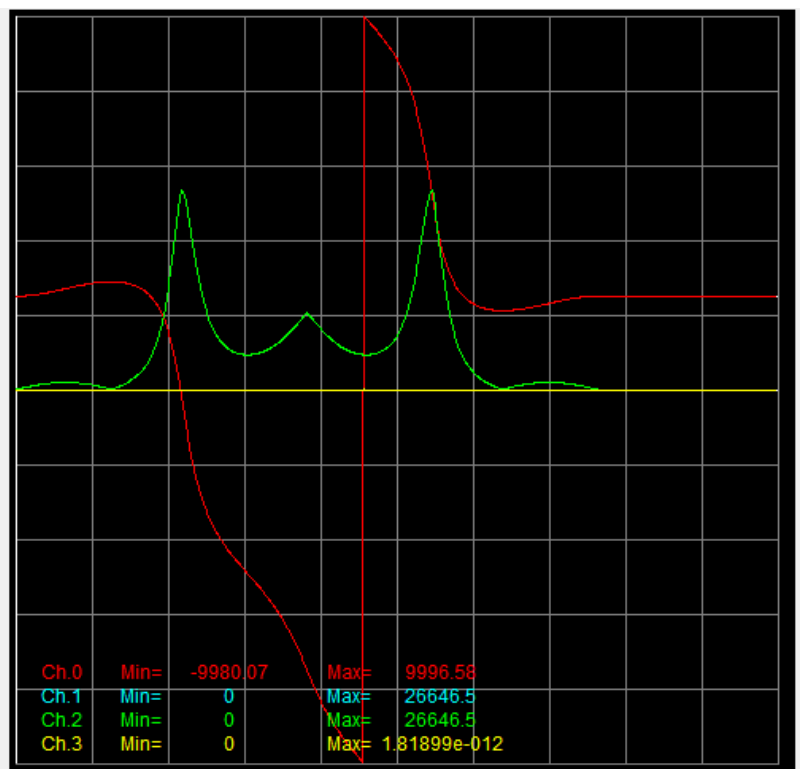
SystemValue3 轴号 6
 自动 1e+04 0 C A

SystemValue2 轴号 7
 手动 1 0 C A

500ms 重复 程序 停止

每格采集点数 100 示波模式 显示游标

☐ 自动保存示波数据 ☒ 显示示波数据



样条插补切向追踪实时数据

红色：目标切向角

青色：刀具旋转轴 VpSpeed

绿色：目标切向角变化速度

黄色：解算位置与目标切向角的差值

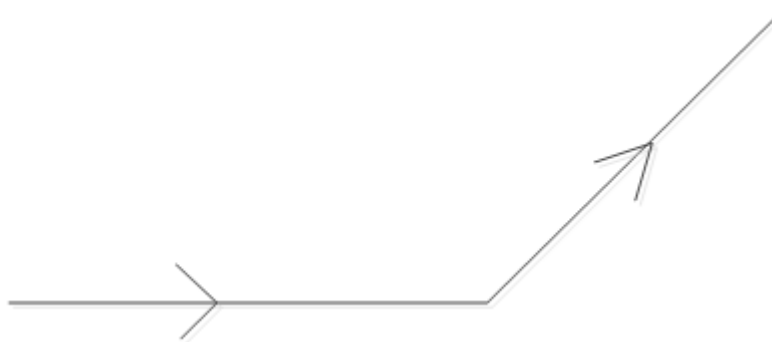
3. 曲线衔接处的处理

在曲线衔接处，若目标切向角或目标切向角变化速度发生阶跃变化，这时 C 轴将按照自身设定的加减速及 speed 朝目标切向角运行，所以不会产生运动冲击，但此处 C 轴实际位置与目标切向角位置会产生一定的偏差，且该偏差不可避免。

设切向追踪刀具旋转轴 C 轴为 7 号轴，跟随 5、6 号轴的合位移的切向角。设定 7 号轴为循环模式 2，循环行程区间为[-10000,10000]。首先投送指令 hmc_movetang(7,5,6,0)。

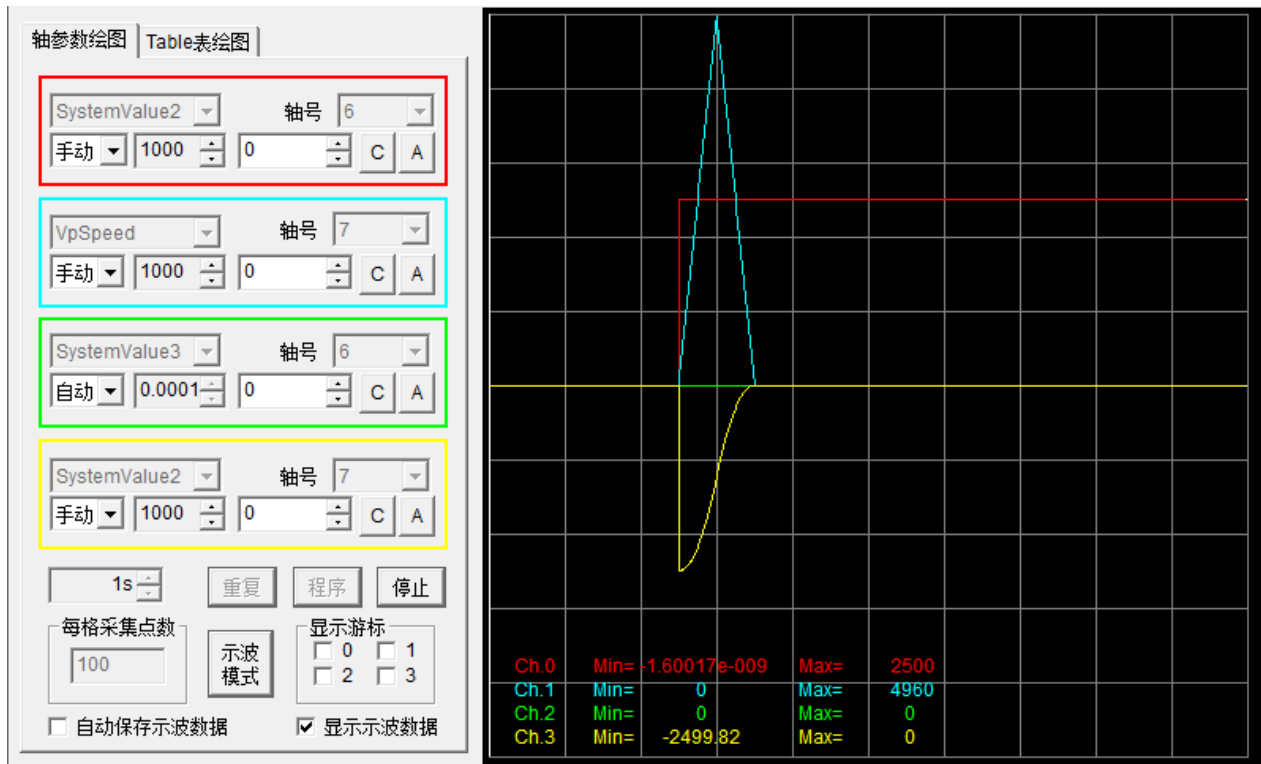
这里仅举例说明直线之间的衔接。折线处由于目标角度位置发生阶跃变化，这时 C 轴将以自身的加减速朝目标位置及目标速度运行，MoveTang 刀具旋转轴无法无差跟随。

投送指令：hmc_move2d(4,5,6,2000,000); hmc_move2d(4,5,6,2000,2000); 轨迹如下：



XY 轴运行轨迹

C 轴参数：accel = decel = 10000，speed = 10000。运行轨迹如下：



切向追踪实时数据（折线，Merge 开启）

红色：目标切向角

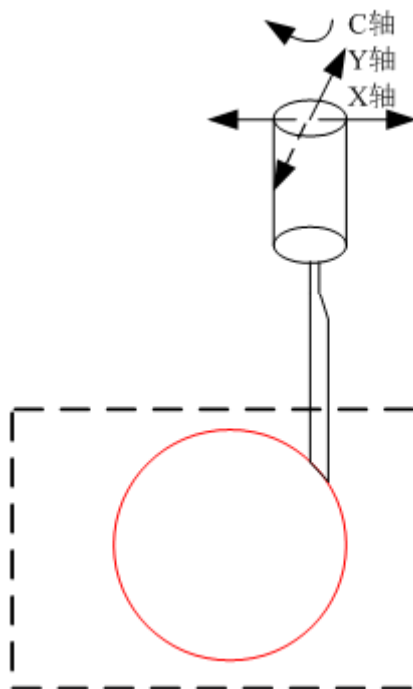
青色：刀具旋转轴 VpSpeed

黄色：解算位置与目标切向角的差值

4. 切割应用场景举例

使用切刀切割一个平面圆弧，为保证切割效果，降低切刀磨损程度，应使得在整个切割过程中实时调整切刀刀刃跟随圆弧轨迹的切线方向变化。

X、Y 轴驱动切刀在 XY 平面执行圆弧插补运动，切割物料；C 轴驱动切刀进行旋转，控制切刀方向跟随切割轨迹实时调整角度：



切割示意图

则 AT 中编写程序如下：

XAxisID := 1; (*圆弧插补分轴 X*)

YAxisID := 2; (*圆弧插补分轴 Y*)

CAxisID := 3; (*切刀旋转轴 C*)

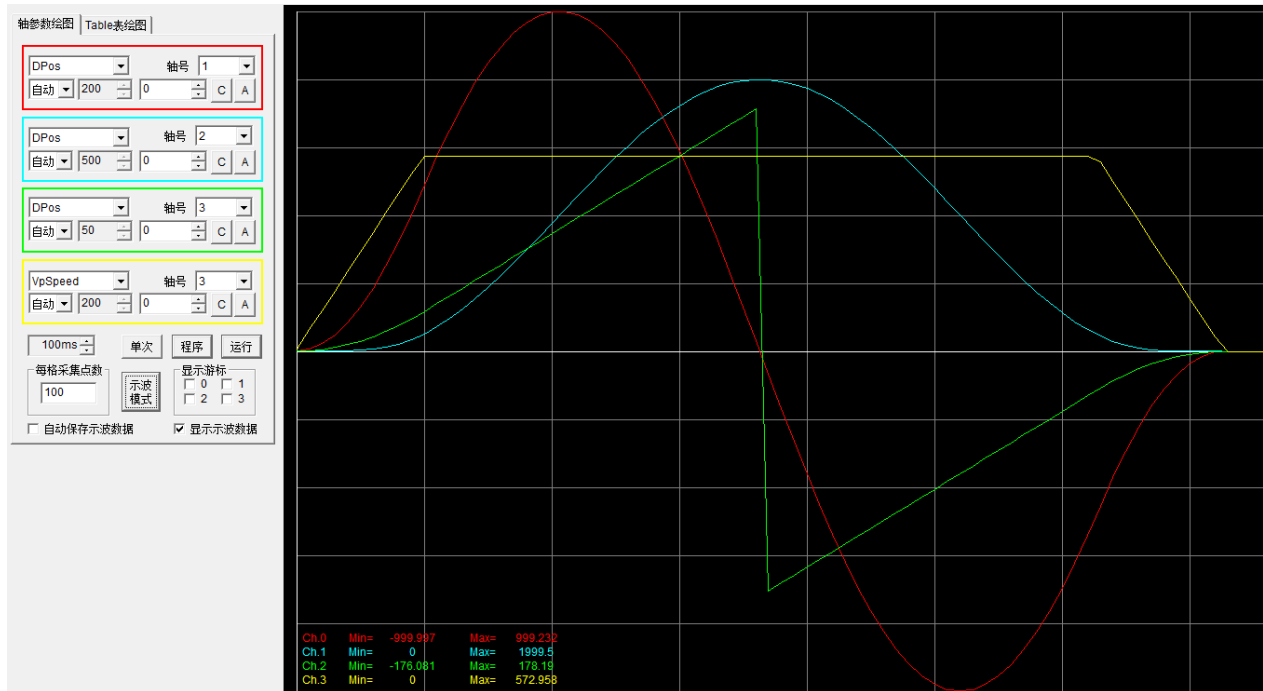
Axis3.RepMode := 2; (*设置切刀旋转轴为循环行程*)

Axis3.RepDist := 180; (*循环行程范围为-180~180，则 C 轴位置显示为旋转角度*)

HMC_MoveCircle(0,XAxisID,YAxisID,1,0,0,0,1000); (*X、Y 轴执行圆弧插补切割*)

HMC_MoveTang(CAxisID,XAxisID,YAxisID,0); (*C 轴跟随 X、Y 轴轨迹旋转，调整切刀角度*)

实际执行结果如下：



运动示意图

可以看出，当圆弧插补轨迹执行至 $1/4$ 圆弧时，切刀角度为 90° ，圆弧插补继续执行至 $1/2$ 圆弧时，切刀角度为 180° ，圆弧插补继续执行至 $3/4$ 圆弧时，切刀角度为 -90° ，圆弧插补执行结束整圆时，切刀角度为 0° 。整个插补过程中，切刀旋转角度实时跟随合运动轨迹与 X 轴正向夹角，及刀具方向与切割轨迹的切向方向（切割速度矢量方向）保持一致，达到最好的切割效果。

3.4.1.5.9.2 HMC_MoveTangConfigCornerMode（切向追踪拐角提刀设定）

函数原型

```
UDINT HMC_MoveTangConfigCornerMode (USINT ucAxisC,
USINT ucCornerMode,
LREAL dCornerAngelLimit,
USINT ucAxisZ,
LREAL dSafePosZ,
USINT ucSafePosZMode)
```

功能说明

当两条指令间衔接处具有角度的跃变时（如折线），可执行拐角提刀功能，以保护刀具，保证加工效果。

提供以下三种抬刀模式供用户选择，按照自动化程度由高到低分别为：自动抬刀模式、手动抬刀模式、不抬刀模式。

自动抬刀模式 ucCornerMode=2

1. 模式介绍

提前判断若指令之间衔接处的转折角大于预设的提刀角度阈值，则会自动执行如下抬刀动作流程：

等待插补主轴 XY、刀具升降轴 Z 停止—>Z 轴提升至安全高度—>C 轴旋转至下条指令的起始切向角—>Z 轴下降至原位置—>XYZ 轴恢复运行

支持任意插补指令之间的衔接。由于非插补类指令的起始切向角不可预判，若 XY 的指令队列中存在非插补类指令，则在该指令的衔接处按照不抬刀处理。

支持以下两种情形：

主轴 XY 为同一插补指令的两个分轴。典型应用为 XY 轴执行二维平面插补指令。

主轴 XY、刀具提升轴 Z 为同一插补指令的三个分轴。典型应用为 XYZ 轴执行三维空间插补指令。

为保证在满足拐角提刀的指令衔接处 XY 轴可靠停止，当 merge 开启时，需保证：

merge 停止角 $\leq$ 拐角提刀角度阈值

2. 配置流程

(1) 使用 HMC_MoveTang(ucAxisC, ucAxisX, ucAxisY, dOffset) 投送切向追踪指令；

(2) 使 用 HMC_MoveTangConfigCornerMode(ucAxisC, ucCornerMode, dCornerAngleLimit, ucAxisZ, dSafePosZ, ucSafePosZMode) 配置拐角抬刀模式为自动抬刀模式，同时配置拐角角度阈值、刀具升降轴 Z 轴轴号、Z 轴抬刀高度、Z 轴抬刀高度坐标模式（增量高度/绝对高度）；

(3) 往主轴 XY 中投送二维平面插补指令，或往轴 XYZ 中投送三维空间插补指令。

3. 补充说明

(1) HMC_MoveTang 指令运行过程中可调用 HMC_GetMoveTangCornerState(ucAxisC)获取实时运行状态（正常运行状态、拐角抬刀状态）；

(2) 若在拐角抬刀状态下使用 HMC_Cancel 撤销 HMC_MoveTang 指令，则指令会同时控制 C 轴、Z 轴按照其自身的减速度停止。

(3) 由于在拐角抬刀状态下 XYZ 指令队列中的指令处于暂停状态，出于安全考虑，撤销完成后并不自动恢复 XYZ 轴中指令队列中的指令运行，若需要恢复 XYZ 中指令队列中后续指令的运行，用户需调用 HMC_SetCmdBufferLoadMode(ucAxisID, ucCmdLoadMode)把指令加载模式设定为正常加载模式（0）；

(4) 若在拐角抬刀状态下 Z 轴遇到限位，则 Z 轴将以 MAX{decle、FastDecel}减速停止，出于安全考虑，HMC_MoveTang 自动切换至手动抬刀模式。

手动抬刀模式 ucCornerMode=1

1. 模式介绍

当指令之间衔接处的转折角大于预设的拐角提刀角度阈值时，XY 轴自动停止，等待用户在 IEC 程序中处理 Z 轴抬刀及 C 轴方向调整动作，抬刀动作流程完成后 XY 轴继续运行后续指令。手动模式下一个完整

的拐角抬刀的处理流程如下（黄色部分为用户 IEC 程序）：

MC 控制器：

检测到拐角抬刀条件满足，插补主轴 XY 停止等待。

IEC 周期任务中（用户编写程序）：

- (1) 调用 HMC_GetMoveTangCornerState 查询到 MoveTang 处于手动抬刀模式下的拐角抬刀等待状态；
- (2) 使用 HMC_Move 提升 Z 轴至安全高度；
- (3) 等待 Z 轴提升动作完成；
- (4) 调用 HMC_MoveTangCornerFollowNextCmd 旋转 C 轴至下条指令的起始切向角；
- (5) 等待 C 轴旋转动作完成；
- (6) 使用 HMC_Move 驱动 Z 轴下降至原位置；
- (7) 等待 Z 轴下降动作完成；
- (8) 调用 HMC_MoveTangCornerContinue 恢复 XY 轴运行后续指令。

支持任意插补指令之间的衔接。由于非插补类指令的起始切向角不可预判，若 XY 的指令队列中存在非插补类指令，则在该指令的衔接处按照不抬刀处理。

支持情形：主轴 XY 为同一插补指令的两个分轴。典型应用为 XY 轴执行二维平面插补指令。

为保证在满足拐角提刀的指令衔接处 XY 轴可靠停止，当 merge 开启时，需保证：

merge 停止角 $\leq$ 拐角提刀角度阈值

2. 配置流程

- (1) 使用 HMC_MoveTang (ucAxisC, ucAxisX, ucAxisY, dOffset) 投送切向追踪指令；
(2) 使用 HMC_MoveTangConfigCornerMode (ucAxisC, ucCornerMode, dCornerAngleLimit, ucAxisZ, dSafePosZ, ucSafePosZMode) 配置拐角抬刀模式为手动抬刀模式，同时配置拐角角度阈值；
- (3) 在一个周期性 IEC 任务中，调用 HMC_GetMoveTangCornerState(ucAxisC)反复查询切向追踪执行状态。若返回值为手动抬刀模式下的拐角停止状态（2），则执行以下 IEC 程序：
- (4) 使用 HMC_Move(ucAxisZ,dSafePos)驱动 Z 轴抬刀至安全平面；
- (5) 等待 Z 轴到达安全平面后，调用 HMC_MoveTangCornerFollowNextCmd(AxisC)，C 轴会运行至下条指令的起始切向角位置；
- (6) 等待 C 轴到达下条指令的起始切向角位置后，使用 HMC_Move(ucAxisZ, -dSafePos)驱动 Z 轴落刀至原位置；
- (7) 等待 Z 轴到达原位置后，调用 HMC_MoveTangCornerContinue (AxisC)恢复切向追踪主轴的运

行。

(8) 往主轴 XY 中投送插补指令：

3. 补充说明

- (1) 与自动抬刀模式不同，手动抬刀模式不支持 主轴 XY、刀具提升轴 Z 作为同一插补指令的三个分轴的情形；
- (2) 若在拐角停止状态下使用 HMC_Cancel 撤销 HMC_MoveTang 指令，由于在拐角抬刀状态下 XY 指令队列中的指令处于暂停状态，出于安全考虑，撤销完成后并不自动恢复 XY 轴中指令队列中的指令运行，若需要恢复 XY 中指令队列中后续指令的运行，用户需调用 HMC_SetCmdBufferLoadMode(ucAxisID, ucCmdLoadMode)把指令加载模式设定为正常加载模式（0）。

不抬刀模式 ucCornerMode=0

不作任何处理，拐角处插补轴 XY 不会停止等待。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号
ucCornerMode	USINT	拐角抬刀模式（0：不抬刀； 1：拐角手动抬刀； 2：拐角自动抬刀）
dCornerAngleLimit	LREAL	拐角抬刀角度阈值（单位：弧度）
ucAxisZ	USINT	拐角自动抬刀 Z 轴轴号 注：ucCornerMode 为 2 时生效。
dSafePosZ	LREAL	拐角自动抬刀 安全提刀高度。 注：ucCornerMode 为 2 时生效。
ucSafePosZMode	USINT	拐角自动提刀 安全提刀高度 坐标模式（0:增量坐标，1:绝对坐标）。 注：ucCornerMode 为 2 时生效。

返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

指令类型

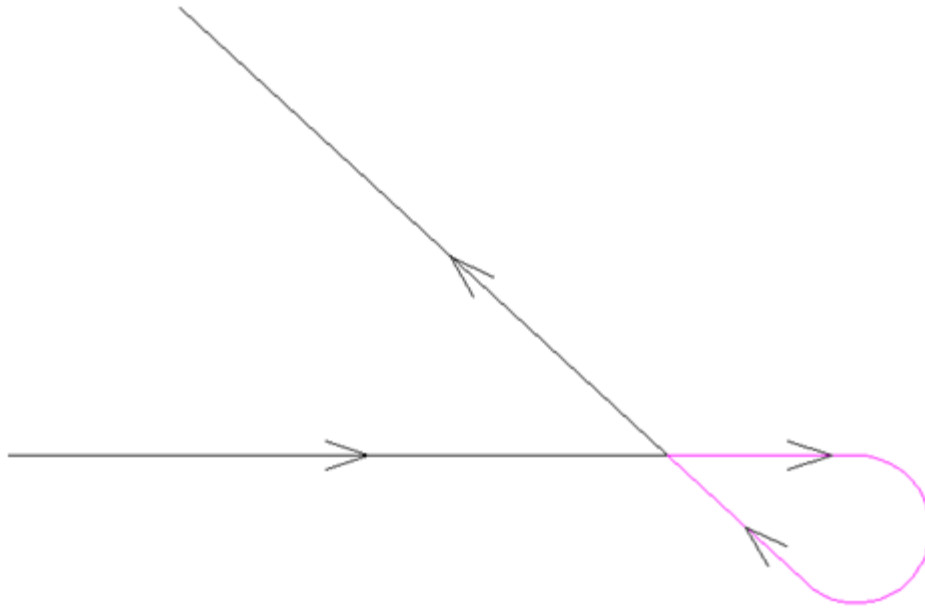
非轴运动缓存指令。

补充说明

如果拐角抬刀动作不能满足用户要求，用户可在拐角处添加工艺路径，以满足加工要求。

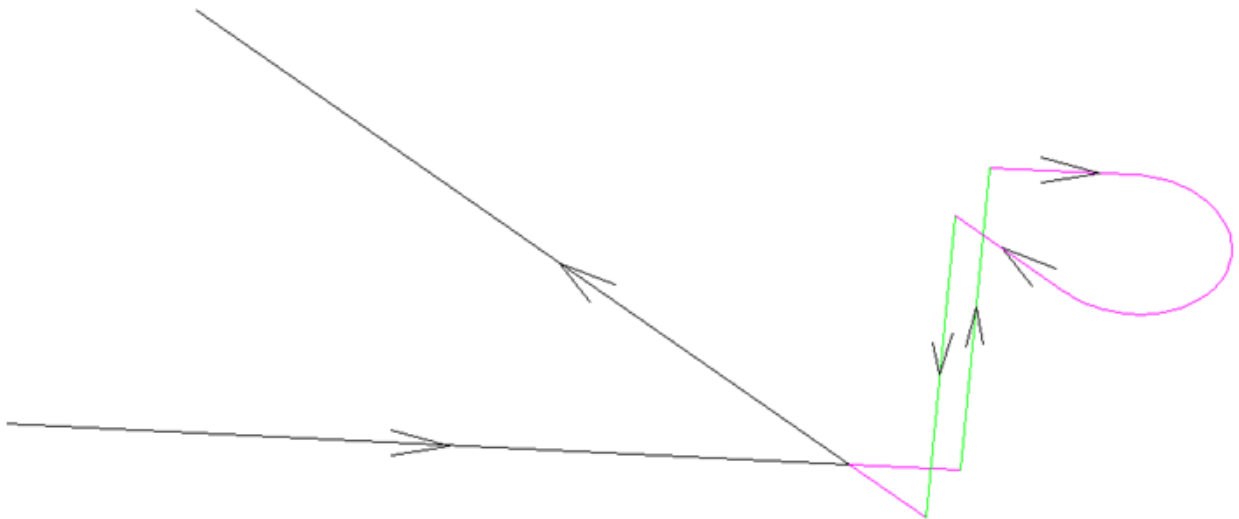
(1) 圆弧工艺路径

在工艺允许的情况下，可在拐角处加入圆弧辅助工艺路径，提高加工效率，避免频繁抬刀,同时亦可提高拐角处的切割质量。黑色折线为加工路径，紫色部分工艺路径。



圆弧工艺路径示意图

(2) 带抬刀的工艺路径



带抬刀的工艺路径示意图

如果用户希望在拐角处刀具的加工路线延出一些，以保证拐角处的切割质量，同时又希望尽可能的降低因工艺路径的引入而对切割原料造成的浪费，则可在（1）中工艺路径中插入 Z 向抬刀路径，如绿色部分。

应用举例

(1) 示例 1：自动抬刀

插补指令轨迹如下，在切割过程中为保护设备，需在指令拐角大于等于 $\pi/4$ 时抬刀。现在分别以自动抬刀和手动抬刀两种方式为例，实现抬刀保护功能。

投送 HMC_MoveTang 指令，为控制设备在拐角大于等于 $\pi/4$ 处自动抬刀，设定拐角抬刀的角度阈值为 $\pi/4 - 10^{-6}$ ， 10^{-6} 为安全余量，因程序中可能会存在浮点数小量偏差，余量大小用户可根据精度要求酌情选取。ST 语言程序如下，主轴 XY 插补轨迹如下图 1 所示，各阶段运动过程曲线如下图 2 所示。

```
AxisXY_ID:=4;(*插补合轴轴号*)
```

```
AxisX_ID:=5; (*X 轴轴号*)
```

```
AxisY_ID:=6; (*Y 轴轴号*)
```

```
AxisZ_ID:=7; (*Z 轴轴号*)
```

```
AxisC_ID:=8; (*C 轴轴号*)
```

```
tangentAngleOffset:=0; (*切向角偏移量*)
```

```
cornerMode :=2; (*拐角自动提刀模式*)
```

```
cornerAngleLimit := ATAN(1) - 1E-6; (*拐角抬刀的角度阈值为 45 度, 1E-6 为安全余量*)
```

```
safePosZ:= 4000; (*拐角抬刀安全高度*)
```

```
SafePosZMode:= 0; (*拐角抬刀安全高度为增量坐标*)
```

```
HMC_MoveTang(AxisC_ID, AxisX_ID, AxisY_ID, tangentAngleOffset); (*投送切向追踪指令*)
```

```
HMC_MoveTangConfigCornerMode(AxisC_ID, cornerMode, cornerAngleLimit, AxisZ_ID, safePosZ,  
SafePosZMode); (*配置拐角提刀模式*)
```

```
Axis4.Merge := 1; (*开启 merge*)
```

```
HMC_SetMergeAngle (AxisXY_ID, ASIN(0.5), cornerAngleLimit); (*配置 merge 减速角、停止角*)
```

```
(**向主轴投送插补指令（圆弧+Move2d）**)
```

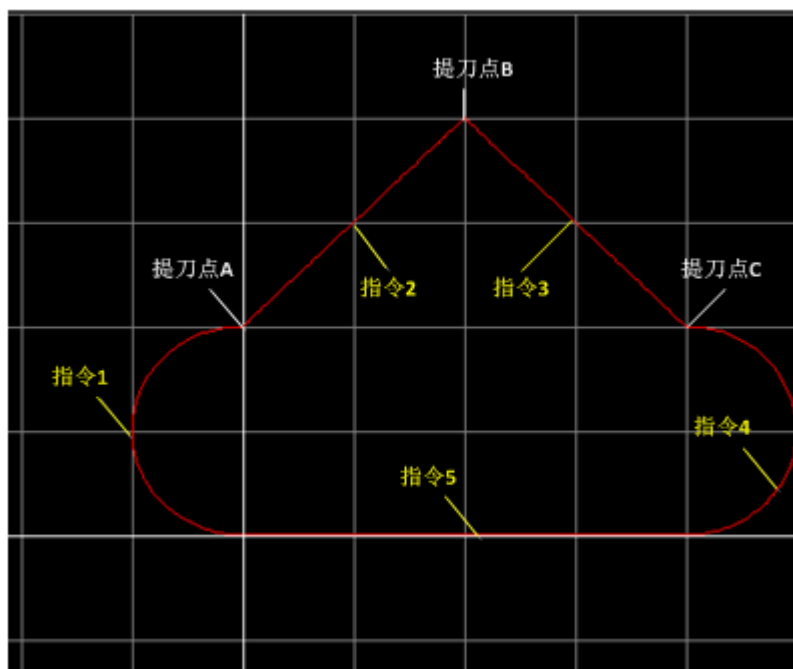
```
HMC_MoveCircle(AxisXY_ID, AxisX_ID, AxisY_ID,3,0,2000,0,1000);
```

```
HMC_Move2d(AxisXY_ID, AxisX_ID, AxisY_ID,2000,2000);
```

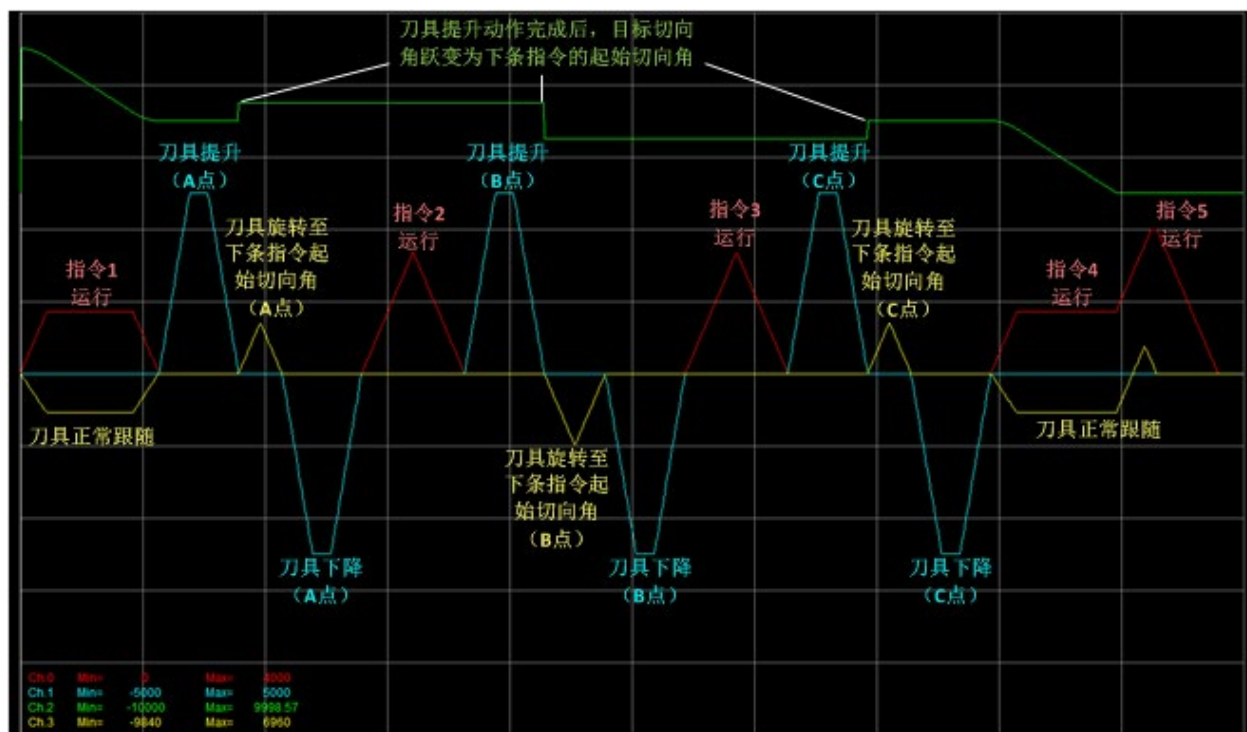
```
HMC_Move2d(AxisXY_ID, AxisX_ID, AxisY_ID,2000,-2000);
```

```
HMC_MoveCircle(AxisXY_ID, AxisX_ID, AxisY_ID,3,0,-2000,0,-1000);
```

```
HMC_Move2d(AxisXY_ID, AxisX_ID, AxisY_ID,-4000,0);
```



切向追踪主轴 XY 插补轨迹



切向追踪自动提刀模式运动过程曲线（主轴 XY 执行 圆弧+Move2d）

绿色：目标切向角

红色：XY 插补合轴 MpSpeed 曲线

黄色：刀具旋转轴 C 轴 MpSpeed 曲线

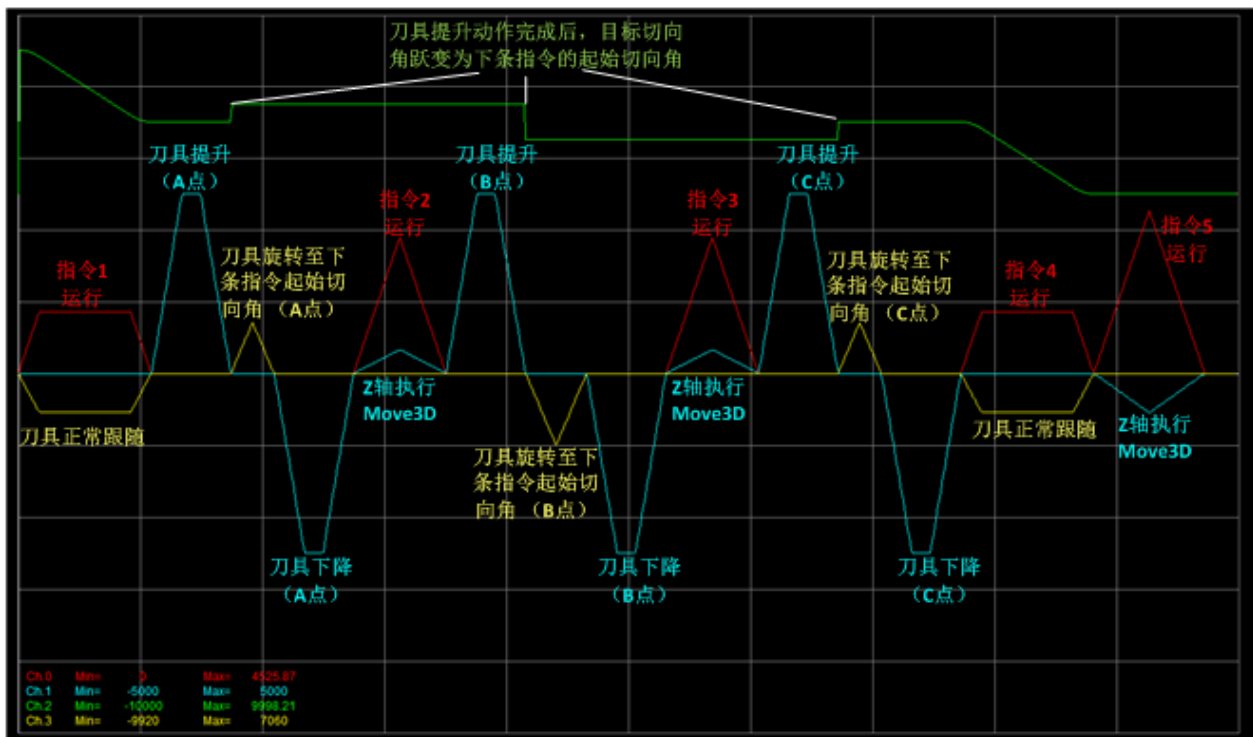
青色：刀具升降轴 Z 轴 MpSpeed 曲线

从上图可以看出，指令衔接处，当目标切向角变化大于设定抬刀阈值时（前四条指令间拐角均大于抬刀阈值），插补 X、Y 轴会自动停止（红色曲线为插补合轴速度曲线），刀具提升轴 Z 轴开始提到至设定位置（青色曲线为刀具提升轴速度），然后切刀旋转轴旋转至下条指令的起始切向角，刀具提升轴 Z 轴再下降至原位，下条插补指令继续恢复执行。指令衔接处目标切向角变换小于设定抬刀阈值时（第四、五条指令间拐角为 0 度），则不进行抬刀动作，不停止插补轴运动。

若把上面的 move2d 指令用 Move3d 代替，即 XYZ 轴执行圆弧+3 轴插补指令，则执行效果如下图所示。由图可知 Z 轴作为 3 轴插补分轴的插补运动与 Z 轴的提刀动作分阶段运行，没有冲突。

(**向主轴投送插补指令(圆弧+Move3d)**)

```
HMC_MoveCircle(AxisXY_ID, AxisX_ID, AxisY_ID, 3, 0, 2000, 0, 1000);
HMC_Move3d(AxisXY_ID, AxisX_ID, AxisY_ID, AxisZ_ID, 2000, 2000, 500);
HMC_Move3d(AxisXY_ID, AxisX_ID, AxisY_ID, AxisZ_ID, 2000, -2000, 500);
HMC_MoveCircle(AxisXY_ID, AxisX_ID, AxisY_ID, 3, 0, -2000, 0, -1000);
HMC_Move3d(AxisXY_ID, AxisX_ID, AxisY_ID, AxisZ_ID, -4000, 0, -1000);
```



切向追踪自动提刀模式运动过程曲线 (XYZ 执行 圆弧+Move3d)

绿色：目标切向角

红色：XYZ 插补合轴 MpSpeed 曲线

黄色：刀具旋转轴 C 轴 MpSpeed 曲线

青色：刀具升降轴 Z 轴 MpSpeed 曲线

(2) 示例 2：手动抬刀

XY 插补路径同 (1)。往 C 轴中投送 HMC_MoveTang 指令, 设定为手动抬刀模式, 其它参数同 (1) 中设置。ST 程序如下。主轴 XY 插补轨迹如下图 1 所示, 各阶段运动过程曲线如下图 2 所示。由图可知与 (1) 中自动提刀模式下的结果完全相同。

任务 1 程序:

AxisXY_ID:=4;(*插补合轴轴号*)

AxisX_ID:=5;(*X 轴轴号*)

AxisY_ID:=6;(*Y 轴轴号*)

AxisZ_ID:=7;(*Z 轴轴号*)

AxisC_ID:=8;(*C 轴轴号*)

tangentAngleOffset:=0;(*切向角偏移量*)

cornerMode :=1;(*拐角手动提刀模式*)

cornerAngleLimit := ATAN(1) - 1E-6;(*拐角抬刀的角度阈值为 45 度, 1E-6 为安全余量*)

HMC_MoveTang(AxisC_ID, AxisX_ID, AxisY_ID, tangentAngleOffset);(*投送切向追踪指令*)

HMC_MoveTangConfigCornerMode(AxisC_ID, cornerMode, cornerAngleLimit, 0, 0, 0);(*配置拐角提刀模式*)

Axis4.Merge := 1;(*开启 merge*)

HMC_SetMergeAngle (AxisXY_ID, ASIN(0.5), cornerAngleLimit);(*配置 merge 减速角、停止角*)

(**向主轴投送插补指令**)

HMC_movecircle(AxisXY_ID, AxisX_ID, AxisY_ID,3,0,2000,0,1000);

HMC_move2d(AxisXY_ID, AxisX_ID, AxisY_ID,2000,2000);

HMC_move2d(AxisXY_ID, AxisX_ID, AxisY_ID,2000,-2000);

HMC_movecircle(AxisXY_ID, AxisX_ID, AxisY_ID,3,0,-2000,0,-1000);

HMC_move2d(AxisXY_ID, AxisX_ID, AxisY_ID,-4000,0);

任务 2 程序 (周期任务):

ManualModeState_Pause:=2;(*拐角抬刀等待状态*)

safePosZ:= 4000;(*拐角抬刀安全高度*)

IF (HMC_GetMoveTangCornerState(AxisC_ID) = ManualModeState_Pause) THEN

(*主轴 XY 处于拐角停止状态，等待提刀处理*)

```
CmdID_AxisZ := HMC_Move(AxisZ_ID,safePosZ);(*Z 轴提刀*)
```

```
HMC_WaitCmdFinish(CmdID_AxisZ);(*等待 Z 轴提刀完成*)
```

```
HMC_MoveTangCornerFollowNextCmd(AxisC_ID); (*C 轴运动至下一条指令的起  
始切向角方向*)
```

```
WHILE (HMC_GetMoveTangDestDirection(AxisC_ID) <> Axis8.dPos ) DO  
(*等待 C 轴动作完成*)
```

```
;
```

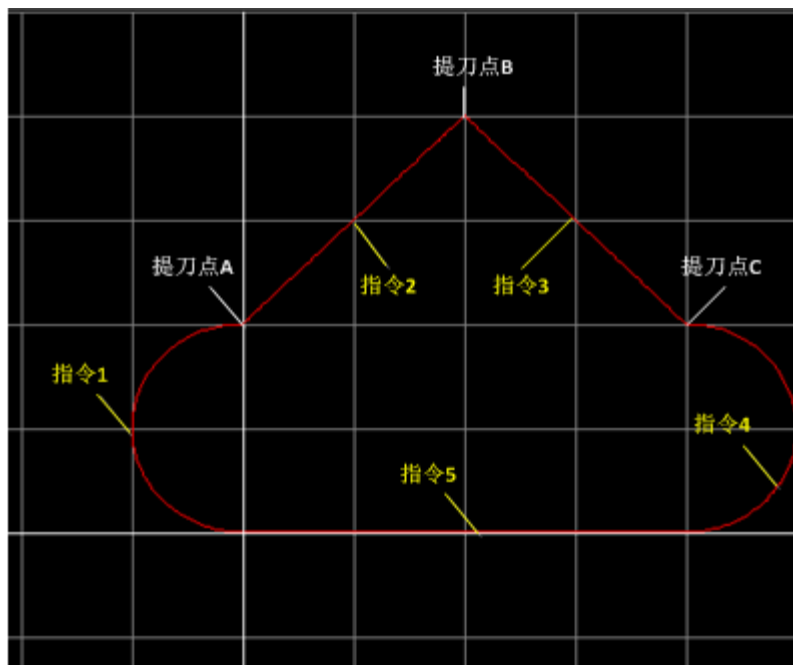
```
END_WHILE
```

```
CmdID_AxisZ := HMC_Move(AxisZ_ID,-safePosZ);(*Z 轴落刀*)
```

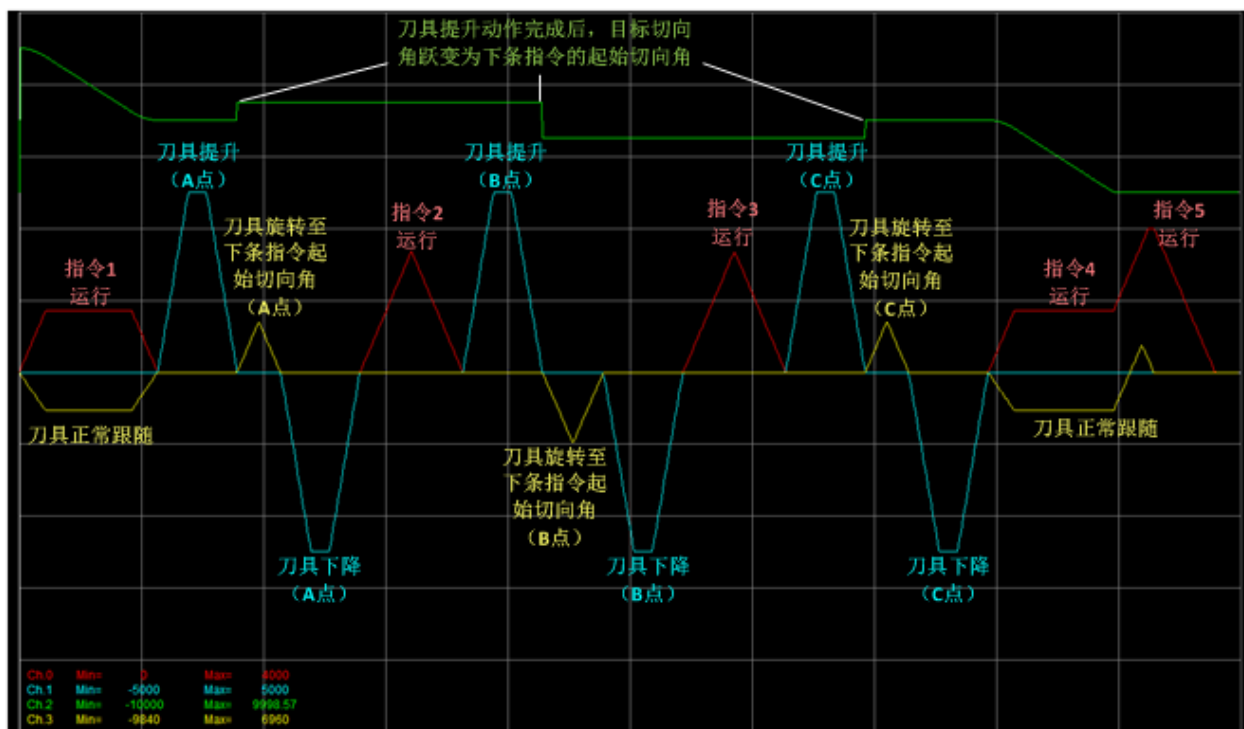
```
HMC_WaitCmdFinish(CmdID_AxisZ);(*Z 轴落刀完成*)
```

```
HMC_MoveTangCornerContinue(AxisC_ID); (*拐角提刀动作全部完成，插补轴恢复  
运行*)
```

```
END_IF
```



切向追踪主轴 XY 插补轨迹



切向追踪手动提刀模式运动过程曲线（主轴 XY 执行 圆弧+Move2d）

绿色：目标切向角

红色：XY 插补合轴 MpSpeed 曲线

黄色：刀具旋转轴 C 轴 MpSpeed 曲线

青色：刀具升降轴 Z 轴 MpSpeed 曲线

3.4.1.5.9.3 HMC_GetMoveTangCornerState（获取切向追踪拐角抬刀状态）

函数原型

USINT HMC_GetMoveTangCornerState(USINT ucAxisC)

功能说明

获取切向追踪拐角抬刀状态。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号

返回值

返回值	类型	参数描述
拐角抬刀状态	USINT	0: 拐角不抬刀; 1: 拐角手动抬刀, 主轴 XY 正常运行状态; 2: 拐角手动抬刀, 主轴 XY 遇到拐角并暂停, 等待工程程序处理抬刀; 3: 拐角自动抬刀, 主轴 XY 正常运行状态; 4: 拐角自动抬刀, 主轴 XY 遇到拐角并暂停, 正在进行自动抬刀处理。 255: 错误

指令类型

非轴运动缓存指令

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

补充说明

当前指令不为切向追踪指令时, 返回错误。

3.4.1.5.9.4 HMC_MoveTangCornerFollowNextCmd（C 轴定位至下一条指令的起始切向角）

函数原型

UDINT HMC_MoveTangCornerFollowNextCmd (USINT ucAxisC)

功能说明

用于切向追踪拐角手动抬刀模式, 在拐角暂停等待抬刀的状态下, 使用该指令可驱动 C 轴定位至下一条指令的起始切向角。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

补充说明

若当前指令不为手动抬刀模式下的切向追踪指令，则返回错误。

3.4.1.5.9.5 HMC_MoveTangCornerContinue（切向追踪继续运行）

函数原型

UDINT HMC_MoveTangCornerContinue (USINT ucAxisC)

功能说明

用于切向追踪拐角手动抬刀模式，在拐角暂停等待抬刀的状态下，使用该指令可恢复切向追踪继续运行。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号

返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

补充说明

若当前指令不为手动抬刀模式下的切向追踪指令，则返回错误。

3.4.1.5.9.6 HMC_GetMoveTangDestDirection（获取切向追踪目标方向位置）

函数原型

LREAL HMC_GetMoveTangDestDirection (USINT ucAxisC)

功能说明

该指令获取切向追踪的目标切向角，单位为 C 轴（ucAxisC，刀具旋转轴）用户单位。该指令执行时，C 轴当前指令需是 HMC_MoveTang 指令。

此时 C 轴工作在循环行程模式下，模式为 2(对称区间)，因此该指令返回的目标切向角范围为[-Repdist, Repdist)，对应于 $[-\pi, \pi)$ 的切向角度。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号

返回值

返回值	类型	参数描述
目标切向角	LREAL	切向追踪指令的目标切向角。若该指令调用错误时返回 0，并报用户错误。

指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

3.4.1.5.9.7 HMC_GetMoveTangDestVelocity（获取切向追踪目标速度）

函数原型

LREAL HMC_GetMoveTangDestVelocity (USINT ucAxisC)

功能说明

该指令获取切向追踪目标切向角的变化速度，单位为 C 轴（ucAxisC，刀具旋转轴）用户单位/S。该指令执行时，C 轴当前指令需是 HMC_MoveTang 指令。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号

返回值

返回值	类型	参数描述
目标速度	LREAL	切向追踪指令的目标切向角的变化速度。函数错误时返回 0，并报用户错误

指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

3.4.1.5.9.8 HMC_GetMoveTangDeviation（获取切向追踪方向偏差）

函数原型

LREAL HMC_GetMoveTangDeviation (USINT ucAxisC)

功能说明

该指令获取切向追踪 C 轴（ucAxisC，刀具旋转轴）当前指令位置与目标切向角的偏差，单位为 C 轴用户单位。该指令执行时，C 轴当前指令需是 HMC_MoveTang 指令。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	刀具旋转轴轴号

返回值

返回值	类型	参数描述
方向偏差	LREAL	切向追踪指令当前指令位置与目标切向角的偏差。函数错误时返回 0，并报用户错误。

指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

3.4.1.5.9.9 HMC_GetMoveTangMasterAxisX（获取切向追踪主轴 X）

函数原型

INT HMC_GetMoveTangMasterAxisX(USINT ucAxisC)

功能说明

获取当前切向追踪指令的 X 轴。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	从轴轴号

返回值

返回值	类型	参数描述
轴号/错误	INT	X 轴轴号/错误返回-1

指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

3.4.1.5.9.10 HMC_GetMoveTangMasterAxisY（获取切向追踪主轴 Y）

函数原型

INT HMC_GetMoveTangMasterAxisY(USINT ucAxisC)

功能说明

获取当前切向追踪指令的 Y 轴。

输入参数

输入参数	类型	参数描述
ucAxisC	USINT	从轴轴号

返回值

返回值	类型	参数描述
轴号/错误	INT	Y 轴轴号/错误返回-1

指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、

3.4.1.5.10 PWM 激光控制

3.4.1.5.10.1 PWM 激光控制

近年来，激光切割应用已被广泛使用于各种加工，例如金属、非金属或复合材料等。**PWM** 类功能块可根据运动轨迹合速度实时调整激光输出的频率与占空比，达到精确控制激光密度的目的，从而实现高质量的切割效果。

提供 5 种控制模式：

(1) 手动控制

该模式下，用户设定的手动占空比、手动频率直接作为最终输出值。

(2) 固定频率变占空比

该模式下，输出的频率固定，由用户设定。根据关联轴的合速度实时调整输出占空比，可实现均匀切割。

(3) 固定脉宽变频率

该模式下，输出脉宽固定由用户设定，根据关联轴的合速度实时调整输出频率，可实现均匀切割。

(4) 固定占空比变频率

该模式下，输出占空比固定由用户设定，根据关联轴的合速度实时调整输出频率，可实现均匀切割。

(5) 随动控制占空比和频率

该模式下，根据关联轴的合速度实时调整输出占空比、频率。可实现均匀切割。

使用 **PWM** 功能时，请按以下步骤：

(1) 先使用 **HMC_PWMConfig** 配置基本参数。

(2) 根据控制模式调用对应的在线控制功能块，使得 **PWM** 进入输出状态。此时可在线调整 **PWM** 输出，也可通过 **Enable** 参数暂停 **PWM** 输出。

(3) 需要重新设置基本参数或关闭 **PWM** 输出时，使用 **HMC_PWMClose**。



状态转移图

可以看出中，通过 HMC_PWMConfig 使得状态机从初始状态进入蓝色的过渡状态，一旦进入蓝色状态，其控制模式确定，但此时 PWM 通道还不进行输出；再根据当前控制模式选择对应的控制函数使得状态机从过渡状态进入黄色输出状态，处于输出状态时。

(4) 还可通过 PWM 的全局变量在线实时调整 PWM 输出，或观察 PWM 状态及实时输出等参数，如下图所示。

序号	变量名	直接地址	变量说明	变量类型	初始值
0001	HMC_PWM0	%SD92952	HMC_PWM0	HMC_PWM	FALSE
0002	HMC_PWM1	%SD93096	HMC_PWM1	HMC_PWM	FALSE

HMC_PWM0					
序号	变量名	直接地址	变量说明	变量类型	初始值
0008	AxisSpeedSrc	%SB92969	速度计算源模式: 0-VpSpeed, 1-M...	USINT	0
0009	Enable	%SD92976	PWM输出使能: 0-禁止, 1-使能	USINT	0
0010	ManualFreq	%SD92984	手动输出频率 (单位: HZ)	LREAL	0
0011	ManualDut...	%SD92992	手动输出占空比 (取值范围0.0~1.0)	LREAL	0
0012	FixedFreq	%SD93000	设定频率值 (单位: HZ)	LREAL	0
0013	FixedPuls...	%SD93008	设定脉宽 (单位: 纳秒)	UDINT	0
0014	FixedDuty...	%SD93016	设定占空比 (取值范围0.0~1.0)	LREAL	0
0015	pFreqMap	%SD93024	自动随动控制模式时频率与速度的...	POINTER T...	0
0016	pDutyCycl...	%SD93028	自动随动控制模式时占空比与速度...	POINTER T...	0
0017	uiFreqTab...	%SD93032	频率与速度映射表的点个数	UDINT	0
0018	uiDutyCyc...	%SD93036	占空比与速度映射表的点个数	UDINT	0
0019	MachineState	%SD93044	当前状态机状态	USINT	0
0020	ParamStatus	%SB93045	参数检测状态: 0-无错误, else-...	USINT	0
0021	Freq	%SD93048	当前输出频率 (单位: HZ)	LREAL	0
0022	DutyCycle	%SD93056	当前输出占空比 (取值范围0.0~1.0)	LREAL	0
0023	Speed	%SD93064	关联轴合速度	LREAL	0

PWM 参数查看与调整

3.4.1.5.10.2 HMC_PWMConfig (PWM 配置)

函数原型

UDINT HMC_PWMConfig (BYTE ucPWMChannel, BYTE ucControlMode, BYTE ucOutputMode, BYTE ucOutputID, POINTER TO BYTE pArrLinkAxis, BYTE ucLinkAxisNum, BYTE ucAxisSpeedSrc)

功能说明

对 PWM 功能进行离线配置。仅当状态机为初始状态时 (MachineState=0)，才可成功调用此功能块进行相关参数配置，其他状态下都不支持。所以一旦 PWM 成功调用 HMC_PWMConfig 退出初始状态后，该函数涉及的配置项则不允许修改，若需修改，必须先通过 HMC_PWMClose 关闭 PWM 通道后重新配置。

该功能块主要配置控制模式、输出方式和通道、随动控制时速度关联轴等信息。

输入参数

输入参数	类型	参数描述
ucPWMChnl	BYTE	PWM 通道号 (0~1)
ucControlMode	BYTE	控制模式 0: 手动 1: 固定频率变占空比 2: 固定脉宽变频率

输入参数	类型	参数描述
		3: 固定占空比变频率 4: 速度随动控制占空比和频率
ucOutputType	BYTE	输出方式 0: Z 相输出 1: DO 输出
ucOutputID	BYTE	DO 通道号或 Z 相输出轴号
pAryLinkAxis	POINTER TO BYTE	速度关联的轴
ucLinkAxisNum	BYTE	速度关联轴个数
ucAxisSpeedSrc	BYTE	速度来源 0: 运动指令计算速度 VpSpeed 1: 实际反馈速度 MpSpeed

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误 1499: 速度关联轴指针所指地址有超出 M、N 区 1553: 当前选择的输出轴的 Z 相已被另一路 PWM 占用 1555: 当前控制模式不为初始状态 0 1564: 当前选择 DO 输出通道已被其它功能占用（如 Switch、StepperDO 等）

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A01

3.4.1.5.10.3 HMC_PWMManual (PWM 手动控制)

函数原型

UDINT HMC_PWMManual (BOOL bEnable, BYTE ucPWMChnl, LREAL dManualFreq, LREAL dManualDutyCycle)

功能说明

设置 PWM 手动控制模式时相关控制参数。当 bEnable 为 0 时，PWM 输出停止；bEnable 为 1 时，直接输出用户设定的手动输出频率、手动输出占空比。

输入参数

输入参数	类型	参数描述
bEnable	BOOL	0-PWM 输出停止，1-使能 PWM 输出
ucPWMChnl	BYTE	PWM 通道号（0~1）
dManualFreq	LREAL	手动输出频率（单位 Hz）
dManualDutyCycle	LREAL	手动输出占空比（取值范围 0.0~1.0）

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误

返回值	类型	参数描述
		1555: 当前控制模式错误

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

3.4.1.5.10.4 HMC_PWMFixedFreq (PWM 固定频率变占空比控制)

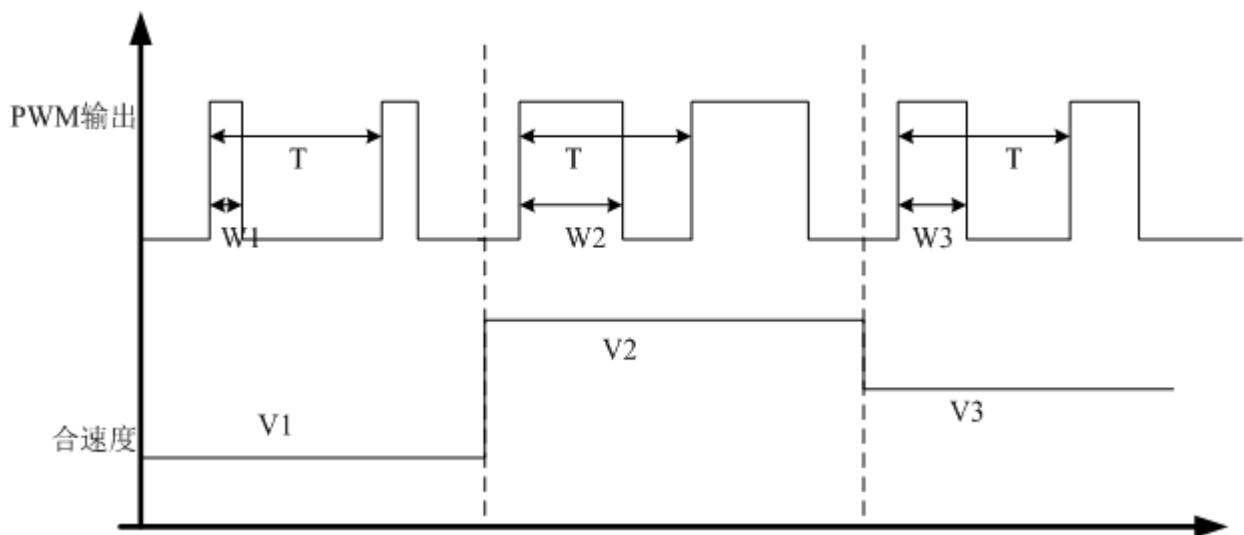
函数原型

UDINT HMC_PWMFixedFreq (BOOL bEnable, BYTE ucPWMChnl, LREAL dFixedFreq, POINTER TO LREAL pDutyCycleTable, UDINT uiDutyCycleTableNum)

功能说明

设置 PWM 固定频率随动调整占空比控制模式的相关控制参数。

当 bEnable 为 0 时，PWM 输出停止；bEnable 为 1 时，输出频率为用户设定的 dFixedFreq，根据速度关联轴的合速度查表计算得到输出占空比。



PWM 输出与速度示意图

输入参数

输入参数	类型	参数描述
bEnable	BOOL	0: PWM 输出停止 1: 使能 PWM 输出
ucPWMChnl	BYTE	PWM 通道号 (0~1)
dFixedFreq	LREAL	设定频率值 (单位 Hz)
pDutyCycleTable	POINTER TO LREAL	占空比与速度关系。在二维数组 pDutyCycleTable[2][uiNum]中，pMapTable[0]为占空比数组，pMapTable[1]为速度数组，两者一一对应，描述出占空比相对于

输入参数	类型	参数描述
		速度的变化曲线
uiDutyCycleTableNum	UDINT	占空比与速度数组的长度，即点个数

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误 1499: 占空比与速度映射表所占地址有超出 M、N 区 1555: 当前控制模式错误

指令类型

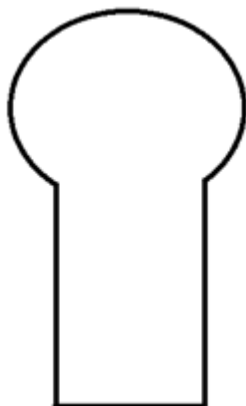
非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

应用举例

使用激光器切割金属材料：



切割形状示意图

在切割过程中，为了保证切割轨迹精准，在圆弧段和短直线段调整速度小一些，在两个长的直线段速度大一些，为保证切割效率，开启速度前瞻。因此整个切割过程中，合运动轨迹的速度会实时变化，指令连接处会出现加速、减速阶段。

由于整个切割过程中，合运动轨迹速度会变化，为了保证激光切割的均匀，激光输出功率应跟随合运动速度实时调整，以达到均匀切割得效果。如速度减速阶段，激光输出功率也应跟随减小，避免出现过融现象；当速度加速阶段，激光输出功率也应跟随调大，避免出现欠融现象。

程序如下：

AxisAry[0] := 0; (*合运动轴*)

(*占空比与速度随动调整数组*)

MapTable[0,0] := 0;

```
MapTable[0,1] := 0.2;
```

```
MapTable[0,2]:=0.6;
```

```
MapTable[0,3]:=0.8;
```

```
MapTable[0,4] := 1.0;
```

```
MapTable[1,0]:=0;
```

```
MapTable[1,1] := 1000;
```

```
MapTable[1,2]:=2000;
```

```
MapTable[1,3]:=3000;
```

```
MapTable[1,4] := 4000;
```

```
axis0.Accel:=3000;
```

```
axis0.Decel:=3000;
```

ConfigTemp := HMC_PWMConfig(0,1,1,0,ADR(AxisAry),1,0); (*先配置 PWM 基本参数，控制模式配置为固定频率调整占空比*)

ControlTemp := HMC_PWMFixedFreq (1 ,0,dFixFreq,ADR(MapTable),5); (*再通过实时控制函数，开启 PWM 输出，设置固定频率、占空比与速度关联关系*)

```
HMC_SetMerge (0, 1);
```

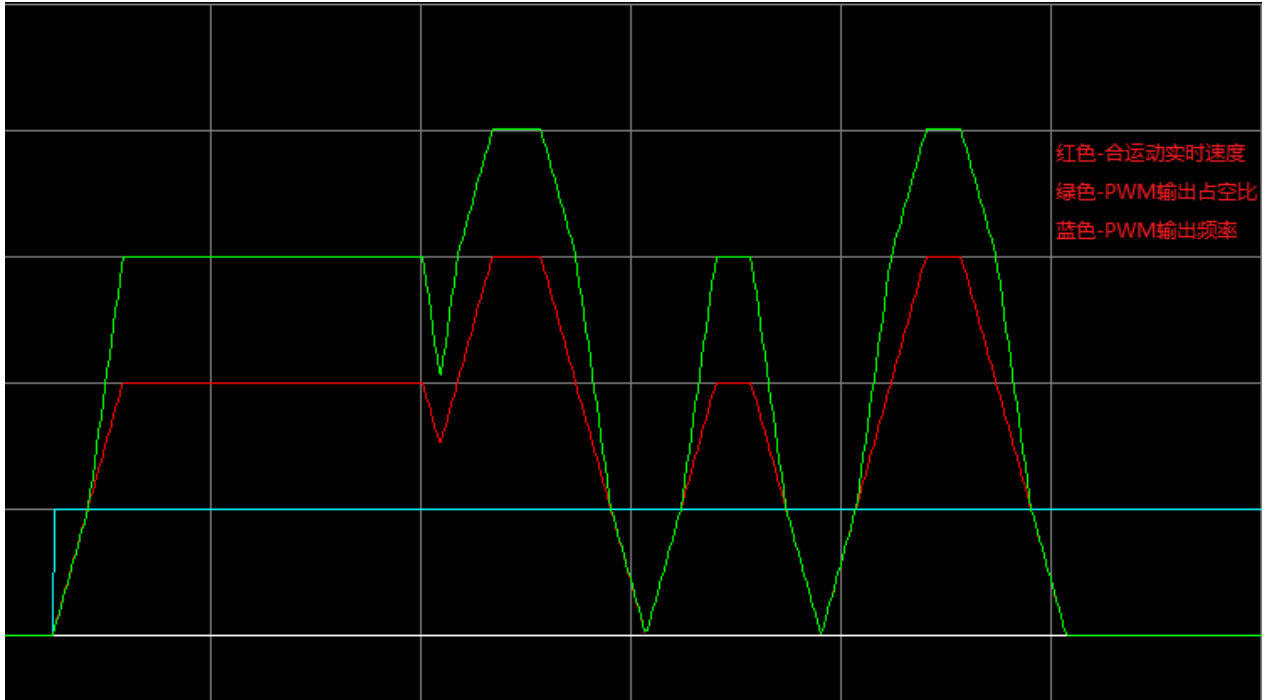
```
HMC_MoveCircleEx (0,1,2,3,2000,0,1000,1000,2000); (*切割圆弧段*)
```

```
HMC_Move2DEx (0,1,2,0,-4000,3000); (*切割长线段 1*)
```

```
HMC_Move2DEx (0,1,2,-2000,0,2000); (*切割短线段*)
```

```
HMC_Move2DEx (0,1,2,0,4000,3000); (*切割长线段 2*)
```

使用 AT 软件示波器记录合运动速度与 PWM 实时输出的占空比、频率关系：



运动轨迹示意图

可以看出，合运动轨迹（红色）在整个切割过程中的指令连接处出现加减速，在速度发生变化时，为了保证切割均匀，避免过融和欠融，需通过 PWM 调整输出占空比以控制激光器输出功率。在速度加速阶段，PWM 输出占空比也跟随增加；在速度减速阶段，PWM 输出占空比也跟随减小。

3.4.1.5.10.5 HMC_PWMFixedPulse（PWM 固定脉宽变频率控制）

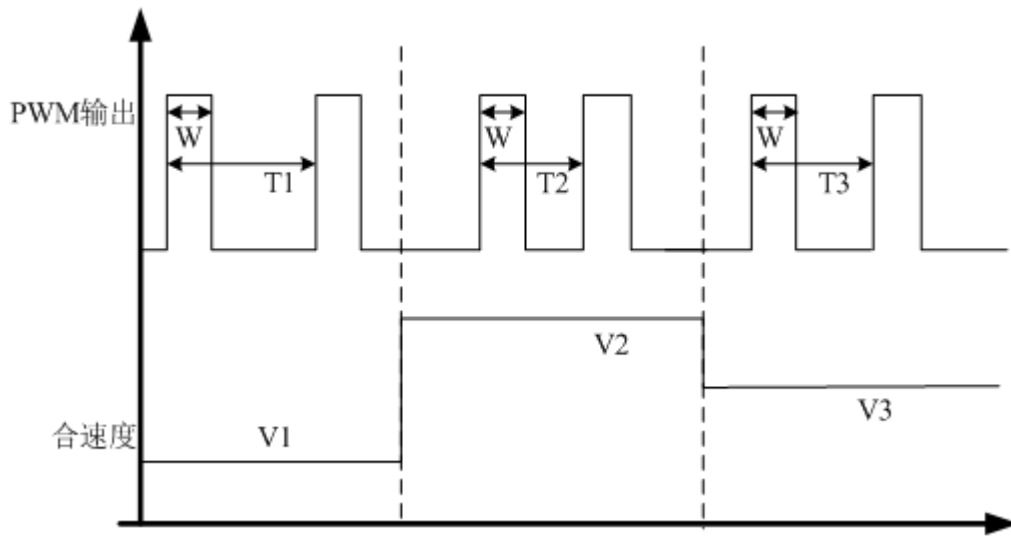
函数原型

UDINT HMC_PWMFixedPulse (BOOL bEnable, BYTE ucPWMChnl, UDINT uiFixedPulseWidth, POINTER TO LREAL pFreqTable, UDINT uiFreqTableNum)

功能说明

设置 PWM 固定脉宽随动调整频率控制模式的相关控制参数。

当 bEnable 为 0 时，PWM 输出停止；bEnable 为 1 时，输出脉宽为用户设定的 uiFixedPulseWidth，根据速度关联轴的合速度查表计算得到输出频率。



PWM 输出与速度示意图

输入参数

输入参数	类型	参数描述
bEnable	BOOL	0: PWM 输出停止 1: 使能 PWM 输出
ucPWMChnl	BYTE	PWM 通道号 (0~1)
uiFixedPulseWidth	UDINT	设定脉宽 (单位: ns)
pFreqTable	POINTER TO LREAL	频率与速度关系。在二维数组 pFreqTable[2][uiNum]中, pFreqTable[0]为频率比数组, pFreqTable[1]为速度数组, 两者一一对应, 描述出频率相对于速度的变化曲线
uiFreqTableNum	UDINT	频率与速度数组的长度, 即点个数

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误 1499: 频率与速度映射表所占地址有超出 M、N 区 1555: 当前控制模式错误

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

3.4.1.5.10.6 HMC_PWMFixedDutyCycle (PWM 固定占空比变频率控制)

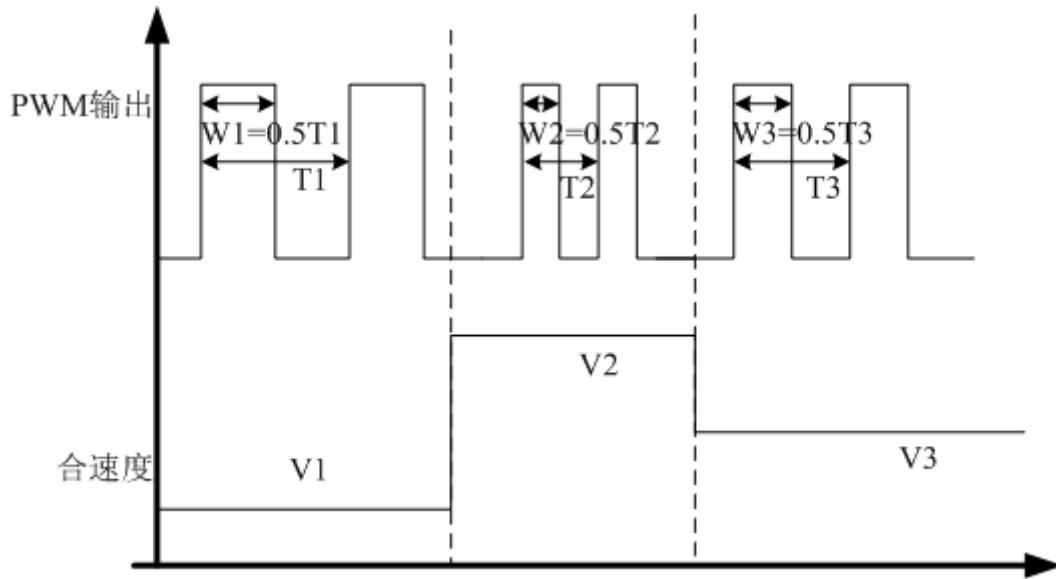
函数原型

UDINT HMC_PWMFixedDutyCycle (BOOL bEnable, BYTE ucPWMChnl, LREAL dFixedDutyCycle, POINTER TO LREAL pFreqTable, UDINT uiFreqTableNum)

功能说明

设置 PWM 固定占空比随动调整频率控制模式的相关控制参数。

当 bEnable 为 0 时，PWM 输出停止；bEnable 为 1 时，输出占空比为用户设定的 dFixedDutyCycle，根据速度关联轴的合速度查表计算得到输出频率。



PWM 输出与速度示意图

输入参数

输入参数	类型	参数描述
bEnable	BOOL	0: PWM 输出停止 1: 使能 PWM 输出
ucPWMChnl	BYTE	PWM 通道号 (0~1)
dFixedDutyCycle	LREAL	设定占空比 (取值范围 0.0~1.0)
pFreqTable	POINTER TO LREAL	频率与速度关系。在二维数组 pFreqTable[2][uiNum] 中，pFreqTable[0] 为频率数组，pFreqTable[1] 为速度数组，两者一一对应，描述出频率相对于速度的变化曲线
uiFreqTableNum	UDINT	频率与速度数组的长度，即点个数

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误 1499: 频率与速度映射表所占地址有超出 M、N 区 1555: 当前控制模式错误

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

3.4.1.5.10.7 HMC_PWMChangeFreqAndDutyCycle (PWM 根据速度调整频率和占空比控制模式)

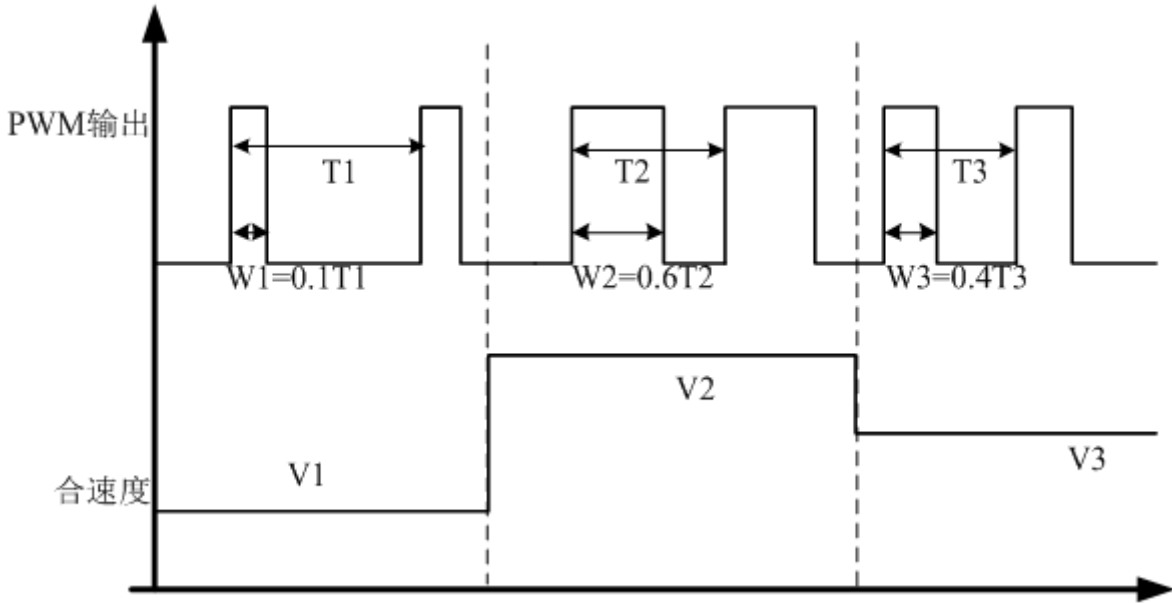
函数原型

UDINT HMC_PWMChangeFreqAndDutyCycle (BOOL bEnable, BYTE ucPWMChnl, POINTER TO LREAL pFreqTable, UDINT uiFreqTableNum, POINTER TO LREAL pDutyCycleTable, UDINT uiDutyCycleTableNum)

功能说明

设置 PWM 速度随动调整频率、占空比控制模式的相关控制参数。

当 bEnable 为 0 时，PWM 输出停止；bEnable 为 1 时，根据速度关联轴的合速度查频率速度映射表计算得到输出频率，根据速度关联轴的合速度查占空比速度映射表计算得到输出占空比。



PWM 输出与速度示意图

输入参数

输入参数	类型	参数描述
bEnable	BOOL	0: PWM 输出停止 1: 使能 PWM 输出
ucPWMChnl	BYTE	PWM 通道号 (0~1)
pFreqTable	POINTER TO LREAL	频率与速度关系。在二维数组 pFreqTable[2][uiNum]中，pFreqTable[0]为频率数组，pFreqTable[1]为速度数组，两者一一对应，描述出频率相对于速度的变化曲线
uiFreqTableNum	UDINT	频率与速度数组的长度，即点个数
pDutyCycleTable	POINTER TO LREAL	占空比与速度关系。在二维数组 pDutyCycleTable[2][uiNum]中，pMapTable [0]为占空比数组，pMapTable[1]为速度数组，两者一一对应，描述出占空比相对于速度的变化曲线
uiDutyCycleTableNum	UDINT	占空比与速度数组的长度，即点个数

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误 1499: 占空比与速度映射表，或频率与速度映射表所占地址有超出 M、N 区 1555: 当前控制模式错误

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

3.4.1.5.10.8 HMC_PWMSetDOCompensation (设置 DO 输出脉宽补偿值)

函数原型

UDINT HMC_PWMSetDOCompensation (BYTE ucPWMChnl, DINT iDOCompensation)

功能说明

当输出方式为 DO 输出时，可通过此功能块对输出计算的结果进行补偿，在原有计算的输出脉宽宽度基础上叠加补偿值。

输入参数

输入参数	类型	参数描述
ucPWMChnl	BYTE	PWM 通道号 (0~1)
iDOCompensation	DINT	DO 输出补偿值 (单位: ns)

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误 1555: 当前控制模式错误, 不为 1 或 3

指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

3.4.1.5.10.9 HMC_PWMClose (PWM 关闭)

函数原型

UDINT HMC_PWMClose (BYTE ucPWMChnl)

功能说明

关闭指定 PWM 通道。使得状态机复位到初始状态，输出停止。

输入参数

输入参数	类型	参数描述
ucPWMChnl	BYTE	PWM 通道号 (0~1)

输出参数

返回值	类型	参数描述
0	UDINT	成功
其它值	UDINT	0xffffffff: 函数参数错误

返回值	类型	参数描述
		1555: 当前控制模式错误

指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A001

3.4.1.5.11 多轴同步控制

3.4.1.5.11.1 HMC_GantrySynchro（龙门同步控制）

函数原型

UDINT HMC_GantrySynchro (USINT ucMasterAxis, USINT* pSlaveAxis, USINT ucSlaveAxisNum, LREAL lrPosWaring, LREAL lrPosErr)

功能说明

一个龙门组最多支持 8 个龙门轴，其中包括一个龙门主轴，其余轴作为从轴，即最多支持 7 个从轴。主轴运动缓存中不显示龙门同步指令。对主轴投送运动控制指令，对从轴投送龙门同步控制指令并指定主轴，龙门同步控制指令会对主轴和从轴进行动态同步。

龙门同步控制指令通过开环和闭环两种方式结合实现对一组龙门轴的动态同步控制：

- 开环同步：将龙门主轴解算的目标位置同步至各个龙门从轴，作为从轴的目标位置。从前端指令保证轴间位置同步；
- 闭环同步：动态监测各龙门轴的实际反馈位置值，通过轴间交叉耦合算法对各个龙门进行动态补偿，实现动态纠偏以消除各龙门轴轴间的偏差。闭环同步要求各龙门轴必须带有位置反馈值。
- 在动态同步过程中，还支持轴间实时偏差监测，设置两组阈值：
- 轴间偏差报警阈值：实时监测龙门组内两两轴间偏差，当发生偏差超出报警阈值时，在对应轴参 AxisStatus 的 bit5 置 1，当偏差恢复时，轴参 AxisStatus 的 bit5 自动恢复。
- 轴间偏差停车保护阈值：实时监测龙门组内两两轴间偏差，当发生偏差超出报警阈值时，在对应轴参 AxisStatus 的 bit6 置 1，并关闭伺服使能开关，无论 AxisErrMask 的 bit6 位是否为 1。当偏差恢复时，轴参 AxisStatus 的 bit6 不自动恢复，需手动清除。
- 可使用 HMC_Cancel 指令撤消龙门同步指令，轴号为从轴数组中任一轴号；
- 主轴轴号必须小于从轴数组中任一轴号；
- 从轴数组轴号必须从小到大排列。

输入参数

输入参数	类型	含义
UCMASTERAXIS	USINT	龙门组主动同步轴号（需小于任一从动轴号）

输入参数	类型	含义
PSLAVEAXIS	POINTER TO USINT	龙门组从动同步轴号数组
UCSLAVEAXISNUM	USINT	龙门组从动同步轴个数 (≤ 7)
UCMODE	USINT	龙门同步模式: 0: 轴间开环同步 1: 轴间闭环同步
LRPOSWARING	LREAL	轴间偏差报警阈值
LRPOSERR	LREAL	轴间偏差停车保护阈值

返回值

返回值	类型	含义
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

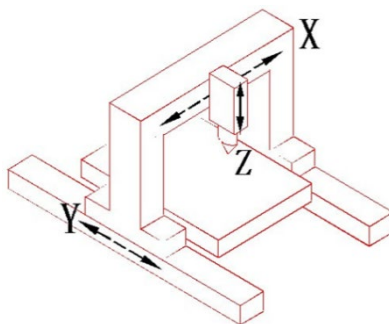
轴运动缓存指令。

补充说明

- 同一龙门组内，所有龙门轴类型必须一致；
- 当选择龙门同步模式为开环模式时，轴类型为带实际输出的轴类型即可，如直连步进轴、伺服轴、总线轴等；当选择为闭环模式时，轴类型为带实际输出且有实际反馈的位置闭环控制轴类型，即直连伺服轴、总线速度控制模式轴类型；
- 龙门轴的循环模式必须为有限行程，即 RepMode=0。

应用示例

如图结构，其中 Y 轴为龙门双驱，可以在工作平台执行一个圆弧轨迹。



结构示意图

ucMaster := 1; (*龙门主轴号*)

pSlaveAxisAry[0] := 2; (*龙门从轴号*)

ucSlaveNum := 1; (*一个龙门从轴*)

ucMode := 1; (*轴间闭环同步*)

lrPosWaring := 100;

lrPosErr := 500;

HMC_GantrySynchro (ucMaster,ADR(pSlaveAxisAry),ucSlaveNum,ucMode,lrPosWaring,lrPosErr); (*
投送龙门指令，使得轴 0 和轴 1 建立龙门关系*)

HMC_MoveCircle (10,0,1,1,0,0,100,100); (*轴 0 为 X 轴，轴 1 为 Y 轴，轴 2 与轴 1 为龙门关系*)

指令投送后如图所示：

Basic				
AxisID	USINT	0	1	2
AxisType	EmAxisType	Servo_A0	Servo_A0	Servo_A0
Units	LREAL	1	1	1
Status				
MCmdType	EmCmdType	HMC__MoveCircle	HMC__MoveCircle	HMC__GantrySynchro
NCmdType	EmCmdType	HMC__McIdle	HMC__McIdle	HMC__McIdle
AxisStatus	UDINT	0	0	0
AxisErrMask	UDINT	1	1	1

参数示意图

3.4.1.5.11.2 HMC_GantryInit（龙门同步回参考点位置对齐初始化）

函数原型

UDINT HMC_GantryInit (USINT* pAxis, USINT ucAxisNum, USINT ucMode, USINT ucHomeMode, USINT* pDiChl, USINT ucCaptureChl, LREAL dReferPointPos, LREAL dPosErr)

功能说明

龙门在开始运行时，须完成对位以及回原点的动作。对位需由安装在龙门各轴侧的对位检测器配合完成，对位检测器的机械位置安装必须准确，因为这是龙门可以校正平行对准的唯一地方。整个龙门初始化过程如下：

- 1. 各轴依次单独回到参考点，在一轴回参考点过程中，另一龙门轴同步跟随；
- 2. 所有轴完成回参考点后，同时对所有龙门轴进行位置重定义；
- 3. 计算轴间偏差，当偏差超出阈值时，指令结束；偏差在阈值范围内时，对各轴进行补偿移位，消除偏差实现对位。

龙门轴回参考点与回原点功能类似，模式支持 Home 低速和 HomeEx 高速两种方式；选择 HomeEx 高速时必须配合使用高速 DI。

输入参数

输入参数	类型	含义
PAXIS	POINTER TO USINT	龙门组同步轴号数组
UCAXISNUM	USINT	龙门组同步轴个数（≤8）
UCMODE	USINT	龙门同步回参考点模式： 0: Home 方式 1: HOMEEX 方式
UCHOMEMODE	USINT	龙门同步回参考点的信号源模式，可参见 Home（0~5）
PDICHL	POINTER TO USINT	回参考点使用的 DI 信号（不可重复）

输入参数	类型	含义
		Mode 为 0 时：DI 可为高速、低速、虚拟 DI Mode 为 1 时：DI 只能为高速 DI
UCCAPTURECHL	USINT	高速回参考点使用的高速捕捉通道号
DREFERPOINTPOS	LREAL	参考点绝对位置值
DPOSERR	LREAL	轴间偏差阈值

返回值

返回值	类型	含义
轴指令 ID	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

轴运动缓存指令。

补充说明

- 龙门轴回参考点使用的原点信号源不可重复；
- 同一龙门组内，所有龙门轴类型必须一致；
- 当选择龙门同步模式为开环模式时，轴类型为带实际输出的轴类型即可，如直连步进轴、伺服轴、总线轴等；当选择为闭环模式时，轴类型为带实际输出且有实际反馈的位置闭环控制轴类型，即直连伺服轴、总线速度控制模式轴类型；
- 龙门轴的循环模式必须为有限行程，即 RepMode=0。
- 可使用 HMC_Cancel 指令撤消龙门初始化指令，轴号必须为轴数组中最小轴号；
- 轴数组轴号顺序无要求。

应用示例

龙门双驱结构停机重启后，两龙门轴处于自然连接状态，存在偏差。此时可先通过龙门初始化对位指令，使得两轴依次回到对位参考点，对轴位置进行初始化，以此获取龙门轴间偏差后，对两轴消除偏差，完成对龙门轴的位置初始化以及对齐。

在此之后再对两轴投送龙门同步控制指令，建立龙门关联关系；之后对龙门主轴投送运动控制指令，实现龙门轴动态同步跟随控制。

```

pAxisAry[0] := 1;
pAxisAry[1] := 2;
ucAxisNum := 2;
ucMode := 1;
ucHomeMode := 0;
pDIChI[0] := 0;
pDIChI[1] := 1;
ucCaptureChI := 0;
dReferPointPos := -1000;

```

dPosErr := 10;

HMC_GantryInit (ADR(pAxisAry) , ucAxisNum, ucMode, ucHomeMode, ADR(pDIChI) , ucCaptureChI, dReferPointPos, dPosErr);

3.4.1.5.11.3 HMC_SetGantryAxisPID (设置龙门同步轴补偿 PID 参数)

函数原型

UDINT HMC_SetGantryAxisPID (USINT ucAxisID, LREAL dKpGain, LREAL dKiGain, LREAL dKdGain)

功能说明

配合龙门同步控制指令 HMC_GantrySynchro 使用。设置龙门组内指定龙门轴的动态纠偏 PID 参数。

若指定轴号当前不为龙门同步指令，则设置不成功。

输入参数

输入参数	类型	含义
UCAXISID	USINT	龙门组轴号
DKPGAIN	LREAL	动态补偿 PID 比例系数（默认值为 1）
DKIGAIN	LREAL	动态补偿 PID 积分系数（默认值为 0）
DKDGAIN	LREAL	动态补偿 PID 微分系数（默认值为 0）

返回值

返回值	类型	含义
0	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

补充说明

不调用此指令设置 PID 时，各轴动态补偿 PID 为默认值。

3.4.1.5.11.4 HMC_GetGantryAxisPID (读取龙门同步轴补偿 PID 参数)

函数原型

UDINT HMC_GetGantryAxisPID (USINT ucAxisID, LREAL dKpGain, LREAL dKiGain, LREAL dKdGain)

功能说明

配合龙门同步控制指令 HMC_GantrySynchro 使用。读取龙门组内指定龙门轴的动态纠偏 PID 参数。

输入参数

输入参数	类型	含义
UCAXISID	USINT	龙门组轴号

输入输出参数

输入输出参数	类型	含义
DKPGAIN	LREAL	动态补偿 PID 比例系数

输入输出参数	类型	含义
DKIGAIN	LREAL	动态补偿 PID 积分系数
DKDGAIN	LREAL	动态补偿 PID 微分系数

返回值

返回值	类型	含义
0	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

补充说明

轴号不为龙门轴时，返回错误。

3.4.1.5.11.5 HMC_GantryGroupDefPos（设置龙门同步轴组当前位置）

函数原型

UDINT HMC_GantryGroupDefPos (USINT* pAxis, USINT ucAxisNum, LREAL dMpos)

功能说明

配合龙门同步控制指令 HMC_GantrySynchro 使用。对龙门组内所有龙门轴的位置重定义。

输入参数

输入参数	类型	含义
PAXIS	POINTER TO USINT	龙门组同步轴
UCAXISNUM	USINT	龙门组同步轴个数（≤8）
DMPOS	LREAL	预定义位置

返回值

返回值	类型	含义
0	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

补充说明

当前指令不为龙门同步指令时，返回错误。

3.4.1.5.11.6 HMC_GetGantryInitState（读取龙门同步控制函数当前状态）

函数原型

UDINT HMC_GetGantryInitState (USINT* pAxis, USINT ucAxisNum)

功能说明

配合龙门同步会对位参考点初始化对齐指令 HMC_GantryInit 使用。读取该指令当前执行状态。

输入参数

输入参数	类型	含义
PAXIS	POINTER TO USINT	龙门组同步轴
UCAXISNUM	USINT	龙门组同步轴个数

返回值

返回值	类型	含义
1: Homing 2: ReturnBack 3: Defpos 4: Compsate 5: Finish	UDINT	龙门初始化指令执行状态
0xFFFFFFFF	UDINT	函数参数错误

指令类型

非轴运动缓存指令。

3.4.1.5.12 轴位置补偿功能

3.4.1.5.12.1 HMC_SetAxisCompenPos（轴位置补偿量设定）

函数原型

UDINT HMC_SetAxisCompenPos (USINT ucAxisID, LREAL dCompenVal)

功能说明

设定轴位置补偿量。

注意：补偿量的设定需放在 HMC_TrigAxisCompens 指令之前，才能在触发下次补偿动作时生效。该值的改变并不影响正在进行中的补偿动作。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dCompenPos	LREAL	位置补偿值

返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

指令类型

非轴运动缓存指令。

3.4.1.5.12.2 HMC_SetAxisCompenPara（轴位置补偿运动参数设定）

函数原型

UDINT HMC_SetAxisCompenPara (USINT ucAxisID, USINT ucJerkMode, LREAL dVmax, LREAL dAccel, LREAL dDecel, LREAL dJerk)

功能说明

设定轴位置补偿运动的运动学参数，实时生效。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

输入参数	类型	参数描述
dJerkMode	USINT	轴位置补偿动作采用的加减速类型： 0：梯形加减速 1：S 形加减速
dVmax	LREAL	位置补偿动作最大速度
dAccel	LREAL	位置补偿动作加速度
dDecel	LREAL	位置补偿动作减速度
dJerk	LREAL	S 形加减速加加速度

返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

指令类型

非轴运动缓存指令。

3.4.1.5.12.3 HMC_TrigAxisCompens（触发位置补偿）

函数原型

UDINT HMC_TrigAxisCompens (USINT ucAxisID)

功能说明

触发轴位置补偿动作。每运行一次该指令，都会使用当前设定的轴位置补偿量进行一次位置补偿动作。如果调用该指令时当前补偿动作尚未结束，则放弃剩余尚未完成的补偿，立即开始新的补偿动作。可用 HMC_GetAxisCompensState 指令查询轴的补偿状态，以确认当前补偿动作是否已经完成。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

指令类型

非轴运动缓存指令。

3.4.1.5.12.4 HMC_StopAxisCompens（停止位置补偿）

函数原型

UDINT HMC_StopAxisCompens (USINT ucAxisID)

功能说明

立即停止正在进行中的轴位置补偿动作。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

指令类型

非轴运动缓存指令。

3.4.1.5.12.5 HMC_GetAxisCompenState（获取轴位置补偿状态）

函数原型

UDINT HMC_GetAxisCompenState (USINT ucAxisID)

功能说明

获取轴补偿状态。

输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

返回值

返回值	类型	参数描述
0	UDINT	补偿动作执行完毕
1	UDINT	补偿动作进行中
0xFFFFFFFF	UDINT	轴参数错误

指令类型

非轴运动缓存指令。

3.4.1.6 机器人控制指令

3.4.1.6.1 Stewart 6 自由度并联机构

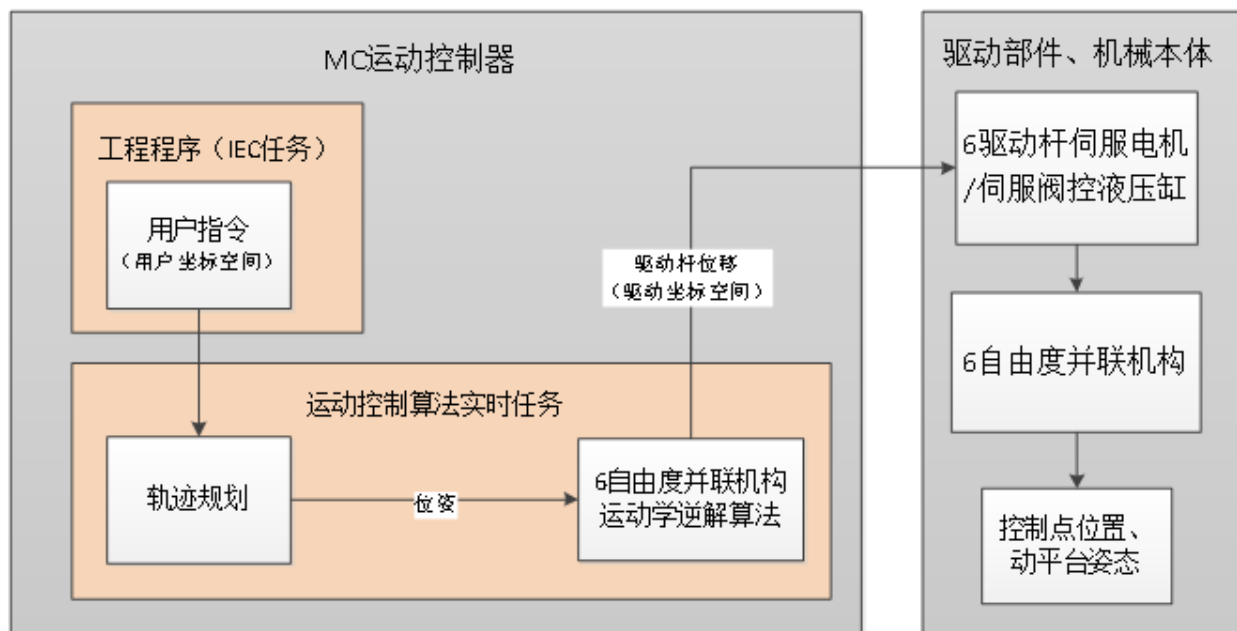
3.4.1.6.1.1 HMC_StewartControl（Stewart 机构控制）

函数原型

UDINT HMC_StewartControl (POINTER TO USINT UserSpaceAxisID, POINTER TO USINT DriveSpaceAxisID, POINTER TO LREAL StewartPara)

功能说明

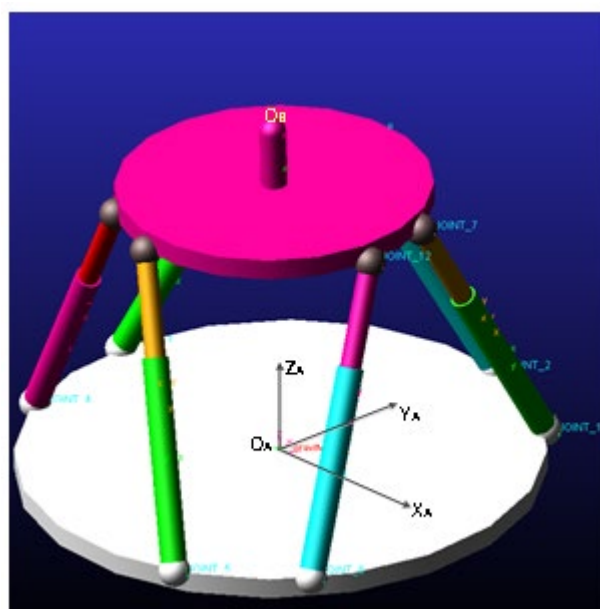
该指令完成 Gough-Stewart 6 自由度并联机构的运动学变换（由用户坐标空间到驱动坐标空间）。指令生效后，便可在用户坐标空间（直角坐标系）编写用户工程程序，控制器自动完成机构的运动学逆解变换，在驱动坐标空间内控制 6 个驱动杆运动。过程如图所示（有关 Gough-Stewart 平台的详细信息参见[补充说明](#)部分）。



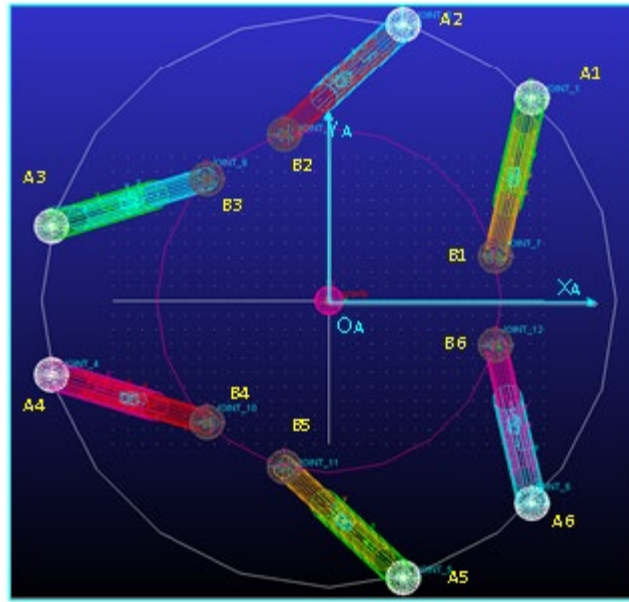
六自由度并联机构控制系统框图

姿态描述采用 RPY 角，即绕固定坐标系的旋转定义方式，旋转顺序为 XYZ。

在 Gough-Stewart 平台 6 个驱动杆回零操作完成，并且 6 个驱动杆实际杆长均等于 StewartPara 数组中的 l_{init} 参数时，调用该指令完成 6 自由度并联平台的初始化，配置 6 自由度并联机构的机构参数、驱动坐标空间的轴号（6 个）、用户坐标空间的轴号（6 个）。



(a)



(b)

并联六自由度平台坐标与机构参数示意图

如上图所示，目标控制点 O_B 位于通过动平台外接圆圆心并与该圆所在平面垂直的直线上，距离动平台平面的距离为 H 。

定义固定于静平台的绝对坐标系 $O_A-X_A Y_A Z_A$ 如下：

- (1) 坐标原点位于静平台铰点外接圆圆心 O_A ；
- (2) Z_A 轴垂直与静平台平面向上；
- (3) X_A 轴位于底面，且垂直下铰点 A_1 和 A_6 的连线；
- (4) Y_A 轴方向可由右手法则确定。

设 x 、 y 、 z 为目标控制点 O_B 在 $O_A-X_A Y_A Z_A$ 中的位置坐标， ϕ_x 、 ϕ_y 、 ϕ_z 为动平台在 $O_B-X_B Y_B Z_B$ 中的姿态坐标。则动平台在绝对坐标系 $O_A-X_A Y_A Z_A$ 内的位姿坐标为：

$$\mathbf{X} = [x, y, z, \phi_x, \phi_y, \phi_z]^T$$

调用该指令初始化完成后，用户坐标空间内的各轴坐标被自动初始化为初始位姿：

$$\mathbf{X}_{init} = [0, 0, z_{init}, 0, 0, 0]^T$$

驱动坐标空间的各轴坐标则被初始化为初始杆长参数 l_{init} 。

使用该指令时需注意以下几点：

(1) 在 6 个驱动杆实际杆长均等于 **StewartPara** 数组中的 l_{init} 参数时方可调用该函数，否则初始位姿会产生偏差；

(2) 可使用 **HMC_Cancel(AxisID, CancelMode)** 撤销该指令，撤销时传入的轴号必须为 6 个驱动轴中

的最小轴号；

(3) 指令调用成功后，首次被执行时会自动把用户坐标空间的各轴的位移设定为动平台在绝对坐标系 $O_A-X_A Y_A Z_A$ 中的位姿，把驱动坐标空间各轴的位移设定为初始杆长 l_{init} ，之后在此初始位姿基础上运动；

(4) 指令首次被执行时会把驱动坐标空间及用户坐标空间各轴的 **RepMode** 设定为有限行程，并根据驱动杆长参数 l_{min} 与 l_{max} 自动设定各驱动轴的软限位参数。在指令运行过程中不允许修改 **RepMode** 轴参，否则可能会发生坐标转换错误；

(5) 调用该指令初始化成功后，用户在绝对坐标系 $O_A-X_A Y_A Z_A$ 内编程，不允许调用 **HMC_DefPos** 或 **HMC_DefOrigin** 指令重新设定用户坐标空间或驱动坐标空间内的各轴坐标，否则坐标转换时会产生严重的偏差。如果用户需要使用相对用户坐标系，可在工程程序中自行进行坐标偏移换算；

(6) 该指令一旦调用成功，在用户坐标空间内不允许使用 **HOME** 指令，因 **HOME** 指令在回零完成之时会有 **DPOS** 和 **Mpos** 的跃变，从而造成坐标转换后驱动轴输出的跃变。

(7) 该并联机构在可达工作空间内存在奇异位形，在奇异位形附近机构力学性能较差，甚至失去承载能力。请根据机构参数，合理规划工作空间，避开奇异位形及机械干涉位形。

输入参数

输入参数	类型	参数描述
UserSpaceAxisID	POINTER TO USINT	用户坐标空间轴号数组，共 6 个元素，依次对应平移轴 XYZ 与旋转轴 $\phi_x \phi_y \phi_z$ 。单位为度 一般为虚拟轴，用户对该 6 个轴进行编程，对动平台位姿进行规划
DriveSpaceAxisID	POINTER TO USINT	驱动轴轴号数组，共 6 个元素，依次对应驱动杆 1~6。如图所示， $A_i B_i$ 对应编号为 i 的驱动杆 物理输出轴，驱动机构 6 个驱动杆运动
StewartPara	POINTER TO LREAL	Gough-Stewart 机构参数数组，共 8 个元素，依次为： 1、静平台铰点外接圆半径 R_0 2、静平台铰点 短边对应圆心角度 θ_0 ($0^\circ \leq \theta_0 < 60^\circ$) 3、动平台铰点外接圆半径 R_1 4、动平台铰点 短边对应圆心角度 θ_1 ($0^\circ \leq \theta_1 < 60^\circ$) 5、目标控制点距离动平台各铰点所在平面的距离 H (目标控制点位于通过动平台外接圆圆心并与该圆所在平面垂直的直线上)。 $H \geq 0$ 6、驱动杆初始杆长 l_{init} 7、驱动杆最短杆长 l_{min} 8、驱动杆最长杆长 l_{max}

 输入参数长度单位为用户单位，角度单位为角度制 ($^\circ$)。

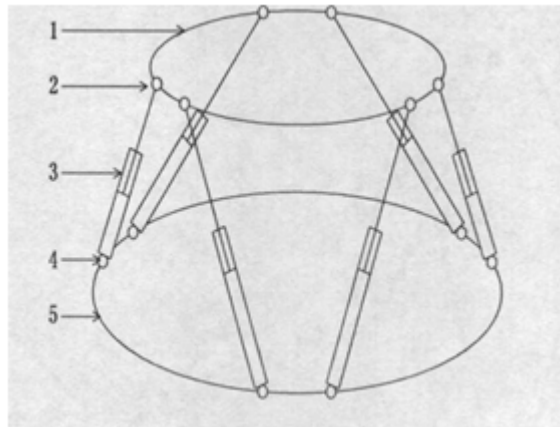
返回值

返回值	类型	参数描述
成功：指令 ID	UDINT	--
错误：16#FFFFFFFF		

指令类型

轴运动缓存指令。

补充说明



并联六自由度平台的机构原理

- 1: 动平台
- 2: 上球铰
- 3: 液压缸/电动缸
- 4: 下球铰
- 5: 静平台

六自由度并联平台的机构原理如上图所示，系统由动、静平台和 6 个伺服电动缸(或伺服阀控液压缸)组成，各支链电动缸与动、静平台分别由球铰连接（实际使用中为消除局部自由度，平台的上铰点通常使用虎克铰连接）。一般静平台固定于基座，通过控制支链电动缸的伸缩运动，动平台可以实现空间运动，完成空间六自由度的位置及姿态调整。并联机构本身的刚度大，当使用液压缸驱动时，特别适用于重载情况下对各种空间运动的模拟。

使用示例

任务一：投送 StewartControl 指令

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	UserSpaceAxisID		用户轴号数组	ARRAY[0..5] OF USINT		FALSE
0002	DriveSpaceAxisID		驱动轴号数组	ARRAY[0..5] OF USINT		FALSE
0003	StewartPara		机构参数	ARRAY[0..7] OF LREAL		FALSE
0004	Err		返回值	DWORD	0	FALSE

```

0001 (*投送StewartControl指令*)
0002 Err := HMC_StewartControl (ADR(UserSpaceAxisID),ADR(DriveSpaceAxisID),ADR(StewartPara));
0003

```

程序示例

1. 变量定义

用户轴号数组：（不允许存在轴号重复或与驱动轴号重复的情况）

Stewart_ST.UserSpaceAxisID

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	UserSpaceAxisID[0]		X轴_位置	USINT	10	FALSE
0002	UserSpaceAxisID[1]		Y轴_位置	USINT	11	FALSE
0003	UserSpaceAxisID[2]		Z轴_位置	USINT	12	FALSE
0004	UserSpaceAxisID[3]		X轴_姿态	USINT	13	FALSE
0005	UserSpaceAxisID[4]		Y轴_姿态	USINT	14	FALSE
0006	UserSpaceAxisID[5]		Z轴_姿态	USINT	15	FALSE

变量定义

驱动轴号数组：（不允许存在编码器轴类型或未使用轴类型）

Stewart_ST.DriveSpaceAxisID

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	DriveSpaceAxisID[0]		驱动杆1	USINT	0	FALSE
0002	DriveSpaceAxisID[1]		驱动杆2	USINT	1	FALSE
0003	DriveSpaceAxisID[2]		驱动杆3	USINT	2	FALSE
0004	DriveSpaceAxisID[3]		驱动杆4	USINT	3	FALSE
0005	DriveSpaceAxisID[4]		驱动杆5	USINT	4	FALSE
0006	DriveSpaceAxisID[5]		驱动杆6	USINT	5	FALSE

变量定义

机构参数：（当机构参数设置不合理时，指令投送不成功）

Stewart_ST.StewartPara

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	StewartPara[0]		动平台半径	LREAL	10000	FALSE
0002	StewartPara[1]		动平台短边角度	LREAL	30	FALSE
0003	StewartPara[2]		静平台半径	LREAL	20000	FALSE
0004	StewartPara[3]		静平台短边角度	LREAL	30	FALSE
0005	StewartPara[4]		控制点距离动平台高度	LREAL	1000	FALSE
0006	StewartPara[5]		初始杆长	LREAL	20000	FALSE
0007	StewartPara[6]		最小杆长	LREAL	10000	FALSE
0008	StewartPara[7]		最大杆长	LREAL	30000	FALSE

变量定义

任务二：用户轴中投送运动控制指令（单轴绝运动、6 轴绝对值直线插补(N 轴插补)、圆弧、球弧、样条等指令），驱动轴将会调整位置和姿态。

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Dis_x		X轴坐标位置	LREAL	1000	FALSE
0002	Dis_y		Y轴坐标位置	LREAL	2000	FALSE
0003	Dis_z		Z轴坐标位置	LREAL	3000	FALSE
0004	Ang_x		X轴姿态_角度	LREAL	10	FALSE
0005	Ang_y		Y轴姿态_角度	LREAL	-20	FALSE
0006	Ang_z		Z轴姿态_角度	LREAL	30	FALSE

```

0001 (*X轴位置调整*)
0002 HMC_MoveAbs(10,Dis_x);
0003 (*Y轴位置调整*)
0004 HMC_MoveAbs(11,Dis_y);
0005 (*Z轴位置调整*)
0006 HMC_MoveAbs(12,Dis_z);
0007 (*X轴姿态调整*)
0008 HMC_MoveAbs(13,Ang_x);
0009 (*Y轴姿态调整*)
0010 HMC_MoveAbs(14,Ang_y);
0011 (*Z轴姿态调整*)
0012 HMC_MoveAbs(15,Ang_z);
0013 (*也可投送球弧或样条指令，来规划用户轴的位置和姿态*)
0014

```

程序示例

任务三：撤销 StewartControl 指令（AxisID 必需是驱动轴号数组中的最小轴号）

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	AxisID		驱动轴数组第一个元素轴号	BYTE	0	FALSE

```

0001 (*撤销StewartControl指令*)
0002 HMC_Cancel (AxisID ,1);
0003

```

程序示例

3.4.1.6.2 5 自由度 SCARA 机械臂

3.4.1.6.2.1 HMC_Scara5Control（5 自由度 SCARA 机械臂控制）

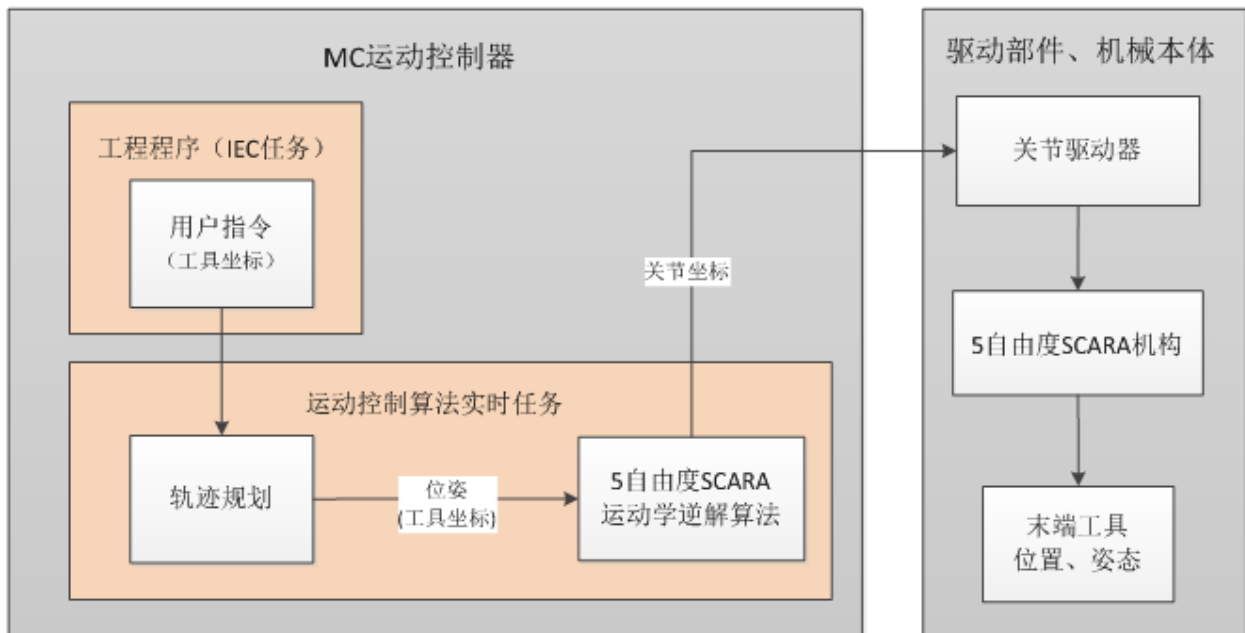
3.4.1.6.2.2 函数原型

UDINT HMC_Scara5Control(

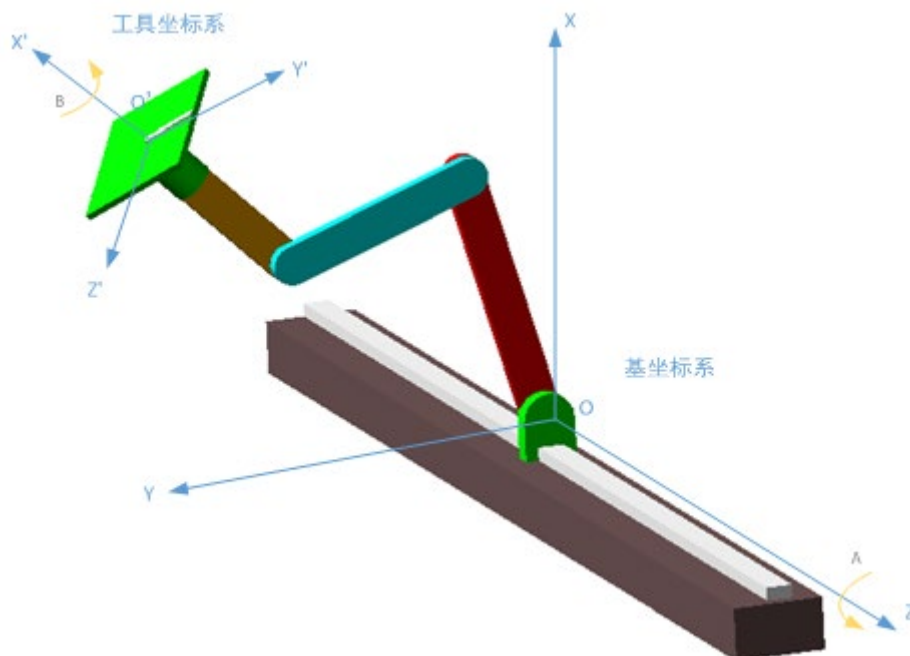
POINTER TO USINT arrBaseCoordAxisID,
 POINTER TO USINT arrJointCoordAxisID,
 POINTER TO LREAL arrScaraLinkLength)

3.4.1.6.2.3 功能说明

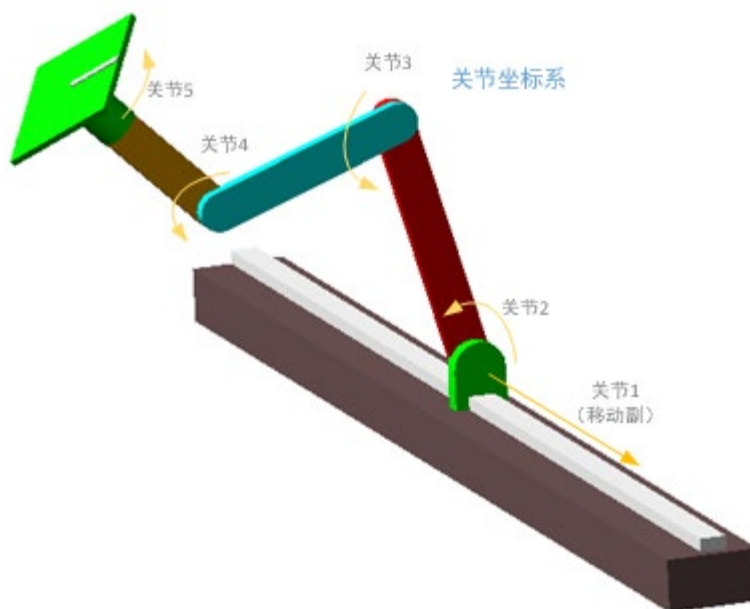
该指令完成 5 自由度 SCARA 机构的运动学变换（由基坐标系到关节坐标系）。指令生效后，便可在基坐标系（直角坐标系）中编写用户工程程序，控制器自动完成机构的运动学逆解变换，驱动各关节运动。过程如图所示。



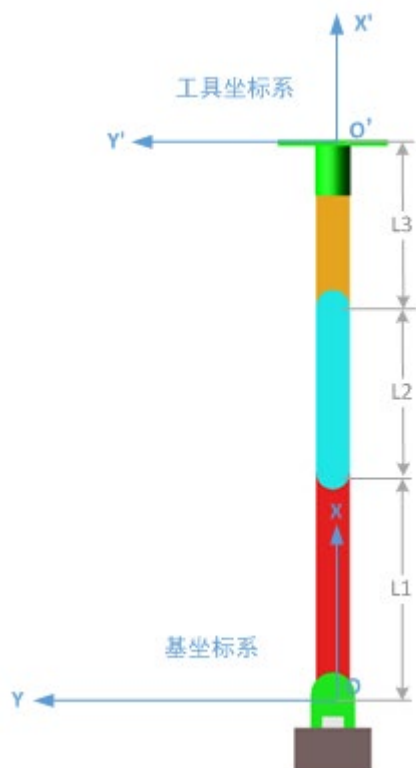
5 自由度 SCARA 串联机构控制系统框图



5 自由度 SCARA 机械手基坐标系、工具坐标系定义（笛卡尔坐标）



5 自由度 SCARA 机械手关节坐标系定义（广义坐标）



5 自由度 SCARA 机械手关节角零点位姿

定义基坐标系、工具坐标系、关节坐标系如图 [5 自由度 SCARA 机械手基坐标系 工具坐标系定义 笛卡尔坐标](#)、图 [5 自由度 SCARA 机械手关节坐标系定义 广义坐标](#) 所示，关节角零点位置如图 [5 自由度 SCARA 机械手关节角零点位姿](#) 所示。

定义工具坐标：工具坐标系 在 基坐标系 中的位置和姿态描述机构目标位姿，即机构的工具坐标。共 5 个坐标分量，即 $(x, y, z, \phi_A, \phi_B)$ 。

姿态描述采用欧拉角，即绕动坐标系的旋转定义方式，旋转顺序为 Z-X'，分别对应图 [5 自由度 SCARA 机械手基坐标系 工具坐标系定义 笛卡尔坐标](#) 中 A 轴、B 轴。

关节角的零点位置如图 [5 自由度 SCARA 机械手关节角零点位姿](#) 中所示。在回零操作完成后，只要零点位置未发生改变，则可在任意位置调用该指令。

使用该指令时需注意以下几点：

- (1) 长度单位为用户单位，角度单位为角度制 ($^{\circ}$)；
- (2) 务必保证关节角的零点位置如图 [5 自由度 SCARA 机械手关节角零点位姿](#) 中所示；
- (3) 可使用 `HMC_Cancel(AxisID, CancelMode)` 撤销该指令，撤销时传入的轴号必须为 5 个驱动轴中的最小轴号；
- (4) 指令调用成功后，首次被执行时，会根据 当前关节角 初始化 工具坐标，之后在此初始位姿基础上运动；

- (5) 指令首次被执行时会把驱动坐标空间及用户坐标空间各轴的 **RepMode** 设定为有限行程，用户可根据机构具体结构设定机构各关节角的软限位参数。在指令运行过程中不允许修改 **RepMode**，否则可能会发生坐标转换错误。
- (6) 可根据机构实际情况设定机构各关节角的限位值；
- (7) 调用该指令初始化成功后，用户在 基坐标系 内编程，不允许调用 **HMC_DefPos** 或 **HMC_DefOrigin** 指令重新设定基坐标系或关节坐标系内的各轴坐标值，否则坐标转换时会产生严重的偏差。如果用户需要使用相对坐标系，可在工程程序中自行进行坐标偏移换算；
- (8) 该指令运行时不允许对基坐标系中的轴使用 **HOME** 指令，因 **HOME** 指令在回零完成之时会有 **DPOS** 和 **Mpos** 的跃变，从而造成坐标转换后关节轴输出的跃变；
- (9) 机构存的可达工作空间是有限的，如果规划的路径超出了其可达工作空间，出于安全考虑，指令会被撤销。请根据机构参数，合理规划工作空间，确保工作空间位于机构可达工作空间之内（可用 **HMC_Scara5ReverseKinematics** 指令辅助校验规划的工具坐标是否合法）。

3.4.1.6.2.4 输入参数

输入参数	类型	参数描述
arrBaseCoordAxisID	POINTER TO USINT	基坐标系轴号数组，共 5 个元素，依次对应平移轴 X、Y、Z 与旋转轴 A、B，如图 5 自由度 SCARA 机械手基坐标系 工具坐标系定义 笛卡尔坐标 、图 5 自由度 SCARA 机械手关节坐标系定义 广义坐标 所示。单位为度 一般为虚拟轴，用户对该 5 个轴进行编程，对工具坐标系的位姿进行规划
arrJointCoordAxisID	POINTER TO USINT	关节坐标系各关节轴轴号数组，共 5 个元素，依次对应关节 1~5。如图 5 自由度 SCARA 机械手基坐标系 工具坐标系定义 笛卡尔坐标 、图 5 自由度 SCARA 机械手关节坐标系定义 广义坐标 所示 物理输出轴，驱动机构 5 个关节运动
arrScaraLinkLength	POINTER TO LREAL	连接杆杆长数组，共 3 个元素，依次对应图 5 自由度 SCARA 机械手关节角零点位姿 中 L1、L2、L3

3.4.1.6.2.5 返回值

返回值	类型	参数描述
成功：指令 ID 错误：16#FFFFFFFF	UDINT	--

3.4.1.6.2.6 指令类型

轴运动缓存指令。

3.4.1.6.2.7 使用示例

任务一：投送 **Scara5Control** 指令

序号△	变量名	直接地址	变量说明	变量类型
0001	Err0		返回值	DWORD
0002	UserAxis		用户基坐标轴数组	ARRAY[0..4] OF BYTE
0003	DriveAxis		关节驱动轴数组	ARRAY[0..4] OF BYTE
0004	LinkLen		杆长数组	ARRAY[0..2] OF LREAL

0001	(*投送Scara5Control指令*)			
0002	Err0 := HMC_Scara5Control (ADR(UserAxis),ADR(DriveAxis),ADR(LinkLen));			

1. 变量定义

用户基坐标轴号数组：（不允许存在轴号重复或与关节驱动轴号重复的情况）

Scara5Control.UserAxis					
序号	变量名	直接地址	变量说明	变量类型	初始值
0001	UserAxis[0]		用户基坐标X	BYTE	6
0002	UserAxis[1]		用户基坐标Y	BYTE	7
0003	UserAxis[2]		用户基坐标Z	BYTE	8
0004	UserAxis[3]		用户旋转坐标A	BYTE	9
0005	UserAxis[4]		用户旋转坐标B	BYTE	10

关节驱动轴号数组：（不允许存在编码器轴类型或未使用轴类型）

Scara5Control.DriveAxis					
序号	变量名	直接地址	变量说明	变量类型△	初始值
0001	DriveAxis[0]		关节1驱动轴	BYTE	1
0002	DriveAxis[1]		关节2驱动轴	BYTE	2
0003	DriveAxis[2]		关节3驱动轴	BYTE	3
0004	DriveAxis[3]		关节4驱动轴	BYTE	4
0005	DriveAxis[4]		关节5驱动轴	BYTE	5

杆长参数：（当杆长设置不合理时，指令投送不成功）

Scara5Control.LinkLen					
序号	变量名	直接地址	变量说明	变量类型	初始值
0001	LinkLen[0]		杆1长度	LREAL	10000
0002	LinkLen[1]		杆2长度	LREAL	11000
0003	LinkLen[2]		杆3长度	LREAL	12000

任务二：用户基坐标轴中投送运动控制指令(MoveAbsND)，给定用户基坐标系中的工具坐标。

序号	变量名	直接地址	变量说明	变量类型
0001	UserAxis		用户基坐标轴数组	ARRAY[0..4] OF BYTE
0002	ToolCoor		工具坐标	ARRAY[0..4] OF LREAL

```
0001 (*用户基坐标投送MoveAbsND指令，给定用户基坐标系中的工具坐标*)
0002 HMC_MoveAbsND (15,ADR(UserAxis),ADR(ToolCoor),5,0);
```

任务三：撤销 Scara5Control 指令（AxisID 必需是驱动轴号数组中的最小轴号）

序号	变量名	直接地址	变量说明	变量类型	初始值
0001	AxisID		驱动轴最小轴号	BYTE	0
0002	Modex		撤销模式	BYTE	1

```
0001 (*撤销指令*)
0002 HMC_Cancel(AxisID,Modex);
```

3.4.1.6.2.8 HMC_Scara5ForwardKinematics（5 自由度 SCARA 运动学正解）

函数原型

```
UDINT HMC_Scara5ForwardKinematics(
    POINTER TO LREAL arrScaraLinkLength,
    POINTER TO LREAL arrToolCoordinate,
    POINTER TO LREAL arrJointAngle)
```

功能说明

该指令完成 5 自由度 SCARA 机构的运动学正解（由关节坐标系到基坐标系）。

输入参数

输入参数	类型	参数描述
arrScaraLinkLength	POINTER TO LREAL	连接杆杆长数组，共 3 个元素，依此对应图 5 自由度 SCARA 机械手关节角零点位姿 中 L1、L2、L3
arrToolCoordinate	POINTER TO LREAL	工具坐标数组，共 5 个元素，保存正解结果
arrJointAngle	POINTER TO LREAL	关节坐标数组，共 5 个元素

返回值

返回值	类型	参数描述
成功: 0 错误: 16#FFFFFFFF	UDINT	--

指令类型

非轴运动缓存指令。

3.4.1.6.2.9 HMC_Scara5ReverseKinematics (5 自由度 SCARA 运动学逆解)

函数原型

```
UDINT HMC_Scara5ReverseKinematics(
    POINTER TO LREAL arrScaraLinkLength,
    POINTER TO LREAL arrToolCoordinate,
    POINTER TO LREAL arrReferenceJointAngle,
    POINTER TO LREAL arrJointAngle)
```

功能说明

该指令完成 5 自由度 SCARA 机构的运动学逆解（由基坐标系到关节坐标系）。当输入的工具坐标超出机构的可达工作空间时，返回错误。

该函数可用在如下方面：

- 检验关键点的工具坐标是否位于可达工作空间之内；
- 实现关节坐标空间编程。

关节坐标空间编程可避免直角坐标系下编程时关键点之间的运动轨迹可能穿过机构不可达工作空间的问题。

如果仅要求点到点之间的定位运动，对中间运动轨迹不作要求，则可在基坐标系中规划好关键位置点的工具坐标，然后使用该指令对关键点进行合法性检查，将合法的关键点的工具坐标转换为关节坐标，此后便可在关节坐标系内编程。

输入参数

输入参数	类型	参数描述
arrScaraLinkLength	POINTER TO LREAL	连接杆杆长数组，共 3 个元素，依此对应图 5 自由度 SCARA 机械手关节角零点姿态 中 L1、L2、L3
arrToolCoordinate	POINTER TO LREAL	工具坐标数组，共 5 个元素，即 $(x, y, z, \phi_A, \phi_B)$
arrReferenceJointAngle	POINTER TO LREAL	关节坐标数组（参照值），共 5 个元素，依次为关节 1~5 的旋转角度 由于逆解的结果不止一个，选择距离该参照位置 较近的一组解作为逆解结果
arrJointAngle	POINTER TO LREAL	关节坐标数组，共 5 个元素，保存逆解结果，依次为关节 1~5 的旋转角度

返回值

返回值	类型	参数描述
成功: 0 错误: 16#FFFFFFFF	UDINT	当用户输入参数错误 或 工具坐标超出机构的可达工作空间时，返回错误

指令类型

非轴运动缓存指令。

3.4.1.6.3 4 自由度串联机械臂

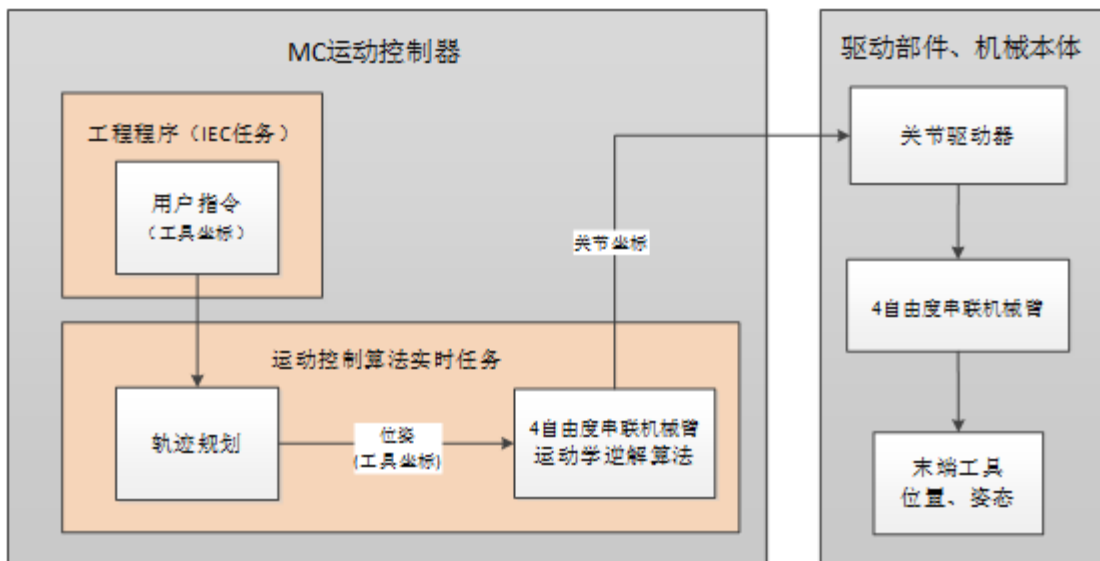
3.4.1.6.3.1 HMC_Serial4Control (4 自由度串联机械臂控制)

函数原型

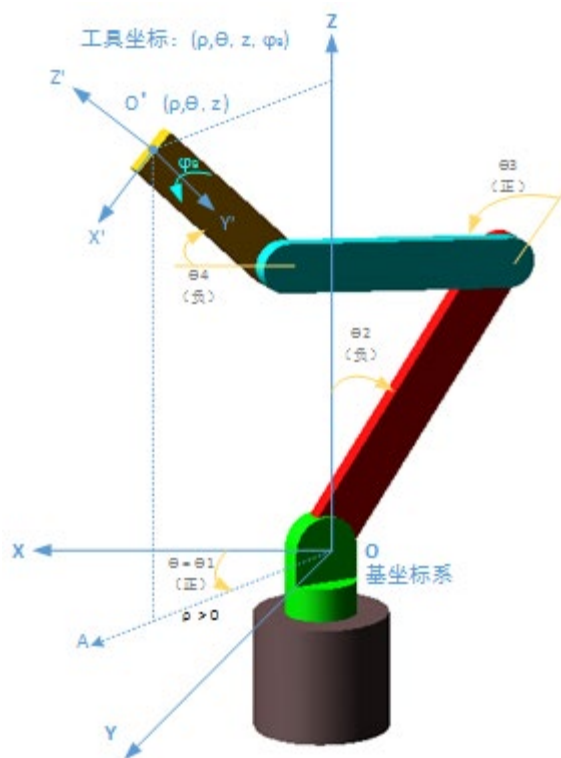
```
UDINT HMC_Serial4Control (
    POINTER TO USINT arrToolCoordAxisID,
    POINTER TO USINT arrJointCoordAxisID,
    POINTER TO LREAL arrLinkLength)
```

功能说明

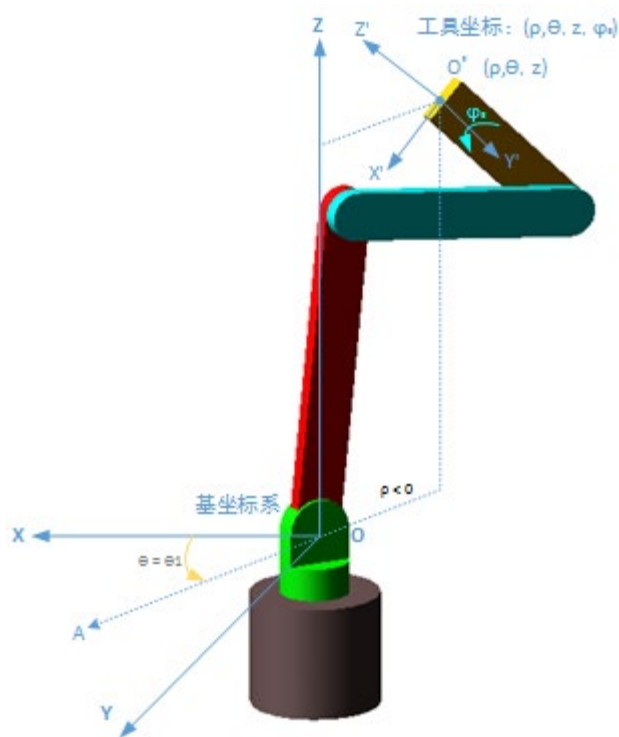
该指令完成 4 自由度串联机械臂的运动学变换（由工具坐标到关节坐标）。指令生效后，用户便可使用工具坐标（圆柱坐标）编写工程程序，控制器自动完成机构的运动学逆解变换，驱动各关节运动。过程如下图所示。



4 自由度串联机械臂控制系统框图



(a)

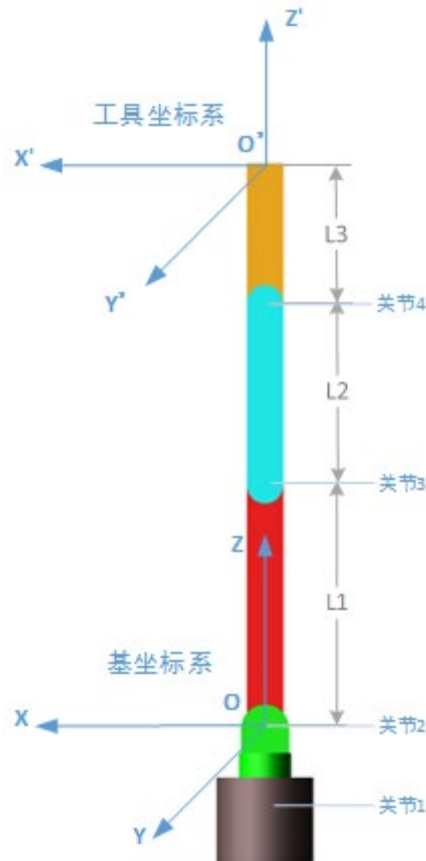


(b)

4 自由度机械手 工具坐标

(1) 图 (a) 为 4 自由度机械手 工具坐标 (圆柱坐标, $\rho > 0$ 的情形)、关节坐标示意;

(2) 图 (b) 为 4 自由度机械手 工具坐标（圆柱坐标， $\rho < 0$ 的情形）、关节坐标示意。



自由度串联机械臂关节角零点位姿

定义工具坐标系 $O'-X'Y'Z'$ ，如图 [4 自由度机械手 工具坐标](#)、图[自由度串联机械臂关节角零点位姿](#)所示。

定义基坐标系为圆柱坐标系，因此工具坐标采用圆柱坐标描述，记为 $(\rho, \Theta, z, \varphi_B)$ 。其中 φ_B 为末端执行工具绕 Y' 轴的旋转角度，旋转正方向参照右手定则。 Θ 为关节 1 的旋转角度， ρ 为极坐标半径。为全面的描述末端工具的位姿，这里对 ρ 进行了扩展，可正可负，对应的位姿分别如图 [4 自由度机械手 工具坐标](#) (a)、图 [4 自由度机械手 工具坐标](#) (b) 所示。

关节 1、2、3、4 的旋转角度称为关节坐标，记作 $(\theta_1, \theta_2, \theta_3, \theta_4)$ ，单位为度，如图 [4 自由度机械手 工具坐标](#) 所示。各关节角零点位置如图[自由度串联机械臂关节角零点位姿](#)所示。

在回零操作完成后，只要关节角零点位置未发生改变，则可在任意位置调用该指令，而不必调整到图[自由度串联机械臂关节角零点位姿](#)中位置。

使用该指令时需注意以下几点：

- (1) 长度单位为用户单位，角度单位为角度制 ($^{\circ}$)；
- (2) 务必保证关节角的零点位置在图[自由度串联机械臂关节角零点位姿](#)中位置；
- (3) 可使用 `HMC_Cancel(AxisID, CancelMode)` 撤销该指令，撤销时传入的轴号必须为 4 个关节轴中

的最小轴号；

- (4) 指令调用成功后，首次被执行时，会根据 当前关节角 初始化 工具坐标，之后在此初始位姿基础上运动；
- (5) 指令首次被执行时会把关节坐标轴及工具坐标轴的 **RepMode** 设定为有限行程，在指令运行过程中不允许修改 **RepMode**，否则可能会发生坐标转换错误；
- (6) 可根据机构实际情况设定机构各关节角的软限位值，触发限位后指令自动撤销；
- (7) 调用该指令初始化成功后，用户使用 工具坐标（圆柱坐标）编程；
- (8) 不允许调用 **HMC_DefPos** 或 **HMC_DefOrigin** 指令重新设定工具坐标的坐标值，或对工具坐标轴使用 **HOME** 指令，否则坐标转换时会产生严重的偏差。如果用户需要自定义相对坐标系，可在工程程序中自行进行坐标偏移换算；
- (9) 机构存的可达工作空间是有限的，如果规划的路径超出了其可达工作空间，出于安全考虑，指令会被撤销。请根据机构参数，合理规划工作空间，确保工作空间位于机构可达工作空间之内（可用 **HMC_Serial4ReverseKinematics** 指令校验规划的工具坐标是否合法）。

输入参数

输入参数	类型	参数描述
arrToolCoordAxisID	POINTER TO USINT	工具坐标对应的轴号数组，共 4 个元素，依次对应圆柱坐标中的 ($\rho, \theta, z, \varphi_B$) 所使用的坐标轴 一般为虚拟轴，用户在这 4 个轴中使用圆柱坐标编程，规划工具坐标
arrJointCoordAxisID	POINTER TO USINT	关节轴轴号数组，共 4 个元素，依次对应关节 1~4。如图 自由度串联机械臂关节角零点位姿 所示 物理输出轴，驱动 4 个关节运动
arrLinkLength	POINTER TO LREAL	连接杆杆长数组，共 3 个元素，依此对应图 自由度串联机械臂关节角零点位姿 中 L1、L2、L3

返回值

返回值	类型	参数描述
成功：指令 ID 错误：16#FFFFFFFF	UDINT	--

指令类型

轴运动缓存指令。

使用示例

指令使用步骤如下：

(1) 参数定义

(*杆长参数数组*)

```
arrLinkLength[0] := 200; (*L1*)
```

```
arrLinkLength[1] := 150; (*L2*)
```

```
arrLinkLength[2] := 100; (*L3*)
```

(*关节轴轴号数组*)

arrJointCoordAxisID[0] := 0; (*关节 1*)

arrJointCoordAxisID[1] := 1; (*关节 2*)

arrJointCoordAxisID[2] := 2; (*关节 3*)

arrJointCoordAxisID[3] := 3; (*关节 4*)

(*工具坐标轴号数组*)

arrToolCoordAxisID[0] := 10;(*工具坐标 ρ 对应轴号*)

arrToolCoordAxisID[1] := 11;(*工具坐标 Θ 对应轴号*)

arrToolCoordAxisID[2] := 12;(*工具坐标 z 对应轴号*)

arrToolCoordAxisID[3] := 13;(*工具坐标 ϕ_B 对应轴号*)

(2) 定义各关节轴的零点在如图[自由度串联机械臂关节角零点位姿](#)所示的位置（关节轴实际位置不必在零位）

(3) 向关节轴中投送 HMC_Serial4Control 指令

HMC_Serial4Control(arrToolCoordAxisID, arrJointCoordAxisID, arrLinkLength);

(4) 使用工具坐标轴编程，坐标为圆柱坐标系

- 移动到图 [4 自由度机械手 工具坐标](#) (a) 中位姿， $(\rho, \Theta, z, \phi_B) = (100, 30^\circ, 250, 45^\circ)$

HMC_MoveAbs(arrToolCoordAxisID[0], 100); (*移动到 $\rho = 100$ *)

HMC_MoveAbs(arrToolCoordAxisID[1], 30); (*移动到 $\Theta = 30^\circ$ *)

HMC_MoveAbs(arrToolCoordAxisID[2], 250); (*移动到 $z = 250$ *)

HMC_MoveAbs(arrToolCoordAxisID[3], 45); (*末端工具绕 Y' 轴旋转 $\phi_B = 45^\circ$ *)

- 移动到图 [4 自由度机械手 工具坐标](#) (b) 中位姿， $(\rho, \Theta, z, \phi_B) = (-100, 30^\circ, 250, 45^\circ)$

HMC_MoveAbs(arrToolCoordAxisID[0], -100); (*移动到 $\rho = -100$ *)

HMC_MoveAbs(arrToolCoordAxisID[1], 30); (*移动到 $\Theta = 30^\circ$ *)

HMC_MoveAbs(arrToolCoordAxisID[2], 250); (*移动到 $z = 250$ *)

HMC_MoveAbs(arrToolCoordAxisID[3], 45); (*末端工具绕 Y' 轴旋转 $\phi_B = 45^\circ$ *)

- 如果需要经过一系列中间点，可使用样条插补指令。

3.4.1.6.3.2 HMC_Serial4ForwardKinematics (4 自由度串联机械臂运动学正解)

函数原型

UDINT HMC_Serial4ForwardKinematics(

POINTER TO LREAL arrLinkLength,
 POINTER TO LREAL arrToolCoordinate,
 POINTER TO LREAL arrJointAngle)

功能说明

该指令完成 4 自由度串联机械臂的运动学正解（由关节坐标到工具坐标）。

输入参数

输入参数	类型	参数描述
arrLinkLength	POINTER TO LREAL	连接杆杆长数组，共 3 个元素，依此对应图 自由度串联机械臂关节角零点位姿 中 L1、L2、L3
arrToolCoordinate	POINTER TO LREAL	工具坐标数组，共 4 个元素（ $\rho, \theta, z, \varphi_B$ ）。保存正解结果
arrJointAngle	POINTER TO LREAL	关节坐标数组，共 4 个元素

返回值

返回值	类型	参数描述
成功：0 错误：16#FFFFFFFF	UDINT	--

3.4.1.6.3.3 HMC_Serial4ReverseKinematics（4 自由度串联机械臂运动学逆解）

函数原型

UDINT HMC_Serial4ReverseKinematics(
 POINTER TO LREAL arrLinkLength,
 POINTER TO LREAL arrToolCoordinate,
 POINTER TO LREAL arrReferenceJointAngle,
 POINTER TO LREAL arrJointAngle)

功能说明

该指令完成 4 自由度串联机械臂的运动学逆解（由工具坐标到关节坐标）。当输入的工具坐标超出机构的可达工作空间时，返回错误。

该函数可用在如下方面：

- 检验关键点的工具坐标是否位于可达工作空间之内；
- 实现关节坐标空间编程。

关节坐标编程可避免工具坐标编程时的诸多问题，如关键点之间的运动轨迹可能穿过机构不可达工作空间的问题、奇异位形问题等。

如果仅要求点到点之间的定位运动，对中间运动轨迹不作要求，则可首先规划好关键位置点的工具坐标，然后使用该指令对关键点进行合法性检查，将合法的關鍵点的工具坐标转换为关节坐标，此后便可使用关节坐标编程。

输入参数

输入参数	类型	参数描述
arrLinkLength	POINTER TO LREAL	连接杆杆长数组，共 3 个元素，依此对应图 自由度串联机械臂关节角零点位姿 中 L1、L2、L3
arrToolCoordinate	POINTER TO LREAL	工具坐标数组，共 4 个元素 ($\rho, \theta, z, \varphi_B$)
arrReferenceJointAngle	POINTER TO LREAL	关节坐标数组（参照值），共 4 个元素，依次为关节 1~4 的旋转角度 由于逆解的结果不止一个，选择距离该参照位置 较近的一组解作为逆解结果
arrJointAngle	POINTER TO LREAL	关节坐标数组，共 4 个元素，保存逆解结果，依次为关节 1~4 的旋转角度

返回值

返回值	类型	参数描述
成功: 0	UDINT	当用户输入参数错误或工具坐标超出机构的可达工作空间时，返回错误。
错误: 16#FFFFFFFF		

指令类型

非轴运动缓存指令。

3.4.1.7 自定义运动控制功能

3.4.1.7.1 HMC_SetDposSwitch（Dpos 规划开关切换）

3.4.1.7.1.1 函数原型

UDINT HMC_SetDposSwitch (USINT ucAxisID, USINT ucDposSwitch)

3.4.1.7.1.2 功能说明

该指令切换指定轴的 Dpos 规划开关，开关设定为 0 时 Dpos 由 MC 内部运动控制算法规划，为 1 时 Dpos 由用户通过 HMC_SetDposVal 指令规划。

Dpos 规划开关设定为 1 时，该轴运动缓存中的运动指令不再执行，也无法通过 HMC_Cancel 清除，因此最好在 Dpos 规划开关切换为 1 前清空该轴的运动缓存，并在开关设定为 1 期间不再往其中投送运动指令。

3.4.1.7.1.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucDposSwitch	USINT	Dpos 规划开关 0: Dpos 由内部运动控制算法规划 1: Dpos 由用户规划

3.4.1.7.1.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.4.1.7.1.5 指令类型

非轴运动缓存指令。

3.4.1.7.2 HMC_SetDposVal (Dpos 规划)

3.4.1.7.2.1 函数原型

UDINT HMC_SetDposVal (USINT ucAxisID, LREAL dSetDposVal)

3.4.1.7.2.2 功能说明

Dpos 规划开关设定为 1 时，该指令设定轴的 Dpos 为指定值；Dpos 规划开关设定为 0 时，该指令无作用。

使用该指令设定 Dpos 后，如果想要等到设定的 Dpos 生效后再执行下一步动作，可在其后调用 HMC_SynchroToAlm 指令，HMC_SynchroToAlm 会与 MC 内部的硬实时任务有一个同步操作，这样直到设定的 Dpos 值被硬实时任务处理生效后才退出指令（硬实时任务每个伺服周期执行一次）。

因此，如果想要使用 HMC_SetDposVal 达到每个伺服周期更新一次 Dpos 的目的，可在一个循环中反复调用 HMC_SetDposVal，并在其后调用 HMC_SynchroToAlm 指令，同时使系统的负荷尽量降低（把用不到的轴全部关闭（轴类型设定为-1），并尽可能的减少处于运行状态的 IEC 任务的数量）。

3.4.1.7.2.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dSetDposVal	LREAL	设定的 Dpos 值

3.4.1.7.2.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.4.1.7.2.5 指令类型

非轴运动缓存指令。

3.4.1.7.2.6 示例

设定 Dpos 规划开关为 1，合理规划 Dpos，使 0 轴和 1 轴位置同步，并且速度曲线满足正弦曲线规律。代码如下：

(1) 初始化，任务 1。

程序:

(*关闭用不到的轴*)

FOR AxisID:=2 TO 15 BY 1 DO

HMC_SetAxisType(AxisID, -1);

END_FOR

HMC_SetAxisType(0, 0); (*0 轴为虚拟轴*)

HMC_SetAxisType(1, 0); (*1 轴为虚拟轴*)

HMC_SetDposSwitch(0,1); (*0 轴 Dpos 规划开关设为 1*)

HMC_SetDposSwitch(1,1); (*1 轴 Dpos 规划开关设为 1*)

HMC_DefPos(0,0); (*0 轴位置初始化*)

HMC_DefPos(1,0); (*0 轴位置初始化*)

(2) Dpos 规划, 任务 2。

程序:

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Dpos			LREAL ▾	0	FALSE ▾
0002	count			DINT ▾	0	FALSE ▾
0003	MaxNum		运行的伺服周期数	DINT ▾	500	FALSE ▾
0004	PI		π	LREAL ▾	3.1415926	FALSE ▾
0005	A		Dpos幅值	LREAL ▾	1000	FALSE ▾

count := 0;

WHILE TRUE DO

count := count + 1;

Dpos := A * (1 - COS(2*PI*count/MaxNum)); (*目标位置计算*)

HMC_SetDposVal(0, Dpos);(*0 轴 Dpos 规划*)

HMC_SetDposVal(1, Dpos);(*1 轴 Dpos 规划*)

HMC_SynchroToAlm (); (*同步到算法硬实时任务*)

IF count = MaxNum THEN (*控制运行次数, 如果不需要控制次数也可一直循环*)

EXIT;

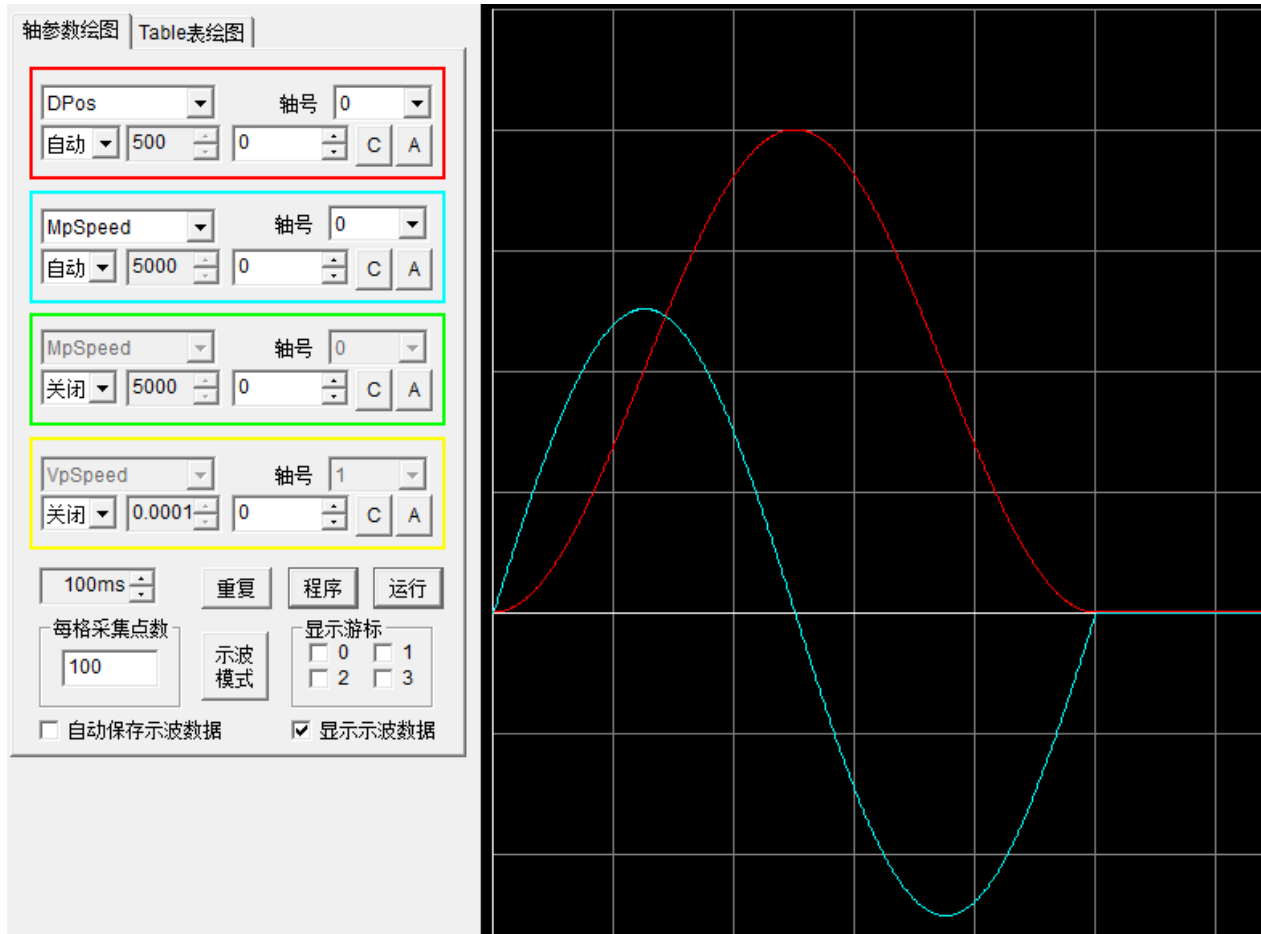
END_IF

END_WHILE

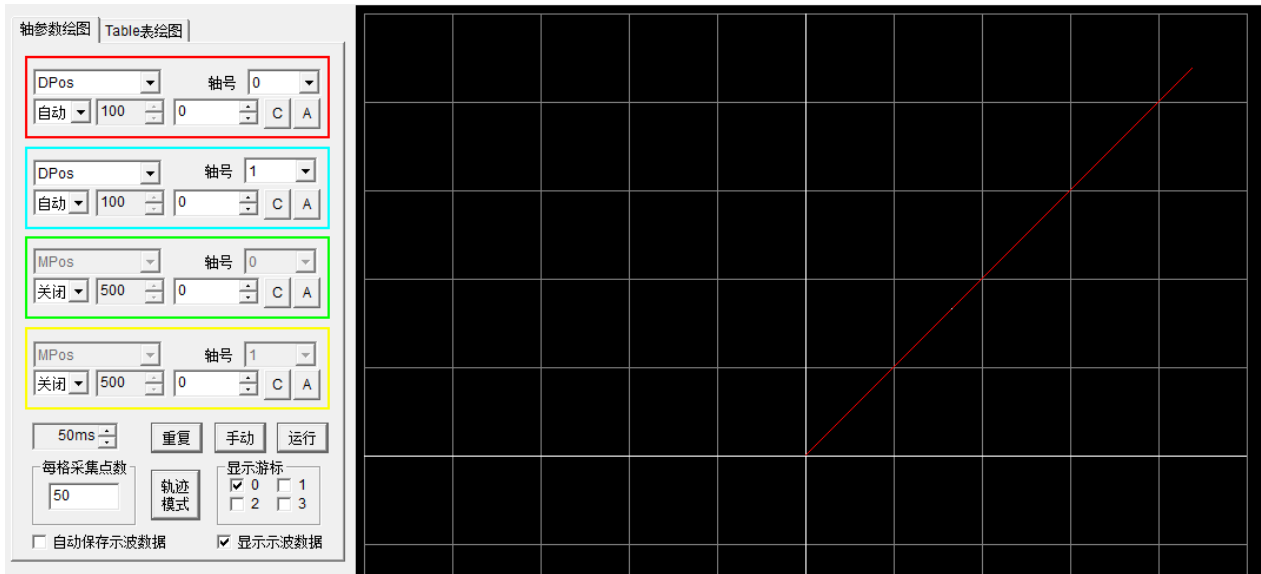
首先运行初始化任务 1，然后在终端输入如下指令：

```
HMC_TriggerScope();TaskStart(2);
```

运行结果如下（伺服周期 1ms）：



轴 0 Dpos 及 MpSpeed



轴 0、轴 1 合运动轨迹

3.4.1.7.3 HMC_SynchroToAlm（同步到算法任务）

3.4.1.7.3.1 函数原型

UDINT HMC_SynchroToAlm ()

3.4.1.7.3.2 功能说明

该指令可用于与 MC 内部的算法硬实时任务的同步。调用该指令后，会等到硬实时任务执行过一次后才退出指令（硬实时任务每个伺服周期执行一次）。

3.4.1.7.3.3 返回值

返回值	类型	参数描述
0	UDINT	执行成功

3.4.1.7.3.4 指令类型

非轴运动缓存指令。

3.4.1.8 轴参数设定指令

3.4.1.8.1 HMC_SetAxisType（设置轴类型）

3.4.1.8.1.1 函数原型

UDINT HMC_SetAxisType (USINT ucAxisID, EmAxisType ucAxisType)

3.4.1.8.1.2 功能说明

设置指定轴的轴类型，各轴类型含义详见轴参数 AxisType。

1. 轴号 0~63 的不同区间范围所支持的轴类型不同。

(1) $0 \leq \text{轴号} < \text{控制器支持的直连轴总数}$

不同型号控制器支持的直连轴总数不同，如 MC1008 型号控制器的直连轴总数为 8，MC1004 型号控制器的直连轴总数为 4，MC1002E 型号控制器的直连轴总数为 2。

轴号在此范围时，可设置轴类型为该控制器型号所支持的任意轴类型。

(2) $\text{直连轴总数} \leq \text{轴号} < (\text{直连轴总数} + \text{控制器支持的手轮总数})$

不同型号控制器支持的手轮总数不同，MC1008 型号控制器的直连轴总数为 2，其他型号控制器的直连轴总数为 1。

轴号在此范围时，可设置轴类型为虚轴、未使用轴、三种编码器轴类型。若为总线型号控制器，也支持该总线类型对应的总线轴类型。

(3) $(\text{直连轴总数} + \text{控制器支持的手轮总数}) \leq \text{轴号} < 64$

轴号在此范围时，可设置轴类型为虚轴或未使用轴，若为总线型号控制器，也支持该总线类型对应的总线轴类型。

各控制器型号的不同轴号范围支持的轴类型请参见《MC 系列运动控制器系统手册》中的附录 3 MC 控制器支持的轴类型。

2. 当该轴的轴指令缓存中有指令时，调用此函数设置轴类型不成功。

3. 轴类型设置成功后，系统会自动根据不同的轴类型会关联修改轴参数 RepMode。

当用户设置某个轴类型为虚轴或编码器轴时，系统将设置 RepMode=2；当用户设置某个轴类型为其他轴类型时，系统将设置 RepMode=0。

3.4.1.8.1.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
EmAxisType	ucAxisType	轴类型

3.4.1.8.1.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.1.5 指令类型

非轴运动缓存指令。

3.4.1.8.1.6 应用举例

HMC_SetAxisType(0,10); (*设置轴 0 为直连步进轴*)

HMC_SetAxisType(1,20); (*设置轴 1 为直连伺服轴*)

HMC_SetAxisType(60,-1); (*设置轴 60 为未使用轴*)

- 不支持的模块版本（直连步进轴：脉冲从 Do0~3 输出）

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A01

- 不支持的模块版本（增量式编码器：脉冲从 DI0~3 输入）

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.8.2 HMC_HwLimitConfig（设置硬限位开关）

3.4.1.8.2.1 函数原型

UDINT HMC_HwLimitConfig (USINT ucAxisID, USINT ucLimitType, INT sDiChan)

3.4.1.8.2.2 功能说明

设置指定轴的硬限位开关，限位功能仅对有限行程轴有效。

该函数配置指定某路 DI 为轴的前向或后向硬限位开关，当前（后）向限位 DI 通道为高电平时，认为该轴到达了前（后）向限位位置，系统将根据 FsLimitMode/RsLimitMode 值来进行有限行程限位处理。

限位开关触发后，轴的运动方式详见《MC 系列运动控制器指令手册》第 9 章的有限行程轴超限处理。

3.4.1.8.2.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucLimitType	USINT	0: 设置前向硬限位 1: 设置后向硬限位
sDiChan	INT	关联的 DI 通道（0~63） 当 sDiChan 为-1 时，表示取消硬限位开关，仅对有限行程轴有效，指定轴将不再关联原来的前

输入参数	类型	参数描述
		(后) 向限位 DI 通道, 即使当前处于限位状态也会立即解除

3.4.1.8.2.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.2.5 指令类型

非轴运动缓存指令

3.4.1.8.2.6 应用举例

配置第 5 路 DI 为轴 0 的前向限位开关, 第 6 路 DI 为轴 0 的后向限位开关。

HMC_HwLimitConfig (0 ,0, 5);

HMC_HwLimitConfig (0 ,1, 6);

3.4.1.8.3 HMC_SetUnits (设置用户单位)

3.4.1.8.3.1 函数原型

UDINT HMC_SetUnits (USINT ucAxisID, LREAL dUnits)

3.4.1.8.3.2 功能说明

设置指定轴的单位转换因子轴参 Units, 该参数只可为正值, 单位: 线/用户单位, "线"是指轴走一个脉冲对应的刻度单位。

通过设置轴参 Units, 可将编码器反馈的计数值或脉冲输出值转换为用户方便使用的单位数值, 例如 mm、角度等。

3.4.1.8.3.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dUnits	LREAL	用户单位, 不可为 0 或负数

3.4.1.8.3.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.3.5 指令类型

非轴运动缓存指令。

3.4.1.8.3.6 应用举例

例如某轴走 10000 个脉冲正好转一圈，即转一圈对应 10000 线；假如说，通过机械结构的设计，该轴每转一圈（10000 线）带动终端走 10 mm，那么如下两种方法是等价的，都可以让终端走 50 mm（即电机转动 5 圈）：

方法 1：

```
HMC_SetUnits(0,1000); (*10000 线/10mm=1000 线/mm*)
```

```
HMC_Move(0,50); (*终端走 50 mm*/)
```

方法 2：

```
HMC_SetUnits(0,1);
```

```
HMC_Move(0,50000); (*电机走 50,000 线；终端走 50 mm*/)
```

上述两种方法中选择哪一种取决于用户的使用习惯。

3.4.1.8.4 HMC_SetJerkMode（设置梯形/S 形加速度）

3.4.1.8.4.1 函数原型

```
UDINT HMC_SetJerkMode(USINT ucAxisID, USINT ucJerkMode)
```

3.4.1.8.4.2 功能说明

设置 JerkMode。JerkMode 为 0 时运动过程速度曲线为梯形加减速曲线，JerkMode 为 1 时为 S 形加减速曲线。

3.4.1.8.4.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucJerkMode	USINT	JerkMode（默认值 0）。 0：运动过程速度曲线为梯形加减速曲线； 1：运动过程速度曲线为 S 形加减速曲线。

3.4.1.8.4.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.4.5 指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01、MC1002E-A01

3.4.1.8.5 HMC_SetKe (设置 Ke)

3.4.1.8.5.1 函数原型

UDINT HMC_SetKe (USINT ucAxisID, DINT iKeNumerator, DINT iKeDenominator)

3.4.1.8.5.2 功能说明

设置轴参数 Ke 的分子（轴参数 KeNumerator）和分母（轴参数 KeDenominator），参考轴参数 KeNumerator 和 KeDenominator 的说明。

该参数仅对伺服轴和编码器轴生效。该参数和 Units 有类似的作用，(Ke /Units)共同构成了伺服轴或编码器轴的“单位转换因子”，其单位为“线/用户单位”，参见 [Units（单位转换因子）](#)。

3.4.1.8.5.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
iKeNumerator	DINT	Ke 分子
iKeDenominator	DINT	Ke 分母

3.4.1.8.5.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.5.5 指令类型

非轴运动缓存指令。

3.4.1.8.5.6 补充说明

Ke 可设置为负，设置为负可对实际反馈速度及位置方向取反（部分低版本固件不支持）。

Ke 不可为 0。

3.4.1.8.5.7 应用举例

转盘由一个电机驱动控制旋转，另有编码器连接至该转盘，测量其旋转角度。

转盘旋转一周，该编码器输出 8192 个脉冲。期望在软件上直接显示该转盘的旋转角度值，由于 8192/360 无法整除，为了达到最高精度，此时可通过设置 Ke 实现。

```
HMC_SetAxisType(0,30);
```



```
HMC_SetUnits(0,1);
```

```
HMC_SetKe(0,360,8192);
```

 (*配合 Units，使得该轴单位为角度，Mpos 直接显示角度*)

3.4.1.8.6 HMC_SetKs（设置 Ks）

3.4.1.8.6.1 函数原型

```
UDINT HMC_SetKs (USINT ucAxisID, DINT iKsNumerator, DINT iKsDenominator)
```

3.4.1.8.6.2 功能说明

设置轴参数步进输出比例 Ks 的分子（轴参数 KsNumerator）和分母（轴参数 KsDenominator），参考轴参数 KsNumerator 和 KsDenominator 的说明。

该参数仅对步进轴有效。该参数和 Units 有类似的作用，(Units*Ks)共同构成了步进轴的“单位转换因子”，其单位为“线/用户单位”，参见 [Units（单位转换因子）](#)。

3.4.1.8.6.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
iKsNumerator	DINT	Ks 分子
iKsDenominator	DINT	Ks 分母

3.4.1.8.6.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.6.5 指令类型

非轴运动缓存指令。

3.4.1.8.6.6 补充说明

Ks 需 > 0。

3.4.1.8.6.7 应用举例

转盘由一个步进电机驱动控制旋转。该步进电机输出 8192 个脉冲可驱动转盘旋转 1 圈。期望在软件上直接显示该转盘的旋转角度值，由于 8192/360 无法整除，为了达到最高精度，此时可通过设置 Ks 实现。

```
HMC_SetAxisType(0,10);
```

```
HMC_SetUnits(0,1);
```

```
HMC_SetKs(0,8192,360);
```

HMC_Move(0,180); (*驱动转盘旋转 180 度*)

3.4.1.8.7 HMC_SetFeedBackSrcAddr（设置用户自定义反馈输入地址）

3.4.1.8.7.1 函数原型

UDINT HMC_SetFeedBackSrcAddr (USINT ucAxis, POINTER TO LREAL pAddr, USINT DataType)

3.4.1.8.7.2 功能说明

配置轴实际位置 Mpos 的用户自定义输入源。当轴参数 FeedBackSrcMode=1 时，该轴的实际位置 Mpos 由用户自定义输入源增量计算更新，此时 Mpos 与该轴的实际物理输入无任何关系。

当轴 Mpos 计算来自用户自定义输入源时，依据自定义输入源的变化增量累积计算 Mpos，轴参 Units 同样参与 Mpos 计算。

注意：该功能仅对有实际物理输入的轴类型生效，如编码器轴、伺服轴、总线轴等。

3.4.1.8.7.3 输入参数

输入参数	类型	参数描述
ucAxis	USINT	轴号
pAddr	POINTER TO LREAL	用户自定义变量地址
DataType	USINT	用户自定义变量的数据类型：USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、F32=7、LREAL=8

3.4.1.8.7.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.7.5 指令类型

非轴运动缓存指令

3.4.1.8.7.6 应用举例

现有一位置传感器，其输出信号为电压信号，待测量位置在 0~100000，对应位置传感器输出电压信号 0~10V。

将位置传感器输出的电压信号连接至第 1 路 AI。用轴 0 实时计算该位置信息。

HMC_SetAxisType (0,30);

axis0.FeedBackSrcMode := 1;

HMC_SetUnits(0, 10.0/100000.0); (*设置位置和电压信号转换系数为单位*)

HMC_SetFeedBackSrcAddr(0, ADR(AI0), 4); (*设置第 1 路 AI 为轴 0 的 Mpos 计算源*)

HMC_Defpos(0, AI0*(100000/10)); (*初始化轴 0 的位置为当前测量的绝对位置，后续在此基础上增量计算位置值*)

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.8.8 HMC_SetOutDstAddr（设置用户自定义输出地址）

3.4.1.8.8.1 函数原型

UDINT HMC_SetOutDstAddr (USINT ucAxis, POINTER TO DINT pAddr, USINT DataType)

3.4.1.8.8.2 功能说明

设置位置闭环输出值 DACOut 的自定义输出地址。位置闭环模式时，计算的输出值 DACOut 输出给该轴的物理输出通道，如伺服 20 模式时，输出给该轴对应的 AO 通道；如总线速度模式时，输出给该轴对应的总线输出数据区。但当轴参数 OutDstMode=1 时，则停止将 DACOut 值输出至该轴物理输出通道，而是输出给该轴的自定义输出地址。

当 OutDstMode=1 时，如果为直连伺服轴，此时算法不修改该轴对应 AO 通道值，则此时若用户通过 AT 在线修改或 IEC 程序修改 AO 通道值会生效；对于总线速度模式轴，算法会立即输出速度 0，将对应驱动器停止。

注意：该功能仅位置闭环模式的轴类型生效，如伺服轴、总线速度模式等。

3.4.1.8.8.3 输入参数

输入参数	类型	参数描述
ucAxis	USINT	轴号
pAddr	POINTER TO DINT	用户自定义变量地址
DataType	USINT	用户自定义变量的数据类型：USINT=1、SINT=2、UINT=3、INT=4、UDINT=5、DINT=6、F32=7、LREAL=8

3.4.1.8.8.4 返回值

返回值	类型	参数描述
0	UDINT	设置成功
其他值	UDINT	错误码

3.4.1.8.8.5 指令类型

非轴运动缓存指令。

3.4.1.8.8.6 应用举例

设置伺服轴轴 0 的输出目标为系统变量 diOutPut。

```
HMC_SetOutDstAddr(0, ADR(diOutPut), 6);
```

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.8.9 HMC_SetSSIEncoderBit (设置 SSI 编码器位数)

3.4.1.8.9.1 函数原型

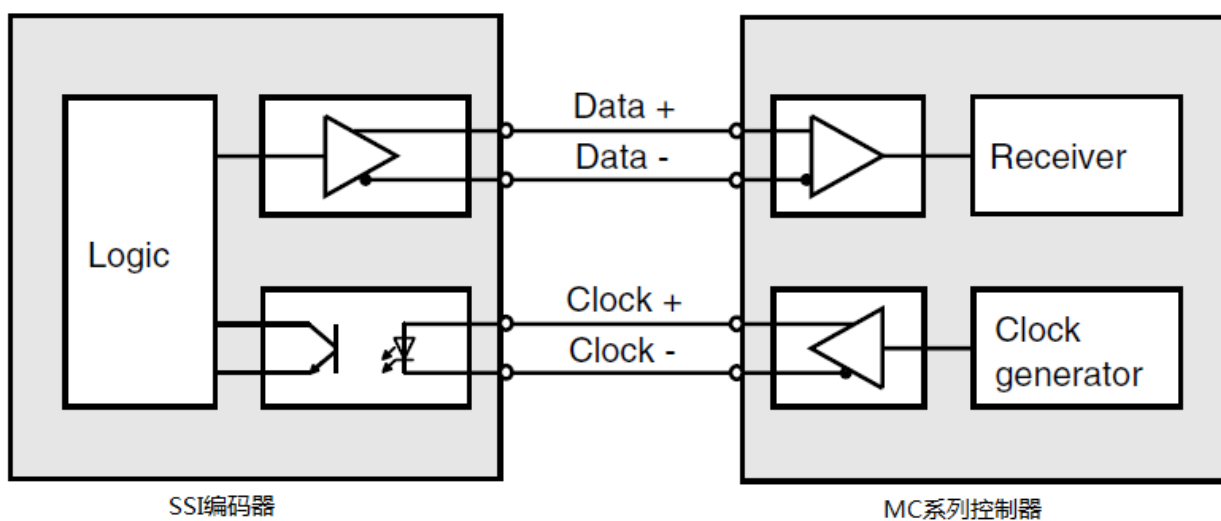
```
UDINT HMC_SetSSIEncoderBit (USINT ucAxisID, USINT ucSSIEncoderBit)
```

3.4.1.8.9.2 功能说明

设置 SSI 编码器位数，默认值为 12。

如果该轴类型已经为 SSI 轴类型（33），则该指令执行成功后会重新初始化 Mpos/Dpos 值为当前 SSI 编码器的绝对位置。

在使用 SSI 绝对值编码器时，应按以下步骤进行操作
硬件接线



硬件接线示意图

软件操作

1. 配置 SSI 编码器相关参数

根据编码器实际规格，配置对应的 SSI 编码器相关参数

配置项	相关函数	实时修改是否初始化位置
编码器位数	HMC_SetSSIEncoderBit	是

配置项	相关函数	实时修改是否初始化位置
编码器码制	SetSSIEncoderDataType	是
SSI 总线时钟频率	HMC_SetSSIEncoderFreq	否
SSI 总线数据锁存时间	HMC_SetSSIEncoderTp	否

2. 设置关联轴的轴类型为 SSI 编码器轴类型

参数配置成功后，设置轴类型 HMC_SetAxisType(AxisID, 33)。

轴类型成功为 SSI 编码器轴类型后，会根据读取到当前编码器的实际位置，并依此初始化当前轴位置 MPos。再此之后，则在此初始位置基础上增量更新每个伺服周期位置增量，计算轴的实时位置值。

其它注意事项：在线修改 SSI 配置参数

轴类型成功为 SSI 编码器轴类型后，位置采集更新过程中，仍可调用步骤 1 的函数重新配置 SSI 编码器相关参数时，且实时生效，但根据不同参数，会决定是否影响重新初始化轴的实际位置 MPos，即上表格第三列。行程设置

如果需要 MPos 的值始终与编码器绝对位置一致，则可设定该轴为循环行程（RepMode=1），且循环行程的行程长度与编码器的位置数据范围一致，如 12 位编码器，在 Units、Ke 均为 1 的情况下，RepMode=1 时应设置 RepDist = 2^{12} 。

在设置 RepMode=2、RepDist=（编码器位置数据范围/2）的情况下，可实现该轴 Mpos 区间长度=编码器位置数据区间长度，并且 Mpos 的范围区间是关于零点对称的，如 12 位编码器，在 Units、Ke 均为 1 的情况下，应设置 RepMode=2、RepDist = $2^{12}/2$ 。

如果设定该轴为有限行程，则如果编码器未发生数据溢出时，Mpos 与编码器绝对位置一致；当编码器发生数据溢出时，则控制器仍按实际增量位移计算 Mpos（不跳变），而此时编码器内部的绝对位置数据则因数据溢出而发生了跳变。

3.4.1.8.9.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucSSIEncoderBit	USINT	编码器位数(默认值 12) 范围：1~127

3.4.1.8.9.4 返回值

返回值	类型	参数描述
0	UDINT	设定成功
其它	UDINT	错误码

3.4.1.8.9.5 指令类型

非轴运动缓存指令

3.4.1.8.9.6 补充说明

MC1008 支持 4 个 SSI 轴，MC1004 支持 2 个 SSI 轴，MC1002R/MC1002E 暂不支持。支持 SSI 轴的

轴号，MC1008 限定为 0、1、2、3，MC1004 限定为 0、1。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-XXX、MC1002E-XXX

3.4.1.8.9.7 应用举例

使用 SSI 单圈绝对值编码器其规格：

接口电气参数	
传感器	
供电电压	5 (+0.4) V DC ¹⁾
功耗 (无负载)	< 40 mA
电源反极性保护	yes
测量范围	360°
分辨率/编码	12位/格雷码
线性度 (25℃)	< 1.5°
重复精度	≤ 0.4°
数据刷新速率	典型 100 μs
符合RoHS标准，据	EU标准 2002/95/EG
符合CE标准，据	EN 61000-6-2, EN 55011 Class B
SSI接口	
时钟脉冲速度	100 kHz ... 750 kHz
输出驱动	RS485
单稳态时间/最大	16 μs / 20 μs
输出短路保护	有 ²⁾
允许负载/通道	典型 60 Ohm (对应 RS485)

SSI 单圈绝对值编码器规格

则编写程序如下：

```

SSIEncoderAxisID := 0;    (*SSI 绝对值编码器关联轴 0*)

HMC_SetSSIEncoderBit (SSIEncoderAxisID ,12);    (*设置位数*)
HMC_SetSSIEncoderDataType (SSIEncoderAxisID ,1); (*设置编码方式*)
HMC_SetSSIEncoderFreq (SSIEncoderAxisID ,100); (*设置总线时钟频率*)
HMC_SetSSIEncoderTp (SSIEncoderAxisID ,100);    (*设置数据锁存时间*)

HMC_SetAxiStype(SSIEncoderAxisID, 33);    (*正确配置参数后，设置轴类型为 SSI*)

```

3.4.1.8.10 HMC_SetSSIEncoderDataType (设置 SSI 编码器码制)

3.4.1.8.11 函数原型

```

UDINT HMC_ SetSSIEncoderDataType (USINT ucAxisID,
USINT ucSSIEncoderDataType)

```

3.4.1.8.12 功能说明

配置 SSI 编码器轴码制。0 为二进制编码，1 为格雷码；默认值为 1。

如果该轴类型已经为 SSI 轴类型（33），则该指令执行成功后会重新初始化 Mpos/Dpos 值为当前 SSI 编码器的绝对位置。

3.4.1.8.13 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucSSIEncoderDataType	USINT	码制（0 为二进制编码；1 为格雷码。默认值 1）

3.4.1.8.14 返回值

返回值	类型	参数描述
0	UDINT	设定成功
其它	UDINT	错误码

3.4.1.8.15 指令类型

非轴运动缓存指令

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-XXX、MC1002E-XXX

3.4.1.8.16 应用举例

参考 [HMC_SetSSIEncoderBit](#)（设置 SSI 编码器位数）

3.4.1.8.17 HMC_SetSSIEncoderFreq (设置 SSI 总线时钟频率)

3.4.1.8.17.1 函数原型

UDINT HMC_SetSSIEncoderFreq (USINT ucAxisID, LREAL dSSIEncoderFreq)

3.4.1.8.17.2 功能说明

配置 SSI 总线时钟频率。默认值为 200 kHz。

如果该轴类型已经为 SSI 轴类型（33），则可在线修改时钟频率，实时生效，且不影响当前

Mpos/Dpos 值。

3.4.1.8.17.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
dSSIEncoderFreq	LREAL	时钟频率(单位 KHZ, 默认值 200KHZ) 范围: 40~10000

3.4.1.8.17.4 返回值

返回值	类型	参数描述
0	UDINT	设定成功
其它	UDINT	错误码

3.4.1.8.17.5 指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-XXX、MC1002E-XXX

■ 备注

将频率换算为周期, 如果周期不是 0.1 μ s 的整数倍, 则就近取整。再折算为频率, 如果两次设置频率相同, 则返回错误码。

3.4.1.8.17.6 应用举例

参考 [HMC_SetSSIEncoderBit \(设置 SSI 编码器位数\)](#)

3.4.1.8.18 HMC_SetSSIEncoderTp (设置 SSI 总线数据锁存时间)

3.4.1.8.19 函数原型

UDINT HMC_SetSSIEncoderTp (USINT ucAxisID, USINT ucSSIEncoderTp)

3.4.1.8.20 功能说明

配置 SSI 总线数据锁存时间。默认值为 35 μ s。

如果该轴类型已经为 SSI 轴类型 (33), 则可在线修改该参数, 实时生效, 且不影响当前 Mpos/Dpos 值。

3.4.1.8.21 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucSSIEncoderTp	USINT	数据锁存时间（单位 us，默认值 35 μ s）

3.4.1.8.22 返回值

返回值	类型	参数描述
0	UDINT	设定成功
其它	UDINT	错误码

3.4.1.8.23 指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-XXX、MC1002E-XXX

3.4.1.8.24 应用举例

参考 [HMC_SetSSIEncoderBit（设置 SSI 编码器位数）](#)

3.4.1.8.25 HMC_SetCmdBufferLoadMode（设置轴指令加载模式）

3.4.1.8.25.1 函数原型

UDINT HMC_SetCmdBufferLoadMode (USINT ucAxisID, USINT ucCmdLoadMode)

3.4.1.8.25.2 功能说明

设置轴指令的加载模式。主要用于切向追踪拐角抬刀模式下的辅助处理。

在切向追踪指令处于拐角暂停抬刀状态下时，若使用 HMC_Cancel 撤销指令，由于在此时 XY 指令队列中的指令处于暂停状态，出于安全考虑，撤销完成后并不自动恢复 XY 轴中指令队列中的指令运行，若需要恢复 XY 中指令队列中后续指令的运行，用户需调用 HMC_SetCmdBufferLoadMode 把指令加载模式设定为正常加载模式（0）。

3.4.1.8.25.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucCmdLoadMode	USINT	0: 正常加载模式; 1: 当前指令运行完成后, 不加载下一条指令。

3.4.1.8.25.4 返回值

返回值	类型	参数描述
0	UDINT	成功
其它	UDINT	错误码

3.4.1.8.25.5 指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

3.4.1.8.26 HMC_GetCmdBufferLoadMode（获取轴指令加载模式）

3.4.1.8.26.1 函数原型

USINT HMC_GetCmdBufferLoadMode (USINT ucAxisID)

3.4.1.8.26.2 功能说明

获取轴指令的加载模式。

3.4.1.8.26.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.4.1.8.26.4 返回值

返回值	类型	参数描述
指令加载模式	USINT	0: 正常加载模式; 1: 当前指令运行完成后, 不加载下一条指令。
255	USINT	错误码

3.4.1.8.26.5 指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1008-C01、MC1004-A01、MC1004-B01、MC1002R-A01

3.4.1.8.27 HMC_SetCmdStopMaxTime (设置指令结束最大等待时间)

3.4.1.8.27.1 函数原型

UDINT HMC_SetCmdStopMaxTime (UDINT uiTime)

3.4.1.8.27.2 功能说明

设置指令结束最大等待时间。

3.4.1.8.27.3 输入参数

输入参数	类型	参数描述
uiTime	UDINT	时间 (ms)

3.4.1.8.27.4 返回值

返回值	类型	参数描述
-	UDINT	0: 成功; 其它值: 失败。

3.4.1.8.27.5 指令类型

非轴运动缓存指令。

■ 不支持的模块版本

MC1008-D03 以下版本、MC1004-C03 以下版本、MC1004L-A04 以下版本、MC1002R-B02 以下版本、MC1002E-A04 以下版本

3.4.1.9 状态读取指令

3.4.1.9.1 HMC_GetSysErr (获取系统错误码)

3.4.1.9.1.1 函数原型

UDINT HMC_GetSysErr ()

3.4.1.9.1.2 功能说明

获取系统最近一次发生的系统错误，若未发生系统错误，则返回 0。

算法实时运行时，会对检测出的异常报错误，错误类型包括系统错误和用户错误。

正常运行时不会产生系统错误，除非算法所用数据被外部程序模块篡改，导致进入一些不可能进入的代码分支，从而产生系统错误。

系统错误是严重的错误类型，所以当产生系统错误时，会自动关闭 watchdog，且算法停止解算（“超时错误”除外），排查错误原因后，需要重启控制器才可恢复。

系统错误中的“超时错误”比较特殊，错误 ID 为 1025，该错误是指算法模块单次运行所耗时间超出当前伺服周期，当发生该系统错误时，只是产生报警，不会关闭 watchdog，算法仍会正常解算。

3.4.1.9.1.3 返回值

返回值	类型	参数描述
0	UDINT	没有错误
其他值	UDINT	当前发生系统错误

3.4.1.9.1.4 指令类型

非轴运动缓存指令。

3.4.1.9.2 HMC_GetUserErrID（获取用户错误码）

3.4.1.9.2.1 函数原型

UDINT HMC_GetUserErrID (UDINT uiErrIndex)

3.4.1.9.2.2 功能

返回用户错误码。

用户错误是因为参数设置非法导致，如循环模式时 RepDist 设置小于 0，采样功能块设置采样通道号超出系统支持的采样通道范围等，即可以看出，用户错误是因为用户设置参数错误人为导致。

发生用户错误时，系统不会有其它保护动作，如停止解算、关闭伺服使能等。

目前只能返回当前用户错误码，暂不支持按索引返回错误码。

3.4.1.9.2.3 输入参数

输入参数	类型	参数描述
uiErrIndex	UDINT	错误索引 取值范围： 0~（HMC_GetUserErrNum()-1）

3.4.1.9.2.4 返回值

返回值	类型	参数描述
用户错误码	UDINT	-

3.4.1.9.2.5 指令类型

非轴运动缓存指令。

3.4.1.9.3 HMC_GetUserErrNum（获取用户错误数）

3.4.1.9.3.1 函数原型

UDINT HMC_GetUserErrNum ()

3.4.1.9.3.2 功能

返回已发生的用户错误数。

当历史未发生用户错误时，返回值为 0，若发生过用户错误，目前只持者返回 1，暂不支持累计错误数量。

3.4.1.9.3.3 返回值

返回值	类型	参数描述
已发生的用户错误数量	UDINT	-

3.4.1.9.3.4 指令类型

非轴运动缓存指令。

3.4.1.9.4 HMC_GetUserErrMes（获取用户错误码附加信息）

3.4.1.9.4.1 函数原型

UDINT HMC_GetUserErrMes (UDINT uiErrIndex)

3.4.1.9.4.2 功能

返回用户错误码附加信息。通过附加信息结合固件版本其它信息，可以确定产生该用户错误的函数。

目前只能返回当前用户错误码附加信息，暂不支持按索引返回错误码附加信息。

3.4.1.9.4.3 输入参数

输入参数	类型	参数描述
uiErrIndex	UDINT	错误索引 取值范围：0~（HMC_GetUserErrNum()-1）

3.4.1.9.4.4 返回值

返回值	类型	参数描述
用户错误码附加信息	UDINT	-

3.4.1.9.4.5 指令类型

非轴运动缓存指令。

3.4.1.9.5 HMC_GetAllAxisErr（获取所有轴的总体异常状态）

3.4.1.9.5.1 函数原型

UDINT HMC_GetAllAxisErr ()

3.4.1.9.5.2 功能

获得所有轴的总体异常状态，当 64 个轴中存在超过 1 个以上轴的轴状态 AxisStatus 与其掩码 AxisErrMask 进行逻辑与运算结果不为 0，则认为存在轴异常。即如下表达式非 0 时，认为所有轴总体异常，该函数返回-1（0xFFFFFFFF）：((Axis0.AxisStatus and Axis0.AxisErrMask) or (Axis1.AxisStatus and Axis1.AxisErrMask) or or (Axis63.AxisStatus and Axis63.AxisErrMask))。

3.4.1.9.5.3 返回值

返回值	类型	参数描述
0	UDINT	所有轴正常
其他值	UDINT	轴异常

3.4.1.9.5.4 指令类型

非轴运动缓存指令。

3.4.1.9.6 HMC_GetCmdErr（获取轴指令错误）

3.4.1.9.6.1 函数原型

UDINT HMC_GetCmdErr (UDINT uiAxisCmdID)

3.4.1.9.6.2 功能

根据轴指令 ID，返回该轴指令错误。没有错误时返回 0，轴指令 ID 不存在时返回 0xffffffff，指令 ID 存在错误时，返回错误类型。

轴指令 ID 是投放运动控制指令时，该函数的返回值。

3.4.1.9.6.3 输入参数

输入参数	类型	参数描述
uiAxisCmdID	UDINT	轴指令 ID

3.4.1.9.6.4 返回值

返回值	类型	参数描述
0	UDINT	没有错误

返回值	类型	参数描述
其他值	UDINT	轴指令 ID 不在轴缓存中或指令 ID 非法

3.4.1.9.6.5 指令类型

非轴运动缓存指令。

3.4.1.9.7 HMC_GetCmdState（获取轴指令状态）

3.4.1.9.7.1 函数原型

USINT HMC_GetCmdState (UDINT uiAxisCmdID)

3.4.1.9.7.2 功能

根据轴指令 ID，返回该轴指令状态。包括以下状态：

- 0：等待运行
- 1：运行中
- 2：已经完成或无指令
- 3：正在投送
- 其它值：参数错误

3.4.1.9.7.3 输入参数

输入参数	类型	参数描述
uiAxisCmdID	UDINT	轴指令 ID

3.4.1.9.7.4 返回值

返回值	类型	参数描述
0	USINT	等待运行：已经被投放到运动缓存中但尚未开始执行
1	USINT	运行中
2	USINT	已经完成或无指令
3	USINT	正在投送：正在向运动缓存投送
其他值	USINT	错误码（0xFF 为用户参数错误）

3.4.1.9.7.5 指令类型

非轴运动缓存指令。

3.4.1.9.8 HMC_WaitCmdFinish（等待指令完成）

3.4.1.9.8.1 函数原型

USINT HMC_WaitCmdFinish (UDINT uiAxisCmdID)

3.4.1.9.8.2 功能

等待直到“某运动指令运行完成后”才返回。在该指令未执行完成之前，HMC_WaitCmdFinish 会一直处于阻塞，不返回。

3.4.1.9.8.3 输入参数

输入参数	类型	参数描述
uiAxisCmdID	UDINT	轴指令 ID

3.4.1.9.8.4 返回值

返回值	类型	参数描述
0	USINT	该指令运行完成
其他值	USINT	错误码（0xFF 为轴指令 ID 非法）

3.4.1.9.8.5 指令类型

非轴运动缓存指令。

3.4.1.9.9 HMC_WaitCmdLoaded（等待指令装载）

3.4.1.9.9.1 函数原型

USINT HMC_WaitCmdLoaded (UDINT uiAxisCmdID)

3.4.1.9.9.2 功能

等待直到“某运动指令（uiAxisCmdID）装载后”即指令刚开始执行才返回。在该指令装载之前，HMC_WaitCmdLoaded 会一直处于阻塞，不返回。指令装载指该指令成功投送至轴运动缓存中。

3.4.1.9.9.3 输入参数

输入参数	类型	参数描述
uiAxisCmdID	UDINT	轴指令 ID

3.4.1.9.9.4 返回值

返回值	类型	参数描述
0	USINT	该指令开始运行
其他值	USINT	错误码（0xFF 为轴指令 ID 非法）

3.4.1.9.9.5 指令类型

非轴运动缓存指令。

3.4.1.9.10 HMC_WaitAxisIdle（等待轴空闲）

3.4.1.9.10.1 函数原型

USINT HMC_WaitAxisIdle (USINT uiAxis)

3.4.1.9.10.2 功能

等待直到“某轴（ucAxisID）的运动指令队列为空”才返回，否则一直处于阻塞状态，不返回。

3.4.1.9.10.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.4.1.9.10.4 返回值

返回值	类型	参数描述
0	USINT	该轴空闲
其他值	USINT	错误码（0x17 为轴号超范围）

3.4.1.9.10.5 指令类型

非轴运动缓存指令。

3.4.1.9.11 HMC_GetCmdBufferState（获取轴指令缓存状态）

3.4.1.9.11.1 函数原型

UDINT HMC_GetCmdBufferState (USINT uiAxis)

3.4.1.9.11.2 功能

获取某轴（ucAxis）的指令缓存状态。判断当前轴缓存是否未滿，并返回该轴缓存状态；主要是为了防止“程序阻塞不往下执行”的现象发生。

3.4.1.9.11.3 输入参数

输入参数	类型	参数描述
ucAxis	USINT	轴号

3.4.1.9.11.4 返回值

返回值	类型	参数描述
0	UDINT	指令缓存未满，可以正常投放指令
1	UDINT	指令缓存已满，若投放指令则会出现阻塞现象
2	UDINT	指令缓存被锁定（当前有指令正在投放进该缓存中）
0xFFFFFFFF	UDINT	函数参数错误

3.4.1.9.11.5 指令类型

非轴运动缓存指令。

- 不支持的模块版本

MC1008-A01、MC1008-A02、MC1008-B01、MC1004-A01、MC1002R-A01

3.4.1.10 故障清除指令

3.4.1.10.1 HMC_ClearAxisErr（消除轴错误）

3.4.1.10.1.1 函数原型

UDINT HMC_ClearAxisErr (USINT uiAxis)

3.4.1.10.1.2 功能

清除某个轴的错误，设置某个轴 AxisStatus=0。

清除错误前，需先解除错误才可清除成功，否则无法清除错误。

3.4.1.10.1.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.4.1.10.1.4 返回值

返回值	类型	参数描述
0	UDINT	成功
其他值	UDINT	错误码（0xFFFFFFFF 为轴号超范围）

3.4.1.10.1.5 指令类型

非轴运动缓存指令。

3.4.1.10.2 HMC_ClearAllErr（消除所有轴错误）

3.4.1.10.2.1 函数原型

UDINT HMC_ClearAllErr ()

3.4.1.10.2.2 功能

清除所有轴的错误，设置所有轴的 AxisStatus=0。

清除错误前，需先解除错误才可清除成功，否则无法清除错误。

3.4.1.10.2.3 输入参数

无

3.4.1.10.2.4 返回值

返回值	类型	参数描述
0	UDINT	成功

3.4.1.10.2.5 指令类型

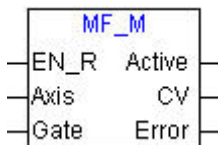
非轴运动缓存指令。

3.4.2 运动控制指令

3.4.2.1 MF_M（测频,5128）

3.4.2.1.1 功能描述

测定由 Axis 指定输入通道的脉冲频率。



MF_M 功能块

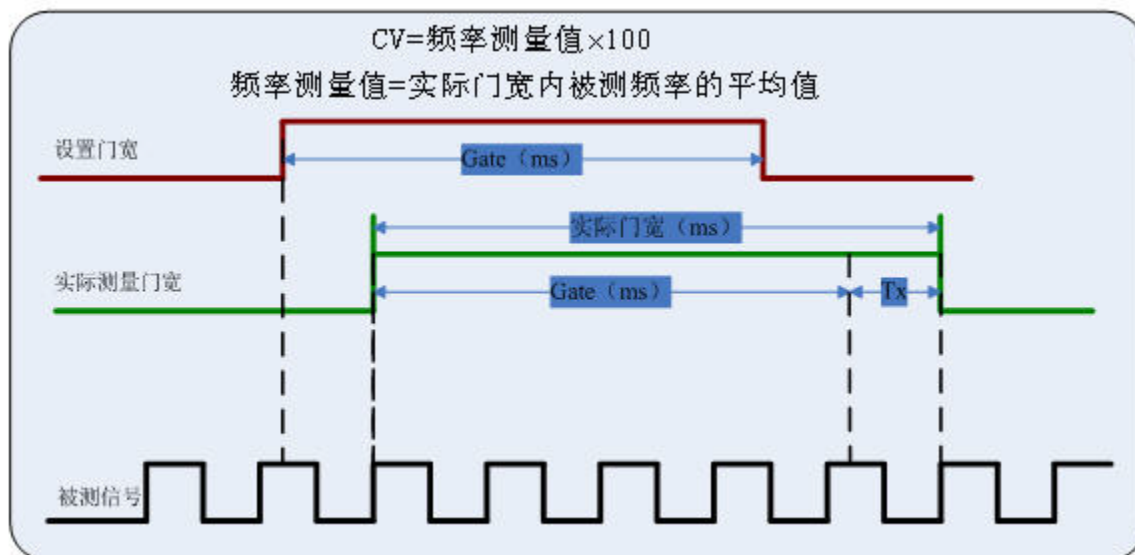
3.4.2.1.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1: 上升沿使能, 高电平有效
Axis	BYTE	输入轴选择	=1、2、3、4 分别表示输入轴为: 轴 1、轴 2、轴 3、轴 4
Gate	WORD	测频门宽	设置范围为 1~100 (ms), 大于 100 时, 以 100 为设置值; 小于 1 时, 以 1 为设置值

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	测频状态标志	0: 测频停止 1: 正在测频
CV	DWORD	当前频率值	1000~10,000,000
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误

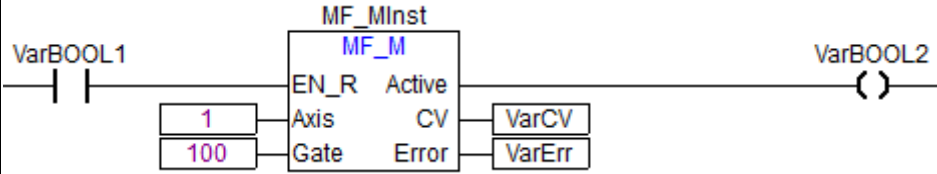
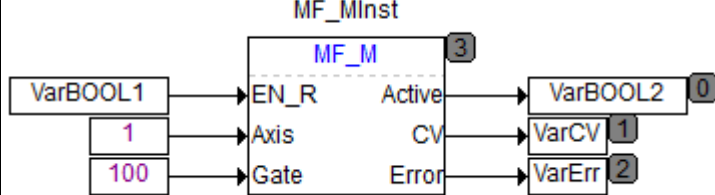
3.4.2.1.3 指令说明

- 该功能块使能信号有效时, 实时检测脉冲频率; 使能信号无效时停止检测, 同时 CV 显示为 0。
- 假如设定门宽范围内, 没有检测到一个完整脉冲, 则自动延长门宽直至检测到下一个边沿为止。
若在 100ms 范围内都无法检测到下一个边沿, 则不再检测, CV 显示为 0。
- 针对同一通道的脉冲输入, 允许高速计数指令和测频指令同时应用。
- $CV \text{ 值} = \text{输入脉冲频率} \times 100$ 。



3.4.2.1.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MF_MInst			MF_M		FALSE
0002	VarCV			DWORD	0	FALSE
0003	VarErr			BYTE	0	FALSE
0004	VarBOOL1			BOOL	FALSE	FALSE
0005	VarBOOL2			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>MF_MInst(EN_R:=VarBOOL1, Axis:=1, Gate:=100, Active=> VarBOOL2, CV=>VarCV, Error=>VarErr);</pre>
连续功能图 (CFC)	

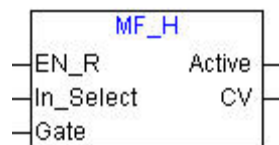
程序说明：

EN_R 置位并保持时，轴 1 通道接收到脉冲，CV 显示当前接收到脉冲的脉冲频率值。

3.4.2.2 MF_H（测频,5106/5107/5108/5109/5118）

3.4.2.2.1 功能描述

测定由 In_Select 指定输入通道的脉冲频率，有效测量范围 10 Hz~100 kHz。



MF_H 功能块

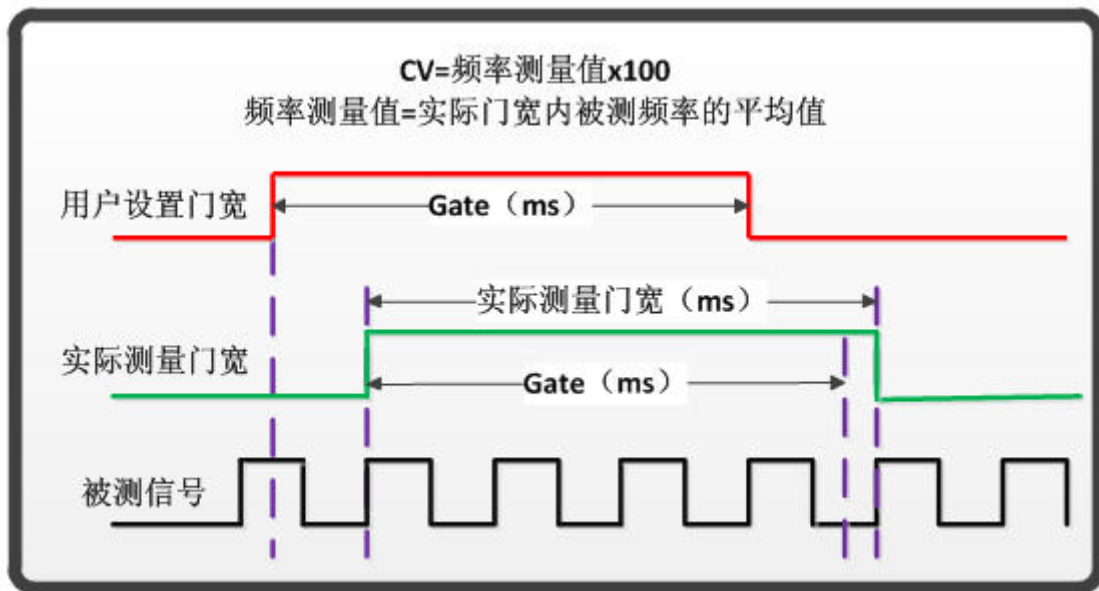
3.4.2.2.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
In_Select	BYTE	输入通道选择（如模块只支持 4 通道测频，则为前 4 个通道）	=1: I0.0 为脉冲输入端 =2: I0.1 为脉冲输入端 =3: I0.2 为脉冲输入端 =4: I0.3 为脉冲输入端 =5: I1.4 为脉冲输入端 =6: I1.5 为脉冲输入端 =7: I1.6 为脉冲输入端 =8: I1.7 为脉冲输入端
Gate	WORD	测频门宽	设置范围为 1~2,000 (ms) 大于 2,000 时，以 2,000 为设置值 小于 1 时，以 1 为设置值

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	测频状态标志	0: 测频停止 1: 正在测频
CV	DWORD	当前频率值	0~10,000,000（分辨率：0.01Hz），数据小于 1000 没有意义。

3.4.2.2.3 指令说明

- 该功能块使能信号有效时，实时检测脉冲频率；使能信号无效时停止检测，同时 CV 显示为 0。
- 假如设定门宽范围内，没有检测到一个完整脉冲，则自动延长门宽直至检测到下一个边沿为止。若在 2000ms 范围内都无法检测到下一个边沿，则不再检测，CV 显示为 0。
- 针对同一通道的脉冲输入，允许高速计数指令和测频指令同时应用。
- $CV \text{ 值} = \text{频率测量值} \times 100$ 。

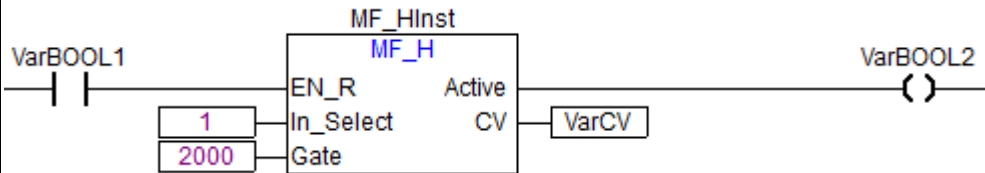


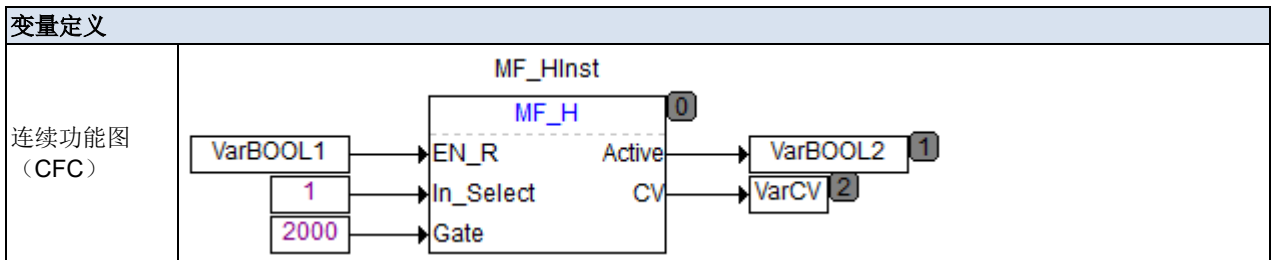
模块支持的测频通道数说明

模块型号	LE5106	LE5107	LE5108	LE5109	LE5128
MF 通道数	4	8	4	8	8

3.4.2.2.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MF_HInst			MF_H		FALSE
0002	VarCV			DWORD	0	FALSE
0003	VarBOOL1			BOOL	FALSE	FALSE
0004	VarBOOL2			BOOL	FALSE	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> MF_HInst(EN_R:=VarBOOL1, In_Select:=1, Gate:=2000, Active=> VarBOOL2, CV=>VarCV); </pre>



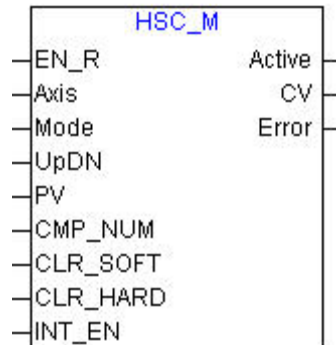
程序说明：

EN_R 置位并保持时，I0.0 通道接收到一 50Hz 脉冲，CV 显示当前接收到脉冲的脉冲频率值，VarCV=4999(约 50Hz)。

3.4.2.3 HSC_M（高速计数,5128）

3.4.2.3.1 功能描述

高速计数功能块可以实现 A、B 双相计数或单相增减计数。用户可以设定多达 21 个不同计数目标值，且达到设定值后可以选择是否启动中断。在 A、B 相计数时，可接收 $90^\circ \pm 60^\circ$ 范围的相差。计数值范围要求在 -1073741824~1073741823。



HSC_M 功能块

3.4.2.3.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1：上升沿使能，高电平保持
Axis	BYTE	输入轴选择	单相（bit5=0）： =1、2、3、4 分别表示输入轴为：轴 1、轴 2、轴 3、轴 4 双相（bit5=1）： =1、3 分别表示输入轴为：轴 1、轴 3 若输入 2 仍对应轴 1 输入，若输入 4 则仍对应轴 3 输入

输入参数	数据类型	功能描述	参数值说明
Mode	BYTE	模式选择	详见注 1
UpDN	BOOL	单相计数时，计数方向控制	0：向上计数 1：向下计数
PV	ARRAY[21] OF DINT	设定计数值	-1073741824~1073741823 ARRAY[0]~ARRAY[20]
CMP_NUM	BYTE	比较个数	1~21
CLR_SOFT	BOOL	软件清零	由 Mode 设定（bit2=0）为软件清零（若 bit4=0：上升沿清零、bit4=1：下降沿清零）时实现 CV 值清零。
CLR_HARD	BYTE	硬件清零	由 Mode 设定（bit2=1）为硬件清零（若 bit4=0：上升沿清零、bit4=1：下降沿清零）时实现 CV 值清零。 0：无效；1~16：分别表示硬件清零通道为 I0.0 ~I1.7
INT_EN	BOOL	中断使能	0：计数到达后不中断 1：计数到达后中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	计数到达标志	0：计数未到设定值 1：计数到达设定值
CV	DINT	当前计数值	-1073741824~1073741823
Error	BYTE	错误信息	=0：正确 =1：输入通道被占用 =2：通道选择错误 =3：比较个数设置错误 =4：PV 设置错误 =5：Mode 设置错误 =6：硬件清零通道设置错误 =7：设置的计数值错误（超出 31 位有效值）

注 1：Mode 参数详细说明

位序号	功能概述	说明
Bit7	-	备用
Bit6	计数连续性选择	0：计数到 PV 值后停止计数 1：计数到 PV 值后继续计数
Bit5	单双相计数选择	0：单相计数 1：双相计数
Bit4	有效清零沿选择	0：上升沿清零 1：下降沿清零
Bit3	计数方向选择	0：软件控制，参数 UpDN 控制计数方向 1：硬件控制，指定 DI 通道（I0.4、I0.5、I0.6、I0.7）控制计数方向
Bit2	清零方式选择	0：软件清零，参数 CLR_SOFT 控制清零 1：硬件清零，CLR_HARD 指定 DI 通道控制清零
Bit1~Bit0	倍频模式选择	00：双（单）相 1 倍频计数 01：双（单）相 2 倍频计数 10：双相 4 倍频计数，单相时无效 11：单双相均无效

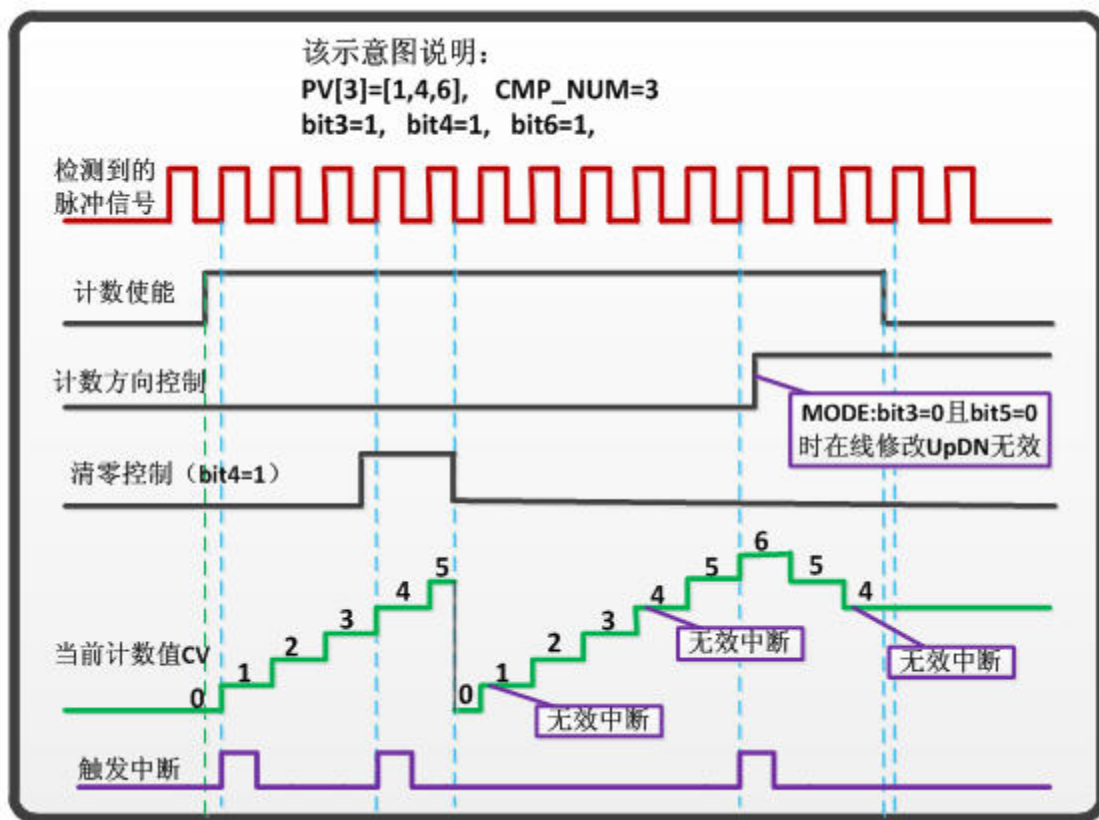
注 2：清零说明

用户可选择硬件清零、软件清零以及有效清零跳变沿类型（上升沿有效、下降沿有效）；若选择硬件

清零时，通道可用户指定。清零时全局清零（相当于定义当前输入点为 0 点）。

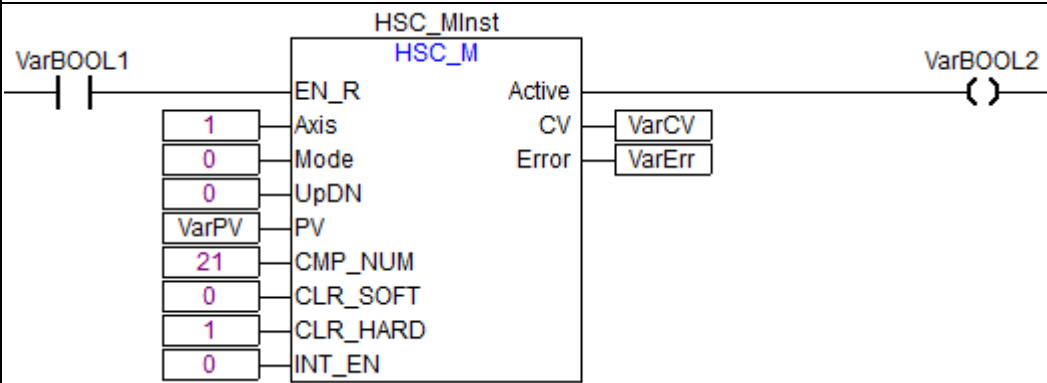
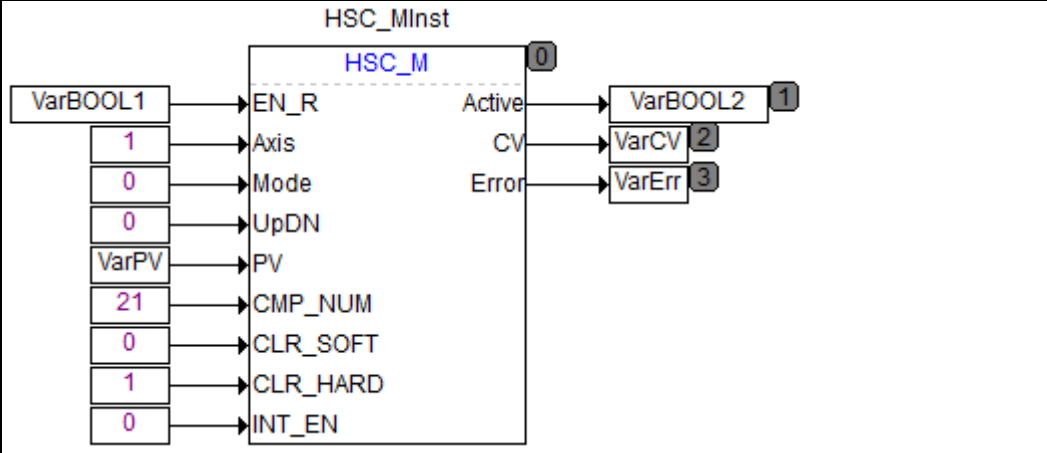
注 3: HSC 指令与内部轴位置关系说明

- 当连接外部编码器，将高速脉冲输入当做一个编码器输入轴，需要实时记录编码器位置时，在启动时刻，就要将 HSC 指令启动，并一直执行。
- HSC 的计数值与特殊寄存器内记录轴位置的参数无关，两个值相互独立。特殊寄存器记录轴位置的值在 HSC 指令启动后开始记录，特殊寄存器的计数方向仅与当前时刻 HSC 的计数方向有关，倍频模式只与第一 HSC 的倍频参数有关，其由 DEFPOS 或 HOME 指令设置或清 0。HSC 计数值只与 HSC 指令设置有关。



3.4.2.3.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HSC_MInst			HSC_M		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarPV			ARRAY[0..20] OF DINT		FALSE
0005	VarCV			DINT	0	FALSE
0006	VarErr			BYTE	0	FALSE

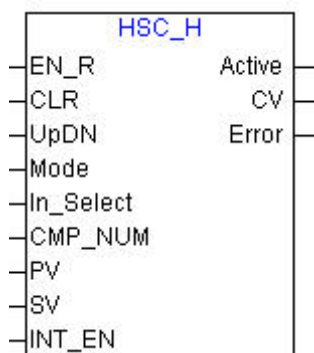
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> HSC_MInst(EN_R:=VarBOOL1, Axis:=1, Mode:=0, UpDN:=0, PV:=VarPV, CMP_NUM:=21, CLR_SOFT:=0, CLR_HARD:=1, INT_EN:=0, Active=>VarBOOL2, CV=>VarCV, Error=>VarErr); </pre>
连续功能图 (CFC)	

3.4.2.4 HSC_H（高速计数,5106/5107/5108/5109/5118）

在不同的 CPU 模块中，HSC 指令有所不同。

3.4.2.4.1 功能描述

高速计数功能块可以实现 A、B 双相计数或单相增减计数。用户可以设定多达 21 个不同计数目标值，且达到设定值后可以选择是否启动中断。在 A、B 相计数时，可接收 $90^{\circ} \pm 60^{\circ}$ 范围的相差。计数值范围要求在 -2147483648~2147483647。



HSC_H 功能块

3.4.2.4.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，电平保持
CLR	BOOL	清零	0→1：上升沿清零
UpDN	BOOL	计数方向控制	0：向上计数 1：向下计数
Mode	BYTE	模式选择	详见注 1
In_Select	BYTE	输入点选择	详见注 2
CMP_NUM	BYTE	比较个数	1~21
PV	ARRAY[21] OF DINT	设定计数值	-2147483648~2147483647 ARRAY[0]~ARRAY[20]
SV	DINT	触发后 CV 改变值	-2147483648~2147483647
INT_EN	BOOL	中断使能	0：计数到达后不中断 1：计数到达后中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	计数到达标志	0：计数未到设定值 1：计数到达设定值
CV	DINT	当前计数值	-2147483648~2147483647
Error	BYTE	错误信息	详见注 3

注 1: Mode 参数详细说明

位序号	功能概述	说明
Bit7~Bit4	-	备用
Bit3	计数方向选择	0: 软件控制, 由功能块参数 UpDN 的值控制计数方向 1: 硬件控制, 指定 DI 通道 (I0.4~I0.7、I2.0~I2.3) 的值控制计数方向
Bit2	清零方式选择	0: Reset 的上升沿实现计数值清零 1: 由对应 DI 通道 (I1.0~I1.3、I2.4~I2.7) 的上升沿实现计数值清零
Bit1~Bit0	倍频模式选择	00: 双(单)相 1 倍频计数 01: 双(单)相 2 倍频计数 10: 对于 LE5118 控制器, 为双相 4 倍频计数, 对于其他控制器, 单双相均无效 11: 单双相均无效

注 2: 输入通道选择 and 对应通道说明

=1: 第 1 路单相计数

=2: 第 2 路单相计数

=3: 第 3 路单相计数

=4: 第 4 路单相计数

=5: 第 5 路单相计数

=6: 第 6 路单相计数

=7: 第 7 路单相计数

=8: 第 8 路单相计数

=21: 第 1 路双相计数

=22: 第 2 路双相计数

=23: 第 3 路双相计数

=24: 第 4 路双相计数

高速计数通道	单相计数 / 双相计数	对应方向通道	方向控制 (仅限单相计数器)	对应清零通道	清零控制
I0.0	第 1 路单相 / 第 1 路双相 A 相	I0.4	第 1 路单相方向控制	I1.0	第 1 路单相清零 / 第 1 路双相清零
I0.1	第 2 路单相 / 第 1 路双相 B 相	I0.5	第 2 路单相方向控制	I1.1	第 2 路单相清零
I0.2	第 3 路单相 / 第 2 路双相 A 相	I0.6	第 3 路单相方向控制	I1.2	第 3 路单相清零 / 第 2 路双相清零
I0.3	第 4 路单相 / 第 2 路双相 B 相	I0.7	第 4 路单相方向控制	I1.3	第 4 路单相清零
I1.4	第 5 路单相 / 第 3 路双相 A 相	I2.0	第 5 路单相方向控制	I2.4	第 5 路单相清零 / 第 3 路双相清零
I1.5	第 6 路单相 / 第 3 路双相 B 相	I2.1	第 6 路单相方向控制	I2.5	第 6 路单相清零
I1.6	第 7 路单相 / 第 4 路双相 A 相	I2.2	第 7 路单相方向控制	I2.6	第 7 路单相清零 / 第 4 路双相清零

高速计数通道	单相计数 / 双相计数	对应方向通道	方向控制 (仅限单相计数器)	对应清零通道	清零控制
I1.7	第 8 路单相 / 第 4 路双相 B 相	I2.3	第 8 路单相方向控制	I2.7	第 8 路单相清零

注 3: Error 错误标志

=0: 正确

=1: 输入通道被占用

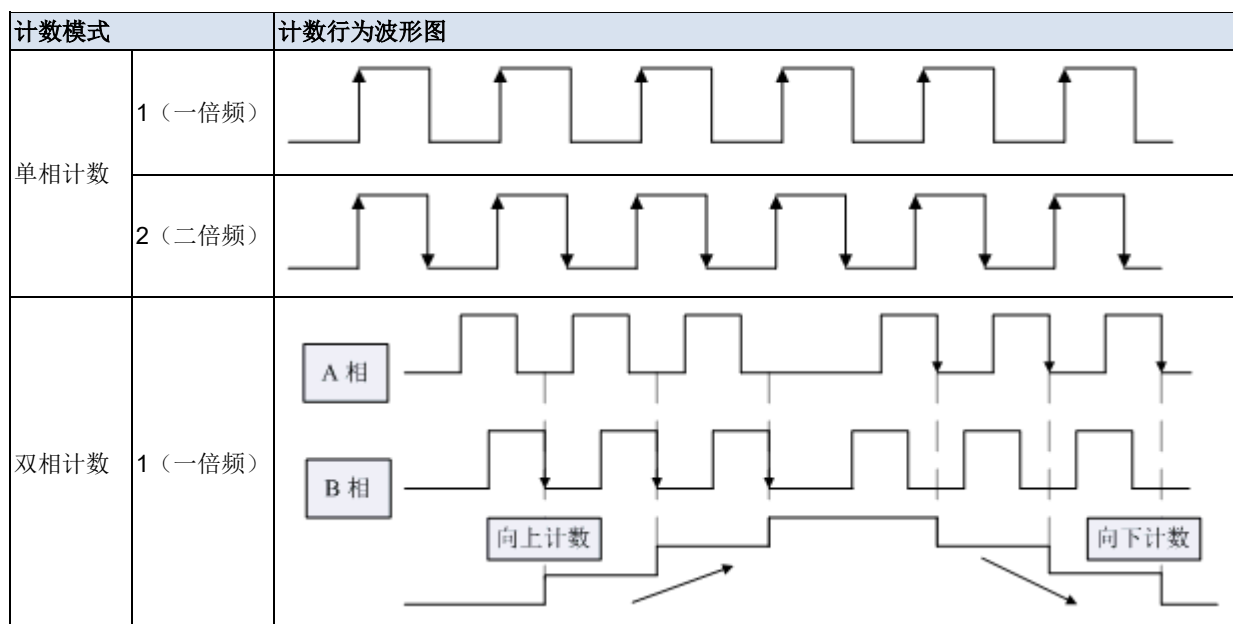
=2: 通道选择错误

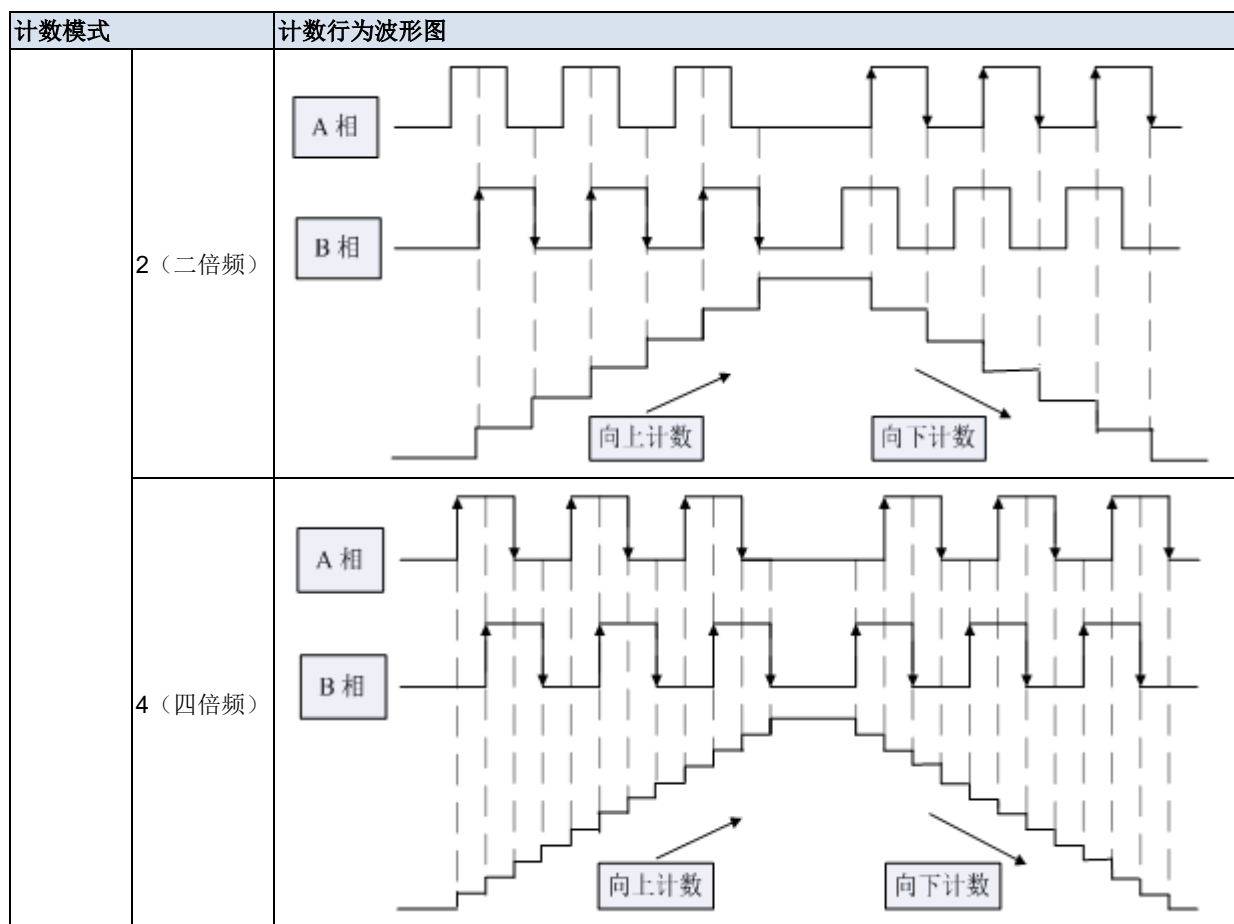
=3: 比较个数设置错误

=4: SV、PV 设置错误

=5: Mode 设置错误

3.4.2.4.3 高速计数计数模式示意图

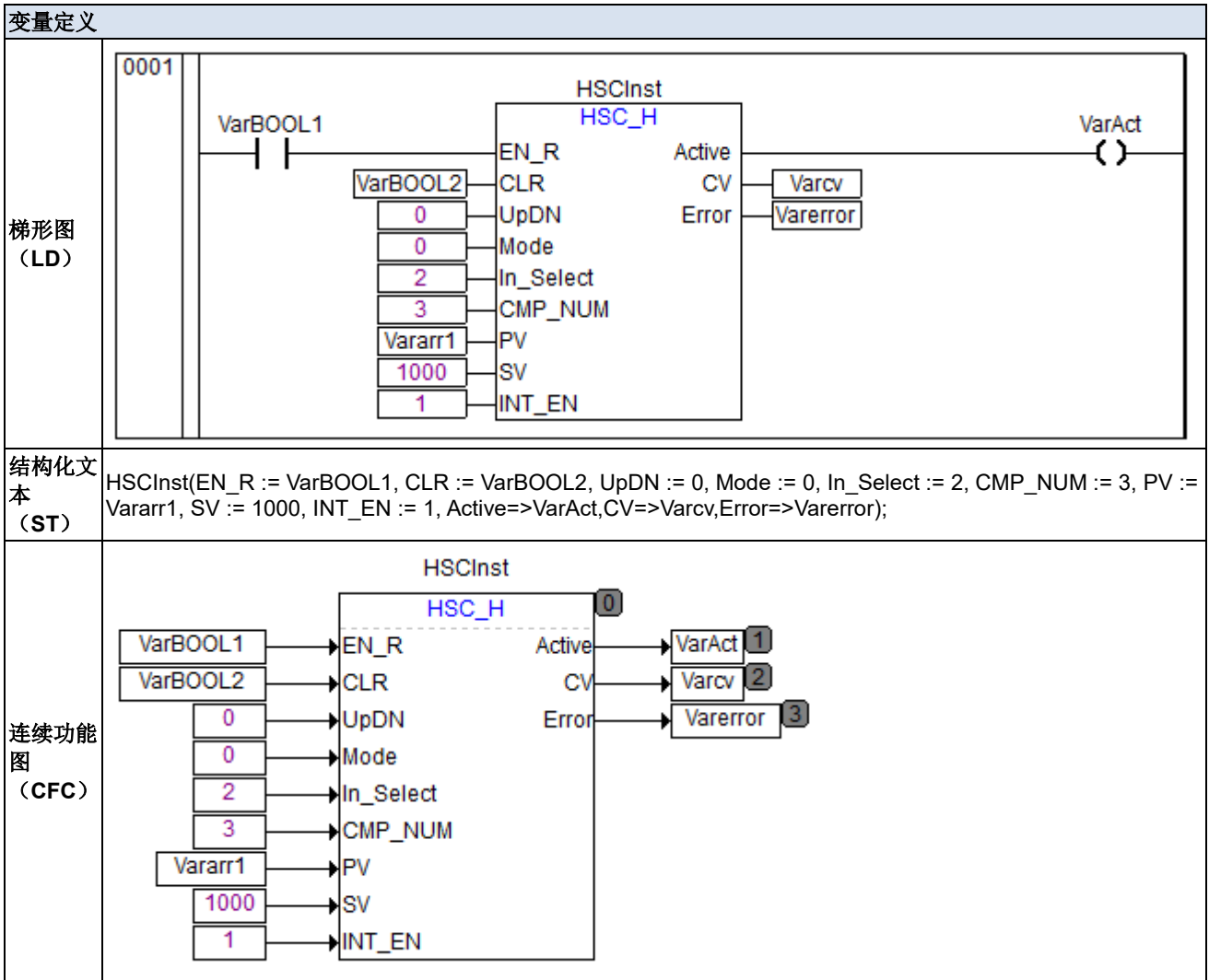




3.4.2.4.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HSCInst			HSC_H		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	Vararr1			ARRAY[0..20] OF DINT		FALSE
0005	VarAct			BOOL	FALSE	FALSE
0006	Varcv			DINT	0	FALSE
0007	Varerror			BYTE	0	FALSE

编程语言 程 序



3.4.2.4.5 程序说明

EN_R 置位并保持时，第 2 路单相计数器向上计数，在计数过程中产生 3 个中断，计数从 1000 开始。CLR 为 1 时，计数值复位。



- 当高速计数器使能端无效时，立即停止高速计数，CV 在重新触发高速计数器时复位。
- 清零端 CLR 由“0”跳变为“1”，即上升沿到来，CV 值变为 0，如果继续有脉冲输入时，CV 值继续从 0 开始累计。
- SV 设定值在高速计数使能后生效，若设定值不合理则报错，报错对应值见 Error 说明。
- 计数方向控制功能端 UpDN 仅在单相计数时有意义。

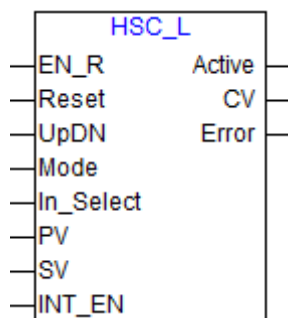
- CV 值等于 2147483647 再往上计数时，CV 值变为-2147483648；同理高速计数指令现在 CV 值等于-2147483648 再往下计数时，CV 值变为 2147483647。
- 单相计数通道支持最高计数频率为 100,000Hz，双相计数通道最高支持计数频率为 50,000Hz。设定计数值 PV 数组中的数据应满足条件：向上计数时， $PV[i+1] > PV[i]$ ；
- 向下计数时， $PV[i+1] < PV[i]$ 否则不能正常计数。
- 每个单相高速计数器占用一个高速输入通道，双相（A/B）高速计数占用两个高速输入通道。若使用双相高速计数器时，则对应两个高速通道的单相计数器无法使用，且双相（A/B）高速计数的经过值、目标值和计数标志存放在对应的第一个高速输入通道的特殊寄存器中。

3.4.2.5 HSC_L（高速计数,5104/5105/5107E/5107L/5109L）

在不同的 CPU 模块中，HSC 指令有所不同。

3.4.2.5.1 功能描述

高速计数功能块可以实现 A、B 双相计数或单相增减计数。计数值达到设定值后可以选择是否启动中断，在 A、B 相计数时，可支持 $90^\circ \pm 60^\circ$ （按占空比 50%计）相差的脉冲输入计数。



HSC_L 功能块

3.4.2.5.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，电平保持
Reset	BOOL	清零	0→1：上升沿清零
UpDN	BOOL	计数方向控制	0：向上计数 1：向下计数
In_Select	BYTE	输入点选择	=1：I0.0 为脉冲输入端（双相计数 A 相），I0.2 为清零端，I0.4 为计数方向控制（双相计数 B 相） =2：I0.1 为脉冲输入端，I0.3 为清零端，I0.5 为计数方向控制
Mode	BYTE	模式选择	详见注 1

输入参数	数据类型	功能描述	参数值说明
PV	DINT	设定计数值	-2147483648~2147483647
SV	DINT	触发后 CV 改变值	-2147483648~2147483647
INT_EN	BOOL	中断使能	0: 计数到达后不中断 1: 计数到达后中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	计数到达标志	0: 计数未到设定值 1: 计数到达设定值
CV	DINT	当前计数值	-2147483648~2147483647
Error	BYTE	错误信息	=0: 正确 =1: In_Select 参数配置错误 =2: mode 参数配置错误 =3: 硬件相关参数 =4: 硬件相关参数 =5: 硬件相关参数 =6: 硬件相关参数 =7: 硬件相关参数 =8: 资源被占用（例如：DI 通道已被配置为脉冲捕获或快速外部中断。）

注 1: Mode 参数详细说明

位序号	功能概述	说明
Bit7~Bit5	-	备用
Bit4	计数方向选择	0: 软件控制, 由功能块参数 UpDN 的值控制计数方向 1: 硬件控制, 指定 DI 通道 (I0.4、I0.5) 的值控制计数方向
Bit3	清零方式选择	0: Reset 的上升沿实现计数值清零 1: 由对应 DI 通道 (I0.2、I0.3) 的上升沿实现计数值清零
Bit2~Bit1	倍频模式选择	00: 单相计数 1 倍频, 双相计数 2 倍频 01: 单相计数 2 倍频, 双相计数 4 倍频 10、11: 单双相均无效
Bit0	单双相计数选择	0: 单相计数 1: 双相计数

3.4.2.5.3 指令说明

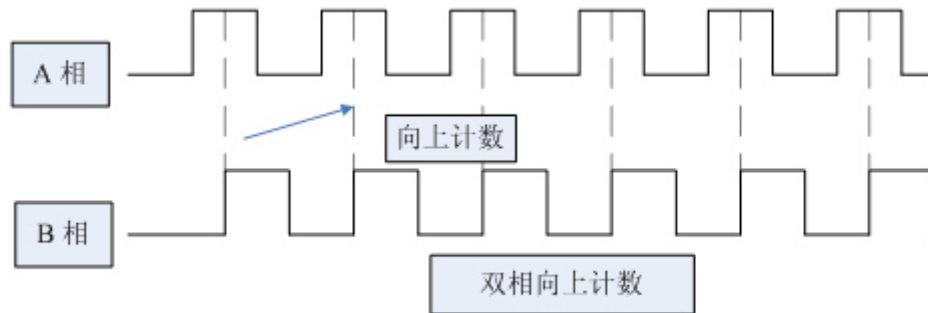
- 清零端 Reset: 当 Reset 由 False 变为 True 后, CV 值变为 0, 如果继续有脉冲输入时, CV 值继续从 0 开始累计, 清零端可以是外部输入点或变量。
- 输入点选择功能端 In_Select: In_Select 有 2 种取值。当 In_Select =1 时, 可以通过设置 mode 的 bit0 选择单相或双相计数; 当 In_Select =2 时, 只有单相计数, mode 的 bit0 只能为 0。

变量设定	0	向上计数
	1	向下计数
外部输入点	低电平	向上计数
	高电平	向下计数

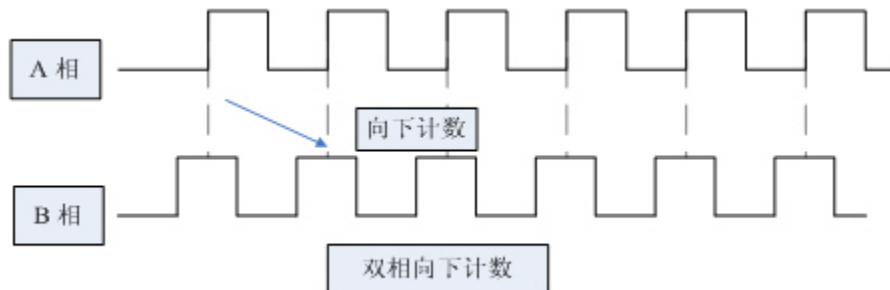
- 单相计数时, 计数方向由 UpDN 输入变量或者外部输入点决定。当 UpDN 不绑定外部输入点时

（即 mode 的 bit4=0），计数方向由 UpDN 端输入变量决定；当 UpDN 选择绑定外部输入点时（即 mode 的 bit4=1），计数方向由绑定的输入端的高低电平决定。

- 双相计数时，UpDN 功能端无意义，计数方向由 A 相和 B 相输入决定。当 A 相超前 B 相时，计数方向为向上计数，当 B 相超前 A 相时，计数方向为向下计数。如下图所示。



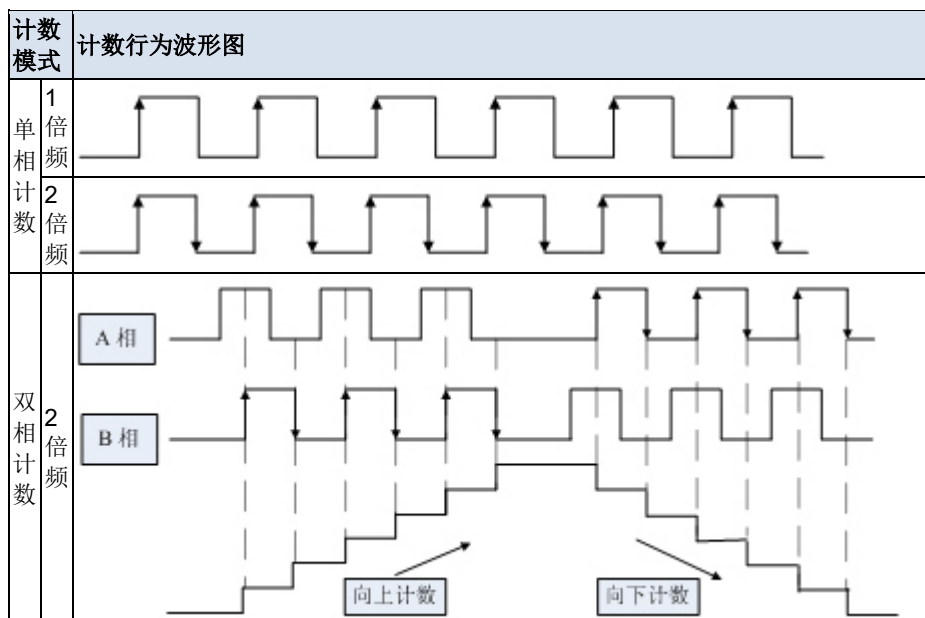
(a)

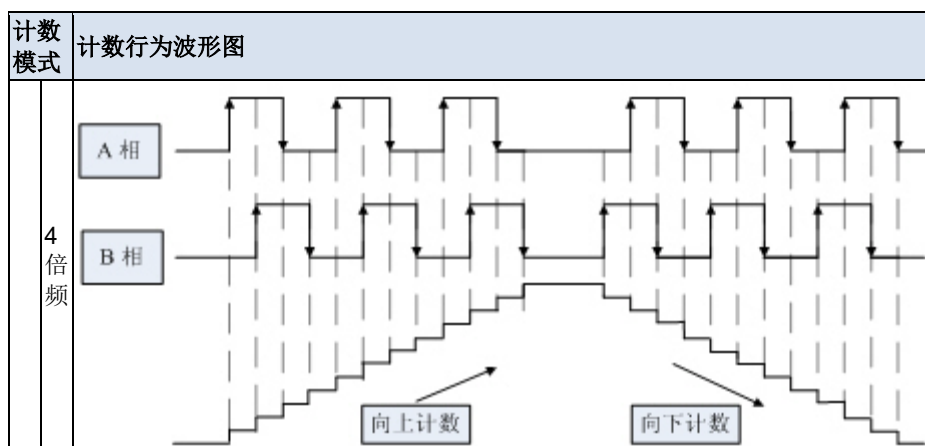


(b)

双向计数示意图

倍频模式功能端 Mode 计数行为图（↑↓表示计数动作产生）。





高速计数指令 CV 值等于 2147483647 再往上计数时，CV 值变为-2147483648；同理高速计数指令现在 CV 值等于-2147483648 再往下计数时，CV 值变为 2147483647。

高数计数器是电平触发的，当前面能量线断掉时，停止高速计数并给高速计数器复位，CV 在重新触发高速计数器时清零。

3.4.2.5.4 特别说明

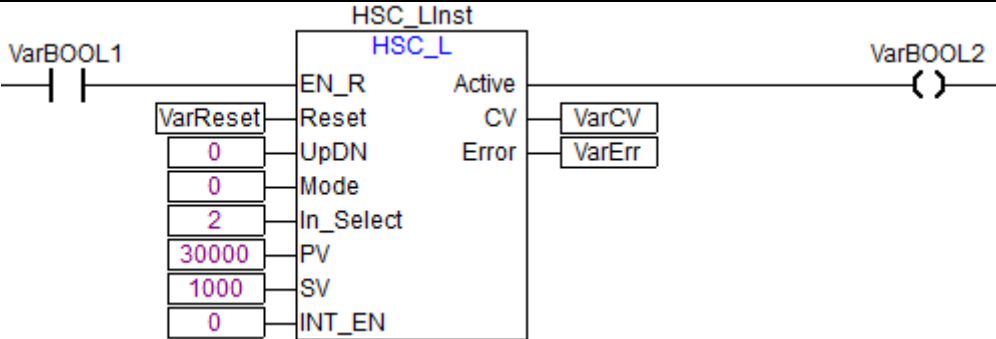
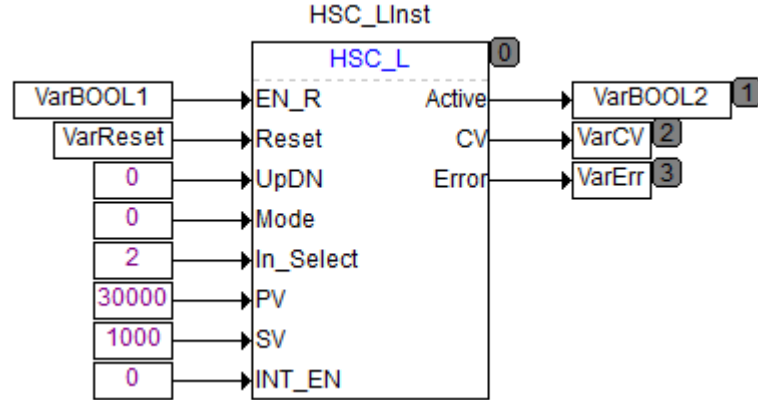
每个单相高速计数器占用一个高速输入通道，每个双相高速计数器占用两个高速输入通道。若使用双相高速计数器时，则对应两个高速通道的单相计数器无法使用。

某一通道已被一个 HSC 功能块占用则其它 HSC 功能块不能再使用此通道，否则会导致计数出错。

HSC 功能块与通道滤波参数（在 AutoThink 界面中配置）两个功能占用 CPU 硬件资源较多，同时开启会导致 CPU 负荷较重，可能出现计数出错，因此在使用中请不要同时开启。

3.4.2.5.5 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HSC_Llnst			HSC_L		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarReset			BOOL	FALSE	FALSE
0005	VarCV			DINT	0	FALSE
0006	VarErr			BYTE	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> HSC_LInst(EN_R:=VarBOOL1, Reset:=VarReset, UpDN:=0, Mode:=0, In_Select:=2, PV:=30000, SV:=1000, INT_EN:=0, Active=> VarBOOL2, CV=>VarCV, Error=>VarErr); </pre>
连续功能图 (CFC)	

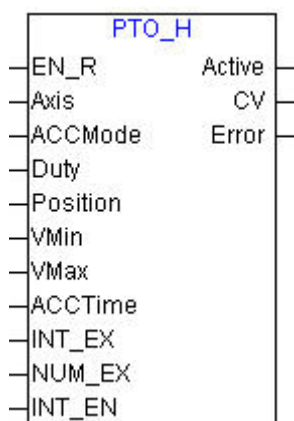
程序说明：

EN_R 置位并保持时，第 2 路单相计数器向上计数，计数范围为 30000 个脉冲，计数初值从 1000 开始。Reset 为 1 时，计数值复位。

3.4.2.6 PTO_H（脉冲串输出,5106/5108/5118）

3.4.2.6.1 功能描述

从由 Axis 指定的端口中输出由 VMin、VMax、ACCTime 指定的频率、Duty 指定的占空比与 Position 设定数目的脉冲。



PTO_H 功能块

3.4.2.6.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出通道选择	=1: 输出轴 1，输出脉冲端 Q0.0 =2: 输出轴 2，输出脉冲端 Q0.1 =3: 输出轴 3，输出脉冲端 Q0.2 =4: 输出轴 4，输出脉冲端 Q0.3
ACCMode	BOOL	加减速模式选择	0: 梯形 1: S 型
Duty	BOOL	占空比	0: 50% 1: 25%
Position	DWORD	输出脉冲数设定	1~4294967295
VMin	DWORD	初始频率	10Hz~100,000Hz
VMax	DWORD	最高频率	10Hz~100,000Hz
ACCTime	WORD	加减速时间	0~32767ms (0 表示匀速)
INT_EX	BYTE	中断输入点选择	0~24 详见注
NUM_EX	DWORD	中断输入后再发个数	0~4294967295
INT_EN	BOOL	中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送 1: 正在发送
CV	DWORD	已发送脉冲个数	1~4294967295
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置输出轴错误 =3: 频率输出错误

注：各通道中断输入说明

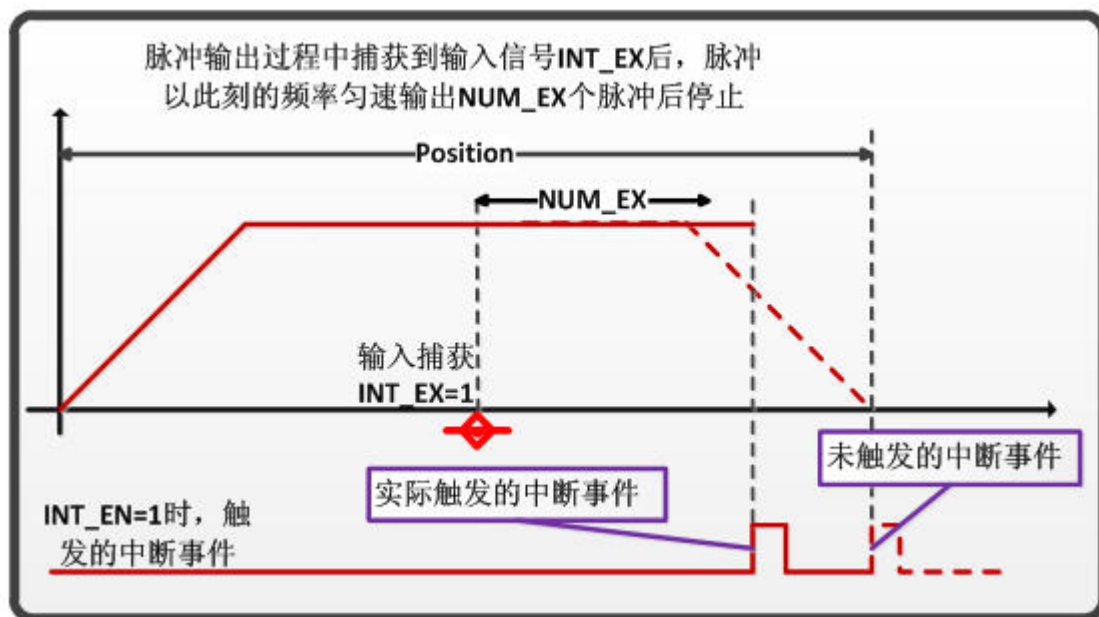
- 触发选择：外部输入点。

- 触发条件：上升沿触发，高电平不保持。
- INT_EX 参数对应含义：0：不选择；1~24：I0.0~I2.7。
- 过程描述：当前使能状态只支持一次有效触发，输出立即以触发上升沿到来时执行 NUM_EX 指定的脉冲数。
- 中断使能：若中断使能（INT_EN=1）则再发脉冲数发送完毕后响应相应中断事件。

特别说明：

出现下列情况之一时匀速输出以 VMax 指定的频率、Duty 指定的占空比与 Position 设定数目的脉冲。

- (1) 加减速时间 ACCTime 为 0。
- (2) 初始频率 VMin、最高频率 VMax 相等。
- (3) 输出脉冲数 Position 小于 6。
- (4) PTO_H 功能的输出变化不会改变特殊寄存器值的状态。

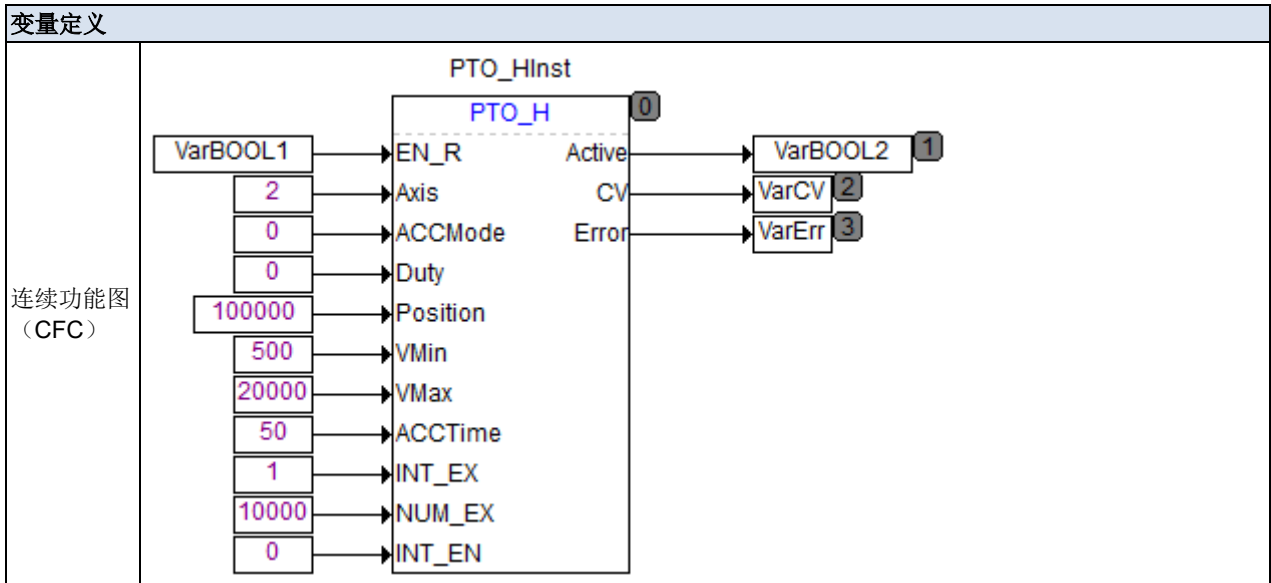


脉冲输出过程

3.4.2.6.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	PTO_HInst			PTO_H		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DWORD	0	FALSE
0005	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	PTO_HInst PTO_H VarBOOL1 EN_R Axis 2 ACCMODE 0 Duty 0 Position 100000 VMin 500 VMax 20000 ACCTime 50 INT_EX 1 NUM_EX 10000 INT_EN 0 Active CV Error VarCV VarErr VarBOOL2 ()
结构化文本 (ST)	PTO_HInst(EN_R:=VarBOOL1, Axis:=2, ACCMODE:=0, Duty:=0, Position:=100000, VMin:=500, VMax:=20000, ACCTime:=50, INT_EX:=1, NUM_EX:=10000, INT_EN:=0, Active=> VarBOOL2, CV=> VarCV, Error=> VarErr);



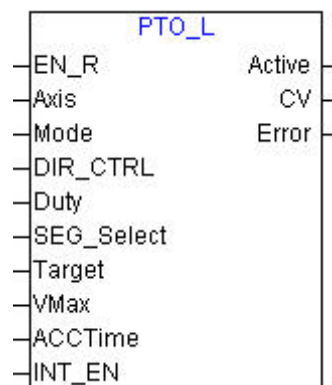
程序说明:

EN_R 置位并保持时, 轴 2 (Q0.1) 发送占空比为 50 的脉冲 100000 个, 按照梯形加速模式从 500Hz 加速到 20000Hz, 加速时间为 50ms。若 I0.0 有脉冲输入, 则产生中断, 中断后再发 10000 个脉冲, 结束。CV 显示出已发送脉冲数量。

3.4.2.7 PTO_L (脉冲串输出,5104)

3.4.2.7.1 功能描述

设定输出脉冲个数和频率后, 由特定的开关量输出点输出已设定个数和频率的脉冲。



PTO_L 功能块

3.4.2.7.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，电平保持
Axis	BYTE	输出轴选择	=1: 轴 1 输出脉冲（包括方向轴） =2: 轴 2 输出脉冲（包括方向轴）
Mode	BYTE	动作模式	动作模式及输出逻辑关系 1: 增量型 PLS+SIGN（正向 OFF/反向 ON） 2: 增量型 PLS+SIGN（正向 ON/反向 OFF）
DIR_CTRL	BOOL	输出方向点是否关联的输入端配置	0: 不关联 1: 关联
Duty	BOOL	占空比	0: 50% 1: 25%
SEG_Select	BYTE	分段数 ^{■1} 选择	SEG_SELECT =0: 10 段 1: 30 段 2: 60 段 3: 100 段
Target	DINT	目标位置设定	-2147483648~2147483647，最少脉冲数的绝对值不能小于 20
VMax	DWORD	最高频率	10Hz~20,000Hz
ACCTime	WORD	加减速时间	0~32767ms（0 表示匀速）
INT_EN	BOOL	中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送（脉冲发送完毕） 1: 正在发送
CV	DINT	已发送脉冲个数	-2147483648~2147483647
Error	BYTE	错误信息	详见注 2

■ 注 1：段数

段数表示：输出频率加速到设定频率值时理论上需要经过的频率跳变次数，选择较大的段数会使加减速过程更加平滑，但可能造成加速时间误差较大。

■ 注 2：Error 错误标志

- =0: 正确
- =1: 此路通道被 PWM 占用，直接退出功能块
- =2: 配置输出轴错误，直接退出功能块
- =3: 配置动作模式错误，直接退出功能块
- =4: 配置加速段数错误，直接退出功能块
- =5: 配置输出脉冲数错误，直接退出功能块
- =6: 配置输出频率错误，直接退出功能块
- =7: 配置加减速时间错误，直接退出功能块

3.4.2.7.3 指令说明

1. 脉冲输出控制指令带有方向端，运动控制模块为 2 个轴；描述:当使能信号由 OFF 变成 ON 时，从指定通道输出脉冲。

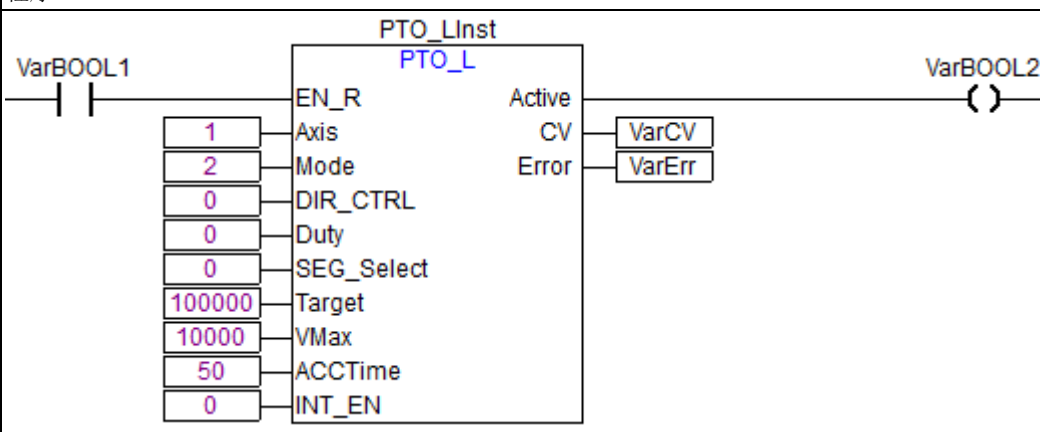
输出轴选择	脉冲输出通道	脉冲输出方向通道
1	Q0.0	Q0.2
2	Q0.1	Q0.3

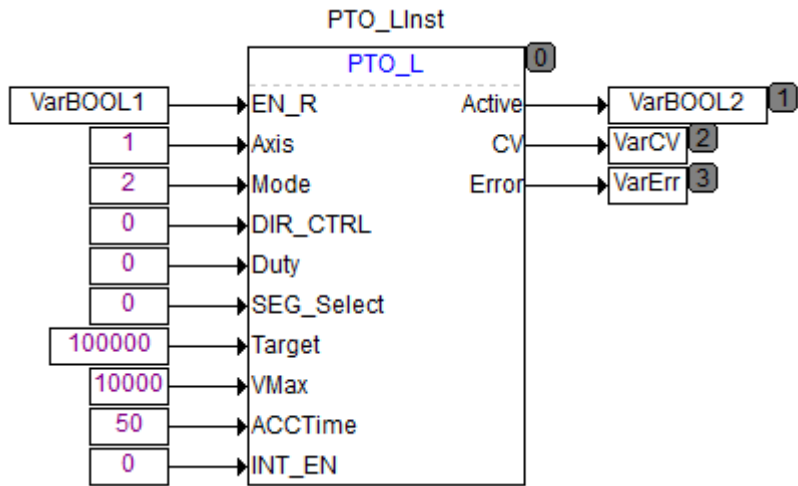
2. 动作模式的增量型以及脉冲发送方式；增量型相对值控制:输出由目标值设定的数量的脉冲。

选择方式 目标值	PLS+SIGN 正向: OFF 反向: ON	PLS+SIGN 正向: ON 反向: OFF	经过值
正数	脉冲输出: ON 方向输出: OFF	脉冲输出: ON 方向输出: ON	递增
负数	脉冲输出: ON 方向输出: ON	脉冲输出: ON 方向输出: OFF	递减

3.4.2.7.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	PTO_LInst			PTO_L		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DINT	0	FALSE
0005	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>PTO_LInst(EN_R:=VarBOOL1, Axis:=1, Mode:=2, DIR_CTRL:=0, Duty:=0,</pre>

变量定义	
	<pre> SEG_Select:=0, Target:=100000, VMax:=10000, ACCTime:=50, INT_EN:=0, Active=> VarBOOL2, CV=> VarCV, Error=> VarErr); </pre>
连续功能图 (CFC)	

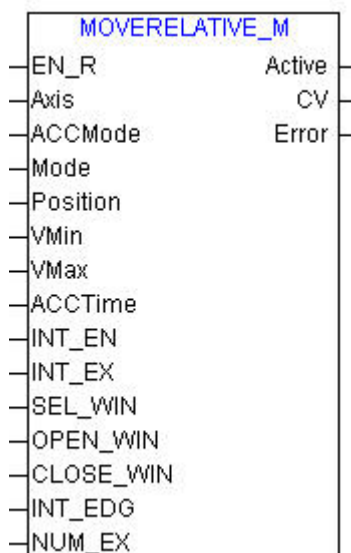
程序说明:

EN_R 置位并保持时, 轴 1 (Q0.0) 按照增量型 PLS+SIGN (正向 ON/反向 OFF) 发送占空比为 50 的脉冲 (Q0.2 用于控制电机转动方向), 共发送 100000 个, 最高输出频率为 10000Hz, 加减速时间为 50ms, CV 显示出已发送脉冲数量。

3.4.2.8 MOVERELATIVE_M (相对定位,5128)

3.4.2.8.1 功能描述

用于控制电机进行定位, 以当前点作为参考点输出已设定个数和频率的脉冲, 达到定位的功能。本文暂不支持运行中修改 MOVERELATIVE 的参数值, 暂不支持脉冲输出过程触发过程值匹配中断。



MOVERELATIVE_M 功能块

3.4.2.8.2 参数说明

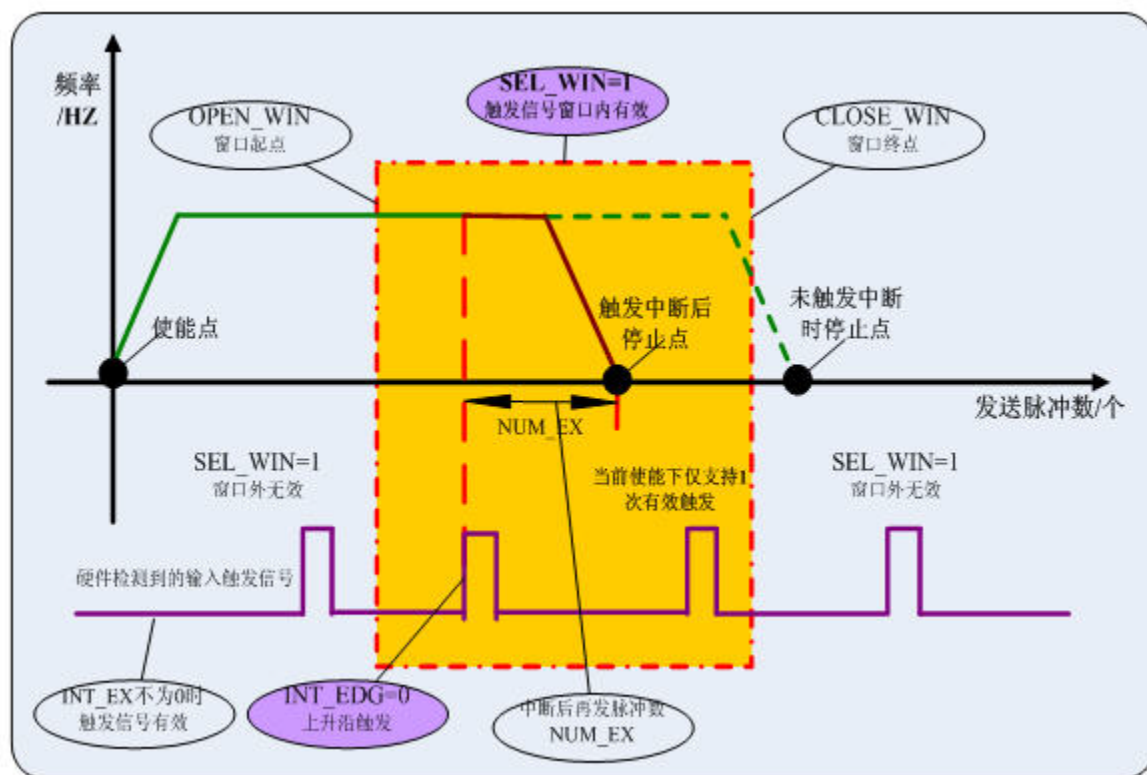
输入参数	数据类型	功能描述	参数值说明	
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持	
Axis	BYTE	输出轴选择	PLS+SIGN 模式 =5: 输出轴 5, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 =6: 输出轴 6, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 =7: 输出轴 7, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 =8: 输出轴 8, 脉冲输出端 Q0.3, 脉冲方向 Q0.7 CW/CCW 模式 =5、7 分别表示输出为: 轴 5、轴 7 若输出 6 仍对应轴 5 输出, 若输出 8 则仍对应轴 7 输出	
ACCMode	BOOL	加减速模式选择	0: 梯形曲线 1: S 型曲线	
Mode	BOOL	脉冲模式控制	0: PLS+SIGN 模式 1: CW/CCW 模式	
Position	DINT	相对位置设定	-1073741824~1073741823	
VMin	DWORD	初始频率	10 Hz~100, 000 Hz	
VMax	DWORD	最高频率	10 Hz~100, 000 Hz	
ACCTime	WORD	加减速时间	0~32, 767 ms (0 表示匀速)	
INT_EN	BOOL	输出结束后, 是否中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断	
INT_EX	BYTE	输出过程中中断输入点选择	0~16 详见注, 若该参数为 0, 则以下与中断相关参数均无效	
SEL_WIN	BYTE	窗口选择	约束 INT_EX 硬件参数的有效性 0: 不使用窗口 1: 窗口内有效 2: 窗口外有效	INT_EX=0 时该组参数均无效, 功能示意图见操作示例。
OPEN_WIN	DINT	窗口 1 相对值	-1073741824~1073741823	
CLOSE_WIN	DINT	窗口 2 相对值	-1073741824~1073741823	
INT_EDG	BOOL	INT_EX 中断跳变沿选择	0: 上升沿触发 1: 下降沿触	

输入参数	数据类型	功能描述	参数值说明
			发
NUM_EX	DWORD	中断输入后再发个数	0~2147483647

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送 1: 正在发送
CV	DINT	已发送脉冲个数	-1073741824~1073741823
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误 =3: 频率输出错误 =4: 过程中断输入点选择错误 =5: 相对位置设定参数超范围 =6: 加减速时间参数设置错误

3.4.2.8.3 操作示例

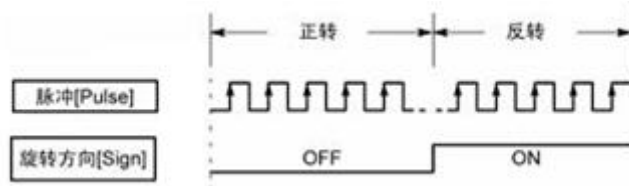
在输出过程中选择外部中断触发时，与 INT_EX 命令相关参数的执行示意图如下。



INT_EX 命令相关参数的执行示意图

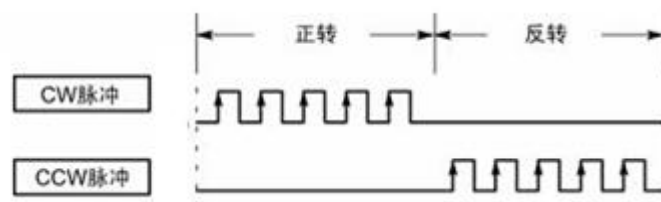
3.4.2.8.4 说明

■ PLS+SIGN 输出模式



速度指定用脉冲输出，旋转方向指定用 ON/OFF 进行指定，在该模式下，旋转方向 OFF 对应电机正转。

■ CW/CCW 输出模式



该方式作用下，正转反转都用脉冲的双路脉冲控制。

3.4.2.8.5 示例

当前位置为 5000 时，作为目标值+7000，执行脉冲输出指令时；

脉冲+方向：从脉冲输出轴输出 7000 脉冲，方向输出低电平；

CW/CCW：CW 轴输出 7000 脉冲，CCW 输出低电平；

以上两种方式后当前位置为 12000。

注：各轴 INT_EX 中断输入说明

- 触发选择：硬件输入触发，触发点用户指定。0：不选择；1~16：I0.0~I1.7；
- 触发条件：可选上升沿或下降沿触发，高电平不保持；
- 过程描述：当前使能状态只支持一次有效触发，触发后按照设置参数输出 NUM_EX 指定的脉冲数。

3.4.2.8.6 特别说明

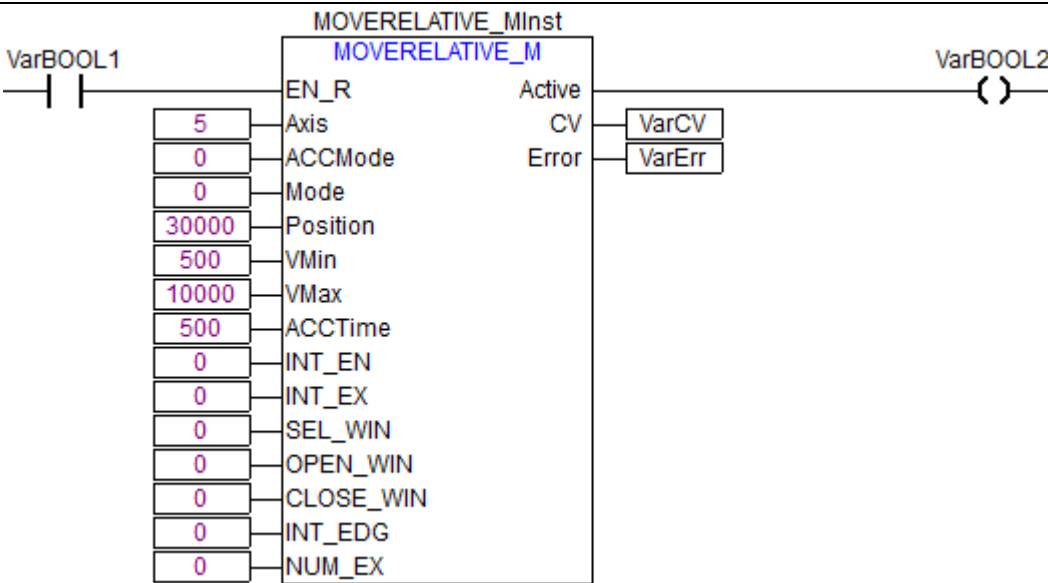
出现下列情况之一时以 VMax 匀速输出指定数目的脉冲。

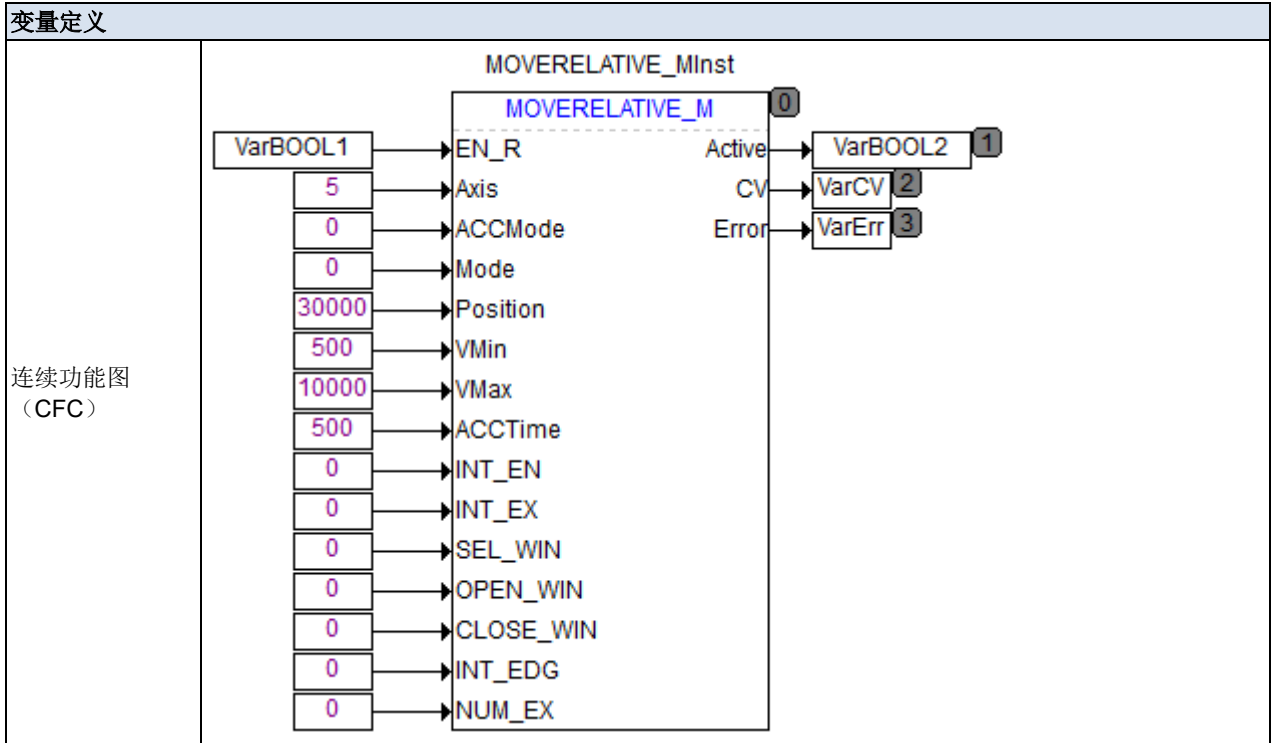
- 加减速时间 ACCTime 为 0；
- 初始频率 VMin、最高频率 VMax 相等；

- 输出脉冲数 Position 小于 6。

3.4.2.8.7 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MOVERELATIVE_MInst			MOVERELATIVE_M		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DINT	0	FALSE
0005	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> MOVERELATIVE_MInst(EN_R:=VarBOOL1, Axis:=5, ACCMode:=0, Mode:=0, Position:=30000, VMin:=500, VMax:=10000, ACCTime:=500, INT_EN:=0, INT_EX:=0, SEL_WIN:=0, OPEN_WIN:=0, CLOSE_WIN:=0, INT_EDG:=0, NUM_EX:=0, Active=> VarBOOL2, CV=> VarCV, Error=> VarErr); </pre>



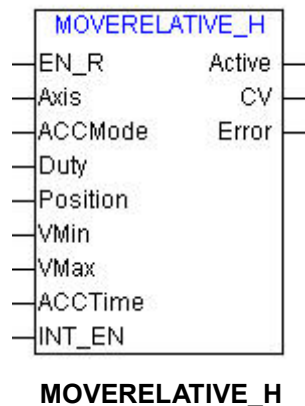
程序说明：

EN_R 置位并保持时，轴 5（Q0.0 和 Q0.4）按照双脉冲模式发送占空比为 50 的脉冲，共发送 30000 个，初始频率为 500Hz，最高输出频率为 10000Hz，加减速时间为 500ms，梯形加减速模式，CV 显示出已发送脉冲数量，无中断产生。

3.4.2.9 MOVERELATIVE_H（相对定位,5106/5108/5118）

3.4.2.9.1 功能描述

用于控制电机进行定位，以当前点作为参考点输出已设定个数和频率的脉冲，达到定位的功能。



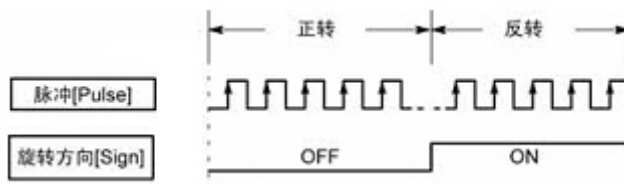
3.4.2.9.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出通道选择	脉冲+方向模式 1: 输出轴 1, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 2: 输出轴 2, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 3: 输出轴 3, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 4: 输出轴 4, 脉冲输出端 Q0.3, 脉冲方向 Q0.7 CW/CCW 模式 5: 输出轴 1, 脉冲输出端 (CW) Q0.0, 脉冲输出端 (CCW) Q0.1 6: 输出轴 2, 脉冲输出端 (CW) Q0.2, 脉冲输出端 (CCW) Q0.3
ACCMode	BOOL	加速模式选择	0: 梯形曲线 1: S 型曲线
Duty	BOOL	占空比	0: 50% 1: 25%
Position	DINT	输出脉冲数设定	-2147483648~2147483647
VMin	DWORD	初始频率	10Hz~100,000Hz
VMax	DWORD	最高频率	10Hz~100,000Hz
ACCTime	WORD	加减速时间	0~32767ms (0 表示匀速)
INT_EN	BOOL	中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送 1: 正在发送
CV	DINT	已发送脉冲个数	-2147483648~2147483647
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置输出轴错误 =3: 频率输出错误

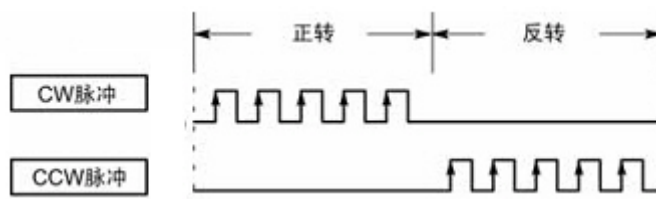
3.4.2.9.3 指令说明

1. PLS+SIGN 输出模式



速度指定用脉冲输出，旋转方向指定用 ON/OFF 进行指定，在该模式下，旋转方向 OFF 对应电机正转。

2. CW/CCW 输出方式



该方式作用下，正转反转都用脉冲的双路脉冲控制。

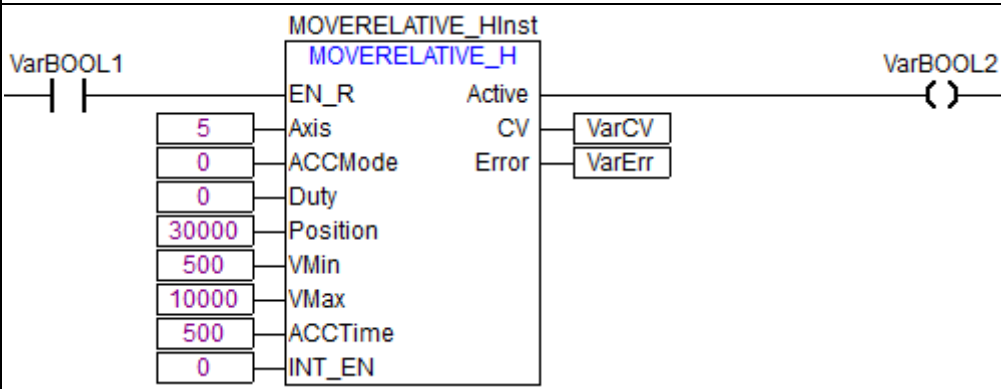
3. 示例

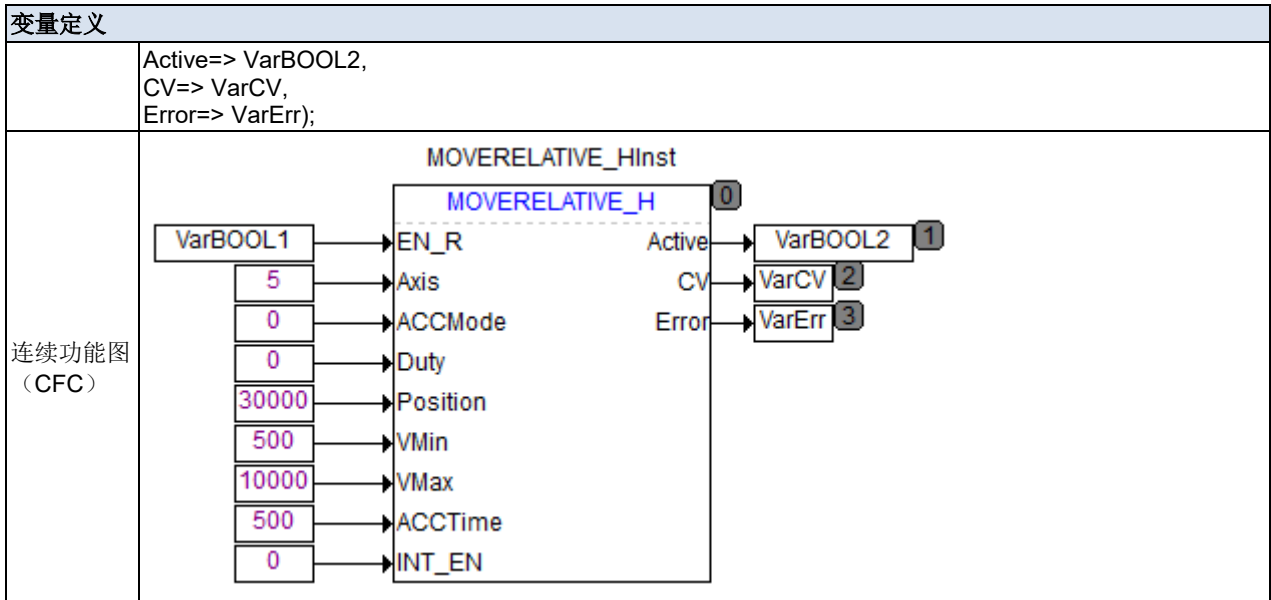
当前位置为 5000 时，作为目标值+7000，执行脉冲输出指令：

- 脉冲+方向：从脉冲输出轴输出 7000 脉冲，方向输出低电平；
- CW/CCW：CW 轴输出 7000 脉冲，CCW 输出低电平；
- 以上两种方式后当前位置为 12000。

3.4.2.9.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MOVERELATIVE_HInst			MOVERELATIVE_H		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DINT	0	FALSE
0005	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> MOVERELATIVE_HInst(EN_R:=VarBOOL1, Axis:=5, ACCMode:=0, Duty:=0, Position:=30000, VMin:=500, VMax:=10000, ACCTime:=500, INT_EN:=0, </pre>



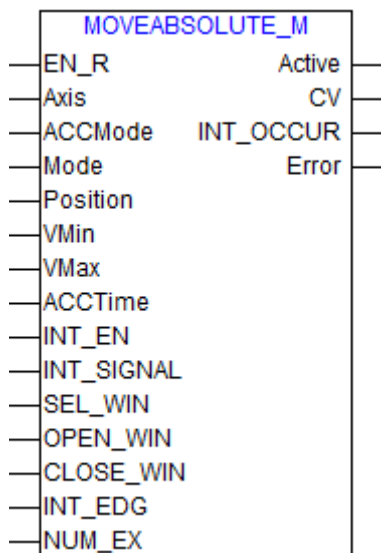
程序说明：

EN_R 置位并保持时，轴 1（Q0.0 和 Q0.1）按照双脉冲模式发送占空比为 50 的脉冲，共发送 30000 个，初始频率为 500Hz，最高输出频率为 10000Hz，加减速时间为 500ms，梯形加减速模式，CV 显示出已发送脉冲数量。

3.4.2.10 MOVEABSOLUTE_M（绝对定位,5128）

3.4.2.10.1 功能描述

用于控制电机进行定位，以原点作为参考点输出已设定个数和频率的脉冲，达到定位的功能。并支持中断功能。



MOVEABSOLUTE_M 功能块

3.4.2.10.2 参数说明

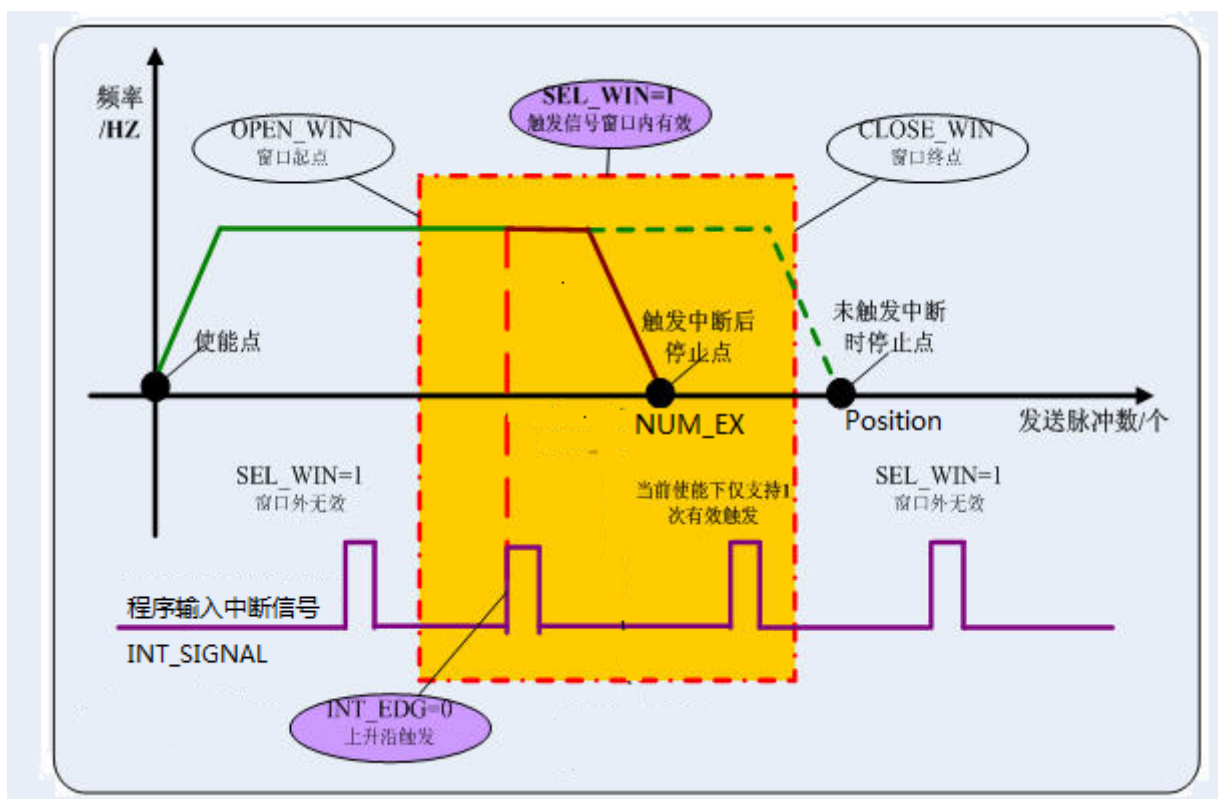
输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出轴选择	PLS+SIGN 模式 =5, 输出轴 5, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 =6, 输出轴 6, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 =7, 输出轴 7, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 =8, 输出轴 8, 脉冲输出端 Q0.3, 脉冲方向 Q0.7 CW/CCW 模式 =5、7 分别表示输出为：轴 5、轴 7 若输出 6 仍对应轴 5 输出，若输出 8 则仍对应轴 7 输出
ACCMode	BOOL	加减速模式选择	0: 梯形曲线 1: S 型曲线
Mode	BOOL	脉冲模式控制	0: PLS+SIGN 模式 1: CW/CCW 模式
Position	DINT	绝对位置值设定	-1,073,741,824~1,073,741,823
VMin	DWORD	初始频率	10 Hz~100,000 Hz
VMax	DWORD	最高频率	10 Hz~100,000 Hz
ACCTime	WORD	加减速时间	0~32,767 ms (0 表示匀速)
INT_EN	BOOL	中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断
INT_SIGNAL	BOOL	中断输入信号	
SEL_WIN	BYTE	窗口选择	约束 INT_EX 硬件参数的有效性 0: 不使用窗口 1: 窗口内有效 2: 窗口外有效
OPEN_WIN	DINT	窗口 1 绝对值	-1,073,741,824~1,073,741,823
CLOSE_WIN	DINT	窗口 2 绝对值	-1,073,741,824~1,073,741,823
INT_EDG	BOOL	跳变沿选择	INT_SIGNAL 中断跳变沿选择

输入参数	数据类型	功能描述	参数值说明
			0: 上升沿触发 1: 下降沿触发
NUM_EX	DINT	中断触发后的目标停止位置	-1,073,741,824~1,073,741,823

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送 1: 正在发送
CV	DINT	已发送脉冲个数	-1,073,741,824~1,073,741,823
INT_OCCUR	BOOL	中断执行生效标志	0: 中断未生效 1: 中断生效
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误 =3: 频率输出错误 =4: 加减速时间参数设置错误 =5: 绝对位置设定参数超范围 =6: 中断信号到达时, 当前位置等于中断停止目标位置 =7: 中断信号到达时, 当前位置已跨过中断停止目标位置

3.4.2.10.3 操作示例

在输出过程中程序中断信号触发时, 执行示意图如下图所示。



INT_SIGNAL 命令相关参数的执行示意图

本功能块支持中断停止功能, 中断信号通过输入参数 INT_SIGNAL 接收。可通过参数 INT_EDG 指定

中断输入信号 INT_SIGNAL 的边沿触发方式。

当中断输入信号同时满足以下条件时，才能触发执行中断：

- 功能块在重新使能后，未执行过有效中断，即 INT_OCCUR 参数为 FALSE；
- 轴还未达到绝对定位目标位置前，INT_SIGNAL 产生有效的跳边沿；
- 中断信号达到时，轴的当前位置满足窗口选择条件。例如当选择 SEL_WIN=1 时，轴当前位置应在 OPEN_WIN 和 CLOSE_WIN 指定的窗口区间内；
- 轴当前位置还未跃过中断目标停止位置。当轴在正向运动时，中断目标停止位置 NUM_EX>该轴当前绝对位置；或当轴负向运动时，中断目标停止位置 NUM_EX<该轴当前位置。



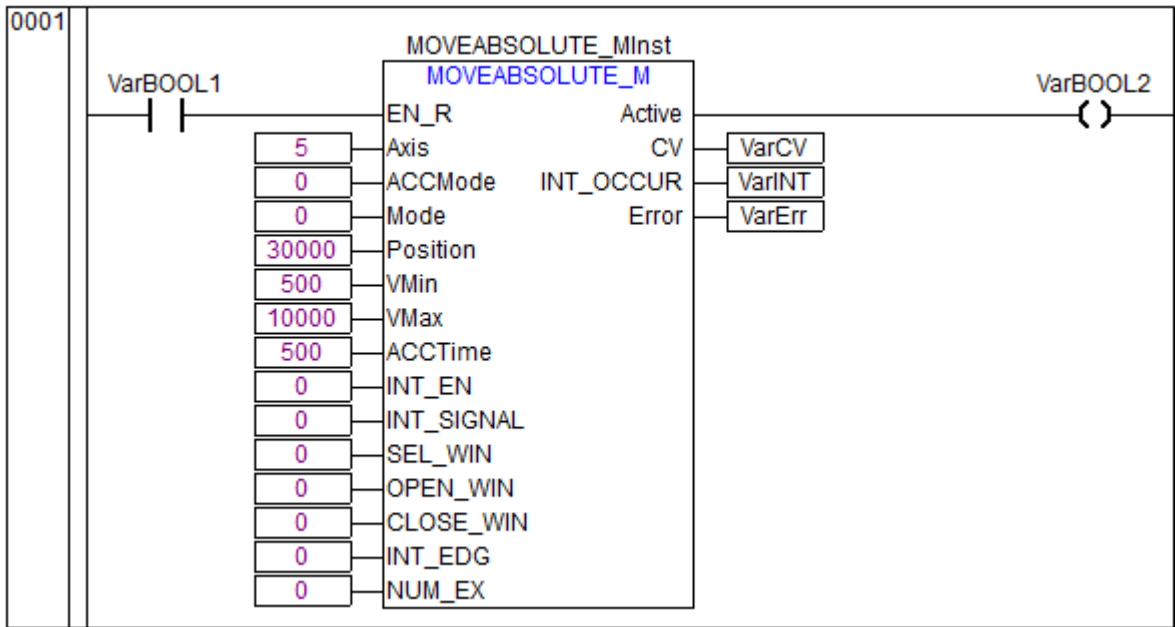
- 若没有使用 HOME 指令设置原点，则默认以第一次调用 MOVERELATIVE、MOVEABSOLUTE、JOG 或 HOME 中任意一个指令控制电机运动之前的位置作为初始原点。PWM 指令不包含其中，若在调用以上功能块之前调用了这两个指令使电机运动，则会使初始原点发生改变。
- 绝对定位指令所参照的原点，与 HOME 指令所设置的原点是同一原点。若程序中未使用 HOME 指令设置原点，则使用初始原点作为绝对定位指令的原点。
- 绝对定位功能块与相对定位功能块相似，只是参考点选择不同，绝对定位时点位点、窗口始终以原点作为参考点，而相对定位始终以当前坐标点作为参考点。若当前位置为 5000 时，作为目标值+7000，执行脉冲输出指令时：
- 脉冲+方向：从脉冲输出轴输出 2000 脉冲，方向输出低电平；
- CW/CCW：CW 轴输出 2000 脉冲，CCW 输出低电平。

3.4.2.10.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MOVEABSOLUTE_MInst			MOVEABSOLUTE_M		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DINT	0	FALSE
0005	VarINT			BOOL	FALSE	FALSE
0006	VarErr			BYTE	0	FALSE
编程语言		程序				

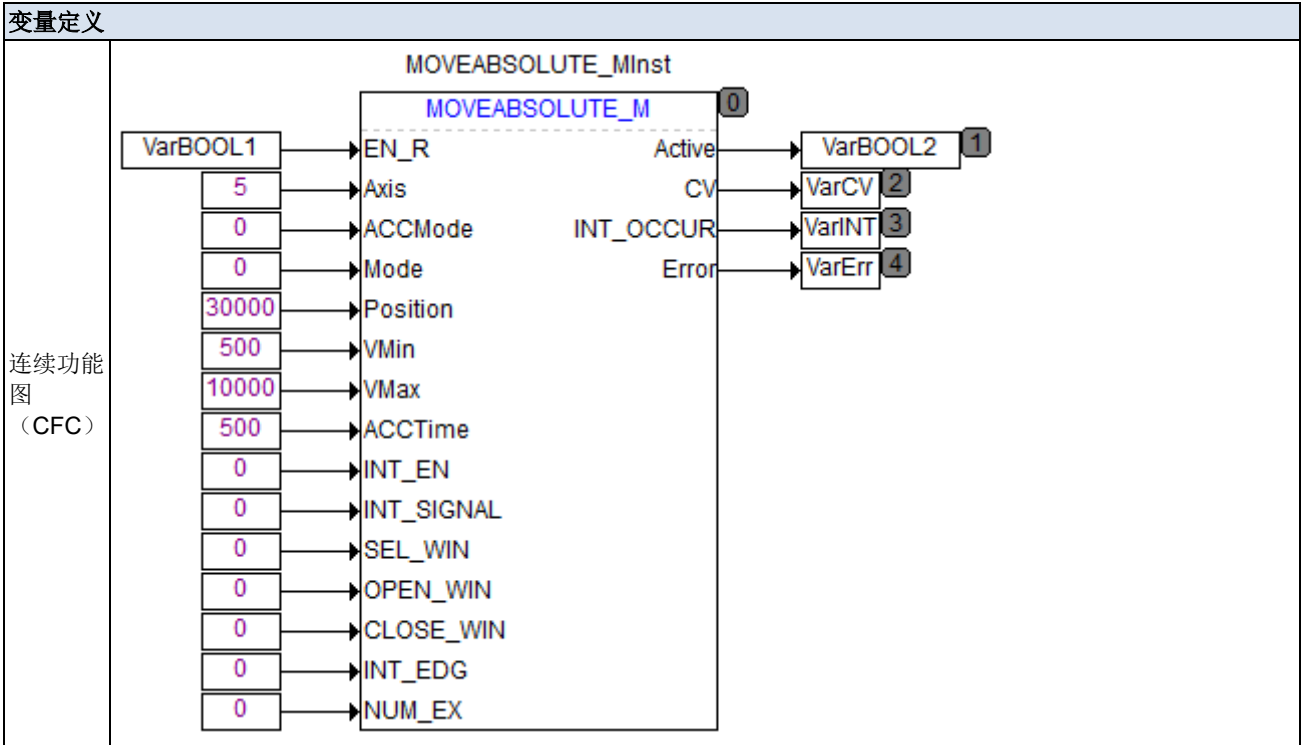
变量定义

梯形图
(LD)



结构化文
本
(ST)

```
MOVEABSOLUTE_MInst(
EN_R:=VarBOOL1,
Axis:=5,
ACCMODE:=0,
Mode:=0,
Position:=30000,
VMin:=500,
VMax:=10000,
ACCTime:=500,
INT_EN:=0,
INT_SIGNAL:=0,
SEL_WIN:=0,
OPEN_WIN:=0,
CLOSE_WIN:=0,
INT_EDG:=0,
NUM_EX:=0,
Active=>VarBOOL2,
CV=>VarCV,
INT_OCCUR=>VarINT,
Error=>VarErr);
```

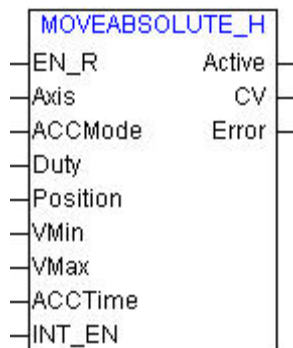
程序说明:

EN_R 置位并保持时，轴 5（Q0.0 和 Q0.1）按照双脉冲模式发送占空比为 50 的脉冲，移动到 30000 脉冲的位置（根据原点计算脉冲数量），初始频率为 500Hz，最高输出频率为 10000Hz，加减速时间为 500ms，梯形加减速模式，CV 显示出已发送脉冲数量。

3.4.2.11 MOVEABSOLUTE_H（绝对定位,5106/5108/5118）

3.4.2.11.1 功能描述

用于控制电机进行定位，以原点作为参考点输出已设定个数和频率的脉冲，达到定位的功能。



MOVEABSOLUTE_H 功能块

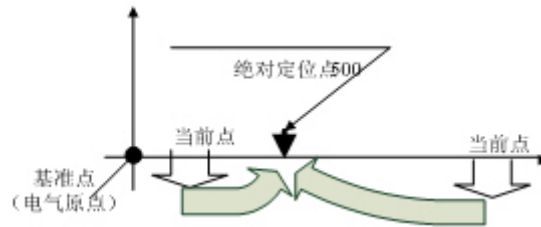
3.4.2.11.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出通道选择	脉冲+方向模式 1: 输出轴 1, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 2: 输出轴 2, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 3: 输出轴 3, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 4: 输出轴 4, 脉冲输出端 Q0.3, 脉冲方向 Q0.7 CW/CCW 模式 5: 输出轴 1, 脉冲输出端 (CW) Q0.0, 脉冲输出端 (CCW) Q0.1 6: 输出轴 2, 脉冲输出端 (CW) Q0.2, 脉冲输出端 (CCW) Q0.3
ACCMODE	BOOL	加速模式选择	0: 梯形曲线 1: S 型曲线
Duty	BOOL	占空比	0: 50% 1: 25%
Position	DINT	输出脉冲数设定	-2147483648~2147483647
VMin	DWORD	初始频率	10Hz~100,000Hz
VMax	DWORD	最高频率	10Hz~100,000Hz
ACCTime	WORD	加减速时间	0~32767ms (0 表示匀速)
INT_EN	BOOL	中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送 1: 正在发送
CV	DINT	已发送脉冲个数	-2147483648~2147483647
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置输出轴错误 =3: 频率输出错误

3.4.2.11.3 指令说明

绝对定位功能块与相对定位功能块相似，只是参考点和调用次数不同，绝对定位始终以原点作为参考点，而相对定位始终以当前坐标点作为参考点。



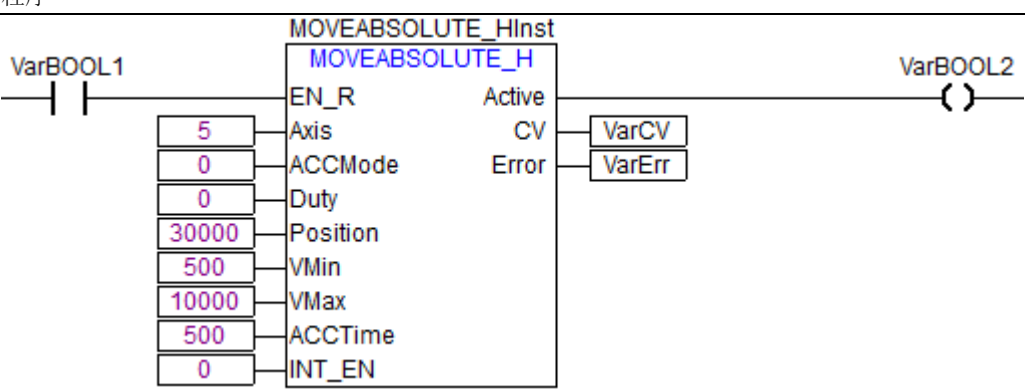
3.4.2.11.4 示例

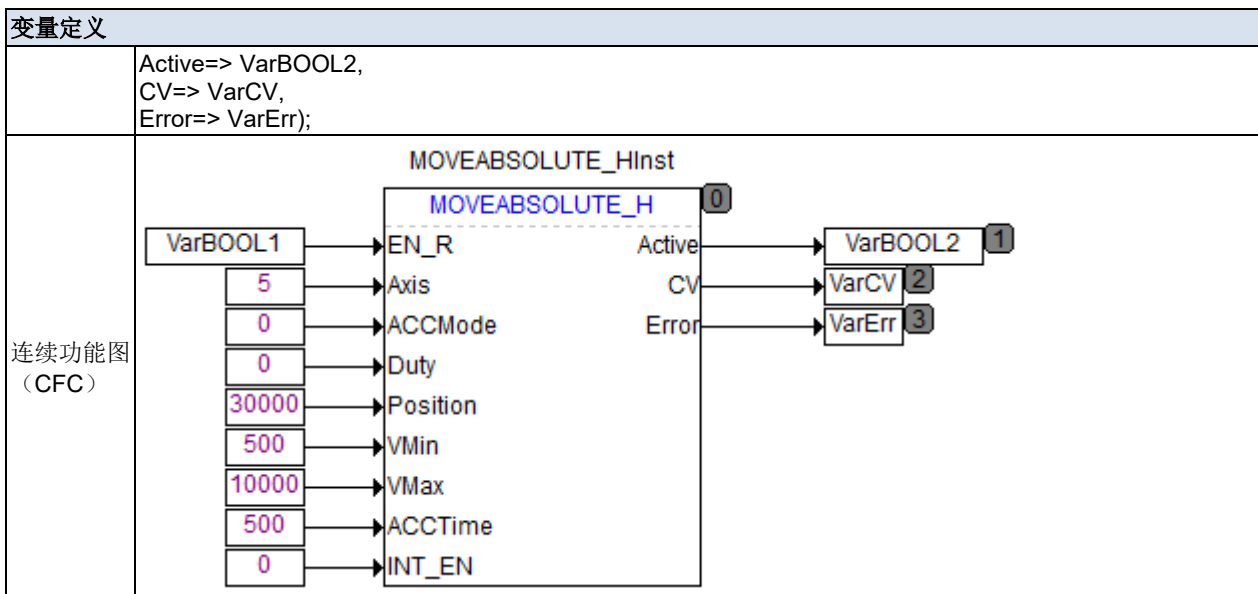
当前位置为 5000 时，作为目标值+7000，执行脉冲输出指令：

- 脉冲+方向：从脉冲输出轴输出 2000 脉冲，方向输出低电平；
- CW/CCW：CW 轴输出 2000 脉冲，CCW 输出低电平；
- 以上两种方式后当前位置为 7000。

3.4.2.11.5 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	MOVEABSOLUTE_HInst			MOVEABSOLUTE_H		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DINT	0	FALSE
0005	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> MOVEABSOLUTE_HInst(EN_R:=VarBOOL1, Axis:=5, ACCMode:=0, Duty:=0, Position:=30000, VMin:=500, VMax:=10000, ACCTime:=500, INT_EN:=0, </pre>



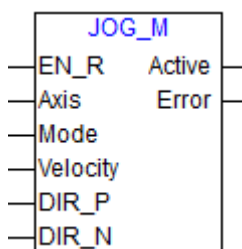
程序说明：

EN_R 置位并保持时，轴 1（Q0.0 和 Q0.1）按照双脉冲模式发送占空比为 50 的脉冲，移动到 30000 脉冲的位置（根据原点计算脉冲数量），初始频率为 500Hz，最高输出频率为 10000Hz，加减速时间为 500ms，梯形加减速模式，CV 显示出已发送脉冲数量。

3.4.2.12 JOG_M（点动,5128）

3.4.2.12.1 功能描述

进行高速输出点的点动控制。当指定的正转开关为 ON 时电机正转，当反转开关为 ON 时电机反转。若正转、反转开关同时为 ON 或者 OFF 时电机静止不动。



JOG_M 功能块

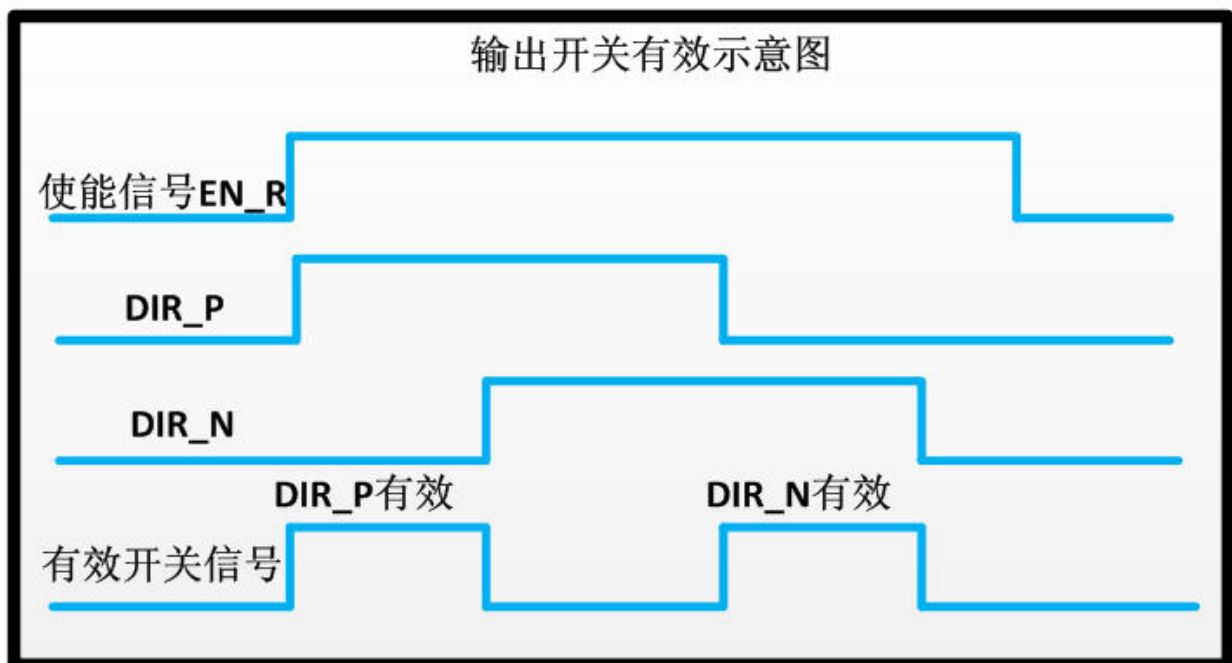
3.4.2.12.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平有效

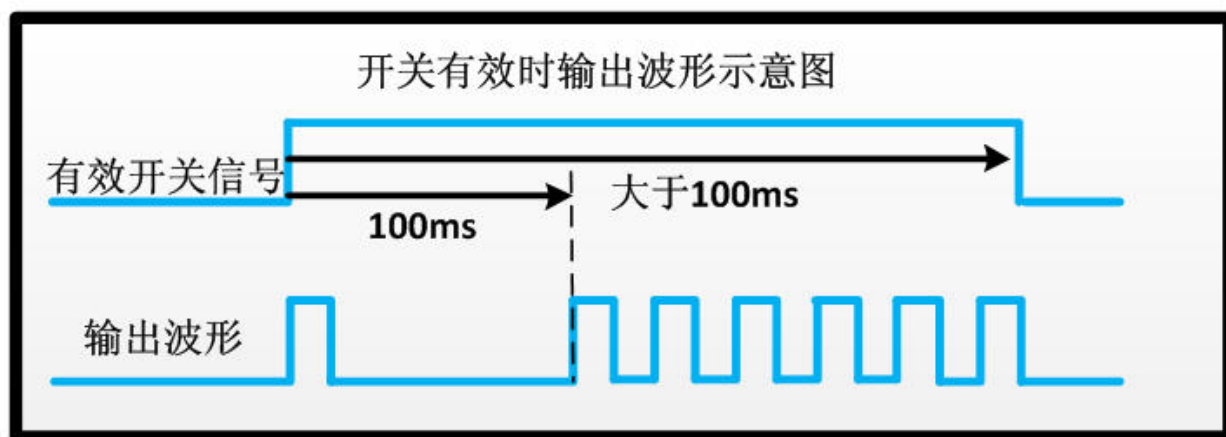
输入参数	数据类型	功能描述	参数值说明
Mode	BOOL	模式控制	0: PLS+SIGN 模式 1: CW/CCW 模式
Axis	BYTE	输出轴选择	PLS+SIGN 模式 =5, 输出轴 5, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 =6, 输出轴 6, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 =7, 输出轴 7, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 =8, 输出轴 8, 脉冲输出端 Q0.3, 脉冲方向 Q0.7 CW/CCW 模式 =5: 轴 5 =7: 轴 7 若输出 6 仍对应轴 5 输出, 若输出 8 则仍对应轴 7 输出
Velocity	DWORD	输出频率	10Hz~100,000Hz
DIR_P	BOOL	正向输出控制端	正向输出控制端为 ON 时, 正向脉冲输出或 CW 脉冲输出
DIR_N	BOOL	反向输出控制端	反向输出控制端为 ON 时, 反向脉冲输出或 CCW 脉冲输出

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送脉冲 1: 正在发送脉冲
Error	BYTE	报错信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误 =3: 频率输出错误

❶操作概述: DIR_P、DIR_N 使能信号接通时, 产生一个与最小命令单位 (Velocity) 相对应的正向或反向脉冲。若使能时间超过 100ms, 将持续输出脉冲。



(a)

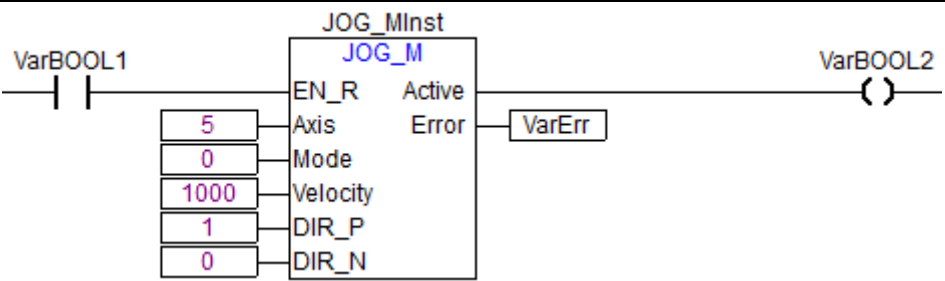


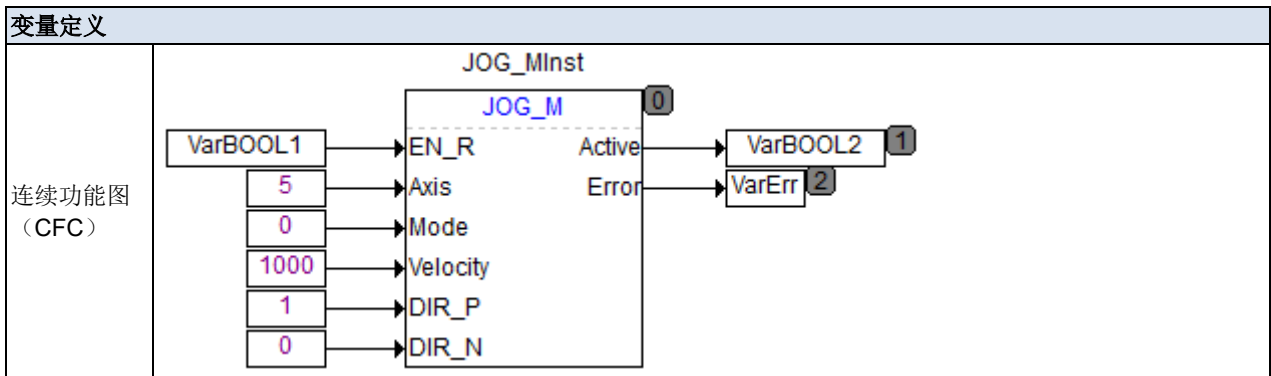
(b)

点动命令执行示意图

3.4.2.12.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	VarErr			BYTE	0	FALSE
0004	JOG_MInst			JOG_M		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> JOG_MInst(EN_R:=VarBOOL1, Axis:=5, Mode:=0, Velocity:=1000, DIR_P:=1, DIR_N:=0, Active=> VarBOOL2, Error=> VarErr); </pre>



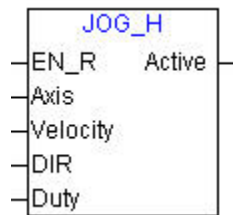
程序说明：

EN_R 置位并保持时，轴 1（Q0.0 和 Q0.4）按照双脉冲模式发送占空比为 50 的脉冲，进行点动，输出频率为 1000Hz。若 EN_R 保持时间 ≥ 0.1 秒，则脉冲将持续发送直至 EN_R 变为 FALSE。

3.4.2.13 JOG_H（点动,5106/5108/5118）

3.4.2.13.1 功能描述

进行高速输出点的点动控制。



JOG_H 功能块

3.4.2.13.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，电平保持
Axis	BYTE	输出轴选择	脉冲+方向模式 1: 输出轴 1，脉冲输出端 Q0.0，脉冲方向 Q0.4 2: 输出轴 2，脉冲输出端 Q0.1，脉冲方向 Q0.5 3: 输出轴 3，脉冲输出端 Q0.2，脉冲方向 Q0.6 4: 输出轴 4，脉冲输出端 Q0.3，脉冲方向 Q0.7 CW/CCW 模式 5: 输出轴 1，脉冲输出端（CW）Q0.0，脉冲输出端（CCW）Q0.1 6: 输出轴 2，脉冲输出端（CW）Q0.2，脉冲输出端（CCW）Q0.3
Velocity	DWORD	输出频率	10Hz~100,000Hz
DIR	BOOL	方向模式	动作模式及输出逻辑关系 1: 方向输出 ON，CCW 方向 0: 方向输出 OFF，CW 方向

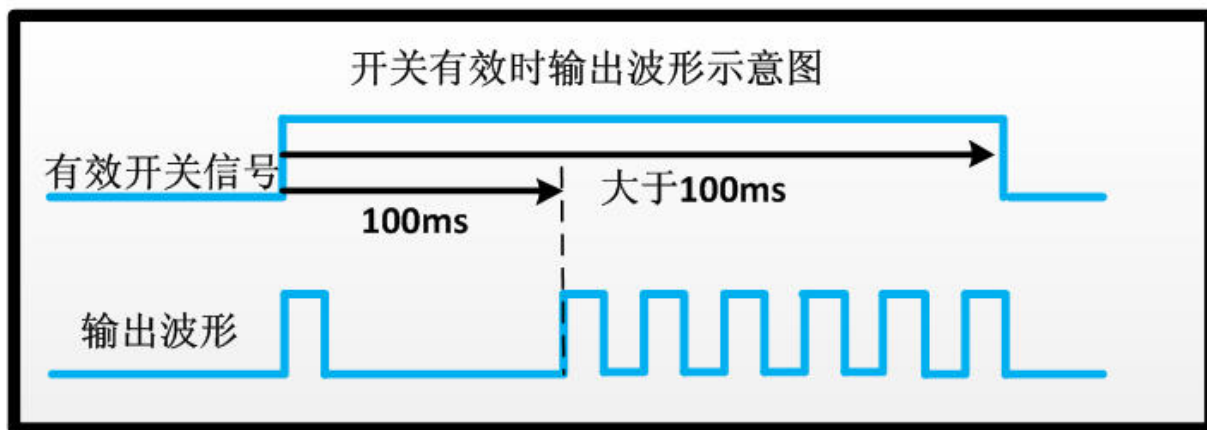
输入参数	数据类型	功能描述	参数值说明
Duty	BOOL	占空比	0: 50% 1: 25%

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送脉冲 1: 正在发送脉冲



- 参数 DIR 可由用户设定，进行点动方向控制；
- 操作概述：输入使能信号接通时，产生一个与最小命令单位相对应的正向或反向脉冲。若使能时间超过 0.1s，将持续输出脉冲。

3.4.2.13.3 指令示例

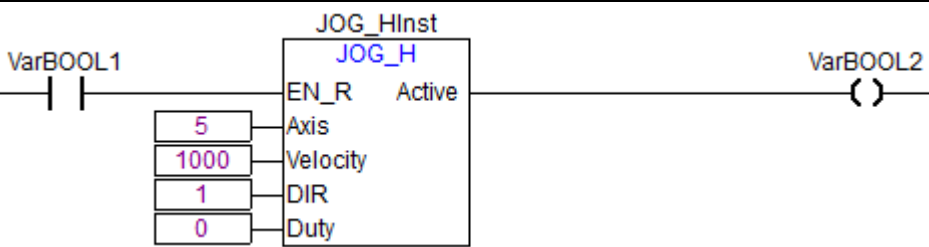
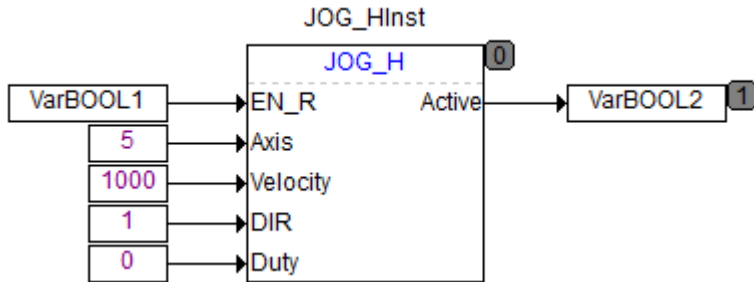


点动命令执行示意图

3.4.2.13.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	JOG_HInst			JOG_H		FALSE

编程语言 程序

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> JOG_HInst(EN_R:=VarBOOL1, Axis:=5, Velocity:=1000, DIR:=1, Duty:=0, Active=> VarBOOL2); </pre>
连续功能图 (CFC)	

程序说明：

EN_R 置位并保持时，轴 1（Q0.0 和 Q0.1）按照双脉冲模式发送占空比为 50 的脉冲，进行点动，输出频率为 1000Hz。若 EN_R 保持时间≥0.1 秒，则脉冲将持续发送直至 EN_R 变为 FALSE。

3.4.2.14 STOP_M（停止,5128）

3.4.2.14.1 功能描述

指令取消所有轴的当前运动，并停止。



STOP_M 功能块

3.4.2.14.2 参数说明

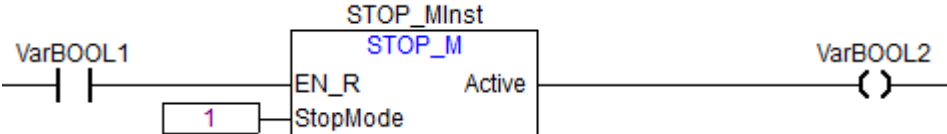
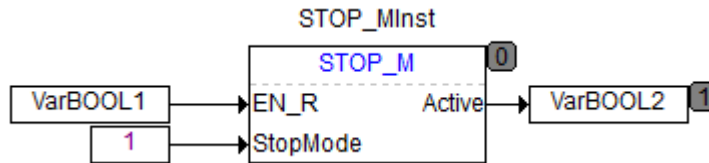
输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平有效
StopMode	BOOL	停止模式	=0: 按照设定的减速率、或者减速时间所相当的减速率实现停止

输入参数	数据类型	功能描述	参数值说明
			=1: 立刻停止输出

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	停止完成标志	1: 所有停止完成 0: 正在停止

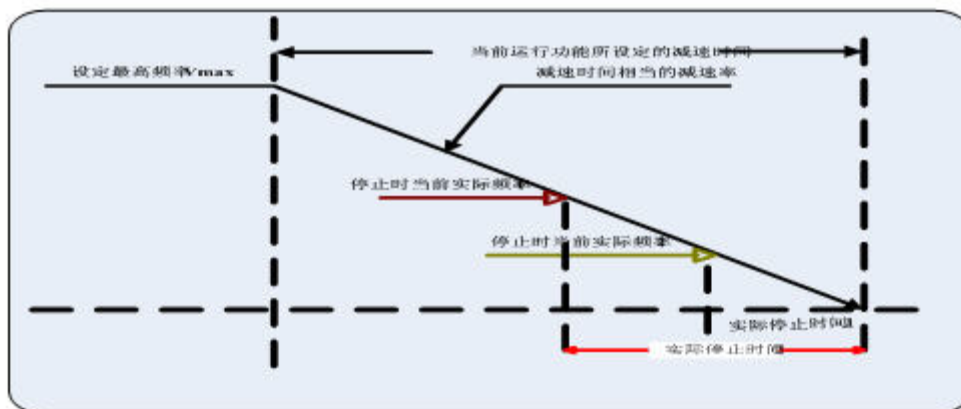
3.4.2.14.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	STOP_MInst			STOP_M		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>STOP_MInst(EN_R:=VarBOOL1, StopMode:=1, Active=> VarBOOL2);</pre>
连续功能图 (CFC)	

3.4.2.14.4 特别说明

STOP_M、CANCEL 两功能块在减速停止时（StopMode=0），是按照设定的减速率、速率实现停止（减速时间所相当的减速率）。如下图所示。



减速停止示意图

3.4.2.15 STOP_H（停止,5106/5108/5118）

3.4.2.15.1 功能描述

实现脉冲的立即停止。



STOP_H 功能块

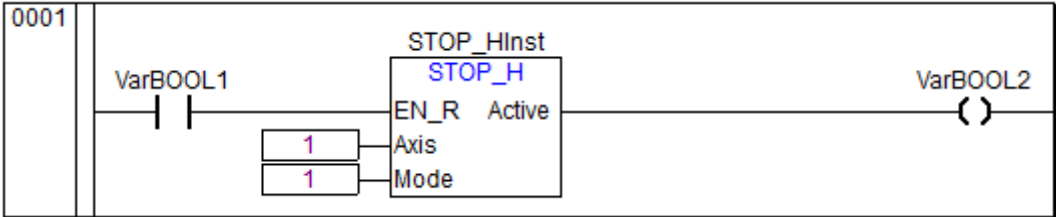
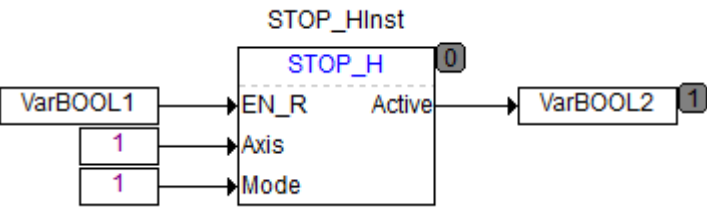
3.4.2.15.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，电平保持
Mode	BOOL	模式选择	0: 不绑定偏差计数器复位输出端 1: 绑定偏差计数器复位输出端
Axis	BYTE	输出轴选择	PLS+SIGN 模式 =1: 输出轴 1, 脉冲 Q0.0, 方向 Q0.4, 偏差计数清零 Q1.0 =2: 输出轴 2, 脉冲 Q0.1, 方向 Q0.5, 偏差计数清零 Q1.1 =3: 输出轴 3, 脉冲 Q0.2, 方向 Q0.6, 偏差计数清零 Q1.2 =4: 输出轴 4, 脉冲 Q0.3, 方向 Q0.7, 偏差计数清零 Q1.3 CW/CCW 模式 =5: 输出轴 1, 脉冲输出端 (CW) Q0.0 脉冲输出端 (CCW) Q0.1 =6: 输出轴 2, 脉冲输出端 (CW) Q0.2 脉冲输出端 (CCW) Q0.3

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	1: 脉冲发送停止 0: 脉冲正在发送

3.4.2.15.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	STOP_HInst			STOP_H		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>STOP_HInst(EN_R:=VarBOOL1, Axis:=1, Mode:=1, Active=> VarBOOL2);</pre>
连续功能图 (CFC)	

程序说明：

EN_R 置位并保持时，将轴 1（Q0.0，按照脉冲+方向模式发送脉冲）的运动停止，并复位电机的偏差计数器（Q1.0）。

3.4.2.15.4 指令说明

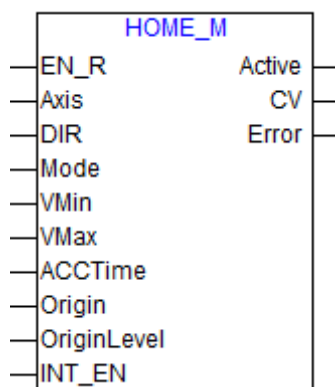
例：若 Mode 设定为 1 时，绑定了偏差计数器复位输出端。如在输入参数 Axis 设为 1 的情况下，本功能块使能后，脉冲输出端 Q0.0 会停止发脉冲，同时对应的偏差计数器输出端 Q1.0 输出信号，使伺服电机中的偏差计数器清零。

若 Mode 设定为 0，则偏差计数器复位端无输出信号，即此时停止指令和偏差计数器输出端无任何关联。

3.4.2.16 HOME_M（原点回归,5128）

3.4.2.16.1 功能描述

控制相应高速输出通道，使电机从当前位置移动到设定原点。



HOME_M 功能块

3.4.2.16.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出轴选择	PLS+SIGN 模式 =5, 输出轴 5, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 =6, 输出轴 6, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 =7, 输出轴 7, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 =8, 输出轴 8, 脉冲输出端 Q0.3, 脉冲方向 Q0.7 CW/CCW 模式 =5、7 分别表示输出为：轴 5、轴 7 若输出 6 仍对应轴 5 输出，若输出 8 则仍对应轴 7 输出
DIR	BOOL	原点搜索方向	0: CW 方向、正向脉冲输出 1: CCW 方向、负向脉冲输出
Mode	BYTE	模式	Bit0 状态 0: 原点信号的上升沿有效 1: 原点信号的下降沿有效 Bit1 状态 0: 返回原点模式 1: 不返回原点模式 Bit2 状态 0: PLG+SIGN 1: CW/CCW
VMin	DWORD	爬行速度	10 Hz~100,000 Hz
VMax	DWORD	回归速度	10 Hz~100,000 Hz
ACCTime	WORD	加减速时间	0~32,767 ms（0 表示匀速）
Origin	BYTE	原点输入点选择	1~16: I0.0~I1.7

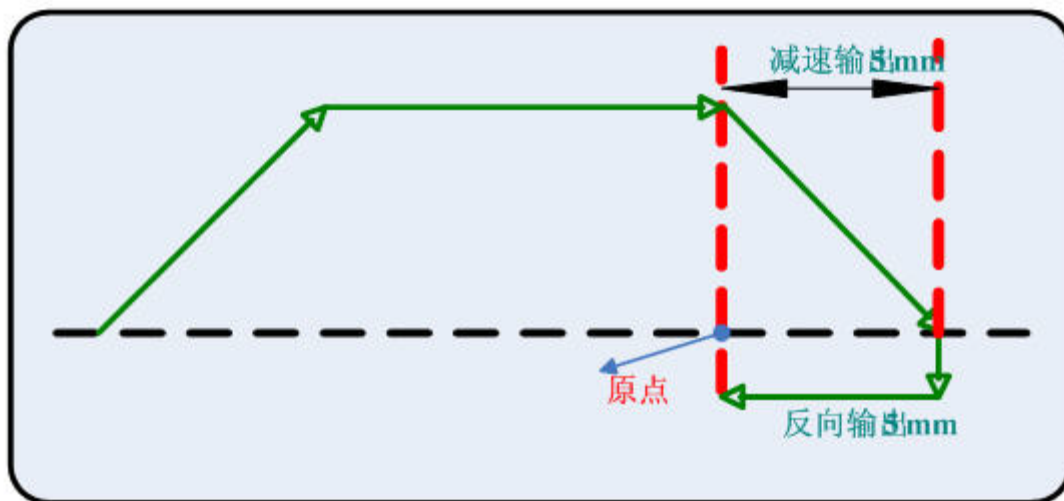
输入参数	数据类型	功能描述	参数值说明
OriginLevel	BOOL	原点有效电平	0: 低电平有效 1: 高电平有效
INT_EN	BOOL	中断使能	0: 回零完成后不产生中断 1: 回零完成后产生中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	原点搜索完成标志	1: 正在回归 0: 未回原点或回归完成
CV	DINT	已发送脉冲个数	-1,073,741,824~1,073,741,823 找到原点前,CV 值为累计发送脉冲个数 找到原点后,CV 值为找到原点后的发送脉冲个数
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误 =3: 频率输出错误 =4: 原点输入点选择错误 =5: 加减速时间参数设置错误

3.4.2.16.3 指令说明

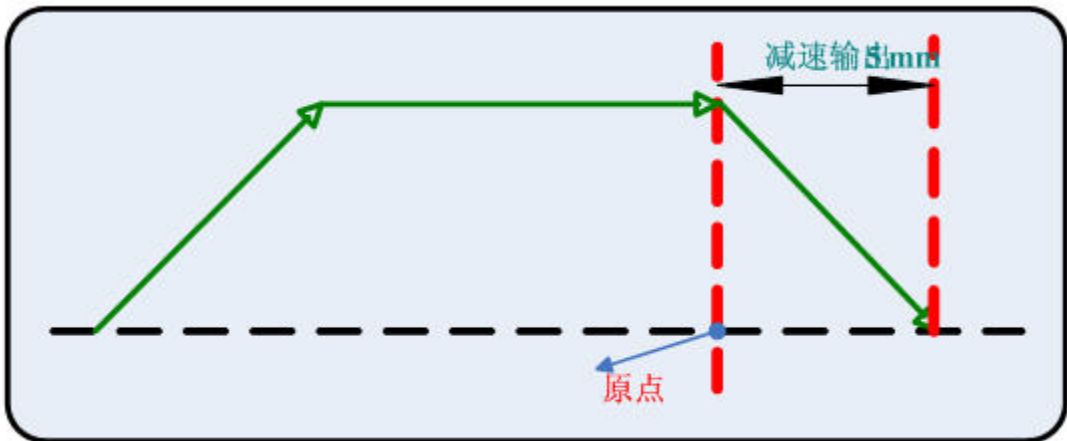
参见 [HOME H（原点 回归, 5106/5108）](#) 的[指令说明](#)。

1. 原点搜索返回原点模式实例



原点搜索返回原点模式实例

2. 原点搜索不返回原点模式实例

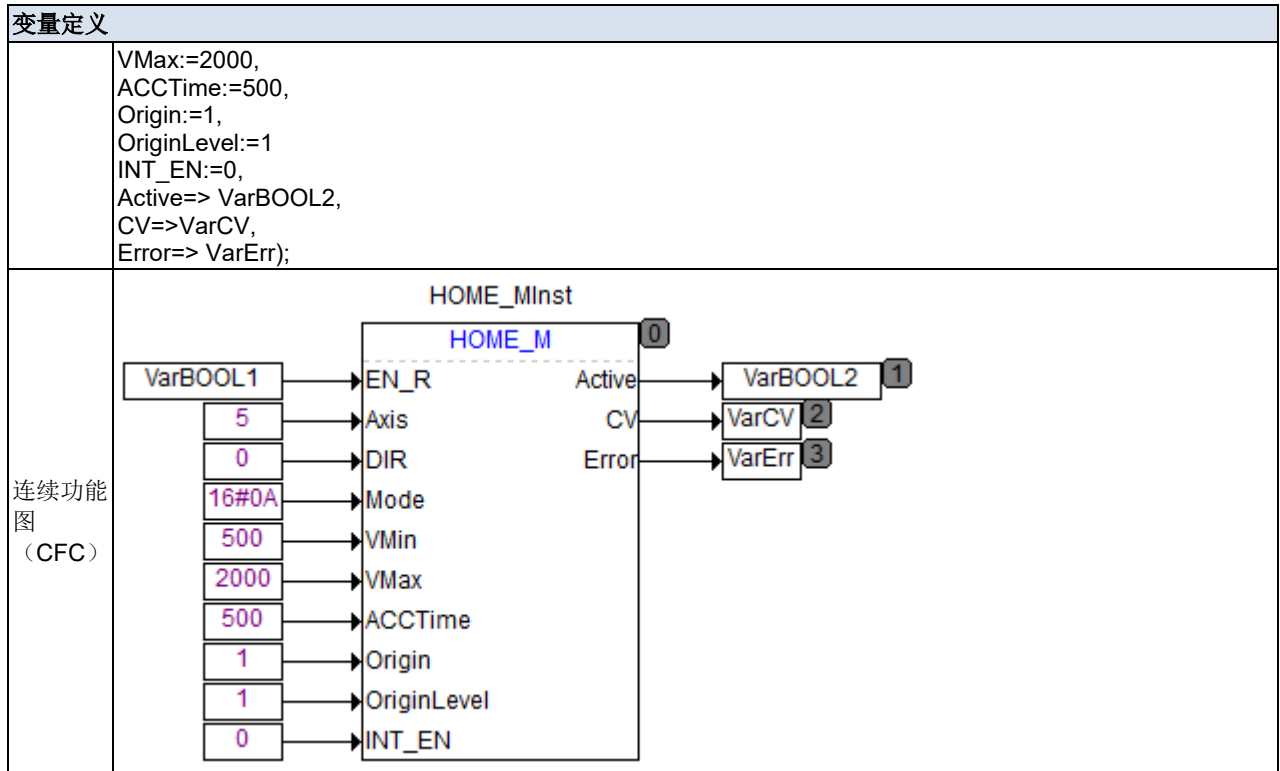


原点搜索不返回原点模式实例

3.4.2.16.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	VarCV			DINT	0	FALSE
0004	VarErr			BYTE	0	FALSE
0005	HOME_MInst			HOME_M		FALSE

编程语言	程序
梯形图 (LD)	0001
结构化文本 (ST)	<pre> HOME_MInst(EN_R:=VarBOOL1, Axis:=5, DIR:=0, Mode:=16#0A, VMin:=500, </pre>



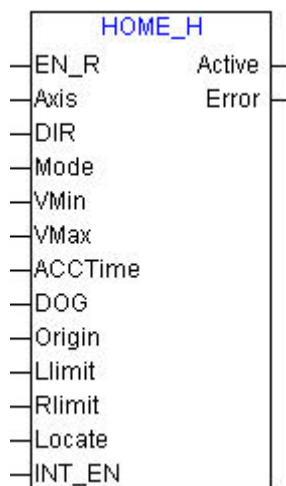
程序说明：

EN_R 置位并保持时，伺服电机的轴 5 按照脉冲+方向模式向正方向进行原点搜索，搜索方式遵循工作模式 2、原点检测方法 2 和反转模式 1（详细说明请参照 [HOME_H（原点 回归, 5106/5108）](#) 相关文字），其中爬行速度 500Hz，回归速度 2000Hz，加减速时间为 500ms。原点脉冲输入由 I0.0 提供，不产生中断。

3.4.2.17 HOME_H（原点 回归,5106/5108/5118）

3.4.2.17.1 功能描述

控制相应高速输出通道，使电机从当前位置移动到机械原点或者通过定位信号将当前点设定为电气原点。



HOME_H 功能块

3.4.2.17.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出通道选择	PLS+SIGN 模式 =1: 输出轴 1，脉冲 Q0.0，方向 Q0.4，偏差计数清零 Q1.0 =2: 输出轴 2，脉冲 Q0.1，方向 Q0.5，偏差计数清零 Q1.1 =3: 输出轴 3，脉冲 Q0.2，方向 Q0.6，偏差计数清零 Q1.2 =4: 输出轴 4，脉冲 Q0.3，方向 Q0.7，偏差计数清零 Q1.3 CW/CCW 模式 =5: 输出轴 1，脉冲输出端（CW）Q0.0 脉冲输出端（CCW）Q0.1 =6: 输出轴 2，脉冲输出端（CW）Q0.2 脉冲输出端（CCW）Q0.3
DIR	BOOL	原点搜索方向	0: CW 方向 1: CCW 方向
Mode	BYTE	模式	参见 Mode 参数详细说明
VMin	DWORD	爬行速度	10Hz~100,000Hz
VMax	DWORD	回归速度	10Hz~100,000Hz
ACCTime	WORD	加减速时间	0~32767ms（0 表示匀速）
DOG	BYTE	近点输入点选择	0: 不用 1~24: I0.0~I2.7
Origin	BYTE	原点输入点选择	1~24: I0.0 ~I2.7
L_Limit	BYTE	左极限输入点选择	0: 不用 1~24: I0.0~I2.7
R_Limit	BYTE	右极限输入点选择	0: 不用 1~24: I0.0~I2.7
Locate	BYTE	定位输入点选择	0: 不用 1~24: I0.0~I2.7
INT_EN	BOOL	中断使能	0: 脉冲输出完成后不产生中断 1: 脉冲输出完成后产生中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	原点回归完成标志	1: 正在回归 0: 回归完成
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置输出轴错误 =3: 频率输出错误 =4: Mode 设置错误

3.4.2.17.3 指令说明

1. Mode 参数详细说明

位序号	功能概述	说明
Bit7	-	备用
Bit6~Bit5	动作模式	00: 原点搜索 01: 原点定位 10: 原点复位 11: 无效
Bit4~Bit3	工作模式	00: 工作模式 1 (步进电机) 01: 工作模式 2 (伺服电机) 10: 工作模式 3 (伺服电机) 11: 无效
Bit2~Bit1	原点检测方法	00: 原点检测方法 1 01: 原点检测方法 2 10: 原点检测方法 3 11: 无效
Bit0	反转动作	0: 反转模式 1 1: 反转模式 2

2. 动作模式选择

- 原点搜索: 执行原点搜索时, 输出脉冲使电动机实际动作, 使用近原点、原点的输入信号来决定原点, 检测到的该原点为系统的机械原点。
- 原点定位: 执行原点定位时, 将电机的当前值设定为坐标原点, 即电气原点。若只执行了原点搜索, 没有原点定位时, 系统的电气原点和机械原点重合。
- 原点复位: 执行原点复位时, 在执行原点搜索或者原点定位后, 即存在机械原点或者电气原点的情况下, 电机根据当前所处的位置向所设定的电气原点运动。



- 即使原点未决定 (没执行原点搜索、原点定位) 情况下, 也可以使电动机动作, 但会出现以下现象:
- 原点复位和绝对定位指令不可用;
- 相对定位指令可以正常使用;
- 原点搜索的回归速度 $\leq$ 原点搜索的爬行速度时, 不执行原点搜索。

3. 工作模式选择

工作模式是决定原点搜索时使用输入信号的参数。根据偏差计数器复位输出信号、定位结束输入的有无，分以下三种模式。

工作模式	输入输出信号		
	原点输入信号	偏差计数器复位	定位结束输入
1	检测原点输入信号的上升沿，进行原点确定	不使用	不使用
2		原点检测时，20~30ms（高电平持续时间）	原点检测后，来自驱动器的定位结束输入进入为止，原点搜索动作结束。
3			

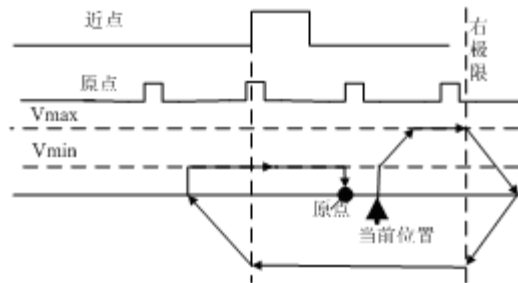
上述三种模式在实际使用中，根据使用的驱动器及用途，设定相应的工作模式。

步进电动机驱动器一般使用工作模式 1

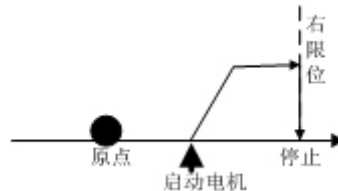
伺服电动机驱动器（或者带有定位结束信号的步进电机驱动器）使用时，如果希望缩短操作时间而不关心定位精度的情况下使用工作模式 2（该模式不使用伺服驱动器侧的定位结束信号）；如果希望提高定位精度的情况下使用工作模式 3（使用伺服驱动器自带的定位结束信号）。

4. 反转动作：即原点搜索过程中，电机行进方向远离原点信号运行。

- 反转模式 1、若电机先检测到限位开关，则电机停止后反向运行继续检测原点，直至检测到为止。示意图如下：



- 反转模式 2、若电机检测到限位开关，则电机立即停止原点搜索。示意图如下：

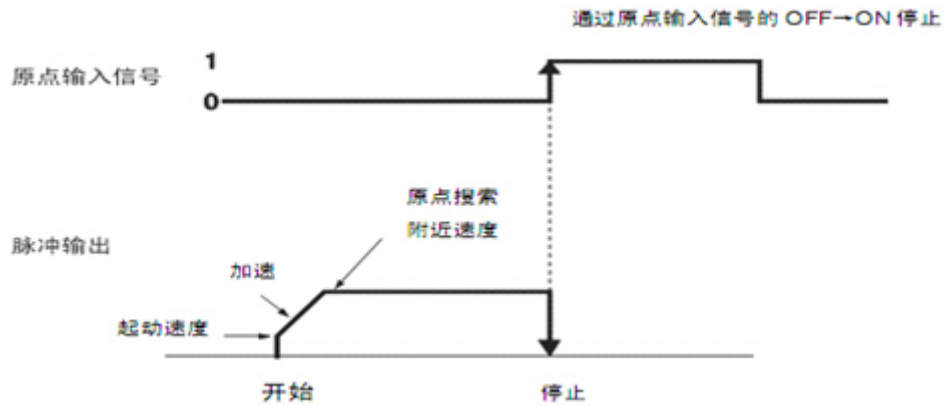


5. 原点检测方法：

- 原点检测方法 1（不使用近原点信号）、具体步骤如下：

原点回归启动后，先以回归速度 V_{Max} 向原点回归方向匀速前进。

当受感应点检测到原点输入信号时脉冲输出立即停止。

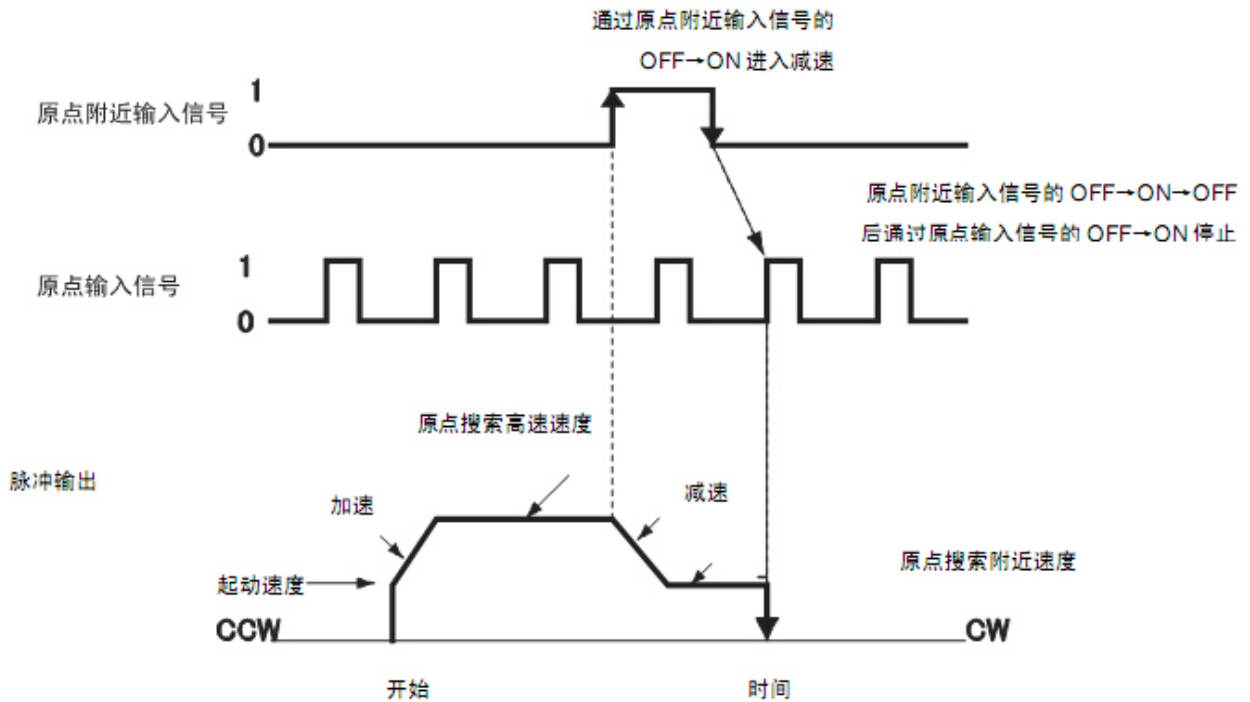


■ 原点检测方法 2、具体步骤如下：

原点回归启动后，先以回归速度 V_{Max} 向原点回归方向匀速前进。

当遇到近点感应开关 DOG sensor 时，以减速斜率作减速动作，减速到达爬行速度（面前设定为 V_{Min} ）后匀速前进。

当受感应点一离开近点感应开关 DOG sensor 后，则当检测到原点信号时脉冲输出立即停止，结束原点搜索动作。

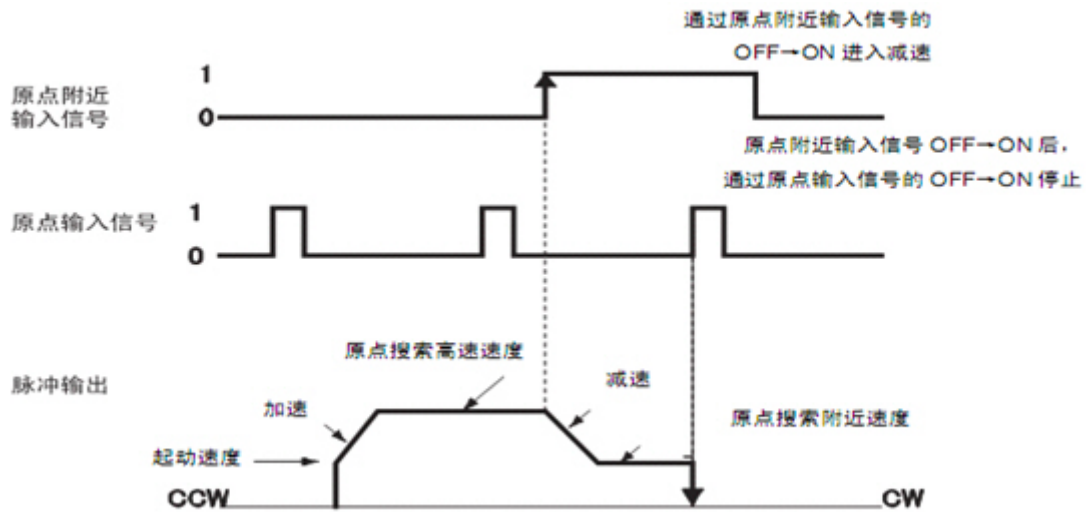


■ 原点检测方法 3：具体执行步骤如下：

原点回归启动后，先以回归速度 V_{Max} 向原点回归方向匀速前进。

当遇到近点感应开关 DOG sensor 时，以减速斜率作减速动作，减速到达爬行速度（面前设定为 V_{Min} ）后匀速前进。

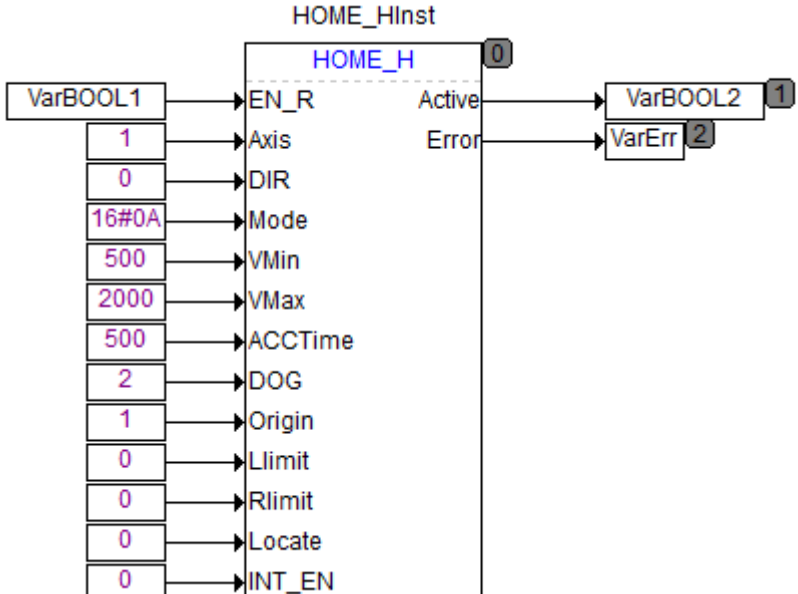
当检测到原点信号时脉冲输出立即停止，结束原点搜索动作。



3.4.2.17.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	HOME_HInst			HOME_H		FALSE
0004	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	<pre> 0001 ----- VarBOOL1 ----- HOME_HInst HOME_H EN_R Axis=1 DIR=0 Mode=16#0A VMin=500 VMax=2000 ACCTime=500 DOG=2 Origin=1 Llimit=0 Rlimit=0 Locate=0 INT_EN=0 Active Error ----- VarErr ----- VarBOOL2 ----- </pre>
结构化文本 (ST)	<pre> HOME_HInst(EN_R:=VarBOOL1, Axis:=1, DIR:=0, Mode:=16#0A, </pre>

变量定义	
	VMin:=500, VMax:=2000, ACCTime:=500, DOG:=2, Origin:=1, Llimit:=0, Rlimit:=0, Locate:=0, INT_EN:=0, Active=> VarBOOL2, Error=> VarErr);
连续功能图 (CFC)	

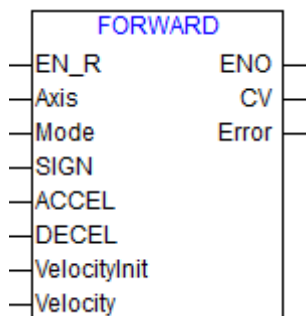
程序说明：

EN_R 置位并保持时，伺服电机的轴 1（Q0.0）按照脉冲+方向模式向正方向进行原点搜索，搜索方式遵循工作模式 2、原点检测方法 2 和反转模式 1（详细说明请参照本节相关文字），其中爬行速度 500Hz，回归速度 2000Hz，加减速时间为 500ms。近原点脉冲输入由 I0.1 通道提供，原点脉冲输入由 I0.0 提供。

3.4.2.18 FORWARD（连续脉冲输出,5128）

3.4.2.18.1 功能描述

FORWARD 指令使能时，高速脉冲输出会按照脉冲给定的值一直输出，直到 CANCEL 或 STOP 指令使能使脉冲输出停止。第一次启动该模块使能时，默认最低频率 10Hz。此功能块的输出频率 Velocity、ACCEL、DECEL 支持在线修改，其它参数均不支持在线修改。



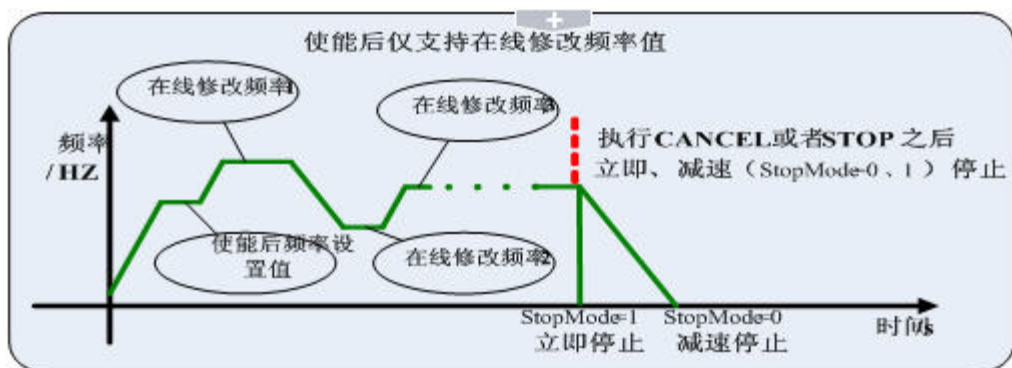
FORWARD 功能块

3.4.2.18.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平有效
Axis	BYTE	输出轴选择	PLS+SIGN 模式 =5: 输出轴 5，脉冲输出端 Q0.0，脉冲方向 Q0.4 =6: 输出轴 6，脉冲输出端 Q0.1，脉冲方向 Q0.5 =7: 输出轴 7，脉冲输出端 Q0.2，脉冲方向 Q0.6 =8: 输出轴 8，脉冲输出端 Q0.3，脉冲方向 Q0.7 CW/CCW 模式 =5、7 分别表示输出轴 5（Q0.0、Q0.1）、轴 7（Q0.2、Q0.3） 若输入 6 仍对应轴 5 输出，若输入 8 则仍对应轴 7 输出。
Mode	BOOL	脉冲模式控制	0: PLS+SIGN 模式；1: CW/CCW 模式
SIGN	BOOL	方向	0: 正向；1: 反向
ACCEL	DWORD	加速度	0~5000000Hz/s，该参数表示第一次启动后加速度和每次修改输出频率的加速度(0 表示 10Hz 匀速)
DECEL	DWORD	减速度	0~5000000Hz/s，该参数表示第一次启动后减速度和每次修改输出频率的减速度(0 表示 10Hz 匀速)
VelocityInit	DWORD	初始频率	10 Hz~100,000 Hz，该参数表示第一次启动后的初始频率
Velocity	DWORD	输出频率	10 Hz~100, 000 Hz

输出参数	数据类型	功能描述	参数值说明
CV	DINT	已发脉冲数	-1073741824~1073741823
Error	BYTE	错误信息	=0: 正确 =1: 输出轴被占用 =2: 配置输出轴错误 =3: 配置输出频率错误 =4: 加、减速度设置错误 =5: 初始频率范围错误 =6: 初始频率大于目标频率

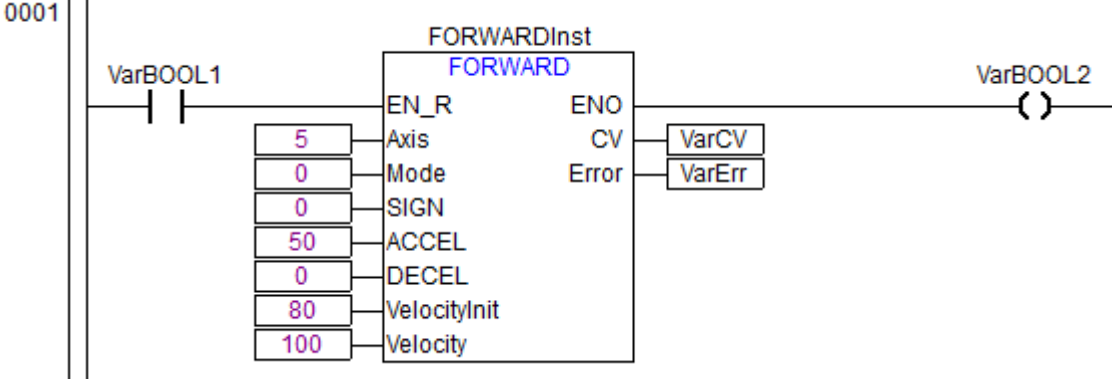
3.4.2.18.3 指令说明

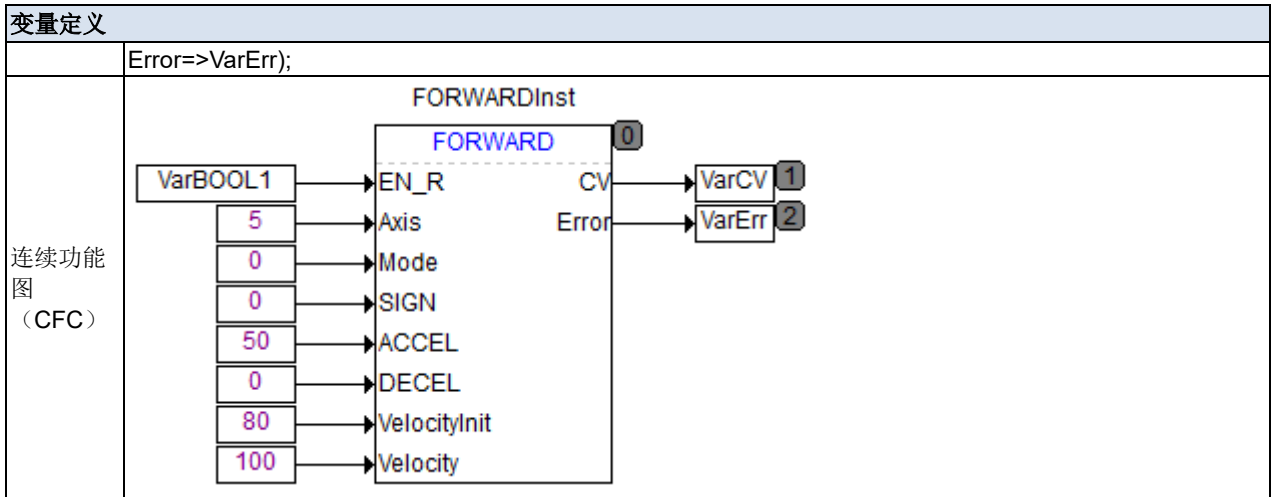


在线修改频率值

3.4.2.18.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	VarCV			DINT	0	FALSE
0004	VarErr			BYTE	0	FALSE
0005	FORWARDInst			FORWARD		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>FORWARDInst(EN_R:=VarBOOL1, Axis:=5, Mode:=0, SIGN:=0, ACCEL:=50, DECEL:=0, VelocityInit:=80, Velocity:=100, CV=>VarCV,</pre>



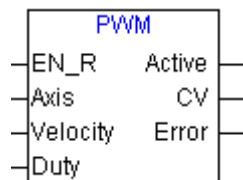
程序说明：

EN_R 置位并保持时，轴 5 按照脉冲输出给定频率 100Hz、同时以 50Hz/s 的加速度一直输出。

3.4.2.19 PWM（脉冲调制输出,5104/5106/5108/5128/5118）

3.4.2.19.1 功能描述

从由 Axis 指定的端口中输出由 Velocity 指定频率、Duty 指定占空比的脉冲。



PWM 功能块

3.4.2.19.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，高电平保持
Axis	BYTE	输出轴选择	LE5104/LE5106/LE5108/LE5118: =1: 输出轴 1, 脉冲输出端 Q0.0 =2: 输出轴 2, 脉冲输出端 Q0.1 =3: 输出轴 3, 脉冲输出端 Q0.2 =4: 输出轴 4, 脉冲输出端 Q0.3 5128: =5、6、7、8 分别表示输出轴为：轴 5、轴 6、轴 7、轴 8 =5: 轴 5, 脉冲输出端 Q0.0 =6: 轴 6, 脉冲输出端 Q0.1 =7: 轴 7, 脉冲输出端 Q0.2

输入参数	数据类型	功能描述	参数值说明
			=8: 轴 8, 脉冲输出端 Q0.3
Velocity	DWORD	输出频率	10Hz~100,000Hz 可在线修改
Duty	WORD	占空比	占空比输入值: 0~1000 (对应 0~100.0) 占空比分辨率: 10Hz≤输出频率≤1,000Hz 时: 1/1000 1,000Hz<输出频率≤10,000Hz 时: 1/100 10,000Hz<输出频率≤100,000Hz 时: 1/10 可在线修改

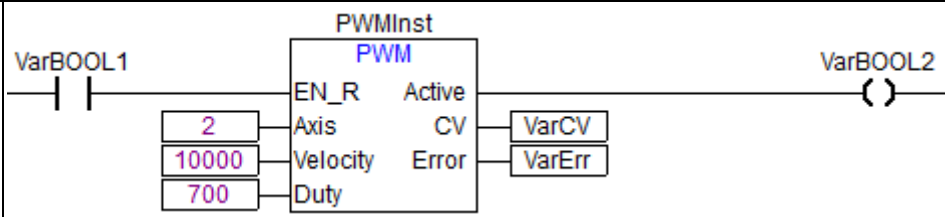
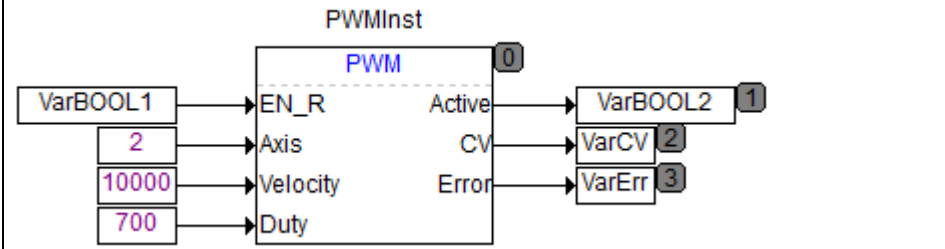
输出参数	数据类型	功能描述	参数值说明
Active	BOOL	脉冲发送标志	0: 停止发送 1: 正在发送
CV	DWORD	已发送脉冲个数	0~2147483647
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误 =3: 频率设置错误 =4: 占空比设置错误

3.4.2.19.3 指令说明

- 功能块使能时，输出频率与占空比可在线修改。
- 占空比设定为 0 时，一直为低电平输出；设定为 1000 时，一直为高电平输出。
- PWM 不影响 S 区特殊寄存器轴绝对位置输出。

3.4.2.19.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	PWMIInst			PWM ▼		FALSE ▼
0002	VarBOOL1			BOOL ▼	FALSE ▼	FALSE ▼
0003	VarBOOL2			BOOL ▼	FALSE ▼	FALSE ▼
0004	VarCV			DWORD ▼	0	FALSE ▼
0005	VarErr			BYTE ▼	0	FALSE ▼
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> PWMInst(EN_R:=VarBOOL1, Axis := 2, Velocity :=10000, Duty :=700, Active =>VarBOOL2, CV =>VarCV, Error=> VarErr); </pre>
连续功能图 (CFC)	

程序说明：

EN_R 置位并保持时，第 2 路高速脉冲输出开始以 10KHz 的频率发送脉冲，占空比为 70.0。

3.4.2.20 DEFPOS（定义位置,5128）

3.4.2.20.1 功能描述

DEFPOS 用于定义指定轴的当前位置，并使轴的当前位置记录在特殊寄存区，从而改变原点的位置。



DEFPOS 功能块

3.4.2.20.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1： 上升沿使能，高电平有效
Axis	BYTE	轴号选择	=1、2、3、4、5、6、7、8 分别表示指定轴为：轴 1 至轴 8
SetValue	DINT	绝对位置设定值	-1073741824~1073741823

输出参数	数据类型	功能描述	参数值说明
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 配置轴错误

3.4.2.20.3 指令说明

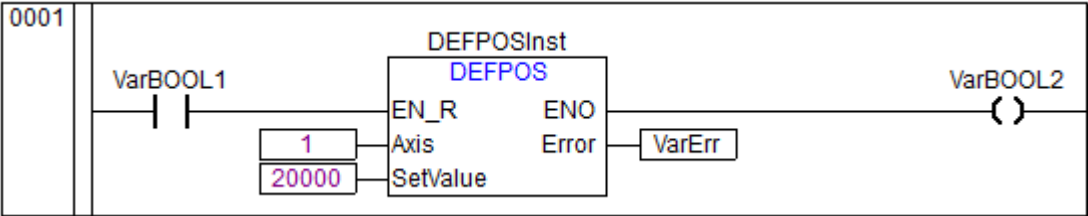
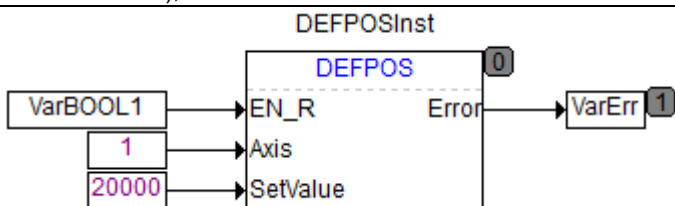
在输入输出位置改变时，以下特殊寄存器实时记录对应轴的绝对位置。除 PWM 之外，所有高速输出指令的使能工作，都可以改变输出轴所对应寄存器的绝对位置值。

轴号	1	2	3	4	5	6	7	8
特殊寄存器	%SD108	%SD112	%SD116	%SD120	%SD140	%SD144	%SD148	%SD152

当轴 1、轴 3 为 AB 相计数时，轴 2、轴 4 对应的计数值不进行数据累计。当轴 5、轴 7 为 CW/CCW 脉冲输出模式时，轴 6、轴 8 对应的计数值不进行数据累计。

3.4.2.20.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	DEFPOSInst			DEFPOS		FALSE
0004	VarErr			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>DEFPOSInst(EN_R:=VarBOOL1, Axis:=1, SetValue:=20000, Error=>VarERR);</pre>
连续功能图 (CFC)	

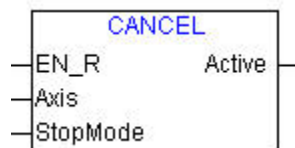
程序说明：

EN_R 置并保持时，把绝对位置设定值赋给对应特殊寄存器，作为当前绝对位置。

3.4.2.21 CANCEL（解除连接,5128）

3.4.2.21.1 功能描述

指令解除当前指定轴与其它轴的一切连接关系，并停止。



CANCEL 功能块

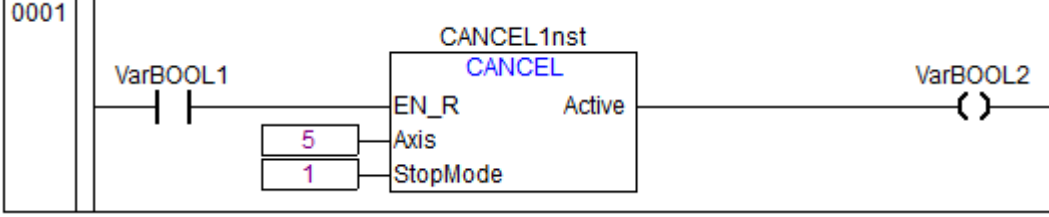
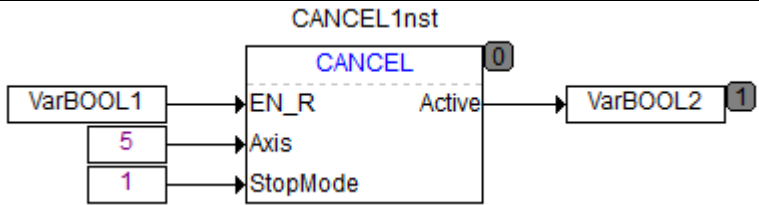
3.4.2.21.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1: 上升沿使能，高电平有效
Axis	BYTE	停止轴选择	=5、6、7、8 当前轴为：PLS+SIGN 模式时 分别表示停止轴为：轴 5、轴 6、轴 7、轴 8 当前轴为：CW/CCW 模式 5、6 表示停止轴 5（Q0.0、Q0.1）； 7、8 表示停止轴 7（Q0.2、Q0.3）
StopMode	BOOL	停止模式	=0: 按照设定的减速率、或者减速时间所相当的减速率实现停止 =1: 立刻停止输出

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	停止完成标志	1: 停止完成 0: 正在停止

3.4.2.21.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	CANCELinst			CANCEL		FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre>CANCEL1nst(EN_R:=VarBOOL1, Axis:=5, StopMode:=1, Active=> VarBOOL2);</pre>
连续功能图 (CFC)	

程序说明：

EN_R 置位并保持时，轴 5 立刻停止输出。

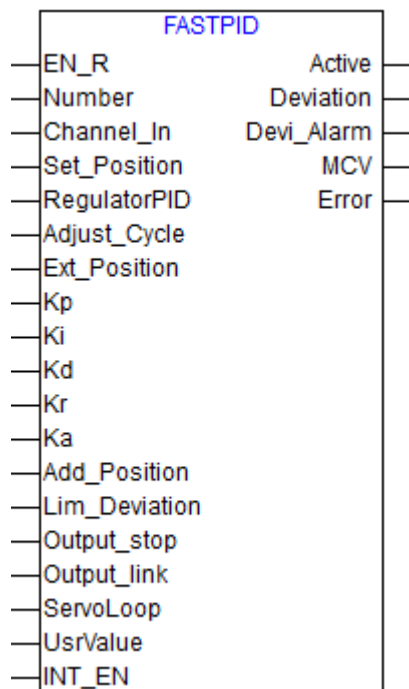
3.4.2.21.4 特别说明

STOP_M、CANCEL 两功能块在减速停止时（StopMode=0），是按照设定的减速率、速率实现停止（减速时间所相当的减速率）。

3.4.2.22 FASTPID（快速 PID,5128）

3.4.2.22.1 功能描述

对现场返回的过程值与设置目标值比较，对其进行 PID 调节控制，将调节值输出给现场设备从而达到及时响应的调节任务，同时对输出值进行限幅操作，以保证输出在许可的范围内工作。



FASTPID 功能块

3.4.2.22.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1: 上升沿使能, 高电平有效
Number	BYTE	当前 PID 功能块编号	=1、2、3、4 表示: 第 1、2、3、4 路模拟量输出
Channel_In	BYTE	反馈输入通道 (注: 本模块仅支持正的模拟量输入)	=1、2、3、4 分别表示: 第 1、2、3、4 路数字量输入 =5、6 分别表示: 第 1、2 路模拟量输入 =7 分别表示: 外部算法输入
Set_Position	DINT	设置目标值	数据范围-1,073,741,824~1,073,741,823 的整数
RegulatorPID	BYTE	叠加 PID 编号	0 表示不叠加; X 表示叠加第 X 个 PID 的调节值, X 为 1、2、3、4 其中之一
Adjust_Cycle	BYTE	PID 调节周期	=1~50 ms 可设置 调节误差 0.1 ms。此调解周期为: 通道输入时间+PID 运算时间+AO 输出时间
Ext_Position	DINT	反馈来源于外部算法	数据范围-1,073,741,824~1,073,741,823 的整数
Kp	REAL	比例系数	0~63.999
Ki	REAL	积分系数	0~63.999
Kd	REAL	微分系数	0~63.999
Kr	REAL	前馈系数	0~63.999
Ka	REAL	叠加系数	1/4095~4095
Add_Position	DINT	PID 调节值额外增加量	数据范围-1,073,741,824~1,073,741,823 的整数
Lim_Deviation	DWORD	偏差值限幅	0~2,147,483,647

输入参数	数据类型	功能描述	参数值说明
Output_stop	BOOL	偏差过大是否断开输出	0：表示断开；1：表示闭合
Output_link	BOOL	是否输出给执行机构	0：表示闭合；1：表示断开
ServoLoop	BOOL	闭环输出开关	0：PID 输出开环，MCV 输出 UsrValue 与 Add_Position 的和 1：PID 输出闭环，MCV 根据反馈量计算输出值
UsrValue	WORD	用户指定输出值	用户输入值，ServoLoop=0 时有效
INT_EN	BOOL	偏差量过大是否中断	0：不触发中断；1：触发中断

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	位置值设定成功标志	0：位置设定失败；1：位置设定成功
Deviation	DINT	偏差值	数据范围的整数
Devi_Alarm	BOOL	偏差过大报警	0：偏差值正常；1：偏差值超过设定值
MCV	DWORD	控制量输出	0~65,535
Error	BYTE	错误信息	=0：正确 =1：通道被占用标志 =2：功能块编号错误 =3：反馈输入通道错误 =4：PID 调节周期设置错误 =5：PID 调节系数设置错误 =6：叠加系数 Ka 设置错误 =7：模拟量通道被禁止 =8：31 位输入数据设置错误 =9：叠加 PID 编号设置错误

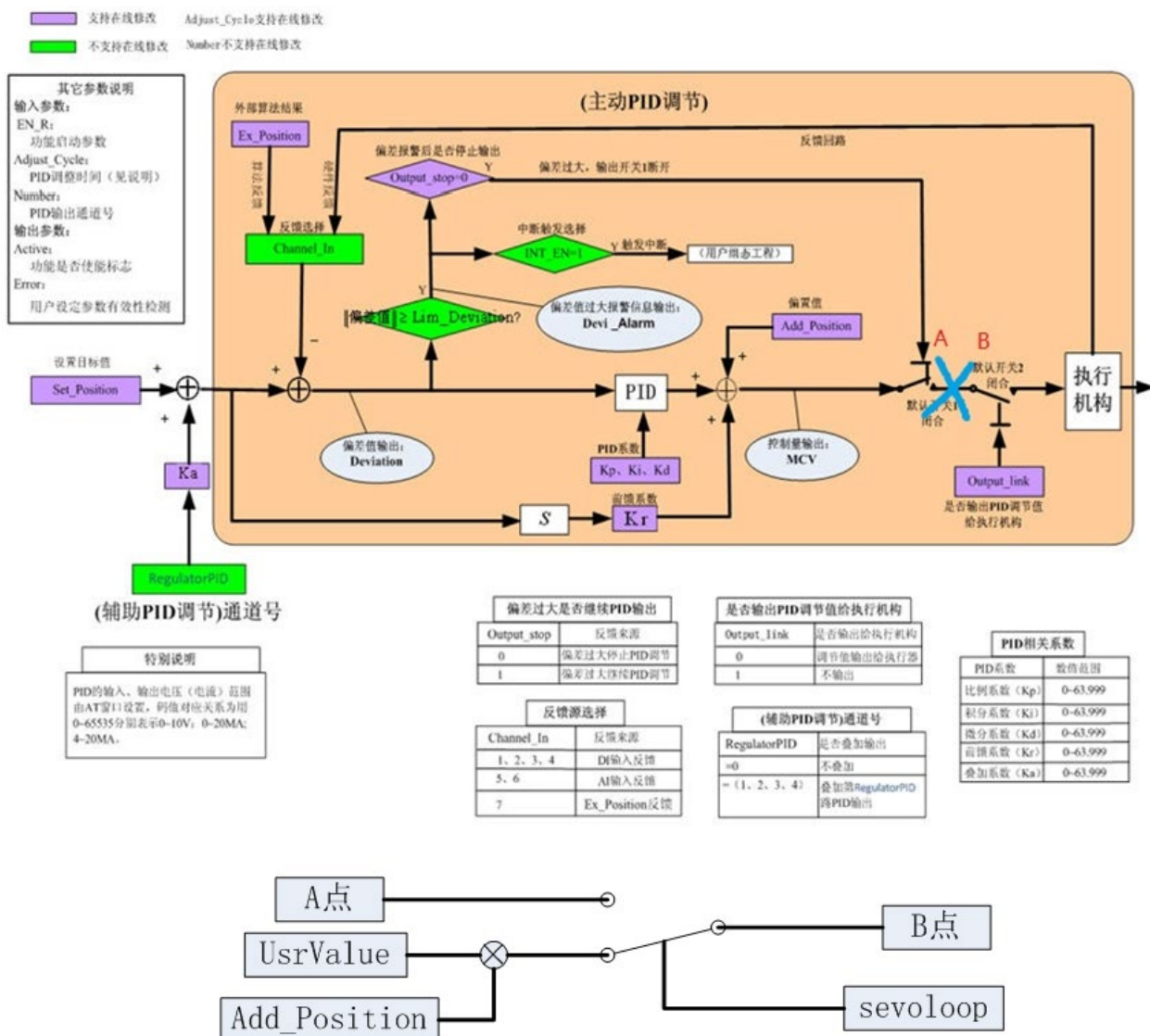
3.4.2.22.3 指令说明

该功能支持调节周期可设置的 PID 运算功能。PID 个数为 4 个，每个 PID 可单独设置是否启动。

- PID 的调节周期，在 1 个调节周期内，需将所有启动的 PID 都运算一遍，调节周期准确，误差不超过 0.1ms。例如：设置启动其中的 3 个 PID，调节周期为 1ms，则在 1ms 之内，将 3 个 PID 都运算一遍。
- PID 参数，可在线实时调整。调整之后的参数，可自动保存到工程文件中。
- PID 算法指标：
 - 多个 PID 可同时设置相同的目标参数。多个 PID 功能相互独立；
 - PID 反馈物理来源有两种：一种来源于模拟量输入，另一种来源于高速脉冲计数通道。PID 反馈来源还可来自外部算法结果；
 - PID 前馈调节算法：目标给定值的变化量乘以前馈系数，与 PID 算法结果进行最终的叠加输出；
 - 在设定的调节周期内，可对 PID 的运算结果，叠加一个额外的偏置量；
 - 在设定的调节周期内，可对两个 PID 的运算结果，叠加输出；

- PID 偏差过大触发中断，偏差值过大触发中断后，默认停止 PID 调节输出，也可以根据选择不停止，只做偏差报警。
- ServoLoop=0 时，PID 不参与运算，输出值为 $UsrValue + Add_Position$ ，并对其做限幅处理。当结果大于 65,535 时，输出 65,535；当结果小于 0 时输出为 0。
- Output_link 控制 PID 的输出，当 Output_link=0 时，PID 只进行运算，并将结果显示在 MCV。当 Output_link=1 时，PID 运算结果除了显示在 MCV 外，同时从 AO 输出。
- 四路 PID 对应四路 AO 输出，AO 通道与 PID 通道号绑定。AO 输出每运算周期更新一次。
- 当 PID 运行时，Q 区的 AO 数据不变化，AO 输出与 Q 区不保持一致。
- 当先择 DI 为反馈信号时，需调用高速计数功能。高速计数的 CV 值作为 PID 运算的反馈参与运算。

PID 功能框图如图所示。



PID 功能框图

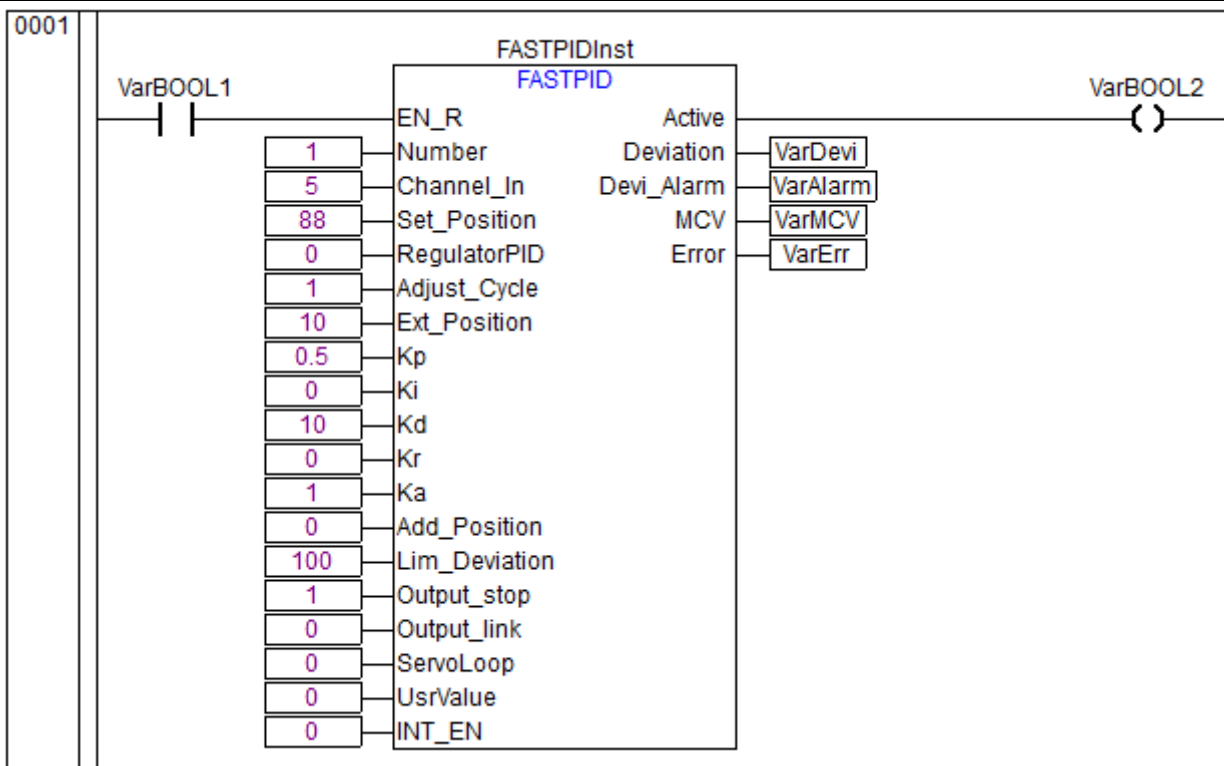
3.4.2.22.4 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarBOOL1			BOOL	FALSE	FALSE
0002	VarBOOL2			BOOL	FALSE	FALSE
0003	VarDevi			DINT	0	FALSE
0004	VarAlarm			BOOL	FALSE	FALSE
0005	VarMCV			DWORD	0	FALSE
0006	VarErr			BYTE	0	FALSE
0007	FASTPIDInst			FASTPID		FALSE

变量定义

编程语言 程序

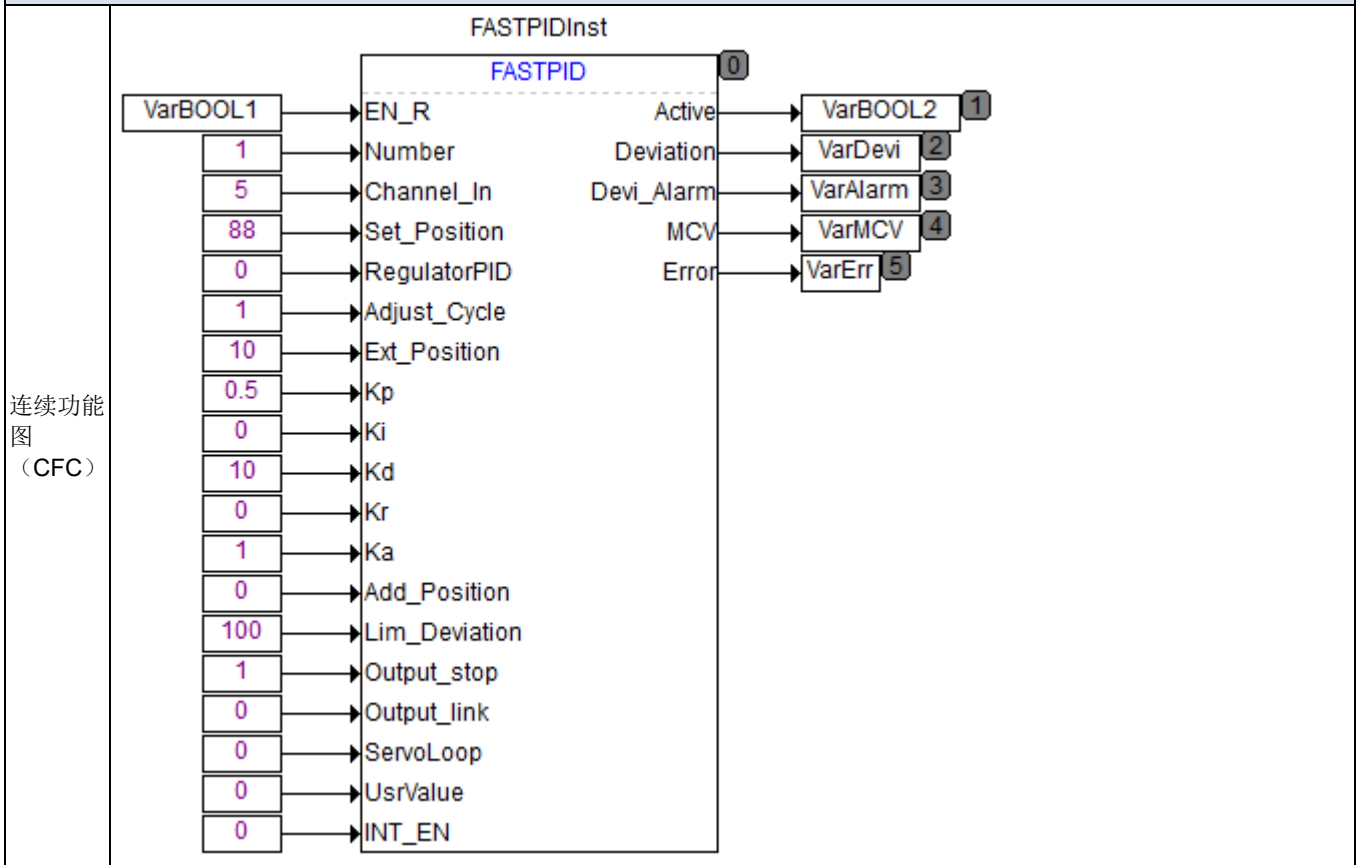
梯形图
(LD)



结构化文
本
(ST)

```
FASTPIDInst(
EN_R:=VarBOOL1,
Number:=1,
Channel_In:=5,
Set_Position:=88,
RegulatorPID:=0,
Adjust_Cycle:=1,
Ext_Position:=10,
Kp:=0.5,
Ki:=0,
Kd:=10,
Kr:=0,
Ka:=1,
Add_Position:=0,
Lim_Deviation:=100,
Output_stop:=1,
Output_link:=0,
ServoLoop:=0,
UsrValue:=0,
INT_EN:=0,
Active=>VarBOOL2,
Deviation=>VarDevi,
Devi_Alarm=>VarAlarm,
MCV=>VarMCV,
Error=> VarErr);
```

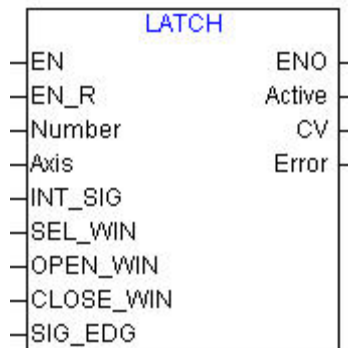
变量定义



3.4.2.23 LATCH（数据锁存,5128）

3.4.2.23.1 功能描述

启动该功能过程中，收到一个开关量信号的上升沿（或下降沿）时候，获取当前轴从参考零点开始到此刻发送（或接收）的总脉冲数。最多支持 4 个 Latch 功能块使能，可以同一个触发点锁存不同的轴的位置。



LATCH 功能块

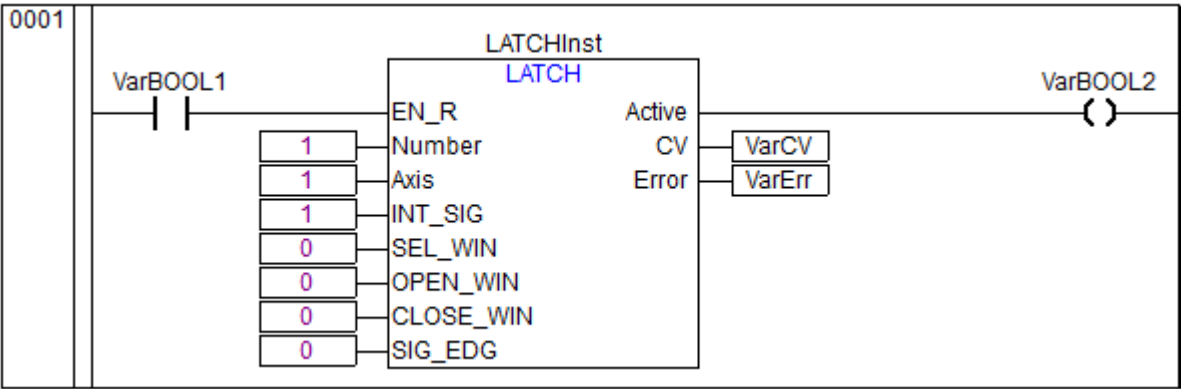
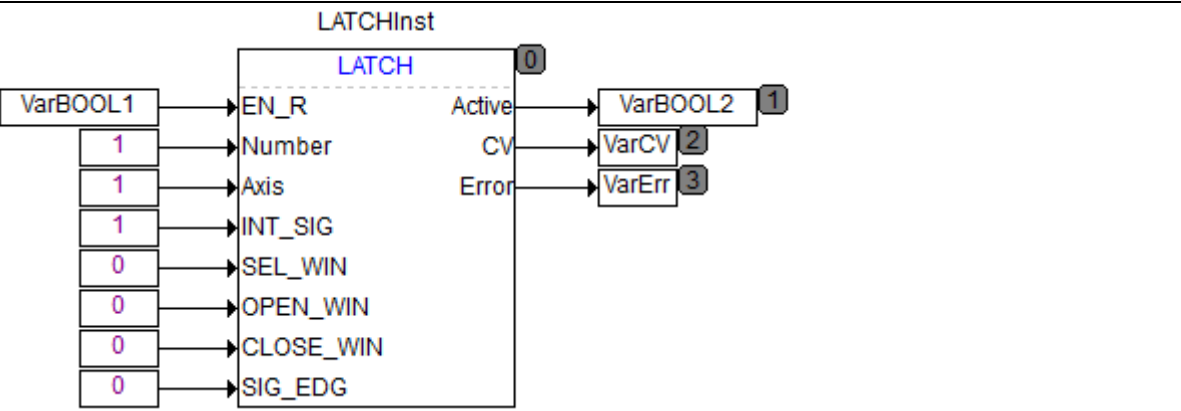
3.4.2.23.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1: 上升沿使能, 高电平有效
Number	BYTE	LATCH 编号	1、2、3、4
Axis	BYTE	指定锁存轴	=1、2、3、4、5、6、7、8
INT_SIG	BYTE	锁存点输入选择	=1、2、3、4、5、6 表示 I0.0 ~ I0.5
SEL_WIN	BYTE	窗口选择	约束 INT_SIG 硬件参数的有效性 0: 不使用窗口 1: 窗口内有效 2: 窗口外有效
OPEN_WIN	DINT	窗口 1	-1,073,741,824~1,073,741,823
CLOSE_WIN	DINT	窗口 2	-1,073,741,824~1,073,741,823
SIG_EDG	BOOL	INT_SIG 有效沿选择	0: 上升沿有效 1: 下降沿有效

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	设定成功标志	0: 未设定成功或者捕捉完成 1: 捕捉参数设定成功
CV	DINT	数据锁存值	-1,073,741,824~1,073,741,823
Error	BYTE	错误信息	=0: 正确 =1: 此路通道被占用 =2: 锁存轴选择错误 =3: 中断输入点选择错误 =4: LATCH 编号设置错误 =5: 31 位数据设置错误

3.4.2.23.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	LATCHInst			LATCH		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarCV			DINT	0	FALSE
0005	VarErr			BYTE	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> LATCHInst(EN_R:=VarBOOL1, Number:=1, Axis:=1, INT_SIG:=1, SEL_WIN:=0, OPEN_WIN:=0, CLOSE_WIN:=0, SIG_EDG:=0, Active=> VarBOOL2, CV=>VarCV, Error=> VarErr); </pre>
连续功能图 (CFC)	

程序说明：

EN_R 置位并保持时，IX0.0 锁存点输入使能上升沿时，输出总脉冲数。

3.4.2.23.4 指令说明

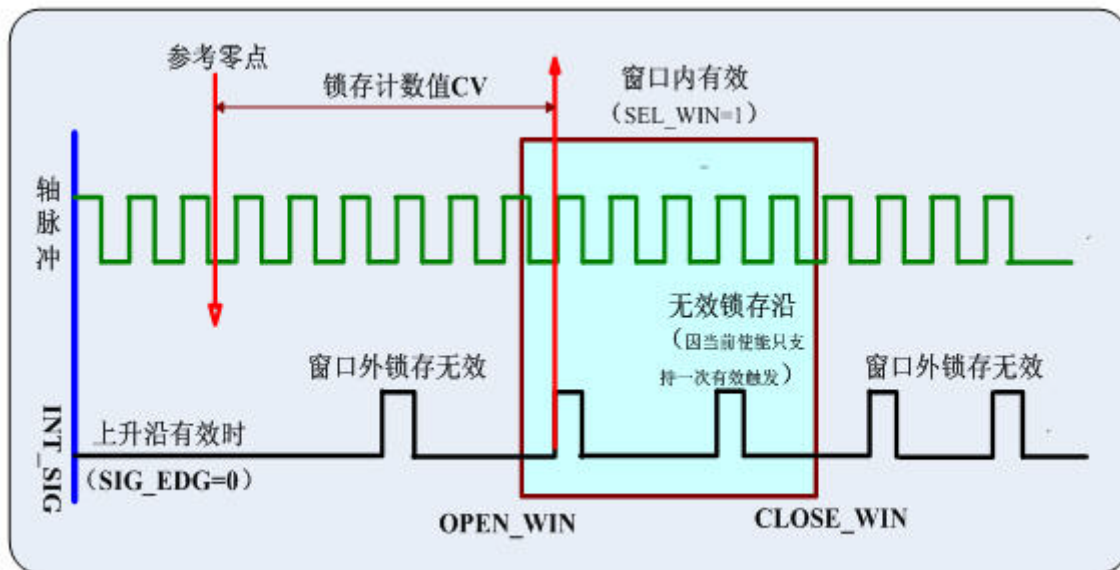
- 精度要求：开关量信号 INT_SIG 触发一个上升沿（或下降沿）的时刻，与锁存轴的脉冲时刻，两个时刻之间的差，不能超过 10 微秒。
- 设置开关量信号与轴之间的锁存关系。
- 在功能块使能时，若存在多次不同触发开关量信号，以第一个有效触发开关量信号为准。
- 中断输入通道不可与锁存轴对应统一硬件通道。如 Axis=1（I0.0）时 INT_SIG=1（I0.0）出错。

例如：下图所示。

SEL_WIN=1: INT_SIG 信号在窗口内有效。

SIG_EDG=0: INT_SIG 信号的上升沿有效。

说明：在当前使能下，窗口内的第一个 INT_SIG 上升沿有效。



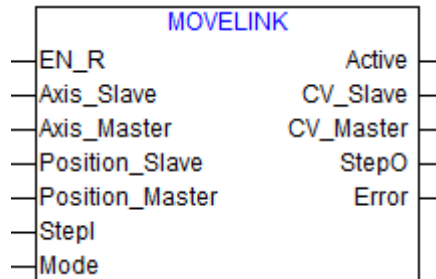
指令示例

①在 3.1.2B1 版本之前的版本中，此功能块为 CATCH。

3.4.2.24 MOVELINK（联动功能,5128）

3.4.2.24.1 功能描述

该功能建立轴 1（跟随轴）、轴 2（主动轴）之间的一种分段线性比例关系，让从动轴跟随主动轴的运动而运动。



MOVELINK 功能块

3.4.2.24.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	连接使能	0→1: 上升沿使能, 高电平有效
Axis_Slave	BYTE	指定跟随轴	=5: 输出轴 5, 脉冲输出端 Q0.0, 脉冲方向 Q0.4 =6: 输出轴 6, 脉冲输出端 Q0.1, 脉冲方向 Q0.5 =7: 输出轴 7, 脉冲输出端 Q0.2, 脉冲方向 Q0.6 =8: 输出轴 8, 脉冲输出端 Q0.3, 脉冲方向 Q0.7
Axis_Master	BYTE	指定主动轴	=1、2、3、4、5、6、7、8 分别表示设定值为: 轴 1、轴 2、...、轴 7、轴 8
Position_Slave	ARRAY[50] OF DINT	跟随轴位置设定值	-1073741824~1073741823
Position_Master	ARRAY[50] OF DINT	主轴位置设定值	-1073741824~1073741823
StepI	BYTE	跟随轴最大有效步数	1~50
Mode	BOOL	运行方式	0: 单次运行 1: 循环运行

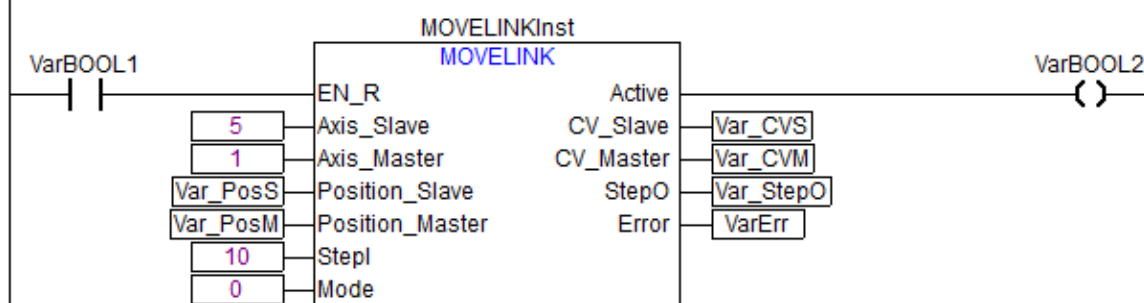
输出参数	数据类型	功能描述	参数值说明
Active	BOOL	设定成功标志	0: 设定失败 1: 设定成功
CV_Slave	DINT	跟随轴当前位置值	-1073741824~1073741823
CV_Master	DINT	主轴当前位置值	-1073741824~1073741823
StepO	BYTE	跟随轴当前步数	1~50
Error	BYTE	错误信息	=0: 正确 =1: 输出通道占用标志 =2: 指定跟随轴错误 =3: 指定主轴错误 =4: 有效步数设置错误 =5: 设置的主从轴相同 =6: 有效步数内的主轴数据为零 =7: 有效步数内的主轴、跟随轴数据设置错误

3.4.2.24.3 指令使用举例

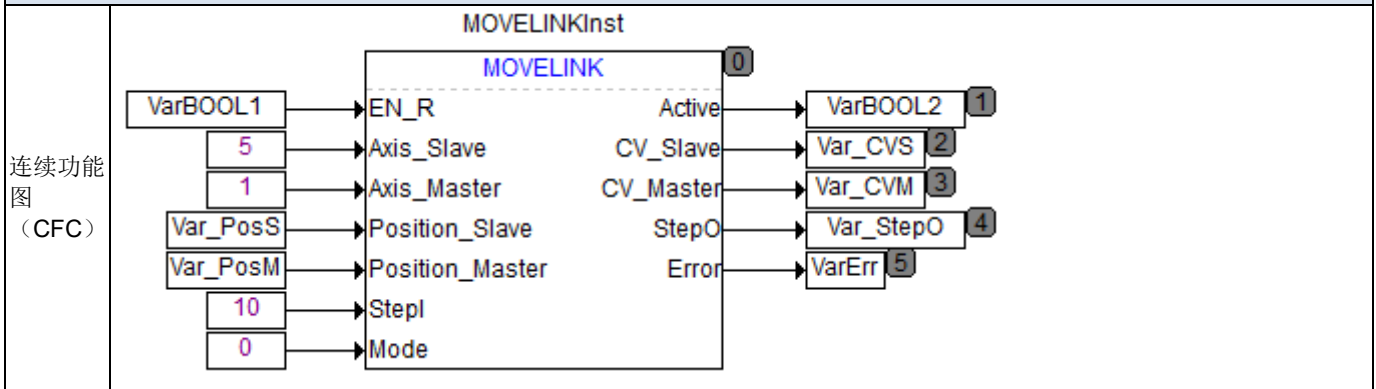
变量定义

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	Var_PosS			ARRAY[0..49] OF DINT		FALSE
0002	Var_PosM			ARRAY[0..49] OF DINT		FALSE
0003	Var_CVS			DINT	0	FALSE
0004	Var_CVM			DINT	0	FALSE
0005	Var_Step0			BYTE	0	FALSE
0006	VarErr			BYTE	0	FALSE
0007	MOVELINKInst			MOVELINK		FALSE
0008	VarBOOL2			BOOL	FALSE	FALSE
0009	VarBOOL1			BOOL	FALSE	FALSE

编程语言 程序

梯形图 (LD)	0001 
结构化文本 (ST)	<pre> MOVELINKInst(EN_R:= VarBOOL1, Axis_Slave:=5, Axis_Master:=1, Position_Slave:=Var_PosS, Position_Master:=Var_PosM, Step1:=10, Mode:=0, Active=> VarBOOL2, CV_Slave=>Var_CVS, CV_Master=>Var_CVM, StepO=> Var_StepO, Error=> VarErr); </pre>

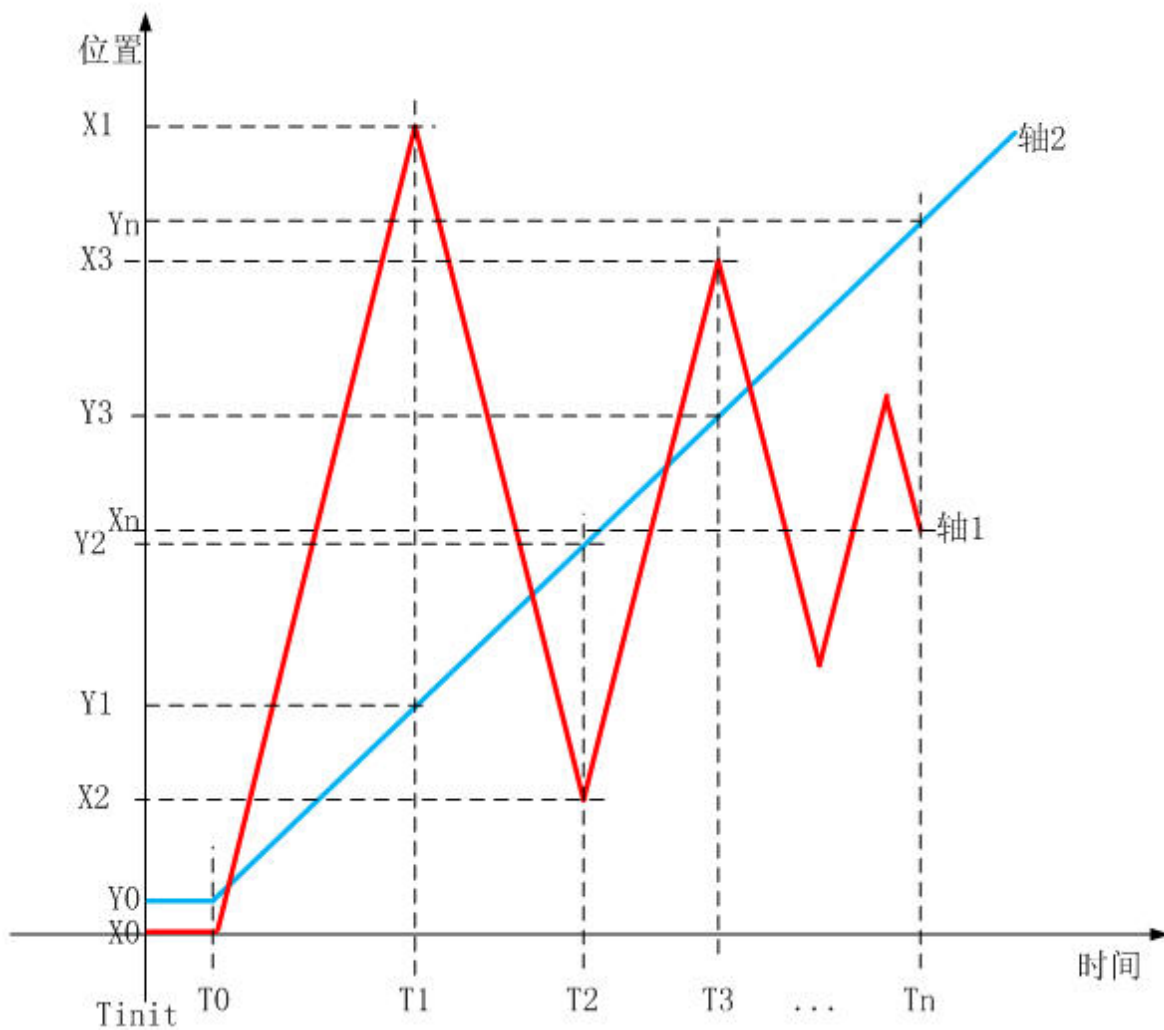
变量定义



3.4.2.24.4 指令说明

通过该功能，用户可重新设定跟随轴（轴 1）的当前位置值，也可重新设定主动轴（轴 2）的当前位置值。实现运动控制轴参考零点的灵活改变。

例如：下图。



绕线机位置连接图

- (1) T_{init} 时刻，通过命令，建立轴 1，轴 2 之间的连接关系，轴 1 开始跟随轴 2。但此时轴 2 正处于静止状态，因此轴 1 也静止。
- (2) T_0 时刻：轴 2 开始运动。轴 1 也跟随轴 2 按预定关系运动。
- (3) T_1 时刻，轴 2 运行到 Y_1 位置时，轴 1 要运行到 X_1 位置。
- (4) T_2 时刻，轴 2 运行到 Y_2 位置时，轴 1 要运行到 X_2 位置。
- (5) T_3 时刻，轴 2 运行到 Y_3 位置时，轴 1 要运行到 X_3 位置。
- (6) T_n 时刻，轴 2 运行到 Y_n 位置时，轴 1 要运行到 X_n 位置。

描述两轴之间运动关系的 2 维表。

其中 $X_1 - X_n$ ， $Y_1 - Y_n$ 的值，由用户填写，且可在线实时调整。或由其它算法赋值。

以下二维表中，用户设定的 X_n 、 Y_n 的值是相对于 X_{n-1} 、 Y_{n-1} 的增量值。

如若设定 $x_1=10$ ， $x_2=-20$ ，则表示：

第 1 步轴 1 从当前位置正向走 10 个脉冲，

第 2 步轴 1 从当前位置反向走 20 个脉冲。

n 最大为 50.

步数	轴 1	轴 2
1	X1	Y1
2	X2	Y2
3	X3	Y3
...	...	...
n	Xn	Yn

Table 中最大有效步数。例如：step =2，则 Table 的有效步数为前 2 步。Step=3，则 Table 的有效步数为前 3 步。

- mode: 运行方式。提供两种运行方式：单次运行，循环运行。
 - 单次运行：执行到 step，就自动停止，自动解除两个轴之间的连接关系。
 - 循环运行：执行到 n，并回到表格第一行再从头执行。直至通过“解除连接”命令才停止运动。

3.4.2.25 CONNECT（电子齿轮,5128）

3.4.2.25.1 功能描述

当前轴的目标位置与被连接轴定义的测量位置通过电子齿轮联接。



CONNECT 功能块

3.4.2.25.2 参数说明

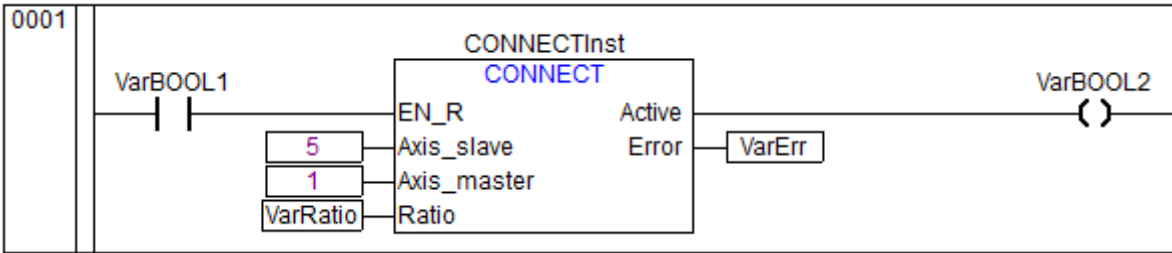
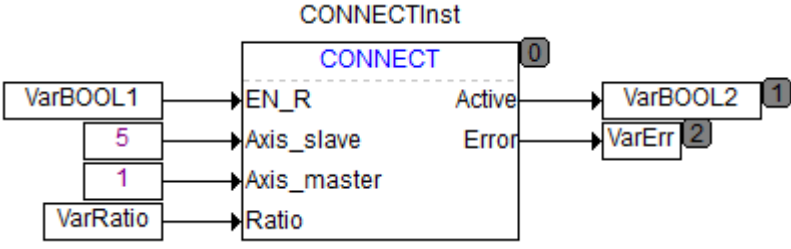
输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1: 上升沿使能，高电平有效
Axis_slave	BYTE	指定跟随轴	=5: 输出轴 5，脉冲输出端 Q0.0，脉冲方向 Q0.4 =6: 输出轴 6，脉冲输出端 Q0.1，脉冲方向 Q0.5 =7: 输出轴 7，脉冲输出端 Q0.2，脉冲方向 Q0.6 =8: 输出轴 8，脉冲输出端 Q0.3，脉冲方向 Q0.7
Axis_master	BYTE	指定主动轴	=1、2、3、4、5、6、7、8 分别表示设定值为：轴 1、轴 2、、轴 7、轴 8
Ratio	REAL	设定比率	电子齿轮的比率，比率可以随时改变，不需取消以前的比率。

输入参数	数据类型	功能描述	参数值说明
			取值范围（-1000.000~1000.000）

输出参数	数据类型	功能描述	参数值说明
Active	BOOL	设定成功标志	0: 设定失败 1: 设定成功
Error	BYTE	错误信息	=0: 正确 =1: 跟随轴占用标志 =2: 指定跟随轴错误 =3: 指定主轴错误 =4: 设定比率错误 =5: 设置的主从轴相同

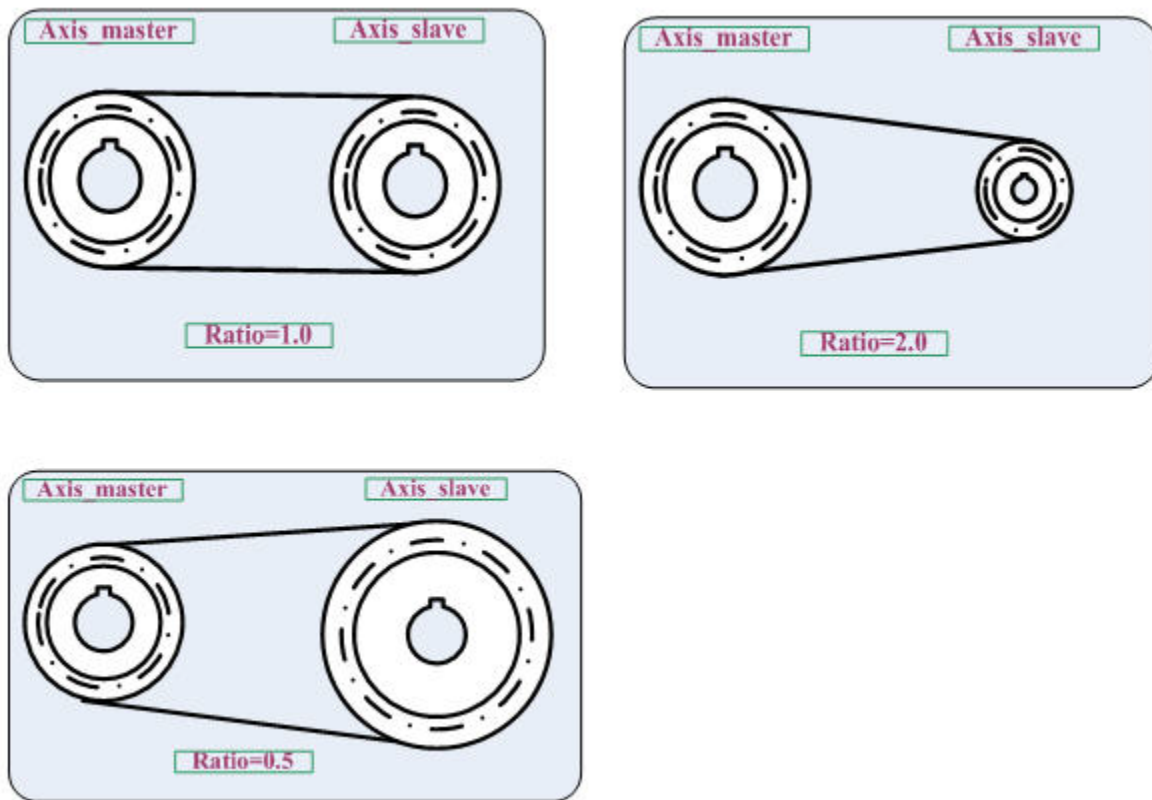
3.4.2.25.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	VarRatio			REAL	0	FALSE
0002	VarErr			BYTE	0	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarBOOL1			BOOL	FALSE	FALSE
0005	CONNECTInst			CONNECT		FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>CONNECTInst(EN_R:= VarBOOL1, Axis_slave:=5, Axis_master:=1, Ratio:=VarRatio, Active=>VarBOOL2, Error=>VarErr);</pre>
连续功能图 (CFC)	

3.4.2.25.4 指令说明

Axis_master 为被检测轴，Axis_slave 为检测到的转速与齿轮比相乘后的输出轴。



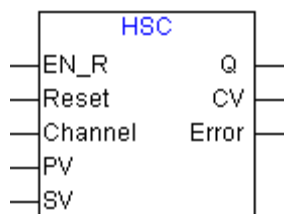
检测轴与输出轴的比率关系

3.4.3 计数指令

3.4.3.1 HSC(高速计数,5109i)

3.4.3.1.1 功能

HSC 是一个 32 位单相高速计数器，通过调用该指令可以实现对输入脉冲的计数（增计数）。



HSC 功能块

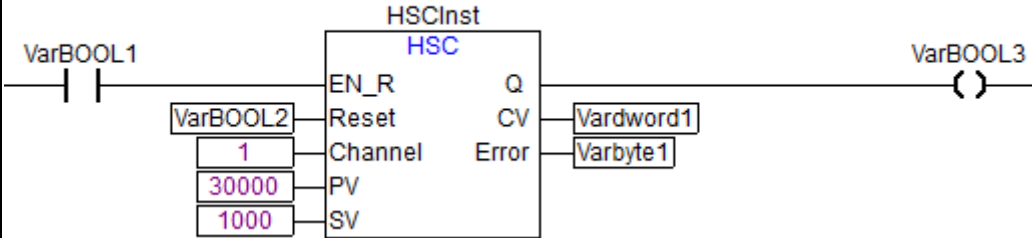
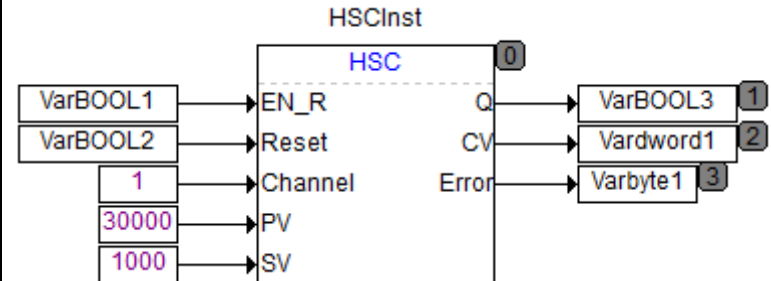
3.4.3.1.2 参数说明

输入参数	数据类型	功能描述	参数值说明
EN_R	BOOL	使能	0→1 上升沿使能，电平保持
Reset	BOOL	清零	0→1: 上升沿使能，ERROR=4 时 ERROR 值清零且 CV 清零，这时 CV 清零后 Reset 高电平仍可以正常计数
Channel	BYTE	通道选择	=1: 通道 1, %IX2.0 =2: 通道 2, %IX3.0
PV	DWORD	目标值	0~2 ³² -1
SV	DWORD	初始值	0~2 ³² -1, 且 SV<PV

输出参数	数据类型	功能描述	值
Q	BOOL	计数到达标志	0: CV<PV 1: CV>=PV
CV	DWORD	当前计数值	0~2 ³² -1
Error	BYTE	错误信息	=0: 正确 =1: Channel 参数错误 =2: 当前通道已被调用 =3: PV<=SV =4: CV 值溢出

3.4.3.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	HSCInst			HSC		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	VarBOOL3			BOOL	FALSE	FALSE
0005	VarDword1			DWORD	0	FALSE
0006	Varbyte1			BYTE	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	
结构化文本 (ST)	<pre> HSCInst(EN_R:=VarBOOL1, Reset:=VarBOOL2, Channel:=0, PV:=30000, SV:=1000, Q=>VarBOOL3, CV=>Vardword1, Error=>Varbyte1); </pre>
连续功能图 (CFC)	

程序说明：

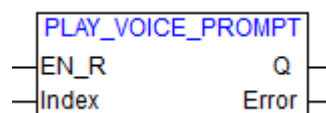
EN_R 置位并保持时，通道 1 脉冲进行向上计数，计数范围为 30000 个脉冲，计数初值从 1000 开始。Reset 为 1 时，计数值复位。

3.4.4 语音控制指令

3.4.4.1 PLAY_VOICE_PROMPT（播放指定语音段,5109i）

3.4.4.1.1 功能

提示音播放指令可以实现 CPU 模块内置语音的播放功能。



PLAY_VOICE_PROMPT 指令模块示意图

3.4.4.1.2 参数说明

输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0→1 上升沿使能，电平保持	0
Index	BYTE	提示音索引	0~255，取值必须与 CPU 模块 Flash 存储的语音信息相对应	0

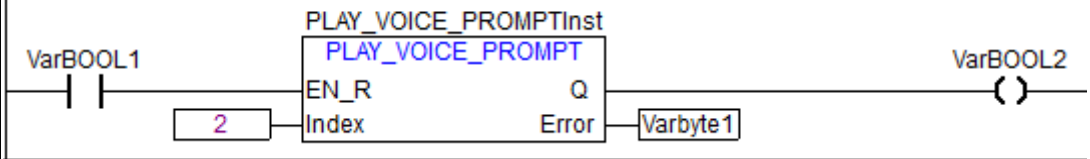
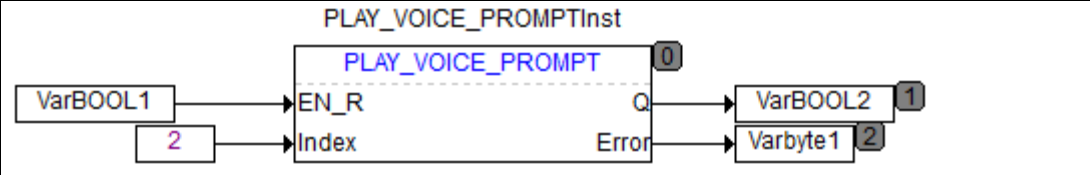
输出参数	数据类型	功能描述	参数值说明
Q	BOOL	播放完成标志	播放完成标志，完成输出 1，否则输出 0
Error	BYTE	错误信息	=0：播放成功 =1：播放失败



- Index 的范围必须与实际 Flash 中的语音索引对应。
- 如果 Q 未输出 1 重新使能该指令模块，则语音播放将停止，并开始播放新语音。

3.4.4.1.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	PLAY_VOICE_PROMPTInst			PLAY...		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	Varbyte1			BYTE	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>PLAY_VOICE_PROMPTInst(EN_R:=VarBOOL1, Index:=2, Q=>VarBOOL2, Error=>Varbyte1);</pre>
连续功能图 (CFC)	

程序说明：

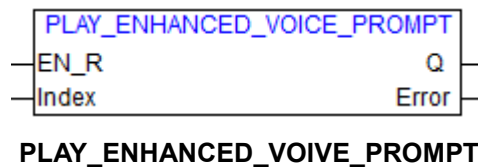
EN_R 置位并保持时，将 CPU 模块中实际 Flash 中的语音索引为 2 的一段语音信息播放一次。

3.4.4.2 PLAY_ENHANCED_VOIVE_PROMPT（播放“滴”提示音后播放语音段,5109i）

在 V3.1.4B3 版本中，该指令名称为“PLAY_VOICE（播放“滴”提示音后播放语音段,5109i）”。

3.4.4.2.1 功能

增强型提示音播放指令可以实现 CPU 模块内置语音的播放功能，并在播放前自动播放 Index 为 0 的打点音。



3.4.4.2.2 参数说明

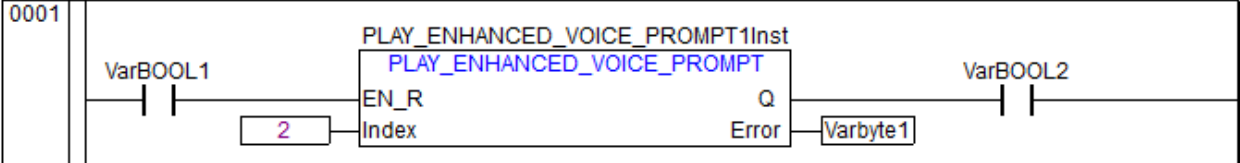
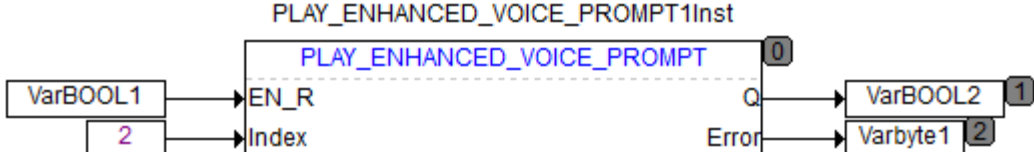
输入参数	数据类型	功能描述	参数值说明	默认值
EN_R	BOOL	使能	0→1 上升沿使能，电平保持	0
Index	BYTE	提示音索引	0~255，取值必须与 CPU 模块 Flash 存储的语音信息相对应	0

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	播放完成标志	播放完成标志，完成输出 1，否则输出 0
Error	BYTE	错误信息	=0：播放成功 =1：播放失败

3.4.4.2.3 指令使用举例

变量定义

序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	PLAY_ENHANCED_VOICE_PROMPT1Inst			PLAY_ENHANCED_VOICE_PROMPT		FALSE
0002	VarBOOL1			BOOL	FALSE	FALSE
0003	VarBOOL2			BOOL	FALSE	FALSE
0004	Varbyte1			BYTE	0	FALSE

变量定义	
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>PLAY_ENHANCED_VOICE_PROMPT1Inst(EN_R:=VarBOOL1, Index:=2, Q=>VarBOOL2, Error=>Varbyte1);</pre>
连续功能图 (CFC)	

程序说明：

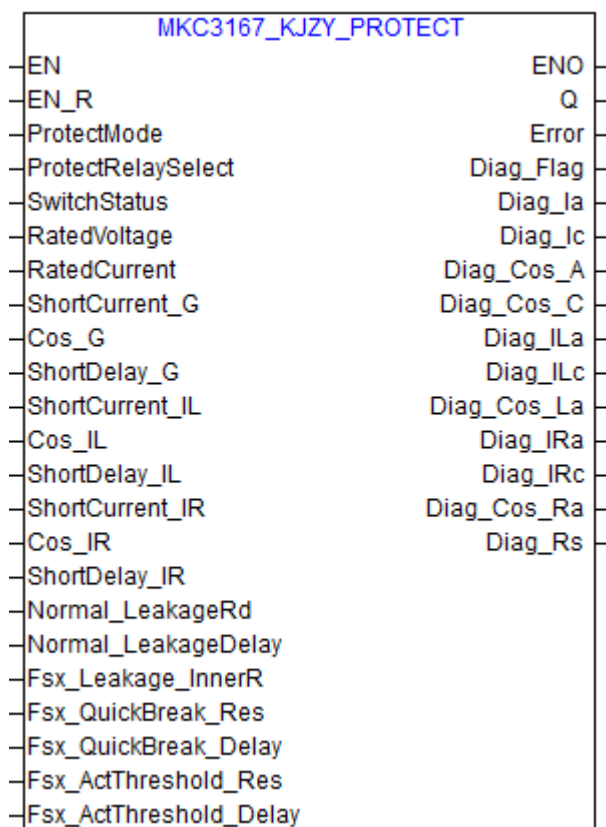
EN_R 置位并保持时，将 CPU 模块中播放完“滴”声之后，再播放实际 Flash 中的语音索引为 2 的一段语音信息播放一次。

3.4.5 MKC3167 保护指令

3.4.5.1 MKC3167_KJZY_PROTECT（一体化移变保护算法,3167）

3.4.5.1.1 功能

实现 KJZY 应用模式下，一体化移变保护算法功能。



MKC3167_KJZY_PROTECT 指令

3.4.5.1.2 参数说明

输入参数	数据类型	参数说明	默认值
EN_R	BOOL	模块使能	0
ProtectMode	BYTE	模块保护功能使能选择 普通漏电和反时限漏电二选一 ProtectMode 字节的详细定义请参见表 20	0
ProtectRelaySelect	BYTE	参与保护动作的继电器选择 缺省值≤63	0
SwitchStatus	BYTE	合闸状态 0: 分闸 1: 合闸	0
RatedVoltage	WORD	额定电压, 取值范围: 不超过类型取值范围	0
RatedCurrent	WORD	额定电流, 取值范围: 不超过类型取值范围>0 非零值	0
ShortCurrent_G	WORD	高压侧电流保护, 电流阈值设置 取值范围: 不超过类型取值范围	0
Cos_G	BYTE	高压侧电流保护, 相敏阈值设置 取值范围: ≤100	0
ShortDelay_G	WORD	高压侧电流保护, 延时设置 取值范围: <65535	0
ShortCurrent_IL	WORD	低压 I 路电流保护, 电流阈值设置	0

输入参数	数据类型	参数说明	默认值
		取值范围：不超过类型取值范围	
Cos_IL	BYTE	低压 I 路电流保护，相敏阈值设置 取值范围：≤100	0
ShortDelay_IL	WORD	低压 I 路电流保护，延时设置 取值范围：<65535	0
ShortCurrent_IR	WORD	低压 II 路电流保护，电流阈值设置 取值范围：不超过类型取值范围	0
Cos_IR	BYTE	低压 II 路电流保护，相敏阈值设置 取值范围：≤100	0
ShortDelay_IR	WORD	低压 II 路电流保护，延时设置 取值范围：<65535	0
Normal_LeakageRd	WORD	普通漏电电阻阈值设置 取值范围：不超过类型取值范围	0
Normal_LeakageDelay	WORD	普通漏电延时设置 取值范围：<65535	0
Fsx_Leakage_InnerR	WORD	反时限漏电设备内阻设置 取值范围：≤10000	0
Fsx_QuickBreak_Res	WORD	反时限漏电速断门限值设定 取值范围：<65535	0
Fsx_QuickBreak_Delay	WORD	反时限漏电速断延时设定 取值范围：<65535	0
Fsx_ActThreshold_Res	WORD	反时限漏电动作门限值设定 取值范围：<65535 且 ≥速断门限值	0
Fsx_ActThreshold_Delay	WORD	反时限漏电动作延时设定 取值范围：<65535 且 ≥速断延时值	0

ProtectMode 字节定义

位	ProtectMode	说明
BIT0	HighVolCurrAmp	高压电流幅值
BIT1	HighVolPhSense	高压电流相敏
BIT2	LowVolCH1CurrAmp	低压 1 路电流幅值
BIT3	LowVolCH1PhSense	低压 1 路电流相敏
BIT4	LowVolCH2CurrAmp	低压 2 路电流幅值
BIT5	LowVolCH2PhSense	低压 2 路电流相敏
BIT6	NorLeakage	普通漏电
BIT7	FsxLeakage	反时限漏电

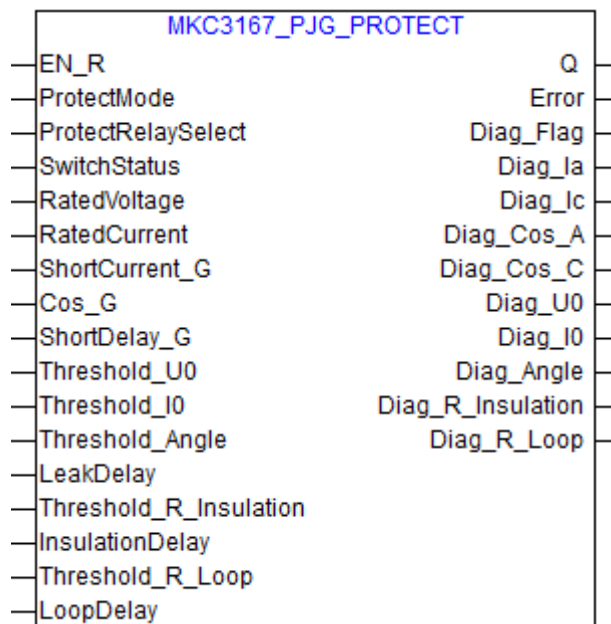
输出参数	数据类型	功能描述	默认值
Q	BOOL	保护功能块输出	FALSE
Error	BYTE	错误代码输出	0
Diag_Flag	BYTE	保护诊断 Diag_Flag 每个位的含义和 ProtectMode 每个位代表的保护类型含义一样，请参见表 ProtectMode 字节定义	0
Diag_Ia	WORD	高压侧短路保护 IA 电流诊断数据	0
Diag_Ic	WORD	高压侧短路保护 IC 电流诊断数据	0
Diag_Cos_A	BYTE	高压侧 A 相功率因数诊断数据	0
Diag_Cos_C	BYTE	高压侧 C 相功率因数诊断数据	0

输出参数	数据类型	功能描述	默认值
Diag_ILa	WORD	低压 I 路短路保护 IA 电流诊断数据	0
Diag_ILc	WORD	低压 I 路短路保护 IC 电流诊断数据	0
Diag_Cos_La	BYTE	低压 I 路 A 相功率因数诊断数据	0
Diag_IRa	WORD	低压 II 路短路保护 IA 电流诊断数据	0
Diag_IRc	WORD	低压 II 路短路保护 IC 电流诊断数据	0
Diag_Cos_Ra	BYTE	低压 II 路 A 相功率因数诊断数据	0
Diag_Rs	WORD	漏电阻	0

3.4.5.2 MKC3167_PJG_PROTECT（高爆保护算法,3167）

3.4.5.2.1 功能

实现 PJG 应用模式下，高爆保护算法功能。



MKC3167_PJG_PROTECT 指令

3.4.5.2.2 参数说明

输入参数	数据类型	参数说明	默认值
EN_R	BOOL	模块使能	FALSE
ProtectMode	BYTE	模块保护功能使能选择 取值范围：≤16 且绝缘监视和回路监视二选一	0
ProtectRelaySelect	BYTE	参与保护动作的继电器选择 取值范围：≤63	0
SwitchStatus	BYTE	合闸状态 0：分闸	0

输入参数	数据类型	参数说明	默认值
		1: 合闸	
RatedVoltage	WORD	额定电压, 取值范围: 不超过类型取值范围	0
RatedCurrent	WORD	额定电流, 取值范围: 不超过类型取值范围 >0 非零值	0
ShortCurrent_G	WORD	高压侧电流保护, 电流阈值设置 取值范围: 不超过类型取值范围	0
Cos_G	BYTE	高压侧电流保护, 相敏阈值设置 取值范围: ≤ 100	0
ShortDelay_G	WORD	高压侧电流保护, 延时设置 取值范围: < 65535	0
Threshold_U0	WORD	功率方向漏电零序电压设置 取值范围: 不超过类型取值范围 ≤ 12000	0
Threshold_I0	WORD	功率方向漏电零序电流设置 取值范围: ≤ 1000	0
Threshold_Angle	WORD	功率方向漏电相位角设置 取值范围: $\leq 360, 65535$	0
LeakDelay	WORD	功率方向漏电级差延时设置 取值范围: < 65535	0
Threshold_R_Insulation	WORD	绝缘电阻设置 取值范围: ≤ 2000	0
InsulationDelay	WORD	绝缘监视延时设置 取值范围: < 65535	0
Threshold_R_Loop	WORD	回路监视电阻设置 取值范围: ≤ 2000	0
LoopDelay	WORD	回路监视延时设置 取值范围: < 65535	0

输出参数	数据类型	功能描述	默认值
Q	BOOL	保护功能块输出	FALSE
Error	BYTE	错误代码输出	0
Diag_Flag	BYTE	保护诊断	0
Diag_Ia	WORD	高压侧短路保护 IA 电流诊断数据	0
Diag_Ic	WORD	高压侧短路保护 IC 电流诊断数据	0
Diag_Cos_A	BYTE	高压侧 A 相功率因数诊断数据	0
Diag_Cos_C	BYTE	高压侧 C 相功率因数诊断数据	0
Diag_U0	WORD	功率方向漏电零序电压诊断数据	0
Diag_I0	WORD	功率方向漏电零序电流诊断数据	0
Diag_Angle	WORD	功率方向漏电零序相位角诊断数据	0
Diag_R_Insulation	WORD	绝缘电阻监视保护诊断数据	0
Diag_R_Loop	WORD	回路电阻监视保护诊断数据	0

保护功能定义

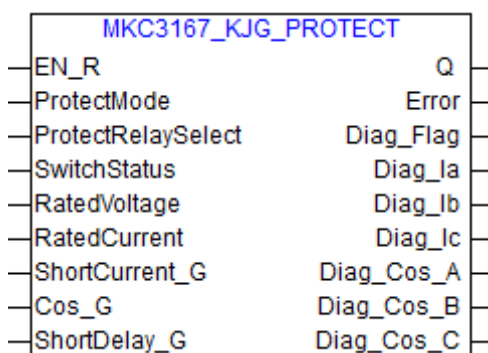
ProtectMode	说明
BIT0 HighVolCurrAmp	高压电流幅值
BIT1 HighVolPhSense	高压电流相敏
BIT2 ZeroPowVectLeakage	零序功率方向性选漏
BIT3 InsulationMonitoring	绝缘监视保护
BIT4 LoopMonitoring	回路监视保护

ProtectMode		说明
BIT5		保留
BIT6		保留
BIT7		保留

3.4.5.3 MKC3167_KJG_PROTECT（移变高压侧保护算法,3167）

3.4.5.3.1 功能

实现 KJG 应用模式下，移变高压侧保护算法功能。



MKC3167_KJG_PROTECT 指令

3.4.5.3.2 参数说明

输入参数	数据类型	参数说明	默认值
EN_R	BOOL	模块使能	FALSE
ProtectMode	BYTE	模块保护功能使能选择 取值范围：≤7	0
ProtectRelaySelect	BYTE	参与保护动作的继电器选择 取值范围：≤63	0
SwitchStatus	BYTE	合闸状态 0：分闸 1：合闸	0
RatedVoltage	WORD	额定电压 取值范围：不超过类型取值范围	0
RatedCurrent	WORD	额定电流 取值范围：不超过类型取值范围>0	0
ShortCurrent_G	WORD	高压侧电流保护，电流阈值设置 取值范围：不超过类型取值范围	0
Cos_G	BYTE	高压侧电流保护，相敏阈值设置 取值范围：≤100	0
ShortDelay_G	WORD	高压侧电流保护，延时设置 取值范围：<65535	0

输出参数	数据类型	功能描述	默认值
Q	BOOL	保护功能块输出	FALSE
Error	BYTE	错误代码输出	0
Diag_Flag	BYTE	保护诊断	0
Diag_Ia	WORD	高压侧短路保护 IA 电流诊断数据	0
Diag_Ib	WORD	高压侧短路保护 IB 电流诊断数据	0
Diag_Ic	WORD	高压侧短路保护 IC 电流诊断数据	0
Diag_Cos_A	BYTE	高压侧 A 相功率因数诊断数据	0
Diag_Cos_B	BYTE	高压侧 B 相功率因数诊断数据	0
Diag_Cos_C	BYTE	高压侧 C 相功率因数诊断数据	0

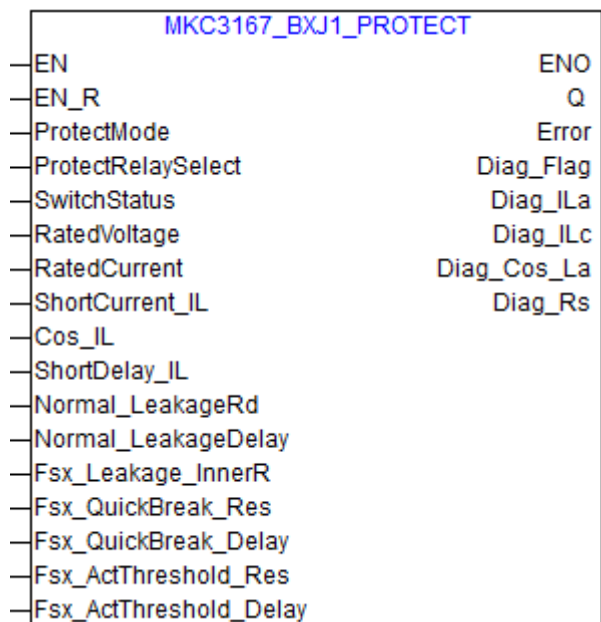
保护功能使能位说明

ProtectMode	说明
BIT0 HighVolCurrAmp	高压电流幅值
BIT1 HighVolPhSense	高压电流相敏
BIT2 Pt100Short	PT100 短接保护
BIT3	保留
BIT4	保留
BIT5	保留
BIT6	保留
BIT7	保留

3.4.5.4 MKC3167_BXJ1_PROTECT（移变低压侧保护算法,3167）

3.4.5.4.1 功能

实现 BXJ1 应用模式下，移变高压侧保护算法功能。



MKC3167_BXJ1_PROTECT 指令

3.4.5.4.2 参数说明

输入参数	数据类型	参数说明	默认值
EN_R	BOOL	模块使能	FALSE
ProtectMode	BYTE	模块保护功能使能选择 取值范围：≤15 且普通漏电和反时限漏电二选一	0
ProtectRelaySelect	BYTE	参与保护动作的继电器选择 取值范围：≤63	0
SwitchStatus	BYTE	合闸状态 0：分闸 1：合闸	0
RatedVoltage	WORD	额定电压 取值范围：不超过类型取值范围 >0 非零值	0
RatedCurrent	WORD	额定电流 取值范围：不超过类型取值范围	0
ShortCurrent_IL	WORD	低压 I 路电流保护，电流阈值设置 取值范围：不超过类型取值范围	0
Cos_IL	BYTE	低压 I 路电流保护，相敏阈值设置 取值范围：≤100	0
ShortDelay_IL	WORD	低压 I 路电流保护，延时设置 取值范围：<65535	0
Normal_LeakageRd	WORD	普通漏电电阻阈值设置 取值范围：不超过类型取值范围	0
Normal_LeakageDelay	WORD	普通漏电延时设置 取值范围：<65535	0
Fsx_Leakage_InnerR	WORD	反时限漏电设备内阻设置 取值范围：≤10000	0
Fsx_QuickBreak_Res	WORD	反时限漏电速断门限值设定 取值范围：<65535	0

输入参数	数据类型	参数说明	默认值
Fsx_QuickBreak_Delay	WORD	反时限漏电速断延时设定 取值范围：<65535	0
Fsx_ActThreshold_Res	WORD	反时限漏电动作门限值设定 取值范围：<65535 且 ≥速断门限值	0
Fsx_ActThreshold_Delay	WORD	反时限漏电动作延时设定 取值范围：<65535 且 ≥速断延时值	0

输出参数	数据类型	功能描述	默认值
Q	BOOL	保护功能块输出	FALSE
Error	BYTE	错误代码输出	0
Diag_Flag	BYTE	保护诊断	0
Diag_ILa	WORD	低压 I 路短路保护 IA 电流诊断数据	0
Diag_ILc	WORD	低压 I 路短路保护 IC 电流诊断数据	0
Diag_Cos_La	BYTE	低压 I 路 A 相功率因数诊断数据	0
Diag_Rs	WORD	漏电电阻	0

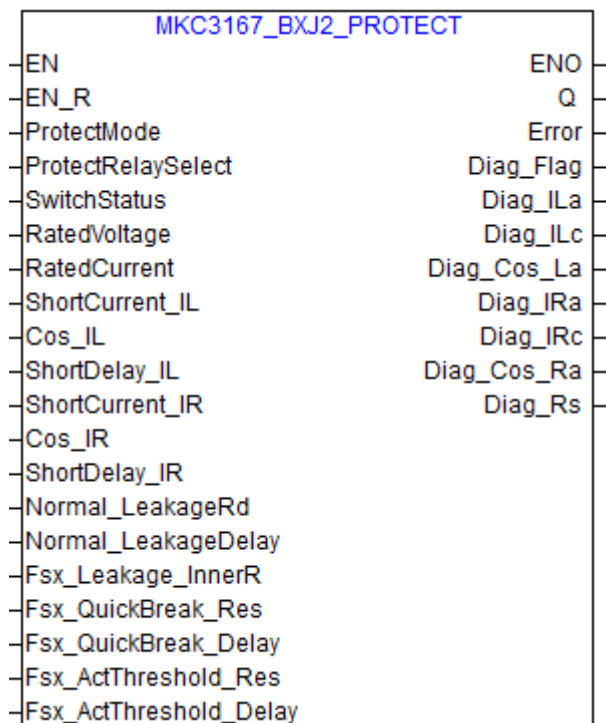
保护功能使能位说明

ProtectMode		说明
BIT0	LowVolCH1CurrAmp	低压 1 路电流幅值
BIT1	LowVolCH1PhSense	低压 1 路电流相敏
BIT2	NorLeakage	普通漏电
BIT3	FsxLeakage	反时限漏电
BIT4		保留
BIT5		保留
BIT6		保留
BIT7		保留

3.4.5.5 MKC3167_BXJ2_PROTECT（移变低压侧双回路保护算法,3167）

3.4.5.5.1 功能

实现 BXJ2 应用模式下，移变低压侧双回路保护算法功能。



MKC3167_BXJ2_PROTECT 指令

3.4.5.5.2 参数说明

输入参数	数据类型	参数说明	默认值
EN_R	BOOL	模块使能	FALSE
ProtectMode	BYTE	模块保护功能使能选择 取值范围：≤63 且普通漏电和反时限漏电二选一	0
ProtectRelaySelect	BYTE	动作继电器选择 取值范围：≤63	0
SwitchStatus	BYTE	合闸状态 0：分闸 1：合闸	0
RatedVoltage	WORD	额定电压 取值范围：不超过类型取值范围 >0 非零值	0
RatedCurrent	WORD	额定电流 取值范围：不超过类型取值范围	0
ShortCurrent_IL	WORD	低压 I 路电流保护，电流阈值设置 取值范围：不超过类型取值范围	0
Cos_IL	BYTE	低压 I 路电流保护，相敏阈值设置 取值范围：≤100	0
ShortDelay_IL	WORD	低压 I 路电流保护，延时设置 取值范围：≤65535	0
ShortCurrent_IR	WORD	低压 II 路电流保护，电流阈值设置 取值范围：不超过类型取值范围	0
Cos_IR	BYTE	低压 II 路电流保护，相敏阈值设置 取值范围：≤100	0

输入参数	数据类型	参数说明	默认值
ShortDelay_IR	WORD	低压 II 路电流保护，延时设置 取值范围：<65535	0
Normal_LeakageRd	WORD	普通漏电电阻阈值设置 取值范围：不超过类型取值范围	0
Normal_LeakageDelay	WORD	普通漏电延时设置 取值范围：<65535	0
Fsx_Leakage_InnerR	WORD	反时限漏电设备内阻设置 取值范围：≤10000	0
Fsx_QuickBreak_Res	WORD	反时限漏电速断门限值设定 取值范围：<65535	0
Fsx_QuickBreak_Delay	WORD	反时限漏电速断延时设定 取值范围：<65535	0
Fsx_ActThreshold_Res	WORD	反时限漏电动作门限值设定 取值范围：<65535 且 ≥速断门限值	0
Fsx_ActThreshold_Delay	WORD	反时限漏电动作延时设定 取值范围：<65535 且 ≥速断延时值	0

输出参数	数据类型	功能描述	默认值
Q	BOOL	保护功能块输出	FALSE
Error	BYTE	错误代码输出	0
Diag_Flag	BYTE	保护诊断	0
Diag_ILa	WORD	低压 I 路短路保护 IA 电流诊断数据	0
Diag_ILc	WORD	低压 I 路短路保护 IC 电流诊断数据	0
Diag_Cos_La	BYTE	低压 I 路 A 相功率因数诊断数据	0
Diag_IRa	WORD	低压 II 路短路保护 IA 电流诊断数据	0
Diag_IRc	WORD	低压 II 路短路保护 IC 电流诊断数据	0
Diag_Cos_Ra	BYTE	低压 II 路 A 相功率因数诊断数据	0
Diag_Rs	WORD	漏电电阻	0

保护功能使能位说明

ProtectMode		说明
BIT0	LowVolCH1CurrAmp	低压 1 路电流幅值
BIT1	LowVolCH1PhSense	低压 1 路电流相敏
BIT2	LowVolCH2CurrAmp	低压 2 路电流幅值
BIT3	LowVolCH2PhSense	低压 2 路电流相敏
BIT4	NorLeakage	普通漏电
BIT5	FsxLeakage	反时限漏电
BIT6		保留
BIT7		保留

3.4.6 故障录波

3.4.6.1 MKC3167_IS_ENABLE_CHANNEL(是否使能 IO 板数据上传与下发通道号)

3.4.6.1.1 功能描述

实现通道实时数据上传使能功能，将保护 IO 采集到的指定通道实时数据上传到 CPU 模块。

3.4.6.1.2 参数描述

输入参数	数据类型	参数值说明	初始值
EN_R	BOOL	使能，上升沿有效	FALSE
ChannelID	BYTE	通道号：1-16，1 代表第一个通道，2 代表第二个通道，以此类推，总共 16 个通道	0
IsEnable	BOOL	是否使能 IO 板上报数据 TRUE：使能 FALSE：未使能	FALSE
输出参数	数据类型	参数值说明	初始值
Q	BOOL	使能功能块输出，执行完成置 TRUE	FALSE
Error	BYTE	错误码 0:运行完成 1:通道 ID 输入错误	0

目前通道数据采集录波功能可支持高压侧(KJG)场景、低压侧双回路(BXJ2)场景使用。具体的通道号和硬件通道对应关系如下表所示：

通道号和硬件通道对应关系

硬件通道	ChannelID	高压侧(KJG)	低压单回路(BXJ2)	低压双回路(BXJ2)
L/N	12	电压 U _{sys}	\	\
A1	11	\	电压	第一回路电压
A2	13	\	\	第二回路电压
A3	14	\	\	\
A4	1	\	\	\
A5	3	电流 A	\	\
A6	5	电流 C	\	\
A7	7	\	电流 A	第一回路电流 A
A8	8	\	电流 C	第一回路电流 C
A9	15	\	\	第二回路电流 A
A10	16	\	\	第二回路电流 C
RS	9	\	漏电电阻 RS	漏电电阻 RS

3.4.6.2 MKC3167_NORMAL_READ（正常模式数据读取）

3.4.6.2.1 功能描述

实现指定通道实时数据读取功能，将保护 IO 上传给 CPU 模块的通道实时数据拷贝到指定地址。

-  注意事项：MKC3167_NORMAL_READ 功能块在同一组态工程中仅支持调用一次，每次只读取一个通道数据。

3.4.6.2.2 参数描述

输入参数	数据类型	参数值说明	默认值
In	BOOL	使能，高电平有效	FALSE
ChannelID	BYTE	需要获取实时数据的通道号：1-16，1 代表第一个通道，2 代表第二个通道，以此类推，总共 16 个通道	0
DataLen	WORD	数据长度，地址空间大小不超过 120 个采样点	0
Address	POINTER TO WORD	数据存放地址	0
输出参数	数据类型	参数值说明	默认值
Done	BOOL	执行结果： 执行成功置 TRUE	FALSE
Busy	BOOL	TRUE：忙 FALSE：空闲	FALSE
Error	BYTE	错误码 0:运行完成； 1:读取通道 ID 输入错误； 2:数据长度非法； 3:读取地址非法； 4:未使能通道上传功能；	0
RealDataLen	WORD	数据长度： 实际拷贝数据长度，最大不超过 DataLen	0

3.4.6.3 MKC3167_GET_ERR_CHNL_ID（获取故障通道 ID）

3.4.6.3.1 功能描述

上报故障通道 ID 号。

-  注意事项：目前通道故障录波功能仅支持高压侧(KJG)场景和低压侧双回路(BXJ2)场景使用。
具体获取的故障通道号和模块硬件通道对应关系见表通道号和硬件通道对应关系。

3.4.6.3.2 参数描述

输入参数	数据类型	参数值说明	默认值
EN_R	BOOL	是否使能，上升沿有效	FALSE
输出参数	数据类型	参数值说明	默认值
Q	BOOL	执行结果，执行成功置 TRUE	FALSE
Error	BYTE	错误码 0:运行完成 1:无故障通道	0
ChannelID	BYTE	故障通道号，通道号：1-16，1 代表第一个通道，2 代表第二个通道，以此类推，总共 16 个通道	0

3.4.6.4 MKC3167_READ_FAULT_DATA（故障模式数据读取）

3.4.6.4.1 功能描述

获取故障通道数据，将保护 IO 获取到的故障通道数据上传到 CPU 模块。

3.4.6.4.2 参数描述

输入参数	数据类型	参数值说明	默认值
In	BOOL	使能，高电平使能	FALSE
ChannelID	BYTE	需要获取故障数据的通道号：1-16，1 代表第一个通道，2 代表第二个通道，以此类推，总共 16 个通道	0
DataLen	WORD	数据长度，地址空间大小（用户指定长度，最大 240 个采样点，不能小于 192 个采样点）	0
Address	POINTER TO WORD	数据存放地址	0
输出参数	数据类型	参数值说明	默认值
Done	BOOL	执行结果： 执行成功置 TRUE，读取成功，FALSE 失败	FALSE
Busy	BOOL	是否在读取数据： TRUE：忙 FALSE：空闲	FALSE
Error	BYTE	错误码 0:运行完成 1:没有故障通道 2:输入通道 ID 非法 3:地址非法 4:输入数据长度非法 5:故障数据读取错误	0
RealDataLen	WORD	数据长度： 实际拷贝数据长度，最大不超过 DataLen	0

3.5 现场总线库

3.5.1 ETHERCAT 协议相关指令库

3.5.1.1 EtherCAT_BusReset（复位 EtherCAT 总线）

3.5.1.1.1 功能说明

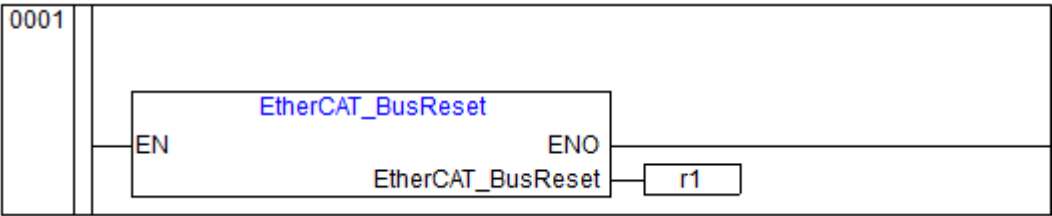
通过外部库的方式复位 EtherCAT 总线，整个过程是先复位 comX 协议板卡、重新配置从站、初始化协议栈，最终达到整个 EtherCAT 总线复位的目的。

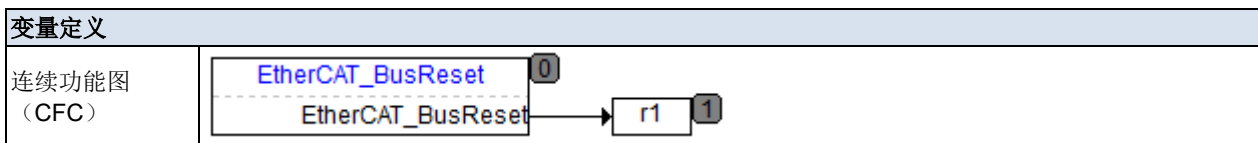
3.5.1.1.2 返回值

返回值	类型	含义
0	UDINT	复位 EtherCAT 总线成功
0x800A0001	UDINT	comX 通讯通道无效（内部错误，用户无需关注）
0x800C0011	UDINT	comX 通讯通道状态错误（内部错误，用户无需关注）
0x800A000A	UDINT	EtherCAT 配置文件无效（内部错误，用户无需关注）
0x800A0004	UDINT	EtherCAT 配置文件加载失败（内部错误，用户无需关注）
0x800A0019	UDINT	复位 comX 通讯通道失败
0x800C0020	UDINT	复位 comX 通讯通道超时

3.5.1.1.3 示例

复位 EtherCAT 总线。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	r1			UDINT	0	FALSE
编程语言		程序				
梯形图（LD）						
结构化文本（ST）		<pre>0001 r1 := EtherCAT_BusReset();</pre>				



3.5.1.2 EtherCAT_SlaveFaultReset（清除从站错误）

3.5.1.2.1 功能说明

通过外部函数库的形式清除 EtherCAT 从站显示的错误码，整个过程是先通过 EtherCAT 协议消息清除从站显示的错误码，再对从站进行初始化，最终达到从站准备就绪，可以进入使能状态的目的。

3.5.1.2.2 输入参数

输入参数	类型	含义
usSlaveAddress	UINT	从站号

3.5.1.2.3 传参说明

参数	值	描述
usSlaveAddress	0x0001~0x37FF	从站号

3.5.1.2.4 返回值

返回值	类型	含义
0	UDINT	清除从站错误码执行失败
1	UDINT	清除从站错误码执行成功

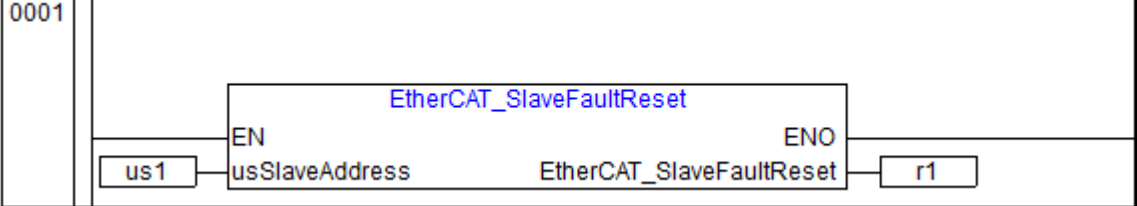
■ 备注

清除从站错误码执行成功表示命令执行成功，不代表从站错误被清除成功。部分从站错误无法清除，详见从站设备说明书。

清除从站错误码执行失败的原因可能是：从站号错误或者 EtherCAT 总线通信中断。

3.5.1.2.5 示例

清除 16#100 号从站错误。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	us1			UINT	16#100	FALSE
0002	r1			UDINT	0	FALSE
编程语言	程序					
梯形图 (LD)						
结构化文 本 (ST)	<pre>0001 r1 := EtherCAT_SlaveFaultReset(us1);</pre>					
连续功能 图 (CFC)						

3.5.1.3 EtherCAT_DriveEnable（伺服使能）

3.5.1.3.1 功能说明

打开/关闭伺服使能分开关，可以独立控制每个从站轴的使能状态。

3.5.1.3.2 输入参数

输入参数	类型	含义
usSlaveAddress	UINT	从站号（0x0001~0x37FF）
bMode	BOOL	使能值 1：使能 0：不使能（默认）

3.5.1.3.3 返回值

返回值	类型	含义
0	UDINT	设置成功
其他	UDINT	设置失败

- 备注：阻塞指令，非轴运动缓存指令。

只有伺服使能总开关开启的情况下，伺服使能分开关才能生效。

3.5.1.3.4 示例

设定 16#100 号站伺服使能分开关为打开。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	us1			UINT	16#100	FALSE
0002	mode1			BOOL	TRUE	FALSE
0003	r1			UDINT	0	FALSE
编程语言		程序				
梯形图（LD）	0001 EtherCAT_DriveEnable us1usSlaveAddressmode1bMode ENOEtherCAT_DriveEnable r1					
结构化文本（ST）	0001r1:=EtherCAT_DriveEnable(us1, mode1);					
连续功能图（CFC）	us1mode1 EtherCAT_DriveEnable 0 r11					

3.5.1.4 EtherCAT_GetDriveEnableState (获取伺服使能状态)

3.5.1.4.1 功能说明

获取某个从站轴的伺服使能状态。

3.5.1.4.2 输入参数

输入参数	类型	含义
usSlaveAddress	UINT	从站号 (0x0001~0x37FF)

3.5.1.4.3 返回值

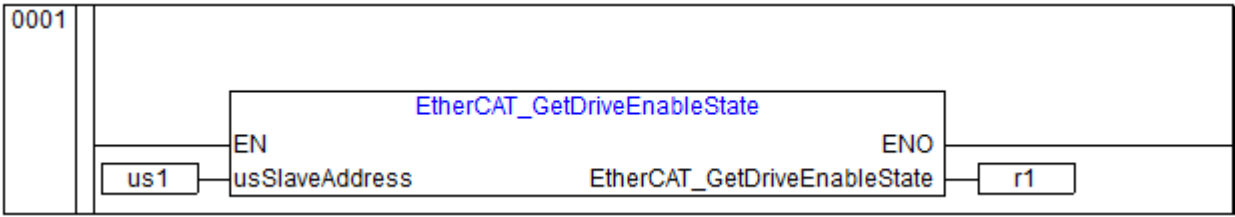
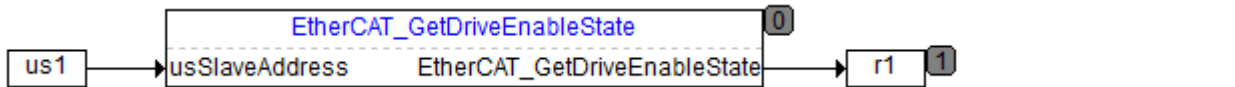
返回值	类型	含义
0	USINT	伺服使能关闭
1	USINT	伺服使能开启

- 备注：阻塞指令，非轴运动缓存指令。

只有当伺服使能总开关与对应从站伺服使能分开关同时开启的情况下，从站的伺服使能状态才为开启状态，否则为关闭状态。

3.5.1.4.4 示例

获取 16#100 号站驱动器状态。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	us1			UINT	16#100	FALSE
0002	r1			UDINT	0	FALSE
编程语言 程序						
梯形图 (LD)						
结构化文本 (ST)	<pre>0001 r1 := EtherCAT_GetDriveEnableState(us1);</pre>					
连续功能图 (CFC)						

3.5.1.5 EtherCAT_Read（读从站参数）

3.5.1.5.1 功能说明

读指定从站、指定参数的参数长度和参数内容。

3.5.1.5.2 输入参数

输入参数	类型	参数描述
ulNodeId	UDINT	从站号
ulIndex	UDINT	参数对应的对象索引
ulSubIndex	UDINT	参数对应的对象子索引
pucDataCnt	POINTER TO BYTE	指向参数长度的指针
pucData	POINTER TO BYTE	指向参数内容的指针

3.5.1.5.3 返回值

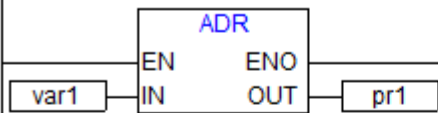
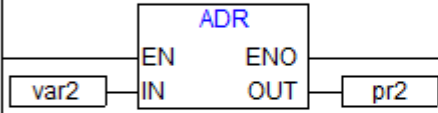
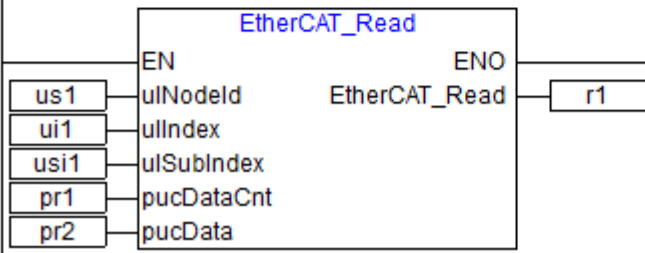
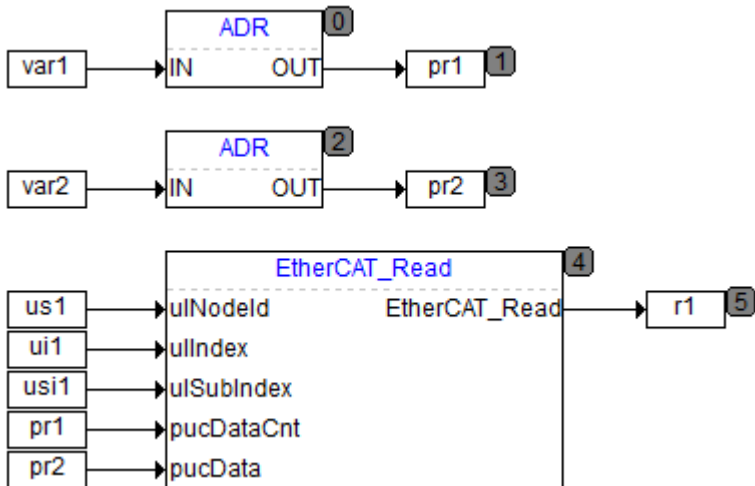
返回值	类型	参数描述
0	UDINT	读取成功
其他	UDINT	读取失败 详见《MC1000 系列多轴运动控制器指令手册》中的附录 3EtherCAT 读写参数错误码

- 备注：如果在 IEC 周期任务中调用 EtherCAT_Read 指令，则要求任务周期必须设置成大于任务执行的实际时间。

3.5.1.5.4 示例

读取 16#100 号站的设备名称（Omron 为例）。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	var1			BYTE	0	FALSE
0002	pr1			POINTER TO BYTE	0	FALSE
0003	var2			STRING(160)	''	FALSE
0004	pr2			POINTER TO BYTE	0	FALSE
0005	us1			UDINT	16#100	FALSE
0006	ui1			UDINT	16#1008	FALSE
0007	us11			UDINT	16#00	FALSE
0008	r1			UDINT	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	0001 
	0002 
	0003  16#100: 从站号 16#1008: 对象索引 16#00: 对象子索引 运行结果: var1=20, 表示参数长度占用 20 字节 var2=R88D-KN04H-ECT-L, 表示设备名称
结构化文本 (ST)	<pre> 0001 pr1 :=ADR(var1); 0002 pr2 :=ADR(var2); 0003 r1 :=EtherCAT_Read(us1, ui1, usi1, pr1, pr2); </pre>
连续功能图 (CFC)	

3.5.1.6 EtherCAT_Write（写从站参数）

3.5.1.6.1 功能说明

写指定从站、指定参数的参数内容。

3.5.1.6.2 输入参数

输入参数	类型	参数描述
ulNodeId	UDINT	从站号
ulIndex	UDINT	参数对应的对象索引
ulSubIndex	UDINT	参数对应的对象子索引
ucDataCnt	BYTE	待写入参数的长度
pucData	POINTER TO BYTE	指向待写入参数内容的指针

3.5.1.6.3 返回值

返回值	类型	参数描述
0	UDINT	写入成功
其他	UDINT	写入失败 详见附录 3 EtherCAT 读写参数错误码

■ 备注

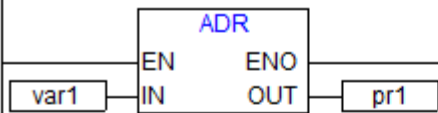
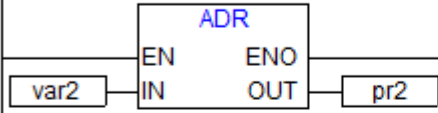
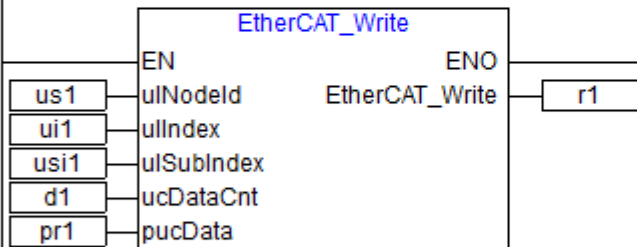
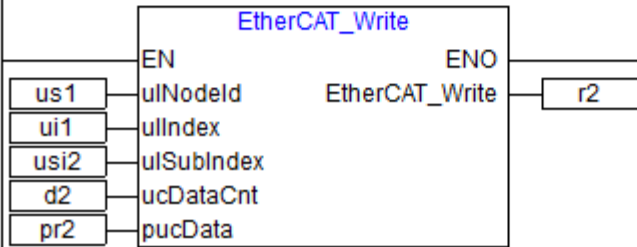
待写入参数内容的实际长度与 ucDataCnt 的值一定要严格保持一致。

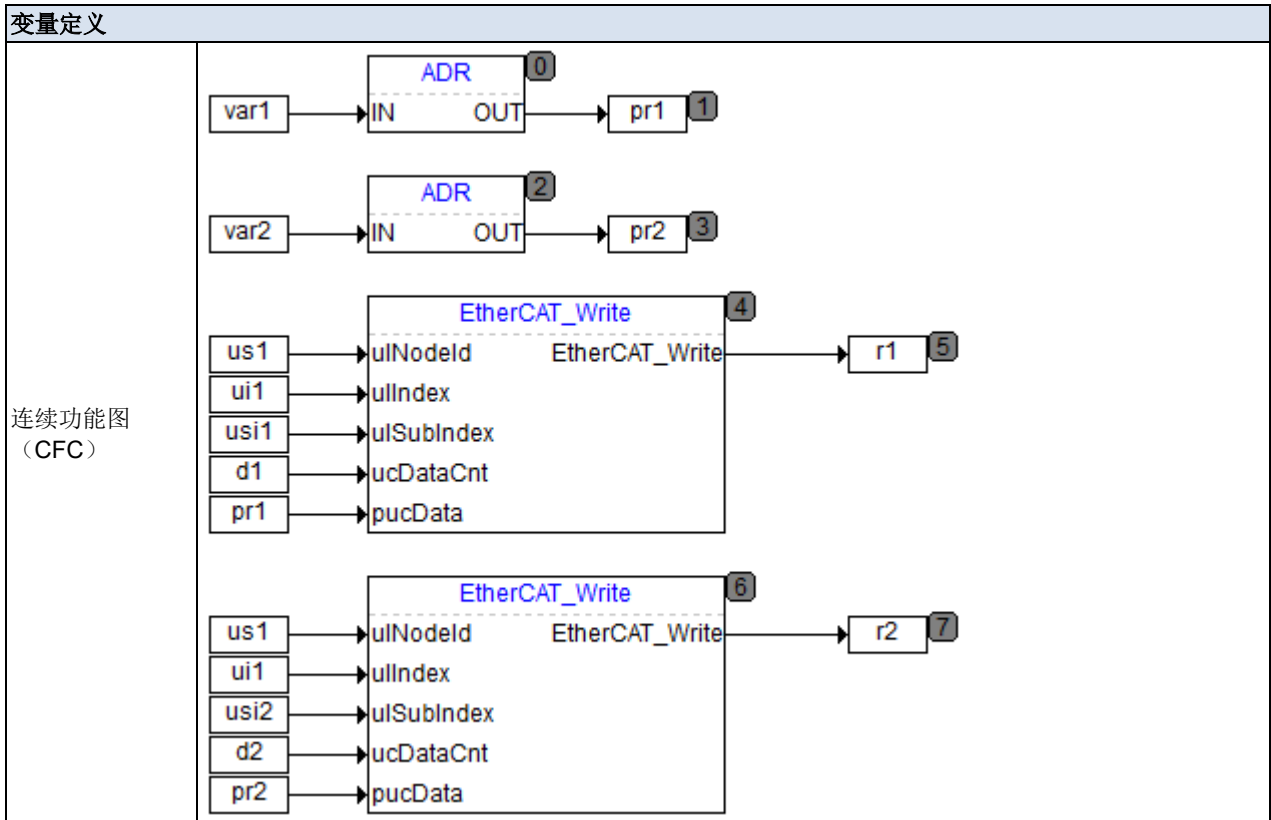
如果在 IEC 周期任务中调用 EtherCAT_Write 指令，则要求任务周期必须设置成大于任务执行的实际时间。

3.5.1.6.4 示例

设置 16#100 号从站的齿轮比（Omron 为例）。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	var1			DINT	1048576	FALSE
0002	pr1			POINTER TO BYTE	0	FALSE
0003	var2			DINT	10000	FALSE
0004	pr2			POINTER TO BYTE	0	FALSE
0005	us1			UDINT	16#100	FALSE
0006	ui1			UDINT	16#6091	FALSE
0007	usi1			UDINT	16#01	FALSE
0008	d1			BYTE	4	FALSE
0009	r1			UDINT	0	FALSE
0010	usi2			UDINT	16#02	FALSE
0011	d2			BYTE	4	FALSE
0012	r2			UDINT	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	0001 
	0002 
	0003 
	0004 
16#100: 从站号 16#6091: 对象索引 16#01: 对象子索引 16#02: 对象子索引 4: 参数长度占用 4 字节 运行结果: 设置从站 10000 个脉冲转一圈。	
结构化文本 (ST)	<pre> 0001 pr1 :=ADR(ivar1); 0002 pr2 :=ADR(ivar2); 0003 r1 :=EtherCAT_Write(us1, ui1, usi1, d1, pr1); 0004 r2 :=EtherCAT_Write(us1, ui1, usi2, d2, pr2); </pre>



3.5.1.7 EtherCat Touch Probe 功能

3.5.1.7.1 EtherCat Touch Probe 功能

该功能可实现 EtherCAT 总线驱动器的高精度位置捕获，捕获由硬件完成，无软件延时偏差。捕获的触发源可选择驱动器的 touch probe 功能所对应的 IO 输入，也可选择由编码器的 Z 相 0 脉冲触发。

目前每个驱动器支持一路高速位置捕获（touch probe1），touch probe2 暂不支持。

3.5.1.7.2 EtherCAT_CaptureConfig（Touch Probe 功能配置）

3.5.1.7.2.1 函数原型

UDINT EtherCAT_CaptureConfig (USINT AxisID, UINT Mode)

3.5.1.7.2.2 功能说明

配置 EtherCAT 总线伺服从站高速捕获功能。

注意：在配置完 capture 功能后，最好在 IEC 中调用 EtherCAT_CaptureGetStatus 循环扫描捕获状态，当检测到捕获完成时，立即调用 EtherCAT_CaptureGetMpos 读取所需的捕获位置。

3.5.1.7.2.3 输入参数

输入参数	类型	参数描述
AxisID	USINT	轴号
Mode	UINT	捕获模式，见下表

伺服 Mode 参数配置如下表（具体参见所用伺服驱动器 touch probe 功能的相关对象，index: 0x60B8）：

捕获通道	Bit	参数描述
Touch probe1	0	0: 关闭捕获通道 1 1: 开启捕获通道 1
	1	0: 单次触发 1: 多次触发
	2	0: 通道 1 的 input 信号触发（touch probe1 input） 1: Z 相 0 脉冲触发
	3	保留
	4	0: 关闭上升沿捕获 1: 开启上升沿捕获
	5	0: 关闭下降沿捕获 1: 开启下降沿捕获
	6	保留
	7	保留

捕获通道	Bit	参数描述
Touch probe2	8	0: 关闭捕获通道 2 1: 开启捕获通道 2
	9	0: 单次触发 1: 多次触发
	10	0: 通道 2 的 input 信号触发（touch probe2 input） 1: Z 相 0 脉冲触发
	11	保留
	12	0: 关闭上升沿捕获 1: 开启上升沿捕获
	13	0: 关闭下降沿捕获 1: 开启下降沿捕获
	14	保留
	15	保留

3.5.1.7.2.4 返回值

返回值	类型	参数描述
0	UDINT	配置成功
其它	UDINT	错误码

3.5.1.7.2.5 指令类型

非轴运动缓存指令。

3.5.1.7.2.6 应用示例

当轴 0 对应的伺服驱动器的 touch probe1 针脚输入上升沿时，捕获该轴的位置值。

程序如下：

序号	变量名	变量说明	变量类型	初始值	掉电保护
0001	AxisID	待捕获轴ID	USINT	0	FALSE
0002	CaptureMode	capture模式	UINT	0	FALSE
0003	CaptureFinishStatus	capture完成时状态	WORD	0	FALSE
0004	CaptureChannel	capture通道。1: touch probe1 ; 2: touch probe2	USINT	0	FALSE
0005	CapturePosID	capture位置ID。1: 上升沿所捕获的位置 ; 2: 下降沿所捕获的位置	USINT	0	FALSE
0006	Err	错误码	UDINT	0	FALSE
0007	CapturePos	捕获的位置	LREAL	0	FALSE

```

0001 AxisID := 0; (*待捕获轴*)
0002 CaptureMode := 16#1; (*capture模式配置：使用通道1，单次触发，触发信号来自touch probe1 输入，仅捕获上升沿。*)
0003 CaptureFinishStatus := 16#3; (*capture完成时状态。*)
0004 Err := EtherCAT_CaptureConfig (AxisID ,CaptureMode); (*配置capture：配置成功时返回0*)
0005 IF 0 = Err THEN (*配置成功*)
0006   WHILE TRUE DO
0007     IF CaptureFinishStatus = (UINT_TO_WORD (EtherCAT_CaptureGetState (AxisID)) AND CaptureFinishStatus ) THEN (*查询capture是否完成*)
0008       CaptureChannel := 1; (*capture通道。1: touch probe1 ; 2: touch probe2*)
0009       CapturePosID := 1; (*capture位置ID。1: 上升沿所捕获的位置 ; 2: 下降沿所捕获的位置*)
0010       CapturePos := EtherCAT_CaptureGetMpos(AxisID, CaptureChannel, CapturePosID); (*获取捕获到的位置*)
0011       EXIT; (*获取到捕获位置后退出循环*)
0012     END_IF
0013   END_WHILE
0014 END_IF
0015

```

程序示例

3.5.1.7.3 EtherCAT_CaptureGetStatus（获取 Touch Probe 状态）

3.5.1.7.3.1 函数原型

UINT EtherCAT_CaptureGetStatus(USINT AxisID)

3.5.1.7.3.2 功能说明

返回指定从站高速捕获功能的状态。

3.5.1.7.3.3 输入参数

输入参数	类型	参数描述
AxisID	USINT	轴号

3.5.1.7.3.4 返回值

返回值	类型	参数描述
高速捕获功能状态 (指令调用失败时返回 0)	UINT	高速捕获功能状态，见下表

返回值状态含义如下表（具体参见所用伺服驱动器 touch probe 功能的相关对象，index: 0x60B9）。

捕获通道	Bit	参数描述
Touch probe1	0	0: 捕获通道 1 关闭 1: 获通道 1 开启捕
	1	0: 捕获通道 1 上升沿捕获未完成 1: 捕获通道 1 上升沿捕获完成
	2	0: 捕获通道 1 下降沿捕获未完成 1: 捕获通道 1 下降沿捕获完成
	3	保留
	4	保留
	5	保留
	6	测试用
	7	测试用

捕获通道	Bit	参数描述
Touch probe2	8	0: 捕获通道 2 关闭 1: 获通道 2 开启
	9	0: 捕获通道 2 上升沿捕获未完成 1: 捕获通道 2 上升沿捕获完成
	10	0: 捕获通道 2 下降沿捕获未完成 1: 捕获通道 2 下降沿捕获完成
	11	保留
	12	保留
	13	保留
	14	测试用
	15	测试用

3.5.1.7.3.5 指令类型

非轴运动缓存指令。

3.5.1.7.4 EtherCAT_CaptureGetMpos（获取 Touch Probe 捕获位置）

3.5.1.7.4.1 函数原型

LREAL EtherCAT_CaptureGetMpos(USINT AxisID, USINT Channel ,USINT PosID)

3.5.1.7.4.2 功能说明

返回指定从站高速捕获通道获取到的 Mpos。

3.5.1.7.4.3 输入参数

输入参数	类型	参数描述
AxisID	USINT	轴号
Channel	USINT	通道号 1: 通道 1（touch probe1） 2: 通道 2（touch probe2）
PosID	USINT	欲获取的位置 ID

输入参数	类型	参数描述
		1: 上升沿 2: 下降沿（某些驱动器不支持下降沿捕获，如松下 A5B）

3.5.1.7.4.4 返回值

返回值	类型	参数描述
捕获到的 Mpos 值（指令调用失败时返回 0）	LREAL	捕获到的 Mpos 值

3.5.1.7.4.5 指令类型

非轴运动缓存指令。

3.5.1.7.4.6 补充说明

获取该值时请首先使用 EtherCAT_CaptureGetStatus 指令检查捕获状态，确认要获取的值已经捕获完成。

3.5.2 RTEX 协议相关指令库

3.5.2.1 RTEX_Read（读驱动器信息）

3.5.2.1.1 功能说明

通过外部库的方式获取从站信息。

3.5.2.1.2 参数说明

输入参数	类型	含义
StaNo	USINT	从站号
ParaID	UDINT	参数 ID
DataLen	USINT	读取的数据长度
pValue	POINTER TO BYTE	存放读出信息的地址

3.5.2.1.3 传参说明

参数	值	描述	举例
StaNo	0~31	从站号	0
ParaID	0x00000+Class*100+ParNum	读 Para0~8 类参数信息	725: 为 Pr7.25
	0x10000+Type_Code	读 system ID 信息	0x10001: 读取生产厂家名称（Panasonic）

参数	值	描述	举例
	$0x20000 + \text{Index} * 0x1000$	读 Alarm 信息	0x21000: 读取 1 号历史报警信息
	$0x30000 + \text{Index} * 0x1000 + \text{Type_Code}$	读 Monitor 信息	0x30005: 读取时间速度
DataLen	给下来的数据空间大小	system ID 为 20 字节 除了 system ID 外都为 4 字节	0x04
pValue	存放读出信息的首地址	-	-

■ system ID 的 Type_Code

Type_Code	值	ParaID
VENDOR_NAME	0x01	65537
DEVICE_TYPE	0x05	65541
DRIVER_MODE_NO	0x12	65554
DRIVER_SERIAL_NO	0x13	65555
SERVODVSFV	0x14	65556
DRIVER_TPPE	0x15	65557
MOTOR_DODEL_NO	0x22	65570
MOTOR_SERIAL_NO	0x23	65571
VENDOR_ID_EX	0x31	65585
MODEL_ID_EX	0x32	65586

■ Alarm 的 Index

取值范围：0~14。如下表所示。

当前与历史报警列表

Index	Alarm Code	Warning Code
当前的 0	代表当前报警	代表当前警示
历史的	1 代表倒数第 1 个报警	0
	2 代表倒数第 2 个报警	0
		...
	14 代表倒数第 14 个报警	0

i 报警未发生时，Alarm Code 为 0。

■ Monitor 的 Index 和 Type_Code

详见《MC1000 系列多轴运动控制器指令手册》中附录 2 的 Type_Code 列表。

■ DataLen

表示调用者为读取结果而预先开辟的空间，一般由读取的内容决定。

驱动器参数为单字节、双字节、四字节等数据类型，长度分别为 1、2、4 字节。

如果读取 system ID 命令，则需要预先开辟大于等于 20 字节的空间，以便存放字符串，一般取值 20 即可。

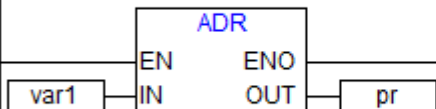
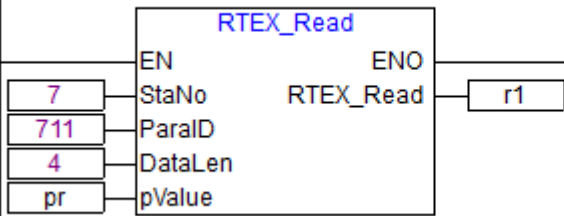
3.5.2.1.4 返回值

返回值	类型	含义
0	UINT	获取成功
0x1000	UINT	读取的类别不存在
0x2000	UINT	数据长度不对
0x3000	UINT	形参指针为空
0x4000	UINT	没有对应的从站号
0x5000	UINT	系统没有进入运行状态
0x6000	UINT	正在执行复位
0x7000	UINT	协议栈异常
0x8000	UINT	只允许制造商读取的值
0x9000	UINT	超时
0x9100	UINT	缓冲区已满
其他值	UINT	参见《MC1000 系列多轴运动控制器指令手册》中附录 1 的 RTEX 指令错误码

3.5.2.1.5 指令使用举例

读取 7 号站 Pr7.11 参数值。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	var1			DINT	0	FALSE
0002	pr			POINTER TO BYTE	0	FALSE
0003	r1			UINT	0	FALSE

编程语言		程序
梯形图 (LD)	0001	
	0002	

7 是站号；
711 为 Pr7.11 参数（输入值的计算为 $7 \times 100 + 11$ ）；
var1 中为运行结果，比如该事例的运行结果为 500000，即 Pr7.11 参数值为 500,000。

变量定义						
	运行结果:					
	0001	var1			DINT	500000 FALSE
结构化文本 (ST)	0001	pr :=ADR (var1);				
	0002	r1 := RTEX_Read (7, 711, 4, pr);				
连续功能图 (CFC)						

3.5.2.2 RTEX_Write (写驱动器参数)

3.5.2.2.1 功能说明

通过外部函数库的形式给总线从站写参数。

3.5.2.2.2 参数说明

输入参数	类型	含义
StaNo	USINT	从站号
ParaID	UDINT	参数 ID
Value	DINT	参数值

传递参数	值	描述
StaNo	0-31	从站号
ParaID	Class*100+ParNum	写 Para0~8 类参数信息
	0x1000	存储所有的参数到 EEPROM
	0x2000	将 C 类参数生效
Value	参数值	Class*100+ParNum: 根据不同参数取值范围不同 ParaID 为 0x1000 或 0x2000 时, 此参数无意义

3.5.2.2.3 返回值

返回值	类型	含义
0	UINT	获取成功

返回值	类型	含义
0x1000	UINT	读取的类别不存在
0x4000	UINT	没有对应的从站号
0x5000	UINT	系统没有进入运行状态
0x6000	UINT	正在执行复位
0x7000	UINT	协议栈异常
0x9000	UINT	超时
0x9100	UINT	缓冲区已满
其他值	UINT	参见《MC1000 系列多轴运动控制器指令手册》中附录 1 的 RTEX 指令错误码

3.5.2.2.4 备注

C 类参数说明详见松下手册“SX-DSV02203_R2_2E”。

3.5.2.2.5 指令使用举例

修改 7 号站 Pr7.11 参数值为 100,000。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	r1			UINT	0	FALSE
编程语言		程序				
梯形图（LD）	0001 7 711 100000 RTEX_Write EN StaNo ParalD Value ENO RTEX_Write r1					
	7 是站号； 711 为 Pr7.11 参数（值的计算为 7*100+11）； 运行结果为：将 Pr7.11 参数值写为 100,000。					
结构化文本（ST）	0001r1 := RTEX_Write (7 , 711, 100000);					
连续功能图（CFC）	7 711 100000 RTEX_Write StaNo ParalD Value RTEX_Write 0 r1 1					

3.5.2.3 RTEX_DriveErrorClear（清除驱动器错误）

3.5.2.3.1 功能说明

通过外部库的方式清除告警信息。

3.5.2.3.2 参数说明

输入参数	类型	含义
StaNo	USINT	从站号
ParaID	UDINT	参数 ID

传递参数	值	描述
StaNo	0~31	从站号
ParaID	0x001	清除当前报警
	0x011	清除历史报警
	0x021	清除外部报警

3.5.2.3.3 返回值

返回值	类型	含义
0	UINT	获取成功
0x1000	UINT	读取的类别不存在
0x4000	UINT	没有对应的从站号
0x5000	UINT	系统没有进入运行状态
0x6000	UINT	正在执行复位
0x7000	UINT	协议栈异常
0x9000	UINT	超时
0x9100	UINT	缓冲区已满
其他值	UINT	参见《MC1000 系列多轴运动控制器指令手册》中附录 1 的 RTEX 指令错误码

i 清除指令，需要在电机处于 CP、CV 等运动控制模式下才有效。

3.5.2.3.4 指令使用举例

清除 7 号站当前报警。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	us1			USINT	7	FALSE
0002	ui1			UDINT	1	FALSE
0003	r1			UINT	0	FALSE
编程语言		程序				
梯形图（LD）	0001 us1 ui1 RTEX_DriveErrorClear ENStaNoParalDRTEX_DriveErrorClear ENO r1 运行结果为：将 7 号站的当前警报清除。					
结构化文本（ST）	0001r1 := RTEX_DriveErrorClear (us1 , ui1);					
连续功能图（CFC）	us1 ui1 RTEX_DriveErrorClear StaNoParalDRTEX_DriveErrorClear 0 r1 1					

3.5.2.4 RTEX_BusReset (复位协议主栈)

3.5.2.4.1 功能说明

通过外部库的方式复位主协议栈及其所有从站，为了避免驱动器复位时电机产生大的冲击，在使用此指令时，必须先清除所有轴缓存指令，使所有轴停止运动。

3.5.2.4.2 返回值

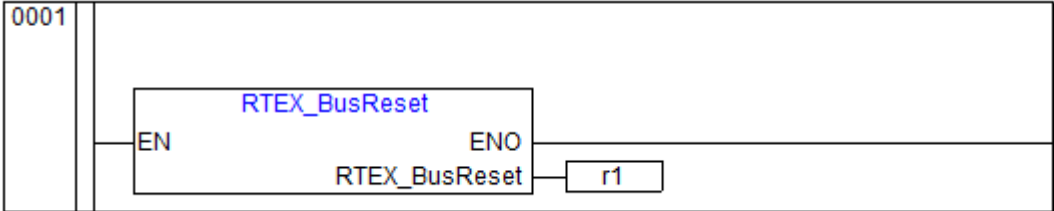
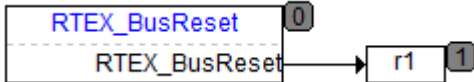
返回值	类型	含义
0	UINT	获取成功
0x5000	UINT	系统没有进入运行状态
0x6000	UINT	正在执行复位
0x7000	UINT	协议栈异常
0x9000	UINT	超时
0x9100	UINT	缓冲区已满
其他值	UINT	参见《MC1000 系列多轴运动控制器指令手册》中附录 1 的 RTEX 指令错误码



- 由于总线断线或从站驱动器断电等原因导致协议栈异常时，可使用该指令对议栈主站进行复位。
- MC1002R-A01 不支持。

3.5.2.4.3 指令使用举例

复位协议主站。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	r1			UINT	0	FALSE
编程语言		程序				
梯形图 (LD)						
结构化文本 (ST)		<pre>0001 r1 := RTEX_BusResel ();</pre>				
连续功能图 (CFC)						

3.5.2.5 RTEX_GetDriveState (获取驱动器状态)

3.5.2.5.1 功能说明

通过外部库的方式获取从站状态标识。

3.5.2.5.2 参数说明

输入参数	类型	含义
StaNo	USINT	从站号
pFlagValue	POINTER TO UINT	读取结果存放地址

3.5.2.5.3 返回值

返回值	类型	含义
0	UINT	获取成功
0x3000	UINT	形参指针为空
0x4000	UINT	没有对应的从站号
0x5000	UINT	系统没有进入运行状态
0x6000	UINT	正在执行复位
0x7000	UINT	协议栈异常
0x9000	UINT	超时
0x10000	UINT	缓冲区已满
其他值	UINT	参见《MC1000 系列多轴运动控制器指令手册》中附录 1 的 RTEX 指令错误码

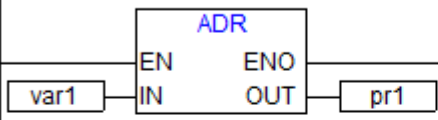
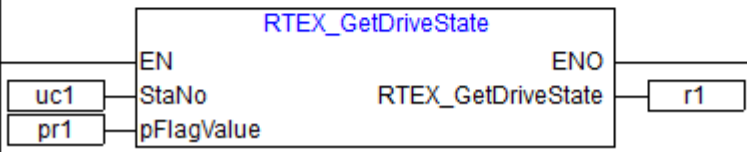
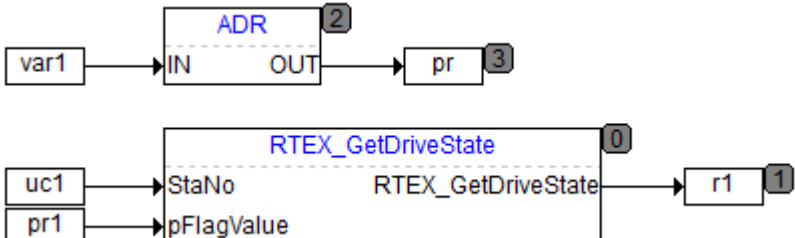
3.5.2.5.4 接收内容

字节号	1	2
Bit 7	Servo_Active	E-STOP
Bit 6	Servo_Ready	SI-MON4/EX-SON
Bit 5	Alarm	SI-MON3/EXT3
Bit 4	Warning	SI-MON2/EXT2
Bit 3	Torque_Limited	SI-MON1/EXT1
Bit 2	Homing_Complete	HOME
Bit 1	In_Progress	POT/NOT
Bit 0	In_Position	NOT/POT

3.5.2.5.5 指令使用举例

获取 7 号站驱动器状态。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	var1			UINT	960	FALSE
0002	pr1			POINTER TO UINT	207644652	FALSE
0003	uc1			BYTE	7	FALSE
0004	r1			UINT	0	FALSE
编程语言		程序				

变量定义	
梯形图 (LD)	0001 
	0002  运行结果：r1 为 0 表示获取成功。 var1=960 转换为十六位二进制 0000 0011 1100 0000，各个位对应接收内容参见接收内容。
结构化文本 (ST)	<pre> 0001 pr1 := ADR (var1); 0002 r1 := RTEX_GetDriveState (uc1 , pr1); </pre>
连续功能图 (CFC)	

3.5.2.6 RTEX_GetDriveState（获取驱动器状态）

3.5.2.6.1 功能说明

打开/关闭伺服使能分开关，可以独立控制每个从站轴的使能状态。

3.5.2.6.2 参数说明

输入参数	类型	含义
ucStackNo	BYTE	从站号
bMode	BOOL	使能值 1: 使能 0: 不使能（默认）

3.5.2.6.3 返回值

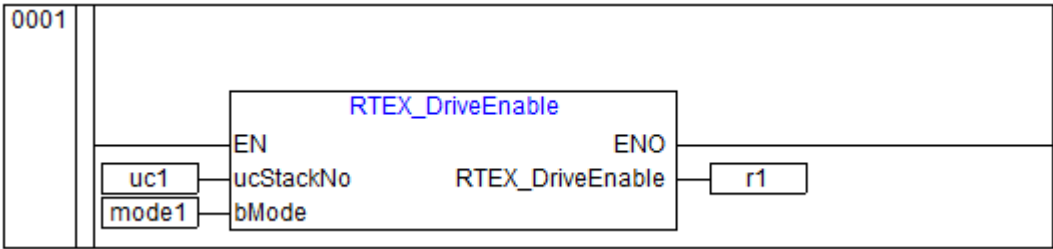
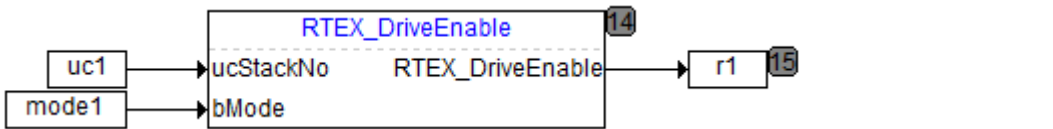
返回值	类型	含义
0	UDINT	设置成功
其他	UDINT	设置失败

❗阻塞指令，非轴运动缓存指令。

3.5.2.6.4 指令使用举例

设定 7 号站伺服使能分开关打开。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	uc1			BYTE	7	FALSE
0002	mode1			BOOL	TRUE	FALSE
0003	r1			UDINT	0	FALSE

编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre>0001 r1:= RTEX_DriveEnable (uc1,mode1);</pre>
连续功能图 (CFC)	

3.5.2.7 RTEX_GetDriveEnableState (获取伺服使能状态)

3.5.2.7.1 功能说明

获取某个从站轴的伺服使能状态。

3.5.2.7.2 参数说明

输入参数	类型	含义
ucStackNo	BYTE	从站号

3.5.2.7.3 返回值

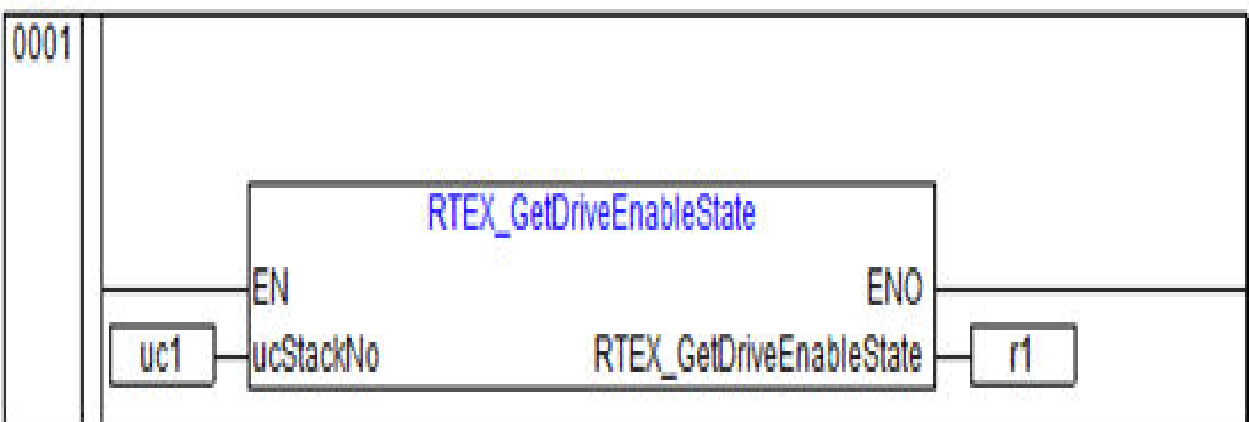
返回值	类型	含义
0	USINT	伺服使能关闭
1	USINT	伺服使能开启

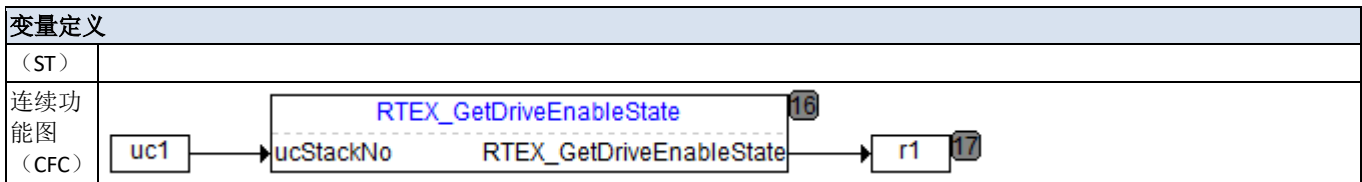


- 阻塞指令，非轴运动缓存指令。
- 只有当伺服使能总开关（通过 HMC_DriveEnable 设定）与对应从站伺服使能分开关同时开启的情况下，从站的伺服使能状态才为开启状态，否则为关闭状态。

3.5.2.7.4 指令使用举例

获取 7 号站伺服使能分开关状态。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	uc1			BYTE	7	FALSE
0002	r1			USINT	0	FALSE
编程语言	程序					
梯形图 (LD)	 r1 为伺服使能总开关&总线驱动器使能分开关。 运行结果：r1 为 1，即为伺服使能总开关和总线驱动器使能分开关均为 1。					
结构化文本	<pre>0001 r1:= RTEX_GetDriveEnableState (uc1);</pre>					



3.5.2.8 RTEX_DriveResetAll（复位所有驱动器）

3.5.2.8.1 功能说明

通过外部库的方式复位所有从站驱动器。为了避免驱动器复位时电机产生大的冲击，在使用此指令时，必须先清除所有轴缓存指令，使所有轴停止运动。

3.5.2.8.2 返回值

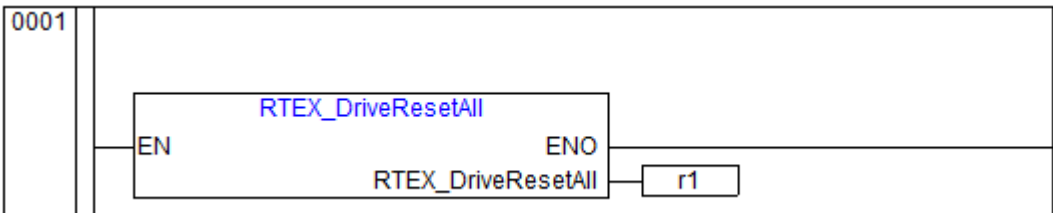
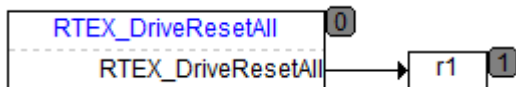
返回值	类型	参数描述
0	UINT	成功
0x5000	UINT	系统没有进入运行状态
0x6000	UINT	正在执行复位
0x7000	UINT	协议栈异常
0x9000	UINT	超时
0x9100	UINT	缓冲区已满
其他值	UINT	参见《MC1000 系列多轴运动控制器指令手册》中附录 1 的 RTEX 指令错误码

i 在进行参数配置，并执行写 EEPROM 之后，可以用此指令复位伺服器，达到参数生效的目的。也可以采用掉电，再上电的方式达到参数生效的目的。

3.5.2.8.3 指令使用举例

复位驱动器。

变量定义						
序号	变量名	直接地址	变量说明	变量类型	初始值	掉电保护
0001	r1			UINT	0	FALSE
编程语言		程序				

变量定义		
梯形图 (LD)	0001	
结构化文本 (ST)	0001	r1 := RTEX_DriveResetAll ();
连续功能图 (CFC)		

3.5.2.9 RTEX_ClearAbsEncoderData (清除绝对值编码器多圈数据)

3.5.2.9.1 函数原型

UDINT RTEX_ClearAbsEncoderData(USINT ucAxisID)

3.5.2.9.2 功能说明

对 RTEX 伺服配套的绝对值编码器多圈数据清零，执行完毕后需断电重启驱动器。

3.5.2.9.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.5.2.9.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.5.2.9.5 指令类型

非轴运动缓存指令。

3.5.2.10 RTEX 总线轴高速位置锁存

该功能可实现 RTEX 总线轴的高精度位置锁存，锁存由硬件完成，无软件延时偏差。锁存的触发源可选择驱动器的 latch 功能所对应的 EXT IO 引脚，也可选择由编码器的 Z 相零脉冲触发。

每个驱动器支持 2 路高速位置锁存通道。

3.5.2.10.1 RTEX_StartAixsPosLatch（启动 RTEX 轴位置锁存）

3.5.2.10.1.1 函数原型

```
UDINT RTEX_StartAixsPosLatch(USINT ucAxisID, USINT ucLatchNo, USINT ucTriggerSignal)
```

3.5.2.10.1.2 功能说明

配置并启动 RTEX 总线轴位置锁存功能，每个驱动器支持 2 路锁存通道。

注意：在配置完锁存功能后，应在 IEC 中循环调用 RTEX_CheckAixsPosLatchStatus 检查锁存状态，当检测到锁存完成时，立即调用 RTEX_GetLatchedAixsPos 读取锁存到的位置。

3.5.2.10.1.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucLatchNo	USINT	锁存通道编号，1 或 2
ucTriggerSignal	USINT	锁存触发信号源： 0: Z 相 ■ 全闭环时外部位移传感器的 Z 相 ■ 串行通讯类型的绝对式外部位移传感器时返回指令错误（005Ah） 1: EXT1 的逻辑上升沿 2: EXT2 的逻辑上升沿 3: EXT3 的逻辑上升沿 4~8: 禁止使用 ■ 选择时返回指令错误（0032h） 9: EXT1 的逻辑下降沿 10: EXT2 的逻辑下降沿 11: EXT3 的逻辑下降沿 12~15: 禁止使用 ■ 选择时返回指令错误（0032h） 有关 EXT 信号的引脚定义，请参见松下 A6N 伺服驱动器手册

3.5.2.10.1.4 返回值

返回值	类型	参数描述
0	UDINT	配置成功
0xFFFFFFFF	UDINT	参数错误

3.5.2.10.1.5 指令类型

非轴运动缓存指令。

3.5.2.10.2 RTEX_CancelAixsPosLatch（撤销 RTEX 轴位置锁存）

3.5.2.10.2.1 函数原型

UDINT RTEX_CancelAixsPosLatch (USINT ucAxisID, USINT ucLatchNo)

3.5.2.10.2.2 功能说明

撤销指定的位置锁存通道。

3.5.2.10.2.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucLatchNo	USINT	锁存通道编号，1 或 2

3.5.2.10.2.4 返回值

返回值	类型	参数描述
0	UDINT	配置成功
0xFFFFFFFF	UDINT	参数错误或锁存通道未启动

3.5.2.10.2.5 指令类型

非轴运动缓存指令。

3.5.2.10.3 RTEX_CheckAixsPosLatchStatus（获取位置锁存状态）

3.5.2.10.3.1 函数原型

UDINT RTEX_CheckAixsPosLatchStatus (USINT ucAxisID, USINT ucLatchNo)

3.5.2.10.3.2 功能说明

返回指定的总线轴高速位置锁存通道的状态。

3.5.2.10.3.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucLatchNo	USINT	锁存通道编号，1 或 2

3.5.2.10.3.4 返回值

返回值	类型	参数描述
高速锁存通道状态	UDINT	0: 等待锁存触发信号 1: 锁存完成 其他: 参数错误或锁存通道未启动

3.5.2.10.3.5 指令类型

非轴运动缓存指令。

3.5.2.10.4 RTEX_GetLatchedAixsPos（获取锁存位置）

3.5.2.10.4.1 函数原型

LREAL RTEX_GetLatchedAixsPos(USINT ucAxisID, USINT ucLatchNo)

3.5.2.10.4.2 功能说明

返回指定轴的高速锁存通道锁存的位置。

3.5.2.10.4.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucLatchNo	USINT	高速位置锁存通道号，1 或 2

3.5.2.10.4.4 返回值

返回值	类型	参数描述
锁存到的 Mpos 值（发生错误时返回 0）	LREAL	锁存到的 Mpos 值（发生错误时返回 0）

3.5.2.10.4.5 指令类型

非轴运动缓存指令。

3.5.2.10.4.6 补充说明

获取该值时请首先使用 RTEX_CheckAixsPosLatchStatus 指令检查锁存通道的状态，确认已锁存动作已完成。

3.5.2.11 RTEX_SetMotorActualPos（设定 RTEX 电机实际位置）

3.5.2.11.1 函数原型

UDINT RTEX_SetMotorActualPos(USINT ucAxisID, DINT iPos)

3.5.2.11.2 功能说明

设定 RTEX 电机当前实际位置（编码器位置）。

3.5.2.11.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
iPos	DINT	设定位置

3.5.2.11.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.5.2.11.5 指令类型

非轴运动缓存指令。

3.5.2.12 RTEX_Home（RTEX 伺服回零）

3.5.2.12.1 函数原型

UDINT RTEX_Home(USINT ucAxisID, USINT ucHomeMode, USINT ucHomeDir)

3.5.2.12.2 功能说明

调用 RTEX 伺服内部原点回归功能。原点回归的加减速、第一阶段查找速度、爬行速度对应轴参中相应参数。

该指令只能在轴类型为 RTEX_PP（43）时使用。

3.5.2.12.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号
ucHomeMode	USINT	回归模式，详见松下 RTEX 伺服文档。以下为 A6N 支持的模式： 1: DI+Z 相 2: DI 3: Z 相 4: 正反限位+DI 6: 正反限位+Z 相
ucHomeDir	USINT	查找方向，详见松下 RTEX 伺服文档。以下为 A6N 支持的模式： 0: 正方向查找原点 1: 负方向查找原点（ucHomeMode 为 2 时不支持）

3.5.2.12.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.5.2.12.5 指令类型

非轴运动缓存指令。

3.5.2.12.6 补充说明

- (1) 使用该指令前，首先设定轴类型为 43（RTEX_PP），开启伺服总使能（HMC_DriveEnable）和从站分使能（RTEX_DriveEnable）。轴类型为 43（RTEX_PP）时，不允许调用除 RTEX_Home/RTEX_ProfileSmoothStop/RTEX_ProfileHardStop 之外的运动控制指令；
- (2) 该指令配置成功后即退出，原点回归过程中可通过 RTEX_GetHomeStatus 指令查看原点回归状态；
- (3) 原点回归动作执行过程中，如果发生意外情况，可调用 RTEX_ProfileSmoothStop（减速停止）或 RTEX_ProfileHardStop（急停）终止原点回归操作。但这样会导致 DPOS 不等于 Mpos 的，所以请务必确保 RTEX_Home 最终执行成功后再切换为所需的轴类型；
- (4) RTEX_Home 指令动作完成后，切换轴类型至需要的轴类型（如 40）。
- (5) 最好把驱动器单位设定为 指令单位/s（设定 Pr7.25=1），这样可以对原点回归速度进行更为精细的设定。
- (6) 当指令运行过程中，对应轴的从站在线参数将不会刷新，RTEX_Read、RTEX_Write 等 RTEX

部分相关指令将返回错误。RTEX_Home 指令结束后，将恢复正常。

3.5.2.13 RTEX_GetHomeStatus（获取 RTEX 伺服回零状态）

3.5.2.13.1 函数原型

UDINT RTEX_GetHomeStatus (USINT ucAxisID)

3.5.2.13.2 功能说明

通过该指令可获取 RTEX 总线原点回归状态。

3.5.2.13.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.5.2.13.4 返回值

返回值	类型	参数描述
0	USINT	原点回归未完成
1	USINT	原点回归完成

3.5.2.13.5 指令类型

非轴运动缓存指令。

3.5.2.14 RTEX_ProfileSmoothStop（RTEX PP 模式减速停止）

3.5.2.14.1 函数原型

UDINT RTEX_ProfileSmoothStop (USINT ucAxisID)

3.5.2.14.2 功能说明

RTEX_PP 模式减速停止，主要用于终止 RTEX_Home 动作。

RTEX 原点回归动作执行过程中，如果发生意外情况，可调用 RTEX_ProfileSmoothStop 终止原点回归操作。但这样会导致 DPOS 不等于 Mpos 的，所以请务必确保 RTEX_Home 最终执行成功后再切换为所需的轴类型。

3.5.2.14.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.5.2.14.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.5.2.14.5 指令类型

非轴运动缓存指令。

3.5.2.15 RTEX_ProfileHardStop（RTEX PP 模式急停）

3.5.2.15.1 函数原型

UDINT RTEX_ProfileHardStop (USINT ucAxisID)

3.5.2.15.2 功能说明

RTEX_PP 模式急停，主要用于终止 RTEX_Home 动作。

RTEX 原点回归动作执行过程中，如果发生意外情况，可调用 RTEX_ProfileHardStop 终止原点回归操作。但这样会导致 DPOS 不等于 Mpos 的，所以请务必确保 RTEX_Home 最终执行成功后再切换为所需的轴类型。

3.5.2.15.3 输入参数

输入参数	类型	参数描述
ucAxisID	USINT	轴号

3.5.2.15.4 返回值

返回值	类型	参数描述
0	UDINT	执行成功
0xFFFFFFFF	UDINT	执行失败

3.5.2.15.5 指令类型

非轴运动缓存指令。

3.5.3 CANopen 协议相关指令

3.5.3.1 CANopenReadDict（获取 CANopen 从节点对象字典参数）

3.5.3.1.1 功能说明

读取 CANopen 远程节点对象字典。

3.5.3.1.2 参数说明

输入参数	数据类型	参数描述	初始值
EN_R	BOOL	上升沿触发	0
NodeID	BYTE	远程节点：1~126	0
Index	WORD	参数索引：0x1000~0x9FFF	0
SubIndex	BYTE	参数子索引	0

输出参数	数据类型	参数描述	初始值
Done	BOOL	读完成 0：未完成 1：成功完成	0
Busy	BOOL	读操作进行中 0：未开始或已结束 1：进行中	0

输出参数	数据类型	参数描述	初始值
Error	BOOL	错误标志 0: 无错误 1: 有错误	0
ErrorID	DWORD	错误码, 请参见错误码	0
ReadLen	WORD	读取的对象字典参数大小 1: 读取的对象字典参数大小为 1 字节 2: 读取的对象字典参数大小为 2 字节 4: 读取的对象字典参数大小为 4 字节	0
ReadData	DWORD	读取的对象字典参数值	0

3.5.3.2 CANopenWriteDict(更新 CANopen 从节点对象字典参数)

3.5.3.2.1 功能说明

写 CANopen 远程节点对象字典。

3.5.3.2.2 参数说明

输入参数	数据类型	参数描述	初始值
EN_R	BOOL	上升沿触发	0
NodeID	BYTE	远程节点: 1~126	0
Index	WORD	参数索引: 0x1000~0x9FFF	0
SubIndex	BYTE	参数子索引	0
DataLen	WORD	写入的对象字典值参数大小: 1: 写入的对象字典参数大小为 1 字节 2: 写入的对象字典参数大小为 2 字节 4: 写入的对象字典参数大小为 4 字节	0
WriteData	DWORD	写入的对象字典参数值	0

输出参数	数据类型	参数描述	初始值
Done	BOOL	写完成 0: 未完成 1: 成功完成	0
Busy	BOOL	写操作进行中 0: 未开始或已结束 1: 进行中	0
Error	BOOL	错误标志 0: 无错误 1: 有错误	0
ErrorID	DWORD	错误码, 请参见错误码	0

3.5.3.3 CANopenNodeNMT（发送 NMT 命令）

3.5.3.3.1 功能说明

向 CANopen 远程节点设备发送 NMT 命令。

NMT 指令发送完成，仅表示 CANopen 主站已向指定的节点发送 NMT 命令，不代表该点已经按照命令字完成操作。

3.5.3.3.2 参数说明

输入参数	数据类型	参数描述	初始值
EN_R	BOOL	上升沿触发	0
NodeID	BYTE	远程节点： 0:广播所有连接从站 1~126:指定对应从站	0
Command	BYTE	NMT 命令字设定值： 0x01:启动远程节点 0x02:停止远程节点 0x80:节点进入预运行状态 0x81:应用复位 0x82:通信复位	0

输出参数	数据类型	参数描述	初始值
Done	BOOL	命令已发送完成 0: 未完成 1: 完成	0
Busy	BOOL	正在发送中 0: 未开始或已结束 1: 进行中	0
Error	BOOL	错误标志 0: 无错误 1: 有错误	0
ErrorID	DWORD	错误码，请参见错误码	0

3.5.3.4 CANopenResetNodeDiag（清除从节点诊断信息）

3.5.3.4.1 功能说明

清除指定节点的诊断信息。

3.5.3.4.2 参数说明

输入参数	数据类型	参数描述	初始值
EN_R	BOOL	上升沿触发	0
NodeID	BYTE	远程节点: 1~126	0

输出参数	数据类型	参数描述	初始值
Done	BOOL	清除完成 0: 未完成 1: 完成	0
Busy	BOOL	正在清除中 0: 未开始或已结束 1: 进行中	0
Error	BOOL	错误标志 0: 无错误 1: 有错误	0
ErrorID	DWORD	错误码, 请参见错误码	0

3.6 产品扩展库

3.6.1 DI 置反

3.6.1.1 SetInvertDI (设置 DI 反转标志)

3.6.1.1.1 功能

DI 状态掩码, 32 个 bit 位与本体 DI 的输入通道对应。uDIMask 的某个指定 bit 位与所对应的 DI 通道位的异或结果作为上位机、LED 屏的显示结果。DI 状态掩码只对本体物理 DI 有效, 对扩展 DI 以及虚拟 DI 掩码无效。

3.6.1.1.2 输入参数

输入参数	类型	参数描述
uDIMask	UDINT	0: 所显示的 DI 状态与实际输入的电平状态一致 1: 所显示的 DI 状态与实际输入的电平状态相反

3.6.1.1.3 返回值

返回值	类型	参数描述
0xFFFFFFFF	UDINT	函数参数错误

- 不支持的模块版本
MC1008-A01
- 备注：阻塞指令，非轴运动缓存指令。

3.6.1.2 GetInvertDI（获取 DI 反转标志）

3.6.1.2.1 功能

获取 DI 状态掩码值。

3.6.1.2.2 返回值

返回值	类型	参数描述
DI 的状态掩码值	UDINT	成功
0xFFFFFFFF	UDINT	函数参数错误

- 不支持的模块版本
MC1008-A01
- 备注：阻塞指令，非轴运动缓存指令。

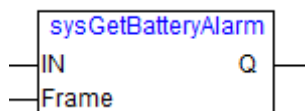
3.6.2 产品内部状态

3.6.2.1 sysGetBatteryAlarm（获取电池电量报警）

3.6.2.1.1 功能

获取主控模块电池电量信息。

当备机为从机状态时，功能块输出值有效。



sysGetBatteryAlarm 指令

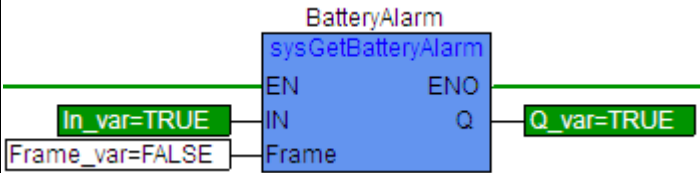
3.6.2.1.2 参数说明

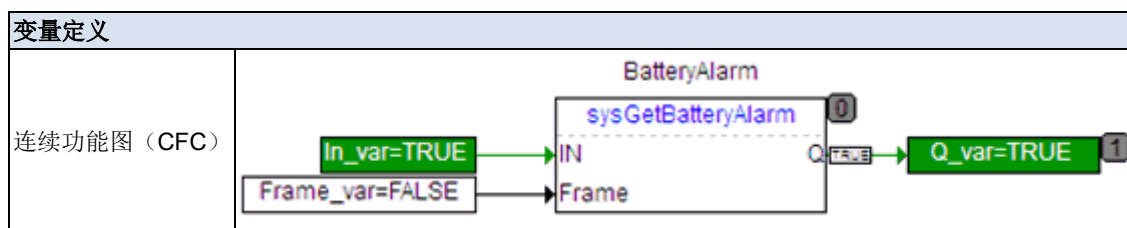
输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
Frame	BOOL	机架选择（默认值 FALSE） TRUE：非主机架 FALSE：主机架

输出参数	类型	含义
Q	BOOL	电池电量 TURE：没电池或者电池电量不足，需更换电池 FALSE：电池电量正常

3.6.2.1.3 指令使用举例

变量定义					
序号	变量名	直接地址	变量说明	变量类型	在线值
0001	In_var			BOOL	TRUE
0002	Frame_var			BOOL	FALSE
0003	Q_var			BOOL	TRUE
0004	BatteryAlarm			SYSGETBATTERYALARM	

编程语言	程序
梯形图（LD）	 The ladder logic diagram shows a network for 'BatteryAlarm'. It starts with a green box 'In_var=TRUE' connected to the 'IN' input of the 'sysGetBatteryAlarm' block. Below it, a white box 'Frame_var=FALSE' is connected to the 'Frame' input. The 'ENO' output of the block is connected to a green box 'Q_var=TRUE'.
结构化文本（ST）	<pre> 0001 BatteryAlarm(0002 IN:=In_var, 0003 Frame:=Frame_var, 0004 Q=>Q_var); 0005 0006 </pre> <pre> BatteryAlarm In_var = TRUE Frame_var = FALSE Q_var = TRUE </pre>



程序说明

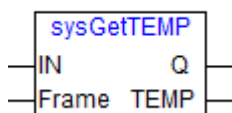
示例为主机架电池电量不足，触发 Q 端输出为 TRUE，此时需要更换 CPU 模块电池。

3.6.2.2 sysGetTEMP（获取主控 CPU 温度）

3.6.2.2.1 功能

获取主控模块温度信息。

当备机为从机状态时，功能块输出值有效。



sysGetTEMP 指令

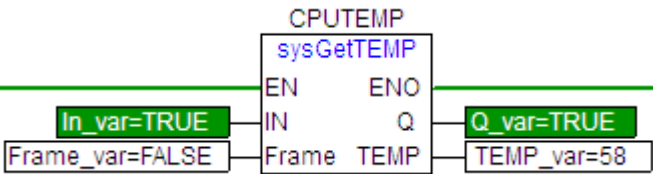
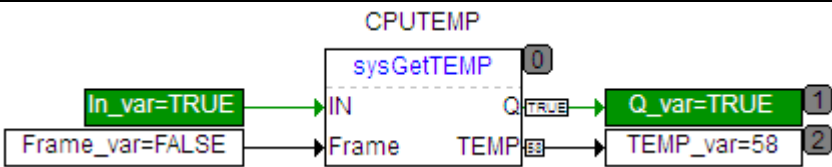
3.6.2.2.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能（默认值 FALSE）
Frame	BOOL	机架选择（默认值 FALSE） TRUE：非主机架 FALSE：主机架

输出参数	类型	含义
Q	BOOL	输出（默认值 FALSE）
TEMP	SINT	温度值，单位为摄氏度（默认值为 0）

3.6.2.2.3 指令使用举例

变量定义						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电
0001	In_var			BOOL	TRUE	FALSE
0002	Frame_var			BOOL	FALSE	FALSE
0003	Q_var			BOOL	TRUE	FALSE
0004	TEMP_var			SINT	58	FALSE
0005	CPUTEMP			SYSGETTEMP		FALSE

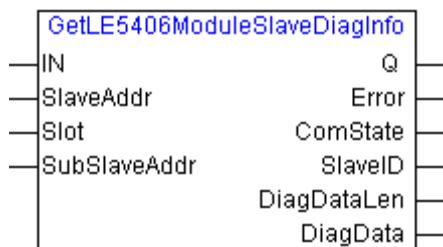
编程语言	程序
梯形图 (LD)	
结构化文本 (ST)	<pre> 0001 CPUTEMP(0002 IN:=In_var, 0003 Frame:=Frame_var, 0004 Q=>Q_var, 0005 TEMP=>TEMP_var); 0006 0007 0008 </pre> CPUTEMP In_var = TRUE Frame_var = FALSE Q_var = TRUE TEMP_var = 58
连续功能图 (CFC)	

3.6.3 模块扩展信息

3.6.3.1 GetLE5406ModuleSlaveDiagInfo（获取 LE5406 网关模块从站诊断信息）

3.6.3.1.1 功能

获取 LE5406 网关模块下挂接 LE IO 从站的诊断信息。



GetLE5406ModuleSlaveDiagInfo 指令

3.6.3.1.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能，默认 FALSE
SlaveAddr	BYTE	LE5406 地址，有效值为 2~125
Slot	BYTE	DP 主卡槽位号
SubSlaveAddr	BYTE	LE IO 从站地址

输出参数	类型	含义
Q	BOOL	输出使能标志
Error	BYTE	错误码： 0：正确 1：槽号错误 2：网关模块的从站地址配置错误 3：网关模块下 IO 从站地址配置错误 4：网关模块未上线 128：没有网关模块的上级主站或槽号不匹配 255：模块类型不匹配
ComState	BOOL	LE IO 从站通讯状态： TRUE：通讯正常 FALSE：通讯异常
SlaveID	LEIOTYPE	LE IO 从站 ID 号
DiagDataLen	BYTE	从站诊断信息长度
DiagData	ARRAY OF BYTE	LE IO 从站诊断信息，请参见 LE 特殊寄存器区

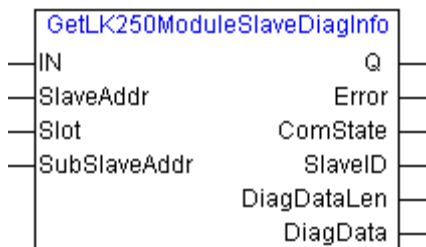
i 当模块在线时（即输出参数 Q、Error、ComState 均为正常状态），LE IO 从站诊断信息有效。

3.6.3.2 GetLK250ModuleSlaveDiagInfo（获取 LK250 网关模块从站诊断信息）

3.6.3.2.1 功能

获取 LK250 网关模块下 IO 从站的诊断信息。AutoThink V3.1.8 及之后版本建议使用

GetLK250ModuleSlaveDiagInfoExt（获取 LK250 网关模块从站诊断信息）功能块获取诊断信息。



GetLK250ModuleSlaveDiagInfo 指令

3.6.3.2.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能，默认 FALSE
SlaveAddr	BYTE	LK250 地址，有效值为 2~125
Slot	BYTE	LK249 槽位号
SubSlaveAddr	BYTE	LK250 下挂从站地址

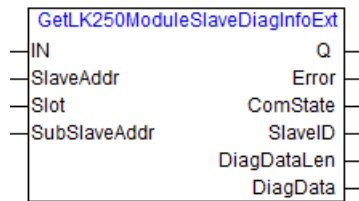
输出参数	类型	含义
Q	BOOL	输出使能标志
Error	BYTE	错误码： 0：正确 1：槽号错误 2：网关模块的从站地址配置错误 3：网关模块的子从站地址配置错误 4：网关模块未上线 128：没有网关模块的上级主站或槽号不匹配 255：模块类型不匹配
ComState	BOOL	DP 从站通讯状态： TRUE：通讯正常 FALSE：通讯异常
SlaveID	DPMODULETYPE	LK250 下挂从站 ID 号
DiagDataLen	BYTE	从站扩展诊断信息长度
DiagData	ARRAY OF BYTE	LK250 下 DP 从站扩展诊断信息，各模块诊断信息不同，请参见各模块的诊断信息说明

 当模块在线时（即输出参数 Q、Error、ComState 均为正常状态），IO 从站诊断信息有效。

3.6.3.3 GetLK250ModuleSlaveDiagInfoExt（获取 LK250 网关模块从站诊断信息）

3.6.3.3.1 功能

获取 LK250 网关模块下 IO 从站的诊断信息。AutoThink V3.1.8 及之后版本建议使用该功能块获取诊断信息。



GetLK250ModuleSlaveDiagInfoExt 指令

3.6.3.3.2 参数说明

输入参数	类型	含义
IN	BOOL	高电平使能，默认 FALSE
SlaveAddr	BYTE	LK250 地址，有效值为 2~125
Slot	BYTE	LK249 槽位号
SubSlaveAddr	BYTE	LK250 下挂从站地址

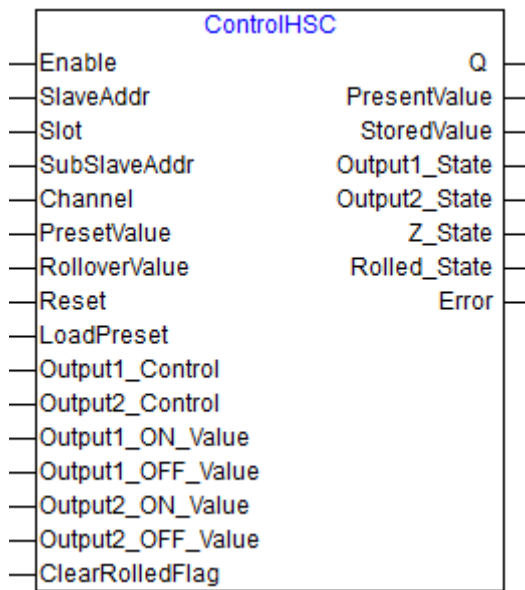
输出参数	类型	含义
Q	BOOL	输出使能标志
Error	BYTE	错误码： 0：正确 1：槽号错误 2：网关模块的从站地址配置错误 3：网关模块的子从站地址配置错误 4：网关模块未上线 128：没有网关模块的上级主站或槽号不匹配 255：模块类型不匹配
ComState	BOOL	DP 从站通讯状态： TRUE：通讯正常 FALSE：通讯异常
SlaveID	DPMODULETYPE_N	LK250 下挂从站 ID 号
DiagDataLen	BYTE	从站扩展诊断信息长度
DiagData	ARRAY OF BYTE	LK250 下 DP 从站扩展诊断信息，各模块诊断信息不同，请参见各模块的诊断信息说明

①当模块在线时（即输出参数 Q、Error、ComState 均为正常状态），IO 从站诊断信息有效。

3.6.3.4 ControlHSC（LK620 控制监视功能块）

3.6.3.4.1 功能说明

设置 LK620 模块的参数以及读取 LK620 模块的运算状态。



ControlHSC 指令

3.6.3.4.2 参数说明

输入参数	数据类型	功能描述	参数值说明
Enable	BOOL	使能输入	0: 无效 1: 有效
SlaveAddr	BYTE	站地址	LK620 模块从站地址，地址范围 2~125，默认值 2
Slot	BYTE	槽位号	通信板卡槽位号（LK249/LK241），范围值 2~5，默认值 2
SubSlaveAddr	BYTE	从站站地址	LK235/LK271 模块从站地址，地址范围 1~239，默认值 1 当前通讯模块为 LK241 时，需要设置该地址。当通讯模块为 LK249 时，该参数不设置，保持默认值 1
Channel	BYTE	通道号	=1，通道 1（计数器 1） =2，通道 2（计数器 2）
PresetValue	DWORD	预置值	计数器从该值开始，此值必须小于翻转值，否则会发生计数错误，预置值的取值范围 0~4,294,967,295
RolloverValue	DWORD	翻转值	计数模式下，作为计数的上限，取值范围为 1~4,294,967,295。在计数过程中，当计数值 =（翻转值 - 1）时，计数器回到 0，重新开始计数
Reset	BOOL	复位计数器	0: 无效 0→1: 上升沿，计数器复位并从 0 开始计数 1: 两个输出通道输出 OFF，不按照组态设置输出
LoadPreset	BOOL	预置值控制位	0: 无效 1: 上升沿有效，计数器载入预置值并从预置值开始计数

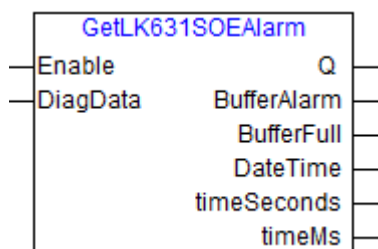
输入参数	数据类型	功能描述	参数值说明
Output1_Control	BYTE	输出控制	用于控制计数器各输出点的状态，输出 1 当前输出值修改： =0x00，根据计数结果输出 =0x02，强制修改 OUT1 输出值为 OFF =0x03，强制修改 OUT1 输出值为 ON
Output2_Control	BYTE	输出控制	用于控制计数器各输出点的状态，输出 2 当前输出值修改： =0x00，根据计数结果输出 =0x02，强制修改 OUT1 输出值为 OFF =0x03，强制修改 OUT1 输出值为 ON
Output1_ON_Value	DWORD	输出 ON 触发值	输出 1 输出 ON 触发值(0~4,294,967,295)
Output1_OFF_Value	DWORD	输出 OFF 触发值	输出 1 输出 OFF 触发值(0~4,294,967,295)
Output2_ON_Value	DWORD	输出 ON 触发值	输出 2 输出 ON 触发值(0~4,294,967,295)
Output2_OFF_Value	DWORD	输出 OFF 触发值	输出 2 输出 OFF 触发值(0~4,294,967,295)
ClearRolledFlag	BOOL	翻转标志清除位	清除翻转标志位

输出参数	数据类型	功能描述	参数值说明
Q	BOOL	输出完成标志	0: 输出未完成 1: 输出完成
PresentValue	DWORD	当前计数值	存放所选计数器的当前计数值
StoredValue	DWORD	存储计数值	所选计数器的当前存储计数值
Output1_State	BOOL	输出 1 状态	1: ON (通道闭合)，即当前计数值达到 Output1_ON_Value 0: OFF (通道断开，停止输出)，即当前计数达到 Output1_OFF_Value
Output2_State	BOOL	输出 2 状态	1: ON (通道闭合)，即当前计数值达到 Output2_ON_Value 0: OFF (通道断开，停止输出)，即当前计数达到 Output2_OFF_Value
Z_State	BOOL	Z 状态	0: 低电平 1: 高电平
Rolled_State	BOOL	翻转标志	计数器是否达到翻转值并翻转 0: 未翻转 1: 已翻转
Error	DWORD	错误信息	值为 0，无错误 bit1: 槽位不在范围内 2~5 bit2: 模块 ID 不在范围内 3~4 bit3: LK235/LK271 站号地址不在范围内 1~239 (LK241) bit4: 输入 IO 从站不在范围内 2~125 (LK249/LK235/LK271) bit5: 此号站配置的不是 LK620 模块 bit6: 通道值错位 bit7: 预设值错误 bit8: 输出控制位错误 bit9: 输出触发值错误

3.6.3.5 GetLK631SOEAlarm(LK631 缓存告警信息查询功能块)

3.6.3.5.1 功能

获取 LK631 模块的 SOE 事件告警信息，包括 SOE 记录条数 70%告警信息、100%告警信息以及事件记录时间信息。



GetLK631SOEAlarm 指令

3.6.3.5.2 参数说明

输入参数	类型	含义
Enable	BOOL	使能输入 0: 无效 1: 有效
DiagData	ARRAY[0..237] OF BYTE	诊断数据。当模块上报诊断数据时，记录一次 SOE 事件 模块在 Profibus-DP 下组态时，该参数关联到 sysGetDPSlaveStateExt 指令的输出引脚变量“DiagData” 模块在 POWERLINK 下组态时，您需要将 LK631 模块 S 区诊断变量的首地址+6 赋值给“DiagData”

输出参数	类型	含义
Q	BOOL	输出完成标志 0: 输出未完成 1: 输出完成
BufferAlarm	BOOL	70%告警标志 LK631 模块 SOE 缓存条数超过最大存储的 70%时，置位告警标志 1，最大存储 3072 条
BufferFull	BOOL	100%告警标志 LK631 模块 SOE 缓存条数超过最大存储的 100%时，置位告警标志 1，最大存储 3072 条
DateTime	DT	SOE 告警事件发生的记录时间
timeSeconds	DWORD	分钟及以上的事件记录时间对应的秒时间
timeMs	WORD	分钟以下的事件记录时间对应的毫秒时间

3.6.3.6 COMM_SET_CONFIG（LK238 设置通讯指令参数功能块）

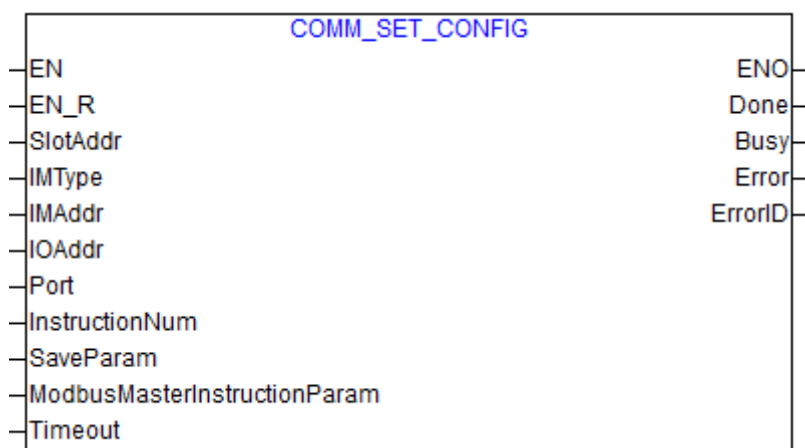
3.6.3.6.1 功能

设置 LK238 模块的通讯指令参数，一次最多设置 16 条指令。该指令有示例工程，可直接导入使用。

导入算法示例时，可通过以下两种方式导入：

- 菜单栏中，单击【工程】—【导入示例工程】菜单。详细说明请参见导入示例工程。
- 库管理器中，右击功能块，选择【导入示例工程】命令。详细说明请参见 LK238 串口通信模块。

导入的示例工程需要在程序中调用或任务下调用才能正常使用。



COMM_SET_CONFIG 指令

3.6.3.6.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿执行
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1: 接口模块类型为 LK231 或 LK232，此时， IMAAddr 参数无意义 2: 接口模块类型为 LK235/LK271 3: 接口模块类型为 LK250 4: 接口模块类型为 LK234/LK235M，此时， SlotAddr 参数无意义
IMAAddr	DWORD	IO 模块所在背板使用的接口模块地址 当接口模块为 LK231 或 LK232 时，IMAAddr 值无意义，无需设置 当接口模块为 LK235/LK271 时，IMAAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时，IMAAddr 与扩展背板拨码开关设置值保持一致

输入参数	类型	含义
		当接口模块为 LK234/LK235M 时, IMAAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果, 例如 IP 地址 192.168.1.250, IMAAddr 参数为 16#C0A801FA 若 IMAAddr 配置错误时, ErrorID 会报 7
IOAddr	BYTE	IO 模块所在背板的 DP 从站地址 (2~125)
Port	BYTE	端口号 (1~4)
InstructionNum	BYTE	配置指令数目 (1~16)
SaveParam	BOOL	下发指令参数是否掉电保持
ModbusMasterInstructionParam	ARRAY[0..15] OF MASTERINSTRUCTIONPARAM	Modbus 主站指令参数结构体 Modbus 主站指令参数结构体中存在重复指令 ID 时, 结构体成员中最后一个指令 ID 的指令参数有效
Timeout	WORD	执行超时时间(单位:1ms) (0~65535), 当设置为 0 时采用默认值 2000; 该参数表示, 控制器与串口模块通讯异常后多长时间报错, 与实际通讯是否超时无关

输出参数	类型	含义
Done	BOOL	通讯参数配置完成
Busy	BOOL	正在进行通信参数配置
Error	BOOL	设置通信参数过程中发生错误
ErrorID	WORD	错误码: =0: 无错误 =1: IMType 参数错误 =2: SlotAddr 指定槽位范围错误 =3: SlotAddr 指定槽位的通信板卡类型不支持 =4: IOAddr 指定的 IO 模块 DP 从站地址错误 =5: IMAAddr 指定的接口模块地址错误 =7: IOAddr 指定地址模块类型错误 (此号站配置的不是 LK238 模块, 请检查 IMType、SlotAddr、IMAAddr 以及 IOAddr 参数配置是否正确) =8: 输入/输出字节长度错误 (请检查模块配置页面输入/输出模块) =9: Port 端口参数配置错误 =10: InstructionNum 配置指令个数错误 =11: 配置指令参数长度超过输出字节长度 (请检查模块配置页面输出模块) =12: ModbusMasterInstructionParam 参数配置错误 =13: 该端口的用户参数配置的不是 Modbus 主站 =15: 控制器与串口模块通讯异常

MasterInstructionParam 结构体成员

成员变量	数据类型	参数说明
InstructionID	BYTE	指令 ID 号 (1~32)
InstructionStatus	BYTE	指令状态 (0: 指令无效, 1: 指令生效)
StationAddress	BYTE	从站地址 (1~247)
FunctionCode	BYTE	0x01 (读线圈)、0x02 (读离散量输入)、0x03 (读保持寄存器)、0x04 (读输入寄存器)、0x05 (写单个线圈)、0x06 (写单个寄存器)、0x0F (写多个线圈)、0x10 (写多个寄存器)
OffsetAddress	WORD	读写偏移地址
DataLength	WORD	读写开关量/模拟量数据长度 开关量: 1~1600

成员变量	数据类型	参数说明
		模拟量：1~100
Timeout	WORD	执行超时时间（单位：1ms），当设置为 0 时采用默认值 1000
TriggerMode	BYTE	触发模式 0：周期触发，当参数配置成功，且协议端口使能，则 Modbus 按照 Period 设定的轮询周期开始通讯 1：事件触发，当参数配置成功，且协议端口使能，通过 COMM_RW_MODBUS_MASTER（LK238 Modbus 主站读写通讯数据功能块）中 EN_R 上升沿触发一次 Modbus 通讯
Period	BYTE	轮询周期时间（单位：10ms），当设置为 0 时采用默认轮询方式为死兜型

3.6.3.7 COMM_GET_CONFIG（LK238 获取通讯指令参数功能块）

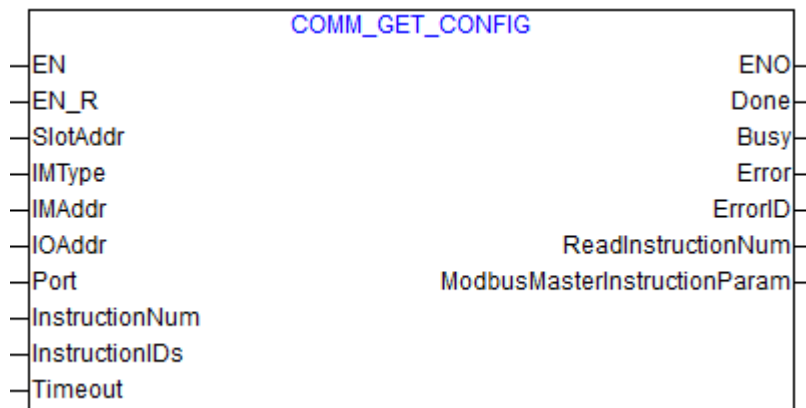
3.6.3.7.1 功能

获取 LK238 模块的通讯指令参数，一次最多获取 16 条指令。该指令有示例工程，可直接导入使用。

导入算法示例时，可通过以下两种方式导入：

- 菜单栏中，单击【工程】—【导入示例工程】菜单。详细说明请参见导入示例工程。
- 库管理器中，右击功能块，选择【导入示例工程】命令。详细说明请参见 LK238 串口通信模块。

导入的示例工程需要在程序中调用或任务下调用才能正常使用。



COMM_GET_CONFIG 指令

3.6.3.7.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿执行
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1：接口模块类型为 LK231 或 LK232，此时，IMAAddr 参数无意义 2：接口模块类型为 LK235/LK271

输入参数	类型	含义
		3: 接口模块类型为 LK250 4: 接口模块类型为 LK234/LK235M, 此时, SlotAddr 参数无意义
IMAddr	DWORD	IO 模块所在背板使用的接口模块地址 当接口模块为 LK231 或 LK232 时, IMAddr 值无意义, 无需设置 当接口模块为 LK235/LK271 时, IMAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时, IMAddr 与扩展背板拨码开关设置值保持一致 当接口模块为 LK234/LK235M 时, IMAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果, 例如 IP 地址 192.168.1.250, IMAddr 参数为 16#C0A801FA 若 IMAddr 配置错误时, ErrorID 会报 7
IOAddr	BYTE	IO 模块所在背板的 DP 从站地址 (2~125)
Port	BYTE	端口号 (1~4)
InstructionNum	BYTE	配置指令数目 (1~16)
InstructionIDs	ARRAY[0..15] OF BYTE	指令 ID 号 (1~32)
Timeout	WORD	执行超时时间(单位:1ms) (0~65535), 当设置为 0 时采用默认值 2000; 该参数表示, 控制器与串口模块通讯异常后多长时间报错, 与实际通讯是否超时无关

输出参数	类型	含义
Done	BOOL	读取通讯参数完成
Busy	BOOL	正在进行读取参数配置
Error	BOOL	读取通信参数过程中发生错误
ErrorID	WORD	功能块错误码 =0: 无错误 =1: IMType 协议类型错误 =2: SlotAddr 指定槽位范围错误 =3: SlotAddr 指定槽位的通信板卡类型不支持 =4: IOAddr 指定的 IO 模块 DP 从站地址错误 =5: IMAddr 指定的接口模块地址错误 =7: IOAddr 指定地址模块类型错误 (此号站配置的不是 LK238 模块, 请检查 IMType、SlotAddr、IMAddr 以及 IOAddr 参数配置是否正确) =8: 输入/输出字节长度错误 (请检查模块配置页面输入/输出模块) =9: Port 端口参数配置错误 =10: InstructionNum 参数配置错误 =11: 获取指令参数长度超过输入字节长度 (请检查模块配置页面输入模块) =12: InstructionIDs 参数配置错误 =13: 该端口的用户参数配置的不是 Modbus 主站 =15: 控制器与串口模块通讯异常
ReadInstructionNum	BYTE	读取配置总个数
ModbusMasterInstructionParam	ARRAY[0..15] OF MASTERINSTRUCTIONPARAM	Modbus 主站指令参数结构体

备注 1: 当 IMType 为 1 时, IMAddr 无意义;

备注 2: 当 IMType 为 4 时, SlotAddr 无意义, IMAddr 是 IPv4 的 4 段分别转换为 16 进制后合并的结果, 例如 IP 地址 192.168.1.250, 输入参数为 16#C0A801FA; 若 IMAddr 配置错误时, ErrorID 会报 7。

MasterInstructionParam 结构体成员

成员变量	数据类型	参数说明
InstructionID	BYTE	指令 ID 号（1~32）
InstructionStatus	BYTE	指令状态（0：指令无效，1：指令生效）
StationAddress	BYTE	从站地址（1~247）
FunctionCode	BYTE	0x01（读线圈）、0x02（读离散量输入）、0x03（读保持寄存器）、0x04（读输入寄存器）、0x05（写单个线圈）、0x06（写单个寄存器）、0x0F（写多个线圈）、0x10（写多个寄存器）
OffsetAddress	WORD	读写偏移地址
DataLength	WORD	读写开关量/模拟量数据长度 开关量：1~1600 模拟量：1~100
Timeout	WORD	执行超时时间（单位：1ms）
TriggerMode	BYTE	触发模式（0：周期触发，1：事件触发）
Period	BYTE	轮询周期时间（单位：10ms）

3.6.3.8 COMM_RW_MODBUS_MASTER（LK238 Modbus 主站读写通讯数据功能块）

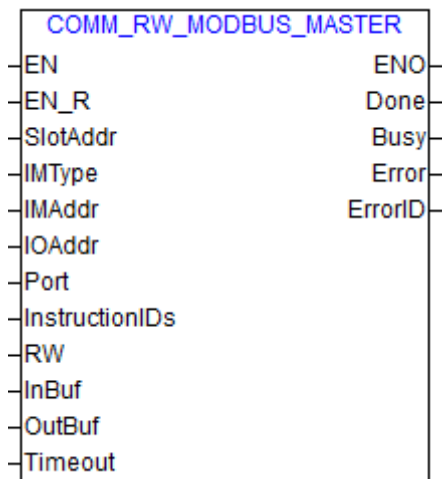
3.6.3.8.1 功能

读取或写入 LK238 Modbus 主站数据。该指令有示例工程，可直接导入使用。

导入算法示例时，可通过以下两种方式导入：

- 菜单栏中，单击【工程】—【导入示例工程】菜单。详细说明请参见导入示例工程。
- 库管理器中，右击功能块，选择【导入示例工程】命令。详细说明请参见 LK238 串口通信模块。

导入的示例工程需要在程序中调用或任务下调用才能正常使用。



COMM_RW_MODBUS_MASTER 指令

3.6.3.8.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿执行
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1：接口模块类型为 LK231 或 LK232，此时，IMAddr 参数无意义 2：接口模块类型为 LK235/LK271 3：接口模块类型为 LK250 4：接口模块类型为 LK234/LK235M，此时，SlotAddr 参数无意义
IMAddr	DWORD	IO 模块所在背板使用的接口模块地址 当接口模块为 LK231 或 LK232 时，IMAddr 值无意义，无需设置 当接口模块为 LK235/LK271 时，IMAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时，IMAddr 与扩展背板拨码开关设置值保持一致 当接口模块为 LK234/LK235M 时，IMAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果，例如 IP 地址 192.168.1.250，IMAddr 参数为 16#C0A801FA 若 IMAddr 配置错误时，ErrorID 会报 7
IOAddr	BYTE	IO 模块所在背板的 DP 从站地址（2~125）
Port	BYTE	端口号（1~4）
InstructionIDs	BYTE	指令 ID 号（1~32）
RW	BOOL	读写方向（RW 为 0 时读数据，1 时写数据）
InBuf	POINTER TO BYTE	输入数据缓冲区，指向的缓冲区大小必须≥224 Bytes
OutBuf	POINTER TO BYTE	输出数据缓冲区，指向的缓冲区大小必须≥224 Bytes
Timeout	WORD	执行超时时间(单位:1ms)（0~65535），当设置为 0 时采用默认值 2000；该参数表示，控制器与串口模块通讯异常后多长时间报错，与实际通讯是否超时无关

输出参数	类型	含义
Done	BOOL	读写主站数据完成
Busy	BOOL	正在进行读写主站数据
Error	BOOL	读写主站数据过程中发生错误

输出参数	类型	含义
ErrorID	WORD	功能块错误码 =0: 无错误 =1: IMType 协议类型错误 =2: SlotAddr 指定槽位范围错误 =3: SlotAddr 指定槽位的通信板卡类型不支持 =4: IOAddr 指定的 IO 模块 DP 从站地址错误 =5: IMAAddr 指定的接口模块地址错误 =7: IOAddr 指定地址模块类型错误（此号站配置的不是 LK238 模块，请检查 IMType、SlotAddr、IMAAddr 以及 IOAddr 参数配置是否正确） =8: 输入/输出字节长度错误（请检查模块配置页面输入/输出模块） =9: Port 端口参数配置错误 =10: InstructionIDs 参数配置错误 =15: 控制器与串口模块通讯异常 主站错误信息 =1001:数据帧错误 =1002:超时 =1004:当前指令无效 =1005:端口未工作（当用户参数页面指定端口 Com Status 为 Disable） =1006:端口协议不匹配 =1128+1:从站不支持的功能码 =1128+2: 数据地址溢出 =1128+3: 数据范围溢出 =1128+4: 从站设备故障 =1128+6: 从站设备忙

3.6.3.9 COMM_RW_MODBUS_SLAVE（LK238 Modbus 从站读写通讯数据功能块）

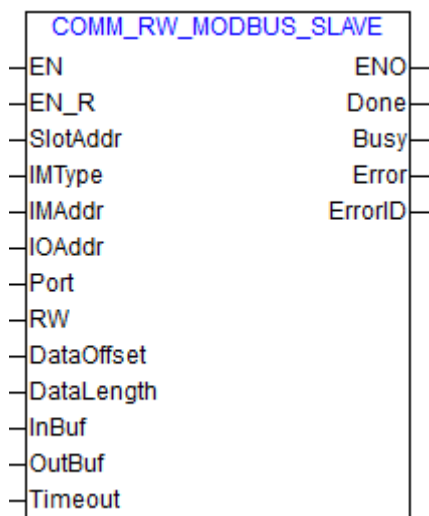
3.6.3.9.1 功能

读取或写入 LK238 Modbus 从站数据。该指令有示例工程，可直接导入使用。

导入算法示例时，可通过以下两种方式导入：

- 菜单栏中，单击【工程】—【导入示例工程】菜单。详细说明请参见导入示例工程。
- 库管理器中，右击功能块，选择【导入示例工程】命令。详细说明请参见 LK238 串口通信模块。

导入的示例工程需要在程序中调用或任务下调用才能正常使用。



COMM_RW_MODBUS_SLAVE 指令

3.6.3.9.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿执行
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1：接口模块类型为 LK231 或 LK232，此时，IMAAddr 参数无意义 2：接口模块类型为 LK235/LK271 3：接口模块类型为 LK250 4：接口模块类型为 LK234/LK235M，此时，SlotAddr 参数无意义
IMAAddr	DWORD	IO 模块所在背板使用的接口模块地址 当接口模块为 LK231 或 LK232 时，IMAAddr 值无意义，无需设置 当接口模块为 LK235/LK271 时，IMAAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时，IMAAddr 与扩展背板拨码开关设置值保持一致 当接口模块为 LK234/LK235M 时，IMAAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果，例如 IP 地址 192.168.1.250，IMAAddr 参数为 16#C0A801FA 若 IMAAddr 配置错误时，ErrorID 会报 7
IOAddr	BYTE	IO 模块所在背板的 DP 从站地址（2~125）
Port	BYTE	端口号（1~4）
RW	BOOL	读写方向（RW 为 0 时读数据，1 时写数据）
DataOffset	WORD	读写数据偏移地址
DataLength	BYTE	读取数据长度
InBuf	POINTER TO BYTE	输入数据缓冲区，指向的缓冲区大小必须≥224 Bytes
OutBuf	POINTER TO BYTE	输出数据缓冲区，指向的缓冲区大小必须≥224 Bytes
Timeout	WORD	执行超时时间(单位:1ms)（0~65535），当设置为 0 时采用默认值 2000；该参数表示，控制器与串口模块通讯异常后多长时间报错，与实际通讯是否超时无关

输出参数	类型	含义
Done	BOOL	读写从站数据完成

输出参数	类型	含义
Busy	BOOL	正在进行读写从站数据
Error	BOOL	读写从站数据过程中发生错误
ErrorID	WORD	功能块错误码 =0: 正确 =1: IMType 协议类型错误 =2: SlotAddr 指定槽位范围错误 =3: SlotAddr 指定槽位的通信板卡类型不支持 =4: IOAddr 指定的 IO 模块 DP 从站地址错误 =5: IMAddr 指定的接口模块地址错误 =7: IOAddr 指定地址模块类型错误（此号站配置的不是 LK238 模块，请检查 IMType、SlotAddr、IMAddr 以及 IOAddr 参数配置是否正确） =8: 输入/输出字节长度错误（请检查模块配置页面输入/输出模块） =9: Port 端口参数配置错误 =10: 读取数据长度超过输入字节长度（请检查模块配置页面输入模块） =11: 写数据长度超过输出字节长度（请检查模块配置页面输出模块） =12: 读写数据长度设定值过小（为 0） =13: 读写数据长度超过 220 字节 =14: 偏移加长度大于 60*1024 =15: 控制器与串口模块通讯异常 从站错误信息 =1001: CRC 校验错误; =1005: 端口未工作（当用户参数页面指定端口 Com Status 为 Disable） =1006: 端口协议不匹配 =1128+2: 数据地址溢出; =1128+3: 数据范围溢出;

3.6.3.10 COMM_RW_FREEPORT（LK238 自由口协议读写通讯数据功能块）

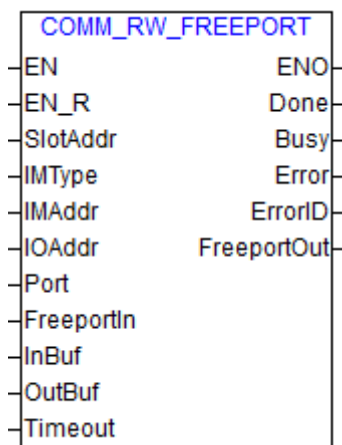
3.6.3.10.1 功能

读取或写入 LK238 自由口数据。该指令有示例工程，可直接导入使用。

导入算法示例时，可通过以下两种方式导入：

- 菜单栏中，单击【工程】—【导入示例工程】菜单。详细说明请参见导入示例工程。
- 库管理器中，右击功能块，选择【导入示例工程】命令。详细说明请参见 LK238 串口通信模块。

导入的示例工程需要在程序中调用或任务下调用才能正常使用。



COMM_RW_FREEPORT 指令

3.6.3.10.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿使能
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1: 接口模块类型为 LK231 或 LK232，此时， IMAAddr 参数无意义 2: 接口模块类型为 LK235/LK271 3: 接口模块类型为 LK250 4: 接口模块类型为 LK234/LK235M，此时， SlotAddr 参数无意义
IMAAddr	DWORD	IO 模块所在背板使用的接口模块地址 当接口模块为 LK231 或 LK232 时， IMAAddr 值无意义，无需设置 当接口模块为 LK235/LK271 时， IMAAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时， IMAAddr 与扩展背板拨码开关设置值保持一致 当接口模块为 LK234/LK235M 时， IMAAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果，例如 IP 地址 192.168.1.250， IMAAddr 参数为 16#C0A801FA 若 IMAAddr 配置错误时， ErrorID 会报 7
IOAddr	BYTE	IO 模块所在背板的 DP 从站地址（2~125）
Port	BYTE	端口号（1~4）
FreeportIn	FREEPORTINPARAM	自由口协议输入数据
InBuf	POINTER TO BYTE	输入数据缓冲区，指向的缓冲区大小必须≥224 Bytes
OutBuf	POINTER TO BYTE	输出数据缓冲区，指向的缓冲区大小必须≥224 Bytes
Timeout	WORD	执行超时时间(单位:1ms)（0~65535），当设置为 0 时采用默认值 2000；该参数表示，控制器与串口模块通讯异常后多长时间报错，与实际通讯是否超时无关

输出参数	类型	含义
Done	BOOL	读写自由口数据完成
Busy	BOOL	正在进行读写数据
Error	BOOL	读写数据过程中发生错误
ErrorID	WORD	功能块错误码 =0: 正确 =1: IMType 协议类型错误 =2: SlotAddr 指定槽位范围错误

输出参数	类型	含义
		=3: SlotAddr 指定槽位的通信板卡类型不支持 =4: IOAddr 指定的 IO 模块 DP 从站地址错误 =5: IMAddr 指定的接口模块地址错误 =7: IOAddr 指定地址模块类型错误（此号站配置的不是 LK238 模块，请检查 IMType、SlotAddr、IMAddr 以及 IOAddr 参数配置是否正确） =8: 输入/输出字节长度错误（请检查模块配置页面输入/输出模块） =9: Port 端口参数配置错误 =10: 发送数据长度超过输出长度字节（发送数据长度包括数据头的 10 字节长度） =11: 接收数据长度超过输入长度字节（接收数据长度包括数据头的 7 字节长度） =12: 发送长度设定异常（等于 0 或大于 210） =13: 接收长度设定异常（大于 210） =14: 端口未工作（当用户参数页面指定端口 Com Status 为 Disable; COMM_CTL_MODULE 功能块中，ComEnable 参数中对对应端口未使能） =15: 控制器与串口模块通讯异常 =16: 端口协议不匹配 =17: 接收使能和发送使能均设置为 0(收发模式下接收和发送至少有一个需要使能)
FreeportOut	FREEPOROUTPARAM	自由口协议输出数据

FreeportInParam 结构体成员

成员变量	数据类型	参数说明
SendEn	BOOL	发送使能（高电平使能）
SendLength	BYTE	设置发送长度（1~210）
SendTimeout	BYTE	设置发送超时时间（1~255，单位：10ms）
RecvEn	BOOL	接收使能（用户参数中配置为收发模式，RecvEn 高电平使能；用户参数中配置为只收模式，RecvEn 上升沿使能，高电平持续收数据）
RecvLen	BYTE	设置接收长度（0~210）（当使能开始字符和结束字符的时候，接收长度可以设置为 0）
StartChar	BYTE	设置开始字符（1~255）
EndChar	BYTE	设置结束字符（1~255）
RecvTimeout	BYTE	设置接收超时时间（1~255，单位：10ms）
RecvMode	BYTE	接收模式 bit0:超时时间使能 bit1:忽略 bit2:结束字符使能 bit3:开始字符使能 bit4-7:忽略
AckID	BYTE	主站确认包号

FreeportOutParam 结构体成员

成员变量	数据类型	参数说明
SendQ	BOOL	发送结束
SendErr	BYTE	发送状态 =0: 正确 =2: 超时
RecvQ	BOOL	接收结束
RecvCount	BYTE	接收字节数
RecvErr	BYTE	结束状态 =0: 正确

成员变量	数据类型	参数说明
		=2: 缺少开始条件（使能开始字符但开始字符设置为 0；使能结束字符，但是未使能开始字符，同时接收长度为 0） =3: 缺少结束条件（使能结束字符但结束字符设置为 0；使能开始字符，但是未使能结束字符，同时接收长度为 0） =4: 超时时间（Timeout）设定过小 =5: 超时 =6: 未找到结束字符
PackID	BYTE	LK238 返回命令号

3.6.3.11 COMM_CTL_MODULE（LK238 模块控制功能块）

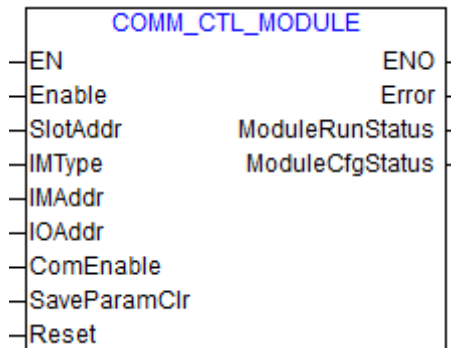
3.6.3.11.1 功能

该功能块可以使能四个串口，清空 Modbus 主站指令，复位交互接口。该指令有示例工程，可直接导入使用。

导入算法示例时，可通过以下两种方式导入：

- 菜单栏中，单击【工程】—【导入示例工程】菜单。详细说明请参见导入示例工程。
- 库管理器中，右击功能块，选择【导入示例工程】命令。详细说明请参见 LK238 串口通信模块。

导入的示例工程需要在程序中调用或任务下调用才能正常使用。



COMM_CTL_MODULE 指令

3.6.3.11.2 参数说明

输入参数	类型	含义
Enable	BOOL	高电平使能
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1: 接口模块类型为 LK231 或 LK232，此时，IMAddr 参数无意义 2: 接口模块类型为 LK235/LK271 3: 接口模块类型为 LK250

输入参数	类型	含义
		4: 接口模块类型为 LK234/LK235M, 此时, SlotAddr 参数无意义
IMAddr	DWORD	IO 模块所在背板使用的接口模块地址 当接口模块为 LK231 或 LK232 时, IMAddr 值无意义, 无需设置 当接口模块为 LK235/LK271 时, IMAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时, IMAddr 与扩展背板拨码开关设置值保持一致 当接口模块为 LK234/LK235M 时, IMAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果, 例如 IP 地址 192.168.1.250, IMAddr 参数为 16#C0A801FA 若 IMAddr 配置错误时, ErrorID 会报 7
IOAddr	BYTE	IO 模块所在背板的 DP 从站地址 (2~125)
ComEnable	MODULECONTROLPARAM	COM1~COM4 的工作使能位
SaveParamClr	BOOL	清空掉电保存的通讯指令参数
Reset	BOOL	复位控制

输出参数	类型	含义
Error	BYTE	功能块错误码 =0: 正确 =1: IMType 协议类型错误 =2: SlotAddr 指定槽位范围错误 =3: SlotAddr 指定槽位的通信板卡类型不支持 =4: IOAddr 指定的 IO 模块 DP 从站地址错误 =5: IMAddr 指定的接口模块地址错误 =7: IOAddr 指定地址模块类型错误 (此号站配置的不是 LK238 模块, 请检查 IMType、SlotAddr、IMAddr 以及 IOAddr 参数配置是否正确)
ModuleRunStatus	MODULERUNSTATUSPARAM	模块工作状态
ModuleCfgStatus	MODULECFGSTATUSPARAM	模块配置状态

ModuleControlParam 结构体成员

成员变量	数据类型	参数说明
COM1_EN	BOOL	COM1 工作使能
COM2_EN	BOOL	COM2 工作使能
COM3_EN	BOOL	COM3 工作使能
COM4_EN	BOOL	COM4 工作使能

ModuleRunStatusParam 结构体成员

成员变量	数据类型	参数说明
ComRunStatus1	BOOL	COM1 工作状态
ComRunStatus2	BOOL	COM2 工作状态
ComRunStatus3	BOOL	COM3 工作状态
ComRunStatus4	BOOL	COM4 工作状态

ModuleCfgStatusParam 结构体成员

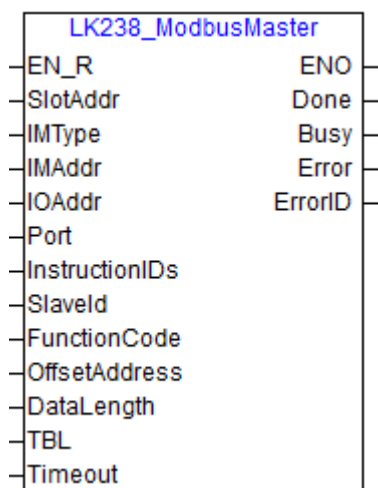
成员变量	数据类型	参数说明
ComCfgStatus1	BOOL	COM1 配置状态
ComCfgStatus2	BOOL	COM2 配置状态
ComCfgStatus3	BOOL	COM3 配置状态
ComCfgStatus4	BOOL	COM4 配置状态

3.6.3.12 LK238_ModbusMaster (LK238 主站配置读写功能块)

3.6.3.12.1 功能

通过该功能块读取或写入 LK238 Modbus 主站数据。当 EN_R 检测到上升沿时，读取或写入一次数据。读/写数据存放在 TBL 指针指向的数据缓存区，缓存区大小定义必须 ≥ 224 字节。

注：该指令在 FA-AT V3.1.10B1Patch1 及以上版本支持使用。



LK238_ModbusMaster 指令

3.6.3.12.2 参数说明

输入参数	类型	含义
EN_R	BOOL	上升沿执行
SlotAddr	BYTE	通信板卡槽位（2~5）
IMType	BYTE	IO 模块所在背板使用的接口模块类型： 1：接口模块类型为 LK231 或 LK232，此时，IMAAddr 参数无意义 2：接口模块类型为 LK235/LK271 3：接口模块类型为 LK250 4：接口模块类型为 LK234/LK235M，此时，SlotAddr 参数无意义
IMAAddr	DWORD	IO 模块所在背板使用的接口模块地址： 当接口模块为 LK231 或 LK232 时，IMAAddr 值无意义，无需设置 当接口模块为 LK235/LK271 时，IMAAddr 与模块地址拨码开关设置值保持一致 当接口模块为 LK250 时，IMAAddr 与扩展背板拨码开关设置值保持一致 当接口模块为 LK234/LK235M 时，IMAAddr 是模块 IP 地址的 4 段分别转换为 16 进制后合并的结果，例如 IP 地址 192.168.1.250，IMAAddr 参数为 16#C0A801FA 若 IMAAddr 配置错误时，ErrorID 会报 7
IOAddr	BYTE	LK238 模块从站地址（2~125）
Port	BYTE	端口号（1~4）
InstructionIDs	BYTE	指令 ID 号（1~32），同一 Port 端口下，指令 ID 号不能重复
SlaveID	BYTE	从站地址（1~247）

输入参数	类型	含义
FunctionCode	BYTE	功能码 0x01（读线圈）、0x02（读离散量输入）、0x03（读保持寄存器）、0x04（读输入寄存器）、0x05（写单个线圈）、0x06（写单个寄存器）、0x0F（写多个线圈）、0x10（写多个寄存器） 设置时，写入对应功能码十进制值。例如 0x01（读线圈）功能码，设置值为 1，0x0F（写多个线圈）功能码，设置值为 15
OffsetAddress	WORD	读写偏移地址，设置范围 0~65535
DataLength	WORD	读写开关量/模拟量数据长度。设置时，数据长度+偏移地址（OffsetAddress）≤65536，例如，地址设置为 65535，则数据长度必须设置为 1 开关量：1~1600 模拟量：1~100
TBL	POINTER TO BYTE	主站存放数据地址，指向的缓冲区大小必须≥224 Bytes
Timeout	WORD	Modbus 通讯超时时间（单位：1ms），当设置为 0 时采用默认值 1000 设置时，超时时间需满足：Timeout>Tran_Byte/(Baud rate/(8*1000))ms，其中 Tran_Byte 为输入/输出字节总长度，Baud rate 为设置的 Modbus 通讯波特率

输出参数	类型	含义
Done	BOOL	读写主站数据完成
Busy	BOOL	正在进行读写主站数据
Error	BOOL	读写主站数据过程中发生错误
ErrorID	WORD	功能块错误码 =0：无错误 =1：IMType 协议类型错误 =2：SlotAddr 指定槽位范围错误 =3：SlotAddr 指定槽位的通信板卡类型不支持 =4：IOAddr 指定的 IO 模块 DP 从站地址错误 =5：IMAddr 指定的接口模块地址错误 =7：IOAddr 指定地址模块类型错误（此号站配置的不是 LK238 模块，请检查 IMType、SlotAddr、IMAddr 以及 IOAddr 参数配置是否正确） =8：输入/输出字节长度错误（请检查模块配置页面输入/输出模块） =9：Port 端口参数配置错误 =10：从站地址不合法（1~247） =11：访问地址不合法（OffsetAddress+ DataLength 超出 0x10000） =12：功能码不合法（01 02 03 04 05 06 15 16） =13：该端口的用户参数配置的不是 Modbus 主站 =14：数据长度不合法（开关量：1~1600 模拟量：1~100） =15：控制器与串口模块通讯异常 =16：InstructionIDs 参数配置错误 主站错误信息 =1001:数据帧错误 =1002:超时 =1005:端口未工作（当用户参数页面指定端口 Com Status 为 Disable） =1128+1:从站不支持的功能码 =1128+2: 数据地址溢出 =1128+3: 数据范围溢出 =1128+4: 从站设备故障 =1128+6: 从站设备忙

3.6.3.12.3 指令使用举例

示例说明：

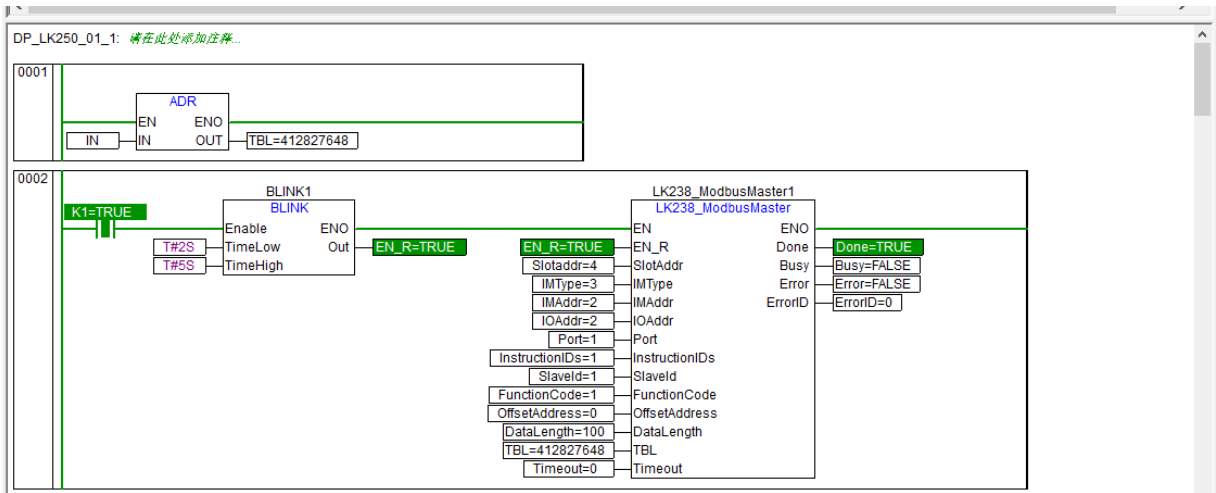
LK238 通过 COM1 口读取从站数据。从站 ID 号为 1，读取偏移地址从 0 开始的 100 个线圈数据。

序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护	SOE例
0001	IN			ARRAY[0..223] OF BYTE		FALSE	☐
0002	TBL			POINTER TO BYTE	412827648	FALSE	☐
0003	K1			BOOL	TRUE	FALSE	☐
0004	BLINK1			BLINK		FALSE	☐
0005	LK238_ModbusMaster1			LK238_MODBUSMASTER		FALSE	☐
0006	EN_R			BOOL	FALSE	FALSE	☐
0007	SlotAddr			BYTE	4	FALSE	☐
0008	IMType			BYTE	3	FALSE	☐
0009	IMAddr			DWORD	2	FALSE	☐
0010	IOAddr			BYTE	2	FALSE	☐
0011	Port			BYTE	1	FALSE	☐
0012	InstructionIDs			BYTE	1	FALSE	☐
0013	SlaveId			BYTE	1	FALSE	☐
0014	FunctionCode			BYTE	1	FALSE	☐
0015	OffsetAddress			WORD	0	FALSE	☐
0016	DataLength			WORD	100	FALSE	☐
0017	Timeout			WORD	0	FALSE	☐
0018	Done			BOOL	FALSE	FALSE	☐
0019	Busy			BOOL	FALSE	FALSE	☐
0020	Error			BOOL	FALSE	FALSE	☐
0021	ErrorID			WORD	0	FALSE	☐

变量定义

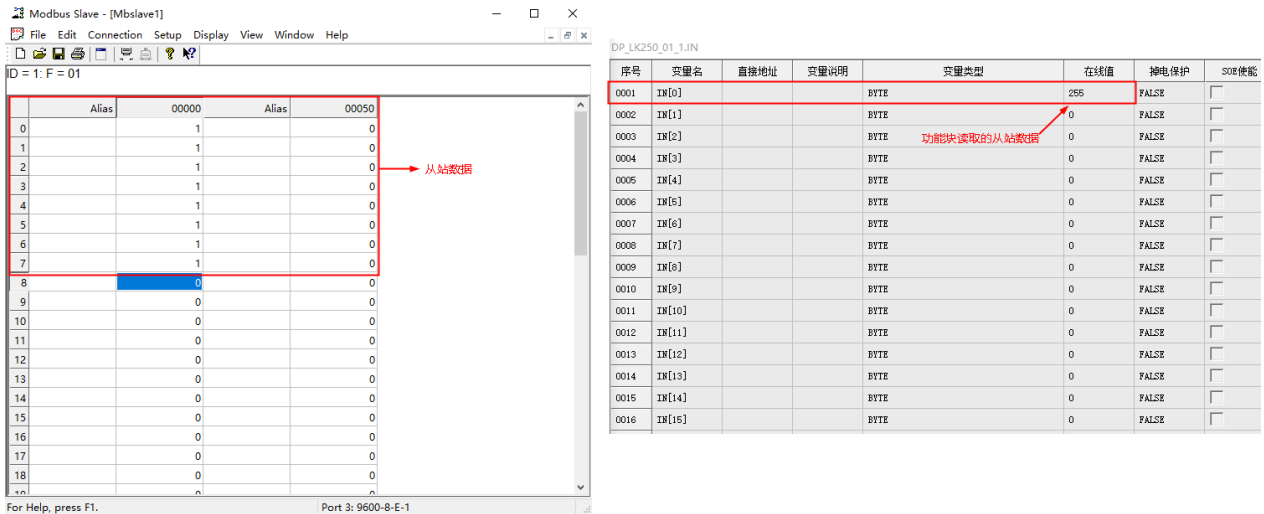
程序通过 ADR 功能块获取数组 IN 的地址，TBL 指针指向该数组地址。

当 K1 为 TRUE 时，触发 2ms 脉冲信号，LK238_ModbusMaster 功能块读取从站数据，并存放到数组 IN 中。



程序示例

从站偏移地址 0~7 数据为 1，读取到数组 IN[0]变量中，值为 255。



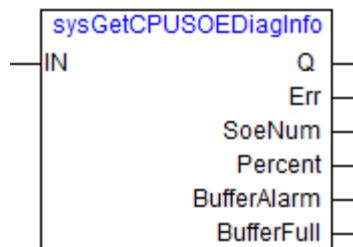
读取从站数据示意

3.6.4 SOE 相关指令

3.6.4.1 sysGetCPUSOEInfo(获取主控 SOE 诊断信息)

3.6.4.1.1 功能

获取主控 SOE 事件存储区诊断信息。



sysGetCPUSOEInfo 指令

3.6.4.1.2 参数说明

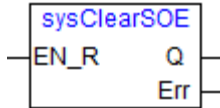
输入参数	类型	含义
IN	BOOL	使能输入信号，电平触发 1: 有效 0: 无效

输出参数	类型	含义
Q	BOOL	输出完成标志 0: 输出未完成 1: 输出完成
Err	BYTE	错误值 0: 获取 SOE 诊断信息成功
SoeNum	WORD	主控当前缓存的 SOE 条数
Percent	REAL	主控 SOE 缓存条数占最大存储条数的百分比, 最大存储 6000 条
BufferAlarm	BOOL	主控 SOE 缓存条数超过最大存储的 70%时, 置位告警标志 1
BufferFull	BOOL	主控 SOE 缓存条数达到最大存储的 100%时, 置位告警标志 1

3.6.4.2 sysClearSOE(清除主控 SOE 数据)

3.6.4.2.1 功能

清空主控及 SOE 模块存储的 SOE 事件数据。



sysClearSOE 指令

3.6.4.2.2 参数说明

输入参数	类型	含义
EN_R	BOOL	使能信号, 上升沿触发

输出参数	类型	含义
Q	BOOL	输出完成标志 0: 输出未完成 1: 输出完成
Err	BYTE	错误值 0: 清空 SOE 数据成功

3.6.4.3 sysReadSOE (读取主控 SOE 信息)

3.6.4.3.1 功能

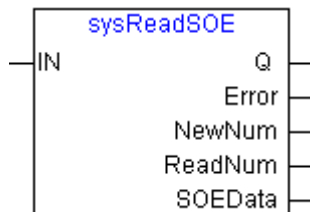
读取控制器 SOE 事件信息, 最多一次读取 50 条。当 IN 为上升沿时, 触发一次数据读取。读取的数据存放到 SOEData 结构体数组中。

支持 SOE 事件上报的数据有: LK631 SOE 模块的 DI 通道数据和设置了 SOE 使能的 BOOL 型变量。

LK631 模块需要在“用户参数”中将 DI 通道功能设置为 SOE，其 SOE 功能才能被启用。

该功能块仅能调用一个。

注：该指令在 FA-AT V3.1.10B1Patch1 及以上版本支持，兼容的固件版本 LK220-G02。



sysReadSOE 指令

3.6.4.3.2 参数说明

输入参数	类型	含义
IN	BOOL	上升沿使能

输出参数	类型	含义
Q	BOOL	输出 TRUE：指令执行成功 FALSE：指令执行失败
Error	BYTE	错误码 0：正确 1：内部 SOE 队列错误 2：组态 SOE 数据接收缓冲区错误 3：未产生新 SOE 事件数据 200：功能块重复调用
NewNum	WORD	自上次读取后，剩余未读取的 SOE 事件条数
ReadNum	WORD	本次读取的 SOE 事件条数，每次读取不超过 50 条
SOEData	ARRAY [0 49] of STSOEDATA	存放 50 条 SOE 事件的缓存区，最多存放 50 条 SOE 事件结构体 STSOEDATA 定义详见下表

下表为 SOE 数据的详细定义：

STSOEDATA 结构体成员

成员变量	类型	含义	字节	
			序号	说明
Seconds	DWORD	SOE 事件发生的秒时间	Byte1	时间（秒）最低字节
			Byte2	时间（秒）低字节
			Byte3	时间（秒）高字节
			Byte4	时间（秒）最高字节
T1	WORD	SOE 事件发生的百微秒时间	Byte1	时间（百微秒）低字节
			Byte2	时间（百微秒）高字节
ulOffset	DWORD	SOE 事件发生的数据区偏移地址。例如，数据的通道地址是 %IX4.6，则偏移	Byte1	偏移地址最低字节
			Byte2	偏移地址低字节

成员变量	类型	含义	字节	
			序号	说明
		地址为：4*8+6=38 字节	Byte3	偏移地址高字节
			Byte4	偏移地址最高字节
ChannelData	BYTE	SOE 事件发生的通道信息	Byte1	Bit0~Bit3：数据区索引，表示 SOE 点属于哪个数据区，例如：0 表示 M 区；1 表示 I 区；3 表示 R 区；4 表示 N 区 Bit4：有效位。1 表示有效，0 表示可能无效。例如校时无效时，记录的 SOE 事件有效位是 0，表明这个记录数据的时间可能是不准确的 Bit5：偏移地址类型，0 表示位偏移，1 表示字节偏移 Bit6~Bit7：通道当前值，值是 0 或 1

查看 SOE 数据信息时，可通过 ChannelData 变量的 Bit0~Bit3 值先确定数据区，再通过 ulOffset 确定偏移地址，找到 SOE 变量。

- 对于通道数据，可直接通过偏移计算出通道地址，找到对应 SOE 通道变量。

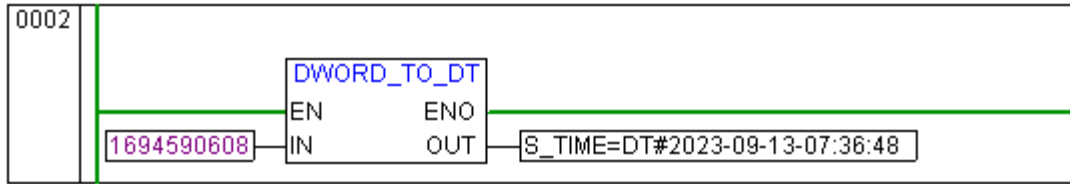
如下图所示，ChannelData 变量当前值为 17，对应二进制值为 00010001。Bit0~Bit3 值为 1，由此确定数据区为 I 区。Bit4 为 1，当前 SOE 数据有效。Bit6~Bit7 值为 0，则当前值为 0（代表值从 1 跳变为 0）。

PRG_SOE.SOEData3							
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护	
0001	SOEData3[0]			STSOEDATA		FALSE	
0002	SOEData3[1]			STSOEDATA		FALSE	
PRG_SOE.SOEData3[0]							
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护	
0001	Seconds		SOE事件发生的秒时间	DWORD	1694590608	FALSE	
0002	T1		SOE事件发生的百微秒时间	WORD	6416	FALSE	
0003	ulOffset		SOE事件发生的数据区偏移	DWORD	9931	FALSE	
0004	ChannelData		SOE事件发生的通道信息	BYTE	17	FALSE	
0005							
0006							
0007							
0008							
0009							
0010							
0011							
0012							
0013							
0014							
0015							
0016							

SOE 数据信息—通道数据

如下图所示，Seconds 秒时间解析为：2023-09-13-07:36:48。上图中读取的百微秒时间 T1 为 6416，

则 SOE 事件发生时间为：2023-09-13-07:40:23（641.6 ms）



秒时间解析为 DT 数据

ulOffset 偏移地址为 9931，由此计算通道地址为：%IX1241.3。整个工程查找地址%IX1241.3，定位到对应通道地址，如下图所示。

LK631_284(37:LK631)				
设备配置 通道 诊断信息 信息				
0_14 Channels Digital Input Module				
通道号	通道名称	通道类型	通道地址	通道说明
1	DPIO_261_1_37_1	BOOL	%IX1240.0	
2	DPIO_261_1_37_2	BOOL	%IX1240.1	
3	DPIO_261_1_37_3	BOOL	%IX1240.2	
4	DPIO_261_1_37_4	BOOL	%IX1240.3	
5	DPIO_261_1_37_5	BOOL	%IX1240.4	
6	DPIO_261_1_37_6	BOOL	%IX1240.5	
7	DPIO_261_1_37_7	BOOL	%IX1240.6	
8	DPIO_261_1_37_8	BOOL	%IX1240.7	
9	DPIO_261_1_37_9	BOOL	%IX1241.0	
10	DPIO_261_1_37_10	BOOL	%IX1241.1	
11	DPIO_261_1_37_11	BOOL	%IX1241.2	
12	DPIO_261_1_37_12	BOOL	%IX1241.3	
13	DPIO_261_1_37_13	BOOL	%IX1241.4	
14	DPIO_261_1_37_14	BOOL	%IX1241.5	

通道数据

SOE 事件信息解析如下：LK631 模块 DPIO_261_1_37_12 通道在 2023-09-13-07:40:23（641.6 ms）时刻发生跳变，值从 1 跳变为 0。

SOE 事件信息

跳变时间	POU 名/模块节点名	变量名	值跳变	SOE 有效位
2023-09-13-07:40:23（641.6 ms）	LK631_284(37:LK631)	DPIO_261_1_37_12	从 1 跳变为 0	Bit4=1，当前 SOE 事件有效

- 对于 PRG 中的 BOOL 型变量，需要导出变量表，在变量表中找到对应的数据区，找到 ulOffset 偏移地址对应的 BOOL 型变量。

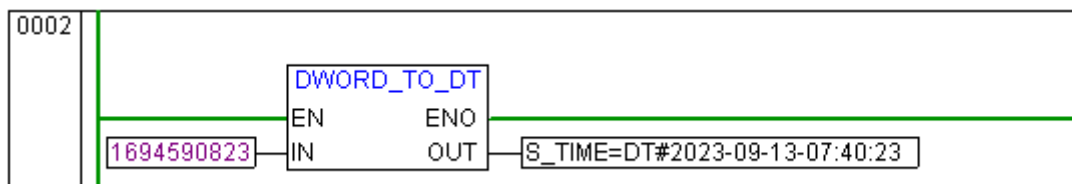
如下图所示，ChannelData 变量当前值为 52，对应二进制值为 00110100。Bit0~Bit3 值为 4，由

此确定数据区为 N 区。Bit4 为 1，当前 SOE 数据有效。Bit6~Bit7 值为 0，则当前值为 0（代表值从 1 跳变为 0）。

PRG_SOE.SOEData3							
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护	
0001	SOEData3[0]			STSOEDATA		FALSE	
0002	SOEData3[1]			STSOEDATA		FALSE	
0003	PRG_SOE.SOEData3[0]						
序号	变量名	直接地址	变量说明	变量类型	在线值	掉电保护	
0004	0001	Seconds	SOE事件发生的秒时间	DWORD	1694590823	FALSE	
0005	0002	T1	SOE事件发生的百微秒时间	WORD	7110	FALSE	
0006	0003	ulOffset	SOE事件发生的数据区偏移	DWORD	1894	FALSE	
0007	0004	ChannelData	SOE事件发生的通道信息	BYTE	52	FALSE	
0008							
0009							
0010							
0011							
0012							
0013							
0014							
0015							
0016							

SOE 数据信息—PRG 变量

如下图所示，Seconds 秒时间解析为：2023-09-13-07:40:23。上图中读取的百微秒时间 T1 为 7110，则 SOE 事件发生时间为：2023-09-13-07:40:23（711.0 ms）。



秒时间解析为 DT 数据

ulOffset 偏移地址为 1894，在变量表中找到 N 区 1894 偏移地址对应的变量，如下图所示。

ProjectName: TEST1						
DateTime: 2023-09-13 15-14-40						
变量名	变量组名	变量说明	变量类型	数据区	变量偏移	变量大小
SOE_BOOL10	PRG_SOE		BOOL	G	1894	1
Percent1	PRG_SOE	主控SOE缓	REAL	G	18028	4
sysGetCPUSOEDiagInfo1	PRG_SOE	获取主控S	SYSGETCPI	G	18012	16
sysGetCPUSOEDiagInfo1.IN	PRG_SOE	使能	BOOL	G	18012	1
sysGetCPUSOEDiagInfo1.Q	PRG_SOE	输出	BOOL	G	18013	1
sysGetCPUSOEDiagInfo1.Err	PRG_SOE	错误	BYTE	G	18014	1
sysGetCPUSOEDiagInfo1.SoeNum	PRG_SOE	主控当前缓	WORD	G	18016	2
sysGetCPUSOEDiagInfo1.Percent	PRG_SOE	主控SOE缓	REAL	G	18020	4
sysGetCPUSOEDiagInfo1.BufferAlarm	PRG_SOE	主控SOE缓	BOOL	G	18024	1
sysGetCPUSOEDiagInfo1.BufferFull	PRG_SOE	主控SOE缓	BOOL	G	18025	1
SOE_BOOL9	PRG_SOE		BOOL	G	1893	1
SOE_BOOL8	PRG_SOE		BOOL	G	1892	1

变量表中找到 SOE 数据点

SOE 事件信息解析如下：PRG_SOE 程序中的变量 SOE_BOOL10 在 2023-09-13-07:40:23 (711.0 ms) 时刻发生跳变，值从 1 跳变为 0。

SOE 事件信息

跳变时间	POU 名/模块节点名	变量名	值跳变	SOE 有效位
2023-09-13-07:40:23 (711.0 ms)	PRG_SOE	SOE_BOOL10	从 1 跳变为 0	Bit4=1, 当前 SOE,事件有效



北京和利时智能技术有限公司

Beijing HollySys Intelligent Technologies Co., Ltd..

地址：北京经济技术开发区地盛中路2号院

邮编：100176

电话：010-5898 1588

热线：4008111999

传真：010-5898 1558

<http://www.hollysys.com>